Core Java

In Depth

From the JVM up: how Java really works, the language and its libraries, collections, streams, concurrency, memory and garbage collection, modern Java, projects you run yourself, and a complete interview and self-test kit.

JVM · Language · OOP · Strings & Time · Exceptions & I/O · Collections · Streams · Concurrency
JVM internals · Diagnostics · Modern Java · Patterns · Projects · Interviews · Study plan

```
public class Main {
    public static void main(String[] args) {
        var jvm = JVM.start();
        jvm.run(bytecode);
    }
}
```

Version basis

Java 25 (the current long-term-support release); notes on Java 21 LTS and on Java 26 and 27 where they differ
Code is written to run on Java 21 or newer unless a listing says otherwise; features after 21 are marked
Examples are single files you can run with the java command; no build tool needed until Part 11

How to use this guide

- **Read it in order the first time.** The guide starts with the JVM itself (how a `.java` file becomes a running program), then the language, object-oriented design, strings and time, exceptions and I/O, collections, lambdas and streams, concurrency, and a deep look at memory and garbage collection. Later parts cover diagnostics, the changes from Java 8 to today, design patterns, four projects, and an interview and self-test kit.
- **Every chapter follows the same rhythm:** why it exists, how it works, how to use it (with code you can run), what goes wrong, a short key-points box, and a **Check yourself** quiz. Coloured boxes mean: NOTE (extra detail), TIP (practical advice), WATCH OUT and GOTCHA (mistakes that cost people days), REAL WORLD (how it appears in production), INTERVIEW ANGLE (what interviewers like to ask).
- **The daily loop: read, code, test.** Read a chapter section, type the example yourself and change it until it breaks, then answer the Check yourself questions without looking. The study-plan chapter at the end maps a 25-day plan onto the chapters.
- **Running the code.** Most listings are complete single-file programs. Save one as `Demo.java` and run `java Demo.java` (Java 11 or newer launches a single source file directly). A dark box is a terminal session: lines beginning with `$` are shell commands, lines beginning with `jshell>` are typed into `jshell`, and the other lines are output. Output shown was produced by running the listings on Java 21 unless marked; timings, hash codes and memory addresses on your machine will differ.
- **Versions.** Java 25 is the long-term-support release this guide targets. Features that arrived after Java 21 are marked with the version that finalised them, and listings that need them were not run on Java 21.
- **Conventions.** `monospace` is code, a class, a method or a command. Package names are lower case, classes `UpperCamelCase`, methods and variables `lowerCamelCase`, constants `UPPER_SNAKE_CASE`.

Contents

Part 1: The Java Platform and the JVM 6

1.1 What Java Is 6
1.2 From Source to Running Program 10
1.3 JVM Architecture 15
1.4 Your First Programs and the Toolchain 24
1.5 How to Study This Book 30

Part 2: Language Fundamentals 33

2.1 Variables, Primitive Types and Literals 33
2.2 Operators and Expressions 38
2.3 Type Conversion, Casting and Promotion 42
2.4 Control Flow 46
2.5 Arrays 50
2.6 Methods 54
2.7 Text Blocks and Basic I/O 58

Part 3: Object-Oriented Java 62

3.1 Classes and Objects 62
3.2 Static, Access Control and Packages 68
3.3 Inheritance 73
3.4 Polymorphism and Dynamic Dispatch 77
3.5 Abstract Classes and Interfaces 82

3.6 The Object Class . . . 86
3.7 Nested, Inner, Local and Anonymous Classes . . . 91
3.8 Enums . . . 96
3.9 Records . . . 101
3.10 Sealed Classes and Interfaces . . . 106
3.11 Pattern Matching . . . 109
3.12 Wrapper Classes, Autoboxing and Identity . . . 113
3.13 Designing Classes Well . . . 118

Part 4: Strings, Text and Time . . . 126

4.1 Strings in depth . . . 126
4.2 Formatting, regex and text . . . 134
4.3 Unicode, charsets and Locale . . . 140
4.4 Numbers beyond primitives . . . 145
4.5 Date and time with java.time . . . 151

Part 5: Exceptions, I/O and Utilities . . . 160

5.1 Exceptions . . . 160
5.2 Logging and debugging basics . . . 170
5.3 Classic I/O . . . 175
5.4 NIO.2: Paths, Files and Channels . . . 181
5.5 Serialization and Its Dangers . . . 188
5.6 Properties, System and Runtime . . . 192

Part 6: Generics and Collections . . . 198

6.1 Generics . . . 198
6.2 The Collections Framework . . . 205
6.3 Lists in depth . . . 211
6.4 Sorting and comparing . . . 216
6.5 Immutable, unmodifiable and sequenced collections . . . 222
6.6 Sets . . . 227
6.7 Other maps: LinkedHashMap, TreeMap and the specialists . . . 233
6.8 HashMap in depth . . . 238
6.9 Queues and deques . . . 246
6.10 Choosing a data structure, and Big-O . . . 251
6.11 Implement your own: array list, linked list, hash map, LRU cache . . . 255

Part 7: Functional Java: Lambdas and Streams . . . 263

7.1 Lambdas and Functional Interfaces . . . 263
7.2 Optional: Modelling "No Result" . . . 270
7.3 Streams: Fundamentals . . . 273
7.4 Collectors . . . 279

7.5 Parallel Streams and Performance . . . 284
7.6 Stream Gatherers . . . 289
7.7 Functional Style in Practice . . . 292

Part 8: Concurrency . . . 296

8.1 Concurrency Concepts . . . 296
8.2 Threads . . . 300
8.3 Races and the Java Memory Model . . . 305
8.4 synchronized and Monitors . . . 310
8.5 Thread Confinement, Immutability and Liveness . . . 316
8.6 Locks and Atomics . . . 321
8.7 Executors and Thread Pools . . . 330
8.8 CompletableFuture . . . 340
8.9 Synchronizers and Concurrent Collections . . . 347
8.10 Fork/Join and Parallelism . . . 356
8.11 Virtual Threads . . . 361
8.12 Structured Concurrency, Scoped Values and Practice . . . 367

Part 9: The JVM in Depth . . . 375

9.1 Objects in Memory . . . 375
9.2 Heap Structure and Reference Types . . . 380
9.3 Garbage Collection Fundamentals . . . 385
9.4 The Collectors . . . 389
9.5 Reading GC Logs and Tuning . . . 394
9.6 Memory Leaks and OutOfMemoryError . . . 398
9.7 The JIT Compiler and Warm-Up . . . 404

Part 10: Runtime Features and Diagnostics . . . 411

10.1 Reflection and Annotations . . . 411
10.2 Dynamic Proxies and Method Handles . . . 416
10.3 Class Loaders and ServiceLoader . . . 420
10.4 The Module System (JPMS) . . . 424
10.5 Calling Native Code: JNI and the Foreign Function and Memory API . . . 427
10.6 Troubleshooting Playbook . . . 430
10.7 The Diagnostics Toolbox . . . 435

Part 11: Modern Java and Tooling . . . 444

11.1 From Java 8 to 25: what changed . . . 444
11.2 Build tools: Maven and Gradle . . . 448
11.3 Testing Java code . . . 451
11.4 Packaging and running Java applications . . . 456
11.5 Java 25 LTS highlights in depth . . . 459

11.6 Java 26, Java 27 and the road ahead . . . 463

Part 12: Design Patterns and Best Practices . . . 468

12.1 Creational patterns in modern Java . . . 468
12.2 Structural and behavioural patterns . . . 475
12.3 Effective Java in practice . . . 485
12.4 Clean code and code smells in Java . . . 491
12.5 Secure coding in Java . . . 497
12.6 Performance rules of thumb . . . 503

Part 13: Projects . . . 510

13.1 How to work through the projects . . . 510
13.2 Project A: A bank account manager . . . 511
13.3 Project B: A log file analyzer . . . 524
13.4 Project C: A concurrent order processor . . . 534
13.5 Project D: An in-memory key-value store . . . 548

Part 14: Interview Preparation . . . 563

14.1 How Java Interviews Work . . . 563
14.2 Interview Q&A;: Language, OOP, Strings, Exceptions and I/O . . . 565
14.3 Interview Q&A;: Collections, Generics, Streams and Functional Java . . . 572
14.4 Interview Q&A;: Concurrency, the Memory Model and the JVM . . . 579
14.5 Interview Q&A;: Modern Java . . . 587
14.6 Interview Q&A;: Design and Best Practices . . . 592
14.7 What Does This Print? Fifty Puzzles . . . 596
14.8 Coding Problems: Strings and Arrays . . . 624
14.9 Coding Problems: Collections, Recursion and Dynamic Programming . . . 631
14.10 Concurrency Coding Tasks . . . 639
14.11 Output Prediction and Debugging Drills . . . 646

Part 15: Study Plan, Cheat Sheets and Quizzes . . . 652

15.1 The 25-Day, 4-Week Study Plan . . . 652
15.2 Cheat Sheets . . . 656
15.3 Part-by-Part Self-Test Quizzes . . . 664
15.4 Final Mock Interview and What to Do Next . . . 674

PART 1

The Java Platform and the JVM

Before you write serious Java, know what Java is made of: a language, a compiler, a virtual machine and a very large standard library.

In this part

1.1 What Java Is
1.2 From Source to Running Program
1.3 JVM Architecture
1.4 Your First Programs and the Toolchain
1.5 How to Study This Book

1.1 What Java Is

You have probably heard "Java is a programming language". That is true but incomplete, and the missing part is what makes Java interesting to an engineer. Java is a **platform**: a language (the syntax you type), a compiler (turns that syntax into a portable intermediate form), a runtime called the **Java Virtual Machine** (JVM, runs that intermediate form), and a standard library of thousands of ready-made classes. When you say "I know Java" in an interview, the interviewer expects you to know all four, and especially how they fit together. This chapter gives you that map. The rest of the book fills it in.

1.1.1 A short history that explains the design

Java began at Sun Microsystems in the early 1990s (James Gosling led the team) and was released publicly in 1996. The goal of the platform was practical: let programmers write code once and run it on many kinds of machines, with automatic memory management and no pointer arithmetic, so that whole classes of crashes disappear. Sun was acquired by Oracle in 2010, and since then the language is developed in the open as the **OpenJDK** project, with Oracle as the largest contributor and many companies (Red Hat, Amazon, Microsoft, Azul, SAP, Alibaba and others) contributing too.

Three design decisions from the early days still shape every Java program today:

- **Compiled to bytecode, not to machine code.** Your `.java` files are compiled into `.class` files that contain *bytecode*, an instruction set for an imaginary machine. Any computer with a JVM can run that bytecode.
- **Automatic memory management.** You create objects with `new`; you never free them. A garbage collector reclaims objects that nothing can reach any more.
- **Strong, static typing with a safety net.** The compiler checks types, the JVM verifies bytecode before running it, and array accesses and casts are checked at run time. A bug in Java usually produces an exception with a stack trace, not a silent memory corruption.

1.1.2 Write once, run anywhere: what it really means

The slogan "write once, run anywhere" (WORA) is explained by one picture: the compiler targets the JVM, not the CPU. The JVM is itself a program, compiled separately for Windows, Linux, macOS, x64, ARM and so on. Your bytecode does not change; the JVM underneath does.

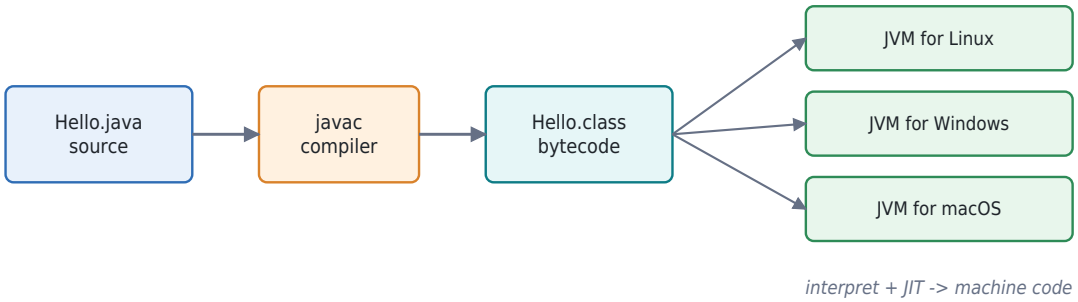


Figure 1.1. The same bytecode runs on any machine that has a JVM. Notice that the JVM, not your program, is platform-specific, and that the JVM turns hot bytecode into machine code at run time.

Two details that interviewers like:

- Primitive sizes are fixed by the language (`int` is always 32 bits, `long` always 64 bits), unlike C, where `int` depends on the platform. This is part of why the same program gives the same answers everywhere.
- "Anywhere" has limits. Your program still meets operating-system differences when it touches files (path separators), native libraries, or the number of CPUs. The JVM hides the instruction set; it cannot hide everything.

1.1.3 JDK, JRE and JVM

These three acronyms are asked about constantly. They are nested like Russian dolls.

Name	What it is	What it contains
JVM	The virtual machine: loads classes, runs bytecode, manages memory, compiles hot code.	Class loader, interpreter, JIT compilers, garbage collectors. Specified by the JVM Specification.
JRE	The Java Runtime Environment: what you need to *run* programs.	JVM + the standard class library (`java.base` and other modules).
JDK	The Java Development Kit: what you need to *build* programs.	JRE + `javac`, `jar`, `javadoc`, `javap`, `jshell`, `jcmd`, `jlink` and other tools.

The three layers. Pick the JDK on a development machine.

A practical update to the textbook answer: since Java 9 the runtime is modular, and since Java 11 Oracle and most vendors stopped shipping a separate "JRE download". You install a JDK, and if you want a small runtime for deployment you build one with `jlink` (covered in the Part on modern Java and tooling). The word "JRE" is still used for "the runtime part of a JDK".

One more distinction. Java the **language** is defined by the *Java Language Specification* (JLS). The **JVM** is defined by the *Java Virtual Machine Specification*. Other languages target the same JVM, such as Kotlin, Scala and Groovy, because they also compile to class files. That is why "JVM languages" exist, and why understanding the JVM pays off even if you later write Kotlin.

1.1.4 Editions

Java comes in "editions", which are really sets of libraries on top of the same language:

- **Java SE** (Standard Edition): the language, the JVM and the core libraries (`java.lang`, `java.util`, `java.io`, `java.time`, `java.net.http` and so on). This book is about Java SE.
- **Jakarta EE** (formerly Java EE): extra specifications for enterprise servers (servlets, persistence, REST). It moved from Oracle to the Eclipse Foundation, and its package names changed from `javax.*` to `jakarta.*` (from Jakarta EE 9). Spring Boot 3 and later use the `jakarta.*` names.
- **Java ME** (Micro Edition): a small-device platform, now niche. Android uses the Java language but a different runtime and class library; do not mix the two up.

> **REAL WORLD**
> When you build a Spring Boot service, you write Java SE code (collections, streams, concurrency, records) and Spring supplies the framework on top. Every Spring problem that looks mysterious, such as a `ClassNotFoundException`, a proxy that fails, or an out-of-memory crash, is in the end a Core Java problem. That is why this book exists.

1.1.5 Releases, versions and LTS

Until Java 8 (2014) a new version arrived whenever it was ready, which meant years of waiting. Since Java 9 (2017) there is a fixed rhythm: **a feature release every six months, in March and September.** Every second year one of them is designated **LTS** (Long-Term Support): vendors promise to ship security fixes for it for years. The LTS releases are 8, 11, 17, 21 and 25. Non-LTS releases are supported only until the next release comes out.

Release	Date	Why it matters
Java 8 (LTS)	Mar 2014	Lambdas, streams, `Optional`, `java.time`. Still common in old code bases.
Java 11 (LTS)	Sep 2018	`HttpClient`, `var` in lambdas, running a source file directly with `java File.java`.
Java 17 (LTS)	Sep 2021	Records, sealed classes, pattern matching for `instanceof`, text blocks, switch expressions.
Java 21 (LTS)	Sep 2023	Virtual threads, pattern matching for `switch`, record patterns, sequenced collections.
Java 25 (LTS)	Sep 2025	Scoped values, module import declarations, compact source files, flexible constructor bodies, compact object headers (opt-in).
Java 26, 27	Mar 2026, Sep 2026	Non-LTS. Java 27 makes G1 the default garbage collector everywhere and turns compact object headers on by default.

Versions you will meet. "Final" means the feature is permanent, not a preview.

> **NOTE**
> The book targets **Java 25 LTS**. Every listing was run on JDK 21 (the machine used to build this book has no JDK 25), so a listing that needs a feature newer than 21 is labelled "(Java 25; not run on 21)". The features of 22 to 25 are collected in the Part on modern Java.

How to choose in practice: new projects should start on the newest LTS your frameworks support (25 now, 21 if a library is not ready). Many companies still run 17 and 11, and a few run 8, so an engineer should be able to read all of them. The language is strongly backward compatible: a program compiled for Java 8 usually still runs on Java 25.

1.1.6 Installing a JDK and checking it

Almost everyone uses a build of OpenJDK. The builds are functionally the same source code; they differ in vendor, support contract and packaging. Common free choices are **Eclipse Temurin**, **Amazon Corretto**, **Microsoft Build of OpenJDK**, **Azul Zulu**, **Red Hat build of OpenJDK**, **BellSoft Liberica** and Oracle's own OpenJDK builds. Pick one LTS build from one of them; for learning, the choice does not matter.

Typical installation routes:

- A **package manager**: `sudo apt install openjdk-21-jdk` on Ubuntu or Debian, `brew install openjdk@21` on macOS, `winget` on Windows.
- **SDKMAN!** (`sdk list java`, then `sdk install java <identifier>`), which lets you switch versions per shell. It is the most convenient way on Linux and macOS to keep several JDKs.
- A downloaded archive that you unpack anywhere and point to with `JAVA_HOME`.

After installing, verify *both* the runtime and the compiler, because they can come from different places:

Check the installation TERMINAL

```
$ java -version
openjdk version "21.0.12.1" 2026-08-18
OpenJDK Runtime Environment (build 21.0.12.1+1-1-24.04.4-Ubuntu)
OpenJDK 64-Bit Server VM (build 21.0.12.1+1-1-24.04.4-Ubuntu, mixed mode, sharing)
$ javac -version
javac 21.0.12.1
```

What to read in that output: `21.0.12.1` is the exact build (feature release 21, update 12); `mixed mode` means the JVM both interprets and JIT-compiles code; `sharing` means it uses a pre-built archive of core classes to start faster; "Ubuntu" is the vendor of this build. If `java -version` says 21 but `javac -version` says "command not found", you have only a runtime or your `PATH` is wrong.

Where is this JDK, and what version does the program see? TERMINAL

```
$ java -XshowSettings:properties -version 2>&1 | \
    grep -E "java.home|java.version |java.specification.version"
    java.home = /usr/lib/jvm/java-21-openjdk-amd64
    java.specification.version = 21
    java.version = 21.0.12.1
```

WATCH OUT

The most common beginner failure is two JDKs on one machine, where `javac` is 21 and `java` is 8 (or the reverse). Symptom: `UnsupportedClassVersionError: ... has been compiled by a more recent version of the Java Runtime`. Fix: make `JAVA_HOME` and `PATH` point at the same JDK, and check both `java -version` and `javac -version`.

INTERVIEW ANGLE

What is the difference between JDK, JRE and JVM? The JVM runs bytecode (loading, verification, interpretation, JIT, garbage collection). The JRE is the JVM plus the standard libraries needed to run a program. The JDK is the JRE plus development tools such as `javac`, `jar`, `javadoc` and `jshell`. You install the JDK to develop; a deployed application only needs a runtime.

Check yourself

1. Name the four parts of the Java platform that this chapter lists.
2. A colleague says "Java is interpreted". Why is that incomplete?
3. Which of 8, 11, 12, 17, 20, 21, 25 are LTS releases?
4. Your machine has a Java 21 runtime. A teammate sends you a `.class` file compiled with JDK 25 and it fails with `UnsupportedClassVersionError`. Why?
5. Is `int` 32 bits on every JVM, or does it depend on the CPU? Why does that help portability?
6. What did the package prefix `javax` change to in Jakarta EE 9 and later?

Answers

1. The language, the compiler (`javac`), the virtual machine (JVM) and the standard class library.
2. Source is compiled to bytecode first. The JVM then interprets bytecode at first, and the JIT compiler turns hot methods into machine code, so most time is spent in compiled code.
3. 8, 11, 17, 21 and 25. The others are six-month, non-LTS releases.
4. Each class file records the class-file version of the compiler that produced it. A JVM refuses class files newer than itself, and a Java 21 runtime cannot load Java 25 class files (major version 69).
5. Always 32 bits; the language specification fixes it. A program therefore computes the same results (for example integer overflow) on every platform.
6. `jakarta`, for example `javax.servlet` became `jakarta.servlet`.

KEY POINTS

Java is a platform: language, compiler, JVM and class library, not just a language.

- The compiler produces bytecode; the JVM, which is platform-specific, runs it. That is "write once, run anywhere".
- JDK contains the JRE, which contains the JVM. Install a JDK to develop, and check both `java -version` and `javac -version`.
- A new feature release arrives every March and September; LTS releases (8, 11, 17, 21, 25) get long support. This book targets Java 25.
- Java SE is the core; Jakarta EE and Spring build on it, so Core Java knowledge is the foundation for backend work.

1.2 From Source to Running Program

Typing `java Hello.java` feels like one step, but several things happen between your text file and the output on the screen. Knowing them makes error messages readable (is this a compile-time or a run-time problem?) and makes tools such as `javap`, `jar` and the classpath understandable instead of magical. In this chapter you will compile a small class, open the resulting file and read its bytecode instruction by instruction.

1.2.1 The pipeline: javac, class files, java

There are two commands, and they do different jobs.

- `javac` is the **compiler**. It reads `.java` source, checks syntax and types, and writes one `.class` file for every class it finds (nested classes get their own file).
- `java` is the **launcher**. It starts a JVM, finds the class you named, loads it and calls `public static void main(String[] args)`.

Here is the class we will dissect. It is deliberately small, and it has an instance field, a method with a local variable, and a `main`.

Counter.java JAVA

```java
public class Counter {
    private int count;

    public void increment() {
        count++;
    }

    public int sum(int a, int b) {
        int total = a + b;
        return total * 2;
    }

    public static void main(String[] args) {
        Counter c = new Counter();
        c.increment();
        System.out.println(c.sum(3, 4) + c.count);
    }
}
```

Compile, run, and look at what was produced TERMINAL

```
$ javac Counter.java
$ ls
Counter.class  Counter.java
$ java Counter
15
```

The result is 15 because `sum(3, 4)` is `(3 + 4) * 2 = 14` and `count` is 1. Note the asymmetry: you give `javac` a **file name** (`Counter.java`) but you give `java` a **class name** (`Counter`, no `.class`). The launcher looks the class up through the *classpath*, which we meet later in this chapter.

> **GOTCHA**
> Errors come in two families. A **compile-time error** (missing semicolon, wrong type) stops `javac`; no class file is produced. A **run-time error** (an exception, `ClassNotFoundException`, `NoSuchMethodError`) happens in `java`, after a successful compile. When you see a message, first ask which tool printed it.

1.2.2 What is inside a class file

A `.class` file is a compact binary format defined by the JVM Specification. Its first four bytes are always the hexadecimal number `CAFEBABE` (a joke by the original designers), followed by the version:

The first 16 bytes of Counter.class TERMINAL

```
$ od -A d -t x1 -N 16 Counter.class
0000000 ca fe ba be 00 00 00 41 00 27 0a 00 02 00 03 07
0000016
```

After `ca fe ba be` come the *minor version* (`00 00`) and the *major version* (`00 41`, which is 65 in decimal). The major version equals the Java release plus 44: Java 8 is 52, Java 17 is 61, Java 21 is 65 and Java 25 is 69. A JVM refuses to load a class whose major version is higher than it supports; that is the source of `UnsupportedClassVersionError`.

The rest of the file is structured, and `javap -v` prints it. In order:

- **Constant pool**: a table of every name, string literal, number and member reference the class uses. Instructions refer to entries by number (`#7`) instead of repeating names, which keeps the file small.
- **Access flags and identity**: public or final, the name of this class and of its superclass, the list of interfaces.
- **Fields**: name, type, flags.
- **Methods**: name, descriptor, flags and a `Code` attribute holding the bytecode, the maximum operand stack depth and the number of local-variable slots.
- **Attributes**: extra data such as the source file name, line numbers for stack traces, and, for records or nested classes, their special metadata.

Header of "javap -v Counter" (trimmed) TERMINAL

```
$ javap -v Counter
Classfile /.../Counter.class
  size 598 bytes
  Compiled from "Counter.java"
public class Counter
  minor version: 0
  major version: 65
  flags: (0x0021) ACC_PUBLIC, ACC_SUPER
  this_class: #8                          // Counter
  super_class: #2                         // java/lang/Object
  interfaces: 0, fields: 1, methods: 4, attributes: 1
Constant pool:
   #1 = Methodref          #2.#3          // java/lang/Object."<init>":()V
   #2 = Class              #4             // java/lang/Object
   ...
```

Notice `methods: 4`. We wrote three methods, and the compiler added a fourth: a **default constructor**, `Counter()`, because we wrote none. Any class without a constructor gets one that simply calls the superclass constructor.

1.2.3 Reading bytecode with javap

`javap -c -p Counter` disassembles the code (`-c`) and includes private members (`-p`). Bytecode is stack-based: instructions push values onto an **operand stack** and pop them to compute. Local variables live in numbered slots; in an instance method slot 0 is `this`, then the parameters follow.

The methods of Counter TERMINAL

```
$ javap -c -p Counter
Compiled from "Counter.java"
public class Counter {
  private int count;

  public Counter();
    Code:
       0: aload_0
       1: invokespecial #1     // Method java/lang/Object."<init>":()V
       4: return

  public void increment();
    Code:
       0: aload_0
       1: dup
       2: getfield      #7     // Field count:I
       5: iconst_1
       6: iadd
       7: putfield      #7     // Field count:I
      10: return

  public int sum(int, int);
    Code:
       0: iload_1
       1: iload_2
       2: iadd
       3: istore_3
       4: iload_3
```

The methods of Counter (continued) TERMINAL

```
       5: iconst_2
       6: imul
       7: ireturn
...
```

Walk through `sum(3, 4)`. Slots: 0 = `this`, 1 = `a`, 2 = `b`, 3 = `total`.

Offset	Instruction	Effect (operand stack, top on the right)
0	`iload_1`	push a: [3]
1	`iload_2`	push b: [3, 4]
2	`iadd`	pop two ints, push their sum: [7]
3	`istore_3`	pop into slot 3 (total = 7): []
4	`iload_3`	push total: [7]
5	`iconst_2`	push the constant 2: [7, 2]
6	`imul`	multiply: [14]
7	`ireturn`	return the top of the stack (14) to the caller

Executing sum(3, 4) step by step.

The prefix letter tells the type: `i` is int, `a` is a reference, `l` long, `d` double. That is why the JVM verifier can check types without running the code.

Now look at `count++` in `increment()`: `aload_0` pushes `this`, `dup` duplicates it, `getfield` reads `count`, `iconst_1` and `iadd` add one, `putfield` writes it back. A single Java statement became **three separate steps** (read, add, write). If two threads run these steps at once they can overwrite each other's update, and this is exactly why `count++` is not thread-safe. The Part on concurrency returns to this.

1.2.4 Four ways to invoke a method

Look at the `main` method of the same class and notice that calls use different instructions:

The call instructions in main (trimmed) TERMINAL

```
$ javap -c -p Counter
...
  public static void main(java.lang.String[]);
    Code:
       0: new           #8     // class Counter
       3: dup
       4: invokespecial #13    // Method "<init>":()V
       7: astore_1
       8: aload_1
       9: invokevirtual #14    // Method increment:()V
      12: getstatic     #17    // Field java/lang/System.out:Ljava/io/PrintStream;
...
      18: invokevirtual #23    // Method sum:(II)I
...
      26: invokevirtual #27    // Method java/io/PrintStream.println:(I)V
```

- `invokespecial`: constructors, `super` calls and private methods; no overriding is possible, so the target is fixed.
- `invokevirtual`: ordinary instance methods. The target is chosen from the *run-time* class of the object, which is how overriding (polymorphism) works.
- `invokestatic`: static methods.
- `invokeinterface`: a method called through an interface type.
- `invokedynamic` (since Java 7, used by the compiler since 8 and 9): lambdas and string concatenation. The call site is linked the first time it runs. The chapters on lambdas and strings explain it.

`new` only allocates an uninitialised object; `invokespecial <init>` is what runs the constructor. That is why `new` is followed by `dup`: one copy of the reference is consumed by the constructor call and one remains to be stored.

> **TIP**
> When something in Java surprises you (string concatenation, `i++` inside an expression, autoboxing, a `switch` on strings) compile a tiny class and run `javap -c -p`. Bytecode is the ground truth of what the compiler did.

1.2.5 jshell: try code without a project

`jshell` is an interactive Java shell (a REPL, Read-Eval-Print Loop, since Java 9). You type expressions, statements, methods or class declarations, and each is evaluated at once. No `main`, no semicolons needed for expressions, and the most common packages are already imported.

A short jshell session TERMINAL

```
$ jshell
jshell> int x = 5
x ==> 5
jshell> x * 2
$2 ==> 10
jshell> String greet(String n) { return "Hi " + n; }
|  created method greet(String)
jshell> greet("Asha")
$4 ==> "Hi Asha"
jshell> var list = new ArrayList<Integer>()
list ==> []
jshell> list.add(3)
$6 ==> true
jshell> int y = "a"
|  Error:
|  incompatible types: java.lang.String cannot be converted to int
|  int y = "a";
|          ^-^
jshell> /exit
|  Goodbye
```

Expressions that are not assigned get a generated name such as `$2` that you can reuse. The command `/vars` lists variables, `/methods` lists methods, `/imports` shows what is imported (by default `java.util.*`, `java.io.*`, `java.util.stream.*`, `java.util.function.*`, `java.math.*` and a few more), `/list` shows your history and `/reset` clears the state. Use jshell for quick questions such as "what does `Integer.parseInt(" 5")` do?"; it is the best way to test your understanding while reading this book.

1.2.6 Launching a source file directly

Since Java 11 you can run a single source file without a separate compile step:

Single-file source launch TERMINAL

```
$ java Counter.java
15
$ ls
Counter.java
```

The launcher compiles the file **in memory** and runs the first class it finds. No `.class` file appears. This is ideal for exercises, scripts and the listings in this book. Its limits: one file (Java 22 added support for launching a program made of several source files), no separate output directory, and the class loader is a special in-memory one, so some reflection or class-loading experiments behave differently from a normal `javac` plus `java` run. When a program grows beyond a few files, compile normally.

1.2.7 Packages, directories and the classpath

A **package** is a namespace for classes. A class `Book` in package `com.example.lib` has the full name `com.example.lib.Book`, begins with a `package` declaration, and must live in a matching directory structure. By convention package names are reversed domain names, all lower case.

src/com/example/lib/Book.java JAVA

```java
package com.example.lib;

public record Book(String title, int pages) {
    public static void main(String[] args) {
        System.out.println(new Book("Core Java", 500));
    }
}
```

Use `-d` to tell `javac` where to put class files; it creates the package directories for you:

Compile into an output directory and run with the classpath TERMINAL

```
$ javac -d out src/com/example/lib/Book.java
$ find out -type f
out/com/example/lib/Book.class
$ java -cp out com.example.lib.Book
Book[title=Core Java, pages=500]
$ java -cp out Book
Error: Could not find or load main class Book
Caused by: java.lang.ClassNotFoundException: Book
```

The **classpath** (`-cp` or `--class-path`) is a list of directories and jar files where the JVM looks for classes; separate entries with `:` on Linux/macOS and `;` on Windows. The class is found by turning its full name into a path below each classpath entry. That is why `Book` fails: the real name is `com.example.lib.Book`, and the JVM would look for `out/Book.class`. The most frequent classpath problems are the wrong name, the wrong root directory, and a missing dependency jar, which shows up as `ClassNotFoundException` or `NoClassDefFoundError`.

If you do not set a classpath, the default is the current directory. Never rely on the `CLASSPATH` environment variable; it silently affects every program on the machine.

1.2.8 Jar files

A **jar** (Java ARchive) is a ZIP file of class files and resources, with an optional `META-INF/MANIFEST.MF` describing it. You ship libraries and applications as jars.

Build and run a jar TERMINAL

```
$ jar --create --file lib.jar --main-class com.example.lib.Book -C out .
$ jar tf lib.jar
META-INF/
META-INF/MANIFEST.MF
com/
com/example/
com/example/lib/
com/example/lib/Book.class
$ java -jar lib.jar
Book[title=Core Java, pages=500]
```

`-C out .` means "change into `out` and add everything there"; `--main-class` writes the `Main-Class` manifest entry so that `java -jar` knows where to start. In real projects Maven or Gradle builds jars for you, and Spring Boot produces an "executable jar" that contains all dependencies inside it. Because jar files are just ZIP archives, `unzip -l` also works on them.

INTERVIEW ANGLE

What happens when you run `javac Hello.java` and then `java Hello`? `javac` parses and type-checks the source and writes `Hello.class` containing bytecode. `java` starts a JVM, which loads `Hello` through the class loaders (searching the classpath), links and initialises it, and invokes `main`. The interpreter executes the bytecode; methods that run often are compiled to machine code by the JIT.

Check yourself

1. What are the first four bytes of every class file, and what does major version 65 mean?
2. In `sum(int a, int b)` of an instance method, which local-variable slot holds `b`?
3. What is wrong here? The file is `Greeter.java` in package `com.acme`, compiled with `javac Greeter.java`, then run with `java Greeter`.
4. Why does the single statement `count++` compile to several bytecode instructions, and why does that matter?
5. Which instruction runs a constructor, and which one only allocates the object?
6. `java -cp out Book` fails although `out/com/example/lib/Book.class` exists. What is the fix?

Answers

1. `CA FE BA BE`. Major version 65 means the file was produced for Java 21 (release number plus 44).
2. Slot 2: slot 0 is `this`, slot 1 is `a`, slot 2 is `b`.
3. The class name is `com.acme.Greeter`, so the run command must be `java com.acme.Greeter` (from a directory where `com/acme/Greeter.class` exists, or with `-cp` pointing at that root). Compiling with `-d out` keeps the layout correct.

4. It reads the field, adds one, and writes it back as separate steps. Another thread can interleave between them, so the increment is not atomic.
5. `invokespecial` of `<init>` runs the constructor; `new` only allocates an uninitialised object.
6. Use the full name: `java -cp out com.example.lib.Book`.

KEY POINTS

`javac` turns source into `.class` files of bytecode; `java` loads a class by its full name from the classpath and runs `main`.

- A class file starts with `CAFEBABE` and a version (release + 44); it holds a constant pool, fields, methods and attributes.
- `javap -c -p` shows the bytecode: a stack machine with numbered local slots and typed instructions.
- `jshell` is for quick experiments and `java File.java` runs one source file without a class file.
- Package names must match directories; the classpath lists the roots where classes are searched; a jar is a ZIP of class files.

1.3 JVM Architecture

The JVM is the part of Java that most developers use daily and understand least. It decides when your classes are loaded, where objects live, how a method call is executed, and why an application sometimes becomes slow or crashes with `OutOfMemoryError`. This chapter gives you the structure of the machine. Later Parts (memory, garbage collection, JIT, diagnostics) go deeper into each box; here you learn what the boxes are and how data flows between them.

1.3.1 The big picture

Think of the JVM as three cooperating subsystems around one shared memory:

- The **class loading subsystem** finds `.class` data, checks it and turns it into a runtime representation of a class.
- The **runtime data areas** are the memory the JVM manages: heap, metaspace, thread stacks and so on.
- The **execution engine** runs bytecode: an interpreter, JIT compilers and a garbage collector.

Native method interfaces (JNI and the newer Foreign Function and Memory API) connect it to the operating system and C libraries.

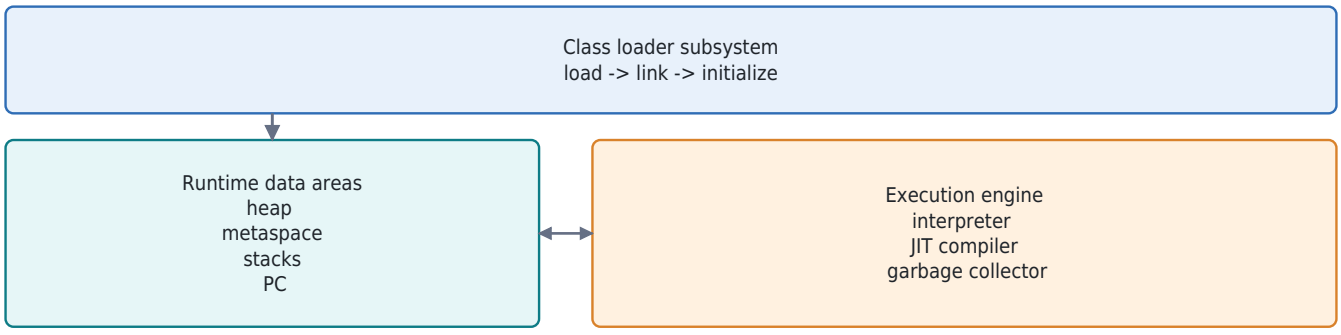
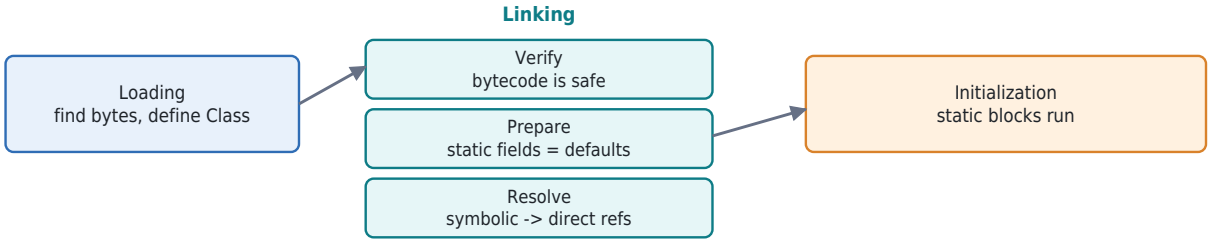


Figure 1.2. The JVM's three subsystems. Follow one class: the loader brings it in, its data lands in the memory areas, and the execution engine runs it.

1.3.2 Class loading: load, link, initialize

The JVM does not load every class at start-up. A class is loaded **lazily**, the first time the program needs it. For each class the JVM goes through three phases.



initialization runs your code, once, the first time the class is actively used

Figure 1.3. The three phases of class loading and the steps inside linking. Notice that initialization is the only phase that runs your code.

1. Loading

A class loader locates the binary data (a file on the classpath, a jar entry, bytes from a network or generated in memory), parses it and creates a `Class` object that represents the class in the heap, with the class's metadata stored in metaspace. Superclasses and interfaces are loaded first.

2. Linking

Linking has three steps:

- **Verification**: the JVM checks that the bytecode is well formed and type-safe: valid jumps, correct operand types, no illegal access. This is what makes it safe to run code from an untrusted source and it is a core reason why Java programs do not corrupt memory.
- **Preparation**: memory for `static` fields is allocated and set to default values (0, `false`, `null`). Not yet to the values you wrote.
- **Resolution**: symbolic references in the constant pool (like "method `sum` of class `Counter`") are replaced by direct references. HotSpot does this lazily, the first time an instruction needs it.

3. Initialization

The JVM runs the class's **static initializer**: the `static { }` blocks and the static field initializers in source order. It happens at most once per class, and the JVM guarantees it is thread-safe: if two threads trigger initialization together, one runs it and the other waits.

A class is initialized when it is first *actively used*: an instance is created with `new`, a static method is called, a non-constant static field is read or written, `Class.forName` is called, a subclass is initialized, or it is the class holding `main`. Merely declaring a variable of the type, or reading a compile-time constant, does not trigger it. This program proves it:

LazyInit.java JAVA

```java
public class LazyInit {
    static class Lazy {
        static { System.out.println("Lazy initialized"); }
        static final int CONST = 7;      // compile-time constant, inlined by javac
        static int value = 42;
    }

    public static void main(String[] args) {
        System.out.println("Constant: " + Lazy.CONST);
        System.out.println("Before value");
        System.out.println("Value: " + Lazy.value);
    }
}
```

Run LazyInit TERMINAL

```
$ java LazyInit.java
Constant: 7
Before value
Lazy initialized
Value: 42
```

What to notice: reading `CONST` printed 7 without running the static block, because `javac` copied the literal 7 into `main`'s bytecode. Reading `value` forced the class to initialize, so the message appears just before "Value: 42". You can see the same events in the JVM's own log with `-Xlog:class+load` and `-Xlog:class+init`.

GOTCHA
A static initializer that throws an exception makes the class unusable. The first use fails with `ExceptionInInitializerError`, and every later use fails with `NoClassDefFoundError: Could not initialize class X`. If you see that second message in a log, search backwards for the first failure; that is the real cause.

1.3.3 Class loaders and delegation

A **class loader** is an object that turns a class name into a `Class`. The important thing to understand is that a class's identity in the JVM is its **name plus the loader that defined it**. Two loaders can each load a class named `com.acme.Plugin`, and the JVM treats them as two unrelated types (a cast between them fails with `ClassCastException`, a classic problem in application servers and plugin systems).

Since Java 9 the JVM has three built-in loaders arranged in a parent-child chain:

Loader	Loads	Notes
Bootstrap	Core modules such as `java.base` (`String`, `Object`, `List`)	Written in native code. `getClassLoader()` returns `null` for its classes.
Platform	Other JDK modules (for example `java.sql`)	Name `platform`. Replaced the old "extension" loader.
Application (system)	Your classes on the classpath/module path	Name `app`. Default loader for `main`.

Built-in class loaders (Java 9 and later).

The loaders follow the **parent delegation model**: when asked to load a class, a loader first asks its parent; only if the parent cannot find it does the loader try itself.

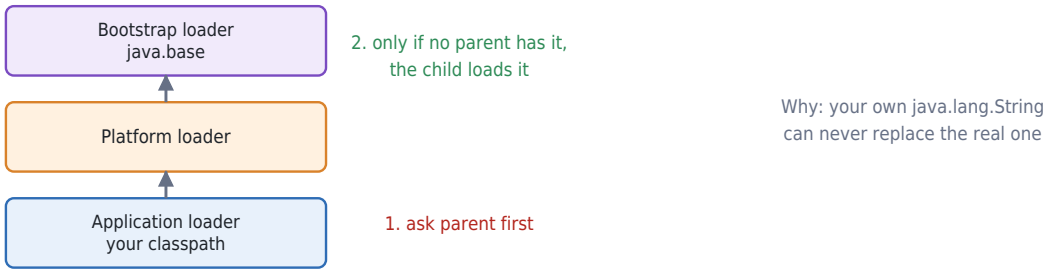


Figure 1.4. Delegation: a request starts at the application loader, goes up to the bootstrap loader, and the first loader that can load the class wins. Notice the guarantee this gives.

Delegation matters for two reasons. It is a **security guarantee**: you cannot replace `java.lang.String` with your own class, because the bootstrap loader will always supply the real one first. And it avoids duplicates: a class is normally loaded by the topmost loader that can supply it. You can see the chain yourself (compiled with `javac`, not the source launcher):

Loaders.java JAVA

```java
public class Loaders {
    public static void main(String[] args) {
        System.out.println("String loader : " + String.class.getClassLoader());
        System.out.println("Logger loader : "
            + java.util.logging.Logger.class.getClassLoader());
        ClassLoader app = Loaders.class.getClassLoader();
        System.out.println("App loader    : " + app.getName());
        System.out.println("Parent        : " + app.getParent().getName());
        System.out.println("Grandparent   : " + app.getParent().getParent());
    }
}
```

Compile with javac and run TERMINAL

```
$ javac Loaders.java && java Loaders
String loader : null
Logger loader : null
App loader    : app
Parent        : platform
Grandparent   : null
```

Both `String` and `Logger` print `null`: they belong to modules loaded by the bootstrap loader, whose "object" does not exist in Java. The application loader's parent is `platform`, and the platform loader's parent is the bootstrap loader, shown as `null`. If you run the same code with `java Loaders.java` the application loader's name is `null`, because the source launcher uses its own in-memory loader. That is the difference mentioned in the previous chapter.

Frameworks create their own loaders and sometimes break delegation on purpose: a web server gives each deployed web application its own loader so that two applications can use different versions of the same library; Spring Boot's executable jar uses a custom loader to read nested jars. A class loader holds references to all classes it defined, which is why an application that is redeployed repeatedly can leak metaspace if something keeps a reference to an old loader (see the Part on the JVM in depth).

INTERVIEW ANGLE
Explain the parent delegation model. Why does it exist? A loader first delegates a load request to its parent and only loads the class itself if the parent fails. This guarantees that core classes such as `java.lang.String` always come from the bootstrap loader (so they cannot be spoofed) and that a class is not loaded twice by different loaders in the same hierarchy. Frameworks may deliberately bypass it to isolate applications.

INTERVIEW ANGLE
What is the difference between `ClassNotFoundException` and `NoClassDefFoundError`? `ClassNotFoundException` is a checked exception thrown when code explicitly asks for a class by name (`Class.forName`, `loadClass`) and it is not found. `NoClassDefFoundError` is an `Error` thrown by the JVM when a class that was present at compile time cannot be found (or its initialization failed) at run time; typically a dependency is missing from the classpath or a static initializer threw.

1.3.4 Runtime data areas

When the JVM runs a program it organises memory into areas with different lifetimes. This list is one of the most-asked topics in interviews, and the picture is worth learning.

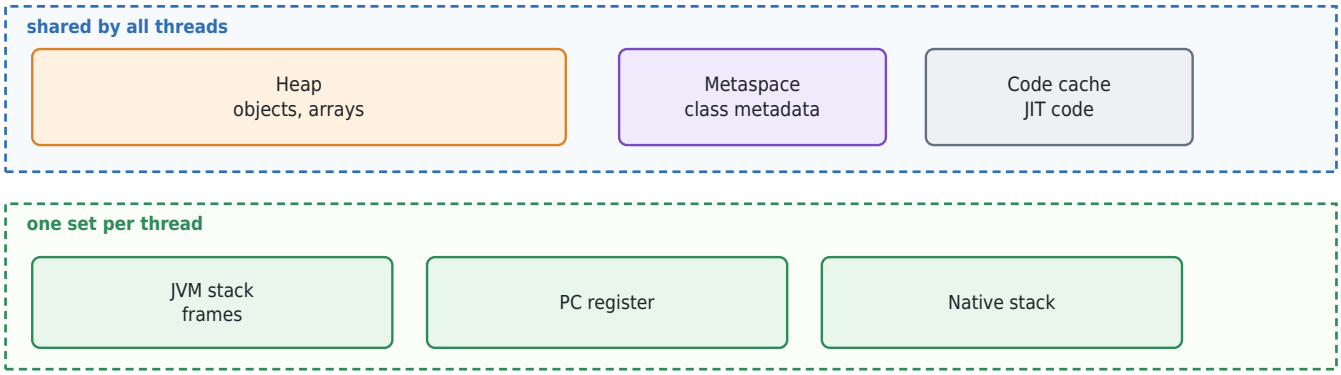


Figure 1.5. The runtime data areas. Notice which are shared by all threads (heap, metaspace) and which exist once per thread (stack, PC register, native stack).

Area	Shared?	What lives there
Heap	All threads	Every object and array created with `new`. Managed by the garbage collector. Sized by `-Xms` (initial) and `-Xmx` (maximum).
Metaspace	All threads	Class metadata: method code, constant pools, field and method descriptors. Lives in native memory, not in the heap. Replaced PermGen in Java 8.
Java thread stack	One per thread	Frames for method calls: local variables, partial results. Size set by `-Xss`.
PC register	One per thread	The address of the bytecode instruction the thread is executing.
Native method stack	One per thread	State of calls into native (C/C++) code.
Code cache	All threads	Machine code produced by the JIT compilers. Not part of the Java heap.

Runtime data areas.

Heap

The heap holds objects. When you write `new Counter()`, the object is allocated on the heap and you receive a **reference** to it. Local variables hold either primitive values or references; the objects themselves are not on the stack. The heap is divided into regions for young and old objects (generations), which is what makes garbage collection efficient; the Part on the JVM in depth explains it.

You can see the sizes the JVM chose for your machine. If you do not set `-Xms` and `-Xmx`, the JVM picks them from the memory it sees:

Default heap sizing on the build machine (8 GB RAM, 2 CPUs; your numbers will differ) TERMINAL

```
$ java -XX:+PrintFlagsFinal -version | grep -E "InitialHeapSize|MaxHeapSize|ThreadStackSize"
    size_t InitialHeapSize                          = 98566144          {product} {ergonomic}
    size_t MaxHeapSize                              = 1568669696        {product} {ergonomic}
     intx ThreadStackSize                           = 1024              {pd product} {default}
```

`{ergonomic}` means the JVM computed the value from the machine; `{default}` means a built-in constant. Here the heap starts at about 94 MB and may grow to about 1.5 GB. `ThreadStackSize = 1024` is in KB, so each thread's stack may grow to 1 MB on 64-bit Linux. `PrintFlagsFinal` lists every HotSpot flag with its final value, and it is the quickest way to see what the JVM actually decided.

> **NOTE**
> Heap sizing is derived from the physical memory (or the container's limit when running in Docker or Kubernetes); by default the maximum is a fraction of it, commonly a quarter. In containers always check the real number with `PrintFlagsFinal` or set `-Xmx` or `-XX:MaxRAMPercentage` explicitly.

Metaspace

Since Java 8, class metadata lives in **metaspace** in native memory. By default it can grow until the operating system refuses (the `MaxMetaspaceSize` flag is effectively unlimited), which is why a class-loader leak ends in `OutOfMemoryError: Metaspace` only when you set a limit or the machine runs out. Before Java 8 this area was the fixed-size PermGen, and `PermGen space` errors were common.

Thread stacks, PC register and native stacks

Every thread gets its own stack, so local variables of one thread are invisible to other threads (which is why local variables are inherently thread-safe, but fields of shared objects are not). The PC register holds the current instruction of a thread that is running Java code. The native method stack serves calls into native code through JNI.

1.3.5 Stack frames and the operand stack

Each time a method is called, the JVM pushes a **frame** onto the calling thread's stack; when the method returns, the frame is popped. A frame holds three things:

- the **local variable array**: `this` (for instance methods), the parameters and the local variables, in numbered slots;
- the **operand stack**: the working area bytecode instructions use (you saw `iload`, `iadd` in the previous chapter);
- **frame data**: a reference to the constant pool of the method's class, and what is needed to return a value or throw an exception.

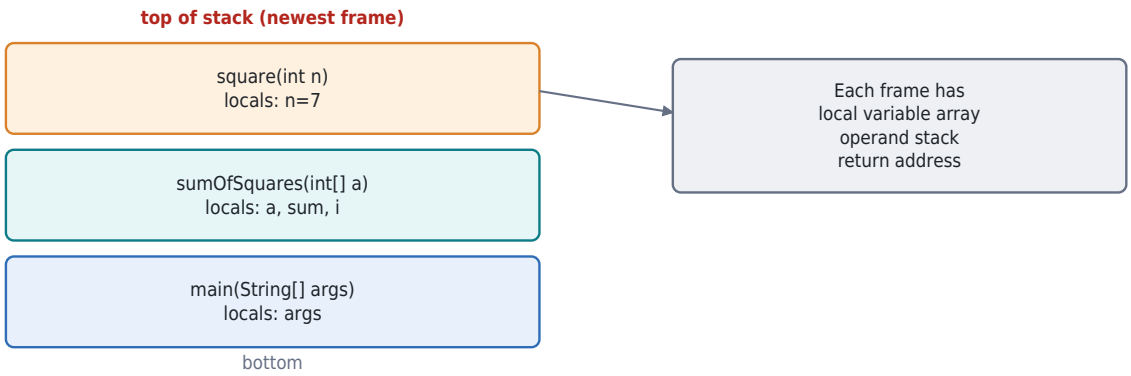


Figure 1.6. A thread's stack with frames for main, sumOfSquares and square. Notice that the newest frame is on top and that each frame has its own locals and operand stack.

Here is a program whose frames are easy to follow. The compiler decided how large each frame must be and recorded it in the class file (`stack=` and `locals=` in `javap -v`), so the JVM allocates exactly that much.

Frames.java JAVA

```java
public class Frames {
    static int square(int n) { return n * n; }

    static int sumOfSquares(int a, int b) {
        int x = square(a);
        int y = square(b);
        return x + y;
    }

    public static void main(String[] args) {
        System.out.println(sumOfSquares(3, 4));
        new Throwable("where am I?").printStackTrace();
    }
}
```

Run Frames; the second output is the live call stack at that moment TERMINAL

```
$ javac Frames.java && java Frames
25
java.lang.Throwable: where am I?
        at Frames.main(Frames.java:10)
```

`printStackTrace()` is simply a readout of the stack of frames. Here only `main` is on the stack, so one line is printed. When `square(a)` runs, there would be three frames: `main`, `sumOfSquares`, `square`. The bytecode of `sumOfSquares` shows the pattern: push the argument, `invokestatic`, store the result in a local:

sumOfSquares in bytecode (javap -c -p Frames)

```
  static int sumOfSquares(int, int);
    Code:
       0: iload_0
       1: invokestatic  #7        // Method square:(I)I
       4: istore_2
       5: iload_1
       6: invokestatic  #7        // Method square:(I)I
       9: istore_3
      10: iload_2
      11: iload_3
      12: iadd
      13: ireturn
```

The method has 4 local slots (a, b, x, y); it needs an operand stack of depth 2 (two ints to add). Because every thread's stack has a limited size (1 MB by default here), unbounded recursion eventually exhausts it:

Deep.java JAVA

```java
public class Deep {
    static int depth = 0;

    static void recurse() {
        depth++;
        recurse();
    }

    public static void main(String[] args) {
        try {
            recurse();
        } catch (StackOverflowError e) {
            System.out.println("StackOverflowError after many calls: " + (depth > 1000));
        }
    }
}
```

Run Deep TERMINAL

```
$ java Deep.java
StackOverflowError after many calls: true
```

The exact depth varies with the JIT state and the frame size (typically some thousands to tens of thousands of calls), so the program prints only a boolean. If you need deeper recursion you can raise the stack with `-Xss`, but the better fix is

usually to turn the recursion into a loop. Note that `StackOverflowError` is an `Error` (a JVM-level problem), but, as shown, it can be caught.

> **GOTCHA**
> Primitives and references are stored in frames, but objects are not. After `int[] a = new int[1000];` the frame holds only a reference (a few bytes); the array's 1000 ints are on the heap. This is why passing a large array to a method is cheap, and why two variables can refer to one object.

1.3.6 The execution engine

The execution engine turns bytecode into actions. The JVM uses a combination of three techniques that together are called **adaptive optimisation**.

The interpreter

At first every method is run by the **bytecode interpreter**: it reads each instruction and performs it. Interpretation starts instantly and needs no compilation time, but it is slow, roughly an order of magnitude slower than compiled code.

The JIT compilers

The JVM counts how often each method is called and how many times loops iterate. Methods that become "hot" are compiled to native machine code by the **JIT** (just-in-time) compiler while the program runs. HotSpot has two compilers:

- **C1** (the client compiler) compiles quickly and produces reasonably fast code.
- **C2** (the server compiler) compiles slowly but applies heavy optimisations (inlining, escape analysis, loop unrolling), using profile data collected earlier, such as which branches are taken and which types actually appear at a call site.

By default HotSpot uses **tiered compilation**, which combines them:

```
Tier 0   interpreter          runs the code and starts collecting counters
Tier 1-3 C1 compiled code     tier 3 adds full profiling (collects data for C2)
Tier 4   C2 compiled code     the fastest code; produced for the hottest methods
```

You can watch the compiler work with `-XX:+PrintCompilation`. In this tiny program `square` is called 200,000 times:

Hot.java JAVA

```java
public class Hot {
    static int square(int x) { return x * x; }

    public static void main(String[] args) {
        long sum = 0;
        for (int i = 0; i < 200_000; i++) {
            sum += square(i);
        }
        System.out.println(sum);
    }
}
```

Typical output (timestamps vary); columns: ms, id, flags, tier, method, bytecode size TERMINAL

```
$ java -XX:+PrintCompilation Hot | grep -E "Hot::|^[0-9]{6,}$"
188    6       3       Hot::square (4 bytes)
188    7       4       Hot::square (4 bytes)
189    6       3       Hot::square (4 bytes)   made not entrant
201    8 %     3       Hot::main @ 4 (32 bytes)
202    9       3       Hot::main (32 bytes)
202   10 %     4       Hot::main @ 4 (32 bytes)
17450708985056
217    8 %     3       Hot::main @ 4 (32 bytes)   made not entrant
```

How to read it: `square` was compiled first at tier 3 (C1 with profiling) and almost immediately at tier 4 (C2); once the tier-4 version is ready, the tier-3 version is "made not entrant", meaning new calls no longer use it. The `%` marks **OSR** (on-stack replacement): `main`'s loop was so hot that the JVM swapped the running interpreted loop for compiled code in the middle of its execution. The big number at the end is the program's own output. Running with `-Xint` forces interpretation only; the answer is identical, only slower, which shows the JIT never changes meaning, only speed.

The thresholds that decide "hot" are tuning flags (on this JDK `Tier3InvocationThreshold` is 200 and `Tier4InvocationThreshold` is 5000 calls), but the real decision also depends on loop back-edges and compile-queue

load, so do not memorise numbers; know the idea. This is also why **benchmarks need warm-up**: the first few seconds of a program are not representative.

> **REAL WORLD**
> When a freshly deployed service is slow for the first minute and then fast, that is the JIT warming up (plus caches filling). Large systems mitigate it with warm-up requests before routing traffic, or with ahead-of-time caches of class data and profiles, which recent JDKs support. You tune this in the Part on JIT and tuning.

1.3.7 Garbage collection in one page

You never call `free`. The **garbage collector** (GC) periodically finds objects that can no longer be reached from the program and reclaims their memory. "Reachable" means: starting from the **GC roots** (local variables in all thread stacks, static fields, JNI references and a few others), follow references; everything visited is live, everything else is garbage. An object does not have to be set to `null` to be collected; it only has to become unreachable.

Most objects die young (a temporary string, an iterator, a request object). The heap is therefore organised in **generations**: new objects are allocated in the young generation, which is collected often and cheaply, and the few survivors are promoted to the old generation, which is collected rarely. The Part on the JVM in depth covers the algorithms.

HotSpot offers several collectors; you select one with a flag.

Flag	Collector	Best for
`-XX:+UseSerialGC`	Serial	Tiny heaps, single CPU, small tools.
`-XX:+UseParallelGC`	Parallel	Throughput: batch jobs where short pauses do not matter.
`-XX:+UseG1GC`	G1	The default on normal servers (since Java 9): balanced throughput and pause times.
`-XX:+UseZGC`	ZGC	Very low pauses on big heaps; generational by default since Java 23.

HotSpot collectors you will meet.

If you do not choose, the JVM picks: G1 on a "server-class" machine and Serial on a small one (fewer than 2 CPUs or less than about 1.8 GB of memory). Java 27 makes G1 the default in all environments. You can ask for a log of what the collector does with `-Xlog:gc`:

Garbage.java JAVA

```java
import java.util.ArrayList;
import java.util.List;

public class Garbage {
    public static void main(String[] args) {
        List<byte[]> keep = new ArrayList<>();
        for (int i = 0; i < 3000; i++) {
            byte[] junk = new byte[100_000];     // becomes garbage immediately
            if (i % 500 == 0) {
                keep.add(junk);                  // stays reachable
            }
        }
        System.out.println("kept " + keep.size());
    }
}
```

Typical output (timings vary); 7M->1M(64M) = heap used before -> after (total heap) TERMINAL

```
$ java -Xmx64m -Xlog:gc Garbage
[0.044s][info][gc] Using G1
[0.335s][info][gc] GC(0) Pause Young (Normal) (G1 Evacuation Pause) 7M->1M(64M) 4.193ms
[0.387s][info][gc] GC(1) Pause Young (Normal) (G1 Evacuation Pause) 15M->1M(64M) 9.999ms
[0.502s][info][gc] GC(2) Pause Young (Normal) (G1 Evacuation Pause) 38M->2M(64M) 9.321ms
...
kept 6
```

The program allocated about 300 MB in total, yet the heap never exceeded 64 MB: after each young collection only 1 to 5 MB remained live, because the `junk` arrays were unreachable. Only `keep` and what it references survived. The first line, `Using G1`, confirms which collector the JVM chose on this 2-CPU, 8 GB machine.

If live data really exceeds the heap, the collector cannot help:

Heap exhaustion — TERMINAL

```
$ java -Xmx32m Oom
Exception in thread "main" java.lang.OutOfMemoryError: Java heap space
        at Oom.main(Oom.java:6)
```

where `Oom` adds `new long[1_000_000]` to a list forever. The usual causes in real systems are not infinite loops but unbounded caches, static collections that only grow, and per-request data kept alive too long. See the Part on diagnostics for how to find them with heap dumps.

WATCH OUT
`System.gc()` does not "free memory now". It is only a request, the JVM may ignore it, and calling it in production code usually hurts latency. Let the collector decide when to run.

1.3.8 Putting it together: one program's life

Follow `java Frames` from start to end. The launcher starts a JVM and sizes the heap. The application loader loads `Frames` (loading, verification, preparation), the JVM initializes it and finds `main`. The interpreter runs `main` in a frame on the main thread's stack; `invokestatic` calls create further frames. `System.out` is looked up, its class loaded from the bootstrap loader (already in a shared archive, so cheap). Objects such as the `Throwable` go on the heap and become garbage afterwards. Hot methods, in a longer-running program, would be compiled by C1 and C2 into the code cache. When `main` returns and no non-daemon threads remain, the JVM exits. Every later Part zooms into one of these steps.

Check yourself

1. Which runtime areas are shared by all threads, and which exist once per thread?
2. What does this print, and why?

JAVA

```java
public class Q2 {
    public static void main(String[] args) {
        System.out.println(Config.NAME);
        System.out.println(Config.port);
    }
}

class Config {
    static { System.out.println("init Config"); }
    static final String NAME = "app";
    static int port = 8080;
}
```

1. `String.class.getClassLoader()` returns `null`. Does that mean `String` has no class loader?
2. Why can an application with a memory leak in its static collections still run for hours before dying with `OutOfMemoryError`, while a runaway recursion fails within milliseconds?
3. What are the three phases of class loading, and which one runs your static blocks?
4. Name the JIT tiers' purpose: why does HotSpot start with an interpreter instead of compiling everything up front?

Answers

1. Shared: heap, metaspace, code cache. Per thread: Java stack, PC register, native method stack.
2. It prints `app`, then `init Config`, then `8080`. `NAME` is a compile-time constant inlined by `javac`, so reading it does not initialize `Config`; reading `port` does. (The same pattern was run in this chapter with `LazyInit`.)
3. No. It means the class was loaded by the bootstrap loader, which is native code and is represented by `null`.
4. A leak slowly fills the heap, which can be 1 GB or more, so it takes hours. Recursion fills a thread stack of about 1 MB with a frame per call, so `StackOverflowError` comes within a fraction of a second.
5. Loading, linking (verify, prepare, resolve) and initialization. Initialization runs static initializers.
6. Most code runs only a few times, so compiling it all would waste start-up time. The interpreter starts instantly; only hot methods are compiled, at higher optimisation levels as they prove their worth.

KEY POINTS

The JVM = class loading subsystem + runtime data areas + execution engine.

- Classes load lazily: load, link (verify, prepare, resolve), then initialize on first active use. Class identity is name plus loader; loaders delegate to their parent first.
- Heap (objects) and metaspace (class metadata) are shared; each thread has its own stack of frames with locals and an operand stack.
- Code starts in the interpreter; the JIT (C1 then C2) compiles hot methods to machine code, so warm-up matters.
- The garbage collector reclaims unreachable objects; G1 is the default on server-class machines. Know the common `OutOfMemoryError` and `StackOverflowError` causes.

1.4 Your First Programs and the Toolchain

You now know what happens between source and output. This chapter makes you productive: programs that take input, the compiler options worth knowing, how a real project is laid out, naming conventions that every Java reader expects, how to write documentation comments, and above all how to read a stack trace, the skill that separates people who debug from people who guess.

1.4.1 Hello with arguments

The smallest useful program uses the command-line arguments that arrive in `main`'s `String[] args` parameter.

Hello.java JAVA

```java
public class Hello {
    public static void main(String[] args) {
        if (args.length == 0) {
            System.out.println("Hello, world");
        } else {
            for (String name : args) {
                System.out.println("Hello, " + name);
            }
        }
    }
}
```

Run it with and without arguments TERMINAL

```
$ java Hello.java
Hello, world
$ java Hello.java Asha Ravi
Hello, Asha
Hello, Ravi
```

What to notice about the signature `public static void main(String[] args)`: it must be `public` (the launcher is outside your class), `static` (no object exists yet), `void` (the return value is not used; exit status is set with `System.exit`), and take a `String[]`. The array is empty, never `null`, when no arguments are given. Arguments are separated by spaces; quote an argument that contains a space (`"Asha Rao"`). Everything arrives as a `String`; use `Integer.parseInt(args[0])` to convert and be ready for `NumberFormatException`.

NOTE

Java 25 finalised "instance main methods" and "compact source files" (JEP 512): a file can be just `void main() { IO.println("Hi"); }`. This is meant for beginners and scripts. It cannot be run on the JDK 21 used to build this book, so the listings here use the classic form, which works on every version and is what you will see in real code.

1.4.2 Reading input with Scanner

`java.util.Scanner` reads and parses text from a source; `System.in` is the standard input stream.

Greeter.java JAVA

```java
import java.util.Scanner;

public class Greeter {
    public static void main(String[] args) {
        try (Scanner in = new Scanner(System.in)) {
```

Greeter.java (continued) JAVA

```
            System.out.print("Name? ");
            String name = in.nextLine();
            System.out.print("Age? ");
            int age = in.nextInt();
            System.out.println(name + " will be " + (age + 1) + " next year");
        }
    }
}
```

Feeding input from a pipe (typing at the prompt works the same) TERMINAL

```
$ printf 'Asha\n20\n' | java Greeter.java
Name? Age? Asha will be 21 next year
```

`try (Scanner in = ...)` is a *try-with-resources* statement: it closes the scanner at the end of the block, even on an exception (the Part on exceptions explains it fully). `nextLine()` reads a whole line; `nextInt()` reads the next whitespace-separated token and parses it. If the token is not a number you get an exception, which is also our first stack trace:

Typing a word where a number is expected TERMINAL

```
$ printf 'Asha\ntwenty\n' | java Greeter.java
Name? Age? Exception in thread "main" java.util.InputMismatchException
        at java.base/java.util.Scanner.throwFor(Scanner.java:947)
        at java.base/java.util.Scanner.next(Scanner.java:1602)
        at java.base/java.util.Scanner.nextInt(Scanner.java:2267)
        at java.base/java.util.Scanner.nextInt(Scanner.java:2221)
        at Greeter.main(Greeter.java:9)
```

Read it from the bottom: your code `Greeter.main` at line 9 called `nextInt`, which called `next`, which called `throwFor`, which threw. We will make that a systematic habit later in this chapter.

GOTCHA

The most famous Scanner trap: `nextInt()` leaves the newline after the number unread, so a following `nextLine()` returns an empty string instead of the next line. With input `20` then `Asha`, the code `int age = in.nextInt(); String name = in.nextLine();` gives `name` equal to `""`. Fix: call `in.nextLine()` once after `nextInt()` to consume the rest of the line, or read every line with `nextLine()` and parse it with `Integer.parseInt`.

Since `System.in` is a stream of bytes, there are other ways to read input (`BufferedReader` for speed, `java.io.Console` for passwords); the Part on I/O covers them. For programs that handle large input, `Scanner` is noticeably slower than `BufferedReader`, which matters in programming contests.

1.4.3 Terminal, IDE or both?

Both. An **IDE** (IntelliJ IDEA, Eclipse, VS Code with the Java extensions) gives code completion, instant error highlighting, refactoring, a visual debugger and project management. The **terminal** shows you what the tools actually do. A learner should do the first chapters of any book on the terminal (as here), so that `javac`, `java`, classpath and packages are not hidden, and then switch to an IDE for larger work. The IDE runs the same `javac` and `java` underneath; whatever it does you can reproduce on the command line, which is how you debug a build that "works in the IDE but not in CI".

Whichever you use, make sure it uses the JDK you intended: the project's SDK setting in the IDE has to match `java -version` in the terminal.

1.4.4 javac flags worth knowing

You will rarely need many options. These are the ones that matter:

Option	Purpose
`-d dir`	Directory for generated class files (creates the package folders).
`-cp path` or `--class-path path`	Where to find already compiled classes and libraries.
`--release N`	Compile against the Java N API and class-file format. Fails if you use an API newer than N. Prefer it to `-source`/`-target`.
`-Xlint:all`	Turn on all optional warnings (raw types, unchecked casts, deprecation).
`-Werror`	Treat warnings as errors; good for CI.

Option	Purpose
`-g`	Include debug information (variable names). Default is line numbers and source file only.
`-parameters`	Keep method parameter names in the class file (used by frameworks such as Spring and by reflection).
`-encoding UTF-8`	Source file encoding; on the current JDK it already defaults to UTF-8 (since Java 18).
`-proc:none`	Disable annotation processors (Lombok, MapStruct need them, so use with care).

Common javac options.

`--release` is the one that prevents production surprises. If your servers run Java 17, compile with `--release 17` even on a JDK 25 laptop, and `javac` will reject code that uses a method which does not exist in 17. Plain `-source 17 -target 17` does not do that check.

Warnings help: raw types compile but are flagged TERMINAL

```
$ javac Warn.java
Note: Warn.java uses unchecked or unsafe operations.
Note: Recompile with -Xlint:unchecked for details.
$ javac -Xlint:all Warn.java
Warn.java:4: warning: [rawtypes] found raw type: List
        List raw = new ArrayList();
        ^
  missing type arguments for generic class List<E>
...
Warn.java:5: warning: [unchecked] unchecked call to add(E) as a member of the raw type List
        raw.add("x");
            ^
3 warnings
```

Compile errors read the same way: file, line, message, the offending line, a caret.

A syntax error: the compiler reports only what it can understand first TERMINAL

```
$ javac Bad.java
Bad.java:4: error: ';' expected
        System.out.println(n)
                           ^
1 error
```

Here line 3 also has a type error (`int n = "five";`), but `javac` reports the syntax error first and the type error appears only after you fix it. Compile errors can hide behind earlier ones, and one missing brace can cause a screen full of messages, so always fix the first error and recompile.

1.4.5 Project layout, Maven and Gradle

Real projects have more than one file, dependencies, tests and a build. Two build tools dominate. This book uses them only as concepts, because the Core Java material runs with plain `javac`.

Maven describes a project in a `pom.xml` and follows "convention over configuration": the layout is fixed, so every Maven project looks the same.

```
my-app/
  pom.xml                    build description, dependencies, plugins
  src/
    main/
      java/com/example/app/  production code (package = directory)
        App.java
      resources/             config files, templates (copied to the classpath)
    test/
      java/com/example/app/  test code (JUnit)
        AppTest.java
  target/                    build output (generated; never commit it)
```

pom.xml XML

```
<project xmlns="http://maven.apache.org/POM/4.0.0">
  <modelVersion>4.0.0</modelVersion>
  <groupId>com.example</groupId>
  <artifactId>my-app</artifactId>
  <version>1.0.0</version>
  <properties>
```

pom.xml (continued) XML

```
    <maven.compiler.release>21</maven.compiler.release>
  </properties>
  <dependencies>
    <dependency>
      <groupId>org.junit.jupiter</groupId>
      <artifactId>junit-jupiter</artifactId>
      <version>5.10.2</version>
      <scope>test</scope>
    </dependency>
  </dependencies>
</project>
```

The commands you will see most are `mvn compile`, `mvn test`, and `mvn package` (builds the jar into `target/`). A dependency is identified by `groupId:artifactId:version`; Maven downloads it and its own dependencies from a repository (Maven Central) into a local cache, then puts them on the classpath for you. **Gradle** does the same job with a script (`build.gradle` or `build.gradle.kts`) and uses the identical `src/main/java` layout. Its commands are `gradle build` and `gradle test`, and most projects ship a wrapper script `./gradlew` so that everyone uses the same Gradle version. (The Part on tooling shows both in more detail.)

REAL WORLD
Spring Boot applications are Maven or Gradle projects whose main class carries `@SpringBootApplication`. The `src/main/resources` directory holds `application.properties` or `application.yml`. When the build fails with "package does not exist", it is almost always a missing dependency in `pom.xml`, which is the Maven version of a `ClassNotFoundException` at compile time.

1.4.6 Naming conventions

Java code is read far more often than it is written, and the community has settled on conventions that everyone follows. Following them is part of being understood.

What	Rule	Examples
Class, interface, record, enum	UpperCamelCase, a noun	`BankAccount`, `Comparable`, `OrderStatus`
Method	lowerCamelCase, a verb	`deposit`, `findById`, `isEmpty`
Variable, parameter, field	lowerCamelCase, meaningful	`balance`, `studentCount`
Constant (`static final`)	UPPER_SNAKE_CASE	`MAX_RETRIES`, `DEFAULT_PORT`
Package	lowercase, reversed domain, no underscores	`com.example.library`
Type parameter	one capital letter	`T`, `E`, `K`, `V`

Naming conventions.

Other habits: one public top-level class per file, and the file name equals the class name (`BankAccount.java`); a boolean is named like a question (`isActive`, `hasStock`); avoid abbreviations (`customerName`, not `custNm`); use braces even for one-line `if` bodies. Java is case-sensitive: `Main` and `main` are different names, and `String` with a capital S is the class while `string` does not exist.

1.4.7 Comments and Javadoc

Java has three kinds of comments:

Acct.java JAVA

```
/**
 * A simple bank account.
 *
 * @author Maruthi
 */
public class Acct {
    private long balance;   // end-of-line comment: balance in paise

    /**
     * Adds money to the account.
     *
```

Acct.java (continued) JAVA

```java
     * @param amount the amount to deposit, must be positive
     * @throws IllegalArgumentException if amount is not positive
     */
    public void deposit(long amount) {
        /* A block comment may span
           several lines. */
        if (amount <= 0) throw new IllegalArgumentException("amount must be > 0");
        balance += amount;
    }
}
```

`//` comments to the end of a line and `/* ... */` comments a block. The `/** ... */` form is a **Javadoc comment**: the `javadoc` tool turns it into HTML API documentation, and IDEs show it when you hover over a method. Tags used most: `@param`, `@return`, `@throws`, `@see`, `{@code x}` for inline code and `{@link Type}` for links. Since Java 23 documentation comments may also be written in Markdown (lines starting with `///`).

Generate documentation into the folder jd TERMINAL

```
$ javadoc -quiet -d jd Acct.java
$ ls jd
Acct.html  allclasses-index.html  allpackages-index.html  ...
```

A good comment explains *why*, not what: `// retry because the gateway drops the first request after idle` is useful, `// add one to i` is noise. Public API methods deserve Javadoc; most private code deserves clear names instead of comments.

1.4.8 How to read a stack trace

A **stack trace** is the call stack at the moment an exception was created. It is the single most useful piece of information in a failing program, and many beginners skip it. Here is a program with a bug three calls deep:

Trace.java JAVA

```java
public class Trace {
    static int parse(String s) {
        return Integer.parseInt(s);
    }

    static int total(String[] items) {
        int sum = 0;
        for (String s : items) {
            sum += parse(s);
        }
        return sum;
    }

    public static void main(String[] args) {
        System.out.println(total(new String[] {"1", "2", "x3"}));
    }
}
```

Run Trace (compiled with javac) TERMINAL

```
$ javac Trace.java && java Trace
Exception in thread "main" java.lang.NumberFormatException: For input string: "x3"
        at java.base/java.lang.NumberFormatException.forInputString(NumberFormatException.java:67)
        at java.base/java.lang.Integer.parseInt(Integer.java:662)
        at java.base/java.lang.Integer.parseInt(Integer.java:778)
        at Trace.parse(Trace.java:3)
        at Trace.total(Trace.java:9)
        at Trace.main(Trace.java:15)
```

Method for reading any trace:

1. **First line**: the thread (`main`), the exception class (`NumberFormatException`) and its message (`For input string: "x3"`). The message usually contains the bad value. Often this alone tells you the problem.
2. **Frames, from the top**: the top frame is where the exception was created; the lines below are the callers, in order, down to `main` at the bottom. Each line is `class.method(File.java:line)`.

3. **Find the first frame that is your code.** The first three lines are JDK internals (`java.base/...`): the JDK correctly rejected a bad input. The first line of *your* code is `Trace.parse(Trace.java:3)`: you passed it a bad string. Open that line.
4. **Then climb** to see who supplied the bad value: `total` at line 9 passed `s` from the array, and `main` at line 15 built that array. The bug is the data `"x3"`, not the JDK.

If a trace ends with `Caused by: ...` that is a **chained exception**: the lower exception is the root cause, and its own trace follows. Always read the last `Caused by` first, because it is where the problem began. Frameworks such as Spring produce traces with hundreds of lines; the same method works: skim for the `Caused by` lines and for frames in your own package (`com.example...`).

TIP

`NullPointerException` messages are helpful in modern Java. Since Java 14 (on by default since 15) they say what was null: `Cannot invoke "String.length()" because "name" is null`. Read the message before reaching for the debugger. (If the class was compiled without `-g`, a local variable shows as `<local1>` instead of its name; compile with `-g`, which build tools do by default for the variable names.)

INTERVIEW ANGLE

What is the difference between a compile-time error and a run-time exception, and where does a stack trace come from? A compile-time error is found by `javac` before the program runs (syntax, type mismatch, unreported checked exception) and prevents a class file. A run-time exception occurs while the JVM executes; when it is created the JVM records the call stack, which is printed if nothing catches it. So the trace shows the path of calls that led to the throw, not necessarily where the faulty data was produced.

Check yourself

1. What is the output of `java Hello.java "Asha Rao" Ravi` (using the Hello of this chapter)?
2. A program reads `int age = in.nextInt(); String name = in.nextLine();` with input `20`, newline, `Asha`. What does `name` contain and how do you fix it?
3. You must produce a jar that runs on Java 17 servers but you build on JDK 25. Which flag and why?
4. In the trace above, which line should you open first, and why not the first line of the trace?
5. Classify the names: `MAX_SIZE`, `orderTotal`, `OrderService`, `com.acme.billing`. Which kind of identifier is each?
6. What is wrong with `class Student { void Print() { } }` by convention, and does it compile?

Answers

1. Two lines: `Hello, Asha Rao` and `Hello, Ravi`. Quotes keep the two words together as one argument.
2. The empty string. `nextInt()` does not consume the end of the line, so `nextLine()` returns the rest of line one. Call `nextLine()` once more before reading the name.
3. `--release 17`. It compiles against the Java 17 API and class-file format and rejects newer APIs; `-source`/`-target` do not check the API.
4. `Trace.parse(Trace.java:3)`: it is the first frame in your own code; the JDK frames above it only report the failure. The root cause (a bad string) was introduced by the data in `main`.
5. Constant, local variable or field, class name, package name.
6. The method should be `print` (lowerCamelCase). It compiles; conventions are not enforced by the compiler.

KEY POINTS

Programs receive input through `args` and `Scanner`; every value from `args` is a `String`.

- Use `javac -d`, `--release`, and `-Xlint:all`; compile errors are reported file, line, message, caret, and the first one matters.
- Maven and Gradle give a standard layout (`src/main/java`, `src/test/java`) and manage dependencies; the Core Java material needs only `javac`.
- Follow the naming conventions and write Javadoc (`/** */`) for public API.
- Read a stack trace by the first line (type and message), then the first frame of your own code, then the `Caused by` chain.

1.5 How to Study This Book

Reading about Java does not teach Java. Programming is a skill, like playing an instrument, and it is learned by doing, failing and fixing. This short chapter gives you a method that works with every other chapter, so that after finishing the book you can also *use* what you know and *explain* it in an interview.

1.5.1 The read-predict-run-break loop

For every listing, follow four steps:

1. **Read** the code and the explanation once, without typing anything.
2. **Predict** the output on paper (or in your head) before you run it. If you cannot predict it, you do not yet understand it, and that gap is exactly what to study.
3. **Run** it. If your prediction was wrong, do not just accept the real output. Find out *why*, using the text, `javap` or the documentation.
4. **Break** it: change one thing (remove a line, change a type, make a value `null`) and predict again. Errors teach more than successes. Every "what goes wrong" callout in this book is a ready-made experiment.

Then close the book and write a small variation from memory. If you cannot, repeat the chapter's examples, not by copying, but by retyping them. Typing slowly forces you to notice details that reading hides.

All listings here are small and complete so that this works. Listings marked "(Java 25; not run on 21)" need a newer JDK than the one used to run the book's code; if you have JDK 25 installed you can run those too.

1.5.2 Using jshell as a notebook

jshell is the fastest way to answer "what happens if...?" while studying. Keep it open in a second terminal window. Useful commands:

Command	Use
`` `/vars`, `/methods`, `/types` ``	List what you have defined in this session.
`` `/list` ``	Show everything you typed, with ids; `/list -all` includes failed snippets.
`` `/edit` ``	Open a snippet in an editor to correct it.
`` `/open File.java` ``	Load a source file into the session.
`` `/save file.jsh` ``	Save the session as a script for later.
`` `/imports`, `/reset`, `/exit` ``	Show the imports, start fresh, leave.
`` `/help` ``	Everything else. Tab completion works on classes and methods; press Tab twice after `Math.` to list methods.

jshell commands for a study session.

Typical question you answer in 10 seconds: what does `7 / 2` give, what is `-7 % 3`, what is `"a" == new String("a")`, what is the result of `Integer.valueOf(127) == Integer.valueOf(127)`. Predict first, then check.

1.5.3 Debugging basics

When a program does not behave, resist the urge to change code randomly. Debug like a scientist: form a hypothesis, test it, narrow down.

- **Read the error.** A compile error tells you the file and line. A stack trace gives the exact path to the failure (see the previous chapter). Many "mysterious" bugs are solved by reading the message slowly.
- **Reproduce it** with the smallest input that fails. Delete code until the problem disappears, then put back the last piece. This is the quickest way to find the cause, and it gives you the example you need to ask for help.
- **Print debugging.** `System.out.println("x=" + x)` at key points is crude, fine for small programs, and teaches you to think about values. Remove the prints after use; in real applications use a logging framework with levels.
- **Use the debugger.** Every IDE lets you set a **breakpoint** on a line, run in debug mode, and then step: *step over* (run this line), *step into* (enter the called method), *step out* (finish this method). In the variables panel you see the values; the call stack panel shows exactly the frames described in the JVM chapter. The command-line debugger `jdb` exists in every JDK, but an IDE debugger is far more convenient. Spend one hour learning your IDE's debugger; it pays back for years.

- **Question your assumptions.** Is it really the version of the file you think? Did the code compile (a stale `.class` file runs old code)? Is the right JDK used? Many hours are lost on things that are not bugs in the code.

1.5.4 Reading documentation

Professional developers look things up constantly. The standard library is documented in **Javadoc**, and reading it is a skill to practise from day one.

- The API reference for the current release is at `docs.oracle.com/en/java/javase/25/docs/api/`; every release has the same structure under its version number. Search by class name, for example `java.util.ArrayList`.
- On a class page read in this order: the **class description** (what it is for and key guarantees, often with the important caveats), then the **method summary** table, then the details of the methods you use. Note `@since` (the version that added it), `@throws`, and phrases such as "not thread-safe", "optional operation", or "implementation note".
- The **Java Language Specification** (JLS) and **JVM Specification** are the final authority for language rules and bytecode behaviour. They are dry, but for a precise question ("is this initializer order guaranteed?") the answer is there.
- **JEPs** (JDK Enhancement Proposals, at `openjdk.org/jeps/<number>`) explain why a feature exists and how it works. They are excellent for understanding newer features such as virtual threads (JEP 444) or records.
- `dev.java/learn` is the official tutorial site. Use blog posts and answers on forums as hints, not as truth; check them against Javadoc.

You can open the Javadoc for any JDK class from the command line too: `javap java.util.ArrayList` prints the public API of a class as signatures, which is often quicker than a browser when you only need a method name.

The public API of a class in your terminal (excerpt) TERMINAL

```
$ javap java.util.Optional | head -6
Compiled from "Optional.java"
public final class java.util.Optional<T> {
  public static <T> java.util.Optional<T> empty();
  public static <T> java.util.Optional<T> of(T);
  public static <T> java.util.Optional<T> ofNullable(T);
  public T get();
```

1.5.5 How the book is organised and how to use it

The Parts build on one another. Part 1 (this one) gives the mental model of the platform. Parts 2 and 3 teach the language and object-oriented design; 4 and 5 cover strings, time, exceptions and I/O; 6 and 7 the collections and the functional features; 8 concurrency; 9 and 10 the JVM internals and diagnostics; 11 and 12 modern features, tooling and good design; and 13 to 15 give projects, interview preparation and a study plan with cheat sheets.

How to use the callouts: `INTERVIEW` boxes contain a question and a crisp answer; try answering aloud before reading. `GOTCHA` and `WARN` boxes are the bugs that cost people days. `REAL` boxes link a topic to how it appears in Spring and production systems. Each chapter ends with "Check yourself" questions; answer them in writing *before* reading the answers, and re-do the ones you missed after a week (spaced repetition beats cramming). If you are short of time, read the Parts in order but spend your effort where interviews concentrate: collections, concurrency, the JVM memory model, and Java 8 to 21 features.

Check yourself

1. Why is predicting the output before running a program more useful than just running it?
2. What is the quickest way to test what `-7 % 3` evaluates to without creating a file?
3. A program prints a stack trace. In which order do you read it, and what do you look for?
4. What is the difference between step over and step into in a debugger?
5. Where do you find out in which Java version a method was added?

Answers

1. Prediction exposes the exact point where your mental model is wrong, and that is the thing to learn. Just running only confirms what you already know.
2. Type it into `jshell`; the result (`$1 ==> -1`) appears immediately.

3. First line (exception type and message), then find the first frame of your own code from the top, then follow the callers; if there is a `Caused by`, read the last one first for the root cause.
4. Step over executes the current line as a whole, including any method it calls. Step into enters the called method so you can watch inside it.
5. The `@since` tag in the method's Javadoc entry.

KEY POINTS

Learn by the loop: read, predict, run, break; typing code beats reading code.

- `jshell` answers small questions in seconds; keep it open while you study.
- Debug with hypotheses: read the message, reproduce minimally, print or use a debugger, check your assumptions.
- Javadoc, the JLS and JEPs are primary sources; use `@since` and `@throws` and read the class description first.
- Do the Check yourself questions in writing and revisit the ones you missed.

PART 2

Language Fundamentals

The building blocks every Java program is made of: data, operators, control flow, arrays, methods and text.

In this part

- 2.1 Variables, Primitive Types and Literals
- 2.2 Operators and Expressions
- 2.3 Type Conversion, Casting and Promotion
- 2.4 Control Flow
- 2.5 Arrays
- 2.6 Methods
- 2.7 Text Blocks and Basic I/O

2.1 Variables, Primitive Types and Literals

Every program manipulates data, and the first decision Java forces on you is the *type* of that data. A type tells the compiler how many bits a value occupies, how those bits are interpreted, and which operations are legal. Java is **statically typed**: the type of every variable and expression is known at compile time, so a whole class of mistakes (adding a boolean to a string-length, passing a name where a count is expected) is rejected before the program ever runs. This chapter covers the eight primitive types, how variables are declared and initialised, the literal forms you can write in source code, and the most common first bug in any language with fixed-size integers: overflow.

2.1.1 Variables and the two kinds of types

A **variable** is a named storage location with a declared type. You declare it once and then assign to it:

Declarations.java — JAVA

```java
public class Declarations {
    public static void main(String[] args) {
        int age = 21;              // declaration with initialiser
        double gpa;                // declaration only
        gpa = 8.7;                 // assignment later
        int a = 1, b = 2, c;       // several in one statement (use sparingly)
        final int MAX_SEATS = 60;  // cannot be reassigned
        System.out.println(age + " " + gpa + " " + (a + b) + " " + MAX_SEATS);
    }
}
```

Java has exactly two families of types.

- **Primitive types** hold the value itself. There are eight of them, fixed by the language: `byte`, `short`, `int`, `long`, `float`, `double`, `char`, `boolean`. They are not objects, have no methods, and can never be `null`.
- **Reference types** (classes, interfaces, arrays, enums, records) hold a *reference* to an object on the heap. `String`, `ArrayList` and `int[]` are reference types. A reference variable can be `null`, meaning "refers to nothing".

Every primitive has a matching **wrapper class** (`Integer`, `Long`, `Double`, `Character`, `Boolean` and so on) used where an object is required, for example inside `List<Integer>`. Boxing and unboxing between the two is covered in the chapter on type conversion and in Part 6.

2.1.2 The eight primitives

Type	Size	Range	Typical use
`byte`	8 bits	`-128 to 127`	raw bytes, binary I/O
`short`	16 bits	`-32,768 to 32,767`	rarely used
`int`	32 bits	`about -2.1 billion to 2.1 billion`	the default for whole numbers
`long`	64 bits	`about -9.2e18 to 9.2e18`	ids, timestamps, large counts
`float`	32 bits	`about 3.4e38, 6-7 digits`	graphics, large arrays of decimals
`double`	64 bits	`about 1.8e308, 15-16 digits`	the default for decimals
`char`	16 bits	`0 to 65,535 (unsigned)`	one UTF-16 code unit
`boolean`	not specified	`true or false`	conditions

The primitive types. Sizes are fixed by the language and do not depend on the machine.

Do not memorise the ranges; derive them. An n-bit signed integer in two's complement holds -2^(n-1) to 2^(n-1) - 1, so `byte` is -128..127 and `int` is -2^31..2^31-1. The wrapper classes expose the limits as constants, and you should print them instead of trusting memory:

Limits.java JAVA

```java
public class Limits {
    public static void main(String[] args) {
        System.out.println("byte   " + Byte.MIN_VALUE + " .. " + Byte.MAX_VALUE);
        System.out.println("short  " + Short.MIN_VALUE + " .. " + Short.MAX_VALUE);
        System.out.println("char   " + (int) Character.MIN_VALUE + " .. "
            + (int) Character.MAX_VALUE);
        System.out.println("int    " + Integer.MIN_VALUE + " .. " + Integer.MAX_VALUE);
        System.out.println("long   " + Long.MIN_VALUE + " .. " + Long.MAX_VALUE);
        System.out.println("float  " + Float.MIN_VALUE + " .. " + Float.MAX_VALUE);
        System.out.println("double " + Double.MIN_VALUE + " .. " + Double.MAX_VALUE);
        System.out.println("bits: " + Byte.SIZE + " " + Short.SIZE + " " + Character.SIZE + " "
            + Integer.SIZE + " " + Long.SIZE + " " + Float.SIZE + " " + Double.SIZE);
    }
}
```

Run it TERMINAL

```
$ java Limits.java
byte   -128 .. 127
short  -32768 .. 32767
char   0 .. 65535
int    -2147483648 .. 2147483647
long   -9223372036854775808 .. 9223372036854775807
float  1.4E-45 .. 3.4028235E38
double 4.9E-324 .. 1.7976931348623157E308
bits: 8 16 16 32 64 32 64
```

GOTCHA

`Float.MIN_VALUE` and `Double.MIN_VALUE` are the smallest *positive* values, not the most negative. The most negative double is `-Double.MAX_VALUE`. Integer `MIN_VALUE` constants are genuinely negative, which is a common source of confusion.

Notice that the range is asymmetric for integers: there is one more negative value than positive. That is why `Math.abs(Integer.MIN_VALUE)` is still negative, as you will see shortly.

NOTE

The size of `boolean` is not defined by the language. The JVM typically uses a full byte for a `boolean` field or local and one byte per element in a `boolean[]`, but the specification leaves it open. Do not reason about it in bits.

2.1.3 Default values and definite assignment

Fields (variables declared in a class) and array elements receive a default value if you do not initialise them. **Local variables never do.** The compiler refuses to let you read a local before it is *definitely assigned*:

Defaults.java JAVA

```java
public class Defaults {
    static int count;
    static boolean ready;
    static double ratio;
    static char letter;
    static String name;

    public static void main(String[] args) {
        System.out.println(count + " " + ready + " " + ratio + " [" + (int) letter + "] " + name);
        int[] a = new int[2];
        System.out.println(a[0]);
        int local;
        // System.out.println(local);  // compile error: might not have been initialized
    }
}
```

Run it TERMINAL

```
$ java Defaults.java
0 false 0.0 [0] null
0
```

The rule: numeric types default to zero, `char` to the character with code 0 (`'\u0000'`), `boolean` to `false`, and every reference type to `null`. The compiler's insistence on initialising locals is a feature: a stale default would hide a bug, whereas a compile error points at it.

2.1.4 Literals

A **literal** is a value written directly in source code. Java gives you several notations:

Kind	Example	Notes
`decimal int`	`42`	type int; add L for long: 42L
`hex, binary, octal`	`0x1F, 0b1010, 017`	31, 10, 15; a leading 0 means octal
`underscores`	`1_000_000`	readability only; ignored by the compiler
`floating point`	`3.14, 1e3, 2.5f`	double by default; f or F makes it float
`char`	`'A', '\n', '\u0041'`	single quotes; escapes and Unicode escapes
`String`	`"hello"`	double quotes; a reference type, not a primitive
`boolean`	`true, false`	the only two values

Literal forms.

Two details cause real bugs. First, an integer literal is an `int` unless it ends in `L`; if the value does not fit in an `int` the compiler rejects it (`long x = 3000000000;` does not compile, `3000000000L` does). Second, a literal with a decimal point is a `double`, so `float f = 3.14;` is a compile error (a double does not fit in a float); write `3.14f`.

Use `L`, never lowercase `l`, because `l` looks like the digit 1. And avoid a leading zero on an integer: `017` is 15, not 17.

2.1.5 Type inference with var

Since Java 10 you can write `var` for a local variable and let the compiler infer the type from the initialiser:

VarDemo.java JAVA

```java
import java.util.ArrayList;

public class VarDemo {
    public static void main(String[] args) {
        var n = 10;                       // int
        var d = 10.0;                     // double
        var l = 10L;                      // long
        var list = new ArrayList<String>();
```

VarDemo.java (continued) JAVA

```
        System.out.println(((Object) n).getClass().getSimpleName() + " "
            + ((Object) d).getClass().getSimpleName() + " "
            + ((Object) l).getClass().getSimpleName() + " "
            + list.getClass().getSimpleName());
    }
}
```

Run it TERMINAL

```
$ java VarDemo.java
Integer Double Long ArrayList
```

`var` is not dynamic typing. The type is fixed at compile time; you simply did not spell it out. It works only for local variables with an initialiser (not fields, parameters or return types), and `var x = null;` or `var a = {1, 2};` do not compile because there is nothing to infer from. Use it when the type is obvious from the right-hand side (`var users = new ArrayList<User>()`) and avoid it when it hides information (`var result = service.process()`).

2.1.6 final, constants and naming

`final` on a variable means it can be assigned exactly once. For a primitive that makes it a constant; for a reference it means the *reference* cannot change, while the object it points to can still be mutated. A class-level constant is conventionally `static final` with an UPPER_SNAKE_CASE name. Variables and methods use `camelCase`, types use `PascalCase`.

2.1.7 Overflow: when the number does not fit

Integer arithmetic in Java **wraps around silently**. There is no exception and no warning; the result is the true value modulo 2^n, reinterpreted as a signed number. This follows from two's complement: after `Integer.MAX_VALUE` comes `Integer.MIN_VALUE`.

Overflow.java JAVA

```
public class Overflow {
    public static void main(String[] args) {
        int max = Integer.MAX_VALUE;
        System.out.println(max + 1);
        System.out.println(Integer.MIN_VALUE == max + 1);
        System.out.println(Math.abs(Integer.MIN_VALUE));
        byte b = (byte) 130;
        System.out.println(b);
        long big = 1_000_000 * 1_000_000;
        long right = 1_000_000L * 1_000_000;
        System.out.println(big + " " + right);
        try {
            Math.addExact(max, 1);
        } catch (ArithmeticException e) {
            System.out.println(e.getMessage());
        }
        char c = 'A';
        c += 2;
        System.out.println(c + " " + (int) c + " " + (char) (c + 1) + " " + (c + 1));
        System.out.println(0x1F + " " + 0b1010 + " " + 017 + " " + 1e3 + " " + 10_000L);
    }
}
```

Run it TERMINAL

```
$ java Overflow.java
-2147483648
true
-2147483648
-126
-727379968 1000000000000
integer overflow
C 67 D 68
31 10 15 1000.0 10000
```

What to notice:

- `max + 1` wraps to the minimum. `Math.abs(Integer.MIN_VALUE)` is still negative because +2^31 does not exist in an `int`.
- `(byte) 130` keeps only the low 8 bits: 130 is `1000 0010`, which as a signed byte is -126.
- `1_000_000 * 1_000_000` is computed entirely in `int` arithmetic and overflows *before* being widened to `long`. The `L` on one operand makes the multiplication happen in `long`. Assigning to a `long` variable does not rescue an `int` expression.
- `Math.addExact`, `subtractExact`, `multiplyExact` and `toIntExact` throw `ArithmeticException` instead of wrapping. Use them whenever overflow would be a bug (money in cents, quantities, sizes).
- `char` is a number: `c += 2` turns `'A'` into `'C'`, and `c + 1` is an `int` (68), not a char.

WATCH OUT
Overflow is silent. A classic production bug is computing a size, a timestamp difference or a total in `int` and getting a negative or tiny number. Decide the type from the largest value the quantity can plausibly reach, not from what feels natural, and use the `*Exact` methods at boundaries you care about.

2.1.8 char is a number, and Unicode in one paragraph

A `char` is an unsigned 16-bit integer holding a UTF-16 code unit. `'A'` is 65, `'a'` is 97, `'0'` is 48. That is why `'a' + 1` is the `int` 98, and why `(char) ('a' + 1)` is `'b'`. Characters outside the Basic Multilingual Plane (most emoji, some rare scripts) need two `char`s, a *surrogate pair*, so a single `char` cannot always represent one visible character. The chapters on strings in Part 4 deal with code points properly; for now remember that `char` means "UTF-16 unit", not "letter".

INTERVIEW ANGLE
What is the difference between `int` and `Integer`? `int` is a primitive: it stores the value directly, cannot be `null`, and has no methods. `Integer` is its wrapper class: an object on the heap that can be `null`, can be used in generics (`List<Integer>`), and carries utility methods. Java converts between them automatically (autoboxing/unboxing), but boxing allocates objects (apart from a small cache for -128..127), so prefer primitives in hot loops and a `null Integer` throws `NullPointerException` when unboxed.

Check yourself

1. What does this print, and why? `byte b = 127; b++; System.out.println(b);`
2. What is wrong with `long total = 1_000_000 * 3_000;`, and how do you fix it?
3. Why does `float f = 3.14;` fail to compile, and which two changes fix it?
4. A local `int count;` is declared but not assigned. A field `int count;` is declared but not assigned. What does reading each give?
5. What does `char c = 'a'; System.out.println(c + 1);` print? What would print `b`?
6. Why is `Math.abs(Integer.MIN_VALUE)` negative?

Answers

1. `-128`. The increment wraps: 127 + 1 overflows an 8-bit signed byte to -128.
2. Both operands are `int`, so the product (3,000,000,000) overflows to -1294967296 before it is widened to `long`. Fix: `1_000_000L * 3_000`.
3. `3.14` is a `double`, and narrowing double to float needs an explicit cast. Write `3.14f`, or `(float) 3.14`.
4. The local is a compile error ("might not have been initialized"); the field is 0.
5. It prints `98` because `c + 1` is an `int`. `System.out.println((char) (c + 1))` prints `b`.
6. There is no +2147483648 in an `int`; the negation wraps back to `MIN_VALUE`.

KEY POINTS
- Java has eight primitives with fixed sizes; `int` is the default integer and `double` the default decimal.
- Fields get default values (0, false, null); locals must be assigned before use or the code does not compile.
- Integer overflow wraps silently; use a wider type or `Math.addExact` style methods when it matters.
- Literals are `int` and `double` unless suffixed (`L`, `f`); underscores, `0x`, `0b` help readability; a leading 0 means octal.
- `char` is an unsigned 16-bit number, so `char` arithmetic yields `int`.
- `var` infers a local's type at compile time; it is not dynamic typing.

2.2 Operators and Expressions

An **expression** combines values and operators into a new value; a **statement** does something. Most of the surprises in Java code come from a handful of operator behaviours: integer division truncates, `%` can be negative, `i++` and `++i` differ, compound assignment hides a cast, and `==` means different things for primitives and references. This chapter walks through each with a program you can run.

2.2.1 Operator groups and precedence

Precedence decides which operator binds first when there are no parentheses. You do not need the whole table by heart; you need the shape of it.

Level	Operators	Associativity
1	`a++ a-- (postfix), method call, array index, dot`	left to right
2	`++a --a (prefix), unary + - ~ ! , cast, new`	right to left
3	`* / %`	left to right
4	`+ - (also string concatenation)`	left to right
5	`<< >> >>>`	left to right
6	`< <= > >= instanceof`	left to right
7	`== !=`	left to right
8	`bitwise AND, then XOR, then OR`	left to right
9	`&& , then ||`	left to right
10	`?: (ternary)`	right to left
11	`= += -= *= and the other assignments`	right to left

Operators from highest to lowest precedence (simplified).

Memorise three consequences. Multiplicative beats additive. Comparison beats logical (`a < b && c < d` needs no parentheses). Bitwise `&` binds *looser* than `==`, so `x & 1 == 0` is parsed as `x & (1 == 0)` and fails to compile; write `(x & 1) == 0`. When in doubt, add parentheses; readers will thank you.

2.2.2 Arithmetic: division, remainder, increment

When both operands of `/` are integers, the result is an integer and the fraction is **truncated toward zero**. `%` is the matching remainder, defined so that `(a / b) * b + a % b == a`. Consequently the remainder takes the sign of the dividend (the left operand):

Ops.java JAVA

```java
public class Ops {
    public static void main(String[] args) {
        System.out.println(7 / 2 + " " + -7 / 2 + " " + 7 % 3 + " " + -7 % 3 + " "
            + 7 % -3 + " " + Math.floorMod(-7, 3));
        System.out.println(7 / 2.0 + " " + (double) (7 / 2));
        int i = 5;
        i = i++ + ++i;
        System.out.println(i);
        int j = 5;
        j = j++;
        System.out.println(j);
        byte b = 10;
        b += 300;
        System.out.println(b);
        int x = 3;
        x *= 2.5;
        System.out.println(x);
        String s = null;
        System.out.println(s != null && s.length() > 0);
        int calls = 0;
        boolean r = (calls++ > 0) & (calls++ > 0);
        System.out.println(calls + " " + r);
        calls = 0;
```

Ops.java (continued) JAVA

```
        r = (calls++ > 0) && (calls++ > 0);
        System.out.println(calls + " " + r);
        System.out.println(1 + 2 + "3" + 4 + 5);
        System.out.println(true ? 1 : 2.0);
    }
}
```

Run it TERMINAL

```
$ java Ops.java
3 -3 1 -1 1 2
3.5 3.0
12
5
54
7
false
2 false
1 false
3345
1.0
```

Going through the output line by line:

- `7 / 2` is 3 and `-7 / 2` is -3 (truncation toward zero, not floor). `-7 % 3` is -1 and `7 % -3` is 1. If you need a non-negative modulo, for example a circular array index, use `Math.floorMod(-7, 3)`, which gives 2.
- `7 / 2.0` is 3.5 because one operand is a `double`. `(double) (7 / 2)` is 3.0: the division already happened in integers, and the cast came too late.
- `i = i++ + ++i` with `i = 5`: the left operand `i++` yields 5 (then `i` becomes 6), the right operand `++i` makes `i` 7 and yields 7, so the sum is 12. Java evaluates operands strictly left to right, so this is well defined, but never write it.
- `j = j++` leaves `j` at 5. The old value is saved, `j` is incremented, then the assignment overwrites it with the saved old value.

Prefix versus postfix

`++i` increments first and the expression value is the *new* value. `i++` yields the *old* value and increments afterwards. As a standalone statement (`i++;`) there is no difference. In a `for` update clause, either is fine. Use the value of an increment inside a larger expression only when it is truly idiomatic, for example `array[count++] = x;`.

2.2.3 Compound assignment hides a cast

`a += b` is not exactly `a = a + b`. It is `a = (T) (a + b)` where `T` is the type of `a`. The implicit cast silently narrows:

- `byte b = 10; b += 300;` gives 54, because 310 truncated to 8 bits is 54. Writing `b = b + 300;` would not compile, because `b + 300` is an `int`.
- `int x = 3; x *= 2.5;` computes 7.5 as a `double` and casts it to the `int` 7.

This is convenient for `char c += 1` and `short s *= 2`, and dangerous when you did not intend a narrowing. If a compound assignment target is smaller than the result type, ask yourself whether the cast is what you want.

2.2.4 Boolean logic and short-circuiting

The operators `&&` and `||` are **short-circuit**: the right operand is evaluated only if the left does not already decide the result. That makes guards like `s != null && s.length() > 0` safe, because `s.length()` is never reached when `s` is `null`. The non-short-circuit forms `&` and `|`, applied to booleans, always evaluate both sides. In the demo, `(calls++ > 0) & (calls++ > 0)` ran both increments (`calls` is 2), whereas with `&&` only the first ran (`calls` is 1).

Use `&&` and `||` by default. Reserve `&` and `|` for bit manipulation, and be wary of side effects (`calls++`, method calls that modify state) inside conditions: whether they run depends on short-circuiting, which makes code hard to read.

2.2.5 The ternary operator and string +

`cond ? a : b` is an expression and returns a value. Its result type follows promotion rules: in `true ? 1 : 2.0` one branch is `double`, so the whole expression is `double` and prints `1.0`, not `1`. A more dangerous version mixes a boxed `Integer` that is `null` with an `int`: `cond ? nullInteger : 0` unboxes the `null` and throws `NullPointerException`. Keep ternaries short and do not nest them.

The `+` operator doubles as string concatenation. If either operand is a `String`, the other is converted and the strings are joined. Because `+` is left-associative, `1 + 2 + "3" + 4 + 5` is `(((1 + 2) + "3") + 4) + 5`, which is `"3" + "3"` then `"33" + 4` then `"334" + 5`: the output `3345`. To add numbers inside a message, parenthesise: `"sum=" + (a + b)`.

2.2.6 Bitwise operators and shifts

Bitwise operators work on the individual bits of integer types. They are used for flags, masks, hashing, compact storage and low-level protocols.

Op	Meaning	Example
&	AND: 1 only if both bits are 1	0b1100 & 0b1010 = 0b1000
or (pipe)	OR: 1 if either bit is 1	0b1100 or 0b1010 = 0b1110
^	XOR: 1 if the bits differ	0b1100 ^ 0b1010 = 0b0110
~	NOT: flips every bit	~12 = -13
<<	shift left, fill with zeros (multiplies by 2^n)	1 << 3 = 8
>>	arithmetic shift right, keeps the sign bit	-16 >> 2 = -4
>>>	logical shift right, fills with zeros	-16 >>> 28 = 15

Bitwise operators.

Bits.java JAVA

```java
public class Bits {
    public static void main(String[] args) {
        int a = 0b1100, b = 0b1010;
        System.out.println(Integer.toBinaryString(a & b) + " " + Integer.toBinaryString(a | b)
            + " " + Integer.toBinaryString(a ^ b) + " " + (~a));
        System.out.println((-16 >> 2) + " " + (-16 >>> 28) + " " + (-16 >>> 2) + " " + (1 << 33)
            + " " + (1L << 33));
        System.out.println(Integer.toBinaryString(-16));
        int flags = 0;
        final int READ = 1, WRITE = 2, EXEC = 4;
        flags |= READ | EXEC;
        System.out.println(flags + " " + ((flags & WRITE) != 0) + " " + ((flags & EXEC) != 0));
        flags &= ~EXEC;
        System.out.println(flags);
        System.out.println((5 & 1) == 1 ? "odd" : "even");
        System.out.println(Integer.bitCount(255) + " " + Integer.toHexString(255) + " "
            + Integer.highestOneBit(100));
    }
}
```

Run it TERMINAL

```
$ java Bits.java
1000 1110 110 -13
-4 15 1073741820 2 8589934592
11111111111111111111111111110000
5 false true
1
odd
8 ff 64
```

What to notice:

- `-16` in binary is `11111111111111111111111111110000`. `>>` copies the sign bit in from the left, so `-16 >> 2` stays negative (-4). `>>>` fills with zeros, so `-16 >>> 28` leaves just the top four bits, 15.
- `1 << 33` is 2, not 8589934592. For an `int`, the shift distance is masked to its low 5 bits (33 & 31 = 1). For a `long` it is masked to 6 bits. Use `1L << 33` when you want a 64-bit result.

- The flags idiom is the heart of permission sets and `EnumSet`: set a bit with `|=`, clear it with `&= ~MASK`, test it with `(flags & MASK) != 0`.
- `(n & 1) == 1` tests for odd numbers and works for negatives too (`n % 2 == 1` fails for negative odd `n`, because `-3 % 2` is -1).

2.2.7 Equality: == on primitives, references and floating point

For primitives, `==` compares values. For references it compares *identity*: do both variables refer to the same object? To compare content you call `equals`. The string and boxing consequences are treated in Part 4 and later; the key point here is that `==` on objects is almost never what you want.

For floating point, `==` is also a trap, because most decimal fractions have no exact binary representation. Compare with a tolerance:

FloatEq.java — JAVA

```java
public class FloatEq {
    public static void main(String[] args) {
        System.out.println(0.1 + 0.2 == 0.3);
        System.out.println(0.1 + 0.2);
        System.out.println(Math.abs(0.1 + 0.2 - 0.3) < 1e-9);
        System.out.println(0.5 + 0.25 == 0.75);
    }
}
```

Run it — TERMINAL

```
$ java FloatEq.java
false
0.30000000000000004
true
true
```

The last line is `true` because 0.5 and 0.25 are sums of powers of two and are exactly representable. The next chapter explains why 0.1 is not.

> **TIP**
> When you cannot remember how an expression evaluates, put it in jshell. `jshell> -7 % 3` answers instantly and never lies.

> **INTERVIEW ANGLE**
> What is the difference between `&&` and `&`? Between `>>` and `>>>`? `&&` is the short-circuit logical AND on booleans: it does not evaluate the right side when the left is false. `&` evaluates both sides when applied to booleans, and on integers is the bitwise AND. `>>` is the arithmetic right shift that preserves the sign bit, `>>>` the logical shift that fills with zero bits; they differ only for negative numbers.

Check yourself

1. What does `System.out.println(-7 % 3 + " " + -7 / 2);` print?
2. What does this print: `int x = 10; x = x-- - --x; System.out.println(x);`
3. `byte q = 10; q *= 30;` compiles. What is `q` afterwards, and why does `q = q * 30;` not compile?
4. What is `1 << 33` for an `int`? How do you get 2^33?
5. Why is `if (n % 2 == 1)` a buggy odd test, and what is a correct one?
6. What does `System.out.println(1 + 2 + "3" + 4 + 5);` print?

Answers

1. `-1 -3`. The remainder takes the sign of the dividend; division truncates toward zero.
2. `2`. `x--` yields 10 (x becomes 9), `--x` makes x 8 and yields 8, so 10 - 8 = 2, which then overwrites x.
3. `44`. The product 300 is cast back to `byte` (300 - 256). `q * 30` is an `int`, and plain assignment does not narrow implicitly.
4. `2`, because the shift count is masked to 5 bits (33 becomes 1). Use `1L << 33`.
5. For negative odd `n`, `n % 2` is -1. Use `n % 2 != 0` or `(n & 1) == 1`.
6. `3345`: the numbers add first (3), then everything after the String is concatenation.

KEY POINTS

- Integer `/` truncates toward zero and `%` takes the dividend's sign; use `Math.floorMod` for a non-negative modulo.
- `a op= b` means `a = (T)(a op b)`, a hidden narrowing cast.
- `&&` and `||` short-circuit; `&` and `|` do not (on booleans) and are bit operators on integers.
- Shift distances are masked (5 bits for `int`, 6 for `long`); `>>` keeps the sign, `>>>` fills zeros.
- Never compare floating-point results with `==`; compare within a tolerance, or use `BigDecimal`.
- When precedence is unclear, use parentheses: clarity beats cleverness.

2.3 Type Conversion, Casting and Promotion

Real programs constantly mix types: an `int` index into a `double` average, a `char` into a number, a `long` timestamp into an `int` field. Java has precise rules for what converts automatically, what needs your permission (a cast), and what the compiler flatly refuses. Knowing those rules lets you predict results such as `'a' + 1` or `0.1 + 0.2` instead of being surprised by them, and it explains the most common compile error beginners meet: "incompatible types: possible lossy conversion".

2.3.1 Widening and narrowing

A **widening** conversion goes to a type that can represent every value of the source (or close to it). It happens automatically. A **narrowing** conversion may lose information, so Java makes you write a **cast** to confirm you know.

Widening order (automatic, left to right)

```
byte → short → int → long → float → double
              char → int
```

Two oddities worth remembering. `long` to `float` (and `int` to `float`) is classed as widening even though it can lose precision: a `float` has only 24 bits of mantissa, so large integers are rounded. And `char` widens to `int` but `short`/`byte` do not widen to `char`, because `char` is unsigned and they are signed.

Narrowing casts behave differently depending on the kinds involved:

- **Integer to narrower integer**: the high bits are discarded, so only the low-order bits remain. `(byte) 200` is -56.
- **Floating point to integer**: the fraction is truncated toward zero; out-of-range values *saturate* to the integer's min or max; `NaN` becomes 0.
- **double to float**: rounded to the nearest float, possibly becoming infinity.

Casts.java JAVA

```
public class Casts {
    public static void main(String[] args) {
        int i = 'A' + 1;
        long l = i;
        float f = l;
        System.out.println(i + " " + l + " " + f);
        System.out.println((int) 3.99 + " " + (int) -3.99);
        System.out.println(Math.round(3.5) + " " + Math.round(-3.5) + " " + Math.round(2.5));
        System.out.println((int) 1e20 + " " + (long) 1e20 + " " + (int) Double.NaN);
        System.out.println((byte) 200 + " " + (short) 70000 + " " + (char) 65 + " " + (int) 'z');
        float big = 16_777_217;
        System.out.println(big + " " + (int) big);
        System.out.println(123456789 + " -> " + (float) 123456789);
        byte x = 10, y = 20;
        int sum = x + y;                // x and y are promoted to int
        // byte bad = x + y;            // error: possible lossy conversion from int to byte
        byte ok = (byte) (x + y);
        final byte fx = 10, fy = 20;
        byte c = fx + fy;               // constant expression that fits: compiles
        System.out.println(sum + " " + ok + " " + c);
        char ch = 'a';
        ch++;
        System.out.println(ch + " " + (char) (ch + 1) + " " + (ch - 'a'));
        System.out.println(Character.getNumericValue('7') + " " + ('7' - '0'));
        System.out.println((char) ('a' + 25) + " " + (char) ('A' + 3));
    }
}
```

Run it TERMINAL

```
$ java Casts.java
66 66 66.0
3 -3
4 -3 3
2147483647 9223372036854775807 0
-56 4464 A 122
1.6777216E7 16777216
123456789 -> 1.2345679E8
30 30 30
b c 1
7 7
z D
```

What to notice:

- `(int) 3.99` and `(int) -3.99` truncate toward zero (3 and -3). They do not round. `Math.round` rounds half *up* (toward positive infinity): 3.5 becomes 4, -3.5 becomes -3, and 2.5 becomes 3. This differs from "banker's rounding" used by some languages and by `BigDecimal` with `HALF_EVEN`.
- `(int) 1e20` is `Integer.MAX_VALUE`, not garbage: float-to-int casts saturate. `(int) Double.NaN` is 0. Integer-to-integer casts, in contrast, wrap (`(short) 70000` is 4464).
- A `float` cannot hold 16,777,217 (2^24 + 1). The value is rounded to 16,777,216, which is why using `float` for counters or money is a mistake. `123456789` becomes `1.2345679E8`.
- The line `byte c = fx + fy;` compiles because `fx + fy` is a **constant expression** (both are `final` with constant initialisers) whose value, 30, fits in a `byte`. The compiler narrows it for you. With non-final `x + y` you need the cast.

2.3.2 Binary numeric promotion

When an arithmetic operator has operands of different types, or of types smaller than `int`, Java **promotes** them before computing. The rules for binary operators (`+ - * / % < >` and so on):

1. If either operand is `double`, the other is converted to `double`.
2. Otherwise, if either is `float`, convert to `float`.
3. Otherwise, if either is `long`, convert to `long`.
4. Otherwise, **both are converted to** `int`.

Rule 4 is the important one. `byte`, `short` and `char` operands never stay small: `x + y` for two bytes is an `int`. This is why `byte c = x + y;` fails, why `ch + 1` is an `int`, and why `'a' + 'b'` is 195, not a two-letter string. The result type of the expression is the promoted type, so `int / int` is an `int` division even if you then assign the result to a `double` (the cast must happen *before* the division: `(double) a / b`).

Compound assignment and `++`/`--` are exempt from the compile error, because they include an implicit cast back to the variable's type (see the operators chapter). That is why `ch++` on a `char` compiles and gives `'b'`.

2.3.3 char arithmetic

Because `char` is a number, some very practical idioms work:

- `(char) ('a' + 25)` is `'z'`; `(char) ('A' + 3)` is `'D'`. Cast back to `char` when you want a character.
- `c - '0'` converts a digit character to its numeric value: `'7' - '0'` is 7. `Character.getNumericValue('7')` does the same and also handles non-ASCII digits and letters in other bases.
- `c - 'a'` gives a zero-based letter index. This is the basis of the counting-array trick for anagram problems: `counts[s.charAt(i) - 'a']++`.
- `Character.isDigit`, `isLetter`, `toUpperCase` handle classification and case; prefer them to hand-rolled range checks when the input is not guaranteed to be ASCII.

2.3.4 Why 0.1 + 0.2 is not 0.3

`float` and `double` follow the IEEE 754 standard: a sign bit, an exponent and a binary fraction. A number is stored exactly only if it is a sum of powers of two (0.5, 0.25, 0.375). The decimal 0.1 is not; in binary it repeats forever (0.000110011001100...), so the stored `double` is the *closest* representable number, slightly more than 0.1. Adding two such approximations gives a result that is not the closest double to 0.3. `BigDecimal` can show you the exact stored value:

Special.java JAVA

```java
import java.math.BigDecimal;
import java.math.RoundingMode;

public class Special {
    public static void main(String[] args) {
        System.out.println(1 / 0.0 + " " + -1 / 0.0 + " " + 0.0 / 0.0);
        double nan = Double.NaN;
        System.out.println((nan == nan) + " " + Double.isNaN(nan));
        System.out.println((0.0 == -0.0) + " " + Double.valueOf(0.0).equals(-0.0));
        try {
            System.out.println(1 / 0);
        } catch (ArithmeticException e) {
            System.out.println(e.getMessage());
        }
        try {
            Math.multiplyExact(1 << 20, 1 << 20);
        } catch (ArithmeticException e) {
            System.out.println(e.getMessage());
        }
        try {
            Math.toIntExact(1L << 40);
        } catch (ArithmeticException e) {
            System.out.println(e.getMessage());
        }
        System.out.println(new BigDecimal("0.1").add(new BigDecimal("0.2")));
        System.out.println(new BigDecimal(0.1));
        BigDecimal a = new BigDecimal("1.10"), b = new BigDecimal("1.1");
        System.out.println(a.equals(b) + " " + a.compareTo(b));
        long cents = 0;
        for (int k = 0; k < 10; k++) cents += 10;
        double dsum = 0;
        for (int k = 0; k < 10; k++) dsum += 0.1;
        System.out.println(cents + " " + dsum);
        System.out.println(new BigDecimal("10").divide(new BigDecimal("4")));
        try {
            new BigDecimal("1").divide(new BigDecimal("3"));
        } catch (ArithmeticException e) {
            System.out.println(e.getMessage());
        }
        BigDecimal third = new BigDecimal("1").divide(new BigDecimal("3"), 4, RoundingMode.HALF_UP);
        System.out.println(third);
    }
}
```

Run it TERMINAL

```
$ java Special.java
Infinity -Infinity NaN
false true
true false
/ by zero
integer overflow
integer overflow
0.3
0.1000000000000000055511151231257827021181583404541015625
false 0
100 0.9999999999999999
2.5
Non-terminating decimal expansion; no exact representable decimal result.
0.3333
```

Infinity, NaN and signed zero

Floating-point division by zero does **not** throw. `1 / 0.0` is `Infinity`, `-1 / 0.0` is `-Infinity`, and `0.0 / 0.0` is `NaN` ("not a number"). Integer division by zero, in contrast, throws `ArithmeticException: / by zero`.

`NaN` is unordered: `NaN == NaN` is `false`, and every comparison with `NaN` (except `!=`) is false. Test with `Double.isNaN(x)`. There is also a negative zero: `0.0 == -0.0` is `true` with the primitive operator, but `Double.valueOf(0.0).equals(-0.0)` is `false`, because `equals` compares the bits. This is a reason `Double` objects in collections can behave unexpectedly.

Detecting overflow instead of ignoring it

`Math.addExact`, `subtractExact`, `multiplyExact`, `incrementExact`, `negateExact` and `toIntExact` do the operation and throw `ArithmeticException("integer overflow")` if the result does not fit. They have `int` and `long` overloads. They cost almost nothing and turn silent corruption into a loud failure; use them for anything representing money, sizes or quantities.

Money and BigDecimal

Never use `double` for currency. Ten additions of 0.1 give 0.9999999999999999, not 1.0 (the last line of the output). Two correct approaches:

- Store the smallest unit as an integer: `long cents`. Fast, exact, and you format on output. The loop above ends with exactly 100.
- Use `java.math.BigDecimal`, an arbitrary-precision decimal. Construct it from a **String** (`new BigDecimal("0.1")`), never from a double: `new BigDecimal(0.1)` faithfully captures the double's binary error, as the long digit string shows.

`BigDecimal` is immutable (`add` returns a new object), `equals` compares value *and scale* (`1.10` is not equal to `1.1`) so use `compareTo` to compare numerically, and `divide` throws if the quotient has no finite decimal expansion unless you supply a scale and a `RoundingMode`. `BigDecimal` is slower than `double`; that is the price of exactness. The sibling `BigInteger` provides unbounded integers (factorials, cryptography).

WATCH OUT
`double balance = 0; balance += 0.1;` ten times will not reach 1.0, and a test `balance == 1.0` fails. If you ever see `double` in a class named `Money` or `Invoice`, treat it as a bug.

GOTCHA
`Math.round(float)` returns an `int` and `Math.round(double)` returns a `long`. `int r = Math.round(2.5);` does not compile (the argument is a double, so the result is a long), but `int r = Math.round(2.5f);` does.

REAL WORLD
In Spring Boot backends, monetary fields are almost always `BigDecimal` (mapped to `NUMERIC`/`DECIMAL` in SQL) or a `long` of minor units. JSON libraries serialise `BigDecimal` as a number; some teams transport it as a string to avoid precision loss in JavaScript clients.

INTERVIEW ANGLE
Why does `0.1 + 0.2 == 0.3` return false in Java, and how do you compare floating-point numbers? Because `0.1`, `0.2` and `0.3` are not exactly representable in binary floating point, and the sum of the two approximations differs from the double nearest 0.3 (it is 0.30000000000000004). Compare with a tolerance (`Math.abs(a - b) < epsilon`) chosen for the problem, or avoid floating point by using `BigDecimal` or integer minor units.

Check yourself

1. Does `byte a = 1, b = 2; byte c = a + b;` compile? Why or why not? What changes if `a` and `b` are `final`?
2. What do `(int) -3.99`, `Math.round(-3.5)` and `(byte) 200` evaluate to?
3. What does this print: `System.out.println('a' + 'b' + "" + 'a' + 'b');`
4. `double avg = total / count;` with `int total = 7, count = 2;` gives 3.0. What is the bug and the fix?
5. What is `0.0 / 0.0`? What does `x == x` give if `x` is that value?
6. Why must you write `new BigDecimal("0.1")` and not `new BigDecimal(0.1)`?

Answers

1. No: `a + b` is an `int` after binary numeric promotion, and narrowing to `byte` needs a cast. If both are `final` with constant initialisers, `a + b` is a constant expression (3) that fits, so it compiles.
2. `-3` (truncation), `-3` (round half up), and `-56` (low 8 bits of 200).
3. `195ab`. `'a' + 'b'` is the int 195, then string concatenation appends `""`, `'a'` and `'b'`.
4. The division is done in `int` (7 / 2 = 3) before the widening. Fix: `(double) total / count`, giving 3.5.
5. `NaN`. `x == x` is `false`; use `Double.isNaN(x)`.
6. The double constructor receives the already-rounded binary value of 0.1 and reproduces its exact (inexact-looking) expansion; the String constructor parses the decimal exactly.

KEY POINTS

- Widening is automatic; narrowing needs a cast and can wrap (integers) or saturate (float to int).
- Binary numeric promotion: anything smaller than `int` becomes `int`; mixed operands go to the wider type, so `byte + byte` is `int`.
- A `final` constant expression that fits the target type may be assigned without a cast.
- Floating point is binary: 0.1 is approximate, `NaN != NaN`, division by 0.0 gives Infinity or NaN, but integer `/ 0` throws.
- Use `Math.*Exact` to catch overflow, `long` minor units or `BigDecimal` (built from Strings) for money.

2.4 Control Flow

Control flow decides which statements run and how many times. Java offers the usual `if`, `switch`, `while`, `do-while` and `for`, plus three modern additions that every current codebase uses: the **switch expression** (Java 14), the **enhanced for** loop over arrays and `Iterable`s (Java 5), and **pattern matching for switch** (Java 21). The old forms still matter because you will read plenty of code written with them, and because their traps are classic interview material.

2.4.1 if, else and the dangling else

`if (cond) stmt; else stmt;` takes any `boolean` condition. Unlike C, an `int` is not a condition: `if (x = 5)` is a compile error (you cannot assign an int where a boolean is required), which removes a whole category of C bugs. The only remaining assignment trap is with booleans: `if (done = true)` compiles and always passes.

Always use braces, even for one-line bodies. Without braces an `else` binds to the **nearest** `if`, and indentation does not matter to the compiler (see the last lines of the pitfalls demo below). A missing brace is also the cause of the famous "goto fail" security bug in C code.

2.4.2 The classic switch statement

The old `switch` statement jumps to the matching `case` label and then keeps running every statement below it until it meets a `break` or the end. This **fall-through** is the main hazard:

Flow.java JAVA

```
public class Flow {
    enum Day { MON, TUE, SAT, SUN }

    static int oldSwitch(int n) {
        int r = 0;
        switch (n) {
            case 1:
                r += 1;
            case 2:
                r += 2;
                break;
            case 3:
                r += 3;
                break;
            default:
                r = -1;
        }
        return r;
    }

    static String kind(Day d) {
```

Flow.java (continued) JAVA

```
        return switch (d) {
            case SAT, SUN -> "weekend";
            case MON, TUE -> {
                String s = "week";
                yield s + "day";
            }
        };
    }

    static int oldYield(String s) {
        return switch (s) {
            case "a":
            case "b":
                yield 1;
            default:
                yield 0;
        };
    }

    static String describe(Object o) {
        return switch (o) {
            case null -> "null!";
            case Integer i when i > 100 -> "big int " + i;
            case Integer i -> "int " + i;
            case String s -> "string of length " + s.length();
            default -> "something else: " + o;
        };
    }

    public static void main(String[] args) {
        System.out.println(oldSwitch(1) + " " + oldSwitch(2) + " " + oldSwitch(3)
            + " " + oldSwitch(9));
        System.out.println(kind(Day.SUN) + " " + kind(Day.MON) + " " + oldYield("b"));
        System.out.println(describe(5) + "; " + describe(500) + "; " + describe("hey"));
        System.out.println(describe(2.5) + "; " + describe(null));
        int total = 0;
        for (int i = 1; i <= 10; i++) {
            if (i % 2 == 0) continue;
            if (i > 7) break;
            total += i;
        }
        System.out.println(total);
        outer:
        for (int r = 1; r <= 3; r++) {
            for (int c = 1; c <= 3; c++) {
                if (c == 2) continue outer;
                if (r == 3) break outer;
                System.out.print(r + "" + c + " ");
            }
        }
        System.out.println();
        int n = 0;
        do { n++; } while (n < 0);
        System.out.println(n);
        String[] names = {"Asha", "Ravi", "Meera"};
        for (String nm : names) System.out.print(nm.charAt(0));
        System.out.println();
        for (int i = 0, j = 10; i < j; i += 3, j -= 3) System.out.print(i + ":" + j + " ");
        System.out.println();

    }
}
```

Run it TERMINAL

```
$ java Flow.java
3 2 3 -1
weekend weekday 1
int 5; big int 500; string of length 3
something else: 2.5; null!
16
11 21
1
ARM
0:10 3:7
```

In `oldSwitch(1)` the label `case 1` adds 1 and, with no `break`, falls into `case 2`, which adds 2: the result is 3. Fall-through is occasionally useful (several labels sharing code) but is mostly accidental. `switch` accepts `byte`, `short`, `char`, `int` (and their wrappers), `String`, enums and, since Java 21, any reference type with patterns. It does not accept `long`, `float`, `double` or `boolean`. Case labels must be compile-time constants. Matching on a `String` uses `equals`, so it is case sensitive, and a `null` selector throws `NullPointerException` unless you add `case null` (Java 21).

2.4.3 Switch expressions (arrow form and yield)

A **switch expression** produces a value, so you can return it or assign it. It comes in the arrow form, which never falls through, and the colon form, which uses `yield`. In `kind` above:

- `case SAT, SUN -> "weekend"` lists several constants on one arm.
- A block arm computes something and delivers the result with `yield`.
- The expression must be **exhaustive**: every possible value is covered. For an enum with all constants listed, no `default` is needed, and adding a new enum constant later makes the compiler flag every switch that is missing it. That is a safety benefit you lose if you add a catch-all `default`.
- `oldYield` shows that the colon form also works inside a switch expression, with `yield` instead of `return`. Stick to the arrow form in new code.

An arrow arm can be an expression, a block or a `throw`. There is no fall-through, so no `break` is needed. When the switch is used as a *statement* with arrows, it is also fall-through free but need not be exhaustive (unless it uses patterns).

2.4.4 Pattern matching for switch (a first look)

Since Java 21 a `case` label can be a **type pattern** with an optional guard (`when`), and `case null` is allowed. `describe` above dispatches on the runtime class of its argument: `Integer i when i > 100` matches big integers, the next `Integer i` catches the rest, `String s` binds a string, and `default` handles anything else. Order matters: a more general pattern before a specific one is a compile error (dominance). Sealed hierarchies, record patterns and exhaustiveness are developed in the object-oriented and modern Java parts; for now notice that this replaces long `instanceof` chains.

2.4.5 Loops

Loop	Use when	Notes
`while`	repeat while a condition holds; count unknown	condition tested before each pass
`do-while`	body must run at least once	condition tested after; ends with a semicolon
`for`	counted loops with an index	init; condition; update, all optional
`enhanced for`	visit every element of an array or Iterable	no index; cannot replace the element

The four loops.

The `for` header can declare several variables and updates (`for (int i = 0, j = 10; i < j; i += 3, j -= 3)`, which printed `0:10 3:7`). A variable declared in the header lives only inside the loop. `for (;;)` is an infinite loop, as is `while (true)`; leave via `break` or `return`.

The **enhanced for** (`for (String nm : names)`) works on arrays and on anything implementing `Iterable`. It is the best default when you do not need the index. Two limits: assigning to the loop variable changes only your local copy, and you must not structurally modify a collection while iterating it (see the pitfalls below).

break, continue and labels

`break` leaves the nearest enclosing loop or switch; `continue` skips to the next iteration. To affect an *outer* loop, put a **label** before it and name it: `continue outer;` or `break outer;`. In the demo, the inner loop skips to the next row when `c == 2`, and `break outer` ends everything at row 3, so only `11 21` is printed. Labels are legitimate for nested searches, but if you feel you need one often, extract the loops into a method and `return` instead.

2.4.6 Loop pitfalls

Pitfalls.java JAVA

```java
import java.util.ArrayList;
import java.util.List;

public class Pitfalls {
    public static void main(String[] args) {
        int count = 0;
        for (double x = 0.0; x != 1.0; x += 0.1) {
            if (++count > 15) {
                System.out.println("0.1 steps never hit 1.0 exactly; stopped after " + count);
                break;
            }
        }
        int[] a = {1, 2, 3};
        try {
            for (int i = 0; i <= a.length; i++) System.out.print(a[i] + " ");
        } catch (ArrayIndexOutOfBoundsException e) {
            System.out.println("-> " + e.getMessage());
        }
        List<String> names = new ArrayList<>(List.of("a", "b", "c"));
        try {
            for (String s : names) if (s.equals("a")) names.remove(s);
        } catch (java.util.ConcurrentModificationException e) {
            System.out.println("CME");
        }
        names.removeIf(s -> s.equals("b"));
        System.out.println(names);
        int x = 5;
        if (x > 3)
            if (x > 10)
                System.out.println("big");
            else
                System.out.println("dangling else belongs to the inner if");
    }
}
```

Run it TERMINAL

```
$ java Pitfalls.java
0.1 steps never hit 1.0 exactly; stopped after 16
1 2 3 -> Index 3 out of bounds for length 3
CME
[c]
dangling else belongs to the inner if
```

- **Floating-point loop counters.** Because 0.1 is inexact, `x != 1.0` is never false and the loop does not terminate by itself (the guard stopped it). Loop on an integer and compute the double inside the body.
- **Off-by-one.** `i <= a.length` reads one element past the end and throws `ArrayIndexOutOfBoundsException`. The idiom is `i < a.length`. Before writing a loop, say out loud the first and last index you want.
- **Modifying a collection during enhanced for.** Removing from an `ArrayList` inside `for (String s : names)` makes the iterator throw `ConcurrentModificationException` on its next step. (Not always: if you remove the second-to-last element the loop ends quietly without an exception, which is an even nastier bug. Do not rely on either behaviour.) Use `Iterator.remove()` or `removeIf`, as the demo does for `"b"`.
- **Stray semicolon.** `while (cond);` or `for (...);` has an empty body; the following block then runs once, independent of the loop. Editors and linters flag it.
- **Dangling else.** Shown at the end: the `else` pairs with `if (x > 10)`, not `if (x > 3)`, despite the misleading indentation.
- **Infinite loops with narrow types.** `for (byte b = 0; b < 200; b++)` never ends, because a byte wraps at 127 and is always less than 200.

TIP
Prefer the enhanced `for` or a stream (`IntStream.range`) when no index is needed, and keep loop bodies small. If a loop needs a flag variable and three levels of nesting, extract a method.

INTERVIEW ANGLE
What is the difference between a switch statement and a switch expression? A switch statement executes code, falls through from one `case` to the next unless you `break`, and need not cover all cases. A switch expression (Java 14) yields a value, uses arrow arms without fall-through (or `yield` in colon form), must be exhaustive, and can be assigned or returned. For enums and sealed types, exhaustiveness lets the compiler tell you when a new case was forgotten.

Check yourself

1. What does `oldSwitch(1)` return in the listing, and which single keyword would change it to 1?
2. Which of these types can NOT be used as a switch selector: `String`, `long`, `char`, an enum?
3. What is wrong with `for (int i = 0; i <= arr.length; i++) sum += arr[i];`?
4. What does this print? `int n = 0; do { n++; } while (n < 0); System.out.println(n);`
5. You write `if (a > 0) if (b > 0) print("both"); else print("x");` with `a = -1`. What is printed?
6. Why can a switch expression over all constants of an enum omit `default`, and why might you not want to add one?

Answers

1. 3 (it falls through into `case 2`). Adding `break;` after `r += 1;` makes it return 1.
2. `long` (also `float`, `double`, `boolean`). `String`, `char` and enums are allowed.
3. The condition should be `<`; with `<=` the last pass reads `arr[arr.length]` and throws `ArrayIndexOutOfBoundsException`.
4. `1`. A do-while runs its body once before testing the condition.
5. Nothing. The `else` belongs to the inner `if (b > 0)`, which is never reached because `a > 0` is false. Use braces.
6. The compiler can prove every constant is handled, so a `default` is unnecessary. A `default` would hide the compile error you get when a new constant is added later.

KEY POINTS
- Always use braces; an `else` binds to the nearest `if`.
- The old `switch` statement falls through without `break`; the arrow form and switch expressions do not.
- A switch expression must be exhaustive, so enums and sealed types give compile-time safety; patterns and `case null` arrive in Java 21.
- Use the enhanced for when you do not need the index; never modify the collection you are iterating (use `removeIf` or `Iterator.remove`).
- Labelled `break`/`continue` exit outer loops; extracting a method with `return` is usually cleaner.
- Do not drive loops with floating-point counters; use integers and compute the doubles.

2.5 Arrays

An **array** is a fixed-size, ordered block of elements of one type, accessed by a zero-based index. It is the lowest-level collection in Java, the foundation under `ArrayList`, `String` (a `byte[]` inside) and `HashMap` (an array of buckets), and the subject of a large share of interview problems. In Java an array is an **object**: it lives on the heap, it knows its own `length`, every access is bounds-checked, and the variable you hold is a reference to it.

2.5.1 Creating arrays and default values

Three equivalent ways to create an array: allocate with `new` and a size, use an initialiser list, or build a multi-dimensional one. `new` fills the elements with the default value for the element type, just like fields: 0, `false`, `'\u0000'`, or `null` for references. The size is fixed for the life of the array. `length` (a field, not a method, unlike `String.length()` and `List.size()`) never changes.

Arrays1.java JAVA

```java
import java.util.Arrays;

public class Arrays1 {
    public static void main(String[] args) {
        int[] a = new int[3];
        String[] s = new String[2];
        boolean[] f = new boolean[2];
        int[] lit = {5, 3, 9, 1};
        System.out.println(Arrays.toString(a) + " " + Arrays.toString(s) + " "
            + Arrays.toString(f));
        System.out.println(lit.length + " " + lit.getClass().getSimpleName());
        int[][] grid = new int[3][4];
        grid[1][2] = 7;
        System.out.println(Arrays.deepToString(grid));
        int[][] jag = new int[3][];
        for (int i = 0; i < jag.length; i++) {
            jag[i] = new int[i + 1];
            Arrays.fill(jag[i], i);
        }
        System.out.println(Arrays.deepToString(jag));
        int[] sorted = lit.clone();
        Arrays.sort(sorted);
        System.out.println(Arrays.toString(lit) + " " + Arrays.toString(sorted));
        System.out.println("idx of 9: " + Arrays.binarySearch(sorted, 9)
            + " missing 4: " + Arrays.binarySearch(sorted, 4));
        int[] bigger = Arrays.copyOf(lit, 6);
        int[] part = Arrays.copyOfRange(lit, 1, 3);
        System.out.println(Arrays.toString(bigger) + " " + Arrays.toString(part));
        int[] alias = lit;
        alias[0] = 99;
        System.out.println(lit[0] + " " + (alias == lit) + " " + lit.equals(lit.clone())
            + " " + Arrays.equals(lit, lit.clone()));
        int[][] shallow = jag.clone();
        shallow[2][0] = -1;
        System.out.println(jag[2][0]);
        int[] src = {1, 2, 3, 4, 5};
        System.arraycopy(src, 0, src, 1, 4);
        System.out.println(Arrays.toString(src));
        Object[] objs = new String[2];
        try {
            objs[0] = Integer.valueOf(1);
        } catch (ArrayStoreException e) {
            System.out.println("ArrayStoreException: " + e.getMessage());
        }
        System.out.println(Arrays.stream(lit).sum() + " " + Arrays.asList(1, 2, 3));
        String[] words = {"pear", "fig", "apple"};
        Arrays.sort(words, (x, y) -> x.length() - y.length());
        System.out.println(Arrays.toString(words) + " " + Arrays.hashCode(new int[]{1, 2}));
        try {
            int[] neg = new int[-1];
        } catch (NegativeArraySizeException e) {
            System.out.println("NegativeArraySizeException: " + e.getMessage());
        }
    }
}
```

Run it TERMINAL

```
$ java Arrays1.java
[0, 0, 0] [null, null] [false, false]
4 int[]
[[0, 0, 0, 0], [0, 0, 7, 0], [0, 0, 0, 0]]
[[0], [1, 1], [2, 2, 2]]
[5, 3, 9, 1] [1, 3, 5, 9]
idx of 9: 3 missing 4: -3
[5, 3, 9, 1, 0, 0] [3, 9]
99 true false true
-1
[1, 1, 2, 3, 4]
ArrayStoreException: java.lang.Integer
112 [1, 2, 3]
[fig, pear, apple] 994
NegativeArraySizeException: -1
```

Printing an array directly gives something like `[I@5656be13` (type code and identity hash, different on each run), which is why `Arrays.toString` and `Arrays.deepToString` exist. Declare with the type before the name (`int[] a`), not the C style `int a[]`, which Java allows but readers dislike.

2.5.2 The java.util.Arrays toolbox

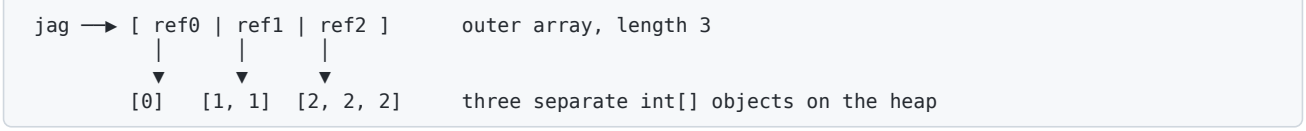
Never write these by hand; `java.util.Arrays` provides them:

- `toString`, `deepToString` for printing; `equals`, `deepEquals`, `hashCode` for content comparison. The array's own `equals` is identity: `lit.equals(lit.clone())` is `false`, `Arrays.equals(...)` is `true`.
- `sort` (a dual-pivot quicksort for primitives; a stable merge-sort variant, TimSort, for objects and comparators), `binarySearch` (requires a sorted array; a missing key returns `-(insertionPoint) - 1`, so `-3` for 4 in `[1, 3, 5, 9]`), `fill`.
- `copyOf(a, newLength)` (pads with defaults or truncates) and `copyOfRange(a, from, to)` with an exclusive upper bound. `System.arraycopy(src, i, dst, j, n)` is the low-level primitive and handles overlapping ranges correctly, as the shift in the demo shows.
- `Arrays.asList(...)` wraps an array as a fixed-size `List` view, and `Arrays.stream(a)` makes a stream.

2.5.3 Multi-dimensional and jagged arrays

Java has no true 2D array; `int[][]` is an **array of arrays**. `new int[3][4]` creates 3 row objects of length 4. Because each row is its own object, rows can differ in length (a *jagged* array): `new int[3][]` followed by assigning each row. The jagged example above produced a triangle. This also means a "2D" array is not contiguous in memory, so row-major traversal (`for row, for col`) is faster than column-major because it follows the cache.

Memory picture of int[][] jag



> **NOTE**
> The sketch is schematic: the outer array holds three references to three separate row arrays on the heap. The real object also has a header holding the class pointer and the length, but its size is JVM-specific (see the object layout chapter in the JVM part).

2.5.4 Reference semantics: aliasing and shallow copies

Assigning an array to another variable copies the **reference**, not the elements: `alias = lit` makes both names point to one array, so `alias[0] = 99` changes `lit[0]` too. `clone()` makes a **shallow** copy: a new array whose elements are copied one by one. For a primitive array that is a full copy; for an array of arrays or objects the new array shares the same inner objects. The demo shows it: after `shallow = jag.clone()`, writing `shallow[2][0] = -1` changed `jag[2][0]`. For a deep copy of a 2D array, copy each row (`Arrays.copyOf` per row) or use streams.

2.5.5 Covariance and ArrayStoreException

Arrays are **covariant**: `String[]` is a subtype of `Object[]`. That lets you write `Object[] objs = new String[2];`, but the real array still only accepts strings, and the JVM checks every store at runtime, throwing `ArrayStoreException` for the integer in the demo. Generics took the opposite approach (invariance: `List<String>` is not a `List<Object>`) so such errors are caught at compile time. Part 6 explains why. Covariance is a legacy design decision you should know about and not exploit.

2.5.6 Varargs

A method parameter declared `int... nums` is **variable arity**: callers may pass zero or more `int`s, or an `int[]`, and inside the method `nums` is an ordinary array. It must be the last parameter, and a method has at most one.

Varargs.java JAVA

```java
import java.util.Arrays;

public class Varargs {
    static int sum(int... nums) {
        int t = 0;
        for (int n : nums) t += n;
        return t;
    }
    static String show(String label, Object... items) {
        return label + ":" + (items == null ? "null array" : items.length);
    }
    public static void main(String[] args) {
        System.out.println(sum() + " " + sum(1) + " " + sum(1, 2, 3) + " " + sum(new int[]{4, 5}));
        System.out.println(show("a") + " " + show("b", 1, 2) + " " + show("c", (Object[]) null));
        String[] xs = {"x", "y", "z"};
        System.out.println(show("d", (Object[]) xs) + " " + show("e", (Object) xs));
        int[][] m = {{1, 2}, {3, 4}, {5, 6}};
        long total = 0;
        for (int[] row : m) for (int v : row) total += v;
        System.out.println(total + " " + m.length + " " + m[0].length + " "
            + Arrays.toString(m[2]));
    }
}
```

Run it TERMINAL

```
$ java Varargs.java
0 1 6 9
a:0 b:2 c:null array
d:3 e:1
21 3 2 [5, 6]
```

`show("c", (Object[]) null)` passes a `null` array, not an array containing `null`, so the method must guard against it. `show("e", (Object) xs)` forces the whole `String[]` to be a single element. Without the casts, passing a `String[]` where `Object...` is expected spreads its elements, and the compiler warns about the ambiguity. Varargs create a new array on each call, so avoid them in tight loops, and avoid overloading varargs methods (see the methods chapter).

2.5.7 Bounds checks and cost

Every index is checked against `length`; the JIT removes checks it can prove redundant, so you pay little in tight loops. A bad index throws `ArrayIndexOutOfBoundsException` (message `Index 3 out of bounds for length 3`), a negative size throws `NegativeArraySizeException`. Array size is limited by `int`, so about 2.1 billion elements at most, in practice slightly fewer, and by the heap size (`OutOfMemoryError: Java heap space` or `Requested array size exceeds VM limit`).

WATCH OUT

Do not write `if (arr == null || arr.length == 0)` as `arr.length == 0 || arr == null`; the null check must come first or the code throws `NullPointerException`. Also remember `arr.length` (field), `str.length()` (method) and `list.size()` (method).

INTERVIEW ANGLE

Array versus ArrayList: when would you use each? An array has a fixed size, can hold primitives without boxing, and has minimal overhead, so it suits fixed-size data, performance-critical numeric code and low-level APIs. `ArrayList` is backed by an array but grows automatically, offers the whole `List` API and works with generics and streams, at the cost of boxing for primitives. Use `ArrayList` by default and arrays when size is fixed or primitive performance matters.

Check yourself

1. What are the contents of `new boolean[2]`, `new String[2]` and `new char[1]` (as `(int) c[0]`)?
2. What does `Arrays.binarySearch(new int[]{1, 3, 5, 9}, 4)` return, and what does the result mean?
3. After `int[] a = {1, 2}; int[] b = a; b[0] = 9;` what is `a[0]`? What would it be if `b = a.clone()`?
4. What is wrong here, and when is it caught? `Object[] o = new Integer[1]; o[0] = "text";`
5. What does `int[][] g = new int[2][]; System.out.println(g[0]);` print?
6. Is `a.equals(b)` for two arrays with the same contents true or false? What should you call?

Answers

1. `[false, false]`, `[null, null]`, and `0`.
2. `-3`: the key is absent; the value is `-(insertionPoint) - 1` with insertion point 2.
3. 9, because both names refer to one array. With `clone()` it stays 1.
4. It compiles but throws `ArrayStoreException` at runtime, because the actual array is `Integer[]`.
5. `null`. The outer array exists but its two row references are not yet allocated.
6. `false`, because `equals` on arrays is identity. Call `Arrays.equals` (or `deepEquals` for nested arrays).

KEY POINTS

- Arrays are fixed-size objects with default-initialised elements and a `length` field; indexing is bounds-checked.
- `int[][]` is an array of arrays, so rows can be jagged; `clone()` and assignment are shallow or aliasing.
- Use `Arrays.toString`, `sort`, `copyOf`, `equals`, `fill`, `binarySearch`; `System.arraycopy` is the fast primitive.
- Arrays are covariant, so a bad store fails at runtime with `ArrayStoreException`.
- Varargs are sugar for an array parameter; the call allocates an array and `null`/array arguments need care.

2.6 Methods

A method is a named block of code that takes inputs, does work and optionally returns a result. Methods are how you split a program into pieces small enough to understand, test and reuse. Most bugs that beginners hit around methods come from three misunderstandings: which overload the compiler picks, what "pass by value" really means for objects, and how recursion uses the stack. This chapter fixes all three.

2.6.1 Anatomy and signature

Anatomy.java JAVA

```java
public class Anatomy {
    // modifiers  return type  name   parameters          throws
    public static int clamp(int value, int min, int max) {
        if (min > max) {
            throw new IllegalArgumentException("min > max");
        }
        return Math.max(min, Math.min(max, value));
    }

    public static void main(String[] args) {
        System.out.println(clamp(15, 0, 10));
    }
}
```

The **method signature** is the name plus the parameter types in order: `clamp(int, int, int)`. The return type, parameter names, modifiers and `throws` clause are not part of the signature. That matters because two methods in one class cannot share a signature, and it is the signature the compiler uses to decide which method you meant.

A method with return type `void` returns nothing; any other method must return a value of a compatible type on every path (the compiler checks this: "missing return statement"). Parameters are local variables initialised from the arguments, and they can be marked `final` to stop reassignment inside the body.

2.6.2 Overloading and how the compiler chooses

Overloading means several methods with the same name and different parameter lists. The compiler picks one at compile time, in three phases, stopping at the first phase that finds a match:

1. Exact match or widening primitive conversion / subtype widening, with no boxing and no varargs.
2. Boxing or unboxing (plus widening of references).
3. Varargs (`int...`).

Inside a phase, if several methods apply, the **most specific** one wins. If none is clearly more specific, the call is ambiguous and fails to compile.

Overload.java JAVA

```java
public class Overload {
    static void f(int x)       { System.out.println("f(int)"); }
    static void f(long x)      { System.out.println("f(long)"); }
    static void f(Integer x)   { System.out.println("f(Integer)"); }
    static void f(Object x)    { System.out.println("f(Object)"); }
    static void f(int... xs)   { System.out.println("f(int...)"); }

    static void g(long x)      { System.out.println("g(long)"); }
    static void g(Integer x)   { System.out.println("g(Integer)"); }

    static void h(Object o)    { System.out.println("h(Object)"); }
    static void h(String s)    { System.out.println("h(String)"); }

    public static void main(String[] args) {
        byte b = 1;
        f(b);                  // widening byte -> int
        f(5);
        f(5L);
        f(Integer.valueOf(5));
        f("text");
        f();
        g(5);                  // widening beats boxing
        h(null);               // most specific: String
        h((Object) "x");
    }
}
```

Output TERMINAL

```
$ java Overload.java
f(int)
f(int)
f(long)
f(Integer)
f(Object)
f(int...)
g(long)
h(String)
h(Object)
```

What to notice. `f(b)` with a `byte` picks `f(int)`, the closest widening, not `f(long)`. `g(5)` picks `g(long)` because widening (phase 1) is tried before boxing (phase 2). `h(null)` picks `h(String)` because `String` is more specific than `Object`. Overloads are resolved on the **static** (declared) type, so `h((Object) "x")` calls `h(Object)` even though the object is a string. Overriding, by contrast, is resolved at run time; that distinction is covered in the polymorphism chapter in Part 3.

GOTCHA

Overloading `f(int...)` next to `f(int, int)` or `f(Integer...)` next to `f(int...)` is a recipe for ambiguity errors and surprises. Prefer distinct names when the meanings differ (`removeAt(int)` versus `remove(Object)` in `List` is a famous trap: `list.remove(1)` removes the element at index 1, not the value 1).

2.6.3 Pass by value, always

Java passes **everything by value**. For a primitive the value is the number itself. For an object, the value is the **reference** (the pointer-like handle), so the method gets its own copy of the reference, pointing at the same object. Two consequences follow: the method can change the object's contents through its copy, but it can never make the caller's variable point somewhere else.

PassByValue.java JAVA

```java
import java.util.ArrayList;
import java.util.List;

public class PassByValue {
    static void reassign(StringBuilder sb) { sb = new StringBuilder("new"); }
    static void mutate(StringBuilder sb)   { sb.append("!"); }
    static void bump(int n)                { n++; }
    static void swap(int[] a)              { int t = a[0]; a[0] = a[1]; a[1] = t; }

    public static void main(String[] args) {
```

PassByValue.java (continued) JAVA

```
        int n = 1;
        bump(n);
        System.out.println("n = " + n);

        StringBuilder sb = new StringBuilder("old");
        reassign(sb);
        System.out.println("after reassign: " + sb);
        mutate(sb);
        System.out.println("after mutate:   " + sb);

        int[] pair = {1, 2};
        swap(pair);
        System.out.println(java.util.Arrays.toString(pair));
    }
}
```

Output TERMINAL

```
$ java PassByValue.java
n = 1
after reassign: old
after mutate:   old!
[2, 1]
```

`bump` changes only its own copy. `reassign` overwrites its copy of the reference, so the caller still sees `old`. `mutate` and `swap` follow the copied reference to the shared object and change it. This is why you cannot write a general `swap(a, b)` for two `int` or `String` variables in Java: the method never sees the caller's variables, only values.

```
caller: sb ──┐
             ├──►  [ StringBuilder "old" ]   (one object on the heap)
callee: sb ──┘     (two separate reference variables, same target)

After "sb = new StringBuilder(...)" inside the callee:
caller: sb ─────►  [ "old" ]
callee: sb ─────►  [ "new" ]
```

INTERVIEW ANGLE

Is Java pass-by-value or pass-by-reference? Always pass-by-value. For objects, the value that is copied is the reference. The callee can mutate the shared object, but reassigning the parameter does not affect the caller's variable. Java has no pass-by-reference (no `&` parameters like C++).

2.6.4 Static versus instance methods

An **instance method** belongs to an object and can use that object's fields through the implicit `this`. A **static method** belongs to the class, has no `this`, and is called as `ClassName.method()`. A static method cannot touch instance fields or call instance methods directly, because there is no object to act on. Make a method static when it depends only on its parameters (`Math.max`, `Integer.parseInt`, `clamp` above); make it an instance method when it reads or changes object state. A static method also cannot be overridden (only hidden); the details are in the OOP part.

2.6.5 Recursion and the stack

A **recursive** method calls itself. It needs a **base case** that stops the recursion and a step that moves toward it. Every call pushes a stack frame (parameters, locals, return address) on the thread's stack, which is why unbounded recursion ends in `StackOverflowError`.

Recur.java JAVA

```
public class Recur {
    static int depth = 0;
    static void down() { depth++; down(); }
    static long fact(int n) { return n <= 1 ? 1 : n * fact(n - 1); }
    static long factLoop(int n) { long r = 1; for (int i = 2; i <= n; i++) r *= i; return r; }
    static long fib(int n) { return n < 2 ? n : fib(n - 1) + fib(n - 2); }

    public static void main(String[] args) {
        System.out.println(fact(5) + " " + factLoop(20) + " " + fib(20));
        try { down(); }
        catch (StackOverflowError e) {
```

Recur.java (continued) JAVA

```
            System.out.println("overflow after " + (depth > 1000 ? "thousands of" : "few")
                + " calls: " + e.getClass().getName());
        }
    }
}
```

Output TERMINAL

```
$ java Recur.java
120 2432902008176640000 6765
overflow after thousands of calls: java.lang.StackOverflowError
```

How deep can you go? With the default 1 MB thread stack on 64-bit Linux (`-XX:+PrintFlagsFinal` shows `ThreadStackSize = 1024` KB), a trivial method like `down()` overflowed after about 10,700 calls on the build machine (typical; it depends on frame size and JIT state). Running with `-Xss256k` cut that to about 1,800. Bigger frames overflow sooner.

Two warnings about recursion. First, `fib` above recomputes the same sub-problems exponentially many times; `fib(50)` would take minutes. Cache results (memoisation) or loop. Second, **Java does not do tail-call optimisation**, so a tail-recursive method still grows the stack. For deep structures, such as a linked list of 100,000 nodes, use a loop or an explicit stack (`ArrayDeque`).

2.6.6 Method design

- **One job, one name.** If you need "and" to describe a method, split it. A method that fits on a screen is easier to test.
- **Few parameters.** More than three or four is a smell; group related values in a `record` (see the records chapter in Part 3).
- **Avoid output parameters and flag arguments.** `render(true)` is unreadable; two methods `renderCompact()` and `renderFull()` are not.
- **Validate at the boundary.** Throw `IllegalArgumentException` for bad arguments, as `clamp` does, or use `Objects.requireNonNull(x, "x")`.
- **Return values, not mutation, when you can.** Methods without side effects are easy to reason about.
- **Name verbs for actions, nouns or `is/has` for queries**: `withdraw`, `balance`, `isEmpty`.

> **REAL WORLD**
> In Spring applications, controller and service methods are mostly short instance methods whose parameters are validated at the boundary (for example with `@Valid`) and whose results are returned rather than written to shared state. The same design rules apply, just with annotations on top.

Check yourself

1. Which is chosen for `f(5)` when the class has `f(long)` and `f(Integer)`, and why?
2. What does this print? `static void m(StringBuilder s){ s = new StringBuilder("x"); s.append("y"); }` called as `StringBuilder a = new StringBuilder("a"); m(a); System.out.println(a);`
3. What is the signature of `public static int clamp(int v, int lo, int hi)`, and does the return type belong to it?
4. Why can a static method not read an instance field directly?
5. Name two ways to rewrite a recursive method that overflows the stack for large input.
6. With `h(Object)` and `h(String)`, which is called by `h(null)`, and what happens if the second overload is `h(Integer)` instead?

Answers

1. `f(long)`: widening (phase 1) beats boxing (phase 2).
2. `a`. The parameter is a copy of the reference; reassigning it does not change `a`, and the append hits the new object.
3. `clamp(int, int, int)`. The return type is not part of the signature.
4. A static method has no `this`; there is no object whose field it could mean.
5. Convert to a loop, use an explicit `Deque` as stack, or (if sub-problems repeat) memoise; raising `-Xss` only postpones the problem.

6. `h(String)`, the most specific. With `h(String)` and `h(Integer)` both present, `h(null)` is ambiguous and does not compile.

KEY POINTS

- The signature is name plus parameter types; overload resolution is done at compile time on static types: widening, then boxing, then varargs, then most specific.
- Java is always pass-by-value; for objects the copied value is the reference, so mutation is visible but reassignment is not.
- Static methods have no `this`; use them for behaviour that depends only on arguments.
- Recursion needs a base case, uses one stack frame per call and ends in `StackOverflowError` when too deep; Java has no tail-call optimisation.
- Keep methods short, with few parameters, validated inputs and no hidden side effects.

2.7 Text Blocks and Basic I/O

Almost every beginner program talks to a console: it prints results and reads input. This chapter covers the everyday tools (`System.out`, `printf`, `Scanner`, `BufferedReader`) and text blocks for multi-line strings. It is a quick tour; the full story of `String`, formatting and the `java.io`/`java.nio` file APIs is in Parts 4 and 5.

2.7.1 Printing: System.out and System.err

`System.out` is a `PrintStream` connected to standard output. `println` prints and adds a line break, `print` does not, and `printf` formats. `System.err` is the same kind of stream for standard error, which the shell can redirect separately (`java App 2> errors.txt`). Use `err` for diagnostics so they do not pollute data that another program reads from `out`. `String.valueOf(x)` is applied to anything you print, so objects print via `toString()`.

2.7.2 printf and String.format

`printf` and `String.format` take a format string with **specifiers** of the form `%[flags][width][.precision]conversion`. The most useful conversions are `%d` (integer), `%f` (floating point), `%s` (any object as string), `%c` (char), `%b` (boolean), `%x` (hex), `%n` (platform line separator) and `%%` (a literal percent sign).

Specifier	Meaning	Example result
`%5d`	integer, width 5, right-aligned	" 42"
`%-5d`	left-aligned	"42 "
`%05d`	zero-padded	"00042"
`%,d`	grouping separators	"1,234,567"
`%.2f`	two decimals (rounds half up)	"3.14"
`%8.3f`	width 8, three decimals	" 2.500"
`%10s / %-10s`	string padded right / left	" right"
`%x`	hexadecimal	"ff" for 255

Common format specifiers.

TextOut.java JAVA

```java
import java.util.Scanner;

public class TextOut {
    public static void main(String[] args) {
        String json = """
            {
              "title": "Dune",
              "year": 1965
            }
            """;
        System.out.print(json);
        String oneLine = """
            Hello \
            World""";
```

TextOut.java (continued) JAVA

```
        System.out.println(oneLine);
        String trailing = """
            keep   \s
            end""";
        System.out.println("[" + trailing.lines().findFirst().get() + "]");
        String indented = """
                deeper
              base
            """;
        System.out.print(indented);

        System.out.printf("%d|%5d|%-5d|%05d%n", 42, 42, 42, 42);
        System.out.printf("%.2f|%8.3f|%,d%n", 3.14159, 2.5, 1234567);
        System.out.printf("%s|%10s|%-10s|%c|%b|%x%n", "a", "right", "left", 'z', true, 255);
        System.out.println(String.format("%08.3f", -3.5));
        System.err.println("this goes to stderr");

        Scanner sc = new Scanner("12 3.5 hello\nsecond line here\n");
        int i = sc.nextInt();
        double d = sc.nextDouble();
        String w = sc.next();
        sc.nextLine();
        String line = sc.nextLine();
        System.out.println(i + " " + d + " " + w + " | " + line);
    }
}
```

Output TERMINAL

```
$ java TextOut.java
{
  "title": "Dune",
  "year": 1965
}
Hello World
[keep    ]
    deeper
  base
42|   42|42   |00042
3.14|   2.500|1,234,567
a|     right|left      |z|true|ff
-003.500
this goes to stderr
12 3.5 hello | second line here
```

2.7.3 Text blocks

A **text block** (final in Java 15) is a string literal delimited by `"""`, written over several lines. It exists so that JSON, SQL, HTML and similar snippets do not need `\n`, `\"` and `+` everywhere. The rules are short:

- The opening `"""` must be followed by a line break; the content starts on the next line.
- **Incidental indentation** is removed: the compiler finds the smallest indentation among all content lines *and the closing delimiter line* and strips that much from each line. In `indented` above, `base` is the least-indented line, so `deeper` keeps 2 extra spaces. Moving the closing `"""` further left would keep more indentation.
- Trailing spaces on each line are stripped. Use `\s` to keep a space, as in `keep \s`.
- A `\` at the end of a line joins it with the next (no newline), which gives you long single-line strings.
- If the closing `"""` is on its own line, the string ends with a newline; if it follows the last text directly (as in `World"""`), it does not.
- Quotes inside need no escaping (except a run of three), and the usual escapes (`\t`, `\n`, `\\`) still work.
- The type is an ordinary `String`; there is nothing special at run time. You can call `.formatted(args)` on it: `""" Hi %s """.formatted(name)`.

2.7.4 Reading input: Scanner

`java.util.Scanner` splits input into tokens at whitespace and parses them with `nextInt()`, `nextDouble()`, `next()` and similar, or reads a whole line with `nextLine()`. Construct it on `System.in` for keyboard input (`new Scanner(System.in)`) or, as above, on a `String` for testing. `hasNextInt()` and `hasNextLine()` let you loop until the input ends.

The classic pitfall is mixing token reads with `nextLine()`:

Pitfall.java JAVA

```java
import java.util.Scanner;

public class Pitfall {
    public static void main(String[] args) {
        Scanner sc = new Scanner("7\nAda Lovelace\n");
        int age = sc.nextInt();
        String name = sc.nextLine();
        System.out.println("[" + name + "]");
        String real = sc.nextLine();
        System.out.println("[" + real + "]");
        try { new Scanner("abc").nextInt(); }
        catch (java.util.InputMismatchException e) { System.out.println("mismatch"); }
    }
}
```

Output TERMINAL

```
$ java Pitfall.java
[]
[Ada Lovelace]
mismatch
```

`nextInt()` consumes `7` but leaves the line break behind, so the first `nextLine()` returns the empty rest of that line. Fix: call `nextLine()` once to discard it, or read every line with `nextLine()` and parse with `Integer.parseInt(line.trim())`. A bad token throws `InputMismatchException` and does not consume the token, so always validate or catch.

2.7.5 BufferedReader for speed and System.console

`Scanner` is convenient but slow for large input (it uses regular expressions internally). For competitive-style programs and big files use `BufferedReader`:

Rd.java JAVA

```java
import java.io.BufferedReader;
import java.io.IOException;
import java.io.InputStreamReader;

public class Rd {
    public static void main(String[] args) throws IOException {
        BufferedReader br = new BufferedReader(new InputStreamReader(System.in));
        String line;
        int sum = 0;
        while ((line = br.readLine()) != null) {
            sum += Integer.parseInt(line.trim());
        }
        System.out.println("sum=" + sum);
    }
}
```

Output (input piped in) TERMINAL

```
$ printf '3\n4\n5\n' | java Rd.java
sum=12
```

`readLine()` returns `null` at end of input, which is the loop's stop condition. Piping with `|` or redirecting with `< file.txt` lets you test console programs without typing. `System.console()` can read passwords without echo but returns `null` when input is redirected, so do not rely on it for general input. Java 25 also adds `IO.println` and `IO.readln` in `java.lang.IO` for small programs (not run on 21); the classic APIs above remain the portable choice.

WATCH OUT
Do not create a new `Scanner(System.in)` per method call, and do not close one wrapped around `System.in` and then create another: closing it closes `System.in` for good. Create one at the top of `main` and pass it around.

TIP
Use `%n`, not `\n`, in `printf` format strings when the output is for people on any platform; and never use `%d` with a `double` (it throws `IllegalFormatConversionException`).

Check yourself

1. What does `System.out.printf("%5.1f|%-4d|%04d%n", 2.25, 7, 7)` print? (Run it in jshell to confirm.)
2. In a text block, what decides how much leading whitespace is stripped?
3. What is wrong here? `int n = sc.nextInt(); String s = sc.nextLine();` with input `5` then `hello` on the next line.
4. Why is `System.err` used for error messages instead of `System.out`?
5. What does `"""` followed by `a \` newline `b"""` produce?

Answers

1. `2.3|7   |0007` followed by a line break. (`2.25` rounds half up to `2.3` in `Formatter`.)
2. The least-indented non-blank line, counting the line holding the closing `"""`.
3. `nextInt()` leaves the line break, so `s` is the empty string; call `nextLine()` first to consume it.
4. It is a separate stream, so a shell can redirect or filter errors independently of the program's real output.
5. The string `a b` (the trailing `\` removes the line break; the space before it is kept).

KEY POINTS
- `System.out` is for results, `System.err` for diagnostics; `printf` uses `%d %f %s %c %b %x %n` with width, flags and precision.
- Text blocks (`"""`) are plain `String`s with indentation stripped relative to the least-indented line or the closing delimiter.
- `Scanner` parses tokens; mixing `nextInt()` with `nextLine()` leaves a stray line break.
- Use `BufferedReader.readLine()` for large inputs; it returns `null` at end of input.
- Create one `Scanner` on `System.in` and never close it prematurely.

PART 3

Object-Oriented Java

Classes, objects, inheritance, interfaces and the design rules that keep a growing code base understandable.

In this part

- 3.1 Classes and Objects
- 3.2 Static, Access Control and Packages
- 3.3 Inheritance
- 3.4 Polymorphism and Dynamic Dispatch
- 3.5 Abstract Classes and Interfaces
- 3.6 The Object Class
- 3.7 Nested, Inner, Local and Anonymous Classes
- 3.8 Enums
- 3.9 Records
- 3.10 Sealed Classes and Interfaces
- 3.11 Pattern Matching
- 3.12 Wrapper Classes, Autoboxing and Identity
- 3.13 Designing Classes Well

3.1 Classes and Objects

Every Java program you will ever read is built from classes. A class is a blueprint that bundles data (fields) with the operations on that data (methods), and an object is one concrete thing made from that blueprint. This sounds simple, and it is, until the day a bug turns out to be "two variables pointed at the same object" or "a field was read before it was assigned". Both bugs come from not having an accurate mental picture of what the JVM does when you write `new`. This chapter builds that picture, including the exact order in which Java initialises a class and its objects.

3.1.1 State, behaviour and identity

An object has three things: **state** (the values stored in its fields), **behaviour** (the methods that read or change that state) and **identity** (it is this object and not another one that happens to hold the same values). Identity is the part beginners forget. Two `Book` objects with the same title and page count are still two objects; `==` on two references compares identity, not content.

A class declares what every object of that type has:

Book.java JAVA

```
public class Book {
    private final String title;   // state: set once, never changes
    private int pagesRead;        // state: changes as we read

    public Book(String title) {   // constructor: runs once per new Book
        this.title = title;
    }

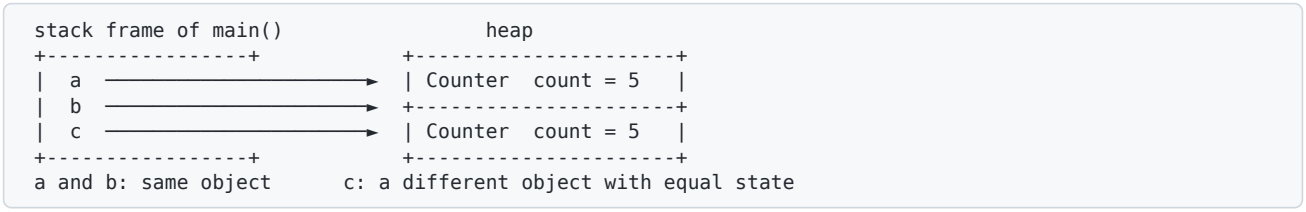
    public void read(int pages) { // behaviour
        pagesRead += pages;
    }

    public String progress() {
        return title + ": " + pagesRead + " pages read";
    }
}
```

The fields `title` and `pagesRead` exist once per object. The methods exist once per class: the JVM does not copy method code into every object. When you call `book.read(10)`, the JVM passes `book` to the method as a hidden first argument, which is what the keyword `this` refers to inside it.

3.1.2 References versus objects

When you write `Book b = new Book("Dune");` two different things happen. The `new` expression allocates an object on the **heap** and returns where it lives. The variable `b` is a **reference**: a small value (typically 4 or 8 bytes) that lets the JVM find the object. A local variable like `b` lives in the stack frame of the running method; the object it refers to lives on the heap. Copying a reference copies the arrow, never the object.



The next program makes this concrete. Read it first and predict the output of each line.

Refs.java JAVA

```java
public class Refs {
    static class Counter {
        int count;
        Counter(int start) { count = start; }
    }
    static void bump(Counter c) { c.count++; }
    static void replace(Counter c) { c = new Counter(100); c.count++; }

    public static void main(String[] args) {
        Counter a = new Counter(1);
        Counter b = a;                  // copies the reference, not the object
        b.count = 5;
        System.out.println("a.count = " + a.count);
        System.out.println("a == b: " + (a == b));
        Counter c = new Counter(5);
        System.out.println("a == c: " + (a == c));
        bump(a);
        System.out.println("after bump: " + a.count);
        replace(a);
        System.out.println("after replace: " + a.count);
        Counter d = null;
        try { d.count++; } catch (NullPointerException e) { System.out.println("NPE caught"); }
    }
}
```

Run it TERMINAL

```
$ java Refs.java
a.count = 5
a == b: true
a == c: false
after bump: 6
after replace: 6
NPE caught
```

What to notice. Changing `b.count` changed what `a` sees, because there is only one object. `a == c` is false although both hold 5, because `==` compares references. `bump` changed the caller's object, but `replace` did not: Java passes the *reference by value*. The method received its own copy of the arrow, repointed that copy at a brand new object, and the caller's arrow was untouched. "Java is pass-by-value, and for objects the value is a reference" is the exact, correct statement (Part 2 covers methods in more detail).

null and NullPointerException

A reference can hold `null`, meaning "no object". Using a null reference to reach a field or call a method throws `NullPointerException` (NPE). Since Java 14 the message says which expression was null, for example `Cannot invoke "String.length()" because "s" is null`; the variable name appears when the class was compiled with `javac -g` (otherwise you see `<local3>`). Primitives (`int`, `double`) can never be null; only references can.

When does an object die?

There is no `delete`. An object becomes eligible for garbage collection when no live reference can reach it any more. In `Object a = new Object(); Object b = a; a = null;` the object is still reachable through `b`, so it stays. Part 9 explains how the collector decides.

3.1.3 Constructors

A **constructor** is code that runs when `new` creates an object, before the reference is returned to you. It has the class name and no return type. Its job is to put the object into a valid state. If you write no constructor at all, the compiler adds a public no-argument **default constructor**; the moment you write any constructor, that free one disappears.

Constructors can call each other with `this(...)`, which avoids duplicating validation:

Ctors.java JAVA

```
public class Ctors {
    static class Book {
        private final String title;
        private final int pages;
        Book(String title, int pages) {
            if (pages <= 0) throw new IllegalArgumentException("pages must be positive: " + pages);
            this.title = title;
            this.pages = pages;
        }
        Book(String title) { this(title, 100); }   // delegates to the main constructor
        Book() { this("Untitled"); }               // delegates again
        @Override public String toString() { return title + " (" + pages + " pages)"; }
    }
    static class Empty { int v; boolean flag; String s; double d; }

    public static void main(String[] args) {
        System.out.println(new Book());
        System.out.println(new Book("Dune"));
        System.out.println(new Book("Dune", 612));
        Empty e = new Empty();
        System.out.println(e.v + " " + e.flag + " " + e.s + " " + e.d);
        try { new Book("Bad", 0); }
        catch (IllegalArgumentException ex) { System.out.println(ex.getMessage()); }
    }
}
```

Run it TERMINAL

```
$ java Ctors.java
Untitled (100 pages)
Dune (100 pages)
Dune (612 pages)
0 false null 0.0
pages must be positive: 0
```

What to notice. Only one constructor contains real logic; the others funnel into it. A `this(...)` call must be the first statement of the constructor (Java 25 relaxes this: code that does not touch `this` may precede it; see Part 11). The class `Empty` shows the **default values** the JVM gives *fields* before any code runs: `0` for numeric types, `false` for `boolean`, `'\u0000'` for `char`, `null` for references. **Local variables get no default**; the compiler refuses to compile a read of an unassigned local.

The keyword this

Inside an instance method or constructor, `this` is the reference to the current object. You need it in two situations: to tell a field from a parameter of the same name (`this.title = title`), and to pass the current object to another method. A constructor that fails by throwing leaves no usable object: the `new` expression never completes.

3.1.4 Initialisation order

An object does not appear fully formed. Java runs a precise sequence, and getting it wrong causes some of the most confusing bugs: a method called from a superclass constructor that sees a field as `null`. The sequence has two phases.

Class initialisation happens once per class, the first time the class is actively used (first `new`, first call of a static method, first access to a non-constant static field). The JVM first initialises the superclass, then runs the class's `static` field initialisers and `static { }` blocks, in the order they appear in the source.

Instance initialisation happens for every `new`. Memory is allocated and zeroed (all fields get their default values), then the constructor runs in this order: the call to the superclass constructor first (`super(...)`, implicit if you wrote none); then the class's instance field initialisers and instance initialiser blocks `{ }` in source order; then the rest of the constructor body. The same rule applies at every level of the hierarchy, from `Object` downwards.

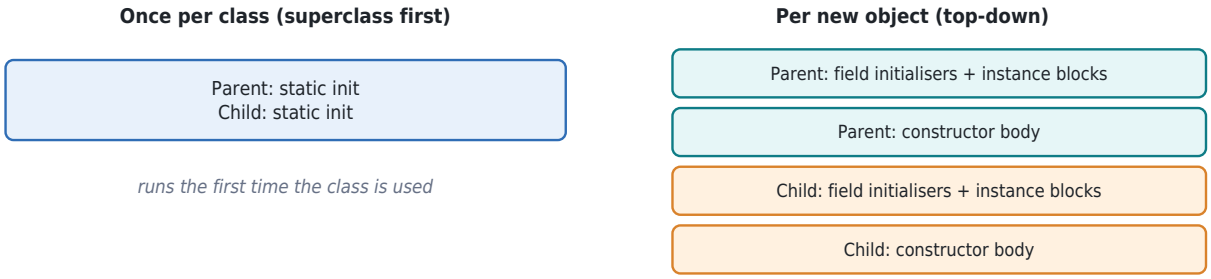


Figure 3.1. Class initialisation runs once, superclass first. Each new object then repeats the instance steps from the top of the hierarchy to the bottom.

This program prints the order. The numbers in the messages are the order we expect; the output proves it.

InitOrder.java JAVA

```java
public class InitOrder {
    static class Base {
        static { System.out.println("1 Base static block"); }
        int x = log("3 Base field initialiser");
        { System.out.println("4 Base instance block"); }
        Base() {
            System.out.println("5 Base constructor");
            show();                       // virtual call from a constructor: dangerous
        }
        void show() { System.out.println("Base.show"); }
    }
    static class Derived extends Base {
        static { System.out.println("2 Derived static block"); }
        int y = log("6 Derived field initialiser");
        String name = "set";
        { System.out.println("7 Derived instance block"); }
        Derived() {
            super();
            System.out.println("8 Derived constructor");
        }
        @Override void show() { System.out.println("Derived.show name=" + name + " y=" + y); }
    }
    static int log(String s) { System.out.println(s); return 1; }

    public static void main(String[] args) {
        System.out.println("main starts");
        new Derived();
        System.out.println("--- second object ---");
        new Derived();
    }
}
```

Run it TERMINAL

```
$ java InitOrder.java
main starts
1 Base static block
2 Derived static block
3 Base field initialiser
4 Base instance block
```

Run it (continued) TERMINAL

```
5 Base constructor
Derived.show name=null y=0
6 Derived field initialiser
7 Derived instance block
8 Derived constructor
--- second object ---
3 Base field initialiser
4 Base instance block
5 Base constructor
Derived.show name=null y=0
6 Derived field initialiser
7 Derived instance block
8 Derived constructor
```

What to notice. The static blocks ran once, superclass first, and not on the second `new`. The line `Derived.show name=null y=0` is the trap: `Base`'s constructor called `show()`, which is overridden, so the **Derived** version ran while `Derived`'s field initialisers had not yet executed. It saw `name` as `null` and `y` as `0`, the zeroed defaults, even though `name` is assigned `"set"` in the source. The rule: never call an overridable method from a constructor.

You can see the mechanism in the bytecode. `javap -c -p 'InitOrder$Derived'` (trimmed) shows that the compiler copied the field initialisers and the instance block *into the constructor*, right after the call to the superclass constructor, and gathered the static block into a method called `static {}` (the class initialiser, `<clinit>`):

Trimmed javap output TERMINAL

```
$ javap -c -p 'InitOrder$Derived'
  InitOrder$Derived();
    Code:
       0: aload_0
       1: invokespecial #1     // Method InitOrder$Base."<init>":()V
       4: aload_0
       5: ldc           #7     // String 6 Derived field initialiser
       7: invokestatic  #9     // Method InitOrder.log:(Ljava/lang/String;)I
      10: putfield      #15    // Field y:I
      13: aload_0
      14: ldc           #21    // String set
      16: putfield      #23    // Field name:Ljava/lang/String;
      19: getstatic     #27    // Field java/lang/System.out:...
      ...
  static {};
    Code:
       0: getstatic     #27    // Field java/lang/System.out:...
       3: ldc           #47    // String 2 Derived static block
```

Constants do not trigger class initialisation

A `static final` field whose value is a compile-time constant (a literal or constant expression of a primitive or `String`) is inlined by the compiler into every user. Reading it does not initialise the class at all:

Lazy.java JAVA

```java
public class Lazy {
    static class Config {
        static final int MAX = 10;                // compile-time constant: inlined
        static final String NAME = load("NAME");  // computed: not a constant
        static { System.out.println("Config initialised"); }
        static String load(String s) { return s.toLowerCase(); }
    }
    public static void main(String[] args) {
        System.out.println("MAX = " + Config.MAX);
        System.out.println("before NAME");
        System.out.println("NAME = " + Config.NAME);
    }
}
```

Run it TERMINAL

```
$ java Lazy.java
MAX = 10
before NAME
Config initialised
NAME = name
```

`MAX` printed without "Config initialised" appearing; reading `NAME` forced the class to initialise first. This laziness is also what makes the "initialisation on demand holder" singleton idiom in Part 12 work.

WATCH OUT
Never call an overridable method from a constructor. The subclass override runs before the subclass's own fields are initialised, so it reads defaults (`null`, `0`). Make the method `private`, `static` or `final`, or move the work out of the constructor into a factory method or an explicit `init()` step.

GOTCHA
Field initialisers run in source order, and that order matters. `int a = b + 1; int b = 5;` does not compile (illegal forward reference), but `int a = getB() + 1; int b = 5;` does, and `a` ends up as 1 because `getB()` ran while `b` was still 0.

3.1.5 Putting it together: a small class design

A well-formed class keeps its fields `private`, sets every field exactly once in a constructor that validates its input, and exposes only the operations callers need. Mark fields `final` whenever the value never changes after construction; the compiler then guarantees it is assigned exactly once, and the JVM guarantees other threads see it correctly (Part 8). Prefer a constructor that produces a fully valid object over a no-argument constructor followed by a chain of setters, because the latter lets other code observe a half-built object.

INTERVIEW ANGLE
What is the difference between a class and an object, and where do they live in memory? A class is a type definition (loaded once into the JVM, its metadata in Metaspace); an object is an instance created with `new`, allocated on the heap. A reference variable holds the address of the object, not the object, and the reference itself lives in a stack frame (local variable) or inside another object (field).

INTERVIEW ANGLE
In what order do static blocks, instance blocks, field initialisers and constructors run? Superclass static initialisation, then subclass static initialisation (once per class). For each `new`: superclass constructor chain first (its field initialisers and instance blocks in source order, then its constructor body), then the subclass's initialisers and blocks, then the subclass constructor body.

Check yourself

1. What does this print, and why? `Book a = new Book("X"); Book b = a; b = new Book("Y"); System.out.println(a == b);` (assume `Book` has the constructor shown above).
2. A class has no constructors. Another programmer adds `Book(String t)`. Existing code `new Book()` stops compiling. Why?
3. In `InitOrder`, why did `Derived.show` print `name=null` even though the field is assigned `"set"`?
4. What is wrong here? `class T { int a = b; int b = 2; }`
5. Does reading `Config.MAX` (a `static final int MAX = 10;`) run the static block of `Config`? What if the field is `static final int MAX = compute();`?
6. Can a Java method change which object the caller's variable refers to? Explain with the `replace` method from `Refs`.

Answers

1. It prints `false`. `b` was repointed at a new object; `a` still refers to the first one.
2. The compiler only supplies the default no-argument constructor when the class declares no constructor. Declaring any constructor removes it.
3. The superclass constructor called the overridden `show()` before the subclass field initialisers ran, so the fields still held default values.
4. Illegal forward reference: `a`'s initialiser reads `b` before `b` is declared. It is a compile-time error.
5. No for a compile-time constant (the value is inlined, the class is not initialised). Yes for `compute()`: the class initialises on first access.
6. No. The method gets a copy of the reference. Reassigning the parameter changes only the copy; mutating the object it points to is visible to the caller.

KEY POINTS

Objects live on the heap; variables hold references. Assignment copies the reference, `==` compares identity.

- Fields get default values (`0`, `false`, `null`); locals do not.
- Order: superclass static, subclass static (once); then per object: super constructor, field initialisers and blocks in source order, constructor body.
- The compiler copies instance initialisers into each constructor after `super(...)`; static ones go to `<clinit>`.
- Do not call overridable methods from constructors; prefer final fields and constructors that fully validate.

3.2 Static, Access Control and Packages

The previous chapter treated every field as belonging to an object. Some data and behaviour belong to the class as a whole instead: a counter of how many accounts have been opened, a constant, a utility function like `Math.max`. The keyword `static` expresses that. A second, equally important question is who is allowed to touch what. Access modifiers and packages let a class hide its internals so they can change later without breaking its users. Together these two ideas are the foundation of encapsulation.

3.2.1 Static members

A `static` field exists once per class, shared by all instances (and usable with no instance at all). A `static` method belongs to the class and has no `this`. Static members are accessed with the class name: `Account.issued()`.

Statics.java JAVA

```java
public class Statics {
    static class Account {
        private static int nextId = 1000;           // one copy, shared by all accounts
        private static final double MIN_BALANCE = 0.0;
        private final int id;                       // one copy per account
        private double balance;

        Account(double opening) { this.id = nextId++; this.balance = opening; }

        static int issued() { return nextId - 1000; }   // no 'this' here
        int id() { return id; }
        double balance() { return balance; }
        void withdraw(double amt) {
            if (balance - amt < MIN_BALANCE) throw new IllegalStateException("insufficient funds");
            balance -= amt;
        }
    }

    public static void main(String[] args) {
        Account a = new Account(50), b = new Account(70);
        System.out.println(a.id() + " " + b.id() + " issued=" + Account.issued());
        a.withdraw(20);
        System.out.println(a.balance());
        try { a.withdraw(100); }
        catch (IllegalStateException e) { System.out.println(e.getMessage()); }
    }
}
```

Run it TERMINAL

```
$ java Statics.java
1000 1001 issued=2
30.0
insufficient funds
```

What to notice. `nextId` is shared, so each constructor call hands out the next number. `id` is per object and `final`. A static method cannot read instance fields directly (there is no object), but an instance method can freely read static fields. The reverse mistake, calling an instance method from `static void main`, is the first compile error most beginners meet: `main` has no object, so you must create one first.

Static initialisers and constants

Constants are written `static final` with UPPER_SNAKE_CASE names. A static field that holds a mutable object (`static final List<String> NAMES = new ArrayList<>()`) is only a constant reference; the list can still change. Use `List.of(...)` or `Collections.unmodifiableList` for a genuinely fixed collection. Complex static setup goes in a `static { }` block, which runs once at class initialisation (see the previous chapter).

Static members are not polymorphic

You can write `someAccount.issued()`, and it compiles, but it is resolved from the *declared type* of the variable and never looks at the object. It is confusing, and linters flag it; always call statics through the class name. A subclass can declare a static method with the same signature, which *hides* rather than overrides the original (see the inheritance chapter).

WATCH OUT

Static mutable state is global state. A `static int counter` incremented by many threads is a race condition, and static state makes tests depend on each other, because it survives from one test to the next. Use static for constants, pure helper functions and factory methods. For shared mutable data, pass an object in (dependency injection, which Spring does for you).

3.2.2 Static nested classes: a preview

A class declared inside another class with the `static` modifier is a **static nested class**. It is an ordinary class that happens to live in the namespace of its outer class and can see the outer class's private static members. This book's examples use them (`Counter`, `Account` above) so that one file can hold several small classes. They do not hold a hidden reference to an outer object. The other kinds of nested class (inner, local, anonymous) are covered later in this Part.

3.2.3 Access modifiers

Java has four access levels. They answer: from where in the program can this class, field, method or constructor be reached?

Modifier	Same class	Same package	Subclass in other package	Anywhere
`private`	✓	✗	✗	✗
(none, package-private)	✓	✓	✗	✗
`protected`	✓	✓	✓ (via inheritance)	✗
`public`	✓	✓	✓	✓

Who can access a member? (✓ = allowed)

Top-level classes can only be `public` or package-private; the other two levels apply to members. The compiler enforces these rules. The following test class sits in the default package and extends a class from `com.example.lib`:

com/example/lib/Widget.java — JAVA

```java
package com.example.lib;

public class Widget {
    public int pub = 1;
    protected int prot = 2;
    int pkg = 3;                // package-private
    private int priv = 4;
    public int sum() { return pub + prot + pkg + priv; }
}
```

Other.java JAVA

```java
import com.example.lib.Widget;

public class Other extends Widget {
    void test(Widget w) {
        System.out.println(pub);      // public: fine
        System.out.println(prot);     // protected, via inheritance: fine
        System.out.println(w.prot);   // protected, via a plain Widget reference: error
        System.out.println(pkg);      // package-private: error
        System.out.println(priv);     // private: error
    }
}
```

Compile it TERMINAL

```
$ javac -d out com/example/lib/Widget.java Other.java
Other.java:7: error: prot has protected access in Widget
        System.out.println(w.prot);   // protected via a Widget reference: error
                           ^
Other.java:8: error: pkg is not public in Widget; cannot be accessed from outside package
        System.out.println(pkg);      // package-private: error
                           ^
Other.java:9: error: priv has private access in Widget
        System.out.println(priv);     // private: error
                           ^
3 errors
```

What to notice. `protected` is the subtle one. A subclass in another package may use the member only on *its own* instances (`this.prot`, or an object of the subclass type), not on an arbitrary `Widget` reference, which is why `w.prot` fails while plain `prot` works. Within the same package, `protected` behaves like package-private plus the inheritance rule.

Choosing a level

Start with `private`. Widen only when a real caller needs it: package-private for helpers shared inside one package, `public` for the actual API, `protected` rarely (it publishes your internals to every subclass author forever). Every public member is a promise you must keep.

3.2.4 Packages

A **package** is a named group of related classes, and it maps to a directory: class `com.example.lib.Widget` lives in `com/example/lib/Widget.java`. Packages do three jobs: they avoid name clashes (two libraries can both have a class `Date`), they define the package-private access boundary, and they organise code. By convention package names are all lower case and start with a reversed domain name you own (`com.example.shop`), which makes them globally unique.

A file starts with `package com.example.lib;`. To use a class from another package, import it (`import java.util.List;`) or write its full name. `import java.util.*;` imports every class in that package on demand but not sub-packages (`java.util.concurrent` needs its own import). Everything in `java.lang` (`String`, `Math`, `Object`) is imported automatically. A `static import` (`import static java.lang.Math.max;`) lets you call a static member without the class name; use it sparingly, mostly for constants and test assertions.

Using a package is a build concern too: `javac -d out ...` writes class files into a matching directory tree, and `java -cp out com.example.lib.Main` runs one. Part 1 covered the classpath; Part 11 covers modules, which add a stronger boundary above packages.

3.2.5 Encapsulation

Encapsulation means a class hides how it stores its data and offers a small set of operations instead. The reward is freedom to change: the class `Temperature` can store Celsius today and Kelvin tomorrow, and no caller breaks, provided they only used the methods. The cost of not doing it is that every caller becomes coupled to every field.

Getters and setters: a fair debate

The textbook pattern is private fields with public `getX()` and `setX()`. Used mechanically it adds nothing: a class with a getter and setter for every field is just a struct with extra typing. The question to ask is "what operation does the caller want?". A `BankAccount` should offer `deposit(amount)` and `withdraw(amount)`, which can check rules, rather than `setBalance(...)`, which lets anyone put in any number. Guidelines:

- Expose a getter only when callers truly need to read the value; do not generate them by reflex.

- Prefer behaviour methods that enforce invariants over setters. If a setter must exist, validate in it.
- Prefer immutable objects (next section) and, for plain data carriers, records (covered later in this Part), which give you accessors with no boilerplate.
- Frameworks matter in practice: JavaBeans conventions (`getX`/`isX`/`setX`) are what Spring, Jackson and JPA use to read and write properties, so entity and DTO classes often keep them. That is a framework requirement, not a design ideal.

3.2.6 Immutability and defensive copies

An **immutable** object never changes after construction. `String`, `Integer`, `BigDecimal` and `LocalDate` are immutable. Immutable objects are easy to reason about, safe to share between threads without locking, and safe as `HashMap` keys. The recipe for an immutable class:

1. Declare the class `final` (so no subclass can add mutable state).
2. Make all fields `private final`.
3. Provide no setters; "modifying" methods return a new object.
4. Make a **defensive copy** of any mutable object received in the constructor, and return copies (or unmodifiable views) from getters.

Step 4 is the one people miss. If the constructor stores the caller's list, the caller still holds a reference to it and can change it behind your back:

Defensive.java JAVA

```java
import java.util.ArrayList;
import java.util.List;

public class Defensive {
    static final class LeakyShelf {
        private final List<String> books;
        LeakyShelf(List<String> books) { this.books = books; }   // stores the caller's list
        List<String> books() { return books; }                   // hands out the internal list
    }
    static final class SafeShelf {
        private final List<String> books;
        SafeShelf(List<String> books) { this.books = List.copyOf(books); }  // copy in
        List<String> books() { return books; }   // safe: List.copyOf is unmodifiable
    }

    public static void main(String[] args) {
        List<String> src = new ArrayList<>(List.of("Dune", "Emma"));
        LeakyShelf leaky = new LeakyShelf(src);
        src.add("Hacked");
        leaky.books().remove("Dune");
        System.out.println("leaky: " + leaky.books());

        src = new ArrayList<>(List.of("Dune", "Emma"));
        SafeShelf safe = new SafeShelf(src);
        src.add("Hacked");
        System.out.println("safe: " + safe.books());
        try { safe.books().add("x"); }
        catch (UnsupportedOperationException e) {
            System.out.println("UnsupportedOperationException");
        }
    }
}
```

Run it TERMINAL

```
$ java Defensive.java
leaky: [Emma, Hacked]
safe: [Dune, Emma]
UnsupportedOperationException
```

What to notice. `final` on `books` only froze the *reference*; the list behind it kept changing in `LeakyShelf`. `List.copyOf` (Java 10+) creates an unmodifiable snapshot, so changes to `src` do not reach `SafeShelf` and the getter can safely return it. For an array field, use `array.clone()` on the way in and out. For a mutable element type such as the legacy `java.util.Date`, copy each element too (`new Date(d.getTime())`), or better, avoid mutable types and use `java.time`.

GOTCHA

`final` means "this variable cannot be reassigned", not "this object cannot change". `final List<String> xs = new ArrayList<>(); xs.add("ok");` compiles and works. Immutability comes from the design of the class, not from the keyword on a field.

REAL WORLD

In Spring applications, immutable configuration and value objects are everywhere. Constructor injection (`final` fields set in the constructor) is preferred over field injection because the bean cannot be observed half-built and tests can create it with plain `new`. DTOs are increasingly records.

INTERVIEW ANGLE

How do you make a class immutable? Is `String` immutable because its field is `final`? `final` class, `private final` fields, no setters, defensive copies of mutable inputs and outputs. `String` is immutable because it exposes no operation that changes its internal state (its array is private and never leaked), not merely because a field is `final`.

Check yourself

1. What is the difference between a static field and an instance field? Which one is stored once per object?
2. Why does this fail to compile? `public class A { int n; public static void main(String[] a) { n = 1; } }`
3. A class in package `p1` has a package-private method. Can a subclass in package `p2` override it? Can code in `p1` call it?
4. What does this print? `final int[] a = {1}; a[0] = 2; System.out.println(a[0]);` Does `final` stop it?
5. Name two things that break the immutability of a class whose fields are all `private final`.
6. Why is `setBalance(double)` a poor method on a bank account class? What would you offer instead?

Answers

1. A static field exists once per class and is shared by all objects; an instance field exists once per object.
2. `main` is static and has no `this`, so it cannot access the instance field `n` without an object.
3. No: package-private members are not visible outside `p1`, so a `p2` method with that signature is a new method, not an override. Yes, code in `p1` can call it.
4. It prints `2`. `final` only prevents reassigning `a`; the array contents can change.
5. Storing a mutable object passed in by the caller without copying it, and returning a reference to an internal mutable object (also: a non-final class that a subclass can make mutable).
6. It lets any caller bypass the rules (overdraft limits, auditing). Offer `deposit` and `withdraw`, which validate and keep the balance consistent.

KEY POINTS

Use `static` for class-level constants, helpers and factories; avoid shared mutable static state.

- Four access levels: `private`, package-private, `protected`, `public`. Start private and widen only when needed.
- A package is a namespace, a directory and an access boundary; names are lower case, reversed-domain.
- Encapsulate behaviour, not just fields: offer operations that enforce invariants, not a setter per field.
- Immutability needs a final class, private final fields, no setters, and defensive copies in and out.
- `final` freezes a reference, not the object it refers to.

3.3 Inheritance

Suppose your library system has `Book`, `Magazine` and `DVD`. All three have a title and can be borrowed. Copying the same fields and methods into three classes is tedious and error-prone: fix a bug in one and the other two keep it. **Inheritance** lets a class reuse another class's members by declaring `extends`: the new class (the **subclass**, or child) automatically has everything the **superclass** (parent) has, and may add to it or change it. Inheritance also creates an *is-a* relationship that the type system understands, which is what makes polymorphism, the next chapter's subject, possible.

Inheritance is also the most over-used feature in object-oriented programming. This chapter teaches the mechanics precisely, and ends with when not to use it.

3.3.1 extends and super

Hier.java JAVA

```java
public class Hier {
    static class Vehicle {
        protected final String name;
        Vehicle(String name) {
            this.name = name;
            System.out.println("Vehicle(" + name + ")");
        }
        Vehicle describe() { System.out.println("Vehicle " + name); return this; }
        static String kind() { return "vehicle"; }
    }
    static class Car extends Vehicle {
        private final int doors;
        Car(String name, int doors) {
            super(name);                        // must come first
            this.doors = doors;
            System.out.println("Car(" + doors + ")");
        }
        Car(String name) { this(name, 4); }
        @Override Car describe() {              // covariant return type
            System.out.println("Car " + name + " doors=" + doors);
            return this;
        }
        static String kind() { return "car"; } // hides, does not override
    }
    static class Truck extends Vehicle {
        Truck() { super("truck"); }
    }

    public static void main(String[] args) {
        Car c = new Car("Mini").describe();
        Vehicle v = c;
        System.out.println(v.describe() == c);
        System.out.println(Vehicle.kind() + " " + Car.kind());
        new Truck();
    }
}
```

Run it TERMINAL

```
$ java Hier.java
Vehicle(Mini)
Car(4)
Car Mini doors=4
Car Mini doors=4
true
vehicle car
Vehicle(truck)
```

What to notice. Each class extends exactly one other class: Java has single inheritance of classes (multiple inheritance of *type* is available through interfaces, covered later). Every class without an `extends` clause extends `java.lang.Object`, so every object has `toString`, `equals`, `hashCode` and friends. The superclass constructor runs first (`Vehicle(Mini)` printed before `Car(4)`), which is why a subclass constructor starts with `super(...)`.

The keyword `super` has two uses: `super(args)` as the first statement of a constructor calls a superclass constructor, and `super.method()` calls the superclass's version of a method from inside an override. `Car.describe()` could have called `super.describe()` to reuse the parent's behaviour and then added to it.

Constructors in a hierarchy

Constructors are **not inherited**. If you write no `super(...)` call, the compiler inserts `super();` for you, and that fails to compile if the parent has no no-argument constructor:

Bad1.java JAVA

```java
public class Bad1 {
    static class Parent {
        Parent(String name) { }
    }
    static class Child extends Parent {
        Child() { }             // implicit super() - Parent has no such constructor
    }
}
```

Compile it TERMINAL

```
$ javac Bad1.java
Bad1.java:6: error: constructor Parent in class Parent cannot be applied to given types;
        Child() { }             // implicit super() - Parent has no such constructor
                ^
  required: String
  found:    no arguments
  reason: actual and formal argument lists differ in length
```

The fix is an explicit `super("something")`. Since Java 25 (JEP 513, flexible constructor bodies) you may run statements *before* `super(...)` as long as they do not use `this`, for example validating an argument or computing a value to pass up. That is not runnable on Java 21, so earlier code resorts to static helper methods for the same job.

3.3.2 Overriding rules

A subclass **overrides** an inherited instance method by declaring one with the same name and the same parameter types. The rules the compiler enforces are the Liskov Substitution Principle in code form: anything that works with the parent must still work with the child.

Aspect	Rule
Access	Cannot be weaker than the parent's (a `public` method cannot become package-private); may be wider.
Return type	Same, or a subtype of the parent's (a **covariant return**: `Car describe()` overrides `Vehicle describe()`). Primitives must match exactly.
Checked exceptions	May throw fewer, or narrower, checked exceptions than the parent; never broader new ones. Unchecked exceptions are unrestricted.
`final` method	Cannot be overridden.
`static` method	Cannot be overridden; a same-signature static method **hides** the parent's.
`private` method	Not inherited, so not overridable; the subclass method is unrelated.
Annotation	Write `@Override` always.

Overriding rules.

Here is one class that breaks four of those rules at once, together with what the compiler says (shortened):

Bad2.java JAVA

```java
public class Bad2 {
    static class Parent {
        public void run() { }
        final void lock() { }
        void load() throws java.io.IOException { }
        Number size() { return 1; }
    }
    static class Child extends Parent {
        void run() { }                  // weaker access
        void lock() { }                 // final in parent
```

Bad2.java (continued) JAVA

```
        void load() throws Exception { } // broader checked exception
        String size() { return ""; }     // not a subtype of Number
    }
}
```

Compile it (shortened) TERMINAL

```
$ javac Bad2.java
Bad2.java:9: error: run() in Child cannot override run() in Parent
  attempting to assign weaker access privileges; was public
Bad2.java:10: error: lock() in Child cannot override lock() in Parent
  overridden method is final
Bad2.java:11: error: load() in Child cannot override load() in Parent
  overridden method does not throw Exception
Bad2.java:12: error: size() in Child cannot override size() in Parent
  return type String is not compatible with Number
```

Always write @Override

`@Override` asks the compiler to verify that you really override something. Without it, a typo (`tostring()`) or a wrong parameter type silently creates a *new* method that is never called polymorphically. The next chapter shows exactly this bug with `equals(Point)` versus `equals(Object)`.

Hiding is not overriding

A `static` method with the same signature in a subclass, and a field with the same name, **hide** the parent's. Which one you get depends on the *declared type* of the expression, not on the object:

Hide.java JAVA

```
public class Hide {
    static class P {
        static String who() { return "P"; }
        String id() { return "P.id"; }
    }
    static class C extends P {
        static String who() { return "C"; }
        @Override String id() { return "C.id+" + super.id(); }
    }
    public static void main(String[] a) {
        P p = new C();
        System.out.println(p.who() + " " + p.id());
    }
}
```

Run it TERMINAL

```
$ java Hide.java
P C.id+P.id
```

The variable `p` is declared `P`, so `p.who()` is `P.who()`, even though the object is a `C`. The instance method `id()` was overridden, so the object's real class decided. Rule of thumb: only instance methods are polymorphic; fields and static methods are looked up using the declared type.

3.3.3 final classes and methods

`final` on a method forbids overriding; on a class forbids extending (`String`, `Integer`, and all records are final). Use `final` when a class is not designed for extension, which is true of most classes. A class that allows subclassing must document how overridable methods are used by other methods (the "self-use" contract), otherwise a subclass author cannot know which override is safe. Joshua Bloch's advice in *Effective Java*, "design and document for inheritance or else prohibit it", is the standing guidance. Sealed classes (a later chapter in this Part) offer a middle path: a closed set of permitted subclasses.

3.3.4 protected and the fragile base class

`protected` lets subclasses reach into a parent's fields and helper methods. It is convenient, and it welds the subclass to the parent's internals: if the parent later changes how it stores its data, every subclass that touched the field breaks. This is the **fragile base class problem**. A classic example, from *Effective Java*: a `CountingSet` that extends `HashSet` and overrides `add` and `addAll` to count insertions.

CS.java JAVA

```java
import java.util.Collection;
import java.util.HashSet;
import java.util.List;

public class CS {
    static class CountingSet<E> extends HashSet<E> {
        int added;
        @Override public boolean add(E e) { added++; return super.add(e); }
        @Override public boolean addAll(Collection<? extends E> c) {
            added += c.size();
            return super.addAll(c);
        }
    }
    public static void main(String[] a) {
        CountingSet<String> s = new CountingSet<>();
        s.addAll(List.of("a", "b", "c"));
        System.out.println(s.added);
    }
}
```

Run it TERMINAL

```
$ java CS.java
6
```

Three elements were added but the counter says 6: the inherited `addAll` implementation calls `add` for each element, so each element was counted twice. The subclass depended on an implementation detail of the parent, which the parent never promised and may change in any release.

The remedy is usually **composition**: instead of `class CountingSet extends HashSet`, write a class that has a `Set` field and forwards calls to it, adding the counting on top. The forwarding class depends only on the public contract of `Set`. A later chapter on class design develops this fully ("favour composition over inheritance").

3.3.5 When to extend

Use inheritance when all of these are true: the relationship really is *is-a* in every context (a `Car` is a `Vehicle` everywhere a vehicle is accepted), the parent was designed for extension, and you want substitutability (callers should be able to use a child where a parent is expected). Do not use it just to reuse code: a `Stack` that extends `ArrayList` can be indexed and cleared from outside, and a `Square` that extends `Rectangle` breaks the promise that setting the width does not change the height. When in doubt, use composition or an interface.

TIP

Run `javac -Xlint:all` to get warnings for risky overriding patterns. It also flags missing `serialVersionUID`, raw types and many other problems at no cost. Many teams make warnings fail the build.

INTERVIEW ANGLE

Can you override a static method? A private method? A constructor? No to all three. A static method with the same signature hides the parent's, resolved by declared type. A private method is not inherited, so a same-named method in the subclass is a different method. Constructors are not inherited and cannot be overridden; you call them via `super(...)`.

INTERVIEW ANGLE

What happens if a superclass constructor calls an overridden method? The overridden version in the subclass runs while the subclass's fields are still at their default values, because the subclass initialisers have not run yet. Avoid calling overridable methods in constructors.

Check yourself

1. What does this print? `class A { A() { System.out.print("A"); } } class B extends A { B() { System.out.print("B"); } } new B();`
2. Why does `class Parent { Parent(int x) {} } class Child extends Parent { }` not compile?
3. Can a subclass override `public Object get()` with `String get()`? With `private Object get()`? Explain.
4. What does this print? `class P { static String s() { return "P"; } } class C extends P { static String s() { return "C"; } } P p = new C(); System.out.println(p.s());`
5. Why is `class Stack extends ArrayList` considered bad design even though it compiles and works?
6. What is the fragile base class problem?

Answers

1. `AB`. The superclass constructor runs first (implicit `super()`), then the subclass constructor body.
2. `Child` has the implicit default constructor, which calls `super()`, but `Parent` has no no-argument constructor. `Child` needs an explicit constructor calling `super(someInt)`.
3. Yes to `String get()`: covariant return types are allowed. No to `private`: access cannot be weaker than `public`.
4. It prints `P`: static methods are hidden, and the declared type `P` decides.
5. It exposes methods like `add(index, e)` and `remove(index)` that break stack semantics, and it ties the class to `ArrayList` forever. A stack should contain a list (composition).
6. A subclass depends on implementation details of its parent, so a harmless-looking change in the parent breaks the subclass.

KEY POINTS

Single inheritance of classes with `extends`; every class ends at `Object`. Constructors are never inherited; `super(...)` chains them, superclass first.

- Overriding: same signature, access not weaker, covariant return, no broader checked exceptions, never `final` or `static`. Always write `@Override`.
- Fields and static methods are hidden, not overridden; the declared type decides which one you get.
- `protected` couples subclasses to parent internals (the fragile base class problem).
- Prefer composition; extend only for a true, designed-for-extension is-a relationship.

3.4 Polymorphism and Dynamic Dispatch

Polymorphism ("many forms") is the payoff of inheritance and interfaces: you write code against a general type, and each object responds in its own way. A method `printAll(List<Shape> shapes)` calls `shape.area()` and gets a circle's area for circles and a rectangle's for rectangles, without a single `if`. New shapes can be added later and `printAll` never changes. To use this safely you must know *which* decisions the compiler makes and which the JVM makes at run time. Mixing them up is the cause of a famous category of interview trick questions and of real bugs.

3.4.1 Static type versus dynamic type

Every reference expression has two types. The **static type** (declared type) is what the source code says: in `Animal a = new Dog();` the static type of `a` is `Animal`. The **dynamic type** (runtime type) is the actual class of the object the reference points to: `Dog`. The compiler knows only static types. It uses them to decide what you are allowed to write: `a.bark()` is rejected if `Animal` has no `bark()`, even though the object could do it.

The JVM knows dynamic types. When a method is invoked on an object, the JVM picks the implementation by the object's real class. That is **dynamic dispatch**, also called late binding or runtime polymorphism.

3.4.2 Overriding versus overloading

These two words sound alike and work in opposite ways.

Aspect	Overloading	Overriding
What	Several methods in one class (or hierarchy) with the same name and different parameter lists	A subclass replaces an inherited instance method with the same signature

Aspect	Overloading	Overriding
Chosen	At **compile time**, from the static types of the arguments	At **run time**, from the dynamic type of the receiver object
Also called	Static / early binding	Dynamic / late binding
Return type	May differ (but cannot be the only difference)	Same or covariant

Overloading vs overriding.

The program below shows all the dispatch decisions in one place: field hiding, overriding, and an overload that looks like an override.

Dispatch.java JAVA

```java
public class Dispatch {
    static class Animal {
        String name = "animal-field";
        String sound() { return "..."; }
        String greet(Animal other) { return "Animal meets Animal"; }
    }
    static class Dog extends Animal {
        String name = "dog-field";
        @Override String sound() { return "Woof"; }
        String greet(Dog other) { return "Dog meets Dog"; }   // overload, NOT an override
    }

    public static void main(String[] args) {
        Animal a = new Dog();
        System.out.println(a.sound());         // dynamic: Woof
        System.out.println(a.name);            // fields use the static type
        System.out.println(((Dog) a).name);    // cast changes the static type
        Dog d = new Dog();
        System.out.println(d.greet(d));        // Dog meets Dog
        System.out.println(a.greet(d));        // overload chosen by static type of a
        Animal plain = new Animal();
        try { Dog bad = (Dog) plain; }
        catch (ClassCastException e) {
            System.out.println("CCE: " + e.getMessage().substring(0, 60));
        }
        Object o = d;
        if (o instanceof Dog dd && dd.name.startsWith("dog")) {
            System.out.println("pattern ok: " + dd.sound());
        }
        System.out.println(null instanceof Animal);
    }
}
```

Run it TERMINAL

```
$ java Dispatch.java
Woof
animal-field
dog-field
Dog meets Dog
Animal meets Animal
CCE: class Dispatch$Animal cannot be cast to class Dispatch$Dog (
pattern ok: Woof
false
```

What to notice, line by line:

- `a.sound()` prints `Woof`: the method is overridden, so the object's class (`Dog`) decides.
- `a.name` prints `animal-field`: **fields are never polymorphic**. A `Dog` object contains two separate `name` fields (one from `Animal`, one that hides it), and the static type picks which.
- `a.greet(d)` prints `Animal meets Animal`. `Dog.greet(Dog)` does not override `Animal.greet(Animal)`; it is an overload with a different parameter type. The compiler looked at `a`'s static type `Animal`, found only `greet(Animal)`, and generated a call to exactly that method. Adding `@Override` to `Dog.greet(Dog)` would have turned this into a compile error and saved you.
- The `ClassCastException` message names both classes. Casting down to a subtype is checked at run time.
- `null instanceof Animal` is `false`; `instanceof` never throws on `null`.

The classic equals bug

The same trap appears in real code. Here `equals(P)` overloads rather than overrides `Object.equals(Object)`:

Over.java JAVA

```java
import java.util.HashSet;
import java.util.Set;

public class Over {
    static class P {
        final int x;
        P(int x) { this.x = x; }
        public boolean equals(P other) { return other != null && x == other.x; }  // overload!
        @Override public int hashCode() { return x; }
    }
    static void show(Object o) { System.out.println("Object"); }
    static void show(String s) { System.out.println("String"); }
    static void pick(long v) { System.out.println("long"); }
    static void pick(Integer v) { System.out.println("Integer"); }
    static void pick(int... v) { System.out.println("varargs"); }

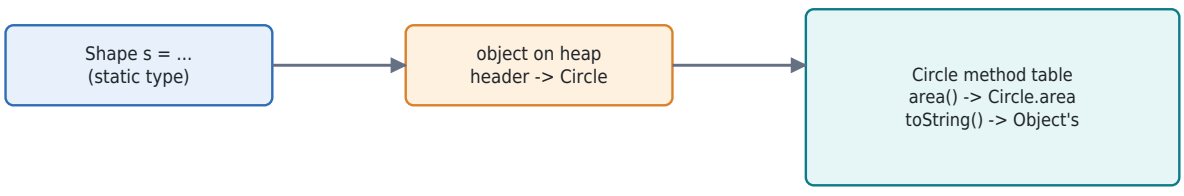
    public static void main(String[] args) {
        P a = new P(1), b = new P(1);
        Object ob = b;
        System.out.println(a.equals(b) + " " + a.equals(ob));
        Set<P> s = new HashSet<>();
        s.add(a);
        System.out.println(s.contains(b));
        Object o = "text";
        show(o);                // static type Object, even though the object is a String
        show("text");
        pick(5);
    }
}
```

Run it TERMINAL

```
$ java Over.java
true false
false
Object
String
long
```

`a.equals(b)` is true (the overload matches), but `a.equals(ob)` is false because the static type of `ob` is `Object`, which selects `Object.equals`, the inherited identity version. `HashSet.contains` calls `equals(Object)`, so the set does not find the "equal" point. With `@Override` on that method the compiler would have refused. The last three lines show overload resolution: it uses the static type (`show(o)` picks the `Object` version), and prefers widening (`int` to `long`) over boxing (`Integer`) over varargs. Phases are tried in that order: no boxing and no varargs first, then boxing, then varargs.

3.4.3 How dynamic dispatch works



s.area(): the object's real class picks the method at run time

Figure 3.2. A call through a superclass reference reaches the method of the object's real class through that class's method table.

Conceptually each loaded class has a **method table** (a *vtable*): an array of pointers to the implementation of each virtual method. A subclass copies its parent's table and replaces the entries for methods it overrides; new methods are appended. Each object's header points to its class, so `a.sound()` becomes "load the class from the object, index into the table at the slot number assigned to `sound`, jump". The slot number is the same in every subclass, which is why the

compiler can emit a fixed instruction while the target varies. Interface methods use a similar structure (an itable), searched by interface. This describes the idea; HotSpot's exact layout is an implementation detail.

You can see the compile-time part in bytecode. Take this small program:

Call.java JAVA

```java
public class Call {
    static class A { void hi() { System.out.println("A"); } }
    static class B extends A { @Override void hi() { System.out.println("B"); } }
    static void run(A a) { a.hi(); }
    public static void main(String[] args) { run(new B()); }
}
```

Bytecode of run(), then the calls in Dispatch TERMINAL

```
$ javac Call.java && javap -c -p Call
  static void run(Call$A);
    Code:
       0: aload_0
       1: invokevirtual #7      // Method Call$A.hi:()V
       4: return
$ javap -c -p Dispatch | grep invokevirtual      # trimmed
 12: invokevirtual #16   // Method Dispatch$Animal.sound:()Ljava/lang/String;
 54: invokevirtual #33   // Method Dispatch$Dog.greet:(LDispatch$Dog;)Ljava/lang/String;
 65: invokevirtual #37   // Method Dispatch$Animal.greet:(LDispatch$Animal;)Ljava/lang/String;
```

The instruction names `Call$A.hi`, the *static* type, and the JVM resolves it to `B.hi` when run with a `B`. The last two lines are the overload story: the compiler wrote the chosen parameter types (`Dog` or `Animal`) into the method descriptor. There was no `invokevirtual` for `Dog.greet(Animal)` because that method does not exist; overload choice is finished before the program runs.

The five invoke instructions tell you how each call is bound:

Instruction	Used for
`invokevirtual`	Ordinary instance method calls on a class type; dispatched at run time.
`invokeinterface`	Calls through an interface type; dispatched at run time.
`invokespecial`	Constructors, `super.method()` calls and private methods: no dispatch, target fixed.
`invokestatic`	Static methods; no receiver object.
`invokedynamic`	Lambdas, string concatenation: target decided on first call by a bootstrap method.

JVM invocation instructions.

NOTE
Is dynamic dispatch slow? Less than you would expect. The JIT compiler (Part 1, Part 9) watches which classes actually reach a call site. If only one does (monomorphic), it inlines that method behind a cheap class check; with two, it inlines both (bimorphic). Only call sites that see many classes (megamorphic) pay for a real table lookup. Class hierarchy analysis lets it skip dispatch entirely when no loaded subclass overrides the method.

3.4.4 Casting and instanceof

An **upcast** (treating a `Dog` as an `Animal`) is always safe and needs no syntax: `Animal a = new Dog();`. A **downcast** goes the other way, `Dog d = (Dog) a;`, and is checked by the JVM at run time: if the object is not a `Dog` (or a subclass), you get `ClassCastException`. The compiler also rejects casts between unrelated classes (`(String) new Dog()` does not compile), but allows any cast between related types and between a class and an interface, because the runtime object might satisfy it.

Before a downcast, test with `instanceof`. Since Java 16, pattern matching for `instanceof` tests and casts in one step:

JAVA

```java
// Old style
if (shape instanceof Circle) {
    Circle c = (Circle) shape;
    System.out.println(c.radius());
}
// Pattern matching (Java 16+): c is in scope where the test is known to be true
if (shape instanceof Circle c && c.radius() > 1.0) {
    System.out.println(c.radius());
}
```

Frequent downcasting is a smell. If you find a chain of `if (x instanceof A) ... else if (x instanceof B) ...` repeated in several places, the behaviour probably belongs in a method on the types themselves (polymorphism), or, for a closed set of types, in a `switch` over a sealed hierarchy with pattern matching, covered later in this Part. Note also that after generics erasure you cannot write `x instanceof List<String>`; only the raw type can be tested at run time.

3.4.5 Making polymorphism work for you

The point is to replace type-testing conditionals with method calls. Here is the standard before-and-after, with a discount rule per customer type:

JAVA

```java
// Before: every new customer type means editing this method
double discount(Customer c) {
    if (c instanceof Regular) return 0.0;
    else if (c instanceof Premium) return 0.10;
    else return 0.25;
}

// After: each type owns its rule; adding a type changes no existing code
abstract class Customer { abstract double discount(); }
class Regular extends Customer { double discount() { return 0.0; } }
class Premium extends Customer { double discount() { return 0.10; } }
```

This is the open/closed principle at work (open for extension, closed for modification), which the class design chapter returns to.

GOTCHA

Calling an overridable method on `this` from a constructor, a field read through the wrong type, an overload that is not an override, a static method called "through an object": all four are the same lesson. Only instance methods are late-bound; everything else is decided by static types or by the order of initialisation.

INTERVIEW ANGLE

What is the difference between method overloading and overriding? Which is compile-time and which is run-time polymorphism? Overloading: same name, different parameters, chosen at compile time by static argument types. Overriding: subclass replaces a parent's instance method with the same signature, chosen at run time by the receiver's actual class. Overriding is the true (run-time) polymorphism.

INTERVIEW ANGLE

What does this print? `Animal a = new Dog(); System.out.println(a.name);` when both classes declare a field `name`. The `Animal` field, because fields are resolved by the static type. Only methods are dispatched dynamically.

Check yourself

1. What is the difference between the static type and the dynamic type of `Animal a = new Dog();`?
2. What does this print? `static void f(Object o){System.out.print("O");} static void f(String s){System.out.print("S");} ... Object x = "hi"; f(x); f("hi"); f(null);`
3. Why does a class with `public boolean equals(Point p)` not work in a `HashSet<Point>`? What one-word fix prevents the mistake?
4. Which of `a.name`, `a.sound()`, `a.greet(d)` (from `Dispatch`) are resolved at run time?
5. When does a downcast throw `ClassCastException`, and how do you avoid it?
6. Which JVM instruction is used for a call to a `private` method, and why does it need no dispatch?

Answers

1. Static: `Animal`, known to the compiler from the declaration. Dynamic: `Dog`, the real class of the object, known only at run time.
2. `OSS`. The first call uses static type `Object`; for `f(null)` the most specific applicable overload is `String`.
3. It overloads instead of overriding `equals(Object)`, and `HashSet` calls the latter. `@Override` makes the compiler reject it.
4. Only `a.sound()` (an overridden instance method). The field read and the overload choice are decided at compile time.
5. When the object's real class is not the target type or a subclass. Test first with `instanceof` (ideally the pattern form), or redesign so the cast is unnecessary.
6. `invokespecial`: a private method cannot be overridden, so the target is known at compile time.

KEY POINTS

Every expression has a static type (compiler) and a dynamic type (JVM). Instance methods dispatch on the dynamic type; fields, static methods and overload choice use the static type.

- Overloading is chosen at compile time; overriding at run time. Always use `@Override`.
- Dispatch is a method-table lookup in concept; the JIT removes most of the cost at monomorphic call sites.
- Upcasts are implicit and safe; downcasts are runtime-checked. Use `instanceof` patterns, but prefer polymorphic methods over type tests.

3.5 Abstract Classes and Interfaces

Inheritance lets a subclass reuse code. But often you want something different: to say "anything that can be drawn", "anything that can be compared", without saying how it is drawn or compared, and without forcing all such things into one family tree. Java gives you two tools for this. An **abstract class** is a partly finished class that cannot be instantiated. An **interface** is a pure contract, a list of capabilities that any class, anywhere in the hierarchy, can promise to provide. Knowing when to use which is a staple of design discussions and interviews.

3.5.1 Abstract classes

An `abstract` class is declared with the keyword and may contain **abstract methods**: a signature with no body. A concrete subclass must implement every abstract method, or itself be declared abstract. You cannot call `new` on an abstract class; its purpose is to be extended.

Why would you want that? To share real code and state between related classes while forcing each to supply the part that differs (the **template method** idea). `Shape` below holds the colour and the drawing routine, but leaves `area()` to the subclasses:

Shapes.java JAVA

```
public class Shapes {
    interface Drawable {
        void draw();
        default String label() { return "Drawable:" + name(); }
        private String name() { return getClass().getSimpleName(); }
        static Drawable of(String text) { return () -> System.out.println("text " + text); }
    }

    static abstract class Shape implements Drawable {
        private final String color;
        Shape(String color) { this.color = color; }       // constructors are allowed
        abstract double area();                           // subclasses must supply this
        public void draw() {
            System.out.println(color + " " + getClass().getSimpleName()
                    + " area=" + String.format("%.2f", area()));
        }
    }
    static class Circle extends Shape {
        private final double r;
        Circle(String c, double r) { super(c); this.r = r; }
        double area() { return Math.PI * r * r; }
    }
    static class Rect extends Shape {
```

Shapes.java (continued) JAVA

```java
        private final double w, h;
        Rect(String c, double w, double h) { super(c); this.w = w; this.h = h; }
        double area() { return w * h; }
    }

    public static void main(String[] args) {
        Shape[] shapes = { new Circle("red", 1.5), new Rect("blue", 2, 3) };
        for (Shape s : shapes) {
            s.draw();
            System.out.println(s.label());
        }
        Drawable.of("hello").draw();
        // new Shape("x");   // compile error: Shape is abstract; cannot be instantiated
    }
}
```

Run it TERMINAL

```
$ java Shapes.java
red Circle area=7.07
Drawable:Circle
blue Rect area=6.00
Drawable:Rect
text hello
```

What to notice. `Shape` is abstract but has a constructor, a `private final` field and a concrete method: abstract classes are real classes with state. The loop is polymorphism in action: `s.draw()` runs `Shape.draw`, which calls `area()`, which is dispatched to the circle's or rectangle's version. An abstract class may have no abstract methods at all (just to block instantiation), and a class with even one abstract method must itself be declared abstract.

3.5.2 Interfaces

An **interface** declares a set of methods that implementing classes must provide. A class says `implements Drawable` and may implement any number of interfaces. Members of an interface are implicitly `public`; fields are implicitly `public static final` constants. An interface has no constructors and no instance state.

Think of an interface as a role. `Comparable` means "can be ordered", `Runnable` means "can be run", `AutoCloseable` means "can be closed". A `String`, an `Integer` and your own `Book` class share nothing in the hierarchy yet all can be `Comparable`. Code written against the interface works with all of them, and tests can substitute simple fake implementations.

Default, static and private methods (Java 8 and 9)

Before Java 8 an interface could not change without breaking every implementor: adding a method broke all existing classes. **Default methods** (Java 8) have a body and are inherited unless a class overrides them, which let the library designers add `List.sort` and `Collection.stream` without breaking decades of existing code. **Static methods** in interfaces (Java 8) hold factories and helpers belonging to the contract (`Drawable.of(...)`, `List.of(...)`); they are not inherited by implementing classes and must be called through the interface name. **Private methods** (Java 9) let default methods share code without exposing it, as `name()` does in `Drawable`.

Multiple inheritance of behaviour: the diamond rules

Because a class may implement several interfaces that now carry default methods, two inherited defaults can collide. The compiler applies three rules, in this order:

1. **A class wins.** A method declared in the class or inherited from a superclass beats any interface default.
2. **The most specific interface wins.** If one interface extends the other and both define the default, the sub-interface's version is used.
3. **Otherwise the class must resolve it.** If two unrelated interfaces provide the same default, the class will not compile until it overrides the method, optionally choosing one with `Interface.super.method()`.

Dia.java JAVA

```java
public class Dia {
    interface A { default String hi() { return "A"; } }
    interface B extends A { default String hi() { return "B"; } }
    interface C { default String hi() { return "C"; } }
    static class Base { public String hi() { return "Base"; } }
```

Dia.java (continued) JAVA

```java
    static class X implements A, B { }              // rule 2: B is more specific than A
    static class Y extends Base implements C { }    // rule 1: the class (Base) wins
    // static class Z implements A, C { }           // rule 3: does not compile, see below

    public static void main(String[] a) {
        System.out.println(new X().hi() + " " + new Y().hi());
    }
}
```

Run it, then uncomment class Z TERMINAL

```
$ java Dia.java
B Base
$ java Dia.java
Dia.java:9: error: types A and C are incompatible;
    static class Z implements A, C { }
           ^
  class Z inherits unrelated defaults for hi() from types A and C
```

The fix for `Z` is to override and pick explicitly: `public String hi() { return A.super.hi() + C.super.hi(); }`. This is the only case where you must write code to resolve the clash.

3.5.3 Functional interfaces: a teaser

An interface with exactly one abstract method is a **functional interface**. A lambda expression or method reference can then stand in for an implementing class. `Drawable.of` above returned `() -> System.out.println(...)` for the same reason (`Drawable` has a single abstract method, `draw`).

Fi.java JAVA

```java
import java.util.function.IntBinaryOperator;

public class Fi {
    @FunctionalInterface interface Op { int apply(int a, int b); }

    public static void main(String[] args) {
        Op add = (a, b) -> a + b;
        Op mul = (a, b) -> a * b;
        System.out.println(add.apply(3, 4) + " " + mul.apply(3, 4));
        IntBinaryOperator max = Math::max;
        System.out.println(max.applyAsInt(3, 9));
    }
}
```

Run it TERMINAL

```
$ java Fi.java
7 12
9
```

`@FunctionalInterface` is optional but makes the compiler check the single-method rule. Part 7 covers lambdas, the standard interfaces in `java.util.function` and streams in depth.

3.5.4 Marker interfaces

An interface with no methods at all is a **marker interface**; it just tags a class. `Serializable` says "this class may be converted to bytes", `Cloneable` changes how `Object.clone` behaves, `RandomAccess` tells algorithms that indexed access is fast. They are checked with `instanceof`. Annotations are the modern way to tag things, but the marker interface has one real advantage: it is a *type*, so a method can demand it in its signature.

3.5.5 Abstract class or interface?

Aspect	Abstract class	Interface
Instance state (fields)	Yes	No (constants only)
Constructors	Yes	No
How many can a class use	One (`extends`)	Any number (`implements`)
Method bodies	Any visibility, any kind	`default`, `static`, `private` only
Method access	Any	Public (abstract/default), private helpers
Models	A family with shared code ("is-a-kind-of")	A capability ("can-do"), usable by unrelated classes

Abstract class versus interface.

Practical guidance: **start with an interface** for the contract. Add an abstract class if several implementations genuinely share state or logic, and have it implement the interface. The JDK does this repeatedly: `List` (interface) with `AbstractList` (skeleton class) with `ArrayList` (concrete). Callers depend on `List`, never on `ArrayList`, which is how you can swap implementations later. Choose an abstract class when you need a constructor, protected hooks, or non-public state; choose an interface when unrelated classes may need to play the role, which is almost always the more flexible choice.

REAL WORLD
Spring and most backend code is written against interfaces. A `UserRepository` interface with a JPA implementation in production and an in-memory fake in tests; a `PaymentGateway` interface with one implementation per provider. The container wires the implementation in, and callers never name the concrete class. Spring Data even generates the implementation of your repository interface at start-up.

TIP
Name interfaces by role (`Comparable`, `Closeable`, `PaymentGateway`), not with an `I` prefix. Java style does not use `IPaymentGateway`. If an implementation has no better name, `Default...` or `...Impl` is common.

INTERVIEW ANGLE
Can an interface have a constructor or instance variables? What happens when two interfaces provide the same default method? No constructor and no instance state; fields are `public static final`. If a class inherits two unrelated defaults with the same signature, it does not compile until it overrides the method (it can delegate with `A.super.m()`). A class method always wins over an interface default, and a more specific sub-interface wins over its parent.

INTERVIEW ANGLE
Abstract class versus interface: which would you choose, and why? Interface by default, because a class can implement many and the contract stays free of implementation. Abstract class when implementations share state or code, needing constructors or protected helpers. Often both: interface plus an abstract skeleton.

Check yourself

1. Can an abstract class have a constructor? If you cannot call `new` on it, what is it for?
2. What does `new X().hi()` return if `X implements A, B`, with `B extends A` and both defining `default String hi()`?
3. This does not compile; what is wrong? `interface P { default void go() {} } interface Q { default void go() {} } class R implements P, Q { }`
4. Are interface fields modifiable? What are `int MAX = 5;` modifiers inside an interface?
5. Is `Runnable r = () -> System.out.println("x");` legal? Why?
6. Give one reason to prefer an interface over an abstract class in a public API.

Answers

1. Yes. A subclass constructor calls it with `super(...)` to initialise the shared state. The class exists to be extended.
2. `B`'s version: the most specific interface wins.
3. `R` inherits two unrelated defaults for `go()`. It must override `go()`, for example calling `P.super.go()`.
4. No. They are implicitly `public static final`, so `MAX` is a constant.
5. Yes: `Runnable` has one abstract method (`run`), so a lambda can implement it.

6. A class can implement many interfaces but extend only one class, and interfaces keep callers independent of any implementation (and make test doubles easy).

KEY POINTS

An abstract class is a partial class with state and constructors; an interface is a contract that unrelated classes can implement.

- Interfaces may carry `default`, `static` and `private` methods; they carry constants but no instance state.
- Diamond rules: class wins, then the most specific interface, otherwise you must override.
- A functional interface has one abstract method and can be implemented by a lambda.
- Design to the interface; add an abstract skeleton class only when implementations share code.

3.6 The Object Class

Every class in Java, whether you wrote `extends` or not, descends from `java.lang.Object`. It is the single root of the hierarchy, so a variable of type `Object` can hold any reference, and every object responds to a small set of methods: `equals`, `hashCode`, `toString`, `getClass`, `clone`, the wait/notify family and the legacy `finalize`. Collections, logging, debuggers and frameworks all rely on these. Overriding them correctly is among the most tested topics in Java interviews and among the most common sources of subtle bugs, so this chapter spends most of its time on `equals` and `hashCode`.

3.6.1 toString

`toString()` returns a text representation of the object. The default implementation returns the class name, an `@`, and the hash code in hexadecimal, such as `java.lang.Object@659e0bfd` (typical output; the number varies between runs). It is useless to humans. `System.out.println(obj)`, string concatenation, loggers and debuggers all call `toString`, so a readable override is the cheapest debugging aid there is:

JAVA

```
@Override
public String toString() {
    return "Point[x=" + x + ", y=" + y + "]";
}
```

Include the fields that identify the object, never secrets (passwords, tokens, card numbers), because `toString` ends up in log files. Records and IDE generators produce this for you.

3.6.2 getClass

`getClass()` returns the `Class` object describing the runtime class of an object; it is `final` and cannot be overridden. `obj.getClass() == Point.class` is true only for *exactly* that class, whereas `obj instanceof Point` is also true for subclasses. `getClass().getName()` gives `java.lang.String`, while `getSimpleName()` gives `String`; arrays have names like `[I` for `int[]`. The distinction between these two tests matters when you write `equals`, as the next section shows.

3.6.3 equals and hashCode

What == means and what equals means

For references, `==` asks "are these the same object?" (identity). `equals` asks "do these represent the same value?" (equality). `Object.equals` is just `==`, so a class that wants value equality must override it. `String`, `Integer`, `List`, `LocalDate` and all records do.

The equals contract

The Javadoc requires `equals` to be an **equivalence relation**, and to be consistent:

- **Reflexive:** `x.equals(x)` is true.
- **Symmetric:** `x.equals(y)` is true exactly when `y.equals(x)` is.
- **Transitive:** if `x.equals(y)` and `y.equals(z)`, then `x.equals(z)`.
- **Consistent:** repeated calls give the same answer while nothing changed.
- **Non-null:** `x.equals(null)` is false, never an exception.

The hashCode contract

`hashCode()` returns an `int` used by hash-based collections (`HashMap`, `HashSet`) to choose a bucket. The rule that ties it to `equals`:

> **DEFINITION**
> hashCode contract If `a.equals(b)` is true then `a.hashCode() == b.hashCode()` must be true. The reverse need not hold (different objects may share a hash code, a collision), and the value must stay the same for the same object while the fields used by `equals` do not change.

Equal objects must produce equal hash codes, otherwise a hash collection looks in the wrong bucket and will not find an object that is "equal" to the one it holds.

Breaking the contract: a demonstration

This class overrides `equals` but forgets `hashCode`. A second class does both correctly, and a third shows what happens to a mutable key:

Equality.java JAVA

```java
import java.util.HashMap;
import java.util.HashSet;
import java.util.Map;
import java.util.Objects;
import java.util.Set;

public class Equality {
    static class BadPoint {
        final int x, y;
        BadPoint(int x, int y) { this.x = x; this.y = y; }
        @Override public boolean equals(Object o) {
            return o instanceof BadPoint p && p.x == x && p.y == y;
        }
        // hashCode NOT overridden: breaks the contract
    }
    static class Point {
        final int x, y;
        Point(int x, int y) { this.x = x; this.y = y; }
        @Override public boolean equals(Object o) {
            if (this == o) return true;
            if (o == null || getClass() != o.getClass()) return false;
            Point p = (Point) o;
            return x == p.x && y == p.y;
        }
        @Override public int hashCode() { return Objects.hash(x, y); }
        @Override public String toString() { return "Point[" + x + "," + y + "]"; }
    }
    static class MutableKey {
        int id;
        MutableKey(int id) { this.id = id; }
        @Override public boolean equals(Object o) {
            return o instanceof MutableKey k && k.id == id;
        }
        @Override public int hashCode() { return Integer.hashCode(id); }
    }

    public static void main(String[] args) {
        BadPoint p1 = new BadPoint(1, 2), p2 = new BadPoint(1, 2);
        System.out.println("bad equals: " + p1.equals(p2));
        Set<BadPoint> bad = new HashSet<>();
        bad.add(p1);
        System.out.println("bad contains equal point: " + bad.contains(p2));
        Set<Point> good = new HashSet<>();
        good.add(new Point(1, 2));
        System.out.println("good contains equal point: " + good.contains(new Point(1, 2)));
        System.out.println(new Point(3, 4));
        Map<MutableKey, String> m = new HashMap<>();
        MutableKey k = new MutableKey(1);
        m.put(k, "one");
        k.id = 2;
        System.out.println("after mutation: " + m.get(k) + " / "
                + m.get(new MutableKey(1)) + " / size " + m.size());
    }
}
```

Run it (the "bad contains" line is false in practice but depends on identity hash codes) TERMINAL

```
$ java Equality.java
bad equals: true
bad contains equal point: false
good contains equal point: true
Point[3,4]
after mutation: null / null / size 1
```

What to notice.

- `BadPoint` objects are `equals`, but the default `hashCode` is derived from object identity, so `p1` and `p2` almost certainly land in different buckets, and `contains` answers false. The set silently misbehaves; no exception tells you.
- `Point` follows the recipe: the same fields feed both `equals` and `hashCode`. `Objects.hash(x, y)` is a convenient way to combine them (it allocates an array; for hot paths, compute `31 * x + y` by hand or use a record, which generates both for you).
- `MutableKey` shows the third rule. After `k.id = 2` the entry sits in the bucket for hash `1`, but a lookup with `k` now searches the bucket for hash `2`, and a lookup with a fresh key `1` finds the entry but `equals` fails against the mutated key. The entry is now unreachable yet still counted in `size`: a memory leak. **Never mutate a field that participates in `hashCode` while the object is a key or set member.** Prefer immutable keys.

equals: getClass or instanceof?

`Point.equals` above compared `getClass() != o.getClass()`, which means a `Point` never equals an instance of a subclass. Using `instanceof` instead lets a subclass instance equal a parent, but then symmetry breaks as soon as a subclass adds a field and its own `equals` considers it (the famous `ColorPoint` problem: `point.equals(colorPoint)` is true but the reverse is false). There is no way to extend an instantiable class, add a value component, and keep the `equals` contract. For a final class, `instanceof` is the clean choice, and for a non-final class, `getClass()` preserves the contract. Prefer composition for value types, or make the value class `final` or a record.

Recipe for equals and hashCode

1. Use the same fields in both methods, and only those fields.
2. In `equals`: check `this == o`, then null and class, then cast and compare. Compare primitives with `==` (`Double.compare` for `double`/`float`, to handle `NaN`), and objects with `Objects.equals(a, b)`, which is null-safe.
3. In `hashCode`: combine the same fields with `Objects.hash(...)` or the `31 * h + field` idiom.
4. Always pair them, always add `@Override` and always use the parameter type `Object`.
5. Better still, use a record (`record Point(int x, int y) {}`) or let the IDE generate it, and test it with a small unit test that puts equal objects into a `HashSet`.

REAL WORLD
JPA entities are a famous trap. An entity whose `hashCode` uses a database-generated `id` changes hash code when it is first saved (null before, a number after), which breaks any `HashSet` that held it. Common guidance is to base `equals`/`hashCode` on a stable business key or return a constant `getClass().hashCode()`. Lombok's `@EqualsAndHashCode` has the same issue if applied naively to entities.

3.6.4 clone, and why to avoid it

`Object.clone()` is `protected` and makes a shallow copy of an object: a new object whose fields hold the same values (the same *references*). It works only if the class implements the marker interface `Cloneable`, otherwise it throws `CloneNotSupportedException`. It creates objects without calling a constructor, interacts badly with `final` fields, and a subclass that forgets to override it correctly breaks the copy. Bloch's verdict in *Effective Java* is blunt: prefer a copy constructor or copy factory. The program shows the central problem, shared mutable state:

Cloning.java JAVA

```java
import java.util.Arrays;

public class Cloning {
    static class Team implements Cloneable {
        String name;
        int[] scores;
        Team(String n, int[] s) { name = n; scores = s; }
        @Override public Team clone() {
            try { return (Team) super.clone(); }          // shallow copy
```

Cloning.java (continued) JAVA

```java
            catch (CloneNotSupportedException e) { throw new AssertionError(e); }
        }
    }
    static class Copyable {
        final String name;
        final int[] scores;
        Copyable(String n, int[] s) { name = n; scores = s.clone(); }
        Copyable(Copyable o) { this(o.name, o.scores); }   // copy constructor: explicit, deep
    }

    public static void main(String[] args) {
        Team a = new Team("A", new int[] {1, 2, 3});
        Team b = a.clone();
        b.scores[0] = 99;
        System.out.println(Arrays.toString(a.scores) + " shared=" + (a.scores == b.scores));
        Copyable c = new Copyable("C", new int[] {1, 2, 3});
        Copyable d = new Copyable(c);
        d.scores[0] = 99;
        System.out.println(Arrays.toString(c.scores));
    }
}
```

Run it TERMINAL

```
$ java Cloning.java
[99, 2, 3] shared=true
[1, 2, 3]
```

The "copy" `b` shares the array with `a`, so changing one changed the other. A correct `clone` must also clone the array (`copy.scores = scores.clone()`), which is why the copy constructor is simpler. Arrays are the one place where `clone()` is routine: `int[] copy = original.clone()` is fine and idiomatic (shallow for arrays of objects).

3.6.5 finalize is on its way out

`Object.finalize()` was meant to release resources just before garbage collection. It is unpredictable (it may run late or never), slows collection, can resurrect objects and has security problems. It has been deprecated for removal since Java 18 (JEP 421), and the compiler now warns when you override it (`warning: [removal] finalize() in Object has been deprecated and marked for removal`). Release resources deterministically with `try`-with-resources and `AutoCloseable` (Part 5); use `java.lang.ref.Cleaner` for the rare native-handle safety net.

3.6.6 wait, notify and the rest

`wait()`, `notify()` and `notifyAll()` are thread-coordination primitives on every object, tied to its monitor lock; Part 8 explains them. `hashCode()` for the default identity version is not the memory address, but a value generated by the JVM and stored in the object header the first time it is requested.

3.6.7 compareTo and consistency with equals

`Comparable.compareTo` defines a natural ordering, used by `Collections.sort`, `TreeSet` and `TreeMap`. It is not part of `Object`, but it must be **consistent with equals**: `a.compareTo(b) == 0` should imply `a.equals(b)`. The JDK itself violates this in one well-known place, which makes a revealing demo:

Cmp.java JAVA

```java
import java.math.BigDecimal;
import java.util.HashSet;
import java.util.Set;
import java.util.TreeSet;

public class Cmp {
    public static void main(String[] args) {
        BigDecimal a = new BigDecimal("1.0"), b = new BigDecimal("1.00");
        System.out.println(a.equals(b) + " " + (a.compareTo(b) == 0));
        Set<BigDecimal> h = new HashSet<>(Set.of(a, b));
        Set<BigDecimal> t = new TreeSet<>(Set.of(a, b));
        System.out.println("HashSet size " + h.size() + ", TreeSet size " + t.size());
    }
}
```

Run it TERMINAL

```
$ java Cmp.java
false true
HashSet size 2, TreeSet size 1
```

`BigDecimal.equals` considers scale (`1.0` is not `1.00`), `compareTo` does not. So a `HashSet` (uses `equals`) holds two elements while a `TreeSet` (uses `compareTo`) holds one. If you write your own `compareTo`, keep it consistent with `equals`, and if you cannot, document it as `BigDecimal` does. Never implement `compareTo` by subtracting integers (`return a - b`): overflow gives wrong signs; use `Integer.compare(a, b)`.

INTERVIEW ANGLE

What is the contract between equals() and hashCode()? What happens if you override only one? Equal objects must have equal hash codes. If you override `equals` only, hash collections cannot find equal objects (they look in the wrong bucket); if only `hashCode`, `equals` still compares identity, so lookups find the right bucket but fail the equality check. Both must use the same fields, and keys must not change while stored.

INTERVIEW ANGLE

Why is `clone()` considered broken? What do you use instead? It is a shallow copy by default, bypasses constructors, needs a marker interface that has no methods, and clashes with `final` fields. Use a copy constructor, a copy factory (`List.copyOf`) or an immutable design; `array.clone()` is the accepted exception.

Check yourself

1. What does `System.out.println(new Object())` print, and which method produces it?
2. A class overrides `equals(Object)` but not `hashCode()`. Name the concrete symptom in a `HashSet`.
3. What does this print? `BigDecimal a = new BigDecimal("2.0"), b = new BigDecimal("2.00"); System.out.println(a.equals(b) + " " + a.compareTo(b));`
4. Why must you not mutate a field used in `hashCode` after putting the object in a `HashMap` as a key?
5. `class Pt { int x; public boolean equals(Pt p) { return p != null && x == p.x; } }` is used as `Set<Pt>`. What is wrong, and what one annotation catches it?
6. Why is `int[] b = a.clone();` acceptable while cloning ordinary objects is discouraged?

Answers

1. Something like `java.lang.Object@659e0bfd` from `Object.toString`: class name, `@`, hex identity hash (varies per run).
2. Equal objects usually land in different buckets, so `contains`, `remove` and `get` fail to find them and duplicates can be added.
3. `false 0`: `equals` compares scale as well as value, `compareTo` compares only numeric value.
4. The entry stays in the bucket for the old hash; lookups now use the new hash and miss it. The entry is lost but still occupies memory.
5. It overloads `equals(Pt)` instead of overriding `equals(Object)`, so the set uses identity equality. `@Override` makes the compiler reject it.
6. Arrays have a well-defined, compiler-supported `clone` returning the right type; the problems of `Cloneable` (constructors, final fields, deep copy) do not apply to a flat array of primitives.

KEY POINTS

All classes extend `Object`; override `toString` for readable logs, and never put secrets in it.

- `equals` must be reflexive, symmetric, transitive, consistent and null-safe; `hashCode` must agree with it (equal objects, equal hashes).
- Override both together, with the same fields, `@Override`, and the parameter type `Object`. Records do this for you.
- Do not mutate hash-relevant fields of objects used as keys; prefer immutable keys.
- Avoid `clone` (use copy constructors) and `finalize` (deprecated for removal); keep `compareTo` consistent with `equals`.

3.7 Nested, Inner, Local and Anonymous Classes

Most classes live in their own file at the top level. But sometimes a class exists only to serve one other class: a node inside a linked list, an iterator that walks one collection, a small callback used in exactly one place. Java lets you declare a class *inside* another class so that the helper stays next to the code that needs it, and so that it can reach private members it would otherwise have no access to. This chapter covers the four flavours, the hidden machinery that makes them work, and the one famous trap (the memory leak through the hidden outer reference).

3.7.1 The four kinds at a glance

Everything declared inside another type is a **nested class**. There are four kinds, and the difference that matters most is whether an instance carries a hidden reference to an instance of the enclosing class.

Kind	Declared	Has outer instance?	Typical use
`static nested`	`static class N` as a member	No	Builder, Node, Entry, helper types
`inner (member)`	`class I` as a non-static member	Yes (`Outer.this`)	Iterators, event handlers tied to one object
`local`	inside a method or block	Yes if the method is non-static	One-off helper, local record/enum/interface
`anonymous`	`new Type() { ... }` expression	Yes if created in non-static code	Single-use implementation of an interface or abstract class

The four kinds of nested class.

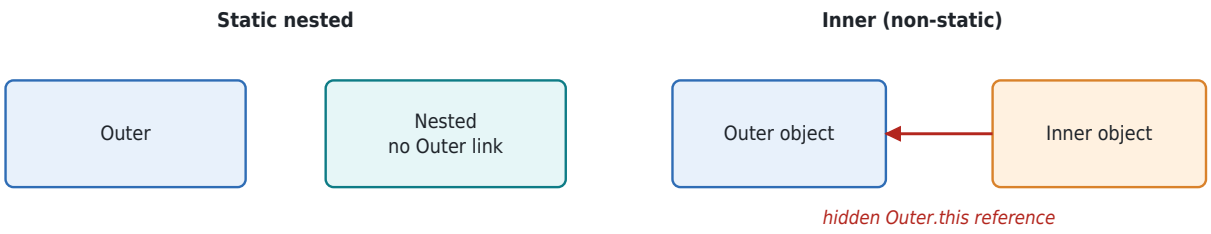


Figure 3.3. Static nested classes stand alone; inner classes are tied to one outer instance through a hidden reference.

The next listing shows a static nested class, an inner class and an anonymous class together, and prints the names the compiler gives them.

Counter.java JAVA

```java
import java.util.function.IntSupplier;

public class Counter {
    private int hits = 0;

    class Ticket {                      // inner class: needs a Counter
        final int number;
        Ticket() { number = ++hits; }    // reads and writes Counter.this.hits
        Counter owner() { return Counter.this; }
    }

    static class Config {               // static nested: no outer instance
        final String name;
        Config(String name) { this.name = name; }
    }

    public static void main(String[] args) {
        Counter c = new Counter();
        Counter.Ticket a = c.new Ticket();
        Counter.Ticket b = c.new Ticket();
        System.out.println(a.number + " " + b.number + " hits=" + c.hits);
        System.out.println(a.owner() == c);
```

Counter.java (continued) JAVA

```
        Counter.Config cfg = new Counter.Config("prod");
        System.out.println(cfg.name);

        int base = 100;                  // effectively final
        IntSupplier s = new IntSupplier() {        // anonymous class
            @Override public int getAsInt() { return base + 1; }
        };
        System.out.println(s.getAsInt());
        System.out.println(s.getClass().getName());
        System.out.println(a.getClass().getName());
        System.out.println(cfg.getClass().getName());
    }
}
```

Output TERMINAL

```
$ java Counter.java
1 2 hits=2
true
prod
101
Counter$1
Counter$Ticket
Counter$Config
```

What to notice

- `c.new Ticket()` is the syntax for creating an inner object: you must supply the outer instance. Inside `Counter`'s own instance methods you can write plain `new Ticket()` and `this` is used.
- `Ticket` updates the private field `hits` of `Counter` directly. Nested classes and their outer class share private access; that is a main reason to nest.
- `Counter.this` names the enclosing instance. If `Ticket` had its own `hits`, `hits` would mean the inner one and `Counter.this.hits` the outer one (shadowing).
- At runtime there is no nesting at all. The compiler produces separate class files named `Counter$Ticket`, `Counter$Config` and `Counter$1` (anonymous classes are numbered).

3.7.2 Static nested versus inner: the hidden reference

Compile `Counter.java` with `javac` and inspect the inner class with `javap -p`. The compiler has added a field and a constructor parameter that you never wrote.

Inside an inner class TERMINAL

```
$ javac -d t Counter.java
$ javap -p -cp t 'Counter$Ticket'
Compiled from "Counter.java"
class Counter$Ticket {
  final int number;
  final Counter this$0;
  Counter$Ticket(Counter);
  Counter owner();
}
```

`this$0` is the hidden reference to the outer object, and the constructor takes the outer instance as its first parameter. A static nested class gets neither. This is why a static nested class is the right default: it costs nothing extra and cannot accidentally keep an outer object alive. **Make a nested class `static` unless it genuinely needs the outer instance.** IDEs will even suggest it.

> **NOTE**
> Since JDK 18 the compiler leaves `this$0` out of an inner class when the class never uses the outer instance (JDK-8271623). So on a modern JDK the field exists only if the inner or anonymous class actually touches the outer object, or if the class is serializable. The leak below therefore depends on how you write the class, which is exactly what makes it hard to notice.

3.7.3 Local classes and capturing variables

A **local class** is declared inside a method, constructor or block. It is visible only there. It can read local variables and parameters of the enclosing method, but only if they are **final or effectively final**: assigned once and never changed. Since Java 16 a local class can also be a record, an enum or an interface (all implicitly static, so they cannot capture locals).

Local.java JAVA

```java
import java.util.ArrayList;
import java.util.List;

public class Local {
    interface Greeter { String greet(); }

    static List<Greeter> make() {
        List<Greeter> list = new ArrayList<>();
        for (int i = 0; i < 3; i++) {
            final int n = i;                // copy captured by value
            list.add(() -> "hello " + n);
        }
        return list;
    }

    static int sum(int[] xs) {
        record Pair(int a, int b) {}        // local record
        int total = 0;
        for (int i = 0; i + 1 < xs.length; i++) {
            Pair p = new Pair(xs[i], xs[i + 1]);
            total += p.a() * p.b();
        }
        return total;
    }

    public static void main(String[] args) {
        for (Greeter g : make()) System.out.println(g.greet());
        System.out.println(sum(new int[]{1, 2, 3, 4}));
        class Local1 { int v = 7; }         // local class
        System.out.println(new Local1().v + " " + Local1.class.getName());
    }
}
```

Output TERMINAL

```
$ java Local.java
hello 0
hello 1
hello 2
20
7 Local$1Local1
```

Why must captured variables be effectively final? A local variable lives on the method's stack frame and dies when the method returns, but the object of the local or anonymous class may live much longer. So the compiler does not share the variable; it **copies the value** into a synthetic field of the new object. If the original could change afterwards, the copy would silently disagree with it, and if the copy could change, the method would not see it. Java resolves the confusion by forbidding both: the value is fixed, so copying is invisible. (Languages such as JavaScript capture the variable itself, which is a different and more confusing model.) The loop above copies `i` into `n` for exactly this reason: `i` changes on every iteration, so it cannot be captured.

GOTCHA

Capturing a mutable *object* is fine; only the variable's reference must not change. `final List<String> log = new ArrayList<>();` can be captured and `log.add(...)` still works. The effectively-final rule protects the variable, not the object it points to. A common workaround, a one-element array `int[] count = {0};`, compiles but is shared mutable state: do not use it with threads, and prefer an `AtomicInteger` or restructure the code.

3.7.4 Anonymous classes

An **anonymous class** declares and instantiates a class in one expression. It has no name, so it is used once, typically to implement an interface or extend a class with a small override:

```java
Comparator<String> byLength = new Comparator<>() {      // diamond allowed since Java 9
    @Override public int compare(String a, String b) {
        return Integer.compare(a.length(), b.length());
    }
};
Thread t = new Thread() {                               // subclass with an override
    @Override public void run() { System.out.println("hi"); }
};
```

Restrictions: no constructor (it has no name; arguments in `new Base(args) { ... }` go to the superclass constructor), it can extend a class **or** implement one interface, not both, and it can declare extra fields and methods but the outside world can call them only through `var`:

```java
var o = new Object() { int x = 5; int twice() { return x * 2; } };
System.out.println(o.twice());       // works: var keeps the anonymous type
```

Anonymous class or lambda?

Since Java 8, a lambda replaces an anonymous class when the type is a **functional interface** (one abstract method). Lambdas are shorter and, as the next section shows, do not carry a hidden outer reference unless they use `this` or an outer member. They are not identical:

Aspect	Anonymous class	Lambda
`this` means	the anonymous object itself	the enclosing instance
Can implement	any interface or abstract/concrete class	functional interfaces only
Can have fields and several methods	yes	no
Class file	a real `Outer$1.class`	none; created at runtime via `invokedynamic`
Name shadowing	its members can shadow outer variables	cannot redeclare an enclosing local name

Anonymous class versus lambda.

Use an anonymous class when you need more than one method, state, or a superclass that is not an interface. Otherwise use a lambda. The mechanism behind lambdas is explained in the chapter on lambdas in the functional Java Part.

3.7.5 The memory leak through the hidden outer reference

An inner or anonymous class created in an instance method may hold `this$0`, which keeps the **entire outer object** reachable for as long as the inner object lives. If the outer object is big and the inner object is long-lived (queued task, registered listener, cached callback), you have a leak. The program below creates tasks that each belong to a `Report` holding a 10 MB array, and keeps only the tasks.

Leak.java JAVA

```java
import java.util.ArrayList;
import java.util.List;

public class Leak {
    static class Report {
        private final byte[] big = new byte[10_000_000];          // 10 MB

        Runnable usesOuter() {                                    // anonymous, reads Report.this
            return new Runnable() {
                public void run() { System.out.println(big.length); }
            };
        }
        Runnable ignoresOuter() {                                 // anonymous, never touches it
            return new Runnable() {
                public void run() { System.out.println("task"); }
```

Leak.java (continued) JAVA

```
            };
        }
    }

    static void fields(Class<?> c) {
        var fs = c.getDeclaredFields();
        for (var f : fs) System.out.println(c.getName() + " has field " + f.getName());
        if (fs.length == 0) System.out.println(c.getName() + " has no fields");
    }

    public static void main(String[] args) throws Exception {
        Report r = new Report();
        fields(r.usesOuter().getClass());
        fields(r.ignoresOuter().getClass());
        List<Runnable> keep = new ArrayList<>();
        for (int i = 0; i < 20; i++) {
            keep.add(new Report().usesOuter());     // each task keeps its 10 MB Report alive
            System.out.println("kept " + keep.size());
        }
    }
}
```

Run with a small heap (output trimmed) TERMINAL

```
$ java -Xmx100m Leak.java
Leak$Report$1 has field this$0
Leak$Report$2 has no fields
kept 1
kept 2
...
kept 8
Exception in thread "main" java.lang.OutOfMemoryError: Java heap space
        at Leak$Report.<init>(Leak.java:6)
        at Leak.main(Leak.java:31)
```

The first anonymous class uses `big`, so the compiler keeps `this$0` and each task pins 10 MB; the heap is exhausted after eight. The second one does not use the outer object, so on JDK 18 and later it carries nothing (on JDK 17 and earlier it would have carried `this$0` anyway). The same trap exists for inner `Iterator`s, listeners, and for the older Android and Swing idioms.

How to avoid it

- Default to `static` nested classes and pass in only what the helper needs.
- Prefer a lambda that uses no instance member, or a static factory method that returns the lambda.
- Do not store inner-class instances (listeners, callbacks) in static fields or long-lived collections without a way to remove them.
- When you suspect it, take a heap dump and look for `this$0` in the path to GC roots; the diagnostics Part shows how.

INTERVIEW ANGLE

What is the difference between a static nested class and an inner class? A static nested class is just a class that lives in another class's namespace; you create it with `new Outer.Nested()` and it has no outer instance. An inner class is non-static, needs an enclosing instance (`outer.new Inner()`), can use `Outer.this` and the outer's private members, and (when it uses them) holds a hidden `this$0` reference that can cause leaks. Prefer static nested unless you need the outer instance.

REAL WORLD

In frameworks you meet nested classes constantly: `Map.Entry` is a nested interface, `ArrayList` has a private inner `Itr` iterator, and Spring or Lombok builders are static nested `Builder` classes. Anonymous classes used to be the way to write callbacks (`new Callback() { ... }`); in modern code they are replaced by lambdas except when state or a superclass is needed.

Check yourself

1. Which of the four nested kinds never has an outer-instance reference?
2. What is wrong with this code, and how do you fix it? `int n = 0; Runnable r = new Runnable() { public void run() { n++; } };`
3. What does `Outer.this` mean inside an inner class, and how do you create an inner object from `main`?
4. Why is a class file named `Outer$1.class` produced for an anonymous class?
5. Why can a long-lived anonymous object cause an `OutOfMemoryError` even if you no longer use the outer object?
6. What does `this` refer to inside a lambda versus inside an anonymous class body?

Answers

1. The static nested class. (Local and anonymous classes created in a static method also have none.)
2. `n` is modified inside the class, so it is not effectively final and the code does not compile. Use an `AtomicInteger`, make the counter a field of an object, or restructure so the value is returned.
3. It is the enclosing instance of the inner object. From `main` you need an outer object: `Outer o = new Outer(); Outer.Inner i = o.new Inner();`.
4. The JVM has no concept of nested classes; the compiler flattens each one into its own class file, naming anonymous classes `Outer$1`, `Outer$2` and so on.
5. The inner object holds `this$0`, so everything reachable from the outer object stays reachable (here the 10 MB array) as long as the inner object is referenced.
6. In a lambda, `this` is the enclosing instance. In an anonymous class, `this` is the anonymous object itself.

KEY POINTS

- Nest a class to keep a helper next to its only user and to share private access. Default to `static` nested.
- An inner class holds a hidden outer reference (`this$0`); the compiler passes it as an extra constructor argument.
- Local and anonymous classes capture local variables by copying their value, so those variables must be effectively final.
- Anonymous classes are numbered class files (`Outer$1`); use a lambda instead when the target is a functional interface.
- An inner object that outlives its outer object and uses it keeps the whole outer graph alive: a classic leak.

3.8 Enums

Many values come from a small, fixed set: days of the week, order states, card suits, HTTP methods. Before Java 5 programmers used `static final int` constants (`int PENDING = 0; int SHIPPED = 1;`), which have no type safety (you can pass `42`), no readable printing, and no namespace. A Java `enum` solves this: it is a class with a fixed, closed set of instances, each one a named constant. The compiler checks the type, you can print the name, and the constants can carry data and behaviour.

3.8.1 Constants are objects

The simplest enum lists its constants. Each constant is a `public static final` instance of the enum class, created once when the class is initialised, so you may compare constants with `==`.

JAVA

```java
enum OrderStatus { PENDING, PAID, SHIPPED, CANCELLED }

OrderStatus s = OrderStatus.PAID;
System.out.println(s);                    // PAID  (toString defaults to name())
System.out.println(s == OrderStatus.PAID); // true: one instance per constant
```

Every enum implicitly extends `java.lang.Enum<E>`, which supplies these methods:

Method	What it does
`values()`	A fresh array of all constants in declaration order (added by the compiler).
`valueOf(String)`	Constant with that exact name; throws `IllegalArgumentException` if none (case-sensitive).

Method	What it does
`name()`	The constant's identifier as written. `final`; use it for stable storage.
`ordinal()`	Zero-based position in the declaration. Fragile: reordering changes it.
`compareTo(E)`	Compares ordinals, so constants sort in declaration order.
`getDeclaringClass()`	The enum type, even for constants with bodies.

Methods every enum has.

WATCH OUT

Do not persist `ordinal()` (in a database column or a file). Inserting a new constant in the middle silently changes the meaning of every stored number. Store `name()` or an explicit code field instead. In JPA/Hibernate prefer `@Enumerated(EnumType.STRING)`, not the default `ORDINAL`.

3.8.2 Fields, constructors and methods

An enum can have fields, a constructor and methods. The constructor is implicitly `private`; it runs once per constant, with the arguments you write after the constant name.

Planets.java JAVA

```java
import java.util.Arrays;

public class Planets {
    enum Level {
        LOW(1), MEDIUM(5), HIGH(10);

        private final int weight;
        Level(int weight) { this.weight = weight; }     // implicitly private
        int weight() { return weight; }
        boolean atLeast(Level other) { return compareTo(other) >= 0; }
    }

    enum Op {
        ADD("+") { int apply(int a, int b) { return a + b; } },
        MUL("*") { int apply(int a, int b) { return a * b; } };

        final String symbol;
        Op(String symbol) { this.symbol = symbol; }
        abstract int apply(int a, int b);
    }

    public static void main(String[] args) {
        System.out.println(Arrays.toString(Level.values()));
        System.out.println(Level.valueOf("HIGH").weight());
        System.out.println(Level.HIGH.ordinal() + " " + Level.HIGH.name());
        System.out.println(Level.HIGH.atLeast(Level.MEDIUM));
        for (Op op : Op.values())
            System.out.println(op + " " + op.symbol + " " + op.apply(6, 7)
                + " " + op.getClass() + " " + op.getDeclaringClass());
        try { Level.valueOf("high"); }
        catch (IllegalArgumentException e) { System.out.println(e.getMessage()); }
        System.out.println(Level.LOW == Level.valueOf("LOW"));
    }
}
```

Output TERMINAL

```
$ java Planets.java
[LOW, MEDIUM, HIGH]
10
2 HIGH
true
ADD + 13 class Planets$Op$1 class Planets$Op
MUL * 42 class Planets$Op$2 class Planets$Op
No enum constant Planets.Level.high
true
```

What to notice

- `Op` declares an **abstract method** and gives each constant its own body. Each such constant is an anonymous subclass (`Planets$Op$1`), which is why `getClass()` differs from `getDeclaringClass()`. This is the strategy pattern with a fixed set of strategies and no `if` chains.
- A field of an enum should be `final`. A mutable field turns the constants into shared global state.
- The enum cannot be extended (`Level` is `final`; constants with bodies make it implicitly sealed), and it cannot be instantiated with `new`.

What the compiler generates

`javap -p` on `Planets$Level` shows the machinery: a `final` class extending `Enum`, one `public static final` field per constant, a private `$VALUES` array, and a static initializer that creates the constants.

Inspecting the compiled enum TERMINAL

```
$ javap -p -cp t 'Planets$Level'
final class Planets$Level extends java.lang.Enum<Planets$Level> {
  public static final Planets$Level LOW;
  public static final Planets$Level MEDIUM;
  public static final Planets$Level HIGH;
  private final int weight;
  private static final Planets$Level[] $VALUES;
  public static Planets$Level[] values();
  public static Planets$Level valueOf(java.lang.String);
  private Planets$Level(int);
  int weight();
  boolean atLeast(Planets$Level);
  private static Planets$Level[] $values();
  static {};
}
```

`values()` returns a **copy** of `$VALUES` each call (to protect the original), so in a hot loop call it once and keep the array, or use `EnumSet.allOf`. Enums can also implement interfaces, which is how you give a family of constants a common type without inheritance:

Cmd.java JAVA

```java
import java.util.Arrays;
import java.util.Map;
import java.util.stream.Collectors;

public class Cmd {
    interface Priced { int cents(); }
    enum Size implements Priced {
        S(100), M(150), L(200);
        private final int cents;
        Size(int c) { cents = c; }
        public int cents() { return cents; }
        static final Map<String, Size> BY_NAME =
            Arrays.stream(values()).collect(Collectors.toMap(s -> s.name().toLowerCase(), s -> s));
    }
    public static void main(String[] args) {
        System.out.println(Size.BY_NAME.get("m").cents());
        System.out.println(Size.M.compareTo(Size.L));
        System.out.println(Size.class.getSuperclass());
    }
}
```

Output TERMINAL

```
$ java Cmd.java
150
-1
class java.lang.Enum
```

The static lookup map is a standard idiom for parsing input. Note that it must be a `static` field declared after the constants, because the constants are created first.

3.8.3 Enums in switch

An enum is the ideal `switch` selector. In a switch **expression** the compiler requires every constant to be covered (or a `default`), so adding a new constant later produces a compile error at every switch that forgot it. That is a real advantage over integer codes. With a classic switch **statement** over an enum, missing constants are silently ignored unless you add a `default` that throws.

JAVA

```
Light next() {
    return switch (this) {
        case RED -> GREEN;
        case GREEN -> YELLOW;
        case YELLOW -> RED;      // no default needed: all constants covered
    };
}
```

Inside a `case` label you write the bare constant name (`RED`), not `Light.RED`. (Since Java 21, pattern-switch also lets you qualify enum constants in case labels when mixing types.)

TIP

Avoid writing a `default` branch in a switch expression over an enum you own. The missing default is what lets the compiler tell you about new constants. Add `default` only for enums you do not control.

3.8.4 EnumSet and EnumMap

Because an enum has a small known number of constants, specialised collections exist and are much faster than `HashSet` and `HashMap` for them.

- `EnumSet<E>` stores membership as bits (one `long` for up to 64 constants). Operations are a few machine instructions. Factories: `of`, `noneOf`, `allOf`, `range`, `complementOf`, `copyOf`.
- `EnumMap<K, V>` stores values in an array indexed by `ordinal()`. Iteration is in declaration order.

Sets.java JAVA

```
import java.util.EnumMap;
import java.util.EnumSet;
import java.util.Map;

public class Sets {
    enum Day { MON, TUE, WED, THU, FRI, SAT, SUN }

    enum Light {
        RED, GREEN, YELLOW;
        Light next() {
            return switch (this) {
                case RED -> GREEN;
                case GREEN -> YELLOW;
                case YELLOW -> RED;
            };
        }
    }

    public static void main(String[] args) {
        EnumSet<Day> weekend = EnumSet.of(Day.SAT, Day.SUN);
        EnumSet<Day> work = EnumSet.complementOf(weekend);
        System.out.println(weekend + " " + work);
        System.out.println(EnumSet.range(Day.TUE, Day.THU));
        Map<Day, Integer> hours = new EnumMap<>(Day.class);
        hours.put(Day.FRI, 6);
        hours.put(Day.MON, 8);
        System.out.println(hours);          // iterates in declaration order
        Light l = Light.RED;
        for (int i = 0; i < 4; i++) { System.out.print(l + " "); l = l.next(); }
        System.out.println();
    }
}
```

Output TERMINAL

```
$ java Sets.java
[SAT, SUN] [MON, TUE, WED, THU, FRI]
[TUE, WED, THU]
{MON=8, FRI=6}
RED GREEN YELLOW RED
```

Note that `EnumMap` printed `MON` before `FRI` although `FRI` was inserted first. `EnumMap` needs the `Class` object (`Day.class`) in its constructor because the map cannot infer the key type at runtime (generics are erased). Use an `EnumSet` instead of an `int` bit-flag field (`READ | WRITE`) whenever you need a set of options.

3.8.5 Singleton via enum

Joshua Bloch's recommendation in *Effective Java* is that a single-element enum is the best way to implement a singleton: the JVM guarantees the constant is created exactly once (class initialisation is thread-safe), and unlike a class with a private constructor, an enum cannot be duplicated by serialization or by reflection (`Constructor.newInstance` refuses to create enum instances).

Registry.java JAVA

```java
public enum Registry {
    INSTANCE;

    private int count;
    public synchronized int register() { return ++count; }

    public static void main(String[] args) {
        System.out.println(Registry.INSTANCE.register());
        System.out.println(Registry.INSTANCE.register());
        System.out.println(Registry.INSTANCE == Registry.valueOf("INSTANCE"));
    }
}
```

Output TERMINAL

```
$ java Registry.java
1
2
true
```

The limits: an enum singleton cannot extend another class (it already extends `Enum`), it is created eagerly at first use of the class, and in a Spring application you normally let the container manage singletons instead. The design patterns Part compares the other singleton forms.

GOTCHA

Enum constants are created in a `static` initializer in declaration order. A constructor or constant body that touches a `static` field of the same enum (other than a compile-time constant) sees its default value, because the field is not yet initialised. The compiler reports a direct reference as an error.

INTERVIEW ANGLE

Why can you compare enums with `==`, and why is a single-element enum a good singleton? Each constant is created once per class loader and the type has no public constructor, so reference equality equals logical equality (and `==` is null-safe and type-checked at compile time). A single-element enum is a good singleton because class initialisation is atomic and thread-safe, serialization of enums preserves identity (it writes the name and uses `valueOf`), and reflection cannot create a second instance.

REAL WORLD

In backend code enums model statuses and types (`OrderStatus`, `PaymentMethod`) and often appear in JSON and database mappings. Jackson serialises an enum by `name()` by default; JPA by ordinal unless told otherwise (see the warning above). Keep an explicit `code` field when external systems need stable values and look it up through a static map.

Check yourself

1. What does `enum E { A, B }` compile to, and what are `A` and `B`?
2. Is this legal? `new Level()` where `Level` is an enum.
3. What does this print? `enum T { X, Y, Z } ... System.out.println(T.Z.compareTo(T.X) + " " + T.valueOf("Y").ordinal());`
4. Why is storing `ordinal()` in a database dangerous?
5. When would you pick `EnumMap` over `HashMap`, and what must you pass to its constructor?
6. Why does `Level.valueOf("high")` fail when the constant is `HIGH`, and what is thrown?

Answers

1. A final class `E extends Enum<E>` with two `public static final` instances `A` and `B`, created in a static initializer.
2. No. Enum constructors are private and you cannot instantiate an enum with `new`; the set of instances is closed.
3. `2 1`. `compareTo` returns the ordinal difference (2 - 0) and `Y` has ordinal 1.
4. Adding or reordering constants changes the ordinals, so stored numbers then point at the wrong constants. Store the name or a stable code.
5. When keys are an enum: it is faster and smaller (an array indexed by ordinal) and iterates in declaration order. Pass the key type's `Class` object, e.g. `new EnumMap<>(Day.class)`.
6. `valueOf` matches the name exactly and is case-sensitive; it throws `IllegalArgumentException` ("No enum constant ...").

KEY POINTS

- An enum is a class with a closed set of singleton instances; compare constants with `==`.
- Constants can have fields, constructors, methods and per-constant bodies (an abstract method per constant replaces `if` chains).
- Never persist `ordinal()`; store `name()` or an explicit code.
- Switch expressions over enums are checked for completeness, so omit `default` for your own enums.
- Use `EnumSet` and `EnumMap` for enum-keyed collections; a single-element enum is a safe singleton.

3.9 Records

A huge amount of Java code is "data carriers": a class with a few fields, a constructor, getters, `equals`, `hashCode` and `toString`. Writing a `Point` with two ints takes about thirty lines, and every line is a chance for a bug (forgetting a field in `equals`, for example). A **record** (Java 16) states the intent in one line and the compiler writes the rest:

JAVA

```
record Point(int x, int y) {}
```

That declares a final class with two private final fields, a constructor, accessors `x()` and `y()`, and `equals`, `hashCode` and `toString` based on both components. A record is a *transparent, shallowly immutable carrier of values*: what you see in the header is its whole state and its public shape.

3.9.1 What you get for free

Records.java JAVA

```java
import java.util.ArrayList;
import java.util.List;
import java.util.Objects;

public class Records {
    record Point(int x, int y) {}

    record Money(long cents, String currency) {
        Money {                                  // compact canonical constructor
            Objects.requireNonNull(currency, "currency");
            if (cents < 0) throw new IllegalArgumentException("negative: " + cents);
            currency = currency.toUpperCase();    // normalise the parameter
        }
        Money plus(Money o) { return new Money(cents + o.cents, currency); }
        static Money ofEuros(int e) { return new Money(e * 100L, "eur"); }
    }

    record Course(String title, List<String> students) {
        Course {
            students = List.copyOf(students);     // defensive copy, unmodifiable
        }
    }

    record Leaky(List<String> items) {}

    public static void main(String[] args) {
        Point p = new Point(3, 4);
        System.out.println(p + " " + p.x() + " " + p.equals(new Point(3, 4))
            + " " + (p.hashCode() == new Point(3, 4).hashCode()));
        System.out.println(Money.ofEuros(5).plus(new Money(250, "Eur")));
        try { new Money(-1, "usd"); }
        catch (IllegalArgumentException e) { System.out.println(e.getMessage()); }

        List<String> src = new ArrayList<>(List.of("Ana", "Raj"));
        Leaky leaky = new Leaky(src);
        Course course = new Course("Java", src);
        src.add("Zed");
        System.out.println(leaky.items() + " " + course.students());
        try { course.students().add("x"); }
        catch (UnsupportedOperationException e) { System.out.println("UOE"); }
        System.out.println(Point.class.isRecord() + " " + Point.class.getSuperclass());
        for (var c : Point.class.getRecordComponents())
            System.out.println(c.getName() + ":" + c.getType());
    }
}
```

Output TERMINAL

```
$ java Records.java
Point[x=3, y=4] 3 true true
Money[cents=750, currency=EUR]
negative: -1
[Ana, Raj, Zed] [Ana, Raj]
UOE
true class java.lang.Record
x:int
y:int
```

What to notice

- The accessor is `x()`, not `getX()`. The generated `toString` is `Point[x=3, y=4]`.
- `Money` has a **compact constructor**: no parameter list, no field assignments. The body runs first and may validate or reassign the *parameters* (`currency = currency.toUpperCase()`); the compiler then assigns the fields from the parameters at the end. You cannot write `this.currency = ...` there.
- `Leaky` stores the caller's list: when `src` changes, the "immutable" record changes too. `Course` copies in its constructor, so it is safe.

3.9.2 Constructors in depth

A record has one **canonical constructor** whose parameters match the components in order. You have three ways to deal with it:

1. Write nothing: the compiler generates it.
2. Write a **compact** constructor (as above) for validation and normalisation. The most common choice.
3. Write the full **explicit canonical** constructor with a parameter list and `this.x = x;` assignments. It must be at least as accessible as the record itself.

You can also add other constructors, but each one must start with `this(...)`, delegating eventually to the canonical one, so every instance passes through the same validation.

Rec2.java JAVA

```java
import java.io.*;
import java.util.*;

public class Rec2 {
    record Range(int lo, int hi) {
        Range {
            if (lo > hi) throw new IllegalArgumentException(lo + " > " + hi);
        }
        Range(int single) { this(single, single); }        // non-canonical must delegate
        static final Range EMPTY = new Range(0, 0);        // static fields allowed
        int length() { return hi - lo; }
        static Range of(String s) {
            String[] p = s.split("-");
            return new Range(Integer.parseInt(p[0]), Integer.parseInt(p[1]));
        }
    }
    record Pair<A, B>(A first, B second) {}
    record Line(Pair<Integer,Integer> from, Pair<Integer,Integer> to) {}
    record Person(String name, int age) implements Serializable {}
    record Version(String tag) implements Comparable<Version> {
        public int compareTo(Version o) { return tag.compareTo(o.tag); }
        @Override public String toString() { return "v" + tag; }
    }
    record Named(String name) {
        @Override public String name() { return name.trim(); }      // custom accessor
    }

    public static void main(String[] args) throws Exception {
        System.out.println(new Range(5) + " " + Range.of("2-9").length() + " " + Range.EMPTY);
        System.out.println(new Line(new Pair<>(0, 0), new Pair<>(3, 4)));
        var bos = new ByteArrayOutputStream();
        try (var out = new ObjectOutputStream(bos)) { out.writeObject(new Person("Ana", 30)); }
        try (var in = new ObjectInputStream(new ByteArrayInputStream(bos.toByteArray()))) {
            System.out.println(in.readObject());
        }
        System.out.println(new Version("1.2") + " " + new Named("  x ").name()
            + "|" + new Named("  x "));
        System.out.println(new Pair<>(null, 1).equals(new Pair<>(null, 1)));
        System.out.println(new Pair<>(1.0, 0.0).equals(new Pair<>(1.0, -0.0)));
    }
}
```

Output TERMINAL

```
$ java Rec2.java
Range[lo=5, hi=5] 7 Range[lo=0, hi=0]
Line[from=Pair[first=0, second=0], to=Pair[first=3, second=4]]
Person[name=Ana, age=30]
v1.2 x|Named[name=  x ]
true
false
```

What to notice

- Records can be generic (`Pair<A, B>`), nested inside each other (`Line`), implement interfaces (`Comparable`, `Serializable`), and have static fields, static methods and ordinary instance methods.
- Records **cannot** declare instance fields beyond the components, cannot extend another class (they already extend `java.lang.Record`), and are implicitly `final`, so they cannot be extended.

- You may override an accessor (`Named.name()`), but the generated `toString`, `equals` and `hashCode` read the **fields**, not the accessors: the last line of output still shows `x` untrimmed. Keep accessors and fields consistent.
- Equality of components: reference components use `equals` (null-safe, so `null` works), primitives are compared by value, and `double`/`float` components compare with `Double.compare`, so `0.0` is not equal to `-0.0`.

3.9.3 How it works under the hood

`javap -p` on a record shows what the compiler added. The three object methods are `final` and are implemented with `invokedynamic`, delegating to `java.lang.runtime.ObjectMethods` at link time instead of bytecode generated per record.

Inspecting a record TERMINAL

```
$ javap -p -cp t 'Records$Point'
final class Records$Point extends java.lang.Record {
  private final int x;
  private final int y;
  Records$Point(int, int);
  public final java.lang.String toString();
  public final int hashCode();
  public final boolean equals(java.lang.Object);
  public int x();
  public int y();
}
```

The class file also stores a `Record` attribute listing the components, which is what `getRecordComponents()` reads (the last lines of the first program's output). Frameworks use it for reflection-based construction. `Jackson` (since 2.12), `Gson`, JPA projections and Spring MVC can all bind to records.

3.9.4 Immutability, honestly

Fields are `final` and there are no setters, so you cannot reassign a component. But the immutability is **shallow**: if a component is a mutable object (an array, an `ArrayList`, a `Date`) the record's content can still change, as `Leaky` showed. To get real immutability:

- Copy mutable inputs in the compact constructor (`List.copyOf`, `Map.copyOf`, `array.clone()`), and
- Return unmodifiable views or copies from accessors if the component is an array, because the default accessor returns the very same array.

Records do not make *wither* operations (change one field, get a new object) automatic. You write them by hand: `Point withX(int nx) { return new Point(nx, y); }`. If a record has many components, a nested builder is still sensible.

> **WATCH OUT**
> A record with an array component has identity-based `equals` for that component, because arrays do not override `equals`. `new R(new int[]{1}).equals(new R(new int[]{1}))` is `false`. Prefer a `List` and copy it, or override `equals`, `hashCode` and `toString` yourself.

3.9.5 When to use a record

Records fit values defined entirely by their data: coordinates, money amounts, DTOs (data transfer objects) returned by a REST controller, keys of a `HashMap` (the generated `hashCode` is correct), tuple-like return values from a method, and the variants of a sealed hierarchy (the next chapter). Local records (declared inside a method) are good for intermediate results in a stream pipeline.

They are the wrong tool when the object has identity or changing state (a bank account, a connection), when you need to hide the representation (a record's header is its public API; changing a component changes the API), or when it must be a JPA `@Entity` (entities need a no-arg constructor, mutable fields and proxies; records do not allow that).

Serialization notes

A record can implement `Serializable`, as `Person` did above (output proved the round trip). Records are serialized by their components only and **deserialized by calling the canonical constructor**, so your validation also runs on untrusted input; a plain class bypasses its constructor during deserialization, which is a classic security hole. `readObject`, `writeObject` and `serialPersistentFields` are ignored for records; only `writeReplace` and `readResolve` work. The default `serialVersionUID` of a record is `0L` unless you declare one. In practice most services serialize records as JSON rather than with Java serialization.

INTERVIEW ANGLE

Are records immutable? What can you not do with them? Records are shallowly immutable: components are `final`, no setters exist, and the class is `final`. A mutable component can still be changed through its own methods, so you copy mutable inputs in the compact constructor. You cannot add instance fields, extend a class, or make the record abstract; it can implement interfaces and have static members and extra methods.

REAL WORLD

In Spring Boot, request and response bodies are very often records (`record CreateOrderRequest(String sku, int qty) {}`), with Bean Validation annotations (`@NotBlank`, `@Min`) placed on the components. Configuration properties can also be bound to records through constructor binding.

Check yourself

1. Name five members the compiler generates for `record Point(int x, int y) {}`.
2. In a compact constructor, why does `this.x = x;` fail to compile, and how do you change the stored value?
3. What does this print? `record R(int[] a) {} ... System.out.println(new R(new int[]{1}).equals(new R(new int[]{1})));`
4. Why does a non-canonical record constructor have to start with `this(...)`?
5. A record has a `List<String>` component. How do you make the record safe against a caller who later mutates their list?
6. Why should you not use a record as a JPA `@Entity`?

Answers

1. Private final fields `x`, `y`; the canonical constructor; accessors `x()`, `y()`; `equals`; `hashCode`; `toString`.
2. In a compact constructor the fields are assigned automatically at the end, so assigning them yourself is an error. To change the stored value, reassign the parameter: `x = Math.abs(x);`.
3. `false`. Arrays use identity `equals`, and the record's `equals` delegates to it for that component.
4. So that every construction path goes through the canonical constructor, and therefore through its validation and field assignment.
5. Copy it in the compact constructor: `items = List.copyOf(items);` (this also rejects `null` elements), and do not expose a mutable list from accessors.
6. Entities need mutable state, a no-argument constructor and non-final classes (for lazy-loading proxies); a record is final, immutable and has no no-arg constructor. Use records for DTOs and projections instead.

KEY POINTS

- A record declares a transparent, final data carrier; the compiler generates fields, canonical constructor, accessors, `equals`, `hashCode`, `toString`.
- Validate and normalise in the compact constructor; every other constructor must delegate to the canonical one.
- Immutability is shallow: copy mutable components, especially collections and arrays.
- Records cannot extend classes or add instance fields, but can implement interfaces, be generic, nested and local.
- Use them for DTOs, keys, value objects and sealed-hierarchy variants, not for entities or objects with identity.

3.10 Sealed Classes and Interfaces

Normally anyone can extend a public class or implement a public interface. That openness is usually good, but sometimes you want the opposite: a type with a **closed, known set of subtypes**. A payment is either a card payment, a bank transfer or a wallet payment, and nothing else. An expression tree has exactly four node kinds. Before Java 17 you could only approximate this with a package-private constructor or an enum. A **sealed** class or interface (finalised in Java 17, JEP 409) lists exactly which classes may extend it, and the compiler enforces and exploits that list.

3.10.1 Declaring a sealed hierarchy

Shapes.java JAVA

```java
public class Shapes {
    sealed interface Shape permits Circle, Rect, Tri {}
    record Circle(double r) implements Shape {}
    record Rect(double w, double h) implements Shape {}
    record Tri(double b, double h) implements Shape {}

    static double area(Shape s) {
        return switch (s) {                      // no default: compiler checks coverage
            case Circle c -> Math.PI * c.r() * c.r();
            case Rect r   -> r.w() * r.h();
            case Tri t    -> 0.5 * t.b() * t.h();
        };
    }

    public static void main(String[] args) {
        for (Shape s : new Shape[]{new Circle(1), new Rect(2, 3), new Tri(4, 5)})
            System.out.printf("%s -> %.2f%n", s, area(s));
        System.out.println(Shape.class.isSealed());
        for (var c : Shape.class.getPermittedSubclasses()) System.out.println(c.getSimpleName());
    }
}
```

Output TERMINAL

```
$ java Shapes.java
Circle[r=1.0] -> 3.14
Rect[w=2.0, h=3.0] -> 6.00
Tri[b=4.0, h=5.0] -> 10.00
true
Circle
Rect
Tri
```

The rules

- `sealed` plus a `permits` list names the allowed direct subtypes. If all of them are in the **same source file**, `permits` may be omitted and the compiler infers them.
- Permitted subtypes must be in the same module (or, in the unnamed module, the same package) as the sealed type.
- Every permitted subtype must say what happens next, by choosing exactly one modifier: `final` (no more subclasses; records and enums are implicitly final), `sealed` (continues the closed hierarchy with its own `permits`) or `non-sealed` (reopens the hierarchy: anyone may extend it).
- The sealed type itself may be `abstract`, an interface or an ordinary class.

Sealed2.java JAVA

```java
public class Sealed2 {
    sealed static abstract class Account permits Savings, Checking {}
    static final class Savings extends Account {}
    static non-sealed class Checking extends Account {}
    static class Premium extends Checking {}          // allowed: Checking is non-sealed

    sealed interface Result<T> {
        record Ok<T>(T value) implements Result<T> {}
        record Err<T>(String msg) implements Result<T> {}
    }

    static String describe(Result<Integer> r) {
```

Sealed2.java (continued) JAVA

```java
        return switch (r) {
            case Result.Ok<Integer> ok -> "ok " + ok.value();
            case Result.Err<Integer> err -> "error " + err.msg();
        };
    }
    public static void main(String[] args) {
        System.out.println(describe(new Result.Ok<>(7)) + ", "
            + describe(new Result.Err<>("boom")));
        System.out.println(new Premium() instanceof Account);
    }
}
```

Output TERMINAL

```
$ java Sealed2.java
ok 7, error boom
true
```

`Result<T>` shows the compact style: the variants are nested records inside the sealed interface, so no `permits` clause is needed. This `Ok`/`Err` shape is the same idea as `Result` types in Rust or Kotlin.

3.10.2 What the compiler does for you

Two things make sealed types worth using. First, a class outside the list cannot extend the type:

Adding a subtype that is not permitted (message shortened) TERMINAL

```
$ javac Bad.java
Bad.java:5: error: class is not allowed to extend sealed class: S (as it is not ...)
    record C() implements S {}
    ^
1 error
```

Second, a `switch` over a sealed type is checked for **exhaustiveness**: if you leave out a variant and have no `default`, it does not compile.

Forgetting a variant TERMINAL

```
$ javac Bad2.java
Bad2.java:5: error: the switch expression does not cover all possible input values
    static int f(S s) { return switch (s) { case A a -> 1; }; }
                              ^
1 error
```

This reverses the usual cost of extension. With a normal interface it is cheap to add a new type and expensive to add a new operation (every implementer must change). With a sealed hierarchy plus `switch`, adding an *operation* is cheap (write one more method with a switch) and adding a *variant* is checked: every switch without a `default` breaks at compile time, pointing at exactly the code to update. That is the right trade for stable data models such as ASTs, events and results.

> **TIP**
> Do not add `default` to a switch over your own sealed type. The `default` silences the exhaustiveness check, which is the main benefit. If you need a catch-all for the future, make the hierarchy non-sealed or accept the compile error as a feature.

3.10.3 Algebraic data types in plain Java

Computer scientists call this pattern an **algebraic data type**: a *sum type* (a value is exactly one of several variants, here the sealed interface) whose variants are *product types* (each bundles several values, here records). Sealed interfaces plus records give Java both halves. Combined with pattern matching (the next chapter) you can take a value apart in the `switch` itself:

Calc.java JAVA

```java
public class Calc {
    sealed interface Expr permits Num, Add, Mul, Neg {}
    record Num(int v) implements Expr {}
    record Add(Expr l, Expr r) implements Expr {}
    record Mul(Expr l, Expr r) implements Expr {}
```

Calc.java (continued) JAVA

```java
    record Neg(Expr e) implements Expr {}

    static int eval(Expr e) {
        return switch (e) {
            case Num n -> n.v();
            case Add a -> eval(a.l()) + eval(a.r());
            case Mul m -> eval(m.l()) * eval(m.r());
            case Neg n -> -eval(n.e());
        };
    }
    static String show(Expr e) {
        return switch (e) {
            case Num(int v) -> Integer.toString(v);
            case Add(Expr l, Expr r) -> "(" + show(l) + " + " + show(r) + ")";
            case Mul(Expr l, Expr r) -> show(l) + " * " + show(r);
            case Neg(Expr x) -> "-" + show(x);
        };
    }
    public static void main(String[] args) {
        Expr e = new Add(new Num(2), new Mul(new Num(3), new Neg(new Num(4))));
        System.out.println(show(e) + " = " + eval(e));
    }
}
```

Output TERMINAL

```
$ java Calc.java
(2 + 3 * -4) = -10
```

The same job in classic object-oriented style would put an `eval()` method in every class (the visitor pattern if operations should live outside). Sealed types plus switch replace the visitor with ordinary code: no `accept` methods and no double dispatch. The design patterns Part returns to this.

Sealed versus enum versus open interface

Use an **enum** when the variants are fixed singletons with the same shape. Use a **sealed interface of records** when variants differ in what data they carry. Use a plain **interface** when third parties must be able to add implementations (plug-ins, `List`, `Comparable`).

GOTCHA

Sealed does not mean secret. Adding a permitted subtype is a source-incompatible change for everyone who switches over the type, even in other projects, so do not seal types in a public library API unless you are ready to treat new variants as breaking. The JDK itself seals very few types.

INTERVIEW ANGLE

What problem do sealed classes solve, and how do they relate to `switch`? They let the author restrict which classes may extend a type, via `permits` (each subtype must be `final`, `sealed` or `non-sealed`). Because the compiler then knows the complete set of subtypes, a `switch` over the sealed type can be checked for exhaustiveness without a `default`, so adding a new variant produces compile errors where the code needs updating.

Check yourself

1. What three modifiers may a permitted subclass of a sealed class use?
2. When can the `permits` clause be left out?
3. What is wrong with this? `sealed interface S permits A {} record A() implements S {} class B implements S {}`
4. Why does a switch over a sealed type not need a `default`, and what is the risk of adding one anyway?
5. A sealed interface is implemented by three records. Is the product or the sum type the interface? And the records?
6. Does `non-sealed` break the exhaustiveness guarantee? Explain briefly.

Answers

1. `final`, `sealed` or `non-sealed` (records and enums count as implicitly final).
2. When all permitted subtypes are declared in the same compilation unit (file), or are nested inside the sealed type; the compiler infers them.

3. `B` is not in the `permits` list, so the compiler rejects it ("class is not allowed to extend sealed class").
4. Because the compiler knows all subtypes, it can verify every case is covered. A `default` would hide a missing new variant until it fails at runtime.
5. The interface is the sum type (one of several alternatives); each record is a product type (several fields together).
6. No. The `non-sealed` subtype is itself still one of the known permitted variants, so the switch covers it by its own case; you just cannot know the types below it.

KEY POINTS

- `sealed ... permits` closes a hierarchy; each permitted subtype must be `final`, `sealed` or `non-sealed`.
- A switch over a sealed type is exhaustive without `default`, so new variants cause compile errors where handling is missing.
- Sealed interfaces plus records give algebraic data types (sum of products) in plain Java.
- Choose enums for same-shaped constants, sealed records for variants with different data, open interfaces for extension by others.
- Sealing a public API makes adding a variant a breaking change.

3.11 Pattern Matching

Code constantly asks "what kind of thing is this, and if it is that kind, give me its parts". Older Java answered with a test, a cast and a variable: three steps, one of which (the cast) the compiler cannot verify against the test. **Pattern matching** merges them: a *pattern* both tests a value and, on success, declares variables bound to its parts. Java has delivered it in stages: `instanceof` patterns (Java 16), `switch` patterns (Java 21), and record patterns (Java 21), with unnamed patterns (`_`) finalised in Java 22.

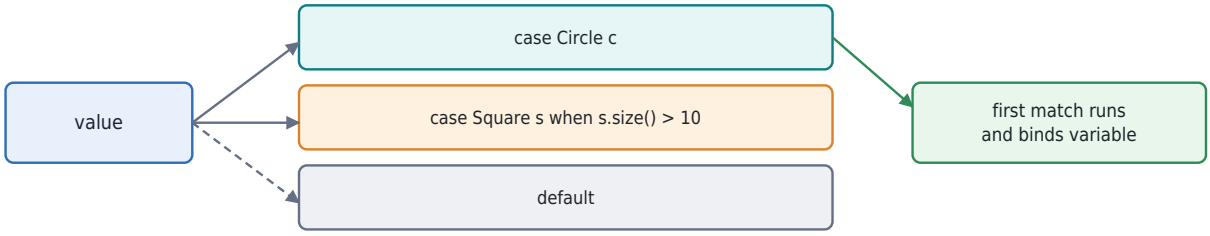


Figure 3.4. How a value flows through a pattern switch: cases are tried top to bottom, each pattern tests then binds, and the first match wins.

3.11.1 Type patterns with instanceof

```java
if (o instanceof String) {
    String s = (String) o;            // test, cast, bind: three steps
    System.out.println(s.length());
}

if (o instanceof String s) {          // test + cast + bind in one
    System.out.println(s.length());
}
```

The binding `s` is in scope exactly where the compiler can prove the match succeeded. That includes the right side of `&&`, and the code after a negated test that exits:

```java
if (o instanceof String s && s.length() > 3) { /* s usable here */ }
if (!(o instanceof Integer i)) return "not an int";
return "int " + (i + 1);              // i is in scope: we only get here if it matched
```

Because `||` could be reached on a failed match, `o instanceof String s || s.isEmpty()` does not compile. A pattern variable is not `final` automatically but it behaves like a local: you may reassign it unless declared `final`, though you rarely should.

3.11.2 Pattern matching for switch

A `switch` (statement or expression, since Java 21) may switch on **any reference type** and use patterns as case labels. Cases are tried top to bottom; the first one that matches runs.

Patterns.java JAVA

```java
import java.util.List;

public class Patterns {
    record Point(int x, int y) {}
    record Line(Point from, Point to) {}

    static String old(Object o) {
        if (o instanceof String) {
            String s = (String) o;            // test, cast, bind: three steps
            return "string of " + s.length();
        }
        return "other";
    }

    static String describe(Object o) {
        if (o instanceof String s && s.length() > 3) return "long string " + s;
        if (!(o instanceof Integer i)) return "not an int: " + o;
        return "int " + (i + 1);                      // i is in scope here
    }

    static String classify(Object o) {
        return switch (o) {
            case null -> "null!";
            case Integer i when i > 100 -> "big int " + i;
            case Integer i -> "int " + i;
            case String s when s.isEmpty() -> "empty string";
            case String s -> "string " + s;
            case Line(Point(var x1, var y1), Point(int x2, int y2)) when x1 == x2
                -> "vertical line";
            case Line(Point a, Point b) -> "line " + a + "->" + b;
            case int[] arr -> "int array of " + arr.length;
            case List<?> l -> "list size " + l.size();
            default -> "something else: " + o.getClass().getSimpleName();
        };
    }

    public static void main(String[] args) {
        System.out.println(old("hello") + " | " + old(5));
        System.out.println(describe("hello") + " | " + describe("hi") + " | " + describe(41));
        Object[] tests = {null, 7, 500, "", "abc", new Line(new Point(1, 1), new Point(1, 9)),
            new Line(new Point(0, 0), new Point(2, 3)), new int[3], List.of(1, 2), 2.5};
        for (Object t : tests) System.out.println(classify(t));
        Object obj = new Line(new Point(1, 2), new Point(3, 4));
        if (obj instanceof Line(Point(var a, var b), Point p2)) System.out.println(a + b + p2.x());
    }
}
```

Output TERMINAL

```
$ java Patterns.java
string of 5 | other
long string hello | not an int: hi | int 42
null!
int 7
big int 500
empty string
string abc
vertical line
line Point[x=0, y=0]->Point[x=2, y=3]
int array of 3
list size 2
something else: Double
6
```

What to notice

- `case null` handles null explicitly. Without it, switching on `null` still throws `NullPointerException`, as with the old switch. You can combine: `case null, default -> ...`.
- `when` adds a **guard**: a boolean condition on the same case. `case Integer i when i > 100` must come before the plain `case Integer i`.
- `case Line(Point(var x1, var y1), Point(int x2, int y2))` is a **record pattern with nested patterns**: it tests for `Line`, calls its accessors, tests the components for `Point`, and binds the ints, all in one line. `var` lets the compiler infer each component type.
- The last line shows a record pattern in `instanceof`.

3.11.3 Record patterns

A **record pattern** `R(P1, P2, ...)` matches a record `R` and then matches each component against its sub-pattern. Nesting is unlimited, which is how you pull apart a tree in one expression (`case Add(Mul(Num(var a), Num(var b)), var rest)` in the expression example of the previous chapter). Under the hood the compiler calls the component accessors; if one throws, the exception is wrapped in a `MatchException`. A sub-pattern whose type is the same as (or wider than) the component type is called *total* and matches everything including `null`:

Nul.java JAVA

```java
public class Nul {
    record Box(Object content) {}
    public static void main(String[] a) {
        Object o = new Box(null);
        System.out.println(o instanceof Box(Object c));      // total pattern matches null
        System.out.println(o instanceof Box(String c));      // type test fails on null
        Object n = null;
        System.out.println(n instanceof Object x);
        try {
            switch (n) {
                case String s -> System.out.println("s");
                default -> System.out.println("d");
            }
        } catch (NullPointerException e) { System.out.println("NPE"); }
    }
}
```

Output TERMINAL

```
$ java Nul.java
true
false
false
NPE
```

The lines correspond to: `new Box(null) instanceof Box(Object c)` is `true` (a total pattern accepts the null component); `... instanceof Box(String c)` is `false` (a type test never matches null); `null instanceof Object x` is `false` (instanceof never matches null); and `switch (n) { case String s ...; default ... }` with `n == null` throws `NullPointerException` because there is no `case null`.

3.11.4 Dominance and exhaustiveness

The compiler checks the order of your labels. A case that can never be reached because an earlier one already matches everything it would is **dominated**, and that is a compile error:

A dominated label TERMINAL

```
$ javac Dom.java
Dom.java:5: error: this case label is dominated by a preceding case label
            case String s -> "s";
                 ^
1 error
```

(`CharSequence` came first, and every `String` is a `CharSequence`.) The rules to remember:

- A more specific type must come **before** a more general one; guarded cases before unguarded ones of the same type.
- `default` (or `case null, default`) should be last. `case null` may be anywhere but is conventionally first.

- A switch expression, or a pattern switch statement, must be **exhaustive**: every possible value must be covered. For an `Object` selector that means a `default` or a total pattern like `case Object o`. For a sealed type, covering every permitted subtype is enough.

The compiler also checks sealed-record exhaustiveness through nested record patterns, which is what let the `show` method in the previous chapter work without `default`.

GOTCHA
Fall-through and patterns do not mix: you cannot fall into a case that declares a pattern variable, because the variable might not be bound. Use arrow (`->`) cases, which never fall through. With colon cases a pattern label must end in `break`, `return` or similar before the next label.

3.11.5 What is new after Java 21

Feature	Status	Note
`instanceof` type patterns	Java 16 final	Binding scope follows flow analysis.
Pattern `switch`, record patterns	Java 21 final	Both in this chapter; guards use `when`.
Unnamed variables and patterns `_`	Java 22 final	`case Box(_)`, `catch (Exception _)`. Needs 22 or later, so not run here.
Primitive types in patterns	Preview in 25, 26 and 27	`case int i` on a `long` or `Object` selector. Not final; do not rely on it.

Pattern-related features by version.

With unnamed patterns (Java 22 and later) you write `case Line(Point(var x, _), _)` to say "I do not care about this component", which removes unused variable names. They are not runnable on this book's JDK 21 and are shown only as a note. Primitive patterns are a preview feature in Java 25 (and still preview in 26 and 27 according to the JEP list), so treat them as experimental.

INTERVIEW ANGLE
What does pattern matching for `instanceof`/`switch` give you over casts, and how does `switch` treat `null`? It combines the type test, the cast and the variable binding, so a wrong cast is impossible. A `switch` with patterns can have guards (`when`), nested record patterns and compile-time exhaustiveness checks. `null` selectors throw `NullPointerException` unless the switch has a `case null` label (possibly `case null, default`).

REAL WORLD
Pattern switches over sealed event or command hierarchies are replacing long `if (x instanceof A) ... else if (x instanceof B)` chains in message handlers, parsers and compilers. They also replace many visitor-pattern implementations. In Spring services the typical use is mapping a sealed `Result` or domain event into an HTTP response.

Check yourself

1. What does this print? `Object o = "hi"; if (!(o instanceof String s)) { System.out.println("no"); return; } System.out.println(s.length());`
2. Why does this not compile? `switch (o) { case CharSequence c -> ..; case String s -> ..; default -> ..; }`
3. How do you make a pattern switch accept `null` without an exception?
4. What does `case Point(var x, var y) when x == y` do?
5. Is `null instanceof Object` true or false?
6. Why do you not need `default` when every subtype of a sealed interface has a case?

Answers

1. `2`. The binding `s` is in scope after the early return because reaching it means the test succeeded.
2. `case String s` is dominated: every `String` is a `CharSequence`, so it can never be reached. Put the more specific case first.
3. Add `case null ->` (or `case null, default ->`). Otherwise a `null` selector throws `NullPointerException`.
4. It matches a `Point` and binds both components, and then only succeeds when the guard (the two coordinates are equal) is true; otherwise the next case is tried.
5. False. `instanceof` never matches `null`.

6. The compiler knows the permitted subtypes, so the cases are provably exhaustive. (An unexpected new subclass compiled later leads to a `MatchException` at runtime, which is why recompiling dependents matters.)

KEY POINTS

- A pattern tests a value and binds variables in one step: `o instanceof String s`, `case Circle c`, `case Point(var x, var y)`.
- Cases run top to bottom; more specific labels must precede general ones (dominance), and `when` adds a guard.
- Switch over any reference type; null needs `case null`, otherwise it throws NPE.
- Record patterns nest, and switches over sealed types are checked for exhaustiveness.
- `_` is final since Java 22; primitive patterns are still preview in 25, 26 and 27.

3.12 Wrapper Classes, Autoboxing and Identity

Java has two worlds of values. The eight **primitives** (`int`, `long`, `double`, `boolean`, ...) are raw numbers stored directly in variables and arrays; they are fast, cannot be `null`, and have no methods. **Objects** live on the heap and are reached through references. Many parts of the language only work with objects: generics (`List<int>` is illegal), collections, `Optional`, reflection, and anything typed `Object`. To bridge the two worlds, every primitive has a matching **wrapper class**, and the compiler converts between them automatically. That convenience hides several traps: `==` that works for 127 but not for 128, a `NullPointerException` on a line that contains no dot, and loops that quietly allocate millions of objects. This chapter shows how the conversion works so you can predict all of it.

3.12.1 The eight wrappers

Primitive	Wrapper	Superclass	Cached by valueOf
`byte`	`Byte`	Number	all 256 values
`short`	`Short`	Number	-128 to 127
`int`	`Integer`	Number	-128 to 127 (upper bound tunable)
`long`	`Long`	Number	-128 to 127
`float`	`Float`	Number	none
`double`	`Double`	Number	none
`char`	`Character`	Object	0 to 127
`boolean`	`Boolean`	Object	TRUE and FALSE only

Primitive types and their wrapper classes (all in java.lang).

Wrapper objects are **immutable**: an `Integer` never changes its value, just like a `String`. They are also `final`, so you cannot subclass them. Each wrapper gives you three kinds of members: constants (`Integer.MAX_VALUE`, `Integer.BYTES`), static utilities that work on the primitive (`Integer.parseInt`, `Integer.compare`, `Integer.toBinaryString`, `Character.isDigit`), and instance methods that unwrap the value (`intValue()`, `doubleValue()`). The utilities are the main reason to know the classes even if you never box anything.

3.12.2 Autoboxing and unboxing

Boxing converts a primitive to its wrapper; **unboxing** is the reverse. Since Java 5 the compiler inserts both for you. Nothing magical happens at run time: the compiler rewrites your code into calls to `valueOf` and `xxxValue`. The bytecode of `static int f(Integer i) { Integer j = i + 1; return j; }` shows it (trimmed `javap -c -p` output):

Boxing is just method calls TERMINAL

```
$ javap -c -p Jp
  static int f(java.lang.Integer);
       0: aload_0
       1: invokevirtual #7    // Method java/lang/Integer.intValue:()I
       4: iconst_1
       5: iadd
       6: invokestatic  #13   // Method java/lang/Integer.valueOf:(I)Ljava/lang/Integer;
       9: astore_1
      10: aload_1
      11: invokevirtual #7    // Method java/lang/Integer.intValue:()I
      14: ireturn
```

So `i + 1` unboxes `i` with `intValue()`, adds in primitive `int`, boxes the result with `Integer.valueOf(int)`, and `return j` unboxes again. Three method calls hide in one innocent line. Two rules follow immediately:

- Unboxing calls a method on the reference. If the reference is `null`, you get a `NullPointerException` even though your source shows no `.`.
- Boxing calls `valueOf`, which may allocate a new object. Where it allocates, and where it reuses an existing one, is the source of the identity puzzles below.

Use `Integer.valueOf(int)` (or let autoboxing do it), never `new Integer(5)`. The constructors of all wrappers have been deprecated since Java 9 and marked for removal since Java 16; they always allocate a fresh object and defeat the cache.

3.12.3 The Integer cache and the == trap

`==` on two references compares **identity** (are they the same object?), not value. For wrappers that is almost never what you want. But a cache makes it appear to work sometimes, which is worse than never working. `Integer.valueOf(int)` keeps a pre-built array of `Integer` objects for -128 to 127 and hands out the same instance every time; outside that range it allocates a new one. The Java Language Specification requires this caching for boxing of small values, so the behaviour is guaranteed, not an implementation accident. Run the following program, which also covers the other traps in this chapter:

Boxing.java JAVA

```java
import java.util.ArrayList;
import java.util.HashMap;
import java.util.List;
import java.util.Map;

public class Boxing {
    public static void main(String[] args) {
        Integer a = 127, b = 127;
        Integer c = 128, d = 128;
        System.out.println("127 == 127 : " + (a == b));
        System.out.println("128 == 128 : " + (c == d));
        System.out.println("128 equals : " + c.equals(d));
        Integer e = Integer.valueOf(1000), f = Integer.valueOf(1000);
        System.out.println("valueOf(1000) == : " + (e == f));
        System.out.println("Long 127 == : " + (Long.valueOf(127) == Long.valueOf(127)));
        System.out.println("Long 128 == : " + (Long.valueOf(128) == Long.valueOf(128)));
        System.out.println("Character 'a' == : "
            + (Character.valueOf('a') == Character.valueOf('a')));
        System.out.println("Boolean == : " + (Boolean.valueOf(true) == Boolean.valueOf("TRUE")));
        System.out.println("Double 1.0 == : " + (Double.valueOf(1.0) == Double.valueOf(1.0)));
        System.out.println("Integer vs Long equals : "
            + Integer.valueOf(5).equals(Long.valueOf(5)));
        System.out.println("Integer == int : " + (c == 128));

        List<Integer> list = new ArrayList<>(List.of(10, 20, 30, 1));
        list.remove(1);
        System.out.println("after remove(1)            : " + list);
        list.remove(Integer.valueOf(1));
        System.out.println("after remove(Integer 1)    : " + list);

        Map<String, Integer> stock = new HashMap<>();
        stock.put("pen", 5);
        try {
            int n = stock.get("ink");
            System.out.println(n);
        } catch (NullPointerException ex) {
```

Boxing.java (continued) JAVA

```
            System.out.println("NPE unboxing a missing map value");
        }
        boolean flag = false;
        try {
            Integer r = flag ? 1 : (Integer) null;
            System.out.println(r);
        } catch (NullPointerException ex) {
            System.out.println("NPE from ternary with int operand");
        }
        Integer ok = flag ? Integer.valueOf(1) : null;
        System.out.println("both boxed: " + ok);

        Integer x = 10;
        Integer y = x;
        x++;
        System.out.println("x=" + x + " y=" + y);

        Object o = true ? Integer.valueOf(1) : Double.valueOf(2.0);
        System.out.println("mixed ternary: " + o);
    }
}
```

Output TERMINAL

```
$ java Boxing.java
127 == 127 : true
128 == 128 : false
128 equals : true
valueOf(1000) == : false
Long 127 == : true
Long 128 == : false
Character 'a' == : true
Boolean == : true
Double 1.0 == : false
Integer vs Long equals : false
Integer == int : true
after remove(1)            : [10, 30, 1]
after remove(Integer 1)    : [10, 30]
NPE unboxing a missing map value
NPE from ternary with int operand
both boxed: null
x=11 y=10
mixed ternary: 1.0
```

What to notice

- `127 == 127` is `true` and `128 == 128` is `false`. Both use autoboxing; only the first lands in the cache. `equals` compares values and is always right.
- `Double` has no cache, so `Double.valueOf(1.0) == Double.valueOf(1.0)` is `false`. `Boolean` and the small `Character` and `Byte` values are always shared.
- `Integer == int` (the line `c == 128`) is a **numeric** comparison: when one operand is a primitive, the other is unboxed and the values are compared. That is the safe form.
- `Integer.valueOf(5).equals(Long.valueOf(5))` is `false`: `equals` first checks that the other object is an `Integer`. Comparing different wrapper types never succeeds.
- `x++` on an `Integer` is `x = Integer.valueOf(x.intValue() + 1)`: a **new** object is assigned to `x`. `y` still points to the old one, as immutability promises.

The cache range is tunable

The lower bound -128 is fixed. The upper bound is 127 by default, but the JVM flag `-XX:AutoBoxCacheMax=<n>` (or the system property `java.lang.Integer.IntegerCache.high`) raises it. Re-running the first lines with `java -XX:AutoBoxCacheMax=1000 Boxing.java` changes the second line:

Same code, different answer TERMINAL

```
$ java -XX:AutoBoxCacheMax=1000 Boxing.java
127 == 127 : true
128 == 128 : true
128 equals : true
```

This is the strongest argument for never using `==` on boxed values: code that "works" in your tests may break when a flag or a value changes. Lint rules (SonarQube, Error Prone, IDE inspections) flag it for the same reason.

INTERVIEW ANGLE
Why does `Integer a = 127, b = 127; a == b` give true but `Integer c = 128, d = 128; c == d` give false? `Integer.valueOf` caches instances for -128 to 127, so boxing both 127s returns the same object and `==` (reference comparison) is true. 128 is outside the default cache, so two separate objects are created. The fix is `equals` (or unboxing: `c.intValue() == d.intValue()`). The cache upper bound can be changed with `-XX:AutoBoxCacheMax`, which is one more reason never to depend on it.

3.12.4 NullPointerException from unboxing

Because unboxing calls `xxxValue()`, any place where a `null` wrapper meets a primitive context throws. The three common sources are in the program above:

- **Map lookups.** `Map.get` returns `null` for a missing key, and `int n = stock.get("ink")` unboxes it. Use `getOrDefault("ink", 0)`, or check `containsKey`, or keep the value as `Integer` and test it for `null`.
- **Mixed ternaries.** In `flag ? 1 : (Integer) null`, one operand is a primitive `int` and the other an `Integer`. The ternary's rules make the result type `int` (binary numeric promotion), so the `null` is unboxed whenever it is chosen. If both operands are wrapper types, as in `flag ? Integer.valueOf(1) : null`, no unboxing happens and the result is simply `null`.
- **Fields and returns.** An `Integer` field that was never set is `null`, and `return count;` from a method declared `int` unboxes it. Database columns that are nullable map naturally to wrapper fields, so this shows up often in entity classes.

The last line of the output, `mixed ternary: 1.0`, shows a quieter ternary rule. When both operands of `?:` are convertible to numbers, the result type is the promoted numeric type, so `true ? Integer.valueOf(1) : Double.valueOf(2.0)` unboxes the `Integer` and widens it to `double`, printing `1.0`, not `1`.

WATCH OUT
Overloads and boxing interact. `list.remove(1)` on a `List<Integer>` removes the element at **index** 1, because `remove(int index)` is an exact match and wins before any boxing is considered. To remove the **value** 1 you must write `list.remove(Integer.valueOf(1))`. The output above shows both results.

3.12.5 Performance cost of boxing

A primitive `int` takes 4 bytes. An `Integer` is an object: a header plus the value, typically 16 bytes on a 64-bit HotSpot JVM with compressed references, and a `List<Integer>` also stores a 4-byte reference to each (the object layout chapter in Part 9 gives the numbers). So a `List<Integer>` of a million numbers needs roughly four times the memory of an `int[]`, and traversing it chases pointers to scattered objects instead of reading a contiguous block, which is slow for the CPU cache. In a hot loop each boxing operation may also allocate, creating garbage for the collector.

The classic demonstration accumulates into a `Long` instead of a `long`:

```java
long sum = 0;
for (long i = 0; i < 100_000_000L; i++) sum += i;      // no allocation

Long boxed = 0L;
for (long i = 0; i < 100_000_000L; i++) boxed += i;    // boxed = Long.valueOf(boxed + i)
```

On the build machine (typical, measured with `System.nanoTime`, one run each, no benchmarking harness) the first loop took about 2.2 seconds and the second about 2.7 seconds. The gap here is smaller than folklore suggests, because the JIT compiler can sometimes remove the allocation (escape analysis); on other machines, JVMs or less simple loops the boxed version can be many times slower. Treat these figures as a hint, not a benchmark; the reliable rules are structural:

- Use primitives for local variables, loop counters, accumulators, and fields that can never be absent.
- Use wrappers only when you need an object: generics, collections, `Optional<Integer>`, nullable database values.
- For large numeric collections consider primitive streams (`IntStream`) and arrays, or specialised libraries, instead of `List<Integer>`. The streams chapter in Part 7 shows `mapToInt` for exactly this reason.

3.12.6 The Number hierarchy and parsing

`java.lang.Number` is an abstract class. Six wrappers extend it (`Byte`, `Short`, `Integer`, `Long`, `Float`, `Double`), and so do `BigInteger`, `BigDecimal` and the atomic classes `AtomicInteger` and `AtomicLong`. `Character` and `Boolean` do **not**: they extend `Object` directly. `Number` declares the conversions `intValue()`, `longValue()`, `floatValue()`, `doubleValue()` (plus `byteValue()` and `shortValue()`); they truncate or narrow exactly like a cast, so `Number n = 3.99; n.intValue()` is `3`.

Nums.java JAVA

```java
import java.math.BigDecimal;
import java.math.BigInteger;
import java.util.concurrent.atomic.AtomicInteger;

public class Nums {
    public static void main(String[] args) {
        Class<?>[] types = {Integer.class, Double.class, BigDecimal.class, BigInteger.class,
            AtomicInteger.class, Character.class, Boolean.class};
        for (Class<?> t : types) {
            System.out.println(t.getSimpleName() + " extends " + t.getSuperclass().getSimpleName()
                + ", Number? " + Number.class.isAssignableFrom(t));
        }
        Number n = 3.99;
        System.out.println(n.intValue() + " " + n.longValue() + " " + n.doubleValue());
        System.out.println(Integer.parseInt("42") + 1);
        System.out.println(Integer.valueOf("42") + 1);
        System.out.println(Integer.toBinaryString(42) + " " + Integer.toHexString(255)
            + " " + Integer.parseInt("ff", 16));
        System.out.println(Integer.compare(3, 7) + " " + Integer.signum(-9)
            + " " + Integer.sum(2, 3) + " " + Integer.MAX_VALUE + " " + Integer.BYTES);
        try { Integer.parseInt("12x"); } catch (NumberFormatException e) {
            System.out.println(e.getMessage());
        }
        System.out.println(Double.compare(0.0, -0.0) + " " + (0.0 == -0.0)
            + " " + Double.valueOf(0.0).equals(-0.0));
        System.out.println(Double.valueOf(Double.NaN).equals(Double.NaN)
            + " " + (Double.NaN == Double.NaN));
    }
}
```

Output TERMINAL

```
$ java Nums.java
Integer extends Number, Number? true
Double extends Number, Number? true
BigDecimal extends Number, Number? true
BigInteger extends Number, Number? true
AtomicInteger extends Number, Number? true
Character extends Object, Number? false
Boolean extends Object, Number? false
3 3 3.99
43
43
101010 ff 255
-1 -1 5 2147483647 4
For input string: "12x"
1 true false
true false
```

Two points are worth remembering. First, `parseInt(String)` returns a primitive `int` and `valueOf(String)` returns an `Integer` (through the cache); use `parseInt` when you want a number to compute with. Both throw the unchecked `NumberFormatException` for bad input, so validate or catch it when parsing user data. Second, the last two lines show that `Double.equals` and `==` disagree. `equals` follows the bit pattern, so `Double.valueOf(Double.NaN).equals(Double.NaN)` is `true` and `0.0` is not equal to `-0.0`, which keeps `HashMap` lookups consistent. The `==` operator follows IEEE 754, where `NaN != NaN` and `0.0 == -0.0`. Use `Double.compare` when you need a total ordering (for sorting) and mind that `==` on primitive doubles is a numeric comparison.

> **TIP**
> Prefer the static helpers over hand-written logic: `Integer.compare(a, b)` instead of `a - b` (which overflows), `Integer.sum`, `Integer.max`, `Integer.toString(n, radix)`, `Long.parseLong`, `Character.isLetter`, `Boolean.parseBoolean`. For money and exact decimals use `BigDecimal`, not `Double`, as the type-conversion chapter explained.

REAL WORLD
In Spring and JPA code, entity ids and nullable columns are usually `Long` or `Integer` (so that "not saved yet" can be `null`), while counters and flags inside business logic are primitives. A very common production bug is `if (order.getId() == otherOrder.getId())` on two `Long` ids: it works in tests with small ids and silently fails once ids pass 127. Use `Objects.equals(a, b)` or `.equals`.

Check yourself

1. What does this print? `Integer a = 200, b = 200; System.out.println((a == b) + " " + a.equals(b) + " " + (a == 200));`
2. `Map<String, Integer> m = new HashMap<>(); int v = m.get("x");` throws. Which exception, and what are two ways to avoid it?
3. What is the output? `List<Integer> l = new ArrayList<>(List.of(5, 6, 7)); l.remove(1); System.out.println(l);` How do you remove the value 6 instead?
4. What does `Long.valueOf(7).equals(7)` return, and why?
5. What does `Object o = true ? 1 : 2.0; System.out.println(o);` print?
6. Name two reasons to prefer `int` over `Integer` for a loop counter.

Answers

1. `false true true`. 200 is outside the cache so `a == b` compares two different objects; `equals` compares values; `a == 200` unboxes `a`.
2. `NullPointerException` (unboxing `null`). Use `m.getOrDefault("x", 0)`, or declare `Integer v` and test it for `null`, or check `containsKey` first.
3. `[5, 7]`: `remove(1)` removes the element at index 1. Use `l.remove(Integer.valueOf(6))` to remove the value.
4. `false`. The argument `7` is boxed to `Integer`, and `Long.equals` returns `false` for any object that is not a `Long`.
5. `1.0`. One operand is `double`, so the ternary has type `double` and the `1` is widened.
6. A primitive avoids an allocation and a `valueOf` call each iteration, uses 4 bytes instead of an object, and can never be `null`, so it cannot throw on unboxing.

KEY POINTS
- Every primitive has an immutable wrapper; autoboxing is compiler-inserted `valueOf` and `xxxValue()` calls.
- `==` on wrappers compares identity. Only -128 to 127 (and `Boolean`, small `Character`) are cached, so always compare with `equals`, or unbox with a primitive.
- Unboxing `null` throws `NullPointerException`: watch map lookups, mixed ternaries and nullable fields.
- Boxing costs memory, indirection and garbage; use primitives for arithmetic and wrappers only where an object is required.
- `Number` is the abstract parent of the numeric wrappers, big numbers and atomics; `Character` and `Boolean` are not `Number`s.

3.13 Designing Classes Well

Knowing the syntax of classes, interfaces, enums and records is the easy half. The harder half is deciding what to build with them: which things become classes, which depend on which, and how to leave room for change without paying for flexibility you never use. Bad class design rarely fails on day one. It fails six months later when a small requirement forces edits in twelve files, or when a bug appears because two parts of the program shared a mutable object. This chapter gathers the design habits that matter most in everyday Java, each with a small example you can run, and finishes with a checklist you can apply in code reviews and interviews.

3.13.1 Composition over inheritance

Inheritance (`extends`) is the strongest coupling Java offers. A subclass depends on the superclass's **implementation**, not only its public contract: which methods call which, in what order, and what they assume. When the superclass changes, subclasses can break even though no signature changed. **Composition** means building a class by holding another object in a field and forwarding calls to it. The class depends only on the other object's public methods.

The classic demonstration is a set that counts how many elements were ever added. The inheritance version looks natural; the composition version is correct:

Fragile.java JAVA

```java
import java.util.Collection;
import java.util.HashSet;
import java.util.List;
import java.util.Set;

public class Fragile {
    // BAD: inheritance couples us to HashSet's internals
    static class CountingHashSet<E> extends HashSet<E> {
        int added = 0;
        @Override public boolean add(E e) { added++; return super.add(e); }
        @Override public boolean addAll(Collection<? extends E> c) {
            added += c.size();
            return super.addAll(c);
        }
    }

    // GOOD: composition + forwarding
    static class CountingSet<E> {
        private final Set<E> inner = new HashSet<>();
        private int added = 0;
        boolean add(E e) { added++; return inner.add(e); }
        boolean addAll(Collection<? extends E> c) {
            boolean changed = false;
            for (E e : c) changed |= add(e);
            return changed;
        }
        int added() { return added; }
    }

    public static void main(String[] args) {
        CountingHashSet<String> bad = new CountingHashSet<>();
        bad.addAll(List.of("a", "b", "c"));
        System.out.println("inheritance: expected 3, got " + bad.added);
        CountingSet<String> good = new CountingSet<>();
        good.addAll(List.of("a", "b", "c"));
        System.out.println("composition: expected 3, got " + good.added());
    }
}
```

Output TERMINAL

```
$ java Fragile.java
inheritance: expected 3, got 6
composition: expected 3, got 3
```

The inheritance version counts every element twice. Its `addAll` adds `c.size()` and then calls `super.addAll`, and `HashSet`'s inherited `addAll` is implemented by calling the overridden `add` for each element, which increments the counter again. Nothing in the documentation promised that, and a future JDK could change it either way. This is the **fragile base class problem**.

The rules of thumb:

- Use inheritance only for a true "is-a" relationship, when the subclass honours everything the superclass promises, and when the superclass was designed for extension (documented, or abstract).
- Otherwise hold the collaborator in a `private final` field. Composition lets you swap the collaborator (pass a `TreeSet` instead), test with a fake, and hide the inner type from callers.
- Share behaviour between unrelated classes with an interface (with `default` methods if useful) and a helper object, not a deep class tree. If you do not intend a class to be extended, make it `final` (as the wrapper classes and `String` are); the "design for inheritance or prohibit it" rule from *Effective Java* is sound advice.

INTERVIEW ANGLE
Why is "favour composition over inheritance" a common design guideline? Inheritance couples a subclass to the superclass's implementation details (fragile base class) and fixes the relationship at compile time. Composition depends only on a public contract, can be changed at run time by passing a different collaborator, and is easier to test. Use inheritance for genuine is-a relationships designed for extension, and composition for reuse.

3.13.2 SOLID in plain Java

SOLID is five guidelines for class design, popularised by Robert C. Martin. They are guidelines, not laws: applying them mechanically produces needless interfaces. Used with judgement they keep code easy to change. Here is each one with a Java illustration.

S: Single Responsibility

A class should have one reason to change. A class `OrderService` that also formats invoices as PDF, talks to SQL and sends email will change whenever any of those change. Split by responsibility: one class for the business rule, one for persistence, one for notification.

O and D: Open/Closed and Dependency Inversion

Software entities should be **open for extension, closed for modification**: add behaviour by adding code (a new class), not by editing a working `if`/`switch` chain. **Dependency inversion** says high-level logic should depend on abstractions (interfaces), not on concrete low-level classes, and receive its collaborators from outside (constructor injection). Together they give you this shape, which is the core idea behind Spring's dependency injection:

Solid.java JAVA

```java
import java.util.ArrayList;
import java.util.List;

public class Solid {
    record Order(String id, double total) {}

    // S: one reason to change each
    interface OrderRepository { void save(Order o); }
    interface Notifier { void send(String message); }

    // D: the service depends on abstractions, receives them through the constructor
    static class OrderService {
        private final OrderRepository repo;
        private final Notifier notifier;
        OrderService(OrderRepository repo, Notifier notifier) {
            this.repo = repo;
            this.notifier = notifier;
        }
        void place(Order o) {
            repo.save(o);
            notifier.send("Order " + o.id() + " placed");
        }
    }

    // O: new discount rules are new classes; the calculator never changes
    interface Discount { double apply(double amount); }
    record Percent(double pct) implements Discount {
        public double apply(double amount) { return amount * (1 - pct / 100); }
    }
    record Flat(double off) implements Discount {
        public double apply(double amount) { return Math.max(0, amount - off); }
    }

    public static void main(String[] args) {
        List<String> log = new ArrayList<>();
        OrderService svc = new OrderService(o -> log.add("saved " + o), log::add);
        svc.place(new Order("A1", 99.5));
        log.forEach(System.out::println);
        for (Discount d : List.of(new Percent(10), new Flat(25), new Flat(500)))
            System.out.println(d + " -> " + d.apply(200));
    }
}
```

Output TERMINAL

```
$ java Solid.java
saved Order[id=A1, total=99.5]
Order A1 placed
Percent[pct=10.0] -> 180.0
Flat[off=25.0] -> 175.0
Flat[off=500.0] -> 0.0
```

`OrderService` has no idea whether the repository is a database or a list, nor whether notifications are email or a log line: in production you pass real implementations, in a unit test two lambdas (both interfaces have one method, so lambdas fit). Adding a `BuyOneGetOne` discount requires a new record and no change to existing code. If you know the sealed types from the previous chapters, note the trade-off: a **sealed** `Discount` makes adding a variant a deliberate, compiler-checked edit everywhere it is handled (closed for extension, open for new operations), while an open interface does the opposite. Choose by which kind of change you expect.

L: Liskov Substitution

Code that works with a base type must keep working with any subtype. Subtypes may not strengthen preconditions or weaken the promises of the base. The standard counter-example is a `Square` that extends a mutable `Rectangle`:

Lsp.java JAVA

```java
public class Lsp {
    static class Rectangle {
        protected int w, h;
        void setWidth(int w) { this.w = w; }
        void setHeight(int h) { this.h = h; }
        int area() { return w * h; }
    }
    static class Square extends Rectangle {
        @Override void setWidth(int w) { this.w = w; this.h = w; }
        @Override void setHeight(int h) { this.w = h; this.h = h; }
    }
    static void stretch(Rectangle r) {
        r.setWidth(5);
        r.setHeight(2);
        System.out.println("expected 10, got " + r.area());
    }
    public static void main(String[] args) {
        stretch(new Rectangle());
        stretch(new Square());
    }
}
```

Output TERMINAL

```
$ java Lsp.java
expected 10, got 10
expected 10, got 4
```

Mathematically a square is a rectangle, but a *mutable* square does not honour the contract "width and height change independently". The fix is not cleverer code but a different model: make shapes immutable values (a `Shape` interface with `area()`; separate `Rectangle` and `Square` records), so nobody can observe the broken promise. Other Liskov violations to avoid: an override that throws `UnsupportedOperationException` for an inherited method, and an override that returns or accepts narrower values than documented.

I: Interface Segregation

Prefer several small interfaces to one large one, so a class is never forced to implement methods it does not need. `Comparable`, `Runnable`, `Iterable` and `AutoCloseable` are one-method interfaces for exactly this reason, and a class can implement as many as it needs. A 15-method `DataStore` interface that every fake has to implement fully is a smell.

> **NOTE**
> Do not create an interface for every class "just in case". One implementation and no prospect of another is a reason to skip the interface. Introduce one when you have a second implementation, a need to substitute in tests, or a module boundary to protect.

3.13.3 Immutability checklist

An **immutable** object cannot change after construction. Immutable objects can be shared freely between threads and callers with no locking, are safe as `HashMap` keys and `HashSet` elements, and cannot be left in a half-updated state. `String`, the wrappers, `BigDecimal` and the `java.time` types are immutable, and so are records (shallowly, as the records chapter explained). To write your own class:

1. Declare the class `final` (or give it only private constructors) so a subclass cannot add mutability.
2. Make all fields `private final`.
3. Provide no setters or other mutators. "Change" operations return a new object (a "wither": `loan.withDue(...)`).
4. Initialise every field in the constructor, validating arguments there, so the object is never visible in an invalid state.
5. **Defensive copy** every mutable argument on the way in, and every mutable field on the way out (or expose only an unmodifiable view).
6. Do not let `this` escape from the constructor (do not register `this` as a listener or start a thread in it).

The following program applies the checklist by hand and shows what a record does and does not do for you:

Imm.java JAVA

```
import java.time.LocalDate;
import java.util.ArrayList;
import java.util.List;

public class Imm {
    // Immutable by the checklist
    static final class Loan {
        private final String member;
        private final LocalDate due;
        private final List<String> notes;
        Loan(String member, LocalDate due, List<String> notes) {
            this.member = member;
            this.due = due;
            this.notes = List.copyOf(notes);          // defensive copy, unmodifiable
        }
        String member() { return member; }
        LocalDate due() { return due; }
        List<String> notes() { return notes; }
        Loan withDue(LocalDate newDue) { return new Loan(member, newDue, notes); }
    }

    // Leaky: record does not copy mutable components
    record LeakyLoan(String member, List<String> notes) {}
    record SafeLoan(String member, List<String> notes) {
        SafeLoan { notes = List.copyOf(notes); }
    }

    public static void main(String[] args) {
        List<String> n = new ArrayList<>(List.of("fragile spine"));
        Loan loan = new Loan("Asha", LocalDate.of(2026, 11, 1), n);
        n.add("late once");
        System.out.println(loan.notes());
        try { loan.notes().add("x"); } catch (UnsupportedOperationException e) {
            System.out.println("UOE");
        }
        Loan later = loan.withDue(LocalDate.of(2026, 11, 15));
        System.out.println(loan.due() + " -> " + later.due());

        List<String> shared = new ArrayList<>(List.of("a"));
        LeakyLoan leaky = new LeakyLoan("Ravi", shared);
        SafeLoan safe = new SafeLoan("Ravi", shared);
        shared.add("b");
        System.out.println(leaky.notes() + " vs " + safe.notes());
        System.out.println(new SafeLoan("Ravi", List.of("a")).equals(safe) + " "
            + (new SafeLoan("Ravi", List.of("a")).hashCode() == safe.hashCode()));
    }
}
```

Output TERMINAL

```
$ java Imm.java
[fragile spine]
UOE
2026-11-01 -> 2026-11-15
[a, b] vs [a]
true true
```

What to notice: the caller's later `n.add("late once")` did not leak into `loan` because the constructor copied the list. `List.copyOf` returns an unmodifiable list, so even `loan.notes().add(...)` fails. The `LeakyLoan` record saw the caller's later change (`[a, b]`) because a record stores the reference it is given; the compact constructor in `SafeLoan` fixed it with one line. `List.copyOf` (Java 10) has a bonus: if the argument is already an immutable list from `List.of`, it returns it without copying.

GOTCHA

`final` on a field makes the *reference* constant, not the object behind it. A `private final List<String> notes` can still be mutated through `notes.add(...)` unless the list itself is unmodifiable. Immutability is a property of the whole object graph reachable from the fields.

3.13.4 equals and hashCode: let records do it

Hand-written `equals`/`hashCode` is easy to get wrong (the Object class chapter showed how a missing `hashCode` breaks `HashMap`). For classes that are plain **values** (a point, money, a customer id, a library loan), make them records: the compiler generates `equals`, `hashCode` and `toString` from all components, consistently and with no maintenance as fields change. Two cautions. Records compare components with `equals`, so an array component is compared by identity; use a `List` instead. And do not make an object with identity (a bank account whose balance changes, a JPA entity with a generated id) a record: two accounts with the same balance are not the same account. Mutable entities keep an explicit `equals` based on a stable business key or id, or they inherit identity equality, which is fine when two distinct objects should never be equal.

If you do write `equals` by hand, the safe recipe on a final class is: check `this == o`, check the type (`o instanceof Loan other`, or `getClass()` if the class is extensible), compare fields with `Objects.equals` (or `Double.compare` for floats), and compute `hashCode` from exactly the same fields using `Objects.hash(...)`.

3.13.5 API design smells

Classes are used by other people (and by you in three months). Common smells, with the fix:

Smell	Better
Boolean flag parameters: `make(t, true, false, 2)`	Named options type, an enum, or two methods with clear names.
Long parameter lists (5 or more)	A record or builder grouping related values.
Returning `null` for "nothing"	`Optional<T>` for a single result, an empty collection for many. Never return `null` for a collection.
Magic ints and strings for modes	An `enum` (Format.PDF) or a sealed type.
Public mutable fields or exposed internal lists	Private fields, copies or unmodifiable views, behaviour methods instead of getters and setters.
Setters that allow a partly built object	Constructor or builder that produces a valid object in one step.
Class that is "data plus a manager" (an `OrderUtil` full of static methods acting on a passive `Order`)	Put behaviour with the data it uses (the "tell, do not ask" idea).
Catch-all `Exception` or `Object` in signatures	The most specific type; generics instead of casts.
Inheritance only to reuse code	Composition, interface plus helper.

Smells in class and method design.

Here the first three smells are fixed in a few lines. Notice that the "before" version compiles and returns `null` to the caller:

Smells.java JAVA

```java
import java.util.List;
import java.util.Optional;

public class Smells {
    enum Format { PDF, CSV }
    record Report(String title) {}

    // BAD: boolean flags, null returns, magic ints, long parameter list
    static Report make(String t, boolean landscape, boolean color, int fmt) { return null; }

    // BETTER: named types, no null, small surface
    record Options(boolean landscape, boolean color, Format format) {
        static Options defaults() { return new Options(false, true, Format.PDF); }
        Options withLandscape() { return new Options(true, color, format); }
    }
    static Report make(String title, Options o) { return new Report(title + " as " + o.format()); }

    static Optional<Report> find(List<Report> all, String title) {
        return all.stream().filter(r -> r.title().equals(title)).findFirst();
    }

    public static void main(String[] args) {
        System.out.println(make("Sales", Options.defaults().withLandscape()));
        System.out.println(find(List.of(new Report("Sales")), "Costs")
            .map(Report::title).orElse("none"));
        System.out.println(make("Q", true, false, 2));
    }
}
```

Output TERMINAL

```
$ java Smells.java
Report[title=Sales as PDF]
none
null
```

The call `make("Q", true, false, 2)` is unreadable (what is `2`? which boolean is colour?) and silently returned `null`; the call `make("Sales", Options.defaults().withLandscape())` states its intent. A tip learned the hard way: a record accessor already uses the component name (`landscape()`), so a method that builds a modified copy cannot reuse that name; the convention is `withLandscape()`.

3.13.6 A short checklist for a new class

- Can I state what this class is for in one sentence, with no "and"?
- Is every field `private`, and `final` unless there is a reason? Could this be a record or an enum?
- Does the constructor leave the object valid? Are arguments validated and mutable ones copied?
- Do callers depend on an interface or on this concrete type? Do I take my dependencies through the constructor?
- Is it `final` or documented for extension? If it extends something, is it really an is-a and does it honour the parent's contract?
- Are `equals`, `hashCode`, `toString` consistent and complete (or generated)?
- Could a caller misuse it with a `null`, a negative number, or the wrong call order? Can I make that impossible in the types?

REAL WORLD
In a Spring Boot service these ideas show up everywhere: services receive repositories and clients through constructor injection (dependency inversion), request and response bodies are records (immutable DTOs), entities use identity equality or an id-based `equals`, and "enum instead of a string status" prevents a class of data bugs. Reviewers routinely reject long boolean-flag methods and `return null` for collections.

Check yourself

1. Why does the `CountingHashSet` in this chapter report 6 for three elements?
2. A class has `private final List<String> items` and a getter `return items;`. Is it immutable? Give one fix.
3. What is wrong with this record? `record Course(String name, String[] tags) {}` (hint: think of `equals`).

4. A `Penguin` extends `Bird`, which has `fly()`. `Penguin.fly()` throws `UnsupportedOperationException`. Which principle does this break, and what is a better design?
5. What does `new SafeLoan("Ravi", List.of("a")).equals(safe)` need to be true, and why did it work in this chapter?
6. Name two smells in `void export(String file, boolean zip, boolean overwrite, int type)`.

Answers

1. `addAll` adds `c.size()` and then `HashSet`'s inherited `addAll` calls the overridden `add` for each element, which counts them again.
2. No: callers can modify the list through the getter. Return `List.copyOf(items)` or `Collections.unmodifiableList(items)`, and copy the list in the constructor too.
3. A record compares components with `equals`, and arrays use identity, so two `Course` objects with equal tag contents are not equal and `hashCode` differs. Use `List<String>` (copied in a compact constructor).
4. Liskov Substitution: code that accepts a `Bird` cannot rely on `fly()`. Model flying separately (a `FlyingBird` interface or a capability), so only birds that fly promise it.
5. All components must be equal: the same member and equal notes. The compact constructor copied the list, and `List.equals` compares contents, so the lists with the same elements are equal.
6. Boolean flag parameters, a magic `int` mode, and a long parameter list. Use an options record and an enum.

KEY POINTS

- Prefer composition to inheritance; inherit only for a real is-a relationship in a class designed for extension, otherwise make the class `final`.
- Depend on small interfaces and receive collaborators through the constructor (SOLID: single responsibility, open/closed, Liskov, interface segregation, dependency inversion).
- Immutability checklist: final class, private final fields, no mutators, validate in the constructor, defensive copies in and out.
- Use records for plain values so `equals`, `hashCode` and `toString` are generated; keep identity-based equality for mutable entities.
- Avoid API smells: boolean flags, long parameter lists, null returns, magic constants and exposed internals.

PART 4

Strings, Text and Time

Text is the most common data your programs handle, and time is the most error-prone. This Part makes both predictable.

In this part

4.1 Strings in depth
4.2 Formatting, regex and text
4.3 Unicode, charsets and Locale
4.4 Numbers beyond primitives
4.5 Date and time with java.time

4.1 Strings in depth

Every Java program is full of strings: names, log lines, JSON, SQL, keys of maps. Because they are everywhere, the design of `java.lang.String` shapes the speed and memory of whole applications. Interviewers love this class for the same reason: a handful of simple facts (immutability, the pool, `==` versus `equals`) explain a surprising number of bugs. In this chapter you will learn what a `String` really is in memory, why it is immutable, how the string pool works, what `+` compiles to, when to use `StringBuilder`, and the methods you should know by heart.

4.1.1 What a String is

A `String` is an immutable sequence of UTF-16 code units. Immutable means that once the object exists, no method can change its content. Every method that looks like a modification (`toUpperCase`, `replace`, `concat`, `substring`) returns a new `String` and leaves the original alone.

Immut.java JAVA

```java
public class Immut {
    public static void main(String[] args) {
        String s = "hello";
        String t = s.toUpperCase();   // new object
        s.concat(" world");           // result thrown away!
        System.out.println(s + " " + t);
        s = s.concat(" world");       // reassign the variable, not the object
        System.out.println(s);
    }
}
```

Output TERMINAL

```
$ java Immut.java
hello HELLO
hello world
```

The second line of `main` is the classic beginner bug: calling `concat` and ignoring the result does nothing. The variable `s` is a reference; the last line moves the reference to a new object. The old `"hello"` object is untouched (and becomes garbage if nothing else points to it).

Why immutable?

The designers chose immutability for reasons that still matter today:

- **Safety as a map key and in sets.** `hashCode` is computed from the content. If content could change, an entry would be lost inside a `HashMap` after mutation. `String` also caches its hash in a field, so repeated lookups are cheap.
- **Security.** Class names, file paths, and network host names are strings. A checked path cannot be changed by another thread between the check and the use.
- **Thread safety for free.** Any number of threads can share a `String` without locks.
- **Sharing.** The JVM can share one object between all places that use the same literal (the string pool, below).

4.1.2 Inside the object: byte[] and coder

Until Java 8, a `String` held a `char[]`, two bytes per character. Most strings in real programs are Latin-1 (English letters, digits, punctuation), so half the memory was wasted. **Compact strings** (JEP 254, Java 9) changed the internals. A `String` now holds:

- `value`: a `byte[]` with the characters.
- `coder`: a byte, `0` = LATIN1 (one byte per char) or `1` = UTF16 (two bytes per char).
- `hash`: the cached hash code (and a flag saying whether it has been computed).

If every character of the string is in the range U+0000 to U+00FF, the JVM stores one byte per character; if even one character is outside that range, the whole string is stored as UTF-16. The feature is on by default (`-XX:+CompactStrings`); the API behaves identically, so you never see it except as lower memory use. You can confirm the flag on your machine:

Check the flag TERMINAL

```
$ java -XX:+PrintFlagsFinal -version | grep CompactStrings
     bool CompactStrings         = true         {pd product} {default}
```

The cost of the feature is a tiny check on each operation to see which encoding applies; the benefit is that heaps full of ASCII text (logs, JSON, identifiers) shrink noticeably. `length()` still counts UTF-16 code units, not bytes.

> **NOTE**
> You can find the private `coder` field with reflection, but `setAccessible(true)` on it fails on a modern JDK because of strong encapsulation (JEP 403, Java 17). That is deliberate. Use `getBytes(StandardCharsets.ISO_8859_1).length` or a heap analyser if you must measure.

4.1.3 The string pool and intern

Programs create the same literal many times: `"OK"`, `"id"`, `"GET"`. The JVM keeps a table of unique strings called the **string pool** (or string intern table). When the class loader resolves a string literal, the JVM looks it up in the pool: if an equal string is there, the existing object is used; if not, a new one is created and added. Since Java 7 the pool lives in the ordinary heap (before that it sat in PermGen), so pooled strings can be garbage-collected when nothing refers to them. The table is a hash table whose size is tunable with `-XX:StringTableSize` (the default here is 65536 buckets).

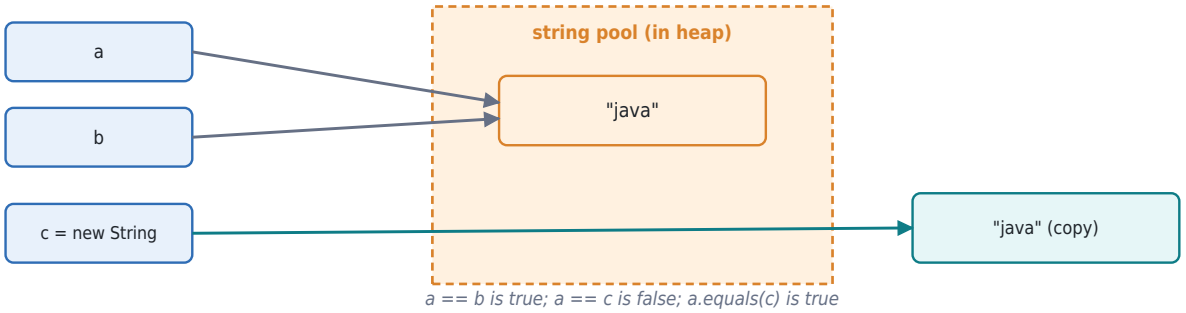


Figure 4.1. Two literals share one pooled object; new String creates a separate heap object with the same content.

Four expressions create strings in different ways. Run this and predict each line before you look at the answer:

PoolDemo.java JAVA

```
public class PoolDemo {
    public static void main(String[] args) {
        String a = "java";
        String b = "java";
        String c = new String("java");
        String d = c.intern();
        String part = "ja";
        String e = part + "va";
        String f = "ja" + "va";
        System.out.println("a == b          : " + (a == b));
        System.out.println("a == c          : " + (a == c));
        System.out.println("a.equals(c)     : " + a.equals(c));
        System.out.println("a == d (intern) : " + (a == d));
```

PoolDemo.java (continued) JAVA

```
        System.out.println("a == e (runtime): " + (a == e));
        System.out.println("a == f (const)  : " + (a == f));
    }
}
```

Output TERMINAL

```
$ java PoolDemo.java
a == b          : true
a == c          : false
a.equals(c)     : true
a == d (intern) : true
a == e (runtime): false
a == f (const)  : true
```

What to notice:

- `a == b`: both literals resolve to the same pooled object.
- `new String("java")` always allocates a fresh object, so `a == c` is false although the content is equal. `new String(String)` is almost never useful; it only creates garbage.
- `intern()` returns the pooled instance with that content (adding one if needed), so `d` is the same object as `a`.
- `"ja" + "va"` is a **constant expression**: the compiler folds it into the literal `"java"` at compile time. `part + "va"` involves a non-final variable, so it is computed at run time and produces a new object. (If `part` were declared `final String part = "ja";` it would be a constant variable and `e` would be pooled too.)

== versus equals

`==` on references asks "are these the same object?". `equals` asks "do these have the same content?". For strings you almost always want `equals`. Comparing with `==` seems to work in tests because literals are pooled, and then fails in production when the value comes from user input, a file, or a database. To avoid a `NullPointerException`, write the literal first: `"admin".equals(role)`, or use `java.util.Objects.equals(a, b)`.

GOTCHA
Never rely on `==` for strings, even "just this once". Whether two equal strings are the same object depends on how each was created. Static analysers flag it for a reason.

When is intern() worth it?

`intern()` can save memory when you load millions of strings drawn from a small set of values (country codes, status names) and keep them for a long time. It has costs: a lookup in the global table on every call, and the table is a shared resource. In most code a `HashMap<String,String>` of your own, or simply not worrying, is better. Modern JVMs also offer string deduplication in G1 (`-XX:+UseStringDeduplication`), which shares the `byte[]` of equal strings during garbage collection without changing identity.

INTERVIEW ANGLE
How many objects does `String s = new String("abc");` create? At most two: the literal `"abc"` in the pool (created when the class is resolved, only if not already there) and one new `String` object on the heap. `s` refers to the heap object. If `"abc"` was already pooled, only one new object is created by this statement.

4.1.4 Concatenation: + and what it compiles to

The `+` operator on strings is syntax sugar; the compiler decides what to emit. Before Java 9 `javac` generated a hidden `StringBuilder` and chained `append` calls. Since Java 9 (JEP 280) it emits a single `invokedynamic` instruction that calls `StringConcatFactory`. The JVM picks the best strategy at the first execution: it can compute the exact final length, allocate the right array once, and copy the parts, which beats a `StringBuilder` chain. You can see it in bytecode:

Indy.java JAVA

```
public class Indy {
    static String f(String a, int b) { return "id=" + a + ", n=" + b; }
}
```

Trimmed javap output TERMINAL

```
$ javac Indy.java && javap -c -p Indy
  static java.lang.String f(java.lang.String, int);
    Code:
       0: aload_0
       1: iload_1
       2: invokedynamic #7,  0   // InvokeDynamic #0:makeConcatWithConstants:(...)
       7: areturn
```

The constant parts of the expression (`"id="` and `", n="`) are stored once as a recipe string in the constant pool, with placeholders for the two arguments. Target 8 (`javac --release 8`) still produces the `StringBuilder` form because indy concatenation needs a Java 9+ runtime.

4.1.5 StringBuilder and StringBuffer

A single `+` is fine. The trouble is `+` (or `+=`) in a loop: each iteration creates a new `String` and copies everything built so far, so building a string of length n costs O(n squared) character copies. `StringBuilder` is a mutable buffer: `append` writes into spare capacity and only grows the array occasionally (the new capacity is roughly double plus 2).

Perf.java JAVA

```java
public class Perf {
    static String plus(int n) {
        String s = "";
        for (int i = 0; i < n; i++) s += "x";
        return s;
    }
    static String sb(int n) {
        StringBuilder b = new StringBuilder();
        for (int i = 0; i < n; i++) b.append('x');
        return b.toString();
    }
    public static void main(String[] args) {
        for (int w = 0; w < 3; w++) { plus(2000); sb(2000); }   // warm up the JIT
        for (int n : new int[]{10_000, 50_000, 100_000}) {
            long t0 = System.nanoTime(); plus(n);
            long t1 = System.nanoTime(); sb(n);
            long t2 = System.nanoTime();
            System.out.printf("n=%,d  +=: %d ms   StringBuilder: %d ms%n",
                              n, (t1 - t0) / 1_000_000, (t2 - t1) / 1_000_000);
        }
    }
}
```

Typical output (measured on the build machine; your numbers will differ) TERMINAL

```
$ java Perf.java
n=10,000  +=: 63 ms   StringBuilder: 0 ms
n=50,000  +=: 1083 ms   StringBuilder: 2 ms
n=100,000  +=: 1783 ms   StringBuilder: 4 ms
```

Look at the shape, not the exact milliseconds: doubling the input makes `+=` roughly quadratic, while `StringBuilder` stays linear. This is a crude measurement (no JMH, one run); the conclusion is the well-known one.

You can watch the buffer grow. A default `StringBuilder` starts with capacity 16 and, on overflow, grows to roughly `old * 2 + 2`:

Observed capacity while appending one char at a time

```
initial capacity 16
len 17 -> capacity 34
len 35 -> capacity 70
len 71 -> capacity 142
```

If you know the approximate final size, pass it to the constructor, `new StringBuilder(4096)`, and avoid the copies.

Class	Mutable?	Thread-safe?
`String`	no	yes (immutable)
`StringBuilder`	yes	no: use inside one thread (Java 5+)
`StringBuffer`	yes	yes: every method is synchronized (legacy, Java 1.0)

String, StringBuilder and StringBuffer compared.

`StringBuffer` is the old thread-safe variant. Its per-call locking is almost never what you want: a buffer shared between threads rarely makes sense, and the lock does not make a sequence of calls atomic. Use `StringBuilder` unless you are maintaining old code. Useful `StringBuilder` methods: `append`, `insert`, `delete`, `deleteCharAt`, `reverse`, `setCharAt`, `setLength`, `indexOf`, `toString`.

> **TIP**
> A loop that builds output with `+=` can often be replaced by `String.join(", ", list)`, `Collectors.joining`, or `String.repeat`. They use a builder internally and say what you mean.

4.1.6 The methods you should know

The next listing exercises the methods that appear most often in code and interviews. Read each line, predict the result, then compare with the output.

Methods.java JAVA

```java
import java.util.*;

public class Methods {
    public static void main(String[] args) {
        String s = "\u2003 Hello, Java  \n";       // starts with an EM SPACE
        System.out.println(s.trim().length() + " " + s.strip().length());
        System.out.println(" ".isBlank() + " " + "".isEmpty());
        System.out.println("ab".repeat(3));
        System.out.println("a\nb\r\nc".lines().toList());
        System.out.println("%s scored %.1f".formatted("Asha", 91.456));
        System.out.println(String.join("|", List.of("a", "b", "c")));
        System.out.println("a,b,,c,,".split(",").length);
        System.out.println("a,b,,c,,".split(",", -1).length);
        System.out.println(Arrays.toString("a1b22c".split("\\d+")));
        System.out.println(Arrays.toString("".split(",")) + " " + "".split(",").length);
        System.out.println(Arrays.toString("a.b".split(".")));
        System.out.println("banana".indexOf("an", 2) + " " + "banana".lastIndexOf('a'));
        System.out.println("Java".compareTo("Jazz") + " " + "a".compareToIgnoreCase("B"));
        System.out.println("hello".hashCode() + " " + "Aa".hashCode() + " " + "BB".hashCode());
        System.out.println(String.valueOf((Object) null) + " " + "x" + null);
        String r = "Hello".transform(x -> x + "!");
        System.out.println(r);
    }
}
```

Output TERMINAL

```
$ java Methods.java
13 11
true true
ababab
[a, b, c]
Asha scored 91.5
a|b|c
4
6
[a, b, c]
[] 1
[]
3 5
-4 -1
99162322 2112 2112
null xnull
Hello!
```

Points worth learning from that output:

- `trim` **vs** `strip`**.** `trim()` (Java 1.0) removes only characters up to U+0020. `strip()` (Java 11) removes all Unicode whitespace as defined by `Character.isWhitespace`, including the em space, so it trimmed two more characters here. Prefer `strip`, `stripLeading`, `stripTrailing`. `isBlank()` is true for empty or whitespace-only strings.
- `lines()` splits on `\n`, `\r`, and `\r\n` and returns a `Stream<String>`.
- `split` **drops trailing empty strings.** `"a,b,,c,,".split(",")` gives 4 parts, not 6; pass a negative limit to keep them. The pattern is a regular expression, so `split(".")` matches every character and returns nothing; use

`split("\\.")` for a literal dot. `"".split(",")` returns an array with one empty string, not an empty array. More on regex in the next chapter.

- `compareTo` returns the difference of the first differing chars (`'v' - 'z'` = -4); rely only on its sign.
- **Equal hash codes do not imply equal strings.** `"Aa"` and `"BB"` collide. The formula is `s[0]*31^(n-1) + ... + s[n-1]`, which is exactly the `hashCode` that `HashMap` buckets use.
- `"x" + null` gives `"xnull"`: concatenation converts `null` to the text `null`.

Other everyday methods: `charAt`, `substring(begin, end)` (end exclusive; copies, not a view, since Java 7u6), `contains`, `startsWith`, `endsWith`, `replace(CharSequence, CharSequence)` (literal), `replaceAll` (regex), `toLowerCase`, `valueOf`, `chars()`, `toCharArray`, `indent`, `stripIndent`, `translateEscapes`, and `String.valueOf` for turning anything into text.

4.1.7 switch on strings

Since Java 7 a `switch` can select on a `String`. It uses `equals` semantics (internally a `hashCode` switch followed by an `equals` check), so it is case-sensitive and a `null` selector throws `NullPointerException`. With Java 14 switch expressions the code is short:

SwitchOnString.java — JAVA

```java
public class SwitchOnString {
    static int level(String day) {
        return switch (day) {
            case "MON", "TUE" -> 1;
            case "WED"        -> 2;
            default           -> 0;
        };
    }
    public static void main(String[] args) {
        System.out.println(level("TUE") + " " + level("WED") + " " + level("tue"));
    }
}
```

Output — TERMINAL

```
$ java SwitchOnString.java
1 2 0
```

Case labels must be constant expressions, so you cannot use a variable as a label. Normalise the selector (`day.strip().toUpperCase(Locale.ROOT)`) when input comes from users; the chapter on Unicode and Locale explains why `Locale.ROOT` matters there.

REAL WORLD

In Spring applications you meet strings mostly at boundaries: request parameters, JSON fields, SQL text, log messages. Two habits pay off. First, log with placeholders (`log.info("order {} failed", id)`) so no string is built when the level is disabled. Second, never build SQL by concatenating user input; use prepared statements. The concatenation itself is cheap, but the injection risk is not.

4.1.8 Strings in everyday text processing

Most string work is a small number of recurring recipes. The program below counts word frequencies in a text block, builds a delimited string three ways, and shows the `split` edge cases that surprise people.

Wf.java — JAVA

```java
import java.util.Arrays;
import java.util.Comparator;
import java.util.Map;
import java.util.StringJoiner;
import java.util.TreeMap;
import java.util.function.Function;
import java.util.stream.Collectors;

public class Wf {
    public static void main(String[] args) {
        String text = """
            To be, or not to be, that is the question:
            Whether 'tis nobler in the mind to suffer
            """;
```

Wf.java (continued) JAVA

```
        Map<String, Long> freq = Arrays.stream(text.toLowerCase().split("\\W+"))
            .filter(w -> !w.isBlank())
            .collect(Collectors.groupingBy(Function.identity(), TreeMap::new,
                Collectors.counting()));
        System.out.println(freq);
        String top = freq.entrySet().stream()
            .sorted(Map.Entry.<String, Long>comparingByValue().reversed()
                .thenComparing(Map.Entry.comparingByKey()))
            .limit(3)
            .map(e -> e.getKey() + "=" + e.getValue())
            .collect(Collectors.joining(", ", "[", "]"));
        System.out.println(top);

        StringJoiner sj = new StringJoiner(", ", "{", "}");
        sj.setEmptyValue("EMPTY");
        System.out.println(sj);
        sj.add("a").add("b");
        System.out.println(sj + " " + String.join("|", "x", "y", "z"));

        System.out.println("hello".chars().filter(c -> c == 'l').count());
        System.out.println("hello".chars().mapToObj(c -> String.valueOf((char) c))
            .collect(Collectors.joining("-")));
        System.out.println(new StringBuilder("racecar").reverse().toString().equals("racecar"));

        // hash caching
        String k = "Aa";
        String k2 = "BB";
        System.out.println(k.hashCode() + " " + k2.hashCode() + " " + k.equals(k2));
        char[] pw = {'s', '3', 'c'};
        Arrays.fill(pw, '\0');
        System.out.println((int) pw[0]);
        System.out.println("a,b,,c,,".split(",").length + " " + "a,b,,c,,".split(",", -1).length
            + " " + Arrays.toString(" x  y ".split(" ")));
    }
}
```

Output (the first line is one line in reality; wrapped here) TERMINAL

```
$ java Wf.java
{be=2, in=1, is=1, mind=1, nobler=1, not=1, or=1, question=1,
 suffer=1, that=1, the=2, tis=1, to=3, whether=1}
[to=3, be=2, the=2]
EMPTY
{a, b} x|y|z
2
h-e-l-l-o
true
2112 2112 false
0
4 6 [, x, , y]
```

What to notice

- `split("\\W+")` splits on runs of non-word characters. The regex is a `String`, so the backslash is doubled. A text block (`"""`) keeps the original line breaks, which `\W+` also treats as separators.
- `Collectors.groupingBy(..., TreeMap::new, counting())` gives a sorted frequency table in one pipeline; Part 7 explains collectors.
- `String.join`, `StringJoiner` and `Collectors.joining` all insert a delimiter *between* elements, so there is no trailing comma to trim. `StringJoiner` also supports a prefix, a suffix and a value for the empty case.
- `chars()` yields an `IntStream` of UTF-16 code units; cast to `char` to rebuild text. Use `codePoints()` when characters outside the BMP are possible (see the Unicode chapter).
- `"Aa"` and `"BB"` have the same `hashCode` (2112) but are not equal. Equal hash codes do not imply equal objects, which is exactly why `HashMap` still calls `equals`. `String` caches its hash code in a field after the first call, so using strings as map keys is cheap.
- `"a,b,,c,,".split(",")` returns 4 elements, because trailing empty strings are removed; the limit `-1` keeps them (6). A leading separator produces a leading empty string (`" x y ".split(" ")` gives `["", "x", "", "y"]`). Prefer `split("\\s+")` after `strip()` for whitespace.

Passwords and char arrays

Because a `String` is immutable, a password held in one stays in memory until garbage collection, and may be visible in a heap dump or in the string pool if it came from a literal. Security-sensitive APIs (`KeyStore.load`, `PBEKeySpec`, `Console.readPassword`) therefore take `char[]`, which you can overwrite with `Arrays.fill(pw, '\0')` as soon as you are done (the last printed `0` above is the code of that overwritten character). This is a mitigation, not a guarantee: the framework that read the password from an HTTP request has often already made `String` copies. Still, never log a password and never keep one in a field longer than necessary.

TIP

Use `isBlank()` to test for empty or whitespace-only text and `strip()` (not `trim()`) to remove whitespace; `strip` understands Unicode white space, `trim` only removes characters up to U+0020. Compare with a constant first to avoid null checks: `"yes".equals(answer)` never throws when `answer` is `null`.

Check yourself

1. What does this print? `String s = "a"; s.concat("b"); s.toUpperCase(); System.out.println(s);`
2. Given `String x = "hi"; String y = new String("hi");`, what are `x == y`, `x.equals(y)` and `x == y.intern()`?
3. Why is `for (...) s += item;` slow for large inputs, and what do you use instead?
4. What does `"a,,b,,".split(",").length` return? How would you keep the trailing empty parts?
5. Name two reasons `String` is immutable and one consequence for the string pool.
6. What is the difference between `trim()` and `strip()`, and between `StringBuilder` and `StringBuffer`?

Answers

1. It prints `a`. Both calls return new strings that are discarded; `String` never changes in place.
2. `false`, `true`, `true`. `==` compares references; `intern()` returns the pooled instance, which is the same object as the literal `x`.
3. Each `+=` allocates a new string and copies all earlier characters, so the total work is quadratic. Use `StringBuilder` (or `String.join`, `Collectors.joining`).
4. `3` (parts `a`, empty, `b`; trailing empties are dropped). Use `split(",", -1)` to keep them (gives 5).
5. Safe map keys / cached hash, security of names and paths, thread safety. Because content never changes, the JVM can safely share one pooled object among all uses of a literal.
6. `trim` removes chars up to U+0020 only; `strip` (Java 11) removes all Unicode whitespace. `StringBuffer` is synchronized and legacy; `StringBuilder` is the unsynchronised, faster default.

KEY POINTS

A `String` is immutable; "modifying" methods return new objects, so always use their return value.

- Since Java 9 a `String` stores a `byte[]` plus a `coder` (Latin-1 or UTF-16), so ASCII text costs one byte per char.
- Literals and compile-time constants are pooled; `new String(...)` and run-time concatenation are not. Compare content with `equals`, never `==`.
- `+` compiles to `invokedynamic` (`StringConcatFactory`) since Java 9; inside loops use `StringBuilder` to avoid quadratic copying.
- Prefer `strip` over `trim`, `StringBuilder` over `StringBuffer`; remember that `split` takes a regex and drops trailing empties.

4.2 Formatting, regex and text

Strings are rarely built by hand. Reports need aligned columns and two decimals; input needs to be validated and pulled apart; multi-line text (JSON, SQL, HTML) needs to sit in source code without a thicket of escapes. Java has a format mini-language (`printf`), a regular-expression engine (`java.util.regex`), and text blocks for exactly these jobs. This chapter shows each, with the traps that cost people hours.

4.2.1 Formatting with printf and Formatter

`String.format`, `System.out.printf`, `PrintStream.format`, `String.formatted` (Java 15) and `java.util.Formatter` all share one syntax: a format string with specifiers of the form `%[argument_index$][flags][width][.precision]conversion`. Each specifier consumes one argument (unless you give an index such as `%2$s`).

Conversion	Meaning	Example result
%d	integer (int, long, BigInteger)	42
%f	floating point, 6 decimals by default	3.141593
%e	scientific notation	1.234568e+04
%s	any object via String.valueOf; %S upper-cases	java
%c	a single character	J
%b	boolean (true unless null/false)	true
%x %X %o	hexadecimal, octal	ff
%n	platform line separator (use instead of)	newline
%%	a literal percent sign	%

The conversions you will use most.

Flags and width tune the layout: `-` left-justify, `0` pad with zeros, `+` always show a sign, `,` group digits, `(` negative numbers in parentheses, `#` alternate form. Width is the minimum field width; precision after the dot means decimals for `%f` and a maximum length for `%s`.

Fmt.java JAVA

```java
import java.util.Locale;

public class Fmt {
    public static void main(String[] args) {
        System.out.printf("[%5d] [%-5d] [%05d] [%,d] [%+d]%n", 42, 42, 42, 1234567, 42);
        System.out.printf("[%8.3f] [%.0f] [%e] [%,.2f]%n", Math.PI, 2.5, 12345.678, 9876543.215);
        System.out.printf("[%s] [%10s] [%-10s] [%.3s] [%S]%n",
            "java", "java", "java", "java", "java");
        System.out.printf("[%x] [%X] [%o] [%08.2f] [%b] [%c] [%%]%n",
            255, 255, 8, 3.14159, true, 'J');
        System.out.printf("%2$s %1$s %2$s%n", "a", "b");
        System.out.println(String.format(Locale.GERMANY, "%,.2f", 1234567.891));
        System.out.println(String.format(Locale.US, "%,.2f", 1234567.891));
        System.out.println(String.format("%.2f %.2f %.0f %.0f", 0.125, 0.135, 0.5, 1.5));
        try { System.out.printf("%d%n", 3.5); } catch (Exception e) { System.out.println(e); }
        try { System.out.printf("%s %s%n", "one"); } catch (Exception e) { System.out.println(e); }
    }
}
```

Output TERMINAL

```
$ java Fmt.java
[   42] [42   ] [00042] [1,234,567] [+42]
[   3.142] [3] [1.234568e+04] [9,876,543.22]
[java] [      java] [java      ] [jav] [JAVA]
[ff] [FF] [10] [00003.14] [true] [J] [%]
b a b
1.234.567,89
1,234,567.89
0.13 0.14 1 2
java.util.IllegalFormatConversionException: d != java.lang.Double
one java.util.MissingFormatArgumentException: Format specifier '%s'
```

What to notice:

- **Locale matters.** Without an explicit `Locale`, `format` uses the default locale of the JVM (`Locale.getDefault(Locale.Category.FORMAT)`), so the same code prints `1.234.567,89` on a German machine. For machine-readable output (files, JSON, logs parsed by tools) pass `Locale.ROOT` or `Locale.US`.
- **Rounding is HALF_UP**, not banker's rounding: `0.125` prints `0.13`, `2.5` with `%.0f` gives `3`. It rounds the decimal expansion of the binary double, which is why `0.135` (really 0.13500000000000000888...) gives `0.14`. Money does not belong in `double`; see the chapter on BigDecimal.
- **Wrong arguments fail at run time**, not compile time: `%d` with a double throws `IllegalFormatConversionException`, too few arguments throw `MissingFormatArgumentException` (note the partial output `one` was already written). IDE inspections and `-Xlint` do not catch all of these.
- Use `%n`, not `\n`, in `printf` strings for portable line endings.

`Formatter` itself is a class that writes formatted text to any `Appendable` (a `StringBuilder`, a file, a `PrintStream`). You rarely instantiate it, but it explains why `String.format` is relatively slow: each call parses the format string. In a hot loop prefer building with `StringBuilder`; elsewhere, readability wins.

4.2.2 Regular expressions

A **regular expression** (regex) is a compact pattern describing a set of strings. Java's engine lives in `java.util.regex` and has two main classes: `Pattern` (a compiled regex, immutable and thread-safe) and `Matcher` (the state of one match attempt on one input, not thread-safe). Compiling is expensive, so keep a `Pattern` in a `static final` field instead of calling `String.matches` or `replaceAll` in a loop (they compile the pattern every time).

The syntax you need

- Character classes: `[abc]`, `[a-z]`, `[^0-9]` (negation); shortcuts `\d` digit, `\w` word char, `\s` whitespace, `.` any char except line end. In a Java string literal every backslash is doubled: `"\\d+"`.
- Quantifiers: `*` zero or more, `+` one or more, `?` optional, `{n}`, `{n,m}`.
- Anchors: `^` start, `$` end, `\b` word boundary.
- Groups: `( ... )` captures, `(?: ... )` does not capture, `(?<name> ... )` captures with a name; `|` alternation.
- Flags: `Pattern.CASE_INSENSITIVE` or inline `(?i)`; `MULTILINE`, `DOTALL`, `UNICODE_CASE`.

Matching, finding and groups

There are three ways to apply a matcher, and mixing them up is a common bug: `matches()` requires the whole input to match, `lookingAt()` requires a match at the start only, and `find()` searches for the next match anywhere (call it in a loop).

Rx.java JAVA

```
import java.util.regex.*;

public class Rx {
    public static void main(String[] args) {
        Pattern p = Pattern.compile("(?<year>\\d{4})-(?<month>\\d{2})-(?<day>\\d{2})");
        Matcher m = p.matcher("Due 2026-10-09, paid 2026-10-12.");
        while (m.find()) {
            System.out.println(m.group() + " year=" + m.group("year")
                + " month=" + m.group(2) + " at " + m.start() + "-" + m.end());
        }
        System.out.println(Pattern.matches("\\d+", "123abc") + " "
            + Pattern.compile("\\d+").matcher("123abc").lookingAt());
        String html = "<b>bold</b> and <i>italic</i>";
        System.out.println(html.replaceAll("<.+>", "#"));
        System.out.println(html.replaceAll("<.+?>", "#"));
        System.out.println("John Smith".replaceAll("(\\w+) (\\w+)", "$2, $1"));
        System.out.println(Pattern.compile("\\d+").matcher("a1b22c333")
            .replaceAll(r -> "<" + r.group().length() + ">"));
        System.out.println(Pattern.compile("[,;]\\s*").splitAsStream("a, b;c").toList());
        System.out.println("a.b".replace(".", "-") + " " + "a.b".replaceAll(".", "-")
            + " " + Pattern.quote("a.b"));
        Matcher e = Pattern.compile("(a)(b)?").matcher("a");
        e.matches();
        System.out.println(e.groupCount() + " " + e.group(2));
    }
}
```

Output TERMINAL

```
$ java Rx.java
2026-10-09 year=2026 month=10 at 4-14
2026-10-12 year=2026 month=10 at 21-31
false true
#
#bold# and #italic#
Smith, John
a<1>b<2>c<3>
[a, b, c]
a-b --- \Qa.b\E
2 null
```

What to notice:

- **Group numbering** counts opening parentheses from left to right; group 0 is the whole match. Named groups also have numbers. A group that did not participate (`(b)?` above) returns `null`, not an empty string.
- **Greedy versus lazy.** `<.+>` is greedy: it grabs as much as possible, so the whole input from the first `<` to the last `>` became one `#`. Appending `?` (`<.+?>`) makes the quantifier lazy: it takes as little as possible, giving two tags per match. Possessive quantifiers (`.++`) never give back characters, which makes failures fast when you know backtracking cannot help.
- `replaceAll` **replacement syntax.** `$1`, `$2` refer to groups; a literal `$` or `\` in the replacement must be escaped, or use `Matcher.quoteReplacement`. The lambda form (`replaceAll(Function<MatchResult,String>)`, Java 9) avoids that problem entirely.
- `replace` **is literal,** `replaceAll` **is regex.** `"a.b".replaceAll(".", "-")` replaced every character. `Pattern.quote` wraps text so it is matched literally.

WATCH OUT

Do not parse HTML, XML, JSON or nested structures with regex. Regular expressions cannot count nesting. Use a real parser; reserve regex for flat patterns such as ids, dates and log lines.

4.2.3 Catastrophic backtracking

Java's engine is a **backtracking** matcher: when a path fails, it returns to the last choice point and tries another. If a pattern contains nested or overlapping repetition, the number of paths can grow exponentially with input length. One bad pattern on a user-supplied string can pin a CPU core for minutes. This is the **ReDoS** (regular-expression denial of service) vulnerability, and it has caused real outages.

The JDK has optimisations for some nested-group cases, so some textbook examples such as `(a+)+b` no longer explode here. The following still does. The pattern `(a?)` repeated n times followed by n times `a` can match the n-letter input only by choosing "empty" for every optional, and the engine tries all 2 to the power n combinations first.

Backtrack.java JAVA

```java
import java.util.regex.Pattern;

public class Backtrack {
    static long ms(String regex, String input) {
        long t0 = System.nanoTime();
        Pattern.compile(regex).matcher(input).matches();
        return (System.nanoTime() - t0) / 1_000_000;
    }
    public static void main(String[] args) {
        for (int n = 14; n <= 22; n += 2) {
            String bad = "(a?)".repeat(n) + "a".repeat(n);   // (a?)(a?)...aaa
            String good = "a{" + n + "}";                     // same language
            String input = "a".repeat(n);
            System.out.printf("n=%d  nested optionals: %d ms   a{n}: %d ms%n",
                              n, ms(bad, input), ms(good, input));
        }
    }
}
```

Typical output (measured on the build machine; timings vary run to run) TERMINAL

```
$ java Backtrack.java
n=14  nested optionals: 70 ms   a{n}: 0 ms
n=16  nested optionals: 116 ms   a{n}: 0 ms
n=18  nested optionals: 430 ms   a{n}: 0 ms
n=20  nested optionals: 645 ms   a{n}: 0 ms
n=22  nested optionals: 1090 ms   a{n}: 0 ms
```

Growth is clearly super-linear; in our extra runs n=26 took several seconds. Real-world culprits look like `(\w+\s?)*$` or `(.*a){12}`, usually written by someone validating an e-mail address or a URL. Defences:

1. Avoid nested quantifiers over the same characters; rewrite the pattern so each input char can be matched in only one way.
2. Use possessive quantifiers (`++`, `*+`) or atomic groups `(?>...)` when backtracking is pointless.
3. Limit input length before matching, and run regex on untrusted input with a timeout (for example in a worker whose result you abandon) because `Matcher` offers no built-in time limit. A custom `CharSequence` that checks a deadline in `charAt` is a known trick.
4. For fixed formats prefer plain parsing (`split`, `indexOf`, `LocalDate.parse`) over a clever regex.

4.2.4 Text blocks

Before Java 15, multi-line text meant `"...\n" +` on every line and escaped quotes. A **text block** (Java 15, JEP 378) starts with three double quotes followed by a line break and ends with three double quotes. The compiler strips the **incidental indentation**: the common leading whitespace of all lines, including the closing delimiter's line, is removed, so you can indent the block to match your code.

Tok.java JAVA

```java
import java.util.StringTokenizer;

public class Tok {
    public static void main(String[] args) {
        String json = """
            {
              "name": "Asha",
              "tags": ["a", "b"]
            }
            """;
        System.out.print(json);
        String t = """
            one \
            line
              indented "quotes" \t tab
            end""";
        System.out.println(t);
        StringTokenizer st = new StringTokenizer("a,b,,c", ",");
        System.out.println(st.countTokens());
    }
}
```

Output TERMINAL

```
$ java Tok.java
{
  "name": "Asha",
  "tags": ["a", "b"]
}
one line
  indented "quotes"      tab
end
3
```

The rules in short: double quotes need no escaping; a trailing `\` joins lines (no newline); `\t`, `\n` and other escapes still work; `\s` (Java 15) keeps a trailing space that would otherwise be stripped; the text ends with a newline only if the closing delimiter is on its own line (compare `json` with `t`). Trailing spaces on each line are removed. For templating use `formatted(...)`: `"""Hello %s""".formatted(name)`. String templates were previewed in Java 21 to 23 but withdrawn, so do not plan on them.

TIP
Put the closing `"""` on its own line, aligned with the code you want as column 0. Moving it left adds indentation to every line; moving it right changes nothing, because the smallest indent wins.

4.2.5 StringTokenizer and split pitfalls

`StringTokenizer` is a legacy class (its Javadoc says new code should use `split` or the regex package). Notice in the last listing that it counted 3 tokens in `"a,b,,c"`: it silently skips empty tokens, whereas `split(",")` keeps the middle empty string. Use `String.split` for simple cases and `Scanner` when you need typed tokens (`nextInt`, `nextDouble`) from a stream.

`split` has the pitfalls we already met: the delimiter is a regex (`split("|")` splits between every char, `split("\\|")` is what you meant); trailing empties are removed; a leading empty string is produced when the input starts with a delimiter (Java 8+ drops it only for a zero-width match at the start); `split(regex, limit)` caps the number of parts, and a negative limit keeps everything. When performance matters and the delimiter is a single non-special char, `split` takes a fast path that avoids the regex engine.

INTERVIEW ANGLE
What is the difference between greedy, lazy and possessive quantifiers? Greedy (`.*`) matches as much as possible and backs off one character at a time if the rest fails. Lazy (`.*?`) matches as little as possible and expands as needed. Possessive (`.*+`) matches as much as possible and never backs off, so it fails fast but can miss matches that greedy would find.

REAL WORLD
Validation of request fields in Spring (`@Pattern(regexp = "...")` from Bean Validation) compiles and runs exactly this `java.util.regex` engine on user input. A badly written regexp in an annotation is a ReDoS hole in a public endpoint: review those patterns as carefully as SQL.

4.2.6 More regex: named groups, lookahead, flags and replacement

The earlier examples used numbered groups. Real patterns grow, and numbers become fragile when you add a group in the middle; use **named groups** `(?<name>...)` and read them with `group("name")`. The next program also shows replacement with back-references (`$1`), a replacement computed by a function, a lookahead, flags, and the quoting helpers.

Rx2.java JAVA

```java
import java.util.regex.*;

public class Rx2 {
    public static void main(String[] args) {
        Pattern p = Pattern.compile("(?<year>\\d{4})-(?<month>\\d{2})-(?<day>\\d{2})");
        Matcher m = p.matcher("Due 2025-08-15, paid 2025-09-01.");
        while (m.find()) {
            System.out.println(m.group("day") + "/" + m.group("month") + "/" + m.group("year")
                + " at " + m.start() + "-" + m.end());
        }
        System.out.println("2025-08-15".replaceAll("(\\d+)-(\\d+)-(\\d+)", "$3.$2.$1"));
        System.out.println(Pattern.compile("\\d+").matcher("a1b22c333")
            .replaceAll(r -> "<" + r.group().length() + ">"));
        System.out.println("price: 100 USD, 250 EUR".replaceAll("\\d+(?= EUR)", "***"));
        System.out.println("passw0rd".matches("(?=.*\\d)(?=.*[a-z]).{8,}"));
        System.out.println(Pattern.compile("hello", Pattern.CASE_INSENSITIVE)
            .matcher("Say HELLO").find());
        System.out.println(Pattern.compile("^b", Pattern.MULTILINE).matcher("a\nb").find());
        System.out.println(Pattern.matches("a.c", "a\nc") + " "
            + Pattern.compile("a.c", Pattern.DOTALL).matcher("a\nc").matches());
        System.out.println(Pattern.quote("1+1") + " "
            + "1+1=2".replaceAll(Pattern.quote("1+1"), "two"));
        System.out.println("a$b".replaceAll("\\$", Matcher.quoteReplacement("\\$")));
        System.out.println(Pattern.compile(",").splitAsStream("x,y,z").count());
        System.out.println(Pattern.compile("\\p{Lu}\\p{L}*").matcher("Émile and Zoé")
            .results().count());
    }
}
```

Output TERMINAL

```
$ java Rx2.java
15/08/2025 at 4-14
01/09/2025 at 21-31
15.08.2025
a<1>b<2>c<3>
price: 100 USD, *** EUR
true
true
true
false true
\Q1+1\E two=2
a\$b
3
2
```

What to notice

- `find()` scans for the next match and `start()`/`end()` give its position; `matches()` demands that the whole input match, `lookingAt()` only the beginning.
- In a replacement string `$3` means "group 3", and a literal `$` or backslash must be escaped. When the replacement text comes from user input, wrap it in `Matcher.quoteReplacement`; when the pattern text comes from user input, wrap it in `Pattern.quote` (which produces `\Q...\E`). Otherwise `"1+1"` is read as a regex and fails to match.
- `replaceAll(Function<MatchResult,String>)` (Java 9) computes each replacement in code, which is clearer than a trick with groups.
- The **lookahead** `(?=...)` checks what follows without consuming it: `\\d+(?= EUR)` matches the number before " EUR" only. Likewise `(?!...)` (negative lookahead) and `(?<=...)` / `(?<!...)` (lookbehind). The password check `(?=.*\\d)(?=.*[a-z]).{8,}` chains two lookaheads from the same start position: "contains a digit" and "contains a lowercase letter", then at least 8 characters.
- Flags: `CASE_INSENSITIVE`, `MULTILINE` (`^` and `$` match at line breaks; the `a\nb` example finds `^b`), `DOTALL` (`.` matches line breaks; without it `a.c` does not match `a`, newline, `c`), and `UNICODE_CASE`. Inline forms such as `(?i)` work inside the pattern.
- `\\p{Lu}` and `\\p{L}` are Unicode categories (uppercase letter, any letter), so `\\p{Lu}\\p{L}*` counts the words "Émile" and "Zoé" correctly where `[A-Z]` would not. Use `matcher.results()` (Java 9) for a stream of matches.

> **TIP**
> A `Pattern` is immutable and thread-safe, while a `Matcher` is not. Compile patterns once into `private static final Pattern` fields; `String.matches`, `replaceAll` and `split` (for multi-character patterns) recompile on every call, which is a measurable cost in hot paths. Prefer a plain `String.replace(CharSequence, CharSequence)` when you do not need a regex at all.

Check yourself

1. What does `System.out.printf("[%6.2f] [%-4s] [%04d]%n", 3.14159, "ab", 7);` print?
2. What does `"2026".matches("\\d")` return, and what would `Pattern.compile("\\d").matcher("2026").find()` return?
3. `"<a><b>".replaceAll("<.+>", "X")` versus `"<a><b>".replaceAll("<.+?>", "X")`: what does each return?
4. Why is `"1.5".split(".")` a bug, and what is the fix?
5. Why should a `Pattern` normally be a `static final` field?
6. What makes this text block end with a newline, and how do you avoid it?

Answers

1. `[ 3.14] [ab  ] [0007]` followed by a line break: width 6 with 2 decimals, left-justified width 4, zero-padded width 4.
2. `false` (the whole string must match one digit); `true` (find searches for a single digit anywhere).
3. `X` (greedy matches `<a><b>` as one) and `XX` (lazy matches each tag separately).
4. The argument is a regex where `.` matches any character, so the result is an empty array. Use `split("\\.")` or `split(Pattern.quote("."))`.
5. Compiling is costly and a `Pattern` is immutable and thread-safe, so one shared instance avoids repeated compilation; `Matcher` instances are created per use.

6. The closing """ sits on its own line after the last text line. Place it at the end of the last text line, or end that line with \, to omit the trailing newline.

KEY POINTS

printf formats are checked only at run time; always pass a Locale for machine-readable output, and note rounding is HALF_UP.

- A Pattern is compiled once and shared; a Matcher is per use. matches needs the whole input, find scans.
- Quantifiers are greedy by default; ? makes them lazy, + possessive. replace is literal, replaceAll and split use regex.
- Nested quantifiers on untrusted input can cause catastrophic backtracking (ReDoS); simplify the pattern and cap input size.
- Text blocks strip incidental indentation, need no quote escaping, and end with a newline only if the delimiter is on its own line.

4.3 Unicode, charsets and Locale

"It works on my machine" is often an encoding or locale problem. The text café turns into cafÃ©, an emoji "has length 2", a Turkish user cannot log in because "I".toLowerCase() produced the wrong letter, and a German server prints 1.234,5 into a file that a US parser rejects. These bugs share one cause: confusing characters, code points, bytes, and cultural conventions. This chapter separates the four layers so you can reason about each.

4.3.1 Characters, code points and bytes

Unicode assigns every character in every writing system a number called a **code point**, written U+0041 for A or U+1F600 for the grinning-face emoji. Code points run from U+0000 to U+10FFFF. Unicode itself says nothing about bytes; an **encoding** maps code points to bytes:

- **UTF-8**: 1 to 4 bytes per code point; ASCII stays one byte. The dominant encoding of files, the web, JSON, and databases.
- **UTF-16**: 2 bytes for code points up to U+FFFF (the Basic Multilingual Plane, BMP), 4 bytes (a **surrogate pair** of two 16-bit units) above that. This is how Java represents text in memory.
- **ISO-8859-1 (Latin-1)** and **US-ASCII**: legacy single-byte sets that can represent only a few hundred or 128 characters.

Java's char is a 16-bit UTF-16 code unit, not a "character". String.length() counts those units. For text inside the BMP the two coincide; for emoji, many historic scripts and some CJK extensions they do not.

Uni.java JAVA

```
import java.nio.charset.Charset;
import java.nio.charset.StandardCharsets;
import java.text.Normalizer;
import java.util.HexFormat;

public class Uni {
    public static void main(String[] args) {
        String s = "A\u00e9\u20ac\uD83D\uDE00";   // A, e-acute, euro sign, grinning face
        System.out.println("length()       = " + s.length());
        System.out.println("codePointCount = " + s.codePointCount(0, s.length()));
        for (int i = 0; i < s.length(); i++) {
            char c = s.charAt(i);
            System.out.printf("char %d: U+%04X  surrogate=%b%n",
                i, (int) c, Character.isSurrogate(c));
        }
        s.codePoints().forEach(cp -> System.out.printf("cp U+%X  %s%n", cp, Character.getName(cp)));

        for (Charset cs : new Charset[] {StandardCharsets.ISO_8859_1, StandardCharsets.UTF_8,
                                         StandardCharsets.UTF_16BE, StandardCharsets.UTF_16}) {
            byte[] b = s.getBytes(cs);
            System.out.println(cs + " -> " + b.length + " bytes " + HexFormat.of().formatHex(b));
        }

        byte[] utf8 = "caf\u00e9".getBytes(StandardCharsets.UTF_8);       // 5 bytes
        String wrong = new String(utf8, StandardCharsets.ISO_8859_1);     // mojibake
        System.out.println(utf8.length + " bytes, wrong decode has " + wrong.length() + " chars, "
```

Uni.java (continued) JAVA

```
            + "right decode equals original: "
            + new String(utf8, StandardCharsets.UTF_8).equals("caf\u00e9"));

        String n1 = "\u00e9", n2 = "e\u0301";   // precomposed vs e + combining acute
        System.out.println(n1.equals(n2) + " " + n1.length() + " " + n2.length() + " "
            + Normalizer.normalize(n2, Normalizer.Form.NFC).equals(n1));
        System.out.println("default charset = " + Charset.defaultCharset());
    }
}
```

Output TERMINAL

```
$ java Uni.java
length()       = 5
codePointCount = 4
char 0: U+0041  surrogate=false
char 1: U+00E9  surrogate=false
char 2: U+20AC  surrogate=false
char 3: U+D83D  surrogate=true
char 4: U+DE00  surrogate=true
cp U+41  LATIN CAPITAL LETTER A
cp U+E9  LATIN SMALL LETTER E WITH ACUTE
cp U+20AC  EURO SIGN
cp U+1F600  GRINNING FACE
ISO-8859-1 -> 4 bytes 41e93f3f
UTF-8 -> 10 bytes 41c3a9e282acf09f9880
UTF-16BE -> 10 bytes 004100e920acd83dde00
UTF-16 -> 12 bytes feff004100e920acd83dde00
5 bytes, wrong decode has 5 chars, right decode equals original: true
false 1 2 true
default charset = UTF-8
```

What to notice:

- Four code points but `length()` is 5: the emoji is stored as the surrogate pair `D83D DE00`. Looping with `charAt` would visit the two halves separately; `codePoints()`, `codePointAt`, and `codePointCount` work on whole code points. `new StringBuilder(s).reverse()` is surrogate-aware (it keeps pairs intact), but `char[]` reversal by hand is not.
- The same text takes 4, 10, 10, or 12 bytes depending on the charset. `UTF-16` (without BE/LE) writes a 2-byte byte-order mark `feff` first. In ISO-8859-1 the characters it cannot represent became `3f`, the question mark. Silent replacement is the default behaviour of `getBytes`; for strict handling use `CharsetEncoder` with `CodingErrorAction.REPORT`.
- Decoding UTF-8 bytes as Latin-1 does not fail; it produces garbage of the wrong length (the "mojibake" `cafÃ©`). Encoding and decoding must use the same charset.
- `é` can be one code point or `e` plus a combining accent. They look identical but `equals` is false. Normalise with `java.text.Normalizer` (NFC) before comparing, hashing or storing user-visible text such as usernames.

Always name the charset

Since Java 18 (JEP 400) the default charset is UTF-8 on every platform, so `new String(bytes)` and `Files.readString(path)` are less dangerous than they were. The old platform charset is still available as the system property `native.encoding`, and it still controls what the console (`System.out`) uses; on the build machine `native.encoding` and `stdout.encoding` are `ANSI_X3.4-1968` (plain ASCII), so non-ASCII output printed as `?` here. That is a terminal problem, not a Java problem. Good habits regardless of version:

- Pass `StandardCharsets.UTF_8` explicitly to `getBytes`, `new String`, `InputStreamReader`, `FileReader` (Java 11+ has a charset constructor), `Files.readString` and `Files.writeString`.
- Set `Content-Type: application/json; charset=UTF-8` (or rely on the framework default of UTF-8) for HTTP.
- Check the encoding of the database connection and of source files (`javac -encoding UTF-8`; UTF-8 is the default from Java 18, but build tools may set their own).

GOTCHA

`String.getBytes()` without arguments uses the default charset. Before Java 18 that was whatever the OS said (`windows-1252`, `ANSI_X3.4-1968`, ...), which is why the same program produced different files on different machines. Never use the no-argument forms in new code.

4.3.2 Locale: culture-sensitive behaviour

A `java.util.Locale` identifies a language and optionally a region (`en-US`, `de-DE`, `hi-IN`). Many JDK methods consult a locale: `String.format`, `toUpperCase`, `NumberFormat`, `DateTimeFormatter.ofLocalizedDate`, `Collator`. The JVM chooses a default from the OS (`Locale.getDefault()`); servers often run in a different locale from developer laptops, so locale-dependent code that was never tested outside your locale is a source of bugs.

Create locales with `Locale.forLanguageTag("en-IN")` or the constants (`Locale.US`, `Locale.GERMANY`). The `Locale` constructors are deprecated since Java 19. `Locale.ROOT` means "no particular locale"; it is what you want for protocol identifiers, keys and anything a program (not a person) reads.

Loc.java JAVA

```java
import java.text.Collator;
import java.text.NumberFormat;
import java.text.ParseException;
import java.util.*;

public class Loc {
    // show every non-ASCII char as \\uXXXX so the output is unambiguous
    static String esc(String s) {
        StringBuilder b = new StringBuilder();
        for (char c : s.toCharArray()) {
            b.append(c < 128 ? String.valueOf(c) : String.format("\\u%04x", (int) c));
        }
        return b.toString();
    }
    public static void main(String[] args) throws ParseException {
        Locale tr = Locale.forLanguageTag("tr");
        System.out.println(esc("TITLE".toLowerCase(tr)) + " " + esc("title".toUpperCase(tr)));
        System.out.println("TITLE".toLowerCase(Locale.ROOT) + " "
            + "title".toUpperCase(Locale.ROOT));
        System.out.println("stra\u00dfe".toUpperCase(Locale.ROOT));

        double v = 1234567.891;
        for (Locale l : List.of(Locale.US, Locale.GERMANY, Locale.forLanguageTag("en-IN"))) {
            String num = NumberFormat.getNumberInstance(l).format(v);
            String cur = NumberFormat.getCurrencyInstance(l).format(v);
            System.out.println(l.toLanguageTag() + "  " + esc(num) + "  " + esc(cur));
        }
        NumberFormat de = NumberFormat.getInstance(Locale.GERMANY);
        System.out.println(de.parse("1.234,5") + " " + Double.parseDouble("1234.5")
            + " " + de.parse("12abc"));

        List<String> w = new ArrayList<>(List.of("zebra", "\u00e4pfel", "apple", "Zoo"));
        Collections.sort(w);
        System.out.println(esc(w.toString()));
        w.sort(Collator.getInstance(Locale.GERMANY));
        System.out.println(esc(w.toString()));
    }
}
```

Output TERMINAL

```
$ java Loc.java
t\u0131tle T\u0130TLE
title TITLE
STRASSE
en-US  1,234,567.891  $1,234,567.89
de-DE  1.234.567,891  1.234.567,89\u00a0\u20ac
en-IN  1,234,567.891  \u20b91,234,567.89
1234.5 1234.5 12
[Zoo, apple, zebra, \u00e4pfel]
[\u00e4pfel, apple, zebra, Zoo]
```

The Turkish i

Turkish has two letters i: dotted (`i`/`İ`) and dotless (`ı`/`I`). With a Turkish locale, `"TITLE".toLowerCase()` yields `tıtle` (a dotless `ı`) and `"title".toUpperCase()` yields `TİTLE`, so `"FILE".toLowerCase().equals("file")` is false on a Turkish machine. This has broken file-extension checks, SQL keyword matching and HTTP header handling in real products. The fix: for identifiers, enum names, protocol tokens and map keys always write `toLowerCase(Locale.ROOT)` or `toUpperCase(Locale.ROOT)`. Note also that upper-casing can change length (`straße` becomes `STRASSE`), so case conversion is not a 1:1 mapping.

NumberFormat, parsing and sorting

`NumberFormat` gives locale-correct output: separators (`1.234.567,891` in Germany), currency symbols and placement (`1.234.567,89 €` with a no-break space,), and in India ₹. Other factories: `getPercentInstance`, `getIntegerInstance`, `getCompactNumberInstance` (Java 12; prints `1M`). `NumberFormat` is not thread-safe: do not share one instance between threads.

Parsing has a trap shown on the `1234.5 1234.5 12` line: `parse` stops at the first character it cannot use and returns what it has, so `"12abc"` parses as 12 with no error. Use `parse(String, ParsePosition)` and check that the whole input was consumed if you need strictness. And `Double.parseDouble` always expects `.` as the decimal point regardless of locale.

Plain `String` ordering (`compareTo`) compares UTF-16 code units, so accented letters sort after `z`. For user-visible lists use `java.text.Collator` for the user's locale, as the last two lines show: in German collation `äpfel` sorts before `apple`.

WATCH OUT

Money amounts should not be formatted with `double` and then parsed back. Format with `NumberFormat.getCurrencyInstance` (or `DecimalFormat` with explicit `RoundingMode`) for display only, and store amounts as `BigDecimal` or integer cents. The next chapter covers `BigDecimal`.

INTERVIEW ANGLE

What is the difference between a `char`, a code point and a byte in Java? A `char` is a 16-bit UTF-16 code unit. A code point is a Unicode number (up to U+10FFFF); code points above U+FFFF need two `char`s (a surrogate pair). A byte only exists once text is encoded with a charset such as UTF-8. So `length()` counts code units, `codePointCount` counts characters in the Unicode sense, and `getBytes(cs).length` counts bytes.

REAL WORLD

Spring Boot serialises JSON as UTF-8 and parses request bodies using the declared charset, but it uses the JVM default locale for things like `@DateTimeFormat` and message bundles unless you configure `LocaleResolver`/`Accept-Language` handling. Docker images often run with the `POSIX`/`C` locale, which is another reason to write `Locale.ROOT` for machine-readable formatting.

4.3.3 Normalization, Base64 and hex: text versus bytes

Two more everyday tools sit between text and bytes. **Normalization** deals with the fact that the same visible text can be encoded in several ways. **Base64** and **hex** are encodings that turn arbitrary bytes into safe ASCII text; they are not encryption.

Nz.java — JAVA

```java
import java.text.Normalizer;
import java.util.Base64;
import java.util.HexFormat;
import java.nio.charset.StandardCharsets;

public class Nz {
    public static void main(String[] args) {
        String composed = "é";
        String decomposed = "e´";
        System.out.println(composed.equals(decomposed) + " " + composed.length()
            + " " + decomposed.length());
        String n = Normalizer.normalize(decomposed, Normalizer.Form.NFC);
        System.out.println(composed.equals(n));
        String stripped = Normalizer.normalize("Café naïve", Normalizer.Form.NFD)
            .replaceAll("\\p{M}", "");
        System.out.println(stripped);
        byte[] data = "Hi é?".getBytes(StandardCharsets.UTF_8);
        byte[] odd = {(byte) 0xfb, (byte) 0xff};
        System.out.println(Base64.getEncoder().encodeToString(data) + " "
            + Base64.getUrlEncoder().withoutPadding().encodeToString(odd));
        System.out.println(HexFormat.of().formatHex(data) + " "
            + HexFormat.ofDelimiter(":").withUpperCase().formatHex(data));
        String back = new String(Base64.getDecoder().decode("SGkgw6k/"), StandardCharsets.UTF_8);
        System.out.println(back.equals("Hi \u00e9?") + " " + back.length());
    }
}
```

Output TERMINAL

```
$ java Nz.java
false 1 2
true
Cafe naive
SGkgw6k/ -_8
486920c3a93f 48:69:20:C3:A9:3F
true 5
```

- `é` can be one code point (U+00E9, "composed") or `e` followed by a combining acute accent (U+0301, "decomposed"). They print identically but `equals` says `false` (lengths 1 and 2). `Normalizer.Form.NFC` composes, `NFD` decomposes; normalise to NFC before comparing, hashing or storing user-supplied text such as usernames.
- NFD followed by removing combining marks (`\\p{M}`) is a common way to strip accents for search or slugs: `Café naïve` becomes `Cafe naive`.
- `java.util.Base64` has three flavours: basic (`+` and `/`, padded with `=`), URL-safe (`-` and `_`, used in JWTs and URLs; add `withoutPadding()` to drop the `=`) and MIME (line-wrapped). The string `Hi é?` is 6 bytes in UTF-8 and becomes 8 Base64 characters. Always convert text to bytes with an explicit charset before encoding.
- `HexFormat` (Java 17) formats and parses hexadecimal with optional delimiters and case. Before it you had to write `String.format("%02x", b)` loops.

WATCH OUT

Base64 is an encoding, not protection. Anyone can decode `SGkgw6k/`. Do not use it to hide passwords or tokens; use hashing (for passwords, with a slow salted algorithm such as bcrypt or Argon2 through a library) or encryption.

Check yourself

1. What do `"a\uD83D\uDE00".length()` and `"a\uD83D\uDE00".codePointCount(0, 3)` return?
2. Why can `"é".equals("e´")` be false although both print as é? How do you compare them correctly?
3. You decode the bytes of a UTF-8 file with `new String(bytes, StandardCharsets.ISO_8859_1)`. What do you get and does Java throw?
4. What is wrong with `if (input.toLowerCase().equals("title"))`, and what is the fix?
5. What does `NumberFormat.getInstance(Locale.GERMANY).parse("12abc")` return?
6. Which default charset applies to `new String(bytes)` in Java 21, and which in Java 11 on Windows?

Answers

1. `3` and `2`: three UTF-16 units (a plus the surrogate pair), two code points.
2. The first is one precomposed code point, the second is `e` plus a combining accent (two code points). Normalise both with `Normalizer.normalize(s, Normalizer.Form.NFC)` and then compare.
3. A string of the wrong content (mojibake such as `cafÃ©`, with one char per byte). No exception: Latin-1 can decode any byte.
4. `toLowerCase()` uses the default locale; in Turkish `TITLE` becomes `tıtle` and the test fails. Use `toLowerCase(Locale.ROOT)` or `equalsIgnoreCase`.
5. `12` (as a `Long`); parsing stops at `a` without error. Use a `ParsePosition` to detect unconsumed input.
6. UTF-8 in Java 18 and later (JEP 400); in Java 11 the platform charset, usually `windows-1252` on a Western Windows installation.

KEY POINTS

A `char` is a UTF-16 code unit; a code point may need two. Use `codePoints()` and `codePointCount` when characters outside the BMP matter.

- Encoding and decoding must use the same charset; always pass `StandardCharsets.UTF_8` explicitly. Java 18+ defaults to UTF-8.
- Normalise (`Normalizer` NFC) before comparing user text; use `Collator` to sort for humans.
- Case conversion and number formatting depend on `Locale`; use `Locale.ROOT` for identifiers and protocol data.
- `NumberFormat` is locale-aware, not thread-safe, and its `parse` can stop early without an error.

4.4 Numbers beyond primitives

Primitive `int`, `long` and `double` are fast and fine for most arithmetic, but they have three hard limits. They overflow silently (`Long.MAX_VALUE + 1` wraps to a negative number), they cannot represent most decimal fractions exactly (`0.1 + 0.2` is not `0.3`), and they have a fixed size. This chapter covers the tools for the cases where those limits matter: `BigDecimal` for exact decimal arithmetic (money, rates, measurements), `BigInteger` for integers of any size, the `Math` class for everything in between, and the random-number and identifier classes you will meet in real programs.

4.4.1 Why double is wrong for money

A `double` stores a binary fraction: a sign, an exponent and 52 bits of mantissa. A number like 0.1 has no finite binary expansion (just as 1/3 has no finite decimal expansion), so the stored value is the nearest binary fraction. Add a few such values and the tiny errors become visible. The chapter on type conversion in Part 2 showed `0.1 + 0.2`; here is what is actually stored for 0.1, and the fix.

Bd.java JAVA

```java
import java.math.BigDecimal;
import java.math.MathContext;
import java.math.RoundingMode;

public class Bd {
    public static void main(String[] args) {
        System.out.println(0.1 + 0.2);
        BigDecimal x = new BigDecimal("0.1").add(new BigDecimal("0.2"));
        System.out.println(x);
        System.out.println(new BigDecimal(0.1));
        System.out.println(BigDecimal.valueOf(0.1));

        BigDecimal a = new BigDecimal("2.0");
        BigDecimal b = new BigDecimal("2.00");
        System.out.println(a.equals(b) + " " + a.compareTo(b) + " " + a.hashCode()
            + " " + b.hashCode());
        System.out.println("scale=" + b.scale() + " precision=" + b.precision()
            + " unscaled=" + b.unscaledValue());

        BigDecimal price = new BigDecimal("19.99");
        BigDecimal qty = new BigDecimal("3");
        BigDecimal total = price.multiply(qty);
        System.out.println(total + " scale " + total.scale());
        BigDecimal rate = new BigDecimal("0.075");
        BigDecimal tax = total.multiply(rate);
        System.out.println(tax + " -> " + tax.setScale(2, RoundingMode.HALF_UP));

        try {
            BigDecimal.ONE.divide(new BigDecimal("3"));
        } catch (ArithmeticException e) {
            System.out.println(e.getMessage());
        }
        System.out.println(BigDecimal.ONE.divide(new BigDecimal("3"), 5, RoundingMode.HALF_EVEN));
        System.out.println(BigDecimal.ONE.divide(new BigDecimal("3"), MathContext.DECIMAL32));
        System.out.println(new BigDecimal("1.10").stripTrailingZeros() + " "
            + new BigDecimal("100").stripTrailingZeros() + " "
            + new BigDecimal("100").stripTrailingZeros().toPlainString());
    }
}
```

Output TERMINAL

```
$ java Bd.java
0.30000000000000004
0.3
0.1000000000000000055511151231257827021181583404541015625
0.1
false 0 621 6202
scale=2 precision=3 unscaled=200
59.97 scale 2
4.49775 -> 4.50
Non-terminating decimal expansion; no exact representable decimal result.
0.33333
0.3333333
1.1 1E+2 100
```

What to notice

- `new BigDecimal(0.1)` takes the already-wrong `double` and exposes its exact value (0.1000000000000000055511...). Use the `String` constructor, or `BigDecimal.valueOf(double)`, which goes through `Double.toString` and so gives `0.1`.
- A `BigDecimal` is an unscaled integer plus a **scale** (the number of digits after the decimal point). `2.00` is unscaled 200, scale 2. Multiplying adds the scales: `19.99 × 3` has scale 2 + 0.
- `equals` compares value **and** scale, so `2.0` and `2.00` are not equal and have different hash codes. `compareTo` compares numeric value only. This matters for `HashMap` keys and `HashSet`: use `compareTo` for numeric equality, or normalise the scale with `setScale` first.
- `divide` without a rounding instruction throws `ArithmeticException` when the exact quotient has infinitely many digits. Always say how to round: give a scale and a `RoundingMode`, or a `MathContext`.
- `BigDecimal` is immutable. `price.add(x)` returns a new object; ignoring the result is a classic bug (the same trap as `String`).

4.4.2 Scale, precision and rounding modes

Three words are easy to mix up. The **scale** counts digits right of the decimal point (it can be negative: `1E+2` has scale -2). The **precision** counts all significant digits of the unscaled value (`2.00` has precision 3). A `MathContext` bundles a precision and a `RoundingMode`; `MathContext.DECIMAL32`, `DECIMAL64` and `DECIMAL128` mirror the IEEE 754 decimal formats (7, 16 and 34 digits). Use `setScale(n, mode)` when you care about digits after the point (money: scale 2), and `round(MathContext)` or `divide(x, mc)` when you care about significant digits (scientific values).

Java defines eight rounding modes in `java.math.RoundingMode`. The table below was produced by calling `new BigDecimal(s).setScale(0, mode)` for each input; read it column by column, noticing what each mode does to the halves (2.5, -2.5) and to negative numbers.

Rm.java JAVA

```java
import java.math.BigDecimal;
import java.math.RoundingMode;

public class Rm {
    public static void main(String[] args) {
        String[] inputs = {"5.5", "2.5", "1.6", "1.1", "-1.1", "-1.6", "-2.5"};
        System.out.printf("%-10s", "mode");
        for (String s : inputs) System.out.printf("%5s", s);
        System.out.println();
        for (RoundingMode m : RoundingMode.values()) {
            if (m == RoundingMode.UNNECESSARY) continue;
            System.out.printf("%-10s", m);
            for (String s : inputs) {
                System.out.printf("%5s", new BigDecimal(s).setScale(0, m));
            }
            System.out.println();
        }
    }
}
```

Output TERMINAL

```
$ java Rm.java
mode        5.5  2.5  1.6  1.1 -1.1 -1.6 -2.5
UP            6    3    2    2   -2   -2   -3
DOWN          5    2    1    1   -1   -1   -2
CEILING       6    3    2    2   -1   -1   -2
FLOOR         5    2    1    1   -2   -2   -3
HALF_UP       6    3    2    1   -1   -2   -3
HALF_DOWN     5    2    2    1   -1   -2   -2
HALF_EVEN     6    2    2    1   -1   -2   -2
```

`UP` rounds away from zero and `DOWN` towards zero (truncation); `CEILING` and `FLOOR` round towards positive and negative infinity, so they differ from `UP`/`DOWN` only for negatives. The three `HALF_` modes differ only on exact ties (the `.5` columns): `HALF_UP` is the "school" rule (and the one `printf` uses), `HALF_DOWN` the opposite, and `HALF_EVEN`, also called banker's rounding, sends ties to the even neighbour (2.5 → 2, 5.5 → 6, 3.5 → 4). Over many operations `HALF_EVEN` has no systematic upward bias, which is why accounting standards and IEEE 754 prefer it. The ninth value, `UNNECESSARY`, asserts that no rounding is needed and throws `ArithmeticException` if it is.

GOTCHA
`Math.round(-2.5)` returns `-2` (it rounds half towards positive infinity), while `BigDecimal HALF_UP` turns -2.5 into -3. `Math.rint(2.5)` is `2.0` (half-even). Three rounding rules with similar names exist in one language, so write the mode down explicitly whenever the result is a bill.

Practical rules for money

- Store money as `BigDecimal` with an agreed scale (or as `long` minor units such as cents; many payment APIs do this). Never as `double`.
- Create from `String` or `valueOf`, never `new BigDecimal(double)`.
- Round once, at the end of a calculation or at a defined business step, with an explicit `RoundingMode`.
- Compare with `compareTo`; use `signum()` for "is it positive, zero or negative".
- Do not use `stripTrailingZeros()` for display without `toPlainString()`: `new BigDecimal("100").stripTrailingZeros()` prints `1E+2`.
- JSON, JDBC and Spring map `NUMERIC`/`DECIMAL` columns to `BigDecimal` naturally; keep the type end to end.
- `BigDecimal` is much slower than `double` (it allocates objects). Use it where exactness matters, not for physics simulations.

INTERVIEW ANGLE
Why shouldn't you use `double` for money, and what do you use instead? A `double` is a binary floating-point value; most decimal fractions such as 0.10 cannot be represented exactly, and errors accumulate and make totals off by a cent. Use `BigDecimal` (constructed from a `String`, with an explicit scale and `RoundingMode`) or integer minor units. Mention that `equals` on `BigDecimal` compares scale too, so use `compareTo`.

4.4.3 BigInteger: integers without a ceiling

`java.math.BigInteger` represents an integer of arbitrary size, limited only by memory. It is immutable, has no operators (you call `add`, `multiply`, `pow`, `mod`, `gcd`), and has number-theory helpers such as `modPow`, `modInverse` and `isProbablePrime`. It is what cryptography code, factorials and exact combinatorics use.

Bi.java — JAVA

```java
import java.math.BigInteger;

public class Bi {
    static BigInteger fact(int n) {
        BigInteger r = BigInteger.ONE;
        for (int i = 2; i <= n; i++) r = r.multiply(BigInteger.valueOf(i));
        return r;
    }
    public static void main(String[] args) {
        System.out.println(fact(25));
        System.out.println(fact(25).bitLength() + " bits, "
            + fact(100).toString().length() + " digits in 100!");
        BigInteger p = BigInteger.TWO.pow(127).subtract(BigInteger.ONE);
        System.out.println(p + " prime? " + p.isProbablePrime(50));
        System.out.println(BigInteger.valueOf(48).gcd(BigInteger.valueOf(180)));
        try {
            BigInteger.valueOf(Long.MAX_VALUE).add(BigInteger.ONE).longValueExact();
        } catch (ArithmeticException e) {
            System.out.println(e.getMessage());
        }
        System.out.println(Long.MAX_VALUE + 1);
    }
}
```

Output — TERMINAL

```
$ java Bi.java
15511210043330985984000000
84 bits, 158 digits in 100!
170141183460469231731687303715884105727 prime? true
12
BigInteger out of long range
-9223372036854775808
```

`25!` already needs 84 bits, so it cannot fit in a `long` (63 bits of magnitude). The number `2^127 - 1` is a Mersenne prime; `isProbablePrime(50)` runs Miller-Rabin style tests with an error probability below 2^-50. Convert back to primitives with the `*Exact` methods (`longValueExact`, `intValueExact`), which throw instead of silently truncating like `longValue()`. `BigInteger` multiplication switches algorithm with size (schoolbook, Karatsuba, Toom-Cook), so it is fast for hundreds of digits but not free; the last line shows that a plain `long` just wraps.

4.4.4 The Math class and exact arithmetic

`java.lang.Math` holds static numeric helpers. Most are thin wrappers over hardware instructions or intrinsics. The ones worth knowing, with traps, are in the run below.

Mth.java JAVA

```java
public class Mth {
    public static void main(String[] args) {
        System.out.println(Math.round(2.5) + " " + Math.round(-2.5) + " " + Math.round(3.5)
            + " " + Math.rint(2.5) + " " + Math.rint(3.5));
        System.out.println(Math.floorDiv(-7, 2) + " " + (-7 / 2) + " "
            + Math.floorMod(-7, 2) + " " + (-7 % 2));
        System.out.println(Math.abs(Integer.MIN_VALUE));
        try {
            Math.absExact(Integer.MIN_VALUE);
        } catch (ArithmeticException e) {
            System.out.println(e.getMessage());
        }
        try {
            Math.toIntExact(1L << 40);
        } catch (ArithmeticException e) {
            System.out.println(e.getMessage());
        }
        System.out.println(Math.hypot(3, 4) + " " + Math.cbrt(27) + " " + Math.pow(2, 10)
            + " " + Math.sqrt(-1) + " " + Math.clamp(15, 0, 10));
        System.out.println(Math.ceilDiv(7, 2) + " " + Math.fma(2.0, 3.0, 1.0));
        System.out.println(Math.ulp(1.0) + " " + Math.nextUp(1.0));
    }
}
```

Output TERMINAL

```
$ java Mth.java
3 -2 4 2.0 4.0
-4 -3 1 -1
-2147483648
Overflow to represent absolute value of Integer.MIN_VALUE
integer overflow
5.0 3.0 1024.0 NaN 10
4 7.0
2.220446049250313E-16 1.0000000000000002
```

- `Math.floorDiv` and `floorMod` round towards negative infinity. Java's `/` and `%` truncate towards zero, so `-7 % 2` is `-1`. For array indices wrapped around a ring buffer (`index % size` when index may be negative) use `floorMod`. (`Math.floorMod(x, n)` is always in `0..n-1` for positive `n`.)
- `Math.abs(Integer.MIN_VALUE)` returns `Integer.MIN_VALUE`, still negative, because +2147483648 does not fit. This even breaks the classic `Math.abs(hash) % n` bucket-picking code when `hash` happens to be `MIN_VALUE`. Use `floorMod(hash, n)`.
- The `*Exact` family (`addExact`, `subtractExact`, `multiplyExact`, `incrementExact`, `negateExact`, `toIntExact`, `absExact` since 15) throws `ArithmeticException` on overflow instead of wrapping. Use them wherever silent wrap-around would be a bug (balances, quantities, sizes).
- `Math.clamp(value, min, max)` is new in Java 21; `ceilDiv` since 18; `fma` (fused multiply-add) computes `a*b+c` with a single rounding.
- `Math.sqrt(-1)` is `NaN`, no exception. Floating-point failures are values, not errors (see the NaN discussion in Part 2).
- `Math.ulp(x)` is the gap between `x` and the next representable `double`: for 1.0 it is about 2.2e-16, which explains why comparing doubles with `==` is fragile.

Math versus StrictMath

`StrictMath` guarantees bit-for-bit identical results on every platform by using the fdlibm algorithms. `Math` is allowed a few ulps of error for `sin`, `cos`, `exp`, `log`, `pow` and so on, so the JIT can use faster hardware instructions. Since Java 17 (JEP 306) all floating-point arithmetic is strict (`strictfp` is redundant and gives a warning), but the transcendental functions in `Math` are still allowed to differ slightly from `StrictMath`. Use `StrictMath` only if you need reproducible results across machines, for example in a test oracle or a game simulation synchronised between clients.

4.4.5 Random numbers: Random, ThreadLocalRandom, SecureRandom

Which random class you pick is a design decision, not a detail.

Class	Use for	Do not use for
`java.util.Random`	Simple simulations, tests that need a reproducible seed. Thread-safe but contended (one atomic seed).	Anything security related; heavy multi-threaded use.
`ThreadLocalRandom`	Random numbers in concurrent code; each thread has its own generator, no contention. Get it with `current()` at the point of use.	Reproducible runs (it cannot be seeded); storing in a field or sharing across threads.
`SecureRandom`	Tokens, session ids, password salts, keys, nonces. Seeded from the OS entropy source.	Hot loops where speed matters more than unpredictability.

Choosing a random number generator.

Rnd.java JAVA

```java
import java.security.SecureRandom;
import java.util.HexFormat;
import java.util.Random;
import java.util.UUID;
import java.util.concurrent.ThreadLocalRandom;
import java.util.random.RandomGenerator;
import java.util.stream.Collectors;

public class Rnd {
    public static void main(String[] args) {
        Random r1 = new Random(42);
        Random r2 = new Random(42);
        System.out.println(r1.nextInt(100) + " " + r1.nextInt(100) + " " + r1.nextInt(100));
        System.out.println(r2.nextInt(100) + " " + r2.nextInt(100) + " " + r2.nextInt(100));
        System.out.println(r1.ints(5, 1, 7).boxed().collect(Collectors.toList()));
        int roll = ThreadLocalRandom.current().nextInt(1, 7);
        System.out.println("die in range: " + (roll >= 1 && roll <= 6));
        SecureRandom sr = new SecureRandom();
        byte[] token = new byte[16];
        sr.nextBytes(token);
        System.out.println("token hex length " + HexFormat.of().formatHex(token).length());
        UUID u = UUID.randomUUID();
        String us = u.toString();
        System.out.println(us.length() + " v" + u.version() + " " + us.charAt(14));
        System.out.println(UUID.nameUUIDFromBytes("alice".getBytes()).equals(
            UUID.nameUUIDFromBytes("alice".getBytes())));
        System.out.println(RandomGenerator.of("L64X128MixRandom").getClass().getSimpleName());
        System.out.println(RandomGenerator.getDefault().getClass().getSimpleName());
        System.out.println(sr.getAlgorithm());
    }
}
```

Output (the seeded lines are deterministic; the SecureRandom algorithm depends on the OS) TERMINAL

```
$ java Rnd.java
30 63 48
30 63 48
[3, 1, 2, 6, 3]
die in range: true
token hex length 32
36 v4 4
true
L64X128MixRandom
L32X64MixRandom
NativePRNG
```

Two `Random` objects with the same seed produce the same sequence: that is the point of a pseudo-random generator (a deterministic algorithm that looks random), and it is why `Random` is wrong for secrets. `Random` is a 48-bit linear congruential generator, so anyone who sees a few outputs can predict the rest. Java 17 added `java.util.random.RandomGenerator` (JEP 356), a common interface implemented by `Random`, `ThreadLocalRandom`, `SecureRandom` and the new families such as `L64X128MixRandom` (shown above, which has good statistical quality and supports splitting for parallel streams). `RandomGenerator.getDefault()` returns a recommended general-purpose generator (`L32X64MixRandom` on this JDK).

WATCH OUT

Never use `Random` or `Math.random()` for passwords, tokens, reset links or anything an attacker benefits from guessing. Use `SecureRandom` (and `new SecureRandom()` is thread-safe; reuse one instance). Another classic mistake: `new Random()` inside a loop or per request. Create one instance and reuse it, or use `ThreadLocalRandom.current()`.

UUID

A `java.util.UUID` is a 128-bit identifier, written as 36 characters (`8-4-4-4-12` hex digits). `UUID.randomUUID()` creates a version 4 UUID from `SecureRandom`: 122 random bits, so collisions are, for practical purposes, impossible. The version digit is the first character of the third group (the 15th character, index 14: `4` above). `UUID.nameUUIDFromBytes` creates a version 3 (MD5, name-based) UUID, which is deterministic: the same input always yields the same UUID, useful for derived ids. The JDK has no built-in generator for version 7 (time-ordered) UUIDs, to my knowledge; if you need index-friendly ids in a database, use a library or a separate id scheme, because random v4 values scatter inserts across a B-tree index.

REAL WORLD

Spring Data and JPA entities often use `UUID` primary keys; remember they are 16 bytes in a native `uuid` column, but 36 characters if stored as `varchar`. For one-time tokens in a Spring Security flow, generate them with `SecureRandom` bytes encoded using `Base64.getUrlEncoder().withoutPadding()`.

Check yourself

1. What does this print: `System.out.println(new BigDecimal("2.0").equals(new BigDecimal("2.00")));`? What would you use instead?
2. Why is `new BigDecimal(0.1)` a bad idea? Give two correct alternatives.
3. What happens with `BigDecimal.ONE.divide(new BigDecimal("3"))` and how do you fix it?
4. What do `Math.round(-2.5)`, `Math.abs(Integer.MIN_VALUE)` and `-7 % 2` return?
5. Which random class do you use to generate a password-reset token, and which for dice in a multi-threaded simulation?
6. What is the difference between the scale and the precision of `new BigDecimal("12.340")`?

Answers

1. `false`: `equals` compares scale as well as value. Use `compareTo(...) == 0`.
2. It converts the inexact `double` literally and gives 0.1000000000000000055511151231257827.... Use `new BigDecimal("0.1")` or `BigDecimal.valueOf(0.1)`.
3. It throws `ArithmeticException` (non-terminating decimal expansion). Supply a scale and rounding mode, for example `divide(d, 5, RoundingMode.HALF_EVEN)`, or a `MathContext`.
4. `-2` (rounds half up towards +infinity), `-2147483648` (overflow, still negative), and `-1` (the remainder takes the sign of the dividend).
5. `SecureRandom` for the token; `ThreadLocalRandom.current()` for the simulation.
6. Scale is 3 (digits after the point); precision is 5 (significant digits: 1, 2, 3, 4, 0).

KEY POINTS

Use `BigDecimal` (built from `String` or `valueOf`) with an explicit `RoundingMode` for money; compare with `compareTo`, not `equals`.

- `BigInteger` removes the size limit of `long`; convert back with the `*Exact` methods so overflow is an error, not a surprise.
- `Math.floorMod`, `Math.absExact` and the `*Exact` methods avoid the silent traps of `%`, `abs` and `+` on primitives.
- `Random` is predictable, `ThreadLocalRandom` is fast in threads, `SecureRandom` is for secrets.
- A random `UUID` (v4) is a good unique id, but not a secret token and not index-friendly.

4.5 Date and time with java.time

Time looks simple until you ship software to more than one place. "Midnight" depends on a time zone, a day is sometimes 23 or 25 hours long, February has 28 or 29 days, and a month is not a fixed number of days. Java's `java.time` package (added in Java 8, JSR 310) models these facts with separate types for separate ideas. Learning which type means what is most of the work.

4.5.1 Why Date and Calendar failed

Before Java 8 you had `java.util.Date` and `java.util.Calendar`. Both are still in the JDK, and you will meet them in legacy code and in older libraries, so you must recognise their problems. The program below shows several of them. It is run with the default zone set to UTC so the output is reproducible.

Old.java JAVA

```java
import java.text.SimpleDateFormat;
import java.time.Instant;
import java.time.LocalDate;
import java.util.Calendar;
import java.util.Date;
import java.util.GregorianCalendar;
import java.util.TimeZone;

public class Old {
    @SuppressWarnings("deprecation")
    public static void main(String[] args) throws Exception {
        TimeZone.setDefault(TimeZone.getTimeZone("UTC"));
        Date d = new Date(2024, 1, 31);
        System.out.println(d);
        Calendar c = new GregorianCalendar(2024, 1, 31);
        System.out.println(c.get(Calendar.MONTH) + " " + c.get(Calendar.DAY_OF_MONTH)
            + " " + c.getTime());
        SimpleDateFormat sdf = new SimpleDateFormat("yyyy-MM-dd");
        sdf.setLenient(true);
        System.out.println(sdf.parse("2025-13-45"));
        Date legacy = new Date(1_700_000_000_000L);
        Instant i = legacy.toInstant();
        Date back = Date.from(i);
        System.out.println(legacy + " | " + i + " | " + back.equals(legacy));
        LocalDate ld = LocalDate.of(2024, 2, 29);
        java.sql.Date sqlDate = java.sql.Date.valueOf(ld);
        System.out.println(sqlDate + " " + sqlDate.toLocalDate());
    }
}
```

Output TERMINAL

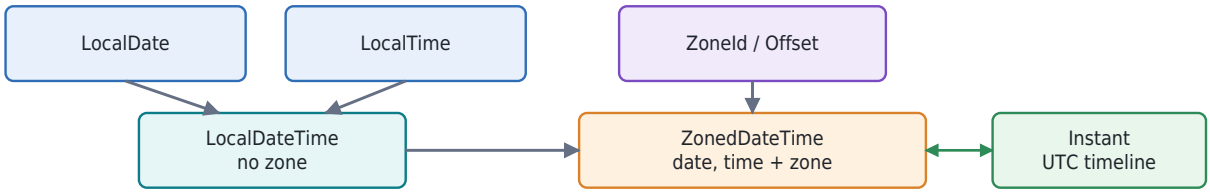
```
$ java Old.java
Sun Mar 02 00:00:00 UTC 3924
2 2 Sat Mar 02 00:00:00 UTC 2024
Sat Feb 14 00:00:00 UTC 2026
Tue Nov 14 22:13:20 UTC 2023 | 2023-11-14T22:13:20Z | true
2024-02-29 2024-02-29
```

What went wrong in the first three lines: `new Date(2024, 1, 31)` takes the year as an offset from 1900 (so the year is 3924) and months from 0 (so 1 means February), and February 31 silently rolls over to March 2. `GregorianCalendar` also counts months from 0 (`MONTH` is 2 for what a human calls March, after the roll-over). `SimpleDateFormat` is lenient by default and happily turned "2025-13-45" into 14 February 2026. In addition:

- `Date` and `Calendar` are mutable, so passing one around invites defensive copies and bugs; `SimpleDateFormat` is mutable *and* not thread-safe, so a shared static instance corrupts output under load.
- `Date` is really an instant (milliseconds since 1970-01-01T00:00:00Z) that prints in the default zone, which confuses people into thinking it holds a zone.
- There is no type for "just a date" or "just a time"; `java.sql.Date` is a subclass that pretends to be one.

The `java.time` types fix all of these: they are immutable (so thread-safe), months are 1 to 12, invalid dates throw, each type represents one concept, and the formatter is thread-safe. Convert at the edges with `Date.toInstant()`, `Date.from(instant)`, `java.sql.Date.valueOf(localDate)` and `GregorianCalendar.toZonedDateTime()`.

4.5.2 The core types



Duration: time-based amount; Period: date-based amount

Figure 4.2. The java.time types: a LocalDate/LocalTime pair has no zone and no instant; add a ZoneId or ZoneOffset to place it on the timeline.

Think of two worlds. The first is **machine time**: a single point on the timeline, with no calendar and no zone. `Instant` is that. The second is **human time**: the fields on a wall clock or calendar, which mean nothing until you say where. `LocalDate`, `LocalTime` and `LocalDateTime` are human time without a place; `ZonedDateTime` and `OffsetDateTime` are human time pinned to a place, so they correspond to exactly one instant.

Type	Holds	Typical use
`Instant`	Seconds and nanoseconds since 1970-01-01T00:00:00Z	Timestamps, event times, logs, database timestamps, durations between events
`LocalDate`	Year, month, day	Birthdays, due dates, holidays: things that have a date but no time or zone
`LocalTime`	Hour, minute, second, nano	Opening hours, alarm at 07:30
`LocalDateTime`	Date and time, no zone	Appointment in the location's own time; never an exact moment
`ZonedDateTime`	LocalDateTime + ZoneId (rules)	User-facing times that must respect DST (meetings, flights)
`OffsetDateTime`	LocalDateTime + fixed offset	Interchange with databases and JSON/REST (ISO 8601 with `+05:30`)
`Duration`	Seconds and nanos (time-based amount)	Elapsed time, timeouts
`Period`	Years, months, days (date-based amount)	Ages, subscription length, "3 months from now"
`ZoneId / ZoneOffset`	Region rules (`Asia/Kolkata`) / fixed offset (`+05:30`)	Choosing the zone

Which type for which job.

The next program creates each of them, does arithmetic and shows the results.

T1.java JAVA

```java
import java.time.*;
import java.time.format.*;
import java.time.temporal.ChronoUnit;
import java.time.temporal.TemporalAdjusters;
import java.util.Locale;

public class T1 {
    public static void main(String[] args) {
        LocalDate d = LocalDate.of(2024, 1, 31);
        System.out.println(d.plusMonths(1) + " " + d.plusMonths(1).isLeapYear()
            + " " + d.getDayOfWeek());
        System.out.println(d.with(TemporalAdjusters.lastDayOfMonth()) + " "
            + d.with(TemporalAdjusters.next(DayOfWeek.MONDAY)));
        LocalDateTime ldt = LocalDateTime.of(d, LocalTime.of(9, 30));
        System.out.println(ldt + " " + LocalTime.of(9, 30, 15));
        try {
            LocalDate.of(2023, 2, 29);
        } catch (DateTimeException e) {
            System.out.println(e.getMessage());
        }
```

T1.java (continued) JAVA

```
        Instant i = Instant.ofEpochSecond(1_700_000_000L);
        System.out.println(i + " " + i.toEpochMilli());
        ZonedDateTime ny = i.atZone(ZoneId.of("America/New_York"));
        ZonedDateTime kol = i.atZone(ZoneId.of("Asia/Kolkata"));
        System.out.println(ny + "\n" + kol);
        System.out.println(ny.toInstant().equals(kol.toInstant()) + " " + ny.toOffsetDateTime());

        Period p = Period.between(LocalDate.of(2020, 2, 29), LocalDate.of(2024, 3, 15));
        Duration du = Duration.between(LocalTime.of(8, 15), LocalTime.of(17, 45, 30));
        System.out.println(p + " " + p.getYears() + "y " + p.getMonths() + "m "
            + p.getDays() + "d");
        System.out.println(du + " " + du.toMinutes() + " min " + du.toHoursPart() + "h"
            + du.toMinutesPart() + "m");
        System.out.println(ChronoUnit.DAYS.between(
            LocalDate.of(2024, 1, 1), LocalDate.of(2025, 1, 1)));
    }
}
```

Output TERMINAL

```
$ java T1.java
2024-02-29 true WEDNESDAY
2024-01-31 2024-02-05
2024-01-31T09:30 09:30:15
Invalid date 'February 29' as '2023' is not a leap year
2023-11-14T22:13:20Z 1700000000000
2023-11-14T17:13:20-05:00[America/New_York]
2023-11-15T03:43:20+05:30[Asia/Kolkata]
true 2023-11-14T17:13:20-05:00
P4Y15D 4y 0m 15d
PT9H30M30S 570 min 9h30m
366
```

What to notice

- `plusMonths(1)` on 31 January gives 29 February: when the target day does not exist, `java.time` clamps to the last valid day instead of rolling over like `Calendar`. Every `java.time` object is immutable, so `d.plusMonths(1)` returns a new `LocalDate` and `d` stays `2024-01-31`.
- `TemporalAdjusters` express calendar logic ("last day of month", "next Monday") without hand-written loops.
- The `Instant` `1700000000` is a single moment. The same moment is 17:13 in New York (UTC-5 in November) and 03:43 the next day in Kolkata (+05:30). `equals` on the two `Instant`s is true. Compare `ZonedDateTime`s through `toInstant()` or `isBefore`/`isAfter`.
- `Period` is calendar-based (years, months, days). `Duration` is exact time-based (hours, minutes, seconds). Calling `Duration.between` on two `LocalDate`s throws `UnsupportedTemporalTypeException` because dates have no time-of-day; use `Period.between` or `ChronoUnit.DAYS.between`.
- The `PT9H30M30S` and `P4Y15D` strings are ISO 8601 duration syntax (`P` period, `T` time part).

NOTE
Why `Instant.toString()` ends in `Z`: it is ISO 8601 for UTC ("Zulu"). An `Instant` has no zone; it is printed in UTC. Call `atZone(zoneId)` to view it in another zone, and use `ZoneId.of("Region/City")`, never abbreviations like "IST" which are ambiguous.

4.5.3 Formatting and parsing with DateTimeFormatter

`DateTimeFormatter` is immutable and thread-safe, so keep formatters in `static final` fields. There are three ways to get one: the predefined ISO constants (`ISO_LOCAL_DATE`, `ISO_INSTANT`, `ISO_OFFSET_DATE_TIME`), localized styles (`ofLocalizedDate(FormatStyle.MEDIUM)`) and pattern strings via `ofPattern`.

Letter	Meaning	Example
`yyyy` / `uuuu`	year-of-era / proleptic year	`2025`
`MM` / `MMM` / `MMMM`	month number / short name / full name	`08` / `Aug` / `August`
`dd`	day of month	`05`

Letter	Meaning	Example
`EEE / EEEE`	day of week short / full	`Mon / Monday`
`HH / hh a`	hour 0-23 / hour 1-12 with am/pm marker	`17 / 05 PM`
`mm, ss, SSS`	minute, second, millisecond	`09, 07, 123`
`XXX / zzz / VV`	offset / zone abbreviation / zone id	`+05:30 / IST / Asia/Kolkata`

Common pattern letters (case matters).

The usual mistakes are `mm` (minutes) vs `MM` (months), `hh` vs `HH`, and `yyyy` vs `YYYY` (week-based year, which gives the wrong year around 31 December; never use `YYYY` unless you mean it). For names of months and days always pass a `Locale` so the output does not depend on the server.

Parsing has a subtle default called the **resolver style**. The default, `SMART`, repairs impossible values; `STRICT` rejects them; `LENIENT` rolls them over. The next program shows the effect of `31/04/2025`.

DateFmt.java JAVA

```java
import java.time.LocalDate;
import java.time.LocalDateTime;
import java.time.format.DateTimeFormatter;
import java.time.format.DateTimeParseException;
import java.time.format.ResolverStyle;
import java.util.Locale;

public class DateFmt {
    public static void main(String[] args) {
        LocalDateTime ldt = LocalDateTime.of(2024, 1, 31, 9, 30);
        DateTimeFormatter f = DateTimeFormatter.ofPattern("dd MMM yyyy, HH:mm", Locale.ENGLISH);
        System.out.println(ldt.format(f) + " | " + DateTimeFormatter.ISO_LOCAL_DATE.format(ldt));
        DateTimeFormatter smart = DateTimeFormatter.ofPattern("dd/MM/uuuu");
        System.out.println(LocalDate.parse("15/08/2025", smart));
        System.out.println(LocalDate.parse("31/04/2025", smart));
        DateTimeFormatter strict = smart.withResolverStyle(ResolverStyle.STRICT);
        try {
            LocalDate.parse("31/04/2025", strict);
        } catch (DateTimeParseException e) {
            System.out.println(e.getMessage());
        }
        DateTimeFormatter strictY = DateTimeFormatter.ofPattern("dd/MM/yyyy")
            .withResolverStyle(ResolverStyle.STRICT);
        try {
            LocalDate.parse("15/08/2025", strictY);
        } catch (DateTimeParseException e) {
            System.out.println(e.getMessage());
        }
    }
}
```

Output (the last message is one line; field order may vary; wrapped here) TERMINAL

```
$ java DateFmt.java
31 Jan 2024, 09:30 | 2024-01-31
2025-08-15
2025-04-30
Text '31/04/2025' could not be parsed: Invalid date 'APRIL 31'
Text '15/08/2025' could not be parsed: Unable to obtain LocalDate from
    TemporalAccessor: {MonthOfYear=8, YearOfEra=2025, DayOfMonth=15},ISO of type
    java.time.format.Parsed
```

With the default `SMART` style, `31/04/2025` is silently clamped to 30 April, which is rarely what validating user input wants. Use `ResolverStyle.STRICT` for input validation. The last line is a second trap: with `STRICT`, the pattern letter `yyyy` (year-of-era) cannot produce a date without an era, so parsing fails with a confusing message; with `STRICT`, write `uuuu`.

GOTCHA
Parsing a date-time with a formatter that lacks the needed fields fails at the end, not at the start: `LocalDate.parse("2025-01-01T10:00")` throws because the text contains a time, and `LocalDateTime.parse("2025-01-01")` throws because it has none. Pick the type that matches the text, or use `parseBest`.

4.5.4 Time zones and DST traps

A `ZoneId` such as `America/New_York` carries the zone's complete history of rules from the IANA time-zone database that ships inside the JDK. Daylight-saving transitions create two awkward situations: a **gap** (clocks jump forward, some local times do not exist) and an **overlap** (clocks jump back, some local times happen twice). `java.time` resolves both deterministically, as the program below shows for New York in 2024.

Dst.java JAVA

```java
import java.time.Duration;
import java.time.LocalDate;
import java.time.LocalDateTime;
import java.time.ZoneId;
import java.time.ZonedDateTime;
import java.time.temporal.ChronoUnit;

public class Dst {
    public static void main(String[] args) {
        ZoneId ny = ZoneId.of("America/New_York");
        ZonedDateTime before = ZonedDateTime.of(2024, 3, 9, 12, 0, 0, 0, ny);
        System.out.println("plusDays(1)  : " + before.plusDays(1));
        System.out.println("plusHours(24): " + before.plusHours(24));
        System.out.println("hours between: "
            + ChronoUnit.HOURS.between(before, before.plusDays(1)));
        ZonedDateTime gap = ZonedDateTime.of(LocalDateTime.of(2024, 3, 10, 2, 30), ny);
        System.out.println("gap  : " + gap);
        LocalDateTime amb = LocalDateTime.of(2024, 11, 3, 1, 30);
        ZonedDateTime first = ZonedDateTime.of(amb, ny);
        System.out.println("overlap first : " + first);
        System.out.println("overlap second: " + first.withLaterOffsetAtOverlap());
        System.out.println("day length: " + Duration.between(
            LocalDate.of(2024, 11, 3).atStartOfDay(ny),
            LocalDate.of(2024, 11, 4).atStartOfDay(ny)));
    }
}
```

Output TERMINAL

```
$ java Dst.java
plusDays(1)  : 2024-03-10T12:00-04:00[America/New_York]
plusHours(24): 2024-03-10T13:00-04:00[America/New_York]
hours between: 23
gap  : 2024-03-10T03:30-04:00[America/New_York]
overlap first : 2024-11-03T01:30-04:00[America/New_York]
overlap second: 2024-11-03T01:30-05:00[America/New_York]
day length: PT25H
```

Reading the output:

- On 10 March 2024 at 02:00 New York clocks jumped to 03:00. `plusDays(1)` keeps the wall-clock time (12:00 the next day) and `plusHours(24)` adds exactly 24 hours (13:00 on the wall clock). Between the two results there are only 23 real hours. Choose by intent: "same time tomorrow" is `plusDays`; "24 hours from now" is `plusHours` or `Duration`.
- 02:30 on that day does not exist. `ZonedDateTime.of` moves it forward by the length of the gap to 03:30.
- On 3 November 01:30 occurs twice. By default you get the earlier offset (-04:00); `withLaterOffsetAtOverlap()` selects the later one (-05:00). That day lasts 25 hours.

Rules that keep you out of trouble

1. Store and transmit moments as `Instant` (or UTC `OffsetDateTime`); convert to a zone only for display.
2. For future events in a user's local time (a meeting at 09:00 in Berlin next March), store the `LocalDateTime` plus the `ZoneId`, not the UTC instant, because zone rules can change before the event.

3. Never use `LocalDateTime` for a moment that must be unambiguous (log timestamps, payments, tokens). It has no zone, so it is not a point on the timeline.
4. Set the JVM or process zone deliberately (`-Duser.timezone=UTC` or `TZ`) and prefer to pass explicit `ZoneId`s.
5. Use `ZoneId.of("Asia/Kolkata")`, not `"IST"` and not fixed offsets, when you need DST or historical rules.
6. Do not compute age or durations by dividing milliseconds by 86,400,000. Use `Period`, `ChronoUnit.DAYS.between` or `Duration` on the right types.

INTERVIEW ANGLE

Which `java.time` class would you use to store the creation time of an order, and why? `Instant`, stored in UTC (a `timestamp with time zone` or `timestamptz` column, or an `OffsetDateTime` at UTC). It is an unambiguous point on the timeline and is independent of server and user time zones. Convert to `ZonedDateTime` for display in the user's zone. `LocalDateTime` is wrong because the same value means different moments in different zones.

4.5.5 Clock: testing code that uses the current time

Calling `LocalDate.now()` or `Instant.now()` directly inside business logic makes that logic impossible to test deterministically ("is the token expired?" depends on when the test runs). Every `now()` has an overload that accepts a `java.time.Clock`. Inject the `Clock` into your service: production uses `Clock.systemUTC()`, tests use `Clock.fixed(...)` or a small mutable clock.

Svc.java JAVA

```java
import java.time.Clock;
import java.time.Duration;
import java.time.Instant;
import java.time.ZoneId;
import java.time.ZoneOffset;

public class Svc {
    record Token(String value, Instant expires) {}

    static class TokenService {
        private final Clock clock;
        TokenService(Clock clock) { this.clock = clock; }
        Token issue(String v) { return new Token(v, clock.instant().plus(Duration.ofMinutes(30))); }
        boolean isValid(Token t) { return clock.instant().isBefore(t.expires()); }
    }

    static class MutableClock extends Clock {
        private Instant now;
        MutableClock(Instant start) { this.now = start; }
        void advance(Duration d) { now = now.plus(d); }
        @Override public ZoneId getZone() { return ZoneOffset.UTC; }
        @Override public Clock withZone(ZoneId z) { return this; }
        @Override public Instant instant() { return now; }
    }

    public static void main(String[] args) {
        MutableClock clock = new MutableClock(Instant.parse("2025-01-01T10:00:00Z"));
        TokenService svc = new TokenService(clock);
        Token t = svc.issue("abc");
        System.out.println(t);
        System.out.println("valid now: " + svc.isValid(t));
        clock.advance(Duration.ofMinutes(29));
        System.out.println("valid after 29 min: " + svc.isValid(t));
        clock.advance(Duration.ofMinutes(2));
        System.out.println("valid after 31 min: " + svc.isValid(t));
    }
}
```

Output TERMINAL

```
$ java Svc.java
Token[value=abc, expires=2025-01-01T10:30:00Z]
valid now: true
valid after 29 min: true
valid after 31 min: false
```

The test needed no `Thread.sleep` and runs in microseconds. In a Spring application declare a `Clock` bean and inject it; the same trick works for `Random` and UUID generators. For measuring elapsed time inside one JVM use

`System.nanoTime()` (monotonic, unaffected by clock adjustments), not `Instant.now()` differences, which can jump when the OS clock is corrected.

4.5.6 Mapping to databases, JSON and legacy code

- **JDBC 4.2 and later** can read and write `java.time` directly: `rs.getObject("created", OffsetDateTime.class)` and `ps.setObject(1, instant.atOffset(ZoneOffset.UTC))`. Map `DATE` to `LocalDate`, `TIME` to `LocalTime`, `TIMESTAMP` to `LocalDateTime` and `TIMESTAMP WITH TIME ZONE` to `OffsetDateTime`. Support for `Instant` directly varies by driver; going through `OffsetDateTime` at UTC is portable. The old `java.sql.Timestamp` converts with `Timestamp.from(instant)` and `toInstant()`.
- **JPA/Hibernate 5.2+** maps `java.time` types natively. Avoid the old `@Temporal` annotation, which is for `Date` and `Calendar`.
- **JSON with Jackson**: register `JavaTimeModule` (Spring Boot does) and disable `WRITE_DATES_AS_TIMESTAMPS` to get ISO 8601 strings like `2025-01-01T10:00:00Z` instead of numeric arrays. ISO strings are self-describing and sortable.
- **REST APIs**: send ISO 8601 with an offset or `Z` for moments, plain `2025-08-15` for dates. Avoid pre-formatted local strings like `15/08/2025`.
- **Legacy code**: convert at the boundary and keep `java.time` inside. `Date.toInstant()`, `Date.from(Instant)`, `Calendar.toInstant()`, `Timestamp.toLocalDateTime()`.

REAL WORLD
The classic production incident: a service stores `LocalDateTime.now()` in a database on a server in UTC, and a reporting job running in a different zone interprets the same values as local time, so every report is off by the zone offset, twice a year by an extra hour. Using `Instant` or UTC `OffsetDateTime` end to end removes the entire category of bug.

4.5.7 A worked example: billing dates and calendar arithmetic

Calendar rules appear everywhere in business code: subscriptions that renew monthly, due dates that move off weekends, birthdays on 29 February. The program below puts the earlier types to work and exposes two classic traps.

Bill.java JAVA

```
import java.time.DayOfWeek;
import java.time.LocalDate;
import java.time.YearMonth;
import java.time.temporal.TemporalAdjusters;
import java.util.ArrayList;
import java.util.List;
import java.util.Set;

public class Bill {
    static final Set<DayOfWeek> WEEKEND = Set.of(DayOfWeek.SATURDAY, DayOfWeek.SUNDAY);

    static LocalDate nextBusinessDay(LocalDate d) {
        LocalDate r = d;
        while (WEEKEND.contains(r.getDayOfWeek())) r = r.plusDays(1);
        return r;
    }
    public static void main(String[] args) {
        LocalDate start = LocalDate.of(2025, 1, 31);
        List<LocalDate> due = new ArrayList<>();
        for (int i = 0; i < 4; i++) due.add(nextBusinessDay(start.plusMonths(i)));
        System.out.println(due);
        // wrong: repeated plusMonths(1) drifts
        LocalDate d = start;
        for (int i = 0; i < 3; i++) d = d.plusMonths(1);
        System.out.println("iterated: " + d + "  direct: " + start.plusMonths(3));
        YearMonth ym = YearMonth.of(2025, 2);
        System.out.println(ym + " " + ym.lengthOfMonth() + " " + ym.atEndOfMonth()
            + " " + ym.atDay(1).getDayOfWeek());
        System.out.println(LocalDate.of(2025, 11, 1)
            .with(TemporalAdjusters.dayOfWeekInMonth(4, DayOfWeek.THURSDAY)));
        LocalDate birthday = LocalDate.of(2000, 2, 29);
        System.out.println(java.time.Period.between(birthday, LocalDate.of(2025, 2, 28)).getYears()
            + " " + java.time.Period.between(birthday, LocalDate.of(2025, 3, 1)).getYears());
    }
}
```

Output TERMINAL

```
$ java Bill.java
[2025-01-31, 2025-02-28, 2025-03-31, 2025-04-30]
iterated: 2025-04-28  direct: 2025-04-30
2025-02 28 2025-02-28 SATURDAY
2025-11-27
24 25
```

What to notice

- A monthly schedule starting on the 31st must be computed as `start.plusMonths(i)` from the *original* date. The result for February is the 28th (clamped), then March is back on the 31st. Adding one month repeatedly (`d = d.plusMonths(1)`) drifts: after 28 February the next addition gives 28 March, not 31 March, and the error never recovers (`2025-04-28` instead of `2025-04-30`).
- `nextBusinessDay` skips weekends using the `DayOfWeek` enum. Real systems add a holiday calendar (a `Set<LocalDate>`) to the same loop. Do not hard-code "add two days if Saturday" rules; loop until the date is valid.
- `YearMonth` models "a month of a year" (credit-card expiry, monthly reports) and knows its length and last day. `Year`, `MonthDay` (for birthdays and anniversaries) and `DayOfWeek` complete the family.
- `TemporalAdjusters.dayOfWeekInMonth(4, THURSDAY)` returns the fourth Thursday of the month: 27 November 2025, the date of US Thanksgiving.
- `Period.between` handles the leap-day birthday rule: someone born on 29 February 2000 is 24 years old on 28 February 2025 and 25 on 1 March 2025. If your business rule differs (some jurisdictions treat 28 February as the birthday), you must code that rule explicitly.

TIP

Write date logic as small pure functions that take a `LocalDate` (or a `Clock`) and return a value, and test them with a table of inputs that includes month ends, 29 February, daylight-saving changeover days and year boundaries. Those are the dates that break production, and they are cheap to test.

Check yourself

1. What does `LocalDate.of(2025, 1, 31).plusMonths(1)` return?
2. Why is `new Date(2025, 0, 1)` a bug, and what does a modern replacement look like?
3. What is the difference between `Duration` and `Period`? Which one can you build from two `LocalDate`s?
4. Which pattern letter pair gives the wrong year around 31 December: `yyyy` or `YYYY`?
5. In `America/New_York`, what is `ZonedDateTime.of(2024, 3, 9, 12, 0, 0, 0, ny).plusDays(1)` compared with `.plusHours(24)`, and why do they differ?
6. Which type would you put in a JSON API field for "when the invoice was paid", and which for "the customer's birthday"?

Answers

1. `2025-02-28`: the day-of-month is clamped to the last valid day.
2. The `Date(int, int, int)` constructor counts the year from 1900 and months from 0, so it means 3925-01-01 (it is deprecated). Use `LocalDate.of(2025, 1, 1)`.
3. `Period` is date-based (years, months, days); `Duration` is time-based (seconds and nanos) and needs a time-of-day. `Period.between(LocalDate, LocalDate)` works; `Duration.between` on two `LocalDate`s throws.
4. `YYYY` is the week-based year; use `yyyy` (or `uuuu`).
5. `plusDays(1)` keeps the wall-clock time (12:00 on 10 March); `plusHours(24)` adds exactly 24 real hours (13:00) because the clocks sprang forward at 02:00 that day.
6. An ISO 8601 instant (`Instant` or UTC `OffsetDateTime`) for the payment time; a `LocalDate` (`1999-04-12`) for the birthday.

KEY POINTS

Use the type that matches the meaning: `Instant` for moments, `LocalDate` for calendar dates, `ZonedDateTime` for user-facing local times, `Duration` and `Period` for amounts.

- All `java.time` types are immutable and thread-safe; arithmetic returns new objects. Months run 1 to 12 and invalid dates throw.
- Store and transmit UTC instants; convert to a zone only for display, and use region ids such as `Asia/Kolkata`.
- `DateTimeFormatter` is thread-safe; use `ResolverStyle.STRICT` and `uuuu` for validation, and pass a `Locale`.
- DST makes some local times missing or doubled and some days 23 or 25 hours long: `plusDays` and `plusHours` differ.
- Inject a `Clock` instead of calling `now()` in business logic so time-dependent code is testable.

PART 5

Exceptions, I/O and Utilities

Failing well, logging what happened, and talking to files, the operating system and other programs.

In this part

5.1 Exceptions
5.2 Logging and debugging basics
5.3 Classic I/O
5.4 NIO.2: Paths, Files and Channels
5.5 Serialization and Its Dangers
5.6 Properties, System and Runtime

5.1 Exceptions

Every program that does real work meets situations it cannot finish: the file is missing, the network drops, the user types "abc" where a number belongs, a bank account has too little money. A language needs a way to say "I could not do what you asked, and here is why", and the caller needs a way to react. Older languages used return codes (`-1`, `null`, `errno`). The trouble with return codes is that they are easy to ignore, they mix the answer with the error signal, and they do not say where the problem started. Java uses **exceptions**: an object that describes the problem is created, the normal flow stops, and the JVM unwinds the call stack looking for code that declared it can handle that kind of problem.

This chapter explains the exception class hierarchy, the checked versus unchecked debate, exactly what happens in `try`/`catch`/`finally` (including a trap that has cost real production systems dearly), automatic resource cleanup, how to design your own exceptions, how to read a stack trace, and how to use exceptions without hurting performance or clarity.

5.1.1 The Throwable hierarchy

Everything that can be thrown is a `java.lang.Throwable`. It has exactly two direct branches that matter:

- `Error`: a serious problem in the JVM or the environment that a normal application is not expected to recover from (`OutOfMemoryError`, `StackOverflowError`, `NoClassDefFoundError`, `AssertionError`).
- `Exception`: a problem your program may reasonably catch and handle. Inside it, `RuntimeException` and its subclasses are the **unchecked** exceptions; every other subclass of `Exception` is **checked**.

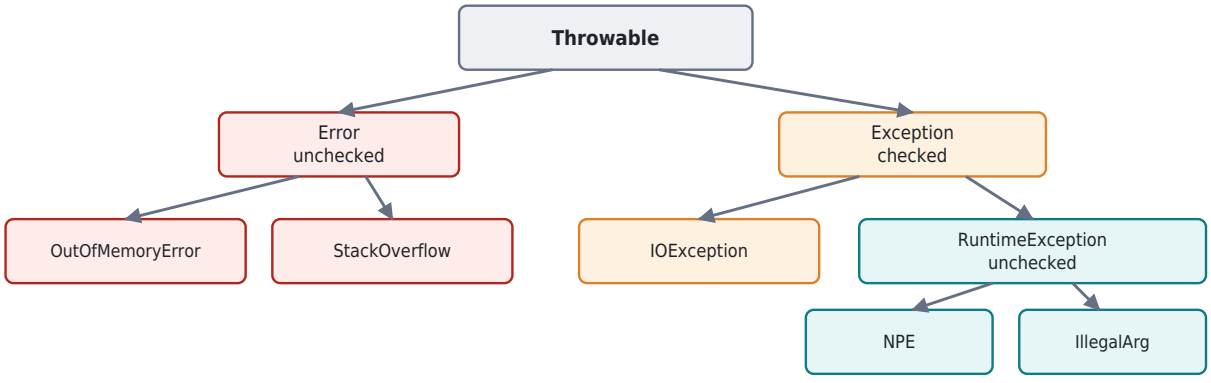


Figure 5.1. Throwable splits into Error and Exception. Only RuntimeException (and Error) are unchecked: the compiler does not force you to declare or catch them.

A `Throwable` carries four things you will use constantly: a **message** (`getMessage()`), an optional **cause** (`getCause()`, another `Throwable`), the **stack trace** captured when the object was created (`getStackTrace()`), and a list of **suppressed** exceptions (`getSuppressed()`, explained later).

Kind	Examples	Compiler rule
`Error`	`OutOfMemoryError`, `StackOverflowError`	None. Do not catch in normal code.

Kind	Examples	Compiler rule
Checked exception	IOException, SQLException, InterruptedException	Must be caught or declared with throws.
Unchecked exception	NullPointerException, IllegalArgumentException, IndexOutOfBoundsException	None. Usually a bug in the caller.

The three kinds of throwable.

5.1.2 Checked versus unchecked

The rule the compiler enforces is called **catch or specify**. If a method can throw a checked exception, it must either wrap the call in a `try`/`catch` or declare `throws` in its signature, which pushes the obligation to its own callers. Unchecked exceptions and errors are exempt.

The original idea was sound: exceptions the caller can reasonably recover from (file not found, network timeout) should be visible in the type system so nobody forgets about them. The intended split was:

- **Checked** for conditions outside the program's control that a caller can recover from: `FileNotFoundException`, `SocketTimeoutException`.
- **Unchecked** for programming errors that should be fixed rather than handled: a `null` where an object was required, an index out of range, an illegal argument.

In practice the debate is lively. Checked exceptions leak implementation details through every layer (a service that declares `throws SQLException` forces its callers to know it uses a database). They interact badly with lambdas, because the standard functional interfaces declare no checked exceptions, so `list.forEach(x -> Files.delete(x))` does not compile. Many frameworks, Spring being the best-known example, therefore wrap checked exceptions in unchecked ones. Others, including the JDK file API designers, still prefer checked exceptions for I/O.

A practical stance that works well in application code:

1. Use unchecked exceptions for violated preconditions (`IllegalArgumentException`, `IllegalStateException`) and for failures the caller cannot meaningfully recover from.
2. Use a checked exception when the caller has a real, different thing to do on failure (retry, ask the user for another file name, fall back to a default) and you want the compiler to remind them.
3. When you call a library that throws checked exceptions you cannot handle at this level, wrap them in an unchecked exception of your own with the original as the cause, instead of declaring `throws Exception` everywhere.

5.1.3 try, catch and finally

The general shape is:

Shape.java JAVA

```java
import java.io.IOException;

public class Shape {
    static void risky() throws IOException { throw new IOException("boom"); }

    public static void main(String[] args) {
        try {
            risky();
        } catch (IOException e) {
            System.out.println("handled: " + e.getMessage());
        } finally {
            System.out.println("always runs");
        }
    }
}
```

The `try` block holds the code that may fail. The first `catch` clause whose parameter type matches the thrown object (the type itself or a superclass of it) runs; later ones are skipped. `finally`, if present, runs afterwards no matter what happened: normal completion, a caught exception, an uncaught exception, or a `return`, `break` or `continue` inside the block.

Order of catch clauses

Catch clauses are tried from top to bottom, so the more specific type must come first. Putting `catch (Exception e)` before `catch (IOException e)` is a compile error ("exception IOException has already been caught"), because the second clause could never run.

Multi-catch

When two exception types need the same handling, join them with `|`:

MultiCatch.java JAVA

```java
public class MultiCatch {
    public static void main(String[] args) {
        String[] inputs = {"42", "abc", null};
        for (String s : inputs) {
            try {
                int n = Integer.parseInt(s.trim());
                System.out.println("parsed " + n);
            } catch (NumberFormatException | NullPointerException e) {
                System.out.println("bad input '" + s + "': " + e.getClass().getSimpleName());
            }
        }
    }
}
```

Run it TERMINAL

```
$ java MultiCatch.java
parsed 42
bad input 'abc': NumberFormatException
bad input 'null': NullPointerException
```

The alternatives in a multi-catch cannot be related by inheritance (you cannot write `IOException | Exception`), and the parameter `e` is implicitly `final`.

The finally trap: return inside finally

Because `finally` always runs, a `return` or a `throw` inside it overrides whatever the `try` block was doing. This is legal Java and almost always a bug. Study this program carefully before running it.

FinallyOrder.java JAVA

```java
public class FinallyOrder {
    static int f() {
        try {
            System.out.println("try");
            return 1;
        } finally {
            System.out.println("finally");
        }
    }

    @SuppressWarnings("finally")
    static int g() {
        try {
            throw new IllegalStateException("lost");
        } finally {
            return 2;   // swallows the exception
        }
    }

    static int h() {
        int x = 10;
        try {
            return x;
        } finally {
            x = 99;     // does not change the value already chosen to return
        }
    }

    public static void main(String[] args) {
        System.out.println(f());
```

FinallyOrder.java (continued) JAVA

```
        System.out.println(g());
        System.out.println(h());
    }
}
```

Run it TERMINAL

```
$ java FinallyOrder.java
try
finally
1
2
10
```

What to notice:

- In `f`, the value `1` is computed first, then `finally` runs, then the method returns `1`. The output order is `try`, `finally`, `1`.
- In `g`, the `IllegalStateException` is thrown, but the `return 2` in `finally` cancels it. The caller never learns anything went wrong. This is the trap: **never `return`, `break`, `continue` or `throw` from a `finally` block**. (javac warns about it with `-Xlint:finally`.)
- In `h`, the return value `10` was copied before `finally` ran, so changing the local variable afterwards has no effect. For a mutable object, however, `finally` could still change the object's contents.

The only situations in which `finally` does not run are the JVM stopping (`System.exit`, a crash, power loss) or the thread being stuck forever in the `try` block.

Execution order, summarised

Situation	Order
No exception	`try` block, then `finally`, then code after
Exception caught	`try` up to the throw, matching `catch`, `finally`, code after
Exception not caught	`try` up to the throw, `finally`, then the exception continues to the caller
Exception inside `catch`	`finally` still runs, then the new exception propagates

What runs, in order.

5.1.4 try-with-resources

A resource is anything that must be released: a file, a socket, a database connection. The old pattern was a `try` with a `finally` that calls `close()`, and it was easy to get wrong: if `close()` itself throws in `finally`, it hides the original exception, and with two resources the second `close()` might never be reached.

Since Java 7, **try-with-resources** handles this. Any class that implements `java.lang.AutoCloseable` (or its sub-interface `java.io.Closeable`) can be declared in the parentheses. The JVM closes the resources automatically when the block ends, **in reverse order of declaration**, whether it ended normally or by exception.

Suppressed.java JAVA

```
public class Suppressed {
    static class Res implements AutoCloseable {
        private final String name;
        Res(String name) { this.name = name; System.out.println("open " + name); }
        void work() { throw new IllegalStateException("work failed on " + name); }
        @Override public void close() {
            System.out.println("close " + name);
            throw new IllegalArgumentException("close failed on " + name);
        }
    }

    public static void main(String[] args) {
        try (Res a = new Res("A"); Res b = new Res("B")) {
            b.work();
        } catch (Exception e) {
            System.out.println("caught: " + e);
            for (Throwable s : e.getSuppressed()) {
                System.out.println("  suppressed: " + s);
```

Suppressed.java (continued) JAVA

```
            }
        }
    }
}
```

Run it TERMINAL

```
$ java Suppressed.java
open A
open B
close B
close A
caught: java.lang.IllegalStateException: work failed on B
  suppressed: java.lang.IllegalArgumentException: close failed on B
  suppressed: java.lang.IllegalArgumentException: close failed on A
```

What to notice:

- Resources close in reverse order: `B` before `A`, the way you unwind nested try/finally blocks.
- Three exceptions happened, but only one can propagate. Java keeps the **primary** exception (the one from the body) and attaches the failures from `close()` to it as **suppressed** exceptions, reachable through `getSuppressed()`. With a hand-written `finally`, the close failure would have replaced the real cause.
- The `catch` and `finally` clauses of a try-with-resources run after the resources are closed.

Since Java 9 you can also use an existing effectively-final variable: `try (res) { ... }`.

GOTCHA

The resource variable is read-only inside the block. Assigning to `a` in the example above is a compile error, because the variable is implicitly `final`. If the constructor of a later resource throws, the earlier ones are still closed. If you declare a resource without try-with-resources, nothing closes it for you, even when the garbage collector later reclaims the object.

INTERVIEW ANGLE

What is the difference between `final`, `finally` and `finalize`? `final` is a modifier (a constant, a class that cannot be extended, a method that cannot be overridden). `finally` is the block that runs after `try`/`catch`. `finalize` was an `Object` method called by the garbage collector before reclaiming an object; it is deprecated for removal since Java 18 and should never be used. Use try-with-resources or `java.lang.ref.Cleaner` instead.

5.1.5 Designing your own exceptions

Create a custom exception when callers need to tell a failure apart from the standard ones, or when you want to carry extra data. Extend `Exception` for a checked exception, `RuntimeException` for an unchecked one. Provide the constructors callers expect, especially one that takes a **cause**.

Chain.java JAVA

```
public class Chain {
    static class AccountException extends Exception {
        private final String accountId;
        AccountException(String message, String accountId, Throwable cause) {
            super(message, cause);
            this.accountId = accountId;
        }
        String accountId() { return accountId; }
    }

    static void low() throws java.io.IOException {
        throw new java.io.IOException("disk full");
    }

    static void mid() throws AccountException {
        try {
            low();
        } catch (java.io.IOException e) {
            throw new AccountException("cannot save account", "ACC-7", e);
        }
    }

    public static void main(String[] args) {
```

Chain.java (continued) JAVA

```
        try {
            mid();
        } catch (AccountException e) {
            System.out.println(e.getMessage() + " for " + e.accountId());
            System.out.println("cause: " + e.getCause());
            e.printStackTrace(System.out);
        }
    }
}
```

Compile and run (so the stack trace has no launcher frames) TERMINAL

```
$ javac Chain.java && java Chain
cannot save account for ACC-7
cause: java.io.IOException: disk full
Chain$AccountException: cannot save account
        at Chain.mid(Chain.java:20)
        at Chain.main(Chain.java:26)
Caused by: java.io.IOException: disk full
        at Chain.low(Chain.java:14)
        at Chain.mid(Chain.java:18)
        ... 1 more
```

Exception chaining

Passing the original exception as the **cause** is called chaining. A layer (the account service) translates a low-level failure (I/O) into one that makes sense at its own level of abstraction, without losing the evidence. The line `Caused by:` in the trace shows the chain, and `... 1 more` means one frame is shared with the trace above, so it is not repeated.

WATCH OUT

Swallowing the cause. `throw new AccountException("cannot save account")` without the cause throws away the only clue about why. The log then says "cannot save account" and nobody can tell whether the disk was full or the path was wrong. Always pass the original exception along.

Rules for good custom exceptions

- Name them after the problem, not the layer: `InsufficientFundsException`, not `BankServiceException2`.
- Remember that every `Throwable` is `Serializable`, so keep fields to simple, serializable types.
- Put useful data in fields (account id, requested amount), but never put secrets such as passwords in the message; messages end up in logs.
- Do not create one exception class per error code. A small hierarchy of three to six classes per module is usually enough.

5.1.6 Reading and producing stack traces

A **stack trace** is the list of method calls that were active when the exception object was **created** (not thrown). Reading it is a core skill. Read from the top: the first line gives the exception type and message, and the first `at` line is where it happened.

A typical trace, annotated

```
Exception in thread "main" java.lang.IllegalStateException: account is closed
    at com.example.bank.Account.withdraw(Account.java:42)      <- where it was created
    at com.example.bank.Transfer.run(Transfer.java:18)         <- who called it
    at com.example.bank.Main.main(Main.java:9)                 <- entry point
Caused by: java.sql.SQLException: connection reset             <- the underlying reason
    at ...
```

Since Java 15 the JVM produces **helpful NullPointerException messages** that name the exact expression that was `null`:

Typical output of a null in a chained call (message wrapped here for width) TERMINAL

```
$ java Npe.java
Exception in thread "main" java.lang.NullPointerException:
  Cannot invoke "java.util.Map.get(Object)" because the return value of
  "Npe$Customer.tags()" is null
        at Npe.main(Npe.java:6)
```

printStackTrace versus logging

`e.printStackTrace()` writes to standard error, which in production is either lost or interleaved with other output, carries no timestamp, level or thread name, and cannot be filtered. It is fine in a five-line experiment and wrong in an application. Log the exception with a logger instead (the next chapter) and pass the exception object itself, so the logger prints the whole trace including `Caused by`:

LogPattern.java JAVA

```java
import java.lang.System.Logger;
import java.lang.System.Logger.Level;

public class LogPattern {
    private static final Logger LOG = System.getLogger("LogPattern");

    public static void main(String[] args) {
        try {
            Integer.parseInt("12x");
        } catch (NumberFormatException e) {
            LOG.log(Level.ERROR, "could not read quantity", e);   // good: message + exception
            // LOG.log(Level.ERROR, e.getMessage());               // bad: loses the stack trace
        }
    }
}
```

Two more rules: **log or throw, not both** (otherwise every layer logs the same failure and the log fills with duplicates), and use `Thread.getAllStackTraces()` or `jstack` when you need the stack of a running thread rather than of an exception (see the Concurrency part).

5.1.7 Rethrowing and precise rethrow

You may catch an exception, do something (log, roll back), and throw it again with `throw e;`. Since Java 7, if `e` is effectively final the compiler tracks the **precise** types that can really come from the `try` block, so you can catch `Exception` and still declare only the specific checked exceptions in `throws`:

Rethrow.java JAVA

```java
import java.io.IOException;

public class Rethrow {
    static void read() throws IOException {
        try {
            if (System.nanoTime() > 0) throw new IOException("unreadable");
        } catch (Exception e) {
            System.out.println("cleaning up");
            throw e;          // compiles: only IOException (and unchecked) can arrive here
        }
    }

    public static void main(String[] args) {
        try { read(); } catch (IOException e) { System.out.println("main got " + e.getMessage()); }
    }
}
```

Run it TERMINAL

```
$ java Rethrow.java
cleaning up
main got unreadable
```

5.1.8 assert

An `assert` statement checks an assumption about your own code during development:

Assert.java JAVA

```java
public class Assert {
    static int half(int n) {
        assert n % 2 == 0 : "n must be even but was " + n;
        return n / 2;
    }
    public static void main(String[] args) {
        System.out.println(half(7));
    }
}
```

Assertions are off by default; -ea turns them on TERMINAL

```
$ java Assert.java
3
$ java -ea Assert.java
Exception in thread "main" java.lang.AssertionError: n must be even but was 7
        at Assert.half(Assert.java:3)
        at Assert.main(Assert.java:7)
```

Assertions are **disabled by default**; enable them with `-ea` (or `-ea:com.example...` for one package, `-esa` for system classes). Because they may be off, never put a side effect in an assertion (`assert list.remove(x);` silently does nothing when assertions are off), and never use them to validate arguments of public methods or user input. For those, throw `IllegalArgumentException` or use `Objects.requireNonNull`. Use `assert` for internal invariants: "this branch cannot happen", "this list is sorted at this point". Testing frameworks run with `-ea` on, which is why assertions help in tests. `AssertionError` is an `Error`; do not catch it.

5.1.9 Performance of exceptions

Throwing is slower than returning a value, and the dominant cost is **capturing the stack trace**: the constructor of `Throwable` calls `fillInStackTrace()`, which walks the whole stack. Deeper stacks cost more. The JIT compiler can sometimes optimise a throw and a catch in the same compiled method into a plain jump, but you cannot rely on that.

The program below throws and catches 200,000 exceptions from a recursion 50 frames deep, once with a normal exception and once with an exception whose stack trace writing is disabled via the four-argument `RuntimeException` constructor (`writableStackTrace = false`).

Cost.java JAVA

```java
public class Cost {
    static class Quiet extends RuntimeException {
        Quiet() { super("q", null, false, false); }
    }
    static int sink;
    static void deep(int n, boolean quiet) {
        if (n == 0) throw quiet ? new Quiet() : new RuntimeException("x");
        deep(n - 1, quiet);
    }
    static long run(boolean quiet) {
        long t = System.nanoTime();
        for (int i = 0; i < 200_000; i++) {
            try { deep(50, quiet); } catch (RuntimeException e) { sink++; }
        }
        return (System.nanoTime() - t) / 1_000_000;
    }
    public static void main(String[] args) {
        run(false); run(true); // warm up
        System.out.println("with stack trace   : " + run(false) + " ms");
        System.out.println("without stack trace: " + run(true) + " ms");
    }
}
```

Typical output (measured on the build machine; numbers vary) TERMINAL

```
$ java Cost.java
with stack trace   : 5496 ms
without stack trace: 2455 ms
```

Even without the trace, unwinding 50 frames is not free, and the exact numbers depend on the machine and JIT state, but the stack trace roughly doubled the cost here. The lessons are practical, not "avoid exceptions":

- Do not use exceptions for normal control flow (for example, parsing every line of a file with `Integer.parseInt` and catching failures as the way to detect non-numbers when 90% of lines are non-numbers). Test first, or use a method that returns a result.
- A try block that never throws costs essentially nothing at run time; the cost is only paid when something is thrown.
- Disabling stack traces (`writableStackTrace = false`) is an advanced trick for exceptions used as high-volume signals inside a library. Never do it for exceptions that people need to debug.

5.1.10 Errors: StackOverflowError and OutOfMemoryError

`Error` subclasses signal that the JVM is in trouble. Two you will meet during study:

Errs.java JAVA

```java
import java.util.ArrayList;
import java.util.List;

public class Errs {
    static int depth = 0;
    static void recurse() { depth++; recurse(); }

    public static void main(String[] args) {
        try { recurse(); } catch (StackOverflowError e) {
            System.out.println("SOE caught, depth > 1000: " + (depth > 1000));
        }
        try {
            List<long[]> l = new ArrayList<>();
            while (true) l.add(new long[10_000_000]);
        } catch (OutOfMemoryError e) {
            System.out.println("OOM: " + e.getMessage());
        }
    }
}
```

Run with a small heap so it fails fast TERMINAL

```
$ java -Xmx64m Errs.java
SOE caught, depth > 1000: true
OOM: Java heap space
```

- `StackOverflowError`: a thread's stack is full, nearly always from runaway recursion. The depth reached depends on the stack size (`-Xss`) and the frame size, so do not print a depth and expect the same number twice.
- `OutOfMemoryError`: the message after the colon says which resource ran out: `Java heap space`, `Metaspace`, `GC overhead limit exceeded`, `unable to create native thread`, `Direct buffer memory`. The Part on the JVM in depth explains how to diagnose each.

Catching these in a demo is fine. In an application, catching `Throwable` or `Error` and carrying on is dangerous because the program may be in an inconsistent state. A better strategy is to let the process die (and let a supervisor restart it), perhaps after writing a heap dump with `-XX:+HeapDumpOnOutOfMemoryError`.

5.1.11 Best practices

- **Catch specific exceptions.** `catch (Exception e)` hides bugs; reserve it for the outermost layer (a request handler, a thread's top level) where you log and turn the failure into a response.
- **Never write an empty catch block.** At minimum log it. If you really intend to ignore an exception, name the variable `ignored` and add a comment explaining why it is safe.
- **Fail fast.** Validate arguments at the start of a method (`Objects.requireNonNull(x, "x")`, `if (amount <= 0) throw new IllegalArgumentException(...)`) so errors appear close to their cause.
- **Include context in the message**: what you tried, with which values. "Cannot withdraw 500 from ACC-7: balance 120" beats "invalid state".
- **Clean up with try-with-resources**, not by hand.
- **Preserve the cause** when wrapping, and **do not both log and rethrow** at every layer.
- **Do not catch** `InterruptedException` **and ignore it.** Either propagate it or restore the flag with `Thread.currentThread().interrupt()` (see the Concurrency part).
- **Prefer standard exceptions** (`IllegalArgumentException`, `IllegalStateException`, `UnsupportedOperationException`, `NoSuchElementException`, `ArithmeticException`) before inventing new ones.

- **Do not return `null` to signal an error** when an `Optional` or an exception would be clearer.

REAL WORLD
In Spring applications, a `@ControllerAdvice` class with `@ExceptionHandler` methods is the single place where exceptions turn into HTTP responses (an `InsufficientFundsException` becomes a 409, a `NoSuchElementException` a 404). The service layer just throws meaningful, mostly unchecked, exceptions with a cause. This is the "catch at the boundary, throw near the problem" rule in action.

INTERVIEW ANGLE
What happens if an exception is thrown in a `finally` block while another is already propagating? The new exception replaces the original; the original is lost (it is not even attached as suppressed, which only try-with-resources does automatically). That is why `finally` should contain only simple cleanup that cannot throw, and why try-with-resources is preferred.

Check yourself

1. What does this print: `static int m() { try { throw new RuntimeException(); } catch (Exception e) { return 1; } finally { System.out.print("F "); } }` followed by `System.out.println(m());`?
2. What is wrong with `catch (Exception e) { ... } catch (IOException e) { ... }` in that order?
3. Which of these are checked: `IOException`, `IllegalStateException`, `OutOfMemoryError`, `InterruptedException`?
4. What does this print: `StringBuilder sb = new StringBuilder("a"); try { return sb.append("b").toString(); } finally { sb.append("c"); }` (the method returns the String)?
5. A try-with-resources body throws A and closing the resource throws B. Which exception does the caller see, and where is the other one?
6. Why does `assert list.remove(x);` count as a bug?

Answers

1. `F 1`. The `catch` chooses the return value `1`, `finally` prints `F ` before the method actually returns, and the caller prints `1`.
2. It does not compile: the second clause is unreachable because `Exception` already catches every `IOException`. Put the specific type first.
3. `IOException` and `InterruptedException` are checked. `IllegalStateException` is unchecked (a `RuntimeException`) and `OutOfMemoryError` is an `Error`, so also unchecked.
4. The method returns `ab`. The `String` was produced before `finally` ran, and appending `c` to the builder afterwards does not change it (the program prints `ab`).
5. The caller sees A (the primary exception). B is attached to A and is available from `A.getSuppressed()`.
6. Assertions are disabled by default, so the `remove` would never run in a normal run. Assertions must have no side effects.

KEY POINTS
Exceptions are objects; `Error` and `RuntimeException` are unchecked, every other `Exception` is checked.

- Use unchecked exceptions for bugs and violated preconditions, checked ones only when callers have a real recovery.
- `finally` always runs; never `return` or `throw` from it, because that discards the pending result or exception.
- Prefer try-with-resources: reverse-order closing, and close failures become suppressed exceptions instead of hiding the real one.
- Wrap with a cause, log with the exception object, and log or throw, not both. Never leave a catch block empty.
- `assert` is for internal invariants and is off unless you pass `-ea`.

5.2 Logging and debugging basics

When a program runs on a server at 3 a.m., you cannot attach your IDE and you cannot ask the user what they clicked. The only witness is the **log**: a time-ordered record of what the program decided to tell you. This chapter shows how Java logs out of the box, how the real-world logging stack is organised (so that Spring Boot configuration stops looking like magic), and how to find a bug interactively with a debugger when logs are not enough.

5.2.1 Why not System.out.println?

`System.out.println` is fine for learning. For anything else it fails in practical ways: no timestamp, no severity, no thread name, no class name, no way to turn it off without editing code, no routing to a file with rotation, and no way to change the amount of detail for one package while the system is running. A logging framework gives you all of that through configuration instead of code changes.

Log levels

Levels rank messages by importance. A logger has a threshold; messages below it are discarded cheaply. Java's built-in `java.util.logging` (JUL) uses seven main levels; most other frameworks use the five-level scheme TRACE, DEBUG, INFO, WARN, ERROR.

JUL level	Common name	Use for
`SEVERE`	ERROR	The operation failed and someone should look. Always include the exception.
`WARNING`	WARN	Something unexpected but handled: a retry, a fallback, a deprecated call.
`INFO`	INFO	Business milestones: server started, order placed. Readable by operators.
`CONFIG`	INFO or DEBUG	Configuration details at startup.
`FINE`	DEBUG	Diagnostic detail for developers. Off in production by default.
`FINER` / `FINEST`	TRACE	Very fine detail such as method entry and exit.

Levels and what they are for.

A sensible default for production is INFO for your own code and WARNING for libraries; raise one package to DEBUG when investigating a problem.

5.2.2 System.Logger: the logging API inside the JDK

Java 9 added `System.Logger`, a minimal facade so that the JDK itself and your code can log without depending on a particular framework. You obtain one with `System.getLogger(name)`. By default it forwards to `java.util.logging`; frameworks can plug in their own backend.

SysLog.java JAVA

```java
import java.lang.System.Logger;
import java.lang.System.Logger.Level;

public class SysLog {
    private static final Logger LOG = System.getLogger("com.example.bank.Transfer");

    public static void main(String[] args) {
        LOG.log(Level.DEBUG, "debug is below the default threshold");
        LOG.log(Level.INFO, "transfer started: {0} -> {1}", "ACC-1", "ACC-2");
        LOG.log(Level.WARNING, "balance low on {0}", "ACC-1");
        try {
            Integer.parseInt("abc");
        } catch (NumberFormatException e) {
            LOG.log(Level.ERROR, "transfer failed", e);
        }
    }
}
```

Typical output (goes to standard error; the date changes) TERMINAL

```
$ java SysLog.java
Oct 10, 2026 8:01:23 PM SysLog main
INFO: transfer started: ACC-1 -> ACC-2
Oct 10, 2026 8:01:23 PM SysLog main
WARNING: balance low on ACC-1
Oct 10, 2026 8:01:23 PM SysLog main
SEVERE: transfer failed
java.lang.NumberFormatException: For input string: "abc"
        at java.base/java.lang.NumberFormatException.forInputString(NumberFormatException.java:67)
        at java.base/java.lang.Integer.parseInt(Integer.java:662)
        at SysLog.main(SysLog.java:12)
        ... (launcher frames omitted)
```

What to notice:

- The `DEBUG` message vanished: the default threshold is INFO.
- `{0}` and `{1}` are placeholders in `java.text.MessageFormat` style. The message is only formatted if the level is enabled, which avoids building strings nobody will read.
- Passing the exception as the last argument prints the full trace, as recommended in the chapter on exceptions.
- The "SysLog main" in the first line is the source class and method, which JUL infers by inspecting the stack: handy, but one reason it is slower than frameworks that do not.

5.2.3 java.util.logging in more detail

JUL has four ideas worth knowing:

- **Logger**: named with dots (`com.example.bank.Transfer`). Loggers form a tree: the logger `com.example.bank` is the parent of `com.example.bank.Transfer`, and a child inherits the parent's level and handlers unless overridden. The root logger (named `""`) sits at the top. Use the class name as the logger name so the tree mirrors your packages.
- **Level**: the threshold on a logger and on each handler. A record is published only if it passes both.
- **Handler**: where records go. `ConsoleHandler` (standard error), `FileHandler`, `StreamHandler`, `SocketHandler`, `MemoryHandler`.
- **Formatter**: turns a record into text: `SimpleFormatter` or `XMLFormatter`, or your own subclass.

Jul.java JAVA

```java
import java.util.logging.ConsoleHandler;
import java.util.logging.Handler;
import java.util.logging.Level;
import java.util.logging.LogRecord;
import java.util.logging.Logger;
import java.util.logging.SimpleFormatter;

public class Jul {
    private static final Logger LOG = Logger.getLogger("com.example.bank");

    public static void main(String[] args) {
        LOG.setUseParentHandlers(false);
        Handler h = new ConsoleHandler();
        h.setLevel(Level.ALL);
        h.setFormatter(new SimpleFormatter() {
            @Override public String format(LogRecord r) {
                return r.getLevel() + " [" + r.getLoggerName() + "] " + r.getMessage() + "\n";
            }
        });
        LOG.addHandler(h);
        LOG.setLevel(Level.FINE);

        LOG.finest("finest: dropped by logger level");
        LOG.fine("fine: kept");
        LOG.info("info: kept");
        LOG.log(Level.SEVERE, "severe: {0}", "payload");
        LOG.fine(() -> "lazy message built only if FINE is on");
    }
}
```

Run it TERMINAL

```
$ java Jul.java
FINE [com.example.bank] fine: kept
INFO [com.example.bank] info: kept
SEVERE [com.example.bank] severe: {0}
FINE [com.example.bank] lazy message built only if FINE is on
```

What to notice:

- `FINEST` was dropped by the logger's level (`FINE`), even though the handler would have accepted it.
- `severe: {0}` printed literally: `r.getMessage()` returns the raw template. The built-in `SimpleFormatter` calls `formatMessage(record)` to substitute parameters; my custom formatter forgot to. If you write a formatter, call `formatMessage(r)`.
- The overload taking a `Supplier<String>` (`LOG.fine(() -> ...)`) builds the message only when the level is enabled. That is the JUL way to avoid the cost of expensive string building, as an alternative to `if (LOG.isLoggable(Level.FINE))`.

Configuration without code

In real use you do not call `addHandler` yourself. JUL reads a properties file, by default `conf/logging.properties` inside the JDK; point to your own with a system property.

logging.properties PROPERTIES

```
handlers = java.util.logging.ConsoleHandler
.level = WARNING
com.example.bank.level = FINE
java.util.logging.ConsoleHandler.level = ALL
java.util.logging.ConsoleHandler.formatter = java.util.logging.SimpleFormatter
java.util.logging.SimpleFormatter.format = %1$tT %4$-7s [%3$s] %5$s%6$s%n
```

Cfg.java JAVA

```
import java.util.logging.Logger;

public class Cfg {
    public static void main(String[] args) {
        Logger bank = Logger.getLogger("com.example.bank.Transfer");
        Logger other = Logger.getLogger("org.thirdparty.Lib");
        bank.fine("bank fine");
        bank.info("bank info");
        other.info("other info (hidden: root level WARNING)");
        other.warning("other warning");
    }
}
```

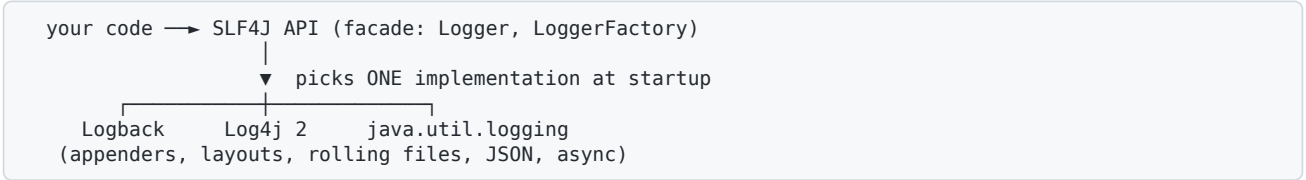
Typical output (times vary) TERMINAL

```
$ java -Djava.util.logging.config.file=logging.properties Cfg.java
20:05:54 FINE    [com.example.bank.Transfer] bank fine
20:05:54 INFO    [com.example.bank.Transfer] bank info
20:05:54 WARNING [org.thirdparty.Lib] other warning
```

The root level is `WARNING`, so the third-party `info` line is hidden, while everything under `com.example.bank` is shown down to `FINE`. This is the same idea as `logging.level.com.example.bank=DEBUG` in a Spring Boot `application.properties`: set a level for a package prefix.

5.2.4 The real-world stack: facade plus implementation

Most Java applications do not call JUL directly. Over the years several logging libraries appeared (Log4j, Logback, Log4j 2, commons-logging), and libraries you depend on each picked a different one, so an application could end up with four log streams. The industry's answer is the **facade** pattern:



SLF4J (Simple Logging Facade for Java) is the interface; Logback or Log4j 2 does the work. Spring Boot uses SLF4J with Logback by default. The code you write looks like this (shown as text because it needs a library that is not part of the JDK):

OrderService.java JAVA

```
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;

public class OrderService {
    private static final Logger log = LoggerFactory.getLogger(OrderService.class);

    void place(String orderId, int items) {
        log.info("placing order {} with {} items", orderId, items);   // {} placeholders
        try {
            charge(orderId);
        } catch (RuntimeException e) {
            log.error("payment failed for order {}", orderId, e);     // exception last
        }
    }
    void charge(String id) { }
}
```

Why `{}` placeholders matter: `log.debug("x=" + x)` builds the string even when debug is off; `log.debug("x={}", x)` builds it only if enabled. Another feature worth knowing is the **MDC** (Mapped Diagnostic Context), a per-thread map (for example a request id) that the layout prints on every line, so you can grep all log lines of one request.

WATCH OUT
Never log secrets or personal data: passwords, tokens, card numbers, full request bodies. Logs are copied to many systems and kept for months. Also avoid logging inside tight loops, and avoid log-and-rethrow at every layer.

INTERVIEW ANGLE
What is SLF4J and why use it instead of Logback directly? SLF4J is a logging facade: application and library code depend only on its API, and the actual implementation (Logback, Log4j 2) is chosen at deployment by what is on the classpath. Libraries can log without forcing a framework on their users, and you can switch implementations without changing code.

5.2.5 Debugging basics

Logs tell you what happened; a **debugger** lets you stop the program and look around. Every Java IDE (IntelliJ IDEA, Eclipse, VS Code with the Java extension) drives the same JVM mechanism, the Java Debug Wire Protocol (JDWP). The vocabulary is the same everywhere:

- **Breakpoint**: a line where execution pauses. A **conditional breakpoint** pauses only when an expression is true (`accountId.equals("ACC-7")`), which saves you from clicking Resume 5,000 times.
- **Step over**: run the current line, staying in this method. **Step into**: enter the method called on this line. **Step out**: run until the current method returns. **Resume**: run to the next breakpoint.
- **Variables / Watches / Evaluate expression**: inspect values, add expressions that are re-evaluated at each stop, or run an arbitrary expression in the current frame.
- **Call stack**: pick any frame to see its local variables. This is the live version of the stack trace.
- **Exception breakpoint**: pause whenever (or only when uncaught) a given exception type is thrown, even deep in a library.

A productive habit is to form a hypothesis first ("the discount is negative because `rate` is `null` here"), set one breakpoint where it can be tested, and check the value, rather than stepping through everything.

Debugging from the terminal and remotely

Compile with `javac -g` to keep local variable names in the class file (IDE builds do this by default). The JDK ships the command-line debugger `jdb`:

jdb basics (commands; interactive, so no output is shown here) TERMINAL

```
$ javac -g Sum.java
$ jdb Sum
> stop in Sum.total
> run
> step
> locals
> print sum
> cont
> quit
```

`jdb` is crude but available on any server with a JDK. More useful in practice is **remote debugging**: start the JVM with the debug agent, then attach your IDE to the port.

Start a JVM that waits for a debugger (port 5005) TERMINAL

```
$ java -agentlib:jdwp=transport=dt_socket,server=y,suspend=n,address=*:5005 -jar app.jar
```

`suspend=y` makes the JVM wait until a debugger connects, which is how you debug start-up code. Treat an open debug port as a remote-code-execution hole: never expose it to an untrusted network, and never leave it on in production.

TIP

A breakpoint can suspend just the hitting thread or every thread in the JVM, and the default differs between IDEs. Suspending all threads can make timeouts fire and heartbeats fail. When debugging a multi-threaded service, consider "suspend thread" rather than "suspend all", and expect time-dependent code to behave differently under the debugger.

Check yourself

1. A logger is set to `INFO`. Which of these appear: `fine`, `info`, `warning`, `severe`?
2. Why is `log.debug("user=" + user)` worse than `log.debug("user={}", user)`?
3. What is wrong with this catch block? `catch (IOException e) { log.error(e.getMessage()); }`
4. In JUL, what happens if the logger allows `FINE` but the `ConsoleHandler` is at its default level?
5. What does a conditional breakpoint save you from?

Answers

1. `info`, `warning` and `severe`. `fine` is below the threshold and is discarded.
2. The first builds the concatenated string (and calls `user.toString()`) every time, even when debug is disabled. The second does the work only if the level is enabled.
3. It logs only the message, not the exception object, so the stack trace and the cause chain are lost. Use `log.error("...", e)`.
4. The record is dropped by the handler. A record must pass the logger level and the handler level; the default `ConsoleHandler` level is `INFO`, so `FINE` messages do not appear until you set the handler to `FINE` or `ALL`.
5. Pausing and resuming manually on every iteration or call; it stops only when the condition (for example a particular id) is true.

KEY POINTS

Log with levels, not println: the level threshold is changed by configuration, not by editing code.

- `System.getLogger` and `java.util.logging` ship with the JDK; most applications use the SLF4J facade over Logback or Log4j 2.
- Pass the exception object to the logger, use `{}` placeholders or suppliers, never log secrets, and do not log-and-rethrow at every layer.
- Loggers form a dotted-name tree, so levels can be set per package.
- A debugger offers breakpoints, stepping, watches and call-stack inspection; remote debugging uses JDWP and must never be exposed publicly.

5.3 Classic I/O

Programs are only useful if they can read input and write output: files, the keyboard, the network. Java has two I/O APIs. The **classic** one in `java.io` has existed since Java 1.0 and is built on streams. The newer one, **NIO.2** in `java.nio.file` (next chapter), is what you should reach for first when you just need to read or write files. You still need to understand `java.io`, because the whole ecosystem uses it: sockets, servlets, compression, serialization, `System.in`, and a lot of existing code.

5.3.1 Streams: bytes versus characters

A **stream** is a one-directional flow of data that you consume or produce sequentially. (This is unrelated to the Streams API for collections.) The first decision is whether you are dealing with raw bytes or with text:

- **Byte streams** (`InputStream`, `OutputStream`) move 8-bit bytes. Use them for images, archives, network protocols, anything that is not text.
- **Character streams** (`Reader`, `Writer`) move 16-bit `char` values. Use them for text. Internally a reader converts bytes to characters using a **charset** (character encoding), and a writer does the reverse.

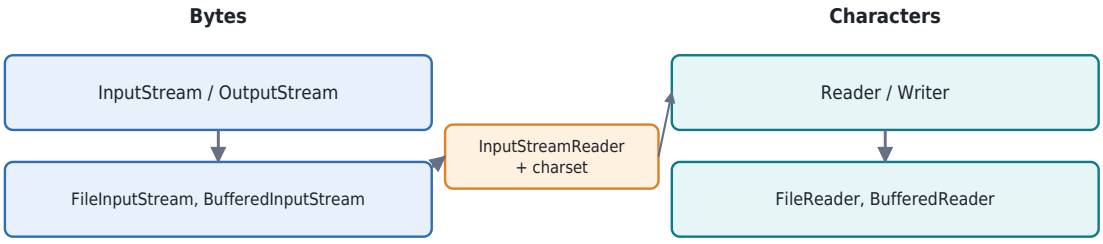


Figure 5.2. Byte streams and character streams. The bridge classes InputStreamReader and OutputStreamWriter turn bytes into characters with a charset; buffering and data classes wrap (decorate) other streams.

Purpose	Byte-oriented	Character-oriented
Read / write a file	`FileInputStream` / `FileOutputStream`	`FileReader` / `FileWriter`
In-memory	`ByteArrayInputStream` / `ByteArrayOutputStream`	`StringReader` / `StringWriter`
Buffering	`BufferedInputStream` / `BufferedOutputStream`	`BufferedReader` / `BufferedWriter`
Bridge bytes and chars	-	`InputStreamReader` / `OutputStreamWriter`
Primitives and objects	`DataInputStream`, `ObjectInputStream`	-
Convenient printing	`PrintStream` (`System.out`)	`PrintWriter`

The main classes at a glance.

Three methods are the heart of every stream: `read()` returns the next byte as an `int` from 0 to 255 or **-1 at end of stream** (that is why it returns `int` and not `byte`); `write(int)` writes the low 8 bits; `close()` releases the operating-system resource. Bulk versions (`read(byte[])`, `write(byte[], int, int)`) are almost always what you want.

5.3.2 The decorator pattern: wrapping streams

The class design is the textbook example of the **decorator** pattern. A `FileInputStream` can read bytes from a file but nothing else. To add buffering you wrap it in a `BufferedInputStream`; to read ints and doubles you wrap that in a `DataInputStream`. Each wrapper has the same interface as the thing it wraps, so they stack like layers:

Decor.java JAVA

```java
import java.io.*;

public class Decor {
    public static void main(String[] args) throws IOException {
        ByteArrayOutputStream sink = new ByteArrayOutputStream();
        try (DataOutputStream out = new DataOutputStream(new BufferedOutputStream(sink))) {
            out.writeInt(42);
            out.writeDouble(3.5);
            out.writeUTF("Asha");
        }
        System.out.println("encoded " + sink.size() + " bytes");
        try (DataInputStream in = new DataInputStream(
                new BufferedInputStream(new ByteArrayInputStream(sink.toByteArray())))) {
            System.out.println(in.readInt() + " " + in.readDouble() + " " + in.readUTF());
        }
    }
}
```

Run it TERMINAL

```
$ java Decor.java
encoded 18 bytes
42 3.5 Asha
```

The 18 bytes are 4 (`int`) + 8 (`double`) + 2 (length of the string) + 4 (the UTF-8 bytes of "Asha"). The reader must read in exactly the order the writer wrote, because there is no label in the data. A `ByteArrayOutputStream` is a handy in-memory destination for tests.

Closing the outermost wrapper closes everything beneath it, which is why you only put the outermost object in the try-with-resources header. Closing also **flushes** buffered output first.

5.3.3 Reading and writing text files

The standard recipe for a text file is: a `FileReader`/`FileWriter` for the file, wrapped in a `BufferedReader`/`BufferedWriter` for efficiency and `readLine()`, optionally wrapped in a `PrintWriter` for `println`/`printf`.

TextFiles.java JAVA

```java
import java.io.BufferedReader;
import java.io.BufferedWriter;
import java.io.File;
import java.io.FileReader;
import java.io.FileWriter;
import java.io.IOException;
import java.io.PrintWriter;
import java.nio.charset.StandardCharsets;

public class TextFiles {
    public static void main(String[] args) throws IOException {
        File f = File.createTempFile("books", ".txt");
        f.deleteOnExit();

        try (PrintWriter out = new PrintWriter(
                new BufferedWriter(new FileWriter(f, StandardCharsets.UTF_8)))) {
            out.println("978-0134685991,Effective Java,45.00");
            out.printf("%s,%s,%.2f%n", "978-1617294945", "Spring in Action", 39.5);
        }

        try (BufferedReader in = new BufferedReader(new FileReader(f, StandardCharsets.UTF_8))) {
            String line;
            while ((line = in.readLine()) != null) {
                String[] p = line.split(",");
                System.out.println(p[1] + " costs " + p[2]);
            }
        }
        System.out.println("size in bytes: " + f.length());
        System.out.println("exists: " + f.exists() + ", isFile: " + f.isFile());
    }
}
```

Run it TERMINAL

```
$ java TextFiles.java
Effective Java costs 45.00
Spring in Action costs 39.50
size in bytes: 74
exists: true, isFile: true
```

What to notice:

- The loop idiom `while ((line = in.readLine()) != null)` assigns and tests in one expression; `readLine()` returns `null` at the end and strips the line terminator.
- The `FileWriter(File, Charset)` constructor exists since Java 11; passing the charset explicitly is the habit to keep (see the next section).
- `PrintWriter` is convenient but **never throws** `IOException`: it records the failure internally. Call `checkError()` if you must know whether the write succeeded. If you need failures to be visible, use `BufferedWriter` directly.
- To append instead of overwrite, use `new FileWriter(f, StandardCharsets.UTF_8, true)`.
- `File` is just a path abstraction with a few queries (`exists`, `length`, `listFiles`, `mkdirs`, `delete`). Many of its methods report failure by returning `false`, with no explanation. NIO.2's `Files` throws informative exceptions instead.

5.3.4 Charset pitfalls

A byte sequence is only text when you know its encoding. If the reader and the writer disagree, you get **mojibake**: garbled characters. The following program writes the text "café ₹" (a French accent and the Indian rupee sign) as UTF-8 and decodes the same bytes with three different charsets. It prints the characters as hexadecimal code units so the result does not depend on the terminal.

Charsets.java JAVA

```java
import java.io.*;
import java.nio.charset.Charset;
import java.nio.charset.StandardCharsets;
import java.util.stream.Collectors;

public class Charsets {
    public static void main(String[] args) throws IOException {
        ByteArrayOutputStream bytes = new ByteArrayOutputStream();
        try (Writer w = new OutputStreamWriter(bytes, StandardCharsets.UTF_8)) {
            w.write("café ₹");
        }
        byte[] data = bytes.toByteArray();
        System.out.println("bytes: " + data.length);

        for (Charset cs : new Charset[] {StandardCharsets.UTF_8, StandardCharsets.ISO_8859_1,
                                         StandardCharsets.US_ASCII}) {
            try (Reader r = new InputStreamReader(new ByteArrayInputStream(data), cs)) {
                char[] buf = new char[20];
                int n = r.read(buf);
                String s = new String(buf, 0, n);
                System.out.println(cs + " -> " + s.chars()
                    .mapToObj(c -> String.format("%04X", c)).collect(Collectors.joining(" ")));
            }
        }
        System.out.println("default charset: " + Charset.defaultCharset());
    }
}
```

Run it TERMINAL

```
$ java Charsets.java
bytes: 9
UTF-8 -> 0063 0061 0066 00E9 0020 20B9
ISO-8859-1 -> 0063 0061 0066 00C3 00A9 0020 00E2 0082 00B9
US-ASCII -> 0063 0061 0066 FFFD FFFD 0020 FFFD FFFD FFFD
default charset: UTF-8
```

What to notice:

- Nine bytes for six characters: `é` takes two bytes in UTF-8 and `₹` takes three.
- Decoded as UTF-8 the text is correct (`00E9`, `20B9`).

- Decoded as ISO-8859-1, every byte becomes its own character, so the two bytes of the accent show up as `00C3 00A9`, which prints as the familiar "Ã©". That is mojibake.
- Decoded as US-ASCII, bytes above 127 are invalid and are replaced by the replacement character `FFFD`.

Defaults and how to stay safe

Before Java 18 the default charset of `FileReader`, `FileWriter`, `new String(bytes)` and `Scanner` came from the operating system, so the same program read a file differently on Windows (often `windows-1252`) and Linux (usually UTF-8). **JEP 400** (Java 18) made UTF-8 the default for the file APIs everywhere, which is why the example above reports UTF-8. Two caveats remain: console output (`System.out`) still uses the platform's `stdout.encoding`, which on some terminals cannot show every character, and old class files and data files may not be UTF-8. The rule that never fails: **always name the charset** (`StandardCharsets.UTF_8`) when converting between bytes and text, in both directions.

Unicode itself, code points and surrogate pairs are covered in the Part on strings and text.

5.3.5 Buffering and performance

Each call to `FileInputStream.read()` is a request to the operating system. A buffered stream reads a block (8,192 bytes by default) in one system call and then serves your single-byte reads from memory. The program below copies a 500,000-byte file three ways.

Copy.java JAVA

```
import java.io.*;

public class Copy {
    static long copyUnbuffered(File src, File dst) throws IOException {
        long t = System.nanoTime();
        try (InputStream in = new FileInputStream(src);
             OutputStream out = new FileOutputStream(dst)) {
            int b;
            while ((b = in.read()) != -1) out.write(b);
        }
        return (System.nanoTime() - t) / 1_000_000;
    }

    static long copyBuffered(File src, File dst) throws IOException {
        long t = System.nanoTime();
        try (InputStream in = new BufferedInputStream(new FileInputStream(src));
             OutputStream out = new BufferedOutputStream(new FileOutputStream(dst))) {
            int b;
            while ((b = in.read()) != -1) out.write(b);
        }
        return (System.nanoTime() - t) / 1_000_000;
    }

    static long copyBlock(File src, File dst) throws IOException {
        long t = System.nanoTime();
        try (InputStream in = new FileInputStream(src);
             OutputStream out = new FileOutputStream(dst)) {
            byte[] buf = new byte[8192];
            int n;
            while ((n = in.read(buf)) != -1) out.write(buf, 0, n);
        }
        return (System.nanoTime() - t) / 1_000_000;
    }

    public static void main(String[] args) throws IOException {
        File src = File.createTempFile("src", ".bin");
        File dst = File.createTempFile("dst", ".bin");
        src.deleteOnExit(); dst.deleteOnExit();
        try (OutputStream o = new BufferedOutputStream(new FileOutputStream(src))) {
            for (int i = 0; i < 500_000; i++) o.write(i);
        }
        copyBuffered(src, dst); // warm-up
        System.out.println("byte by byte, unbuffered : " + copyUnbuffered(src, dst) + " ms");
        System.out.println("byte by byte, buffered   : " + copyBuffered(src, dst) + " ms");
        System.out.println("8 KiB block copy         : " + copyBlock(src, dst) + " ms");
    }
}
```

Typical output (measured on the build machine; absolute numbers vary a lot, the ratios matter) TERMINAL

```
$ java Copy.java
byte by byte, unbuffered : 3848 ms
byte by byte, buffered   : 337 ms
8 KiB block copy         : 10 ms
```

Buffering made the byte-by-byte copy about ten times faster, and copying in blocks was faster still because it avoids even the per-byte method calls. The takeaway is a habit: wrap file streams in buffered ones, or read and write arrays. (The unusually slow absolute times come from the sandbox the example was run in; on your machine they will differ.)

When you wrap a `Writer` in a buffer, remember the data may sit in memory until `flush()` or `close()`. A program that crashes after `write` but before `close` can lose the last block. For a log file you may flush after each line; for a bulk export you do not.

5.3.6 Scanner, console and standard streams

`java.util.Scanner` splits input into tokens using whitespace (or any regular expression delimiter) and parses them into numbers on demand. It is convenient for small inputs and for reading from `System.in`.

Scan.java JAVA

```java
import java.util.Scanner;

public class Scan {
    public static void main(String[] args) {
        String text = "Asha 91 87.5\nRavi 78 90\n";
        try (Scanner sc = new Scanner(text)) {
            while (sc.hasNext()) {
                String name = sc.next();
                int a = sc.nextInt();
                double b = sc.nextDouble();
                System.out.println(name + " average " + (a + b) / 2);
            }
        }
        try (Scanner sc = new Scanner("a,b;c").useDelimiter("[,;]")) {
            while (sc.hasNext()) System.out.print("[" + sc.next() + "]");
            System.out.println();
        }
    }
}
```

Run it TERMINAL

```
$ java Scan.java
Asha average 89.25
Ravi average 84.0
[a][b][c]
```

Notes on `Scanner`:

- It reports parse problems with an `InputMismatchException` (unchecked) and does not consume the offending token, so a loop that does not skip it spins forever. Check with `hasNextInt()` first.
- Mixing `nextInt()` and `nextLine()` is the classic beginner bug: `nextInt()` leaves the newline in the buffer, so the following `nextLine()` returns an empty string. Call `nextLine()` once to consume it.
- It is slow for huge inputs. For competitive programming or large files, read lines with a `BufferedReader` and split them yourself.
- Number parsing uses the default `Locale`, so on a machine with a German locale `2.5` may fail to parse. Use `useLocale(Locale.ROOT)` for data files.

The three standard streams are `System.in` (an `InputStream`), `System.out` and `System.err` (both `PrintStream`). The shell can redirect each independently (`java App > out.txt 2> err.txt`), which is a reason to send diagnostics to `System.err` or a logger and results to `System.out`. `System.console()` can be `null` when input or output is redirected, so do not rely on it for password prompts in anything but an interactive tool.

GOTCHA

`PrintStream.println` and `PrintWriter.println` swallow I/O errors, and `System.out` is a `PrintStream`. Writing a large output to a closed pipe (`java App | head -1`) will not throw; your program simply keeps going.

INTERVIEW ANGLE

What is the difference between `InputStream` and `Reader`? `InputStream` reads raw bytes; `Reader` reads characters and applies a charset to decode them. Use `InputStreamReader` to bridge from a byte stream to a character stream, and always give it the charset explicitly. Using a `Reader` on binary data (an image) corrupts it, and using an `InputStream` for text means you must decode it yourself.

REAL WORLD

In a Spring MVC application you rarely touch `FileInputStream`; you receive a `MultipartFile` or an `InputStream` from the request and pass it on. The mistakes that appear in code review are the ones from this chapter: reading a whole upload into a `byte[]` instead of streaming it, forgetting to close an `InputStream` (a leaked file handle), and relying on the platform default charset when parsing CSV.

Check yourself

1. What does `InputStream.read()` return at the end of the stream, and why is its return type `int`?
2. In `new BufferedReader(new FileReader(f))`, which object do you close, and what happens to the other?
3. What is wrong with this snippet? `PrintWriter pw = new PrintWriter("out.txt"); pw.println("hi");` (no close).
4. A file written by `new FileWriter(f, StandardCharsets.UTF_8)` is read with ISO-8859-1. The text contains the single character `é`. What do you see?
5. `Scanner sc` reads `42\nAsha\n` with `int n = sc.nextInt(); String s = sc.nextLine();`. What is `s`?
6. Why does `copyBlock` in the listing beat the buffered byte-by-byte copy?

Answers

1. It returns `-1`. The type is `int` so that all 256 byte values (0 to 255) can be distinguished from the end marker; a `byte` could not represent -1 as "end" without clashing with the valid byte 0xFF.
2. Close only the outer `BufferedReader` (the try-with-resources variable). Closing it closes the wrapped `FileReader` and the file.
3. Without `close()` or `flush()` the data may stay in the buffer and never reach the file; and the file handle leaks. Use try-with-resources.
4. Two characters, shown as "Ã©", because UTF-8 stored the character as the two bytes C3 A9 and ISO-8859-1 maps each byte to its own character (mojibake).
5. The empty string `""`: `nextInt()` consumed `42` but left the newline, and `nextLine()` returned the rest of that first line.
6. It makes one method call and one system call per 8 KiB instead of one method call per byte, so there is far less overhead, even though both avoid a system call per byte.

KEY POINTS

Choose the stream family by data type: bytes (`InputStream`/`OutputStream`) or text (`Reader`/`Writer`).

- Streams are decorators: wrap file streams in buffered ones and close only the outermost in try-with-resources.
- Always name the charset (`StandardCharsets.UTF_8`); Java 18+ defaults to UTF-8 for file APIs, but console encoding can differ.
- `PrintWriter` and `PrintStream` swallow `IOException`; `read()` returns -1 at the end.
- Buffer or use arrays: per-byte unbuffered I/O is one to two orders of magnitude slower.
- `Scanner` is convenient but slow, locale-sensitive and prone to the `nextInt`/`nextLine` trap.

5.4 NIO.2: Paths, Files and Channels

Classic `java.io.File` has long-standing weaknesses: most methods signal failure by returning `false` with no reason, there is no way to copy or move a file, no symbolic-link support, poor metadata access and no way to walk a directory tree efficiently. Java 7 introduced a replacement API, informally called **NIO.2**, in the package `java.nio.file`. In day-to-day code it comes down to two types: `Path` (where a file is) and `Files` (what you can do to it).

5.4.1 Path: a location, not a file

A `Path` is an immutable sequence of names that may or may not exist on disk. Creating or manipulating a path touches no file; it is string-like manipulation, so it is cheap and cannot fail with an `IOException`. Create one with `Path.of(...)` (Java 11; `Paths.get(...)` is the older spelling and does the same).

PathDemo.java JAVA

```java
import java.nio.file.Path;
import java.nio.file.Paths;

public class PathDemo {
    public static void main(String[] args) {
        Path base = Path.of("/data/library");
        Path p = base.resolve("books/../books/java.txt");
        System.out.println("path        : " + p);
        System.out.println("normalized  : " + p.normalize());
        System.out.println("file name   : " + p.getFileName());
        System.out.println("parent      : " + p.normalize().getParent());
        System.out.println("name count  : " + p.normalize().getNameCount());
        System.out.println("absolute?   : " + p.isAbsolute());
        Path a = Path.of("/data/library/books/java.txt");
        Path b = Path.of("/data/library/media");
        System.out.println("relativize  : " + a.relativize(b));
        System.out.println("resolve abs : " + base.resolve("/etc/passwd"));
        System.out.println("same as Paths.get: " + Paths.get("a", "b").equals(Path.of("a", "b")));
        System.out.println("ends with   : " + a.endsWith("java.txt") + " " + a.startsWith("/data"));
    }
}
```

Run it TERMINAL

```
$ java PathDemo.java
path        : /data/library/books/../books/java.txt
normalized  : /data/library/books/java.txt
file name   : java.txt
parent      : /data/library/books
name count  : 4
absolute?   : true
relativize  : ../../media
resolve abs : /etc/passwd
same as Paths.get: true
ends with   : true true
```

The four operations to master:

- `resolve(other)` appends `other` to this path (like `cd`), **except** when `other` is absolute, in which case the result is `other`. The `resolve abs` line shows this; it is the root of many path-traversal vulnerabilities when `other` comes from a user.
- `normalize()` removes `.` and `..` segments lexically (it does not look at the disk, so it is wrong in the presence of symbolic links).
- `relativize(other)` computes the path that gets from this path to `other`.
- `getFileName()`, `getParent()`, `getRoot()`, `startsWith`, `endsWith` take paths apart. `endsWith("java.txt")` compares whole name elements, not characters.

WATCH OUT

Path traversal. If a user supplies a file name such as `../../etc/passwd` and you resolve it against your upload folder, the result escapes the folder. Defend with `Path target = base.resolve(name).normalize(); if (!target.startsWith(base)) throw new SecurityException(...)`.

5.4.2 The Files class

`Files` is a utility class of static methods that operate on paths. It throws specific `IOException` subclasses (`NoSuchFileException`, `FileAlreadyExistsException`, `DirectoryNotEmptyException`, `AccessDeniedException`) instead of returning `false`. The most useful methods:

Method	What it does
`readString(path)` / `writeString(path, text, options...)`	Whole file as one `String` (UTF-8 by default). Good for small files.
`readAllLines(path)` / `write(path, lines)`	Whole file as a `List<String>`.
`lines(path)`	Lazy `Stream<String>`; must be closed (use try-with-resources).
`newBufferedReader` / `newBufferedWriter`	Buffered text I/O when you need control. Also `newInputStream`/`newOutputStream`.
`copy(src, dst, options...)` / `move(src, dst, options...)`	Copy or move; fails if the target exists unless `REPLACE_EXISTING`.
`delete(path)` / `deleteIfExists(path)`	Remove a file or an empty directory.
`createDirectories(path)`	Like `mkdir -p`; no error if it already exists.
`createTempFile` / `createTempDirectory`	Safe unique temporary files.
`exists`, `isDirectory`, `size`, `getLastModifiedTime`, `readAttributes`	Queries and metadata.
`list(dir)` / `walk(dir)` / `find(dir, depth, matcher)`	Lazy streams of directory entries (close them).

Everyday Files methods.

FilesDemo.java JAVA

```java
import java.io.IOException;
import java.nio.charset.StandardCharsets;
import java.nio.file.*;
import java.util.List;
import java.util.stream.Stream;

public class FilesDemo {
    public static void main(String[] args) throws IOException {
        Path dir = Files.createTempDirectory("lib");
        Path books = dir.resolve("books.txt");

        Files.writeString(books, "Effective Java\nClean Code\nJava Concurrency\n");
        Files.writeString(books, "Refactoring\n", StandardOpenOption.APPEND);
        System.out.println(Files.readString(books).lines().count() + " lines, "
            + Files.size(books) + " bytes");

        List<String> all = Files.readAllLines(books, StandardCharsets.UTF_8);
        System.out.println(all);

        try (Stream<String> lines = Files.lines(books)) {
            lines.filter(l -> l.contains("Java")).forEach(l -> System.out.println("match: " + l));
        }

        Path copy = dir.resolve("copy.txt");
        Files.copy(books, copy);
        try {
            Files.copy(books, copy);
        } catch (FileAlreadyExistsException e) {
            System.out.println("already exists: " + dir.relativize(Path.of(e.getFile())));
        }
        Files.copy(books, copy, StandardCopyOption.REPLACE_EXISTING);

        Files.createDirectories(dir.resolve("archive/2026"));
        Files.move(copy, dir.resolve("archive/2026/old.txt"), StandardCopyOption.ATOMIC_MOVE);

        try (Stream<Path> walk = Files.walk(dir)) {
            walk.map(p -> dir.relativize(p).toString()).sorted().forEach(s ->
                System.out.println("  [" + s + "]"));
        }
```

FilesDemo.java (continued) JAVA

```
        try {
            Files.readString(dir.resolve("missing.txt"));
        } catch (NoSuchFileException e) {
            System.out.println("NoSuchFileException: " + dir.relativize(Path.of(e.getMessage())));
        }
        System.out.println("exists? " + Files.exists(copy) + ", dir? " + Files.isDirectory(dir));
    }
}
```

Run it TERMINAL

```
$ java FilesDemo.java
4 lines, 55 bytes
[Effective Java, Clean Code, Java Concurrency, Refactoring]
match: Effective Java
match: Java Concurrency
already exists: copy.txt
  []
  [archive]
  [archive/2026]
  [archive/2026/old.txt]
  [books.txt]
NoSuchFileException: missing.txt
exists? false, dir? true
```

What to notice:

- `StandardOpenOption.APPEND` adds to the file; without options `writeString` creates or truncates it.
- The second `Files.copy` threw `FileAlreadyExistsException` because the default is "never overwrite silently". Copy options make the intent explicit: `REPLACE_EXISTING`, `COPY_ATTRIBUTES`.
- `ATOMIC_MOVE` asks the file system to make the move all-or-nothing. It is the right tool for "write to a temp file, then move it into place" so readers never see a half-written file. It fails with `AtomicMoveNotSupportedException` if the file system cannot do it (for example across devices).
- `Files.walk` produced a path for the starting directory itself (the empty line `[]`), then all descendants. `walk`, `list` and `lines` return streams that hold an open directory or file handle, so close them with try-with-resources as the listing does.
- `readString` and `readAllLines` load everything into memory. Fine for a config file, wrong for a 5 GB log: stream with `Files.lines` or `newBufferedReader`.

5.4.3 Walking trees with a FileVisitor

`Files.walk` is the easy way to see every file. When you need to react to directories entering and leaving, handle errors per file, skip subtrees or delete directories after their contents, use `Files.walkFileTree` with a `FileVisitor`. It has four callbacks: `preVisitDirectory`, `visitFile`, `visitFileFailed` and `postVisitDirectory`. Extend `SimpleFileVisitor` and override only the ones you need; each returns a `FileVisitResult` (`CONTINUE`, `SKIP_SUBTREE`, `SKIP_SIBLINGS`, `TERMINATE`).

Visitor.java JAVA

```
import java.io.IOException;
import java.nio.file.*;
import java.nio.file.attribute.BasicFileAttributes;

public class Visitor {
    static void deleteTree(Path root) throws IOException {
        Files.walkFileTree(root, new SimpleFileVisitor<>() {
            @Override public FileVisitResult visitFile(Path f, BasicFileAttributes a)
                    throws IOException {
                Files.delete(f);
                return FileVisitResult.CONTINUE;
            }
            @Override public FileVisitResult postVisitDirectory(Path d, IOException e)
                    throws IOException {
                Files.delete(d);
                return FileVisitResult.CONTINUE;
            }
        });
    }
```

Visitor.java (continued) JAVA

```
    public static void main(String[] args) throws IOException {
        Path root = Files.createTempDirectory("tree");
        Files.createDirectories(root.resolve("a/b"));
        Files.writeString(root.resolve("a/one.txt"), "1");
        Files.writeString(root.resolve("a/b/two.txt"), "22");
        long[] total = {0};
        Files.walkFileTree(root, new SimpleFileVisitor<>() {
            @Override public FileVisitResult visitFile(Path f, BasicFileAttributes a) {
                total[0] += a.size();
                System.out.println("file " + root.relativize(f) + " (" + a.size() + " bytes)");
                return FileVisitResult.CONTINUE;
            }
        });
        System.out.println("total bytes: " + total[0]);
        deleteTree(root);
        System.out.println("deleted, exists? " + Files.exists(root));
    }
}
```

Run it (the order of entries within a directory is not specified and varies) TERMINAL

```
$ java Visitor.java
file a/b/two.txt (2 bytes)
file a/one.txt (1 bytes)
total bytes: 3
deleted, exists? false
```

Deleting a directory tree is the classic use: files must be deleted first (`visitFile`), and each directory afterwards (`postVisitDirectory`), because `Files.delete` refuses to remove a non-empty directory. Note the visitor also shows attribute access for free: the `BasicFileAttributes` argument carries size, timestamps and file type without another system call.

5.4.4 WatchService: reacting to file changes

`WatchService` asks the operating system to tell you when files in a directory are created, modified or deleted, instead of polling. You register a directory, then wait for a `WatchKey` that holds the pending events.

Watch.java JAVA

```
import java.io.IOException;
import java.nio.file.*;
import java.util.concurrent.TimeUnit;

public class Watch {
    public static void main(String[] args) throws IOException, InterruptedException {
        Path dir = Files.createTempDirectory("inbox");
        try (WatchService ws = FileSystems.getDefault().newWatchService()) {
            dir.register(ws, StandardWatchEventKinds.ENTRY_CREATE,
                             StandardWatchEventKinds.ENTRY_MODIFY);
            Files.writeString(dir.resolve("order-1.json"), "{}");
            WatchKey key = ws.poll(5, TimeUnit.SECONDS);
            if (key != null) {
                for (WatchEvent<?> ev : key.pollEvents()) {
                    System.out.println(ev.kind() + " " + ev.context());
                }
                key.reset();
            }
        }
    }
}
```

Typical output (event kinds and counts can vary by operating system) TERMINAL

```
$ java Watch.java
ENTRY_CREATE order-1.json
ENTRY_MODIFY order-1.json
```

Always call `key.reset()` after processing, otherwise the key is never queued again and you receive no more events. Watching is not recursive: register each subdirectory yourself. Events can be coalesced or duplicated (the modify event above arrives because writing the content is a second action after creation), and on some platforms the

implementation polls every few seconds, so build handlers to be idempotent and to wait until a file is completely written (for instance by having the writer move the file into the watched directory atomically).

5.4.5 Channels and buffers: the NIO model

The other half of NIO (Java 1.4) is a different model from streams:

- A **channel** (`FileChannel`, `SocketChannel`) is a connection to a file or socket that supports reading and writing in both directions, bulk transfers, and (for sockets) non-blocking mode with a `Selector`.
- A **buffer** (`ByteBuffer`, `CharBuffer`, `IntBuffer`) is a block of memory with a cursor. Data goes between a channel and a buffer, never directly.

A buffer has three numbers: **capacity** (fixed size), **position** (the next index to read or write) and **limit** (the first index you must not touch). You write into a buffer, then `flip()` it to switch to reading, then `clear()` or `compact()` to write again. Forgetting `flip()` is the standard bug.

Chan.java JAVA

```java
import java.io.IOException;
import java.nio.ByteBuffer;
import java.nio.channels.FileChannel;
import java.nio.charset.StandardCharsets;
import java.nio.file.*;

public class Chan {
    public static void main(String[] args) throws IOException {
        Path f = Files.createTempFile("chan", ".dat");
        try (FileChannel ch = FileChannel.open(f, StandardOpenOption.WRITE)) {
            ByteBuffer buf = ByteBuffer.allocate(16);
            buf.putInt(7).putInt(11);
            System.out.println("after put : " + buf);
            buf.flip();
            System.out.println("after flip: " + buf);
            ch.write(buf);
            System.out.println("after write: " + buf);
            buf.clear();
            buf.put("hello".getBytes(StandardCharsets.UTF_8)).flip();
            ch.write(buf);
        }
        try (FileChannel ch = FileChannel.open(f, StandardOpenOption.READ)) {
            ByteBuffer buf = ByteBuffer.allocate(8);
            ch.read(buf);
            buf.flip();
            System.out.println(buf.getInt() + " " + buf.getInt());

            var map = ch.map(FileChannel.MapMode.READ_ONLY, 8, 5);
            byte[] b = new byte[5];
            map.get(b);
            System.out.println("mapped: " + new String(b, StandardCharsets.UTF_8));
        }
        Files.delete(f);
    }
}
```

Run it TERMINAL

```
$ java Chan.java
after put : java.nio.HeapByteBuffer[pos=8 lim=16 cap=16]
after flip: java.nio.HeapByteBuffer[pos=0 lim=8 cap=16]
after write: java.nio.HeapByteBuffer[pos=8 lim=8 cap=16]
7 11
mapped: hello
```

What to notice:

- After two `putInt` calls (4 bytes each) the position is 8. `flip()` sets the limit to 8 and the position to 0, so the buffer now exposes exactly the 8 bytes just written. The channel `write` consumed them (position 8 again).
- `FileChannel.map` creates a **memory-mapped file**: a region of the file appears as memory, and the operating system pages it in and out. Reads and writes need no `read` system call per access, which makes random access to huge files cheap and is how many databases and search engines (Lucene, Kafka's log segments) get their speed. The costs: the mapping is released only when the buffer is garbage collected (or via the newer `Arena`-based API in

the foreign-memory interface), I/O errors surface as unusual runtime errors, and the mapped size is limited to 2 GB per buffer through this API.

- `ByteBuffer.allocateDirect` allocates memory outside the Java heap, which can avoid a copy when talking to the operating system, but it is slower to allocate and it counts against `-XX:MaxDirectMemorySize`, not `-Xmx`.

For plain file reading and writing, `Files` is simpler and fast enough. Reach for channels when you need non-blocking network I/O (although most applications use a framework such as Netty or the servlet container for that), `transferTo` zero-copy between channels, file locking (`FileChannel.lock`), or random access.

5.4.6 A quick look at HttpClient

Java 11 added `java.net.http.HttpClient`, a modern replacement for the old `HttpURLConnection`. It supports HTTP/1.1 and HTTP/2 (and HTTP/3 from Java 26 per JEP 517, opt-in), synchronous and asynchronous calls, timeouts and redirects. A client is immutable and reusable; create one and share it. The demo starts a tiny server from the JDK (`com.sun.net.httpserver`) in the same program, so it runs without internet access.

Http.java JAVA

```java
import com.sun.net.httpserver.HttpServer;
import java.io.IOException;
import java.net.InetSocketAddress;
import java.net.URI;
import java.net.http.HttpClient;
import java.net.http.HttpRequest;
import java.net.http.HttpResponse;
import java.nio.charset.StandardCharsets;
import java.time.Duration;

public class Http {
    public static void main(String[] args) throws Exception {
        HttpServer server = HttpServer.create(new InetSocketAddress("127.0.0.1", 0), 0);
        server.createContext("/books/1", ex -> {
            String json = "{\"id\":1,\"title\":\"Effective Java\"}";
            byte[] body = json.getBytes(StandardCharsets.UTF_8);
            ex.getResponseHeaders().add("Content-Type", "application/json");
            ex.sendResponseHeaders(200, body.length);
            try (var os = ex.getResponseBody()) { os.write(body); }
        });
        server.start();
        int port = server.getAddress().getPort();

        HttpClient client = HttpClient.newBuilder()
            .connectTimeout(Duration.ofSeconds(3)).build();
        URI uri = URI.create("http://127.0.0.1:" + port + "/books/1");
        HttpRequest req = HttpRequest.newBuilder(uri)
            .header("Accept", "application/json").timeout(Duration.ofSeconds(5)).GET().build();

        HttpResponse<String> res = client.send(req, HttpResponse.BodyHandlers.ofString());
        String type = res.headers().firstValue("content-type").orElse("?");
        System.out.println(res.statusCode() + " " + type);
        System.out.println(res.body());

        client.sendAsync(req, HttpResponse.BodyHandlers.ofString())
              .thenApply(HttpResponse::body)
              .thenAccept(b -> System.out.println("async: " + b.length() + " chars"))
              .join();
        server.stop(0);
    }
}
```

Run it TERMINAL

```
$ java Http.java
200 application/json
{"id":1,"title":"Effective Java"}
async: 33 chars
```

What to notice:

- `send` blocks and returns an `HttpResponse<T>`; the `BodyHandler` decides what `T` is (`ofString()`, `ofByteArray()`, `ofInputStream()`, `ofFile(path)`, `ofLines()`).

- `sendAsync` returns a `CompletableFuture`, which composes with `thenApply`/`thenAccept`. The Part on concurrency covers futures in depth.
- Set `connectTimeout` on the client and `timeout` on every request. Without them a request can wait indefinitely.
- A non-2xx status is **not** an exception: `send` returns normally with `statusCode()` 404 or 500. Check it yourself.
- `HttpClient` does not parse JSON. In applications you pair it with Jackson, or use Spring's `RestClient`/`WebClient`, which sit on top of similar clients.

INTERVIEW ANGLE

Why prefer `Files`/`Path` over `File`? `Path` and `Files` throw descriptive exceptions instead of returning `false`, offer copy, move, atomic move, symbolic-link handling, file attributes, lazy directory streams, tree walking and file watching, and all the helpers handle charsets explicitly (UTF-8 by default). `File` remains for legacy APIs; `file.toPath()` and `path.toFile()` convert between them.

REAL WORLD

Production services avoid reading whole files into memory. A CSV import job reads with `Files.newBufferedReader` or `Files.lines` and processes line by line, writes results through a `BufferedWriter`, and publishes the output file by writing to a temporary name and doing an `ATOMIC_MOVE`, so downstream consumers never see a partial file.

Check yourself

1. What is `Path.of("/a/b").resolve("/c")`, and why does it matter for security?
2. What is wrong with `Files.lines(p).forEach(System.out::println);` in a long-running application?
3. A `ByteBuffer` has just been filled by `put` calls. What must you do before writing it to a channel, and what does that call change?
4. `Files.move(a, b)` where `b` already exists: what happens?
5. Does `HttpClient.send` throw for an HTTP 500 response?
6. Why is `normalize()` unreliable when a path contains symbolic links?

Answers

1. It is `/c`: resolving an absolute path replaces the base. Untrusted names must be normalized and checked with `startsWith(base)`, or an attacker can escape the intended directory.
2. The stream holds an open file handle and is never closed. Use try-with-resources: `try (Stream<String> s = Files.lines(p)) { ... }`.
3. Call `flip()`: it sets the limit to the current position and the position to 0, so the channel reads exactly the bytes you wrote.
4. It throws `FileAlreadyExistsException`; pass `StandardCopyOption.REPLACE_EXISTING` to overwrite.
5. No. It returns a response whose `statusCode()` is 500. Only I/O problems (connection refused, timeout) throw.
6. `normalize()` only edits the text of the path. `a/link/../b` where `link` points elsewhere means a different place on disk than `a/b`, which only `toRealPath()` (it consults the file system) resolves correctly.

KEY POINTS

`Path` is a location and `Files` does the work; prefer them to `File`.

- Files methods throw specific exceptions; use copy/move options (`REPLACE_EXISTING`, `ATOMIC_MOVE`) explicitly.
- Close streams returned by `Files.lines`, `walk`, `list`; use `walkFileTree` for tree operations such as recursive delete.
- Validate user-supplied names with `normalize()` and `startsWith(base)` to stop path traversal.
- Channels and buffers (`flip`!), memory-mapped files and `WatchService` are for specialised needs.
- `HttpClient` (Java 11+) is reusable, supports async and HTTP/2; set timeouts and check status codes yourself.

5.5 Serialization and Its Dangers

Serialization turns an object graph into a sequence of bytes so it can be stored or sent; **deserialization** rebuilds the objects from those bytes. Java has had a built-in mechanism since version 1.1: implement the marker interface `java.io.Serializable`, then use `ObjectOutputStream` and `ObjectInputStream`. It is easy to start with and is the subject of many interview questions. It is also one of the most dangerous features of the platform, which is why this chapter spends as much time on what to avoid as on how it works.

5.5.1 The basic mechanism

`Serializable` has no methods. It only tells the runtime "this class agrees to be serialized". `ObjectOutputStream.writeObject` walks the object and everything it references, writing each non-static, non-transient field. Shared references are written once, so cycles and aliasing survive the round trip.

SerDemo.java JAVA

```
import java.io.*;

public class SerDemo {
    static class Account implements Serializable {
        private static final long serialVersionUID = 1L;
        private final String owner;
        private long balanceCents;
        private transient String sessionToken = "abc123";

        Account(String owner, long balanceCents) {
            this.owner = owner;
            this.balanceCents = balanceCents;
            System.out.println("constructor ran for " + owner);
        }
        @Override public String toString() {
            return owner + " " + balanceCents + " token=" + sessionToken;
        }
    }

    static byte[] write(Object o) throws IOException {
        ByteArrayOutputStream bos = new ByteArrayOutputStream();
        try (ObjectOutputStream out = new ObjectOutputStream(bos)) {
            out.writeObject(o);
        }
        return bos.toByteArray();
    }

    static Object read(byte[] data) throws IOException, ClassNotFoundException {
        try (ObjectInputStream in = new ObjectInputStream(new ByteArrayInputStream(data))) {
            return in.readObject();
        }
    }

    public static void main(String[] args) throws Exception {
        Account a = new Account("Asha", 12_500);
        byte[] bytes = write(a);
        System.out.println("bytes: " + bytes.length);
        System.out.println("starts with: " + String.format("%02x%02x", bytes[0], bytes[1]));
        Account b = (Account) read(bytes);
        System.out.println("copy: " + b);
        System.out.println("same object? " + (a == b));
    }
}
```

Run it TERMINAL

```
$ java SerDemo.java
constructor ran for Asha
bytes: 95
starts with: aced
copy: Asha 12500 token=null
same object? false
```

What to notice:

- "constructor ran" appears **once**. Deserialization created the copy without calling `Account`'s constructor. This single fact explains most of the security and design problems below.

- `sessionToken` came back `null`: `transient` fields are skipped and get the default value (`null`, `0`, `false`). Use `transient` for secrets, caches and anything that cannot be meaningfully saved, such as a `Thread` or a database connection.
- `static` fields belong to the class, not the instance, and are never written.
- Every serialized stream starts with the magic bytes `aced 0005`. Seeing `rO0AB` at the start of a Base64 value in a cookie or request is the classic sign of Java serialization.
- Writing an object whose class (or whose field's class) is not `Serializable` throws `NotSerializableException`, with the class name as its message.

5.5.2 serialVersionUID: versioning

Each serializable class has a 64-bit `serialVersionUID`. When reading, the runtime compares the id stored in the stream with the id of the class currently loaded; a mismatch throws `InvalidClassException`. If you do not declare the field, the id is computed from the class's name, interfaces, fields and methods, so **almost any edit to the class changes it**, including a harmless new method. Printing it for a class that declares nothing gives a long computed number such as `-1512680603108537169`.

Declare it yourself, as `private static final long serialVersionUID = 1L;`. Then compatible changes (adding a field, which reads as the default value from old data) keep working, and you bump the number only when you intend to break old data. The compatibility rules are listed in the Java Object Serialization Specification; removing or changing the type of a field is incompatible.

5.5.3 What happens to the superclass

The first non-serializable superclass in the hierarchy is **re-initialized by running its no-argument constructor**; its fields are not read from the stream. The class must therefore have an accessible no-arg constructor, or deserialization fails.

SerSafe.java (first part) JAVA

```
static class Base {
    String label = "default-label";
    Base() { System.out.println("Base() constructor ran"); }
}
static class Child extends Base implements Serializable {
    private static final long serialVersionUID = 1L;
    int age;
    Child(int age) { this.age = age; label = "custom"; }
}
// write(new Child(30)) then read(...) prints:
//   Base() constructor ran     (creating the original)
//   Base() constructor ran     (deserialization re-runs it)
//   age=30 label=default-label
```

`label` was set to `"custom"` before writing, yet it comes back as `"default-label"`, because `Base` is not serializable and its state is rebuilt, not restored. If a superclass holds state you care about, make it `Serializable` too.

5.5.4 Customizing the format

A class can define three private methods that the runtime finds by name and signature (reflection, not an interface):

- `private void writeObject(ObjectOutputStream out)` and `private void readObject(ObjectInputStream in)` run during writing and reading. Call `out.defaultWriteObject()` / `in.defaultReadObject()` first to handle the normal fields, then handle extras such as a transient cache.
- `private Object readResolve()` lets you replace the freshly built object with another one, for example to restore a singleton or an enum-like canonical instance. `writeReplace()` does the same on the writing side.

The most important use of `readObject` is **validation**. Because the constructor is bypassed, the invariants it enforced are not enforced on read.

SerSafe.java (second part) JAVA

```
static class Temperature implements Serializable {
    private static final long serialVersionUID = 1L;
    private double celsius;
    Temperature(double c) {
        if (c < -273.15) throw new IllegalArgumentException("below absolute zero");
```

SerSafe.java (second part) (continued) JAVA

```
        celsius = c;
    }
    private void readObject(ObjectInputStream in) throws IOException, ClassNotFoundException {
        in.defaultReadObject();
        if (celsius < -273.15) throw new InvalidObjectException("below absolute zero");
    }
}
// The test program writes new Temperature(20.0), patches the 8 bytes of 20.0 to -500.0,
// reads the bytes back, and prints:
//   rejected: below absolute zero
```

Without the `readObject` check, the tampered object would be accepted, with a temperature the constructor would never allow. This is why Effective Java advises treating deserialization as "another public constructor" that needs the same checks.

GOTCHA
A `final` field cannot be fixed up in `readObject`, and a defensive copy of a mutable field (a `Date`, a `List`) cannot be made there either, because the attacker may keep a reference to the stream's own inner object. The robust pattern is the **serialization proxy**: `writeReplace` returns a small private `SerializationProxy` object holding the logical state, and the proxy's `readResolve` builds the real object through the normal constructor. A record gets this behaviour automatically.

5.5.5 Why Java serialization is a security risk

`ObjectInputStream.readObject` reads class names out of the byte stream, loads those classes and creates their instances **before your code can look at the result**. The cast `(Account) in.readObject()` happens after the damage. An attacker who controls the bytes can name any `Serializable` class on your class path and trigger its `readObject` and `readResolve` logic and anything those call, such as `hashCode` when the object is put into a `HashMap` during reading. Chains of such calls found in common libraries are called **gadget chains**; public tools generate payloads for them, and the result has been remote code execution in many real products. OWASP lists insecure deserialization among the major web application risks for that reason.

Besides code execution, crafted streams can cause denial of service: a deeply nested structure or a huge array declaration exhausts memory or stack long before the stream ends.

The rules, in order of preference:

1. **Do not use Java serialization for data that crosses a trust boundary.** Not for HTTP bodies, cookies, message queues that others can write to, or files users upload.
2. Use a data format with explicit types: JSON (Jackson, Gson), Protocol Buffers, Avro. They map text or bytes onto classes you named, and they do not run arbitrary class code. Records work well as the target type, because JSON libraries call the canonical constructor and so your validation runs.
3. If you must read Java-serialized data, **filter it** (next section) and keep the dependencies patched.

5.5.6 ObjectInputFilter

Since Java 9 (backported to 8u121) you can attach a filter to a stream that decides, for every class, array size and nesting depth, whether to continue. A filter pattern is a semicolon-separated list that is checked in order; `!` rejects, and the final `!*` rejects everything not allowed earlier.

ObjectInputFilter usage JAVA

```
ObjectInputFilter allow = ObjectInputFilter.Config.createFilter(
    "com.example.shop.*;java.lang.*;java.util.*;maxdepth=5;maxbytes=65536;!*");

try (ObjectInputStream in = new ObjectInputStream(input)) {
    in.setObjectInputFilter(allow);     // must be set before readObject
    Object o = in.readObject();
}
```

When the test program read a `Child` stream through a filter that did not allow its class, `readObject` threw `InvalidClassException` with the message `filter status: REJECTED`. The filter runs before the class is instantiated.

There is also a JVM-wide filter, set with the system property `jdk.serialFilter` (for example `-Djdk.serialFilter='com.example.**;!*'`) or with `ObjectInputFilter.Config.setSerialFilter`, so that streams

opened by libraries you do not control are covered too. Java 17 added `ObjectInputFilter.Config.setSerialFilterFactory` for per-stream filter composition. Remember a filter reduces risk; an allow-list that includes a class with a gadget chain is still exploitable.

WATCH OUT
Never write `Object o = ois.readObject()` on bytes from a network or a user. Validate the source first (a signature or HMAC over the bytes, checked **before** deserializing) or avoid the mechanism.

5.5.7 Alternatives in practice

Production Java code rarely uses `ObjectOutputStream` directly. Typical replacements:

Need	Use	Why
Web API request/response	JSON via Jackson (Spring Boot default)	Human readable, schema is your class or record, no code execution
Service-to-service, strict schema	Protobuf or Avro	Compact, versioned fields, many languages
HTTP session or cache values	JSON, or Java serialization only with a filter	Spring Session and Redis serializers can be switched to JSON
Deep copy of an object	Copy constructor, `record` `with`-style method, or a mapper	Serialization-based copying is slow and fragile
Persisting your own data	A database or JSON file	Survives class changes much better

Where serialization shows up, and what to use instead.

Java serialization still appears inside the JDK and old frameworks (RMI, some session replication, `Externalizable` in legacy code). The OpenJDK project has long discussed replacing it; do not plan on that. Treat the topic as "know how it works, recognise it in a code review, and decline to add new uses".

INTERVIEW ANGLE
What does `transient` do, what is `serialVersionUID`, and why is Java serialization considered unsafe? `transient` excludes a field from serialization; it comes back as the default value. `serialVersionUID` is the version stamp compared on read, so declare it to control compatibility instead of letting the compiler-computed value change on every edit. It is unsafe because deserialization instantiates classes named by the stream without running their constructors, so crafted bytes can invoke gadget classes on the class path (remote code execution) or exhaust resources. Mitigations: do not deserialize untrusted data, use JSON or Protobuf, set an `ObjectInputFilter` allow-list, and validate in `readObject` or use a serialization proxy.

REAL WORLD
Spring Boot services store sessions and cache entries through serializers. If a team keeps the default JDK serializer for Redis, any process able to write to that Redis can send crafted bytes to every service that reads it. Security reviews therefore ask to switch to JSON serializers or to set `jdk.serialFilter`.

Check yourself

1. What does this print, assuming `class P implements Serializable { transient int n = 5; int m = 7; }` is written and read back after `n` and `m` were both changed to 9 before writing?
2. Is the constructor of a `Serializable` class called when an object is deserialized? What about its non-serializable superclass?
3. Why should you declare `serialVersionUID` explicitly?
4. A class is `Serializable` and has a field `private Thread worker;`. What happens on `writeObject`, and what are two fixes?
5. Explain how deserializing untrusted bytes can run attacker-chosen code even though your program only calls `readObject` and casts to `Account`.
6. Name two safer alternatives and one JDK mechanism that limits the damage when you cannot avoid `ObjectInputStream`.

Answers

1. `n` is `0` (transient fields get default values) and `m` is `9`.
2. No, the constructor of the serializable class is not called. The no-arg constructor of the first non-serializable superclass is called, and that superclass's fields are not restored from the stream.
3. The computed default changes whenever the class changes, even for compatible edits, breaking old data. A declared value lets you decide when compatibility ends.
4. `NotSerializableException: java.lang.Thread`. Mark the field `transient` (and recreate it lazily or in `readObject`), or do not keep the thread in the object.
5. `readObject` creates the object graph the stream describes before your cast; classes on the class path with dangerous `readObject` or `hashCode` logic (gadgets) run while the stream is read, so the cast failing afterwards is too late.
6. JSON (Jackson) or Protobuf into known types; an `ObjectInputFilter` allow-list (per stream or `jdk.serialFilter`) with depth and size limits.

KEY POINTS

Serialization copies fields without calling constructors, which is what makes it fragile and dangerous.

- `Serializable` is a marker; `transient` and `static` fields are not written; the first non-serializable superclass is re-created through its no-arg constructor.
- Declare `serialVersionUID`; a mismatch throws `InvalidClassException`.
- Validate in `readObject` or use a serialization proxy; records deserialize through the canonical constructor.
- Never deserialize untrusted bytes; prefer JSON or Protobuf and use `ObjectInputFilter` allow-lists when unavoidable.

5.6 Properties, System and Runtime

Every Java program lives inside a process, and the process has a context: command-line arguments, environment variables, system properties, a working directory, an exit code. Applications read configuration from that context, clean up when they are told to stop, and sometimes start other programs. This chapter covers the small set of JDK classes for that: `System`, `Properties`, `Runtime` and `ProcessBuilder`.

5.6.1 Command-line arguments and system properties

Arguments after the class name (or after the source file in `java Foo.java`) arrive in `String[] args` of `main`. The shell splits them on spaces, so quote an argument that contains a space.

System properties are string key/value pairs held by the JVM. The JVM fills in many (`java.version`, `os.name`, `user.dir`, `file.separator`, `line.separator`, `java.io.tmpdir`, `user.home`), and you add your own on the command line with `-Dkey=value` placed **before** the class or `-jar`. Read one with `System.getProperty(key)` (returns `null` when missing) or `System.getProperty(key, default)`. `Integer.getInteger(key, default)` and `Boolean.getBoolean(key)` read and parse in one step; `Boolean.getBoolean("x")` reads the *property* named `x`, not the string, a common mix-up.

SysDemo.java — JAVA

```
import java.io.*;
import java.util.*;

public class SysDemo {
    public static void main(String[] args) throws Exception {
        System.out.println("args = " + Arrays.toString(args));
        System.out.println("java.version = " + System.getProperty("java.version"));
        System.out.println("os.name = " + System.getProperty("os.name"));
        System.out.println("file.separator = " + System.getProperty("file.separator"));
        System.out.println("app.mode = " + System.getProperty("app.mode", "dev"));
        System.out.println("Integer.getInteger = " + Integer.getInteger("app.port", 8080));
        System.out.println("HOME set? " + (System.getenv("HOME") != null));
        System.out.println("APP_COLOR = " + System.getenv("APP_COLOR"));
        Runtime rt = Runtime.getRuntime();
        System.out.println("cpus>0: " + (rt.availableProcessors() > 0));
        System.out.println("maxMemory MB: " + rt.maxMemory() / (1024 * 1024));

        Properties p = new Properties();
        p.setProperty("db.url", "jdbc:h2:mem:test");
        p.setProperty("db.pool", "5");
        StringWriter sw = new StringWriter();
```

SysDemo.java (continued) JAVA

```
        p.store(sw, "app config");
        String text = sw.toString().lines().filter(l -> !l.startsWith("#")).sorted()
            .reduce("", (a, b) -> a + b + "\n");
        System.out.print(text);
        Properties q = new Properties();
        q.load(new StringReader("# comment\nname = Library \\\n  App\nport=9090\nempty=\n"));
        System.out.println("name='" + q.getProperty("name") + "' port=" + q.getProperty("port")
            + " missing=" + q.getProperty("x") + " dflt=" + q.getProperty("x", "none"));

        Runtime.getRuntime().addShutdownHook(
            new Thread(() -> System.out.println("hook: cleaning up")));
        System.out.println("exiting with 3");
        System.exit(3);
    }
}
```

Run with a property, an environment variable and two arguments TERMINAL

```
$ APP_COLOR=blue java -Dapp.mode=prod -Dapp.port=9000 SysDemo.java one "two words"
args = [one, two words]
java.version = 21.0.12.1
os.name = Linux
file.separator = /
app.mode = prod
Integer.getInteger = 9000
HOME set? true
APP_COLOR = blue
cpus>0: true
maxMemory MB: 1496
db.pool=5
db.url=jdbc\:h2\:mem\:test
name='Library App' port=9090 missing=null dflt=none
exiting with 3
hook: cleaning up
$ echo $?
3
```

The values of `java.version`, `os.name` and `maxMemory` depend on the machine (typical output). `maxMemory` is the heap ceiling the JVM will try to use; by default it is a fraction (one quarter) of physical memory, and `-Xmx` overrides it. `availableProcessors()` respects container CPU limits in modern JDKs, which matters for sizing thread pools.

5.6.2 Environment variables

`System.getenv("NAME")` returns the value of an environment variable or `null`; `System.getenv()` returns an unmodifiable `Map<String,String>`. You cannot set environment variables of your own process from Java. Compare the two sources of configuration:

Aspect	System property	Environment variable
Set with	`-Dkey=value`, or `System.setProperty`	The shell, container, or Kubernetes manifest
Scope	One JVM	Process and its children
Names	Dotted (`app.mode`)	Upper case with underscores (`APP_MODE`)
Typical use	JVM and library switches	Secrets and per-environment settings in containers

System properties and environment variables.

Spring Boot reads both and maps `APP_MODE` to `app.mode` ("relaxed binding"). Another source is the JAR itself, with `JAVA_TOOL_OPTIONS` as an environment variable that the JVM scans for flags; the banner `Picked up JAVA_TOOL_OPTIONS: ...` on stderr is that mechanism at work.

GOTCHA

`System.setProperty` changes a global. A test that sets `user.timezone` or `file.encoding` can leak into other tests running in the same JVM. Set and restore in a `finally` block, and avoid changing properties after startup in production code.

5.6.3 The Properties class

`java.util.Properties` is a `Hashtable<Object,Object>` specialised for string pairs, with a text file format (the `.properties` file). It predates collections generics; always use `getProperty` and `setProperty` and avoid the inherited `put`/`get`, which accept non-strings. Features worth knowing:

- `load(Reader)` / `load(InputStream)` parse lines of `key=value`, `key:value` or `key value`; `#` and `!` start comment lines; a trailing backslash continues a line (in the demo, `Library \` plus `App` produced `Library App`, leading blanks on the continuation are dropped); a key with no value (`empty=`) yields an empty string.
- `getProperty(key, default)` supplies a fallback. A `Properties` can also be constructed with a *defaults* `Properties`, consulted when the key is missing.
- `store(Writer, comments)` writes the file, preceded by a comment line and a timestamp. Output order is **not** guaranteed (Hashtable order; the demo sorted the lines before printing). Characters `:` and `=` in values are escaped with a backslash, as in `jdbc\:h2\:mem\:test`.
- `stringPropertyNames()` returns the keys whose key and value are strings.

GOTCHA

Encoding. The byte-stream methods `load(InputStream)` and `store(OutputStream, ...)` use ISO-8859-1 and write other characters as `\uXXXX` escapes (the test wrote the letter e-acute as `Café`). The `Reader`/`Writer` versions use whatever charset you give them. Since Java 9, `ResourceBundle` reads `.properties` files as UTF-8; for `Properties.load` you choose by passing `new InputStreamReader(in, StandardCharsets.UTF_8)`.

A typical configuration loader reads a file from the class path, so it works from inside a JAR:

ConfigLoader.java JAVA

```java
import java.io.IOException;
import java.io.InputStream;
import java.io.InputStreamReader;
import java.io.Reader;
import java.nio.charset.StandardCharsets;
import java.util.Properties;

public class ConfigLoader {
    static Properties load(String resource) throws IOException {
        Properties props = new Properties();
        try (InputStream in = ConfigLoader.class.getResourceAsStream(resource)) {
            if (in == null) {
                throw new IOException("resource not found: " + resource);
            }
            try (Reader r = new InputStreamReader(in, StandardCharsets.UTF_8)) {
                props.load(r);
            }
        }
        return props;
    }

    public static void main(String[] args) throws IOException {
        try {
            load("/app.properties");
        } catch (IOException e) {
            System.out.println(e.getMessage());
        }
    }
}
```

Without an app.properties on the class path TERMINAL

```
$ java ConfigLoader.java
resource not found: /app.properties
```

`getResourceAsStream` returns `null` (not an exception) when the resource is missing, so check it. A leading `/` makes the name absolute from the class path root; without it the name is relative to the class's package.

5.6.4 Runtime and shutdown hooks

`Runtime.getRuntime()` gives the single `Runtime` object of the JVM. Useful methods: `availableProcessors()`, `maxMemory()` (limit), `totalMemory()` (currently reserved), `freeMemory()` (free part of that), `addShutdownHook(Thread)`, `exit(int)` and `halt(int)`. `Runtime.version()` returns the structured JDK version. Calling `gc()` is a request only; do not build logic on it.

A **shutdown hook** is a thread that the JVM starts when it begins an orderly shutdown: when the last non-daemon thread ends, when `System.exit` is called, or when the process receives `SIGTERM` or `SIGINT` (Ctrl+C). In the demo it printed `hook: cleaning up` after `System.exit(3)`. It does **not** run if the process is killed with `SIGKILL` (`kill -9`), after `Runtime.halt`, or on a JVM crash.

Rules for hooks:

1. Hooks run concurrently with each other, in no defined order, so a hook must not depend on another hook.
2. Keep them short and robust: closing connections, flushing files, deregistering from a service registry. The JVM exits when they finish, and orchestrators such as Kubernetes follow `SIGTERM` with `SIGKILL` after a grace period (30 seconds by default).
3. Do not call `System.exit` inside a hook (it deadlocks) and do not add hooks while shutdown is under way.
4. Frameworks wrap this: Spring Boot registers its own hook to close the application context, which runs `@PreDestroy` methods and graceful web server shutdown. You rarely write raw hooks in Spring applications.

5.6.5 Exit codes

The exit code of a process tells the caller, a shell or a CI job, whether it succeeded. By convention `0` means success and non-zero means failure. `System.exit(n)` stops the JVM with status `n` (after running hooks). If `main` returns normally and no non-daemon threads remain, the status is 0. If `main` ends with an uncaught exception, the stack trace is printed and the status is **1**.

Prefer to let `main` return a code through one `System.exit` at the top level, not to scatter `System.exit` calls through libraries: a library that exits makes itself impossible to test, and calling it from a servlet container would kill the whole server. Shell scripts read the code with `$?`, as the demo showed (`3`).

5.6.6 Running other programs with ProcessBuilder

`ProcessBuilder` starts an operating-system process. (`Runtime.exec(String)` is the older path; it splits the command string by whitespace, which breaks on quoted arguments, and has been deprecated since Java 18. Use `ProcessBuilder`.) You pass the command and its arguments as separate strings, and **no shell is involved** unless you ask for one with `sh -c`, so there is no wildcard expansion, pipe or `&&`.

Proc.java JAVA

```java
import java.io.*;
import java.nio.charset.StandardCharsets;
import java.util.*;
import java.util.concurrent.TimeUnit;

public class Proc {
    public static void main(String[] args) throws Exception {
        ProcessBuilder pb = new ProcessBuilder("sh", "-c", "echo out; echo err 1>&2; exit 4");
        pb.redirectErrorStream(true);
        Process p = pb.start();
        try (BufferedReader r = new BufferedReader(
                new InputStreamReader(p.getInputStream(), StandardCharsets.UTF_8))) {
            r.lines().forEach(l -> System.out.println("child> " + l));
        }
        System.out.println("exit code " + p.waitFor());

        Process slow = new ProcessBuilder("sleep", "5").start();
        boolean done = slow.waitFor(300, TimeUnit.MILLISECONDS);
        System.out.println("finished in time? " + done);
        slow.destroy();
        slow.waitFor();
        System.out.println("alive after destroy: " + slow.isAlive());

        ProcessBuilder env = new ProcessBuilder("sh", "-c", "echo $GREETING from $PWD")
            .directory(new File("/tmp"));
        env.environment().put("GREETING", "hello");
        env.redirectErrorStream(true);
```

Proc.java (continued) JAVA

```
        Process e = env.start();
        System.out.print(new String(e.getInputStream().readAllBytes(), StandardCharsets.UTF_8));
        e.waitFor();
        System.out.println("pid positive: " + (ProcessHandle.current().pid() > 0));
    }
}
```

Run it (Linux; uses sh and sleep) TERMINAL

```
$ java Proc.java
child> out
child> err
exit code 4
finished in time? false
alive after destroy: false
hello from /tmp
pid positive: true
```

What to notice:

- `getInputStream()` is the child's **standard output** as seen by the parent (the naming is from the parent's point of view). `redirectErrorStream(true)` merges stderr into it, which is why `err` appears in the stream.
- **Read the output.** If the child writes more than the operating system's pipe buffer holds and nobody reads, the child blocks and `waitFor()` hangs forever. Either consume the stream (as above), merge stderr, or redirect output to a file with `redirectOutput(File)`, or to the parent's console with `inheritIO()`.
- `waitFor()` returns the exit code; `waitFor(timeout, unit)` returns `false` on timeout, after which you decide: `destroy()` (polite, SIGTERM) or `destroyForcibly()` (SIGKILL).
- `directory(File)` sets the working directory and `environment()` is a mutable copy of the environment for the child only.
- `ProcessHandle` (Java 9) exposes pids, parent and children, and `onExit()` returns a `CompletableFuture`.

WATCH OUT

Command injection. Never build a command by concatenating user input into a string passed to `sh -c`: `"ls " + userInput` with `; rm -rf ~` as input runs both. Pass user data as a separate argument element (`new ProcessBuilder("ls", userInput)`), which reaches the program without shell interpretation, and validate it anyway (a value beginning with `-` can still be read as an option by the program).

INTERVIEW ANGLE

How do you run code when a Java application is stopped, and when does it not run? Register a shutdown hook with `Runtime.getRuntime().addShutdownHook(thread)`. The JVM runs it on normal exit, `System.exit`, and on SIGTERM or Ctrl+C. It does not run on `kill -9`, `Runtime.halt`, power loss or a JVM crash, so critical data must not depend on it. Hooks run concurrently, so keep them independent and quick.

REAL WORLD

In containers, the platform sends SIGTERM, waits for a grace period and then SIGKILL. A Spring Boot service with graceful shutdown enabled (`server.shutdown=graceful`) stops accepting new requests, finishes in-flight ones, and closes pools from the shutdown hook; a service that ignores this drops requests on each deployment.

Check yourself

1. What is printed by `java -Dflag=true Main` for `System.out.println(Boolean.getBoolean("flag") + " " + Boolean.getBoolean("true"));`?
2. A shutdown hook closes a database pool. In which of these situations does it run: `System.exit(1)`, Ctrl+C, `kill -9`, uncaught exception in `main`?
3. What is wrong with this: `Process p = new ProcessBuilder("producer").start(); p.waitFor();` when `producer` prints several megabytes?
4. Why is `new ProcessBuilder("sh", "-c", "grep " + term + " log.txt")` dangerous, and what is the safer form?
5. Which charset does `Properties.load(InputStream)` use, and how do you read a UTF-8 properties file correctly?
6. What is the exit status of a JVM whose `main` throws an uncaught exception?

Answers

1. `true false`. `Boolean.getBoolean(name)` looks up a system property named `name`; no property called `true` exists.
2. It runs for `System.exit(1)`, Ctrl+C (SIGINT) and the uncaught exception in `main` (once the other non-daemon threads finish and the JVM exits normally). It does not run for `kill -9`.
3. The child blocks when the pipe buffer fills because the parent never reads it, so `waitFor()` hangs. Read the stream, merge or redirect stderr, redirect output to a file, or use `inheritIO()`.
4. The term is interpreted by the shell, so `x; rm -rf ~` runs a second command. Use `new ProcessBuilder("grep", "--", term, "log.txt")`, with no shell, and validate the term.
5. ISO-8859-1, with `\uXXXX` escapes for other characters. Wrap the stream in an `InputStreamReader` with `StandardCharsets.UTF_8` and call `load(Reader)`.
6. 1, after the stack trace is printed.

KEY POINTS

The process context (arguments, properties, environment, exit code) is how a Java program is configured and observed from outside.

- `-Dkey=value` before the class sets system properties; `System.getenv` reads environment variables; both return `null` when missing.
- `Properties` loads `key=value` text; byte streams mean ISO-8859-1, so pass a UTF-8 `Reader` for modern files.
- Shutdown hooks run on exit, `System.exit` and SIGTERM/SIGINT, never on `kill -9` or `halt`; keep them short and independent.
- Exit code 0 is success; an uncaught exception gives 1; call `System.exit` only at the outermost layer.
- With `ProcessBuilder` pass arguments separately, drain the child's output, use timeouts, and never concatenate user input into a shell string.

PART 6

Generics and Collections

Type-safe containers, the Collections Framework, and the data structures behind it.

In this part

- **6.1** Generics
- **6.2** The Collections Framework
- **6.3** Lists in depth
- **6.4** Sorting and comparing
- **6.5** Immutable, unmodifiable and sequenced collections
- **6.6** Sets
- **6.7** Other maps: LinkedHashMap, TreeMap and the specialists
- **6.8** HashMap in depth
- **6.9** Queues and deques
- **6.10** Choosing a data structure, and Big-O
- **6.11** Implement your own: array list, linked list, hash map, LRU cache

6.1 Generics

Before Java 5 every container held `Object`. You put a `String` in, you got an `Object` out, and you cast it back and hoped. If somebody had stored an `Integer` in the same list, the program compiled happily and failed at runtime with a `ClassCastException`, possibly far from the line that did the damage. **Generics** move that check to compile time: you tell the compiler "this list holds `String`", and it refuses to compile code that violates the promise and inserts the casts for you.

Generics are also one of the most misunderstood parts of the language, because they were added in a way that kept old bytecode working. The result is a feature that looks like C++ templates but behaves very differently underneath. This chapter teaches the syntax, then the rules that surprise people (wildcards, erasure, arrays), and finally why those rules exist.

6.1.1 Why generics: the problem and the promise

Here is the pre-generics world, still legal today if you use raw types:

RawDemo.java JAVA

```java
import java.util.ArrayList;
import java.util.List;

public class RawDemo {
    @SuppressWarnings({"rawtypes", "unchecked"})
    public static void main(String[] args) {
        List books = new ArrayList();      // raw type: no element type
        books.add("Dune");
        books.add(42);                     // compiles, nothing complains
        String first = (String) books.get(0);
        String second = (String) books.get(1);   // ClassCastException here
    }
}
```

With `List<String>` the line `books.add(42)` is a compile error, and `books.get(0)` needs no cast. Generics give you three things: **type safety** (errors found by the compiler), **no casts** in client code, and **reusable algorithms** (one `max` method works for every comparable type).

6.1.2 Generic classes, interfaces and methods

A **type parameter** is a placeholder name declared in angle brackets. By convention it is a single capital letter: `T` (type), `E` (element), `K`/`V` (key/value), `R` (result), `N` (number).

Pair.java JAVA

```java
import java.util.List;

public class Pair {
    record Box<T>(T value) {                    // generic class (here a record)
        <R> Box<R> map(java.util.function.Function<? super T, ? extends R> f) {
            return new Box<>(f.apply(value)); // generic method with its own type parameter R
        }
    }

    static <T extends Comparable<T>> T max(List<T> items) {   // bounded type parameter
        T best = items.get(0);
        for (T t : items) {
            if (t.compareTo(best) > 0) best = t;
        }
        return best;
    }

    public static void main(String[] args) {
        Box<String> name = new Box<>("Dune");
        Box<Integer> len = name.map(String::length);
        System.out.println(name + " -> " + len);
        System.out.println(max(List.of(3, 9, 4)));
        System.out.println(max(List.of("pear", "apple", "zebra")));
        // max(List.of(new Object())); // does not compile: Object is not Comparable<Object>
    }
}
```

Run it TERMINAL

```
$ java Pair.java
Box[value=Dune] -> Box[value=4]
9
zebra
```

What to notice:

- `Box<T>` is a **generic class**: `T` is chosen by whoever writes `Box<String>`. The `<>` on the right of `new Box<>(...)` is the **diamond**; the compiler infers the type argument (Java 7+).
- `map` declares its own `<R>`. A **generic method** has its type parameters before the return type. Callers almost never write them out because the compiler **infers** them from the arguments (`max(List.of(3, 9, 4))` infers `T = Integer`). You can force them with `Collections.<String>emptyList()` when inference fails.
- `T extends Comparable<T>` is a **bound**: only types that can compare themselves to their own kind are allowed, which is what lets us call `compareTo`. For several bounds use `&`: `<T extends Number & Comparable<T>>` (a class bound, if any, must come first).
- The same syntax works for interfaces (`interface Repository<T, ID>`) and constructors, and `static` methods can be generic even though a static method cannot use the class's own `T`.

> **NOTE**
> The word `extends` in a bound means "is a subtype of" and covers interfaces too. There is no `implements` in type bounds. There is also no `T super X` for a type parameter declaration; `super` appears only in wildcards.

6.1.3 Generics are invariant

Is a `List<Integer>` a `List<Number>`? Intuition says yes, since `Integer` is a `Number`. The compiler says no, and the reason is worth understanding. Suppose it were allowed:

Bad1.java JAVA

```java
List<Integer> ints = new ArrayList<>();
List<Number> nums = ints;      // imagine this compiled
nums.add(3.14);                // legal for List<Number>... but ints now holds a Double
Integer i = ints.get(0);       // would blow up
```

What javac actually says TERMINAL

```
$ javac Bad1.java
Bad1.java:5: error: incompatible types: List<Integer> cannot be converted to List<Number>
        List<Number> nums = ints;
                            ^
```

Generic types are **invariant**: `List<Integer>` and `List<Number>` are unrelated types even though `Integer` extends `Number`. Arrays, by contrast, are **covariant** (`Integer[]` is an `Object[]`), which is exactly why `Object[] a = new Integer[1]; a[0] = "x";` compiles and throws `ArrayStoreException` at runtime. Generics chose compile-time safety over that convenience.

Invariance is too strict for real code. A method that only sums numbers should accept a `List<Integer>` and a `List<Double>`. That is what wildcards are for.

6.1.4 Wildcards and PECS

A **wildcard** `?` stands for "some unknown type". It comes in three flavours:

Form	Meaning	What you can do
`List<?>`	list of some unknown type	read elements as Object; add only null
`List<? extends Number>`	list of Number or a subtype (upper bound)	read as Number; cannot add (except null)
`List<? super Integer>`	list of Integer or a supertype (lower bound)	add Integer; read only as Object

The three wildcard forms.

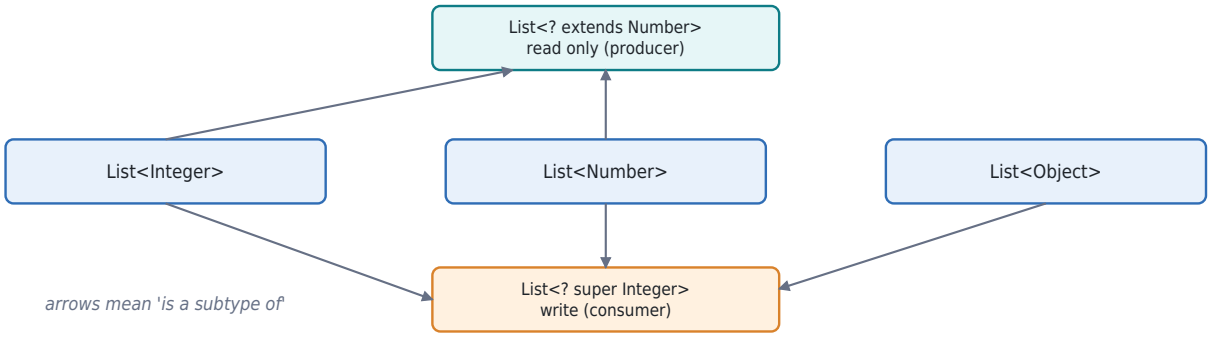


Figure 6.1. The wildcard type hierarchy: List<Integer> and List<Number> are both subtypes of List<? extends Number>, while List<Number> and List<Object> are subtypes of List<? super Integer>. Producers use extends, consumers use super.

The memory aid is **PECS: Producer Extends, Consumer Super** (Joshua Bloch). If a structure *produces* values that your code reads, use `? extends T`. If it *consumes* values that your code writes into it, use `? super T`. If you do both, use an exact `T` and no wildcard.

Wild.java JAVA

```java
import java.util.ArrayList;
import java.util.List;

public class Wild {
    static double sum(List<? extends Number> nums) {          // producer: we only read
        double s = 0;
        for (Number n : nums) s += n.doubleValue();
        return s;
    }

    static void fill(List<? super Integer> sink, int n) {     // consumer: we only write
        for (int i = 0; i < n; i++) sink.add(i);
    }

    static <T> void copy(List<? super T> dest, List<? extends T> src) {   // both, PECS
        for (T t : src) dest.add(t);
    }

    public static void main(String[] args) {
        List<Integer> ints = List.of(1, 2, 3);
        List<Double> dbls = List.of(1.5, 2.5);
        System.out.println(sum(ints) + " " + sum(dbls));
```

Wild.java (continued) JAVA

```java
        List<Number> numbers = new ArrayList<>();
        List<Object> objects = new ArrayList<>();
        fill(numbers, 3);
        fill(objects, 2);
        copy(numbers, dbls);
        System.out.println(numbers + " " + objects);

        List<? extends Number> ro = ints;
        Number first = ro.get(0);          // reading is fine
        System.out.println(first);
        // ro.add(4);                      // compile error: capture of ? extends Number
        List<?> unknown = ints;
        Object o = unknown.get(1);
        System.out.println(o);
    }
}
```

Run it TERMINAL

```
$ java Wild.java
6.0 4.0
[0, 1, 2, 1.5, 2.5] [0, 1]
1
2
```

Why can you not add to a `List<? extends Number>`? The compiler only knows "some subtype of Number". It could be a `List<Double>`, so adding an `Integer` would be wrong, and there is no way to name the actual type. javac reports it with a **capture** variable:

The error for ro.add(4) TERMINAL

```
$ javac Bad1.java
Bad1.java:7: error: incompatible types: int cannot be converted to CAP#1
        ro.add(4);
               ^
  where CAP#1 is a fresh type-variable:
    CAP#1 extends Number from capture of ? extends Number
```

The JDK follows PECS everywhere. `Collections.copy(List<? super T> dest, List<? extends T> src)` and `Comparator<? super T>` in `List.sort` are textbook cases: sorting a `List<Student>` is fine with a `Comparator<Person>`, because anything that can compare any `Person` can compare students.

Wildcard capture helper

`List<?>` has a trap: you cannot write `swap(List<?> l)` using `l.set(0, l.get(1))` because the two `?` are different unknowns to the compiler. The standard trick is a private generic helper, which lets the compiler *capture* the wildcard into a named `T`:

Gen2.java JAVA

```java
import java.util.*;

public class Gen2 {
    static <T> void swapFirst(List<T> l) {
        T t = l.get(0);
        l.set(0, l.get(1));
        l.set(1, t);
    }

    static void swap(List<?> l) { swapFirst(l); }   // capture conversion happens here

    public static void main(String[] a) {
        List<String> l = new ArrayList<>(List.of("a", "b", "c"));
        swap(l);
        System.out.println(l);
    }
}
```

Run it TERMINAL

```
$ java Gen2.java
[b, a, c]
```

INTERVIEW ANGLE
What is the difference between `List<?>`, `List<Object>` and the raw `List`? `List<Object>` is a list whose element type is exactly `Object`; you may add anything, but you cannot pass a `List<String>` to it (invariance). `List<?>` is a list of an unknown type; it accepts any parameterisation but you cannot add to it (only null). Raw `List` switches off generic checking entirely and exists only for backward compatibility; it produces unchecked warnings and can corrupt a list at runtime.

6.1.5 Type erasure

Generics exist only at compile time. After type checking, javac **erases** them: every type parameter is replaced by its bound (or `Object` if unbounded), casts are inserted where needed, and the class file contains no trace of `String` in `List<String>` except in optional signature metadata. This was a deliberate compatibility decision in Java 5, so that old and new code could run together and the JVM did not have to change.

Erasure at work

```
Source                                  After erasure (what the bytecode does)
class Box<T> { T value; }               class Box { Object value; }
class Num<T extends Number> { T v; }    class Num { Number v; }
String s = list.get(0);                 String s = (String) list.get(0);
<T extends Comparable<T>> T max(...)    Comparable max(...)
```

You can see the inserted casts in bytecode. In `Pair.max` above the call `t.compareTo(best)` compiles to an `invokeinterface` on `Comparable`, preceded by `checkcast` instructions:

javap -c -p Pair (trimmed) TERMINAL

```
$ javap -c -p -cp out Pair | grep "checkcast\|compareTo"
       7: checkcast     #13   // class java/lang/Comparable
      33: checkcast     #13   // class java/lang/Comparable
      39: invokeinterface #29,  2  // InterfaceMethod Comparable.compareTo:(Object)I
      79: checkcast     #55   // class java/lang/String
```

Consequences of erasure

These restrictions all follow from "the type argument does not exist at runtime":

- **No primitives as type arguments.** `List<int>` is illegal; use `List<Integer>` (boxing). That has a memory and speed cost, which is why `IntStream` and primitive-specialised classes exist.
- **No `new T()`, `new T[n]`, or `T.class`.** The JVM would not know which class to instantiate. Pass a `Class<T>` or a `Supplier<T>` when you need to create instances.
- **No `instanceof List<String>`.** Only the raw class survives. `o instanceof List<?>` is allowed. Pattern matching with `List<String> l` is rejected unless the cast is provably safe (see the javac message below).
- **No static field of type `T`**, and no generic exceptions (`class MyEx<T> extends Exception` is illegal), because a `catch` clause could not tell `MyEx<String>` from `MyEx<Integer>`.
- **Overloads that erase to the same signature clash**: `void f(List<String>)` and `void f(List<Integer>)` cannot coexist.

javac messages for the forbidden cases

```
Bad1.java:9: error: Object cannot be safely cast to List<String>
        if (o instanceof List<String> l) { }
            ^
Bad1.java:11: error: generic array creation
    static <T> T[] mk() { return new T[3]; }
                                  ^
```

A type whose runtime representation carries full information is called **reifiable**: primitives, non-generic classes, raw types, `List<?>`, and arrays of those. `List<String>` is **non-reifiable**. Arrays are reified (an `Integer[]` knows it is an `Integer[]` and checks every store), generics are not, and that mismatch is the source of the next two topics.

Same class at runtime

Erasure.java JAVA

```java
import java.lang.reflect.Method;
import java.util.ArrayList;
import java.util.Arrays;
import java.util.List;

public class Erasure {
    static class Animal implements Comparable<Animal> {
        final int w;
        Animal(int w) { this.w = w; }
        public int compareTo(Animal o) { return Integer.compare(w, o.w); }
    }

    @SafeVarargs
    static <T> T[] bad(T... xs) { return xs; }

    static <T> T[] makeArray(T a, T b) { return bad(a, b); }

    @SuppressWarnings({"unchecked", "rawtypes"})
    public static void main(String[] args) throws Exception {
        List<String> a = new ArrayList<>();
        List<Integer> b = new ArrayList<>();
        System.out.println(a.getClass() == b.getClass());

        for (Method m : Animal.class.getDeclaredMethods()) {
            System.out.println(m + " bridge=" + m.isBridge());
        }

        List<String> strings = new ArrayList<>();
        List raw = strings;
        raw.add(42);                       // heap pollution: only an unchecked warning
        System.out.println("size " + strings.size());
        try {
            String s = strings.get(0);     // compiler inserted checkcast String here
            System.out.println(s);
        } catch (ClassCastException e) {
            System.out.println("CCE: " + e.getMessage().substring(0, 58) + "...");
        }

        try {
            String[] arr = makeArray("x", "y");
            System.out.println(arr.length);
        } catch (ClassCastException e) {
            System.out.println("CCE from varargs array: " + e.getMessage().substring(0, 60));
        }
        System.out.println(Arrays.toString(new List<?>[2]));
    }
}
```

Run it TERMINAL

```
$ java Erasure.java
true
public int Erasure$Animal.compareTo(Erasure$Animal) bridge=false
public int Erasure$Animal.compareTo(java.lang.Object) bridge=true
size 1
CCE: class java.lang.Integer cannot be cast to class java.lang....
CCE from varargs array: class [Ljava.lang.Object; cannot be cast to class [Ljava.lan
[null, null]
```

Three lessons in one run:

1. **Same runtime class.** `ArrayList<String>` and `ArrayList<Integer>` are both just `ArrayList`. That is why `a.getClass() == b.getClass()` is `true`.
2. **Bridge methods.** `Animal implements Comparable<Animal>` and writes `compareTo(Animal)`. After erasure the interface method is `compareTo(Object)`. To keep polymorphism working javac generates a synthetic **bridge method** `compareTo(Object)` that casts its argument to `Animal` and calls the real one. Reflection shows both; `bridge=true` marks the generated one. Bridge methods also appear for covariant return types. If you ever see an odd extra method in a stack trace or a `javap` listing, this is it.
3. **Heap pollution and where the exception appears.** A variable of type `List<String>` ended up holding an `Integer`. This is **heap pollution**. The `add(42)` through the raw type only produced a compiler warning; the failure

came later at `strings.get(0)`, at a cast the compiler had silently inserted, and the stack trace points at a line that looks perfectly innocent. This is why unchecked warnings should be treated as real bugs, not noise.

6.1.6 Generic arrays and varargs

`new List<String>[10]` and `new T[10]` are compile errors. If they were allowed, covariance of arrays and erasure would combine to defeat type safety: you could store a `List<Integer>` into what the compiler thinks is a `List<String>[]`, because at runtime the array only checks for `List`. The legal alternatives:

- Use a `List<List<String>>` (almost always the right answer).
- Create `new ArrayList[10]` or `(T[]) new Object[10]` and accept an unchecked cast, keeping the array private (this is what `ArrayList` itself does internally with `Object[]`).
- Use `Array.newInstance(componentClass, n)` when the caller supplies a `Class<T>`.

A **generic varargs** parameter (`T... xs`) is secretly a generic array, so javac warns about "possible heap pollution". In the listing, `bad` returns the varargs array, and `makeArray` calls it from a generic context. At runtime the array is really an `Object[]` (the erasure of `T[]`), so the caller's `String[] arr = ...` cast fails with a `ClassCastException`. `@SafeVarargs` is a promise that you only read the array and never leak it; `List.of`, `Arrays.asList` and `EnumSet.of` make that promise honestly. Leaking the array, as `bad` does, breaks it.

6.1.7 Raw types and migration

A **raw type** is a generic class used without type arguments (`List`, `Map`). It exists for old code. Using it turns off generic checking for that variable, and mixing raw and parameterised types gives "unchecked" warnings. Do not use raw types in new code. If you must call a legacy API that returns one, convert at the boundary and isolate the suppression:

Legacy.java — JAVA

```java
import java.util.ArrayList;
import java.util.List;

public class Legacy {
    static List legacyNames() { return new ArrayList(List.of("ada", "linus")); }

    @SuppressWarnings("unchecked")
    static List<String> names() {
        return (List<String>) legacyNames();   // one audited, annotated place
    }

    public static void main(String[] args) {
        for (String n : names()) System.out.println(n.toUpperCase());
    }
}
```

Run it — TERMINAL

```
$ java Legacy.java
ADA
LINUS
```

Keep `@SuppressWarnings("unchecked")` on the smallest scope possible (a single statement or method), with a comment saying why it is safe. Never on a whole class.

GOTCHA

A generic method call can be fine at compile time and still throw `ClassCastException` on a line that has no cast in your source. Whenever a generic return value is used as a concrete type, the compiler inserted a `checkcast`. If the stack trace points at such a line, look for raw types, unchecked casts or `@SuppressWarnings` upstream.

REAL WORLD

In Spring, `RestTemplate.exchange(..., new ParameterizedTypeReference<List<Order>>() {})` exists purely because of erasure: `List<Order>.class` is not a thing, so the library captures the type argument through an anonymous subclass (its superclass signature is stored in the class file and survives erasure). Jackson has the same idea in `TypeReference`. Knowing erasure explains why these odd-looking helpers are needed.

INTERVIEW ANGLE
What is type erasure, and what problems does it cause? The compiler checks generic types, then removes them: type parameters become their bounds (or Object) and casts are inserted. Consequences: no `new T()`, no `T[]`, no `instanceof List<String>`, no primitive type arguments, `List<String>` and `List<Integer>` share one runtime class, bridge methods are generated to keep overriding working, and heap pollution is possible through raw types. The upside is full backward compatibility with pre-5 code.

Check yourself

1. What does this print, and why? `List<Integer> a = new ArrayList<>(); List<String> b = new ArrayList<>(); System.out.println(a.getClass() == b.getClass());`
2. What is wrong with `List<Number> n = new ArrayList<Integer>();`, and how would you change the declaration so a method can read `Number`s from any list of numbers?
3. Which wildcard do you choose for `copy(dest, src)` where `src` only gives you elements and `dest` only receives them? State the rule.
4. Why does `new T[10]` not compile, and what is the usual workaround?
5. In the `Erasure` listing, why does the `ClassCastException` happen at `strings.get(0)` and not at `raw.add(42)`?
6. What is a bridge method and when does javac generate one?

Answers

1. `true`. After erasure both are the plain `ArrayList` class; type arguments exist only at compile time.
2. Generics are invariant, so `List<Integer>` is not a `List<Number>`. Declare `List<? extends Number>` (or make the method generic) to read from any list of numbers.
3. `copy(List<? super T> dest, List<? extends T> src)`: PECS, Producer Extends (src), Consumer Super (dest).
4. The array's component type must be known at runtime but `T` is erased. Use a `List<T>`, or `(T[]) new Object[n]` with a private array, or `Array.newInstance(cls, n)` with a `Class<T>`.
5. The raw `add` has no cast, so it succeeds and the list holds an `Integer`. The compiler inserted a `checkcast String` at the `get`, and that is where the mismatch is detected.
6. A synthetic method generated when erasure would break overriding, for example `compareTo(Object)` in a class implementing `Comparable<Animal>`, or a covariant return type. It casts and delegates to the real method.

KEY POINTS
Generics give compile-time type safety and remove casts; the JVM itself only sees erased types.

- Generic types are invariant: `List<Integer>` is not a `List<Number>`. Use wildcards for flexibility: `? extends` to read, `? super` to write (PECS).
- Erasure replaces type parameters by their bounds; that is why there is no `new T()`, no generic arrays, no `instanceof List<String>`, and no primitives as type arguments.
- Bridge methods keep overriding working after erasure; heap pollution through raw types fails later at an inserted cast.
- Never use raw types in new code; confine unchecked casts to the smallest annotated scope.

6.2 The Collections Framework

Almost every program keeps groups of things: the books on a shelf, the orders waiting to be shipped, the students in a class. Arrays can hold a group, but they have a fixed size, no built-in search, and no way to say "no duplicates" or "keep these sorted". The **Java Collections Framework** (JCF, in `java.util`, since Java 1.2) is a set of interfaces, implementations and algorithms for groups of objects. Learning it well pays off in every interview and every codebase: choosing the right collection is one of the most common design decisions a Java developer makes.

The framework has a simple split worth memorising before anything else:

- **Interfaces** say what a collection can do: `List`, `Set`, `Queue`, `Deque`, `Map`.
- **Implementations** say how it does it, with different speed and memory trade-offs: `ArrayList`, `LinkedList`, `HashSet`, `TreeSet`, `ArrayDeque`, `PriorityQueue`, `HashMap`, `TreeMap`.
- **Algorithms** are static helpers in `Collections` and `Arrays` (sorting, searching, shuffling).

Program to the interface (`List<String> names = new ArrayList<>();`), so you can swap the implementation without touching callers.

6.2.1 The hierarchy

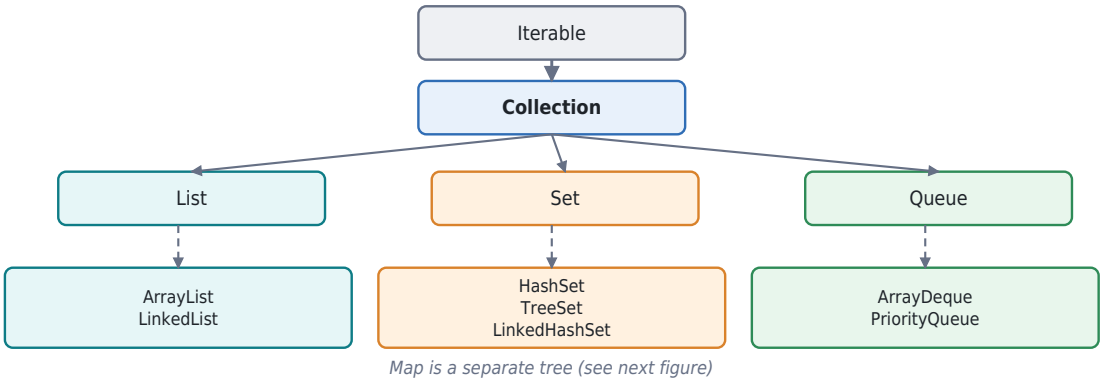


Figure 6.2. The Collection hierarchy. Iterable is the root; List, Set and Queue extend Collection, and the concrete classes (ArrayList, HashSet, ArrayDeque and so on) sit at the bottom. Notice that Map is not part of it.

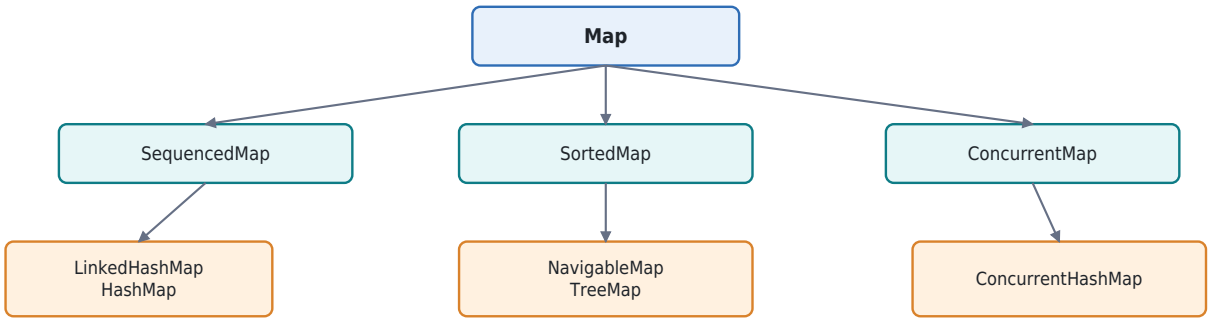


Figure 6.3. The Map hierarchy: a separate tree rooted at Map, with SequencedMap, SortedMap and NavigableMap refining it, and HashMap, LinkedHashMap, TreeMap and ConcurrentHashMap as implementations.

The surprise for beginners: `Map` **does not extend** `Collection`. A collection is a group of single elements; a map is a group of key-value pairs, and operations like `add(E)` make no sense for it. A map exposes three collection **views**: `keySet()`, `values()` and `entrySet()`. We can confirm the hierarchy with reflection:

Iface.java JAVA

```
import java.util.*;

public class Iface {
    public static void main(String[] args) {
        Collection<String> c = new ArrayList<>(List.of("x", "y"));
        System.out.println(c.size() + " " + c.contains("x") + " " + c.isEmpty());
        Iterable<String> it = c;
        it.forEach(System.out::print);
        System.out.println();
        for (Class<?> k : new Class<?>[]{ArrayList.class, LinkedList.class, HashSet.class,
                LinkedHashSet.class, TreeSet.class, ArrayDeque.class, PriorityQueue.class,
                HashMap.class, TreeMap.class}) {
            System.out.println(k.getSimpleName() + " List=" + List.class.isAssignableFrom(k)
                + " Set=" + Set.class.isAssignableFrom(k)
                + " Queue=" + Queue.class.isAssignableFrom(k)
                + " Deque=" + Deque.class.isAssignableFrom(k)
                + " Collection=" + Collection.class.isAssignableFrom(k));
        }
    }
}
```

Run it TERMINAL

```
$ java Iface.java
2 true false
xy
ArrayList List=true Set=false Queue=false Deque=false Collection=true
LinkedList List=true Set=false Queue=true Deque=true Collection=true
HashSet List=false Set=true Queue=false Deque=false Collection=true
LinkedHashSet List=false Set=true Queue=false Deque=false Collection=true
TreeSet List=false Set=true Queue=false Deque=false Collection=true
ArrayDeque List=false Set=false Queue=true Deque=true Collection=true
PriorityQueue List=false Set=false Queue=true Deque=false Collection=true
HashMap List=false Set=false Queue=false Deque=false Collection=false
TreeMap List=false Set=false Queue=false Deque=false Collection=false
```

Note that `LinkedList` is both a `List` and a `Deque`, while `ArrayDeque` is a `Deque` only. Here is the family at a glance:

Interface	Contract	Main implementations
`List`	ordered by position, duplicates allowed, index access	ArrayList, LinkedList
`Set`	no duplicates (by equals)	HashSet, LinkedHashSet, TreeSet
`Queue / Deque`	holds elements for processing; head and tail operations	ArrayDeque, PriorityQueue, LinkedList
`Map`	key to value, unique keys	HashMap, LinkedHashMap, TreeMap

The core interfaces.

Java 21 added a refinement to this picture: **sequenced collections** (`SequencedCollection`, `SequencedSet`, `SequencedMap`) for types that have a defined first and last element. We cover them at the end of this Part's first half; the diagrams above show them in the hierarchy.

6.2.2 What every Collection can do

`Collection<E>` defines the operations shared by lists, sets and queues. The ones you use daily:

Method	Notes
`size(), isEmpty()`	Number of elements; isEmpty() is clearer than size() == 0.
`contains(Object)`	Uses equals(). Cost depends on the implementation (O(1) HashSet, O(n) ArrayList).
`add(E), remove(Object)`	Return boolean: did the collection change? A Set.add of a duplicate returns false.
`addAll, removeAll, retainAll, containsAll`	Bulk operations; retainAll is set intersection, removeAll is set difference.
`removeIf(Predicate)`	Java 8. Removes matching elements safely, without an explicit iterator.
`iterator(), forEach`	Traversal. Every Collection is Iterable, so it works in an enhanced for loop.
`stream(), toArray()`	Bridge to streams (see the Part on lambdas and streams) and to arrays.

Core Collection methods.

Several of these mutators are **optional operations**: an unmodifiable collection (for example `List.of(...)`) implements `add` but throws `UnsupportedOperationException`. This was a design compromise to avoid a separate read-only interface hierarchy. We return to it in the chapter on immutable collections.

6.2.3 Iteration and Iterator

The enhanced `for` loop is syntactic sugar over an `Iterator`. `for (String b : books)` compiles to roughly `Iterator<String> it = books.iterator(); while (it.hasNext()) { String b = it.next(); ... }`. Any class implementing `Iterable<T>` can be the target of a for-each, including your own:

Shelf.java JAVA

```
import java.util.Iterator;
import java.util.Map;
import java.util.NoSuchElementException;
import java.util.TreeMap;

public class Shelf implements Iterable<String> {
    private final String[] books;
```

Shelf.java (continued) JAVA

```java
    Shelf(String... books) { this.books = books; }

    @Override public Iterator<String> iterator() {
        return new Iterator<>() {
            private int next = 0;
            @Override public boolean hasNext() { return next < books.length; }
            @Override public String next() {
                if (!hasNext()) throw new NoSuchElementException();
                return books[next++];
            }
        };
    }

    public static void main(String[] args) {
        for (String b : new Shelf("Dune", "Emma", "Ulysses")) System.out.println(b);

        Map<String, Integer> stock = new TreeMap<>(Map.of("Dune", 3, "Emma", 0, "Ulysses", 5));
        for (Map.Entry<String, Integer> e : stock.entrySet()) {
            System.out.println(e.getKey() + " -> " + e.getValue());
        }
        stock.values().removeIf(n -> n == 0);      // view: removes the entry from the map
        System.out.println(stock + " keys=" + stock.keySet());
    }
}
```

Run it TERMINAL

```
$ java Shelf.java
Dune
Emma
Ulysses
Dune -> 3
Emma -> 0
Ulysses -> 5
{Dune=3, Ulysses=5} keys=[Dune, Ulysses]
```

What to notice: the `Iterator` is a small object with a cursor. It is **single use**: once exhausted, ask the collection for a new one. `Iterator` has three methods (`hasNext`, `next`, optionally `remove`), and `List` adds `ListIterator` with `hasPrevious`, `set` and `add` for bidirectional traversal. The map views are live: `stock.values().removeIf(...)` changed the map itself.

6.2.4 Fail-fast iterators and ConcurrentModificationException

The most common collections bug in beginner code is changing a list while looping over it. Watch what happens:

Cme.java JAVA

```java
import java.util.ArrayList;
import java.util.Iterator;
import java.util.List;

public class Cme {
    public static void main(String[] args) {
        List<String> books = new ArrayList<>(List.of("Dune", "Emma", "Ulysses", "Hamlet"));

        try {
            for (String b : books) {
                if (b.startsWith("E")) books.remove(b);   // structural change during iteration
            }
        } catch (java.util.ConcurrentModificationException e) {
            System.out.println("CME: " + e);
        }
        System.out.println(books);

        Iterator<String> it = books.iterator();
        while (it.hasNext()) {
            if (it.next().startsWith("E")) it.remove();   // the supported way
        }
        System.out.println(books);

        books.removeIf(b -> b.length() > 5);              // Java 8 shortcut
        System.out.println(books);
```

Cme.java (continued) JAVA

```
        List<String> two = new ArrayList<>(List.of("a", "b", "c"));
        for (String s : two) {
            if (s.equals("b")) two.remove(s);             // removing the second-to-last element
        }
        System.out.println(two);
    }
}
```

Run it TERMINAL

```
$ java Cme.java
CME: java.util.ConcurrentModificationException
[Dune, Ulysses, Hamlet]
[Dune, Ulysses, Hamlet]
[Dune]
[a, c]
```

Four things happened:

1. The first loop removed "Emma" with `List.remove` while the loop's iterator was active. The next call to `next()` threw `ConcurrentModificationException` (CME). The element *was* removed before the exception (`[Dune, Ulysses, Hamlet]`).
2. The explicit `Iterator.remove()` is the supported way to delete during traversal. Here nothing started with "E" any more, so the list is unchanged; run it on a fresh list and it removes correctly.
3. `removeIf` does the same job in one line and is the best choice since Java 8.
4. The last example is the famous silent case: removing the second-to-last element makes `hasNext()` return false, so the loop ends without throwing, and the program carries on with no error. Fail-fast is **best-effort**, not a guarantee.

How it works: `ArrayList` keeps a counter `modCount` that is incremented on every **structural modification** (add, remove, clear; changing an element with `set` does not count). An iterator copies `modCount` when created (`expectedModCount`) and `next()` compares them first. `hasNext()` only checks `cursor != size`, which is why case 4 slips through. The iterator's own `remove()` updates `expectedModCount`, so it stays in sync.

> **WATCH OUT**
> Never rely on a CME to detect a threading bug. The documentation says fail-fast behaviour cannot be guaranteed, particularly under unsynchronised concurrent modification. If several threads share a collection, use a concurrent collection (`ConcurrentHashMap`, `CopyOnWriteArrayList`, covered in the Part on concurrency) or a lock. The CME name is misleading: single-threaded code triggers it all the time.

The opposite design is a **weakly consistent** (or snapshot) iterator. `CopyOnWriteArrayList` iterates over a snapshot and `ConcurrentHashMap` iterators tolerate concurrent changes; neither ever throws a CME.

6.2.5 equals and hashCode: what collections need from your classes

Collections do not magically know when two of your objects are "the same". They ask the objects:

- `List.contains`, `indexOf`, `remove(Object)` call `equals`.
- Hash-based collections (`HashSet`, `HashMap` keys, `LinkedHashSet`) call `hashCode` first to find a bucket, then `equals` to compare inside it.
- Tree-based collections (`TreeSet`, `TreeMap`) never call `equals`; they use `compareTo` or a `Comparator` (next chapters).

The contract: equal objects must have equal hash codes. Break it and hash collections silently misbehave:

Eq.java JAVA

```
import java.util.*;

public class Eq {
    static class Isbn {
        final String code;
        Isbn(String code) { this.code = code; }
        @Override public boolean equals(Object o) {
            return o instanceof Isbn other && code.equals(other.code);
        }
        // hashCode forgotten on purpose
    }
    record GoodIsbn(String code) {}
```

Eq.java (continued) JAVA

```
    public static void main(String[] args) {
        Set<Isbn> bad = new HashSet<>();
        bad.add(new Isbn("111"));
        System.out.println(bad.contains(new Isbn("111")));
        List<Isbn> list = new ArrayList<>(List.of(new Isbn("111")));
        System.out.println(list.contains(new Isbn("111")));

        Set<GoodIsbn> good = new HashSet<>();
        good.add(new GoodIsbn("111"));
        System.out.println(good.contains(new GoodIsbn("111")));
    }
}
```

Run it TERMINAL

```
$ java Eq.java
false
true
true
```

The same two `Isbn` objects are "found" by the `ArrayList` (which uses `equals` only) but not by the `HashSet`: the inherited identity `hashCode` sends the lookup to a different bucket, so `equals` is never even called. A `record` gets a correct `equals` and `hashCode` for free. The full story of buckets is in the HashMap chapter later in this Part.

Two more rules: never use a mutable field in `hashCode` for an object you keep in a hash collection (change it and the object is lost in the wrong bucket), and keep `equals` consistent with `compareTo` when you use sorted collections.

INTERVIEW ANGLE
Why does the enhanced for loop throw `ConcurrentModificationException` when you remove an element, and how do you fix it? The list's iterator records the list's modification count when created and checks it on every `next()`. A direct `list.remove` changes the count, so the next `next()` throws. Fix: use `Iterator.remove()`, `removeIf`, collect the items to remove and call `removeAll`, or iterate over a copy. In multi-threaded code use a concurrent collection.

REAL WORLD
Spring Data repositories return `List<T>`, `Set<T>` or `Optional<T>`; Hibernate swaps in its own persistent collection classes behind those interfaces. This only works because you coded to the interface. Entities used as `Set` elements or `Map` keys need a careful `equals`/`hashCode` (often based on a business key, not a generated id), which is a classic JPA interview topic.

Check yourself

1. Is `Map` a `Collection`? Name the three collection views a `Map` provides.
2. What does this print? `List<String> l = new ArrayList<>(List.of("a","b","c")); for (String s : l) { if (s.equals("b")) l.remove(s); } System.out.println(l);`
3. Same code but the test is `s.equals("a")`. What happens and why?
4. A class overrides `equals` but not `hashCode`. Which operation on a `HashSet` of that class fails, and which on an `ArrayList` still works?
5. Which of `ArrayList`, `LinkedList`, `ArrayDeque`, `PriorityQueue` implement `Deque`?
6. What is wrong with `for (Integer i : list) list.add(i * 2);`?

Answers

1. No. `Map` is a separate hierarchy. Views: `keySet()`, `values()`, `entrySet()`.
2. `[a, c]`. Removing the second-to-last element makes `hasNext()` return false, so the loop ends without a CME (verified by running).
3. A `ConcurrentModificationException`: removing "a" changes `modCount`, and the next `next()` call detects it.
4. `contains`/`remove`/duplicate detection in the `HashSet` fail (wrong bucket); `ArrayList.contains` uses only `equals` and works.
5. `LinkedList` and `ArrayDeque`. `ArrayList` is only a `List`; `PriorityQueue` is a `Queue` but not a `Deque`.
6. It modifies the list structurally during iteration, so the next iteration throws a CME. Build a separate result list or use a plain index loop.

KEY POINTS

The framework separates interfaces (List, Set, Queue, Deque, Map) from implementations; code to the interface.

- `Map` is not a `Collection`; it offers `keySet`, `values` and `entrySet` views that write through to the map.
- Iterators are fail-fast and best-effort: use `Iterator.remove()` or `removeIf` to delete while traversing, never `list.remove` inside for-each.
- Hash collections need a consistent `equals` and `hashCode`; sorted collections need a consistent ordering. Records give you both for free.
- Some mutators are optional operations and throw `UnsupportedOperationException` on unmodifiable collections.

6.3 Lists in depth

A `List` is an ordered collection that allows duplicates and lets you reach elements by position. It is the collection you will use most. Two implementations dominate the discussion: `ArrayList`, which stores elements in an array, and `LinkedList`, which stores them in chained nodes. Textbooks teach them as equals with opposite strengths; real-world measurement says something much more one-sided. This chapter shows how `ArrayList` actually works, why `LinkedList` is rarely the right answer, and the half-dozen traps (`remove`, `Arrays.asList`, `subList`, `List.of`) that cost beginners hours.

6.3.1 ArrayList internals

`ArrayList` wraps a plain `Object[] elementData` plus an `int size`. The array length is the **capacity**; `size` is how many slots are in use. `get(i)` is an array read after a bounds check, so it is O(1). `add(e)` writes into the next free slot, O(1) most of the time. When the array is full, `ArrayList` allocates a bigger one (about 1.5 times larger) and copies everything across with `Arrays.copyOf`. This copying is why the growth is called **amortised** O(1): an occasional O(n) copy is spread over many cheap adds.

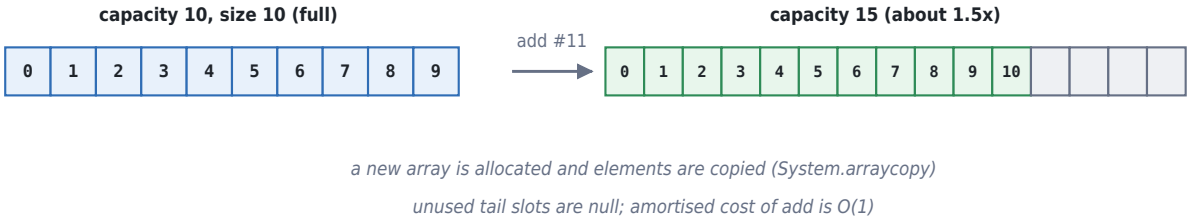


Figure 6.4. ArrayList growth: when size reaches capacity, a new array about 1.5 times larger is allocated and the old elements are copied; the unused tail slots are null.

We can watch it happen. The internal array is private, so this listing reads it with reflection; on Java 17+ that needs `--add-opens`:

Growth.java JAVA

```
import java.lang.reflect.Field;
import java.util.ArrayList;
import java.util.List;

public class Growth {
    static int capacity(ArrayList<?> list) throws Exception {
        Field f = ArrayList.class.getDeclaredField("elementData");
        f.setAccessible(true);
        return ((Object[]) f.get(list)).length;
    }

    public static void main(String[] args) throws Exception {
        ArrayList<Integer> list = new ArrayList<>();
        System.out.println("new ArrayList(): capacity " + capacity(list));
        int last = capacity(list);
        for (int i = 1; i <= 100; i++) {
            list.add(i);
            int c = capacity(list);
            if (c != last) {
                System.out.println("after add #" + i + ": capacity " + last + " -> " + c);
                last = c;
```

Growth.java (continued) JAVA

```
            }
        }
        ArrayList<Integer> sized = new ArrayList<>(50);
        System.out.println("new ArrayList(50): capacity " + capacity(sized));
        List<Integer> copy = new ArrayList<>(List.of(1, 2, 3));
        System.out.println("copy of 3: capacity " + capacity((ArrayList<Integer>) copy));
        list.subList(10, 100).clear();
        list.trimToSize();
        System.out.println("after trimToSize: size " + list.size() + " capacity " + capacity(list));
    }
}
```

Run it (JDK 21) TERMINAL

```
$ java --add-opens java.base/java.util=ALL-UNNAMED Growth.java
new ArrayList(): capacity 0
after add #1: capacity 0 -> 10
after add #11: capacity 10 -> 15
after add #16: capacity 15 -> 22
after add #23: capacity 22 -> 33
after add #34: capacity 33 -> 49
after add #50: capacity 49 -> 73
after add #74: capacity 73 -> 109
new ArrayList(50): capacity 50
copy of 3: capacity 3
after trimToSize: size 10 capacity 10
```

What the numbers tell you:

- A new `ArrayList` allocates **nothing** until the first add: capacity 0, then 10 on the first `add`. The oft-quoted "default capacity 10" is true, but lazy. Thousands of empty lists cost almost no array memory.
- After that, new capacity = old + old/2 (integer division, `old >> 1`): 10, 15, 22, 33, 49, 73, 109. That is the 1.5x growth.
- `new ArrayList<>(50)` reserves exactly 50, and copying a collection sizes to the collection. If you know roughly how many elements are coming, say so in the constructor (or call `ensureCapacity`) to skip the intermediate copies.
- `trimToSize()` shrinks the array to `size`, useful for a long-lived list that has stopped growing. `clear()` does not shrink the array.

Inserting or removing in the middle is O(n): `add(index, e)` and `remove(index)` use `System.arraycopy` to shift every later element by one. It is a fast native block copy, but still proportional to the number of elements moved. `remove` also nulls out the last slot so the removed object can be garbage collected.

Operation	Cost	Why
`get(i), set(i, e)`	O(1)	direct array access
`add(e) at the end`	amortised O(1)	occasional resize and copy
`add(i, e), remove(i)`	O(n)	shifts later elements with arraycopy
`contains(o), indexOf(o)`	O(n)	linear scan using equals
`remove(Object)`	O(n)	scan, then shift
`iteration`	O(n)	sequential memory, very cache friendly

ArrayList operation costs.

> **NOTE**
> `ArrayList` is not thread-safe. Concurrent adds can lose elements or throw `ArrayIndexOutOfBoundsException`. For occasional wrapping use `Collections.synchronizedList`; for read-heavy shared lists see `CopyOnWriteArrayList` below.

6.3.2 LinkedList: the reality

`LinkedList` is a **doubly linked list**: each element lives in its own `Node` object holding the item and `prev`/`next` references. It implements both `List` and `Deque`. On paper it has O(1) insertion and removal at both ends and O(1) insertion "in the middle" once you hold the position. But finding position `i` means walking from the nearer end, so `get(i)` is O(n).

Textbook claim: "use `LinkedList` when you insert and delete a lot." Here is what the numbers look like for a mixed workload, measured on the build machine (a slow shared sandbox, so only the ratios matter; typical output):

Reality.java JAVA

```java
import java.util.*;

public class Reality {
    static long time(Runnable r) {
        long t = System.nanoTime();
        r.run();
        return (System.nanoTime() - t) / 1_000_000;
    }

    public static void main(String[] args) {
        int n = 50_000;
        List<Integer> array = new ArrayList<>();
        List<Integer> linked = new LinkedList<>();
        for (int i = 0; i < n; i++) { array.add(i); linked.add(i); }

        for (int round = 0; round < 3; round++) {   // repeat so the JIT warms up
            long a = time(() -> { long s = 0; for (int i = 0; i < n; i++) s += array.get(i); });
            long l = time(() -> { long s = 0; for (int i = 0; i < n; i++) s += linked.get(i); });
            long af = time(() -> { List<Integer> x = new ArrayList<>();
                for (int i = 0; i < n; i++) x.add(0, i); });
            long lf = time(() -> { List<Integer> x = new LinkedList<>();
                for (int i = 0; i < n; i++) x.add(0, i); });
            System.out.println("get(i) loop: ArrayList " + a + " ms, LinkedList " + l + " ms | "
                + "add(0,x): ArrayList " + af + " ms, LinkedList " + lf + " ms");
        }
    }
}
```

Typical output (measured on the build machine, varies per run) TERMINAL

```
$ java Reality.java
get(i) loop: ArrayList 43 ms, LinkedList 12416 ms | add(0,x): ArrayList 665 ms, LinkedList 169 ms
get(i) loop: ArrayList 24 ms, LinkedList 5797 ms | add(0,x): ArrayList 600 ms, LinkedList 26 ms
get(i) loop: ArrayList 0 ms, LinkedList 5131 ms | add(0,x): ArrayList 488 ms, LinkedList 5 ms
```

Indexing a `LinkedList` in a loop is catastrophically slow (quadratic overall), and it is the classic way the "obvious" code goes wrong. Inserting at the front is where `LinkedList` shines. But the honest summary is:

- **Default to `ArrayList`.** Contiguous arrays are friendly to CPU caches and to the garbage collector; each `LinkedList` node is a separate heap object (on a typical 64-bit HotSpot with compressed references, roughly 24 bytes of node overhead per element on top of the element itself, against 4 bytes per slot in an array).
- Even inserting in the *middle* is usually faster on `ArrayList` for lists of ordinary size, because finding the position in a `LinkedList` costs a pointer walk across scattered memory.
- If you need a queue or a stack (insert and remove at the ends), use `ArrayDeque`, which is faster and lighter than `LinkedList` for that job (see the chapter on queues later in this Part). `LinkedList` remains defensible when you hold a `ListIterator` and do many insertions/removals at the iterator's position, or when you must allow `null` elements in a deque.

INTERVIEW ANGLE

ArrayList vs LinkedList: when would you pick each? ArrayList for almost everything: O(1) random access, compact memory, fast iteration, amortised O(1) append. LinkedList only when you do many insertions/removals at known positions via an iterator or at both ends and never index; even then ArrayDeque is usually better for end operations. Mention that LinkedList.get(i) is O(n) and that node objects cost memory and cache misses; that is the answer that shows you have looked beyond Big-O tables.

6.3.3 Legacy lists: Vector, Stack, CopyOnWriteArrayList

`Vector` (Java 1.0) is an `ArrayList` whose every method is `synchronized`; `Stack` extends `Vector` and adds `push/pop/peek`. Both predate the Collections Framework. Their per-call locking does not make compound actions atomic ("check then add" still races) and costs speed, and `Stack` inherits list methods that break the stack idea (you can `insertElementAt` in the middle). In new code use `ArrayList`, and `ArrayDeque` as a stack (`push`, `pop`, `peek`). You will still meet `Vector` in old libraries and interview questions.

`CopyOnWriteArrayList` (in `java.util.concurrent`) copies the whole array on every write. Reads and iteration take no lock and see a snapshot, so iteration never throws a CME. It is excellent for lists that are read constantly and changed rarely (a list of event listeners, a configuration list) and terrible for write-heavy use. The concurrency Part covers it properly.

6.3.4 Remove pitfalls

`List<Integer>` has two methods both named `remove`: `remove(int index)` and `remove(Object o)`. With an `int` argument Java picks the index version, which is almost never what you meant for a list of numbers:

Pitfalls.java JAVA

```
import java.util.*;

public class Pitfalls {
    public static void main(String[] args) {
        List<Integer> nums = new ArrayList<>(List.of(10, 20, 30, 1, 2));
        nums.remove(1);                       // remove(int index)
        System.out.println(nums);
        nums.remove(Integer.valueOf(1));      // remove(Object)
        System.out.println(nums);

        String[] arr = {"a", "b", "c"};
        List<String> view = Arrays.asList(arr);
        view.set(0, "Z");
        System.out.println(Arrays.toString(arr) + " " + view);
        try {
            view.add("d");
        } catch (UnsupportedOperationException e) {
            System.out.println("asList add: " + e);
        }
        int[] prim = {1, 2, 3};
        System.out.println(Arrays.asList(prim).size());   // List<int[]> with ONE element

        List<Integer> immutable = List.of(1, 2, 3);
        try {
            immutable.add(4);
        } catch (UnsupportedOperationException e) {
            System.out.println("List.of add: " + e);
        }
        try {
            List.of(1, null);
        } catch (NullPointerException e) {
            System.out.println("List.of null: NPE");
        }
        try {
            immutable.contains(null);
        } catch (NullPointerException e) {
            System.out.println("List.of contains(null): NPE");
        }

        List<Integer> base = new ArrayList<>(List.of(0, 1, 2, 3, 4, 5, 6, 7));
        List<Integer> sub = base.subList(2, 5);
        sub.set(0, 99);
        System.out.println(base + " " + sub);
        sub.clear();                          // removes from base too
        System.out.println(base);
        base.add(100);                        // structural change to the parent
        try {
            System.out.println(sub.size());
        } catch (ConcurrentModificationException e) {
            System.out.println("sub after parent change: CME");
        }
    }
}
```

Run it TERMINAL

```
$ java Pitfalls.java
[10, 30, 1, 2]
[10, 30, 2]
[Z, b, c] [Z, b, c]
asList add: java.lang.UnsupportedOperationException
1
List.of add: java.lang.UnsupportedOperationException
List.of null: NPE
List.of contains(null): NPE
[0, 1, 99, 3, 4, 5, 6, 7] [99, 3, 4]
[0, 1, 5, 6, 7]
sub after parent change: CME
```

Go through the output slowly; each block is an interview favourite.

remove(int) versus remove(Object)

`nums.remove(1)` removed the element at index 1 (the value 20), not the value 1. To remove by value pass an object: `nums.remove(Integer.valueOf(1))`.

Arrays.asList is a fixed-size view

`Arrays.asList(arr)` does not copy. It returns a list **backed by the array**: `set` writes through to the array (`[Z, b, c]` in both), the size cannot change (`add` and `remove` throw `UnsupportedOperationException`). With a primitive array `Arrays.asList(prim)` gives a `List<int[]>` containing one element, because `T...` cannot be `int`. To get a normal mutable list write `new ArrayList<>(Arrays.asList(arr))` (or `List.of(arr)` for an immutable copy).

List.of is immutable and null-hostile

`List.of(...)` (Java 9) returns an **immutable** list. Mutators throw `UnsupportedOperationException`, nulls are rejected in the factory (`NullPointerException`), and even `contains(null)` throws. Compare that with the *unmodifiable view* from `Collections.unmodifiableList`, which only blocks changes through the view. The next chapters in this Part separate those ideas properly.

subList is a window, not a copy

`base.subList(2, 5)` is a **view** of the parent over indices 2 to 4. Writes and `clear()` on the view change the parent (the 99 appeared in `base`; `sub.clear()` deleted three elements from it). It is the idiomatic way to delete a range: `list.subList(from, to).clear()`. But if the parent is *structurally* modified directly after the view was created, any use of the view throws a `ConcurrentModificationException`. Treat a `subList` as short-lived; if you need to keep it, copy it: `new ArrayList<>(base.subList(2, 5))`.

> **GOTCHA**
> Do not hold on to a `subList` returned from a long-lived list. It keeps the whole parent alive in memory, and any later structural change to the parent invalidates it.

6.3.5 Everyday list API worth knowing

- `list.sort(comparator)` sorts in place (it delegates to `Arrays.sort` on the backing array, which is a stable merge sort variant; see the chapter on sorting).
- `list.replaceAll(UnaryOperator)` transforms every element in place; `list.removeIf(Predicate)` removes matches in one pass (for `ArrayList` this is a single compaction, O(n), instead of the O(n squared) of repeated `remove`).
- `list.indexOf`, `lastIndexOf`, `contains` use `equals`; `list.equals(other)` is true for any two lists with equal elements in the same order, regardless of implementation.
- `listIterator()` allows moving backwards and `add`/`set` at the cursor.
- `list.toArray(new String[0])` converts to a typed array; the zero-length array is a type token and is the recommended form.
- `Collections.nCopies`, `Collections.emptyList()` and `List.copyOf` build special-purpose immutable lists.

REAL WORLD
In Spring services the overwhelmingly common choice is `ArrayList`: query results, JSON arrays deserialised by Jackson and stream `collect(Collectors.toList())` results are all array-backed. The practical rules that matter at work are: return an empty list rather than `null`, do not expose your internal mutable list from a getter (return `List.copyOf(items)` or an unmodifiable view), and never mutate a list you received from a caller, since it might be an immutable `List.of`.

Check yourself

1. How many elements does `new ArrayList<>()` have room for right after construction, and after the first `add`? By what factor does it grow afterwards?
2. What does this print? `List<Integer> l = new ArrayList<>(List.of(5, 6, 7, 1)); l.remove(1); System.out.println(l);` How do you remove the value 1 instead?
3. What happens with `List<String> l = Arrays.asList("a", "b"); l.set(0, "x"); l.add("c");`?
4. Why is `for (int i = 0; i < list.size(); i++) list.get(i)` fast on `ArrayList` but a performance bug on `LinkedList`?
5. After `List<Integer> s = base.subList(1, 3); s.clear();` what happens to `base`? What if you then call `base.add(0)` and use `s` again?
6. Why prefer `ArrayDeque` over `Stack` and `LinkedList` for a stack?

Answers

1. Capacity 0 (lazy; the shared empty array), 10 after the first add, then roughly 1.5x (10, 15, 22, 33, ...) each time it fills.
2. `[5, 7, 1]` (index 1 removed). Use `l.remove(Integer.valueOf(1))`.
3. `set` works and writes through to the array; `add` throws `UnsupportedOperationException` because the list has a fixed size.
4. `ArrayList.get` is an O(1) array read; `LinkedList.get(i)` walks nodes from an end, O(n), making the whole loop O(n squared). Use an iterator or for-each instead.
5. Elements 1 and 2 are removed from `base` too (it is a view). After `base.add(0)` the view is stale and using it throws `ConcurrentModificationException`.
6. `Stack` is a legacy synchronized `Vector` subclass exposing list operations; `LinkedList` allocates a node per element. `ArrayDeque` is array-based, lighter and faster, and has the proper `push/pop/peek` API.

KEY POINTS
`ArrayList` is an array with lazy first allocation (capacity 10) and about 1.5x growth; `get` is O(1), middle insert/remove is O(n).

- Default to `ArrayList`; `LinkedList.get(i)` is O(n) and its nodes cost memory and cache misses. Use `ArrayDeque` for stacks and queues.
- `remove(int)` versus `remove(Object)` is a classic trap for `List<Integer>`.
- `Arrays.asList` is a fixed-size write-through view of the array; `List.of` is immutable and rejects null; `subList` is a live window onto its parent.
- `Vector` and `Stack` are legacy; `CopyOnWriteArrayList` suits read-mostly shared lists.

6.4 Sorting and comparing

Sorting looks like a solved problem: call `sort`. The real work is telling Java **what order you mean**. Numbers and strings have an obvious order, but a `Student` does not: by name, by year then grade, by grade descending with ties broken by name? Java gives you two tools for that, `Comparable` and `Comparator`, plus a library of ready-made algorithms in `Collections` and `Arrays`. This chapter covers how to define orders, how to combine them without writing loops of `if`s, the sorting algorithms underneath, and the bugs that come from comparators that are almost right.

6.4.1 Comparable: the natural order

A class has at most one **natural order**, defined by implementing `Comparable<T>` and its single method `compareTo`. The contract: `a.compareTo(b)` returns a negative number if `a` sorts before `b`, zero if they are equivalent in ordering, and a positive number if `a` sorts after. `String`, the wrapper classes, `LocalDate`, `BigDecimal` and enums are all `Comparable`. Sorted collections (`TreeSet`, `TreeMap`) and `Collections.sort(list)` use it when you give no comparator.

Sorting.java JAVA

```
import java.util.*;
import java.util.stream.*;

public class Sorting {
    record Student(String name, int year, double gpa) {}

    static class Book implements Comparable<Book> {
        final String title;
        final int pages;
        Book(String title, int pages) { this.title = title; this.pages = pages; }
        @Override public int compareTo(Book o) { return title.compareTo(o.title); }
        @Override public String toString() { return title + "(" + pages + ")"; }
    }

    public static void main(String[] args) {
        List<Book> books = new ArrayList<>(List.of(new Book("Emma", 474), new Book("Dune", 612),
            new Book("Ulysses", 730)));
        Collections.sort(books);                        // natural order via Comparable
        System.out.println(books);
        books.sort(Comparator.comparingInt(b -> b.pages));
        System.out.println(books);

        List<Student> st = new ArrayList<>(List.of(
            new Student("Asha", 2, 8.1), new Student("Ravi", 3, 9.0), new Student("Meena", 2, 9.0),
            new Student("Kiran", 3, 7.5), new Student("Bala", 2, 8.1)));
        st.sort(Comparator.comparingInt(Student::year)
                          .thenComparing(Student::gpa, Comparator.reverseOrder())
                          .thenComparing(Student::name));
        st.forEach(s -> System.out.println(s));

        // stability: sort by year only; equal years keep their previous relative order
        List<Student> byName = new ArrayList<>(st);
        byName.sort(Comparator.comparing(Student::name));
        byName.sort(Comparator.comparingInt(Student::year));
        System.out.println(byName.stream().map(Student::name).collect(Collectors.joining(",")));

        // nulls
        List<String> withNull = new ArrayList<>(Arrays.asList("pear", null, "apple", "fig"));
        withNull.sort(Comparator.nullsFirst(Comparator.naturalOrder()));
        System.out.println(withNull);
        withNull.sort(Comparator.nullsLast(Comparator.comparing(String::length)));
        System.out.println(withNull);
        try {
            Collections.sort(withNull);
        } catch (NullPointerException e) {
            System.out.println("natural order with null: NPE");
        }
    }
}
```

Run it TERMINAL

```
$ java Sorting.java
[Dune(612), Emma(474), Ulysses(730)]
[Emma(474), Dune(612), Ulysses(730)]
Student[name=Meena, year=2, gpa=9.0]
Student[name=Asha, year=2, gpa=8.1]
Student[name=Bala, year=2, gpa=8.1]
Student[name=Ravi, year=3, gpa=9.0]
Student[name=Kiran, year=3, gpa=7.5]
Asha,Bala,Meena,Kiran,Ravi
[null, apple, fig, pear]
[fig, pear, apple, null]
natural order with null: NPE
```

Because `Book` is `Comparable<Book>`, `Collections.sort(books)` just works. The natural order is the order of the *class*: choose it only when one order is clearly "the" order (titles alphabetically, dates chronologically). For anything else, use a comparator.

GOTCHA
`Comparable` is a decision you make once in the class and cannot override per call site. If you cannot edit the class (a library type) or you need several orders, you must use a `Comparator`.

6.4.2 Comparator: orders you pass in

A `Comparator<T>` is a separate object (usually a lambda) with the method `int compare(T a, T b)`. Since Java 8 you rarely write one by hand; you **build** it from small pieces with static and default methods:

Factory or combinator	Meaning
`Comparator.naturalOrder(), reverseOrder()`	the Comparable order, or its reverse
`Comparator.comparing(keyExtractor)`	order by a key, compared with the key's natural order
`Comparator.comparing(key, keyComparator)`	same, with an explicit comparator for the key
`Comparator.comparingInt / Long / Double`	avoids boxing for primitive keys
`cmp.thenComparing(...)`	tie-breaker used only when cmp says equal
`cmp.reversed()`	flips the whole comparator
`Comparator.nullsFirst(cmp), nullsLast(cmp)`	wraps cmp so that null elements are ordered instead of throwing

Building comparators.

The `Student` sort above reads like a sentence: by year ascending, then gpa descending, then name. `thenComparing(Student::gpa, Comparator.reverseOrder())` reverses just the gpa step. Compare it with the hand-written version, which is longer and where a single swapped sign silently gives wrong order:

ByHand.java JAVA

```java
// the old way: correct, but verbose and easy to get subtly wrong
Comparator<Student> byHand = (a, b) -> {
    if (a.year() != b.year()) return Integer.compare(a.year(), b.year());
    int g = Double.compare(b.gpa(), a.gpa());     // note the swapped arguments for descending
    if (g != 0) return g;
    return a.name().compareTo(b.name());
};
```

Three rules for writing comparators by hand:

1. Use `Integer.compare(a, b)`, `Double.compare`, `Long.compare`, `String.compareTo`; do not subtract.
2. To descend, swap the arguments, or use `.reversed()` on a comparator; never negate the result (`-compare` fails when it returns `Integer.MIN_VALUE`).
3. `reversed()` applies to the whole chain before it. `a.thenComparing(b).reversed()` reverses both; use `thenComparing(key, Comparator.reverseOrder())` to reverse only one step.

Nulls

Natural ordering cannot compare `null`: `Collections.sort` threw a `NullPointerException` in the example. `Comparator.nullsFirst(...)` and `nullsLast(...)` wrap another comparator and decide where nulls go. Notice in the output that `nullsLast(comparing(String::length))` put `null` at the end and kept the others in length order (`fig`, `pear`, `apple`).

6.4.3 Stability and the algorithms underneath

A sort is **stable** if elements that compare equal keep their original relative order. Stability makes multi-pass sorting work: sort by name, then sort by year, and within each year the names stay alphabetical. Our `byName` experiment shows it: after the second sort the output is `Asha,Bala,Meena,Kiran,Ravi`, still alphabetical inside each year.

What Java uses:

- `Collections.sort(list)`, `List.sort`, `Arrays.sort(Object[])` and `Arrays.sort(T[], Comparator)` use **TimSort** (since Java 7, by Tim Peters, also in Python): a stable merge sort that detects existing runs of sorted data. It is O(n

log n) worst case and close to O(n) on already-sorted or nearly sorted input. It needs up to n/2 extra memory for merging.

- `Arrays.sort(int[])` and the other primitive arrays use a **dual-pivot quicksort** (Vladimir Yaroslavskiy). It is O(n log n) on average and very fast, but not stable, which does not matter for primitives because equal ints are indistinguishable. `Arrays.parallelSort` splits the work across the common fork-join pool for large arrays.
- `ArrayList.sort` sorts its internal array directly; the default `List.sort` copies to an array, sorts it and writes it back.

> **NOTE**
> Do not try to implement sort order by calling `Collections.sort` on every insert. Use `TreeSet`/`TreeMap` if you need a collection kept sorted, or `PriorityQueue` if you only need the smallest element repeatedly (later in this Part).

6.4.4 Comparators that lie

A comparator must be a consistent total order: if `a < b` and `b < c` then `a < c`; `compare(a, b)` and `compare(b, a)` have opposite signs; equal elements compare equal to the same third elements. Breaking this gives wrong results, and TimSort may even throw `IllegalArgumentException: Comparison method violates its general contract!` (it detects some, not all, violations). The most frequent source is subtraction:

Broken.java JAVA

```java
import java.math.BigDecimal;
import java.util.*;

public class Broken {
    public static void main(String[] args) {
        Comparator<Integer> subtract = (a, b) -> a - b;       // looks fine, is not
        System.out.println(subtract.compare(Integer.MAX_VALUE, -1) + " <- should be positive");
        List<Integer> l = new ArrayList<>(List.of(5, Integer.MIN_VALUE, 3, Integer.MAX_VALUE));
        l.sort(subtract);
        System.out.println(l);
        l.sort(Integer::compare);
        System.out.println(l);

        BigDecimal a = new BigDecimal("1.0"), b = new BigDecimal("1.00");
        System.out.println(a.equals(b) + " " + (a.compareTo(b) == 0));
        Set<BigDecimal> hs = new HashSet<>(List.of(a, b));
        Set<BigDecimal> ts = new TreeSet<>(List.of(a, b));
        System.out.println("HashSet " + hs.size() + ", TreeSet " + ts.size());

        String[] words = {"banana", "Apple", "cherry", "apple"};
        Arrays.sort(words);
        System.out.println(Arrays.toString(words));
        Arrays.sort(words, String.CASE_INSENSITIVE_ORDER);
        System.out.println(Arrays.toString(words));
        Arrays.sort(words, Comparator.comparing(String::length)
                                     .thenComparing(Comparator.reverseOrder()));
        System.out.println(Arrays.toString(words));
    }
}
```

Run it TERMINAL

```
$ java Broken.java
-2147483648 <- should be positive
[3, 5, 2147483647, -2147483648]
[-2147483648, 3, 5, 2147483647]
false true
HashSet 2, TreeSet 1
[Apple, apple, banana, cherry]
[Apple, apple, banana, cherry]
[apple, Apple, cherry, banana]
```

What to notice:

- `a - b` overflows: `Integer.MAX_VALUE - (-1)` wraps to a negative number, so the "sorted" list ends with `-2147483648`. Always use `Integer.compare`.
- **Consistency with equals.** `new BigDecimal("1.0").equals(new BigDecimal("1.00"))` is `false` (different scale) but `compareTo` returns 0. A `HashSet` (equals/hashCode) keeps two elements; a `TreeSet` (compareTo) sees them as duplicates and keeps one. The documentation calls a natural ordering **consistent with equals** when `compareTo ==`

`0` exactly when `equals` is true. Sorted collections use `compareTo`/`compare` for identity, so an inconsistent ordering makes `TreeSet` violate the `Set` contract in a visible way. Strongly prefer comparators consistent with equals; if you cannot, document it.

- The last two lines of `Broken` show the default string order is by UTF-16 code unit: all uppercase letters come before lowercase (`Apple` before `apple`). For human-friendly alphabetical order use `String.CASE_INSENSITIVE_ORDER` or `java.text.Collator`, which respects the language rules. Our case-insensitive sort kept the two ties in their previous order precisely because the sort is stable. The final line sorts by length, then reverse alphabetical.

INTERVIEW ANGLE

What is the difference between `Comparable` and `Comparator`? `Comparable` is implemented by the class itself, defines one natural order (`compareTo(other)`), and lives in `java.lang`. `Comparator` is an external strategy object (`compare(a, b)`) in `java.util`; you can have many, pass them to `sort`, `TreeMap` and friends, and use them on classes you do not own. Add that comparators compose with `comparing`/`thenComparing`, that both must be consistent with `equals` for sorted sets and maps to behave, and that you never sort by subtraction because of overflow.

6.4.5 The Collections and Arrays utility classes

Both classes are static helper holders (you do not instantiate them). A tour of the useful members, all from one runnable program:

Util.java JAVA

```java
import java.util.*;

public class Util {
    public static void main(String[] args) {
        int[] nums = {42, 7, 19, 3, 88, 56};
        Arrays.sort(nums);
        System.out.println(Arrays.toString(nums));
        System.out.println(Arrays.binarySearch(nums, 19) + " " + Arrays.binarySearch(nums, 20)
            + " " + Arrays.binarySearch(nums, 1) + " " + Arrays.binarySearch(nums, 100));

        List<String> names = new ArrayList<>(List.of("kiwi", "apple", "fig", "plum"));
        Collections.sort(names);
        System.out.println(names + " idx " + Collections.binarySearch(names, "fig")
            + " missing " + Collections.binarySearch(names, "grape"));
        Collections.reverse(names);
        System.out.println(names);
        Collections.shuffle(names, new Random(7));
        System.out.println(names);
        Collections.swap(names, 0, 3);
        Collections.rotate(names, 1);
        System.out.println(names + " max=" + Collections.max(names)
            + " min=" + Collections.min(names, Comparator.comparing(String::length)));
        System.out.println(Collections.frequency(List.of(1, 2, 1, 1), 1)
            + " " + Collections.nCopies(3, "ab")
            + " " + Collections.disjoint(List.of(1), List.of(2)));
        List<String> sync = Collections.synchronizedList(new ArrayList<>());
        sync.add("x");
        System.out.println(sync);
        Integer[] boxed = {3, 1, 2};
        Arrays.sort(boxed, Collections.reverseOrder());
        System.out.println(Arrays.toString(boxed));
        System.out.println(Arrays.equals(new int[]{1,2}, new int[]{1,2}) + " "
            + Arrays.mismatch(new int[]{1,2,3}, new int[]{1,2,4}) + " "
            + Arrays.deepToString(new int[][]{{1},{2,3}}));
        int[] filled = new int[4]; Arrays.fill(filled, 9);
        System.out.println(Arrays.toString(Arrays.copyOfRange(filled, 1, 6)));
    }
}
```

Run it (the shuffled order depends on the JDK's Random algorithm; with seed 7 it is repeatable) TERMINAL

```
$ java Util.java
[3, 7, 19, 42, 56, 88]
2 -4 -1 -7
[apple, fig, kiwi, plum] idx 1 missing -3
[plum, kiwi, fig, apple]
[plum, kiwi, apple, fig]
[plum, fig, kiwi, apple] max=plum min=fig
3 [ab, ab, ab] true
[x]
[3, 2, 1]
true 2 [[1], [2, 3]]
[9, 9, 9, 0, 0]
```

Binary search and its return value

`binarySearch` requires a **sorted** input (using the same ordering) and runs in O(log n). Found: returns the index. Not found: returns `-(insertionPoint) - 1`, a negative number that tells you where the key would be inserted. In the output, searching for 20 in `[3, 7, 19, 42, 56, 88]` gave `-4`, so the insertion point is `-(-4) - 1 = 3`. Searching for 1 gave `-1` (insert at 0) and for 100 gave `-7` (insert at 6, the end). Always test `>= 0` rather than `!= -1`. On an unsorted array the result is undefined and no error is raised.

Memorise this short list

- `Arrays.asList`, `Arrays.toString`, `Arrays.deepToString` (nested arrays), `Arrays.equals`/`deepEquals`, `Arrays.fill`, `Arrays.copyOf`/`copyOfRange` (padding with zeros or nulls when the new length is longer, as the `[9, 9, 9, 0, 0]` line shows), `Arrays.mismatch` (first differing index, Java 9), `Arrays.hashCode`, `Arrays.stream`.
- `Collections.sort`, `reverse`, `shuffle`, `swap`, `rotate`, `max`/`min` (with optional comparator), `frequency`, `nCopies`, `disjoint`, `unmodifiableList` and friends, `synchronizedList` and friends, `emptyList`, `singletonList`.
- Do not print an array with `System.out.println(arr)`: you get `[I@1b6d3586`-style type-and-hash text. Use `Arrays.toString`.

GOTCHA

`Collections.synchronizedList(list)` makes each call thread-safe, but iterating over it still needs `synchronized (list) { ... }` around the whole loop. For real concurrency prefer the classes in `java.util.concurrent`.

REAL WORLD

Sorting happens in two places in a typical Spring application: in the database (`ORDER BY`, or `Sort`/`Pageable` in Spring Data) and in memory on small result sets, where `Comparator.comparing(Order::getCreatedAt).reversed()` or stream `sorted(...)` is the idiom. Sort in the database when the result is paged; sorting a full table in Java to take the top ten is a classic performance bug.

Check yourself

1. What is the output of `Comparator<Integer> c = (a, b) -> a - b;` `System.out.println(c.compare(Integer.MAX_VALUE, -1) > 0);` and why?
2. What is wrong with sorting `[3, 1, 2]` into descending order by `list.sort((a, b) -> a - b)`?
3. Which sort is stable: `Arrays.sort(int[])` or `Arrays.sort(String[])`? Which algorithm does each use?
4. `Arrays.binarySearch` returns `-4` for a missing key. Where would the key be inserted?
5. Why do `new HashSet<>(List.of(new BigDecimal("1.0"), new BigDecimal("1.00")))` and the `TreeSet` equivalent have different sizes?
6. How do you sort students by year ascending but gpa descending?

Answers

1. `false`: `Integer.MAX_VALUE - (-1)` overflows to `-2147483648`, which is negative. Use `Integer.compare`.
2. It sorts ascending, not descending (`a - b` orders smallest first), and it also risks overflow. Use `Comparator.reverseOrder()` or `(a, b) -> Integer.compare(b, a)`.
3. `String[]` (objects): TimSort, stable. `int[]` uses dual-pivot quicksort, which is not stable (irrelevant for primitives).
4. At index 3, because the result is `-(insertionPoint) - 1`.
5. `equals` distinguishes the scale (1.0 versus 1.00) so the `HashSet` has 2 elements, but `compareTo` returns 0 so the `TreeSet` treats them as duplicates and keeps 1. `BigDecimal`'s natural order is inconsistent with equals.

6. `Comparator.comparingInt(Student::year).thenComparing(Student::gpa, Comparator.reverseOrder())`.

KEY POINTS

`Comparable` gives a class one natural order; a `Comparator` supplies any other. Build them with `comparing`, `thenComparing`, `reversed`, `nullsFirst`.

- Never compare by subtraction; use `Integer.compare` and friends, and keep comparators consistent with `equals` when used in sorted sets and maps.
- Object sorts (`List.sort`, `Arrays.sort(T[])`) are stable TimSort; primitive array sorts are dual-pivot quicksort.
- `binarySearch` needs sorted input and returns `-(insertionPoint) - 1` when the key is absent.
- `Collections` and `Arrays` hold the everyday algorithms: reverse, shuffle, rotate, max/min, fill, copyOf, mismatch, deepToString.

6.5 Immutable, unmodifiable and sequenced collections

Mutable collections are convenient and dangerous. If a method returns its internal list, any caller can change your object's state behind its back. If two threads share a list that one of them edits, you get races. If a constant lookup table is mutable, someone eventually modifies it. The fix is collections that **cannot change**, and Java offers several kinds that look similar but behave very differently. This chapter sorts them out, then covers the Java 21 addition that finally gave collections with a defined order a common vocabulary: **sequenced collections**.

6.5.1 Three meanings of "read-only"

It is worth being precise, because interviews love this:

- A **mutable** collection (`ArrayList`) allows add and remove.
- An **unmodifiable view** (`Collections.unmodifiableList(list)`) is a wrapper that rejects changes made *through the wrapper*. The underlying list can still change, and the view shows those changes.
- An **immutable** collection (`List.of`, `List.copyOf`, `Set.of`, `Map.of`) never changes after creation, because nothing can reach its storage. This is the safe one.

Immut.java JAVA

```java
import java.util.*;
import java.util.stream.*;

public class Immut {
    public static void main(String[] args) {
        List<String> names = new ArrayList<>(List.of("ada", "linus"));
        List<String> view = Collections.unmodifiableList(names);
        List<String> copy = List.copyOf(names);
        names.add("grace");
        System.out.println("view " + view + " | copy " + copy);
        try {
            view.add("x");
        } catch (UnsupportedOperationException e) {
            System.out.println("view.add -> UnsupportedOperationException");
        }

        List<String> same = List.copyOf(copy);
        System.out.println("copyOf of immutable is same object: " + (same == copy));
        System.out.println("copyOf of mutable is same object: " + (List.copyOf(names) == names));

        Map<String, Integer> m = Map.of("a", 1, "b", 2);
        System.out.println(m.get("a") + " " + m.containsKey("z"));
        try {
            Map.of("k", 1, "k", 2);
        } catch (IllegalArgumentException e) {
            System.out.println("Map.of duplicate: " + e.getMessage());
        }
        try {
            Set.of("x", "x");
        } catch (IllegalArgumentException e) {
            System.out.println("Set.of duplicate: " + e.getMessage());
        }
        Map<String, Integer> big = Map.ofEntries(Map.entry("one", 1), Map.entry("two", 2));
        System.out.println(big.size());
```

Immut.java (continued) JAVA

```
        List<String> fromStream = Stream.of("a", null, "b").toList();
        System.out.println(fromStream);
        try {
            fromStream.add("c");
        } catch (UnsupportedOperationException e) {
            System.out.println("Stream.toList add -> UnsupportedOperationException");
        }
        List<String> collected = Stream.of("a", "b").collect(Collectors.toUnmodifiableList());
        System.out.println(collected);

        record Account(String id, List<String> history) {
            Account {
                history = List.copyOf(history);       // defensive copy in a compact constructor
            }
        }
        List<String> h = new ArrayList<>(List.of("open"));
        Account acc = new Account("A1", h);
        h.add("tamper");
        System.out.println(acc);

        // shallow immutability
        List<StringBuilder> sbs = List.of(new StringBuilder("x"));
        sbs.get(0).append("y");
        System.out.println(sbs);
    }
}
```

Run it TERMINAL

```
$ java Immut.java
view [ada, linus, grace] | copy [ada, linus]
view.add -> UnsupportedOperationException
copyOf of immutable is same object: true
copyOf of mutable is same object: false
1 false
Map.of duplicate: duplicate key: k
Set.of duplicate: duplicate element: x
2
[a, null, b]
Stream.toList add -> UnsupportedOperationException
[a, b]
Account[id=A1, history=[open]]
[xy]
```

Reading the output:

1. **View versus copy.** After `names.add("grace")` the unmodifiable view shows three names (it is a window onto `names`), but `List.copyOf` still has two. The wrapper protects against *callers* modifying, not against the owner modifying.
2. `copyOf` **is smart.** Given an already immutable list it returns the same object (no copying); given a mutable list it makes a snapshot. Use it for **defensive copies** of constructor arguments and getter results.
3. **Factories are strict.** `List.of`, `Set.of`, `Map.of` reject `null` elements and keys, `Set.of` and `Map.of` reject duplicates (`IllegalArgumentException`), and `Map.of` supports up to 10 pairs (for more use `Map.ofEntries(Map.entry(k, v), ...)`).
4. `Stream.toList()` (Java 16) returns an unmodifiable list, but unlike `List.of` it allows `null` elements. `Collectors.toUnmodifiableList()` (Java 10) rejects nulls. The older `Collectors.toList()` makes no promises, in practice an `ArrayList`.
5. **A record with a collection component** is only as immutable as you make it. The compact constructor with `List.copyOf` snapshots the list, so mutating the caller's list afterwards (`tamper`) does not leak in.
6. **Shallow immutability.** The list is immutable but its elements are not: `sbs.get(0).append("y")` changed what the list "contains". Immutable collections of immutable elements (`String`, records of immutable fields, `LocalDate`) are what you want.

WATCH OUT
Do not return a bare internal list from a getter. Return `List.copyOf(items)` (a snapshot, safe), or `Collections.unmodifiableList(items)` (a cheap live view; callers see later changes and the wrapper does not protect against your own concurrent modifications). Choose the snapshot when the list is small and stability matters.

Iteration order of Set.of and Map.of

For `List.of` the order is the order you gave. For `Set.of` and `Map.of` the iteration order is **deliberately unspecified and randomised per JVM run** to stop people depending on it. Run this program twice and you will normally see different orders:

Order.java — JAVA

```java
import java.util.*;

public class Order {
    public static void main(String[] args) {
        // Print the iteration order of Set.of / Map.of; it can differ between JVM runs.
        System.out.println(Set.of("a", "b", "c", "d", "e"));
        System.out.println(Map.of("a", 1, "b", 2, "c", 3, "d", 4));
    }
}
```

Two runs on the build machine (typical; orders change between runs) — TERMINAL

```
$ java Order.java
[e, d, c, b, a]
{d=4, c=3, b=2, a=1}
$ java Order.java
[a, b, c, d, e]
{a=1, b=2, c=3, d=4}
```

If a test compares a `Set.of` rendered as a string, it may pass today and fail tomorrow. Compare as sets, or use `LinkedHashSet` / `TreeSet` when order matters.

Why immutability is worth the effort

- **Thread safety for free.** Immutable objects can be shared across threads without locks (safely published, see the concurrency Part).
- **No aliasing bugs.** Nobody changes your data behind your back, so getters and constructors stay simple.
- **Reliable keys.** An immutable object has a stable `hashCode`, so it is a safe `HashMap` key or `HashSet` element.
- **Efficiency.** The `List.of` implementations are compact and, for up to two elements, use specialised field-based classes without an array.

Costs: every "change" creates a new collection (copy-on-write), so building a large collection by repeated copying is O(n squared). Build it with a mutable collection and freeze once at the end (`List.copyOf`, `stream.toList()`).

INTERVIEW ANGLE
What is the difference between `Collections.unmodifiableList(list)` and `List.copyOf(list)` or `List.of(...)`? `unmodifiableList` is a read-only live view: the original list can still change and the view reflects it. `List.copyOf` and `List.of` are genuinely immutable snapshots with no back door. Also: `List.of` forbids nulls and `contains(null)` throws, while the unmodifiable wrapper delegates to the underlying list. In all cases the elements themselves may still be mutable (shallow immutability).

6.5.2 Sequenced collections (Java 21)

Before Java 21, getting "the first element" or "the last element" had a different spelling for every collection:

Collection	First	Last
List	list.get(0)	list.get(list.size() - 1)
Deque	deque.getFirst()	deque.getLast()
SortedSet	set.first()	set.last()
LinkedHashSet	set.iterator().next()	no direct way (iterate everything)
LinkedHashMap	iterate entrySet	iterate to the end

First and last before Java 21.

Reversing was just as inconsistent (`Collections.reverse` copies/mutates; `descendingSet`, `descendingIterator` only on some). Java 21 (JEP 431) introduced three interfaces for collections that have a **defined encounter order**:

- `SequencedCollection<E>` extends `Collection<E>`: `getFirst`, `getLast`, `addFirst`, `addLast`, `removeFirst`, `removeLast`, and `reversed()`.
- `SequencedSet<E>` extends `SequencedCollection` and `Set`.
- `SequencedMap<K,V>` extends `Map`: `firstEntry`, `lastEntry`, `pollFirstEntry`, `pollLastEntry`, `putFirst`, `putLast`, `reversed()`, plus `sequencedKeySet()`, `sequencedValues()`, `sequencedEntrySet()`.

`List`, `Deque` and `LinkedHashSet` (and sorted sets via `SortedSet`) join the collection side; `LinkedHashMap` and `SortedMap` join the map side. `HashSet` and `HashMap` are not sequenced: they have no meaningful first or last element. This is the "where did it go in the hierarchy diagram" answer from the overview chapter.

Seq.java JAVA

```java
import java.util.*;

public class Seq {
    public static void main(String[] args) {
        List<String> list = new ArrayList<>(List.of("b", "c", "d"));
        list.addFirst("a");
        list.addLast("e");
        System.out.println(list + " first=" + list.getFirst() + " last=" + list.getLast());
        List<String> rev = list.reversed();
        System.out.println(rev);
        list.add("f");
        System.out.println("reversed view sees the change: " + rev);
        rev.removeFirst();
        System.out.println("removed via view: " + list);

        try {
            new ArrayList<String>().getFirst();
        } catch (NoSuchElementException e) {
            System.out.println("getFirst on empty: NoSuchElementException");
        }
        try {
            List.of(1, 2).addFirst(0);
        } catch (UnsupportedOperationException e) {
            System.out.println("List.of addFirst: UnsupportedOperationException");
        }

        LinkedHashSet<String> set = new LinkedHashSet<>(List.of("x", "y", "z"));
        set.addFirst("z");                      // moves an existing element to the front
        System.out.println(set + " " + set.reversed());

        LinkedHashMap<String, Integer> lhm = new LinkedHashMap<>();
        lhm.put("one", 1); lhm.put("two", 2); lhm.put("three", 3);
        lhm.putFirst("zero", 0);
        System.out.println(lhm + " first=" + lhm.firstEntry() + " last=" + lhm.lastEntry());
        System.out.println(lhm.reversed());
        System.out.println(lhm.sequencedKeySet().reversed());
        Map.Entry<String, Integer> polled = lhm.pollLastEntry();
        System.out.println(polled + " -> " + lhm);

        Deque<Integer> dq = new ArrayDeque<>(List.of(1, 2, 3));
        System.out.println(dq.reversed() + " " + (dq instanceof SequencedCollection<Integer>));
        TreeSet<Integer> ts = new TreeSet<>(List.of(5, 1, 3));
        System.out.println(ts.getFirst() + " " + ts.reversed());
        try {
            ts.addFirst(0);
        } catch (UnsupportedOperationException e) {
            System.out.println("TreeSet addFirst: UnsupportedOperationException");
        }
    }
}
```

Run it (JDK 21 or later) TERMINAL

```
$ java Seq.java
[a, b, c, d, e] first=a last=e
[e, d, c, b, a]
reversed view sees the change: [f, e, d, c, b, a]
removed via view: [a, b, c, d, e]
```

Run it (JDK 21 or later) (continued) TERMINAL

```
getFirst on empty: NoSuchElementException
List.of addFirst: UnsupportedOperationException
[z, x, y] [y, x, z]
{zero=0, one=1, two=2, three=3} first=zero=0 last=three=3
{three=3, two=2, one=1, zero=0}
[three, two, one, zero]
three=3 -> {zero=0, one=1, two=2}
[3, 2, 1] true
1 [5, 3, 1]
TreeSet addFirst: UnsupportedOperationException
```

What to notice:

- `reversed()` **is a view, not a copy.** After `list.add("f")` the reversed view already showed `f` at its front, and `rev.removeFirst()` removed the last element of the original list. It is O(1) to create. Copy it (`new ArrayList<>(list.reversed())`) if you need an independent list.
- **Empty collections throw.** `getFirst()` on an empty list throws `NoSuchElementException`, unlike `Deque.peekFirst()` which returns `null`. The old `list.get(0)` threw `IndexOutOfBoundsException`.
- **Mutators are optional operations.** An immutable `List.of` throws `UnsupportedOperationException` on `addFirst`, and so does a `TreeSet`, because its position is dictated by the sort order, not by you. `getFirst` and `reversed` work on both.
- `LinkedHashSet.addFirst("z")` **moved the existing element**, since sets have no duplicates; for `LinkedHashMap`, `putFirst` / `putLast` likewise reposition an existing key.
- `ArrayDeque` is a `SequencedCollection` (the `instanceof` printed `true`); `Deque` already had most of these methods, which is why the new interface fits it with only `reversed()` new.

Java 21 also added matching wrapper factories to `Collections` (`unmodifiableSequencedCollection`, `unmodifiableSequencedSet`, `unmodifiableSequencedMap`) so that read-only views keep the sequenced methods.

> **NOTE**
> Sequenced collections arrived in Java 21, the previous LTS. If your project is on Java 17, none of this compiles: use `list.get(list.size() - 1)`, `Deque` methods, or `Collections.reverse` on a copy.

> **REAL WORLD**
> "Latest N items" and "most recent first" are everyday requirements: a recent-activity feed, a browser history, an audit trail. With Java 21 `history.getLast()` and `history.reversed()` express them directly, and `LinkedHashMap` is the basis of the LRU caches covered in the maps chapter later in this Part.

Check yourself

1. What does this print? `List<Integer> a = new ArrayList<>(List.of(1, 2)); List<Integer> v = Collections.unmodifiableList(a); a.add(3); System.out.println(v);`
2. Which of these throws, and what? `List.of(1, null)`, `Arrays.asList(1, null)`, `Stream.of(1, null).toList()`.
3. Why can the output of `System.out.println(Set.of("a", "b", "c"))` differ between two runs of the same program?
4. Is a `record Team(String name, List<String> members)` immutable? How do you make it so?
5. What do `list.reversed()` and `new ArrayList<>(list).reversed()` have in common, and how do they differ from `Collections.reverse(list)`?
6. Which of `HashSet`, `LinkedHashSet`, `TreeSet`, `ArrayDeque` are `SequencedCollection`s, and what happens on `treeSet.addFirst(x)`?

Answers

1. `[1, 2, 3]`. The unmodifiable wrapper is a live view of `a`.
2. Only `List.of(1, null)` throws (`NullPointerException`); `Arrays.asList` and `Stream.toList()` accept nulls.
3. `Set.of` (and `Map.of`) iteration order is intentionally unspecified and randomised per JVM instance.
4. Not by itself: the list component can be mutated by whoever holds it. Add a compact constructor `members = List.copyOf(members);` (and keep elements immutable).
5. `reversed()` returns a live O(1) reverse-order view; for `new ArrayList<>(list).reversed()` the view is over the copy. `Collections.reverse(list)` physically reverses the list in place, O(n), and changes it.

6. `LinkedHashSet`, `TreeSet` (via `SortedSet`) and `ArrayDeque`; `HashSet` is not. `addFirst` on a `TreeSet` throws `UnsupportedOperationException` because the order is determined by the comparator.

KEY POINTS

Unmodifiable views block changes through the wrapper; immutable collections (`List.of`, `copyOf`, `Stream.toList`) cannot change at all, but their elements may.

- `List.of`/`Set.of`/`Map.of` reject nulls and duplicates; `Set`/`Map` iteration order is randomised, so never depend on it.
- Make defensive copies with `List.copyOf` in constructors (records: compact constructor) and in getters.
- Java 21 sequenced collections give `getFirst`, `getLast`, `addFirst`, `removeLast` and `reversed()` to lists, deques, `LinkedHashSet`, `LinkedHashMap` and sorted collections; `reversed()` is a live view.
- Choose between snapshot copy and live view deliberately; both are "read-only" but behave very differently.

6.6 Sets

A **set** is a collection with one promise: no duplicates. You reach for a set when the question is "have I seen this before?", "which items are in both groups?" or "give me the distinct values". A `List` can answer those questions, but slowly: `list.contains(x)` walks the whole list, so checking a million items against a million-item list is a trillion comparisons. A well-chosen set answers `contains` in constant or logarithmic time.

The `java.util.Set` interface adds almost nothing to `Collection`. It tightens the contract (`add` returns `false` and does nothing if an equal element is already present) and defines what "equal" and "hash" mean for whole sets. The interesting part is the three main implementations, because they differ in ordering and speed. Learn them as a trio.

Class	Order of iteration	add / contains / remove	Needs from elements
`HashSet`	none you may rely on	O(1) average	equals and hashCode
`LinkedHashSet`	insertion order	O(1) average	equals and hashCode
`TreeSet`	sorted (natural or Comparator)	O(log n)	Comparable or a Comparator

The three general-purpose sets.

6.6.1 HashSet: a HashMap in disguise

`HashSet` does not contain its own hashing logic. Open the JDK source and you will find one field: a `HashMap<E, Object>` called `map`, plus a static dummy value shared by every entry.

HashSetInternals.java (simplified from the JDK source) JAVA

```
public class HashSet<E> extends AbstractSet<E> implements Set<E> {
    private transient HashMap<E, Object> map;
    private static final Object PRESENT = new Object();   // one shared dummy value

    public boolean add(E e)      { return map.put(e, PRESENT) == null; }
    public boolean contains(Object o) { return map.containsKey(o); }
    public boolean remove(Object o)   { return map.remove(o) == PRESENT; }
    public int size()            { return map.size(); }
    public Iterator<E> iterator() { return map.keySet().iterator(); }
}
```

That is the whole idea. The element is the map key; the value is a dummy. `add` returns `true` exactly when `put` found no previous value for the key. Everything you learn in the next chapter about `HashMap` (buckets, hash spreading, resizing, treeification, the cost of a bad `hashCode`) is therefore also true for `HashSet`. `LinkedHashSet` is built the same way on top of `LinkedHashMap`, and `TreeSet` on top of `TreeMap`.

Two consequences matter in practice. First, an element's identity inside a hash set is decided by `hashCode()` and `equals()`, never by `==`. Two distinct `Point` objects with equal fields are the same element if the class overrides both methods (a `record` does so for you). Second, iteration order is whatever the table happens to look like.

SetsDemo.java (part 1: the three orders) JAVA

```java
import java.util.*;

public class SetsDemo {
    public static void main(String[] args) {
        List<String> fruit = List.of("pear", "apple", "fig", "kiwi", "banana");
        System.out.println("HashSet:       " + new HashSet<>(fruit));
        System.out.println("LinkedHashSet: " + new LinkedHashSet<>(fruit));
        System.out.println("TreeSet:       " + new TreeSet<>(fruit));
        Set<String> hs = new HashSet<>(fruit);
        System.out.println("add duplicate: " + hs.add("fig"));
    }
}
```

Run it (the HashSet line is typical; do not depend on it) TERMINAL

```
$ java SetsDemo.java
HashSet:       [banana, apple, pear, kiwi, fig]
LinkedHashSet: [pear, apple, fig, kiwi, banana]
TreeSet:       [apple, banana, fig, kiwi, pear]
add duplicate: false
```

The `HashSet` order looks random but is not: it is the order of the buckets in the table, and a bucket is picked from the string's hash code. Because `String.hashCode()` is specified, the same strings in the same-sized table give the same order on every run of the same JDK. Still, a different table size, a different JDK version or a different insertion history can change it. Treat it as unordered. (Sets created with `Set.of(...)` go further: their iteration order is deliberately randomised per JVM start, to stop programs from depending on it.)

TIP
Constructing a set from a collection is the idiom for removing duplicates: `new HashSet<>(list)` loses order, `new LinkedHashSet<>(list)` removes duplicates but keeps first-seen order, `new TreeSet<>(list)` removes duplicates and sorts. Pick by the order you want afterwards.

6.6.2 LinkedHashSet: predictable order for a small price

`LinkedHashSet` is a `HashSet` whose entries are additionally threaded on a doubly linked list. The hash table still does lookups; the linked list only remembers insertion order, so iteration walks the list instead of scanning empty buckets. The costs are two extra references per entry (about 8 bytes with compressed pointers) and slightly slower inserts. Re-inserting an element that is already present does not move it. Use it whenever a user will see the output, in tests where you compare printed results, and for "distinct but keep order" jobs.

From Java 21 it is a `SequencedSet`, so it has `getFirst()`, `getLast()`, `removeFirst()` and `reversed()`; the sequenced collections are covered in the chapter on immutable and sequenced collections.

6.6.3 TreeSet and NavigableSet: sorted, with range queries

`TreeSet` keeps its elements sorted using a red-black tree (a self-balancing binary search tree; the chapter on other maps explains the shape). Because it is ordered, it can answer questions a hash set cannot: what is the smallest element at least 25? Which elements fall between 20 and 40? These live in the `NavigableSet` interface (which extends `SortedSet`).

Method	Meaning
`first() / last()`	smallest / largest element (throws if empty)
`pollFirst() / pollLast()`	remove and return smallest / largest (null if empty)
`floor(e) / ceiling(e)`	greatest element ≤ e / least element ≥ e (null if none)
`lower(e) / higher(e)`	greatest element < e / least element > e
`headSet(to, inclusive)`	view of elements before to
`tailSet(from, inclusive)`	view of elements from from onward
`subSet(from, fi, to, ti)`	view between two bounds
`descendingSet()`	view in reverse order

Main NavigableSet methods. All are O(log n).

NavigableDemo.java JAVA

```java
import java.util.*;

public class NavigableDemo {
    public static void main(String[] args) {
        NavigableSet<Integer> n = new TreeSet<>(List.of(10, 20, 30, 40, 50));
        System.out.println("floor(25)=" + n.floor(25) + " ceiling(25)=" + n.ceiling(25)
            + " lower(30)=" + n.lower(30) + " higher(30)=" + n.higher(30));
        System.out.println("headSet(30)=" + n.headSet(30) + " tailSet(30)=" + n.tailSet(30));
        System.out.println("subSet(20,true,40,true)=" + n.subSet(20, true, 40, true));
        System.out.println("descending=" + n.descendingSet() + " pollFirst=" + n.pollFirst());

        NavigableSet<Integer> head = n.headSet(40, false);   // a VIEW, not a copy
        n.add(35);
        System.out.println("view sees the new element: " + head);
    }
}
```

Run it TERMINAL

```
$ java NavigableDemo.java
floor(25)=20 ceiling(25)=30 lower(30)=20 higher(30)=40
headSet(30)=[10, 20] tailSet(30)=[30, 40, 50]
subSet(20,true,40,true)=[20, 30, 40]
descending=[50, 40, 30, 20, 10] pollFirst=10
view sees the new element: [20, 30, 35]
```

NOTE

`headSet`, `tailSet`, `subSet` and `descendingSet` return live **views** of the original tree, not copies. Adding to the original shows up in the view, and adding an out-of-range element to the view throws `IllegalArgumentException`. Wrap in `new TreeSet<>(view)` if you want a snapshot.

Equality in a TreeSet is decided by compareTo, not equals

`TreeSet` never calls `equals` or `hashCode`. Two elements are "the same" when the comparison returns 0. That is usually what you want, but it surprises people whose comparator looks at only one field.

TreeSetCompare.java JAVA

```java
import java.util.*;

public class TreeSetCompare {
    public static void main(String[] args) {
        Set<String> ci = new TreeSet<>(String.CASE_INSENSITIVE_ORDER);
        ci.add("Java"); ci.add("JAVA"); ci.add("java");
        System.out.println("case-insensitive: " + ci);

        Set<String> byLen = new TreeSet<>(Comparator.comparingInt(String::length));
        byLen.addAll(List.of("cat", "dog", "bird"));
        System.out.println("by length: " + byLen);      // "dog" silently dropped

        try {
            new TreeSet<Object>().add(new Object());
        } catch (ClassCastException e) {
            System.out.println("not Comparable: " + e.getClass().getSimpleName());
        }
    }
}
```

Run it TERMINAL

```
$ java TreeSetCompare.java
case-insensitive: [Java]
by length: [cat, bird]
not Comparable: ClassCastException
```

The second line is the classic bug: `cat` and `dog` have length 3, so the comparator says they are equal and `dog` is dropped. If your comparator orders by one field, add tie-breakers with `thenComparing` until it only returns 0 for elements you truly consider equal. The third case shows that a `TreeSet` of non-`Comparable` elements fails on the very first `add`, not later.

Two more `TreeSet` rules: it rejects `null` (natural ordering cannot compare it), and it is not thread-safe (`ConcurrentSkipListSet` is the sorted concurrent alternative).

6.6.4 EnumSet: the fastest set you will ever use

When the elements are values of one enum, use `EnumSet`. It is abstract; you create instances through factories such as `EnumSet.of(...)`, `noneOf(Class)`, `allOf(Class)`, `range(a, b)`, `complementOf(set)` and `copyOf(collection)`. Internally an enum set of up to 64 constants is a single `long` used as a bit mask (bit number = the constant's `ordinal()`), larger enums use a `long[]`. `add`, `contains` and `remove` are a shift and a mask; `containsAll`, `retainAll` and `addAll` are one bitwise instruction per word. Iteration is in declaration order of the constants.

EnumSetDemo.java JAVA

```java
import java.util.*;

public class EnumSetDemo {
    enum Day { MON, TUE, WED, THU, FRI, SAT, SUN }

    public static void main(String[] args) {
        Set<Day> weekend = EnumSet.of(Day.SAT, Day.SUN);
        Set<Day> workdays = EnumSet.complementOf(EnumSet.copyOf(weekend));
        System.out.println(weekend + " " + workdays);
        System.out.println("range = " + EnumSet.range(Day.TUE, Day.THU));
        System.out.println("allOf size = " + EnumSet.allOf(Day.class).size());
    }
}
```

Run it TERMINAL

```
$ java EnumSetDemo.java
[SAT, SUN] [MON, TUE, WED, THU, FRI]
range = [TUE, WED, THU]
allOf size = 7
```

Use an `EnumSet<Permission>` instead of an `int` full of flag bits: you get the same speed with type safety and readable output.

6.6.5 Set algebra with bulk operations

The bulk methods of `Collection` are exactly the mathematical set operations. The trick is that they modify the receiver, so copy first when you need the originals again.

Operation	Code	Result for a={1..5}, b={4..7}
union	copy.addAll(b)	1 2 3 4 5 6 7
intersection	copy.retainAll(b)	4 5
difference a minus b	copy.removeAll(b)	1 2 3
subset test	b.containsAll(a)	false

Set algebra with Collection methods.

SetAlgebra.java JAVA

```java
import java.util.*;

public class SetAlgebra {
    static <T> Set<T> union(Set<T> a, Set<T> b) {
        Set<T> r = new TreeSet<>(a);
        r.addAll(b);
        return r;
    }
    static <T> Set<T> intersection(Set<T> a, Set<T> b) {
        Set<T> r = new TreeSet<>(a);
        r.retainAll(b);
        return r;
    }
    static <T> Set<T> difference(Set<T> a, Set<T> b) {
        Set<T> r = new TreeSet<>(a);
        r.removeAll(b);
        return r;
    }
```

SetAlgebra.java (continued) JAVA

```
    public static void main(String[] args) {
        Set<Integer> a = Set.of(1, 2, 3, 4, 5);
        Set<Integer> b = Set.of(4, 5, 6, 7);
        System.out.println("union        = " + union(a, b));
        System.out.println("intersection = " + intersection(a, b));
        System.out.println("a minus b    = " + difference(a, b));
        Set<Integer> sym = union(a, b);
        sym.removeAll(intersection(a, b));
        System.out.println("symmetric    = " + sym);
        System.out.println("a contains all of intersection: " + a.containsAll(intersection(a, b)));
    }
}
```

Run it TERMINAL

```
$ java SetAlgebra.java
union        = [1, 2, 3, 4, 5, 6, 7]
intersection = [4, 5]
a minus b    = [1, 2, 3]
symmetric    = [1, 2, 3, 6, 7]
a contains all of intersection: true
```

(The helpers copy into a `TreeSet` only so the printed order is stable; use a `HashSet` copy when you do not care.) A performance trap hides in `removeAll`: if the argument is a `List`, each element of the receiver is looked up with `List.contains`, which is O(n). Pass a set as the argument.

6.6.6 The mutating-element bug

A hash set stores each element in the bucket chosen by its hash code at the time of insertion. If the element changes in a way that alters `hashCode()` while it sits in the set, the set still looks for it in the old bucket and never finds it again. The object is still there, still reachable by iteration, and impossible to look up or remove.

MutationBug.java JAVA

```
import java.util.*;

public class MutationBug {
    static class Tag {
        String name;
        Tag(String name) { this.name = name; }
        @Override public boolean equals(Object o) {
            return o instanceof Tag t && t.name.equals(name);
        }
        @Override public int hashCode() { return name.hashCode(); }
        @Override public String toString() { return name; }
    }

    public static void main(String[] args) {
        Set<Tag> tags = new HashSet<>();
        Tag t = new Tag("java");
        tags.add(t);
        t.name = "jvm";                                  // mutate while inside the set
        System.out.println("contains(t)       = " + tags.contains(t));
        System.out.println("size              = " + tags.size());
        System.out.println("iteration         = " + tags);
        System.out.println("contains(\"java\")  = " + tags.contains(new Tag("java")));
        System.out.println("remove(t)         = " + tags.remove(t));
    }
}
```

Run it TERMINAL

```
$ java MutationBug.java
contains(t)       = false
size              = 1
iteration         = [jvm]
contains("java")  = false
remove(t)         = false
```

Neither `contains(t)` nor `contains(new Tag("java"))` succeeds: the first looks in the bucket for "jvm" (empty), the second finds the bucket for "java" but the stored object's `equals` no longer matches. The element is leaked until the set

is discarded. The same applies to keys of a `HashMap` and to elements of a `TreeSet` whose ordering fields change (the tree is simply mis-ordered).

The rule: **elements of sets and keys of maps must be immutable, or at least their `equals`/`hashCode`/ordering fields must never change while they are in the collection.** Prefer `String`, boxed numbers, enums, `record`s of immutable fields and `java.time` values.

GOTCHA

`Set.of(...)` and `Map.of(...)` reject `null` and also throw `NullPointerException` from `contains(null)`; `HashSet` and `LinkedHashSet` accept one `null`; `TreeSet` accepts none. `Set.of("a", "a")` throws `IllegalArgumentException` for the duplicate. Know which factory you are holding when null is possible.

INTERVIEW ANGLE

How does a HashSet guarantee uniqueness, and what happens if you put objects that do not override equals and hashCode? It delegates to a `HashMap`: the element is the key, so `add` computes the hash, finds the bucket and uses `equals` to look for an existing equal key. Without overrides, `Object`'s identity-based versions are used, so two objects with equal fields are treated as different elements and "duplicates" appear. With `hashCode` overridden but `equals` not (or vice versa) the contract is broken and lookups become unreliable.

INTERVIEW ANGLE

When would you choose TreeSet over HashSet? When you need sorted iteration or range/neighbour queries (`floor`, `ceiling`, `subSet`) and can pay O(log n) per operation. If you only test membership, `HashSet` is faster. If the elements are enum constants, use `EnumSet`.

Check yourself

1. What data structure does `HashSet` use internally, and what is stored as the value?
2. What does this print? `Set<String> s = new TreeSet<>(Comparator.comparingInt(String::length)); s.addAll(List.of("ab", "cd", "efg")); System.out.println(s);`
3. You add a mutable object to a `HashSet`, change a field used by `hashCode`, then call `set.contains(obj)`. What is the likely result, and why?
4. Which set would you use for the days of the week on which a shop is open, and why?
5. `TreeSet<Integer> t = new TreeSet<>(List.of(10, 20, 30)); t.ceiling(21)` and `t.lower(10)` return what?
6. Why is `bigSet.removeAll(someList)` potentially slow, and what is the fix?

Answers

1. A `HashMap<E, Object>`; every entry maps the element to one shared dummy object (`PRESENT`).
2. `[ab, efg]`. "ab" and "cd" compare equal (same length), so "cd" is dropped.
3. Most likely `false`. The object sits in the bucket for its old hash; `contains` now searches the bucket for the new hash.
4. `EnumSet<DayOfWeek>`: a bit mask, so tiny, fast, type-safe and iterated in week order.
5. `30` and `null` (nothing is strictly lower than 10).
6. For each element of `bigSet` it calls `someList.contains`, which is O(n), giving O(n times m). Pass a `HashSet` built from the list.

KEY POINTS

Key points

- A `HashSet` is a `HashMap` whose values are a dummy; uniqueness comes from `hashCode` and `equals`, so both must be consistent and based on immutable data.
- `LinkedHashSet` keeps insertion order for a small memory cost; `TreeSet` keeps sorted order and offers `NavigableSet` range queries in O(log n), with equality decided by the comparator.
- `EnumSet` is a bit mask: use it for sets of enum constants. Copy before using `addAll`/`retainAll`/`removeAll` if you need the original.
- Mutating an element after insertion makes it unreachable in hash sets and corrupts tree order; keep elements immutable.

6.7 Other maps: LinkedHashMap, TreeMap and the specialists

`HashMap` is the default, but the `Map` interface has a family of implementations that each trade something for something else. Choosing the right one often replaces twenty lines of hand-written code with a constructor call. This chapter covers the ones you will meet in real code and in interviews, in order of how often you need them.

Class	Ordering	Null keys	Typical use
`HashMap`	none	one null key	the default
`LinkedHashMap`	insertion or access order	one null key	predictable order, LRU caches
`TreeMap`	sorted by key	none	sorted views, range queries
`EnumMap`	enum declaration order	none	enum-keyed data
`IdentityHashMap`	none	allowed	graph algorithms, serialization internals
`WeakHashMap`	none	allowed	metadata caches keyed by objects
`ConcurrentHashMap`	none	none	shared maps across threads
`Hashtable`	none	none	legacy; do not use in new code

The Map family at a glance.

6.7.1 LinkedHashMap: order and the LRU cache

`LinkedHashMap` extends `HashMap` and additionally threads every entry onto a doubly linked list (each node gets `before` and `after` references). Lookups use the hash table as usual; iteration follows the list. By default the list is in **insertion order**. A re-`put` of an existing key does not move it.

The second mode is **access order**. Construct it with the three-argument constructor `new LinkedHashMap<>(initialCapacity, loadFactor, true)` and every `get`, `put` or `compute` on a key moves its entry to the end of the list, so the first entry is always the least recently used. Combine that with the protected hook `removeEldestEntry(Map.Entry)`, which `put` calls after inserting; return `true` and the map deletes the eldest entry. That is a complete LRU (least-recently-used) cache in a dozen lines.

LruCache.java JAVA

```java
import java.util.*;

public class LruCache {
    static class Lru<K, V> extends LinkedHashMap<K, V> {
        private final int capacity;
        Lru(int capacity) {
            super(16, 0.75f, true);              // true = access order
            this.capacity = capacity;
        }
        @Override protected boolean removeEldestEntry(Map.Entry<K, V> eldest) {
            return size() > capacity;
        }
    }

    public static void main(String[] args) {
        Lru<String, Integer> cache = new Lru<>(3);
        cache.put("a", 1);
        cache.put("b", 2);
        cache.put("c", 3);
        cache.get("a");                          // "a" becomes most recently used
        cache.put("d", 4);                       // evicts "b", the least recently used
        System.out.println("keys, eldest first: " + cache.keySet());
        System.out.println("contains b? " + cache.containsKey("b"));
    }
}
```

Run it TERMINAL

```
$ java LruCache.java
keys, eldest first: [c, a, d]
contains b? false
```

After the `get("a")` the order is `b, c, a`; inserting `d` makes the size 4 which exceeds the capacity, so `b` (the eldest) goes. The result is `c, a, d`.

WATCH OUT
This LRU cache is not thread-safe, and in access-order mode even `get` modifies the list, so two concurrent readers can corrupt it. Wrap it with `Collections.synchronizedMap` or use a purpose-built cache library (Caffeine is the standard choice in Spring applications). The last chapter of this Part builds an LRU from scratch with a hash map plus your own linked list.

From Java 21, `LinkedHashMap` is a `SequencedMap`: it has `firstEntry()`, `lastEntry()`, `putFirst`, `putLast`, `pollFirstEntry()` and `reversed()`. For example `putFirst("z", 0)` on a map holding a, b, c gives `{z=0, a=1, b=2, c=3}`, and `reversed()` shows the entries last-to-first as a view.

6.7.2 TreeMap: the sorted map

`TreeMap` keeps its entries sorted by key, using a **red-black tree**. Sorted order is the whole point: it supports everything a sorted structure can do, and every basic operation (`put`, `get`, `remove`, `containsKey`) costs O(log n) because the tree is balanced.

What a red-black tree is

A binary search tree stores smaller keys in the left subtree and larger keys in the right. Without care, inserting already-sorted keys (1, 2, 3, ...) builds a lopsided chain with O(n) lookups. A red-black tree is a binary search tree whose nodes are coloured red or black, with rules that keep it approximately balanced:

1. The root is black.
2. A red node has only black children (no two reds in a row).
3. Every path from a node down to its empty leaves passes through the same number of black nodes.

Rule 3 and rule 2 together mean that the longest root-to-leaf path is at most twice the shortest, so the height stays at most about 2 log2(n). When an insertion or removal breaks a rule, the tree repairs itself with recolourings and **rotations**, local rearrangements that change the shape but keep the left-smaller-right-larger order. You will not be asked to code deletion, but you should be able to state the rules and the O(log n) guarantee. Each `TreeMap` entry also stores a `left`, a `right`, a `parent` and a colour flag, so it is heavier than a `HashMap` node.

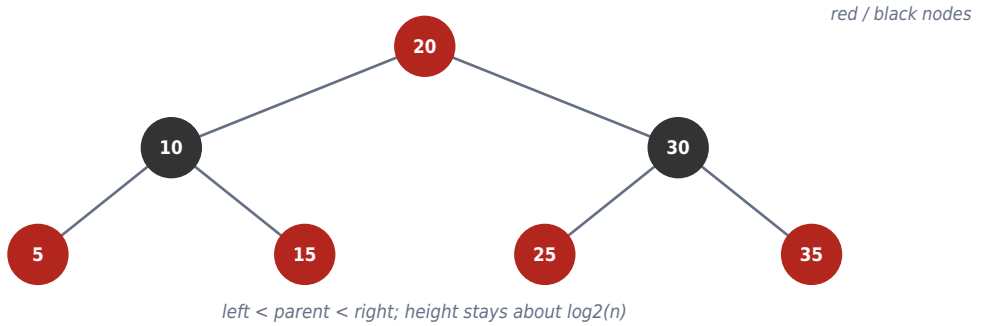


Figure 6.5. A red-black tree keeps keys ordered and the height logarithmic; rotations restore balance after inserts and removes.

NavigableMap in action

`TreeMap` implements `NavigableMap`, the map twin of `NavigableSet`: `firstKey`, `lastKey`, `floorKey/Entry`, `ceilingKey/Entry`, `lowerKey`, `higherKey`, `pollFirstEntry`, `headMap`, `tailMap`, `subMap` and `descendingMap`. The first three examples below are tasks that are awkward to solve any other way.

TreeMapDemo.java JAVA

```java
import java.util.*;

public class TreeMapDemo {
    public static void main(String[] args) {
        // 1. Grade boundaries: find the greatest threshold not above the score
        TreeMap<Integer, String> grades = new TreeMap<>();
        grades.put(0, "F"); grades.put(60, "D"); grades.put(70, "C");
        grades.put(80, "B"); grades.put(90, "A");
        for (int score : new int[] {95, 72, 59, 80})
            System.out.print(score + "=" + grades.floorEntry(score).getValue() + " ");
```

TreeMapDemo.java (continued) JAVA

```
        System.out.println();

        // 2. Range queries (headMap/tailMap are live views)
        System.out.println("headMap(70)        = " + grades.headMap(70));
        System.out.println("tailMap(80, false) = " + grades.tailMap(80, false));
        System.out.println("ceilingKey(61)     = " + grades.ceilingKey(61));

        // 3. Reverse order
        System.out.println("descending keys    = " + grades.descendingMap().keySet());
        System.out.println("firstKey           = " + grades.firstKey());
    }
}
```

Run it TERMINAL

```
$ java TreeMapDemo.java
95=A 72=C 59=F 80=B
headMap(70)        = {0=F, 60=D}
tailMap(80, false) = {90=A}
ceilingKey(61)     = 70
descending keys    = [90, 80, 70, 60, 0]
firstKey           = 0
```

The `floorEntry` trick (a lookup table with ranges, like tax brackets, shipping rates or the highest bid below a price) is the most useful idea here: with a `HashMap` you would loop over all thresholds.

Like `TreeSet`, a `TreeMap` treats two keys as equal when `compareTo` (or your `Comparator`) returns 0, never consulting `equals`. A comparator that is inconsistent with `equals` makes the map violate the general `Map` contract, while still being internally consistent.

6.7.3 EnumMap

An `EnumMap<K extends Enum<K>, V>` stores its values in a plain array indexed by the key's `ordinal()`. `get` and `put` are an array access: no hashing, no collisions, and the smallest footprint of any map. Iteration is in enum declaration order. Use it whenever the key is an enum.

EnumMapDemo.java JAVA

```
import java.util.*;

public class EnumMapDemo {
    enum Level { LOW, MID, HIGH }

    public static void main(String[] args) {
        Map<Level, Integer> alerts = new EnumMap<>(Level.class);
        alerts.put(Level.HIGH, 3);
        alerts.put(Level.LOW, 1);
        System.out.println(alerts);             // ordered by declaration, not insertion
    }
}
```

Run it TERMINAL

```
$ java EnumMapDemo.java
{LOW=1, HIGH=3}
```

6.7.4 IdentityHashMap: reference equality

`IdentityHashMap` compares keys with `==` and hashes with `System.identityHashCode` instead of `equals`/`hashCode`. It deliberately violates the `Map` contract. Two keys that are `equals` but distinct objects are two entries. You almost never want this in business code. It exists for algorithms that must track *objects* rather than values: a deep copy or serializer that records "I have already visited this exact node" to survive cycles, or a proxy and instrumentation framework.

IdentityDemo.java JAVA

```
import java.util.*;

public class IdentityDemo {
    public static void main(String[] args) {
        String s1 = new String("key");
```

IdentityDemo.java (continued) JAVA

```
        String s2 = new String("key");
        Map<String, Integer> id = new IdentityHashMap<>();
        id.put(s1, 1);
        id.put(s2, 2);
        Map<String, Integer> eq = new HashMap<>();
        eq.put(s1, 1);
        eq.put(s2, 2);
        System.out.println("IdentityHashMap size = " + id.size());
        System.out.println("HashMap size         = " + eq.size());
    }
}
```

Run it TERMINAL

```
$ java IdentityDemo.java
IdentityHashMap size = 2
HashMap size         = 1
```

6.7.5 WeakHashMap: entries that disappear

A `WeakHashMap` holds its **keys** through weak references. When a key is no longer strongly reachable from anywhere else, the garbage collector may clear it, and the map silently drops the entry afterwards. It suits caches of metadata keyed by objects whose lifetime you do not control ("extra data about this object, which should vanish when the object does"). Weak, soft and phantom references are explained in the memory chapters of the JVM part.

WeakDemo.java JAVA

```
import java.util.*;

public class WeakDemo {
    public static void main(String[] args) throws Exception {
        WeakHashMap<Object, String> weak = new WeakHashMap<>();
        Object keep = new Object();
        weak.put(keep, "kept alive by a variable");
        weak.put(new Object(), "no other reference");
        System.out.println("before gc: size = " + weak.size());
        System.gc();                        // a request; HotSpot normally honours it
        Thread.sleep(200);
        System.out.println("after gc:  size = " + weak.size());
        System.out.println(keep != null);  // keeps `keep` reachable until here
    }
}
```

Run it (the result depends on the garbage collector acting; this is typical) TERMINAL

```
$ java WeakDemo.java
before gc: size = 2
after gc:  size = 1
true
```

GOTCHA

The *value* is held strongly. If a value refers back to its own key (directly or through a chain), the key never becomes weakly reachable and the entry never goes away. Also, never use `String` literals or small cached boxed values as keys of a `WeakHashMap`: they are always strongly reachable, so the entries never disappear.

6.7.6 ConcurrentHashMap: the thread-safe map

When several threads share a map, `HashMap` can lose updates, return stale data or corrupt itself. The correct default is `java.util.concurrent.ConcurrentHashMap`. The full story belongs to the concurrency part; here is what you should know now.

- **Not one big lock.** (The Java 7 version used segments, the Java 8 rewrite does not.) Reads are lock-free. An insert into an **empty bucket** uses a compare-and-set (CAS). Only when the bucket already has nodes does the writer synchronise on that bucket's first node, so two threads writing to different buckets proceed in parallel. Its bins use the same linked-list-then-tree strategy as `HashMap`.

- **Atomic compound operations.** `putIfAbsent`, `computeIfAbsent`, `compute`, `merge` and `replace(k, old, new)` run atomically for that key. `if (!map.containsKey(k)) map.put(k, v)` is a race even on a concurrent map; `putIfAbsent` is not.
- **No nulls.** Neither null keys nor null values: in a shared map, `get(k) == null` would be ambiguous (absent or mapped to null), and the caller cannot call `containsKey` atomically afterwards.
- **Weakly consistent iterators.** They never throw `ConcurrentModificationException`; they reflect the map at some point at or after the creation of the iterator, and may or may not show later changes.
- `size()` **is an estimate under concurrent change** (it is summed from striped counters, like `LongAdder`). `mappingCount()` returns a `long`.

ConcurrentCounter.java JAVA

```java
import java.util.*;
import java.util.concurrent.*;

public class ConcurrentCounter {
    static void run(Map<String, Integer> map) throws Exception {
        ExecutorService pool = Executors.newFixedThreadPool(4);
        for (int t = 0; t < 4; t++)
            pool.submit(() -> {
                for (int i = 0; i < 10_000; i++) map.merge("hits", 1, Integer::sum);
            });
        pool.shutdown();
        pool.awaitTermination(10, TimeUnit.SECONDS);
    }
    public static void main(String[] args) throws Exception {
        Map<String, Integer> safe = new ConcurrentHashMap<>();
        run(safe);
        System.out.println("ConcurrentHashMap hits = " + safe.get("hits") + " (expected 40000)");
        try {
            new ConcurrentHashMap<String, Integer>().put("k", null);
        } catch (NullPointerException e) {
            System.out.println("null value rejected by ConcurrentHashMap");
        }
        Map<String, Integer> unsafe = new HashMap<>();
        run(unsafe);
        System.out.println("HashMap hits = " + unsafe.get("hits")
            + " (usually wrong; varies by run)");
    }
}
```

Run it (the HashMap number is typical of a broken result and changes between runs) TERMINAL

```
$ java ConcurrentCounter.java
ConcurrentHashMap hits = 40000 (expected 40000)
null value rejected by ConcurrentHashMap
HashMap hits = 10000 (usually wrong; varies by run)
```

Do not "fix" the unsafe counter by wrapping each step in your own check-then-act code; use the atomic methods. When you need a sorted concurrent map, use `ConcurrentSkipListMap`.

6.7.7 Hashtable and Collections.synchronizedMap

`Hashtable` is a Java 1.0 class: every method is `synchronized` on the whole table (one lock for everything), it rejects null keys and values, and it exposes the pre-collections `Enumeration` API. It is kept only for compatibility. Its `Properties` subclass is still common for configuration (see the chapter on properties). In new code use `ConcurrentHashMap` for sharing, or `HashMap` when single-threaded.

`Collections.synchronizedMap(map)` wraps any map with a single mutex. It is safe for individual calls but you must still hold the lock yourself when iterating over its views, and compound operations are not atomic. Prefer `ConcurrentHashMap`.

INTERVIEW ANGLE
How would you implement an LRU cache in Java? Extend `LinkedHashMap` with access order enabled (`super(cap, 0.75f, true)`) and override `removeEldestEntry` to return `size() > capacity`. It is O(1) per operation. For a concurrent cache wrap it in a synchronised wrapper or use Caffeine. If the interviewer wants it "from scratch", combine a `HashMap` from key to node with a doubly linked list; the last chapter of this Part shows that version.

INTERVIEW ANGLE
HashMap vs ConcurrentHashMap vs Hashtable? `HashMap`: no synchronisation, one null key, fastest single-threaded. `Hashtable`: legacy, every method locked, no nulls. `ConcurrentHashMap`: scalable thread-safe map with CAS and per-bucket locking, atomic `merge`/`compute`, no nulls, weakly consistent iterators.

INTERVIEW ANGLE
When would you use TreeMap instead of HashMap? When you need keys in sorted order or nearest-key queries (floor, ceiling, subMap), accepting O(log n) instead of O(1) and the extra memory per entry.

Check yourself

1. How do you make a `LinkedHashMap` iterate in least-recently-used order, and how do you make it evict automatically?
2. Given `TreeMap<Integer,String> m` with keys 10, 20, 30, what do `m.floorKey(25)`, `m.higherKey(30)` and `m.headMap(20).size()` return?
3. What is wrong with `if (!cmap.containsKey(k)) cmap.put(k, new ArrayList<>());` on a `ConcurrentHashMap`, and what replaces it?
4. Why can a `WeakHashMap` entry fail to disappear even though the key is no longer used by your code?
5. Why does `new ConcurrentHashMap<String,String>().put("a", null)` throw?
6. What does `new IdentityHashMap<String,Integer>()` do with `put(new String("a"), 1); put(new String("a"), 2);` and what is the size?

Answers

1. Use the constructor `new LinkedHashMap<>(16, 0.75f, true)` for access order; override `removeEldestEntry` to return `size() > capacity` for automatic eviction.
2. `20`, `null` and `1` (headMap is exclusive of 20, so only key 10).
3. Another thread can insert between the check and the put (a race). Use `cmap.computeIfAbsent(k, x -> new ArrayList<>())`, which is atomic.
4. The value (or something else) still strongly references the key, or the key is a literal or cached object that is always strongly reachable.
5. `ConcurrentHashMap` forbids null keys and values, since `get` returning null would be ambiguous in a concurrent setting.
6. It compares by reference, so the two `new String("a")` objects are different keys: size 2.

KEY POINTS
Key points

- `LinkedHashMap` adds a linked list to `HashMap`: insertion order by default, access order plus `removeEldestEntry` gives an LRU cache.
- `TreeMap` is a red-black tree: sorted keys, O(log n) operations and `NavigableMap` queries (`floor`, `ceiling`, `headMap`, `subMap`); equality is by comparator.
- Use `EnumMap` for enum keys, `IdentityHashMap` for reference semantics, `WeakHashMap` for entries tied to key lifetime.
- For shared maps use `ConcurrentHashMap` with its atomic `merge`/`compute`/`putIfAbsent`; `Hashtable` is legacy.

6.8 HashMap in depth

`HashMap` is the most used class in the Java standard library after `String` and `ArrayList`, and the most asked-about in interviews. Almost every backend program uses one: caches, indexes by id, counters, grouping. Using it is easy. Knowing why `get` is "O(1)", when it stops being O(1), and which mistakes make it silently wrong is what separates a Java user from a Java engineer. This chapter opens the class up, based on the OpenJDK implementation (the structure has been stable since Java 8, and I verified the numbers below by running experiments on JDK 21).

6.8.1 The problem and the core idea

You want to find a value by key without scanning everything. An array gives you instant access by an integer index, so the idea is: turn the key into an integer, and use that integer to pick a slot in an array. The array is called the **table**; each slot is a **bucket**. The conversion function is `hashCode()`.

Two different keys can map to the same bucket (a **collision**, unavoidable: there are far more possible keys than slots). So a bucket holds not one entry but a small collection of entries, and a lookup is: compute the bucket, then search the few entries inside it using `equals`. If the buckets stay short, the lookup costs a constant number of steps regardless of how many entries the map holds. All the engineering in `HashMap` is about keeping buckets short.

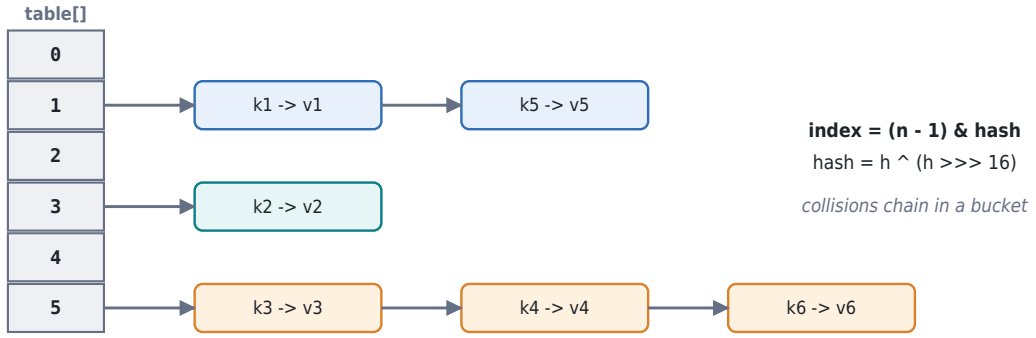


Figure 6.6. Keys are hashed to a bucket index; colliding keys share a bucket as a linked chain of nodes.

6.8.2 From key to bucket: hash spreading and the index

There are two steps, and both are worth knowing exactly.

Step 1, the spread. `HashMap` takes the key's `hashCode()` and mixes the high 16 bits into the low 16 bits:

Spread.java (from the JDK source, HashMap.hash) JAVA

```
static final int hash(Object key) {
    int h;
    return (key == null) ? 0 : (h = key.hashCode()) ^ (h >>> 16);
}
```

A `null` key therefore hashes to 0 and lives in bucket 0 (which is why `HashMap` allows one `null` key). The spread exists because of step 2.

Step 2, the index. The table length is always a power of two, so instead of the slow `hash % length` the map computes `(length - 1) & hash`. With length 16 the mask is binary `1111`, so only the lowest 4 bits of the hash choose the bucket. Without the spread, any two hash codes that differ only in their upper bits would always collide in small tables. A concrete case: the integers 65536, 131072 and 196608 differ only in bits 16 and above; in a 16-slot table all three would land in bucket 0. After `h ^ (h >>> 16)` they land in buckets 1, 2 and 3. The spread is a cheap way (one shift, one XOR) to let every bit of the hash influence the index.

Power-of-two sizing also gives the resize a very nice property, which we reach below.

HashInternals.java JAVA

```
import java.lang.reflect.*;
import java.util.*;

public class HashInternals {
    static int spread(Object key) {
        int h = key.hashCode();
        return h ^ (h >>> 16);
    }
    static Object[] table(HashMap<?, ?> m) throws Exception {
        Field f = HashMap.class.getDeclaredField("table");
        f.setAccessible(true);                      // needs --add-opens java.base/java.util
        return (Object[]) f.get(m);
    }
    static int threshold(HashMap<?, ?> m) throws Exception {
        Field f = HashMap.class.getDeclaredField("threshold");
        f.setAccessible(true);
        return f.getInt(m);
    }
```

HashInternals.java (continued) JAVA

```
    public static void main(String[] args) throws Exception {
        for (String k : new String[] {"apple", "banana", "Aa", "BB"}) {
            int h = k.hashCode();
            int s = spread(k);
            System.out.printf("%-7s hashCode=%11d spread=%11d index(16)=%2d index(no spread)=%2d%n",
                k, h, s, s & 15, h & 15);
        }
        HashMap<Integer, String> m = new HashMap<>();
        System.out.println("new: table=" + table(m) + " threshold=" + threshold(m));
        int last = -1;
        for (int i = 1; i <= 100; i++) {
            m.put(i, "v");
            int len = table(m).length;
            if (len != last) {
                System.out.println("size=" + i + " table=" + len + " threshold=" + threshold(m));
                last = len;
            }
        }
        HashMap<String, Integer> sized = new HashMap<>(100);
        sized.put("a", 1);
        System.out.println("new HashMap(100): table=" + table(sized).length);
        HashMap<String, Integer> sized2 = HashMap.newHashMap(100);
        sized2.put("a", 1);
        System.out.println("HashMap.newHashMap(100): table=" + table(sized2).length);
    }
}
```

Run it (--add-opens lets reflection read the private fields of HashMap) TERMINAL

```
$ java --add-opens java.base/java.util=ALL-UNNAMED HashInternals.java
apple   hashCode=   93029210 spread=   93030097 index(16)= 1 index(no spread)=10
banana  hashCode=-1396355227 spread=-1396317280 index(16)= 0 index(no spread)= 5
Aa      hashCode=       2112 spread=       2112 index(16)= 0 index(no spread)= 0
BB      hashCode=       2112 spread=       2112 index(16)= 0 index(no spread)= 0
new: table=null threshold=0
size=1 table=16 threshold=12
size=13 table=32 threshold=24
size=25 table=64 threshold=48
size=49 table=128 threshold=96
size=97 table=256 threshold=192
new HashMap(100): table=128
HashMap.newHashMap(100): table=256
```

Read the output carefully, it explains a lot.

- `Aa` and `BB` have the same `hashCode` (2112). This is a famous coincidence of Java's string hash, not a bug; it is a handy source of deliberate collisions for experiments.
- A new `HashMap` has **no table at all** (`table=null`). The table is allocated lazily, on the first `put`, with the default capacity 16.
- The **threshold** is `capacity × load factor`: 16 × 0.75 = 12. When the entry count exceeds it, the table doubles.

6.8.3 Buckets, collisions and the node

Each bucket starts as `null`. The first entry stored in a bucket is a `HashMap.Node`, which holds exactly four fields: the cached `hash`, the `key`, the `value` and a `next` reference. With compressed pointers (the default on 64-bit JVMs for heaps under about 32 GB) that is a 12-byte object header plus four 4-byte fields: 28 bytes, padded to 32. That per-entry cost is one reason a `HashMap<Integer,Integer>` is much heavier than the numbers it stores (see the chapter on choosing data structures).

`put(key, value)` does the following.

1. If the table is `null`, allocate it (resize to the default 16).
2. Compute `i = (n - 1) & hash(key)`. If bucket `i` is empty, put a new node there. Done.
3. Otherwise walk the bucket. For each node compare the stored hash first, then the key: `node.hash == hash && (node.key == key || key.equals(node.key))`. Comparing the cached hash first is a cheap filter that avoids most `equals` calls.
4. If an equal key is found, replace its value (and return the old value). Otherwise append a new node at the **tail** of the chain.

5. If the number of entries now exceeds the threshold, resize.

`get` is steps 2 and 3 only. Notice what this means for you: `get` calls your key's `hashCode()` once and `equals()` on the keys in one bucket. If `hashCode` is slow or `equals` is wrong, `HashMap` inherits the problem.

6.8.4 Resizing and rehashing

When `size > threshold` the table doubles (16, 32, 64, ...) and the threshold doubles with it. Every entry must be moved, because its index depends on the table length. Here the power-of-two choice pays off. Going from length 16 to 32 adds one more bit to the mask. For each entry that bit of its hash is either 0 or 1, so an entry in old bucket `j` goes to either new bucket `j` or new bucket `j + 16`, nothing else. The JDK splits every bucket into a "low" list and a "high" list in a single pass, preserving the relative order, without recomputing any hash and without reallocating nodes. That is why the resize loop is simple and why entries from one bucket never mix with others.

Resizing is still O(n) work and allocates a new array, so it is the main source of latency spikes when a map grows. If you know roughly how many entries you will add, size the map up front. Beware the trap in the experiment above: `new HashMap<>(100)` does **not** hold 100 entries without resizing. The constructor rounds 100 up to the next power of two, 128, and the threshold is only 96. Since Java 19 there is a factory that does the arithmetic for you: `HashMap.newHashMap(100)` computes `ceil(100 / 0.75)`, then rounds up to 256 here, and guarantees no resize for 100 entries. On older JDKs write `new HashMap<>((int) (n / 0.75f) + 1)`.

The load factor

The default load factor of 0.75 is a compromise stated in the Javadoc: it balances time and space cost. A lower value (say 0.5) means shorter buckets and faster lookups but a table twice as big; a higher value (say 1.0 or more) saves memory but buckets get longer. With a decent hash function and load factor 0.75 the average bucket holds well under one entry, and chains of 8 or more are astronomically unlikely by chance (the JDK source comment quotes a Poisson distribution with parameter 0.5, giving a chance of about 6 in 100 million for 8 entries). In practice you leave the load factor alone. Tuning it is almost never the right fix.

6.8.5 Treeification: Java 8's defence against bad hashes

Before Java 8, a bucket was always a linked list. Lookup in a long chain is O(n), and a bad `hashCode` (or a deliberate attack: crafting thousands of strings with equal hashes to hit a web server's parameter map) could turn a map into a slow linked list. Java 8 added the **tree bin**: when a bucket grows long, its linked list is converted into a red-black tree (the same balanced tree that `TreeMap` uses), so a single worst-case bucket costs O(log n) instead of O(n).

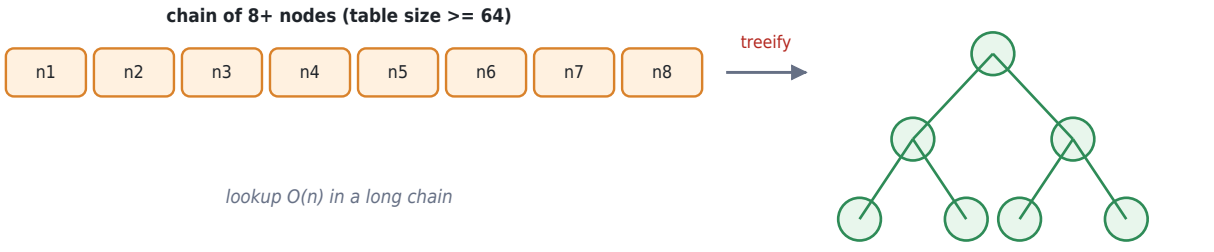


Figure 6.7. A bucket chain converting into a red-black tree when it passes the threshold, and back when it shrinks.

The constants, from the source:

Constant	Value	Meaning
`DEFAULT_INITIAL_CAPACITY`	16	Table size allocated on first put
`DEFAULT_LOAD_FACTOR`	0.75	Resize when size exceeds capacity x 0.75
`TREEIFY_THRESHOLD`	8	A chain this long may become a tree
`UNTREEIFY_THRESHOLD`	6	A tree bin that shrinks to this size is turned back into a list
`MIN_TREEIFY_CAPACITY`	64	Trees only if the table has at least this many buckets

HashMap constants in OpenJDK.

There is a subtlety that most explanations get wrong. Treeification is attempted when a bucket that already holds 8 nodes receives a 9th (the check in `putVal` is on the count of nodes traversed). And then `treeifyBin` first looks at the

table length: if the table has fewer than 64 buckets, it **resizes instead of treeifying**, on the theory that a small table with a long chain simply needs more buckets. Only when the table is at least 64 does the bin become a tree.

You can watch this happen. The next program uses keys whose `hashCode` is the constant 42, so every key goes into the same bucket, and peeks at the table with reflection.

BadHash.java JAVA

```java
import java.lang.reflect.*;
import java.util.*;

public class BadHash {
    // Constant hash, NOT Comparable: worst case
    static final class BadKey {
        final int id;
        BadKey(int id) { this.id = id; }
        @Override public int hashCode() { return 42; }
        @Override public boolean equals(Object o) { return o instanceof BadKey b && b.id == id; }
    }
    // Constant hash but Comparable: tree bins can still binary-search
    static final class BadComparableKey implements Comparable<BadComparableKey> {
        final int id;
        BadComparableKey(int id) { this.id = id; }
        @Override public int hashCode() { return 42; }
        @Override public boolean equals(Object o) {
            return o instanceof BadComparableKey b && b.id == id;
        }
        @Override public int compareTo(BadComparableKey o) { return Integer.compare(id, o.id); }
    }
    static String describe(HashMap<?, ?> m) throws Exception {
        Field f = HashMap.class.getDeclaredField("table");
        f.setAccessible(true);
        Object[] t = (Object[]) f.get(m);
        for (Object n : t)
            if (n != null)
                return "table=" + t.length + ", the used bin holds a "
                    + n.getClass().getSimpleName();
        return "empty";
    }
    public static void main(String[] args) throws Exception {
        HashMap<BadComparableKey, Integer> m = new HashMap<>();
        for (int i = 1; i <= 12; i++) {
            m.put(new BadComparableKey(i), i);
            if (i == 1 || i >= 8) System.out.println("after " + i + " keys: " + describe(m));
        }
        int n = 20_000;
        for (int round = 0; round < 3; round++) {
            long t0 = System.nanoTime();
            HashMap<BadKey, Integer> a = new HashMap<>();
            for (int i = 0; i < n; i++) a.put(new BadKey(i), i);
            long t1 = System.nanoTime();
            HashMap<BadComparableKey, Integer> b = new HashMap<>();
            for (int i = 0; i < n; i++) b.put(new BadComparableKey(i), i);
            long t2 = System.nanoTime();
            HashMap<Integer, Integer> c = new HashMap<>();
            for (int i = 0; i < n; i++) c.put(i, i);
            long t3 = System.nanoTime();
            System.out.printf("round %d: non-Comparable %5d ms | Comparable %4d ms | good %3d ms%n",
                round, (t1 - t0) / 1_000_000, (t2 - t1) / 1_000_000, (t3 - t2) / 1_000_000);
        }
    }
}
```

Run it (the timings are typical on the build machine; this program takes about a minute and a half) TERMINAL

```
$ java --add-opens java.base/java.util=ALL-UNNAMED BadHash.java
after 1 keys: table=16, the used bin holds a Node
after 8 keys: table=16, the used bin holds a Node
after 9 keys: table=32, the used bin holds a Node
after 10 keys: table=64, the used bin holds a Node
after 11 keys: table=64, the used bin holds a TreeNode
after 12 keys: table=64, the used bin holds a TreeNode
round 0: non-Comparable 21528 ms | Comparable  815 ms | good  41 ms
round 1: non-Comparable 14125 ms | Comparable   51 ms | good  17 ms
round 2: non-Comparable 15251 ms | Comparable   21 ms | good   0 ms
```

What to notice:

- The 9th key into the same bucket did not build a tree; it **doubled the table** from 16 to 32. So did the 10th (to 64). Only the 11th key finally found a table of length 64 and converted the bin into a `TreeNode`.
- Inserting 20,000 keys with a constant hash and no ordering took between 14 and 22 **seconds**; the good keys took milliseconds. A single bad `hashCode` is the difference between a fast service and an outage.
- The third column is the surprise: the same constant hash, but the key class implements `Comparable`, and 20,000 inserts take well under a second. A tree bin orders nodes by hash first and, when hashes are equal, by `compareTo` (if the keys share a class and are `Comparable`). Then it can binary-search, O(log n). If the keys are not `Comparable`, the tree cannot order the colliding nodes; a lookup has to search both subtrees, and the tree degrades to O(n) with a worse constant than the plain list. Treeification is a safety net with a hole: **make keys `Comparable` when you cannot guarantee a good hash**, and fix the hash regardless.

When removals or a resize shrink a tree bin to 6 nodes or fewer, it is converted back into a plain linked list (the "untreeify" threshold of 6; the gap between 8 and 6 stops a bin from flipping back and forth at the boundary).

6.8.6 Writing a good hashCode

The `HashMap` contract rests on one rule from `Object`: **equal objects must have equal hash codes.** The reverse need not hold, but the closer hashes of unequal objects are to uniformly distributed, the better. Practical guidance:

- Override `equals` and `hashCode` together, using the same fields.
- Use `Objects.hash(a, b, c)` or, for new code, a `record`, which generates both from all components. `Objects.hash` boxes primitives and allocates an array, so in a very hot path write the arithmetic by hand: `31 * a + b`.
- Never use mutable fields, and never use fields that differ between equal objects (e.g. a timestamp).
- Remember the standard hash codes: `Integer` is its value; `Long` is `(int)(v ^ (v >>> 32))`; `String` is `s[0]*31^(n-1) + ... + s[n-1]` (cached in the string after the first call); `List.hashCode` combines elements the same way as `Objects.hash`. For example `Objects.hash(1, 2, 3)` and `List.of(1, 2, 3).hashCode()` are both 30817.

6.8.7 The map API that every engineer should use

Older Java code does `if (map.containsKey(k)) { map.put(k, map.get(k) + 1); } else { map.put(k, 1); }`. That hashes the key three times and is not atomic in a concurrent map. Java 8 added single-call methods that read better and do the right thing.

Method	Behaviour
`getOrDefault(k, d)`	value for k, or d if k is absent (a key mapped to null returns null, not d)
`putIfAbsent(k, v)`	put only if k is absent or mapped to null; returns the previous value
`computeIfAbsent(k, f)`	if absent, compute the value with f(k) and store it; the idiom for multi-maps
`computeIfPresent(k, f)`	if present, recompute; returning null from f removes the entry
`compute(k, f)`	recompute whether or not present
`merge(k, v, f)`	if absent store v, else store f(old, v); the idiom for counters
`replaceAll(f)`, `forEach`, `removeIf` on views	bulk operations

Main HashMap convenience methods (Java 8+).

MapApi.java JAVA

```java
import java.util.*;

public class MapApi {
    public static void main(String[] args) {
        String text = "to be or not to be that is the question to be";

        Map<String, Integer> counts = new HashMap<>();
        for (String w : text.split(" ")) counts.merge(w, 1, Integer::sum);
        System.out.println("merge counts: " + counts);

        Map<Character, List<String>> byInitial = new TreeMap<>();
        for (String w : text.split(" "))
            byInitial.computeIfAbsent(w.charAt(0), c -> new ArrayList<>()).add(w);
        System.out.println("computeIfAbsent: " + byInitial);
```

MapApi.java (continued) JAVA

```
        Map<String, Integer> m = new HashMap<>();
        m.put(null, 1);
        m.put("a", null);
        System.out.println("null key -> " + m.get(null) + ", containsKey(a)=" + m.containsKey("a"));
        System.out.println("getOrDefault(a, 9)=" + m.getOrDefault("a", 9)
            + " getOrDefault(zz, 9)=" + m.getOrDefault("zz", 9));
        m.putIfAbsent("a", 5);
        System.out.println("putIfAbsent over a null value: a=" + m.get("a"));

        Map<String, Integer> c = new HashMap<>(Map.of("x", 1));
        c.computeIfPresent("x", (k, v) -> null);
        System.out.println("computeIfPresent returning null removes the entry: " + c);
    }
}
```

Run it TERMINAL

```
$ java MapApi.java
merge counts: {the=1, that=1, not=1, or=1, be=3, question=1, is=1, to=3}
computeIfAbsent: {b=[be, be, be], i=[is], n=[not], o=[or], q=[question], t=[to, to, that, the, to]}
null key -> 1, containsKey(a)=true
getOrDefault(a, 9)=null getOrDefault(zz, 9)=9
putIfAbsent over a null value: a=5
computeIfPresent returning null removes the entry: {}
```

Notice `getOrDefault("a", 9)` printed `null`: the key is present (mapped to `null`), so the default is not used. `containsKey` is the way to tell "absent" from "mapped to null". Prefer maps without null values and the confusion disappears.

WATCH OUT

Do not mutate the map from inside `computeIfAbsent`, `compute` or `merge` (for example, a recursive memoised Fibonacci calling `computeIfAbsent` from within its own mapping function). `HashMap` detects it and throws `ConcurrentModificationException` since Java 9.

6.8.8 Iteration order and fail-fast iterators

Iterating a `HashMap` visits buckets from index 0 to the end, and each chain from head to tail. The order depends on key hashes and the table length, so it can change when the map resizes. Look at integer keys inserted as 33, 17, 1, 49, 16, 2 into a map with 16 buckets: 16 goes to bucket 0; 33, 17, 1 and 49 all share bucket 1 (each is 1 modulo 16) and stay in insertion order within it; 2 goes to bucket 2. The key set prints `[16, 33, 17, 1, 49, 2]`, which is neither insertion nor sorted order. If your program prints a `HashMap` and a test compares the text, you have a flaky test. Use `LinkedHashMap` or `TreeMap` when order matters.

Iterators of `HashMap` are **fail-fast**: the map keeps a `modCount` that every structural change (add or remove an entry, not replacing a value) increments. The iterator remembers the count when it was created and throws `ConcurrentModificationException` on the next step if it changed. This is a best-effort bug detector, not a thread-safety mechanism. To remove entries while iterating use `iterator.remove()` or the bulk forms `map.entrySet().removeIf(...)` and `map.values().removeIf(...)`.

6.8.9 The key-mutation bug, again, for maps

The earlier chapter on sets showed that mutating an element makes it unfindable. For maps it is the same with keys, and it is a common source of "the data is in the map but `get` returns `null`" bugs.

KeyMutation.java JAVA

```
import java.util.*;

public class KeyMutation {
    static class Key {
        int id;
        Key(int id) { this.id = id; }
        @Override public boolean equals(Object o) { return o instanceof Key k && k.id == id; }
        @Override public int hashCode() { return Integer.hashCode(id); }
    }
    public static void main(String[] args) {
        Map<Key, String> map = new HashMap<>();
```

KeyMutation.java (continued) JAVA

```
        Key k = new Key(1);
        map.put(k, "one");
        k.id = 2;                                   // key mutated after insertion
        System.out.println("get(k)          = " + map.get(k));
        System.out.println("get(new Key(1)) = " + map.get(new Key(1)));
        System.out.println("size            = " + map.size());
    }
}
```

Run it TERMINAL

```
$ java KeyMutation.java
get(k)          = null
get(new Key(1)) = null
size            = 1
```

`get(k)` looks in the bucket for id 2 (empty); `get(new Key(1))` finds the right bucket but the stored key now has id 2, so `equals` fails. The entry is stuck in the map forever. Fix: make key classes immutable (`final` fields, or a `record`).

6.8.10 What changed in Java 8, and what to say in an interview

Aspect	Java 7 and earlier	Java 8 and later
Bucket structure	linked list only	linked list, converted to red-black tree when long
Insertion into a chain	at the head	at the tail
Resize of a bucket	rehash every node; order of a chain gets reversed	split into low/high lists, order preserved
Hash spreading	several shifts and XORs	single h ^ (h >>> 16)
Worst-case lookup	O(n)	O(log n) if keys are Comparable (else O(n))
Concurrent misuse	could create a cyclic chain and spin forever in get	no infinite loop, but still data loss and wrong sizes

Java 7 versus Java 8 and later HashMap.

The last row deserves a line: `HashMap` is **not thread-safe**. In Java 7, two threads resizing at once could link two nodes into a cycle, after which `get` looped forever at 100 percent CPU. Java 8's tail-preserving resize removed that specific hang, but concurrent writers can still lose updates, corrupt the size or throw exceptions. If several threads write, use `ConcurrentHashMap` (next chapter), not `HashMap` and not a lock around `HashMap` unless you need compound operations across several keys.

REAL WORLD
Spring and most Java frameworks build request headers, JSON object fields, bean registries and caches on `HashMap`, `LinkedHashMap` or `ConcurrentHashMap`. When a service shows high CPU in `HashMap.get` or `HashMap$TreeNode.find` in a thread dump or profiler, the usual causes are a poor `hashCode` (all keys land in one bucket), a mutable key, or a map shared by several threads without synchronisation.

INTERVIEW ANGLE
Explain how HashMap works internally. It is an array of buckets whose length is a power of two. `put` takes `key.hashCode()`, XORs its high 16 bits into the low 16, masks with `length - 1` to get the bucket, and walks the bucket comparing cached hashes and then `equals`. A new entry goes to the end of the chain. When the entry count exceeds capacity times load factor (default 16 and 0.75), the table doubles and entries are redistributed, each staying in its index or moving by the old length. A chain of 8 or more entries in a table of at least 64 buckets is converted to a red-black tree (and back at 6), making the worst case O(log n). Average `get` and `put` are O(1).

INTERVIEW ANGLE
What happens if two different keys have the same hashCode? What if they are also equal? Same hash and not `equals`: they are stored in the same bucket as separate nodes (a collision), found by `equals`. Equal keys: the second `put` replaces the first one's value, because the map treats them as the same key.

Check yourself

1. What are the default capacity and load factor of a `HashMap`, and when is the table created?
2. `new HashMap<>(100)` followed by 100 puts: does the map resize? What about `HashMap.newHashMap(100)`?
3. A key class overrides `equals` but not `hashCode`. What goes wrong with `map.put(new K(1), "a"); map.get(new K(1))`?
4. What is wrong with this loop? `for (String k : map.keySet()) if (k.startsWith("tmp")) map.remove(k);` How do you fix it?
5. Why is the table length always a power of two? Give two reasons.
6. What does this print? `Map<String,Integer> m = new HashMap<>(); m.put("a", null); System.out.println(m.getOrDefault("a", 9));`

Answers

1. 16 and 0.75. The table is allocated lazily on the first `put`.
2. `new HashMap<>(100)` rounds to capacity 128 with threshold 96, so the 97th put resizes. `HashMap.newHashMap(100)` (Java 19+) gives a table of 256 and does not resize.
3. The two keys have different identity-based hash codes, so they usually land in different buckets and `get` returns `null`, even though `equals` says they are the same.
4. Removing from the map while iterating its key set throws `ConcurrentModificationException`. Use `map.keySet().removeIf(k -> k.startsWith("tmp"))` or an explicit `Iterator.remove()`.
5. Index computation is a fast mask `(n - 1) & hash` instead of a division, and resizing splits every bucket into exactly two destinations (same index or index plus old length) without rehashing.
6. `null`: the key is present, so the default is not used.

KEY POINTS

Key points

- A HashMap is an array of buckets; the bucket is `(n - 1) & (h ^ (h >>> 16))`, entries in one bucket are found by cached hash and `equals`.
- The table is allocated on the first put, doubles when size exceeds capacity times 0.75, and splits each bucket in two on resize; presize with `HashMap.newHashMap(n)`.
- Long chains become red-black trees (threshold 8, table at least 64, back at 6), but only `Comparable` keys keep the O(log n) guarantee; a bad `hashCode` can still make a map hundreds of times slower.
- Keys must be immutable with consistent `equals` and `hashCode`; use `merge`, `computeIfAbsent` and `getOrDefault` instead of check-then-put.
- `HashMap` is not thread-safe, its iteration order is unspecified and its iterators are fail-fast.

6.9 Queues and deques

A **queue** is a collection built around *processing order*: things go in at one end and come out at the other, and the interesting question is which element comes out next. Print jobs, web requests waiting for a worker thread, tasks in a scheduler, nodes to visit in a breadth-first search: all are queues. Java gives you three families, and choosing between them is mostly a question of "what order do I want out?".

6.9.1 Queue and Deque: the interfaces

`java.util.Queue<E>` extends `Collection<E>` with six methods, in two flavours. Each operation has a version that throws an exception on failure and a version that returns a special value:

Operation	Throws exception	Returns special value
Insert at tail	add(e)	offer(e) (false if full)
Remove head	remove()	poll() (null if empty)
Examine head	element()	peek() (null if empty)

The two flavours of every Queue operation.

Prefer the `offer`/`poll`/`peek` family when an empty or full queue is a normal situation. Use `remove`/`element` when emptiness would be a bug and you want it to fail loudly. For unbounded queues `add` and `offer` behave the same, because they never fail for lack of space.

`java.util.Deque<E>` (pronounced "deck", a *double-ended queue*) extends `Queue` and lets you insert, examine and remove at **both** ends: `addFirst`/`addLast`, `offerFirst`/`offerLast`, `pollFirst`/`pollLast`, `peekFirst`/`peekLast`, `removeFirst`/`removeLast`, `getFirst`/`getLast`. Since Java 21 a `Deque` is also a `SequencedCollection`, so `reversed()` works on it. A deque can act as a FIFO queue (first in, first out) or a LIFO stack (last in, first out). It also has stack-flavoured names: `push(e)` is `addFirst(e)`, `pop()` is `removeFirst()`, and `peek()` looks at the first element.

6.9.2 ArrayDeque: the default choice

`ArrayDeque` is a resizable circular array with a head index and a tail index. Adding at either end moves an index (wrapping around the end of the array); when the array is full it doubles. There are no node objects, so it is compact and cache-friendly, and every end operation is amortized O(1). It is the right default for both a plain queue and a stack.

QueueBasics.java JAVA

```java
import java.util.ArrayDeque;
import java.util.Deque;
import java.util.LinkedList;
import java.util.NoSuchElementException;
import java.util.Queue;

public class QueueBasics {
    public static void main(String[] args) {
        Queue<String> line = new ArrayDeque<>();
        line.offer("Asha");
        line.offer("Bilal");
        line.offer("Chen");
        System.out.println("peek  = " + line.peek());
        System.out.println("poll  = " + line.poll());
        System.out.println("queue = " + line);

        Queue<String> empty = new ArrayDeque<>();
        System.out.println("poll on empty  = " + empty.poll());
        System.out.println("peek on empty  = " + empty.peek());
        try {
            empty.remove();
        } catch (NoSuchElementException e) {
            System.out.println("remove on empty -> NoSuchElementException");
        }
        try {
            empty.element();
        } catch (NoSuchElementException e) {
            System.out.println("element on empty -> NoSuchElementException");
        }

        Deque<Integer> stack = new ArrayDeque<>();
        stack.push(1);
        stack.push(2);
        stack.push(3);
        System.out.println("stack = " + stack + ", pop = " + stack.pop());

        try {
            new ArrayDeque<String>().add(null);
        } catch (NullPointerException e) {
            System.out.println("ArrayDeque rejects null");
        }
        LinkedList<String> ll = new LinkedList<>();
        ll.add(null);
        System.out.println("LinkedList accepts null: " + ll);
    }
}
```

Run it TERMINAL

```
$ java QueueBasics.java
peek  = Asha
poll  = Asha
queue = [Bilal, Chen]
poll on empty  = null
peek on empty  = null
remove on empty -> NoSuchElementException
element on empty -> NoSuchElementException
stack = [3, 2, 1], pop = 3
ArrayDeque rejects null
LinkedList accepts null: [null]
```

What to notice

- `poll()` and `peek()` on an empty queue return `null`; `remove()` and `element()` throw. This is the single most-asked queue detail.
- `ArrayDeque` rejects `null` elements because `null` is the "empty" signal of `poll`/`peek`. `LinkedList` allows `null`, which makes `poll() == null` ambiguous. That is one more reason to avoid it.
- After three `push` calls the stack prints as `[3, 2, 1]`: the top of the stack is the *first* element of the deque.

WATCH OUT

Do not use `java.util.Stack` for a stack. `Stack` extends `Vector`, so every method is synchronized, it exposes `get(int)` and `add(int, e)` that break the stack idea, and its `push` adds at the *end*, the opposite of `Deque`. Use `Deque<Integer> stack = new ArrayDeque<>();` instead. The `Stack` class remains only for legacy code.

6.9.3 LinkedList as a queue

`LinkedList` implements `Deque` too, with a doubly linked list of node objects. It works, but each element costs a separate node (about 24 bytes) on a typical 64-bit JVM with compressed references, allocation pressure is higher and iteration jumps around memory. It earns its place only when you need `null` elements or to remove through an `Iterator` in the middle of a long list. For queue and stack work, `ArrayDeque` is almost always faster and smaller.

6.9.4 PriorityQueue: the heap

A `PriorityQueue` does not return elements in arrival order; it always returns the **smallest** according to natural ordering (`Comparable`) or a `Comparator` you pass in. It is a *binary min-heap* stored in an array: a complete binary tree in which every parent is less than or equal to its children. The smallest element is therefore always at index 0.

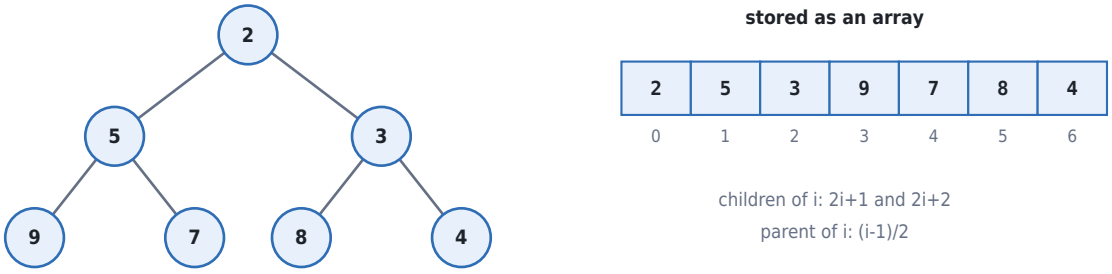


Figure 6.8. A binary min-heap and its array layout. The children of the node at index i are at 2i+1 and 2i+2, so no pointers are needed. offer sifts the new element up; poll moves the last element to the root and sifts it down.

The index arithmetic is what makes the heap fast and compact. For a node at index `i`, the parent is at `(i - 1) / 2` and the children at `2i + 1` and `2i + 2`.

- **offer(e)**: put `e` at the next free array slot (the bottom of the tree), then *sift up*: while it is smaller than its parent, swap them. At most about log2(n) swaps, so O(log n).
- **poll()**: take the root, move the *last* element into the root slot, then *sift down*: swap it with the smaller child until neither child is smaller. O(log n).
- **peek()**: return index 0. O(1).
- **remove(Object)** and **contains**: a linear scan, O(n). A heap is not searchable.

- Building from a collection or array with the constructor uses a bottom-up heapify in O(n), cheaper than n separate offers.

PqDemo.java JAVA

```java
import java.util.Collections;
import java.util.Comparator;
import java.util.List;
import java.util.PriorityQueue;

public class PqDemo {
    record Task(String name, int priority) { }

    public static void main(String[] args) {
        PriorityQueue<Integer> pq = new PriorityQueue<>();
        for (int n : new int[] {50, 20, 40, 10, 30, 5}) {
            pq.offer(n);
        }
        System.out.println("toString (heap array order): " + pq);
        System.out.print("poll order:");
        while (!pq.isEmpty()) {
            System.out.print(" " + pq.poll());
        }
        System.out.println();

        PriorityQueue<Task> tasks =
            new PriorityQueue<>(Comparator.comparingInt(Task::priority).reversed());
        tasks.add(new Task("email", 2));
        tasks.add(new Task("deploy", 9));
        tasks.add(new Task("lunch", 5));
        System.out.println("most urgent: " + tasks.poll());

        List<Integer> data = List.of(7, 3, 99, 41, 8, 56, 12, 77, 1, 64);
        int k = 3;
        PriorityQueue<Integer> minHeap = new PriorityQueue<>();
        for (int x : data) {
            minHeap.offer(x);
            if (minHeap.size() > k) {
                minHeap.poll();
            }
        }
        List<Integer> top = new java.util.ArrayList<>(minHeap);
        Collections.sort(top, Collections.reverseOrder());
        System.out.println("top " + k + " = " + top);
    }
}
```

Run it TERMINAL

```
$ java PqDemo.java
toString (heap array order): [5, 20, 10, 50, 30, 40]
poll order: 5 10 20 30 40 50
most urgent: Task[name=deploy, priority=9]
top 3 = [99, 77, 64]
```

What to notice

- `toString()` and the iterator show the **array order** of the heap, which is only partially sorted: `[5, 20, 10, 50, 30, 40]`. Only the root is guaranteed to be the minimum. The `poll` loop, by contrast, yields fully sorted output. Never iterate a `PriorityQueue` and expect sorted order.
- For "highest first", pass a reversed comparator. A `PriorityQueue` always puts the *smallest by the comparator* at the head.
- The top-k pattern keeps a min-heap of size k: after each offer, if the size exceeds k, poll the smallest. At the end the heap holds the k largest. That costs O(n log k) time and O(k) memory, versus sorting everything in O(n log n).

GOTCHA

Equal priorities are not served in insertion order. `PriorityQueue` is not stable. If two tasks have the same priority, either may come out first. If you need first-come order among equals, add a sequence number to the comparator: `Comparator.comparingInt(Task::priority).thenComparingLong(Task::seq)`.

Other details: a `PriorityQueue` is unbounded (it grows like an `ArrayList`), rejects `null`, is **not thread-safe** (use `PriorityBlockingQueue` across threads), and elements must be mutually comparable or you get a `ClassCastException` at the first `offer`. Mutating an element's priority while it is in the queue silently breaks the heap invariant, the same trap as mutating a `HashMap` key.

6.9.5 A preview: blocking queues

When threads share a queue, polling in a loop wastes CPU. `java.util.concurrent.BlockingQueue` adds `put(e)`, which waits while the queue is full, and `take()`, which waits while it is empty. `ArrayBlockingQueue` is bounded (a fixed array), `LinkedBlockingQueue` is optionally bounded, `PriorityBlockingQueue` orders by priority, and `SynchronousQueue` has no capacity at all (each `put` waits for a `take`). Bounded queues give you **back-pressure**: a fast producer is slowed down instead of exhausting memory.

Bq.java JAVA

```java
import java.util.concurrent.ArrayBlockingQueue;
import java.util.concurrent.BlockingQueue;

public class Bq {
    public static void main(String[] args) throws Exception {
        BlockingQueue<Integer> q = new ArrayBlockingQueue<>(2);
        Thread producer = new Thread(() -> {
            try {
                for (int i = 1; i <= 5; i++) {
                    q.put(i);
                }
                q.put(-1);
            } catch (InterruptedException e) {
                Thread.currentThread().interrupt();
            }
        });
        producer.start();
        int sum = 0;
        while (true) {
            int v = q.take();
            if (v < 0) {
                break;
            }
            sum += v;
        }
        System.out.println("sum = " + sum);
    }
}
```

Run it TERMINAL

```
$ java Bq.java
sum = 15
```

The producer blocks on `put` whenever two items are waiting, and the consumer blocks on `take` when none are. The special value `-1` is a "poison pill" telling the consumer to stop. The full treatment (thread pools, `ExecutorService` work queues, producer-consumer designs) belongs to the concurrency Part.

INTERVIEW ANGLE
How does a PriorityQueue work, and what is the complexity of its operations? It is a binary min-heap stored in an array. `offer` and `poll` are O(log n) because of sift-up and sift-down, `peek` is O(1), `remove(Object)` and `contains` are O(n). Iteration order is not sorted. It is unbounded, not thread-safe and not stable for equal priorities.

REAL WORLD
In services, queues sit between components: a bounded `BlockingQueue` inside a thread pool, a message broker (Kafka, RabbitMQ) between services, a `PriorityQueue` in a scheduler or a "k nearest" search. Spring's `@Async` and `ThreadPoolTaskExecutor` have a `queueCapacity` setting that is exactly the bound of such a queue.

Check yourself

1. What is the difference between `queue.remove()` and `queue.poll()` on an empty queue?
2. What does this print? `PriorityQueue<Integer> q = new PriorityQueue<>(List.of(3, 1, 2));`
   `System.out.println(q.poll() + " " + q.peek());`

3. Why should you prefer `ArrayDeque` over `Stack` and over `LinkedList` for a stack?
4. Why can `System.out.println(priorityQueue)` not be used to show the sorted contents?
5. What is wrong here? `Deque<String> d = new ArrayDeque<>(); d.add(null);`
6. How would you keep the 10 largest numbers from a stream of a billion numbers?

Answers

1. `remove()` throws `NoSuchElementException`; `poll()` returns `null`.
2. `1 2`. `poll` removes the smallest (1) and `peek` then shows the next smallest (2).
3. `Stack` is a synchronized legacy `Vector` subclass whose API breaks the stack contract; `LinkedList` allocates a node per element. `ArrayDeque` is a compact circular array with O(1) amortized ends.
4. `toString` and iteration use the heap's array order, where only index 0 is guaranteed to be the minimum; the rest is only partially ordered.
5. `ArrayDeque` does not permit `null` elements; `add(null)` throws `NullPointerException`.
6. Keep a min-heap `PriorityQueue` of size 10: offer each number and poll when the size exceeds 10. O(n log 10) time, O(10) memory.

KEY POINTS

Key points

- `Queue` is FIFO with two method families (throwing `add/remove/element`, value-returning `offer/poll/peek`); `Deque` adds both ends and doubles as the stack.
- Use `ArrayDeque` for queues and stacks; avoid `Stack` and (usually) `LinkedList`.
- `PriorityQueue` is an array-based binary min-heap: O(log n) offer/poll, O(1) peek, O(n) search; iteration order is not sorted and ties are not stable.
- `BlockingQueue` (`put`/`take`) is the thread-safe, back-pressure-capable cousin, covered in the concurrency Part.

6.10 Choosing a data structure, and Big-O

Most "which collection?" questions have a short answer once you know your access pattern: *how will I mostly read, insert, remove and iterate, and does order matter?* This chapter gives you the table, the reasoning behind it, and the cautions that stop you from over-optimising.

6.10.1 Big-O in thirty seconds

Big-O notation describes how the cost of an operation grows with the number of elements n, ignoring constant factors. O(1) is constant, O(log n) grows very slowly (about 20 steps for a million elements), O(n) grows in proportion, and O(n log n) is the cost of a good sort. "Amortized O(1)" means that an occasional expensive operation (such as an array resize) is spread across many cheap ones. Big-O says nothing about the constant: an O(1) hash lookup may cost far more nanoseconds than an O(n) scan of a tiny array.

6.10.2 The table

Structure	get / contains	add	remove	order	notes
`ArrayList`	O(1) by index; O(n) contains	amortized O(1) at end; O(n) middle	O(n)	insertion	best default list
`LinkedList`	O(n)	O(1) at ends	O(1) at known node	insertion	O(n) to find the node
`ArrayDeque`	O(1) at ends	amortized O(1) at ends	O(1) at ends	insertion	queue / stack, no null
`HashSet / HashMap`	O(1) average	O(1) average	O(1) average	none	O(log n) worst case per bucket
`LinkedHashSet / Map`	O(1)	O(1)	O(1)	insertion or access	extra links per entry
`TreeSet / TreeMap`	O(log n)	O(log n)	O(log n)	sorted	floor, ceiling, range views
`PriorityQueue`	O(n) contains; O(1) peek	O(log n)	O(log n) poll; O(n) remove(o)	heap only	min at head
`EnumSet / EnumMap`	O(1)	O(1)	O(1)	enum order	bit set / array inside

Typical costs; n is the number of elements. Hash costs assume a well-distributed hashCode.

6.10.3 Memory overhead matters as much as speed

Every extra object costs a header (typically 12 bytes with compressed class pointers, rounded up to 8-byte alignment) plus a reference to it. A rough picture on a 64-bit HotSpot JVM with compressed references:

- `ArrayList<Integer>`: one 4-byte slot per element in the backing array, plus the `Integer` object itself (16 bytes) unless it is in the small cache. Primitives would be 4 bytes. This **boxing** overhead is why `int[]` or a primitive-specialised library wins on huge numeric data.
- `LinkedList`: a node per element (header + three references, about 24 bytes) on top of the element.
- `HashMap`: a node per entry (header + hash + key + value + next, about 32 bytes), plus the table array, which is kept at most 75 percent full by default.
- `HashSet` is a `HashMap` underneath, so it pays for the value slot too.
- `TreeMap`: a node per entry with key, value, left, right, parent and a colour flag, about 40 bytes.

These figures are estimates for a typical 64-bit JVM, not guarantees; measure with a tool such as JOL or a heap dump before relying on exact numbers.

6.10.4 Does the complexity show up in practice?

LookupCost.java JAVA

```java
import java.util.ArrayList;
import java.util.HashSet;
import java.util.List;
import java.util.Set;
import java.util.TreeSet;

public class LookupCost {
    public static void main(String[] args) {
        int n = 100_000;
        List<Integer> list = new ArrayList<>();
        for (int i = 0; i < n; i++) {
            list.add(i);
        }
        Set<Integer> hash = new HashSet<>(list);
        Set<Integer> tree = new TreeSet<>(list);
        for (int round = 0; round < 3; round++) {
            System.out.println("round " + round
                + ": list " + time(list::contains) + " ms"
                + ", hash " + time(hash::contains) + " ms"
                + ", tree " + time(tree::contains) + " ms");
        }
    }

    static long time(java.util.function.Predicate<Integer> p) {
        long start = System.nanoTime();
        int hits = 0;
        for (int i = 0; i < 2_000; i++) {
            if (p.test(99_999 - i)) {
                hits++;
            }
        }
        if (hits != 2_000) {
            throw new AssertionError();
        }
        return (System.nanoTime() - start) / 1_000_000;
    }
}
```

Typical output (measured on the build machine; will differ on yours) TERMINAL

```
$ java LookupCost.java
round 0: list 1836 ms, hash 2 ms, tree 21 ms
round 1: list 906 ms, hash 14 ms, tree 0 ms
round 2: list 680 ms, hash 8 ms, tree 0 ms
```

Two thousand lookups in 100,000 elements cost hundreds of milliseconds in the list (each `contains` scans up to n elements) and a few milliseconds in the hash and tree sets. The exact numbers wobble because of JIT warm-up, which is also why you must never take one run as a benchmark.

WATCH OUT

Micro-benchmarks written with `System.nanoTime()` in `main` mislead. The JIT compiler, garbage collector, and dead-code elimination all distort short loops. The program above is only good enough to show a difference of several orders of magnitude. For smaller differences use JMH (the Java Microbenchmark Harness) and compare on realistic data sizes.

6.10.5 Decision guide

Ask the questions in order:

1. **Key-to-value lookup?** Use a `Map`. `HashMap` by default; `LinkedHashMap` if you need insertion or access order (or an LRU cache); `TreeMap` if you need sorted keys or range queries; `EnumMap` for enum keys; `ConcurrentHashMap` when threads share it.
2. **Uniqueness, no values?** Use a `Set`, with the same choices (`HashSet`, `LinkedHashSet`, `TreeSet`, `EnumSet`).
3. **Process in some order?** FIFO or LIFO: `ArrayDeque`. By priority: `PriorityQueue`. Between threads: a `BlockingQueue`.
4. **Otherwise a sequence?** `ArrayList`. Reach for `LinkedList` only if you have measured and know why.
5. **Read-only?** `List.of`, `Set.of`, `Map.of` and `copyOf` (see the chapter on immutable collections).

Declare variables by interface (`List<String> names = new ArrayList<>();`), so changing the implementation later is a one-line edit, and pass the narrowest interface a method needs.

Common interview picks

- Frequency count or "have I seen this?": `HashMap<K,Integer>` with `merge`, or `HashSet`.
- Two-sum, duplicates, anagram groups: `HashMap` / `HashSet` for O(1) lookups.
- Sorted order or "next greater key": `TreeMap`/`TreeSet` with `floorKey`/`ceilingKey`.
- Top-k, merge k sorted lists, running median: `PriorityQueue` (one or two).
- Breadth-first search: `ArrayDeque` as queue. Depth-first or undo history, matching brackets: `ArrayDeque` as stack.
- LRU cache: `LinkedHashMap` with access order, or a `HashMap` plus doubly linked list (built by hand in the next chapter).
- Sliding window maximum: a `Deque` holding indexes in decreasing value order.

INTERVIEW ANGLE

You have a million records to look up by id repeatedly. Which structure, and why? A `HashMap` from id to record: O(1) average lookup versus O(n) for scanning a list or O(log n) for a `TreeMap`. Pre-size it (`new HashMap<>(expected / 0.75f + 1)` or `HashMap.newHashMap(expected)` in Java 19+) to avoid repeated resizing, and make sure the key class has a correct `equals` and a well-distributed `hashCode`. If you also need the records sorted by id, add a `TreeMap` or use it instead.

Check yourself

1. Which structure gives O(log n) `floorKey` and sorted iteration, and which gives O(1) average `get` but no order?
2. Why is `ArrayList.contains` O(n) even though `ArrayList.get(i)` is O(1)?
3. What is the main memory disadvantage of `ArrayList<Integer>` compared with `int[]`?
4. You need the 100 most recent events, discarding older ones automatically. Which collection do you build?
5. True or false: an O(1) algorithm is always faster than an O(n) one. Explain.

Answers

1. `TreeMap` (or `TreeSet`); `HashMap` (or `HashSet`).
2. `get(i)` indexes the array directly, but `contains` must compare the argument with elements one by one with `equals` until it finds a match.
3. Boxing: each element is a separate `Integer` object (about 16 bytes) plus a reference in the array, against 4 bytes in the array of ints; poorer cache locality as well.
4. A bounded `ArrayDeque` used as a ring: after each `addLast`, if `size() > 100` call `removeFirst`. (A `LinkedHashMap` with `removeEldestEntry` also works for keyed data.)
5. False. Big-O hides constants and only describes growth; for small n a linear scan of a compact array can beat a hash lookup. Measure with realistic sizes.

KEY POINTS

Key points

- Choose by access pattern: lookup by key (`HashMap`), sorted (`TreeMap`), FIFO/LIFO (`ArrayDeque`), priority (`PriorityQueue`), sequence (`ArrayList`).
- Know the table: hash O(1) average, tree O(log n), array list O(1) index and O(n) search or middle insert, heap O(log n) offer/poll.
- Memory overhead (node objects, boxing, load factor) and cache locality often matter as much as the Big-O class.
- Declare by interface, trust the JDK defaults, and measure with a real benchmark harness before optimising.

6.11 Implement your own: array list, linked list, hash map, LRU cache

Reading about data structures is not the same as building them. In this chapter you write four small ones from scratch: a dynamic array, a singly linked list, a hash map with chaining, and an LRU cache. Each is a few dozen lines, runs on its own, and checks itself. The goal is not to replace the JDK (never do that in production) but to make the earlier chapters concrete, and because "implement an LRU cache" is a classic interview task.

Each listing uses `assert` statements as tests. Assertions are **disabled by default**, so run with `java -ea File.java` (`-ea` means enable assertions); without the flag a failing check would pass silently. Printed output shows the state so you can follow along.

6.11.1 A dynamic array

An `ArrayList` is an array that grows. The idea is a private `Object[]` plus a `size` counter: when the array is full, allocate a bigger one and copy. Doubling (rather than adding a fixed amount) makes `add` amortized O(1); the JDK grows by about 1.5 times, which trades a little time for less wasted space.

DynArray.java JAVA

```
import java.util.Arrays;

public class DynArray<E> {
    private Object[] items = new Object[4];
    private int size;

    public int size() {
        return size;
    }

    public void add(E e) {
        if (size == items.length) {
            items = Arrays.copyOf(items, items.length * 2);
        }
        items[size++] = e;
    }

    @SuppressWarnings("unchecked")
    public E get(int index) {
        checkIndex(index);
        return (E) items[index];
    }

    public E remove(int index) {
        checkIndex(index);
        E old = get(index);
        System.arraycopy(items, index + 1, items, index, size - index - 1);
        items[--size] = null;
        return old;
    }

    private void checkIndex(int index) {
        if (index < 0 || index >= size) {
            throw new IndexOutOfBoundsException("index " + index + ", size " + size);
        }
    }

    @Override
    public String toString() {
        return Arrays.toString(Arrays.copyOf(items, size));
    }

    public static void main(String[] args) {
        DynArray<String> a = new DynArray<>();
        for (String s : "ant bee cat dog eel fox".split(" ")) {
            a.add(s);
        }
        assert a.size() == 6;
        assert a.get(4).equals("eel");
        assert a.remove(1).equals("bee");
        assert a.size() == 5 && a.get(1).equals("cat");
        try {
```

DynArray.java (continued) JAVA

```
            a.get(5);
            assert false : "should have thrown";
        } catch (IndexOutOfBoundsException expected) {
            System.out.println("caught: " + expected.getMessage());
        }
        System.out.println(a + " size=" + a.size());
        System.out.println("all DynArray checks passed");
    }
}
```

Run with assertions enabled TERMINAL

```
$ java -ea DynArray.java
caught: index 5, size 5
[ant, cat, dog, eel, fox] size=5
all DynArray checks passed
```

What to notice

- The array is `Object[]` and `get` casts to `E`. Generics are erased (see the chapter on generics), so you cannot write `new E[4]`; the unchecked cast is safe because only `add(E)` ever stores into the array.
- `remove` shifts the tail left with `System.arraycopy` (O(n)) and then nulls the vacated slot. Forgetting that last line is a classic memory leak: the array would keep a reference to an object the caller thinks is gone.
- Index checks throw `IndexOutOfBoundsException` with a useful message, as the real class does. Note the check is against `size`, not `items.length`.

6.11.2 A singly linked list

A linked list stores each element in a **node** that points to the next node. There is no shifting and no resizing, but reaching the k-th element means following k links. Our list keeps only a `head` reference, so `addFirst` is O(1) and `addLast` walks the whole list, O(n). (`java.util.LinkedList` is doubly linked and keeps a `tail`, so both ends are O(1).)

SinglyLinkedList.java JAVA

```
public class SinglyLinkedList<E> {
    private static class Node<E> {
        E value;
        Node<E> next;

        Node(E value) {
            this.value = value;
        }
    }

    private Node<E> head;
    private int size;

    public void addFirst(E e) {
        Node<E> n = new Node<>(e);
        n.next = head;
        head = n;
        size++;
    }

    public void addLast(E e) {
        Node<E> n = new Node<>(e);
        if (head == null) {
            head = n;
        } else {
            Node<E> cur = head;
            while (cur.next != null) {
                cur = cur.next;
            }
            cur.next = n;
        }
        size++;
    }

    public boolean remove(E e) {
        Node<E> prev = null;
        for (Node<E> cur = head; cur != null; prev = cur, cur = cur.next) {
```

SinglyLinkedList.java (continued) JAVA

```
            if (java.util.Objects.equals(cur.value, e)) {
                if (prev == null) {
                    head = cur.next;
                } else {
                    prev.next = cur.next;
                }
                size--;
                return true;
            }
        }
        return false;
    }

    public void reverse() {
        Node<E> prev = null;
        Node<E> cur = head;
        while (cur != null) {
            Node<E> next = cur.next;
            cur.next = prev;
            prev = cur;
            cur = next;
        }
        head = prev;
    }

    public int size() {
        return size;
    }

    @Override
    public String toString() {
        StringBuilder sb = new StringBuilder("[");
        for (Node<E> cur = head; cur != null; cur = cur.next) {
            sb.append(cur.value);
            if (cur.next != null) {
                sb.append(" -> ");
            }
        }
        return sb.append("]").toString();
    }

    public static void main(String[] args) {
        SinglyLinkedList<Integer> list = new SinglyLinkedList<>();
        list.addLast(2);
        list.addLast(3);
        list.addFirst(1);
        System.out.println(list + " size=" + list.size());
        assert list.toString().equals("[1 -> 2 -> 3]");
        list.reverse();
        System.out.println("reversed: " + list);
        assert list.toString().equals("[3 -> 2 -> 1]");
        assert list.remove(2) && !list.remove(99);
        assert list.size() == 2;
        System.out.println("after remove(2): " + list);
    }
}
```

Run with assertions enabled TERMINAL

```
$ java -ea SinglyLinkedList.java
[1 -> 2 -> 3] size=3
reversed: [3 -> 2 -> 1]
after remove(2): [3 -> 1]
```

What to notice

- `Node` is a `private static` nested class: it does not need the outer instance, so it does not carry the hidden reference an inner class would (see the chapter on nested classes).
- `remove` tracks a `prev` pointer so it can splice the node out; removing the head is a special case. These pointer cases are exactly where interview bugs happen: always test the head, the middle, the last node, and the empty list.
- `reverse` is the famous three-pointer loop: save `next`, point `cur.next` backwards, advance both. Draw it on paper once and it sticks. It runs in O(n) time and O(1) extra space.

6.11.3 A hash map with chaining

Now the structure behind `HashMap` (see the chapter on HashMap in depth). The pieces are: a table of buckets, a way to turn a key into a bucket index, and a chain of entries per bucket to handle collisions (two keys landing in the same bucket).

MyHashMap.java JAVA

```
public class MyHashMap<K, V> {
    private static class Entry<K, V> {
        final K key;
        V value;
        Entry<K, V> next;

        Entry(K key, V value, Entry<K, V> next) {
            this.key = key;
            this.value = value;
            this.next = next;
        }
    }

    private Entry<K, V>[] table = newTable(16);
    private int size;

    @SuppressWarnings("unchecked")
    private static <K, V> Entry<K, V>[] newTable(int n) {
        return (Entry<K, V>[]) new Entry[n];
    }

    private static int hash(Object key) {
        int h = key == null ? 0 : key.hashCode();
        return h ^ (h >>> 16);
    }

    private static int indexFor(int hash, int length) {
        return hash & (length - 1);
    }

    public V get(K key) {
        for (Entry<K, V> e = table[indexFor(hash(key), table.length)]; e != null; e = e.next) {
            if (java.util.Objects.equals(e.key, key)) {
                return e.value;
            }
        }
        return null;
    }

    public V put(K key, V value) {
        int i = indexFor(hash(key), table.length);
        for (Entry<K, V> e = table[i]; e != null; e = e.next) {
            if (java.util.Objects.equals(e.key, key)) {
                V old = e.value;
                e.value = value;
                return old;
            }
        }
        table[i] = new Entry<>(key, value, table[i]);
        if (++size > table.length * 3 / 4) {
            resize();
        }
        return null;
    }

    private void resize() {
        Entry<K, V>[] old = table;
        table = newTable(old.length * 2);
        for (Entry<K, V> head : old) {
            for (Entry<K, V> e = head; e != null; ) {
                Entry<K, V> next = e.next;
                int i = indexFor(hash(e.key), table.length);
                e.next = table[i];
                table[i] = e;
                e = next;
            }
        }
    }
```

MyHashMap.java (continued) JAVA

```java
    public int size() {
        return size;
    }

    public int capacity() {
        return table.length;
    }

    public static void main(String[] args) {
        MyHashMap<String, Integer> m = new MyHashMap<>();
        m.put("one", 1);
        m.put("two", 2);
        m.put(null, 0);
        assert m.get("two") == 2;
        assert m.get(null) == 0;
        assert m.put("one", 11) == 1 && m.get("one") == 11;
        assert m.size() == 3;
        for (int i = 0; i < 1000; i++) {
            m.put("k" + i, i);
        }
        for (int i = 0; i < 1000; i++) {
            assert m.get("k" + i) == i;
        }
        assert m.get("missing") == null;
        System.out.println("size=" + m.size() + " capacity=" + m.capacity());
        System.out.println("all MyHashMap checks passed");
    }
}
```

Run with assertions enabled TERMINAL

```
$ java -ea MyHashMap.java
size=1003 capacity=2048
all MyHashMap checks passed
```

What to notice

- `hash` spreads the high bits into the low bits with `h ^ (h >>> 16)`, as the JDK does, because the index uses only the low bits.
- `hash & (length - 1)` equals `hash % length` when the length is a power of two, but is faster and never negative. This is why the table size doubles each time instead of growing by an arbitrary amount.
- When `size` exceeds 75 percent of the table length, `resize` doubles the table and **re-inserts every entry** (rehash), because the index depends on the table length. 1,003 entries ended at capacity 2048: 1,003 > 768 (75 percent of 1024) forced the last doubling.
- A `null` key hashes to 0, like in the JDK. `Objects.equals` handles the null comparison.
- This version has no treeification, no `remove`, no iterators and is not thread-safe. Adding `remove` is a good exercise: it is the same `prev` splice as in the linked list.

GOTCHA

If `hashCode` returns a constant, this map degrades to O(n) lookups. Every key lands in one bucket and the chain becomes a linked list. The JDK's `HashMap` softens the damage by converting long chains into red-black trees when the keys are `Comparable`, but a correct, well-spread `hashCode` is the real fix.

6.11.4 An LRU cache

A **Least Recently Used (LRU) cache** holds at most `capacity` entries; when full, adding a new one evicts the entry that was used longest ago. Both `get` and `put` must be O(1). No single simple structure gives that, so we combine two:

- a `HashMap` from key to node, for O(1) lookup;
- a doubly linked list ordered by recency, with the most recently used at the front, for O(1) "move to front" and "remove the last".

Two **sentinel** nodes (`head` and `tail`, holding no data) remove all the null checks at the ends of the list.

LruCache.java JAVA

```java
import java.util.HashMap;
import java.util.Map;

public class LruCache<K, V> {
    private static class Node<K, V> {
        K key;
        V value;
        Node<K, V> prev;
        Node<K, V> next;
    }

    private final int capacity;
    private final Map<K, Node<K, V>> index = new HashMap<>();
    private final Node<K, V> head = new Node<>();
    private final Node<K, V> tail = new Node<>();

    public LruCache(int capacity) {
        this.capacity = capacity;
        head.next = tail;
        tail.prev = head;
    }

    public V get(K key) {
        Node<K, V> n = index.get(key);
        if (n == null) {
            return null;
        }
        unlink(n);
        linkAfterHead(n);
        return n.value;
    }

    public void put(K key, V value) {
        Node<K, V> n = index.get(key);
        if (n != null) {
            n.value = value;
            unlink(n);
            linkAfterHead(n);
            return;
        }
        if (index.size() == capacity) {
            Node<K, V> lru = tail.prev;
            unlink(lru);
            index.remove(lru.key);
        }
        n = new Node<>();
        n.key = key;
        n.value = value;
        index.put(key, n);
        linkAfterHead(n);
    }

    private void unlink(Node<K, V> n) {
        n.prev.next = n.next;
        n.next.prev = n.prev;
    }

    private void linkAfterHead(Node<K, V> n) {
        n.next = head.next;
        n.prev = head;
        head.next.prev = n;
        head.next = n;
    }

    @Override
    public String toString() {
        StringBuilder sb = new StringBuilder("[");
        for (Node<K, V> n = head.next; n != tail; n = n.next) {
            sb.append(n.key).append('=').append(n.value);
            if (n.next != tail) {
                sb.append(", ");
            }
        }
        return sb.append("] (most recent first)").toString();
    }
```

LruCache.java (continued) JAVA

```
    public static void main(String[] args) {
        LruCache<String, Integer> c = new LruCache<>(2);
        c.put("a", 1);
        c.put("b", 2);
        c.get("a");
        c.put("c", 3);
        System.out.println(c);
        assert c.get("b") == null;
        assert c.get("a") == 1 && c.get("c") == 3;
        c.put("a", 10);
        c.put("d", 4);
        System.out.println(c);
        assert c.get("c") == null && c.get("a") == 10;
        System.out.println("all LruCache checks passed");
    }
}
```

Run with assertions enabled TERMINAL

```
$ java -ea LruCache.java
[c=3, a=1] (most recent first)
[d=4, a=10] (most recent first)
all LruCache checks passed
```

What to notice

- Every successful `get` and `put` of an existing key calls `unlink` then `linkAfterHead`: that is the whole "recently used" mechanic.
- The eviction victim is always `tail.prev`, and its key is stored *in the node* so we can remove it from the index map too. That is why `Node` has a `key` field.
- Capacity 2: after `put a`, `put b`, `get a`, `put c`, the cache evicts `b` (not `a`), exactly as the printed state shows.

TIP

In real code you would not hand-write this. `LinkedHashMap` in access-order mode with `removeEldestEntry` is a three-line LRU cache (see the chapter on other maps), and Caffeine, a popular cache library that Spring Boot's cache support can use, adds thread safety, expiry and a better eviction policy. Writing it yourself once is for understanding and for interviews.

INTERVIEW ANGLE

Design an LRU cache with O(1) get and put. Combine a hash map (key to node) with a doubly linked list ordered by recency. On access, unlink the node and re-insert it at the front; on insert into a full cache, remove the node before the tail sentinel and delete its key from the map. Mention thread safety (wrap with a lock or use Caffeine) and that `LinkedHashMap(cap, 0.75f, true)` gives this for free.

Check yourself

1. Run with assertions disabled, a failing `assert` does what? How do you enable them?
2. Why must `DynArray.remove` set `items[size]` to `null` after shifting?
3. In `MyHashMap`, why is `hash & (length - 1)` a valid replacement for `hash % length`, and what requirement does that put on `length`?
4. What is wrong with this `reverse`? `cur.next = prev; prev = cur; cur = cur.next;`
5. Why does the LRU cache keep the key inside each node as well as in the map?
6. What is the time complexity of `SinglyLinkedList.addLast` here, and how would you make it O(1)?

Answers

1. Nothing: assertions are disabled by default and the statement is skipped. Enable with `java -ea` (`-enableassertions`).
2. Otherwise the array keeps a stale reference to the removed element past `size`, preventing garbage collection (a memory leak).
3. For a power-of-two `length`, the mask keeps exactly the low bits, which equals the remainder, and it is never negative. So the table length must always be a power of two.

4. After `cur.next = prev`, `cur.next` points backwards, so `cur = cur.next` moves back to the previous node (or null): the rest of the list is lost. Save `next` first.
5. When evicting `tail.prev`, you only have the node; you need its key to delete the matching entry from the index map.
6. O(n), because it walks to the end. Keep a `tail` reference and update it on insert and remove to get O(1).

KEY POINTS

Key points

- Build structures from a few primitives: arrays that double (dynamic array), nodes with pointers (linked list), an array of chains indexed by a spread hash masked to a power of two (hash map).
- Resizing a hash table means rehashing every entry, since the index depends on the table length; the doubling policy keeps put at amortized O(1).
- An LRU cache is a hash map plus a doubly linked list with sentinels, giving O(1) get and put; `LinkedHashMap` in access order does the same in the JDK.
- Test with `assert` and run with `java -ea`; always cover the empty, single-element, first, middle and last cases.

PART 7

Functional Java: Lambdas and Streams

Behaviour as data: lambdas, Optional, the Stream API, collectors, parallel streams and gatherers.

In this part

- 7.1 Lambdas and Functional Interfaces
- 7.2 Optional: Modelling "No Result"
- 7.3 Streams: Fundamentals
- 7.4 Collectors
- 7.5 Parallel Streams and Performance
- 7.6 Stream Gatherers
- 7.7 Functional Style in Practice

7.1 Lambdas and Functional Interfaces

Before Java 8, passing behaviour to a method meant writing an anonymous class: five lines of ceremony to say "compare by length". Lambda expressions (Java 8, 2014) give you a short syntax for exactly that case: a piece of code you hand to another method to run later. The whole Stream API, `CompletableFuture`, `Optional.map`, `Comparator.comparing` and most of modern Spring code are built on this one idea. If you understand lambdas well enough to predict what they capture, what `this` means inside them and how they compile, the rest of this Part becomes easy.

7.1.1 Functional interfaces: the type of a lambda

A lambda has no type of its own. It is always converted to an instance of a **functional interface**: an interface with exactly one abstract method (default, static and private methods do not count, and neither do abstract methods that merely re-declare `Object` methods such as `equals`). The compiler decides which interface from the context. This is called **target typing**: the type the surrounding code expects is the type of the lambda.

Lam1.java (excerpt) JAVA

```java
interface Validator<T> { boolean test(T t); }

Validator<String> notBlank = s -> !s.isBlank();     // target type: Validator<String>
Function<Integer, Integer> inc  = x -> x + 1;      // same text, different target type
UnaryOperator<Integer>     inc2 = x -> x + 1;      // ... also fine
```

The lambda `x -> x + 1` is not "a function"; it is whatever the left-hand side says it is. The parameter types are inferred from the target type, so you normally write `s -> ...` and not `(String s) -> ...`. You may write the types, or use `var`, but you must not mix styles in one parameter list.

Form	Meaning
`() -> 42`	no parameters, expression body
`x -> x * 2`	one parameter, parentheses optional
`(a, b) -> a + b`	several parameters, inferred types
`(int a, int b) -> a + b`	explicit types (all or none)
`x -> { log(x); return x; }`	block body: needs braces, `return` and semicolons

Lambda syntax forms.

An expression body returns its value (or nothing, for a `void` target). A block body behaves like a method body. A lambda whose body is a single method call such as `s -> System.out.println(s)` is compatible with both value-returning and `void` targets; the compiler picks whichever fits.

7.1.2 The java.util.function toolbox

You rarely need to declare your own functional interface, because `java.util.function` has 43 of them. They look intimidating but follow a naming scheme. Learn the five core shapes and the rest are variations.

Interface	Abstract method	Use it for
`Function<T,R>`	`R apply(T t)`	transform a value (map)
`Predicate<T>`	`boolean test(T t)`	yes/no question (filter)
`Consumer<T>`	`void accept(T t)`	do something with a value (forEach)
`Supplier<T>`	`T get()`	produce a value, often lazily
`UnaryOperator<T>` / `BinaryOperator<T>`	`T apply(T) / T apply(T,T)`	Function where all types are the same

The five core functional interfaces.

The variations are mechanical: a `Bi` prefix means two arguments (`BiFunction<T,U,R>`, `BiConsumer`, `BiPredicate`); `Int`, `Long`, `Double` prefixes are primitive specialisations that avoid boxing (`IntPredicate`, `ToIntFunction<T>`, `IntBinaryOperator`, `IntSupplier`). `Runnable` and `Callable<V>` are the older no-argument shapes from `java.lang` and `java.util.concurrent`.

Lam1.java JAVA

```java
import java.util.*;
import java.util.function.*;

public class Lam1 {
    record Book(String title, int pages) {}

    interface Validator<T> { boolean test(T t); }

    public static void main(String[] args) {
        // 1. a lambda is an instance of a functional interface
        Validator<String> notBlank = s -> !s.isBlank();
        System.out.println(notBlank.test("Java") + " " + notBlank.test("  "));

        // 2. same lambda text, different target types
        Function<Integer, Integer> inc = x -> x + 1;
        UnaryOperator<Integer> inc2 = x -> x + 1;
        System.out.println(inc.apply(1) + " " + inc2.apply(1));

        // 3. java.util.function family
        Predicate<Book> longBook = b -> b.pages() > 300;
        Function<Book, String> title = Book::title;
        Supplier<List<Book>> fresh = ArrayList::new;
        Consumer<Book> printer = b -> System.out.println("Book: " + b.title());
        BiFunction<String, Integer, Book> make = Book::new;
        BinaryOperator<Integer> add = Integer::sum;
        IntBinaryOperator mul = (a, b) -> a * b;

        List<Book> shelf = fresh.get();
        shelf.add(make.apply("Effective Java", 412));
        shelf.add(make.apply("Java Puzzlers", 312));
        shelf.add(make.apply("Tiny Book", 90));
        shelf.stream().filter(longBook).map(title).forEach(System.out::println);
        printer.accept(shelf.get(2));
        System.out.println(add.apply(2, 3) + " " + mul.applyAsInt(4, 5));

        // 4. composition
        Predicate<Book> shortBook = longBook.negate();
        Function<Integer, Integer> dbl = x -> x * 2;
        System.out.println(shortBook.test(shelf.get(2)));
        System.out.println(inc.andThen(dbl).apply(5) + " " + inc.compose(dbl).apply(5));

        // 5. capture
        int base = 100;                         // effectively final
        Function<Integer, Integer> addBase = x -> x + base;
        System.out.println(addBase.apply(5));
        int[] counter = {0};                    // the box is final, contents mutable
        Runnable r = () -> counter[0]++;
```

Lam1.java (continued) JAVA

```
        r.run(); r.run();
        System.out.println(counter[0]);
    }
}
```

Output TERMINAL

```
$ java Lam1.java
true false
2 2
Effective Java
Java Puzzlers
Book: Tiny Book
5 20
true
12 11
105
2
```

What to notice

- `inc.andThen(dbl)` runs `inc` first then `dbl`: (5+1) times 2 = 12. `inc.compose(dbl)` runs `dbl` first: 5 times 2 plus 1 = 11. Mixing these two up is a classic mistake.
- `Predicate` gives you `and`, `or`, `negate` as default methods; `Function` gives `andThen`, `compose`, `identity()`. Small combinators like these are how you build behaviour from parts.
- `Book::new` works as a `BiFunction<String,Integer,Book>` because the record's canonical constructor takes `(String, int)`; the compiler matches shapes, boxing as needed.

7.1.3 Declaring your own: @FunctionalInterface

When none of the standard shapes carries meaning, declare your own, and annotate it with `@FunctionalInterface`. The annotation changes nothing at run time; it tells the compiler "fail the build if someone adds a second abstract method", which would silently break every lambda that targets the interface. Use it for any interface you intend lambdas to implement.

Retry.java JAVA

```
@FunctionalInterface
public interface Retry<T> {
    T attempt() throws Exception;          // checked exceptions are allowed here

    default T withRetries(int times) throws Exception {
        Exception last = null;
        for (int i = 0; i < times; i++) {
            try { return attempt(); } catch (Exception e) { last = e; }
        }
        throw last;
    }
}
```

A custom interface is also the standard answer to a limit of the built-in ones: `Function.apply` cannot throw checked exceptions, but your own `attempt()` can. Section "Functional style in practice" in the last chapter of this Part returns to this.

7.1.4 Capturing variables: effectively final

A lambda body may use local variables from the enclosing method, but only if they are **effectively final**: never reassigned after initialisation (you do not need to write `final`). The lambda captures the variable's *value*, copies it, and carries the copy with it, possibly to another thread, long after the method returned.

Capture.java (fragment) JAVA

```
int total = 0;
// ERROR: local variables referenced from a lambda expression
//        must be final or effectively final
list.forEach(x -> total += x.length());
```

Why the rule? A local variable lives in a stack frame that disappears when the method returns, and may be read by a different thread. Allowing the lambda to write a stack variable would make "shared mutable local" a thing, with all the races that implies. Instance fields and static fields are different: they live on the heap and a lambda can read and write them freely (and then you own the thread-safety problem).

The common workarounds, in order of preference: do not mutate at all (use `mapToInt(...).sum()` or a collector, which is the point of the next chapters); use an `AtomicInteger` or `LongAdder` when several threads are involved; or, for single-threaded throwaway code, a one-element array as in the listing above (`counter[0]++`). The array reference is final; its contents are not. This works but is a smell: if you need it often, you are probably writing a loop in disguise.

7.1.5 What does this mean inside a lambda?

In an anonymous class, `this` refers to the anonymous object itself. In a lambda, `this` refers to the **enclosing instance**: a lambda is not a new scope for `this`, it is just code that lives in the enclosing method. A lambda also cannot declare a local variable with the same name as one in the enclosing scope (an anonymous class can shadow it).

Lam2.java JAVA

```
import java.util.*;
import java.util.function.*;

public class Lam2 {
    private final String name = "outer";

    Runnable lambda() {
        return () -> System.out.println("lambda this = " + this.name + " "
            + getClass().getSimpleName());
    }

    Runnable anon() {
        return new Runnable() {
            public void run() { System.out.println("anon this = " + this.getClass().getName()); }
        };
    }

    static String up(String s) { return s.toUpperCase(); }
    String hello(String who) { return "hello " + who; }

    public static void main(String[] args) {
        new Lam2().lambda().run();
        new Lam2().anon().run();

        // four kinds of method reference
        Function<String, String> a = Lam2::up;                 // static
        Lam2 inst = new Lam2();
        Function<String, String> b = inst::hello;              // bound instance
        Function<String, Integer> c = String::length;          // unbound instance
        Supplier<ArrayList<String>> d = ArrayList::new;        // constructor
        Function<Integer, int[]> e = int[]::new;               // array constructor
        System.out.println(a.apply("x") + " " + b.apply("bob") + " " + c.apply("four")
            + " " + d.get() + " " + e.apply(3).length);

        // class names of lambdas
        Runnable r1 = () -> {};
        Runnable r2 = () -> {};
        System.out.println(r1.getClass().isSynthetic() + " "
            + r1.getClass().getName().contains("$$Lambda"));
        System.out.println(r1.getClass() == r2.getClass());
        System.out.println(r1.getClass().getInterfaces()[0].getSimpleName());

        // non-capturing lambda evaluated twice yields the same instance
        System.out.println(sup() == sup());
        int k = 1;
        System.out.println(cap(k) == cap(k));
    }
    static Supplier<String> sup() { return () -> "x"; }
    static Supplier<Integer> cap(int k) { return () -> k; }
}
```

Output TERMINAL

```
$ java Lam2.java
lambda this = outer Lam2
anon this = Lam2$1
X hello bob 4 [] 3
true true
false
Runnable
true
false
```

What to notice

- Inside the lambda, `this.name` compiled and `getClass()` reported `Lam2`: `this` is the enclosing `Lam2`. In the anonymous class it is a different object of class `Lam2$1`.
- The last two lines show a detail that is **not** a guarantee: a lambda that captures nothing may be a single cached instance (`sup() == sup()` was `true` here), while a capturing one is a fresh object per evaluation. Never depend on identity of lambdas. The Language Specification explicitly leaves it open.

7.1.6 Method references: four kinds

When a lambda does nothing but call one existing method, a **method reference** says it more briefly. There are four kinds and the one everybody gets wrong is the third.

Kind	Example	Equivalent lambda
`Static method`	`Lam2::up`	s -> Lam2.up(s)
`Bound instance (a specific object)`	`inst::hello`	s -> inst.hello(s)
`Unbound instance (any object of a type)`	`String::length`	s -> s.length()
`Constructor`	`ArrayList::new, int[]::new`	() -> new ArrayList<>(), n -> new int[n]

The four kinds of method reference.

In the unbound form the *first* parameter of the functional interface becomes the receiver: `Function<String,Integer> f = String::length` means `s -> s.length()`. That is also why `Comparator.comparing(Book::title)` works. A bound reference such as `System.out::println` evaluates `System.out` once, when the reference is created; the lambda `x -> System.out.println(x)` would read the field every time. Prefer a method reference when it is shorter *and* clearer; if you have to think about which kind it is, write the lambda.

GOTCHA

A method reference is ambiguous if the method is overloaded in a way that fits several shapes. `Integer::toString` fails as a `Function<Integer,String>` because both the static `Integer.toString(int)` and the instance `toString()` fit. Use a lambda, `i -> i.toString()`, or `String::valueOf`.

7.1.7 How lambdas compile: invokedynamic, not inner classes

A natural guess is that the compiler turns each lambda into an anonymous inner class. It does not, and the difference matters for startup time, memory and even stack traces. Compile this tiny program and look at the bytecode.

Lam3.java JAVA

```java
public class Lam3 {
    interface Op { int apply(int a, int b); }
    public static void main(String[] args) {
        Op add = (a, b) -> a + b;
        Runnable r = () -> System.out.println("hi " + add.apply(2, 3));
        r.run();
    }
}
```

javap output, trimmed TERMINAL

```
$ javac -d out Lam3.java && ls out
Lam3$Op.class
Lam3.class
$ javap -c -p -cp out Lam3
  public static void main(java.lang.String[]);
    Code:
       0: invokedynamic #7,  0     // InvokeDynamic #0:apply:()LLam3$Op;
       5: astore_1
       6: aload_1
       7: invokedynamic #11,  0    // InvokeDynamic #1:run:(LLam3$Op;)Ljava/lang/Runnable;
      12: astore_2
      13: aload_2
      14: invokeinterface #15,  1  // InterfaceMethod java/lang/Runnable.run:()V
      19: return

  private static void lambda$main$1(Lam3$Op);
    Code:
       ...
       6: invokeinterface #25,  3  // InterfaceMethod Lam3$Op.apply:(II)I

  private static int lambda$main$0(int, int);
    Code:
       0: iload_0
       1: iload_1
       2: iadd
       3: ireturn
```

There are only two class files: no `Lam3$1.class`. The lambda *bodies* became ordinary private static methods named `lambda$main$0` and `lambda$main$1` in the same class. The place where each lambda is "created" became an `invokedynamic` instruction. Note that `lambda$main$1` takes the captured `Op` as its first parameter: captured values become extra leading arguments.

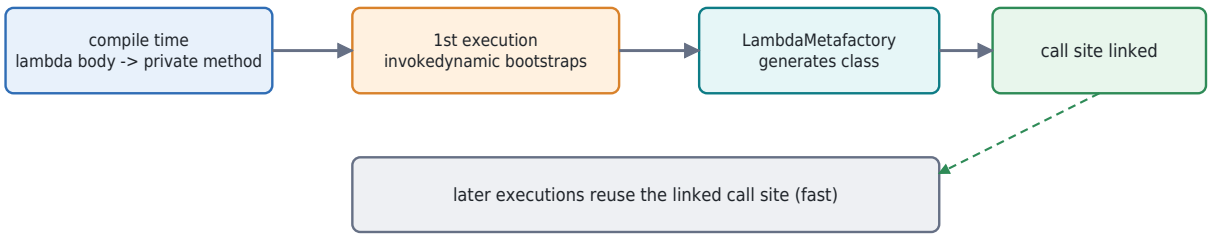


Figure 7.1. First execution links the call site; later executions reuse the linked result. The body is a plain private method; the object that implements the interface is generated at run time.

How it works at run time:

1. The first time the JVM executes an `invokedynamic` instruction, it calls its **bootstrap method**, listed in the class file's `BootstrapMethods` attribute. For lambdas this is `LambdaMetafactory.metafactory`.
2. The metafactory receives the target interface (`Op`), the method handle for the synthetic body (`REF_invokeStatic Lam3.lambda$main$0`) and the captured arguments' types. It spins up a small hidden class at run time that implements the interface and delegates to the body.
3. It returns a `CallSite`. The JVM permanently links that instruction to it, so later executions skip the bootstrap and, with no captured variables, can hand back one shared instance.

Why not inner classes? An anonymous class is fixed at compile time: one `.class` file per lambda to load, verify and keep in Metaspace, even if never run. With `invokedynamic` the compiler emits a *recipe* and the strategy is chosen by the runtime. The translation can improve across JDK releases without recompiling your code; this is the same trick used for string concatenation (`makeConcatWithConstants`, visible in the `javap` output above). The cost is some startup work on the first use of each lambda, which is one reason a Java program that uses many lambdas starts a little slower on a cold JVM and why AOT caching features target it.

NOTE
Lambda classes are *hidden classes*: you cannot find them by name, they do not show up in `Class.forName`, and in a stack trace they appear as frames like `Lam2$$Lambda/0x000...run`, usually filtered out. The synthetic body method `lambda$main$0` does appear in stack traces; that is the frame you will read when a lambda throws.

7.1.8 Pitfalls and good practice

- **Keep lambdas short.** More than three or four lines: extract a named method and use a method reference. A name documents intent and gives you a clean stack frame.
- **Do not capture what you do not need.** A lambda that uses an instance field or method captures `this`. If you store it in a long-lived registry (a listener list, a cache), you keep the whole enclosing object alive: the same leak pattern as an inner class.
- **Checked exceptions** cannot cross a standard functional interface. Wrap in an unchecked exception or use your own interface (see `Retry` above).
- **Avoid overloads that differ only in functional interface type** (`submit(Runnable)` vs `submit(Callable)`): lambdas are often ambiguous between them. `ExecutorService.submit` gets away with it only because a lambda with a return value cannot be a `Runnable`.

INTERVIEW ANGLE

How are lambdas different from anonymous inner classes? (1) `this` refers to the enclosing instance, not the lambda. (2) A lambda is compiled to a private synthetic method plus an `invokedynamic` call site bootstrapped by `LambdaMetafactory`; an anonymous class is a separate `.class` file loaded at class-load time. (3) A lambda can only implement a functional interface (one abstract method); an anonymous class can extend a class or implement any interface and can hold state. (4) A lambda's identity is unspecified (it may be cached); an anonymous class creates a new object each time.

INTERVIEW ANGLE

Why must captured local variables be effectively final? The lambda gets a copy of the value, and it may run later or on another thread after the frame is gone. If the variable could change, the copy and the original would silently disagree, or threads would race on a stack slot. The rule turns the problem into a compile-time error. Fields are not restricted because they live on the heap, not in the frame.

Check yourself

1. What is the type of the expression `x -> x + 1`?
2. What does this print? `Function<Integer,Integer> f = x -> x + 1; Function<Integer,Integer> g = x -> x * 2; System.out.println(f.compose(g).apply(3));`
3. Why does this not compile, and what is a good fix? `int n = 0; list.forEach(s -> n++);`
4. Name the four kinds of method reference and give a one-line example of each.
5. In a class `Box` with a field `int size`, does `this.size` inside a lambda in one of its methods refer to the lambda or to the `Box`? What about inside an anonymous class?
6. How many extra `.class` files does the compiler produce for a class containing three lambdas, and where does the lambda code go?

Answers

1. It has no standalone type; it takes the type of its target, which must be a functional interface (for example `Function<Integer,Integer>` or `UnaryOperator<Integer>`).
2. `7`. `compose` runs `g` first (3 times 2 = 6) and then `f` (6+1 = 7).
3. `n` is modified so it is not effectively final. Fix: avoid mutation, e.g. `int n = list.size()` or `list.stream().mapToInt(String::length).sum()`; for threads use `AtomicInteger`.
4. Static `Integer::parseInt`; bound `System.out::println`; unbound `String::length`; constructor `ArrayList::new` (also `int[]::new`).
5. In a lambda `this` is the enclosing `Box`, so `this.size` works. In an anonymous class `this` is the anonymous object, and you must write `Box.this.size`.
6. None. The bodies become private synthetic methods (`lambda$main$0` and so on) inside the same class; the interface implementations are hidden classes generated at run time via `invokedynamic`.

KEY POINTS

A lambda is an instance of a functional interface, whose type comes from the context (target typing).

- Memorise the five core shapes: `Function`, `Predicate`, `Consumer`, `Supplier`, and the operator variants; the rest are `Bi`/primitive variations.
- Captured locals must be effectively final; `this` in a lambda is the enclosing instance.
- There are four method reference kinds; the unbound one (`String::length`) takes the receiver as the first argument.
- Lambdas compile to a private synthetic method plus `invokedynamic` and `LambdaMetafactory`, not to inner class files.

7.2 Optional: Modelling "No Result"

Tony Hoare called the null reference his "billion dollar mistake". A method that returns `Member` might return a member or `null`, and nothing in the signature tells the caller. The caller who forgets the check gets a `NullPointerException` far from the cause. `java.util.Optional<T>` (Java 8) is a small container that is either empty or holds one non-null value, so the *return type itself* says "this may be absent" and forces the caller to deal with it.

7.2.1 Design intent: a return type, not a field type

The Javadoc is explicit: `Optional` is intended for use as a **method return type** where there is a clear need to represent "no result" and `null` would likely cause errors. It is not meant as a replacement for every nullable reference. In particular, do not use it for fields, method parameters, collection elements or map values. It is not `Serializable`, it adds an object allocation, and a parameter of type `Optional<String>` can itself be `null`, which defeats the purpose.

7.2.2 Creating and consuming

There are exactly three ways to create one: `Optional.of(x)` (x must be non-null, otherwise NPE immediately), `Optional.ofNullable(x)` (empty if x is null) and `Optional.empty()`. Everything else is a way of getting something *out*, or transforming it without leaving the box.

Opt1.java JAVA

```
import java.util.*;

public class Opt1 {
    record Member(String name, String email) {}

    static final Map<Integer, Member> DB = Map.of(
        1, new Member("Asha", "asha@example.com"),
        2, new Member("Ben", null));

    static Optional<Member> find(int id) { return Optional.ofNullable(DB.get(id)); }

    static String expensiveDefault() {
        System.out.println("  (computing default)");
        return "none@example.com";
    }

    public static void main(String[] args) {
        // creation
        Optional<String> e = Optional.empty();
        Optional<String> p = Optional.of("x");
        Optional<String> n = Optional.ofNullable(null);
        System.out.println(e + " " + p + " " + n);
        try { Optional.of(null); }
        catch (NullPointerException ex) { System.out.println("of(null) -> NPE"); }

        // map / flatMap / filter
        for (int id : new int[]{1, 2, 3}) {
            String mail = find(id)
                .map(Member::email)              // map: null result becomes empty
                .filter(s -> s.endsWith(".com"))
                .orElse("(no email)");
            System.out.println(id + " -> " + mail);
        }

        // orElse vs orElseGet
        System.out.println("orElse:");
        System.out.println(find(1).map(Member::email).orElse(expensiveDefault()));
```

Opt1.java (continued) JAVA

```
        System.out.println("orElseGet:");
        System.out.println(find(1).map(Member::email).orElseGet(Opt1::expensiveDefault));

        // orElseThrow, ifPresentOrElse, or, stream
        try { find(3).orElseThrow(() -> new NoSuchElementException("member 3")); }
        catch (NoSuchElementException ex) { System.out.println("thrown: " + ex.getMessage()); }
        find(2).ifPresentOrElse(m -> System.out.println("found " + m.name()),
                                () -> System.out.println("absent"));
        System.out.println(find(3).or(() -> find(1)).map(Member::name).get());
        System.out.println(java.util.stream.Stream.of(1, 2, 3).map(Opt1::find)
            .flatMap(Optional::stream).map(Member::name).toList());

        // flatMap
        Optional<Optional<String>> nested = find(1).map(m -> Optional.ofNullable(m.email()));
        Optional<String> flat = find(1).flatMap(m -> Optional.ofNullable(m.email()));
        System.out.println(nested + " | " + flat);

        // primitives
        OptionalInt max = java.util.stream.IntStream.of(3, 9, 4).max();
        System.out.println(max + " " + max.getAsInt() + " "
            + java.util.stream.IntStream.empty().max());
        OptionalDouble avg = java.util.stream.IntStream.empty().average();
        System.out.println(avg.orElse(-1));
        // equality
        System.out.println(Optional.of("a").equals(Optional.of("a")));
        try { e.get(); } catch (NoSuchElementException ex) { System.out.println(ex.getMessage()); }
    }
}
```

Output TERMINAL

```
$ java Opt1.java
Optional.empty Optional[x] Optional.empty
of(null) -> NPE
1 -> asha@example.com
2 -> (no email)
3 -> (no email)
orElse:
  (computing default)
asha@example.com
orElseGet:
asha@example.com
thrown: member 3
found Ben
Asha
[Asha, Ben]
Optional[Optional[asha@example.com]] | Optional[asha@example.com]
OptionalInt[9] 9 OptionalInt.empty
-1.0
true
No value present
```

What to notice

- **Transform, do not unwrap.** `find(id).map(Member::email).filter(...).orElse(...)` is a chain with no `if` and no `null`. Member 2 has an email of `null`; `map` turns a `null` result into an empty `Optional`, so the chain keeps working. Member 3 does not exist at all and takes the same path.
- `orElse` **evaluates its argument eagerly.** "(computing default)" printed even though the member had an email. `orElseGet(supplier)` only calls the supplier when the `Optional` is empty. Use `orElse` for cheap constants and `orElseGet` for anything that does work (a database call, object construction, logging).
- `map` **vs** `flatMap`**.** `map(f)` wraps whatever `f` returns, so a function that already returns an `Optional` produces `Optional<Optional<String>>`. Use `flatMap` when your function returns an `Optional`: it flattens one level. This is the same relationship you will see between `Stream.map` and `Stream.flatMap`.
- `Optional::stream` (Java 9) turns an `Optional` into a stream of zero or one element. `flatMap(Optional::stream)` is the idiomatic way to drop the empties from a stream of optionals.
- `orElseThrow()` with no arguments (Java 10) is the clear, intention-revealing replacement for `get()`. It throws `NoSuchElementException`, but its name says so.

Method	Behaviour when empty
`orElse(T)`	returns the given default (argument always evaluated)
`orElseGet(Supplier)`	calls the supplier lazily
`orElseThrow() / orElseThrow(Supplier)`	throws NoSuchElementException / your exception
`ifPresent(Consumer)`	does nothing
`ifPresentOrElse(Consumer, Runnable)`	runs the Runnable (Java 9)
`or(Supplier<Optional>)`	returns the supplied alternative Optional (Java 9)
`get()`	throws NoSuchElementException; avoid

Which method do I use to get the value out?

7.2.3 Primitive optionals

`OptionalInt`, `OptionalLong` and `OptionalDouble` hold a primitive and avoid boxing. They are what `IntStream.max()`, `average()` and friends return. They have `getAsInt()`, `orElse`, `ifPresent`, `isPresent` but no `map`/`filter`, because they exist for the end of a primitive stream rather than for chaining. Note that `average()` of an empty stream is an empty `OptionalDouble`, not `NaN`; that is why the example printed `-1.0` using `orElse(-1)`.

7.2.4 Anti-patterns

1. isPresent() followed by get()

This is a null check wearing a costume, and it gains nothing:

Anti1.java (fragment) JAVA

```
// bad
Optional<Member> m = find(1);
if (m.isPresent()) {
    send(m.get().email());
}

// good
find(1).map(Member::email).ifPresent(this::send);
```

2. Optional as a parameter, field or collection element

`void register(Optional<String> nickname)` forces every caller to wrap, can still be passed `null`, and communicates less than two overloads or a nullable parameter with documentation. A field of type `Optional` is not serialisable and doubles the object count. Return `Optional` from the *getter* of a possibly-absent field instead: `Optional<String> nickname() { return Optional.ofNullable(nickname); }`.

3. Returning null from a method that returns Optional

The worst of both worlds: `return null;` where `Optional.empty()` was meant. Callers trust the type and call `.map` on a null reference.

4. Using Optional for collections

Return an empty `List`, not `Optional<List<T>>`. An empty collection already means "nothing".

5. Optional.get() without a check, and orElse(null)

`get()` in code review is a red flag: prefer `orElseThrow()` so the failure mode is stated. `orElse(null)` re-introduces the null you were avoiding; it is acceptable only at a boundary with an API that demands null.

> **GOTCHA**
> `Optional` is a value-based class. Do not use it as a lock for `synchronized`, do not compare with `==` (use `equals`) and do not rely on identity. Equality is defined by the contained values: `Optional.of("a").equals(Optional.of("a"))` is `true`.

REAL WORLD
In Spring Data, repository methods such as `CrudRepository.findById(id)` return `Optional<T>`, and `Optional` works well with `orElseThrow(() -> new NotFoundException(id))` in services, which a controller advice then maps to HTTP 404. Spring MVC also supports `Optional<T>` for optional `@RequestParam` values, which is one of the few cases of `Optional` as a parameter that the framework itself handles for you.

INTERVIEW ANGLE
What is the difference between `orElse` and `orElseGet`, and between `map` and `flatMap` on `Optional`? `orElse(x)` evaluates `x` eagerly even if a value is present; `orElseGet(supplier)` invokes the supplier only if empty, so use it when the default is expensive or has side effects. `map(f)` wraps the result of `f` in an `Optional` (and turns a `null` result into empty); `flatMap(f)` expects `f` to return an `Optional` already and avoids the double wrapping.

Check yourself

1. What does `Optional.ofNullable(null).map(String::length).orElse(0)` return?
2. What is wrong with `public void setName(Optional<String> name)` as a setter?
3. In `opt.orElse(load())`, when does `load()` run? How do you change that?
4. Given `Optional<String> o = Optional.of("a");` what does `o.map(s -> (String) null).isPresent()` print?
5. You have `List<Integer> ids` and `Optional<Member> find(int id)`. Write one stream expression producing the list of members that exist.
6. Why should you not call `get()` without `isPresent()`, and what should you use?

Answers

1. `0`. The empty `Optional` stays empty through `map`, and `orElse` supplies 0.
2. A parameter of type `Optional` can itself be `null`, forces wrapping on every caller and adds nothing over a nullable parameter or an overload. `Optional` is for return types.
3. Always, before `orElse` is called, even when the value is present. Use `orElseGet(this::load)` to make it lazy.
4. `false`. When the mapping function returns `null`, `map` returns `Optional.empty()`.
5. `ids.stream().map(this::find).flatMap(Optional::stream).toList()`.
6. `get()` throws `NoSuchElementException` when empty and hides that risk. Use `orElseThrow()` (or `orElseThrow(Supplier)`), `orElse`, `orElseGet`, or `ifPresent`.

KEY POINTS
`Optional<T>` is a return type that makes "may be absent" explicit; do not use it for fields, parameters or collections.

- Create with `of`, `ofNullable`, `empty`; transform with `map`, `flatMap`, `filter`; leave with `orElse`, `orElseGet`, `orElseThrow`, `ifPresentOrElse`.
- `orElse` is eager, `orElseGet` is lazy; `flatMap` flattens an `Optional` returned by the function.
- `isPresent()` plus `get()` is a null check in disguise; chain instead. Never return `null` from an `Optional` method.
- `OptionalInt`, `OptionalLong`, `OptionalDouble` avoid boxing and end primitive-stream pipelines.

7.3 Streams: Fundamentals

A loop tells the computer *how* to process data: create an accumulator, iterate, test, add. A stream pipeline states *what* you want: "the titles of the long books, upper-cased". Streams (Java 8, package `java.util.stream`) are a way to describe a computation over a sequence of elements as a pipeline of steps, and then let the library run it. They are not a data structure: a stream stores nothing, it is a view over a source that carries a recipe.

7.3.1 Anatomy of a pipeline

Every pipeline has three parts:

1. A **source** that produces elements: a collection (`list.stream()`), an array (`Arrays.stream(a)`), a generator (`Stream.iterate`), a range (`IntStream.range`), a file (`Files.lines`), and so on.
2. Zero or more **intermediate operations** (`filter`, `map`, `sorted`, `distinct`, `limit`, `flatMap`, `peek` ...). Each returns a new stream. They are lazy: calling them does no work.

3. Exactly one **terminal operation** (`collect`, `toList`, `forEach`, `count`, `reduce`, `findFirst`, `anyMatch`, `sum` ...). It triggers the whole pipeline and produces a result or a side effect.

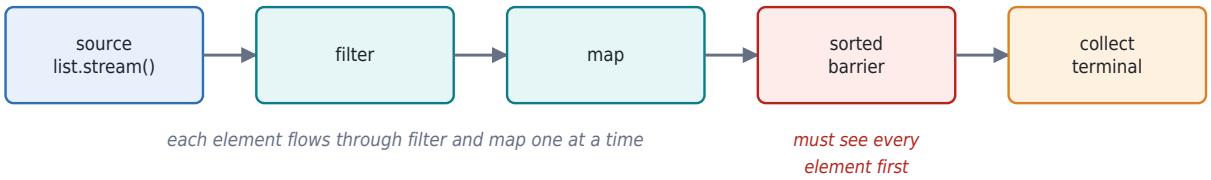


Figure 7.2. Elements flow one at a time through the whole chain (vertical processing); stateful operations such as sorted are barriers that must see every element first.

Because of laziness, the library can fuse all the steps into a single pass over the data and stop early. The following program proves it with `peek`, which calls a consumer on each element as it flows by (intended for debugging; do not put real logic in it).

Str1.java JAVA

```java
import java.util.*;
import java.util.stream.*;

public class Str1 {
    public static void main(String[] args) {
        List<String> names = List.of("kiwi", "apple", "fig", "banana", "cherry", "date");

        // 1. nothing runs until the terminal operation
        Stream<String> pipeline = names.stream()
            .peek(s -> System.out.println("  peek1 " + s))
            .filter(s -> s.length() > 3)
            .peek(s -> System.out.println("  peek2 " + s))
            .map(String::toUpperCase);
        System.out.println("pipeline built, nothing printed yet");
        List<String> out = pipeline.limit(2).toList();
        System.out.println(out);

        // 2. one-shot
        Stream<String> s = names.stream();
        s.count();
        try { s.count(); } catch (IllegalStateException e) { System.out.println(e.getMessage()); }

        // 3. sources
        System.out.println(IntStream.rangeClosed(1, 5).boxed().toList());
        System.out.println(Stream.iterate(1, x -> x * 2).limit(8).toList());
        System.out.println(Stream.iterate(1, x -> x < 100, x -> x * 3).toList());
        System.out.println(Arrays.stream(new int[]{4, 5, 6}).sum());
        System.out.println("a-b-c".chars().filter(Character::isLetter).count());
        System.out.println(Stream.of("x", "y").collect(Collectors.joining("+")));

        // 4. flatMap and mapMulti
        List<List<Integer>> nested = List.of(List.of(1, 2), List.of(), List.of(3));
        System.out.println(nested.stream().flatMap(List::stream).toList());
        System.out.println(Stream.of(1, 2, 3).<Integer>mapMulti((x, sink) -> {
            if (x != 2) { sink.accept(x); sink.accept(x * 10); }
        }).toList());

        // 5. takeWhile / dropWhile (ordered)
        System.out.println(Stream.of(1, 2, 3, 10, 1, 2).takeWhile(x -> x < 5).toList());
        System.out.println(Stream.of(1, 2, 3, 10, 1, 2).dropWhile(x -> x < 5).toList());

        // 6. stateful: sorted, distinct
        System.out.println(names.stream().sorted(Comparator.comparing(String::length)
            .thenComparing(Comparator.naturalOrder())).toList());
        System.out.println(Stream.of(3, 1, 3, 2, 1).distinct().toList());

        // 7. reduce and terminal ops
        System.out.println(Stream.of(1, 2, 3, 4).reduce(0, Integer::sum));
        System.out.println(Stream.of(1, 2, 3, 4).reduce(Integer::max));
        System.out.println(names.stream().anyMatch(x -> x.startsWith("f")));
```

Str1.java (continued) JAVA

```
        System.out.println(names.stream().allMatch(x -> x.length() > 3));
        System.out.println(names.stream().noneMatch(String::isEmpty));
        System.out.println(names.stream().filter(x -> x.length() == 6).findFirst());
        System.out.println(IntStream.of(4, 8, 15).summaryStatistics());
        System.out.println(IntStream.range(0, 5).mapToObj(i -> "i" + i)
            .collect(Collectors.joining(",", "[", "]")));
        System.out.println(names.stream().mapToInt(String::length).average().getAsDouble());

        // 8. toList() is unmodifiable; collect(toList()) is not guaranteed either way
        List<String> t = names.stream().toList();
        try { t.add("x"); }
        catch (UnsupportedOperationException e) { System.out.println("toList: unmodifiable"); }
    }
}
```

Output TERMINAL

```
$ java Str1.java
pipeline built, nothing printed yet
  peek1 kiwi
  peek2 kiwi
  peek1 apple
  peek2 apple
[KIWI, APPLE]
stream has already been operated upon or closed
[1, 2, 3, 4, 5]
[1, 2, 4, 8, 16, 32, 64, 128]
[1, 3, 9, 27, 81]
15
3
x+y
[1, 2, 3]
[1, 10, 3, 30]
[1, 2, 3]
[10, 1, 2]
[fig, date, kiwi, apple, banana, cherry]
[3, 1, 2]
10
Optional[4]
true
false
true
Optional[banana]
IntSummaryStatistics{count=3, sum=27, min=4, average=9.000000, max=15}
[i0,i1,i2,i3,i4]
4.666666666666667
toList: unmodifiable
```

What to notice

- "pipeline built, nothing printed yet": creating the pipeline ran no `peek`. Work started only at `toList()`.
- The order is `peek1 kiwi, peek2 kiwi, peek1 apple, peek2 apple`: each element travels through the *entire* chain before the next one starts. It is not "filter everything, then map everything".
- After `limit(2)` was satisfied, the source stopped being consumed: `fig`, `banana`, `cherry` and `date` never appeared. This is **short-circuiting**.
- The elements that were filtered out never reach `peek2` (in this run only `kiwi` and `apple` got that far, because `limit(2)` ended the traversal).
- Reusing a stream throws `IllegalStateException: stream has already been operated upon or closed`. A stream is **one-shot**. If you need to traverse twice, create a new stream from the source, or keep a `Supplier<Stream<T>>`.

7.3.2 Creating streams

Source	Notes
`collection.stream()`	any Collection; also parallelStream()
`Arrays.stream(array) / Stream.of(a, b, c)`	arrays and literals; int[] gives an IntStream
`IntStream.range(0, n) / rangeClosed(1, n)`	half-open vs closed; the loop replacement
`Stream.iterate(seed, f)`	infinite; use limit or the 3-argument form with a stop predicate
`Stream.iterate(seed, hasNext, next)`	Java 9, like a for loop
`Stream.generate(supplier)`	infinite stream of supplied values
`string.chars() / string.lines()`	IntStream of chars / Stream of lines
`Files.lines(path)`	lazily reads a file; close it with try-with-resources
`Stream.concat(a, b), Stream.empty(), Stream.ofNullable(x)`	small helpers

Common stream sources.

> **TIP**
> Prefer the primitive streams `IntStream`, `LongStream` and `DoubleStream` for numbers. They avoid boxing and have `sum()`, `average()`, `max()` and `summaryStatistics()`. Convert with `mapToInt`, `mapToObj`, `boxed()`, `asLongStream()`.

7.3.3 Intermediate operations

The ones you use daily: `filter(Predicate)` keeps matching elements; `map(Function)` transforms each element; `flatMap(Function<T, Stream<R>>)` maps each element to a stream and concatenates them; `distinct()`, `sorted()`, `limit(n)`, `skip(n)`, `peek`, `takeWhile`, `dropWhile`, `mapMulti`.

`flatMap` is where people stall. `map` is one-to-one; `flatMap` is one-to-many (or one-to-none): each element becomes a stream, and the streams are joined into one. With `List<List<Integer>>`, `flatMap(List::stream)` gives a flat stream of integers, and an inner empty list simply contributes nothing. It is the stream counterpart of `Optional.flatMap` from the previous chapter.

`mapMulti` (Java 16) is the imperative cousin. Instead of returning a stream, your function receives each element plus a sink and may push zero, one or many outputs. It avoids allocating a small stream per element and is handy when the "expansion" is conditional. Because of type inference, you often have to give the output type explicitly (`<Integer>mapMulti`).

`takeWhile` **and** `dropWhile` (Java 9) work on the encounter order: `takeWhile` returns the longest prefix satisfying the predicate and stops; `dropWhile` discards that prefix and returns the rest. Unlike `filter`, they stop evaluating the predicate at the first failure, so on `1, 2, 3, 10, 1, 2` with `x < 5`, `takeWhile` gives `[1, 2, 3]` and `dropWhile` gives `[10, 1, 2]` (see the output above). Use them only on ordered streams.

7.3.4 Terminal operations and short-circuiting

A **short-circuiting** operation can finish without consuming the whole stream: `findFirst`, `findAny`, `anyMatch`, `allMatch`, `noneMatch`, and the intermediate `limit`/`takeWhile`. This is what makes infinite streams usable: `Stream.iterate(1, x -> x * 2).limit(8)` is fine; `Stream.iterate(...).count()` never returns.

`reduce` folds elements into one value. `reduce(0, Integer::sum)` takes an *identity* and an associative function and returns the value itself; `reduce(Integer::max)` has no identity, so it returns an `Optional` (the stream might be empty). The three-argument form `reduce(identity, accumulator, combiner)` exists for parallel streams and for when the result type differs from the element type (see `Str2` below, where it sums lengths). For most real tasks a collector (next chapter) or a primitive `sum()` is clearer.

7.3.5 Statefulness, ordering and what "behavioural parameters" must obey

Intermediate operations are either **stateless** (`filter`, `map`, `flatMap`, `peek`: each element is handled on its own) or **stateful** (`sorted`, `distinct`, `limit`, `skip`: need to remember or see other elements). `sorted` is a full barrier: it cannot emit anything until the upstream is exhausted. The second program shows that, and the other rules the Javadoc imposes on lambdas passed to stream operations.

Str2.java JAVA

```java
import java.util.*;
import java.util.stream.*;

public class Str2 {
    public static void main(String[] args) {
        // short-circuit visits
        Stream.of("a", "b", "c", "d").peek(x -> System.out.println("visit " + x))
              .filter(x -> x.compareTo("b") >= 0).findFirst().ifPresent(System.out::println);

        // sorted is a barrier
        Stream.of(3, 1, 2).peek(x -> System.out.println("before sort " + x)).sorted()
              .peek(x -> System.out.println("after sort " + x)).limit(1).forEach(x -> {});

        // vertical processing
        Stream.of("a", "b").peek(x -> System.out.println("p1 " + x)).map(String::toUpperCase)
              .forEach(x -> System.out.println("p2 " + x));

        // interference: modifying the source during the pipeline
        List<Integer> src = new ArrayList<>(List.of(1, 2, 3));
        try { src.stream().forEach(x -> { if (x == 2) src.add(99); }); }
        catch (ConcurrentModificationException e) { System.out.println("CME"); }

        // stateful lambda: wrong
        List<Integer> sink = new ArrayList<>();
        IntStream.range(0, 5).forEach(sink::add);
        System.out.println(sink);
        // unordered source: HashSet
        TreeSet<Integer> sorted = Stream.of(5, 3, 8)
            .collect(Collectors.toCollection(TreeSet::new));
        System.out.println(sorted);
        // infinite stream with limit
        System.out.println(Stream.generate(() -> "z").limit(3).collect(Collectors.joining()));
        // reduce with identity and combiner
        System.out.println(Stream.of("a", "bb", "ccc")
            .reduce(0, (acc, s) -> acc + s.length(), Integer::sum));
        // primitive boxed
        System.out.println(IntStream.rangeClosed(1, 5).mapToObj(Integer::toString)
            .collect(Collectors.joining("*")));
        System.out.println(LongStream.rangeClosed(1, 20).reduce(1L, (a, b) -> a * b));
    }
}
```

Output TERMINAL

```
$ java Str2.java
visit a
visit b
b
before sort 3
before sort 1
before sort 2
after sort 1
p1 a
p2 A
p1 b
p2 B
CME
[0, 1, 2, 3, 4]
[3, 5, 8]
zzz
6
1*2*3*4*5
2432902008176640000
```

What to notice

- `findFirst` stopped as soon as the first element that matched (`b`) arrived: `visit c` and `visit d` never printed.
- With `sorted()`, all three "before sort" lines print before any "after sort" line, and `limit(1)` then needed only one element downstream. Every element had to be pulled upstream first. This is why `sorted()` on a huge or infinite stream is a trap.
- Vertical processing is visible in the third block: `p1 a, p2 A, p1 b, p2 B`.
- Modifying the source collection while the pipeline runs is **interference** and breaks the contract. For an `ArrayList` source the stream throws `ConcurrentModificationException` (the message is "CME" in our output, printed by the catch block). Do not rely on this detection; just do not do it.
- `forEach(sink::add)` into an `ArrayList` works sequentially but is a side effect on shared state; on a parallel stream it would corrupt the list. Use `collect` or `toList()`.

The rules for behavioural parameters (lambdas you pass in) are short and worth memorising:

- They should be **non-interfering**: do not modify the stream's source during the pipeline.
- They should be **stateless**: the result must not depend on anything that may change during execution, such as a counter or a `HashSet` you are filling. Otherwise results differ between sequential and parallel runs and between runs.
- Side effects are best confined to `forEach` at the end; `peek` is for debugging.

7.3.6 Encounter order

A stream has an **encounter order** if its source does (a `List`, an array, `Stream.of`; not a `HashSet`). `sorted` imposes one; `unordered()` removes the requirement. Most operations preserve it. This matters in two places: `findFirst`, `limit` and `takeWhile` are defined in terms of order (so they are costlier on parallel streams), and `forEach` on a parallel stream does not respect it (`forEachOrdered` does, at a price).

7.3.7 toList() versus collect(Collectors.toList())

`Stream.toList()` (Java 16) returns an **unmodifiable** list and permits `null` elements; the last line of the first program showed `UnsupportedOperationException` on `add`. `collect(Collectors.toList())` returns "a `List`" with no guarantees about mutability or type (in practice an `ArrayList`). Use `toList()` by default; use `collect(Collectors.toCollection(ArrayList::new))` when you really need a mutable list. Both are covered in the next chapter.

7.3.8 Streams versus loops

Streams shine for filter-transform-collect pipelines, grouping and aggregation. A plain loop is better when you need to mutate local state, early-`return` from a method, handle checked exceptions, or when the pipeline would have more than five or six steps with nested lambdas. Streams are not faster: for small collections an equivalent loop is typically the same speed or quicker; the benefit is readability. The final chapter of this Part returns to this trade-off.

WATCH OUT

Do not use a stream when you only need `forEach`. `list.stream().forEach(...)` is just `list.forEach(...)` with extra steps. And never write `stream.map(x -> { sideEffect(x); return x; })` expecting it to run: with a terminal operation like `count()` the JVM may legitimately skip the whole pipeline when it can compute the answer directly from the source size (since Java 9 `list.stream().map(...).count()` does not run `map` on a sized source).

INTERVIEW ANGLE

What is the difference between intermediate and terminal operations, and why are streams lazy? Intermediate operations return a new stream and only describe the work; the terminal operation triggers execution. Laziness lets the library fuse the stages into one pass with no intermediate collections, and short-circuit (`limit`, `findFirst`, `anyMatch`) so only as many elements are processed as needed. It also makes infinite streams usable.

INTERVIEW ANGLE

Can you reuse a stream? What is the difference between `map` and `flatMap`? No: a stream can be consumed only once; a second terminal operation throws `IllegalStateException`. `map` transforms each element one-to-one; `flatMap` maps each element to a stream and flattens the results into one, so it is one-to-many.

Check yourself

1. What does this print? `Stream.of(1, 2, 3).peek(x -> System.out.print(x)).map(x -> x * 2);`
2. What does this print? `Stream.of(5, 6, 7, 1, 8).takeWhile(x -> x > 4).toList()` and what would `filter(x -> x > 4)` give instead?
3. A colleague writes `Stream<String> s = list.stream(); s.forEach(...); s.forEach(...);`. What happens?
4. Why is `Stream.iterate(1, x -> x + 1).sorted().limit(3).toList()` a bug?
5. Choose `map` or `flatMap`: turning `List<Order>` into all `OrderLine`s when `Order.lines()` returns a `List<OrderLine>`.
6. Why is `IntStream.rangeClosed(1, 5).sum()` preferable to `Stream.of(1,2,3,4,5).reduce(0, Integer::sum)`?

Answers

1. Nothing. There is no terminal operation, so no element is ever processed.
2. `[5, 6, 7]`; `filter` would give `[5, 6, 7, 8]` because it keeps testing after the first failure.
3. The second `forEach` throws `IllegalStateException: stream has already been operated upon or closed`.
4. `sorted()` is a barrier needing all elements, and the source is infinite, so it never finishes (it eventually runs out of memory).
5. `flatMap(o -> o.lines().stream())`, because each order maps to many lines.
6. It uses primitive `int` all the way (no boxing), states the intent directly and is shorter.

KEY POINTS

A stream is a lazy, one-shot pipeline: source, intermediate operations, one terminal operation.

- Nothing runs until the terminal operation; elements flow through the chain one at a time and short-circuit operations stop early.
- `map` is one-to-one, `flatMap` is one-to-many, `mapMulti` is the push-style alternative; `takeWhile`/`dropWhile` act on prefixes.
- Stateful operations (`sorted`, `distinct`) are barriers; lambdas must be non-interfering and stateless.
- Prefer `toList()`, primitive streams and collectors over manual mutation; use a loop when a loop is clearer.

7.4 Collectors

`collect` is the terminal operation that turns a stream back into something concrete: a list, a map, a string, a summary. It takes a **Collector**, a recipe for mutable reduction: create a container, add elements to it, merge containers (for parallel runs), and optionally transform the container into the final result. The `Collectors` utility class provides about forty ready-made recipes. Knowing five of them well covers most of what a backend developer writes: `toList`/`toSet`, `toMap`, `groupingBy`, `partitioningBy` and `joining`.

The examples use a small bookshop.

Col1.java JAVA

```java
import java.util.*;
import java.util.stream.*;

public class Col1 {
    enum Category { FICTION, SCIENCE, HISTORY }
    record Book(String title, String author, Category cat, int pages, double price) {}

    static void show(Object o) { System.out.println(o); }

    public static void main(String[] args) {
        List<Book> books = List.of(
            new Book("Dune", "Herbert", Category.FICTION, 612, 12.5),
            new Book("Emma", "Austen", Category.FICTION, 474, 8.0),
            new Book("Cosmos", "Sagan", Category.SCIENCE, 396, 15.0),
            new Book("Origin", "Darwin", Category.SCIENCE, 502, 11.0),
            new Book("SPQR", "Beard", Category.HISTORY, 608, 18.0),
            new Book("Persuasion", "Austen", Category.FICTION, 249, 7.5));

        // groupingBy: classifier only -> Map<K, List<T>>
        Map<Category, List<String>> titlesByCat = books.stream().collect(
            Collectors.groupingBy(Book::cat, TreeMap::new,
                Collectors.mapping(Book::title, Collectors.toList())));
        show(titlesByCat);
```

Col1.java (continued) JAVA

```
        // downstream collectors
        show(books.stream().collect(
            Collectors.groupingBy(Book::cat, TreeMap::new, Collectors.counting())));
        show(books.stream().collect(
            Collectors.groupingBy(Book::cat, TreeMap::new, Collectors.averagingInt(Book::pages))));
        Map<Category, Optional<Book>> priciest = books.stream().collect(
            Collectors.groupingBy(Book::cat, TreeMap::new,
                Collectors.maxBy(Comparator.comparingDouble(Book::price))));
        show(priciest.get(Category.FICTION).map(Book::title).orElse("?"));

        // partitioningBy: always two keys
        Map<Boolean, Long> longVsShort = books.stream()
            .collect(Collectors.partitioningBy(b -> b.pages() > 450, Collectors.counting()));
        show(longVsShort);

        // joining
        show(books.stream().map(Book::title).sorted()
            .collect(Collectors.joining(", ", "{", "}")));

        // summarizing
        DoubleSummaryStatistics st = books.stream()
            .collect(Collectors.summarizingDouble(Book::price));
        System.out.printf("min=%.1f max=%.1f avg=%.2f sum=%.1f n=%d%n",
            st.getMin(), st.getMax(), st.getAverage(), st.getSum(), st.getCount());

        // toMap and its duplicate-key pitfall
        try {
            books.stream().collect(Collectors.toMap(Book::author, Book::title));
        } catch (IllegalStateException e) { show(e.getMessage()); }
        Map<String, String> merged = books.stream().collect(
            Collectors.toMap(Book::author, Book::title, (a, b) -> a + "|" + b, TreeMap::new));
        show(merged);

        // teeing: two collectors, one pass
        var minMax = books.stream().collect(Collectors.teeing(
            Collectors.minBy(Comparator.comparingInt(Book::pages)),
            Collectors.maxBy(Comparator.comparingInt(Book::pages)),
            (mn, mx) -> mn.get().title() + ".." + mx.get().title()));
        show(minMax);

        // collectingAndThen, toUnmodifiableList, filtering, flatMapping
        List<String> frozen = books.stream().map(Book::author).distinct()
            .collect(Collectors.collectingAndThen(Collectors.toList(),
                                                  Collections::unmodifiableList));
        show(frozen);
        show(books.stream().collect(
            Collectors.groupingBy(Book::cat, TreeMap::new,
                Collectors.filtering(b -> b.price() > 10, Collectors.counting()))));

        // groupingBy produces HashMap (order unspecified) unless you pass a factory
        Map<String, Integer> pagesByAuthor = books.stream().collect(
            Collectors.groupingBy(Book::author, TreeMap::new, Collectors.summingInt(Book::pages)));
        show(pagesByAuthor);
    }
}
```

Output TERMINAL

```
$ java Col1.java
{FICTION=[Dune, Emma, Persuasion], SCIENCE=[Cosmos, Origin], HISTORY=[SPQR]}
{FICTION=3, SCIENCE=2, HISTORY=1}
{FICTION=445.0, SCIENCE=449.0, HISTORY=608.0}
Dune
{false=2, true=4}
{Cosmos, Dune, Emma, Origin, Persuasion, SPQR}
min=7.5 max=18.0 avg=12.00 sum=72.0 n=6
Duplicate key Austen (attempted merging values Emma and Persuasion)
{Austen=Emma|Persuasion, Beard=SPQR, Darwin=Origin, Herbert=Dune, Sagan=Cosmos}
Persuasion..Dune
[Herbert, Austen, Sagan, Darwin, Beard]
{FICTION=1, SCIENCE=2, HISTORY=1}
{Austen=723, Beard=608, Darwin=502, Herbert=612, Sagan=396}
```

7.4.1 groupingBy: the workhorse

`groupingBy(classifier)` builds a `Map<K, List<T>>`: the classifier function computes the key for each element, equal keys share a bucket. The real power is the second argument, a **downstream collector** applied to each bucket. The elements of a group are not just collected into a list; they flow into whatever collector you give:

- `counting()` gives `Map<K, Long>` (group sizes).
- `summingInt(f)`, `averagingInt(f)`, `summarizingDouble(f)` give per-group statistics.
- `mapping(f, downstream)` transforms elements before collecting them (titles instead of books).
- `filtering(p, downstream)` and `flatMapping(f, downstream)` (Java 9) filter or flatten inside the group. Note the difference from a `filter` before the grouping: `filtering` keeps a group with a count of 0, a pre-`filter` drops the group entirely. In the output above, `HISTORY=1` stays even though only one book matched; with a pre-filter on a category that had no match the key would be missing.
- `maxBy(comparator)` / `minBy` give `Optional<T>` per group.
- Another `groupingBy` gives multi-level grouping, `Map<K1, Map<K2, V>>`.

The three-argument form `groupingBy(classifier, mapFactory, downstream)` lets you choose the map type. The default is a `HashMap`, with no ordering guarantee; the listing used `TreeMap::new` to keep the output stable. An `EnumMap` factory (`() -> new EnumMap<>(Category.class)`) is the right choice for an enum key.

> **GOTCHA**
> If you pass `collect(groupingBy(f, TreeMap::new, ...))` directly as the argument of `System.out.println(...)`, the compiler reports "inference variable R has incompatible bounds", because `println` is overloaded and inference gets confused. That is why `Col1` prints through a tiny `show(Object)` helper. In your own code, assign the result to a typed variable first.

7.4.2 partitioningBy

`partitioningBy(predicate)` is a `groupingBy` with a boolean classifier, with two differences: the map always contains both keys `false` and `true` (even when a side is empty, with an empty list or count 0), and it accepts a downstream collector in the same way. Use it when you want "the ones that match and the ones that do not".

7.4.3 joining, summarizing, collectingAndThen

`joining(delimiter, prefix, suffix)` concatenates `CharSequence` elements using a `StringBuilder`, so map to strings first. `summarizingInt/Long/Double` return a statistics object with count, sum, min, max and average in one pass. `collectingAndThen(downstream, finisher)` applies a function to the final result: the classic use is wrapping a list in an unmodifiable view. Since Java 10 there are also `toUnmodifiableList/Set/Map` and, since 16, `Stream.toList()`.

7.4.4 toMap and its pitfalls

`toMap(keyFn, valueFn)` is convenient and has three well-known traps, two of which the listing triggers:

1. **Duplicate keys** throw `IllegalStateException: Duplicate key Austen (attempted merging values Emma and Persuasion)`. If duplicates are possible, provide a *merge function* as the third argument: `(a, b) -> a + "|" + b`, or `(a, b) -> a` to keep the first.
2. **Null values** throw `NullPointerException` (the implementation uses `HashMap.merge`). Wrap with `Optional` or filter them out first. (`Col2` below demonstrates this.)
3. **The result map is a** `HashMap` with unspecified order. Use the four-argument form with `TreeMap::new` or `LinkedHashMap::new` when order matters.

If you also need to group and the values are lists, prefer `groupingBy`, which handles duplicate keys by design.

7.4.5 teeing: two collectors, one pass

`Collectors.teeing(c1, c2, merger)` (Java 12) feeds every element to two collectors and combines their results. It removes the need to traverse a stream twice, which is impossible anyway since a stream is one-shot. The listing used it to get the shortest and longest book in one pass; common uses are "count and sum", "min and max", or `(sum, count)` to compute an average by hand.

7.4.6 Writing your own Collector

A `Collector<T, A, R>` has four functions: **supplier** (create the mutable container `A`), **accumulator** (add one element `T`), **combiner** (merge two containers, used by parallel streams) and **finisher** (convert `A` to the result `R`). `Collector.of(...)` builds one from lambdas. If the container is already the result, use the overload without a finisher; it marks the collector `IDENTITY_FINISH`.

Col2.java JAVA

```java
import java.util.*;
import java.util.function.*;
import java.util.stream.*;

public class Col2 {
    /** Collects strings into a "a;b;c" string, built by hand to show the four parts. */
    static Collector<String, StringJoiner, String> semicolon() {
        return Collector.of(
            () -> new StringJoiner(";"),          // supplier: new mutable container
            StringJoiner::add,                    // accumulator: fold one element in
            StringJoiner::merge,                  // combiner: merge two containers (parallel)
            StringJoiner::toString);              // finisher: container -> result
    }

    /** Keeps the N largest values. */
    static Collector<Integer, ?, List<Integer>> topN(int n) {
        return Collector.of(
            () -> new PriorityQueue<Integer>(),
            (pq, x) -> { pq.add(x); if (pq.size() > n) pq.poll(); },
            (a, b) -> { a.addAll(b); while (a.size() > n) a.poll(); return a; },
            pq -> pq.stream().sorted(Comparator.reverseOrder()).toList());
    }

    public static void main(String[] args) {
        System.out.println(Stream.of("a", "b", "c").collect(semicolon()));
        System.out.println(Stream.of(5, 1, 9, 3, 7, 8).collect(topN(3)));
        System.out.println(IntStream.rangeClosed(1, 1000).parallel().boxed().collect(topN(3)));
        System.out.println(semicolon().characteristics());
        // toMap with null value
        try { Stream.of("a").collect(Collectors.toMap(s -> s, s -> null)); }
        catch (NullPointerException e) { System.out.println("toMap: null value -> NPE"); }
        // groupingBy ignores null? no: null key is rejected
        try { Stream.of("a").collect(Collectors.groupingBy(s -> (String) null)); }
        catch (NullPointerException e) { System.out.println("groupingBy: " + e.getMessage()); }
        // unmodifiable
        List<Integer> u = Stream.of(1, 2).collect(Collectors.toUnmodifiableList());
        try { u.add(3); }
        catch (UnsupportedOperationException e) { System.out.println("unmodifiable"); }
    }
}
```

Output TERMINAL

```
$ java Col2.java
a;b;c
[9, 8, 7]
[1000, 999, 998]
[]
toMap: null value -> NPE
groupingBy: element cannot be mapped to a null key
unmodifiable
```

What to notice

- `topN` keeps a bounded min-heap: every time the heap grows past `n`, the smallest element is removed. The combiner merges two heaps and trims, which is why the parallel run over 1000 numbers returns the same `[1000, 999, 998]`.
- A correct combiner is not optional. If it were wrong (say, it returned the first container), the sequential run would still look fine and only the parallel one would give wrong answers. Test collectors with `.parallel()`.
- `groupingBy` rejects a `null` key with `element cannot be mapped to a null key`.
- The empty characteristics set `[]` means: not concurrent, not unordered, and the finisher is needed.

TIP

Before writing a custom collector, check whether `teeing`, `mapping`, `reducing`, `collectingAndThen` or a chain of existing ones already does the job. Custom collectors are mainly for a library or for a hot path where a purpose-built accumulator matters. Java 24's gatherers (see the last chapter of this Part) cover the "intermediate operation" side that collectors cannot.

REAL WORLD

Reports and API responses are mostly `groupingBy` pipelines: "orders per customer", "revenue per day", "top 5 products per category". In a Spring service you typically convert entities to DTO records in a `map`, group, and return the `Map<String, List<Dto>>` as JSON. If the data lives in a database, do the aggregation in SQL: a stream pipeline over one million rows loaded into memory is a code smell.

INTERVIEW ANGLE

How do you group a list of employees by department and get the average salary per department? `employees.stream().collect(Collectors.groupingBy(Employee::dept, Collectors.averagingDouble(Employee::salary)))` gives `Map<Dept, Double>`. The first argument classifies, the second is a downstream collector applied per group. Pass `TreeMap::new` as the middle argument if you need sorted keys.

INTERVIEW ANGLE

What can go wrong with `Collectors.toMap`? Duplicate keys throw `IllegalStateException` unless a merge function is given; a `null` value throws `NullPointerException`; the map type is an unspecified `HashMap` unless you pass a supplier.

Check yourself

1. What does `Stream.of("a", "bb", "cc").collect(Collectors.groupingBy(String::length, Collectors.counting()))` return?
2. What is the difference between `groupingBy(f)` and `partitioningBy(p)` regarding keys in the result when no element satisfies `p`?
3. What goes wrong here, and how do you fix it? `Stream.of("x", "x").collect(Collectors.toMap(s -> s, s -> 1))`
4. Name the four functions of a `Collector`. Which one only matters for parallel streams?
5. What does `r` hold? `String r = Stream.of(1, 2, 3, 4).collect(Collectors.teeing(Collectors.counting(), Collectors.summingInt(i -> i), (n, s) -> s + "/" + n));`
6. Why is `groupingBy(Employee::dept)` a bad choice if `dept` may be `null`?

Answers

1. A `Map<Integer, Long>` equal to `{1=1, 2=2}`.
2. `partitioningBy` always has both `true` and `false` keys (the empty side maps to an empty list / zero); `groupingBy` only has keys for which at least one element exists.
3. `IllegalStateException: Duplicate key x`. Pass a merge function: `Collectors.toMap(s -> s, s -> 1, Integer::sum)`.
4. Supplier, accumulator, combiner, finisher. The combiner is used to merge partial results of parallel streams.
5. `10/4`.
6. A `null` classifier result throws `NullPointerException: element cannot be mapped to a null key`. Map nulls to a sentinel such as `"UNKNOWN"` first.

KEY POINTS

A `Collector` is a four-part recipe (supplier, accumulator, combiner, finisher) for mutable reduction.

- `groupingBy` with a downstream collector (`counting`, `summingInt`, `mapping`, `filtering`...) covers most aggregation; pass a map factory for ordering.
- `partitioningBy` always yields both `true` and `false`; `joining` builds strings; `teeing` computes two things in one pass.
- `toMap` fails on duplicate keys and null values unless you handle them; `toList()` on a stream gives an unmodifiable list.
- Custom collectors need a correct combiner; test them with `.parallel()`.

7.5 Parallel Streams and Performance

Adding `.parallel()` to a stream looks like free speed: one word, and the pipeline runs on all cores. Sometimes it is. Often it is slower, and occasionally it silently returns the wrong answer. This chapter explains what actually happens when a stream goes parallel, so that you can predict which of the three outcomes you will get instead of guessing.

7.5.1 What .parallel() really does

A parallel stream changes only the *execution strategy* of the terminal operation. The pipeline you wrote stays the same. When the terminal operation starts, the runtime does three things.

1. **Split.** The source's `Spliterator` is asked to split itself with `trySplit()`, again and again, into smaller chunks, until the chunks are small enough or there are enough of them to keep all workers busy.
2. **Process.** Each chunk runs the whole pipeline (source to last intermediate operation, plus a partial terminal operation) as a task in a `ForkJoinPool`.
3. **Combine.** Partial results are merged pairwise on the way back up, using the *combiner* of the terminal operation: the combiner of a `Collector`, the second function of a three-argument `reduce`, and so on.

This is the same fork/join idea you will meet in the concurrency Part: divide, run independently, merge. A parallel stream is a fork/join computation whose "divide" step comes from the source and whose "merge" step comes from the terminal operation.

Which pool, how many workers? TERMINAL

```
$ java Par1.java
cores: 2
common pool parallelism: 1
threads used: [ForkJoinPool.commonPool-worker-#, main]
```

Output above is from the build machine, which has only 2 cores (typical output elsewhere will differ). By default, parallel streams use the JVM-wide **common pool**, `ForkJoinPool.commonPool()`, whose parallelism defaults to the number of cores minus one. The calling thread helps too, which is why `main` appears in the list of threads: with one worker thread plus the caller, both cores are busy. Note what this implies: *every* parallel stream in the JVM, including those inside libraries and frameworks, shares this single pool.

You can change the common pool's size with `-Djava.util.concurrent.ForkJoinPool.common.parallelism=N` at startup. You cannot change it later.

7.5.2 Ordering: what stays ordered and what does not

A common worry is that parallel means "random order". It does not, for most operations. A stream from a `List`, an array or `IntStream.range` is **ordered**, and the library preserves *encounter order* of the result for operations that are defined in terms of it: `collect` into a list, `toList()`, `findFirst`, `limit`, `skip`, `sorted`. What is *not* ordered is the **side effects**: the order in which `forEach` or `peek` lambdas happen to run.

Par1.java (excerpt) JAVA

```
System.out.print("forEach        : ");
IntStream.rangeClosed(1, 10).parallel().forEach(i -> System.out.print(i + " "));
System.out.println();
System.out.print("forEachOrdered : ");
IntStream.rangeClosed(1, 10).parallel().forEachOrdered(i -> System.out.print(i + " "));
System.out.println();
System.out.println(IntStream.rangeClosed(1, 10).parallel().boxed().toList());
```

Typical output (the forEach line changes from run to run) TERMINAL

```
$ java Par1.java
forEach        : 6 7 9 10 8 3 4 5 1 2
forEachOrdered : 1 2 3 4 5 6 7 8 9 10
[1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
```

`forEachOrdered` forces the action to run in encounter order, but it gives away much of the benefit: elements may be computed in parallel, yet their actions are serialised and buffered. Use it when you must print or write in order; prefer collecting into a list and then iterating when you can.

Respecting order has a price. `limit(n)`, `skip(n)`, `findFirst` and `distinct` on an ordered parallel stream must coordinate chunks so that "the first n" really are the first n. If order does not matter to you, declare it with `unordered()`; the library can then use cheaper algorithms (for example `limit` may take *any* n elements, and `distinct` can use a concurrent set). `findAny` exists for the same reason: it may return any element, so it needs no coordination.

7.5.3 The rules your lambdas must obey

Parallel streams make three promises, and each is conditional on your code. Break the condition and the promise is gone.

Rule 1: no shared mutable state

Lambdas passed to `map`, `filter`, `forEach` and friends run on several threads at once. If they write to a plain `ArrayList`, a `HashMap` or a counter, you have a data race (a topic of the concurrency Part).

Race.java JAVA

```java
import java.util.*;
import java.util.stream.*;

public class Race {
    public static void main(String[] args) {
        for (int run = 1; run <= 5; run++) {
            List<Integer> bad = new ArrayList<>();
            try {
                IntStream.range(0, 1_000_000).parallel().forEach(bad::add);
                System.out.println("run " + run + ": size = " + bad.size());
            } catch (Exception e) {
                System.out.println("run " + run + ": " + e.getClass().getSimpleName()
                    + " (size " + bad.size() + ")");
            }
        }
    }
}
```

Typical output with three common-pool workers (numbers vary every run) TERMINAL

```
$ java -Djava.util.concurrent.ForkJoinPool.common.parallelism=3 Race.java
run 1: size = 632690
run 2: size = 705532
run 3: size = 569106
run 4: size = 387279
run 5: size = 592587
```

One million elements were added; between 39 and 71 percent survived, and no exception was thrown. Two threads read the same `size`, both write to the same slot, and one of the adds is lost. Sometimes the same race instead throws `ArrayIndexOutOfBoundsException` when the backing array is being grown. Worse, with a small input and few cores the race often does not show at all: on our two-core machine, 100 000 elements gave the correct size on every try. Code that "works on my laptop" is the typical way this bug reaches production.

The fix is not a synchronised list. It is to let the stream do the accumulating: `collect(Collectors.toList())` or `toList()`. Each worker fills its own private container and the combiner merges them, with no locks.

Rule 2: reduce functions must be associative, and the identity must be a true identity

`reduce(identity, op)` may be evaluated as `op(op(a, b), op(c, d))` instead of `op(op(op(a, b), c), d)`. That is only the same result when `op` is **associative**. The `identity` value is also used once per chunk, so it must satisfy `op(identity, x) == x`.

Par1.java (excerpt) JAVA

```java
System.out.println("sequential: " + IntStream.rangeClosed(1, 100).boxed()
    .reduce(0, (a, b) -> a - b));
System.out.println("parallel  : " + IntStream.rangeClosed(1, 100).boxed().parallel()
    .reduce(0, (a, b) -> a - b));
System.out.println("sequential: " + Stream.of(1, 2, 3, 4).reduce(10, Integer::sum));
System.out.println("parallel  : " + Stream.of(1, 2, 3, 4).parallel().reduce(10, Integer::sum));
```

Typical output (parallel results depend on how the source was split) TERMINAL

```
$ java Par1.java
sequential: -5050
parallel  : 0
sequential: 20
parallel  : 50
```

Subtraction is not associative, so the parallel result is meaningless. The second pair is subtler: `10` is not an identity for addition (`0` is), so it is added once per chunk: sequentially once (giving 20), in parallel four times here (giving 50). A sequential run hides both mistakes. Rule of thumb: if you can write the same fold with `Collectors` or with `sum`, `max`, `count`, do that; hand-written `reduce` is where parallel bugs live.

Rule 3: behavioural parameters should be stateless and non-interfering

A lambda in a parallel pipeline must not depend on how many times it was called before, and must not modify the source while the stream runs. The first requirement is what makes `map(x -> counter++)` wrong; the second is what makes `list.stream().forEach(x -> list.add(x))` throw `ConcurrentModificationException` even sequentially. The previous chapter covered both; parallelism simply turns "wrong" into "wrong and random".

7.5.4 When does parallel help?

Splitting, scheduling and merging cost something. A parallel stream wins only when the useful work outweighs that overhead. A simple rule of thumb is **N × Q**: the number of elements times the cost per element. If `N × Q` is large, parallelism pays. Five factors decide it.

Factor	Good for parallel	Bad for parallel
Work per element	CPU-heavy (primes, image filters, parsing, hashing)	Trivial (`x + 1`, getters)
Number of elements	Many (thousands and up for heavy work, millions for light work)	Few
Source structure	`ArrayList`, arrays, `IntStream.range`, `HashSet`/`HashMap` (split well)	`LinkedList`, `Stream.iterate`, `BufferedReader.lines()` (split poorly)
Operations	Stateless: `map`, `filter`, `reduce`, `collect`	Order-bound or stateful: `limit`, `findFirst`, `sorted`, `distinct`
Boxing	Primitive streams (`IntStream`)	`Stream<Integer>` with many boxes

Factors that decide whether a parallel stream is faster.

Here are measurements from the build machine (2 cores, JDK 21; best of five runs after one warm-up; typical figures, not benchmarks you should quote).

Par2.java (excerpt) JAVA

```java
static boolean isPrime(long n) {
    if (n < 2) return false;
    for (long d = 2; d * d <= n; d++) if (n % d == 0) return false;
    return true;
}
```

The timed calls, each wrapped in a small `time(Runnable)` helper that warms up once and keeps the best of five runs:

Par2.java (excerpt, inside main) JAVA

```java
// CPU-heavy: count primes below 400000
LongStream.range(0, 400_000).filter(Par2::isPrime).count();
LongStream.range(0, 400_000).parallel().filter(Par2::isPrime).count();
// Cheap: sum 10 million ints
IntStream.range(0, 10_000_000).sum();
IntStream.range(0, 10_000_000).parallel().sum();
// Same boxed sum over 2 million Integers, parallel, two different sources
arrayList.parallelStream().mapToLong(i -> i).sum();
linkedList.parallelStream().mapToLong(i -> i).sum();
```

Typical output (measured on the build machine, 3 common-pool workers on 2 cores) TERMINAL

```
$ java -Djava.util.concurrent.ForkJoinPool.common.parallelism=3 Par2.java
CPU-heavy (count primes below 400000):
  sequential ms: 376
  parallel   ms: 357
Cheap work (sum 10 million ints):
  sequential ms: 15
  parallel   ms: 4
Boxed sum, ArrayList vs LinkedList source (parallel):
  ArrayList  ms: 6
  LinkedList ms: 29
```

Read these numbers carefully, because they teach three different things.

- **The prime count barely improved (376 to 357 ms).** The work is CPU-heavy, so it should scale, but two things hold it back here. First, the machine has two cores and was also running the harness. Second, the cost per number is wildly uneven (large primes take far longer than even numbers), so one chunk ends up with far more work than another and the others sit idle. Real speed-ups need free cores and evenly sized work; on an 8-core desktop the same code typically does much better.
- **The cheap sum improved even though it is trivial.** `IntStream.range(...).sum()` over primitives splits perfectly (a range is just two numbers) and has no boxing, so the overhead stays low. Do not generalise: 4 ms versus 15 ms is both tiny in absolute terms, and the sequential version here is already fast enough that nobody would care.
- **The source matters.** The same boxed sum over a `LinkedList` took about five times as long as over an `ArrayList`. An `ArrayList` splits in O(1) by index. A `LinkedList` has to walk to find the middle, so its spliterator splits by *copying batches* into arrays first.

WATCH OUT

Never trust a parallel speed-up (or slow-down) from a single `main` run Timings need warm-up, repetition and a quiet machine; the JIT compiler changes the cost of the same loop between its first and fifth run. For real decisions use JMH, the OpenJDK micro-benchmark harness, and measure in an environment shaped like production. And measure the right thing: a parallel stream that is 3x faster in isolation can make a loaded web server slower overall, since it steals cores from other requests.

7.5.5 Pitfalls in production code

The shared common pool

The common pool has few threads and no per-caller isolation. If one request runs a parallel stream containing a blocking call (a database query, an HTTP request), it occupies common-pool workers for as long as that call takes, and every other parallel stream and `CompletableFuture.supplyAsync` without an explicit executor in the JVM queues behind it. This is one of the most common causes of "the whole service froze under load" stories involving parallel streams. Rule: **never block inside a parallel stream.** For I/O-bound fan-out use virtual threads or an executor you control (see the concurrency Part).

Using your own pool

There is no public `parallel(pool)` method. A widely-used trick is to start the terminal operation from inside a task running in your own `ForkJoinPool`; the stream's subtasks then run in that pool.

Par2.java (excerpt) JAVA

```java
ForkJoinPool pool = new ForkJoinPool(4);
String name = pool.submit(() ->
    IntStream.range(0, 100).parallel()
        .mapToObj(i -> Thread.currentThread().getName().replaceAll("\\d+$", "#"))
        .distinct().sorted().collect(Collectors.joining(", "))).get();
System.out.println(name);
pool.shutdown();
```

Output TERMINAL

```
$ java Par2.java
ForkJoinPool-1-worker-#
```

The only thread names seen are `ForkJoinPool-1-worker-N`, not `commonPool`. Be aware that this behaviour is an implementation detail rather than a documented guarantee, so treat it as a pragmatic workaround and not a contract. It also does not solve the underlying issue that the work is CPU-bound and you still need to choose a sensible pool size.

Mixing parallel and sequential

A pipeline is either parallel or sequential as a whole. `parallel()` and `sequential()` just set a flag, and the **last call before the terminal operation wins**, whatever position it appears in. `stream.parallel().map(...).sequential().forEach(...)` runs sequentially from start to end.

Exceptions and debugging

An exception thrown in one worker propagates to the caller, but which one if several fail is not defined. Stack traces show `ForkJoinPool` frames and you lose the simple "one thread, one call stack" picture. Reproduce a problem sequentially first.

INTERVIEW ANGLE

When would you use a parallel stream, and when would you avoid it? Use it when the pipeline is CPU-bound, the data set is large, the source splits cheaply (array, `ArrayList`, range), the operations are stateless and associative, and there are spare cores. Avoid it for small data, I/O or blocking calls (they starve the shared common pool), `LinkedList` or `iterate` sources, order-sensitive operations, and anything that touches shared mutable state. Always measure; in a server also consider the effect on other requests.

REAL WORLD

In Spring Boot services you rarely see `parallelStream()` on request paths, because request threads should not fight over the common pool. Teams that need parallelism for heavy batch work typically use a dedicated executor, `CompletableFuture` with a named pool, or virtual threads for I/O fan-out, and keep streams sequential inside each task.

Check yourself

1. What does the following print the first time you run it, and is the output guaranteed? `IntStream.rangeClosed(1, 5).parallel().forEach(System.out::print);`
2. Why does `Stream.of(1, 2, 3, 4).parallel().reduce(10, Integer::sum)` not return 20 in general?
3. A colleague "fixes" a data race in a parallel `forEach` by switching the target to `Collections.synchronizedList`. Why is this still a poor design?
4. Which is better for parallel processing, an `ArrayList` of 5 million elements or a `LinkedList` of the same, and why?
5. What does this pipeline do: `list.stream().parallel().map(f).sequential().forEach(g)`?
6. Why is a blocking database call inside `parallelStream().forEach(...)` dangerous?

Answers

1. It prints the digits 1 to 5 in some order (e.g. `31245` or `12345`); the order of side effects in `forEach` is not guaranteed on a parallel stream. Use `forEachOrdered` for encounter order.
2. The identity `10` is applied once per chunk, not once overall, and the number of chunks depends on the split. Sequentially you get 20; in parallel you can get 50 or other values. The identity must be `0` for addition.
3. The result is now correct but contended: all workers queue on one lock, so you lose most of the parallel speed-up, and the element order is nondeterministic. Collecting with `toList()` or a `Collector` uses per-worker containers and a combiner.
4. The `ArrayList`: it splits by index in constant time and has good cache locality. A `LinkedList` must traverse to split.

5. The whole pipeline runs sequentially, because the last of `parallel()`/`sequential()` before the terminal operation decides the mode for the entire pipeline.
6. Blocked workers occupy the shared common pool, so other parallel streams and async tasks in the whole JVM stall behind them.

KEY POINTS

`.parallel()` only changes how the terminal operation executes: split the source, run chunks in the common `ForkJoinPool`, merge with the combiner.

- Correctness first: no shared mutable state, associative reductions with a true identity, stateless lambdas.
- Order of results is preserved for ordered sources; the order of side effects is not. `forEachOrdered` and ordered `limit`/`findFirst` cost extra; `unordered()` and `findAny` relax this.
- Parallel helps for large, CPU-bound work on well-splitting sources (arrays, `ArrayList`, ranges) with free cores; it hurts for small or blocking work and `LinkedList`/`iterate` sources.
- The common pool is shared by the whole JVM: never block in it. Measure with JMH, not with one `main` run.

7.6 Stream Gatherers

Intermediate operations such as `map`, `filter` and `limit` are a fixed menu. If you need "group every three elements", "running total" or "drop consecutive duplicates", the menu has nothing to offer, and you end up collecting to a list and looping, which defeats the pipeline. Collectors extend the *terminal* end of a stream; until recently there was no equivalent for the middle. **Stream Gatherers** (JEP 485) fill that gap: a `Gatherer` is a user-defined intermediate operation. They were previewed in Java 22 and 23 and became final in **Java 24**, so they are part of Java 25 LTS.

NOTE

The build machine runs JDK 21, so none of the code in this chapter was executed. Listings are marked "(Java 24+; not run on 21)". They follow the Javadoc for `java.util.stream.Gatherer` and `Gatherers` (Java 25), and the sample results in comments are the ones given in that documentation. Run them yourself on JDK 24 or later before relying on them.

7.6.1 Using the built-in gatherers

You attach a gatherer with the new `Stream.gather(Gatherer)` method. The class `java.util.stream.Gatherers` supplies five ready-made ones.

Factory	What it does
`` `windowFixed(int n)` ``	Groups elements into lists of `n`; the last list may be shorter
`` `windowSliding(int n)` ``	Lists of `n` consecutive elements, each window dropping the oldest and adding the next
`` `fold(initial, folder)` ``	Like `reduce` but as an intermediate step: emits one element, the folded result, at the end
`` `scan(initial, scanner)` ``	Prefix scan: emits every intermediate accumulated value
`` `mapConcurrent(max, mapper)` ``	Maps with up to `max` concurrent virtual threads, keeping encounter order

The built-in gatherers (Java 24+).

GatherDemo.java (Java 24+; not run on 21) JAVA

```java
import java.util.List;
import java.util.stream.Gatherers;
import java.util.stream.Stream;

public class GatherDemo {
    public static void main(String[] args) {
        // batches of three (e.g. for bulk inserts): [[1, 2, 3], [4, 5, 6], [7, 8]]
        System.out.println(Stream.of(1, 2, 3, 4, 5, 6, 7, 8)
            .gather(Gatherers.windowFixed(3)).toList());

        // moving window: [[1, 2], [2, 3], [3, 4], ...]
        System.out.println(Stream.of(1, 2, 3, 4, 5)
            .gather(Gatherers.windowSliding(2)).toList());

        // running totals: [1, 3, 6, 10, 15]
```

GatherDemo.java (Java 24+; not run on 21) (continued) JAVA

```
        System.out.println(Stream.of(1, 2, 3, 4, 5)
            .gather(Gatherers.scan(() -> 0, Integer::sum)).toList());

        // fold emits exactly one element: Optional["12345"]
        System.out.println(Stream.of(1, 2, 3, 4, 5)
            .gather(Gatherers.fold(() -> "", (s, n) -> s + n)).findFirst());
    }
}
```

What to notice: each result is still a stream, so gatherers compose with everything else (`filter(...).gather(...).map(...)`). `windowFixed` is the answer to the classic interview question "split a list into chunks of n" without index arithmetic. `windowSliding` gives you moving averages and "compare each element with the previous one" (adjacent pairs) checks. `scan` and `fold` differ in what they emit: `scan` one output per input, `fold` one output in total.

`mapConcurrent` deserves a note. It runs the mapper on virtual threads, at most `maxConcurrency` at a time, and still emits results in the original order. It is aimed at blocking work (calling a service per element) where a parallel stream would be the wrong tool, since it would block the shared common pool. It is a small, bounded fan-out that does not need `CompletableFuture`.

7.6.2 How a gatherer works

A `Gatherer<T, A, R>` consumes elements of type `T`, may keep private state of type `A`, and pushes elements of type `R` downstream. It is built from up to four functions, which deliberately resemble a `Collector`:

Part	Role
`initializer`	Creates the private state `A` for one run (for example a buffer or a counter). Omit it for stateless gatherers
`integrator`	Called for each element with `(state, element, downstream)`. May push zero, one or many results. Returns `true` to continue, `false` to say "no more input wanted" (short-circuit)
`combiner`	Merges two states, only needed so the gatherer can run in parallel. Omit it to make the gatherer sequential
`finisher`	Called once at the end with `(state, downstream)` to flush anything still buffered

The four parts of a Gatherer.

Two points make this more powerful than a plain `Collector` or `flatMap`. First, the integrator can push *several* elements or none, so it can express filtering, mapping, one-to-many and many-to-one in one place. Second, the integrator returns a boolean, so a gatherer can **short-circuit**, which a `Collector` never can. The factories are `Gatherer.ofSequential(...)` (no combiner, always run sequentially) and `Gatherer.of(...)` (with a combiner, may run in parallel); each has overloads that omit the initializer, the finisher or both.

7.6.3 Writing a custom gatherer

A typical need that `Stream` lacks: remove *consecutive* duplicates (`1 1 2 2 2 1` becomes `1 2 1`). `distinct()` is wrong because it removes the later 1 too. The gatherer needs one piece of state, the previous element.

Dedupe.java (Java 24+; not run on 21) JAVA

```
import java.util.List;
import java.util.Objects;
import java.util.stream.Gatherer;
import java.util.stream.Stream;

public class Dedupe {
    /** Drops an element when it equals the one just before it. */
    static <T> Gatherer<T, ?, T> dropConsecutiveDuplicates() {
        class Last { T value; boolean seen; }          // private state for one run

        return Gatherer.<T, Last, T>ofSequential(
            Last::new,                                 // initializer
            Gatherer.Integrator.ofGreedy((state, element, downstream) -> {
                if (!state.seen || !Objects.equals(state.value, element)) {
                    state.value = element;
                    state.seen = true;
                    return downstream.push(element);    // forward the element
```

Dedupe.java (Java 24+; not run on 21) (continued) JAVA

```
                }
                return true;                            // skip it, keep consuming
            }));
    }

    public static void main(String[] args) {
        List<Integer> out = Stream.of(1, 1, 2, 2, 2, 1, 3, 3)
            .gather(dropConsecutiveDuplicates())
            .toList();
        System.out.println(out);                        // expected: [1, 2, 1, 3]
    }
}
```

What to notice: the state class `Last` is created freshly per stream run by the initializer, so the gatherer object itself is reusable and holds no mutable fields. `downstream.push(x)` sends one result on and returns `false` if the downstream does not want more (a later `limit`, for example), and returning that value from the integrator propagates the short-circuit. `Integrator.ofGreedy` marks an integrator that never stops the stream by itself, which lets the library skip checking the result. Use `Integrator.of` when your integrator can return `false`.

Gatherers also compose with `andThen`, and a gatherer that is stateless can be parallel. For anything else, remember the rule from the parallel chapter: if state depends on *encounter order* (like "previous element"), the gatherer must be sequential, and `ofSequential` states that honestly.

INTERVIEW ANGLE

How do Gatherers differ from Collectors? A `Collector` is a terminal operation: it consumes the whole stream and produces one final result. A `Gatherer` is an intermediate operation: it can emit many elements, keep state between elements, and short-circuit, and the pipeline continues after it. `windowFixed`, `scan` and custom stateful steps are the typical uses. They became final in Java 24 (JEP 485).

Check yourself

1. What is the difference between `Gatherers.scan` and `Gatherers.fold`?
2. What would `Stream.of(1, 2, 3, 4, 5).gather(Gatherers.windowFixed(2)).toList()` return?
3. Why can a `Gatherer` short-circuit while a `Collector` cannot?
4. In which Java release did gatherers become a final feature, and which method attaches one to a stream?
5. When must you create a gatherer with `ofSequential` and not `of`?

Answers

1. `scan` emits every intermediate accumulation (one output per input); `fold` emits only the final accumulated value (exactly one element).
2. `[[1, 2], [3, 4], [5]]`: the last window is smaller.
3. The integrator returns a boolean, and `false` signals "stop sending me input", while a `Collector`'s accumulator returns nothing and the pipeline always feeds it every element.
4. Java 24 (JEP 485); `Stream.gather(Gatherer)`.
5. When the logic depends on encounter order or on state that cannot be merged by a combiner, as with "previous element" state.

KEY POINTS

Gatherers (final in Java 24) are user-defined intermediate stream operations, attached with `Stream.gather`.

- Built-ins: `windowFixed`, `windowSliding`, `fold`, `scan`, `mapConcurrent` in `java.util.stream.Gatherers`.
- A custom gatherer has an optional initializer, an integrator that pushes zero or more outputs and may short-circuit, an optional combiner and an optional finisher.
- Use `ofSequential` when order-dependent state cannot be merged; `of` with a combiner permits parallel evaluation.
- Not runnable on JDK 21: test on 24 or later.

7.7 Functional Style in Practice

Lambdas and streams are tools, and tools can be used well or badly. This closing chapter of the Part is about judgement: when does a functional style make code simpler, when does it make it worse, and how do you handle the awkward cases (checked exceptions, state, errors) that the tutorials skip.

7.7.1 Immutability and pure functions

A **pure function** returns a value that depends only on its arguments and has no side effects (no printing, no writing to fields, no I/O). Pure functions are easy to test (call and compare), easy to reuse, safe to run in parallel and safe to cache. Streams are designed around them: `map`, `filter`, `reduce` and `collect` assume their lambdas are pure, and the parallel chapter showed what happens when they are not.

Immutability supports this. Records, `List.of`, `Map.of`, `Stream.toList()` and `Collections.unmodifiableList` give you values that cannot change under you. A method that receives an immutable list and returns a new list cannot corrupt anything, and a lambda that captures only immutable values is trivially thread-safe. Practical rules:

- Transform instead of mutate: `names.stream().map(String::toUpperCase).toList()` rather than looping and calling `set`.
- Keep side effects at the edges: compute with streams, then print, save or send once, at the end, from ordinary code.
- A lambda that does more than a line or two should become a named method and be passed as a method reference. The name is documentation, and you can unit-test the method on its own.

7.7.2 Composing behaviour

Functional interfaces carry default methods for composition. `Function.andThen` and `compose` chain functions, `Predicate.and`, `or` and `negate` combine conditions, `Comparator.thenComparing` chains sort keys. A function that returns a function is a way to "pre-fill" an argument (currying).

Fx.java (excerpt) JAVA

```java
Function<Integer, Function<Integer, Integer>> add = a -> b -> a + b;
Function<Integer, Integer> add5 = add.apply(5);
Function<Integer, Integer> pipeline = add5.andThen(x -> x * 2).compose(x -> x - 1);
System.out.println(pipeline.apply(10));
```

Output TERMINAL

```
$ java Fx.java
28
```

Read `pipeline` from the inside out: `compose(x -> x - 1)` runs first (10 becomes 9), then `add5` (14), then `andThen(x -> x * 2)` (28). Mixing `compose` and `andThen` in one expression is correct but easy to misread; in your own code prefer one direction.

7.7.3 The checked exception problem

`java.util.function` interfaces declare no checked exceptions. The following does not compile, because `Files.readString` throws `IOException`:

Broken.java JAVA

```java
// does not compile: unreported exception java.io.IOException
paths.stream().map(p -> Files.readString(p)).toList();
```

There is no perfect answer, only trade-offs. These are the options, in the order you should reach for them.

1. **Handle it inside the lambda.** If you can recover (use a default, log and skip), write a `try/catch` in a small named helper method that returns `Optional` or a default, and use a method reference to it.
2. **Wrap in an unchecked exception.** Convert to `UncheckedIOException` or your own runtime exception and let the whole pipeline abort. This is the right choice when the failure means the operation cannot continue.
3. **Return a result type.** Make each step return a value that represents success or failure and decide later. This keeps the pipeline going and lets you report all failures.

The listing below shows options 2 and 3 together.

Fx.java JAVA

```java
import java.io.*;
import java.util.*;
import java.util.function.*;
import java.util.stream.*;

public class Fx {
    @FunctionalInterface
    interface IOFunction<T, R> { R apply(T t) throws IOException; }

    /** Adapts a throwing function into a plain Function that wraps the checked exception. */
    static <T, R> Function<T, R> unchecked(IOFunction<T, R> f) {
        return t -> {
            try { return f.apply(t); }
            catch (IOException e) { throw new UncheckedIOException(e); }
        };
    }

    static int readLength(String name) throws IOException {
        if (name.startsWith("missing")) throw new FileNotFoundException(name);
        return name.length();
    }

    sealed interface Result<T> {
        record Ok<T>(T value) implements Result<T> {}
        record Fail<T>(String error) implements Result<T> {}
    }

    static Result<Integer> attempt(String name) {
        try { return new Result.Ok<>(readLength(name)); }
        catch (IOException e) { return new Result.Fail<>(e.getMessage()); }
    }

    static String describe(Result<Integer> r) {
        return switch (r) {
            case Result.Ok<Integer>(var v) -> "ok " + v;
            case Result.Fail<Integer>(var msg) -> "failed: " + msg;
        };
    }

    public static void main(String[] args) {
        List<String> names = List.of("a.txt", "bb.txt", "missing.txt", "c.txt");

        try {
            names.stream().map(unchecked(Fx::readLength)).forEach(System.out::println);
        } catch (UncheckedIOException e) {
            System.out.println("stopped: " + e.getCause());
        }

        List<Result<Integer>> results = names.stream().map(Fx::attempt).toList();
        results.forEach(r -> System.out.println(describe(r)));
        System.out.println("total of successes = " + results.stream()
            .flatMap(r -> r instanceof Result.Ok<Integer> ok
                ? Stream.of(ok.value()) : Stream.<Integer>empty())
            .mapToInt(Integer::intValue).sum());
    }
}
```

Run it TERMINAL

```
$ java Fx.java
5
6
stopped: java.io.FileNotFoundException: missing.txt
ok 5
ok 6
failed: missing.txt
ok 5
total of successes = 16
```

What to notice: with the wrapper, elements before the failure were processed (5, 6), the failure aborted the stream, and the original checked exception is preserved as the cause. With `Result`, all four names were processed, the failure is just a value, and the sealed interface plus pattern `switch` makes the compiler check that you handled both cases. The result-type approach is how most functional codebases do error handling in pipelines. (The record-pattern `switch` is in

its own method because a `javac` 21 code-generation bug crashed the compiler when the same `switch` sat inside a lambda; this is fixed in later versions, so you usually need not worry.)

WATCH OUT
Do not hide checked exceptions with "sneaky throw" tricks Generic tricks that throw a checked exception without declaring it compile, but callers cannot catch it by type in a way the compiler understands, and tools and colleagues will be surprised. Wrap in an unchecked exception with the original as `cause`.

7.7.4 Streams versus loops

A stream is not automatically better than a loop. Choose by readability, and by what has to happen to the data.

Prefer a stream when	Prefer a loop when
Data flows through filter, map, group, aggregate steps	You need several results from one pass with shared state
The steps read as a sentence	You need `break`, `continue`, labelled jumps or early `return` in the middle
Collectors express the result (group, partition, join)	You modify local variables (streams require effectively final)
The data is large and pure parallelism may help	You throw or handle checked exceptions per element

Choosing between a stream pipeline and a loop.

An honest test: if the pipeline has more than about five stages, or any lambda is multi-line, extract methods or reconsider. Compare these two ways to find the first long name in uppercase:

Style.java JAVA

```java
import java.util.List;
import java.util.Optional;

public class Style {
    static Optional<String> loop(List<String> names) {
        for (String n : names) {
            if (n.length() > 4) {
                return Optional.of(n.toUpperCase());
            }
        }
        return Optional.empty();
    }

    static Optional<String> stream(List<String> names) {
        return names.stream().filter(n -> n.length() > 4).findFirst().map(String::toUpperCase);
    }

    public static void main(String[] args) {
        List<String> names = List.of("Ann", "Charlotte", "Raj", "Benjamin");
        System.out.println(loop(names));
        System.out.println(stream(names));
    }
}
```

Run it TERMINAL

```
$ java Style.java
Optional[CHARLOTTE]
Optional[CHARLOTTE]
```

Both are fine here; the stream version states *what* without index or flag. The advantage grows with the number of stages. Other common mistakes:

- **Side effects in `map` or `peek`** to "do work" (for example `map(x -> { list.add(x); return x; })`). Streams may skip, reorder or repeat such calls (the streams chapter warned that `count()` may skip `map` entirely when it can compute the size directly from the source).
- `stream().forEach(list::add)` instead of `collect` or `toList()`: harmless sequentially, a bug in parallel.
- **Streaming a tiny collection in a hot path.** A stream pipeline allocates several objects. Irrelevant in business code, relevant in a tight inner loop; profile before optimising.
- **Overusing `Optional.get()`, and `Optional` fields**: see the Optional chapter.

- **Boxing in numeric pipelines.** Prefer `mapToInt`, `IntStream` and `sum()` over `Stream<Integer>` with `reduce(0, Integer::sum)`.

REAL WORLD
In a Spring Boot codebase you will see streams in service-layer mapping code (`entities.stream().map(this::toDto).toList()`), in building maps for lookups (`Collectors.toMap`, `groupingBy`), and in tests. You will see plain loops in performance-sensitive and I/O-heavy code, and `Optional` returned by repository methods such as `findById`. Method references to small mapping methods are the idiom that keeps these readable.

Check yourself

1. What does `Function<Integer,Integer> f = x -> x + 1; f.compose((Integer x) -> x * 2).andThen(x -> x * 10).apply(3)` return?
2. Why does `names.stream().map(n -> Files.readString(Path.of(n)))` not compile, and what are two ways to fix it?
3. What is wrong here, and what is the idiomatic fix? `List<String> out = new ArrayList<>(); names.stream().filter(n -> n.length() > 3).forEach(out::add);`
4. Name two situations where a plain loop is the better choice than a stream.
5. Why is a pure function easier to run in a parallel stream?

Answers

1. `compose` runs first: 3 * 2 = 6, then `f`: 7, then `andThen`: 70. The result is 70.
2. `readString` throws the checked `IOException` and `Function.apply` declares none. Fix by catching inside a helper method, wrapping in `UncheckedIOException`, or returning a result type.
3. It mutates an external list from inside the pipeline, which is a data race if the stream becomes parallel. Use `.collect(Collectors.toList())` or `.toList()`.
4. Examples: you need `break`/early exit mixed with several local mutable variables, or per-element checked exception handling, or the logic is simply clearer as a loop.
5. It has no shared state or side effects, so chunks cannot interfere with each other and the order in which they run does not change the result.

KEY POINTS
Prefer pure functions and immutable data in pipelines; keep side effects at the edges.

- Extract non-trivial lambdas into named methods and use method references.
- Checked exceptions: catch in a helper, wrap in an unchecked exception, or return a `Result`-like value.
- Streams are best for filter/map/group/aggregate flows; loops win when you need early exits, several results or local mutation.
- Avoid side effects in `map`/`peek`, and avoid `forEach(list::add)`; use `collect`/`toList()`.

PART 8

Concurrency

Threads, the memory model, locks, executors, futures and virtual threads: how to make a program do several things at once and still be correct.

In this part

8.1 Concurrency Concepts
8.2 Threads
8.3 Races and the Java Memory Model
8.4 synchronized and Monitors
8.5 Thread Confinement, Immutability and Liveness
8.6 Locks and Atomics
8.7 Executors and Thread Pools
8.8 CompletableFuture
8.9 Synchronizers and Concurrent Collections
8.10 Fork/Join and Parallelism
8.11 Virtual Threads
8.12 Structured Concurrency, Scoped Values and Practice

8.1 Concurrency Concepts

Almost every real Java program is concurrent. A web server handles thousands of requests at once; a desktop app keeps its window responsive while it downloads a file; a batch job splits a large input across all CPU cores. Concurrency is also the area of Java where the most subtle bugs live: bugs that appear once a week in production, vanish under a debugger, and corrupt data silently. This chapter builds the vocabulary and the mental model you need before you write your first thread. The rest of this Part builds on it.

8.1.1 Processes and threads

A **process** is a running program. The operating system gives each process its own private virtual address space, its own open files and its own resources. Two processes cannot read each other's memory unless they use an explicit mechanism (pipes, sockets, shared memory). A Java program started with `java Main` is one process containing one JVM.

A **thread** is a path of execution inside a process. All threads of a process share the same memory (in Java: the same heap), but each thread has its own program counter and its own call stack holding its local variables. Every Java program has at least one thread, called `main`, plus several JVM-internal threads (garbage collector workers, JIT compiler threads, the signal dispatcher).

Aspect	Process	Thread
Memory	Own address space, isolated	Shares the heap with other threads of the process
Creation cost	High (OS must set up a new address space)	Lower, but still an OS resource with its own stack (platform threads)
Communication	Pipes, sockets, files, shared memory	Plain reads and writes of shared objects
Failure	A crash is contained to the process	An uncaught error can corrupt state shared by all threads

Process versus thread.

The sharing is what makes threads both powerful and dangerous. Communication between threads is nearly free (you just read a field), but nothing stops two threads from changing the same field at the same moment.

8.1.2 Concurrency versus parallelism

These two words are often used as if they meant the same thing. They do not.

- **Concurrency** is about *structure*: dealing with many things at once. Several tasks are in progress and their execution overlaps in time. A single CPU core can run a concurrent program by switching rapidly between tasks (time slicing).
- **Parallelism** is about *execution*: doing many things at the same instant. It needs more than one CPU core.

A concurrent program may run in parallel when cores are available, but it is still correct on one core. A good mental picture: one cook switching between three pans is concurrent; three cooks each with a pan is parallel.

There are two quite different reasons to use threads:

1. **To hide waiting (I/O-bound work).** A thread that waits for a database or a network reply does not need a CPU. While it waits, other threads can run. Here threads improve throughput and responsiveness even on one core.
2. **To use more cores (CPU-bound work).** Splitting a computation across cores shortens the wall-clock time, up to the limits described next.

Choosing the wrong model is a classic mistake. Adding 200 threads to a CPU-bound task on 4 cores does not make it 50 times faster; it adds scheduling overhead. Adding threads to an I/O-bound task, on the other hand, can multiply throughput until the remote system becomes the bottleneck.

8.1.3 Why concurrent programming is hard

Sequential code has one property you rely on without noticing: statement N finishes before statement N+1 starts, and each statement sees the effects of all earlier ones. With threads, three things break that comfort.

Atomicity

An operation is **atomic** if no other thread can observe it half-done. `count++` looks like one operation but is three: read the field, add one, write it back. Two threads can both read 5 and both write 6, so one increment is lost. You will reproduce this in the next chapters.

Visibility

When one thread writes a variable, there is no guarantee, by default, that another thread will ever see the new value. Each CPU core has caches and registers, and the compiler and the JIT are free to keep a value in a register for the whole length of a loop. Visibility is the property that a write by one thread becomes observable by another.

Ordering

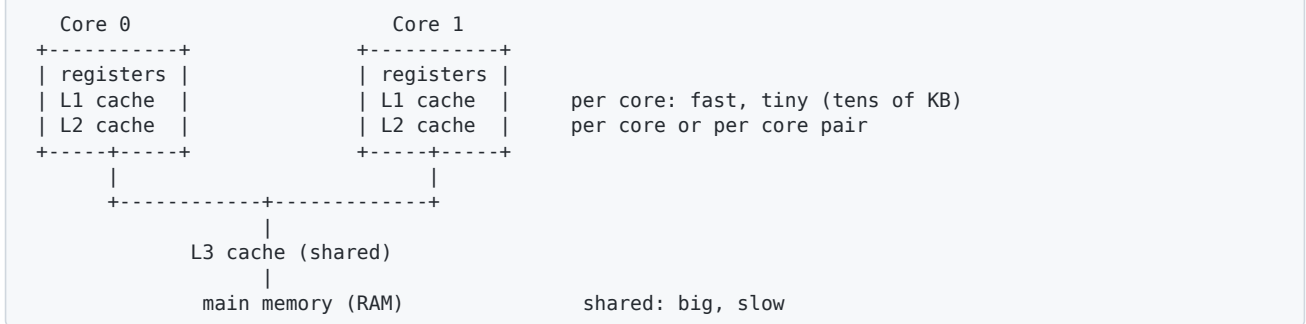
The JIT compiler and the CPU may reorder independent instructions to run faster. Within one thread the result is always as if the code ran in order, but a second thread watching the writes can see them in a different order from the one you wrote. The Java Memory Model (next chapters) defines exactly when you can rely on an order.

These three (atomicity, visibility, ordering) are the entire reason for every tool in this Part: `synchronized`, `volatile`, atomics, locks and concurrent collections each guarantee some combination of them.

8.1.4 CPU caches in one picture

Why can a value be invisible? Because a modern CPU does not read main memory on every access. It reads through small, very fast caches.

Simplified view of a two-core machine



Hardware keeps caches *coherent* (a write on one core eventually invalidates stale copies on others), but "eventually" and "in which order" are exactly what the hardware is allowed to be loose about, in exchange for speed. Typical latencies, as orders of magnitude: a register access is under a nanosecond, L1 about a nanosecond, L3 tens of nanoseconds, main memory around a hundred nanoseconds. A thread that reads a variable repeatedly will happily keep using its cached or register copy.

The practical consequence: Java does not promise that threads share a single consistent "current value" of every variable. It promises something weaker and precisely specified, the Java Memory Model, that you opt into with `synchronized`, `volatile`, `final` and the `java.util.concurrent` classes.

8.1.5 How much faster can parallelism make things? (Amdahl's law)

Suppose a fraction `p` of a job can be parallelised and the rest, `1 - p`, must run sequentially. With `n` cores the best possible speed-up is

DEFINITION

Amdahl's law speedup(n) = 1 / ((1 - p) + p / n) Even with unlimited cores the speed-up is capped at 1 / (1 - p). A job that is 90 percent parallel can never go more than 10 times faster.

Amdahl.java JAVA

```java
public class Amdahl {
    static double speedup(double p, int n) { return 1.0 / ((1 - p) + p / n); }
    public static void main(String[] args) {
        System.out.println("cores available here: "
            + Runtime.getRuntime().availableProcessors() + " (varies)");
        double[] ps = {0.50, 0.90, 0.95};
        int[] ns = {2, 4, 8, 16, 1000};
        for (double p : ps) {
            StringBuilder sb = new StringBuilder(String.format("parallel fraction %.2f:", p));
            for (int n : ns) sb.append(String.format("  n=%d -> %.2fx", n, speedup(p, n)));
            System.out.println(sb);
        }
    }
}
```

Run it TERMINAL

```
$ java Amdahl.java
cores available here: 2 (varies)
parallel fraction 0.50:  n=2 -> 1.33x  n=4 -> 1.60x  n=8 -> 1.78x  n=16 -> 1.88x  n=1000 -> 2.00x
parallel fraction 0.90:  n=2 -> 1.82x  n=4 -> 3.08x  n=8 -> 4.71x  n=16 -> 6.40x  n=1000 -> 9.91x
parallel fraction 0.95:  n=2 -> 1.90x  n=4 -> 3.48x  n=8 -> 5.93x  n=16 -> 9.14x  n=1000 -> 19.63x
```

What to notice: going from 16 to 1000 cores buys a 90-percent-parallel job only 6.4x to 9.9x. The sequential part, often a lock that all threads must pass through, dominates. This is why reducing the time spent holding locks (a theme of the synchronization chapters) matters more than adding hardware. Amdahl's law is optimistic: it ignores the cost of coordination, which grows with the number of threads.

8.1.6 The vocabulary you will meet

Term	Meaning
Race condition	The result depends on the unlucky timing of threads. `count++` from two threads is the textbook case.
Data race	Two threads access the same variable, at least one writes, and nothing orders the accesses. A data race is what the Java Memory Model forbids you to rely on.
Thread-safe	A class that behaves correctly when used from several threads without extra synchronisation by the caller.
Critical section	A block of code that only one thread at a time may execute.
Mutual exclusion	The guarantee that only one thread is inside a critical section.
Deadlock	Two or more threads wait for each other forever, each holding what the other needs.
Starvation	A thread is ready to run but never gets the resource (CPU or a lock) it needs.
Context switch	The OS saves one thread's state and loads another's. Costs microseconds and pollutes CPU caches.
Throughput / latency	Work done per second / time for one piece of work. They often pull in opposite directions.

Terms used throughout this Part.

INTERVIEW ANGLE

What is the difference between concurrency and parallelism? Concurrency is about managing several tasks whose execution overlaps in time, which a single core can do by time slicing. Parallelism is executing several tasks at the same instant on multiple cores. Concurrency is a property of the program's structure; parallelism is a property of its execution. You can have concurrency without parallelism (one core), and the reason you design for concurrency is partly to be able to exploit parallel hardware.

REAL WORLD

In a Spring Boot service every incoming HTTP request is handled on a thread taken from a pool. Your singleton beans, such as services and repositories, are therefore shared by many threads at once. That is why "keep singleton beans stateless" is the first rule of Spring code: a mutable field in a service is a data race waiting to happen.

Check yourself

1. A program has three independent tasks and runs on a machine with one CPU core. Can it be concurrent? Can it be parallel?
2. Name the three problems (the "big three") that make shared-memory threading hard.
3. A job is 80 percent parallelisable. What is the maximum speed-up with unlimited cores? (Use the formula.)
4. What is wrong with this reasoning: "my task is CPU-bound, so I will start 100 threads on my 4-core laptop to make it faster"?
5. What does this print, assuming `count` is a plain `int` field shared by two threads that each run the loop? `for (int i = 0; i < 1000; i++) count++;` Is it always 2000?

Answers

1. It can be concurrent (the OS or JVM time-slices the tasks), but not parallel: parallel execution needs at least two cores.
2. Atomicity (compound operations can interleave), visibility (writes may not be seen by other threads) and ordering (instructions can be reordered).
3. 1 / (1 - 0.8) = 5, so at most 5 times faster, no matter how many cores.
4. A CPU-bound task cannot use more than the 4 cores; the extra threads only add context-switch and memory overhead. The thread count should be about the number of cores.
5. Not always 2000. `count++` is read, add, write; two threads can lose updates, so the result can be anything from 1000 to 2000 and typically varies between runs.

KEY POINTS

Concurrency is the structure of overlapping tasks; parallelism is simultaneous execution on several cores.

- Threads share the heap, which makes communication cheap and bugs easy.
- The three hazards are atomicity, visibility and ordering; every concurrency tool in Java addresses some of them.
- CPU caches and the optimiser mean threads do not automatically see each other's writes.
- Amdahl's law: the sequential fraction caps the speed-up, so shorten critical sections.
- Use threads to hide I/O waits or to spread CPU work over about as many threads as cores.

8.2 Threads

A **thread** in Java is an object of class `java.lang.Thread` that represents an operating-system thread (a *platform thread*; Java 21 adds cheap *virtual threads*, covered in a later chapter). This chapter shows how to create threads, how their life cycle works, how to wait for them, and, most importantly, how to stop them politely.

8.2.1 Creating and starting threads

You tell a thread *what to run* by giving it a `Runnable` (a functional interface with one method, `void run()`), either by passing it to the `Thread` constructor or by subclassing `Thread` and overriding `run`. Passing a `Runnable` is almost always better: it separates the task from the mechanism that runs it, so the same task can later be handed to an executor.

Basics.java JAVA

```java
public class Basics {
    public static void main(String[] args) throws Exception {
        Thread t1 = new Thread(
            () -> System.out.println("lambda in " + Thread.currentThread().getName()));
        t1.start();
        t1.join();
        Thread t2 = new Thread(new Task(), "worker-A");
        t2.start();
        t2.join();
        Thread t3 = new Thread() {
            @Override public void run() { System.out.println("subclass in " + getName()); }
        };
        t3.setName("worker-B");
        t3.start();
        t3.join();
        // calling run() directly does NOT start a thread
        new Thread(new Task(), "never-started").run();
        System.out.println("main is " + Thread.currentThread().getName()
            + ", id>0: " + (Thread.currentThread().threadId() > 0));
    }
    static class Task implements Runnable {
        public void run() { System.out.println("Task in " + Thread.currentThread().getName()); }
    }
}
```

Run it TERMINAL

```
$ java Basics.java
lambda in Thread-0
Task in worker-A
subclass in worker-B
Task in main
main is main, id>0: true
```

What to notice:

- `start()` asks the JVM to create the OS thread and call `run()` on it. The thread that called `start()` continues immediately.
- Calling `run()` directly (the `never-started` line) executes the method on the *calling* thread. The output says `Task in main`. This is one of the most common beginner bugs: no concurrency happens at all.
- A thread can be started only once. A second `start()` throws `IllegalThreadStateException`.
- Unnamed threads get names `Thread-0`, `Thread-1`, and so on. Always name threads you create (constructor argument or `setName`); names show up in thread dumps and logs, which is where you will need them. `Thread.threadId()` (Java 19+) replaces the deprecated `getId()`.

In Java 21 there are also builders: `Thread.ofPlatform().name("worker-", 0).start(task)` and `Thread.ofVirtual().start(task)`. In application code you will usually not create threads by hand at all but submit tasks to an `ExecutorService`, which is the topic of a later chapter. Understanding raw threads is still essential, because executors are built from them.

8.2.2 The thread life cycle

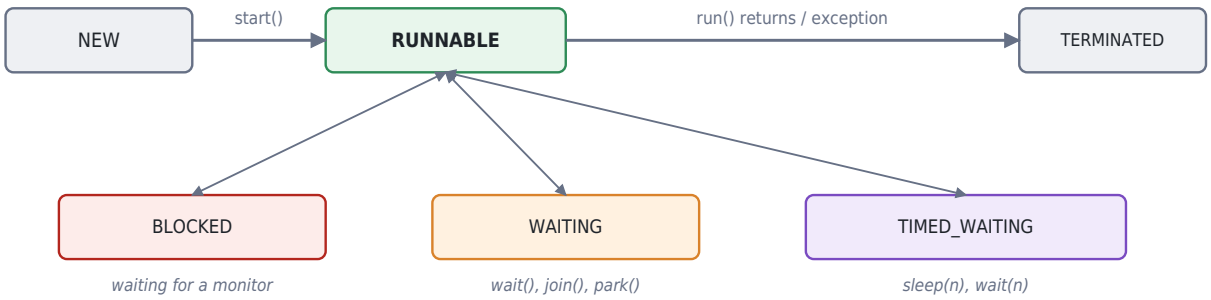


Figure 8.1. The six states of `Thread.State` and the events that move a thread between them. Note that RUNNABLE covers both "running on a CPU" and "ready to run".

`Thread.getState()` returns one of six values:

State	Meaning
NEW	Created but `start()` not yet called.
RUNNABLE	Executing, or ready to execute when the scheduler gives it a core. The JVM does not distinguish "running" from "ready". A thread blocked in native I/O is also reported RUNNABLE.
BLOCKED	Waiting to acquire an intrinsic lock (entering a `synchronized` block or method that another thread holds).
WAITING	Waiting indefinitely for another thread's action: `Object.wait()`, `Thread.join()`, `LockSupport.park()`.
TIMED_WAITING	Waiting with a timeout: `Thread.sleep(ms)`, `wait(ms)`, `join(ms)`, `parkNanos`.
TERMINATED	`run()` has finished (normally or by an exception).

Thread.State values.

The following program drives one thread through every state and prints what `getState()` reports. The `sleep` calls here are only to give the thread time to reach the next state; real code must never coordinate threads with sleeps.

States.java JAVA

```java
public class States {
    static final Object lock = new Object();
    public static void main(String[] args) throws Exception {
        Thread t = new Thread(() -> {
            synchronized (lock) { }
            try { Thread.sleep(300); } catch (InterruptedException e) { }
            synchronized (lock) {
                try { lock.wait(); } catch (InterruptedException e) { }
            }
        }, "demo");
        System.out.println("before start : " + t.getState());
        synchronized (lock) {
            t.start();
            Thread.sleep(100);
            System.out.println("lock held    : " + t.getState());
        }
        Thread.sleep(100);
        System.out.println("sleeping     : " + t.getState());
        Thread.sleep(400);
        System.out.println("in wait()    : " + t.getState());
        synchronized (lock) { lock.notify(); }
```

States.java (continued) JAVA

```
        t.join();
        System.out.println("after finish : " + t.getState());
    }
}
```

Run it TERMINAL

```
$ java States.java
before start : NEW
lock held    : BLOCKED
sleeping     : TIMED_WAITING
in wait()    : WAITING
after finish : TERMINATED
```

Main holds `lock` while it starts `demo`, so `demo` is BLOCKED at its first `synchronized`. After main releases the lock, `demo` sleeps (TIMED_WAITING), then calls `lock.wait()` (WAITING) until main calls `notify()`. A thread in `BLOCKED` is waiting for a *lock*; in `WAITING` it is waiting for a *signal*. Telling these apart in a thread dump is the first step in diagnosing a hung application.

8.2.3 Waiting for a thread: join

`t.join()` blocks the calling thread until `t` terminates, and it also creates a happens-before edge (everything `t` wrote is visible to the caller afterwards; see the memory model chapter). `join(millis)` gives up after a timeout; check `isAlive()` afterwards to see whether the thread really finished. Always prefer `join` (or a `Future`, later) over sleeping and hoping.

8.2.4 Daemon threads and JVM exit

The JVM exits when all *non-daemon* threads have finished. A **daemon thread** is a background helper (a cache cleaner, a heartbeat) that must not keep the JVM alive. Call `setDaemon(true)` *before* `start()`. A thread inherits the daemon status of the thread that creates it. Daemon threads are stopped abruptly at exit: `finally` blocks and `try`-with-resources cleanup do not run, so never use a daemon for work that must complete or for anything holding files or locks that need orderly release.

8.2.5 Uncaught exceptions

If `run()` throws, the exception does not propagate to the thread that called `start()`; there is no one to catch it. By default the JVM prints the stack trace to `System.err` and the thread dies. You can install a handler per thread (`setUncaughtExceptionHandler`) or globally (`Thread.setDefaultUncaughtExceptionHandler`) to log and alert. In thread pools the story differs (exceptions are captured into the `Future`), which a later chapter shows.

Daemon.java JAVA

```
public class Daemon {
    public static void main(String[] args) throws Exception {
        Thread.setDefaultUncaughtExceptionHandler((th, ex) ->
            System.out.println("handler got " + ex + " from " + th.getName()));
        Thread bad = new Thread(() -> { throw new IllegalStateException("boom"); }, "bad-thread");
        bad.start();
        bad.join();
        Thread d = new Thread(() -> {
            try { Thread.sleep(5000); } catch (InterruptedException e) { }
            System.out.println("daemon never prints this");
        }, "background");
        d.setDaemon(true);
        d.start();
        System.out.println("main ends; daemon=" + d.isDaemon());
    }
}
```

Run it TERMINAL

```
$ java Daemon.java
handler got java.lang.IllegalStateException: boom from bad-thread
main ends; daemon=true
```

The daemon thread would sleep 5 seconds and print, but `main` ends first, no non-daemon thread remains, and the JVM exits at once; the message never appears.

8.2.6 Interruption: the cooperative cancellation protocol

How do you stop a thread? Not with `Thread.stop()`: it was deprecated for removal because it kills a thread at an arbitrary point, leaving objects half-updated and locks released in an inconsistent state; in Java 20 it was changed to throw `UnsupportedOperationException`. Java's mechanism is **interruption**, which is a polite request. Interrupting a thread sets its *interrupt flag*; it is up to the thread's code to notice and stop.

The rules form a small protocol:

1. `t.interrupt()` sets the flag on `t`.
2. A thread in a blocking method that supports it (`sleep`, `wait`, `join`, `BlockingQueue.put/take`, most `java.util.concurrent` waits) wakes up, **clears the flag**, and throws `InterruptedException`.
3. A busy loop must poll `Thread.currentThread().isInterrupted()` itself.
4. `Thread.interrupted()` (static) returns the flag *and clears it*; avoid it unless you mean to consume the interrupt.

Interrupt.java JAVA

```java
public class Interrupt {
    public static void main(String[] args) throws Exception {
        Thread sleeper = new Thread(() -> {
            try {
                Thread.sleep(10_000);
            } catch (InterruptedException e) {
                System.out.println("sleeper: caught, flag now "
                    + Thread.currentThread().isInterrupted());
                Thread.currentThread().interrupt();
                System.out.println("sleeper: restored, flag now "
                    + Thread.currentThread().isInterrupted());
            }
        });
        sleeper.start();
        sleeper.interrupt();
        sleeper.join();

        Thread spinner = new Thread(() -> {
            long n = 0;
            while (!Thread.currentThread().isInterrupted()) n++;
            System.out.println("spinner: saw interrupt, stopping");
        });
        spinner.start();
        Thread.sleep(50);
        spinner.interrupt();
        spinner.join();
        System.out.println("done");
    }
}
```

Run it TERMINAL

```
$ java Interrupt.java
sleeper: caught, flag now false
sleeper: restored, flag now true
spinner: saw interrupt, stopping
done
```

What to notice: when `InterruptedException` is thrown the flag is already cleared (`flag now false`). If you swallow the exception the request to stop is lost forever. The correct handling is one of two things: **propagate** the exception (declare `throws InterruptedException`), or **restore the flag** with `Thread.currentThread().interrupt()` so callers up the stack can still see it.

> **WATCH OUT**
> Never write `catch (InterruptedException e) { }` (an empty catch) or just print and continue. The thread was asked to stop and you ignored it. Executors that try to shut down will hang, and tasks that should be cancelled will keep running. Either rethrow, or call `Thread.currentThread().interrupt()` and return.

8.2.7 Priorities, yield and other myths

`setPriority(1..10)` is only a hint to the OS scheduler. On Linux, JVM thread priorities are ignored by default, and on other systems they have platform-specific effects. Never use priorities for correctness (for example to make "the important thread always win"). `Thread.yield()` is likewise a hint, and `Thread.sleep(0)` is not a reliable way to let others run. If one thread must run before another, use synchronisation (`join`, latches, locks), never timing or priorities.

`Thread.sleep(ms)` pauses the current thread without releasing any monitor it holds. This makes sleeping inside a `synchronized` block a good way to stall every other thread that needs that lock. (`Object.wait()`, by contrast, releases the monitor.)

8.2.8 Naming and reading a thread dump

A **thread dump** is a snapshot of all threads, their states and stacks. You get it with `jstack <pid>` or `jcmd <pid> Thread.print`, or by sending the process a `SIGQUIT` with `kill -3 <pid>` (Linux/macOS), which prints the dump to the program's standard output. Every thread appears with its name, daemon flag, `prio`, native id and `java.lang.Thread.State`. A later chapter reads one in anger when diagnosing a deadlock. For now: if your threads are all called `Thread-17`, you cannot tell the order-processing thread from the e-mail thread. Name them.

> **TIP**
> Create threads with a descriptive name and, in library code, a stable prefix (for example `inventory-sync-1`). When something hangs at 3 a.m. the name is the first clue.

> **INTERVIEW ANGLE**
> What is the difference between `start()` and `run()`? `start()` creates a new OS thread and makes the JVM call `run()` on it; control returns immediately to the caller. Calling `run()` directly is just an ordinary method call on the current thread, so there is no concurrency. `start()` can be called only once per `Thread` object.

> **INTERVIEW ANGLE**
> How do you stop a thread safely? Use cooperative interruption: call `interrupt()`, and make the target thread check `isInterrupted()` in loops and handle `InterruptedException` by cleaning up and exiting (or restoring the interrupt flag). `Thread.stop()` is unsafe and no longer works. A `volatile boolean` flag also works for loops that never block, but interruption additionally wakes blocked threads.

Check yourself

1. What does this print, and why? `Thread t = new Thread(() -> System.out.println(Thread.currentThread().getName())); t.run();`
2. A thread calls `lock.wait()`. Which `Thread.State` is it in? What if it calls `Thread.sleep(500)`? What if it tries to enter a `synchronized` block held by another thread?
3. Why is `catch (InterruptedException e) {}` considered a bug?
4. When does the JVM exit with respect to daemon and non-daemon threads? What happens to a daemon thread's `finally` block at that moment?
5. What happens if you call `start()` twice on the same `Thread` object?
6. A thread throws an uncaught `RuntimeException` inside `run()`. Does `main`'s `try/catch` around `t.start()` catch it?

Answers

1. It prints `main`: `run()` called directly executes on the calling thread; nothing is started.
2. `WAITING`; `TIMED_WAITING`; `BLOCKED`.
3. It swallows the cancellation request. The interrupt flag was cleared when the exception was thrown, so nobody up the stack can learn that the thread was asked to stop. Rethrow, or restore the flag with `Thread.currentThread().interrupt()`.
4. It exits when the last non-daemon thread ends. Daemon threads are killed abruptly, so their `finally` blocks may not run.
5. `IllegalThreadStateException`. Create a new `Thread` (or submit the task again to a pool).
6. No. The exception happens on the other thread. It goes to that thread's uncaught-exception handler, by default printing the stack trace; `main` is unaffected.

KEY POINTS

Thread basics to remember.

- Create threads from a `Runnable`; call `start()`, never `run()`; name your threads.
- Six states: NEW, RUNNABLE, BLOCKED (lock), WAITING and TIMED_WAITING (signal or time), TERMINATED.
- `join()` waits for termination and gives visibility of the thread's writes.
- Stop threads with interruption: handle `InterruptedException` by propagating it or restoring the flag; never swallow it.
- Priorities and `yield` are hints; never rely on them. The JVM exits when only daemon threads remain.

8.3 Races and the Java Memory Model

Two threads, one counter, no locks. What could go wrong? This chapter shows, by running code, three different ways it goes wrong, and then introduces the rulebook that tells you exactly when a multi-threaded Java program is guaranteed to behave: the **Java Memory Model** (JMM), defined in chapter 17 of the Java Language Specification.

8.3.1 A race condition you can run

A **race condition** exists when the correctness of a computation depends on the relative timing of threads. The simplest case is a lost update. The program below increments four counters from two threads, one million times each. One counter is a plain `int`, one is `volatile`, one is guarded by `synchronized`, and one is an `AtomicInteger`.

Race.java — JAVA

```java
import java.util.concurrent.atomic.AtomicInteger;

public class Race {
    static int plain = 0;
    static volatile int vol = 0;
    static int locked = 0;
    static final Object LOCK = new Object();
    static final AtomicInteger atomic = new AtomicInteger();

    public static void main(String[] args) throws Exception {
        final int perThread = 1_000_000;
        Runnable r = () -> {
            for (int i = 0; i < perThread; i++) {
                plain++;
                vol++;
                synchronized (LOCK) { locked++; }
                atomic.incrementAndGet();
            }
        };
        Thread a = new Thread(r), b = new Thread(r);
        a.start(); b.start();
        a.join(); b.join();
        System.out.println("expected          : " + 2 * perThread);
        System.out.println("plain int         : " + plain);
        System.out.println("volatile int      : " + vol);
        System.out.println("synchronized      : " + locked);
        System.out.println("AtomicInteger     : " + atomic.get());
    }
}
```

Compile it and run it a few times (we use `javac` here because the single-file launcher shows slightly different timing):

Typical output (varies between runs) — TERMINAL

```
$ javac Race.java && java Race
expected          : 2000000
plain int         : 1999592
volatile int      : 1895976
synchronized      : 2000000
AtomicInteger     : 2000000
$ java Race
expected          : 2000000
plain int         : 1999886
volatile int      : 1999889
synchronized      : 2000000
AtomicInteger     : 2000000
```

What to notice:

- The plain `int` loses updates. How many it loses changes from run to run (this build machine has only 2 cores, so the loss is small; on a busier or bigger machine it is much larger). A bug whose symptom changes every run is typical of a race.
- `volatile` **does not fix it.** `vol++` is still read, add, write. Volatility makes each individual read and write visible, but the three steps can interleave. (This is the most common misconception about `volatile`.)
- `synchronized` and `AtomicInteger` give exactly 2000000, for different reasons that later chapters explain: mutual exclusion and a hardware compare-and-swap loop.
- Races may not show if you test once. A program that passes a hundred test runs can still be wrong. You prove thread safety by reasoning with the rules below, not by testing.

What count++ really is

Three steps in bytecode, interleaved by two threads

```
Thread A: getstatic count   -> 5
Thread B: getstatic count   -> 5
Thread A: iconst_1, iadd    -> 6
Thread B: iconst_1, iadd    -> 6
Thread A: putstatic count   <- 6
Thread B: putstatic count   <- 6      one increment is lost; count is 6, not 7
```

This is a **check-then-act** / **read-modify-write** problem. The cure is to make the whole compound action atomic.

8.3.2 Visibility: the flag that never flips

The second hazard needs no interleaving at all. One thread writes a flag; another loops until it sees it.

Visibility.java JAVA

```java
public class Visibility {
    static boolean stop = false;           // try adding volatile

    public static void main(String[] args) throws Exception {
        Thread worker = new Thread(() -> {
            long spins = 0;
            while (!stop) spins++;
            System.out.println("worker saw stop flag; spins=" + (spins > 0));
        });
        worker.start();
        Thread.sleep(500);
        stop = true;
        System.out.println("main set stop = true");
        worker.join(2000);
        System.out.println("worker alive after 2s: " + worker.isAlive());
        System.exit(0);
    }
}
```

Compile with javac and run (typical; HotSpot with default JIT) TERMINAL

```
$ javac Visibility.java && java Visibility
main set stop = true
worker alive after 2s: true
```

The worker never stops. The JIT compiler may legally hoist the read of `stop` out of the loop (it sees no code in the loop that changes it, so it assumes it can read it once). Make the field `volatile` and the program behaves:

VisibilityFixed.java JAVA

```java
public class VisibilityFixed {
    static volatile boolean stop = false;

    public static void main(String[] args) throws Exception {
        Thread worker = new Thread(() -> {
            long spins = 0;
            while (!stop) spins++;
            System.out.println("worker saw stop flag; spins=" + (spins > 0));
        });
        worker.start();
        Thread.sleep(500);
```

VisibilityFixed.java (continued) JAVA

```
        stop = true;
        System.out.println("main set stop = true");
        worker.join(2000);
        System.out.println("worker alive after 2s: " + worker.isAlive());
        System.exit(0);
    }
}
```

Run it TERMINAL

```
$ javac VisibilityFixed.java && java VisibilityFixed
main set stop = true
worker saw stop flag; spins=true
worker alive after 2s: false
```

GOTCHA

This bug is not guaranteed to appear. Under the source launcher (`java Visibility.java`) on the build machine the loop happened to terminate, while the javac-compiled class hung. Whether the JIT optimises the loop depends on timing, flags and JVM version. Without `volatile` the program is *incorrect* even on the days it works.

8.3.3 Reordering

The third hazard is order. Consider two threads where `x`, `y`, `r1`, `r2` start at 0:

Classic reordering litmus test

```
Thread A:  x = 1;  r1 = y;        Thread B:  y = 1;  r2 = x;
```

Intuition says at least one thread must see the other's write, so `r1 == 0 && r2 == 0` is impossible. On real hardware it *is* possible, because each core buffers its write in a store buffer and may execute the following read before the write becomes visible to others (and the JIT may also swap the two statements, since they touch different variables). Seeing the effect in a Java test needs millions of precise attempts on a multi-core machine, and our 2-core build machine did not show it reliably, so we do not print a number here; tools such as the JCStress harness exist for exactly this. The lesson is that you may not infer anything about cross-thread order from the order of statements in the source.

8.3.4 The Java Memory Model in plain words

The JMM does not describe caches and store buffers. It defines a relation between actions called **happens-before** (written hb). If action A happens-before action B, then A's effects (all its writes) are visible to B, and A is ordered before B. If there is no happens-before edge between a write and a read of the same variable, that is a **data race**, and the read may return the latest write, an older write, or a value written by some interleaving; the specification only promises each read sees *some* write, not which one.

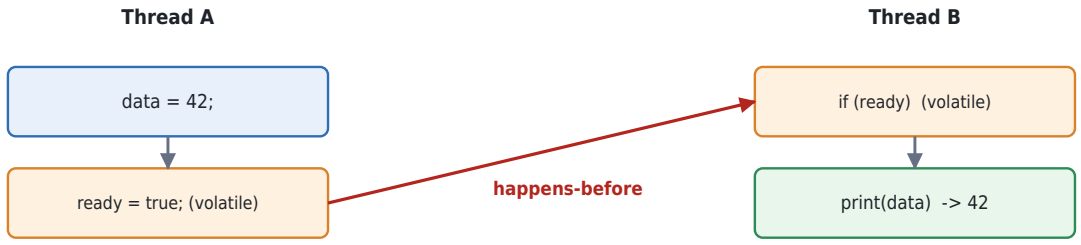


Figure 8.2. Happens-before edges: an unlock-lock pair or a volatile write-read pair connects two threads, so everything before the edge is visible after it.

The main happens-before rules (what you may rely on):

Rule	Meaning
Program order	Each action in a thread happens-before every later action in the same thread.
Monitor lock	An `unlock` of a monitor happens-before every later `lock` of that same monitor.

Rule	Meaning
Volatile variable	A write to a volatile field happens-before every later read of that field.
Thread start	`t.start()` happens-before every action in `t`.
Thread termination	Every action in `t` happens-before another thread detects that `t` ended (`t.join()` returns, or `t.isAlive()` is false).
Interruption	`t.interrupt()` happens-before `t` detects the interrupt.
Final fields	Writes in a constructor to `final` fields happen-before any thread reads them through a reference obtained after the constructor ends (the freeze rule).
Transitivity	If A hb B and B hb C then A hb C.

Happens-before rules.

Transitivity is the magic that makes this usable. Thread A writes ordinary data, then writes a volatile flag. Thread B reads the volatile flag (true), then reads the ordinary data. Because the data write hb the flag write (program order), flag write hb flag read (volatile rule), and flag read hb data read (program order), B is guaranteed to see the data. This is called *piggybacking* on the volatile.

Many library classes establish hb edges for you: submitting a task to an `ExecutorService` happens-before the task runs, the task's completion happens-before `Future.get()` returns, putting an object in a `BlockingQueue` happens-before taking it, `CountDownLatch.countDown()` happens-before a successful `await()`.

8.3.5 volatile in detail

A `volatile` field gives you:

1. **Visibility:** a read always sees the most recent write to that variable by any thread.
2. **Ordering:** no instruction before a volatile write is moved after it, and none after a volatile read is moved before it. It acts like a one-way fence at each access.
3. **Atomic reads and writes of `long` and `double`.** (The JLS allows a non-volatile 64-bit read or write to be split into two 32-bit halves; in practice 64-bit HotSpot does not tear them, but only `volatile` guarantees it.)

It does **not** give you atomic compound operations (`++`, check-then-act). Rules for using it: a single writer or writes that do not depend on the previous value (a status flag, a published reference, a configuration snapshot) are good fits.

INTERVIEW ANGLE
What does `volatile` guarantee, and what does it not? It guarantees visibility (a read sees the last write) and ordering (via the happens-before edge from a write to later reads), and makes reads and writes of long and double atomic. It does not make compound actions such as `count++` atomic, because those are separate read and write steps; use `synchronized` or an atomic class for that.

8.3.6 final fields and safe publication

Publishing an object means making a reference to it reachable by another thread. **Unsafe publication** is storing the reference in a plain shared field with no hb edge:

Fragment: unsafe publication JAVA

```
class Holder {
    static Config current;                      // plain field
    static void init() { current = new Config(8080, "prod"); }
}
class Config {
    int port; String env;                       // non-final fields
    Config(int port, String env) { this.port = port; this.env = env; }
}
```

A reader thread that sees `Holder.current != null` may still see `port == 0` or `env == null`: the reference write can become visible before the field writes inside the constructor. The object is *observed partially constructed*.

There are four standard ways to publish safely:

- Initialise the reference from a **static initialiser** (the class-initialisation lock gives an hb edge).
- Store it in a `volatile` **field** or an `AtomicReference`.
- Store it in a field guarded by a **lock** (readers lock the same monitor).

- Make the fields `final`. The freeze rule guarantees that any thread that sees the reference also sees the correctly constructed values of the final fields (and objects reachable only through them), even if the reference itself was published by a race. This is why immutable objects built from final fields are inherently thread-safe, as the last chapter of this Part discusses. The rule holds only if `this` does not escape from the constructor (do not start a thread or register a listener inside a constructor).

Also safe: placing the object in a thread-safe collection (`ConcurrentHashMap`, `BlockingQueue`) or handing it over through an executor.

8.3.7 Double-checked locking

Lazy initialisation of a shared object is a famous trap. The "obvious" optimisation avoids locking after initialisation: check, lock, check again, create. Before Java 5 this was broken. It is correct today if and only if the field is `volatile`.

Singleton.java JAVA

```java
public class Singleton {
    private static volatile Singleton instance;
    private final int[] data = new int[]{1, 2, 3};
    private Singleton() { }
    public static Singleton get() {
        Singleton local = instance;           // one volatile read on the fast path
        if (local == null) {
            synchronized (Singleton.class) {
                local = instance;
                if (local == null) instance = local = new Singleton();
            }
        }
        return local;
    }
    static class Holder {                      // initialization-on-demand holder
        static final Singleton INSTANCE = new Singleton();
    }
    public static void main(String[] args) {
        System.out.println(get() == get());
        System.out.println(Holder.INSTANCE == Holder.INSTANCE);
        System.out.println(get().data.length);
    }
}
```

Run it TERMINAL

```
$ java Singleton.java
true
true
3
```

What to notice: without `volatile`, another thread could see `instance != null` yet see the `data` array not yet initialised, since the reference store may be reordered before the constructor's writes. The local variable `local` is a micro-optimisation (one volatile read on the fast path instead of two). The `Holder` class shows the simplest correct alternative: the **initialisation-on-demand holder idiom**. The JVM initialises `Holder.INSTANCE` lazily, the first time `Holder` is used, under the class-initialisation lock, so no explicit synchronisation is needed. For singletons, an `enum` with one constant is a third option. Prefer these over double-checked locking.

8.3.8 Beyond volatile: VarHandle and fences

Since Java 9, `java.lang.invoke.VarHandle` offers finer-grained access modes to a field or array element: plain, opaque, acquire/release, and volatile, plus `compareAndSet`. The JDK's own atomic classes and concurrent collections are built on it (earlier on `sun.misc.Unsafe`). Application code should use `AtomicInteger`, `AtomicReference`, locks and the concurrent collections; reach for `VarHandle` only when you are writing a library data structure and have measured a need. Acquire/release semantics are weaker than full volatile but cheaper on some CPUs, and much easier to get wrong.

> **NOTE**
> Safe code needs only two habits. First, every shared mutable variable must have a clear owner: guarded by one lock, declared volatile, or confined to one thread. Second, communicate between threads using library constructs that already carry the happens-before edges.

REAL WORLD
A Spring `@Configuration` bean with `private final` fields is safely published to all request threads; a hand-rolled `static Map` cache written by one request thread and read by others is not. Frameworks initialise singletons under internal locks, which gives you an hb edge for the container's own beans, but not for state you mutate afterwards.

Check yourself

1. Two threads each execute `counter++` 1000 times on a shared `volatile int counter` starting at 0. Can the final value be less than 2000? Why?
2. A worker loops `while (running) { process(); }` and another thread sets `running = false`. Name the bug if `running` is a plain `boolean`, and give the smallest fix.
3. State the happens-before rule for `Thread.join()`. Why does it let you read a result field written by the joined thread without any other synchronisation?
4. What is wrong with this code? Fix it. `private static Resource r; static Resource get() { if (r == null) r = new Resource(); return r; }`
5. Why is a class whose fields are all `final` easier to share between threads? Which condition must still hold in the constructor?
6. True or false: a `volatile` write happens-before every later read of the same variable, so after thread B reads `ready == true` it also sees everything thread A wrote before setting `ready`.

Answers

1. Yes. `volatile` makes each read and write visible, but `counter++` is read-modify-write; two threads can read the same value and both store value+1. Use `AtomicInteger` or `synchronized`.
2. A visibility bug: the worker may never see the write (the JIT can hoist the read). Declare the field `volatile`.
3. Everything the thread did happens-before the return of `join()` in the waiting thread. So after `join()` returns, all of the thread's writes are visible.
4. Two threads can both see null and create two instances (a check-then-act race), and a thread may see a partially built object. Fix: make `get()` `synchronized`, or use the holder idiom, or `volatile` plus double-checked locking.
5. The freeze rule makes final fields visible after construction even if the reference was published by a race, and nothing changes afterwards. The constructor must not let `this` escape before it finishes.
6. True. Program order plus the volatile rule plus transitivity make all earlier writes of A visible to B (the piggybacking effect).

KEY POINTS
The memory model in short.

- `count++` is three steps; without mutual exclusion or an atomic, updates are lost, and tests will not reliably show it.
- Without a happens-before edge, a thread may never see another thread's write; the JIT may legally hoist a read out of a loop.
- Rules to learn: program order, monitor lock, volatile, thread start and join, final-field freeze, transitivity.
- `volatile` gives visibility and ordering, not atomic compound actions. Use it for flags and safely published references.
- Publish objects safely: static initialisers, volatile, locks or final fields. Prefer the holder idiom over double-checked locking.

8.4 synchronized and Monitors

The previous chapter showed that shared mutable state needs two things: **mutual exclusion** (only one thread at a time in a critical section) and **visibility** (what one thread wrote, the next sees). Java's built-in tool for both is the `synchronized` keyword, backed by an **intrinsic lock**, also called a **monitor**, that every Java object carries.

8.4.1 How synchronized works



only the owner runs the synchronized code; wait() releases the lock

Figure 8.3. The monitor: a lock owner plus an entry set of blocked threads and a wait set of threads that called wait().

Every object has an associated monitor. Entering a `synchronized` block means acquiring that object's monitor; leaving it (normally *or by exception*) releases it. If another thread owns it, the entering thread becomes BLOCKED until the owner releases it. The compiler turns a block into `monitorenter` and `monitorexit` bytecodes (with an exception handler that guarantees the exit); a `synchronized` method is marked with the `ACC_SYNCHRONIZED` flag and the JVM acquires the lock on entry.

Because unlock happens-before the next lock of the same monitor, `synchronized` gives you visibility as well as exclusion: everything done inside the block is visible to the next thread that locks the same object.

Which object is the lock?

Form	Lock object
`synchronized void m()` (instance method)	`this`
`static synchronized void m()`	the `Class` object (`Counter.class`)
`synchronized (obj) { ... }`	`obj`

What each form locks on.

The two counters in the next listing use different locks: the instance counter locks the `Counter` object, the static one locks `Counter.class`. Both give the exact total.

Counter.java JAVA

```java
public class Counter {
    private int count;
    public synchronized void increment() { count++; }       // lock = this
    public synchronized int get() { return count; }

    private static int shared;
    public static synchronized void bump() { shared++; }     // lock = Counter.class

    public static void main(String[] args) throws Exception {
        Counter c = new Counter();
        Runnable r = () -> { for (int i = 0; i < 1_000_000; i++) { c.increment(); bump(); } };
        Thread a = new Thread(r), b = new Thread(r);
        a.start(); b.start(); a.join(); b.join();
        System.out.println(c.get() + " " + shared);
    }
}
```

Run it TERMINAL

```
$ java Counter.java
2000000 2000000
```

The rule that matters: **two critical sections exclude each other only if they use the same lock object.** A static `synchronized` method and an instance `synchronized` method do not exclude each other, because they lock different objects. Likewise, if one thread updates a field under lock A and another reads it under lock B (or with no lock), there is no happens-before edge and you have a data race.

8.4.2 Reentrancy

Intrinsic locks are **reentrant**: a thread that already owns a monitor can enter another block guarded by the same monitor, and it succeeds immediately. The JVM keeps a hold count and releases the monitor only when the count returns to zero. Without reentrancy a `synchronized` method could not call another `synchronized` method of the same object (or a subclass's overriding method) without deadlocking itself.

Reentrant.java JAVA

```java
public class Reentrant {
    synchronized void outer() { System.out.println("outer"); inner(); }
    synchronized void inner() {
        System.out.println("inner, holds lock: " + Thread.holdsLock(this));
    }
    public static void main(String[] args) { new Reentrant().outer(); }
}
```

Run it TERMINAL

```
$ java Reentrant.java
outer
inner, holds lock: true
```

8.4.3 Method versus block, and lock scope

A `synchronized` method locks the whole method. A `synchronized` block locks only a few lines and lets you choose the lock object. Two guidelines:

1. **Keep critical sections short.** By Amdahl's law, time spent holding a lock is the sequential part of your program. Do not perform I/O, call unknown code or `sleep` while holding a lock; compute outside, then lock just to update shared state.
2. **Lock on a private object**, not on `this` or a public object, when you want to control who can participate:

Fragment: a private lock JAVA

```java
public class Inventory {
    private final Object lock = new Object();      // nobody else can lock this
    private int stock;

    public void add(int n) {
        synchronized (lock) { stock += n; }
    }
}
```

If you synchronise on `this`, any code holding a reference to your object can take the same lock (`synchronized (inventory) { ... }`) and stall or deadlock you. Never lock on a `String` literal, a boxed `Integer`, or any value that might be shared or interned: unrelated code in the same JVM may be locking the same cached object. Never lock on a field you reassign, because threads would then lock different objects.

> **WATCH OUT**
> `synchronized` on only the setter, or only on writes, is not thread-safe. Every access (read and write) to the shared state must use the same lock, otherwise readers may see stale or inconsistent values.

8.4.4 wait, notify and notifyAll

Mutual exclusion is not enough when a thread must wait for a *condition* ("the buffer has room", "the queue is not empty"). Spinning on the condition wastes CPU. The monitor provides three methods on `Object`:

- `wait()`: the calling thread must own the monitor; it **releases the monitor**, joins the object's wait set (WAITING) and sleeps until notified (or interrupted, or the timeout of `wait(ms)` passes). When it wakes, it must **re-acquire the monitor** before continuing.
- `notify()`: wakes one thread from the wait set (an arbitrary one).
- `notifyAll()`: wakes all threads in the wait set; they then compete for the monitor.

Calling any of them without owning the monitor throws `IllegalMonitorStateException`.

The rules of the wait loop

1. Always call `wait` **inside a loop that re-tests the condition**: `while (!condition) wait();`. Never `if`.
2. Always change the condition and notify while holding the same lock.
3. Prefer `notifyAll()` unless you can prove only one waiter kind exists and one wake-up suffices.

The loop is needed for three reasons. A thread woken by `notifyAll` may find that another thread already consumed the thing it wanted. The condition may be different for different waiters on the same monitor (producers and consumers). And the JVM is allowed to wake a waiting thread **spuriously**, without any notify, a behaviour inherited from OS threading libraries and documented in the Javadoc of `Object.wait`.

8.4.5 Producer-consumer by hand

A bounded buffer is the standard example: producers wait when it is full, consumers wait when it is empty.

BoundedBuffer.java JAVA

```
import java.util.ArrayDeque;
import java.util.Queue;

public class BoundedBuffer<T> {
    private final Queue<T> queue = new ArrayDeque<>();
    private final int capacity;
    public BoundedBuffer(int capacity) { this.capacity = capacity; }

    public synchronized void put(T item) throws InterruptedException {
        while (queue.size() == capacity) wait();     // loop, never "if"
        queue.add(item);
        notifyAll();
    }
    public synchronized T take() throws InterruptedException {
        while (queue.isEmpty()) wait();
        T item = queue.remove();
        notifyAll();
        return item;
    }

    public static void main(String[] args) throws Exception {
        BoundedBuffer<Integer> buf = new BoundedBuffer<>(3);
        Thread producer = new Thread(() -> {
            try {
                for (int i = 1; i <= 8; i++) buf.put(i);
                buf.put(-1);                         // poison pill
            } catch (InterruptedException e) { Thread.currentThread().interrupt(); }
        }, "producer");
        Thread consumer = new Thread(() -> {
            try {
                int sum = 0;
                for (int v; (v = buf.take()) != -1; ) sum += v;
                System.out.println("consumer sum = " + sum);
            } catch (InterruptedException e) { Thread.currentThread().interrupt(); }
        }, "consumer");
        producer.start(); consumer.start();
        producer.join(); consumer.join();
    }
}
```

Run it TERMINAL

```
$ java BoundedBuffer.java
consumer sum = 36
```

The producer puts 1..8 (sum 36) and a poison pill. With capacity 3 the producer must repeatedly wait for room, and the consumer must wait for items, so the example exercises both waits. What to notice:

- Both methods are `synchronized`, so the queue (an unsynchronised `ArrayDeque`) is only touched under the monitor, and both conditions are tested in `while` loops.
- `notifyAll()` is used because producers and consumers wait for *different* conditions on the same monitor. A plain `notify()` could wake a producer when only a consumer can make progress, and then the wake-up is "lost": all threads sleep forever.
- `put` and `take` declare `InterruptedException` and propagate it, honouring the interruption protocol.

In real code you would use a `BlockingQueue` (a later chapter) which does all of this; but you must understand `wait`/`notify` because it is the foundation of those classes and a favourite interview topic.

INTERVIEW ANGLE
Why must `wait()` be called in a loop? Because a thread can wake without the condition being true: another thread may have consumed it first (a "stolen" wake-up), several conditions may share one monitor, and spurious wake-ups are permitted by the specification. The loop re-checks the condition after each wake-up and waits again if it is still false.

INTERVIEW ANGLE
What is the difference between `sleep()` and `wait()`? `sleep` is a static `Thread` method; it pauses the current thread and does NOT release any lock. `wait` is an `Object` method; it must be called while holding that object's monitor, it releases the monitor while waiting, and it ends on notify, interrupt or timeout.

8.4.6 Deadlock

A **deadlock** is a cycle of threads each waiting for a lock held by the next. All four conditions must hold: mutual exclusion, hold-and-wait, no pre-emption (locks cannot be taken away), and circular wait. Break any one and deadlock is impossible. The classic cause is two threads taking two locks in opposite order. Here are two transfers between accounts, `A to B` and `B to A`, each locking "from" and then "to":

Deadlock.java JAVA

```java
public class Deadlock {
    static class Account {
        final String name; int balance = 100;
        Account(String name) { this.name = name; }
    }
    static void transfer(Account from, Account to, int amount) throws InterruptedException {
        synchronized (from) {
            Thread.sleep(100);                      // widen the race window
            synchronized (to) {
                from.balance -= amount; to.balance += amount;
            }
        }
    }
    public static void main(String[] args) throws Exception {
        Account a = new Account("A"), b = new Account("B");
        new Thread(() -> {
            try { transfer(a, b, 10); } catch (InterruptedException e) { }
        }, "t1-A-to-B").start();
        new Thread(() -> {
            try { transfer(b, a, 20); } catch (InterruptedException e) { }
        }, "t2-B-to-A").start();
        Thread.sleep(60_000);
    }
}
```

The `sleep(100)` simply makes the unlucky interleaving certain. With real code the same bug waits months for the right timing. Start it and attach `jstack` from a second terminal (get the pid with `jcmd -l`):

Diagnose a hang with jstack (object addresses and Java version vary) TERMINAL

```
$ javac Deadlock.java && java Deadlock &
$ jstack $(jcmd -l | grep Deadlock | cut -d' ' -f1)
Found one Java-level deadlock:
=============================
"t1-A-to-B":
  waiting to lock monitor 0x00007fdf6404fb70 (object 0x00000000a8518120, a Deadlock$Account),
  which is held by "t2-B-to-A"

"t2-B-to-A":
  waiting to lock monitor 0x00007fdf700015b0 (object 0x00000000a85180d8, a Deadlock$Account),
  which is held by "t1-A-to-B"

Java stack information for the threads listed above:
===================================================
"t1-A-to-B":
        at Deadlock.transfer(Deadlock.java:10)
        - waiting to lock <0x00000000a8518120> (a Deadlock$Account)
```

Diagnose a hang with jstack (object addresses and Java version vary) (continued) TERMINAL

```
        - locked <0x00000000a85180d8> (a Deadlock$Account)
        at Deadlock.lambda$main$0(Deadlock.java:17)
...
Found 1 deadlock.
```

`jstack` detects monitor deadlocks for you and prints the cycle: thread t1 holds the account at `...80d8` and waits for `...8120`, thread t2 holds `...8120` and waits for `...80d8`. The line number points to the inner `synchronized (to)`. Without that summary, you would search for threads in state `BLOCKED (on object monitor)` with `waiting to lock` lines and follow the "held by" chain. A thread that is hung but is not part of a reported cycle is blocked for another reason (I/O, a lock-free spin, a stuck `wait`). Take two or three dumps a few seconds apart: threads that are in the same place in all of them are the ones stuck.

The cure: a global lock order

If every thread acquires locks in the same global order, no cycle can form. For two accounts, order by a unique id:

DeadlockFixed.java JAVA

```java
public class DeadlockFixed {
    static class Account {
        private static long next = 0;
        final long id = next++;
        int balance = 100;
    }
    static void transfer(Account from, Account to, int amount) throws InterruptedException {
        Account first = from.id < to.id ? from : to;     // global order by id
        Account second = first == from ? to : from;
        synchronized (first) {
            Thread.sleep(100);
            synchronized (second) {
                from.balance -= amount;
                to.balance += amount;
            }
        }
    }
    public static void main(String[] args) throws Exception {
        Account a = new Account(), b = new Account();
        Thread t1 = new Thread(() -> {
            try { transfer(a, b, 10); } catch (InterruptedException e) { }
        });
        Thread t2 = new Thread(() -> {
            try { transfer(b, a, 20); } catch (InterruptedException e) { }
        });
        t1.start(); t2.start(); t1.join(); t2.join();
        System.out.println("a=" + a.balance + " b=" + b.balance);
    }
}
```

Run it TERMINAL

```
$ java DeadlockFixed.java
a=110 b=90
```

Both transfers now lock the lower-id account first. A transfer of 10 from a to b and of 20 from b to a nets a=110, b=90 and always terminates. Other strategies: acquire all locks at once through one coarser lock, use `tryLock` with a timeout (explicit locks, next chapter) and back off, or avoid holding one lock while calling code that takes another ("open calls"). Calling an overridable or callback method while holding a lock is a frequent hidden cause.

NOTE
The `static long next` id counter in `DeadlockFixed` is itself not thread-safe; it is fine here because both accounts are created by `main` before any thread starts. In real code use an `AtomicLong`.

REAL WORLD
Production deadlocks usually involve one lock in your code and one in a framework or database: a request thread holds a synchronized cache lock and waits for a connection from an exhausted connection pool, while the thread holding the last connection waits for that cache lock. A thread dump shows both stacks immediately. Many production services capture `jstack` or `jcmd Thread.print` automatically when health checks fail.

Check yourself

1. What object does each lock on? `static synchronized void a()`, `synchronized void b()`, `synchronized (this) { }`.
2. Why does this fail? `synchronized void push(int x) { list.add(x); } int size() { return list.size(); }` (the list is a shared `ArrayList`)
3. What exception is thrown by `lock.wait()` when the caller does not own the lock's monitor? What does `wait` do with the monitor it owns?
4. What is wrong with `synchronized (lock) { if (queue.isEmpty()) lock.wait(); item = queue.remove(); }` and how do you fix it?
5. Thread 1 locks A then B; thread 2 locks B then A. Name the problem and two fixes.
6. You get a thread dump of a hung service. Which states and lines indicate a deadlock?

Answers

1. `a()`: the `Class` object. `b()` and the block: `this`. So `b()` and the block exclude each other, but `a()` does not exclude either.
2. `size()` is not synchronised, so it may read the list without a happens-before edge from `push` (stale or inconsistent size, possible failure during a resize). All accesses to shared state must use the same lock.
3. `IllegalMonitorStateException`. `wait` releases the monitor while waiting and reacquires it before returning.
4. `if` instead of `while`: after a spurious or stolen wake-up the queue may still be empty and `remove()` throws `NoSuchElementException`. Use `while (queue.isEmpty()) lock.wait();`.
5. Deadlock through opposite lock ordering. Fixes: acquire in a single global order; use one lock for both; use `tryLock` with timeout and back off.
6. Threads in `BLOCKED (on object monitor)` with `waiting to lock <addr>`, where the lock is held by another waiting thread in a cycle; `jstack` prints "Found one Java-level deadlock".

KEY POINTS

Intrinsic locks in short.

- Every object is a monitor; `synchronized` gives mutual exclusion and a happens-before edge from unlock to the next lock of the same object.
- Only critical sections using the same lock object exclude each other; guard all reads and writes of shared state with one lock; prefer a private lock object.
- Monitors are reentrant. Keep critical sections short; never do I/O or call foreign code while holding a lock.
- Use `wait`/`notifyAll` in a `while` loop, holding the lock; wake-ups can be spurious or stolen.
- Deadlock needs a lock cycle. Prevent it with a global lock order; diagnose it with `jstack`.

8.5 Thread Confinement, Immutability and Liveness

The cheapest way to make shared data safe is to not share it, or to share it only when it can never change. This chapter covers those two strategies, and then the failure modes that remain even when your locking is correct: your threads can stop making progress.

8.5.1 Do not share: thread confinement

An object that is only ever reachable from one thread needs no synchronisation. This is **thread confinement**, and it is the foundation of most safe concurrent code. There are three levels:

- **Stack confinement.** Local variables live on the thread's own stack. An object created in a method and never stored in a field or passed elsewhere is private to that thread. Primitive locals are always confined. This is why most of your code is already thread-safe.
- **Ad hoc confinement.** You *promise* by convention that only one thread touches the object (a Swing/JavaFX UI component touched only by the UI thread). The compiler cannot check it, so use it sparingly and document it.
- `ThreadLocal` **confinement.** Each thread gets its own copy of a variable, enforced by the library.

8.5.2 ThreadLocal

A `ThreadLocal<T>` holds one value *per thread*. Under the hood each `Thread` object has a small map from `ThreadLocal` objects to values; `get()` and `set()` look up the current thread's map. Use cases:

1. **Per-thread instances of non-thread-safe helpers.** The classic example is `SimpleDateFormat`, which keeps parsing state in the object and corrupts output if shared (today you would use the immutable `DateTimeFormatter` instead).
2. **Implicit per-request context** that you do not want to pass through every method: a request id for logging, the current user, a transaction. (This is how Spring's transaction and security contexts reach deep code.)

Confine.java JAVA

```java
import java.text.SimpleDateFormat;
import java.util.Date;

public class Confine {
    // SimpleDateFormat is NOT thread-safe; give each thread its own copy.
    private static final ThreadLocal<SimpleDateFormat> FMT =
        ThreadLocal.withInitial(() -> new SimpleDateFormat("yyyy-MM-dd"));

    private static final ThreadLocal<String> REQUEST_ID = new ThreadLocal<>();

    static void handle(String id) {
        REQUEST_ID.set(id);
        try {
            step();
        } finally {
            REQUEST_ID.remove();                // essential in thread pools
        }
    }
    static void step() {
        System.out.println(Thread.currentThread().getName() + " handling " + REQUEST_ID.get()
            + ", date " + FMT.get().format(new Date(0L)));
    }
    public static void main(String[] args) throws Exception {
        Thread a = new Thread(() -> handle("req-1"), "t-a");
        Thread b = new Thread(() -> handle("req-2"), "t-b");
        a.start(); b.start(); a.join(); b.join();
        System.out.println("main sees: " + REQUEST_ID.get());
    }
}
```

Run it TERMINAL

```
$ java Confine.java
t-a handling req-1, date 1970-01-01
t-b handling req-2, date 1970-01-01
main sees: null
```

What to notice: `ThreadLocal.withInitial` creates each thread's value lazily on first `get()`. The line order of `t-a` and `t-b` varies. `main` sees `null` because it never called `set`: the values are invisible to other threads. The `try`/`finally` with `remove()` is not decoration; it is the most important line.

ThreadLocal leaks in thread pools

Pool threads live as long as the application. If a task calls `set` and never `remove`, the value stays attached to that pool thread, long after the task ended. Two problems follow:

1. **Stale data bleeding between requests.** The next task on that thread calls `get()` and sees the previous request's user or transaction. This is a security bug, not just a leak.
2. **Memory leaks.** A value that references a large object graph (or a class loader, in an application server) is retained as long as the thread lives. After a web-app redeploy, the old class loader cannot be collected.

Always `remove()` in a `finally` when the thread is not yours. Prefer to avoid `ThreadLocal` for passing data when an explicit parameter works: hidden state makes code harder to test and reason about. Also note `InheritableThreadLocal` copies the parent's value into a child thread at creation, which interacts badly with pools (the value is copied once, when the pool creates the thread, not per task).

Virtual threads (Java 21, a later chapter) support `ThreadLocal`, but a program can have millions of virtual threads, so a per-thread cache of heavy objects becomes expensive. Java 25 finalised **scoped values** (JEP 506), an immutable, bounded-lifetime alternative for passing context to called code and child tasks; it is designed with virtual threads and

structured concurrency in mind. (Scoped values are not available on Java 21 without preview, so we do not run an example here.)

> **WATCH OUT**
> In an application server or any pool, a `ThreadLocal` without `remove()` is a bug waiting for load. If you see `ThreadLocal` in code review, look for the `remove()` call.

8.5.3 Do not mutate: immutability

An **immutable** object never changes state after construction. All its fields are set once; nothing can see it half-built (given safe construction, see the final-field rule of the memory model chapter). Immutable objects are thread-safe by construction, without locks: any number of threads can read them, and they can be shared and cached freely. `String`, the boxed numbers, `BigDecimal`, `java.time` types, records of immutable components and `List.of(...)` are all immutable.

The checklist for an immutable class:

1. Declare the class `final` (or make constructors private), so no subclass can add mutable state.
2. Make all fields `private final`.
3. Do not provide setters or any method that changes state.
4. Make **defensive copies** of mutable inputs in the constructor and of mutable outputs in getters (or store unmodifiable collections).
5. Do not let `this` escape from the constructor.

Immutable.java JAVA

```java
import java.util.List;

public final class Immutable {
    record Point(int x, int y) { }                      // shallowly immutable, safe to share
    static final class Order {
        private final String id;
        private final List<String> items;
        Order(String id, List<String> items) {
            this.id = id;
            this.items = List.copyOf(items);             // defensive copy, unmodifiable
        }
        List<String> items() { return items; }
        String id() { return id; }
    }
    public static void main(String[] args) {
        var src = new java.util.ArrayList<>(List.of("book", "pen"));
        Order o = new Order("A1", src);
        src.add("laptop");                                // does not leak into the order
        System.out.println(o.items());
        try {
            o.items().add("x");
        } catch (UnsupportedOperationException e) {
            System.out.println("read-only");
        }
        System.out.println(new Point(1, 2));
    }
}
```

Run it TERMINAL

```
$ java Immutable.java
[book, pen]
read-only
Point[x=1, y=2]
```

`List.copyOf(items)` takes a snapshot: the later `src.add("laptop")` does not leak into the order, and the list the order exposes rejects modification. A record such as `Point` is *shallowly* immutable: its fields are final, but if a component is a mutable object (for example an `ArrayList`) the record is not deeply immutable. Immutability costs allocation when you "change" something (you create a new object), but modern allocation and garbage collection are cheap, and the safety is usually well worth it.

Thread-safety strategies at a glance

Strategy	Idea	Typical tool
Confinement	Only one thread ever sees the data	Locals, `ThreadLocal`, one-thread executors
Immutability	Data never changes after publication	`final` fields, records, `List.of`
Mutual exclusion	One thread at a time touches the data	`synchronized`, `ReentrantLock`
Atomic variables	Lock-free single-variable updates	`AtomicInteger`, `AtomicReference`
Concurrent data structures	Thread-safe building blocks	`ConcurrentHashMap`, `BlockingQueue`
Message passing	Hand data over, then stop touching it	Queues, executors, futures

Strategies for sharing state.

Pick them in this order: confine if possible; make immutable if possible; otherwise lock or use a ready-made concurrent class.

8.5.4 Liveness problems

A program is **safe** if nothing bad happens (no corrupted data) and **live** if something good eventually happens (work gets done). Locks buy safety, and each one adds a risk to liveness. We already met deadlock. The three classic liveness failures:

Deadlock

Threads wait on each other in a cycle and none can proceed. All four Coffman conditions must hold: mutual exclusion, hold and wait, no pre-emption, circular wait. Prevention: lock ordering, a single coarse lock, `tryLock` with timeouts. Detection: `jstack` or `ThreadMXBean.findDeadlockedThreads()` for monitors and ownable synchronizers. Deadlocks are not limited to locks: a thread pool whose tasks wait for other tasks submitted to the same (full) pool can deadlock too ("thread-starvation deadlock"), for example a fixed pool of 2 threads where both tasks block waiting for subtasks that are queued behind them.

Livelock

Threads are not blocked; they are busy reacting to each other and make no progress. The standard picture is two people in a corridor, each stepping aside in the same direction, repeatedly. In code, two threads that each take a lock, fail to get the second lock with `tryLock`, release the first, and retry immediately in lock-step can livelock. The remedy is **randomised backoff**: sleep a random short time before retrying, which breaks the symmetry. A livelock burns CPU, so in a dump the threads show RUNNABLE or TIMED_WAITING and the CPU is high, which distinguishes it from a deadlock (BLOCKED, CPU idle).

Starvation

A thread is ready to run but never gets the resource it needs, because others keep taking it. Causes: a thread holding a lock for a very long time; a low-priority task behind a flood of high-priority ones; **unfair locks**. Intrinsic locks make no fairness promise, and neither does `ReentrantLock` by default, which allows *barging* (a new arrival may take the lock ahead of queued threads) because that is faster. `new ReentrantLock(true)` is fair (first come, first served) at a cost to throughput. Avoid long critical sections and let queues, not priorities, order work.

Symptom	Thread states	CPU
`Deadlock`	BLOCKED or WAITING in a cycle	idle
`Livelock`	RUNNABLE or TIMED_WAITING, changing	high
`Starvation`	one thread waiting long, others busy	busy
`Lost wake-up`	all WAITING, nothing holds any lock	idle

Telling liveness failures apart in a thread dump.

INTERVIEW ANGLE
How can you avoid deadlock? Break one of its four conditions. The practical approaches: acquire locks in a fixed global order; hold as few locks as possible and for as short a time as possible; do not call foreign (overridable) code while holding a lock; use `tryLock` with a timeout and back off; and prefer higher-level constructs (concurrent collections, executors, immutable data) that avoid explicit locking. To diagnose, take a thread dump with `jstack`/`jcmd Thread.print`, which reports Java-level deadlocks.

INTERVIEW ANGLE
What is the difference between a deadlock and a livelock? In a deadlock the threads are blocked forever waiting for each other and use no CPU. In a livelock they keep running and responding to each other (retrying, backing off in lock-step) but make no progress, so CPU use is high.

Check yourself

1. What does this program print, and why does it matter in a thread pool? `static ThreadLocal<String> user = new ThreadLocal<>();` Task 1 on pool thread P calls `user.set("alice")` and ends. Task 2 later runs on P and calls `user.get()`.
2. Is a `record Order(String id, List<String> items)` immutable? How would you make it so?
3. List the three levels of thread confinement. Which one is checked by the compiler?
4. What does each of these look like in a thread dump: deadlock, livelock? What differs in CPU usage?
5. Why can a fixed thread pool of 2 threads deadlock when each task submits a subtask to the same pool and waits for its result?
6. Why is `new ReentrantLock()` not fair by default? What does `new ReentrantLock(true)` change?

Answers

1. Task 2 prints `alice` (the stale value), because the value is stored on the thread, not the task. This leaks identity between requests; always call `remove()` in `finally`.
2. Only shallowly: the `items` list may be mutated by whoever holds it. Make a defensive copy in a compact constructor: `items = List.copyOf(items);`.
3. Stack confinement (locals), ad hoc confinement (convention), `ThreadLocal`. None of them is enforced by the type system; the nearest is stack confinement by never letting a local object escape.
4. Deadlock: BLOCKED/WAITING threads in a cycle, CPU idle, and `jstack` reports the deadlock. Livelock: threads RUNNABLE or alternating, CPU busy, no progress.
5. Both pool threads are occupied by the parent tasks, blocked in `Future.get()`; the subtasks wait in the queue for a free thread that never comes. Use a bigger or separate pool, or avoid blocking on subtasks.
6. Fairness costs throughput, since a barging thread can take a just-released lock without the cost of waking a queued thread. `true` makes the lock grant to the longest-waiting thread, reducing starvation but lowering throughput.

KEY POINTS
Safe sharing comes before locking.

- Prefer confinement (locals, `ThreadLocal`) and immutability (final fields, defensive copies, records, `List.copyOf`) to locks.
- `ThreadLocal` needs `remove()` in `finally` on pooled threads, or values leak and bleed across requests.
- Liveness failures: deadlock (cycle, idle), livelock (busy, no progress), starvation (never scheduled or never gets the lock).
- Prevent deadlock with lock ordering, short critical sections and timeouts; diagnose with thread dumps.
- Java 25 scoped values (JEP 506) are the modern alternative to `ThreadLocal` for context passing.

8.6 Locks and Atomics

The `synchronized` keyword is simple and safe, but it is a blunt tool. You cannot give up waiting for it, you cannot ask "is it free right now?", you cannot interrupt a thread that is blocked on it, and a monitor has exactly one wait set. The `java.util.concurrent.locks` package (Java 5) and the `java.util.concurrent.atomic` package give you finer instruments. This chapter covers both: explicit locks, which still block threads, and atomic variables, which usually do not block at all.

The rule of thumb for the whole chapter: **start with `synchronized` or a ready-made concurrent class; reach for a `Lock` only when you need one of its extra abilities; reach for atomics for a single shared variable.** A higher-level class from the next chapters (executors, concurrent collections) is better still.

8.6.1 ReentrantLock

`ReentrantLock` is a mutual-exclusion lock with the same basic semantics as an intrinsic monitor (one owner at a time, reentrant), plus extras. Reentrant means the owning thread can lock again without deadlocking itself; the lock counts how many times it was taken and is free only when `unlock()` was called the same number of times.

The price of the extras is discipline. A monitor is released automatically at the end of the `synchronized` block, even on exceptions. A `Lock` is released only when you call `unlock()`. The idiom is non-negotiable: call `lock()` **before** the `try`, and `unlock()` in `finally`.

LockDemo.java JAVA

```
import java.util.concurrent.TimeUnit;
import java.util.concurrent.locks.ReentrantLock;

public class LockDemo {
    static class Account {
        private final ReentrantLock lock = new ReentrantLock();
        private long balance;

        Account(long balance) { this.balance = balance; }

        void deposit(long amount) {
            lock.lock();                // acquire BEFORE the try block
            try {
                balance += amount;
            } finally {
                lock.unlock();          // always release in finally
            }
        }

        boolean tryDeposit(long amount, long millis) throws InterruptedException {
            if (!lock.tryLock(millis, TimeUnit.MILLISECONDS)) {
                return false;           // could not get the lock in time
            }
            try {
                balance += amount;
                return true;
            } finally {
                lock.unlock();
            }
        }

        long balance() {
            lock.lock();
            try { return balance; } finally { lock.unlock(); }
        }

        ReentrantLock lock() { return lock; }
    }

    public static void main(String[] args) throws Exception {
        Account acc = new Account(100);
        acc.deposit(50);
        System.out.println("balance = " + acc.balance());

        // Reentrancy: the same thread may lock again; hold count goes up.
        acc.lock().lock();
        acc.lock().lock();
        System.out.println("hold count = " + acc.lock().getHoldCount());
```

LockDemo.java (continued) JAVA

```
        acc.lock().unlock();
        acc.lock().unlock();
        System.out.println("locked after unlocks? " + acc.lock().isLocked());

        // Another thread holds the lock; tryLock with a timeout gives up.
        Thread holder = new Thread(() -> {
            acc.lock().lock();
            try {
                Thread.sleep(500);
            } catch (InterruptedException e) {
                Thread.currentThread().interrupt();
            } finally {
                acc.lock().unlock();
            }
        });
        holder.start();
        Thread.sleep(100);              // let the holder take the lock
        System.out.println("tryLock(100ms) = " + acc.lock().tryLock(100, TimeUnit.MILLISECONDS));
        System.out.println("tryDeposit while held = " + acc.tryDeposit(10, 50));
        holder.join();
        System.out.println("tryDeposit after release = " + acc.tryDeposit(10, 50));
        System.out.println("final balance = " + acc.balance());
    }
}
```

Output TERMINAL

```
$ java LockDemo.java
balance = 150
hold count = 2
locked after unlocks? false
tryLock(100ms) = false
tryDeposit while held = false
tryDeposit after release = true
final balance = 160
```

What to notice

- `lock()` sits before `try`. If `lock()` itself threw and you had put it inside the `try`, the `finally` would call `unlock()` on a lock you do not hold and throw `IllegalMonitorStateException`, hiding the original error.
- `getHoldCount()` shows reentrancy: two `lock()` calls need two `unlock()` calls.
- `tryLock(timeout, unit)` returns `false` instead of waiting forever. This is the practical cure for the deadlocks you saw in the liveness chapter: if you cannot get the second lock in time, release the first and retry or fail.

What a Lock can do that synchronized cannot

Ability	synchronized	ReentrantLock
Try without blocking	no	`tryLock()`
Give up after a timeout	no	`tryLock(time, unit)`
Respond to interruption while waiting	no	`lockInterruptibly()`
Fair (longest waiter first) option	no	`new ReentrantLock(true)`
Several wait queues per lock	one wait set	`newCondition()` many
Released automatically	yes	no, you call `unlock()`
Scope	block or method	any code, even across methods
Inspect state (owner, queue length)	no	`isLocked()`, `getQueueLength()`

Intrinsic monitor versus ReentrantLock.

Fairness

`new ReentrantLock(true)` creates a **fair** lock: when the lock is released it goes to the thread that has waited longest. The default is **non-fair**: a thread that calls `lock()` just as the lock is released may "barge" in front of queued threads. That sounds unjust, but it is faster, because the queued thread must be woken up, which takes microseconds, while the running thread can take the lock at once. Fair locks cost noticeably less throughput under contention. Use them only if you have observed starvation. Note that fairness of the lock does not guarantee fair thread scheduling, and the untimed `tryLock()` ignores fairness (it barges even on a fair lock).

8.6.2 Condition: many wait sets per lock

A monitor has one wait set, used by `wait/notify`. In a bounded buffer that single set is shared by producers waiting for space and consumers waiting for items, so you must use `notifyAll()` and wake everyone. A `Condition` is created from a lock with `lock.newCondition()`, and one lock can have several. `await()`, `signal()` and `signalAll()` correspond to `wait()`, `notify()` and `notifyAll()`, and as with monitors you must hold the lock when you call them.

BoundedBuffer.java JAVA

```
import java.util.ArrayDeque;
import java.util.Deque;
import java.util.concurrent.locks.Condition;
import java.util.concurrent.locks.ReentrantLock;

public class BoundedBuffer {
    static class Buffer<T> {
        private final Deque<T> items = new ArrayDeque<>();
        private final int capacity;
        private final ReentrantLock lock = new ReentrantLock();
        private final Condition notFull = lock.newCondition();
        private final Condition notEmpty = lock.newCondition();

        Buffer(int capacity) { this.capacity = capacity; }

        void put(T item) throws InterruptedException {
            lock.lock();
            try {
                while (items.size() == capacity) {
                    notFull.await();        // releases the lock while waiting
                }
                items.addLast(item);
                notEmpty.signal();          // wake one consumer only
            } finally {
                lock.unlock();
            }
        }

        T take() throws InterruptedException {
            lock.lock();
            try {
                while (items.isEmpty()) {
                    notEmpty.await();
                }
                T item = items.removeFirst();
                notFull.signal();           // wake one producer only
                return item;
            } finally {
                lock.unlock();
            }
        }
    }

    public static void main(String[] args) throws Exception {
        Buffer<Integer> buf = new Buffer<>(3);
        int n = 1000;
        long[] sum = new long[1];

        Thread producer = new Thread(() -> {
            try {
                for (int i = 1; i <= n; i++) buf.put(i);
            } catch (InterruptedException e) { Thread.currentThread().interrupt(); }
        });
        Thread consumer = new Thread(() -> {
            try {
                for (int i = 1; i <= n; i++) sum[0] += buf.take();
```

BoundedBuffer.java (continued) JAVA

```
            } catch (InterruptedException e) { Thread.currentThread().interrupt(); }
        });
        producer.start();
        consumer.start();
        producer.join();
        consumer.join();
        System.out.println("consumed sum = " + sum[0] + " (expected " + (n * (n + 1L) / 2) + ")");
    }
}
```

Output TERMINAL

```
$ java BoundedBuffer.java
consumed sum = 500500 (expected 500500)
```

The two rules from the monitor chapter carry over unchanged. First, always `await()` in a `while` loop that rechecks the condition, because of spurious wakeups and because another thread may have consumed the item between the signal and your wake-up. Second, `await()` releases the lock while it waits and re-acquires it before returning. With two conditions a producer only ever wakes consumers and vice versa, so `signal()` (one thread) is enough here. In real code you will rarely hand-write this; `ArrayBlockingQueue` (later in this Part) is exactly this class, already tested.

GOTCHA

Do not call `wait()`/`notify()` on a `Lock` or `Condition` object. `Condition` is an ordinary object, so it also has the `Object.wait()` and `notify()` methods. Calling them compiles, but they use the object's own monitor, not the lock. The real methods are `await()` and `signal()`. If you see `IllegalMonitorStateException` from a `Condition`, check which one you called.

8.6.3 ReadWriteLock

Many data structures are read far more often than they are written. A plain lock makes readers wait for each other for no reason. A `ReadWriteLock` has two locks: the **read lock** can be held by any number of threads as long as nobody holds the write lock, and the **write lock** is exclusive. `ReentrantReadWriteLock` is the standard implementation.

RwDemo.java JAVA

```
import java.util.HashMap;
import java.util.Map;
import java.util.concurrent.locks.ReentrantReadWriteLock;

public class RwDemo {
    static class PriceCache {
        private final Map<String, Integer> prices = new HashMap<>();
        private final ReentrantReadWriteLock rw = new ReentrantReadWriteLock();

        Integer get(String sku) {
            rw.readLock().lock();            // many readers may hold this at once
            try {
                return prices.get(sku);
            } finally {
                rw.readLock().unlock();
            }
        }

        void put(String sku, int price) {
            rw.writeLock().lock();           // exclusive: no readers, no other writers
            try {
                prices.put(sku, price);
            } finally {
                rw.writeLock().unlock();
            }
        }

        ReentrantReadWriteLock lock() { return rw; }
    }

    public static void main(String[] args) {
        PriceCache cache = new PriceCache();
        cache.put("book", 499);
        System.out.println("book = " + cache.get("book"));
```

RwDemo.java (continued) JAVA

```
        var rw = cache.lock();
        rw.readLock().lock();
        rw.readLock().lock();
        System.out.println("read locks held = " + rw.getReadLockCount());
        rw.readLock().unlock();
        rw.readLock().unlock();

        // Downgrading is allowed: take the read lock while still holding the write lock.
        rw.writeLock().lock();
        rw.readLock().lock();
        rw.writeLock().unlock();
        System.out.println("downgraded: write locked = " + rw.isWriteLocked()
                + ", readers = " + rw.getReadLockCount());
        rw.readLock().unlock();

        // Upgrading is NOT: a thread holding the read lock that asks for the write lock
        // would wait for itself. We show it with tryLock so the program does not hang.
        rw.readLock().lock();
        boolean upgraded = rw.writeLock().tryLock();
        System.out.println("upgrade with tryLock succeeded? " + upgraded);
        rw.readLock().unlock();
    }
}
```

Output TERMINAL

```
$ java RwDemo.java
book = 499
read locks held = 2
downgraded: write locked = false, readers = 1
upgrade with tryLock succeeded? false
```

What to notice

- Two read locks were held at the same time (`read locks held = 2`). That is the whole benefit.
- **Downgrading** (write lock, then take the read lock, then release the write lock) is supported. It lets you update data and then read it consistently without a window where another writer sneaks in.
- **Upgrading** (holding a read lock and asking for the write lock) is not supported. With a blocking `writeLock().lock()` the thread would wait for all readers to leave, including itself, so it deadlocks. Release the read lock first, take the write lock, and recheck your condition, since the world may have changed in between.

A read-write lock is not automatically faster. Locking costs more than a plain lock, so it pays off only when reads dominate, the read section is not tiny, and there is real contention. Measure before you adopt it. For a map shared between threads, `ConcurrentHashMap` is almost always the better answer than a `HashMap` behind a read-write lock.

8.6.4 StampedLock

`StampedLock` (Java 8) adds a third mode, the **optimistic read**: you read without locking, then check afterwards whether any write happened. If not, you got a consistent snapshot for free. If a write did happen, you retry or fall back to a real read lock. Each lock acquisition returns a `long` **stamp** that you pass back to release or validate.

StampedDemo.java JAVA

```
import java.util.concurrent.locks.StampedLock;

public class StampedDemo {
    static class Point {
        private double x, y;
        private final StampedLock sl = new StampedLock();

        void move(double dx, double dy) {
            long stamp = sl.writeLock();
            try {
                x += dx;
                y += dy;
            } finally {
                sl.unlockWrite(stamp);
            }
        }
```

StampedDemo.java (continued) JAVA

```
        double distanceFromOrigin() {
            long stamp = sl.tryOptimisticRead();     // no lock taken, just a version stamp
            double cx = x, cy = y;                   // read the fields into locals
            if (!sl.validate(stamp)) {               // did a write happen meanwhile?
                stamp = sl.readLock();               // fall back to a real read lock
                try {
                    cx = x;
                    cy = y;
                } finally {
                    sl.unlockRead(stamp);
                }
            }
            return Math.hypot(cx, cy);
        }
    }

    public static void main(String[] args) throws Exception {
        Point p = new Point();
        p.move(3, 4);
        System.out.println("distance = " + p.distanceFromOrigin());

        StampedLock sl = new StampedLock();
        long s = sl.tryOptimisticRead();
        System.out.println("valid before write? " + sl.validate(s));
        long w = sl.writeLock();
        sl.unlockWrite(w);
        System.out.println("valid after write?  " + sl.validate(s));
    }
}
```

Output TERMINAL

```
$ java StampedDemo.java
distance = 5.0
valid before write? true
valid after write?  false
```

The pattern in `distanceFromOrigin` is the one to memorise: take the stamp, copy fields into **local variables**, call `validate(stamp)`, and use only the locals. Between the copy and the validation the fields may be half-updated (a torn pair of `x` and `y`), but if validation succeeds, no write intervened, so the copies were consistent.

WATCH OUT

StampedLock is not reentrant and has no Conditions. Locking the same `StampedLock` twice from one thread (for example a write lock followed by another write lock) deadlocks, because it does not track owners. Stamps are not tied to threads, so any thread can release with the stamp. It is also easy to misuse: forgetting to validate, or using fields after a failed validation. Use it for small, hot, read-mostly classes like the `Point` above, and prefer simpler tools elsewhere.

8.6.5 Atomic variables and compare-and-swap

Consider `count++` on a shared `int`. It is three steps (read, add, write), so two threads can both read 7 and both write 8, losing an update. You saw the fix with `synchronized`. Modern CPUs offer a cheaper primitive: **compare-and-swap (CAS)**, a single indivisible instruction that says "set this memory location to NEW only if it currently holds EXPECTED, and tell me whether it worked". In Java it is exposed as `compareAndSet(expected, new)`.

The classes in `java.util.concurrent.atomic` wrap a single variable with CAS-based operations: `AtomicInteger`, `AtomicLong`, `AtomicBoolean`, `AtomicReference<V>`, plus arrays (`AtomicIntegerArray`), and field updaters. Their reads and writes have `volatile` semantics, so they also give visibility and ordering.

The usage pattern is a **CAS loop**: read the current value, compute the new value, try to CAS, and if another thread got there first, loop and recompute. Nobody ever blocks, so a stalled thread cannot hold others up; this style is called **lock-free**.

CasDemo.java JAVA

```
import java.util.concurrent.atomic.AtomicInteger;
import java.util.concurrent.atomic.AtomicReference;

public class CasDemo {
    // A lock-free "set to the maximum" built from compareAndSet.
```

CasDemo.java (continued) JAVA

```java
    static void updateMax(AtomicInteger max, int candidate) {
        int current = max.get();
        while (candidate > current) {
            if (max.compareAndSet(current, candidate)) return;   // we won the race
            current = max.get();                                  // someone changed it: retry
        }
    }

    // A tiny lock-free stack (Treiber stack) built on AtomicReference.
    static class Stack<T> {
        private record Node<T>(T value, Node<T> next) {}
        private final AtomicReference<Node<T>> head = new AtomicReference<>();

        void push(T v) {
            Node<T> oldHead, newHead;
            do {
                oldHead = head.get();
                newHead = new Node<>(v, oldHead);
            } while (!head.compareAndSet(oldHead, newHead));
        }

        T pop() {
            Node<T> oldHead;
            do {
                oldHead = head.get();
                if (oldHead == null) return null;
            } while (!head.compareAndSet(oldHead, oldHead.next()));
            return oldHead.value();
        }
    }

    public static void main(String[] args) throws Exception {
        AtomicInteger n = new AtomicInteger(5);
        System.out.println("CAS 5->6: " + n.compareAndSet(5, 6) + ", value " + n.get());
        System.out.println("CAS 5->7: " + n.compareAndSet(5, 7) + ", value " + n.get());
        System.out.println("updateAndGet(x*10) = " + n.updateAndGet(x -> x * 10));
        System.out.println("accumulateAndGet(max with 3) = " + n.accumulateAndGet(3, Math::max));

        AtomicInteger max = new AtomicInteger(Integer.MIN_VALUE);
        Thread[] ts = new Thread[4];
        for (int t = 0; t < ts.length; t++) {
            final int base = t * 1000;
            ts[t] = new Thread(() -> { for (int i = 0; i < 1000; i++) updateMax(max, base + i); });
            ts[t].start();
        }
        for (Thread t : ts) t.join();
        System.out.println("max = " + max.get());

        Stack<String> s = new Stack<>();
        s.push("a"); s.push("b"); s.push("c");
        System.out.println(s.pop() + s.pop() + s.pop() + " " + s.pop());
    }
}
```

Output TERMINAL

```
$ java CasDemo.java
CAS 5->6: true, value 6
CAS 5->7: false, value 6
updateAndGet(x*10) = 60
accumulateAndGet(max with 3) = 60
max = 3999
cba null
```

What to notice

- `compareAndSet(5, 7)` returned `false` because the value was already 6. A failed CAS is normal, not an error: it means "someone else changed it first".
- Prefer the built-in loops: `updateAndGet(fn)`, `getAndUpdate(fn)`, `accumulateAndGet(x, fn)`, `incrementAndGet()`. The function you pass must be **side-effect free**, because it can run more than once when the CAS fails.
- `updateMax` and the `Stack` show the shape of every lock-free algorithm: snapshot, compute, CAS, retry. The stack's `push` creates a new immutable `Node` each time; immutability is what makes the CAS safe.

Atomics versus synchronized versus volatile

Now the race from the memory-model chapter, measured with four threads of 500,000 increments each:

CounterRace.java JAVA

```java
import java.util.concurrent.atomic.AtomicLong;
import java.util.concurrent.atomic.LongAdder;

public class CounterRace {
    static int plain;
    static volatile int vol;
    static int locked;
    static final Object LOCK = new Object();
    static final AtomicLong atomic = new AtomicLong();
    static final LongAdder adder = new LongAdder();

    static void runThreads(int threads, Runnable task) throws InterruptedException {
        Thread[] ts = new Thread[threads];
        for (int i = 0; i < threads; i++) { ts[i] = new Thread(task); ts[i].start(); }
        for (Thread t : ts) t.join();
    }

    public static void main(String[] args) throws Exception {
        int threads = 4, perThread = 500_000;
        long expected = (long) threads * perThread;

        runThreads(threads, () -> { for (int i = 0; i < perThread; i++) plain++; });
        runThreads(threads, () -> { for (int i = 0; i < perThread; i++) vol++; });
        runThreads(threads, () -> {
            for (int i = 0; i < perThread; i++) synchronized (LOCK) { locked++; }
        });
        runThreads(threads, () -> {
            for (int i = 0; i < perThread; i++) atomic.incrementAndGet();
        });
        runThreads(threads, () -> {
            for (int i = 0; i < perThread; i++) adder.increment();
        });

        System.out.println("expected      = " + expected);
        System.out.println("plain int     correct? " + (plain == expected));
        System.out.println("volatile int  correct? " + (vol == expected));
        System.out.println("synchronized  = " + locked);
        System.out.println("AtomicLong    = " + atomic.get());
        System.out.println("LongAdder     = " + adder.sum());
    }
}
```

Output TERMINAL

```
$ java CounterRace.java
expected      = 2000000
plain int     correct? false
volatile int  correct? false
synchronized  = 2000000
AtomicLong    = 2000000
LongAdder     = 2000000
```

Both the plain and the `volatile` counters lose updates (the exact counts differ on every run, so the program only prints whether the result was correct; it was not). `volatile` fixes visibility, not atomicity: `vol++` is still read-modify-write. `synchronized`, `AtomicLong` and `LongAdder` are all exact.

LongAdder: atomics under heavy contention

When many threads hammer one `AtomicLong`, they all spin on CAS against the same memory location, and most attempts fail and retry. `LongAdder` (Java 8) avoids this by keeping a **set of cells**: under contention each thread updates a different cell, and `sum()` adds the cells up on demand. `increment()` is much cheaper under contention, at the price of a `sum()` that is not a single atomic snapshot and extra memory. Use `LongAdder` (or `LongAccumulator` for max/min) for statistics counters such as request counts and metrics that are written often and read rarely. Use `AtomicLong` when you need the exact current value atomically, for example to generate unique IDs with `incrementAndGet()`.

The ABA problem

CAS asks "is the value still A?". If another thread changed it A to B and back to A, the CAS succeeds although the world changed. For a counter that is harmless. For a linked structure it can be fatal: a node may have been popped, reused and pushed again, so your idea of what `next` points to is stale. In Java, garbage collection removes the commonest form of the bug (a node cannot be recycled while you still hold a reference to it), but it can still appear with values, or with pooled and reused objects. `AtomicStampedReference` pairs the reference with an `int` stamp (a version number) and CASes both; `AtomicMarkableReference` pairs it with a boolean flag.

AbaDemo.java JAVA

```java
import java.util.concurrent.atomic.AtomicInteger;
import java.util.concurrent.atomic.AtomicStampedReference;

public class AbaDemo {
    public static void main(String[] args) {
        // Plain CAS cannot see A -> B -> A.
        AtomicInteger value = new AtomicInteger(100);
        int seen = value.get();                 // thread 1 reads 100 and is paused
        value.compareAndSet(100, 50);           // thread 2: 100 -> 50
        value.compareAndSet(50, 100);           // thread 2: 50 -> 100 (looks untouched)
        System.out.println("plain CAS succeeds despite A-B-A: " + value.compareAndSet(seen, 999));

        // A stamp (version) makes the change visible.
        AtomicStampedReference<Integer> ref = new AtomicStampedReference<>(100, 0);
        int[] stampHolder = new int[1];
        Integer seenValue = ref.get(stampHolder);
        int seenStamp = stampHolder[0];
        ref.compareAndSet(100, 50, ref.getStamp(), ref.getStamp() + 1);
        ref.compareAndSet(50, 100, ref.getStamp(), ref.getStamp() + 1);
        boolean ok = ref.compareAndSet(seenValue, 999, seenStamp, seenStamp + 1);
        System.out.println("stamped CAS succeeds: " + ok + ", stamp now " + ref.getStamp());
    }
}
```

Output TERMINAL

```
$ java AbaDemo.java
plain CAS succeeds despite A-B-A: true
stamped CAS succeeds: false, stamp now 2
```

The plain CAS "succeeded" although the value had changed twice. The stamped version failed, correctly, because the stamp had moved from 0 to 2.

Under the hood

Atomics are implemented with `VarHandle` (Java 9, which replaced the internal `Unsafe` calls). On x86-64 a CAS compiles to a single `lock cmpxchg` instruction; on ARM, to a load-linked/store-conditional pair or a dedicated atomic instruction. You can use `VarHandle` directly for fine-grained control (`compareAndSet`, `getAcquire`, `setRelease`), but that is library-author territory. `AtomicIntegerFieldUpdater` and friends let you do CAS on a `volatile` field of another class when you cannot afford an `AtomicInteger` object per instance.

INTERVIEW ANGLE

What is CAS, and what is its downside compared with a lock? CAS is a single hardware instruction that atomically sets a variable to a new value only if it still holds the expected value. Atomic classes loop on it (read, compute, CAS, retry) so threads never block. Downsides: under heavy contention many CAS attempts fail and burn CPU (use `LongAdder`), it protects only one variable (a multi-field invariant needs a lock or an immutable snapshot in an `AtomicReference`), and it is exposed to the ABA problem (use `AtomicStampedReference`).

INTERVIEW ANGLE

Is `volatile int count; count++` thread-safe? No. `volatile` guarantees that writes are visible and ordered, but `count++` is three operations (read, add, write), so two threads can interleave and lose an update. Use `AtomicInteger.incrementAndGet()` or `synchronized`.

REAL WORLD

In a Spring Boot service you rarely write `ReentrantLock` yourself. You meet atomics in metrics counters (Micrometer uses `LongAdder`-style counters), in ID generators, and in "run once" flags (`AtomicBoolean.compareAndSet(false, true)`). Locks appear inside libraries: connection pools, caches and `ConcurrentHashMap` itself.

Check yourself

1. Why must `lock.lock()` come before the `try` block, and `unlock()` be in `finally`?
2. What is wrong with this code? `rw.readLock().lock(); cache.put(k, v); rw.writeLock().lock();` (the thread already holds the read lock).
3. Two threads each run `for (int i = 0; i < 1000; i++) volatile_counter++;`. Can the final value be less than 2000? Why?
4. When would you choose `LongAdder` over `AtomicLong`, and when not?
5. In `updateAndGet(x -> x + sideEffect())` why is a side effect in the lambda a bug?
6. What does the stamp in `AtomicStampedReference` protect against?

Answers

1. If `lock()` is inside the `try` and fails, `finally` would unlock a lock you do not own (`IllegalMonitorStateException`), masking the real error. `finally` guarantees release on every exit path, including exceptions.
2. A read lock cannot be upgraded. The blocking `writeLock().lock()` waits for all readers to leave, including this thread, so it deadlocks. Release the read lock first, then take the write lock and recheck.
3. Yes, it can be less than 2000 (typically is). `volatile` gives visibility but `++` is read-modify-write, so increments are lost when threads interleave.
4. `LongAdder` for write-heavy counters (metrics) read rarely, where contention makes CAS retry. `AtomicLong` when you need the exact value atomically, such as an ID sequence, or when contention is low.
5. The function may be re-executed each time the CAS fails, so the side effect would happen an unpredictable number of times.
6. The ABA problem: the value changing A to B and back to A between your read and your CAS. The stamp changes on every update, so the CAS fails.

KEY POINTS

Explicit locks and atomics are the toolbox beyond `synchronized`.

- `ReentrantLock`: `lock()` before `try`, `unlock()` in `finally`; extras are `tryLock`, `lockInterruptibly`, fairness and multiple `Condition`s. Default is non-fair, which is faster.
- `ReadWriteLock` helps only when reads dominate; downgrading works, upgrading deadlocks. `StampedLock` adds optimistic reads but is non-reentrant and easy to misuse.
- CAS is one atomic instruction; atomic classes build lock-free CAS loops. Pass side-effect-free functions to `updateAndGet`.
- `volatile` is not atomic for `x++`. Use `AtomicLong`, or `LongAdder` for write-heavy counters.
- ABA is real for reference-based structures; `AtomicStampedReference` adds a version.

8.7 Executors and Thread Pools

Creating a `new Thread` per task is the first thing every beginner does, and the first thing that breaks in production. A platform thread reserves memory for its stack (typically up to 1 MB of address space), costs microseconds to start, and the operating system can only schedule so many of them. A burst of 50,000 requests becomes 50,000 threads, and the machine dies of memory pressure and context switching long before the work is done.

The fix is to **separate the task from how it runs**. You describe the work as a `Runnable` or `Callable`, and hand it to an **executor**, which owns a small pool of reusable worker threads. The pool bounds resource use, removes thread start-up cost from the request path, and gives you a single place to configure queueing, rejection, naming and shutdown. This is the Executor framework (Java 5), the workhorse of server-side Java before virtual threads and still the right tool for CPU-bound work and for bounding concurrency.

8.7.1 The interfaces

Three interfaces form a ladder.

- `Executor` has one method, `void execute(Runnable)`. It decouples submission from execution policy.
- `ExecutorService` adds lifecycle (`shutdown`, `shutdownNow`, `awaitTermination`, `close`) and task submission that returns a `Future`: `submit`, `invokeAll`, `invokeAny`.
- `ScheduledExecutorService` adds delayed and periodic tasks.

`Runnable` returns nothing and cannot throw a checked exception. `Callable<V>` returns a `V` and may throw `Exception`. A `Future<V>` is a handle to a result that may not exist yet: `get()` blocks until it does, `get(timeout, unit)` gives up, `cancel(mayInterrupt)` tries to stop the task, `isDone()` and `isCancelled()` report state. Behind the scenes `submit` wraps your task in a `FutureTask`, which is both a `Runnable` (the pool runs it) and a `Future` (you wait on it).

PoolBasics.java JAVA

```
import java.util.ArrayList;
import java.util.List;
import java.util.concurrent.Callable;
import java.util.concurrent.ExecutionException;
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;
import java.util.concurrent.Future;
import java.util.concurrent.TimeUnit;
import java.util.concurrent.TimeoutException;

public class PoolBasics {
    record Order(int id, int amount) {}

    static int price(Order o) throws InterruptedException {
        Thread.sleep(50);                       // pretend to call a pricing service
        if (o.id() == 3) throw new IllegalStateException("order 3 is invalid");
        return o.amount() * 2;
    }

    public static void main(String[] args) throws Exception {
        ExecutorService pool = Executors.newFixedThreadPool(3);
        try {
            List<Future<Integer>> futures = new ArrayList<>();
            for (int i = 1; i <= 4; i++) {
                Order order = new Order(i, i * 10);
                Callable<Integer> task = () -> price(order);
                futures.add(pool.submit(task));
            }
            for (int i = 0; i < futures.size(); i++) {
                try {
                    System.out.println("order " + (i + 1) + " -> " + futures.get(i).get());
                } catch (ExecutionException e) {
                    System.out.println("order " + (i + 1) + " failed: " + e.getCause());
                }
            }

            Future<String> slow = pool.submit(() -> { Thread.sleep(2000); return "late"; });
            try {
                slow.get(100, TimeUnit.MILLISECONDS);
            } catch (TimeoutException e) {
                System.out.println("timed out, cancelling: " + slow.cancel(true));
                System.out.println("isCancelled = " + slow.isCancelled()
                        + ", isDone = " + slow.isDone());
            }

            List<Callable<String>> jobs = List.of(() -> "A", () -> "B", () -> "C");
            for (Future<String> f : pool.invokeAll(jobs)) System.out.print(f.get() + " ");
            System.out.println("<- invokeAll keeps submission order");
            System.out.println("invokeAny returned one of A/B/C: "
                    + List.of("A", "B", "C").contains(pool.invokeAny(jobs)));
        } finally {
            pool.shutdown();
        }
    }
}
```

Output TERMINAL

```
$ java PoolBasics.java
order 1 -> 20
order 2 -> 40
order 3 failed: java.lang.IllegalStateException: order 3 is invalid
order 4 -> 80
timed out, cancelling: true
isCancelled = true, isDone = true
A B C <- invokeAll keeps submission order
invokeAny returned one of A/B/C: true
```

What to notice

- A task's exception does not escape on the worker thread. It is stored in the `Future` and re-thrown by `get()` wrapped in an `ExecutionException`; the real failure is `getCause()`. Order 3 failed while orders 1, 2 and 4 were unaffected.
- `get(100, MILLISECONDS)` throws `TimeoutException`, but the task keeps running unless you call `cancel(true)`, which interrupts the worker. The task must respond to interruption (here `Thread.sleep` does) for the cancel to take effect quickly.
- `invokeAll` waits for all tasks and returns futures in the order of the input list. `invokeAny` returns the result of one task that completed successfully and cancels the rest.
- `shutdown()` is in a `finally`. A pool whose threads are non-daemon keeps the JVM alive until it is shut down.

8.7.2 Inside ThreadPoolExecutor

Every standard pool is a `ThreadPoolExecutor` configured with seven parameters. Understanding them is the difference between a pool that behaves and one that deadlocks the service at 2 a.m.

Parameter	Meaning
`corePoolSize`	Threads kept alive even when idle (unless `allowCoreThreadTimeOut(true)`).
`maximumPoolSize`	Upper bound on threads. Extra threads above the core size are created only when the queue is full.
`keepAliveTime, unit`	How long a thread above the core size may sit idle before it is retired.
`workQueue`	A `BlockingQueue<Runnable>` holding tasks waiting for a worker.
`threadFactory`	Creates the threads: use it to set names, daemon status, an uncaught-exception handler.
`handler`	The `RejectedExecutionHandler` used when the pool cannot accept a task.

The ThreadPoolExecutor constructor parameters.

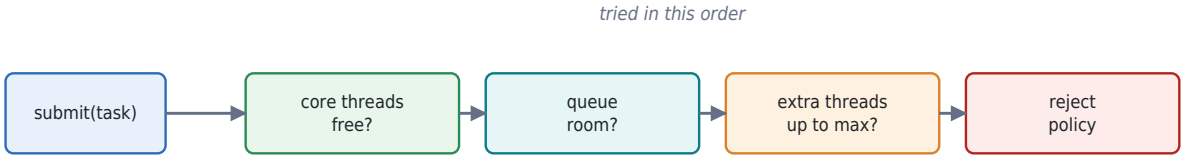


Figure 8.4. The submit path: a task first goes to a core thread, then to the queue, then to extra threads up to the maximum, and only then to the rejection handler.

The order of decisions on each `execute(task)` is the single most important fact about the class:

1. If fewer than `corePoolSize` threads are running, start a new thread and give it this task (even if other core threads are idle).
2. Otherwise try to put the task in the queue. If that works, a worker will pick it up.
3. If the queue refuses (it is full), and fewer than `maximumPoolSize` threads exist, start a new non-core thread.
4. Otherwise the pool is saturated: call the rejection handler.

Notice what this means: **the pool grows beyond the core size only after the queue is full.** With an unbounded queue (the default of `newFixedThreadPool`) step 3 never happens, so `maximumPoolSize` is irrelevant. The

demonstration below uses core 2, max 4 and a queue of capacity 2, and keeps every task blocked on a latch so you can watch the state evolve (the `active` count is read while workers are starting, so it can occasionally lag by one; the pool sizes and queue lengths are the point).

TpeDemo.java JAVA

```java
import java.util.concurrent.ArrayBlockingQueue;
import java.util.concurrent.CountDownLatch;
import java.util.concurrent.RejectedExecutionException;
import java.util.concurrent.ThreadPoolExecutor;
import java.util.concurrent.TimeUnit;

public class TpeDemo {
    static void fill(ThreadPoolExecutor pool, int tasks, CountDownLatch gate) {
        for (int i = 1; i <= tasks; i++) {
            final int id = i;
            try {
                pool.execute(() -> {
                    try { gate.await(); } catch (InterruptedException e) { /* ignore */ }
                });
                System.out.printf("task %d accepted: pool=%d active=%d queued=%d%n",
                        id, pool.getPoolSize(), pool.getActiveCount(), pool.getQueue().size());
            } catch (RejectedExecutionException e) {
                System.out.printf("task %d REJECTED (pool=%d, queued=%d)%n",
                        id, pool.getPoolSize(), pool.getQueue().size());
            }
        }
    }

    static void scenario(String title, ThreadPoolExecutor pool) throws Exception {
        System.out.println("--- " + title);
        CountDownLatch gate = new CountDownLatch(1);   // keeps every task busy until we open it
        fill(pool, 7, gate);
        gate.countDown();
        pool.shutdown();
        pool.awaitTermination(5, TimeUnit.SECONDS);
        System.out.println("completed tasks = " + pool.getCompletedTaskCount());
    }

    public static void main(String[] args) throws Exception {
        // core 2, max 4, queue of 2: tasks 1-2 start core threads, 3-4 are queued,
        // 5-6 create extra threads (up to max), 7 has nowhere to go.
        scenario("AbortPolicy (default)", new ThreadPoolExecutor(2, 4, 30, TimeUnit.SECONDS,
                new ArrayBlockingQueue<>(2), new ThreadPoolExecutor.AbortPolicy()));
        scenario("DiscardPolicy (task 7 is silently dropped)",
                new ThreadPoolExecutor(2, 4, 30, TimeUnit.SECONDS,
                new ArrayBlockingQueue<>(2), new ThreadPoolExecutor.DiscardPolicy()));
    }
}
```

Output TERMINAL

```
$ java TpeDemo.java
--- AbortPolicy (default)
task 1 accepted: pool=1 active=1 queued=0
task 2 accepted: pool=2 active=2 queued=0
task 3 accepted: pool=2 active=2 queued=1
task 4 accepted: pool=2 active=2 queued=2
task 5 accepted: pool=3 active=3 queued=2
task 6 accepted: pool=4 active=4 queued=2
task 7 REJECTED (pool=4, queued=2)
completed tasks = 6
--- DiscardPolicy (task 7 is silently dropped)
task 1 accepted: pool=1 active=1 queued=0
task 2 accepted: pool=2 active=2 queued=0
task 3 accepted: pool=2 active=2 queued=1
task 4 accepted: pool=2 active=2 queued=2
task 5 accepted: pool=3 active=3 queued=2
task 6 accepted: pool=4 active=4 queued=2
task 7 accepted: pool=4 active=4 queued=2
completed tasks = 6
```

Tasks 1 and 2 start the two core threads. Tasks 3 and 4 wait in the queue. Only when the queue is full do tasks 5 and 6 create threads 3 and 4. Task 7 finds a full queue and a full pool: with `AbortPolicy` it throws

`RejectedExecutionException`, with `DiscardPolicy` it silently vanishes (note the count of completed tasks is 6 in both runs). Silent loss is rarely what you want.

8.7.3 Queues

The queue choice defines the pool's behaviour more than any other parameter.

Queue	Effect
`` `LinkedBlockingQueue` (unbounded) ``	Never full, so the pool never grows past the core size and never rejects. Under overload the queue grows until `OutOfMemoryError`. Used by `newFixedThreadPool` and `newSingleThreadExecutor`.
`` `ArrayBlockingQueue(n)` ``	Bounded. Gives back-pressure: when full, the pool grows to the maximum, then rejects. The usual production choice.
`` `SynchronousQueue` ``	Capacity zero: a hand-off. Every task needs a free worker or a new thread. Used by `newCachedThreadPool` (core 0, max `Integer.MAX_VALUE`, 60 s keep-alive), which can create thousands of threads under load.
`` `PriorityBlockingQueue` ``	Unbounded, ordered tasks (tasks must be `Comparable`). Not FIFO. Beware submit-wrapped tasks, which are `FutureTask`s, not your comparable class.
`` `DelayQueue` ``	Elements become available after a delay; the basis of scheduled pools.

Work queues and what they do to a pool.

8.7.4 Rejection policies

When the queue is full and the pool is at its maximum, the `RejectedExecutionHandler` decides:

- `AbortPolicy` (default): throw `RejectedExecutionException`. The caller learns about overload immediately. Often the right choice, because you can convert it into an HTTP 503.
- `CallerRunsPolicy`: the submitting thread runs the task itself. This is built-in **back-pressure**: while the caller is busy running a task it cannot submit more, so the producer slows down to the speed of the consumers.
- `DiscardPolicy`: silently drop the new task. Dangerous.
- `DiscardOldestPolicy`: drop the oldest queued task and retry. Also loses work silently.
- A custom handler: log, count a metric, or block with `queue.put`.

CallerRunsDemo.java JAVA

```java
import java.util.concurrent.CountDownLatch;
import java.util.concurrent.SynchronousQueue;
import java.util.concurrent.ThreadPoolExecutor;
import java.util.concurrent.TimeUnit;

public class CallerRunsDemo {
    public static void main(String[] args) throws Exception {
        // One worker, hand-off queue, CallerRunsPolicy: when the worker is busy, the
        // submitting thread runs the task itself, which slows the producer down (back-pressure).
        ThreadPoolExecutor pool = new ThreadPoolExecutor(1, 1, 0, TimeUnit.SECONDS,
                new SynchronousQueue<>(), new ThreadPoolExecutor.CallerRunsPolicy());
        CountDownLatch gate = new CountDownLatch(1);
        pool.execute(() -> {
            try { gate.await(); } catch (InterruptedException e) { /* ignore */ }
        });
        pool.execute(() ->
                System.out.println("task B ran on: " + Thread.currentThread().getName()));
        gate.countDown();
        pool.shutdown();
        pool.awaitTermination(2, TimeUnit.SECONDS);
    }
}
```

Output TERMINAL

```
$ java CallerRunsDemo.java
task B ran on: main
```

Task B ran on `main`, the submitting thread, because the only worker was busy and the hand-off queue could not hold it.

WATCH OUT
CallerRunsPolicy can run tasks on a thread that must not block. If the submitter is an event-loop or request-accepting thread, running a slow task on it stalls everything that thread does. And if the submitter is itself a worker of the same pool, you get unexpected recursion. Know who calls `submit` before choosing it.

8.7.5 The factory methods and why teams ban them

`Executors` offers shortcuts:

Factory	Configuration	Hidden risk
`newFixedThreadPool(n)`	core = max = n, unbounded `LinkedBlockingQueue`	Unbounded queue: memory exhaustion, unbounded latency under overload.
`newCachedThreadPool()`	core 0, max unlimited, `SynchronousQueue`	Unbounded thread creation: `unable to create native thread`.
`newSingleThreadExecutor()`	one thread, unbounded queue	Same unbounded queue; tasks run strictly in order.
`newScheduledThreadPool(n)`	n core threads, delay queue	Periodic task that throws is cancelled silently.
`newVirtualThreadPerTaskExecutor()`	a new virtual thread per task	No limit on concurrency; bound external resources yourself.
`newWorkStealingPool()`	`ForkJoinPool` with work stealing	Daemon threads; for many small independent tasks.

Executors factory methods.

The well-known Alibaba Java guidelines and several style guides forbid `newFixedThreadPool` and `newCachedThreadPool` in production code for exactly the two hidden risks above. The remedy is to construct `ThreadPoolExecutor` directly with a **bounded queue**, an explicit **rejection policy**, and a **named thread factory**. For learning code and short-lived tools the factories are perfectly fine.

8.7.6 Sizing a pool

There is no magic number, but there is a method.

- **CPU-bound work** (parsing, hashing, computation) never waits: more threads than cores only add context switching. Start with `Runtime.getRuntime().availableProcessors()` (or cores + 1).
- **I/O-bound work** (database, HTTP calls) spends most of its time waiting, so more threads keep the CPUs busy. A classical estimate is `threads = cores × (1 + wait time / compute time)`. If a task computes 10 ms and waits 90 ms, that is about 10 times the core count.
- **Little's law** is a good cross-check: concurrent requests in the system = arrival rate × average time in the system. If you serve 200 requests per second and each takes 50 ms, about 10 are in flight at any moment, so a pool of 10 to 20 is plenty.
- Mix workloads in one pool and a slow dependency can starve fast work. Use **separate pools** (bulkheads) for different downstream systems.
- Verify with measurements under realistic load, and watch `getQueue().size()`, `getActiveCount()` and rejection counts.

Also remember that a pool size is not your real limit. If the database allows 20 connections, a 200-thread pool only moves the waiting from the queue to the connection pool.

8.7.7 Shutdown

A pool must be shut down, or its non-daemon workers keep the JVM running forever.

- `shutdown()`: stop accepting tasks; already submitted ones (running and queued) finish. Returns immediately.
- `shutdownNow()`: stop accepting, try to cancel running tasks by interrupting their threads, and return the list of tasks that never started.
- `awaitTermination(time, unit)`: block until the pool has terminated or the time passes.
- `close()` (Java 19+): calls `shutdown()` and waits for termination (it interrupts the thread via `shutdownNow` if the waiting thread is interrupted). This lets you use `ExecutorService` in try-with-resources.

ShutdownDemo.java JAVA

```java
import java.util.List;
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;
import java.util.concurrent.TimeUnit;

public class ShutdownDemo {
    public static void main(String[] args) throws Exception {
        ExecutorService pool = Executors.newFixedThreadPool(1);
        pool.execute(() -> {
            try {
                Thread.sleep(300);
                System.out.println("task 1 finished normally");
            } catch (InterruptedException e) {
                System.out.println("task 1 interrupted");
            }
        });
        pool.execute(() -> System.out.println("task 2 (was queued)"));

        pool.shutdown();                                  // no new tasks; queued ones still run
        System.out.println("isShutdown = " + pool.isShutdown()
                + ", isTerminated = " + pool.isTerminated());
        try {
            pool.execute(() -> System.out.println("never"));
        } catch (java.util.concurrent.RejectedExecutionException e) {
            System.out.println("submit after shutdown -> RejectedExecutionException");
        }
        boolean done = pool.awaitTermination(1, TimeUnit.SECONDS);
        System.out.println("terminated in time = " + done);

        // The robust shutdown recipe (also what ExecutorService.close() does in Java 19+).
        ExecutorService pool2 = Executors.newFixedThreadPool(1);
        pool2.execute(() -> {
            try { Thread.sleep(10_000); } catch (InterruptedException e) {
                System.out.println("long task interrupted by shutdownNow");
            }
        });
        Thread.sleep(100);
        pool2.shutdown();
        if (!pool2.awaitTermination(200, TimeUnit.MILLISECONDS)) {
            List<Runnable> neverStarted = pool2.shutdownNow();
            System.out.println("never started: " + neverStarted.size());
            pool2.awaitTermination(1, TimeUnit.SECONDS);
        }

        try (ExecutorService pool3 = Executors.newFixedThreadPool(2)) {   // Java 19+
            pool3.submit(() -> System.out.println("try-with-resources waits for this task"));
        }
        System.out.println("after close()");
    }
}
```

Output TERMINAL

```
$ java ShutdownDemo.java
isShutdown = true, isTerminated = false
submit after shutdown -> RejectedExecutionException
task 1 finished normally
task 2 (was queued)
terminated in time = true
long task interrupted by shutdownNow
never started: 0
try-with-resources waits for this task
after close()
```

The recipe in the middle (shutdown, wait a grace period, then `shutdownNow`, then wait again) is the standard graceful-then-forceful shutdown. Running tasks only stop if they react to interruption, so write long tasks to check `Thread.interrupted()` or call blocking methods that throw `InterruptedException`.

8.7.8 Scheduled executors

`ScheduledThreadPoolExecutor` runs tasks after a delay or periodically. Three methods matter:

- `schedule(task, delay, unit)`: run once.
- `scheduleAtFixedRate(task, initialDelay, period, unit)`: runs at `start, start+period, start+2*period...`. If a run takes longer than the period, the next run starts late, but runs never overlap.
- `scheduleWithFixedDelay(task, initialDelay, delay, unit)`: waits `delay` after each run **finishes**. Use it when you want a pause between runs rather than a steady rate.

SchedDemo.java JAVA

```java
import java.util.concurrent.CountDownLatch;
import java.util.concurrent.Executors;
import java.util.concurrent.ScheduledExecutorService;
import java.util.concurrent.ScheduledFuture;
import java.util.concurrent.TimeUnit;
import java.util.concurrent.atomic.AtomicInteger;

public class SchedDemo {
    public static void main(String[] args) throws Exception {
        ScheduledExecutorService ses = Executors.newScheduledThreadPool(1);
        long start = System.nanoTime();

        ScheduledFuture<String> once =
                ses.schedule(() -> "one-shot ran", 200, TimeUnit.MILLISECONDS);
        System.out.println(once.get());

        AtomicInteger runs = new AtomicInteger();
        CountDownLatch three = new CountDownLatch(3);
        ScheduledFuture<?> periodic = ses.scheduleAtFixedRate(() -> {
            int n = runs.incrementAndGet();
            System.out.println("tick " + n);
            three.countDown();
            if (n == 2) throw new RuntimeException("fails on tick 2");   // suppresses later runs!
        }, 0, 100, TimeUnit.MILLISECONDS);
        three.await(1, TimeUnit.SECONDS);                 // times out: tick 3 never happens
        System.out.println("ticks = " + runs.get() + ", periodic.isDone() = " + periodic.isDone());
        ses.shutdownNow();
    }
}
```

Output TERMINAL

```
$ java SchedDemo.java
one-shot ran
tick 1
tick 2
ticks = 2, periodic.isDone() = true
```

The periodic task threw on its second run and **was never run again**, silently: `isDone()` became `true` and no error was printed. For periodic tasks, wrap the body in `try/catch (Throwable)` and log, unless you really want a failure to stop the schedule. For anything calendar-like (cron expressions, "every day at 02:00") use a scheduler library or Spring's `@Scheduled`, not a raw `Timer` (single thread, one failure kills it).

8.7.9 Common mistakes

1. Swallowed exceptions

`submit()` captures a task's exception in its `Future`; if nobody calls `get()`, you never see it. `execute()` lets the exception propagate to the thread's uncaught-exception handler, and the pool replaces the dead worker. This program shows both, using a custom `ThreadFactory` to name the threads and to install a handler:

ExecMistakes.java JAVA

```java
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;
import java.util.concurrent.Future;
import java.util.concurrent.ThreadFactory;
import java.util.concurrent.TimeUnit;
import java.util.concurrent.atomic.AtomicInteger;
```

ExecMistakes.java (continued) JAVA

```java
public class ExecMistakes {
    static ThreadFactory named(String prefix) {
        AtomicInteger n = new AtomicInteger(1);
        return r -> {
            Thread t = new Thread(r, prefix + "-" + n.getAndIncrement());
            t.setUncaughtExceptionHandler((th, ex) ->
                    System.out.println("  handler on " + th.getName() + ": " + ex));
            return t;
        };
    }

    public static void main(String[] args) throws Exception {
        ExecutorService pool = Executors.newFixedThreadPool(1, named("billing"));

        System.out.println("1. submit() with a failing task:");
        Future<?> f = pool.submit(() -> { throw new IllegalStateException("boom"); });
        Thread.sleep(200);
        System.out.println("  nothing was printed; the exception waits inside the Future");
        try {
            f.get();
        } catch (Exception e) {
            System.out.println("  get() throws: " + e);
        }

        System.out.println("2. execute() with a failing task:");
        pool.execute(() -> { throw new IllegalStateException("boom 2"); });
        Thread.sleep(200);

        pool.execute(() -> System.out.println(
                "3. pool keeps working on: " + Thread.currentThread().getName()));
        pool.shutdown();
        pool.awaitTermination(2, TimeUnit.SECONDS);
    }
}
```

Output TERMINAL

```
$ java ExecMistakes.java
1. submit() with a failing task:
  nothing was printed; the exception waits inside the Future
  get() throws: java.util.concurrent.ExecutionException: java.lang.IllegalStateException: boom
2. execute() with a failing task:
  handler on billing-1: java.lang.IllegalStateException: boom 2
3. pool keeps working on: billing-2
```

Note that the replacement worker is `billing-2`: the thread that died with the exception was `billing-1`. Always give pool threads meaningful names; they are what you will read in thread dumps and logs.

2. Blocking on a Future from inside the same pool

A task that submits a subtask to its own pool and calls `get()` holds a worker while waiting for a worker. With a pool of N threads and N such parents, nothing can run: **thread-pool deadlock** (also called thread starvation deadlock, shown in the liveness chapter). Use separate pools, a larger pool, or `CompletableFuture` chaining (next chapter).

3. ThreadLocal and context leaking through pool threads

Workers are reused, so `ThreadLocal` values set by one task are still there for the next. Always `remove()` in `finally`, as covered in the confinement chapter.

4. Forgetting to shut down, or shutting down the shared pool

Create the pool once, share it, and shut it down in one place (for example a `@PreDestroy` method or a JVM shutdown hook). Creating a new pool per request is the same mistake as creating a thread per request, only slower.

5. Unbounded everything

`newFixedThreadPool` plus an unbounded queue plus a slow downstream service is the classic outage: tasks pile up, memory climbs, latency becomes minutes, and the first symptom you see is `OutOfMemoryError`. Bound the queue and decide what the system does when full.

INTERVIEW ANGLE
Explain what happens when you submit a task to a ThreadPoolExecutor. If fewer than `corePoolSize` threads exist, a new thread is started for the task. Otherwise the task is offered to the queue. If the queue is full and fewer than `maximumPoolSize` threads exist, a new thread is started. If both are full, the `RejectedExecutionHandler` runs (default: throw `RejectedExecutionException`). Corollary: with an unbounded queue the pool never grows past the core size, so `maximumPoolSize` is ignored.

INTERVIEW ANGLE
Why is `Executors.newFixedThreadPool` discouraged in production? It uses an unbounded `LinkedBlockingQueue`, so under overload tasks accumulate until memory runs out, and there is no back-pressure or rejection. Build a `ThreadPoolExecutor` with a bounded queue, a rejection policy and a named `ThreadFactory`.

REAL WORLD
Spring Boot hides this. `@Async` uses a `TaskExecutor` (Boot auto-configures a `ThreadPoolTaskExecutor` named `applicationTaskExecutor`) whose core size, max size and queue capacity are properties (`spring.task.execution.pool.*`). The same three-step growth rule applies: with Boot's default unbounded queue capacity, the pool will not grow past its core size. Tomcat's request pool (`server.tomcat.threads.max`, default 200) is also a thread pool, and it is why a slow database makes the whole service unresponsive.

Check yourself

1. A pool is created with core 4, max 16 and `new LinkedBlockingQueue<>()`. A flood of tasks arrives. How many threads will the pool ever have, and why?
2. What is wrong here? `pool.submit(() -> { riskyWork(); });` and nobody keeps the returned Future.
3. What does `shutdown()` do to tasks already waiting in the queue? What about `shutdownNow()`?
4. You run `ses.scheduleAtFixedRate(task, 0, 1, SECONDS)` and `task` throws on its 5th run. What happens to runs 6, 7, ...?
5. Which rejection policy gives back-pressure, and how does it work?
6. A task calls `pool.submit(sub).get()` and the pool has 2 threads. Two such tasks run at once. What happens?

Answers

1. Four. The unbounded queue never reports full, so step 3 (grow toward max) never happens; `maximumPoolSize` is never reached.
2. The exception is captured in the Future and never seen. Either call `get()` / handle failures inside the task, or use `execute()`, or catch and log in the task.
3. `shutdown()` lets queued tasks run to completion but accepts no new ones. `shutdownNow()` does not run queued tasks, returns them as a list, and interrupts the running ones.
4. They never happen. A throwing periodic task cancels its own future silently. Catch exceptions inside the task.
5. `CallerRunsPolicy`: the submitting thread runs the task itself, so it cannot submit more until it finishes, slowing the producer to the pace of the workers.
6. Both workers are blocked in `get()` waiting for subtasks that sit in the queue with no free thread: a thread-pool deadlock. Use another pool or non-blocking composition.

KEY POINTS
Executors turn threads into a managed, bounded resource.

- Prefer executors to raw `Thread`s. Describe work as `Runnable`/`Callable`, wait with `Future`, and always shut the pool down.
- `ThreadPoolExecutor` order: core threads, then queue, then up to max threads, then reject. An unbounded queue means the pool never exceeds the core size.
- In production use a bounded queue, an explicit rejection policy (`AbortPolicy` or `CallerRunsPolicy`) and a named `ThreadFactory`.
- Size by workload: CPU-bound near the core count, I/O-bound larger; separate pools for separate dependencies.
- Exceptions hide in `submit()` Futures and kill periodic schedules silently; handle them deliberately.

8.8 CompletableFuture

A plain `Future` has a blocking design: to use the result you must call `get()` and park a thread until it is ready. Chain three remote calls with `Future` and you hold three threads hostage in turn, or you write nested callbacks by hand. `CompletableFuture` (Java 8) fixes this. It is a `Future` whose result you can also *continue from*: "when this finishes, do that", "when both of these finish, combine them", "if this fails, use a fallback". You build a pipeline of stages, much as the Streams chapters built pipelines of operations, and no thread waits while the stages are pending.

It also implements `CompletionStage`, the interface that defines all the chaining methods, and it is a **promise**: you can create an empty one with `new CompletableFuture<>()` and complete it later from anywhere with `complete(value)` or `completeExceptionally(error)`. That is how adapters turn callback-style APIs into futures.

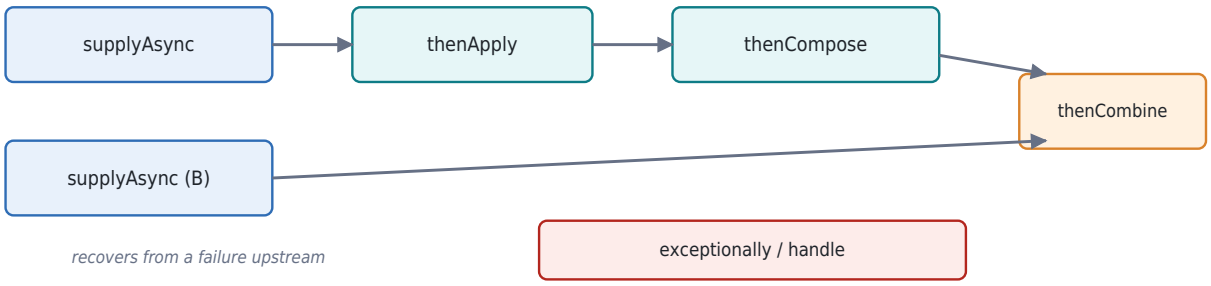


Figure 8.5. A dependent pipeline: supplyAsync starts a stage, thenApply and thenCompose extend it, thenCombine joins two independent branches, and exceptionally or handle recover from a failure anywhere upstream.

8.8.1 Starting a computation

- `CompletableFuture.supplyAsync(Supplier<T>)` runs a task that returns a value, in the common pool.
- `CompletableFuture.supplyAsync(Supplier<T>, Executor)` runs it in **your** executor. This is what you should normally use (see the pitfalls below).
- `runAsync(Runnable)` is the no-result variant.
- `completedFuture(value)` and `failedFuture(ex)` (Java 9) create already-finished futures; useful for tests and for short-circuiting.

8.8.2 Transforming and chaining

The method names follow a pattern: what the stage does to the previous result, and whether it runs on the thread that completed the previous stage or is handed to an executor (`...Async`).

Method	Function type	Use it to
`thenApply`	`T → U`	Transform the value (like `map`).
`thenAccept`	`T → void`	Consume the value (end of the chain).
`thenRun`	`Runnable`	Run an action ignoring the value.
`thenCompose`	`T → CompletionStage<U>`	Chain a dependent async call (like `flatMap`).
`thenCombine`	`other future, (T, U) → V`	Join two independent futures.
`allOf(f1, f2, ...)`	`static, returns Void future`	Wait for all to finish.
`anyOf(f1, f2, ...)`	`static, returns Object future`	First to finish wins.
`exceptionally`	`Throwable → T`	Recover with a fallback value.
`handle`	`(T, Throwable) → U`	Recover or transform: sees both outcomes.
`whenComplete`	`(T, Throwable) → void`	Observe (log) without changing the outcome.

The chaining methods.

The most common beginner confusion is `thenApply` versus `thenCompose`. If your function returns a plain value, use `thenApply`. If it returns another `CompletableFuture` (it makes an async call), use `thenCompose`; `thenApply` would give you a `CompletableFuture<CompletableFuture<U>>`, a nested future you then have to unwrap.

CfBasics.java JAVA

```java
import java.util.List;
import java.util.concurrent.CompletableFuture;
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;

public class CfBasics {
    static void sleep(long ms) {
        try {
            Thread.sleep(ms);
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
        }
    }

    static CompletableFuture<Integer> price(ExecutorService ex, String sku) {
        return CompletableFuture.supplyAsync(() -> { sleep(100); return sku.length() * 100; }, ex);
    }

    static CompletableFuture<String> stock(ExecutorService ex, String sku) {
        return CompletableFuture.supplyAsync(() -> { sleep(100); return "in stock"; }, ex);
    }

    static CompletableFuture<Double> rating(ExecutorService ex, String sku) {
        return CompletableFuture.supplyAsync(() -> { sleep(100); return 4.5; }, ex);
    }

    public static void main(String[] args) {
        ExecutorService ex = Executors.newFixedThreadPool(4);
        try {
            long t0 = System.nanoTime();

            // thenApply: transform the result (like Stream.map / Optional.map)
            CompletableFuture<String> label = price(ex, "book")
                    .thenApply(cents -> cents / 100.0)
                    .thenApply(d -> "price: " + d);
            System.out.println(label.join());

            // thenCompose: the next step itself returns a CompletableFuture (flatMap)
            CompletableFuture<String> chained = price(ex, "pen")
                    .thenCompose(p -> stock(ex, "pen").thenApply(s -> p + " / " + s));
            System.out.println("composed: " + chained.join());

            // thenCombine: wait for two INDEPENDENT futures, then merge
            CompletableFuture<String> both = price(ex, "lamp")
                    .thenCombine(rating(ex, "lamp"), (p, r) -> "lamp " + p + " rated " + r);
            System.out.println(both.join());

            // allOf: wait for many; the result is Void, so read each future afterwards
            long t1 = System.nanoTime();
            var p = price(ex, "desk");
            var s = stock(ex, "desk");
            var r = rating(ex, "desk");
            CompletableFuture.allOf(p, s, r).join();
            long millis = (System.nanoTime() - t1) / 1_000_000;
            System.out.println("allOf: " + p.join() + ", " + s.join() + ", " + r.join());
            System.out.println("three 100 ms calls took less than 250 ms: " + (millis < 250));

            // anyOf: the first one to finish wins (result type is Object)
            CompletableFuture<Object> first = CompletableFuture.anyOf(
                    CompletableFuture.supplyAsync(() -> { sleep(300); return "slow"; }, ex),
                    CompletableFuture.supplyAsync(() -> { sleep(30); return "fast"; }, ex));
            System.out.println("anyOf -> " + first.join());

            // Collect a list of futures into a future of a list
            List<CompletableFuture<Integer>> prices = List.of("a", "bb", "ccc").stream()
                    .map(k -> price(ex, k)).toList();
            CompletableFuture<List<Integer>> all = CompletableFuture
                    .allOf(prices.toArray(new CompletableFuture[0]))
                    .thenApply(v -> prices.stream().map(CompletableFuture::join).toList());
            System.out.println("all prices: " + all.join());
        } finally {
            ex.shutdown();
        }
    }
}
```

Output TERMINAL

```
$ java CfBasics.java
price: 4.0
composed: 300 / in stock
lamp 400 rated 4.5
allOf: 400, in stock, 4.5
three 100 ms calls took less than 250 ms: true
anyOf -> fast
all prices: [100, 200, 300]
```

What to notice

- `price(...).thenApply(...)` returns immediately; the transformations run when the price arrives. The `join()` at the end is where the demo finally blocks, and in an application you would normally return the future to the caller instead.
- `thenCombine` runs the two calls **concurrently** and merges when both are done. Writing `price(...).thenCompose(p -> rating(...))` instead would run them one after the other. Use `thenCompose` only when the second call needs the first result.
- `allOf` returns `CompletableFuture<Void>`, so you read each individual future afterwards with `join()` (cheap, they are already complete). The last block shows the idiom to turn a `List<CompletableFuture<T>>` into a `CompletableFuture<List<T>>`.
- Three 100 ms calls finished in well under 250 ms, because they overlapped. Sequentially it would be 300 ms.
- `anyOf` returns `Object`, so you cast. It also completes with the first *completion*, even if that first completion is a failure.

8.8.3 Which thread runs each stage?

This matters for performance and for correctness. A stage created with `thenApply` runs on whichever thread completes the previous stage, or, **if that stage is already complete when you attach the new one, on the calling thread**. The `...Async` variants always submit the step to an executor (the common pool unless you pass one).

CfThreads.java JAVA

```java
import java.util.concurrent.CompletableFuture;
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;
import java.util.concurrent.ForkJoinPool;

public class CfThreads {
    static String t() { return Thread.currentThread().getName(); }

    public static void main(String[] args) throws Exception {
        System.out.println("cpus = " + Runtime.getRuntime().availableProcessors()
                + ", common pool parallelism = " + ForkJoinPool.commonPool().getParallelism());

        System.out.println("supplyAsync (default executor) ran on: "
                + CompletableFuture.supplyAsync(CfThreads::t).join());

        ExecutorService ex = Executors.newFixedThreadPool(2, r -> new Thread(r, "my-pool"));
        try {
            CompletableFuture<String> f = CompletableFuture.supplyAsync(CfThreads::t, ex);
            f.join();                                          // f is now complete
            // thenApply on an already-completed future runs on the CALLING thread:
            System.out.println("thenApply on completed future ran on: "
                    + f.thenApply(x -> t()).join());
            // thenApplyAsync always hands the step to an executor:
            System.out.println("thenApplyAsync(.., ex) ran on: "
                    + f.thenApplyAsync(x -> t(), ex).join());
        } finally {
            ex.shutdown();
        }
    }
}
```

Output TERMINAL

```
$ java CfThreads.java
cpus = 2, common pool parallelism = 1
supplyAsync (default executor) ran on: Thread-0
thenApply on completed future ran on: main
thenApplyAsync(.., ex) ran on: my-pool
```

Typical output on a 2-CPU build machine (names and parallelism depend on your machine). Two things to read here. First, `thenApply` on a finished future ran on `main`: harmless for a cheap transformation, but a surprise if the step is slow and you thought it ran in the background. Second, with `supplyAsync` and no executor the thread is called `Thread-0`, not `ForkJoinPool.commonPool-worker-1`. The reason: `CompletableFuture` uses the common `ForkJoinPool` only when its parallelism is greater than 1; on machines (or containers) with one or two CPUs, the common pool has parallelism 1 (here, `cpus - 1`), and `CompletableFuture` falls back to creating a **new thread per task**. So the same code behaves differently in a small container than on your laptop.

8.8.4 Errors

An exception thrown inside any stage completes that stage exceptionally, and the failure **propagates through every dependent stage** (they are skipped) until a stage handles it. Nothing is printed, nothing is thrown at the point of failure. If nobody calls `join()`/`get()` and nobody attaches a handler, the failure is silently lost, the same trap as a lost `Future` from `submit`.

Wrapping rules: `get()` throws the checked `ExecutionException`; `join()` throws the unchecked `CompletionException`; both carry the original exception as `getCause()`. Inside `handle`, `exceptionally` and `whenComplete`, the `Throwable` you receive is often already wrapped in a `CompletionException` (it is for failures coming from a previous stage), so unwrap it if you test its type.

CfErrors.java JAVA

```java
import java.util.concurrent.CompletableFuture;
import java.util.concurrent.CompletionException;
import java.util.concurrent.ExecutionException;
import java.util.concurrent.TimeUnit;

public class CfErrors {
    static CompletableFuture<Integer> failing() {
        return CompletableFuture.supplyAsync(() -> {
            if (true) throw new IllegalStateException("pricing service down");
            return 1;
        });
    }

    public static void main(String[] args) throws Exception {
        // 1. Without handling, the failure travels down the chain and surfaces at join()/get().
        CompletableFuture<Integer> doomed = failing().thenApply(x -> x + 1);
        try {
            doomed.join();
        } catch (CompletionException e) {
            System.out.println("join() threw CompletionException, cause: " + e.getCause());
        }
        try {
            doomed.get();
        } catch (ExecutionException e) {
            System.out.println("get()  threw ExecutionException,  cause: " + e.getCause());
        }

        // 2. exceptionally: recover with a fallback value
        int recovered = failing().exceptionally(ex -> -1).join();
        System.out.println("exceptionally -> " + recovered);

        // 3. handle: sees BOTH result and exception, returns a new value
        String msg = failing()
                .handle((v, ex) -> ex == null ? "ok " + v
                        : "failed: " + ex.getClass().getSimpleName()
                        + " / " + ex.getCause().getMessage())
                .join();
        System.out.println("handle -> " + msg);

        // 4. whenComplete: observe (log) but do not change the outcome
        try {
            failing().whenComplete((v, ex) ->
```

CfErrors.java (continued) JAVA

```
                    System.out.println("whenComplete saw: " + ex.getClass().getSimpleName()))
                    .join();
        } catch (CompletionException e) {
            System.out.println("still failed afterwards: " + e.getCause().getMessage());
        }

        // 5. Timeouts (Java 9+)
        CompletableFuture<String> slow = CompletableFuture.supplyAsync(() -> {
            try { Thread.sleep(1000); } catch (InterruptedException e) { /* ignore */ }
            return "slow result";
        });
        String r = slow.completeOnTimeout("default", 100, TimeUnit.MILLISECONDS).join();
        System.out.println("completeOnTimeout -> " + r);

        CompletableFuture<String> slow2 = CompletableFuture.supplyAsync(() -> {
            try { Thread.sleep(1000); } catch (InterruptedException e) { /* ignore */ }
            return "slow result";
        }).orTimeout(100, TimeUnit.MILLISECONDS);
        try {
            slow2.join();
        } catch (CompletionException e) {
            System.out.println("orTimeout -> " + e.getCause().getClass().getSimpleName());
        }

        // 6. You can complete a future by hand: it is also a promise.
        CompletableFuture<String> promise = new CompletableFuture<>();
        promise.completeExceptionally(new RuntimeException("cancelled by user"));
        System.out.println("isCompletedExceptionally = " + promise.isCompletedExceptionally());
    }
}
```

Output TERMINAL

```
$ java CfErrors.java
join() threw CompletionException, cause: java.lang.IllegalStateException: pricing service down
get()  threw ExecutionException,  cause: java.lang.IllegalStateException: pricing service down
exceptionally -> -1
handle -> failed: CompletionException / pricing service down
whenComplete saw: CompletionException
still failed afterwards: pricing service down
completeOnTimeout -> default
orTimeout -> TimeoutException
isCompletedExceptionally = true
```

What to notice

- `handle` and `whenComplete` both received a `CompletionException`, not the `IllegalStateException`: that is the wrapping. `ex.getCause()` is the real error. A common helper: `Throwable root = ex instanceof CompletionException ? ex.getCause() : ex;`.
- `exceptionally` supplies a fallback value but cannot see the successful result. `handle` sees both. `whenComplete` is for side effects such as logging; the outcome continues unchanged, which is why the final `join()` still threw.
- `orTimeout` makes the future fail with `TimeoutException` after the delay; `completeOnTimeout` completes it normally with a default. Both are Java 9 additions. They do **not** stop the underlying work: the slow supplier keeps running. They only decide what the future reports. Likewise `cancel(true)` on a `CompletableFuture` does not interrupt the running task (the Javadoc says the flag has no effect), it only completes the future with `CancellationException`.
- Since Java 12 there are also `exceptionallyCompose` (recover with another async call) and `...Async` forms of `exceptionally`.

8.8.5 Executors, the common pool and blocking

`supplyAsync(task)` without an executor uses the **common pool**, a single JVM-wide `ForkJoinPool` that is also used by parallel streams. It is sized for CPU-bound work (`cores - 1` threads). Three consequences:

1. **Never run blocking I/O in the common pool.** A few tasks waiting for a database or HTTP response occupy all the threads, and every parallel stream and every other `CompletableFuture` in the application stalls.
2. **Pass your own executor** for I/O (a bounded `ThreadPoolExecutor` per downstream dependency, or `Executors.newVirtualThreadPerTaskExecutor()` in Java 21+). It also makes thread names meaningful in dumps.

3. **Do not block a worker waiting for another stage that needs a worker.** Calling `join()` inside a task that runs on a small pool is the thread-pool deadlock from the executors chapter, in a new disguise:

CfStarve.java JAVA

```java
import java.util.concurrent.CompletableFuture;
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;
import java.util.concurrent.TimeUnit;

public class CfStarve {
    public static void main(String[] args) throws Exception {
        // Daemon worker: join() ignores interrupts, so a stuck worker would keep the JVM alive.
        ExecutorService single = Executors.newFixedThreadPool(1, r -> {
            Thread t = new Thread(r, "single");
            t.setDaemon(true);
            return t;
        });
        try {
            // BAD: the outer task occupies the only worker, then waits for an inner task
            // that needs that same worker.
            CompletableFuture<Integer> outer = CompletableFuture.supplyAsync(() -> {
                CompletableFuture<Integer> inner = CompletableFuture.supplyAsync(() -> 42, single);
                return inner.join() + 1;
            }, single);
            try {
                outer.get(500, TimeUnit.MILLISECONDS);
            } catch (java.util.concurrent.TimeoutException e) {
                System.out.println("blocking style: stuck (timed out after 500 ms)");
            }

            // GOOD: compose instead of blocking. No thread waits, so one worker is enough.
            ExecutorService single2 = Executors.newFixedThreadPool(1);
            int ok = CompletableFuture.supplyAsync(() -> 1, single2)
                    .thenCompose(x -> CompletableFuture.supplyAsync(() -> x + 41, single2))
                    .thenApply(x -> x + 1)
                    .get(2, TimeUnit.SECONDS);
            System.out.println("composed style: " + ok);
            single2.shutdown();
        } finally {
            single.shutdownNow();
        }
    }
}
```

Output TERMINAL

```
$ java CfStarve.java
blocking style: stuck (timed out after 500 ms)
composed style: 43
```

The blocking version never finishes: the single worker is held by the outer task, which waits for an inner task queued for the same worker. The composed version needs no waiting thread, because `thenCompose` only *registers* the next step. (The worker in the demo is a daemon thread because a stuck `join()` ignores interrupts and would otherwise prevent the JVM from exiting.) This is the practical reason to prefer `thenCompose` to a nested `join()`.

Useful extras

- `delayedExecutor(delay, unit)` (Java 9) is an `Executor` that starts the task after a delay: `CompletableFuture.supplyAsync(task, CompletableFuture.delayedExecutor(1, TimeUnit.SECONDS))` is a one-line retry back-off.
- `thenApplyAsync(fn, executor)`: hand a heavy transformation to a specific pool.
- `copy()` and `minimalCompletionStage()` (Java 9) give consumers a view they cannot complete or cancel.
- `join()` is the same as `get()` without checked exceptions; `getNow(default)` returns the default if not done; `isDone()`, `isCompletedExceptionally()`.
- `resultNow()` and `exceptionNow()` (Java 19, on `Future`) read a completed result without declared exceptions.

Design guidance

- Return `CompletableFuture<T>` from async service methods, and let the outermost caller decide when (or whether) to block.
- Always supply an executor for anything that is not pure CPU work.
- Attach an error handler at the end of every chain whose result nobody reads (fire-and-forget), or the failure vanishes.
- Put timeouts on remote calls (`orTimeout`), and decide what the fallback is.
- Keep stages small and side-effect free; for sequential-looking business logic with blocking calls, a virtual thread (later in this Part) often produces simpler code than a long chain of lambdas.

INTERVIEW ANGLE
What is the difference between thenApply and thenCompose? And thenCombine? `thenApply` maps the result with a function returning a plain value. `thenCompose` is for a function that itself returns a `CompletableFuture` (flat-map), keeping the chain flat. `thenCombine` joins two independent futures, running both concurrently and applying a function when both are done.

INTERVIEW ANGLE
Which thread runs `supplyAsync` and its `thenApply`? `supplyAsync` without an executor runs in the common `ForkJoinPool` (or a new thread per task if the common pool's parallelism is 1). `thenApply` runs on the thread that completed the previous stage, or on the caller if the stage is already complete; `thenApplyAsync` submits to an executor.

REAL WORLD
Spring's `@Async` methods can return `CompletableFuture<T>`. Web clients and drivers expose callbacks or reactive types, which teams adapt to `CompletableFuture` with a promise (`new CompletableFuture<>()` completed in the callback). Fan-out "gather data from three services" endpoints are the standard use of `thenCombine` and `allOf`.

Check yourself

1. What is the type of `CompletableFuture.supplyAsync(() -> 5).thenApply(x -> CompletableFuture.supplyAsync(() -> x))`, and how would you fix it?
2. What does this print? `CompletableFuture.supplyAsync(() -> 1 / 0).exceptionally(e -> -1).thenApply(x -> x * 10).join()`
3. Why is `supplyAsync(() -> jdbc.query(...))` without an executor a bad idea?
4. A chain of three `thenApply` calls ends without `join()` and without `exceptionally`. Stage two throws. What do you observe?
5. After `future.orTimeout(1, SECONDS)` fails with `TimeoutException`, is the supplier still running?
6. `anyOf(a, b)`: a fails immediately, b would succeed in one second. What is the result?

Answers

1. `CompletableFuture<CompletableFuture<Integer>>`. Use `thenCompose` instead of `thenApply`.
2. `-10`. The division fails, `exceptionally` supplies -1, then `thenApply` multiplies it.
3. A blocking call ties up a common-pool thread, starving parallel streams and other async work; with a small CPU count the pool is tiny. Pass a dedicated executor.
4. Nothing: no output and no exception. The failure sits in the final future and is lost unless something reads it or handles it.
5. Yes. `orTimeout` only completes the future exceptionally; it does not interrupt or stop the computation.
6. The result future completes exceptionally with a's exception, because `anyOf` takes the first completion, successful or not.

KEY POINTS

CompletableFuture composes asynchronous steps without blocking threads.

- `thenApply` maps, `thenCompose` flat-maps, `thenCombine` merges two independent futures, `allOf`/`anyOf` wait for many.
- Failures skip dependent stages until `exceptionally`, `handle` or `whenComplete`; `join` throws `CompletionException`, `get` throws `ExecutionException`.
- Pass your own executor for blocking work. The common pool is small, shared with parallel streams, and a thread per task if parallelism is 1.
- Do not `join()` inside a task running on a small pool; compose instead.
- `orTimeout`/`completeOnTimeout` change what the future reports, not whether the work keeps running.

8.9 Synchronizers and Concurrent Collections

`synchronized`, locks and atomics are the atoms of concurrency. Most real coordination problems, though, are molecules that come up again and again: "wait until all N workers are ready", "allow at most 10 threads in here", "hand work from producers to consumers", "count words from many threads". The `java.util.concurrent` package provides ready-made, heavily tested solutions. Use them before you invent your own with `wait/notify`: they are shorter, faster, and correct in the corner cases you would forget.

This chapter has two halves: **synchronizers**, which make threads wait for each other, and **concurrent collections**, which let threads share data structures safely.

8.9.1 CountDownLatch: wait for N events

A `CountDownLatch` starts with a count. `countDown()` decrements it; `await()` blocks until it reaches zero. It is **one-shot**: the count never resets. Two classic uses are a **start gate** (count 1: workers wait for `countDown()` so they all begin together, useful in tests) and a **completion signal** (count N: the coordinator waits until N workers have finished).

8.9.2 CyclicBarrier: wait for each other, repeatedly

A `CyclicBarrier(parties)` makes `parties` threads wait at `await()` until the last one arrives; then they are all released, an optional **barrier action** runs once, and the barrier resets for the next round. Use it for phased computations: all workers finish step 1 before any starts step 2. If any thread is interrupted or times out while waiting, the barrier breaks and the others get `BrokenBarrierException`.

LatchBarrier.java JAVA

```
import java.util.concurrent.BrokenBarrierException;
import java.util.concurrent.CountDownLatch;
import java.util.concurrent.CyclicBarrier;
import java.util.concurrent.atomic.AtomicInteger;

public class LatchBarrier {
    public static void main(String[] args) throws Exception {
        // CountDownLatch: one-shot. Workers wait for a start signal; main waits for all to finish.
        int workers = 3;
        CountDownLatch startSignal = new CountDownLatch(1);
        CountDownLatch done = new CountDownLatch(workers);
        AtomicInteger finished = new AtomicInteger();
        for (int i = 0; i < workers; i++) {
            new Thread(() -> {
                try {
                    startSignal.await();               // all workers park here
                    finished.incrementAndGet();        // pretend work
                } catch (InterruptedException e) {
                    Thread.currentThread().interrupt();
                } finally {
                    done.countDown();                  // count down even on failure
                }
            }).start();
        }
        System.out.println("finished before the signal = " + finished.get());
        startSignal.countDown();                       // release everybody at once
        done.await();
        System.out.println("finished after the latch   = " + finished.get());
        System.out.println("latch count now " + done.getCount() + " (cannot be reset)");
```

LatchBarrier.java (continued) JAVA

```
        // CyclicBarrier: reusable; every party waits until all arrive, then a barrier action runs.
        CyclicBarrier barrier =
                new CyclicBarrier(3, () -> System.out.println("  -- barrier tripped"));
        Thread[] ts = new Thread[3];
        for (int i = 0; i < 3; i++) {
            ts[i] = new Thread(() -> {
                try {
                    for (int round = 1; round <= 2; round++) {
                        barrier.await();              // phase 1 / phase 2 boundaries
                    }
                } catch (InterruptedException | BrokenBarrierException e) {
                    Thread.currentThread().interrupt();
                }
            });
            ts[i].start();
        }
        for (Thread t : ts) t.join();
        System.out.println("barrier reused for 2 rounds");
    }
}
```

Output TERMINAL

```
$ java LatchBarrier.java
finished before the signal = 0
finished after the latch   = 3
latch count now 0 (cannot be reset)
  -- barrier tripped
  -- barrier tripped
barrier reused for 2 rounds
```

What to notice

- `countDown()` is in a `finally`. If a worker threw before reaching it, `done.await()` would hang forever. This is the number one latch bug.
- The latch's count ends at 0 and stays there; to run the scenario again you need a new latch. The barrier tripped twice: once per round.
- Latch: threads wait for *events* (any thread may count down, waiters and counters can be different threads). Barrier: threads wait for *each other* (the same threads both arrive and wait).

8.9.3 Semaphore: limit concurrent access

A `Semaphore` holds a number of **permits**. `acquire()` takes one (blocking if none), `release()` returns one. It limits how many threads are inside a section at once: a pool of 10 database connections, at most 5 concurrent downloads, a rate limiter. `new Semaphore(n, true)` is fair. `tryAcquire()` and `tryAcquire(timeout)` do not block forever. A semaphore with one permit is a lock, except that it is not owned: any thread may release, which is both its power and its danger.

8.9.4 Phaser and Exchanger

`Phaser` is a more flexible barrier: parties can `register()` and `arriveAndDeregister()` while it runs, it supports many phases, and you can override `onAdvance` to run code at each phase boundary and to terminate. Use it when the number of participants changes (for example a recursive task that spawns subtasks, each registering as a party). For a fixed set of parties, `CyclicBarrier` is simpler.

`Exchanger<V>` is a rendezvous for exactly two threads: each calls `exchange(x)`, blocks until the other arrives, and gets the other's object. It is rare in applications; the textbook use is double-buffering (one thread fills a buffer while the other drains the previous one, then they swap).

SemaphorePhaser.java JAVA

```
import java.util.concurrent.Exchanger;
import java.util.concurrent.Phaser;
import java.util.concurrent.Semaphore;
import java.util.concurrent.atomic.AtomicInteger;

public class SemaphorePhaser {
    public static void main(String[] args) throws Exception {
        // Semaphore: at most 3 threads inside the section at once (e.g. a connection limit).
```

SemaphorePhaser.java (continued) JAVA

```
        Semaphore permits = new Semaphore(3);
        AtomicInteger inside = new AtomicInteger();
        AtomicInteger maxInside = new AtomicInteger();
        Thread[] ts = new Thread[10];
        for (int i = 0; i < ts.length; i++) {
            ts[i] = new Thread(() -> {
                try {
                    permits.acquire();
                    try {
                        int now = inside.incrementAndGet();
                        maxInside.accumulateAndGet(now, Math::max);
                        Thread.sleep(30);
                        inside.decrementAndGet();
                    } finally {
                        permits.release();
                    }
                } catch (InterruptedException e) {
                    Thread.currentThread().interrupt();
                }
            });
            ts[i].start();
        }
        for (Thread t : ts) t.join();
        System.out.println("10 threads, 3 permits -> max inside at once = " + maxInside.get());
        System.out.println("tryAcquire(5) on 3 free permits: " + permits.tryAcquire(5));

        // Phaser: like a barrier, but parties can register/deregister dynamically.
        Phaser phaser = new Phaser(1);                    // main registers itself
        for (int i = 1; i <= 2; i++) {
            final int id = i;
            phaser.register();
            new Thread(() -> {
                phaser.arriveAndAwaitAdvance();           // end of phase 0
                phaser.arriveAndDeregister();             // leave after phase 1
            }, "worker-" + id).start();
        }
        System.out.println("phase before = " + phaser.getPhase());
        phaser.arriveAndAwaitAdvance();                   // main joins the phase-0 rendezvous
        System.out.println("phase after  = " + phaser.getPhase());
        phaser.arriveAndDeregister();

        // Exchanger: two threads swap objects at a meeting point.
        Exchanger<String> ex = new Exchanger<>();
        Thread other = new Thread(() -> {
            try { System.out.println("other got: " + ex.exchange("from-other")); }
            catch (InterruptedException e) { Thread.currentThread().interrupt(); }
        });
        other.start();
        String got = ex.exchange("from-main");
        other.join();
        System.out.println("main got:  " + got);
    }
}
```

Output TERMINAL

```
$ java SemaphorePhaser.java
10 threads, 3 permits -> max inside at once = 3
tryAcquire(5) on 3 free permits: false
phase before = 0
phase after  = 1
other got: from-main
main got:  from-other
```

With 10 threads and 3 permits, never more than 3 ran inside at once (the maximum observed was 3; on a very loaded machine it could be lower, but never higher). `tryAcquire(5)` failed immediately because only 3 permits exist in total.

Class	Waiting for	Reusable?
`CountDownLatch`	N events (count reaches zero)	No, one-shot
`CyclicBarrier`	N threads to arrive together	Yes, resets each round
`Phaser`	Registered parties, with dynamic membership	Yes, many phases

Class	Waiting for	Reusable?
`Semaphore`	A free permit (limits concurrency)	Yes, permits come and go
`Exchanger`	One partner thread to swap objects	Yes

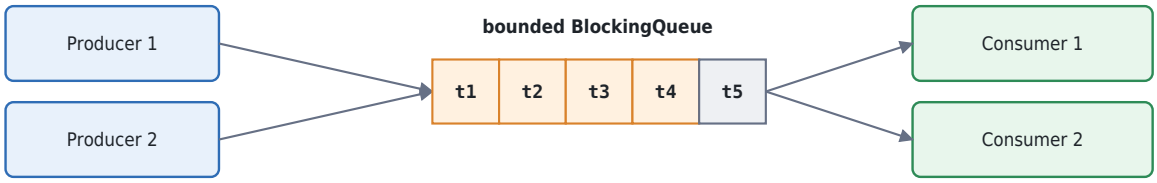
Choosing a synchronizer.

GOTCHA

Always release what you acquired, in `finally`. A semaphore permit that is not released after an exception leaks: after enough leaks every thread blocks at `acquire()` and the service is dead with no error message. Acquire before the `try`, release in `finally`, exactly as with locks. Do the same for `countDown()`.

8.9.5 BlockingQueue: producer-consumer without wait/notify

A `BlockingQueue` is a queue with two extra behaviours: `take()` **blocks** when the queue is empty, and `put(x)` **blocks** when a bounded queue is full. That is the whole producer-consumer pattern in the data structure itself. The producers and consumers need no locks or conditions, and a bounded queue gives you **back-pressure** for free: if consumers are slow, producers are slowed down instead of memory filling up.



full queue: put() blocks, so producers slow to the consumers' speed

Figure 8.6. Producers put into a bounded queue and consumers take from it; the queue absorbs bursts, and when it is full the producers block, which throttles them to the consumers' speed.

Each operation comes in four flavours depending on how it reacts when it cannot proceed:

Reaction	Insert	Remove	Examine	Timed
Throws exception	`add(e)`	`remove()`	`element()`	-
Returns special value	`offer(e)`	`poll()`	`peek()`	-
Blocks	`put(e)`	`take()`	-	-
Gives up after a time	`offer(e, t, unit)`	`poll(t, unit)`	-	-

BlockingQueue method families.

QueuesDemo.java JAVA

```java
import java.util.ArrayList;
import java.util.Collections;
import java.util.List;
import java.util.concurrent.ArrayBlockingQueue;
import java.util.concurrent.BlockingQueue;
import java.util.concurrent.PriorityBlockingQueue;
import java.util.concurrent.SynchronousQueue;
import java.util.concurrent.TimeUnit;

public class QueuesDemo {
    static final String POISON = "<<STOP>>";

    public static void main(String[] args) throws Exception {
        BlockingQueue<String> queue = new ArrayBlockingQueue<>(2);   // bounded: back-pressure
        List<String> consumed = Collections.synchronizedList(new ArrayList<>());

        Thread consumer = new Thread(() -> {
            try {
```

QueuesDemo.java (continued) JAVA

```
                while (true) {
                    String item = queue.take();                    // blocks while empty
                    if (item.equals(POISON)) break;                // poison pill ends the loop
                    consumed.add(item);
                }
            } catch (InterruptedException e) {
                Thread.currentThread().interrupt();
            }
        });
        consumer.start();
        for (int i = 1; i <= 5; i++) queue.put("order-" + i);      // blocks while full
        queue.put(POISON);
        consumer.join();
        System.out.println("consumed: " + consumed);

        // The four flavours of insertion on a full queue.
        BlockingQueue<Integer> small = new ArrayBlockingQueue<>(1);
        small.add(1);
        System.out.println("offer(2)           = " + small.offer(2));
        System.out.println("offer(2, 50ms)     = " + small.offer(2, 50, TimeUnit.MILLISECONDS));
        try {
            small.add(2);
        } catch (IllegalStateException e) {
            System.out.println("add(2) throws     : " + e.getMessage());
        }
        System.out.println("poll()             = " + small.poll());
        System.out.println("poll(50ms) on empty = " + small.poll(50, TimeUnit.MILLISECONDS));

        // PriorityBlockingQueue: ordered, unbounded.
        BlockingQueue<Integer> pq = new PriorityBlockingQueue<>(List.of(5, 1, 3));
        System.out.println("priority order: " + pq.take() + " " + pq.take() + " " + pq.take());

        // SynchronousQueue: no storage, a pure hand-off.
        SynchronousQueue<String> handoff = new SynchronousQueue<>();
        System.out.println("offer with no waiting consumer = " + handoff.offer("x"));
        Thread taker = new Thread(() -> {
            try { System.out.println("taker received " + handoff.take()); }
            catch (InterruptedException e) { Thread.currentThread().interrupt(); }
        });
        taker.start();
        handoff.put("parcel");                      // returns when the taker took it
        taker.join();
    }
}
```

Output TERMINAL

```
$ java QueuesDemo.java
consumed: [order-1, order-2, order-3, order-4, order-5]
offer(2)           = false
offer(2, 50ms)     = false
add(2) throws     : Queue full
poll()             = 1
poll(50ms) on empty = null
priority order: 1 3 5
offer with no waiting consumer = false
taker received parcel
```

What to notice

- The consumer ends with a **poison pill**, a special item that means "stop". It is the simplest shutdown protocol for one consumer; with N consumers, put N pills (or let each consumer re-insert the pill). The alternative is to interrupt the consumer threads.
- On a full queue `offer` returns `false`, `offer` with timeout waits then returns `false`, and `add` throws `IllegalStateException: Queue full`. Pick the reaction that matches your overload strategy; `put` is right when blocking the producer is the strategy.
- `PriorityBlockingQueue` returns elements in priority order, not FIFO, and is unbounded.
- `SynchronousQueue` has no capacity: `offer` fails without a waiting taker, and `put` waits for the `take`. It is the queue behind `newCachedThreadPool`.

Class	Notes	Bounded?
`ArrayBlockingQueue`	Array, one lock for both ends, optional fairness. Predictable memory.	Always
`LinkedBlockingQueue`	Linked nodes, two locks (head and tail) so producers and consumers do not contend. Higher throughput.	Optional (default `Integer.MAX_VALUE`)
`LinkedBlockingDeque`	Double-ended; also supports work-stealing patterns.	Optional
`PriorityBlockingQueue`	Binary heap; ordered by `Comparable` or `Comparator`.	No
`DelayQueue`	Elements implement `Delayed`; `take()` returns an element only after its delay expires. Timers, expiring caches.	No
`SynchronousQueue`	Direct hand-off; no storage.	Zero capacity
`LinkedTransferQueue`	Like a linked queue, plus `transfer(e)` that waits until a consumer receives the element.	No

The BlockingQueue implementations.

8.9.6 ConcurrentHashMap in depth

`ConcurrentHashMap` (CHM) is the map you use when many threads read and write one map. It is not `Collections.synchronizedMap(new HashMap<>())`, which puts a single lock around every method and serialises all threads. CHM allows reads without locking at all and lets writers to different parts of the table run in parallel.

How it works (Java 8 and later)

- The table is an array of bins, like `HashMap`, with the same hash spreading idea and the same tree-ification of long bins (a bin with 8 or more entries in a table of at least 64 becomes a red-black tree).
- **Reads (`get`) take no lock.** Table slots and node values are `volatile`, so a reader sees a consistent value for the entry.
- **Writes** to an empty bin use a CAS to insert the first node. Writes to a non-empty bin use `synchronized` on the **head node of that bin only**. Two writers to different bins never block each other. (Java 7 used 16 fixed lock "segments"; Java 8 replaced them with per-bin locking, which scales with the table size.)
- **Resizing** is cooperative: threads that find a resize in progress help move bins instead of waiting.
- **Size** is not stored in one field (that would be a contention hot spot). It is spread over counter cells, like `LongAdder`, and summed on demand, so `size()` is an estimate under concurrent updates; `mappingCount()` returns the same as a `long` and is preferred for huge maps.
- **Iterators are weakly consistent**: they never throw `ConcurrentModificationException`, and they reflect the map at some point at or after the iterator's creation, possibly including later updates.
- **No null keys or values.** With a plain `HashMap`, `get(k) == null` is ambiguous (absent, or mapped to null?) and you can disambiguate with `containsKey`. In a concurrent map that second call is a race: the answer may change between the two calls. Banning nulls removes the ambiguity.

Compound operations must be atomic operations

Each CHM method is atomic, but a *sequence* of them is not. `get` followed by `put` is a classic check-then-act race. Use the atomic compound methods: `putIfAbsent`, `computeIfAbsent`, `computeIfPresent`, `compute`, `merge`, `replace(k, old, new)`, `remove(k, v)`.

ChmDemo.java JAVA

```java
import java.util.HashMap;
import java.util.Map;
import java.util.concurrent.ConcurrentHashMap;
import java.util.concurrent.atomic.LongAdder;

public class ChmDemo {
    static void inParallel(int threads, Runnable r) throws InterruptedException {
        Thread[] ts = new Thread[threads];
        for (int i = 0; i < threads; i++) { ts[i] = new Thread(r); ts[i].start(); }
        for (Thread t : ts) t.join();
    }
```

ChmDemo.java (continued) JAVA

```java
    public static void main(String[] args) throws Exception {
        int threads = 4, perThread = 50_000;

        // BROKEN: check-then-act with individually atomic get and put.
        Map<String, Integer> broken = new ConcurrentHashMap<>();
        inParallel(threads, () -> {
            for (int i = 0; i < perThread; i++) {
                Integer old = broken.get("hits");
                broken.put("hits", old == null ? 1 : old + 1);   // another thread may interleave
            }
        });

        // CORRECT: one atomic operation.
        ConcurrentHashMap<String, Integer> merged = new ConcurrentHashMap<>();
        inParallel(threads, () -> {
            for (int i = 0; i < perThread; i++) merged.merge("hits", 1, Integer::sum);
        });

        // ALSO CORRECT and scalable: map of LongAdder.
        ConcurrentHashMap<String, LongAdder> adders = new ConcurrentHashMap<>();
        inParallel(threads, () -> {
            for (int i = 0; i < perThread; i++) {
                adders.computeIfAbsent("hits", k -> new LongAdder()).increment();
            }
        });

        long expected = (long) threads * perThread;
        System.out.println("expected             = " + expected);
        System.out.println("get+put lost updates? " + (broken.get("hits") != expected));
        System.out.println("merge                = " + merged.get("hits"));
        System.out.println("computeIfAbsent+Adder = " + adders.get("hits").sum());

        ConcurrentHashMap<String, Integer> stock = new ConcurrentHashMap<>();
        System.out.println("putIfAbsent first:  " + stock.putIfAbsent("pen", 10)); // null: absent
        System.out.println("putIfAbsent second: " + stock.putIfAbsent("pen", 99)); // old value
        stock.compute("pen", (k, v) -> v == null ? 1 : v - 3);
        System.out.println("after compute: " + stock);
        System.out.println("replace(pen, 7, 70): " + stock.replace("pen", 7, 70));
        System.out.println("remove(pen, 1): " + stock.remove("pen", 1));
        stock.put("ink", 5);
        stock.put("tape", 8);
        System.out.println("mappingCount = " + stock.mappingCount() + ", size = " + stock.size());

        // Bulk operations take a parallelism threshold (here 1 = use parallelism when possible).
        long total = stock.reduceValues(1, Integer::sum);
        System.out.println("reduceValues sum = " + total);
        System.out.println("search value > 50 -> " + stock.search(1, (k, v) -> v > 50 ? k : null));

        try {
            stock.put("null-value", null);
        } catch (NullPointerException e) {
            System.out.println("ConcurrentHashMap rejects null values (and null keys)");
        }
        Map<String, Integer> hm = new HashMap<>();
        hm.put(null, null);
        System.out.println("HashMap allows them: " + hm);
    }
}
```

Output TERMINAL

```
$ java ChmDemo.java
expected             = 200000
get+put lost updates? true
merge                = 200000
computeIfAbsent+Adder = 200000
putIfAbsent first:  null
putIfAbsent second: 10
after compute: {pen=7}
replace(pen, 7, 70): true
remove(pen, 1): false
mappingCount = 3, size = 3
```

Output (continued) TERMINAL

```
reduceValues sum = 83
search value > 50 -> pen
ConcurrentHashMap rejects null values (and null keys)
HashMap allows them: {null=null}
```

What to notice

- The `get` + `put` version loses updates even though both methods are individually thread-safe: two threads read the same old value. `merge("hits", 1, Integer::sum)` performs the whole read-modify-write atomically for that key.
- `computeIfAbsent(k, k -> new LongAdder())` is the idiom for a concurrent multi-counter or multi-map (`computeIfAbsent(key, k -> new ArrayList<>())` is **not** safe, because the `ArrayList` itself is not thread-safe; use a concurrent collection as the value).
- `putIfAbsent` returns the previous value, or `null` if the key was absent, so a `null` result means "I inserted it".
- The bulk operations (`forEach`, `reduceValues`, `search`, ...) take a **parallelism threshold**: the map size above which the work is spread over the common pool. `Long.MAX_VALUE` forces sequential execution, `1` allows maximum parallelism.

WATCH OUT

The function passed to compute/computeIfAbsent/merge runs while the bin is locked. Keep it short and quick. It must not touch the same map (no nested `map.compute` or `computeIfAbsent` on it): that can deadlock or throw `IllegalStateException: Recursive update`. Do not call slow I/O inside it; other writers to that bin wait.

8.9.7 Other concurrent collections

- `ConcurrentLinkedQueue` / `ConcurrentLinkedDeque`: unbounded, non-blocking, lock-free queues built on CAS. `poll()` returns `null` when empty (there is no `take`). `size()` walks the list, so it is O(n) and approximate; use `isEmpty()`. Choose them when consumers poll in a loop with other work to do, not when they should sleep until data arrives (use a `BlockingQueue` then).
- `ConcurrentSkipListMap` / `ConcurrentSkipListSet`: the concurrent, sorted `TreeMap`/`TreeSet` equivalent (a skip list gives O(log n) without a global lock). Use them when you need ordering plus concurrency; a `TreeMap` cannot be made concurrent cheaply.
- `CopyOnWriteArrayList` / `CopyOnWriteArraySet`: every mutation copies the whole backing array; reads and iteration need no lock and iterate over an immutable **snapshot**. Excellent for **rarely modified, frequently read** lists such as listener registries or configuration. Terrible for lists that change often (each add is O(n)).

CowDemo.java JAVA

```java
import java.util.ArrayList;
import java.util.ConcurrentModificationException;
import java.util.List;
import java.util.concurrent.CopyOnWriteArrayList;

public class CowDemo {
    public static void main(String[] args) {
        List<String> plain = new ArrayList<>(List.of("a", "b", "c"));
        try {
            for (String s : plain) if (s.equals("a")) plain.add("d");
        } catch (ConcurrentModificationException e) {
            System.out.println("ArrayList: ConcurrentModificationException");
        }

        List<String> cow = new CopyOnWriteArrayList<>(List.of("a", "b", "c"));
        for (String s : cow) {
            if (s.equals("a")) cow.add("d");          // legal: the iterator uses a snapshot
            System.out.print(s + " ");
        }
        System.out.println("-> list is now " + cow);

        var it = cow.iterator();
        cow.add("e");
        int seen = 0;
        while (it.hasNext()) { it.next(); seen++; }
        System.out.println("iterator made before add('e') saw " + seen
                + " elements; list has " + cow.size());
        try {
```

CowDemo.java (continued) JAVA

```
            it = cow.iterator();
            it.next();
            it.remove();
        } catch (UnsupportedOperationException e) {
            System.out.println("snapshot iterator does not support remove()");
        }
    }
}
```

Output TERMINAL

```
$ java CowDemo.java
ArrayList: ConcurrentModificationException
a b c -> list is now [a, b, c, d]
iterator made before add('e') saw 4 elements; list has 5
snapshot iterator does not support remove()
```

The `CopyOnWriteArrayList` loop added an element during iteration without error, but the loop did not see it (it printed `a b c`, not `d`): the iterator works on the snapshot taken when it was created. The snapshot iterator also cannot modify the list (`remove()` throws `UnsupportedOperationException`).

Synchronized wrappers versus concurrent collections

`Collections.synchronizedList/Map/...` wrap a collection so that each method takes one lock. That makes single calls thread-safe but still leaves compound actions and iteration unsafe (you must hold the wrapper's lock while iterating), and everything is serialised. The `java.util.concurrent` collections are designed for concurrency from the start: finer-grained locking or lock-free algorithms, iterators that tolerate modification, atomic compound methods. Prefer them. The legacy `Hashtable` and `Vector` are synchronised like the wrappers and are best left in the past.

Choosing

Need	Use
Shared map with updates from many threads	`ConcurrentHashMap` (+ `merge`/`compute`)
Sorted shared map or set	`ConcurrentSkipListMap` / `ConcurrentSkipListSet`
Read-mostly list, event listeners	`CopyOnWriteArrayList`
Hand work to consumers, wait when empty	`LinkedBlockingQueue` / `ArrayBlockingQueue`
Bounded buffer with back-pressure	`ArrayBlockingQueue` or `LinkedBlockingQueue(capacity)`
Non-blocking queue, poll in a loop	`ConcurrentLinkedQueue`
Delayed or scheduled elements	`DelayQueue`
Frequent counters	`LongAdder`, or `ConcurrentHashMap<K, LongAdder>`

Which concurrent structure for which job.

INTERVIEW ANGLE

How does ConcurrentHashMap achieve thread safety, and how is it different from Hashtable? `Hashtable` (and `synchronizedMap`) lock the whole map on every call. In Java 8+, `ConcurrentHashMap` reads without locks (volatile reads), inserts into an empty bin with a CAS, and otherwise synchronises only on the head node of the affected bin, so writes to different bins proceed in parallel. It forbids null keys and values, has weakly consistent iterators (no `ConcurrentModificationException`), and offers atomic compound operations (`merge`, `compute`, `putIfAbsent`). Java 7 used segment locks; Java 8 replaced them with per-bin locking.

INTERVIEW ANGLE

CountDownLatch versus CyclicBarrier? A latch is one-shot and event-based: waiters block until the count, decremented by anyone, reaches zero. A barrier is reusable and party-based: N threads wait for each other, are released together, and the barrier resets; it can run a barrier action when tripped.

REAL WORLD
A Spring Boot application uses `ConcurrentHashMap` in many places without you seeing it: bean registries, caches, `Micrometer` meters. The idiom `cache.computeIfAbsent(key, this::load)` gives a "load once per key" cache, but a slow `load` blocks other writers to the same bin, which is why dedicated cache libraries (Caffeine) use their own design. `BlockingQueue` is the buffer in log appenders and in custom background workers.

Check yourself

1. What is wrong with `if (!map.containsKey(k)) map.put(k, v);` on a `ConcurrentHashMap`, and what is the fix?
2. A worker may throw before it reaches `latch.countDown()`. What happens to `latch.await()`, and how do you prevent it?
3. What does this print? `var l = new CopyOnWriteArrayList<>(List.of(1, 2)); for (int x : l) { l.add(x + 10); } System.out.println(l);`
4. You need at most 4 concurrent calls to a rate-limited API from 50 threads. Which class, and which two calls?
5. Why does `ConcurrentHashMap` reject `null` values?
6. Which `BlockingQueue` method does not block and returns `false` when the queue is full? Which one blocks forever?

Answers

1. Check-then-act race: two threads can both see the key absent and both put. Use `putIfAbsent(k, v)` or `computeIfAbsent(k, fn)`, which are atomic.
2. It hangs forever, since the count never reaches zero. Put `countDown()` in a `finally` block.
3. `[1, 2, 11, 12]`. The loop iterates the snapshot `[1, 2]` (no exception), and each iteration appends `x + 10` to the live list.
4. `Semaphore(4)`: `acquire()` before the call and `release()` in `finally`.
5. `get(k) == null` would be ambiguous (absent versus mapped to null), and in a concurrent map you cannot resolve it with a second `containsKey` call because the state may change in between.
6. `offer(e)` returns `false` immediately when full; `put(e)` blocks until space is available.

KEY POINTS
Prefer the ready-made coordinators and collections to hand-written wait/notify.

- `CountDownLatch` (one-shot events), `CyclicBarrier` (reusable rendezvous), `Phaser` (dynamic parties), `Semaphore` (limit concurrency), `Exchanger` (swap between two threads). Release permits and count down in `finally`.
- `BlockingQueue` is producer-consumer in one object; bounded queues give back-pressure; poison pills stop consumers.
- `ConcurrentHashMap`: lock-free reads, per-bin writes, no nulls, weakly consistent iterators; use `merge`/`compute` instead of `get` + `put`.
- `CopyOnWriteArrayList` for read-mostly data; `ConcurrentSkipListMap` for sorted concurrent maps; `ConcurrentLinkedQueue` for non-blocking queues.

8.10 Fork/Join and Parallelism

Some problems split naturally into smaller copies of themselves: sum a huge array, sort a big list, search a directory tree, render the tiles of an image. Solving the pieces on different cores and combining the answers is **divide and conquer**. The Fork/Join framework (Java 7) is built for exactly that shape, and it is the engine under parallel streams and, by default, `CompletableFuture`.

You could do this with an ordinary `ThreadPoolExecutor`, but it works badly. A parent task that waits for its children holds a pool thread idle while waiting; with deep recursion the pool runs out of threads, the thread-pool deadlock again. Fork/Join solves it with a pool whose threads do useful work *while* waiting.

8.10.1 The idea: fork, compute, join

A `ForkJoinTask` is a lightweight unit of work, much cheaper than a thread (thousands of tasks may be alive on a handful of threads). You normally extend one of two subclasses:

- `RecursiveTask<V>`: `compute()` returns a result.
- `RecursiveAction`: `compute()` returns nothing (it modifies data in place).

The pattern in `compute()` is always the same:

The fork/join recipe

```
if (problem is small enough)        // the THRESHOLD
    solve it directly, sequentially
else
    split into left and right
    left.fork()                      // queue left for any worker (async)
    r = right.compute()              // do right ourselves, in this thread
    l = left.join()                  // wait for left (helping others meanwhile)
    return combine(l, r)
```

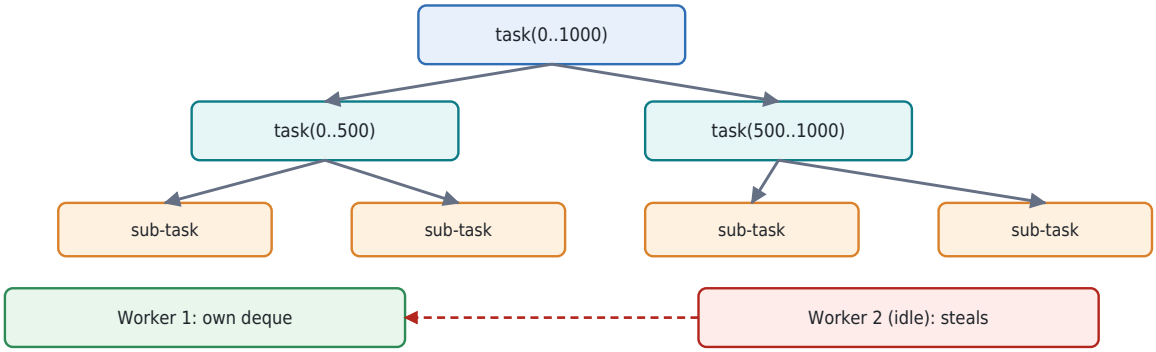


Figure 8.7. A big task is split recursively into subtasks; each worker keeps a deque of its own tasks, and an idle worker steals from the other end of a busy worker's deque.

ForkJoinSum.java JAVA

```java
import java.util.Arrays;
import java.util.concurrent.ForkJoinPool;
import java.util.concurrent.RecursiveAction;
import java.util.concurrent.RecursiveTask;

public class ForkJoinSum {
    static class SumTask extends RecursiveTask<Long> {
        private static final int THRESHOLD = 100_000;
        private final long[] data;
        private final int from, to;                      // half-open range [from, to)

        SumTask(long[] data, int from, int to) {
            this.data = data;
            this.from = from;
            this.to = to;
        }

        @Override
        protected Long compute() {
            if (to - from <= THRESHOLD) {                // small enough: do it directly
                long sum = 0;
                for (int i = from; i < to; i++) sum += data[i];
                return sum;
            }
            int mid = (from + to) >>> 1;
            SumTask left = new SumTask(data, from, mid);
            SumTask right = new SumTask(data, mid, to);
            left.fork();                                 // schedule left asynchronously
            long rightResult = right.compute();          // do right in THIS thread
            return left.join() + rightResult;            // then wait for left
        }
    }

    static class DoubleAction extends RecursiveAction {   // no result: RecursiveAction
        private final long[] data;
```

ForkJoinSum.java (continued) JAVA

```
        private final int from, to;
        DoubleAction(long[] data, int from, int to) {
            this.data = data;
            this.from = from;
            this.to = to;
        }

        @Override
        protected void compute() {
            if (to - from <= 100_000) {
                for (int i = from; i < to; i++) data[i] *= 2;
                return;
            }
            int mid = (from + to) >>> 1;
            invokeAll(new DoubleAction(data, from, mid), new DoubleAction(data, mid, to));
        }
    }

    public static void main(String[] args) {
        long[] data = new long[20_000_000];
        Arrays.setAll(data, i -> i % 100);

        long t0 = System.nanoTime();
        long seq = 0;
        for (long v : data) seq += v;
        long t1 = System.nanoTime();

        ForkJoinPool pool = new ForkJoinPool();           // parallelism = number of CPUs
        long fj = pool.invoke(new SumTask(data, 0, data.length));
        long t2 = System.nanoTime();

        long par = Arrays.stream(data).parallel().sum();  // parallel stream (common pool)
        long t3 = System.nanoTime();

        System.out.println("sequential = " + seq);
        System.out.println("fork/join  = " + fj);
        System.out.println("parallel   = " + par);
        System.out.println("all equal: " + (seq == fj && fj == par));
        System.out.println("pool parallelism = " + pool.getParallelism()
                + ", common pool parallelism = " + ForkJoinPool.commonPool().getParallelism());
        System.out.printf("typical timings (ms): sequential %d, fork/join %d, parallel stream %d%n",
                (t1 - t0) / 1_000_000, (t2 - t1) / 1_000_000, (t3 - t2) / 1_000_000);

        pool.invoke(new DoubleAction(data, 0, data.length));
        System.out.println("after DoubleAction: data[3] = " + data[3] + ", data[99] = " + data[99]);
        pool.shutdown();
    }
}
```

Output TERMINAL

```
$ java ForkJoinSum.java
sequential = 990000000
fork/join  = 990000000
parallel   = 990000000
all equal: true
pool parallelism = 2, common pool parallelism = 1
typical timings (ms): sequential 26, fork/join 39, parallel stream 39
after DoubleAction: data[3] = 6, data[99] = 198
```

What to notice

- Only one half is forked; the other is computed in the current thread (`right.compute()`). Forking both and joining both would waste the current thread, which would just sit waiting. For more than two subtasks use `invokeAll(task1, task2)` as `DoubleAction` does, which forks all but one and runs the last directly.
- `THRESHOLD` is the main tuning knob. Too small and the bookkeeping per task (object creation, queue operations) costs more than the work; too large and there are too few tasks to keep all cores busy and balance the load. A common rule: aim for several times more tasks than cores (so stealing can balance) and make each leaf do at least a few thousand simple operations.
- `pool.invoke(task)` blocks and returns the result. `pool.submit(task)` returns the task (it is a `Future`), and `execute(task)` is fire-and-forget. From inside a task use `fork()`/`join()`, not `get()`.

- The three results agree, but look at the timings (measured on a 2-CPU build machine, typical): they are almost equal. Summing an array is **memory-bound**: the CPU waits for RAM, not for arithmetic, so extra cores help little, and on two cores with a warm-up cost there is nothing to gain. Parallelism pays when each element needs substantial computation (parsing, hashing, image filters). Never assume it is faster: measure.

8.10.2 Work stealing

Each worker thread owns a **double-ended queue (deque)** of tasks. A worker pushes the tasks it forks onto one end and pops from the same end (LIFO, so it works on the most recently split, smallest, cache-warm piece). When a worker's deque is empty, it becomes a **thief**: it takes a task from the *opposite end* (the oldest task, which is usually the biggest remaining chunk) of another worker's deque. Stealing from the other end means the thief and the owner rarely touch the same element, so contention is low.

Two properties follow. Load balances itself: a worker that drew easy tasks finishes early and steals work from a worker with hard ones, without a central queue becoming a bottleneck. And `join()` is cheap: when you `join()` a task that has not finished, the framework first tries to run it (if it is still in your deque) or finds other work to do ("helping"), instead of just parking. If it truly must block, the pool can add a compensation thread. That is why deep recursion does not exhaust the pool.

8.10.3 Pools: the common pool and your own

- `new ForkJoinPool()` (parallelism = number of CPUs) or `new ForkJoinPool(n)`. Close it with `shutdown()`; its workers are daemon threads.
- `ForkJoinPool.commonPool()` is a shared static pool, parallelism `cpus - 1` by default, adjustable with `-Djava.util.concurrent.ForkJoinPool.common.parallelism=N`. It is used by **parallel streams**, `CompletableFuture.supplyAsync` (when parallelism > 1), and `ForkJoinTask.fork()` called from a non-pool thread.
- Calling `fork()` outside any pool submits to the common pool. That is convenient and also how you accidentally mix unrelated work into a shared resource.

8.10.4 Parallel streams are Fork/Join in disguise

`list.parallelStream()` or `stream.parallel()` splits the source with a `Spliterator`, processes chunks as Fork/Join tasks on the common pool, and combines the results. You saw `Arrays.stream(data).parallel().sum()` above; it did the same work as `SumTask` with one line. The Streams chapters list the rules: stateless, non-interfering functions, an associative combiner, no shared mutable state. What matters here are the operational consequences:

- All parallel streams in the JVM compete for one small pool. A slow parallel stream (or one that does blocking I/O) delays every other.
- Splitting must be cheap: arrays, `ArrayList` and ranges split well; `LinkedList` and `Stream.iterate` do not.
- There is no supported way to choose the pool per stream. The well-known trick (submitting the stream pipeline from a task running in your own `ForkJoinPool`) works in current implementations, but it is not documented behaviour; for guaranteed control write the Fork/Join tasks yourself.
- It is a good fit for CPU-bound work over large in-memory collections. It is a poor fit for blocking calls, small collections, and server code where requests already provide parallelism.

8.10.5 Mistakes

1. **Blocking I/O inside tasks.** Workers are few. A task that waits on a socket stalls a core; use virtual threads (next chapter) for blocking work.
2. **Shared mutable state.** Tasks should work on disjoint slices of data (as `DoubleAction` does: each task owns a range of indices) or return values to combine. Writing to a shared `ArrayList` or a non-atomic counter is a race.
3. **Threshold too low.** Splitting an array of 100 elements into one-element tasks creates more garbage than work.
4. **Forking and never joining**, or joining in the wrong order: a task's exception only surfaces at `join()`/`invoke()`. An unjoined task's failure is silently lost.
5. **Thinking `ForkJoinPool` replaces `ThreadPoolExecutor`.** It is optimised for many small, recursive, CPU-bound tasks with dependencies. For independent, longer or blocking tasks, a normal executor is simpler.

INTERVIEW ANGLE
What is work stealing and why does ForkJoinPool use it? Each worker has its own deque of tasks. It takes work from one end; when it runs dry it steals from the opposite end of another worker's deque. That keeps all cores busy when task sizes are uneven, avoids one shared queue as a bottleneck, and keeps contention low because owner and thief work on opposite ends.

INTERVIEW ANGLE
When is a parallel stream slower than a sequential one? For small data, for cheap per-element work (splitting and merging cost more than the work), for poorly splittable sources (`LinkedList`), for memory-bound operations like summing a big array, with blocking operations, and with ordered or stateful operations that force coordination. Measure with realistic data before adopting it.

REAL WORLD
In a web service the request threads already provide parallelism, so a parallel stream inside each request can oversubscribe the machine and let one heavy request starve the common pool for the rest. Fork/Join shines in batch jobs and libraries: `Arrays.parallelSort`, `ConcurrentHashMap` bulk operations, and image or data processing pipelines.

Check yourself

1. In `compute()`, why `left.fork(); right.compute(); left.join();` instead of forking both halves and joining both?
2. What does the THRESHOLD control and what happens if it is 1?
3. From which end does a worker take its own tasks, and from which end does a thief steal? Why different ends?
4. What is wrong with this task? `protected Void compute() { ... total += data[i]; ... }` where `total` is a shared `long` field updated by many subtasks.
5. Why was summing 20 million longs not much faster in parallel in the demo?
6. Which pool does `list.parallelStream()` use, and why is blocking I/O inside it harmful?

Answers

1. The current thread would otherwise idle while both children run elsewhere. Computing one half itself uses the thread productively and halves the number of tasks created.
2. The size at which a task stops splitting and solves the problem directly. With 1 you create a task per element, so overhead (allocation, queue operations) dwarfs the work.
3. An owner works at the newest end (LIFO, small cache-warm tasks); a thief takes from the oldest end (big pieces, and far from the owner's end so there is little contention).
4. A data race: `total += x` is not atomic and not visible between threads. Return partial sums from `compute()` and combine them, or use a `LongAdder`.
5. The work is memory-bound (the loop is limited by RAM bandwidth), and the machine had only two cores, so splitting overhead and memory limits cancel the gain.
6. The shared common pool (`cpus - 1` threads). Blocked workers leave too few threads, delaying every other parallel stream and async task in the JVM.

KEY POINTS
Fork/Join gives divide-and-conquer parallelism on a few threads.

- Extend `RecursiveTask` (result) or `RecursiveAction` (none); split until a threshold, `fork()` one half, `compute()` the other, `join()`.
- Work stealing: each worker has a deque; idle workers steal old, large tasks from the other end. `join()` helps instead of idling.
- The common pool (`cpus - 1`) serves parallel streams and default `CompletableFuture`; keep it free of blocking calls.
- Parallel is not automatically faster: measure, especially for memory-bound or small workloads.
- Tasks should own disjoint data or return results to combine; never share mutable state.

8.11 Virtual Threads

For two decades, Java servers faced an awkward choice. The natural way to write server code is **thread per request**: one thread handles one request from start to finish, calling blocking methods (database, HTTP, files) in plain sequential style. It is easy to read, easy to debug (the stack trace is the story of the request), and easy to profile. But a platform thread is an operating-system thread, and that costs memory (its stack), start-up time and scheduler work, so a server can afford only a few thousand of them. The throughput of such a server is capped by the number of threads, not by the CPU, even though most threads spend most of their time just *waiting* for I/O.

The industry answer was asynchronous programming: callbacks, `CompletableFuture` chains, reactive frameworks. They scale, but the code is harder to write, harder to debug (stack traces show the framework, not your logic), and virally changes every method signature.

Virtual threads (final in Java 21, JEP 444) give back the simple model. A virtual thread is a `Thread` that is managed by the JVM, not the operating system. It is so cheap that you create one per task, even millions, and when it blocks on I/O, it steps off its carrier thread and lets another virtual thread run. You write blocking code; the JVM makes it scale.

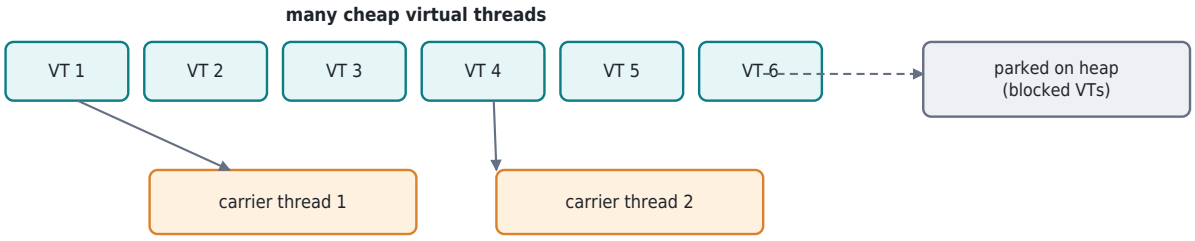


Figure 8.8. Many virtual threads are mounted on a few carrier (platform) threads one at a time; when a virtual thread blocks, it is unmounted and its stack is parked on the heap, freeing the carrier for another.

8.11.1 How they work

- **Carrier threads.** A virtual thread runs by being **mounted** on a platform thread called its carrier. The carriers are the threads of a dedicated `ForkJoinPool` (in FIFO mode), by default as many as there are CPUs (`jdk.virtualThreadScheduler.parallelism`, with a ceiling of `jdk.virtualThreadScheduler.maxPoolSize`, 256 by default).
- **Unmounting.** When the virtual thread calls a blocking operation that the JDK has been adapted for (`Thread.sleep`, socket and file-channel I/O, `java.util.concurrent` locks, `BlockingQueue.take`, `Future.get`, and so on), the JVM copies the virtual thread's stack frames into a heap object, unmounts it, and the carrier immediately picks another runnable virtual thread. When the I/O completes, the virtual thread becomes runnable again and is mounted on *some* carrier (not necessarily the same one).
- **Cheap stacks.** The stack of a virtual thread lives on the heap as stack chunk objects and starts tiny, growing with the actual call depth. A platform thread reserves a fixed stack up front (typically about 1 MB of address space). That is the difference between thousands and millions of threads.
- **Preemption is not time-sliced.** The scheduler switches virtual threads when they block, not on a timer. A virtual thread in a tight CPU loop holds its carrier until it finishes. Virtual threads are for waiting, not for computing.

They are still real `java.lang.Thread` objects. `Thread.currentThread()`, `ThreadLocal`, interruption, `join`, thread dumps and debuggers all work. Existing blocking code runs unchanged.

8.11.2 Creating them

VtBasics.java JAVA

```java
import java.time.Duration;
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;
import java.util.concurrent.atomic.AtomicInteger;
import java.util.stream.IntStream;

public class VtBasics {
    static long runTasks(ExecutorService ex, int tasks, AtomicInteger done) {
        long t0 = System.nanoTime();
        try (ex) {                                          // close() waits for all tasks
            IntStream.range(0, tasks).forEach(i -> ex.submit(() -> {
                try { Thread.sleep(Duration.ofMillis(100)); }   // simulated blocking I/O
                catch (InterruptedException e) { Thread.currentThread().interrupt(); }
                done.incrementAndGet();
            }));
        }
        return (System.nanoTime() - t0) / 1_000_000;
    }

    public static void main(String[] args) throws Exception {
        Thread vt = Thread.ofVirtual().name("order-handler").start(() ->
                System.out.println("inside: isVirtual=" + Thread.currentThread().isVirtual()
                        + ", daemon=" + Thread.currentThread().isDaemon()));
        vt.join();
        Thread unstarted = Thread.ofVirtual().unstarted(() -> {});
        System.out.println("state before start: " + unstarted.getState());
        Thread pt = Thread.ofPlatform().name("legacy").start(() -> {});
        pt.join();
        System.out.println("virtual thread name when unnamed: '"
                + Thread.startVirtualThread(() -> {}).getName() + "'");

        int tasks = 2000;
        AtomicInteger platformDone = new AtomicInteger();
        long platformMs = runTasks(Executors.newFixedThreadPool(100), tasks, platformDone);
        AtomicInteger virtualDone = new AtomicInteger();
        long virtualMs = runTasks(Executors.newVirtualThreadPerTaskExecutor(), tasks, virtualDone);

        System.out.println("platform pool of 100: " + platformDone + " tasks done");
        System.out.println("virtual threads:      " + virtualDone + " tasks done");
        System.out.printf("typical timings: platform pool %d ms, virtual threads %d ms%n",
                platformMs, virtualMs);
    }
}
```

Output TERMINAL

```
$ java VtBasics.java
inside: isVirtual=true, daemon=true
state before start: NEW
virtual thread name when unnamed: ''
platform pool of 100: 2000 tasks done
virtual threads:      2000 tasks done
typical timings: platform pool 2026 ms, virtual threads 131 ms
```

What to notice

- `Thread.ofVirtual().name("...").start(task)`, `Thread.startVirtualThread(task)`, and `Executors.newVirtualThreadPerTaskExecutor()` are the three ways in. `Thread.ofVirtual().unstarted(task)` gives a `Thread` in state `NEW` that you start yourself. There is also a `ThreadFactory` via `Thread.ofVirtual().factory()` for frameworks.
- Virtual threads are always **daemon** threads (they do not keep the JVM alive; `setDaemon(false)` is not allowed) and have an empty name unless you set one. Their priority is fixed.
- `ExecutorService` implements `AutoCloseable` since Java 19. The `try (ex)` block waits for all tasks at the end, so the timer around it measures real completion.
- The numbers (typical, measured on a 2-CPU build machine; yours will differ): 2000 tasks of 100 ms of blocking each took about 2 seconds on a pool of 100 platform threads (2000 / 100 × 100 ms) but about 0.13 seconds on virtual threads, which overlap all of them. The same code, a different executor.

8.11.3 Thread per request, finally

The following program starts an HTTP server using only JDK classes (`com.sun.net.httpserver`), tells it to run each request on a new virtual thread, and then fires 300 concurrent requests from virtual threads. Each handler blocks for 200 ms. Written in the plain blocking style, with no callbacks.

VtServer.java JAVA

```java
import com.sun.net.httpserver.HttpServer;
import java.net.InetSocketAddress;
import java.net.URI;
import java.net.http.HttpClient;
import java.net.http.HttpRequest;
import java.net.http.HttpResponse;
import java.nio.charset.StandardCharsets;
import java.util.ArrayList;
import java.util.List;
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;
import java.util.concurrent.Future;

public class VtServer {
    public static void main(String[] args) throws Exception {
        HttpServer server = HttpServer.create(new InetSocketAddress("127.0.0.1", 0), 0);
        // Thread-per-request: every request gets its own virtual thread.
        server.setExecutor(Executors.newVirtualThreadPerTaskExecutor());
        server.createContext("/order", exchange -> {
            try {
                Thread.sleep(200);                      // pretend to call a database
            } catch (InterruptedException e) {
                Thread.currentThread().interrupt();
            }
            byte[] body = ("virtual=" + Thread.currentThread().isVirtual())
                    .getBytes(StandardCharsets.UTF_8);
            exchange.sendResponseHeaders(200, body.length);
            try (var out = exchange.getResponseBody()) { out.write(body); }
        });
        server.start();
        URI uri = URI.create("http://127.0.0.1:" + server.getAddress().getPort() + "/order");

        HttpClient client = HttpClient.newHttpClient();
        long t0 = System.nanoTime();
        int ok = 0;
        try (ExecutorService callers = Executors.newVirtualThreadPerTaskExecutor()) {
            List<Future<HttpResponse<String>>> results = new ArrayList<>();
            for (int i = 0; i < 300; i++) {
                results.add(callers.submit(() -> client.send(HttpRequest.newBuilder(uri).build(),
                        HttpResponse.BodyHandlers.ofString())));
            }
            for (var f : results) {
                var r = f.get();
                if (r.statusCode() == 200 && r.body().equals("virtual=true")) ok++;
            }
        }
        long ms = (System.nanoTime() - t0) / 1_000_000;
        System.out.println(ok + " of 300 requests answered by a virtual thread");
        System.out.println("each request blocks 200 ms; total " + ms + " ms (serial: 60000 ms)");
        server.stop(0);
    }
}
```

Output TERMINAL

```
$ java VtServer.java
300 of 300 requests answered by a virtual thread
each request blocks 200 ms; total 1324 ms (serial: 60000 ms)
```

300 requests, each blocking 200 ms, completed in well under a second (the printed time is typical and varies from run to run; serial execution would need 60 seconds). Every request was handled on a virtual thread. In a framework the setup is one line: in Spring Boot 3.2+, `spring.threads.virtual.enabled=true` makes the embedded server and the async executors use virtual threads.

8.11.4 Rules for using virtual threads well

1. **Do not pool them.** A pool exists to ration an expensive resource; virtual threads are not expensive. Create one per task. If you want to *limit concurrency* (say, 20 calls into a rate-limited service), use a `Semaphore`, not a pool of virtual threads.
2. **Use them for blocking I/O-bound work**, not CPU-bound loops. For CPU-bound work a pool sized to the core count (or Fork/Join) is still right; virtual threads add nothing there.
3. **Keep code blocking and simple.** Do not wrap blocking calls in `CompletableFuture` just to make them "async"; a virtual thread already gives you that.
4. **Mind the resources behind the threads.** A million virtual threads can open a million connections; your database has a connection pool of 20. The pool becomes the bottleneck, which is how it should be, but make sure waiting there has a timeout.
5. **Prefer `ReentrantLock` and other `java.util.concurrent` tools** to `synchronized` around blocking calls on Java 21 (see pinning).

8.11.5 Pinning

A virtual thread is **pinned** when it cannot unmount while blocked; it then holds its carrier hostage. In Java 21 the two causes are (1) blocking while inside a `synchronized` block or method, and (2) blocking in a native method or a foreign-function call. Pinning is not a correctness bug; the program works, but you lose scalability. Because there are only as many carriers as CPUs, a few pinned threads can stall the whole scheduler.

The program below runs eight virtual threads that each sleep 200 ms while holding a private lock (so there is no lock contention at all), once with `ReentrantLock` and once with `synchronized`. We limit the carriers to two with system properties set at the top of `main` (the command-line `-D` flags do the same):

VtPinning.java JAVA

```
import java.time.Duration;
import java.util.ArrayList;
import java.util.List;
import java.util.concurrent.locks.ReentrantLock;

public class VtPinning {
    static void sleep() {
        try { Thread.sleep(Duration.ofMillis(200)); }
        catch (InterruptedException e) { Thread.currentThread().interrupt(); }
    }

    static long run(boolean useSynchronized) throws Exception {
        List<Thread> threads = new ArrayList<>();
        long t0 = System.nanoTime();
        for (int i = 0; i < 8; i++) {
            final Object monitor = new Object();             // private lock: no contention
            final ReentrantLock lock = new ReentrantLock();
            threads.add(Thread.ofVirtual().start(() -> {
                if (useSynchronized) {
                    synchronized (monitor) { sleep(); }      // blocks while holding a monitor
                } else {
                    lock.lock();
                    try { sleep(); } finally { lock.unlock(); }
                }
            }));
        }
        for (Thread t : threads) t.join();
        return (System.nanoTime() - t0) / 1_000_000;
    }

    public static void main(String[] args) throws Exception {
        // Must run before the first virtual thread starts (or pass -D flags on the command line).
        System.setProperty("jdk.virtualThreadScheduler.parallelism", "2");
        System.setProperty("jdk.virtualThreadScheduler.maxPoolSize", "2");
        System.out.println("8 virtual threads, each sleeps 200 ms, only 2 carrier threads");
        System.out.println("typical: with ReentrantLock  -> " + run(false) + " ms");
        System.out.println("typical: with synchronized   -> " + run(true) + " ms");
    }
}
```

Output TERMINAL

```
$ java VtPinning.java
8 virtual threads, each sleeps 200 ms, only 2 carrier threads
typical: with ReentrantLock  -> 215 ms
typical: with synchronized   -> 805 ms
```

With the `ReentrantLock`, a sleeping thread unmounts, so all eight overlap: about 200 ms. With `synchronized`, a sleeping thread keeps its carrier, so only two run at a time: 8 / 2 × 200 ms = about 800 ms. On Java 21 you can see the culprits with `-Djdk.tracePinnedThreads=short`, which printed lines like this during the synchronized run:

Pinning trace on Java 21 (excerpt) TERMINAL

```
$ java -Djdk.tracePinnedThreads=short VtPinning.java
VirtualThread[#30]/runnable@ForkJoinPool-1-worker-1 reason:MONITOR
    VtPinning.lambda$run$0(VtPinning.java:20) <== monitors:1
```

Java 24 changed this. JEP 491 (*Synchronize Virtual Threads without Pinning*, Java 24) lets a virtual thread that blocks inside `synchronized` release its carrier, so on Java 24 and 25 the second line above would be about as fast as the first. Pinning for native and foreign calls remains. (The listing and its output are Java 21 behaviour; the Java 24 change is described from the JEP and was not run here.) The practical consequences: on Java 21 audit libraries and your code for blocking calls inside `synchronized` (older JDBC drivers and some logging code were the usual suspects), and on 24+ you can mostly stop worrying. The JFR event `jdk.VirtualThreadPinned` reports pinned blocking that lasts longer than a threshold.

8.11.6 ThreadLocal and virtual threads

Virtual threads support `ThreadLocal`, including inheritance. It works, but think before using it. A `ThreadLocal` was designed for the world of few, long-lived, pooled threads, where caching an expensive object (a `SimpleDateFormat`, a buffer) in each thread is a good deal. With a million short-lived virtual threads each holding its own copy, the "cache" becomes a million copies. For **per-request context** (user, trace ID, transaction) prefer a parameter, or a `ScopedValue` (next chapter), which is immutable, automatically cleaned up and cheap. Never keep a `ThreadLocal` value that is large or that expects `remove()` to be called in a thread that gets reused: virtual threads are not reused, so the value simply dies with the thread, but the memory is held while it is alive.

8.11.7 Observability

- **Thread dumps**: the classic `jstack` and `jcmd <pid> Thread.print` do *not* list virtual threads (there could be millions). Use `jcmd <pid> Thread.dump_to_file -format=json file.json` to get a dump that includes them, grouped by their owner in structured-concurrency scopes.
- **JFR**: events `jdk.VirtualThreadStart`, `jdk.VirtualThreadEnd`, `jdk.VirtualThreadPinned` and `jdk.VirtualThreadSubmitFailed`.
- A debugger and stack traces work as for platform threads: the stack trace shows your own code path, one of the main wins over async styles.

Aspect	Platform thread	Virtual thread
Managed by	Operating system	JVM (user-mode scheduler)
Stack	Fixed reservation (about 1 MB virtual address space)	Heap object, grows with call depth
Typical count	Hundreds to thousands	Hundreds of thousands to millions
Creation cost	Microseconds, kernel call	Very cheap, no kernel call
Pooling	Yes, always	No, one per task
Best for	CPU-bound work, long-lived workers	Blocking I/O, thread-per-request
Daemon	Configurable	Always daemon
Time slicing	OS preempts	Switches only when blocking

Platform threads versus virtual threads.

INTERVIEW ANGLE
What problem do virtual threads solve, and how do they differ from reactive programming? They remove the limit on the number of concurrent blocking tasks. Blocking calls unmount a virtual thread from its carrier, so a handful of OS threads can run millions of them, while code stays in simple sequential style with intact stack traces. Reactive programming achieves similar scalability by never blocking at all, at the price of callback/operator-chain code and awkward debugging. Virtual threads do not make CPU-bound code faster.

INTERVIEW ANGLE
What is pinning? A virtual thread is pinned when it blocks but cannot unmount, so it keeps occupying its carrier. In Java 21 this happens when blocking inside `synchronized` or in a native/foreign call. Java 24 (JEP 491) removed the `synchronized` case. Detect with `-Djdk.tracePinnedThreads` (21) or the `jdk.VirtualThreadPinned` JFR event.

REAL WORLD
Spring Boot 3.2 and later switch Tomcat and `@Async` to virtual threads with one property, `spring.threads.virtual.enabled=true`. The usual wins are services that spend their time waiting on databases and downstream HTTP calls. The usual surprises are a connection pool that becomes the true bottleneck, libraries that pin on Java 21, and monitoring tools that count platform threads.

Check yourself

1. Why is `Executors.newVirtualThreadPerTaskExecutor()` not a "pool"? What would you use to cap concurrency?
2. What does this print on Java 21? `Thread t = Thread.ofVirtual().unstarted(() -> {}); System.out.println(t.getState() + " " + t.isDaemon());`
3. A virtual thread does `synchronized (lock) { Thread.sleep(1000); }` on Java 21 with 4 CPUs. What happens to its carrier, and what changed in Java 24?
4. Will 10,000 virtual threads each running `while (true) { x++; }` all make progress on a 4-CPU machine? Why or why not?
5. Why is a thread-per-request design with platform threads limited, and what limit remains with virtual threads?
6. Why is putting a 500 KB buffer in a `ThreadLocal` a worse idea with virtual threads than with a 200-thread pool?

Answers

1. It does not reuse or limit threads: every task gets a fresh virtual thread. Cap concurrency with a `Semaphore` (acquire before the call, release in `finally`).
2. `NEW true`. Virtual threads are always daemon threads, and an unstarted one is in state `NEW`.
3. The carrier is pinned and blocked for the full second, unusable by other virtual threads. Java 24 (JEP 491) lets the virtual thread release the carrier even inside `synchronized`.
4. No. Scheduling is cooperative: a CPU-bound virtual thread never blocks, so it never unmounts. Only 4 run at a time, each holding its carrier until it ends (never, here); the other 9,996 starve.
5. Each platform thread costs OS resources and memory, so a few thousand is the practical ceiling. With virtual threads the remaining limits are the downstream resources (database connections, sockets, rate limits) and CPU for the actual work.
6. Each virtual thread would get its own 500 KB copy; with 100,000 concurrent virtual threads that is about 50 GB. A pool of 200 threads holds 200 copies, about 100 MB.

KEY POINTS
Virtual threads make blocking code scale.

- A virtual thread (Java 21, JEP 444) is a JVM-managed `Thread` mounted on a few carrier threads; blocking operations unmount it so the carrier can run another.
- Create one per task with `Executors.newVirtualThreadPerTaskExecutor()` or `Thread.ofVirtual()`. Never pool them; limit concurrency with a `Semaphore`.
- They help I/O-bound workloads; they do not speed up CPU-bound work and are not time-sliced.
- Pinning (blocking inside `synchronized` or native code) holds the carrier; fixed for `synchronized` in Java 24 (JEP 491). Use `ReentrantLock` on Java 21.
- Prefer parameters or `ScopedValue` to `ThreadLocal` for per-request context; use `jcmd Thread.dump_to_file` for dumps.

8.12 Structured Concurrency, Scoped Values and Practice

Two more ideas finish the modern toolbox, and then we turn to the practical side: how to design, test and debug concurrent code. The first idea, **structured concurrency**, says that if a task splits into concurrent subtasks, those subtasks should live and die inside the task's own block of code, exactly like local variables and method calls do. The second, **scoped values**, gives the subtasks an easy, safe way to see the caller's context.

8.12.1 Why structured concurrency

Look at what unstructured concurrency does when you fetch a user and their orders in parallel with an `ExecutorService`:

Unstructured style (fragment) JAVA

```
Future<User> user = executor.submit(() -> findUser(id));
Future<List<Order>> orders = executor.submit(() -> findOrders(id));
User u = user.get();            // if this throws, 'orders' keeps running: a leak
List<Order> o = orders.get();   // if this is slow, nobody cancels anything
```

If `findUser` fails, `findOrders` continues to run for nothing. If the calling thread is interrupted, the subtasks are not. The relationship between the parent and the children exists only in your head; the thread dump shows unrelated pool threads, and nothing guarantees that a child has finished when the method returns. This is the same flaw that `goto` had: control flow that escapes its block.

Structured concurrency (`StructuredTaskScope`) restores the guarantees:

- Subtasks are **forked inside a scope** (a try-with-resources block) and cannot outlive it.
- `join()` waits for them, and the scope's `close()` ensures none is still running when the block exits.
- If one fails (in the shutdown-on-failure policy) the siblings are **cancelled**.
- An interrupt of the owner thread cancels the subtasks.
- Each subtask runs in its own virtual thread, so blocking is cheap, and thread dumps show the parent-child tree.

8.12.2 Status: a preview feature

Be precise about the status, because it determines whether you can use it in production. `StructuredTaskScope` has been a **preview API** since Java 21 (JEP 453) and was still a preview in Java 25 (JEP 505, fifth preview), Java 26 (JEP 525, sixth) and Java 27 (JEP 533, seventh). Preview APIs need `--enable-preview` at compile and run time and are allowed to change between releases, and this one has changed: the Java 21 shape uses constructors (`new StructuredTaskScope.ShutdownOnFailure()`), while the Java 25 preview uses a factory `StructuredTaskScope.open(...)` with pluggable **joiners**. Learn the concept now, write production code against it only if you accept recompiling for each release, and check the current JEP before copying any code.

8.12.3 The API in practice

This program runs on Java 21 with `--enable-preview` (the Java 21 preview API). It forks two subtasks, waits for both, and then shows what happens when one of them fails.

StructuredDemo.java (Java 21 preview API) JAVA

```
import java.util.concurrent.StructuredTaskScope;
import java.util.concurrent.StructuredTaskScope.Subtask;

public class StructuredDemo {
    static String fetchUser() throws InterruptedException {
        Thread.sleep(100);
        return "maya";
    }

    static int fetchOrders() throws InterruptedException {
        Thread.sleep(150);
        return 3;
    }

    static int failing() throws Exception {
        Thread.sleep(50);
        throw new IllegalStateException("inventory service down");
    }
```

StructuredDemo.java (Java 21 preview API) (continued) JAVA

```
    public static void main(String[] args) throws Exception {
        // Java 21 preview API (JEP 453). All subtasks succeed.
        try (var scope = new StructuredTaskScope.ShutdownOnFailure()) {
            Subtask<String> user = scope.fork(StructuredDemo::fetchUser);
            Subtask<Integer> orders = scope.fork(StructuredDemo::fetchOrders);
            scope.join().throwIfFailed();                  // wait for both; rethrow first failure
            System.out.println(user.get() + " has " + orders.get() + " orders");
        }

        // One subtask fails: the sibling is cancelled (interrupted) and join reports the failure.
        try (var scope = new StructuredTaskScope.ShutdownOnFailure()) {
            Subtask<String> slow = scope.fork(() -> {
                try {
                    Thread.sleep(5000);
                    return "slow done";
                } catch (InterruptedException e) {
                    System.out.println("sibling was cancelled");
                    throw e;
                }
            });
            scope.fork(StructuredDemo::failing);
            scope.join();
            try {
                scope.throwIfFailed();
            } catch (java.util.concurrent.ExecutionException e) {
                System.out.println("failed: " + e.getCause().getMessage());
            }
            System.out.println("slow state: " + slow.state());
        }
    }
}
```

Output TERMINAL

```
$ java --enable-preview --source 21 StructuredDemo.java
maya has 3 orders
sibling was cancelled
failed: inventory service down
slow state: UNAVAILABLE
```

What to notice

- Both `fork` calls happen inside the `try` block; the scope closes at the closing brace, so no subtask outlives the method. Compare with the unstructured fragment above.
- `scope.join().throwIfFailed()` waits for all subtasks and rethrows the first failure. Only after a successful `join()` do you call `Subtask.get()` (calling it earlier throws `IllegalStateException`).
- In the second block `failing()` throws after 50 ms. `ShutdownOnFailure` then **interrupts the slow sibling** (the message "sibling was cancelled" was printed by the sibling itself) instead of letting it run for 5 seconds. The cancelled subtask's state is `UNAVAILABLE`.
- Because it uses virtual threads, forking thousands of subtasks that block on I/O is fine.

The same code in the Java 25 preview API looks like this. It is a sketch to show the shape; I could not run it on the JDK 21 installed here.

Java 25 preview API (sketch; not run on 21) JAVA

```
import java.util.concurrent.StructuredTaskScope;
import java.util.concurrent.StructuredTaskScope.Subtask;

class ProfileService {
    record Profile(String user, int orders) {}

    Profile load(long id) throws Exception {
        try (var scope = StructuredTaskScope.open()) {   // fail fast, cancel siblings
            Subtask<String> user = scope.fork(() -> fetchUser(id));
            Subtask<Integer> orders = scope.fork(() -> fetchOrders(id));
            scope.join();                                // throws if any subtask failed
            return new Profile(user.get(), orders.get());
        }
    }
```

Java 25 preview API (sketch; not run on 21) (continued) JAVA

```
    String fastest() throws Exception {                  // race two replicas
        try (var scope = StructuredTaskScope.open(
                StructuredTaskScope.Joiner.<String>anySuccessfulResultOrThrow())) {
            scope.fork(() -> queryReplica("eu"));
            scope.fork(() -> queryReplica("us"));
            return scope.join();                         // first success; the loser is cancelled
        }
    }

    String fetchUser(long id) { return "maya"; }
    int fetchOrders(long id) { return 3; }
    String queryReplica(String region) { return region; }
}
```

In this version the **joiner** passed to `open` decides the policy: the no-argument `open()` waits for all subtasks and fails if any fails; `anySuccessfulResultOrThrow()` returns the first successful result and cancels the rest; `allSuccessfulOrThrow()` collects all results. A timeout can be configured when opening the scope. The method names and signatures have moved between previews, so treat the names above as accurate to the Java 25 Javadoc at the time of writing and verify them against the release you use.

8.12.4 Scoped values

`ThreadLocal` has three well-known problems: it is **mutable** (any code can `set` it), it **lives as long as the thread** (so you must remember `remove()`), and every child thread must **copy** inherited values, which is expensive with millions of virtual threads. A `ScopedValue<T>` fixes all three. A value is bound for the duration of a call; inside it, any code can read it; when the call returns, the binding is gone.

`ScopedValue` was a preview in Java 21 to 24 and became **final in Java 25** (JEP 506). The program below ran on Java 21 with `--enable-preview`; on Java 25 the same source compiles without the flag (the one change in 25 is that `orElse` no longer accepts `null`).

ScopedDemo.java (Java 21 preview; final in 25) JAVA

```
public class ScopedDemo {
    record RequestContext(String user, String traceId) {}

    // Like a ThreadLocal, but immutable, bounded to a block, and cheap for virtual threads.
    static final ScopedValue<RequestContext> CTX = ScopedValue.newInstance();

    static void handle() {
        System.out.println("handling for " + CTX.get().user() + " trace=" + CTX.get().traceId());
        audit();
    }

    static void audit() {                                // no parameter passing needed
        System.out.println("audit: " + CTX.get().user());
    }

    public static void main(String[] args) throws Exception {
        System.out.println("bound before? " + CTX.isBound());
        ScopedValue.where(CTX, new RequestContext("maya", "t-1")).run(() -> {
            handle();
            // Rebinding for an inner call; the outer binding returns afterwards.
            ScopedValue.where(CTX, new RequestContext("batch", "t-2")).run(ScopedDemo::audit);
            System.out.println("back to " + CTX.get().user());
        });
        System.out.println("bound after?  " + CTX.isBound());
        String result = ScopedValue.where(CTX, new RequestContext("lee", "t-3")).call(() -> {
            return "call() returned for " + CTX.get().user();
        });
        System.out.println(result);
        try {
            CTX.get();
        } catch (java.util.NoSuchElementException e) {
            System.out.println("get() outside a binding -> NoSuchElementException");
        }
    }
}
```

Output TERMINAL

```
$ java --enable-preview --source 21 ScopedDemo.java
bound before? false
handling for maya trace=t-1
audit: maya
audit: batch
back to maya
bound after?  false
call() returned for lee
get() outside a binding -> NoSuchElementException
```

What to notice

- `ScopedValue.where(KEY, value).run(task)` binds the value for the extent of `task`; `call(...)` is the variant that returns a value and may throw. Several bindings chain: `where(A, a).where(B, b).run(...)`.
- No parameter passing: `audit()` reads `CTX.get()` even though `handle()` never passed it. This is what makes it useful for request context, security principals and trace IDs.
- **Rebinding** is a nested scope: the inner `where(...).run(...)` shadows the outer binding, and the outer value is back afterwards (`back to maya`).
- The value is **immutable**: there is no `set`. To change context, bind a new value for an inner call. `isBound()` tells you whether a binding exists, and `get()` outside a binding throws `NoSuchElementException`.
- `StructuredTaskScope` subtasks automatically **inherit** the scoped values of the owner, with no copying, which is why the two features were designed together.

Aspect	ThreadLocal	ScopedValue
Mutability	`set()` anywhere	Immutable; rebind for an inner scope
Lifetime	Until `remove()` or thread death	The `run`/`call` block
Cleanup bugs	Leaks in pooled threads	Not possible
Inheritance by child threads	Copy (inheritable variant)	Shared, no copy (structured scopes)
Cost with millions of virtual threads	High	Low
Status	Since Java 1.2	Final in Java 25

ThreadLocal versus ScopedValue.

GOTCHA
A scoped value is not inherited by arbitrary threads. If you bind a value and then submit a task to an ordinary `ExecutorService`, the task does not see the binding: the pool thread was not created inside the scope. Only subtasks forked through a `StructuredTaskScope` inherit it. For everything else, pass the value as a parameter.

8.12.5 A concurrency design checklist

Before you write `synchronized` or `new Thread`, walk through these questions. Most concurrency bugs are design bugs.

1. **Do you need shared mutable state at all?** Prefer immutability, confinement to one thread, or message passing (queues) before sharing.
2. **If you must share, who owns it?** Name the lock or the single thread that guards each piece of state; document it (`@GuardedBy`-style comments). Guard *all* accesses, reads included.
3. **What is the invariant?** If two fields must change together, one lock must cover both, or the pair must be one immutable object swapped atomically through an `AtomicReference`.
4. **Use the highest-level tool that fits**: a concurrent collection, `ExecutorService`, `CompletableFuture`, a virtual thread per task, before locks, before `wait/notify`, before atomics and `volatile`.
5. **Keep critical sections short, with no alien calls.** Do not call unknown code (callbacks, listeners, overridable methods) or do I/O while holding a lock.
6. **Order your locks** (always take A before B) or use `tryLock` with a timeout.
7. **Bound everything**: queue sizes, pool sizes, wait times, retries. Unbounded queues and untimed waits are outages waiting for a trigger.

8. **Handle interruption and shutdown**: every blocking loop must exit on interrupt; every pool must be shut down; every `Future` failure must be observed.
9. **Name your threads** and pools, so dumps and logs are readable.
10. **Document the thread-safety** of each class: immutable, thread-safe, conditionally thread-safe, or not thread-safe.

Situation	Reach for	Avoid
Count or accumulate	`LongAdder`, `AtomicLong`, `merge` on a CHM	`volatile` with `++`
Share a map	`ConcurrentHashMap`	`HashMap` plus ad-hoc locks
Hand work between threads	`BlockingQueue`	Hand-written `wait/notify`
Many blocking tasks	Virtual threads	Thousands of platform threads
CPU-bound batch	Fixed pool of cores, Fork/Join, parallel streams	Over-sized pools
Compose async results	`CompletableFuture` with own executor	Blocking `get()` inside pool tasks
Fan-out and fan-in	`StructuredTaskScope` (preview) or `invokeAll`	Orphaned futures
Per-request context	Parameter, `ScopedValue` (25)	Unremoved `ThreadLocal`

Picking a tool.

8.12.6 Testing concurrent code

Concurrency bugs are rare events: a test can pass a thousand times and fail in production. You cannot prove the absence of races by testing, but you can raise your chances of finding them.

- **Design for testability.** Keep concurrency in a thin layer; test the logic single-threaded. Pass in the `Executor` so a test can use a direct (same-thread) executor.
- **Maximise overlap.** Start threads together with a `CountDownLatch` or `CyclicBarrier` start line, loop many times, and use more threads than cores.
- **Repeat.** A race that appears once in 10,000 runs shows up in a loop of 100,000.
- **Always use timeouts** in tests (`future.get(5, SECONDS)`, `@Timeout` in JUnit 5). A deadlocked test that hangs the build is worse than a failing one.
- **Assert on invariants**, not on timing. Never use `Thread.sleep` to "wait until done"; wait on a latch or a future, or poll with a library such as Awaitility.
- **Use tools** built for the job: JCStress (the OpenJDK concurrency stress-testing harness) for low-level memory-model questions, and Lincheck-style linearizability checkers for concurrent data structures.

The harness below applies the first three rules: four threads released by a barrier, 100 trials of 80,000 increments, once for a plain `int` and once for an `AtomicInteger`.

StressHarness.java JAVA

```
import java.util.concurrent.CyclicBarrier;
import java.util.concurrent.atomic.AtomicInteger;
import java.util.function.Supplier;

public class StressHarness {
    interface Counter { void inc(); int get(); }

    static class Unsafe implements Counter {
        private int n;
        public void inc() { n++; }
        public int get() { return n; }
    }

    static class Safe implements Counter {
        private final AtomicInteger n = new AtomicInteger();
        public void inc() { n.incrementAndGet(); }
        public int get() { return n.get(); }
    }

    /** Runs one trial: all threads start together (barrier), each increments `each` times. */
    static boolean trial(Supplier<Counter> factory, int threads, int each) throws Exception {
        Counter c = factory.get();
```

StressHarness.java (continued) JAVA

```
        CyclicBarrier startLine = new CyclicBarrier(threads);
        Thread[] ts = new Thread[threads];
        for (int i = 0; i < threads; i++) {
            ts[i] = new Thread(() -> {
                try {
                    startLine.await();                          // maximise overlap
                    for (int k = 0; k < each; k++) c.inc();
                } catch (Exception e) {
                    throw new RuntimeException(e);
                }
            });
            ts[i].start();
        }
        for (Thread t : ts) t.join();
        return c.get() == threads * each;
    }

    static int failures(Supplier<Counter> factory, int trials) throws Exception {
        int bad = 0;
        for (int i = 0; i < trials; i++) if (!trial(factory, 4, 20_000)) bad++;
        return bad;
    }

    public static void main(String[] args) throws Exception {
        int trials = 100;
        boolean unsafeFailed = failures(Unsafe::new, trials) > 0;
        System.out.println("Unsafe counter: failed trials > 0 ? " + unsafeFailed);
        System.out.println("Safe counter:   failed trials = " + failures(Safe::new, trials));
    }
}
```

Output TERMINAL

```
$ java StressHarness.java
Unsafe counter: failed trials > 0 ? true
Safe counter:   failed trials = 0
```

The plain counter failed at least one trial (typically most of them, which is what you want from a stress test of a known bug); the atomic one never failed. Note what the test can and cannot prove. The failure is conclusive. The success only says "not found in 100 trials on this machine".

WATCH OUT

A passing concurrent test is not a proof. Different CPUs, JIT states and loads change interleavings. Run stress tests in CI on more than one machine, keep them fast enough to run often, and complement them with code review against the checklist above.

8.12.7 Debugging concurrent code

When a service hangs or crawls, you need to see what its threads are doing.

Thread dumps

`jstack <pid>` or `jcmd <pid> Thread.print` prints every platform thread with its state and stack (the monitor chapter shows reading one). Take **three dumps a few seconds apart**: threads that are in the same place in all three are your suspects. What to look for:

- Many threads `BLOCKED` on the same monitor, with one owner doing slow work: lock contention.
- A cycle of "waiting to lock X held by thread Y": a deadlock. `jstack` prints "Found one Java-level deadlock" for monitors and `java.util.concurrent` locks.
- Pool threads all `WAITING` in `Future.get()` or `CountDownLatch.await()`: thread-pool starvation.
- Threads `RUNNABLE` in `java.net.SocketInputStream.read` or similar: waiting on the network, not computing; the CPU is idle.
- Virtual threads: use `jcmd <pid> Thread.dump_to_file -format=json out.json`.

You can also detect deadlocks from inside the program, which is useful for a watchdog thread that logs and alerts. This one is the same two-lock deadlock as the liveness chapter, found with `ThreadMXBean`:

DeadlockWatch.java JAVA

```java
import java.lang.management.ManagementFactory;
import java.lang.management.ThreadInfo;
import java.lang.management.ThreadMXBean;
import java.util.concurrent.CountDownLatch;

public class DeadlockWatch {
    static final Object A = new Object();
    static final Object B = new Object();

    static Thread locker(String name, Object first, Object second, CountDownLatch bothHoldFirst) {
        Thread t = new Thread(() -> {
            synchronized (first) {
                bothHoldFirst.countDown();
                try { bothHoldFirst.await(); } catch (InterruptedException e) { return; }
                synchronized (second) {
                    System.out.println("never reached");
                }
            }
        }, name);
        t.setDaemon(true);                                  // so the demo can exit
        return t;
    }

    public static void main(String[] args) throws Exception {
        CountDownLatch latch = new CountDownLatch(2);
        locker("transfer-1", A, B, latch).start();
        locker("transfer-2", B, A, latch).start();
        Thread.sleep(300);

        ThreadMXBean mx = ManagementFactory.getThreadMXBean();
        long[] ids = mx.findDeadlockedThreads();            // null when there is no deadlock
        System.out.println("deadlocked threads found: " + (ids == null ? 0 : ids.length));
        if (ids != null) {
            for (ThreadInfo info : mx.getThreadInfo(ids)) {
                System.out.println(info.getThreadName() + " is " + info.getThreadState()
                        + ", waiting for a lock held by " + info.getLockOwnerName());
            }
        }
    }
}
```

Output TERMINAL

```
$ java DeadlockWatch.java
deadlocked threads found: 2
transfer-1 is BLOCKED, waiting for a lock held by transfer-2
transfer-2 is BLOCKED, waiting for a lock held by transfer-1
```

`findDeadlockedThreads()` returns `null` when there is no deadlock and the thread IDs when there is. (The threads here are daemons so the demo can exit; in a real deadlock they would hold the JVM hostage.)

JFR and logging

Java Flight Recorder (`-XX:StartFlightRecording`, see the diagnostics Part) records lock contention (`jdk.JavaMonitorEnter`, `jdk.JavaMonitorWait`), parking (`jdk.ThreadPark`), thread start/end and, with virtual threads, pinning events. Open the recording in JDK Mission Control or print events with the `jfr` tool. In logs, always print the thread name, and add a request or trace ID, so you can follow one request across threads.

Reproducing the problem

- Reduce threads or add them; shrink pools to 1 to provoke starvation.
- Insert deliberate delays (`Thread.sleep`, `Thread.yield`) at suspected race points in a test build to widen the window.
- Run on fewer or more cores (`-XX:ActiveProcessorCount=N`) to change scheduling.
- Remember that adding logging or a debugger changes timing: a bug that disappears under the debugger is a classic sign of a race (a Heisenbug).

INTERVIEW ANGLE

How would you debug a Java service that is hung in production? Take several thread dumps (`jcmd <pid> Thread.print` or `jstack`) a few seconds apart and compare. Look for a reported deadlock; for many threads BLOCKED on one monitor (contention); for pool threads waiting on futures (starvation); for threads stuck in socket reads (downstream slowness). Correlate with CPU use (high CPU plus RUNNABLE: spinning or busy; low CPU plus waiting: blocked), GC logs (a long pause looks like a hang) and, if available, a JFR recording. Then reproduce with a smaller pool or injected delay and fix the root cause: lock ordering, a bounded queue, timeouts.

INTERVIEW ANGLE

What is structured concurrency? A discipline where concurrent subtasks are forked inside a block (a scope) and must finish before the block exits, like method calls. Failures and cancellation propagate between the parent and children, so there are no leaked tasks, and thread dumps show the hierarchy. Java offers it as the `StructuredTaskScope` preview API (JEP 505 and successors); it is not final yet.

Check yourself

1. In the unstructured fragment at the start of this chapter, `user.get()` throws. What happens to the `orders` task?
2. Is `StructuredTaskScope` a final feature in Java 25? And `ScopedValue`?
3. What does this print (Java 25 or 21 with preview)? `ScopedValue<String> X = ScopedValue.newInstance(); ScopedValue.where(X, "a").run(() -> { ScopedValue.where(X, "b").run(() -> System.out.print(X.get())); System.out.print(X.get()); });`
4. A task is submitted to a normal `ExecutorService` from inside a `ScopedValue.where(...).run(...)` block and calls `X.get()`. What happens?
5. Why is `Thread.sleep(1000)` a poor way to wait for a concurrent test to finish? What do you use instead?
6. A service is slow, CPU is at 5 percent, and the thread dumps show 190 of 200 pool threads in `Socket.read`. What does that suggest?

Answers

1. It keeps running, and nobody cancels it. Its result is simply discarded, wasting resources and possibly causing side effects. Structured concurrency cancels siblings when the scope fails.
2. No for `StructuredTaskScope`: it is still a preview API in 25 (fifth preview, JEP 505) and in later releases. Yes for `ScopedValue`: final in Java 25 (JEP 506).
3. `ba`. The inner binding `b` shadows the outer one for the inner call; afterwards the outer value `a` is visible again.
4. The pool thread has no binding, so `X.get()` throws `NoSuchElementException`. Only subtasks forked through a `StructuredTaskScope` inherit scoped values. Pass the value as a parameter.
5. It is either too long (slow tests) or too short (flaky tests) and tests nothing about completion. Wait on a `CountDownLatch`, a `Future.get(timeout)`, or poll with a timeout (Awaitility).
6. The threads are waiting on a remote service, not computing. The downstream dependency is slow; look at its latency, timeouts and connection limits. A bigger pool or virtual threads would only add more waiting threads.

KEY POINTS

Structure your concurrency, scope your context, and test and observe it.

- Structured concurrency keeps subtasks inside a scope, cancels siblings on failure and shows parent-child relations in dumps; `StructuredTaskScope` is still a preview API (JEP 505 and successors), so its API can change.
- `ScopedValue` (final in Java 25) is an immutable, scope-bound replacement for `ThreadLocal`; only structured subtasks inherit it.
- Design first: avoid shared mutable state, guard every access with one named lock, use high-level tools, bound everything, order locks, handle interruption.
- Test by maximising overlap and repeating, with timeouts; a pass is evidence, not proof.
- Debug with repeated thread dumps (`jstack`, `jcmd`), `ThreadMXBean.findDeadlockedThreads`, and JFR.

PART 9

The JVM in Depth

How objects sit in memory, how the garbage collector finds and frees them, and how the JIT turns hot bytecode into fast machine code.

In this part

- **9.1** Objects in Memory
- **9.2** Heap Structure and Reference Types
- **9.3** Garbage Collection Fundamentals
- **9.4** The Collectors
- **9.5** Reading GC Logs and Tuning
- **9.6** Memory Leaks and OutOfMemoryError
- **9.7** The JIT Compiler and Warm-Up

9.1 Objects in Memory

Every `new` allocates bytes on the heap, and those bytes are more than your fields. When a service holds ten million small objects, the difference between 16 and 24 bytes per object is 80 MB, and that decides whether the application fits in its container. This chapter shows exactly what the JVM stores for an object or an array, how to measure it, and how the JIT sometimes avoids allocating at all.

The Part 1 chapter on JVM architecture gave the big picture of the runtime data areas. Here we zoom into one object. Everything below was measured on OpenJDK 21 on a 64-bit Linux machine with default flags; HotSpot (the JVM implementation inside OpenJDK) is the reference.

9.1.1 The anatomy of an object

An ordinary object on HotSpot has three parts: a header, the instance fields, and padding.

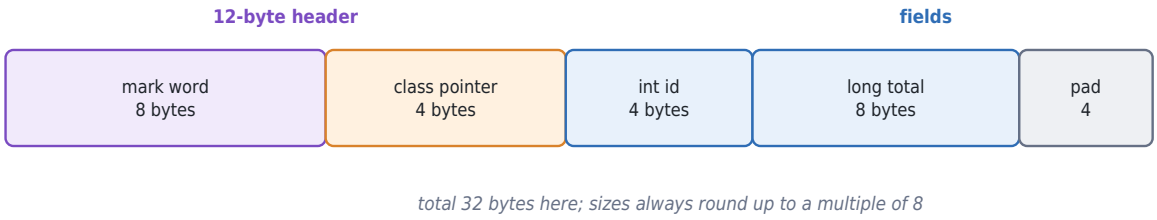


Figure 9.1. Object layout on a 64-bit HotSpot JVM with compressed class pointers: a 12-byte header (mark word plus class pointer), then fields, then padding to a multiple of 8 bytes.

The **header** has two words:

- The **mark word** (8 bytes) holds the identity hash code once it has been requested, the GC age (how many young collections the object survived), and the lock state used by `synchronized`. The same bits are reused for different purposes depending on the object's state.
- The **class pointer** (4 bytes with compressed class pointers, the default) points to the class metadata in Metaspace. It is how the JVM finds the vtable, the field layout and the type of any object.

So the header is 12 bytes. The fields follow, and the total size is rounded up to a multiple of 8 bytes (the object alignment, `-XX:ObjectAlignmentInBytes=8` by default). That rounding is **padding**. An object with no fields is therefore 16 bytes, not 12.

Measuring it: jcmd class histogram

You do not have to guess. The `GC.class_histogram` command of `jcmd` prints instance counts and total bytes per class. The program below allocates a million objects of four shapes, keeps them alive in an array, then asks the JVM about itself.

Histo.java JAVA

```java
import java.io.*;

public class Histo {
    static class Empty {}
    static class OneInt { int a; }
    static class Mixed { byte b; long l; int i; boolean f; }
    static class Ref { Object o; }
    static Object[] keep = new Object[5_000_000];

    public static void main(String[] args) throws Exception {
        int n = 1_000_000, k = 0;
        for (int i = 0; i < n; i++) keep[k++] = new Empty();
        for (int i = 0; i < n; i++) keep[k++] = new OneInt();
        for (int i = 0; i < n; i++) keep[k++] = new Mixed();
        for (int i = 0; i < n; i++) keep[k++] = new Ref();
        long pid = ProcessHandle.current().pid();
        Process p = new ProcessBuilder("jcmd", String.valueOf(pid), "GC.class_histogram")
            .redirectErrorStream(true).start();
        try (BufferedReader r = new BufferedReader(new InputStreamReader(p.getInputStream()))) {
            String line;
            while ((line = r.readLine()) != null)
                if (line.contains("Histo$") || line.contains("#instances"))
                    System.out.println(line);
        }
    }
}
```

Default flags (compressed oops on), then with -XX:-UseCompressedOops TERMINAL

```
$ java -Xmx3g -XX:+UseSerialGC Histo.java
 num     #instances         #bytes  class name (module)
   1:       1000000       32000000  Histo$Mixed
   3:       1000000       16000000  Histo$Empty
   4:       1000000       16000000  Histo$OneInt
   5:       1000000       16000000  Histo$Ref
$ java -Xmx3g -XX:+UseSerialGC -XX:-UseCompressedOops Histo.java
 num     #instances         #bytes  class name (module)
   2:       1000000       32000000  Histo$Mixed
   3:       1000000       24000000  Histo$Ref
   4:       1000000       16000000  Histo$Empty
   5:       1000000       16000000  Histo$OneInt
```

Read the byte column as bytes per object times a million. `Empty` is 16 bytes (12 header + 4 padding). `OneInt` is also 16: 12 header + 4 for the `int` fits exactly, then 0 padding. `Ref` is 16 with compressed references (12 + a 4-byte reference) and 24 without (12 + 4 padding to align the 8-byte reference + 8). `Mixed` has 1 + 8 + 4 + 1 = 14 bytes of fields but costs 32 bytes in total. Why?

Field layout: the JVM reorders your fields

The JVM is free to place fields in any order, and it does. To waste as little as possible it puts the biggest fields first and fills the gap after the 12-byte header with a smaller field. You can see the real offsets through `sun.misc.Unsafe` (an internal class kept for experiments like this one; never use it in production code):

Offsets.java JAVA

```java
import java.lang.reflect.Field;

public class Offsets {
    static class Mixed { byte b; long l; int i; boolean f; }
    static class Child extends Mixed { short s; }
    public static void main(String[] args) throws Exception {
        Field f = Class.forName("sun.misc.Unsafe").getDeclaredField("theUnsafe");
        f.setAccessible(true);
        sun.misc.Unsafe u = (sun.misc.Unsafe) f.get(null);
        for (Class<?> c : new Class<?>[] {Mixed.class, Child.class})
            for (Field x : c.getDeclaredFields())
                System.out.println(c.getSimpleName() + "." + x.getName()
                    + " (" + x.getType() + ") offset " + u.objectFieldOffset(x));
        System.out.println("int[] base offset " + u.arrayBaseOffset(int[].class)
            + ", long[] base " + u.arrayBaseOffset(long[].class)
```

Offsets.java (continued) JAVA

```
            + ", Object[] base " + u.arrayBaseOffset(Object[].class)
            + ", Object[] scale " + u.arrayIndexScale(Object[].class));
    }
}
```

Typical output on JDK 21 (a deprecation warning for Unsafe is also printed to the error stream) TERMINAL

```
$ java Offsets.java
Mixed.b (byte) offset 24
Mixed.l (long) offset 16
Mixed.i (int) offset 12
Mixed.f (boolean) offset 25
Child.s (short) offset 26
int[] base offset 16, long[] base 16, Object[] base 16, Object[] scale 4
```

The declared order was `b, l, i, f`; the layout is `i` at 12 (it fills the 4-byte hole right after the header), `l` at 16 (8-aligned), then `b` at 24 and `f` at 25. The object ends at byte 26, and padding rounds it to 32. Two lessons follow:

- Declaration order does not matter for size; HotSpot packs fields itself. Do not reorder fields by hand to save memory.
- A subclass starts its fields after the superclass's fields. `Child.s` sits at 26, in the tail of `Mixed`. Inheritance never shrinks an object, and a deep hierarchy of classes each holding one `boolean` can still waste bytes.

9.1.2 Compressed oops

An **oop** (ordinary object pointer) is the JVM's name for a reference. On a 64-bit JVM a reference would normally be 8 bytes, which doubles the cost of every pointer-heavy structure such as `ArrayList<Object>`, `HashMap` nodes and linked structures. HotSpot avoids this with **compressed oops**: because objects are 8-byte aligned, the low 3 bits of every object address are zero, so the JVM stores `address >> 3` in 32 bits and shifts back when it dereferences. Thirty-two bits times eight bytes covers 32 GB.

That gives the best-known sizing rule in Java: **keep the heap below 32 GB**. Crossing the line turns compressed oops off and every reference doubles to 8 bytes. A 32 GB heap can hold fewer objects than a 30 GB heap. Check it yourself:

Compressed oops switch off as the maximum heap crosses 32 GB TERMINAL

```
$ for m in 30g 31g 32g 33g; do echo -n "$m: "; \
    java -Xmx$m -XX:+PrintFlagsFinal -version 2>/dev/null \
    | grep UseCompressedOops | awk '{print $4}'; done
30g: true
31g: true
32g: false
33g: false
```

Compressed class pointers are a second, separate switch (`UseCompressedClassPointers`, true by default). In the histogram above, turning off compressed oops left the header at 12 bytes, because the class pointer stayed compressed.

9.1.3 Arrays

An array is an object with one extra header field: the **length**, 4 bytes, stored right after the 12-byte header. Elements start at offset 16 (`arrayBaseOffset` above printed 16 for `int[]`, `long[]` and `Object[]`). Element size is the primitive's size, or the reference size (4 with compressed oops, 8 without; `arrayIndexScale` printed 4 and 8 in our two runs). The total is again rounded up to a multiple of 8.

Array	Calculation	Bytes
`new int[0]`	16 header+length	16
`new int[3]`	16 + 12 = 28, rounded up	32
`new long[10]`	16 + 80	96
`new byte[1000]`	16 + 1000 = 1016	1016
`new Object[100]`	16 + 400 = 416	416

Array sizes with compressed oops (typical HotSpot 64-bit).

Note that an `Object[]` or `Integer[]` stores only references; each element is a separate object elsewhere in the heap. A `List<Integer>` with a million entries is therefore not 4 MB of ints: each `Integer` costs 16 bytes (12 header + 4 value), plus a 4-byte slot in the backing array, so about 20 MB, plus the cost of cache misses when you walk it. This is the main reason numeric code prefers `int[]` (or `IntStream`) over `List<Integer>`. The wrapper-class chapter in Part 3 covers the boxing side.

A multi-dimensional array is an array of arrays. `new int[1000][2]` creates 1001 objects: an outer `Object[]`-like array of 1000 references and 1000 two-element `int[]` arrays of 24 bytes each (16 + 8), with no padding needed. A flat `int[2000]` is one object of 8016 bytes; the nested version is about 28 KB.

9.1.4 Estimating without a tool: Runtime totals

A cheaper but coarser way is `Runtime.totalMemory() - freeMemory()` is the used heap. It is noisy (other allocations, TLABs, concurrent GC threads), so you must allocate many objects, call `System.gc()` before each reading, and divide. Treat it as a sanity check, not a measurement. For serious sizing use the class histogram above, a heap dump, or the JOL library (Java Object Layout, an OpenJDK project that prints exactly the tables we derived by hand). JOL is a separate dependency, so we do not use it here.

9.1.5 Compact object headers (JEP 519, JEP 534)

The 12-byte header is the target of one of the recent footprint projects. **Compact object headers** squeeze the mark word and class pointer into a single 8-byte word. Typical Java heaps are full of small objects (boxed values, strings, list nodes, small records), so 4 bytes saved per object is a real saving. An object with 8 bytes of fields drops from 24 bytes (12 + 8, padded) to 16 (8 + 8).

The status, from the JEPs:

- Java 24 shipped it as an experimental feature; Java 25 made it a product feature, still opt-in with `-XX:+UseCompactObjectHeaders` (JEP 519).
- Java 27 turns it on by default (JEP 534).

We cannot run this on JDK 21 (the flag is rejected: `Unrecognized VM option 'UseCompactObjectHeaders'`), so treat the numbers above as the Java 21 baseline and measure again on your target version. The practical effect is that on Java 25 you can try the flag on a service and compare heap usage and GC time with the class histogram; on Java 27 expect it as the default.

9.1.6 Where do objects live: heap, stack and escape analysis

The language says objects live on the heap. The JIT compiler is allowed to do anything that preserves behaviour, and sometimes it avoids the allocation. The analysis is called **escape analysis**: for each `new`, C2 (the optimizing compiler, covered in the JIT chapter at the end of this Part) asks whether the reference can leave the method (stored in a field, returned, passed to code that is not inlined). If it cannot, the object is not needed as an object. The compiler then performs **scalar replacement**: the fields become local variables (often just registers) and no memory is allocated at all. Note that HotSpot does not allocate such objects "on the stack" as C++ would; it deletes them.

Escape.java JAVA

```java
import java.lang.management.*;

public class Escape {
    record Point(int x, int y) {}

    static int sumDistance(int a, int b) {
        Point p = new Point(a, b);          // never leaves this method
        return p.x() * p.x() + p.y() * p.y();
    }

    public static void main(String[] args) {
        long total = 0;
        for (int i = 0; i < 300_000_000; i++) total += sumDistance(i & 1023, 3);
        long gcs = 0;
        for (GarbageCollectorMXBean b : ManagementFactory.getGarbageCollectorMXBeans())
            gcs += b.getCollectionCount();
        System.out.println("total=" + total + " gcCount=" + gcs);
    }
}
```

Typical output (counts vary a little between runs) TERMINAL

```
$ java -Xmx256m -XX:+UseSerialGC -XX:+DoEscapeAnalysis Escape.java
total=104706632657792 gcCount=41
$ java -Xmx256m -XX:+UseSerialGC -XX:-DoEscapeAnalysis Escape.java
total=104706632657792 gcCount=274
```

Three hundred million `Point` objects would be 4.8 GB of garbage. With escape analysis off the JVM ran 274 young collections. With it on, it ran 41; those remaining collections come from the first iterations, which run in the interpreter and the quick C1 compiler before C2 takes over (the same loop reaches C2 by on-stack replacement, explained in the JIT chapter). The answer is identical in both runs; only the cost differs.

GOTCHA

Escape analysis is an optimization, not a guarantee. It fails when the object is stored anywhere shared, when a method on it is too big to inline, or when control flow merges two possible objects. You cannot write code that "requires" scalar replacement, and you cannot see it in the source. If allocation matters, prove it with a GC log or a profiler (JFR allocation events) instead of assuming.

INTERVIEW ANGLE

How much memory does `new Object()` take, and why? 16 bytes on a 64-bit HotSpot JVM with default flags: a 12-byte header (8-byte mark word plus 4-byte compressed class pointer) padded to the 8-byte alignment. With compact object headers (Java 25 opt-in, default in 27) the header is 8 bytes and the object is 8 bytes. Follow-up answer: `Integer` is 16 bytes (12 + 4); `long[0]` is 16 (12 + 4 length); a `String` is a 24-byte object pointing to a separate `byte[]`.

REAL WORLD

In a Spring Boot service the heap is dominated by small objects: `String` and their `byte[]`, `HashMap$Node` (32 bytes each: header 12 + hash 4 + key 4 + value 4 + next 4 = 28, padded to 32), boxed numbers and DTO records. That is why teams measure with a heap histogram before choosing a container memory limit, and why the 32 GB compressed-oops boundary is a standard sizing rule.

Check yourself

1. How many bytes does `class P { int x; int y; }` occupy on a default 64-bit HotSpot (compressed oops)? What about `class Q { long a; }`?
2. Why is a `new int[3]` 32 bytes and not 28?
3. You raise `-Xmx` from 31g to 33g to "get more room" and the service holds fewer objects. What happened?
4. What is wrong with the claim "escape analysis puts objects on the stack"?
5. `List<Integer>` with 1,000,000 distinct values versus `int[1_000_000]`: roughly how many bytes each on a default JVM?

Answers

1. `P` is 12 + 8 = 20, padded to 24. `Q` is 12 + 4 (gap, filled by padding because a `long` must be 8-aligned) + 8 = 24.
2. Header 12 + length 4 = 16, plus 12 bytes of data = 28, rounded up to the next multiple of 8, which is 32.
3. Above about 32 GB compressed oops are disabled, so every reference grows from 4 to 8 bytes and object headers/arrays of references get larger.
4. HotSpot does not allocate on the stack. It proves the object does not escape and scalar-replaces it: fields become locals or registers and the allocation vanishes.
5. About 20 MB (16-byte `Integer` objects plus 4-byte array slots; values above 127 are not cached) versus about 4 MB (16 + 4,000,000 bytes).

KEY POINTS

An object is a 12-byte header (mark word + compressed class pointer), fields, and padding to a multiple of 8 bytes.

- An empty object is 16 bytes; HotSpot reorders fields to pack them; arrays add a 4-byte length and start data at offset 16.
- Compressed oops keep references at 4 bytes up to a heap just under 32 GB; beyond that every reference doubles.
- Measure with `jcmd <pid> GC.class_histogram` or a heap dump, not with guesses; `Runtime` totals are only a rough check.
- Compact object headers (8-byte header) are opt-in in Java 25 and default in Java 27; escape analysis can remove allocations entirely but is never guaranteed.

9.2 Heap Structure and Reference Types

"The heap" is not one blob. It is divided into regions with different jobs, and the JVM also uses memory outside the heap that no `-Xmx` limits. If you only know `-Xmx`, you cannot explain why a container with a 512 MB heap gets killed at 700 MB, or why a cache "leaks" even though you never hold a strong reference. This chapter maps the memory the JVM uses, and then covers the four reference strengths that let your code cooperate with the garbage collector.

9.2.1 The generational heap

Measurements of real programs show the **weak generational hypothesis**: most objects die young (request-scoped data, iterators, temporary strings), and the few that survive tend to live a long time (caches, connection pools, configuration). Collectors exploit this by splitting the heap by age so that the frequent cheap collection touches only the young part.

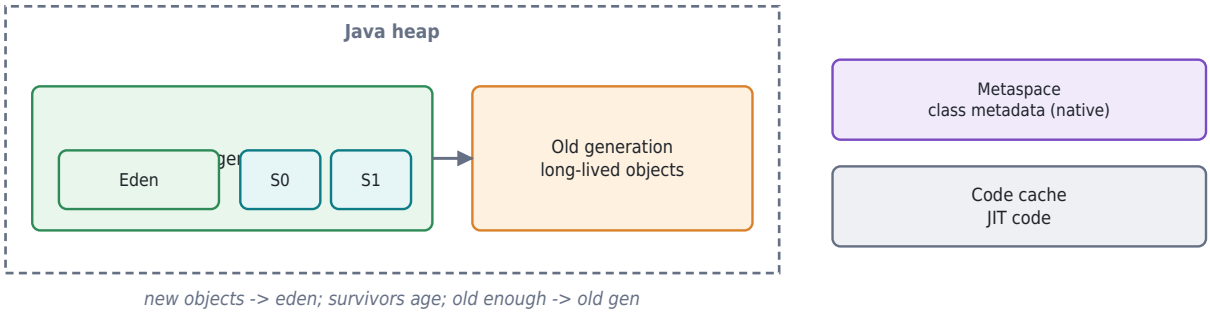


Figure 9.2. The generational heap: young generation (eden plus two survivor spaces) and old generation, with the path an object takes as it ages; Metaspace and the code cache live outside the heap.

- **Eden**: where new objects are allocated. When it fills, a **young (minor) collection** runs.
- **Survivor spaces** (S0 and S1, or "from" and "to"): live objects from eden are copied here. At each young collection the survivors of the previous round are copied again into the other survivor space, and their **age** (stored in the header's mark word) goes up by one.
- **Old generation (tenured)**: objects that reach the **tenuring threshold** (`-XX:MaxTenuringThreshold`, 15 by default, the maximum the header's 4-bit age field can hold) or do not fit in the survivor space are promoted here. The old generation is collected less often, by a more expensive collection.
- **Humongous objects** (G1 only): objects of at least half a region are allocated directly in special old-generation regions. The chapter on the collectors returns to this.

The defaults you can read from `-XX:+PrintFlagsFinal` on this machine: `NewRatio=2` (old is twice the size of young for the classic collectors), `SurvivorRatio=8` (eden is 8 times one survivor space), `MaxTenuringThreshold=15`. G1 does not use fixed boundaries; it resizes young dynamically within its pause goal, so these flags matter mostly for Serial and Parallel.

9.2.2 TLABs: why allocation is fast

Allocating in a shared eden would need a lock or a CAS per `new`. Instead each thread gets a private slice of eden, a **thread-local allocation buffer (TLAB)**. Allocation inside the TLAB is a pointer bump: add the object size to the pointer, check against the limit. No synchronization is needed, which is why `new` in Java can be as fast as a few instructions. When a TLAB is full the thread asks eden for a new one (this is the synchronized slow path); an object too large for the remaining TLAB space may be allocated directly in eden. TLABs are on by default (`UseTLAB`), and `ResizeTLAB` lets the JVM adapt each thread's TLAB size to its allocation rate. You rarely tune them; know the mechanism, because it explains why allocation is cheap and why the cost of an object is paid later, at collection time.

9.2.3 Memory the JVM uses outside the heap

Look at what the JVM itself reports. This program lists every memory pool and the buffer pools:

Pools.java JAVA

```java
import java.lang.management.*;
import java.nio.ByteBuffer;

public class Pools {
    public static void main(String[] args) {
        ByteBuffer direct = ByteBuffer.allocateDirect(8 * 1024 * 1024);
        for (MemoryPoolMXBean p : ManagementFactory.getMemoryPoolMXBeans()) {
            MemoryUsage u = p.getUsage();
            System.out.printf("%-30s %-14s used=%6d KB max=%s%n", p.getName(), p.getType(),
                u.getUsed() / 1024, u.getMax() < 0 ? "undefined" : (u.getMax() / 1024) + " KB");
        }
        for (BufferPoolMXBean b : ManagementFactory.getPlatformMXBeans(BufferPoolMXBean.class))
            System.out.printf("buffer pool %-8s count=%d used=%d KB%n",
                b.getName(), b.getCount(), b.getMemoryUsed() / 1024);
        System.out.println(direct.capacity());
    }
}
```

Typical output (JDK 21, default G1, -Xmx512m); sizes vary by run TERMINAL

```
$ java -Xmx512m Pools.java
CodeHeap 'non-nmethods'        Non-heap memory used=  1437 KB max=5692 KB
Metaspace                      Non-heap memory used= 10459 KB max=undefined
CodeHeap 'profiled nmethods'   Non-heap memory used=  2413 KB max=120032 KB
Compressed Class Space         Non-heap memory used=  1256 KB max=1048576 KB
G1 Eden Space                  Heap memory    used=  7168 KB max=undefined
G1 Old Gen                     Heap memory    used=  3634 KB max=524288 KB
G1 Survivor Space              Heap memory    used=  2048 KB max=undefined
CodeHeap 'non-profiled nmethods' Non-heap memory used=   496 KB max=120036 KB
buffer pool mapped   count=0 used=0 KB
buffer pool direct   count=2 used=8200 KB
buffer pool mapped - 'non-volatile memory' count=0 used=0 KB
8388608
```

What to notice:

- **Heap pools** carry the collector's name (`G1 Eden Space`; with `-XX:+UseSerialGC` they are `Eden Space`, `Survivor Space`, `Tenured Gen`). The maximum of the old pool is the `-Xmx` value, 512 MB here.
- **Metaspace** holds class metadata (the runtime representation of every loaded class: methods, constant pools, field layouts). It lives in native memory, not the heap, and since Java 8 replaced PermGen it grows on demand. Its limit is `-XX:MaxMetaspaceSize`, which is unlimited by default (`max=undefined`), so a classloader leak shows up as native memory growth. **Compressed Class Space** is the part of it holding the class structures that the 4-byte class pointers address; its default reservation is 1 GB.
- **Code cache** (the `CodeHeap` pools) stores JIT-compiled machine code. It is segmented into non-method code, profiled code (C1 with profiling) and fully optimized code. The default reserved size here is about 240 MB (`ReservedCodeCacheSize`); if it fills up the JIT stops compiling and performance collapses with a log message.
- **Direct buffers** from `ByteBuffer.allocateDirect` are native memory outside the heap. Our 8 MB buffer shows in the `direct` pool (the count of 2 and the extra few KB come from a small buffer the JDK allocated itself). The ceiling is `-XX:MaxDirectMemorySize`, which defaults to the maximum heap size. NIO networking, Netty and many databases drivers use them heavily.

Not in the list: thread stacks (each platform thread reserves its stack, typically 1 MB for Java threads on 64-bit Linux, controlled with `-Xss`), the JVM's own C++ data structures, GC bookkeeping (for example remembered sets), and memory used by native libraries. This is the reason **process memory is always larger than** `-Xmx`.

WATCH OUT

Setting `-Xmx` equal to the container memory limit gets the process killed by the operating system's out-of-memory killer. The heap is only one slice. As a starting point, leave 25 to 50 percent of the limit for Metaspace, code cache, thread stacks, direct buffers and GC structures, then watch the real figure with Native Memory Tracking (`-XX:NativeMemoryTracking=summary` and `jcmd <pid> VM.native_memory`). Using `-XX:MaxRAMPercentage` instead of a fixed `-Xmx` is covered in the tuning chapter.

9.2.4 Reference types

An ordinary variable holds a **strong reference**: while it can be reached from a GC root, the object is kept. The package `java.lang.ref` adds three weaker kinds. A **reachability** level is the strongest path from a root to the object, and the GC decides what to clear based on it. All four levels, from strongest to weakest:

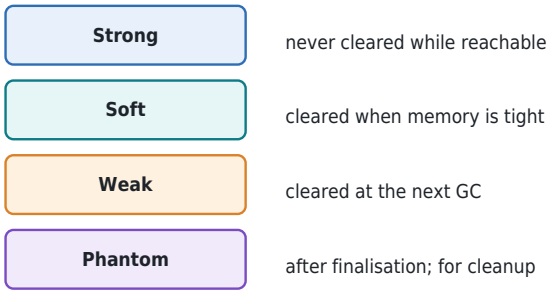


Figure 9.3. The four reference strengths and when the collector clears each; the reference queue notifies you after clearing.

Type	Cleared when	Typical use
strong (`Object o = ...`)	Never, while reachable	All normal code
`SoftReference`	When memory is short (before an `OutOfMemoryError`)	Memory-sensitive caches
`WeakReference`	At the next GC after only weak paths remain	Canonicalizing maps, metadata keyed by objects (`WeakHashMap`)
`PhantomReference`	After the object is finalizable; `get()` always returns `null`	Cleanup actions after death (`Cleaner`)

Reference types.

The next program shows all of them. Run it with a small heap so the behaviour is predictable.

Refs.java JAVA

```
import java.lang.ref.*;
import java.util.*;

public class Refs {
    static void gc() throws InterruptedException {
        System.gc();
        Thread.sleep(200);
    }

    public static void main(String[] args) throws Exception {
        Object strong = new Object();
        WeakReference<Object> weak = new WeakReference<>(new byte[1024]);
        SoftReference<Object> soft = new SoftReference<>(new byte[1024]);
        ReferenceQueue<Object> queue = new ReferenceQueue<>();
        Object victim = new Object();
        PhantomReference<Object> phantom = new PhantomReference<>(victim, queue);

        System.out.println("before gc: weak=" + (weak.get() != null)
            + " soft=" + (soft.get() != null) + " phantom.get()=" + phantom.get());
        victim = null;
        gc();
        System.out.println("after gc : weak=" + (weak.get() != null)
            + " soft=" + (soft.get() != null) + " strong=" + (strong != null));
        System.out.println("phantom enqueued: " + (queue.remove(1000) == phantom));

        WeakHashMap<Object, String> cache = new WeakHashMap<>();
        Object key = new Object();
        cache.put(key, "metadata");
        cache.put(new Object(), "orphan");
        gc();
        System.out.println("WeakHashMap size after gc: " + cache.size());
        key = null;
        gc();
        System.out.println("after dropping last key : " + cache.size());
    }
}
```

Output on JDK 21 TERMINAL

```
$ java -Xmx64m Refs.java
before gc: weak=true soft=true phantom.get()=null
after gc : weak=false soft=true strong=true
phantom enqueued: true
WeakHashMap size after gc: 1
after dropping last key : 0
```

The weak target was cleared by the very next collection even though memory was plentiful; the soft target survived because there was no memory pressure. `phantom.get()` is `null` even before the object dies, because a phantom reference never gives the object back; it only tells you, through the queue, that the object is gone. The `WeakHashMap` entry whose key we kept stayed, the orphan entry disappeared.

Soft references under pressure

The JVM clears soft references when the heap runs low, and, before that, by a policy: a soft reference not used for roughly `SoftRefLRUPolicyMSPerMB` (1000 ms by default) times the free megabytes in the heap is eligible. The guarantee is only that all soft references are cleared before the JVM throws `OutOfMemoryError`.

SoftDemo.java JAVA

```java
import java.lang.ref.SoftReference;
import java.util.*;

public class SoftDemo {
    public static void main(String[] args) {
        List<SoftReference<byte[]>> cache = new ArrayList<>();
        for (int i = 0; i < 100; i++) cache.add(new SoftReference<>(new byte[1024 * 1024]));
        long alive = cache.stream().filter(r -> r.get() != null).count();
        System.out.println("100 MB via soft refs, -Xmx32m; still alive: " + alive);
    }
}
```

Typical output (the exact number depends on the collector and heap) TERMINAL

```
$ java -Xmx32m SoftDemo.java
100 MB via soft refs, -Xmx32m; still alive: 4
```

The program never died: the collector reclaimed about 90 of the 100 soft megabytes (our two runs kept 10 and 4) to make room instead of throwing. That is the behaviour a cache wants. It is also the weakness: the collector, not your code, decides what stays, and clearing happens in bursts that cost GC time. In practice most production code uses a bounded cache with an explicit eviction policy (the Caffeine library in Spring applications) instead of `SoftReference`.

WeakHashMap and its trap

`WeakHashMap` holds its **keys** weakly. When no one else references a key, the entry disappears at some later point (stale entries are removed lazily on the next access). It is meant for attaching extra data to objects you do not own. The trap: if the **value** strongly references its own key (directly or through a chain), the key is never weakly-only reachable and the entry is never removed.

9.2.5 Cleaner: replacing finalize

Before Java 9 people closed native resources in `finalize()`. Finalization is deprecated for removal (since Java 18): it runs at an unpredictable time, on one thread, can resurrect objects and slows down GC. The replacement is `java.lang.ref.Cleaner`, built on phantom references. You register an object together with a **cleaning action**, and the action runs once, either when you call `clean()` explicitly or after the GC finds the object unreachable.

CleanerDemo.java JAVA

```java
import java.lang.ref.Cleaner;

public class CleanerDemo {
    static final Cleaner CLEANER = Cleaner.create();

    static class NativeHandle implements AutoCloseable {
        // The cleaning action must NOT reference the NativeHandle itself.
        private static class State implements Runnable {
            final String name;
            State(String name) { this.name = name; }
            public void run() { System.out.println("cleaned " + name); }
```

CleanerDemo.java (continued) JAVA

```
        }
        private final Cleaner.Cleanable cleanable;
        NativeHandle(String name) { cleanable = CLEANER.register(this, new State(name)); }
        @Override public void close() { cleanable.clean(); }
    }

    public static void main(String[] args) throws Exception {
        try (NativeHandle h = new NativeHandle("explicit")) {
            System.out.println("using " + h.getClass().getSimpleName());
        }
        new NativeHandle("forgotten");
        System.gc();
        Thread.sleep(300);
        System.out.println("done");
    }
}
```

Typical output: the second "cleaned" line depends on the GC running in time TERMINAL

```
$ java CleanerDemo.java
using NativeHandle
cleaned explicit
cleaned forgotten
done
```

What to notice: the cleaning action is a separate static nested class holding only the data it needs (here a name; in real code a native pointer or file descriptor). If it were a lambda or inner class capturing `this`, the handle would stay strongly reachable from the cleaner and would never be collected. And the action runs once: calling `close()` first made the later GC a no-op. The rule is the same as for `finalize`, only safer: **Cleaner is a safety net, not the primary mechanism**. Provide `AutoCloseable` and make users close in try-with-resources.

INTERVIEW ANGLE

What are the four reference types in Java and where have you seen them used? Strong (default), soft (cleared before OOM, used for caches), weak (cleared at the next GC when only weakly reachable, used by `WeakHashMap` and for listener registries), phantom (`get()` always null, enqueued after the object is dead, the basis of `Cleaner` and direct-buffer cleanup). Add that `ThreadLocal` keys are weak references, which is part of why `ThreadLocal` leaks happen on thread pools (covered in the leaks chapter).

Check yourself

1. In which generation is a freshly allocated `new ArrayList<>()` placed, and what makes that allocation cheap?
2. Name three kinds of JVM memory that are not limited by `-Xmx`.
3. What does this print, and why? `var r = new WeakReference<>(new Object()); System.gc(); System.out.println(r.get());`
4. A `WeakHashMap<Key, Value>` never shrinks although you drop all other references to the keys. What is the likely cause?
5. Why must a `Cleaner` action not be a lambda that uses `this`?
6. What guarantee do soft references give, and what do they not guarantee?

Answers

1. In eden (unless it is a huge object that goes straight to old). The allocation is a pointer bump inside the thread's TLAB with no locking.
2. Any of: Metaspace, code cache, direct buffers, thread stacks, GC bookkeeping, native library memory.
3. Almost always `null`: the object is only weakly reachable, so the collector clears it. The snippet is not guaranteed because `System.gc()` is only a hint, but with default HotSpot settings it does run a full collection.
4. A value (or something it reaches) holds a strong reference to its own key, so the key is never weakly reachable.
5. The lambda captures `this`, which keeps the object strongly reachable from the Cleaner's registry, so it never becomes unreachable and the action never runs by GC.
6. All soft references are cleared before the JVM throws `OutOfMemoryError`. There is no guarantee when, or in which order, they are cleared before that point, nor that they stay while memory is fine.

KEY POINTS

The heap is split by age: eden, two survivor spaces, old; the young collection is cheap because most objects are dead, and TLABs make allocation a pointer bump.

- Process memory is more than the heap: Metaspace, code cache, direct buffers, thread stacks and GC structures all count against a container limit.
- Strong, soft, weak and phantom references give the collector different permission to clear; weak clears at the next GC, soft under memory pressure, phantom signals via a queue.
- `WeakHashMap` entries vanish when the key is unreachable elsewhere, but a value that references its key keeps the entry alive.
- Use `Cleaner` (not `finalize`) as a safety net behind `AutoCloseable`, with a cleaning action that does not capture the object.

9.3 Garbage Collection Fundamentals

Java has no `free`. The JVM decides when an object is no longer needed and reclaims its memory. That removes whole classes of bugs (dangling pointers, double free), but it moves a cost into the runtime: the collector competes with your code for CPU and sometimes stops your code altogether. To tune a Java service, and to answer interview questions about GC, you need a precise model of what the collector does, rather than the slogan "it removes unused objects". This chapter builds that model; the next one applies it to the real collectors.

9.3.1 What counts as garbage: reachability

The JVM does not track "unused". It tracks **reachability**. An object is **live** if some chain of references leads to it from a **GC root**; everything else is garbage, even if it still points to other objects. The roots are the references the runtime itself holds:

- local variables and parameters in the frames of every live thread's stack (including values in registers and the operand stack of running frames);
- static fields of loaded classes (the classes themselves are held by their class loader);
- live threads (a running `Thread` object is a root until it terminates);
- JNI references held by native code;
- internal JVM structures: the string table, classes in use, synchronization monitors being held, and so on.



Two consequences matter in practice. First, **cycles are not a problem**: D and E point at each other, yet both are garbage because no root reaches them. This is the difference from reference counting (used by CPython and Swift, for example), which cannot free a cycle by counting alone. Second, **a leak in Java means unwanted reachability**: an object you no longer need is still reachable from a root (a static `Map`, a listener list, a `ThreadLocal`), so the collector must keep it. That is the topic of the memory-leak chapter.

You can prove the first point on the real JVM. A weak reference observes whether the cycle was collected:

CycleReal.java JAVA

```java
import java.lang.ref.WeakReference;

public class CycleReal {
    static class Node { Node other; byte[] pad = new byte[1000]; }

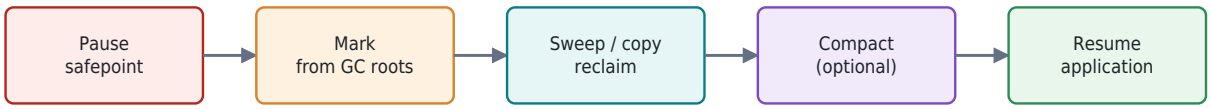
    public static void main(String[] args) throws Exception {
        Node x = new Node(), y = new Node();
        x.other = y; y.other = x;               // reference cycle
        WeakReference<Node> probe = new WeakReference<>(x);
        x = null; y = null;
        System.gc();
```

CycleReal.java (continued) JAVA

```
        Thread.sleep(100);
        System.out.println("cycle collected: " + (probe.get() == null));
    }
}
```

Output TERMINAL

```
$ java CycleReal.java
cycle collected: true
```



concurrent collectors do most marking while the application keeps running

Figure 9.4. One GC cycle: stop (or not) the application, mark from roots, reclaim or copy, optionally compact, resume; the cost of each phase depends on live data or on heap size.

9.3.2 Mark-sweep-compact

The oldest and simplest tracing algorithm has two phases (plus one optional one):

1. **Mark**: start at the roots, follow every reference, and set a mark on each object reached. (Real collectors keep mark bits in a side bitmap or in the header, and use an explicit work queue instead of recursion so deep structures do not overflow the stack.)
2. **Sweep**: walk the whole heap and reclaim every unmarked object, adding its space to a free list.
3. **Compact** (optional): slide the live objects together to remove the holes left behind.

This toy collector runs the same algorithm over a small graph, using exactly the picture above:

ToyGc.java JAVA

```
import java.util.*;

public class ToyGc {
    static class Node {
        final String name;
        final List<Node> refs = new ArrayList<>();
        boolean marked;
        Node(String name) { this.name = name; }
    }

    static void mark(Node n) {
        if (n == null || n.marked) return;      // the marked flag also stops cycles
        n.marked = true;
        for (Node r : n.refs) mark(r);
    }

    public static void main(String[] args) {
        Node a = new Node("A"), b = new Node("B"), c = new Node("C");
        Node d = new Node("D"), e = new Node("E"), f = new Node("F");
        a.refs.add(b);  b.refs.add(c);          // root -> A -> B -> C
        d.refs.add(e);  e.refs.add(d);          // D <-> E : a cycle nobody points to
        f.refs.add(c);                          // F -> C, but nobody points to F
        List<Node> heap = List.of(a, b, c, d, e, f);
        List<Node> roots = List.of(a);          // a local variable holds A

        for (Node r : roots) mark(r);           // phase 1: mark everything reachable
        List<Node> survivors = new ArrayList<>();
        for (Node n : heap) {                   // phase 2: sweep the rest
            if (n.marked) survivors.add(n);
            else System.out.println("sweep " + n.name);
        }
        System.out.println("survivors: " + survivors.stream().map(n -> n.name).toList());
    }
}
```

Output TERMINAL

```
$ java ToyGc.java
sweep D
sweep E
sweep F
survivors: [A, B, C]
```

Note that the work in the mark phase is proportional to the number of **live** objects, while the sweep phase is proportional to the heap size. Garbage is never visited by the marker. This is why tracing GC cost depends on how much survives, not on how much you allocated.

Plain mark-sweep leaves **fragmentation**: free space exists but in holes too small for a big array, and allocation has to search a free list instead of bumping a pointer. **Compaction** fixes both, at the price of moving objects, which means every reference to them must be updated. That is cheap if the world is stopped and expensive if the application is running (the next chapter shows how ZGC does it concurrently using load barriers).

9.3.3 Copying collection

Instead of marking and then sweeping in place, a **copying collector** divides a space into two halves. It allocates in one half ("from-space"), and when that fills it copies every live object into the empty other half ("to-space"), then declares the whole from-space free in one step. After copying, the survivors sit contiguously, so there is no fragmentation and allocation is again a pointer bump.

The cost is proportional to the number of survivors only: dead objects cost nothing, because they are never touched. The price is wasted space (you reserve room to copy into). Copying is therefore perfect when most objects are dead, which is exactly what the generational hypothesis says about young objects. That is why the young generation uses eden plus two small survivor spaces and a copying collector, while the old generation, where most objects survive, uses mark-compact or a region-based variant.

9.3.4 The generational hypothesis in practice

Putting it together, a **young collection** (also called minor GC):

- marks from the roots, **plus** from the old objects that point into young ones. Searching the whole old generation for such pointers would defeat the purpose, so the JVM keeps a **card table**: a write barrier marks a small region (a "card") of the heap as dirty whenever a reference is stored into an old object. At a young collection only dirty cards are scanned;
- copies live eden and survivor objects into the other survivor space, or promotes them to old if old enough;
- frees eden and the old survivor space wholesale.

The cost of a young collection scales with the live data in young and the number of dirty cards, not with the heap size. A **full GC** (or major collection) examines the whole heap and is the expensive event you want to avoid. When the old generation fills up, or a promotion fails, you get one. Even so, on modern JVMs the line between "young" and "full" is blurry for concurrent collectors (G1 adds "mixed" collections that clean part of the old generation), as the next chapter shows.

9.3.5 Stop-the-world and safepoints

Moving an object is only safe if no thread is looking at it at that moment. Most collectors therefore stop all application threads for at least part of each cycle; this is a **stop-the-world (STW) pause**. The JVM does not interrupt a thread at an arbitrary instruction. A thread has to reach a **safepoint**: a point where its state (where its references are on the stack and in registers) is exactly known to the JVM. JIT-compiled code contains safepoint polls at method returns and loop back-edges. When a collection is requested, the JVM raises a flag, each thread notices it at the next poll, parks, and the collector runs once all threads have stopped.

You can see this in the unified logging system. This run used a 64 MB heap, the Serial collector and `-Xlog:gc,safepoint`; the interesting lines are trimmed:

Typical output, slow shared machine (pause times are inflated) TERMINAL

```
$ java -Xmx64m -XX:+UseSerialGC -Xlog:gc,safepoint Safepoint.java
[6.402s][info][gc] GC(0) Pause Young (Allocation Failure) 18M->4M(61M) 175.498ms
[6.406s][info][safepoint] Safepoint "GenCollectForAllocation", Reaching safepoint: 3649 ns,
    Cleanup: 34419 ns, At safepoint: 179615774 ns, ...
[8.660s][info][gc] GC(4) Pause Full (Allocation Failure) 45M->24M(61M) 132.744ms
```

Read the safepoint line from left to right: "Reaching safepoint" is how long it took to get all threads to stop (here microseconds), "At safepoint" is the time the VM operation, the GC, actually took. In a healthy system "reaching" is tiny. A long reaching time means one thread was in a long loop without a safepoint poll (classically a counted `int` loop with no calls, which older JITs left unpolled), and everyone else waited for it. Also, other VM operations besides GC use safepoints: biased-lock revocation in old versions, deoptimization, heap dumps, `jstack` thread dumps in some cases. If your pauses are not GC pauses, `-Xlog:safepoint` is where to look.

GOTCHA
`System.gc()` is a request for a full collection, not a command. HotSpot honours it by default (and runs it as a stop-the-world full GC) unless `-XX:+DisableExplicitGC` is set. Libraries sometimes call it behind your back; direct `ByteBuffer` cleanup and RMI's distributed GC are the classic examples. If you see periodic `Pause Full (System.gc())` lines in a GC log, search for the caller instead of tuning the collector.

9.3.6 The three goals: throughput, latency, footprint

No collector can be best at everything. Three measures pull against each other:

Goal	What it means	How you lose it
Throughput	Percentage of total time spent running your code rather than GC	Small heaps and frequent collections; barrier overhead in concurrent collectors
Latency	Length of the longest pause (matters for request tail latency, p99)	A big stop-the-world collection of a large live set
Footprint	Memory the JVM needs beyond your live data	Headroom for copying, region bookkeeping, concurrent collectors that need slack to keep up

The GC trade-off triangle.

A batch job that only cares about finishing sooner wants throughput; a trading or interactive system wants low latency and will pay with memory and CPU; a small container wants footprint. More heap generally buys both throughput and lower pause frequency, but costs memory. A concurrent collector works alongside your threads and so keeps pauses short, but spends CPU on barriers and background threads. The collectors in the next chapter are different points on this triangle. A useful working rule is: choose the goal first, then the collector, and set the heap before touching any other flag.

Two small numbers help you reason. **Allocation rate** (MB per second) determines how often young collections happen: a 600 MB eden with a 1.2 GB/s allocation rate is collected twice a second. **Survival rate** determines how long each one takes. Reducing allocation rate (fewer temporary objects) is usually the cheapest GC tuning there is.

INTERVIEW ANGLE
How does the JVM decide an object is eligible for garbage collection, and can a reference cycle leak memory? An object is eligible when it is not reachable from any GC root (thread stack variables, static fields, JNI references, live threads). Reachability is computed by tracing from the roots, so a cycle with no path from a root is collected; Java does not use reference counting. A "leak" in Java is an object that is still reachable but no longer needed.

Check yourself

1. Which of these are GC roots: a local variable in `main`, a field of an object in the old generation, a `static` field, a running thread?
2. What does this print, and what does it show? `Object a = new Object(); WeakReference<Object> w = new WeakReference<>(a); a = null; System.gc(); System.out.println(w.get());`
3. Why is the cost of a copying collection proportional to live data, not to garbage?
4. What is the card table for, and what would a young collection need to do without it?
5. Explain "Reaching safepoint" versus "At safepoint" in a `-Xlog:safepoint` line.
6. Which of throughput, latency and footprint does a bigger heap usually improve, and which does it cost?

Answers

1. A local variable in `main`, a static field and a running thread are roots. A field of an old object is not a root itself; it is only reachable if the old object is.
2. It prints `null` in the typical case: after `a = null` the object has only a weak path, so the full collection triggered by `System.gc()` clears it. It shows that reachability, not variable scope, decides.

3. The collector copies only what it reaches from the roots; dead objects are never visited, and the whole from-space is released at once.
4. It records which old-generation regions had a reference written into them. Without it, each young collection would have to scan the entire old generation to find old-to-young pointers.
5. "Reaching safepoint" is the time to bring all threads to a stop; "At safepoint" is the time the operation (such as the GC) took while they were stopped.
6. It usually improves throughput and the frequency of pauses, at the cost of footprint (memory); a very large heap can also increase the length of a single full collection for non-concurrent collectors.

KEY POINTS

The GC finds live objects by tracing from roots (stack variables, statics, live threads, JNI); unreachable objects, including cycles, are garbage.

- Mark-sweep costs time proportional to live objects plus heap size and fragments memory; compaction and copying fix fragmentation by moving objects.
- Copying costs only the survivors, which is why the young generation uses it; a card table lets young collections skip the old generation.
- Stop-the-world pauses need all threads at safepoints; `-Xlog:safepoint` shows time to reach it versus time spent in the operation.
- Throughput, latency and footprint trade against each other; reducing allocation rate is the cheapest tuning.

9.4 The Collectors

HotSpot ships several garbage collectors because no single design wins on every axis from the previous chapter. You choose one with a command-line flag, and the choice changes pause times, CPU use and memory overhead more than any other flag. This chapter describes each collector by how it works, what it is good at, and when to pick it. All measurements are from OpenJDK 21 on a 2-CPU build machine; the log lines are real, but absolute pause times on a small shared machine are not representative of production hardware.

9.4.1 The lineup at a glance

Collector	Flag	Strength	Weakness
Serial	`-XX:+UseSerialGC`	Tiny footprint, no GC threads	Single-threaded, stop-the-world for everything
Parallel	`-XX:+UseParallelGC`	Highest raw throughput for batch work	Pauses grow with the live set
G1	`-XX:+UseG1GC`	Balanced; pause-time goal; the default	Barrier and bookkeeping overhead; pauses still scale with the live young set
ZGC	`-XX:+UseZGC`	Pauses of well under a millisecond, independent of heap size	More CPU and memory overhead; needs headroom
Shenandoah	`-XX:+UseShenandoahGC`	Low pauses, concurrent compaction	Not in every vendor's build (check yours)
Epsilon	`-XX:+UseEpsilonGC`	Allocates, never frees	The JVM exits when the heap is full

The HotSpot collectors (Java 21 flags; Java 25 availability noted in the text).

Confirm which one you are on in code or by logging:

WhichGc.java JAVA

```java
import java.lang.management.*;

public class WhichGc {
    public static void main(String[] args) {
        Runtime rt = Runtime.getRuntime();
        System.out.println("cpus=" + rt.availableProcessors()
            + " maxHeapMB=" + rt.maxMemory() / (1024 * 1024));
        for (GarbageCollectorMXBean b : ManagementFactory.getGarbageCollectorMXBeans())
            System.out.println("  collector: " + b.getName());
    }
}
```

Output on the build machine (2 CPUs, 6 GB RAM) TERMINAL

```
$ java WhichGc.java
cpus=2 maxHeapMB=1496
  collector: G1 Young Generation
  collector: G1 Concurrent GC
  collector: G1 Old Generation
$ java -XX:+UseSerialGC WhichGc.java
cpus=2 maxHeapMB=1446
  collector: Copy
  collector: MarkSweepCompact
$ java -XX:+UseParallelGC WhichGc.java
cpus=2 maxHeapMB=1330
  collector: PS MarkSweep
  collector: PS Scavenge
```

The first line is the **default** here, which brings us to the question of which GC you get without asking.

9.4.2 Which collector is the default?

- Java 9 to 26: **G1** on a "server-class" machine, which means at least 2 available CPUs and at least about 1.8 GB of memory (1792 MB); otherwise **Serial**. A small container with one CPU or a low memory limit therefore silently gets Serial. This catches people out: the same jar behaves differently in a 1-CPU Kubernetes pod and on a laptop.
- Java 27 (JEP 523): **G1 everywhere**, regardless of CPU count or memory. Explicit flags still override.
- The JVM looks at the container's limits when counting CPUs and memory (container awareness), not the host's.

`java -XX:+PrintFlagsFinal -version | grep Use.*GC` shows which collectors are enabled on this machine (`UseG1GC = true` above in our flag listing).

9.4.3 Serial and Parallel

Both are **generational, stop-the-world, copying-young / mark-compact-old** collectors, exactly the model from the previous chapter. **Serial** does everything on one thread. It has the lowest overhead and memory use, and is the right choice for small heaps (up to a few hundred MB), single-CPU containers, command-line tools and short-lived programs. **Parallel** (the Java 8 default, also called the throughput collector) runs the same algorithms on several GC threads (`-XX:ParallelGCThreads`). For a batch job where no one cares about an occasional two-second pause, Parallel often completes the job fastest because it has no concurrent bookkeeping and no read/write barriers beyond the card mark. Its pauses grow with the amount of live data, so it is a poor fit for latency-sensitive services with big heaps. Here is Parallel in the log, for a program `Churn` that allocates mostly short-lived 1 KB arrays and keeps about 2 percent (we reuse it below; compile once with `javac Churn.java`):

Churn.java JAVA

```java
import java.util.*;

public class Churn {
    public static void main(String[] args) {
        List<byte[]> retained = new ArrayList<>();
        long sum = 0;
        for (int i = 0; i < 3_000_000; i++) {
            byte[] tmp = new byte[1024];            // short-lived
            sum += tmp.length;
            if (i % 50 == 0) {                      // 2% of the data survives
                retained.add(new byte[1024]);
                if (retained.size() > 20_000) retained.subList(0, 10_000).clear();
            }
        }
        System.out.println(sum + " retained=" + retained.size());
    }
}
```

Parallel: every young collection is a stop-the-world copy; 128 MB heap TERMINAL

```
$ java -Xms128m -Xmx128m -XX:+UseParallelGC -Xlog:gc Churn
[0.034s][info][gc] Using Parallel
[0.377s][info][gc] GC(0) Pause Young (Allocation Failure) 33M->1M(123M) 1.846ms
[0.425s][info][gc] GC(1) Pause Young (Allocation Failure) 34M->2M(123M) 0.974ms
[0.473s][info][gc] GC(3) Pause Young (Allocation Failure) 35M->11M(123M) 29.196ms
```

9.4.4 G1: the default

G1 (Garbage-First) is designed for heaps from a few hundred MB to tens of GB and for a configurable pause-time goal. It still has generations, but not as contiguous blocks.

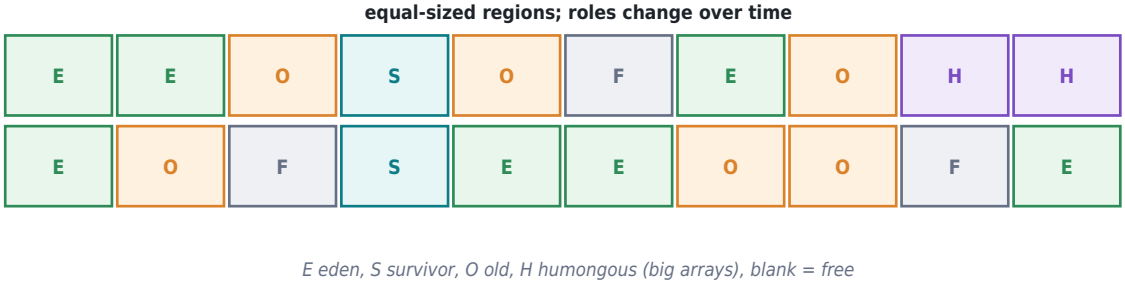


Figure 9.5. G1 divides the heap into equal-sized regions that are each eden, survivor, old or humongous; collection picks the regions with the most garbage first.

Regions

The heap is divided into about 2000 equal **regions** (1 MB to 32 MB, a power of two chosen from the heap size; the log of our 128 MB heap printed `Heap Region Size: 1M`). Each region plays a role at a given time: eden, survivor, old, or **humongous** (an object at least half a region large gets its own contiguous run of regions; that is expensive, so avoid frequent allocation of multi-megabyte arrays under G1). Because the role is per region, G1 can resize the young generation by giving or taking regions, and can collect any set of regions it likes.

Young collections

When eden regions are full G1 stops the world and evacuates the live objects of the **collection set** (all young regions) into new survivor or old regions, in parallel across GC threads. It sizes the number of young regions to meet the pause goal `-XX:MaxGCPauseMillis` (200 ms by default). Each region has a **remembered set** that records which other regions point into it, so a region can be collected without scanning the rest of the heap.

Concurrent marking and mixed collections

To clean the old generation without a long pause, G1 starts a **concurrent marking cycle** when the heap occupancy reaches the **initiating heap occupancy** (`-XX:InitiatingHeapOccupancyPercent`, 45 by default; G1 also adapts this value at run time). Marking runs alongside the application and uses a snapshot-at-the-beginning (SATB) technique: a write barrier records the old value of any reference that is overwritten, so objects alive when marking started are not lost. After marking, G1 knows how much garbage each old region holds. The next few young collections become **mixed collections**: they collect all young regions plus the old regions with the **most garbage** (hence the name Garbage-First), as many as the pause goal allows. The old generation is thus compacted gradually, in small steps, rather than in one big pause. If G1 cannot keep up and runs out of space, it falls back to a **full GC** which is single-threaded in older releases and parallel since Java 10; a full GC in a G1 log is a sign that the heap is too small or the allocation rate too high.

The following trimmed log comes from `Fill`, which is like `Churn` but retains one array in eight, so the old generation really fills up. It is the same code with different numbers:

Fill.java JAVA

```java
import java.util.*;

public class Fill {
    public static void main(String[] args) {
        List<byte[]> retained = new ArrayList<>();
        long sum = 0;
        for (int i = 0; i < 4_000_000; i++) {
            byte[] tmp = new byte[1024];
            sum += tmp.length;
            if (i % 8 == 0) {
                retained.add(new byte[1024]);
                if (retained.size() > 60_000) retained.subList(0, 30_000).clear();
            }
        }
        System.out.println(sum + " retained=" + retained.size());
    }
}
```

On a 128 MB heap it shows each phase:

G1, real output, trimmed — TERMINAL

```
$ java -Xms128m -Xmx128m -XX:+UseG1GC -Xlog:gc Fill
[0.317s][info][gc] GC(0) Pause Young (Normal) (G1 Evacuation Pause) 7M->1M(128M) 15.244ms
[1.039s][info][gc] GC(7) Pause Young (Normal) (G1 Evacuation Pause) 75M->62M(128M) 22.031ms
[1.046s][info][gc] GC(8) Pause Young (Concurrent Start) (G1 Evacuation Pause) 86M->65M(128M) 3.113ms
[1.047s][info][gc] GC(9) Concurrent Mark Cycle
[1.054s][info][gc] GC(9) Pause Remark 80M->53M(128M) 0.305ms
[1.099s][info][gc] GC(9) Pause Cleanup 60M->60M(128M) 0.074ms
[1.100s][info][gc] GC(9) Concurrent Mark Cycle 53.165ms
[1.127s][info][gc] GC(11) Pause Young (Prepare Mixed) (G1 Evacuation Pause) 79M->70M(128M) 23.806ms
[1.140s][info][gc] GC(12) Pause Young (Mixed) (G1 Evacuation Pause) 90M->89M(128M) 9.127ms
```

Reading it: GC(8) is a young pause that also **starts** the concurrent cycle; GC(9) is the marking, mostly concurrent, with only two tiny pauses (Remark and Cleanup, 0.3 and 0.07 ms); GC(11) "Prepare Mixed" and GC(12) "Mixed" then reclaim old regions. Each line has the form `before->after(committed)` followed by the pause.

When G1 is the right choice

For most services with heaps of 4 GB to a few tens of GB and a latency target in the tens to hundreds of milliseconds, G1 with the defaults is the right place to start, and you add flags only when logs show a problem. Its costs: extra memory for remembered sets and other bookkeeping, and extra barrier work for the application. Recent releases keep improving this: Java 26 reduces synchronization in G1's write barriers for better throughput (JEP 522), and Java 27 makes it the default everywhere.

9.4.5 ZGC: pauses that do not grow with the heap

ZGC targets pause times that stay below a millisecond or so and **do not depend on the heap size or the live-set size**; it is built for heaps from a few GB to many TB. Its trick is that nearly everything, including moving objects, is done concurrently with the application:

- It uses **load barriers**: a small piece of code the JIT inserts at every reference load from the heap. If the reference points at an object that is being relocated, or its "color" bits show it is stale, the barrier fixes it on the spot (healing the pointer), so the application never sees an old address.
- Colored pointers carry metadata bits in the 64-bit reference itself, which is why ZGC does not use compressed oops.
- The only stop-the-world phases are short ones that scan roots; their length does not scale with heap size.

In Java 21, ZGC became generational: `-XX:+UseZGC -XX:+ZGenerational` (JEP 439) collects young objects separately, which greatly cuts the CPU and headroom the non-generational version needed. In Java 23 generational became the default mode of ZGC, and in Java 24 the non-generational mode was removed (JEP 490), so on Java 25 `-XX:+UseZGC` simply means generational ZGC. The price: ZGC uses more CPU than G1 (barriers and concurrent threads), needs free headroom (if the application allocates faster than ZGC can reclaim, threads are stalled; you see `Allocation Stall` in the log), and uses more memory. On our small 2-CPU build machine with a 128 MB heap it behaves exactly like that:

ZGC (non-generational, JDK 21 default mode) on a tiny heap; this is a stress case, not typical — TERMINAL

```
$ java -Xms128m -Xmx128m -XX:+UseZGC -Xlog:gc Churn
[0.136s][info][gc] Using The Z Garbage Collector
[0.735s][info][gc] GC(0) Garbage Collection (Warmup) 50M(39%)->48M(38%)
[1.166s][info][gc] Allocation Stall (main) 24.367ms
[1.329s][info][gc] GC(3) Garbage Collection (Allocation Stall) 24M(19%)->24M(19%)
```

The lesson: do not judge ZGC on a 128 MB heap. Use it where it is designed to shine, a multi-GB heap, several spare cores and a hard latency requirement, and give it room.

9.4.6 Shenandoah

Shenandoah (included in many OpenJDK builds, such as those from Linux distributions and Adoptium; not in every vendor's build, and it was present in the JDK 21 used here) also compacts concurrently, with a different mechanism: a **load-reference barrier** ensures that all accesses go to the new copy of a moved object. Its pauses are short and mostly independent of heap size, like ZGC's. A generational mode of Shenandoah became a product feature in Java 25 (JEP 521); check the release notes of your build before relying on it. Shenandoah is a good alternative where ZGC is unavailable or where you want its heap-sizing behaviour; run the same load against both and compare.

Shenandoah log start, JDK 21, trimmed — TERMINAL

```
$ java -Xms128m -Xmx128m -XX:+UseShenandoahGC -Xlog:gc Churn
[0.050s][info][gc] Using Shenandoah
[0.387s][info][gc] Trigger: Learning 1 of 5. Free (90880K) is below initial threshold (91750K)
[0.387s][info][gc] GC(0) Pause Init Mark (unload classes) 0.010ms
[0.392s][info][gc] GC(0) Concurrent marking (unload classes) 4.641ms
```

9.4.7 Epsilon: the collector that does nothing

`-XX:+UnlockExperimentalVMOptions -XX:+UseEpsilonGC` gives you a heap that is allocated but never reclaimed. When it fills up, the JVM exits. It sounds useless, but it has real uses: short-lived programs sized never to fill the heap, performance tests where you want to measure your code without GC noise, and experiments to measure how much of a slowdown is GC.

Epsilon on a 16 MB heap, with Churn — TERMINAL

```
$ java -XX:+UnlockExperimentalVMOptions -XX:+UseEpsilonGC -Xmx16m Churn
Terminating due to java.lang.OutOfMemoryError: Java heap space
```

9.4.8 How to choose

Use this order of questions:

1. **Is the heap tiny or the container limited to one CPU?** Serial.
2. **Is it a batch job, a build, a data-processing run, where total run time matters and pauses do not?** Parallel (or G1 if you are unsure; the difference is usually modest).
3. **Is it an ordinary service (REST, Spring Boot, Kafka consumers) with a heap up to a few tens of GB?** G1, defaults first.
4. **Do you have a hard latency SLO, big heaps, and spare CPU?** Try ZGC (or Shenandoah), and compare p99 latency and CPU against G1 with a load test, not by reasoning.
5. **Experiments or ultra-short tasks?** Epsilon.

Do not switch collector because a blog said so. Switch when a GC log or a tool like JFR shows pauses that violate your latency target, or throughput lost to GC that you cannot fix by raising the heap or reducing allocation.

INTERVIEW ANGLE

Which garbage collector does Java use by default and how do you pick another? G1 since Java 9 on server-class machines (at least 2 CPUs and about 1.8 GB), Serial on smaller ones; Java 27 will make G1 the default everywhere. You pick another with `-XX:+UseSerialGC`, `-XX:+UseParallelGC`, `-XX:+UseZGC`, `-XX:+UseShenandoahGC`. Say what you would pick: G1 for general services, ZGC for strict low latency on large heaps, Parallel for batch throughput, Serial for tiny containers.

REAL WORLD

Most Spring Boot services run happily on the default G1. Teams move to ZGC when they have p99 latency targets that a G1 mixed pause breaks, and they do it behind a flag in the deployment (`JAVA_TOOL_OPTIONS=-XX:+UseZGC`) after a canary comparison. Many small microservices pods (1 CPU, 512 MB) get Serial automatically before Java 27, and that is often the correct choice.

Check yourself

1. Your service runs fine on a 4-CPU VM but in a Kubernetes pod limited to 1 CPU its pauses behave differently. Why might this be (before Java 27)?
2. What does a `Pause Young (Mixed)` line in a G1 log tell you has happened earlier?
3. Why does ZGC not use compressed oops, and what does the load barrier do?
4. What happens if you run `java -XX:+UseEpsilonGC -Xmx16m` against a program that allocates 1 GB in total?
5. A G1 log shows `Pause Full (G1 Compaction Pause)` repeatedly. Name two likely causes.
6. In which direction does a larger `-XX:MaxGCPauseMillis` push throughput and pause length?

Answers

1. With fewer than 2 CPUs the JVM is not "server-class", so it selects Serial instead of G1: a different collector with different pause behaviour.
2. A concurrent marking cycle ran and finished (concurrent start, remark, cleanup), so G1 knows which old regions hold the most garbage.
3. ZGC stores metadata bits inside the 64-bit reference (colored pointers), so references cannot be compressed. The load barrier checks those bits on every reference load and, if the object has moved, fixes the pointer, so mutators never see stale addresses.
4. The program dies with `OutOfMemoryError: Java heap space` once 16 MB is used; Epsilon never reclaims.
5. The heap is too small for the live set, or the allocation rate exceeds what marking and mixed collections can reclaim; humongous allocations fragmenting the heap and a late-starting marking cycle are also common causes.
6. A larger goal lets G1 do more work per pause: fewer, longer pauses and usually higher throughput.

KEY POINTS

Serial, Parallel, G1, ZGC, Shenandoah and Epsilon are points on the throughput, latency and footprint triangle; pick by goal, then verify with a load test.

- Default: G1 on 2 or more CPUs and about 1.8 GB or more, Serial otherwise; Java 27 makes G1 the default everywhere.
- G1 uses equal-size regions, a pause goal (`MaxGCPauseMillis`, 200 ms), concurrent marking and mixed collections that take the most garbage-heavy old regions first.
- ZGC and Shenandoah compact concurrently, keeping pauses very short at the price of CPU, headroom and barriers; ZGC is generational by default from Java 23 and non-generational mode is gone in Java 24.
- Epsilon never collects, which is useful only for tests and very short programs.

9.5 Reading GC Logs and Tuning

Tuning without a log is guessing. The JVM can tell you exactly when each collection ran, how long it stopped your threads, and how much memory it recovered. This chapter shows how to switch that on, how to read a line, which handful of flags matter, and how to size the JVM inside a container. We close with a procedure you can follow on a real service.

9.5.1 Sizing the heap: -Xms and -Xmx

`-Xmx` sets the maximum heap size and `-Xms` the initial size (for example `-Xms512m -Xmx2g`; units `k`, `m`, `g`). If you set nothing, the JVM decides from the machine: the maximum is `MaxRAMPercentage`, **25 percent** of the physical memory by default, and the initial size is `InitialRAMPercentage`, about 1.6 percent. On the 6 GB build machine the default maximum was 1496 MB and the initial heap about 94 MB, both visible in `-XX:+PrintFlagsFinal`.

For servers the usual advice is to set `-Xms` equal to `-Xmx`. The heap then never has to grow or shrink, which avoids resize pauses and makes memory use predictable, and the OS maps the memory only as it is touched anyway. For tiny tools, leave the defaults.

Does a bigger heap help? Run the same allocation-heavy program (a variant of `Fill` that prints the collector statistics from `GarbageCollectorMXBean` at the end) with different heap sizes:

GcStats.java JAVA

```
import java.lang.management.*;
import java.util.*;

public class GcStats {
    public static void main(String[] args) {
        List<byte[]> retained = new ArrayList<>();
        long start = System.nanoTime();
        for (int i = 0; i < 4_000_000; i++) {
            byte[] tmp = new byte[1024];
            if (i % 8 == 0) {
                retained.add(new byte[1024]);
                if (retained.size() > 60_000) retained.subList(0, 30_000).clear();
            }
        }
        long ms = (System.nanoTime() - start) / 1_000_000;
        long count = 0, gcMs = 0;
```

GcStats.java (continued) JAVA

```
        for (GarbageCollectorMXBean b : ManagementFactory.getGarbageCollectorMXBeans()) {
            count += b.getCollectionCount();
            gcMs += b.getCollectionTime();
        }
        System.out.println("maxHeap=" + Runtime.getRuntime().maxMemory() / (1 << 20)
            + "MB wall=" + ms + "ms collections=" + count + " gcTime=" + gcMs + "ms");
    }
}
```

Typical output, G1, measured on the 2-CPU build machine (timings are noisy) TERMINAL

```
$ javac GcStats.java
$ java -Xms80m -Xmx80m GcStats
maxHeap=80MB wall=971ms collections=105 gcTime=406ms
$ java -Xms128m -Xmx128m GcStats
maxHeap=128MB wall=923ms collections=55 gcTime=512ms
$ java -Xms512m -Xmx512m GcStats
maxHeap=512MB wall=2502ms collections=20 gcTime=1562ms
$ java -Xms2g -Xmx2g GcStats
maxHeap=2048MB wall=2695ms collections=7 gcTime=1256ms
```

The number of collections falls steadily as the heap grows (105, 55, 20, 7), exactly as the allocation-rate argument predicts. The total time did not fall: with a larger young generation each collection touches fresh memory and the machine's two cores were shared with the test harness, so the wall time here is dominated by noise and page faults. This is a good reminder that "bigger heap = faster" is not a law. A bigger heap reduces **frequency**; the **duration** of each pause depends on the live data being copied. Always measure with your workload.

9.5.2 Turning the log on: -Xlog

Java 9 replaced the old `-XX:+PrintGCDetails` family with **unified logging**: `-Xlog:<selectors>:<output>:<decorators>`. The pieces:

- **Selectors** are `tag` combinations with a level, such as `gc` (the one-line summary per collection), `gc*` (everything starting with the tag `gc`), `gc+heap=debug`, `safepoint`. The default without any `-Xlog` is only warnings.
- **Output** is `stdout`, `stderr` or `file=name`, optionally with rotation: `file=gc.log:time:filecount=5,filesize=20m`.
- **Decorators** prefix each line: `uptime` (the default), `time` (wall-clock), `level`, `tags`, `pid`.

Common recipes:

Useful GC logging flags TERMINAL

```
$ java -Xlog:gc -jar app.jar                    # one line per collection to stdout
$ java -Xlog:gc*:file=gc.log:time,uptime,level,tags:filecount=5,filesize=20m -jar app.jar
$ java -Xlog:gc+heap=debug,gc+phases=debug,safepoint:file=gc-detail.log -jar app.jar
$ java -Xlog:disable -Xlog:gc:stderr -jar app.jar   # turn everything off, then only gc
```

For production keep `-Xlog:gc*:file=...` with rotation on all the time; the overhead is negligible and the log is the first thing you will want after an incident.

9.5.3 Reading a collection line

The default `-Xlog:gc` line has a fixed shape. Take this G1 line from the earlier run:

```
[0.696s][info][gc] GC(2) Pause Young (Normal) (G1 Evacuation Pause) 76M->3M(128M) 17.474ms
 |         |     |    |     |                  |                        |    |    |     |
 |         |     |    |     |                  |                        |    |    |     └ pause
 |         |     |    |     |                  |                        |    |    └ committed heap
 |         |     |    |     |                  |                        |    └ heap in use AFTER
 |         |     |    |     |                  |                        └ heap in use BEFORE
 |         |     |    |     |                  └ cause (G1 Evacuation Pause)
 |         |     |    |     └ type: Young (Normal), Concurrent Start, Mixed, Full
 |         |     |    └ GC number (collection counter)
 |         |     └ tag
 |         └ level
 └ uptime since JVM start
```

Three numbers answer most questions. **Before and after** show how much was freed, and a growing "after" across collections shows a leak or a fill-up. The **pause** is what your application felt. The **cause** says why it ran: `Allocation`

`Failure` (eden/young was full), `G1 Evacuation Pause`, `G1 Humongous Allocation`, `Metadata GC Threshold` (Metaspace reached its threshold), `System.gc()`, `Ergonomics`.

With more detail (`gc*`, `gc+heap=debug`) you get per-phase times. This is the same G1 collection, trimmed:

G1 collection with phase detail TERMINAL

```
$ java -Xlog:gc*:stdout -Xmx128m Churn.java
[0.679s][info ][gc,start     ] GC(2) Pause Young (Normal) (G1 Evacuation Pause)
[0.679s][info ][gc,task      ] GC(2) Using 2 workers of 2 for evacuation
[0.692s][info ][gc,phases    ] GC(2)   Pre Evacuate Collection Set: 0.10ms
[0.696s][info ][gc,phases    ] GC(2)   Evacuate Collection Set: 13.04ms
[0.696s][info ][gc,phases    ] GC(2)   Post Evacuate Collection Set: 0.15ms
[0.696s][info ][gc,heap      ] GC(2) Eden regions: 75->0(73)
[0.696s][info ][gc,heap      ] GC(2) Survivor regions: 1->3(10)
[0.696s][info ][gc,heap      ] GC(2) Old regions: 2->2
[0.696s][info ][gc,heap      ] GC(2) Humongous regions: 0->0
[0.696s][info ][gc           ] GC(2) Pause Young (Normal) (G1 Evacuation Pause) 76M->3M(128M) 17ms
[0.696s][info ][gc,cpu       ] GC(2) User=0.00s Sys=0.00s Real=0.02s
```

"Evacuate Collection Set" is almost all of the pause: copying the live objects. The region lines show eden emptied (75 regions to 0), survivors rising from 1 to 3 regions, and no humongous objects. If `User` time is much larger than `Real`, the GC threads were working in parallel; if `Real` is far larger than `User + Sys`, the JVM was waiting, often because the machine was swapping or CPU-throttled by a container limit.

9.5.4 Reading the patterns

What to look for in a GC log of a running service:

- **Healthy**: young pauses of a few to a few tens of milliseconds, "after" sizes flat over time, rare mixed collections.
- **Leak or fill-up**: the "after" size of each collection (the live set) grows steadily, and old collections get closer together until a Full GC. More heap only delays it; see the leaks chapter.
- **Heap too small**: collections every few hundred milliseconds with high "after" numbers; `Pause Full` lines; GC time above 10 percent of wall time.
- **Allocation storm**: pauses are fine but the collection rate is huge. The fix is in the code (fewer temporary objects, a profiler's allocation view), not in flags.
- **Humongous allocations in G1**: causes named `G1 Humongous Allocation`. Fix by raising region size (`-XX:G1HeapRegionSize=16m`) or avoiding giant arrays.
- **Allocation stalls in ZGC**: the log shows `Allocation Stall`; give it a bigger heap or more CPU.

To analyse logs beyond reading them by eye, tools such as GCeasy or GCViewer parse them into charts; the flags above produce the input they need.

9.5.5 The flags that matter

There are hundreds of GC flags; a handful covers 95 percent of real tuning.

Flag	Default	What it does
`-Xms` / `-Xmx`	1.6% / 25% of RAM	Initial and maximum heap. Set equal on servers.
`-XX:MaxRAMPercentage`	25	Max heap as a percentage of available (container) memory, when no `-Xmx` is given
`-XX:InitialRAMPercentage`	about 1.6	Initial heap as a percentage
`-XX:MaxGCPauseMillis`	200 (G1)	Pause-time goal for G1. A hint, not a guarantee
`-XX:InitiatingHeapOccupancyPercent`	45 (G1)	Heap occupancy at which a concurrent cycle starts
`-XX:G1HeapRegionSize`	by heap size	Region size 1m to 32m
`-XX:NewRatio` / `-Xmn`	2 (old = 2 x young)	Young generation sizing for Serial and Parallel
`-XX:MaxTenuringThreshold`	15	Number of young collections before promotion
`-XX:+HeapDumpOnOutOfMemoryError`	off	Write a heap dump when an OOM is thrown
`-XX:MaxMetaspaceSize`	unlimited	Cap for class metadata

Flag	Default	What it does
`-XX:MaxDirectMemorySize`	= max heap	Cap for direct buffers
`-Xss`	1m	Thread stack size
`-XX:+AlwaysPreTouch`	off	Touch all heap pages at startup, trading start time for steady latency
`-XX:+UseStringDeduplication`	off	G1, ZGC and others can merge duplicate `String` arrays

The important JVM memory and GC flags.

Notes on the pause goal: `MaxGCPauseMillis` is a target that G1 tries to meet by sizing young and choosing how many old regions to include in mixed collections. Lowering it (say to 50) gives shorter, more frequent pauses and lower throughput; if the goal is unrealistic for your live set, G1 will miss it and pay for trying. Raise it for batch-like services.

WATCH OUT

Do not copy a page of tuning flags from a blog post or an old Java 8 service. Flags such as `-XX:+UseConcMarkSweepGC` (CMS, removed in Java 14), `-XX:+PrintGCDetails` (replaced by `-Xlog:gc*`), `-XX:PermSize` (PermGen is gone) or `-XX:+UseBiasedLocking` (removed) are either rejected or ignored today. The JVM warns on obsolete flags; treat those warnings as a to-do list. Start from defaults plus a heap size and add one flag at a time with a measurement.

9.5.6 Containers

A Java process in a container must stay inside the cgroup memory limit, or the kernel's OOM killer ends it. The JVM is **container-aware**: since Java 10 (and backported to 8u191) it reads the cgroup limits and uses them for `availableProcessors()` and the default heap size. The default maximum heap is 25 percent of the container memory, which is often too small for a dedicated service container. Two ways to choose:

- Fix it: `-Xms1g -Xmx1g` in a 2 GB container.
- Scale with it: `-XX:MaxRAMPercentage=70` (and let the container limit change per environment).

You can simulate a smaller machine without Docker using `-XX:MaxRAM` and `-XX:ActiveProcessorCount`:

Default heap follows the memory the JVM sees (WhichGc from the previous chapter) TERMINAL

```
$ java -XX:MaxRAM=512m WhichGc
cpus=2 maxHeapMB=128
  collector: G1 Young Generation
$ java -XX:MaxRAM=512m -XX:MaxRAMPercentage=75 WhichGc
cpus=2 maxHeapMB=384
$ java -XX:ActiveProcessorCount=1 WhichGc
cpus=1 maxHeapMB=1446
  collector: Copy
  collector: MarkSweepCompact
```

Look at the third run: a single visible CPU turned the default collector from G1 to Serial (the "Copy / MarkSweepCompact" pair), the container trap from the previous chapter.

How much of the container should be heap? The heap is only one part of process memory (previous chapters on heap structure): add Metaspace, code cache, thread stacks (1 MB reserved each; hundreds of threads add up), direct buffers and GC structures. A starting point of 60 to 75 percent of the limit for the heap is typical, lower when the application uses many threads or large direct buffers. Confirm with `jcmd <pid> VM.native_memory summary` after starting with `-XX:NativeMemoryTracking=summary`.

9.5.7 A tuning procedure

1. **Define the goal.** "p99 latency under 200 ms at 500 requests per second" or "nightly job finishes in 40 minutes". Without a goal there is nothing to tune.
2. **Turn the log on** (`-Xlog:gc*:file=...`) and run a realistic load, long enough to see steady state.
3. **Read before changing.** Fix leaks and allocation hot spots first; they beat any flag.
4. **Set the heap** to a size where the live set after a full collection is at most about 30 to 50 percent of the heap, then set `-Xms=-Xmx`.
5. **Change one thing at a time**, then compare p99 latency, GC time percentage and CPU, not only GC pauses.
6. **Only then** consider a different collector.

INTERVIEW ANGLE
Your service has long GC pauses. What do you do? First get data: enable `-Xlog:gc*` and find which kind of pause is long (young, mixed, full) and why (cause). Check whether the live set after GC is growing (leak) or the allocation rate is high (churn), whether the heap is too small, and whether the container limits CPU. Fix the code if it is a leak or allocation storm; resize the heap if it is undersized; consider ZGC or Shenandoah if the pause is inherent to the live-set size and a latency target requires it.

Check yourself

1. What is the default maximum heap of a JVM on a machine with 16 GB of RAM and no flags?
2. In this line, how much memory did the collection free and how long was the pause? `[2.5s][info][gc] GC(7) Pause Young (Normal) (G1 Evacuation Pause) 900M->150M(1024M) 31ms`
3. What is wrong with `-Xmx4g` in a container with a 4 GB memory limit?
4. Why does `-XX:+PrintGCDetails` not work on Java 17?
5. After every collection the "after" figure is larger than the one before, and Full GCs begin. What do you conclude, and does raising `-Xmx` fix it?
6. You set `-XX:MaxGCPauseMillis=20` and throughput drops. Why?

Answers

1. 25 percent of the memory, about 4 GB.
2. 750 MB freed (900M to 150M); the pause was 31 ms.
3. The heap is only part of the process: Metaspace, code cache, thread stacks, direct memory and GC structures push the total above 4 GB and the container is killed by the OOM killer. Leave headroom or use `MaxRAMPercentage`.
4. It belongs to the pre-Java-9 logging system; since Java 9 you use unified logging, for example `-Xlog:gc*`.
5. The live set is growing: a leak or an unbounded cache. A bigger heap only delays the failure.
6. A tiny pause goal forces G1 to use small young generations and fewer old regions per mixed collection, so collections happen more often and GC overhead rises.

KEY POINTS
Set the heap deliberately (`-Xms` = `-Xmx` on servers) and leave room outside the heap, especially in containers; the default max heap is 25 percent of memory.

- Switch on `-Xlog:gc*:file=gc.log` with rotation; read before/after sizes, pause and cause on every line.
- A growing "after" size means a leak or a too-small heap; a high collection rate means allocation churn; both are fixed in code or sizing, not by exotic flags.
- `MaxGCPauseMillis` is a goal that trades throughput for shorter pauses; most other GC flags can stay at their defaults.
- Fewer than 2 visible CPUs or too little memory selects Serial before Java 27, so check the collector actually in use.

9.6 Memory Leaks and OutOfMemoryError

Java has a garbage collector, so how can a Java program leak memory? Because the collector frees only what is **unreachable**. A *leak* in Java is an object your program will never use again but still reaches through some reference, usually from a long-lived place such as a static field, a cache or a thread. The collector is correct to keep it. The bug is yours.

9.6.1 What a leak looks like

The signature is in the GC log from the previous chapter: the heap size **after** each full collection climbs steadily, run after run, until a collection cannot free enough and the JVM throws `OutOfMemoryError` (OOM). A healthy program has a sawtooth that returns to roughly the same floor. A leaking program has a sawtooth on a rising ramp.

Here is the simplest leak: a static list that only grows. We run it with a deliberately small heap so it fails quickly, and ask the JVM to write a heap dump when it dies.

Leak.java JAVA

```java
import java.util.ArrayList;
import java.util.List;

public class Leak {
    static final List<byte[]> CACHE = new ArrayList<>();

    public static void main(String[] args) {
        Runtime rt = Runtime.getRuntime();
        try {
            for (int i = 1; ; i++) {
                CACHE.add(new byte[1024 * 1024]);          // 1 MB that is never removed
                if (i % 20 == 0) {
                    System.gc();
                    long used = (rt.totalMemory() - rt.freeMemory()) >> 20;
                    System.out.println("entries=" + i + " usedAfterGC=" + used + " MB");
                }
            }
        } catch (OutOfMemoryError e) {
            int n = CACHE.size();
            CACHE.clear();                                 // free memory so we can print
            System.out.println("Caught after " + n + " entries: " + e.getMessage());
        }
    }
}
```

Run with a 64 MB heap and a heap dump on OOM TERMINAL

```
$ java -Xmx64m -XX:+HeapDumpOnOutOfMemoryError -XX:HeapDumpPath=leak.hprof Leak.java
entries=20 usedAfterGC=43 MB
java.lang.OutOfMemoryError: Java heap space
Dumping heap to leak.hprof ...
Heap dump file created [39424353 bytes in 0.205 secs]
Caught after 30 entries: Java heap space
```

The line after `entries=20` is the JVM announcing the OOM and writing the dump; our `catch` then runs. Even after an explicit `System.gc()`, 43 MB stayed in use at 20 entries: that floor is the leak. (Catching `OutOfMemoryError` is fine in a demo and in a few well-defined places such as a request boundary, but never a recovery strategy: after an OOM, other threads may have died half-way through updating shared state.)

9.6.2 The usual suspects

Real leaks fall into a small number of patterns. Learn to recognise them in code review.

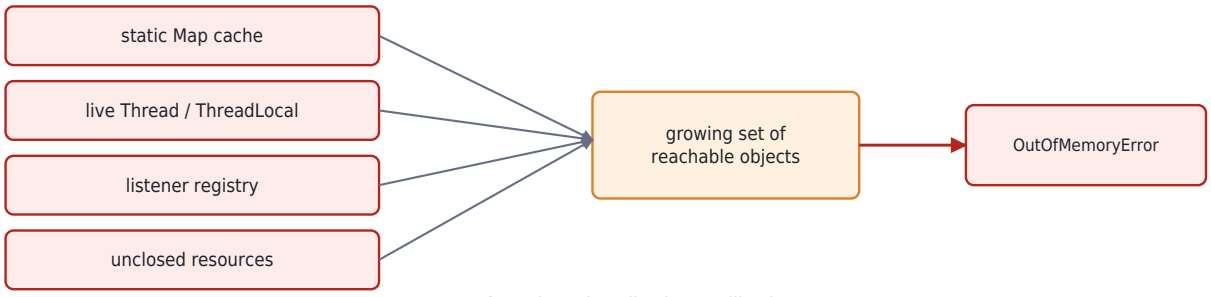


Figure 9.6. The common leak patterns: every one is a long-lived root (static field, live thread, listener registry, open resource) that keeps a growing set of objects reachable.

1. Static and long-lived collections

A `static Map` used as a cache with no size limit or eviction, a registry that is added to on every request, a "history" list. Fix: bound the structure. Use a size-limited LRU (a `LinkedHashMap` with `removeEldestEntry`, or a library cache such as Caffeine with `maximumSize` and expiry), or key by a small set.

2. Listeners and callbacks never removed

An object registers itself with an event bus, a `static` listener list, a `Runtime` shutdown hook or a scheduler, and is never unregistered. The registry holds it forever, and it holds everything it references (often a whole screen, session or request context). Fix: pair every `register` with an `unregister` in a lifecycle method, or hold listeners through weak references so the registry does not keep them alive.

3. ThreadLocal values on pooled threads

A `ThreadLocal` value lives as long as its thread. Application-server and `ExecutorService` threads live for the lifetime of the process, so a value you `set` and forget stays alive. The demo below makes the effect visible: eight pooled threads each store an 8 MB buffer.

TlLeak.java JAVA

```java
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;
import java.util.concurrent.TimeUnit;

public class TlLeak {
    static final ThreadLocal<byte[]> BUFFER = new ThreadLocal<>();

    static long usedMb() {
        System.gc();
        Runtime rt = Runtime.getRuntime();
        return (rt.totalMemory() - rt.freeMemory()) >> 20;
    }

    static void run(boolean remove) throws Exception {
        ExecutorService pool = Executors.newFixedThreadPool(8);
        for (int i = 0; i < 8; i++) {
            pool.submit(() -> {
                try {
                    BUFFER.set(new byte[8 * 1024 * 1024]);
                    // handle a request using BUFFER.get()
                } finally {
                    if (remove) BUFFER.remove();
                }
            });
        }
        Thread.sleep(300);                 // tasks finished, threads stay alive in the pool
        System.out.println("remove=" + remove + "  used after GC: " + usedMb() + " MB");
        pool.shutdown();
        pool.awaitTermination(5, TimeUnit.SECONDS);
    }

    public static void main(String[] args) throws Exception {
        run(false);
        System.out.println("after pool shutdown: " + usedMb() + " MB");
        run(true);
    }
}
```

Run it TERMINAL

```
$ java -Xmx512m TlLeak.java
remove=false  used after GC: 76 MB
after pool shutdown: 3 MB
remove=true  used after GC: 3 MB
```

All eight tasks had finished, yet 64 MB of buffers (plus baseline) was still reachable through the idle worker threads. When the pool shut down, the threads died and the memory returned. The fix is the `finally { BUFFER.remove(); }` pattern, which you should treat as mandatory for any `ThreadLocal` used in a pooled thread. In frameworks this bites with request-scoped data, and with web applications that are redeployed: a `ThreadLocal` that references a class from the old application keeps the old class loader alive, which leaks Metaspace (see below). Virtual threads make per-thread caches in `ThreadLocal` costly for a different reason (there can be millions of them), which is one motivation for scoped values (see the concurrency Part).

4. Unclosed resources

Streams, sockets, JDBC connections, `HttpClient` responses and file handles hold native memory or OS descriptors that the heap size does not show. An unclosed `Connection` also pins its statement and result-set buffers. Fix: `try`-with-resources everywhere. A `Cleaner` (previous chapter) is only a safety net.

5. Inner classes and lambdas capturing `this`

A non-static inner class, an anonymous class or a lambda that touches an instance member holds a hidden reference to the enclosing object. Handing such an object to a long-lived scheduler or cache retains the whole enclosing instance. Prefer `static` nested classes or capture only the values you need.

6. Mutable keys and missing `equals` and `hashCode`

Put objects with a broken `hashCode` (or a mutated key) into a `HashMap`: lookups miss, so code that does "if absent, add" adds again every time and the map grows without bound. `equals`/`hashCode` were covered in the Object chapter; here is the production consequence.

7. Class loader leaks

Metaspace holds class metadata, and a class is unloaded only when its class loader becomes unreachable. Anything that keeps a loader alive (a static reference in a shared library to a class loaded by a web application, a `ThreadLocal`, a registered JDBC driver, a thread started by the application) keeps every class it loaded. Dynamic class generation (proxies, bytecode libraries, scripting engines) in a loop with fresh loaders does the same.

9.6.3 Each OutOfMemoryError message and what it means

`OutOfMemoryError` is not one problem but several. **The message after the colon tells you which memory area is exhausted**, so always read it before touching a flag.

Message	Meaning	First action
`Java heap space`	Heap is full and a collection could not free enough for an allocation.	Leak (growing floor in the GC log) or heap too small; take a heap dump.
`Metaspace`	Class metadata area hit its limit (`MaxMetaspaceSize`, if set, or native memory).	Class loader leak or runaway class generation; look at loaded class counts.
`GC overhead limit exceeded`	Parallel collector only: almost all time in GC, almost nothing freed (default limit about 98 percent time and under 2 percent freed).	Same as heap space: the heap is effectively full.
`Requested array size exceeds VM limit`	Array length near `Integer.MAX_VALUE`.	Bug in size computation, or use a different structure.
`Cannot reserve N bytes of direct buffer memory`	`ByteBuffer.allocateDirect` over `MaxDirectMemorySize`.	Missing release of direct buffers, or a limit too low for the network library.
`unable to create native thread`	The OS refused a new thread (process or memory limit).	Thread leak or unbounded pools; check `ulimit -u`, container limits, thread count.
`Out of swap space`	The OS has no memory left for a native allocation.	Check the container limit and native memory tracking (`-XX:NativeMemoryTracking=summary`).

Common OOM messages and the first thing to check.

We can reproduce several of them on one machine. First the array-size cases and the direct-buffer one, with a 64 MB heap and a 32 MB direct limit (output from JDK 21):

OomMessages.java JAVA

```java
public class OomMessages {
    public static void main(String[] args) {
        try { Object[] a = new Object[Integer.MAX_VALUE]; }
        catch (OutOfMemoryError e) { System.out.println("1: " + e.getMessage()); }
        try { long[] a = new long[Integer.MAX_VALUE - 8]; }
        catch (OutOfMemoryError e) { System.out.println("2: " + e.getMessage()); }
        try { java.nio.ByteBuffer.allocateDirect(200 * 1024 * 1024); }
        catch (OutOfMemoryError e) { System.out.println("3: " + e.getMessage()); }
        try { int[] a = new int[-1]; }
        catch (Throwable e) { System.out.println("4: " + e); }
    }
}
```

Run it TERMINAL

```
$ java -Xmx64m -XX:MaxDirectMemorySize=32m OomMessages.java
1: Requested array size exceeds VM limit
2: Java heap space
3: Cannot reserve 209715200 bytes of direct buffer memory (allocated: 8192, limit: 33554432)
4: java.lang.NegativeArraySizeException: -1
```

Note case 2: a huge `long[]` is not "too big for the VM" but simply larger than the heap, so it reports `Java heap space`. Case 4 is a different exception altogether, a plain bug. Now Metaspace. We create a fresh class loader in a loop and define a proxy class in each; the loaders are kept in a list, so none can be unloaded:

MetaLeak.java JAVA

```java
import java.lang.reflect.Proxy;
import java.util.ArrayList;
import java.util.List;

public class MetaLeak {
    public static void main(String[] args) {
        List<Object> keep = new ArrayList<>();
        try {
            for (int i = 0; ; i++) {
                ClassLoader loader = new ClassLoader(null) { };   // a fresh loader each time
                keep.add(Proxy.newProxyInstance(loader, new Class<?>[] {Runnable.class},
                        (proxy, method, a) -> null));            // defines a new proxy class
                if (i % 2000 == 0) System.out.println("loaders created: " + i);
            }
        } catch (OutOfMemoryError e) {
            int n = keep.size();
            keep.clear();
            System.out.println("Caught after " + n + " loaders: " + e.getMessage());
        }
    }
}
```

Run with Metaspace capped at 16 MB TERMINAL

```
$ java -XX:MaxMetaspaceSize=16m MetaLeak.java
loaders created: 0
Caught after 884 loaders: Metaspace
```

The count (884) depends on JDK and platform; the point is the message. By default Metaspace is **not** limited (it grows into native memory), so in production a runaway loader leak shows up as a process growing until the container is killed. Setting `-XX:MaxMetaspaceSize` turns that into a clean, diagnosable `OutOfMemoryError: Metaspace`.

GOTCHA

GC overhead limit exceeded This message comes from the Parallel collector's safety check, and only when the heap is nearly full and the collector keeps running without freeing anything. We could not trigger it with the default G1 or with Serial on the build machine, even with a 24 MB heap: those collectors report `Java heap space` instead. Treat it as "heap space" with a different label, and do not fix it with `-XX:-UseGCOverheadLimit`, which only makes the program slower before it fails the same way.

9.6.4 Heap dumps and finding the culprit

A **heap dump** is a snapshot of every live object and the references between them. Three ways to get one:

Taking a heap dump — TERMINAL

```
# automatically when an OOM happens (set this on every production JVM)
$ java -XX:+HeapDumpOnOutOfMemoryError -XX:HeapDumpPath=/var/dumps -jar app.jar
# on demand from a running process
$ jcmd <pid> GC.heap_dump /tmp/app.hprof
# a quick class-by-class count without a full dump
$ jcmd <pid> GC.class_histogram | head -20
```

Open the `.hprof` file in Eclipse MAT, VisualVM or IntelliJ's profiler. The question to ask is not "what is the biggest object" but "**what is keeping the objects alive**". The *dominator tree* groups memory by the object that, if it were freed, would free everything beneath it. The *path to GC roots* shows the chain from a root (a static field, a thread, a class loader) to a suspicious instance. A few thousand instances of the same class, all reachable from one `static` map, is the typical answer. Diagnostic tooling (`jcmd`, `jmap`, JFR) gets its own chapter in the next Part.

TIP
Take two class histograms several minutes apart under the same load. Subtract them: the class whose count only goes up is your suspect. This is faster than a full dump analysis and safe to run in production, although `GC.class_histogram` does force a full GC unless you add `-all`.

WATCH OUT
A heap dump file is as large as the live heap and contains everything in memory: passwords, tokens, personal data. Store it on a disk with enough free space (a full disk makes the dump fail silently) and treat it as sensitive data.

INTERVIEW ANGLE
How can a Java application have a memory leak when it has a garbage collector? The collector frees unreachable objects only. A leak is an object that is no longer needed but is still reachable from a GC root, for example through a static collection, a registered listener, a `ThreadLocal` on a pooled thread or an unclosed resource. I confirm it from the GC log (rising heap floor after full GCs), capture a heap dump, and inspect the path from a GC root to the retained instances in a tool such as MAT.

REAL WORLD
In Spring Boot services the classic leaks are an unbounded in-memory cache (use Caffeine or `@Cacheable` with a size and TTL), a `ThreadLocal` set in a filter and not cleared in `finally`, and metrics with unbounded label values (one new time series per user ID). Run with `-XX:+HeapDumpOnOutOfMemoryError` and `-XX:+ExitOnOutOfMemoryError` so the orchestrator restarts a broken instance instead of leaving it half-alive.

Check yourself

1. What is the difference between a memory leak in Java and in C?
2. The service log says `java.lang.OutOfMemoryError: Metaspace` after each redeploy of a web application without restarting the server. What is the likely cause?
3. Why does `BUFFER.remove()` in a `finally` block matter when `BUFFER` is a `ThreadLocal` and tasks run on a thread pool?
4. What is wrong with `catch (OutOfMemoryError e) { retry(); }` as a general strategy?
5. What does this print, and why? `Object[] a = new Object[Integer.MAX_VALUE];` under `-Xmx64m`.
6. You see `unable to create native thread`. Raising `-Xmx` makes it worse. Why?

Answers

1. In C you leak by forgetting to free; in Java the object is still reachable (a forgotten reference), so the collector may not free it. The cause is a lingering reference, not a missing `free`.
2. A class loader leak: something (a `ThreadLocal`, driver, thread or static reference from a shared parent loader) keeps the old application's class loader reachable, so its classes are never unloaded.
3. Pool threads outlive the task, so the value stays reachable through the thread's map; the memory is held until the thread dies, which in a server is the life of the process.
4. After an OOM, state may be half-updated and the retry will likely fail again; fix the cause, or fail fast and restart (`-XX:+ExitOnOutOfMemoryError`).

5. `OutOfMemoryError: Requested array size exceeds VM limit` (we ran it above on JDK 21): the length is above what the VM supports at all, regardless of heap.
6. Thread stacks are native memory outside the heap; a bigger heap leaves less room for the process under the OS or container limit.

KEY POINTS

A Java leak is a reachable-but-unused object; find the GC root path, not just the big objects.

- A rising heap floor after full GCs in the GC log is the signature; `HeapDumpOnOutOfMemoryError` captures the evidence.
- Usual causes: unbounded static caches, forgotten listeners, `ThreadLocal` on pooled threads (use `remove()`), unclosed resources, class loader leaks.
- Read the message: heap space, Metaspace, direct buffer memory, native thread and array-size errors point to different memory areas.
- Set `MaxMetaspaceSize` and `MaxDirectMemorySize` deliberately and treat heap dumps as sensitive files.

9.7 The JIT Compiler and Warm-Up

Java programs start slow and get fast. Run a method once and it is interpreted bytecode, a few dozen times slower than native code. Run it ten thousand times and the JVM compiles it to optimised machine code that can rival C. This chapter explains what the JVM does, how to watch it, why it makes naive benchmarks lie, and the options for making start-up faster.

9.7.1 Interpreter, C1 and C2

The HotSpot execution engine has three levels working together (see the figure).

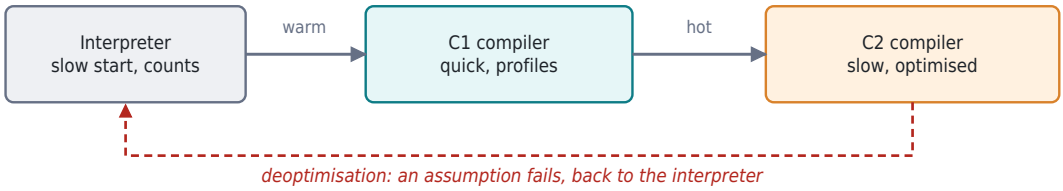


Figure 9.7. Tiered compilation: code starts in the interpreter, moves to C1 (fast to compile, with profiling) and then to C2 (slow to compile, heavily optimised) when counters show it is hot; deoptimisation can send it back.

- The **interpreter** executes bytecode one instruction at a time. It starts instantly and collects counters: how many times each method was invoked and how many times each loop branched backwards.
- **C1**, the client compiler, compiles quickly into reasonably fast code. At tier 3 it also inserts profiling code (which branches are taken, which receiver types appear at each call).
- **C2**, the server compiler, spends longer using the profile to compile heavily optimised code (tier 4).

The tiers are numbered 0 (interpreter), 1 to 3 (C1 variants) and 4 (C2). This is **tiered compilation**, on by default. Compilation happens on background compiler threads, so the application keeps running (in slower code) while the next version is built. The default thresholds, read from `-XX:+PrintFlagsFinal` on JDK 21:

Tier thresholds on the build machine TERMINAL

```
$ java -XX:+PrintFlagsFinal -version | grep -E "Tier[34](Invocation|BackEdge|Compile)Threshold"
     intx Tier3BackEdgeThreshold                    = 60000
     intx Tier3CompileThreshold                     = 2000
     intx Tier3InvocationThreshold                  = 200
     intx Tier4BackEdgeThreshold                    = 40000
     intx Tier4CompileThreshold                     = 15000
     intx Tier4InvocationThreshold                  = 5000
```

Roughly: about 200 invocations (or 2,000 invocations-plus-loop-iterations) trigger C1; about 5,000 (or 15,000) trigger C2. The JVM scales these with compiler queue length, so treat them as orders of magnitude, not exact. You do not normally tune them.

9.7.2 Watching the JIT: PrintCompilation

`-XX:+PrintCompilation` prints a line each time a method is compiled or invalidated. Our test program calls a tiny method in a hot loop and times eight rounds of two million calls:

Warm.java JAVA

```java
public class Warm {
    static int score(int x) { return (x * 31 + 7) % 1000; }

    static long batch(int n) {
        long sum = 0;
        for (int i = 0; i < n; i++) sum += score(i);
        return sum;
    }

    public static void main(String[] args) {
        for (int round = 1; round <= 8; round++) {
            long t0 = System.nanoTime();
            long r = batch(2_000_000);
            long micros = (System.nanoTime() - t0) / 1000;
            System.out.println("round " + round + ": " + micros + " us (result " + r + ")");
        }
    }
}
```

Typical output on the build machine (2 CPUs; times vary run to run) TERMINAL

```
$ java Warm.java
round 1: 13950 us (result 999000000)
round 2: 17647 us (result 999000000)
round 3: 13834 us (result 999000000)
round 4: 11448 us (result 999000000)
round 5: 2353 us (result 999000000)
round 6: 2424 us (result 999000000)
round 7: 2350 us (result 999000000)
round 8: 2421 us (result 999000000)
$ java -Xint Warm.java
round 1: 57511 us (result 999000000)
round 2: 81807 us (result 999000000)
round 3: 67594 us (result 999000000)
```

The first rounds ran 5 to 6 times slower than the last, and `-Xint` (interpreter only) is about 25 to 30 times slower than the compiled steady state. Same code, same machine, same result: the difference is warm-up. Here is what the JIT did (compiled to a class first, with `-XX:+PrintCompilation`, lines for `Warm` only; the columns are time in ms, compile id, flags, tier, method and bytecode size):

What the compiler logged TERMINAL

```
$ javac Warm.java && java -XX:+PrintCompilation Warm
   31    6       3       Warm::score (12 bytes)
   32    7       4       Warm::score (12 bytes)
   32    6       3       Warm::score (12 bytes)   made not entrant
   33    8 %     3       Warm::batch @ 4 (25 bytes)
   37    9       3       Warm::batch (25 bytes)
   38   10 %     4       Warm::batch @ 4 (25 bytes)
   40    8 %     3       Warm::batch @ 4 (25 bytes)   made not entrant
round 1: 8701 us (result 999000000)
   66  120       4       Warm::batch (25 bytes)
   68    9       3       Warm::batch (25 bytes)   made not entrant
   72   10 %     4       Warm::batch @ 4 (25 bytes)   made not entrant
round 2: 5559 us (result 999000000)
```

Reading the lines:

- `score` is compiled at tier 3 (C1) after enough calls, then at tier 4 (C2) a millisecond later. The tier 3 version is marked `made not entrant`: no new calls enter it, because a better version now exists.
- A `%` marks **OSR**, *on-stack replacement*. `batch` is called only once per round, but its loop runs two million times. The JVM compiles the loop (`@ 4` is the bytecode index of the loop head) and switches the running method from interpreter to compiled code in the middle of the loop. Without OSR, a long-running `main` loop would stay slow until the method was entered again.
- After round 1, `batch` gets a normal tier 4 compile (the line at 66 ms), and the OSR versions are retired. From then on calls go straight to optimised code. In this run C2 had not finished until after round 1, which is why rounds are noisy.

9.7.3 What C2 does: inlining and friends

The biggest optimisation is **inlining**: replacing a call with the body of the callee. Calls are cheap, but inlining removes the call and, more importantly, exposes the callee's code to the caller's optimiser, which then performs constant folding, dead-code removal, loop unrolling and **escape analysis** (the previous chapters showed how an object that does not escape can be scalar-replaced and never allocated). That is why small methods and getters cost nothing in hot code and why you should not hand-inline for speed.

Inlining decisions depend on size: methods up to `MaxInlineSize` (35 bytecodes) are inlined if they are called at all often; "hot" methods up to `FreqInlineSize` (325 bytecodes) are inlined too. Both are visible in `PrintFlagsFinal` (35 and 325 on JDK 21). A diagnostic flag lets you see the decisions:

Inlining decisions (diagnostic flag; output trimmed) TERMINAL

```
$ java -XX:+UnlockDiagnosticVMOptions -XX:+PrintInlining Warm
                            @ 11   Warm::score (12 bytes)   inline
                            @ 11   Warm::score (12 bytes)   inline (hot)
```

`score` was inlined into `batch`. Gigantic methods (over 8,000 bytecodes, `HugeMethodLimit`) are never compiled at all, one reason to keep methods short.

A second major technique is **speculative optimisation based on the profile**. If, at a call site `s.area()`, the profile shows that only one class has ever been the receiver (*monomorphic*), C2 compiles a direct, inlined call guarded by a cheap type check. Two types (*bimorphic*) are also inlined with two checks. Three or more (*megamorphic*) fall back to a virtual-table call (see dispatch in the OOP Part) which cannot be inlined. This is why interface-heavy code with many implementations at one call site is a little slower than the same code with one or two.

9.7.4 Deoptimisation

Speculation can turn out wrong. When it does, the JVM **deoptimises**: it throws the compiled code away (marks it `made not entrant`), moves the running frames back to the interpreter, and later recompiles with the new facts. Correctness is never at risk; only speed. Let us trigger it: a method that sees only `Square` for a long time and then starts seeing `Circle` as well.

Deopt.java JAVA

```java
public class Deopt {
    interface Shape { double area(); }
    record Square(double s) implements Shape { public double area() { return s * s; } }
    record Circle(double r) implements Shape { public double area() { return 3.14159 * r * r; } }

    static double total(Shape[] shapes) {
        double t = 0;
        for (Shape s : shapes) t += s.area();
        return t;
    }

    public static void main(String[] args) {
        Shape[] squares = new Shape[1000];
        for (int i = 0; i < squares.length; i++) squares[i] = new Square(i);
        double sink = 0;
        for (int i = 0; i < 20_000; i++) sink += total(squares);
        System.out.println("phase 1 done (only Square seen)");
        Shape[] mixed = new Shape[1000];
        for (int i = 0; i < mixed.length; i++) {
            mixed[i] = (i % 2 == 0) ? new Square(i) : new Circle(i);
        }
        for (int i = 0; i < 20_000; i++) sink += total(mixed);
        System.out.println("phase 2 done (Square and Circle)");
        System.out.println(sink > 0);
    }
}
```

Typical output (timestamps vary): C2 code is invalidated when Circle appears TERMINAL

```
$ javac Deopt.java && java -XX:+PrintCompilation Deopt | grep -E "Deopt::|phase"
  122   13 %     3       Deopt::total @ 11 (42 bytes)
  133   14       3       Deopt::total (42 bytes)
  133   15 %     4       Deopt::total @ 11 (42 bytes)
  140   16       4       Deopt::total (42 bytes)
  166   14       3       Deopt::total (42 bytes)   made not entrant
phase 1 done (only Square seen)
```

Typical output (timestamps vary): C2 code is invalidated when Circle appears (continued) — TERMINAL

```
  219   15 %     4       Deopt::total @ 11 (42 bytes)   made not entrant
  219   16       4       Deopt::total (42 bytes)   made not entrant
  219   20 %     3       Deopt::total @ 11 (42 bytes)
  219   21       3       Deopt::total (42 bytes)
  224   23 %     4       Deopt::total @ 11 (42 bytes)
  248   24       4       Deopt::total (42 bytes)
phase 2 done (Square and Circle)
```

Before "phase 1 done" `total` reaches tier 4 assuming every `Shape` is a `Square`. The moment a `Circle` arrives, the type check in the compiled code fails and both tier 4 versions are marked `made not entrant` at 219 ms. The method restarts in tier 3 (profiling again), and a new tier 4 version, compiled with the knowledge of both classes, appears 5 ms later. With `-Xlog:deoptimization=debug` you see the reason: `class_check`. Other causes: an `if` branch never taken before is taken, a class is loaded that invalidates an assumption (e.g. a new subclass overriding an inlined method), an unexpected null, or an array index out of range.

> **NOTE**
> A burst of deoptimisations right after start-up or after a deployment is normal. A steady stream of them in a long-running service is a clue that the program's behaviour keeps changing shape (a megamorphic call site or a flip-flopping branch).

9.7.5 Microbenchmark traps

Because of warm-up, inlining and dead-code elimination, **hand-written timing loops usually measure the wrong thing**. Look at this loop, which looks like it measures the cost of `Math.sqrt`:

Bad.java — JAVA

```java
public class Bad {
    public static void main(String[] args) {
        for (int round = 0; round < 5; round++) {
            long t0 = System.nanoTime();
            for (int i = 0; i < 100_000_000; i++) {
                Math.sqrt(i);                // result unused
            }
            System.out.println("loop took " + (System.nanoTime() - t0) / 1_000_000 + " ms");
        }
    }
}
```

Typical output on the build machine — TERMINAL

```
$ java Bad.java
loop took 72 ms
loop took 102 ms
loop took 0 ms
loop took 0 ms
loop took 0 ms
```

The "0 ms" is real: once C2 compiled the loop, it saw that the result is never used, that `Math.sqrt` has no side effects, and removed the whole loop (**dead-code elimination**). A hundred million square roots in under a millisecond is not a fast machine, it is no work at all. The first two rounds measured warm-up, the last three measured nothing.

The traps, in short:

- **Warm-up**: the first iterations run interpreted or C1 code. Discard them.
- **Dead-code elimination**: if the result is not consumed, the work may vanish. Consume results.
- **Constant folding**: inputs that are compile-time constants get computed at compile time.
- **Loop optimisation and OSR**: code in `main` behaves differently from the same code in a separately-called method.
- **GC and noise**: allocation rate, other processes, CPU frequency and, on a 2-CPU machine like this one, the JIT threads competing with your code.
- **Profile pollution**: running benchmark A before B changes the type profiles that B is compiled with.

The fix is a harness. **JMH** (Java Microbenchmark Harness, from the OpenJDK project) handles forks, warm-up iterations, measured iterations and a `Blackhole` that consumes results so the JIT cannot discard them. It is a separate dependency, so it is shown here as text only (not run in this book):

SqrtBenchmark.java (JMH, not run here) JAVA

```java
import org.openjdk.jmh.annotations.*;
import org.openjdk.jmh.infra.Blackhole;
import java.util.concurrent.TimeUnit;

@BenchmarkMode(Mode.AverageTime)
@OutputTimeUnit(TimeUnit.NANOSECONDS)
@Warmup(iterations = 5, time = 1)
@Measurement(iterations = 5, time = 1)
@Fork(2)
@State(Scope.Thread)
public class SqrtBenchmark {
    double x = 12345.678;                // a field, so it is not a compile-time constant

    @Benchmark
    public void sqrt(Blackhole bh) {
        bh.consume(Math.sqrt(x));
    }
}
```

Generate a project from the JMH Maven archetype (`org.openjdk.jmh:jmh-java-benchmark-archetype`), build the benchmark jar and run `java -jar target/benchmarks.jar`. Read the score together with its error margin; compare two variants only under identical settings. The performance chapter in the design Part repeats the rule: measure first, with a real tool.

WATCH OUT
Do not decide between two implementations from a `System.nanoTime()` loop in `main`. Either use JMH or measure the whole application under realistic load with a profiler or JFR.

9.7.6 Flags, code cache and the cost of warm-up

You rarely need JIT flags, but know what they are:

Flag	Effect
`-XX:+PrintCompilation`	Log each compile and invalidation (as above).
`-Xint`	Interpreter only; shows the cost of no JIT. Never for production.
`-XX:TieredStopAtLevel=1`	Use C1 only: faster start-up and lower CPU/memory, lower peak speed. Used for short-lived tools and some test runs.
`-XX:ReservedCodeCacheSize=N`	Size of the code cache (251658240 bytes, 240 MB, on this machine with tiered compilation).
`-XX:CICompilerCount=N`	Number of compiler threads (2 here, chosen from the CPU count).
`-Xlog:jit+compilation=debug`	Unified-logging form of compilation output.

JIT-related flags worth knowing.

The **code cache** is a native memory region that stores compiled code. If it fills up (reported in the log as `CodeCache is full. Compiler has been disabled`) the JVM stops compiling new methods and the application slows down permanently. This is rare in modern JDKs, which flush cold code, but it can bite programs that generate lots of classes.

Warm-up costs real money in two situations. **Short-lived processes** (command-line tools, serverless functions) may finish before C2 does any work. **Autoscaled and restarted services** pay the warm-up on every new instance, sending production traffic to code that runs at a fraction of its eventual speed; a gradual load-balancer ramp-up (or a small synthetic warm-up request set) helps.

9.7.7 Class Data Sharing, AOT and native images

Start-up has another cost: loading, verifying and parsing hundreds of classes. **Class Data Sharing (CDS)** stores the parsed, ready-to-use form of classes in a memory-mapped archive. The JDK ships an archive for its own classes (the log line `Opened archive .../classes.jsa` earlier in this Part's logs is exactly that). **Application CDS** adds your classes. On JDK 21 a *dynamic archive* is created at exit of a normal run (the classes must come from a jar):

Application CDS on JDK 21 TERMINAL

```
$ jar cfe warm.jar Warm Warm.class
$ java -XX:ArchiveClassesAtExit=warm.jsa -jar warm.jar
$ java -XX:SharedArchiveFile=warm.jsa -Xlog:class+load -jar warm.jar | grep "Warm "
[0.041s][info][class,load] Warm source: shared objects file (top)
```

The last line shows our class loaded from the shared archive. (When we ran the same experiment with a loose `.class` directory, the JVM printed `Skipping Warm: Unsupported location`; archived classes have to come from a jar or modules file.) Since JDK 19, `-XX:+AutoCreateSharedArchive -XX:SharedArchiveFile=app.jsa` creates and refreshes the archive automatically.

CDS shortens loading, not warm-up of the JIT. The newer **AOT cache** work from Project Leyden goes further: Java 24 and 25 added ahead-of-time class loading and linking (JEP 483), command-line ergonomics for creating the cache (JEP 514) and method profiling stored in the cache (JEP 515), so a later run can start compiling hot code sooner, and Java 26 allows the object cache with any collector (JEP 516). These need JDK 24 and later, so they are described here but not run on the build machine. The point for an engineer: a training run produces an archive that makes the next start faster, without changing your code.

GraalVM Native Image takes the opposite approach: it compiles your whole application ahead of time into a standalone executable. Start-up is in milliseconds and memory is small, which suits serverless functions and CLI tools. The costs: a closed-world assumption (everything reachable must be known at build time, so reflection, proxies and resources need configuration), long build times, no JIT so peak throughput is usually lower than HotSpot for long-running services, and a different set of diagnostic tools. Frameworks like Spring Boot and Quarkus have first-class support. We mention it for completeness; the rest of the book uses the standard JVM.

INTERVIEW ANGLE
Why is Java code slow at the start and fast later? What is a JIT? The JVM first interprets bytecode while counting method calls and loop iterations. Hot code is compiled to machine code by C1 and then, based on a collected profile, by C2, which inlines and speculates (for example that a call site has one receiver type) and falls back to the interpreter through deoptimisation if an assumption fails. The cost is warm-up: early requests run slower. Because of this, benchmarks need warm-up and a harness such as JMH.

Check yourself

1. What do the tiers 0, 3 and 4 stand for, and what does a `%` in `PrintCompilation` output mean?
2. A benchmark loop calls `Math.sqrt(i)` 100 million times and ignores the result, and reports 0 ms. Why?
3. What does "made not entrant" tell you?
4. A call site `shape.area()` has seen `Square` and `Circle` only. A third class `Triangle` starts to appear. What may the JIT do, and what is the effect on speed?
5. Name two situations where warm-up cost matters more than peak speed, and one mitigation for each.
6. What is the difference between CDS and the JIT, and between the JIT and GraalVM Native Image?

Answers

1. Tier 0 is the interpreter, 3 is C1 with full profiling, 4 is C2 (fully optimised). `%` marks an on-stack replacement compile of a loop inside a method that is already running.
2. After warm-up C2 sees that the result is unused and `Math.sqrt` has no side effects, and eliminates the loop (dead-code elimination). Consume the result, or use JMH with `Blackhole`.
3. The compiled version still exists for frames already running it, but no new calls will enter it; it was replaced by a better version or invalidated by deoptimisation.
4. The type profile becomes megamorphic (three or more receivers), so the call falls back to a virtual dispatch that cannot be inlined, and a compiled version based on the old profile may be deoptimised. Result: a modest slowdown.
5. Short-lived command-line tools or serverless functions (use CDS or a native image, or `TieredStopAtLevel=1`); freshly started service instances behind a load balancer (ramp traffic up gradually, or warm up with synthetic requests).
6. CDS speeds up class loading with a pre-parsed archive and leaves execution to the JVM; the JIT compiles hot bytecode to machine code while the program runs. Native Image compiles everything ahead of time to a standalone executable without a JVM or JIT.

KEY POINTS

Java is fast because of runtime compilation, and slow at first for the same reason.

- Tiered compilation: interpreter, then C1 (tier 3, profiles), then C2 (tier 4, optimised); the thresholds are about 200 and 5,000 invocations.
- C2 speculates on the profile (inlining monomorphic and bimorphic calls) and deoptimises when it is wrong; `-XX:+PrintCompilation` shows both.
- Never trust a hand-rolled timing loop: warm-up and dead-code elimination distort it. Use JMH.
- To start faster use CDS or AOT caches; to remove the JVM entirely use GraalVM Native Image, at a cost in flexibility and peak throughput.

PART 10

Runtime Features and Diagnostics

Reflection, proxies, class loaders, modules, native calls, and the tools you use when a running JVM misbehaves.

In this part

10.1 Reflection and Annotations
10.2 Dynamic Proxies and Method Handles
10.3 Class Loaders and ServiceLoader
10.4 The Module System (JPMS)
10.5 Calling Native Code: JNI and the Foreign Function and Memory API
10.6 Troubleshooting Playbook
10.7 The Diagnostics Toolbox

10.1 Reflection and Annotations

Most Java code knows its types at compile time: you write `account.deposit(50)` and the compiler checks that `deposit` exists. Frameworks cannot work that way. Spring must create your `@Service` classes, JUnit must find your `@Test` methods, and Jackson must read the fields of a class it has never seen. They all rely on **reflection**: the ability of a running program to inspect classes, fields, methods and constructors as objects, and to use them without naming them in source code.

Reflection is the foundation of almost every Java framework, so a backend developer meets it constantly, usually hidden behind an annotation. This chapter shows the API, what it costs, how modules restrict it, and how to write your own annotation and a reader for it.

10.1.1 The Class object and its members

Every loaded type has exactly one `java.lang.Class` object per class loader. You obtain it in three ways: the literal `Account.class`, the call `obj.getClass()`, or by name with `Class.forName("com.example.Account")`, which loads and initializes the class at run time. From a `Class` you can ask for its fields, methods, constructors, annotations, superclass, interfaces and modifiers.

There are two families of accessors, and confusing them is the most common beginner error:

Method	Returns	Includes inherited?
`getMethods()` / `getFields()`	public members only	yes, from superclasses and interfaces
`getDeclaredMethods()` / `getDeclaredFields()`	all members declared in this class, any access level	no
`getMethod(name, types)`	one public method	yes
`getDeclaredMethod(name, types)`	one method of this class, any access	no

The two families of reflective lookups.

The listing inspects a small class, creates an instance through its constructor, calls a public and a private method, and overwrites a `private final` field.

Inspect.java JAVA

```java
import java.lang.reflect.*;
import java.util.*;

public class Inspect {
    static class Account {
        private final String id;
        private long balance;
        public Account(String id, long balance) { this.id = id; this.balance = balance; }
        public long balance() { return balance; }
```

Inspect.java (continued) JAVA

```java
        public void deposit(long amount) { balance += amount; }
        private void audit(String msg) { System.out.println("audit: " + msg); }
        @Override public String toString() { return id + ":" + balance; }
    }

    public static void main(String[] args) throws Exception {
        Class<?> c = Account.class;
        System.out.println("name      = " + c.getName());
        System.out.println("simple    = " + c.getSimpleName());
        System.out.println("modifiers = " + Modifier.toString(c.getModifiers()));
        System.out.println("super     = " + c.getSuperclass().getName());

        System.out.println("-- fields");
        for (Field f : c.getDeclaredFields())
            System.out.println(Modifier.toString(f.getModifiers()) + " "
                    + f.getType().getSimpleName() + " " + f.getName());
        System.out.println("-- methods");
        Arrays.stream(c.getDeclaredMethods()).map(Method::getName).sorted()
              .forEach(n -> System.out.println(n));

        Constructor<?> k = c.getDeclaredConstructor(String.class, long.class);
        Object acc = k.newInstance("A-1", 100L);
        Method dep = c.getMethod("deposit", long.class);
        dep.invoke(acc, 50L);
        System.out.println(acc);

        Method audit = c.getDeclaredMethod("audit", String.class);
        audit.setAccessible(true);
        audit.invoke(acc, "hello");

        Field id = c.getDeclaredField("id");
        id.setAccessible(true);
        id.set(acc, "HACKED");
        System.out.println(acc);

        try {
            c.getMethod("nope");
        } catch (NoSuchMethodException e) {
            System.out.println("NoSuchMethodException: " + e.getMessage());
        }
        try {
            Method m = c.getMethod("deposit", long.class);
            m.invoke(acc, "text");
        } catch (IllegalArgumentException e) {
            System.out.println("IllegalArgumentException: " + e.getMessage());
        }
        Method boom = Inspect.class.getDeclaredMethod("boom");
        try { boom.invoke(null); } catch (InvocationTargetException e) {
            System.out.println("wrapped: " + e + " cause=" + e.getCause());
        }
    }
    static void boom() { throw new IllegalStateException("inside"); }
}
```

Output of Inspect.java on JDK 21 TERMINAL

```
$ java Inspect.java
name      = Inspect$Account
simple    = Account
modifiers = static
super     = java.lang.Object
-- fields
private final String id
private long balance
-- methods
audit
balance
deposit
toString
A-1:150
audit: hello
```

Output of Inspect.java on JDK 21 (continued) TERMINAL

```
HACKED:150
NoSuchMethodException: Inspect$Account.nope()
IllegalArgumentException: argument type mismatch
wrapped: java.lang.reflect.InvocationTargetException cause=java.lang.IllegalStateException: inside
```

What to notice. `getName()` of a nested class uses `$`. `getDeclaredMethods()` returns methods in no guaranteed order, so the demo sorts them. Primitive parameter types are looked up with `long.class`, not `Long.class`; `getMethod("deposit", Long.class)` would throw `NoSuchMethodException`. `Method.invoke` boxes arguments and results. And when the called method throws, you do not see the exception directly: reflection wraps it in an `InvocationTargetException`, and the real exception is `getCause()`. Frameworks always unwrap it, as the proxy example in the next chapter does.

10.1.2 Breaking encapsulation: setAccessible and modules

`setAccessible(true)` switches off the access check for that one `Method`, `Field` or `Constructor` object. That is how serialization libraries fill private fields and how Hibernate sets an entity's `id`. In the listing it even rewrote a `final` field. That works for ordinary instance fields; it does not work for `static final` fields, and it never works for record components or hidden classes.

Since Java 9 there is a second gate: **modules**. A package in a named module is reachable by reflection on private members only if the module declares the package as `opens` (the module chapter later in this Part explains the syntax). The JDK's own modules do not open their internals. Java 16 made this strict by default, so code that used to poke into `java.lang` now fails:

Hack.java JAVA

```java
import java.lang.reflect.Field;
public class Hack {
    public static void main(String[] args) throws Exception {
        Field f = String.class.getDeclaredField("value");
        try { f.setAccessible(true); System.out.println("opened"); }
        catch (Exception e) { System.out.println(e.getClass().getName() + ": " + e.getMessage()); }
        System.out.println(Hack.class.getModule() + " / " + String.class.getModule());
    }
}
```

Strong encapsulation of the JDK, and the escape hatch TERMINAL

```
$ java Hack.java
java.lang.reflect.InaccessibleObjectException: Unable to make field private final byte[]
  java.lang.String.value accessible: module java.base does not "opens java.lang"
  to unnamed module @1a052a00
unnamed module @1a052a00 / module java.base
$ java --add-opens java.base/java.lang=ALL-UNNAMED Hack.java
opened
unnamed module @145eaa29 / module java.base
```

(The module hash after `@` changes from run to run, and the error message is a single long line in reality; it is wrapped here.) The flag `--add-opens module/package=target` is the supported escape hatch. You will see it in the JVM arguments of older libraries on a modern JDK. Treat each one as technical debt: it means some library still depends on JDK internals.

WATCH OUT

The error `InaccessibleObjectException ... does not "opens ..." to unnamed module` after upgrading the JDK is not a bug in your code. A library you depend on is reflecting into a JDK or module-private package. First upgrade the library. Add `--add-opens` only as a stopgap, and keep the list short.

10.1.3 What reflection costs

Reflection is slower than a direct call for three reasons: the JVM must look up and check the member, arguments are boxed into an `Object[]`, and the JIT compiler cannot easily inline through a `Method` it does not know statically. Since Java 18 (JEP 416) `Method.invoke` is implemented on top of method handles instead of the old native accessors, which simplified the JDK internals and kept reflection about as fast as before. On the build machine a reflective call to a tiny `inc()` method took roughly 50 ns in a hot loop (typical), many times slower than a JIT-inlined direct call, which can cost well under a nanosecond.

Two practical rules follow. First, do the expensive work (`getDeclaredMethod`, `setAccessible`) once and cache the `Method` object; every framework keeps a cache of its reflection metadata. Second, in hot paths prefer a `MethodHandle` stored in a `static final` field (next chapter), or generated code, or no reflection at all. For a request handler that runs a few thousand times per second, the cost is invisible; in an inner loop over millions of items it is not.

INTERVIEW ANGLE
What is reflection, and what are its drawbacks? Reflection lets a program examine and modify its own structure at run time: load classes by name, list members, create objects and invoke methods without compile-time references. Drawbacks: slower than direct calls, no compile-time type checking (errors become run-time exceptions), it can break encapsulation via `setAccessible`, and it is restricted by the module system. Use it in framework and infrastructure code, not in ordinary business logic.

10.1.4 Annotations

An **annotation** is structured metadata attached to a program element. It does nothing by itself; some other piece of code (the compiler, a build-time tool, or a runtime library) reads it and acts. You declare one with `@interface`. Two meta-annotations control it:

- `@Retention` decides how long it survives: `SOURCE` (discarded by the compiler, like `@Override`), `CLASS` (kept in the `.class` file but not loaded by the JVM; the default), or `RUNTIME` (readable through reflection).
- `@Target` restricts where it may appear: `TYPE`, `FIELD`, `METHOD`, `PARAMETER`, `CONSTRUCTOR` and so on.

Elements of an annotation look like methods and may have defaults. Allowed types are primitives, `String`, `Class`, enums, other annotations, and arrays of those. Only `RUNTIME` annotations can be read by the code below, which implements a miniature version of what Bean Validation does.

MiniValidator.java JAVA

```
import java.lang.annotation.*;
import java.lang.reflect.*;
import java.util.*;

public class MiniValidator {
    @Retention(RetentionPolicy.RUNTIME)
    @Target(ElementType.FIELD)
    @interface NotBlank { String message() default "must not be blank"; }

    @Retention(RetentionPolicy.RUNTIME)
    @Target(ElementType.FIELD)
    @interface Range { int min() default 0; int max() default Integer.MAX_VALUE; }

    @Retention(RetentionPolicy.SOURCE)
    @interface OnlyInSource {}

    static class Student {
        @NotBlank String name;
        @Range(min = 16, max = 100) int age;
        @OnlyInSource String note = "x";
        Student(String name, int age) { this.name = name; this.age = age; }
    }

    static List<String> validate(Object o) throws IllegalAccessException {
        List<String> errors = new ArrayList<>();
        for (Field f : o.getClass().getDeclaredFields()) {
            f.setAccessible(true);
            Object v = f.get(o);
            NotBlank nb = f.getAnnotation(NotBlank.class);
            if (nb != null && (v == null || v.toString().isBlank()))
                errors.add(f.getName() + ": " + nb.message());
            Range r = f.getAnnotation(Range.class);
            if (r != null) {
                int n = (Integer) v;
                if (n < r.min() || n > r.max())
                    errors.add(f.getName() + ": " + n + " not in ["
                            + r.min() + ", " + r.max() + "]");
            }
            System.out.println(f.getName() + " annotations = "
                    + Arrays.toString(f.getAnnotations()));
        }
        return errors;
    }
```

MiniValidator.java (continued) JAVA

```java
    public static void main(String[] args) throws Exception {
        System.out.println(validate(new Student("Asha", 20)));
        System.out.println(validate(new Student(" ", 12)));
    }
}
```

Output of MiniValidator.java TERMINAL

```
$ java MiniValidator.java
name annotations = [@MiniValidator.NotBlank(message="must not be blank")]
age annotations = [@MiniValidator.Range(max=100, min=16)]
note annotations = []
[]
name annotations = [@MiniValidator.NotBlank(message="must not be blank")]
age annotations = [@MiniValidator.Range(max=100, min=16)]
note annotations = []
[name: must not be blank, age: 12 not in [16, 100]]
```

What to notice. The `SOURCE`-retention annotation on `note` is invisible at run time: its list is empty. The order of elements printed in `Range(max=100, min=16)` is not guaranteed. The pattern is the same in every framework: iterate over members, call `getAnnotation(X.class)`, branch on the result. Spring's `@Autowired`, JUnit's `@Test`, and JPA's `@Entity` are all read this way, usually once at start-up and cached.

Annotation processing at compile time

Annotations with `SOURCE` or `CLASS` retention can be consumed by an **annotation processor**: a class implementing `javax.annotation.processing.Processor`, discovered by the compiler (through `ServiceLoader`, see the class loader chapter) and run during `javac`. Processors can report errors and generate new source files, but cannot change existing ones. Lombok (which does change them, by using compiler internals), MapStruct and Dagger work this way. The benefit over run-time reflection is that the work happens at build time, so there is no start-up cost and no reflection at run time. That is why frameworks aiming at fast start-up and native images (Quarkus, Micronaut, Spring AOT) prefer build-time processing.

REAL WORLD
In a Spring Boot application, component scanning reads class-file annotations at start-up, creates beans by reflection through constructors, and injects dependencies into fields or constructor parameters. Slow start-up of large Spring apps is largely the cost of this reflective discovery, which Spring AOT moves to build time.

GOTCHA
Annotations on interfaces are not inherited by implementing classes, and annotations on a method are not inherited by an override. Only class-level annotations marked with `@Inherited` pass from a superclass to a subclass. Frameworks like Spring add their own search logic on top (`AnnotatedElementUtils`) to hide this.

Check yourself

1. Which call returns a private method declared in the class itself: `getMethod` or `getDeclaredMethod`? Which one also finds inherited public methods?
2. What does this print? `Method m = String.class.getMethod("length");` `System.out.println(m.invoke("hello"));`
3. A method called through `Method.invoke` throws `IllegalStateException`. What exception does the caller of `invoke` see, and how do you get the original?
4. Your custom annotation is never found by `field.getAnnotation(...)` although it is clearly present in the source. What is the most likely cause?
5. After moving to a newer JDK a library fails with `InaccessibleObjectException`. Name two ways to deal with it.
6. Why do build-time annotation processors give faster start-up than reflection-based scanning?

Answers

1. `getDeclaredMethod` finds a private method of that class. `getMethod` finds public methods only, but includes inherited ones.
2. It prints `5`: the result is boxed to `Integer`, printed by `println(Object)`.
3. An `InvocationTargetException`; call `getCause()` to get the `IllegalStateException`.

4. The annotation lacks `@Retention(RetentionPolicy.RUNTIME)`. The default is `CLASS`, which is not visible to reflection.
5. Upgrade the library to a version that does not reflect into JDK internals, or add `--add-opens module/package=ALL-UNNAMED` as a temporary workaround.
6. The metadata is processed and code generated during compilation, so at start-up there is nothing to scan or look up reflectively.

KEY POINTS

Reflection turns classes, methods and fields into objects you can inspect and call at run time; it powers every major framework.

- `getX` returns public members including inherited ones; `getDeclaredX` returns every member of that class only.
- `setAccessible(true)` bypasses access checks, but modules can still forbid it unless the package is `opens` or `--add-opens` is given.
- Cache `Method` and `Field` objects; unwrap `InvocationTargetException` to find the real error.
- An annotation is only readable at run time if it has `RUNTIME` retention; processors read `SOURCE`/`CLASS` annotations at compile time.

10.2 Dynamic Proxies and Method Handles

A **proxy** is an object that stands in for another object and intercepts calls to it. Logging every call to a service, opening a transaction around a method, checking permissions, caching results, and recording calls for a mock are all the same idea: do something before and after the real call, without editing the real class. Writing such a wrapper by hand for each interface is tedious, so Java can generate it at run time. This chapter covers `java.lang.reflect.Proxy`, then the lower-level and faster tools that the modern JVM uses internally: `MethodHandle` and `VarHandle`.

10.2.1 java.lang.reflect.Proxy

`Proxy.newProxyInstance(loader, interfaces, handler)` builds a class at run time that implements the given interfaces. Every method of those interfaces is routed to one method of your `InvocationHandler`: `invoke(Object proxy, Method method, Object[] args)`. You decide what happens: call the real object, return something else, throw.

Proxies.java JAVA

```
import java.lang.reflect.*;
import java.util.*;

public class Proxies {
    interface AccountService {
        long balance(String id);
        void deposit(String id, long amount);
    }
    static class RealService implements AccountService {
        private final Map<String, Long> data = new HashMap<>(Map.of("A-1", 100L));
        public long balance(String id) { return data.get(id); }
        public void deposit(String id, long amount) {
            if (amount <= 0) throw new IllegalArgumentException("amount must be positive");
            data.merge(id, amount, Long::sum);
        }
    }
    static class TimingHandler implements InvocationHandler {
        private final Object target;
        TimingHandler(Object target) { this.target = target; }
        @Override
        public Object invoke(Object proxy, Method method, Object[] args) throws Throwable {
            System.out.println("-> " + method.getName() + Arrays.toString(args));
            try {
                Object result = method.invoke(target, args);
                System.out.println("<- " + method.getName() + " = " + result);
                return result;
            } catch (InvocationTargetException e) {
                System.out.println("!! " + method.getName() + " threw " + e.getCause());
                throw e.getCause();
            }
        }
    }
    @SuppressWarnings("unchecked")
```

Proxies.java (continued) JAVA

```java
    static <T> T logged(T target, Class<T> iface) {
        return (T) Proxy.newProxyInstance(iface.getClassLoader(),
                new Class<?>[]{iface}, new TimingHandler(target));
    }
    public static void main(String[] args) {
        AccountService svc = logged(new RealService(), AccountService.class);
        svc.deposit("A-1", 25);
        System.out.println(svc.balance("A-1"));
        try { svc.deposit("A-1", -5); }
        catch (IllegalArgumentException e) { System.out.println("caught " + e.getMessage()); }
        System.out.println(svc.getClass().getName()
                + " isProxy=" + Proxy.isProxyClass(svc.getClass()));
        System.out.println(svc.getClass().getSuperclass().getName());
        System.out.println(svc.getClass().getInterfaces()[0].getSimpleName());
    }
}
```

Output of Proxies.java TERMINAL

```
$ java Proxies.java
-> deposit[A-1, 25]
<- deposit = null
-> balance[A-1]
<- balance = 125
125
-> deposit[A-1, -5]
!! deposit threw java.lang.IllegalArgumentException: amount must be positive
caught amount must be positive
$Proxy0 isProxy=true
java.lang.reflect.Proxy
AccountService
```

What to notice. The caller holds an `AccountService` and cannot tell it is talking to a proxy. The handler unwraps `InvocationTargetException` and rethrows the cause, so callers see the business exception, not a reflection wrapper. The generated class is named `$Proxy0`, extends `Proxy`, and implements only the interfaces you listed. That is the key limitation: **JDK proxies work on interfaces only**. You cannot proxy a concrete class this way, and you cannot cast the proxy to `RealService`.

Proxies in frameworks

- **Spring AOP** wraps beans in proxies to apply `@Transactional`, `@Cacheable`, `@Async` and security advice. If the bean implements an interface, Spring may use a JDK proxy; otherwise it generates a subclass at run time with a bytecode library (CGLIB, now repackaged inside Spring). Spring Boot defaults to subclass proxies. That explains two famous surprises: a `private` or `final` method is not advised (a subclass cannot override it), and **self-invocation** (`this.other()` inside the same bean) bypasses the proxy, so `@Transactional` on `other()` is silently ignored.
- **Mockito** creates mocks by generating subclasses (ByteBuddy) and routing calls to a recording handler: the same interception idea.
- **JPA/Hibernate** returns proxy objects for lazy associations; the first call on one triggers the database query.
- **Retrofit and Spring Data repositories** turn an interface with no implementation into a working client or DAO through a proxy.

GOTCHA
A call from inside the target object to its own method does not go through the proxy, because `this` refers to the real object, not the wrapper. If you need the advice to apply, call through the injected proxy (another bean) or restructure the code so that the annotated method lives in a different bean.

10.2.2 Method handles

A **MethodHandle** (`java.lang.invoke`) is a typed, directly executable reference to a method, constructor or field accessor. Unlike `Method`, the access check happens once, when you look the handle up with a `MethodHandles.Lookup`, not on each call. Because the JVM knows the exact signature, the JIT compiler can treat a handle held in a `static final` field like a direct call and inline it. The handle's type is a `MethodType`: return type plus parameter types.

Handles.java JAVA

```java
import java.lang.invoke.*;

public class Handles {
    static class Box {
        volatile int count;
        String name = "box";
        String hello(String who) { return name + " greets " + who; }
    }
    public static void main(String[] args) throws Throwable {
        MethodHandles.Lookup lookup = MethodHandles.lookup();
        MethodHandle concat = lookup.findVirtual(String.class, "concat",
                MethodType.methodType(String.class, String.class));
        String s = (String) concat.invokeExact("Java", "Handle");
        System.out.println(s);
        MethodHandle hello = lookup.findVirtual(Box.class, "hello",
                MethodType.methodType(String.class, String.class));
        System.out.println((String) hello.invokeExact(new Box(), "Asha"));
        MethodHandle bound = hello.bindTo(new Box());
        System.out.println((String) bound.invokeExact("Ravi"));
        MethodHandle max = lookup.findStatic(Math.class, "max",
                MethodType.methodType(int.class, int.class, int.class));
        MethodHandle max10 = MethodHandles.insertArguments(max, 0, 10);
        System.out.println((int) max10.invokeExact(3) + " " + (int) max10.invokeExact(42));
        System.out.println(hello.type());
        try { Object o = concat.invokeExact((Object) "a", (Object) "b"); }
        catch (WrongMethodTypeException e) {
            System.out.println("WrongMethodTypeException: " + e.getMessage());
        }

        VarHandle COUNT = MethodHandles.lookup().findVarHandle(Box.class, "count", int.class);
        Box b = new Box();
        COUNT.set(b, 5);
        boolean ok = COUNT.compareAndSet(b, 5, 6);
        boolean bad = COUNT.compareAndSet(b, 5, 7);
        int prev = (int) COUNT.getAndAdd(b, 10);
        System.out.println(ok + " " + bad + " prev=" + prev + " now=" + (int) COUNT.getVolatile(b));
        VarHandle ARR = MethodHandles.arrayElementVarHandle(long[].class);
        long[] arr = new long[3];
        ARR.setRelease(arr, 1, 99L);
        System.out.println((long) ARR.getAcquire(arr, 1));
    }
}
```

Output of Handles.java TERMINAL

```
$ java Handles.java
JavaHandle
box greets Asha
box greets Ravi
10 42
(Box,String)String
WrongMethodTypeException: handle's method type (String,String)String but found (Object,Object)Object
true false prev=6 now=16
99
```

What to notice.

- A virtual method handle takes the receiver as its first parameter: the type of `hello` is `(Box,String)String`.
- `invokeExact` demands that the call site's static types match the handle's type exactly, including the cast of the result. A mismatch fails at run time with `WrongMethodTypeException`. `invoke` is lenient and adapts types (boxing, widening) at some cost.
- Handles compose: `bindTo` fixes the receiver, `insertArguments` fixes arguments. The same building blocks implement lambdas (the `invokedynamic` instruction that you met in the Part on the JVM) and string concatenation.
- The compiler derives the call-site type from the static types of the arguments and from the cast on the result, so `Object o = h.invokeExact(...)` on a handle that returns `String` fails; write `(String) h.invokeExact(...)`.

Compared to reflection, handles are faster when cached in `static final` fields, but harder to write and with fewer conveniences (no `getDeclaredMethods`). Use reflection for discovery, handles for fast invocation once discovered.

10.2.3 VarHandle

A **VarHandle** (Java 9) is a typed reference to a variable: a field, an array element or a byte-buffer slot. It replaced `sun.misc.Unsafe` for most uses. It offers access *modes* with precisely defined memory semantics, which is what the concurrency Part describes as the Java memory model's building blocks: plain `get/set`, `getVolatile/setVolatile`, `getAcquire/setRelease` (cheaper ordering), `getOpaque/setOpaque`, and atomic read-modify-write operations `compareAndSet`, `getAndAdd`, `getAndSet`.

In the listing, `compareAndSet(b, 5, 6)` succeeds because the current value is 5; the second attempt with the stale expected value fails and returns `false`; `getAndAdd` returns the previous value (6) and leaves 16. This is exactly what `AtomicInteger` does internally, but without allocating a wrapper object per counter: `ConcurrentHashMap`, `AtomicReferenceArray` and `ConcurrentLinkedQueue` use VarHandles on their own fields. In application code you should normally use `AtomicInteger`, `LongAdder` or a lock; reach for a `VarHandle` only when writing a lock-free data structure.

INTERVIEW ANGLE

Reflection vs. method handles vs. dynamic proxies: how do they differ? Reflection (`Method`) inspects and calls members, checking access on each call and boxing arguments. A `MethodHandle` is a typed, pre-resolved callable that the JIT can inline when held in a `static final`. A dynamic proxy is a generated class implementing interfaces whose methods all delegate to an `InvocationHandler`, used to intercept calls. Proxies often use reflection or handles internally to reach the real target.

Check yourself

1. Can `Proxy.newProxyInstance` create a proxy of the concrete class `ArrayList`? Why or why not?
2. In a Spring bean, method `a()` calls `this.b()`, and `b()` is annotated `@Transactional`. Is a transaction started for `b()`? Why?
3. What does this do? `MethodHandle h = lookup.findVirtual(String.class, "length", MethodType.methodType(int.class)); Object o = h.invokeExact("abc");`
4. Which exception type should a proxy handler rethrow after `method.invoke(target, args)` fails with `InvocationTargetException`?
5. Name the VarHandle operation that atomically changes a value only if it still equals an expected one.

Answers

1. No. JDK proxies can only implement interfaces; for classes you need a subclass-generating library such as CGLIB or ByteBuddy.
2. No. The call is made on `this`, the raw target, so it bypasses the proxy that applies the transaction advice.
3. It throws `WrongMethodTypeException` at run time: the call site's type is `(String)Object` but the handle is `(String)int`. Cast the result: `int n = (int) h.invokeExact("abc");`.
4. The cause (`e.getCause()`), so the caller sees the original business exception instead of a reflection wrapper.
5. `compareAndSet(...)` (or `weakCompareAndSet`, `compareAndExchange`).

KEY POINTS

A dynamic proxy is a run-time generated implementation of interfaces whose calls all go to an `InvocationHandler`.

- JDK proxies need interfaces; class proxies (Spring, Mockito) generate subclasses, so `final` and `private` methods and self-invocation escape them.
- Always unwrap `InvocationTargetException` in handlers.
- `MethodHandle` is a typed, access-checked-once callable the JIT can inline; `invokeExact` requires exact static types.
- `VarHandle` gives typed access to fields and array elements with explicit memory-ordering modes and atomic operations.

10.3 Class Loaders and ServiceLoader

The Part on the JVM introduced class loading as the first stage of a class's life: the JVM finds the bytes, defines a `Class`, links and initializes it. This chapter goes one level deeper. You will see that a class is identified by its name **and** the loader that defined it, write a custom loader, use `ServiceLoader` to discover implementations without hard-coding them, and understand why application servers and plugin systems depend on all this.

10.3.1 The loaders and the delegation rule

Since Java 9 a running JVM has three built-in loaders arranged as a chain:

Loader	Loads	Notes
Bootstrap	`java.base` and other core modules	native code; `getClassLoader()` returns `null`
Platform	other JDK modules (`java.sql`, `java.net.http`, ...)	parent is bootstrap
Application (system)	your class path and module path	parent is platform

Built-in class loaders (Java 9 and later).

Before Java 9 the middle one was the extension loader (`lib/ext`), which was removed. The rule that makes the system predictable is **parent delegation**: when asked to load a class, a loader first asks its parent, and only if the parent cannot find it does the loader try itself. The consequence is that `java.lang.String` always comes from the bootstrap loader, even if a jar on your class path contains its own `java/lang/String.class`. Nobody can replace core classes by putting a look-alike on the class path.

```
loadClass("com.example.Order")                      AppClassLoader
   1. already loaded? no                                 |
   2. ask parent  ------------------------------>  PlatformClassLoader
                                                         |  ask parent
                                                         v
                                                  Bootstrap  -> not found
                                                         |
                                        platform: not in my modules -> not found
   3. parent failed -> findClass(): search class path -> define the class
```

The identity rule is the part interviewers like: **a class is identified by its fully qualified name plus its defining loader**. Two loaders can each load `Plugin` from the same bytes and the result is two unrelated `Class` objects with separate static state; an instance of one cannot be cast to the other.

10.3.2 Writing a custom class loader

You rarely need your own loader, but writing one once makes the model concrete. To write one, extend `ClassLoader` and override `findClass(String name)`: read the bytes, call `defineClass`. That keeps parent-first delegation. The demo overrides `loadClass` to be **child-first** (try my own directory first), which is what servlet containers do for web applications, so that each app can bring its own library versions. Names starting with `java.` must still go to the parent: `defineClass` refuses to define classes in `java.*`.

The setup uses two directories so that the `Plugin` class is compiled on its own:

Build and run the loader demo TERMINAL

```
$ mkdir plugin-src app
$ # Plugin.java goes to plugin-src, LoaderDemo.java to app
$ javac -d plugins plugin-src/Plugin.java
$ javac -cp plugins -d out app/LoaderDemo.java
```

plugin-src/Plugin.java JAVA

```
public class Plugin {
    public String hello() {
        return "Plugin loaded by " + getClass().getClassLoader().getClass().getSimpleName();
    }
}
```

app/LoaderDemo.java JAVA

```java
import java.io.IOException;
import java.lang.ref.WeakReference;
import java.nio.file.*;

public class LoaderDemo {
    /** Child-first loader: tries one directory before asking its parent. */
    static class DirLoader extends ClassLoader {
        private final Path dir;
        DirLoader(Path dir, ClassLoader parent) {
            super("dir-loader", parent);
            this.dir = dir;
        }

        @Override
        protected Class<?> loadClass(String name, boolean resolve)
                throws ClassNotFoundException {
            synchronized (getClassLoadingLock(name)) {
                if (name.startsWith("java.")) return super.loadClass(name, resolve);
                Class<?> c = findLoadedClass(name);
                if (c == null) {
                    try { c = findClass(name); }
                    catch (ClassNotFoundException e) { return super.loadClass(name, resolve); }
                }
                if (resolve) resolveClass(c);
                return c;
            }
        }

        @Override
        protected Class<?> findClass(String name) throws ClassNotFoundException {
            Path file = dir.resolve(name.replace('.', '/') + ".class");
            try {
                byte[] bytes = Files.readAllBytes(file);
                return defineClass(name, bytes, 0, bytes.length);
            } catch (IOException e) {
                throw new ClassNotFoundException(name);
            }
        }
    }

    public static void main(String[] args) throws Exception {
        System.out.println("-- loader chain of this class");
        for (ClassLoader l = LoaderDemo.class.getClassLoader(); l != null; l = l.getParent())
            System.out.println(l.getName() + " : " + l.getClass().getName());
        System.out.println("String loader = " + String.class.getClassLoader());

        Path dir = Path.of("plugins");
        DirLoader loader = new DirLoader(dir, LoaderDemo.class.getClassLoader());
        Class<?> c1 = loader.loadClass("Plugin");
        Object p = c1.getDeclaredConstructor().newInstance();
        System.out.println(c1.getMethod("hello").invoke(p));
        Class<?> c2 = new DirLoader(dir, null).loadClass("Plugin");
        System.out.println("same name, same Class? " + (c1 == c2));
        System.out.println(c1.getClassLoader().getName());

        try {
            Plugin direct = (Plugin) p;
        } catch (ClassCastException e) {
            System.out.println("ClassCastException: " + e.getMessage().substring(0, 70) + "...");
        }

        WeakReference<ClassLoader> ref = new WeakReference<>(loader);
        loader = null; c1 = null; p = null;
        for (int i = 0; i < 5 && ref.get() != null; i++) { System.gc(); Thread.sleep(100); }
        System.out.println("loader collected? " + (ref.get() == null));
    }
}
```

Output (typical; the last line depends on the collector having run) TERMINAL

```
$ java -cp out:plugins LoaderDemo
-- loader chain of this class
app : jdk.internal.loader.ClassLoaders$AppClassLoader
platform : jdk.internal.loader.ClassLoaders$PlatformClassLoader
String loader = null
Plugin loaded by DirLoader
same name, same Class? false
dir-loader
ClassCastException: class Plugin cannot be cast to class Plugin (Plugin is in unnamed modu...
loader collected? true
```

What to notice.

- The chain stops at the platform loader: the bootstrap loader is represented by `null`, and `String.class.getClassLoader()` is `null`.
- `Plugin` is on the class path too (`-cp out:plugins`), so the *application* loader could load it. Our child-first loader loaded its own copy, so casting to `Plugin` in `main` (which the application loader resolved) fails with the famous message *class X cannot be cast to class X*. When you see this message in a server, the same class has been loaded by two loaders.
- Dropping the loader, its classes and its instances made the loader collectable. This is how **class unloading** works: classes are unloaded only when their defining loader becomes unreachable, never individually. A static field, a thread, a `ThreadLocal` value or a registered shutdown hook that still points into the loader keeps all of its classes alive. That is the classic **Metaspace leak** on redeploys of web applications: after ten redeploys you have ten copies of every class.

GOTCHA

`Thread.getContextClassLoader()` is a different thing from the defining loader of a class. Frameworks (JNDI, JDBC drivers, ServiceLoader, logging) that sit in a shared parent loader use the context loader to find classes of the application that called them. If that fails in a thread pool or a custom thread, the symptom is `ClassNotFoundException` for a class that is plainly on the class path.

10.3.3 ServiceLoader and plugin architecture

How do you let someone else add an implementation of your interface without your code naming it? `java.util.ServiceLoader` is the JDK's answer: you define a **service interface**; each provider jar declares its implementation; the application asks `ServiceLoader.load(Service.class)` and iterates over what it finds. JDBC drivers, `javax.script` engines, charsets providers and logging backends use this mechanism.

On the class path a provider jar declares itself in a text file named after the interface: `META-INF/services/<fully.qualified.InterfaceName>`, one implementation class per line. On the module path the declaration is in `module-info.java`: `provides com.example.spi.Greeter with com.example.impl.EnglishGreeter;` in the provider module and `uses com.example.spi.Greeter;` in the consumer. Providers need a public no-argument constructor (or a public static `provider()` method in modules).

com/example/Main.java JAVA

```java
package com.example;

import com.example.spi.Greeter;
import java.util.ServiceLoader;

public class Main {
    public static void main(String[] args) {
        ServiceLoader<Greeter> loader = ServiceLoader.load(Greeter.class);
        for (Greeter g : loader) {
            System.out.println(g.language() + " -> " + g.greet("Asha")
                    + "  [" + g.getClass().getName() + "]");
        }
        System.out.println("found: " + loader.stream().count());
    }
}
```

With `Greeter` (an interface with `language()` and `greet(String)`) in `com.example.spi`, two implementations `EnglishGreeter` and `HindiGreeter` in `com.example.impl`, and the services file below, the result is:

Using META-INF/services (class path) TERMINAL

```
$ cat out/META-INF/services/com.example.spi.Greeter
com.example.impl.EnglishGreeter
com.example.impl.HindiGreeter
$ javac -d out $(find src -name '*.java')
$ java -cp out com.example.Main
en -> Hello, Asha  [com.example.impl.EnglishGreeter]
hi -> Namaste, Asha  [com.example.impl.HindiGreeter]
found: 2
```

Providers are instantiated **lazily**, one per loop iteration; `stream()` lets you inspect provider types without instantiating them, and `findFirst()` returns the first one. `ServiceLoader` is not thread-safe, and the iterator caches what it has found. Since the services file is just a resource, adding a jar to the class path is enough to add a plugin: no recompiling, no registration code. Spring Boot's auto-configuration uses the same idea with its own file (`META-INF/spring/...AutoConfiguration.imports`).

REAL WORLD

A plugin architecture for a backend product (payment gateways, report exporters, storage backends) is typically: an `api` module with the service interface, one jar per provider, and the host loading providers either with `ServiceLoader` on the class path or each in its own `URLClassLoader`. The second option isolates dependency versions between plugins and lets you unload a plugin by dropping its loader; the first is simpler and is enough for most apps.

INTERVIEW ANGLE

Can the same class be loaded twice in one JVM? What is the parent delegation model and why does it exist? Yes: a class is identified by name plus defining loader, so two loaders produce two distinct `Class` objects (and `ClassCastException` between them). Parent delegation makes a loader ask its parent first, which guarantees that core classes always come from the bootstrap loader and cannot be spoofed, and that a shared class is loaded once at the highest level. Containers deliberately break it (child-first) for web applications.

Check yourself

1. What does `String.class.getClassLoader()` return, and why?
2. Two class loaders each load class `Plugin` from the same `.class` file. Is `c1 == c2`? Can an instance of one be assigned to a variable of the other's type?
3. What is wrong here? The class `Config` has `static final Map<String, Object> CACHE`, is loaded by a web-app loader, and a daemon thread started by a library inside that app keeps running after undeploy.
4. Where does a provider jar declare its implementation of `com.example.spi.Greeter` on the class path?
5. Why does `defineClass` for a name starting with `java.` fail even in a custom loader?

Answers

1. `null`: the bootstrap loader is implemented in native code and has no `ClassLoader` object.
2. `c1 != c2`. The instance cannot be assigned or cast; the result is a `ClassCastException` saying class `Plugin` cannot be cast to class `Plugin`.
3. The running thread references the loader (through its classes), so the loader and all its classes, including `Config.CACHE`, can never be unloaded: a Metaspace leak on every redeploy. Stop threads on undeploy.
4. In a resource file `META-INF/services/com.example.spi.Greeter` listing the fully qualified class names, one per line.
5. The JVM prohibits user-defined loaders from defining classes in `java.*` packages (`SecurityException: Prohibited package name: java.lang`); core classes must come from the bootstrap loader.

KEY POINTS

A class's identity is its name plus its defining class loader; delegation to the parent first keeps core classes trustworthy.

- Loaders form a chain: application, platform, bootstrap (shown as `null`); a custom loader overrides `findClass` (parent-first) or `loadClass` (child-first).
- "X cannot be cast to X" means two loaders loaded X.
- Classes are unloaded only together with their loader; anything that still references it causes a Metaspace leak.
- `ServiceLoader` discovers implementations of an interface from `META-INF/services` files or `provides ... with` in `module-info.java`.

10.4 The Module System (JPMS)

Before Java 9 every public class on the class path was visible to every other class. Libraries had to put internal helpers in public classes just so other packages in the same jar could use them, and then users depended on them. The JDK itself suffered most: `sun.misc.Unsafe` and thousands of internal classes were public, so a JDK upgrade broke applications. The Java Platform Module System (JPMS, Project Jigsaw, Java 9) fixes this at the language level with two ideas: **reliable configuration** (a module says what it needs, and the JVM checks at start-up that everything is present) and **strong encapsulation** (a module says what it exposes; everything else is inaccessible, even via reflection).

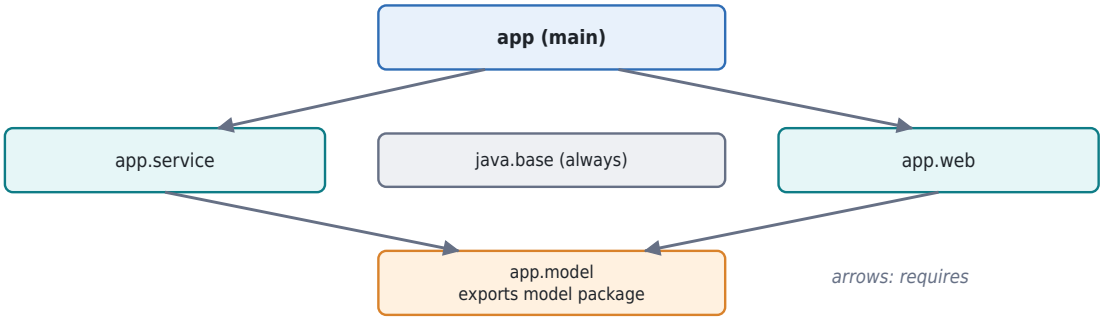


Figure 10.1. The module graph of a modular application: arrows are requires edges; only the exported packages of a module are visible to the modules that read it.

10.4.1 module-info.java

A **module** is a named group of packages described by a file `module-info.java` at the root of the source tree. These are the directives you need:

Directive	Meaning
`requires m;`	this module reads module `m`; compile and run need it
`requires transitive m;`	anyone who reads me also reads `m` (used when `m`'s types appear in my API)
`exports p;`	public types of package `p` are accessible to all modules
`exports p to a, b;`	accessible only to the named modules (qualified export)
`opens p;`	allows deep reflection (private members) into `p` at run time
`provides S with I;`	this module supplies implementation `I` of service `S`
`uses S;`	this module consumes `ServiceLoader.load(S.class)`

Core module directives.

Every module implicitly requires `java.base`, which holds `java.lang`, `java.util`, `java.io`, `java.nio` and friends. Other JDK modules are named, for example `java.sql`, `java.net.http`, `java.logging`; `java --list-modules` shows them all. Note that *accessible* needs both a `requires` and an `exports`: reading a module does not give you its non-exported packages, and `public` on a class inside a non-exported package means "public within the module only".

10.4.2 A tiny modular application

The demo has two modules. `com.example.greet` exports one package and keeps another, `internal`, hidden. `com.example.app` requires it.

```
mod/
  greeter/module-info.java
  greeter/com/example/greet/Greeter.java
  greeter/com/example/greet/internal/Templates.java
  app/module-info.java
  app/com/example/app/Main.java
```

greeter/module-info.java JAVA

```
module com.example.greet {
    exports com.example.greet;
}
```

greeter/com/example/greet/Greeter.java JAVA

```java
package com.example.greet;

import com.example.greet.internal.Templates;

public class Greeter {
    public String greet(String name) {
        return Templates.HELLO.formatted(name);
    }
}
```

greeter/com/example/greet/internal/Templates.java JAVA

```java
package com.example.greet.internal;

public final class Templates {
    private Templates() {}
    public static final String HELLO = "Hello, %s!";
}
```

app/module-info.java JAVA

```java
module com.example.app {
    requires com.example.greet;
}
```

app/com/example/app/Main.java JAVA

```java
package com.example.app;

import com.example.greet.Greeter;

public class Main {
    public static void main(String[] args) throws Exception {
        System.out.println(new Greeter().greet("Asha"));
        Module m = Greeter.class.getModule();
        System.out.println(m + " exports greet? " + m.isExported("com.example.greet"));
        System.out.println("exports internal? " + m.isExported("com.example.greet.internal"));
        Class<?> t = Class.forName("com.example.greet.internal.Templates");
        try {
            System.out.println(t.getField("HELLO").get(null));
        } catch (IllegalAccessException e) {
            System.out.println("IllegalAccessException: " + e.getMessage());
        }
    }
}
```

Compile each module into its own output directory, using `-p` (`--module-path`) to tell `javac` where required modules are, then run with `-m module/mainclass`:

Compile and run a modular app (JDK 21) TERMINAL

```
$ javac -d out/greet $(find greeter -name '*.java')
$ javac -p out/greet -d out/app $(find app -name '*.java')
$ java -p out/greet:out/app -m com.example.app/com.example.app.Main
Hello, Asha!
module com.example.greet exports greet? true
exports internal? false
IllegalAccessException: class com.example.app.Main (in module com.example.app) cannot access
 class com.example.greet.internal.Templates (in module com.example.greet) because module
 com.example.greet does not export com.example.greet.internal to module com.example.app
$ java --describe-module com.example.greet -p out/greet
com.example.greet file:///tmp/w13/mod/out/greet/
exports com.example.greet
requires java.base mandated
contains com.example.greet.internal
```

(The `IllegalAccessException` message is a single long line in reality; it is wrapped here.) **What to notice.** Reflection saw the class (`Class.forName` works: it is a loading question) but could not read its field, because the package is not exported. If you import `com.example.greet.internal.Templates` directly in `Main`, the compile fails with "package com.example.greet.internal is not visible". The `contains` line in `--describe-module` is a package that exists but is not exported. A deliberate escape hatch exists for the tests of legacy libraries: `--add-opens com.example.greet/com.example.greet.internal=com.example.app` made the same run print `Hello, %s!`.

Other things the module system checks at start-up: a missing `requires` fails immediately with `FindException: Module X not found`, rather than a `NoClassDefFoundError` an hour later; **two modules exporting the same package** (a split package) are rejected; and cyclic requires are illegal.

10.4.3 Unnamed and automatic modules: the migration story

Code on the class path lives in the **unnamed module**, which reads every module and exports everything. A plain jar placed on the *module path* becomes an **automatic module**: its name is derived from the `Automatic-Module-Name` manifest entry or the file name, it exports and opens all packages, and it reads all modules. These two rules let a modular application use non-modular libraries and let you migrate one jar at a time. A good library at least sets `Automatic-Module-Name` in its manifest.

10.4.4 jlink: a trimmed runtime

Because modules declare their dependencies, tools can compute exactly which JDK modules an application needs. `jdeps --print-module-deps app.jar` lists them, and `jlink` builds a custom runtime image containing only those modules plus your own:

jlink on the demo (sizes typical) TERMINAL

```
$ jar --create --file mlib/greet.jar -C out/greet .
$ jar --create --file mlib/app.jar --main-class com.example.app.Main -C out/app .
$ jdeps --module-path mlib --print-module-deps mlib/app.jar
com.example.greet,java.base
$ jlink --module-path mlib --add-modules com.example.app --output image \
    --strip-debug --compress zip-6 --no-header-files --no-man-pages
$ image/bin/java --list-modules
com.example.app
com.example.greet
java.base@21.0.12.1
$ du -sh image
39M     image
$ image/bin/java -m com.example.app/com.example.app.Main
Hello, Asha!
module com.example.greet exports greet? true
```

The image (39 MB here, against about 286 MB for the full JDK directory on this machine) is a self-contained runtime you can copy to a container without installing a JDK. `jlink` needs the `jmods` directory of the JDK, which not every distribution ships. The `--compress zip-6` option compresses the image; check `jlink --help` for the exact syntax of your JDK version. `jpackage` goes one step further and wraps an image into an installer; the tooling Part covers packaging.

10.4.5 Why most applications still use the class path

JPMS is used heavily inside the JDK, but most backend applications are not modular. Reasons that are worth stating honestly: the Spring ecosystem, Hibernate and most libraries rely on deep reflection, which requires `opens` everywhere or automatic modules; build tools and IDEs add friction; split packages and `Automatic-Module-Name` gaps make mixed graphs fragile; and a Spring Boot fat jar already gives you a runnable artifact. What every Java developer does feel from JPMS is its consequences: the `--add-opens` errors from the reflection chapter, `jlink`/`jdeps`, the slimmed JDK, and the end of `sun.misc.Unsafe` abuse. Modularity is most valuable for libraries with a clear public API and for runtime images.

> **INTERVIEW ANGLE**
> What do `exports` and `opens` mean, and how do they differ? `exports p` makes the public types of package `p` available at compile time and run time to other modules. `opens p` additionally permits reflection on all members of `p`, including private ones, at run time only. An `exports` without `opens` lets you call public API but not reflect into private fields; frameworks like Hibernate or Jackson need `opens` (or the module declared `open module`).

Check yourself

1. Which module is required by every module without being declared?
2. A class is `public` but lives in a package the module does not export. Who can use it?
3. What is the difference between `requires` and `requires transitive`?
4. What is a split package, and what does the module system do about it?

5. What does this print when run on the class path (no `module-info`): `System.out.println(Main.class.getModule().isNamed());`?
6. Why can a `jlink` image be much smaller than a JDK installation?

Answers

1. `java.base`.
2. Only code inside the same module. Other modules cannot compile against it, and reflection into it is also refused.
3. With `requires transitive m`, any module that requires mine automatically reads `m` too. Use it when `m`'s types appear in your public signatures.
4. The same package name exported by two different modules. The module system refuses to start (or compile) such a configuration to avoid ambiguity.
5. `false`: class-path code is in the unnamed module.
6. It contains only the modules your application (transitively) requires, found with `jdeps`, instead of all ~70 JDK modules (69 on this JDK 21 build).

KEY POINTS

JPMS adds named modules whose `module-info.java` declares dependencies (`requires`) and what they expose (`exports`, `opens`).

- Strong encapsulation: public no longer means visible; only exported packages are accessible, also to reflection unless `opens`.
- `java -p dir -m module/Main` runs a modular app; missing modules fail at start-up, not later.
- Class-path code is in the unnamed module; plain jars on the module path are automatic modules; both ease migration.
- `jdeps` and `jlink` build minimal runtime images; most Spring-style applications still run on the class path.

10.5 Calling Native Code: JNI and the Foreign Function and Memory API

Java runs inside the JVM, but real systems sometimes need things Java does not provide: a C library for image processing or compression, an operating-system call, a GPU driver, or memory laid out exactly as a native struct. Crossing from Java to native code (and back) has been possible since Java 1.1 through the Java Native Interface. Since Java 22 there is a modern replacement, the Foreign Function and Memory (FFM) API, which this chapter focuses on. You will probably rarely write either, but you should know why they exist, what they cost, and why the old way is being phased out.

10.5.1 JNI: the old way

With **JNI** you declare a method `native` in Java, generate a C header with `javac -h`, implement the function in C or C++ following a naming convention (`Java_com_example_Foo_bar`), compile it into a shared library (`.so`, `.dll`, `.dylib`), and load it with `System.loadLibrary("foo")`.

JAVA

```
public class Checksum {
    static { System.loadLibrary("checksum"); }   // finds libchecksum.so / checksum.dll
    public static native long crc(byte[] data);  // implemented in C
}
```

JNI works and is used inside the JDK itself, but it has real costs: you must write and build C code for every platform; mistakes in native code (a bad pointer, a forgotten `DeleteLocalRef`) crash the whole JVM with a segmentation fault instead of throwing an exception; data must be copied between Java arrays and native memory or pinned; and the glue code is boilerplate. These are the reasons the platform needed something better.

10.5.2 The FFM API

The FFM API (package `java.lang.foreign`) lets pure Java code do two things:

- **Call native functions** ("downcalls") in libraries already on the machine, without writing any C or JNI glue.
- **Allocate, read and write native memory** safely, outside the Java heap, with bounds checks and a defined lifetime.

It was incubated in Java 14 to 18, previewed in 19 to 21, and became final and permanent in **Java 22** (JEP 454). The main types:

Type	Role
`Linker`	links Java to native code; `Linker.nativeLinker()` is the platform's C ABI
`SymbolLookup`	finds a function address by name; `linker.defaultLookup()` covers the C standard library
`FunctionDescriptor`	the C signature: return layout and parameter layouts
`ValueLayout`	describes primitive and pointer shapes: `JAVA_INT`, `JAVA_LONG`, `ADDRESS`
`MemorySegment`	a contiguous region of memory (native or heap) with a size, so every access is bounds-checked
`Arena`	owns segments and decides their lifetime; closing the arena frees them

Main types of the FFM API.

A downcall is a `MethodHandle` (see the earlier chapter on dynamic proxies and method handles). You build it once from a symbol and a descriptor, then call it with `invokeExact`.

10.5.3 Running it on JDK 21

On JDK 21 the API is still a preview (third preview, JEP 442), so the program below needs `--enable-preview`. The shape of the code is the same as in Java 22+; only some method names changed (see the box after the output). The program calls `strlen` and `getpid` from the C library, fills a native `int` array, and shows what happens when a segment outlives its arena.

NativeDemo.java (preview API of JDK 21) JAVA

```
import java.lang.foreign.*;
import java.lang.invoke.MethodHandle;

public class NativeDemo {
    public static void main(String[] args) throws Throwable {
        Linker linker = Linker.nativeLinker();
        SymbolLookup stdlib = linker.defaultLookup();

        MethodHandle strlen = linker.downcallHandle(
                stdlib.find("strlen").orElseThrow(),
                FunctionDescriptor.of(ValueLayout.JAVA_LONG, ValueLayout.ADDRESS));
        MethodHandle getpid = linker.downcallHandle(
                stdlib.find("getpid").orElseThrow(),
                FunctionDescriptor.of(ValueLayout.JAVA_INT));

        try (Arena arena = Arena.ofConfined()) {
            MemorySegment cString = arena.allocateUtf8String("Hello, native world");
            long len = (long) strlen.invokeExact(cString);
            System.out.println("strlen = " + len);

            MemorySegment ints = arena.allocateArray(ValueLayout.JAVA_INT, 5);
            for (int i = 0; i < 5; i++) ints.setAtIndex(ValueLayout.JAVA_INT, i, i * i);
            int sum = 0;
            for (int i = 0; i < 5; i++) sum += ints.getAtIndex(ValueLayout.JAVA_INT, i);
            System.out.println("sum of squares = " + sum + ", segment bytes = " + ints.byteSize());
        }
        int pid = (int) getpid.invokeExact();
        System.out.println("same pid as ProcessHandle? " + (pid == ProcessHandle.current().pid()));

        MemorySegment leaked;
        try (Arena a = Arena.ofConfined()) { leaked = a.allocate(8); }
        try { leaked.get(ValueLayout.JAVA_BYTE, 0); }
        catch (IllegalStateException e) { System.out.println("after close: " + e.getMessage()); }
    }
}
```

Run on JDK 21 (Linux) TERMINAL

```
$ java --enable-preview --source 21 --enable-native-access=ALL-UNNAMED NativeDemo.java
Note: NativeDemo.java uses preview features of Java SE 21.
Note: Recompile with -Xlint:preview for details.
strlen = 19
sum of squares = 30, segment bytes = 20
same pid as ProcessHandle? true
after close: Already closed
```

What to notice.

- `strlen` takes a C `char *`, so the descriptor says `ADDRESS`; the Java `MemorySegment` holding a NUL-terminated UTF-8 string is passed directly. No copying, no C code.
- The `try (Arena ...)` block is the lifetime: when it ends, the native memory is freed. A `confined` arena may be used only by the thread that created it. Use `Arena.ofShared()` for several threads, and `Arena.global()` for memory that is never freed.
- The last test is the safety story. In C, reading freed memory is undefined behaviour and may crash or silently succeed. With FFM the access throws `IllegalStateException` ("Already closed"); out-of-range offsets throw `IndexOutOfBoundsException`. Native memory is still outside the garbage collector, but its lifetime is explicit and checked.
- Without `--enable-native-access` the JVM prints a warning, because calling native code can crash the process: it is a *restricted* operation. Libraries should be granted it per module (`--enable-native-access=my.module`), applications on the class path use `ALL-UNNAMED`.

NOTE
In Java 22 and later (final API, not run here since only JDK 21 is installed), the same program drops `--enable-preview` and renames a few methods: `arena.allocateUtf8String(s)` becomes `arena.allocateFrom(s)`, and `arena.allocateArray(layout, n)` becomes `arena.allocate(layout, n)`. Everything else (`Linker`, `FunctionDescriptor`, `ValueLayout`, `downcallHandle`, `invokeExact`) keeps its shape. Check the Javadoc of your JDK when porting.

10.5.4 Tooling, struct layouts and upcalls

Writing descriptors by hand is fine for one function and miserable for a library with hundreds. The **jextract** tool (distributed separately from the JDK) reads a C header and generates Java classes with the descriptors, struct layouts and method wrappers. For C structs you build a `MemoryLayout.structLayout(...)` and read fields through a `VarHandle` obtained from it (the same `VarHandle` type from the earlier chapter). The reverse direction, native code calling back into Java (a comparator for `qsort`, say), is an **upcall stub** created with `linker.upcallStub(...)` from a method handle.

10.5.5 When to use what

- **Pure Java first.** Many things people reach for native code for (compression, HTTP, JSON, crypto, regex) have fast Java libraries. Native calls cost more than a Java method call, and crossing the boundary blocks some JIT optimizations.
- **FFM** for new work: calling an existing C library, talking to OS facilities, or managing large off-heap data (caches, memory-mapped structures bigger than the heap) with deterministic freeing. It replaces JNI and most uses of `sun.misc.Unsafe` and direct `ByteBuffer` for this purpose.
- **JNI** only for existing code. The platform is moving to treat JNI like FFM, as a restricted capability that must be explicitly enabled, so expect warnings from libraries that still use it on newer JDKs.

REAL WORLD
Backend developers meet this indirectly: Netty and some database drivers use native libraries for TLS and compression. Off-heap storage engines (caches, embedded databases) are the typical reason application teams adopt FFM directly.

WATCH OUT
Native memory that is not owned by an arena you close is invisible to heap tools. A process whose `-Xmx` is 2 GB but whose resident memory keeps growing past 4 GB probably leaks native memory (see the diagnostics chapters for `jcmd VM.native_memory`).

Check yourself

1. Name two reasons a JNI bug is more dangerous than a Java bug.
2. In which Java version did the FFM API become final, and what was it before?
3. What does this do after the `try` block ends? `MemorySegment s; try (Arena a = Arena.ofConfined()) { s = a.allocate(8); } s.get(ValueLayout.JAVA_BYTE, 0);`
4. Which class holds the C function signature, and which finds a function by name?
5. What does a `ValueLayout.ADDRESS` parameter in a `FunctionDescriptor` correspond to in C?

Answers

1. A native error can crash the entire JVM (segmentation fault) instead of throwing an exception, and native code is unchecked: no bounds checking, no garbage collection, manual memory management.
2. Java 22 (JEP 454). Before that it was a preview API in Java 19 to 21 and an incubator API in 14 to 18.
3. It throws `IllegalStateException` ("Already closed"): the arena freed the memory and the segment is no longer accessible.
4. `FunctionDescriptor` describes the signature; `SymbolLookup` (for example `linker.defaultLookup()`) finds the symbol.
5. A pointer (`void *`, `char *`, `struct *`).

KEY POINTS

Native code is for existing libraries, OS access and off-heap data; prefer pure Java when it is fast enough.

- JNI needs C glue code, platform-specific builds, and can crash the JVM.
- The FFM API (final in Java 22; preview on 21) calls C functions through `Linker`, `FunctionDescriptor` and `MethodHandle`, with no C code.
- `MemorySegment` plus `Arena` give bounds-checked off-heap memory with an explicit, enforced lifetime.
- Native access is a restricted operation; enable it deliberately with `--enable-native-access`.

10.6 Troubleshooting Playbook

The previous chapter introduced the tools one by one. In an incident you do not want a tour; you want a procedure. This chapter is organised by symptom. For each one: how to confirm it, which tool to reach for, how to read the result, and the usual fix. All of the commands and outputs referred to here come from the demo program in the diagnostics chapter.

10.6.1 The general method

Whatever the symptom, the same five steps apply, and skipping the first two is the usual reason people chase the wrong problem.

1. **Measure before you guess.** What exactly is slow, how slow, since when, for which requests? A graph of CPU, heap and GC time beats an opinion.
2. **Capture evidence before restarting.** A restart destroys the only copy of the broken state. Take a thread dump (three, seconds apart), `jstat`, and if memory is involved a heap dump, then restart if you must.
3. **Form one hypothesis** from the evidence, using the table below.
4. **Change one thing**, and check that the measurement moved.
5. **Write down the cause** and add the metric or alert that would have caught it earlier.

Symptom	First look	Then
CPU at 100%	top -H, then jstack (or JFR hot-methods)	GC share via jstat; read the hot thread's stack
Requests hang, CPU idle	three jstack dumps	deadlock report, threads waiting on a pool, DB or socket
Heap grows until OutOfMemoryError	jstat -gcutil, jmap -histo:live	heap dump, dominator tree, path to GC roots
Long pauses, latency spikes	GC log or jfr view gc	pause cause, heap sizing, collector choice

Symptom	First look	Then
Slow startup	-Xlog:class+load count, JFR from start	class loading, static initialisers, scanning, AOT/CDS
Thread count keeps rising	jstack, then count threads by name	who creates them; missing shutdown or unbounded pool
Process killed by the OS or container	container memory limit, native memory	Native Memory Tracking, direct buffers, thread stacks

Symptom to first tool.

10.6.2 Symptom: one core pinned at 100% CPU

Confirm. `top -H -p <pid>` shows one row per thread. In the patient, the thread burning CPU is obvious:

Per-thread CPU TERMINAL

```
$ top -H -b -n1 -p 2902 | head -12 | tail -6
  PID USER      PR  NI    VIRT    RES    SHR S  %CPU  %MEM     TIME+ COMMAND
 2952 root      20   0 2877116 122804  24364 R  18.2   1.5   0:08.36 cpu-spinn+
 2902 root      20   0 2877116 122804  24364 S   0.0   1.5   0:00.00 java
 2904 root      20   0 2877116 122804  24364 S   0.0   1.5   0:00.36 java
 2905 root      20   0 2877116 122804  24364 S   0.0   1.5   0:00.04 GC Thread+
 2906 root      20   0 2877116 122804  24364 S   0.0   1.5   0:00.00 G1 Main M+
```

(The %CPU was only 18% because the build machine was shared with other jobs; on a quiet machine a spinning thread shows close to 100%. Look at the first row: it is the thread with by far the largest CPU time.) Linux shows thread names, so the name alone (`cpu-spinn...`, truncated) often answers the question.

Read. The thread id from `top` (2952) is the `nid` in a `jstack` dump. The JDK 21 build used here prints the id in decimal (`[2952]` and `nid=2952`); many JDK builds print `nid` in hex, in which case convert with `printf '%x\n' 2952`, which gives `b88`, and search for `nid=0xb88`. The stack of that thread shows the code:

Hot thread in the dump TERMINAL

```
$ jstack 2902 | grep -A4 '"cpu-spinner"'
"cpu-spinner" #25 [2952] daemon prio=5 os_prio=0 cpu=5587.43ms elapsed=18.10s ...
   java.lang.Thread.State: RUNNABLE
        at Victim.lambda$main$0(Victim.java:16)
        at java.lang.Thread.run(java.base@21.0.12.1/Thread.java:1583)
```

If you have no shell on the box, a JFR recording gives the same answer, with statistics. The earlier `jfr view hot-methods` listing put 100% of samples in that lambda.

If the hot threads are named `GC Thread`, `G1 Conc` or similar, the problem is not your code but memory pressure: the collector is running constantly. Check `jstat -gcutil <pid> 1000` for a high and growing `GCT` and an old generation that stays near 100%, then go to the memory section below.

Typical fixes. An accidental infinite or busy loop (a `while (!flag)` poll without sleeping, a retry loop with no back-off), a catastrophic regular expression, a hash table whose `hashCode` is broken so lookups degrade to scans, or a missing `break`. Fix the code; do not add CPUs.

> **GOTCHA**
> A `RUNNABLE` thread is not necessarily using CPU. Threads parked inside native socket or file reads are `RUNNABLE` too. Trust the `cpu=` figure in the header (compare it between two dumps) or the `top -H` numbers, not the state alone.

10.6.3 Symptom: requests hang and CPU is idle

An idle but unresponsive JVM is waiting for something. Take three thread dumps, five seconds apart, and compare.

- **A "Found one Java-level deadlock" section** is a deadlock. The report names each thread, the lock it waits for and the thread that holds it, with line numbers (see the deadlock listing in the diagnostics chapter). Fix: acquire locks in one global order, or hold only one lock at a time, or use `tryLock` with a timeout. `ThreadMXBean.findDeadlockedThreads()` lets your own health endpoint detect it.
- **Many threads `WAITING` or `TIMED_WAITING` on the same pool or queue**, for instance every request thread parked in `HikariPool.getConnection` or `LinkedBlockingQueue.take`: a resource is exhausted. A connection pool of 10 with

a leaked connection per failed request will do this after a while. The fix is to close resources in `try-with-resources` and to size and time-limit the pool.

- **Many threads `RUNNABLE` in `SocketInputStream.socketRead0` or `NioSocketImpl.read`** with the same remote host: a downstream service is slow and there is no read timeout. Set connect and read timeouts on every client.
- **Many threads `BLOCKED` on the same monitor, with one thread holding it:** lock contention. Read what the lock holder is doing; if it is doing I/O inside `synchronized`, move the I/O out. JFR's `jdk.JavaMonitorEnter` events quantify this.

A deadlock involving `ReentrantLock` is also reported by `jstack` (it lists "ownable synchronizers") when you use `jstack -l <pid>`; the default output reports only monitor deadlocks in the summary line.

> **TIP**
> If the JVM does not respond to `jstack` at all (the attach times out), the JVM itself may be stuck in a long safepoint or out of memory. Try `kill -3 <pid>`: it asks the JVM to print the thread dump to its own standard output, which usually ends up in your log file. Unlike most signals it does not terminate the process; it only triggers the dump.

10.6.4 Symptom: memory climbs, then OutOfMemoryError

Confirm. Watch the old generation after collections, not the instantaneous heap. In the patient, young collections happen constantly and that is healthy; the question is whether the post-collection floor rises.

Is the floor rising? TERMINAL

```
$ jstat -gcutil 2902 1000 5
  S0     S1     E      O      M     CCS    YGC     YGCT     FGC    FGCT     CGC    CGCT       GCT
     - 100.00  57.89  54.76  96.65  87.45      5     0.074     0     0.000     0     0.000     0.074
     ...
```

(Our patient clears its list once it holds more than 300 arrays, so it does not climb forever. A true leak shows `O` rising each cycle, and `FGC` rising as the collector fights a full heap.) Also useful: the "GC overhead limit exceeded" message, which means the JVM spent most of its time collecting while freeing almost nothing.

Narrow. `jmap -histo:live <pid>` names the class that is growing. Take it twice, a few minutes apart, and compare the counts; the class whose instance count only goes up is the suspect.

Prove. Take a heap dump (`jcmd <pid> GC.heap_dump /path/x.hprof`), open it in Eclipse MAT, and use the dominator tree and the "path to GC roots, excluding weak references" on a sample object. The path ends at the owner, which in practice is one of the leak patterns from the memory chapters: a static collection (our `Victim.retained`), an unbounded cache, listeners never deregistered, a `ThreadLocal` on a pooled thread, or an unclosed resource.

Fix. Bound the structure (maximum size or time-to-live), unregister listeners, call `ThreadLocal.remove()` in a `finally`, close resources. Raising `-Xmx` only delays the failure and makes the eventual heap dump bigger.

Not the Java heap? If the OutOfMemoryError message says something else, the cause is elsewhere:

Message	Meaning	Look at
Java heap space	live objects exceed -Xmx	heap dump, histogram
GC overhead limit exceeded	nearly all time in GC, little freed	same as heap space
Metaspace	too many classes or class loaders	jcmd VM.classloader_stats, redeploy leaks
unable to create native thread	OS refused another thread	thread count, ulimit -u, container limits
Direct buffer memory	off-heap NIO buffers exhausted	-XX:MaxDirectMemorySize, buffer pooling

OutOfMemoryError messages and where to look.

The process may also be killed by the operating system or the container runtime with no Java exception at all, because the JVM's total footprint (heap plus Metaspace, thread stacks, code cache, direct buffers) exceeded the memory limit. Start the JVM with `-XX:NativeMemoryTracking=summary` and then `jcmd <pid> VM.native_memory summary` to see the non-heap categories. Without that flag the command reports "Native memory tracking is not enabled", as we saw on the patient.

10.6.5 Symptom: latency spikes and long pauses

Check whether the pauses are GC pauses before anything else. Two quick ways: `jfr view gc` on a recording, or the GC log enabled with `-Xlog:gc*:file=gc.log`. In the patient's recording, young pauses were 6.18 ms, 48.2 ms and 6.02 ms. A 48 ms pause among 6 ms ones is worth a look; a pause of seconds is a problem.

- **Frequent young pauses that are long:** too much live data being copied each time, or too few CPUs for the GC threads. Reduce allocation in the hot path, or look at the young generation sizing.
- **Full collections or very long pauses:** the old generation is full (a leak, or a heap that is too small), or the collector cannot keep up with the allocation rate. Check `FGC` in `jstat`.
- **Pauses that are not in the GC log:** time to safepoint (a thread in a long counted loop delaying the stop), or the machine itself is paused (CPU throttling in containers, swapping, a noisy neighbour). Check `-Xlog:safepoint` and the container's CPU throttling statistics.

If the pause target matters more than throughput, the collector choice (G1 with `-XX:MaxGCPauseMillis`, or ZGC) is the lever; see the collectors chapter. Do not tune flags before you have a log that shows the cause.

10.6.6 Symptom: slow startup

Startup time is mostly class loading and initialisation, plus the interpreter running cold code before the JIT warms it up. To see what is happening:

Count loaded classes during startup TERMINAL

```
$ java -Xlog:class+load:file=classes.log -jar app.jar
$ wc -l classes.log
```

A framework application may load many thousands of classes. Start a JFR recording from the first instant (`-XX:StartFlightRecording=filename=start.jfr,settings=profile`) and look at the hot methods and at the time between `main` and readiness. Typical culprits are classpath scanning, reflection-heavy configuration (reflection and annotation processing chapters), a slow connection at start-up (database, remote configuration), and static initialisers that do real work. Remedies include lazy initialisation, narrowing component scans, Class Data Sharing archives and, on newer JDKs, the AOT cache (see the JVM start-up discussion in the JIT chapter), and for short-lived command-line tools a native image. Measure the effect of each change on startup, not on a long-running benchmark.

10.6.7 Symptom: the thread count keeps rising

Count threads from outside, over time:

Count and group threads TERMINAL

```
$ jcmd 2902 Thread.print | grep -c '^"'
$ jcmd 2902 Thread.print | grep '^"' | sed 's/[0-9]*" .*//' | sort | uniq -c | sort -rn | head
```

The second command groups by name prefix, so `pool-7-thread-` repeated 3000 times points at an executor created per request and never shut down. The usual causes are `Executors.newCachedThreadPool()` under a burst of slow tasks, a new `ExecutorService` per call without `shutdown()`, and `new Thread(...)` in a loop whose threads block forever. The fix is one shared, bounded pool with named threads (give a `ThreadFactory` that sets a meaningful name), or virtual threads for blocking I/O work, which are cheap enough that "one per task" is acceptable. In code, `ManagementFactory.getThreadMXBean().getThreadCount()` is the number to export as a metric; we printed it in the `Watchdog` program.

When the operating system refuses a thread, the JVM throws `OutOfMemoryError: unable to create native thread`. That is a limit on processes (`ulimit -u`, the container's `pids` limit) or on address space for stacks, not a shortage of heap.

10.6.8 A worked incident: from alert to fix

Suppose the order service's latency alert fires at 02:10 and the health check starts failing intermittently. The sequence below is the one an experienced engineer follows. Each step is a command from this chapter.

1. `jcmd` finds the pid; `jcmd <pid> VM.uptime` says the process has been up 3 days, so this is not a start-up problem.
2. Metrics show CPU low, so the process is waiting, not computing. `jstack <pid> > t1.txt`, twice more at 5-second intervals.

3. In all three dumps, 180 of 200 request threads are `WAITING` at `HikariPool.getConnection`. The database connection pool is exhausted.
4. The dumps show the remaining 20 threads each holding a connection while in `HttpClient.send`: a downstream payment service is slow, and there is no timeout, so connections are held for minutes.
5. Short-term mitigation: restart the payment service or fail fast; the pool recovers. Fix: add connect and read timeouts, release the connection before making the remote call, and alert on pool wait time.
6. Follow-up: add a metric for threads waiting on the pool, so the next occurrence is visible before customers notice.

Nothing in that story needed a profiler or a heap dump; three thread dumps were enough. Many incidents end this way, which is why thread dumps are the first tool to reach for.

REAL WORLD
In Spring Boot services the same data is exposed by Actuator (`/actuator/threaddump`, `/actuator/metrics/jvm.threads.live`, `/actuator/metrics/hikaricp.connections.pending`) and by Micrometer to Prometheus. Know the manual `jcmd` route anyway: when the web layer is the thing that is stuck, the HTTP endpoints stuck with it.

INTERVIEW ANGLE
Your Java service's heap usage keeps growing and it eventually dies with OutOfMemoryError. Walk me through diagnosing it. Confirm with GC data that the post-collection old-generation floor rises (`jstat -gcutil` or GC logs). Take a class histogram twice to see which class grows. Capture a heap dump (ideally automatically via `-XX:+HeapDumpOnOutOfMemoryError`), open it in Eclipse MAT, and inspect the dominator tree and the path to GC roots of a sample object to find the holder: usually a static collection, cache, listener or ThreadLocal. Fix the holder (bound, evict, deregister, remove), then verify that the floor is flat.

Check yourself

1. A Java process shows `%CPU` near 100 for a single thread in `top -H` with id 2952. How do you find the Java code responsible? What can differ between JDK builds?
2. What does this thread-dump excerpt tell you? Three dumps in a row show `"http-nio-8080-exec-1"` through `-exec-200` all `WAITING` in `HikariPool.getConnection`, and CPU is 3%.
3. Why is raising `-Xmx` not a fix for a leak?
4. `OutOfMemoryError: unable to create native thread` appears while `jstat` shows the heap is only 40% used. Name two possible causes.
5. What is wrong with taking `jmap -histo:live` every second on a busy production server?
6. The JVM was killed with exit code 137 in a container and there was no Java stack trace. Which memory is not covered by `-Xmx`, and which tool shows it?

Answers

1. Search the `jstack` dump for `nid=2952` and read that thread's stack, or use a JFR hot-methods view. Many JDK builds print `nid` in hex, so if the decimal search finds nothing, search for `nid=0xb88` (`printf '%x' 2952`).
2. The database connection pool is exhausted: all request threads wait for a connection while nobody computes. Look for threads holding connections (leaked or stuck on a slow call) and fix timeouts and resource closing.
3. A leak grows without bound, so a bigger heap only delays the failure and makes the dump bigger. Fix what holds the objects.
4. The OS or container refused another thread (process or pid limit), or there is not enough native memory left for thread stacks. Too many threads is the underlying cause, usually an unbounded executor or a thread-per-task loop.
5. `:live` forces a full collection every time, pausing the application and adding GC load. Sample rarely, and prefer a recording or a single dump.
6. Exit code 137 means the process received SIGKILL, typically from the out-of-memory killer. Metaspace, thread stacks, the code cache and direct buffers are outside the heap. Native Memory Tracking (`-XX:NativeMemoryTracking=summary` plus `jcmd <pid> VM.native_memory`) breaks them down.

KEY POINTS

Work from symptom to tool: per-thread CPU to a thread dump, idle hangs to repeated dumps, growth to `jstat`, histogram and heap dump, pauses to the GC log or JFR.

- Capture evidence (three thread dumps, `jstat`, a heap dump if memory is involved) before restarting anything.
- A deadlock is reported by `jstack`; a hung service with idle CPU is usually waiting on an exhausted pool, a missing timeout or a contended lock.
- A leak is proved by a rising post-GC floor and a GC-root path in a heap dump; raising `-Xmx` only hides it.
- Memory outside the heap (Metaspace, thread stacks, direct buffers) causes container kills and different OutOfMemoryError messages; use Native Memory Tracking.
- Build the habit: name your threads, set timeouts, bound your pools and caches, and enable heap-dump-on-OOM before you need it.

10.7 The Diagnostics Toolbox

Sooner or later a Java program in production does something you did not intend: it burns a whole CPU core, stops responding, or grows until the container is killed. The JDK ships with a set of small command-line tools that can look inside a running JVM without stopping it, without a debugger, and usually without a restart. Learning them is the difference between "restart it and hope" and "find the cause in ten minutes".

This chapter walks through the tools against one live program, so every output you see is real. The next chapter turns them into a playbook: symptom, tool, how to read it, fix.

10.7.1 The demo patient

To practise, we need a JVM that misbehaves in three ways at once: one thread spins on the CPU, two threads deadlock, and one thread allocates 100 KB arrays and keeps up to 300 of them alive. The program is deliberately ugly; never write production code like this.

Victim.java JAVA

```java
import java.util.ArrayList;
import java.util.List;

public class Victim {
    static final Object LOCK_A = new Object();
    static final Object LOCK_B = new Object();
    static final List<byte[]> retained = new ArrayList<>();

    static void sleep(long ms) {
        try { Thread.sleep(ms); } catch (InterruptedException e) { return; }
    }

    public static void main(String[] args) {
        Thread spin = new Thread(() -> {
            long x = 0;
            while (true) { x += Math.sqrt(x + 1); if (x < 0) x = 0; }
        }, "cpu-spinner");
        spin.setDaemon(true);
        spin.start();

        Thread t1 = new Thread(() -> {
            synchronized (LOCK_A) { sleep(300); synchronized (LOCK_B) { } }
        }, "worker-1");
        Thread t2 = new Thread(() -> {
            synchronized (LOCK_B) { sleep(300); synchronized (LOCK_A) { } }
        }, "worker-2");
        t1.setDaemon(true); t2.setDaemon(true);
        t1.start(); t2.start();

        Thread leaker = new Thread(() -> {
            while (true) {
                retained.add(new byte[100_000]);
                sleep(20);
                if (retained.size() > 300) retained.clear();
            }
        }, "leaker");
        leaker.setDaemon(true);
```

Victim.java (continued) JAVA

```
        leaker.start();

        System.out.println("Victim running, pid " + ProcessHandle.current().pid());
        sleep(120_000);
    }
}
```

We start it with a 256 MB heap and ask the JVM to record a 20-second Flight Recording from the very first moment (more on that later):

Start the patient TERMINAL

```
$ javac Victim.java
$ java -XX:StartFlightRecording=filename=boot.jfr,duration=20s -Xmx256m Victim
[2.141s][info][jfr,startup] Started recording 1. The result will be written to:
[2.141s][info][jfr,startup]
[2.141s][info][jfr,startup] /work/boot.jfr
Victim running, pid 2902
```

The process id (pid) will differ on your machine. Everything below was run against pid 2902 on JDK 21.0.12.

> **NOTE**
> The diagnostic tools print `Picked up JAVA_TOOL_OPTIONS: ...` on stderr if that environment variable is set (common in containers). It is harmless, and we remove it from the listings below to keep them readable.

10.7.2 Finding and inspecting the process: jps, jcmd, jinfo

Every other tool needs a pid. `jps` lists JVMs owned by your user; `jcmd` with no arguments does the same job and also shows the main class.

Find the JVM TERMINAL

```
$ jps -l
2902 Victim
$ jcmd
2902 Victim
```

(`jps -l` on a launcher-started program such as `java Foo.java` shows `jdk.compiler/com.sun.tools.javac.launcher.Main` instead of your class, which is why we compiled `Victim` first.)

`jcmd` is the Swiss army knife. It sends a named command to the target JVM over a local attach mechanism, and the JVM executes it. `jcmd <pid> help` lists what that particular JVM supports; on JDK 21 the list includes (trimmed):

What can jcmd do? TERMINAL

```
$ jcmd 2902 help
2902:
The following commands are available:
Compiler.codecache
Compiler.queue
GC.class_histogram
GC.heap_dump
GC.heap_info
GC.run
JFR.check
JFR.dump
JFR.start
JFR.stop
JFR.view
Thread.dump_to_file
Thread.print
VM.classloader_stats
VM.flags
VM.native_memory
VM.system_properties
VM.uptime
VM.version
...
```

Start with the cheap, read-only questions: which JVM is this, which flags is it really running with, and how long has it been up?

Identity, flags, uptime TERMINAL

```
$ jcmd 2902 VM.version
OpenJDK 64-Bit Server VM version 21.0.12.1+1-1-24.04.4-Ubuntu
JDK 21.0.12.1
$ jcmd 2902 VM.flags
-XX:CICompilerCount=2 -XX:ConcGCThreads=1 -XX:G1HeapRegionSize=1048576
-XX:InitialHeapSize=98566144 -XX:MaxHeapSize=268435456 -XX:+UseCompressedOops
-XX:+UseG1GC ...
$ jcmd 2902 VM.uptime
54.072 s
```

`VM.flags` shows the flags that differ from the built-in defaults, including values the JVM computed for itself (`G1HeapRegionSize`, `InitialHeapSize`). Here we see our `-Xmx256m` as `MaxHeapSize=268435456` and that the collector is G1 (the build machine has 2 CPUs and enough memory for a "server-class" machine; see the chapter on collector choice in the garbage collection Part).

`jinfo` reads, and for a few "manageable" flags even changes, a single flag on a live JVM:

jinfo TERMINAL

```
$ jinfo -flag MaxHeapSize 2902
-XX:MaxHeapSize=268435456
$ jinfo -flag UseG1GC 2902
-XX:+UseG1GC
```

Most flags are read-only once the JVM runs; `jinfo -flag +HeapDumpOnOutOfMemoryError <pid>` is a useful exception because it is manageable, so you can arm the heap-dump-on-OOM safety net on a process that was started without it. Use `jcmd <pid> VM.system_properties` for the system properties.

10.7.3 Threads: jstack and Thread.print

`jstack <pid>` (or the equivalent `jcmd <pid> Thread.print`) prints a thread dump: the name, state, and stack trace of every thread at one instant. It is the single most useful diagnostic for "the application is stuck" and "the application is busy". A dump of our patient, trimmed to the interesting threads:

Thread dump (trimmed) TERMINAL

```
$ jstack 2902
"cpu-spinner" #25 [2952] daemon prio=5 os_prio=0 cpu=5587.43ms elapsed=18.10s \
    tid=0x00007f9b041be3a0 nid=2952 runnable  [0x00007f9adc7de000]
   java.lang.Thread.State: RUNNABLE
        at Victim.lambda$main$0(Victim.java:16)
        at java.lang.Thread.run(java.base@21.0.12.1/Thread.java:1583)

"worker-1" #26 [2953] daemon prio=5 os_prio=0 cpu=0.55ms elapsed=18.08s \
    tid=0x00007f9b041bfb80 nid=2953 waiting for monitor entry  [0x00007f9adc6df000]
   java.lang.Thread.State: BLOCKED (on object monitor)

"leaker" #28 [2955] daemon prio=5 os_prio=0 cpu=68.85ms elapsed=18.88s \
    tid=0x00007f9b041c1eb0 nid=2955 sleeping [0x00007f9adc4df000]
   java.lang.Thread.State: TIMED_WAITING (sleeping)
        at java.lang.Thread.sleep0(java.base@21.0.12.1/Native Method)
        at Victim.sleep(Victim.java:10)
        at Victim.lambda$main$3(Victim.java:33)
```

(The `\` line continuation is only for this book's page width; in the real dump each header is one line.) Read each header like this:

- The quoted name is `Thread.getName()`. Name your threads; `Thread-17` tells you nothing, `order-sync-3` tells you where to look.
- `cpu=5587.43ms elapsed=18.10s` is CPU time consumed versus wall time since the thread started. The spinner used 5.6 s of CPU in 18 s of life; the leaker used 69 ms. This pair is the quickest way to spot a hot thread in a dump.
- `nid=2952` is the operating-system thread id. This JDK 21 build prints it in decimal, so it matches what `top -H` shows directly; many JDK builds print `nid` in hex (`nid=0xb88`), and then you convert with `printf '%x' 2952`.
- `Thread.State` is the Java state (`RUNNABLE`, `BLOCKED`, `WAITING`, `TIMED_WAITING`), covered in the concurrency Part. `RUNNABLE` also includes threads blocked in native I/O, so "runnable" does not always mean "using CPU".

Deadlock detection comes free

When the dump contains a cycle of threads waiting on each other's monitors, `jstack` says so at the end of the output:

Deadlock section of jstack TERMINAL

```
$ jstack 2902
...
Found one Java-level deadlock:
=============================
"worker-1":
  waiting to lock monitor 0x00007f9ad40041d0 (object 0x00000000f25005f8, a java.lang.Object),
  which is held by "worker-2"

"worker-2":
  waiting to lock monitor 0x00007f9ad00040a0 (object 0x00000000f2500608, a java.lang.Object),
  which is held by "worker-1"

Java stack information for the threads listed above:
===================================================
"worker-1":
        at Victim.lambda$main$1(Victim.java:22)
        - waiting to lock <0x00000000f25005f8> (a java.lang.Object)
        - locked <0x00000000f2500608> (a java.lang.Object)
"worker-2":
        at Victim.lambda$main$2(Victim.java:25)
        - waiting to lock <0x00000000f2500608> (a java.lang.Object)
        - locked <0x00000000f25005f8> (a java.lang.Object)
```

Read it as a picture: worker-1 holds lock `...608` and wants `...5f8`; worker-2 holds `...5f8` and wants `...608`. The `Victim.java:22` and `:25` line numbers point at the inner `synchronized` blocks. The fix is to make both threads acquire locks in the same order (or use `ReentrantLock.tryLock` with a timeout). The same detection is available in code through `ThreadMXBean.findDeadlockedThreads()`, which is how health-check endpoints and watchdogs report deadlocks without anyone running `jstack`:

Watchdog.java JAVA

```
import java.lang.management.ManagementFactory;
import java.lang.management.ThreadInfo;
import java.lang.management.ThreadMXBean;

public class Watchdog {
    static final Object A = new Object();
    static final Object B = new Object();

    static void pause() {
        try { Thread.sleep(200); } catch (InterruptedException e) { return; }
    }

    public static void main(String[] args) throws Exception {
        Thread t1 = new Thread(() -> {
            synchronized (A) { pause(); synchronized (B) { } }
        }, "worker-1");
        Thread t2 = new Thread(() -> {
            synchronized (B) { pause(); synchronized (A) { } }
        }, "worker-2");
        t1.setDaemon(true); t2.setDaemon(true);
        t1.start(); t2.start();
        Thread.sleep(600);
        ThreadMXBean mx = ManagementFactory.getThreadMXBean();
        long[] ids = mx.findDeadlockedThreads();
        System.out.println("deadlocked threads: " + (ids == null ? 0 : ids.length));
        for (ThreadInfo info : mx.getThreadInfo(ids)) {
            System.out.println(info.getThreadName() + " is " + info.getThreadState()
                + " waiting for " + info.getLockName() + " held by " + info.getLockOwnerName());
        }
        System.out.println("live threads: " + mx.getThreadCount());
    }
}
```

Run it (typical output; the object hash codes vary) TERMINAL

```
$ java Watchdog.java
deadlocked threads: 2
worker-1 is BLOCKED waiting for java.lang.Object@1d119efb held by worker-2
worker-2 is BLOCKED waiting for java.lang.Object@4b5d6a01 held by worker-1
live threads: 8
```

TIP

Take thread dumps in a series: three dumps, five seconds apart. One dump is a photograph; three show which threads are moving and which are stuck at the same line every time. Redirect each to a file (`jstack <pid> > dump1.txt`).

NOTE

Applications that use virtual threads create far too many threads for a plain dump. `jcmd <pid> Thread.dump_to_file -format=json <file>` writes a structured dump that includes virtual threads (added in JDK 21), and we created one this way on the patient. The older `jstack` and `Thread.print` show platform threads only.

10.7.4 Memory: jstat, jmap and heap dumps

`jstat` samples the garbage collector's counters from outside, cheaply, at a fixed interval. `jstat -gcutil <pid> 500 4` prints four samples, 500 ms apart, as percentages of each space:

jstat -gcutil TERMINAL

```
$ jstat -gcutil 2902 500 4
  S0     S1     E      O      M     CCS    YGC     YGCT     FGC    FGCT     CGC    CGCT       GCT
    - 100.00  57.89  54.76  96.65  87.45      5     0.074     0     0.000     0     0.000     0.074
    - 100.00  68.42  54.76  96.65  87.45      5     0.074     0     0.000     0     0.000     0.074
    - 100.00   0.00  70.37  97.02  91.52      6     0.079     0     0.000     0     0.000     0.079
    - 100.00   8.33  70.37  97.02  91.52      6     0.079     0     0.000     0     0.000     0.079
```

The columns: `S0`, `S1`, `E`, `O` are survivor, eden and old-generation occupancy in percent; `M` and `CCS` are Metaspace and compressed class space; `YGC`/`YGCT` count and total time (seconds) of young collections; `FGC`/`FGCT` the same for full collections; `CGC`/`CGCT` for concurrent G1 cycles; `GCT` is total GC time. Between rows 2 and 3 the young count went from 5 to 6 and eden dropped back to 0.00: that is one young collection. Watch two things over minutes: does `O` (old) keep climbing after each collection, and does `FGC` increase? A staircase in `O` that never comes down is the signature of a leak.

`jcmd <pid> GC.heap_info` gives a one-shot summary of the heap as the collector sees it:

GC.heap_info TERMINAL

```
$ jcmd 2902 GC.heap_info
 garbage-first heap   total 61440K, used 21731K [0x00000000f0000000, 0x0000000100000000)
  region size 1024K, 5 young (5120K), 0 survivors (0K)
 Metaspace       used 4346K, committed 4480K, reserved 1114112K
  class space    used 410K, committed 448K, reserved 1048576K
```

Class histogram: who owns the memory?

`jmap -histo:live <pid>` (or `jcmd <pid> GC.class_histogram`) lists instance counts and shallow bytes per class. The `:live` option forces a full collection first so that only reachable objects are counted; that pauses the application, so use it with care on a busy production system.

Class histogram (top rows) TERMINAL

```
$ jmap -histo:live 2902
 num     #instances         #bytes  class name (module)
-------------------------------------------------------
   1:         16279       14602984  [B (java.base@21.0.12.1)
   2:            45         632472  [Ljdk.internal.vm.FillerElement; (java.base@21.0.12.1)
   3:         15140         363360  java.lang.String (java.base@21.0.12.1)
   4:          2205         270168  java.lang.Class (java.base@21.0.12.1)
```

`[B` is the JVM's name for `byte[]` (`[I` is `int[]`, `[Ljava.lang.Object;` is `Object[]`). Here `byte[]` dominates at about 14.6 MB, which matches the leaker holding up to 300 arrays of 100 KB. A histogram answers "what kind of object is piling up" but not "who is holding it". For that you need a heap dump.

Heap dumps

A heap dump is a snapshot of every live object and the references between them, written to an `.hprof` file:

Take a heap dump TERMINAL

```
$ jcmd 2902 GC.heap_dump /work/v.hprof
2902:
Dumping heap to /work/v.hprof ...
Heap dump file created [35246298 bytes in 0.037 secs]
$ jmap -dump:live,format=b,file=/work/v2.hprof 2902
```

Both forms do the same job. Expect the dump to be roughly the size of the used heap and the JVM to pause while it is written; for multi-gigabyte heaps that can be many seconds. The file may contain passwords, tokens and personal data that were in memory, so treat it as sensitive and never attach it to a public ticket.

You do not read an `.hprof` file by hand. Open it in Eclipse MAT (Memory Analyzer), VisualVM or IntelliJ's profiler. The views that matter are the same in all of them:

- **Dominator tree**: objects ordered by retained size, the memory that would be freed if that object died. A leak almost always shows up as one object with a huge retained size.
- **Path to GC roots** for a suspicious object: the chain of references (a static field, a thread-local, a listener list) that keeps it alive. That chain is the bug.
- **Histogram** by class, as above, but with retained sizes.

In our dump, selecting a `byte[]` and asking for its path to GC roots ends at `Victim.retained`, a static `ArrayList`: the classic "static collection that only grows" pattern from the memory leak chapter.

> **WATCH OUT**
> `-XX:+HeapDumpOnOutOfMemoryError` and `-XX:HeapDumpPath=/dumps/` should be on every production JVM. The dump you want is the one from the moment of failure, and you cannot ask the process to go back in time. Make sure the path has enough disk space for a dump the size of `-Xmx`.

10.7.5 Flight Recorder: the always-on profiler

Java Flight Recorder (JFR) is an event recorder built into the JVM. It continuously records low-overhead events (sampled stack traces, allocations, GC pauses, lock contention, file and socket I/O, CPU load) into a ring buffer and can write them to a `.jfr` file. It was a commercial feature in Oracle JDK 8 and has been open source and free in OpenJDK since JDK 11. Its overhead is low enough to leave on in production with the default settings; the `profile` setting records more and costs a bit more, so use it for focused investigations.

There are three ways to start a recording:

Starting JFR TERMINAL

```
# at JVM start
$ java -XX:StartFlightRecording=filename=boot.jfr,duration=20s -Xmx256m Victim
# on a running JVM
$ jcmd 2902 JFR.start name=live settings=profile duration=5s filename=/work/live.jfr
Started recording 2. The result will be written to:

/work/live.jfr
# write what is in the buffer now, then keep recording
$ jcmd 2902 JFR.dump name=live filename=/work/now.jfr
```

Two further options matter in production: `maxage=6h` and `maxsize=500m` keep a rolling window, and `-XX:StartFlightRecording=...,dumponexit=true` writes the file when the JVM exits (including a crash caused by OutOfMemoryError with the right flags). Use `jcmd <pid> JFR.check` to list active recordings.

Reading a recording

The `jfr` tool reads `.jfr` files. `jfr summary` shows what the file contains:

jfr summary (trimmed) TERMINAL

```
$ jfr summary boot.jfr

 Version: 2.1
 Chunks: 1
 Start: 2026-10-11 03:48:35 (UTC)
 Duration: 20 s
```

jfr summary (trimmed) (continued) TERMINAL

```
 Event Type                                Count  Size (bytes)
============================================================
 jdk.ThreadSleep                             841         15387
 jdk.ObjectAllocationSample                  621          8748
 jdk.ExecutionSample                         564          5777
 jdk.GCPhaseParallel                         256          6790
 jdk.OldObjectSample                         206          6969
 ...
 jdk.CPULoad                                  20           386
```

`jfr view` (JDK 21) renders ready-made views as text tables. The hot-methods view answers "where is the CPU going?" using the execution samples:

jfr view hot-methods TERMINAL

```
$ jfr view hot-methods boot.jfr
                    Java Methods that Executes the Most

Method                                                          Samples Percent
--------------------------------------------------------------- ------- -------
Victim.lambda$main$0()                                              564 100.00%
```

All 564 samples landed in the lambda at the heart of `cpu-spinner`. On a real service you would see a spread, and the top few entries are your optimisation candidates. The `gc` view lists each collection with its pause:

jfr view gc TERMINAL

```
$ jfr view gc boot.jfr
                             Garbage Collections

Start    GC ID Type                    Heap Before GC Heap After GC Longest Pause
-------- ----- ---------------------- -------------- ------------- -------------
09:18:36     2 Young Garbage Collection        17.4 MB        8.4 MB       6.18 ms
09:18:49     3 Young Garbage Collection        61.4 MB       36.0 MB       48.2 ms
09:18:53     4 Young Garbage Collection        50.0 MB       43.5 MB       6.02 ms
```

(The timestamps in the views are shown in the machine's local zone, which differs from the UTC time in `jfr summary`.) And `allocation-by-class` shows what is producing garbage:

jfr view allocation-by-class TERMINAL

```
$ jfr view allocation-by-class boot.jfr
                           Allocation by Class

Object Type                                                 Allocation Pressure
----------------------------------------------------------- -------------------
byte[]                                                                  78.97%
java.util.HashMap$KeyIterator                                           20.33%
java.util.concurrent.ConcurrentHashMap$Node[]                            0.25%
```

For raw events use `jfr print`, optionally filtered by event name and stack depth:

jfr print TERMINAL

```
$ jfr print --events jdk.ExecutionSample --stack-depth 3 boot.jfr
jdk.ExecutionSample {
  startTime = 09:18:35.655 (2026-10-11)
  sampledThread = "cpu-spinner" (javaThreadId = 25)
  state = "STATE_RUNNABLE"
  stackTrace = [
    Victim.lambda$main$0() line: 16
    java.lang.Thread.run() line: 1583
  ]
}
...
```

For a graphical view, open the file in JDK Mission Control (a separate download, no longer bundled with the JDK) or in IntelliJ IDEA. Its flame graph and "Memory" and "Lock instances" pages are built on exactly these events. Events worth knowing by name: `jdk.ExecutionSample` (CPU sampling), `jdk.JavaMonitorEnter` (lock contention), `jdk.GarbageCollection`, `jdk.ObjectAllocationSample`, `jdk.OldObjectSample` (candidate leaks, with optional reference chains), `jdk.FileRead`, `jdk.SocketRead`, `jdk.CPULoad`.

GOTCHA
JFR's sampler records Java threads at a safe-ish moment roughly every 10 or 20 ms (the exact period depends on the setting). It is a statistical profile: good for finding the top few hot methods, not for timing one particular call.

10.7.6 GUI tools and the remote story

`jconsole` and VisualVM (a separate download) connect to a JVM, locally or over JMX, and show live graphs of heap, threads, classes and CPU, let you invoke a thread dump or heap dump with a button, and browse MBeans. They are good for exploring a development or staging process. For remote production processes, the common pattern is different: expose metrics through Micrometer or the platform MXBeans to Prometheus and Grafana, and keep the command-line tools for the moment you are on the box (or in the container, via `kubectl exec`).

WATCH OUT
Do not open JMX remote access (`-Dcom.sun.management.jmxremote.port=...`) without authentication and TLS. An open JMX port is remote code execution for anyone who can reach it. The local attach used by `jcmd` and `jstack` requires the same operating-system user as the target process, which is why `jcmd` says "Permission denied" if you run it as a different user from the application.

REAL WORLD
In containers the JVM's pid is often 1 and the image may have only a JRE-like runtime without the tools. Either use a JDK-based image (so `jcmd` exists), or add JFR flags to the startup command and copy the `.jfr` out afterwards. Spring Boot Actuator exposes `/actuator/threaddump` and `/actuator/heapdump` for the same purposes; restrict those endpoints, they leak internals.

INTERVIEW ANGLE
How would you find out why a Java process is using 100% CPU? Find the hot thread: `top -H -p <pid>` shows per-thread CPU. Match that thread id to the `nid` in a `jstack` dump (converting to hex if your JDK prints it that way), then read its stack. Take several dumps to confirm it is always in the same place, or record a JFR and look at the hot-methods view. Check GC activity with `jstat -gcutil` as well, since GC threads can be the cause.

Check yourself

1. Which tool gives you a thread dump, and which line at the end of the dump tells you about a deadlock?
2. In `jstat -gcutil` output, the `O` column climbs after every collection and `FGC` keeps increasing. What does it suggest, and what do you take next?
3. Why does `jmap -histo:live` pause the application, and what does `[B` mean in its output?
4. What is wrong with this startup line for a production service? `java -Dcom.sun.management.jmxremote.port=9010 -jar app.jar`
5. You need a profile of a service that is already running and was started without any JFR flags. Give the command.
6. A heap dump shows 2 GB retained by one `HashMap` held in a static field. Which view in MAT finds this, and what do you do with the answer?

Answers

1. `jstack <pid>` or `jcmd <pid> Thread.print`. If a cycle of monitors exists, the dump ends with "Found one Java-level deadlock", followed by who holds and who waits for each lock.
2. Objects are surviving every collection and the old generation fills: a probable leak or an undersized heap. Take a `jmap -histo:live` to see which class grows, then a heap dump and look at the dominator tree and path to GC roots.
3. `:live` triggers a full collection first, which stops the application threads. `[B` is the internal name of `byte[]`.
4. The JMX port is open with no authentication and no TLS (these are off by default when only the port property is set), so anyone who can reach port 9010 can control the JVM. Add `authenticate=true`, `ssl=true` or bind it to localhost and tunnel.
5. `jcmd <pid> JFR.start name=live settings=profile duration=60s filename=/tmp/live.jfr`, then read it with `jfr view hot-methods /tmp/live.jfr` or Mission Control.
6. The dominator tree (sorted by retained size) finds it; "path to GC roots" shows the static field. The fix is to bound or evict the map (size limit, TTL, a cache library) or to stop adding to it.

KEY POINTS

The JDK diagnostic tools attach to a running JVM; start with `jcmd <pid> help`, `Thread.print`, `GC.heap_info` and `JFR.start`.

- A thread dump shows what every thread is doing now; three dumps a few seconds apart show who is stuck; deadlocks are reported automatically.
- `jstat -gcutil` plus `jmap -histo:live` tell you whether memory is leaking and which class is piling up; a heap dump with a dominator tree and GC-root path tells you who holds it.
- JFR is a low-overhead, always-available recorder; read it with `jfr summary`, `jfr view` and `jfr print`, or with Mission Control.
- Heap dumps and JMX ports are security-sensitive: protect both, and arm `HeapDumpOnOutOfMemoryError` before you need it.

PART 11

Modern Java and Tooling

How the language grew from Java 8 to 25, and the build, test and packaging tools around it.

In this part

11.1 From Java 8 to 25: what changed
11.2 Build tools: Maven and Gradle
11.3 Testing Java code
11.4 Packaging and running Java applications
11.5 Java 25 LTS highlights in depth
11.6 Java 26, Java 27 and the road ahead

11.1 From Java 8 to 25: what changed

Java 8 was released in March 2014 and it is still the most common version you will meet in old enterprise code. Java 25, the current long-term-support (LTS) release, came out in September 2025. In between lie seventeen releases and a very different language. If you read a tutorial from 2015 and a codebase from 2025 side by side, they look like two dialects. This chapter is your map: what arrived when, which features change how you write everyday code, and which ones you can safely ignore for now.

The reason to know this is practical. In an interview you will be asked "what is new since Java 8?". At work you will inherit a project on Java 11 or 17 and have to decide what you may use. And when you read someone else's code you need to recognise `record`, `sealed`, `case Point(int x, int y)` and `var` on sight.

11.1.1 How releases work

Since Java 10 a new feature release appears every six months, in March and September. Every second year one of them is designated LTS: 8, 11, 17, 21 and 25. Vendors support LTS releases for years, while the releases in between get about six months of updates. Production systems usually sit on an LTS and move from one to the next; the non-LTS versions are where new features first appear in final form.

Features do not appear fully formed. A new language or library feature usually goes through **preview** (shipped but switched off unless you pass `--enable-preview`, and allowed to change) and sometimes **incubator** (a separate module whose API may change), before it becomes final. A feature that has been "preview" for several releases is not unstable junk; it is being refined with feedback. The unit of change is the **JEP** (JDK Enhancement Proposal), a numbered document at `openjdk.org/jeps/<number>`; when you want the real story of a feature, search for its JEP.

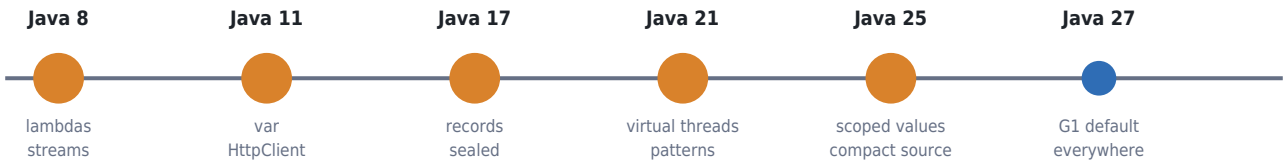


Figure 11.1. Java releases from 8 to 27. The LTS releases (8, 11, 17, 21, 25) are the usual production targets; the releases between them deliver features in preview first.

11.1.2 The version-by-version table

Read this table once for the big picture and return to it as a reference. Entries marked (preview) were not final in that release.

Version	Language and library	Platform and tooling
8 (LTS)	lambdas, method references, streams, `Optional`, default methods, `java.time`	Metaspace replaces PermGen

Version	Language and library	Platform and tooling
9	`List.of`/`Map.of`, private interface methods, `Flow`	modules (JPMS), `jshell`, G1 becomes default GC
10	`var` for local variables	container-aware JVM
11 (LTS)	`String` methods (`strip`, `repeat`, `isBlank`, `lines`), `HttpClient`, `var` in lambdas	`java Foo.java` single-file launch
12-13	switch expressions (preview)	-
14	switch expressions final	helpful `NullPointerException` messages (on by default from 15)
15	text blocks final	ZGC and Shenandoah production-ready
16	records, `instanceof` patterns, `Stream.toList()`	strong encapsulation of JDK internals starts
17 (LTS)	sealed classes final	Security Manager deprecated for removal
18	-	UTF-8 default charset, `jwebserver`
19-20	virtual threads, record patterns (previews)	-
21 (LTS)	virtual threads, record patterns, pattern `switch`, sequenced collections	generational ZGC
22	unnamed variables and patterns _, FFM API	multi-file source launch
23	Markdown in Javadoc comments	-
24	stream gatherers, Class-File API	virtual threads without pinning on `synchronized`
25 (LTS)	compact source files, instance `main`, flexible constructor bodies, module imports, scoped values	compact object headers (opt-in), AOT ergonomics

Features worth knowing, by release.

Java 26 (March 2026) and Java 27 (September 2026) are covered in the last chapter of this Part, after the Java 25 deep dive.

11.1.3 One runnable tour of Java 8 to 21

The following program touches a feature from most of the releases above. It needs only Java 21, so you can run it on your machine with `java Tour.java`. Read it top to bottom; each comment names the release that introduced the feature.

Tour.java JAVA

```java
import java.util.*;
import java.util.stream.*;

public class Tour {
    record Point(int x, int y) {}                          // Java 16
    sealed interface Shape permits Circle, Square {}       // Java 17
    record Circle(double r) implements Shape {}
    record Square(double side) implements Shape {}

    static double area(Shape s) {
        return switch (s) {                                // exhaustive, no default
            case Circle c -> Math.PI * c.r() * c.r();
            case Square q -> q.side() * q.side();
        };
    }

    static String describe(Object o) {                     // pattern switch: Java 21
        return switch (o) {
            case Point(int x, int y) when x == y -> "diagonal point " + x;
            case Point p -> "point " + p;
            case String s -> "string of length " + s.length();
            case null -> "null!";
            default -> "something else";
        };
    }

    public static void main(String[] args) {
        // Java 8: streams
        System.out.println(Stream.of("b", "a", "c").sorted().collect(Collectors.joining(",")));
```

Tour.java (continued) JAVA

```
        // Java 9: collection factories
        List<Integer> nums = List.of(1, 2, 3);
        System.out.println(nums + " " + Map.of("k", 1));
        // Java 10: var
        var names = new ArrayList<String>();
        names.add("Asha");
        System.out.println(names);
        // Java 11: new String methods
        System.out.println("  hi ".strip() + "|" + "ab".repeat(3) + "|" + " ".isBlank());
        // Java 14: switch expression
        int day = 3;
        String kind = switch (day) { case 6, 7 -> "weekend"; default -> "weekday"; };
        System.out.println(kind);
        // Java 15: text block
        String json = """
            {"id": 1,
             "name": "Asha"}
            """;
        System.out.print(json);
        // Java 16: pattern variable in instanceof, record, Stream.toList()
        Object o = "hello";
        if (o instanceof String s && s.length() > 3) System.out.println(s.toUpperCase());
        System.out.println(new Point(1, 2));
        System.out.println(Stream.of(1, 2, 3, 4).filter(n -> n % 2 == 0).toList());
        // Java 21: pattern matching for switch, record patterns, guards
        System.out.println(describe(new Point(2, 2)) + "; " + describe(new Point(1, 2))
            + "; " + describe("abc") + "; " + describe(null) + "; " + describe(4));
        System.out.printf("%.2f%n", area(new Circle(1)));
        // Java 21: sequenced collections
        var seq = new ArrayList<>(List.of(1, 2, 3));
        System.out.println(seq.getFirst() + " " + seq.getLast() + " " + seq.reversed());
        // Java 21: virtual threads
        try (var ex = java.util.concurrent.Executors.newVirtualThreadPerTaskExecutor()) {
            System.out.println(ex.submit(() -> Thread.currentThread().isVirtual()).get());
        } catch (Exception e) { throw new RuntimeException(e); }
    }
}
```

Output (Java 21) TERMINAL

```
$ java Tour.java
a,b,c
[1, 2, 3] {k=1}
[Asha]
hi|ababab|true
weekday
{"id": 1,
 "name": "Asha"}
HELLO
Point[x=1, y=2]
[2, 4]
diagonal point 2; point Point[x=1, y=2]; string of length 3; null!; something else
3.14
1 3 [3, 2, 1]
true
```

What to notice

- `sealed` plus `switch` gives you a compiler-checked "this covers every case". Add a third `Shape` and `area` stops compiling until you handle it. That is the single biggest win of modern Java for domain modelling.
- `case null` is allowed in a pattern switch. Before Java 21 a `switch` on a null reference threw `NullPointerException` unconditionally.
- `reversed()` returns a view, not a copy. Sequenced collections (Java 21) add `getFirst`, `getLast`, `addFirst`, `reversed` to `List`, `Deque`, `LinkedHashSet` and friends so you no longer write `list.get(list.size() - 1)`.

11.1.4 What to adopt and what to ignore

Not every new feature deserves equal attention. A sensible ranking for a backend developer:

Feature	Advice
`var`	Use for locals when the right-hand side makes the type obvious (`var users = new ArrayList<User>()`). Avoid when it hides the type (`var x = compute()`).
Records	Adopt for DTOs, value objects, map keys, multiple return values. Not for JPA entities (they need mutable state and a no-arg constructor).
Text blocks	Adopt for SQL, JSON, HTML snippets in code.
Switch expressions	Adopt. They replace most old `switch` statements and cannot fall through.
Sealed types plus pattern `switch`	Adopt for closed hierarchies: results, events, commands, AST nodes.
`Stream.toList()`	Prefer over `collect(Collectors.toList())` when you do not need a mutable result.
Virtual threads	Adopt for blocking server-style code on Java 21+; not a speed-up for CPU-bound work.
Sequenced collections	Use the new methods when the code is on 21+.
`HttpClient`	Use instead of `HttpURLConnection` for new code.
Modules (JPMS)	Know how they work; most applications still run on the classpath.
Preview features	Learn them, but do not ship them. They require `--enable-preview` and a matching JDK.

Adoption guide.

TIP

Check which Java your project targets before using a feature. In Maven this is `maven.compiler.release`; in Gradle it is the toolchain. Using `--release 17` makes `javac` reject Java 21 APIs, not only syntax. That protects you from calling `List.reversed()` in code that must run on 17.

INTERVIEW ANGLE

What has changed in Java since version 8 that you actually use? Name a handful and say why, not just what: `var` for less noise; records for immutable data carriers; sealed interfaces with pattern `switch` for exhaustive handling of a closed set of cases; text blocks for multi-line strings; `Optional` and streams (Java 8) with `Stream.toList()`; virtual threads in 21 for cheap blocking concurrency; and the new six-month release cadence with LTS every two years. Mention that you target an LTS and that preview features need `--enable-preview`.

Check yourself

1. Which releases are LTS releases between 8 and 25?
2. What does this print on Java 21? `var l = new ArrayList<>(List.of(1, 2, 3)); System.out.println(l.reversed().getFirst());`
3. What is the difference between a preview feature and an incubator module?
4. Why is `case null` useful, and what happened in a pre-21 `switch` on a null value?
5. What is wrong with this code? `record User(String name) { void setName(String n) { this.name = n; } }`

Answers

1. 11, 17, 21 and 25 (plus 8 itself).
2. `3`. `reversed()` returns a reversed view, and the first element of the view is the last of the original.
3. A preview feature is part of the language or core API but disabled unless `--enable-preview`; an incubator module is a separate API module (such as the Vector API) that is not yet part of the standard. Both may change before they become final.
4. It handles null inside the switch instead of requiring a separate check. Before 21 a `switch` on a null reference threw `NullPointerException`.
5. Record fields are `final`; assigning `this.name` in a method does not compile. A record is immutable, so there is no setter.

KEY POINTS

- Java has a feature release every March and September; LTS releases are 8, 11, 17, 21 and 25.
- Features move through preview or incubator stages before they are final; read the JEP for the real story.
- Highest value since 8: `var`, records, text blocks, switch expressions, sealed types, pattern matching, virtual threads, sequenced collections.
- Target an LTS and pin the language level with `--release`; do not ship preview features.

11.2 Build tools: Maven and Gradle

Compiling with `javac` by hand works for one file. A real project has hundreds of source files, tests, and dozens of third-party libraries, each with its own dependencies. A **build tool** automates the repeatable parts: download the right library versions, compile, run tests, package the result, and do it the same way on your laptop and on the build server. In the Java world the two dominant tools are **Maven** and **Gradle**. Spring Boot projects use either; Maven is more common in enterprises, Gradle in Android and many newer projects.

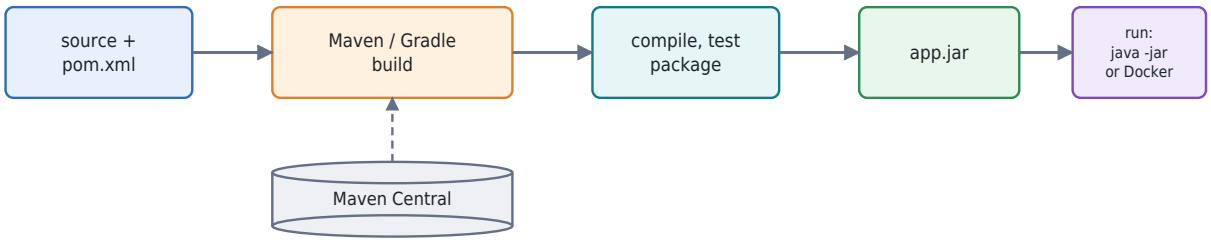


Figure 11.2. From source to a running service: the build tool resolves dependencies from a repository, compiles, tests and packages; the artifact is then run directly, in a container, or deployed.

11.2.1 Maven: convention over configuration

Maven's central idea is that if every project uses the same directory layout and the same sequence of steps, you can describe a project with a short declarative file, the **POM** (`pom.xml`), instead of a script. The standard layout is:

```
project/
├── pom.xml
└── src/
    ├── main/java/com/example/lib/App.java       production code
    ├── main/resources/application.properties    files copied to the jar
    └── test/java/com/example/lib/AppTest.java   test code
```

A minimal POM looks like this. The three values `groupId`, `artifactId` and `version` (the **GAV coordinates**) identify the project, and any library, uniquely.

pom.xml XML

```xml
<project xmlns="http://maven.apache.org/POM/4.0.0">
  <modelVersion>4.0.0</modelVersion>
  <groupId>com.example</groupId>
  <artifactId>lib</artifactId>
  <version>1.0.0</version>
  <packaging>jar</packaging>

  <properties>
    <maven.compiler.release>21</maven.compiler.release>
    <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
  </properties>

  <dependencies>
    <dependency>
      <groupId>org.junit.jupiter</groupId>
      <artifactId>junit-jupiter</artifactId>
      <version>5.11.0</version>
      <scope>test</scope>
    </dependency>
  </dependencies>
</project>
```

The version shown is only an example; look up the current release on Maven Central (`search.maven.org`) before you copy it.

The build lifecycle

Maven has built-in **lifecycles** made of ordered **phases**. You ask for a phase and Maven runs every earlier phase first. The default lifecycle phases you use daily:

Phase	What happens
`validate`	check the POM is sane
`compile`	compile `src/main/java` to `target/classes`
`test`	compile and run unit tests (Surefire plugin)
`package`	build the jar into `target/`
`verify`	run integration tests and checks
`install`	copy the artifact to your local repository `~/.m2`
`deploy`	upload the artifact to a remote repository

Main phases of the default lifecycle, in order.

Two more lifecycles exist: `clean` (deletes `target/`) and `site`. So `mvn clean package` means "delete old output, then run everything up to and including `package`". Each phase is just a set of **plugin goals** bound to it: `compile` runs `compiler:compile`, `test` runs `surefire:test`. When you need extra behaviour, such as building an executable Spring Boot jar, you add a plugin to the POM.

> **NOTE**
> On this book's build machine Maven 3.9 is installed but the sandbox cannot reach Maven Central, so the commands below were not run here. The output you see on your own machine begins with `[INFO] Scanning for projects...` and ends with `BUILD SUCCESS`.

Everyday Maven commands TERMINAL

```
$ mvn -v
$ mvn clean package
$ mvn test
$ mvn -q -DskipTests package
$ mvn dependency:tree
$ mvn -pl service -am package
```

`dependency:tree` is the command you will use most when debugging. It prints what Maven resolved, including the transitive (indirect) dependencies. `-pl service -am` builds only the `service` module and the modules it depends on ("also make").

11.2.2 Dependencies, scopes and conflicts

A dependency is another jar identified by GAV. Maven downloads it from a **repository** (by default **Maven Central**) into a local cache (`~/.m2/repository`) and also downloads that library's own dependencies. These indirect ones are **transitive dependencies**.

The `scope` says when a dependency is on the classpath:

Scope	Meaning
`compile`	default; available everywhere and passed on to consumers
`provided`	needed to compile, supplied by the runtime (for example the servlet API in a server)
`runtime`	not needed to compile, needed to run (for example a JDBC driver)
`test`	only for test code (JUnit, Mockito)
`import`	only in `dependencyManagement`, to import a bill of materials (BOM)

Dependency scopes.

When two paths pull in different versions of the same library, Maven applies **nearest wins**: the version closest to your project in the dependency tree is used (first declaration wins on a tie). This can silently choose an old version. The fix is explicit: declare the version you want in `dependencyManagement`, or exclude the unwanted transitive dependency. A

BOM (bill of materials), such as Spring Boot's, is a POM that lists compatible versions of many libraries so you do not set each one.

> **WATCH OUT**
> `SNAPSHOT` versions (`1.2.0-SNAPSHOT`) mean "work in progress" and can change under you. Never depend on a SNAPSHOT in a release. Pin versions in release builds so the same commit produces the same result next month.

11.2.3 Gradle: the same job as a script

Gradle uses a build script rather than XML. It is more flexible and, with its build cache and daemon, often faster on big projects; the price is that a script is code and so easier to make hard to read. The equivalent of the POM above, in the Groovy-based `build.gradle`:

build.gradle GRADLE

```
plugins {
    id 'java'
    id 'application'
}

group = 'com.example'
version = '1.0.0'

java {
    toolchain { languageVersion = JavaLanguageVersion.of(21) }
}

repositories { mavenCentral() }

dependencies {
    implementation 'com.google.guava:guava:33.0.0-jre'
    runtimeOnly 'org.postgresql:postgresql:42.7.3'
    testImplementation 'org.junit.jupiter:junit-jupiter:5.11.0'
}

application { mainClass = 'com.example.lib.App' }

tasks.named('test') { useJUnitPlatform() }
```

Gradle's configurations map roughly to Maven scopes: `implementation` (compile, not exposed to consumers), `api` (exposed; from the `java-library` plugin), `runtimeOnly`, `compileOnly` (provided) and `testImplementation`. Tasks replace phases: `./gradlew build`, `./gradlew test`, `./gradlew run`, `./gradlew dependencies`. Newer projects often use the Kotlin DSL (`build.gradle.kts`) with the same structure.

The wrapper

Both tools offer a **wrapper**: small scripts (`mvnw`, `gradlew`) committed to the repository that download the exact tool version the project wants. Anyone who clones the repo runs `./mvnw package` or `./gradlew build` and needs no pre-installed Maven or Gradle, only a JDK. This removes "works with my Maven 3.6" problems and is what CI servers should call. Always commit the wrapper files.

11.2.4 Multi-module projects and repositories

Large applications split into modules, each with its own POM, under a parent POM that lists them and shares versions:

parent pom.xml (fragment) XML

```
<packaging>pom</packaging>
<modules>
  <module>domain</module>
  <module>service</module>
  <module>web</module>
</modules>
```

`service` declares a dependency on `domain` like on any other artifact; Maven builds the modules in dependency order. Gradle expresses the same with `settings.gradle`: `include 'domain', 'service', 'web'`. Do not confuse these build modules with Java modules (`module-info.java`) from Part 10; build modules are a grouping for compilation, Java modules are a runtime and compile-time visibility mechanism.

Repositories come in three kinds: **local** (`~/.m2`, your cache), **remote** (Maven Central or a company server), and a **repository manager** such as Nexus or Artifactory that proxies Central for the whole company and stores private artifacts. `mvn deploy` publishes to the last kind.

REAL WORLD
In a Spring Boot service the POM usually has `spring-boot-starter-parent` as its parent (it supplies a BOM and plugin configuration). A dependency like `spring-boot-starter-web` is a "starter": one line that pulls in a consistent set of libraries. Understanding scopes and `dependency:tree` is how you diagnose the classic "NoSuchMethodError at runtime" caused by two versions of one library.

INTERVIEW ANGLE
What is the difference between Maven and Gradle, and what does `mvn install` do? Maven is declarative XML with a fixed lifecycle and strong conventions; Gradle is a script (Groovy or Kotlin) with a task graph, incremental builds and a build cache, so it is more flexible and often faster. `mvn install` runs all phases up to `install`, which copies the built artifact into the local repository `~/.m2` so other local projects can depend on it. `deploy` is the phase that publishes to a remote repository.

Check yourself

1. What do the GAV coordinates stand for?
2. You run `mvn package`. Which earlier phases run, and which later ones do not?
3. A library is needed to compile but the application server supplies it at runtime. Which scope?
4. Two dependencies pull in `lib 1.0` and `lib 2.0`. Which version does Maven choose, by default, and how do you override it?
5. Why commit `mvnw` or `gradlew` to the repository?

Answers

1. groupId, artifactId, version: the unique identity of an artifact.
2. validate, compile, test run before `package` (and others in between); `verify`, `install`, `deploy` do not run.
3. `provided` (Gradle: `compileOnly`).
4. The nearest one in the dependency tree (first declared on a tie). Override it with `dependencyManagement` or an exclusion, and confirm with `mvn dependency:tree`.
5. So everyone, including CI, builds with the same tool version without installing it.

KEY POINTS
- A build tool resolves dependencies, compiles, tests and packages repeatably.
- Maven: `pom.xml`, standard layout, GAV coordinates, lifecycle phases run in order up to the one you ask for.
- Scopes control when a dependency is on the classpath; transitive conflicts resolve to the nearest version, so inspect with `dependency:tree`.
- Gradle uses a script and tasks; both tools provide a wrapper that you should commit.
- Pin versions and avoid SNAPSHOTs in releases.

11.3 Testing Java code

A test is a small program that runs your code and fails loudly when the result is not what you expected. Without tests every change is a gamble: you fix one bug, you break another, and you find out in production. With a good test suite you can refactor, upgrade Java or replace a library and know within minutes whether behaviour changed. For a job interview, "how do you test your code?" is asked almost as often as "how does HashMap work?".

11.3.1 The test pyramid

Tests differ in scope, and scope decides speed and reliability.

```
          /\
         /  \        few   end-to-end / UI tests     slow, brittle, realistic
        /----\
       /      \      some  integration tests         real DB, real HTTP, a Spring context
      /--------\
     /          \    many  unit tests                one class, no I/O, milliseconds
    /------------\
```

A **unit test** exercises one class or method in isolation, runs in milliseconds, and needs no network, database or clock. An **integration test** wires several parts together (your repository class with a real database such as one started in a container). An **end-to-end test** drives the whole system from the outside. The pyramid advice is: write many fast unit tests, fewer integration tests, and very few end-to-end tests, because the higher you go, the slower the feedback and the more often tests fail for reasons that are not bugs.

11.3.2 What makes a test good

- **Fast and deterministic.** The same input gives the same result every time. A test that depends on the current time, random numbers or thread timing is a flaky test; inject a `java.time.Clock` or a seeded `Random` instead.
- **Independent.** A test must not depend on another test having run first. Each test builds its own objects.
- **Tests one behaviour** and is named after it: `depositRejectsNegativeAmount`, not `test1`.
- **Arrange, Act, Assert.** Set up the objects, call the code, check the outcome. Three visible parts.
- **Fails for one reason**, with a message that tells you what was expected and what happened.
- **Tests behaviour, not implementation.** If a harmless refactoring breaks twenty tests, they were too tightly coupled to private details.

11.3.3 JUnit 5

JUnit 5 is the standard test framework. It is built from three parts: the *Platform* (launches tests; IDEs and Maven/Gradle talk to it), *Jupiter* (the API you write tests against) and *Vintage* (runs old JUnit 3/4 tests). Add `org.junit.jupiter:junit-jupiter` with `test` scope and your tests live under `src/test/java`. The listing below is shown as text because the sandbox cannot download JUnit; the next section runs a hand-made equivalent so you can see the mechanism.

AccountTest.java (needs JUnit 5 on the classpath; not run here) JAVA

```
import org.junit.jupiter.api.*;
import org.junit.jupiter.params.ParameterizedTest;
import org.junit.jupiter.params.provider.ValueSource;
import static org.junit.jupiter.api.Assertions.*;

class AccountTest {
    private Account account;

    @BeforeEach
    void setUp() {                       // runs before EACH test, on a fresh test instance
        account = new Account();
    }

    @Test
    @DisplayName("deposits add up")
    void depositIncreasesBalance() {
        account.deposit(50);
        account.deposit(25);
        assertEquals(75L, account.balance());
    }

    @Test
    void depositRejectsZero() {
        var ex = assertThrows(IllegalArgumentException.class, () -> account.deposit(0));
        assertEquals("amount must be positive", ex.getMessage());
    }

    @ParameterizedTest
    @ValueSource(longs = {-1, 0, -100})
```

AccountTest.java (needs JUnit 5 on the classpath; not run here) (continued) JAVA

```
    void depositRejectsNonPositive(long amount) {
        assertThrows(IllegalArgumentException.class, () -> account.deposit(amount));
    }

    @Test
    void severalChecksAtOnce() {
        assertAll(
            () -> assertEquals(0L, account.balance()),
            () -> assertTrue(account.balance() >= 0));
    }

    @AfterEach void tearDown() { /* release resources */ }
}
```

What to notice

- **Lifecycle.** `@BeforeEach` and `@AfterEach` run around every test; `@BeforeAll` and `@AfterAll` run once per class (they must be `static` unless you change the instance lifecycle). JUnit creates a *new instance of the test class for each test method*, which is why fields set in one test do not leak into another.
- **Assertions.** `assertEquals(expected, actual)`: expected first. Swapping them gives a confusing failure message. `assertThrows` returns the exception so you can inspect it. `assertAll` reports every failing check instead of stopping at the first.
- **Parameterized tests** run one method with many inputs: `@ValueSource` for simple values, `@CsvSource` for rows of arguments, `@MethodSource` for arbitrary objects.
- Test methods and classes can be package-private; they do not need to be `public`. `@Disabled` skips a test; `@Nested` groups tests inside an inner class.

You run them with `mvn test` or `./gradlew test`; both print a summary such as `Tests run: 4, Failures: 0`.

11.3.4 A tiny test runner you can run now

To see there is no magic, here is a 60-line framework with a `@Test` annotation, a fresh instance per test, two assertions and a pass/fail report. It uses reflection (covered in Part 10) to find annotated methods, which is essentially what JUnit's engine does, plus a lot of polish. Run it with `java MiniTest.java`.

MiniTest.java JAVA

```
import java.lang.annotation.*;
import java.lang.reflect.*;

public class MiniTest {
    @Retention(RetentionPolicy.RUNTIME) @Target(ElementType.METHOD)
    @interface Test {}

    static class Account {
        private long balance;
        void deposit(long amount) {
            if (amount <= 0) throw new IllegalArgumentException("amount must be positive");
            balance += amount;
        }
        long balance() { return balance; }
    }

    static class AccountTests {
        @Test void depositIncreasesBalance() {
            var a = new Account();
            a.deposit(50);
            a.deposit(25);
            assertEquals(75L, a.balance());
        }
        @Test void depositRejectsZero() {
            assertThrows(IllegalArgumentException.class, () -> new Account().deposit(0));
        }
        @Test void deliberatelyBroken() {
            assertEquals(1L, new Account().balance());
        }
    }

    static void assertEquals(Object expected, Object actual) {
        if (!expected.equals(actual))
```

MiniTest.java (continued) JAVA

```
            throw new AssertionError("expected <" + expected + "> but was <" + actual + ">");
    }
    static void assertThrows(Class<? extends Throwable> type, Runnable code) {
        try { code.run(); }
        catch (Throwable t) {
            if (type.isInstance(t)) return;
            throw new AssertionError("expected " + type.getSimpleName() + " but got " + t);
        }
        throw new AssertionError("expected " + type.getSimpleName() + " but nothing was thrown");
    }

    public static void main(String[] args) throws Exception {
        int passed = 0, failed = 0;
        for (Method m : AccountTests.class.getDeclaredMethods()) {
            if (!m.isAnnotationPresent(Test.class)) continue;
            // a fresh instance per test, like JUnit
            Object instance = AccountTests.class.getDeclaredConstructor().newInstance();
            m.setAccessible(true);
            try {
                m.invoke(instance);
                passed++;
                System.out.println("PASS " + m.getName());
            } catch (InvocationTargetException e) {
                failed++;
                System.out.println("FAIL " + m.getName() + " -> " + e.getCause().getMessage());
            }
        }
        System.out.println(passed + " passed, " + failed + " failed");
        if (failed > 0) System.exit(1);
    }
}
```

Typical output (the method order from reflection is not guaranteed) TERMINAL

```
$ java MiniTest.java
PASS depositIncreasesBalance
PASS depositRejectsZero
FAIL deliberatelyBroken -> expected <1> but was <0>
2 passed, 1 failed
```

Notice the last line: the runner exits with a non-zero status when any test fails. That is how a build server knows to stop the pipeline, and it is exactly what Surefire does for `mvn test`. Also notice that `getDeclaredMethods()` does not promise an order; JUnit likewise says test order is deliberately not obvious, so never rely on it.

11.3.5 Making code testable: inject the clock

The most common reason code is hard to test is that it reaches out to something global: the system clock, a random generator, a static singleton. The cure is to pass the dependency in. For time, `java.time.Clock` is built for this: production code receives `Clock.systemUTC()`, tests receive a fixed clock.

Cl.java JAVA

```
import java.time.*;

public class Cl {
    record Subscription(LocalDate end) {
        boolean expired(Clock clock) {
            return LocalDate.now(clock).isAfter(end);
        }
    }

    public static void main(String[] a) {
        var s = new Subscription(LocalDate.of(2026, 1, 31));
        var before = Clock.fixed(Instant.parse("2026-01-31T10:00:00Z"), ZoneOffset.UTC);
        var after = Clock.fixed(Instant.parse("2026-02-01T00:00:01Z"), ZoneOffset.UTC);
        System.out.println(s.expired(before) + " " + s.expired(after));
    }
}
```

Terminal TERMINAL

```
$ java Cl.java
false true
```

Both boundary cases, the last day and the first day after, are now exact and repeatable, something a test using `LocalDate.now()` could only pass on one particular day. The same trick applies to `Random` (pass a seeded instance), to `UUID` generation (pass a `Supplier<UUID>`) and to file access (pass a `Path`, and use a temporary directory in the test). Constructor injection is nothing more than that: dependencies arrive as parameters. This is also what Spring does for you, and it is why well-designed Spring code is easy to test without Spring at all.

11.3.6 Mocks and fluent assertions

Real classes have collaborators: a service calls a repository, which calls a database. To unit-test the service alone you replace the repository with a **test double**. **Mockito** is the common library for generating one:

OrderServiceTest.java (needs Mockito; not run here) JAVA

```java
@Test
void placeOrderSavesAndReturnsId() {
    OrderRepository repo = Mockito.mock(OrderRepository.class);
    Mockito.when(repo.save(Mockito.any())).thenReturn(42L);
    var service = new OrderService(repo);

    long id = service.place(new Order("book", 2));

    assertEquals(42L, id);
    Mockito.verify(repo).save(Mockito.any());      // the interaction we care about
}
```

The idea is *stubbing* (`when(...).thenReturn(...)` decides what the double answers) and *verification* (`verify(...)` asserts it was called). Use mocks for slow or non-deterministic collaborators (database, HTTP, clock). Do not mock simple value objects, and do not verify every call: a test that mirrors the implementation line by line breaks on every refactoring. Where you can, prefer a small **fake** (an in-memory repository) to a mock; it keeps the test about behaviour.

AssertJ replaces JUnit's `assertEquals` with readable, chainable assertions that give better failure messages: `assertThat(names).containsExactly("Asha", "Ravi").doesNotContain("Zed");`. It is a taste, not a necessity.

REAL WORLD
In Spring Boot, `spring-boot-starter-test` bundles JUnit 5, Mockito and AssertJ. `@SpringBootTest` starts the whole application context (an integration test, slow); `@WebMvcTest` loads only the web layer; plain unit tests with no Spring annotation are fastest. Teams that see 10-minute builds usually have too many `@SpringBootTest` tests where a unit test would do. For database tests, Testcontainers starts a real database in Docker, which is more faithful than an in-memory substitute.

GOTCHA
A test that passes is not proof that the code is right: coverage tools measure which lines ran, not whether you asserted anything. A test with no assertion, or one that catches and swallows the exception, can reach 100% line coverage and verify nothing. Mutation testing and code review are the antidotes.

INTERVIEW ANGLE
How do you decide what to test, and what is the difference between a mock and a stub? Test behaviour that matters through public methods: normal cases, boundaries (0, 1, empty, maximum), and failure paths (invalid input, exceptions). A stub supplies canned answers to the code under test; a mock additionally records calls so that the test can verify an interaction happened. Prefer state-based assertions on results; verify interactions only when the interaction itself is the requirement (for example "an email was sent").

Check yourself

1. Why does JUnit create a new instance of the test class for every test method?
2. What is wrong with this assertion order, and why does it matter? `assertEquals(account.balance(), 75L);`
3. Why is a test that calls `LocalDate.now()` and compares it to a hard-coded date a bad test, and what is the fix?
4. In `MiniTest`, what exit status does the program return, and who benefits?

5. When is `@SpringBootTest` the wrong tool?

Answers

1. To isolate tests: state from one cannot leak into another, so test order does not matter.
2. Expected must come first. The failure message says "expected: <actual value> but was: <75>", which misleads you when debugging.
3. It depends on the real clock, so it passes today and fails tomorrow. Inject a `Clock` (fixed in tests) and use `LocalDate.now(clock)`.
4. 1 when any test failed (0 otherwise); the build server uses the status to fail the pipeline.
5. For logic that needs no Spring: a plain unit test is much faster. Use it for integration of wiring only.

KEY POINTS

- Follow the pyramid: many fast unit tests, fewer integration tests, very few end-to-end tests.
- Good tests are fast, deterministic, independent, focused on one behaviour and named for it.
- JUnit 5: `@Test`, `@BeforeEach`, `@ParameterizedTest`, `assertEquals(expected, actual)`, `assertThrows`; a fresh instance per test.
- Use doubles (Mockito or hand-written fakes) for slow collaborators; do not over-verify interactions.
- Coverage numbers do not prove correctness.

11.4 Packaging and running Java applications

Writing code is half of the job; the other half is getting it onto a server and running it reliably. This chapter follows an application from compiled classes to a runnable artifact, to a trimmed runtime, to a native installer, and finally to a container. Each step answers the question "what does the person running this need to have installed?"

11.4.1 Jars

A **JAR** (Java ARchive) is a zip file of class files, resources and a `META-INF/MANIFEST.MF`. It is how libraries are distributed and, with a `Main-Class` entry in the manifest, how simple applications are run. Create one with the `jar` tool:

com/example/Greeter.java — JAVA

```java
package com.example;

public class Greeter {
    public static void main(String[] a) {
        System.out.println("Hello " + (a.length > 0 ? a[0] : "world"));
    }
}
```

Build and run an executable jar — TERMINAL

```
$ javac -d out com/example/Greeter.java
$ jar --create --file greeter.jar --main-class com.example.Greeter -C out .
$ java -jar greeter.jar Maruthi
Hello Maruthi
$ unzip -p greeter.jar META-INF/MANIFEST.MF
Manifest-Version: 1.0
Created-By: 21.0.12.1 (Ubuntu)
Main-Class: com.example.Greeter
```

`--main-class` writes the `Main-Class` line, so `java -jar` knows where to start. `-C out .` means "change into `out` and add everything". If your program needs libraries, a plain jar does not contain them: you need `-cp app.jar:lib/*` (on Windows `;`), or a **fat jar** (also called uber jar) that bundles the dependencies. Spring Boot builds an executable fat jar with its Maven or Gradle plugin and a custom class loader that reads the nested jars. The `maven-shade-plugin` and the Gradle `shadow` plugin are the generic alternatives.

NOTE

`java -jar` ignores any `-cp` you pass; the jar's manifest `Class-Path` entry decides. This surprises people who try to add a driver jar on the command line.

11.4.2 jlink: a runtime made for your application

Normally a user needs a whole JDK or JRE installed. The module system (Part 10) lets you ship only the parts of the JDK your program uses. `jdeps` finds which modules a jar needs, and `jlink` builds a custom runtime image containing just those.

jdeps and jlink TERMINAL

```
$ jdeps --print-module-deps --ignore-missing-deps greeter.jar
java.base
$ jlink --add-modules java.base --output rt --strip-debug --no-header-files \
    --no-man-pages --compress zip-6
$ rt/bin/java --list-modules
java.base@21.0.12.1
$ rt/bin/java -jar greeter.jar Asha
Hello Asha
$ du -sh rt /usr/lib/jvm/java-21-openjdk-amd64
39M     rt
286M    /usr/lib/jvm/java-21-openjdk-amd64
```

The custom runtime is 39 MB against 286 MB for the full JDK on this build machine (sizes depend on platform and vendor). A real application adds modules such as `java.sql` or `java.net.http`. `jlink` needs your dependencies to be modular or automatic modules; with big classpath applications (Spring Boot) it is awkward, and teams more often use the full JRE in a container image instead.

11.4.3 jpackage: a native installer

`jpackage` takes your application and a runtime and produces a platform installer: `.msi` or `.exe` on Windows, `.dmg` or `.pkg` on macOS, `.deb` or `.rpm` on Linux. It is for desktop applications, not servers. A typical call:

BASH

```
jpackage --name Greeter --input target --main-jar greeter.jar \
         --main-class com.example.Greeter --type deb
```

It must be run on the operating system you are targeting (no cross-building) and needs platform packaging tools. It bundles a jlink-style runtime, so the installed application needs no separate Java. The tool is installed here (`jpackage --version` prints 21.0.12.1), but producing a package requires the OS tooling, so the command above was not run.

11.4.4 Java in containers

Most backend Java now runs in Docker or Kubernetes. A container image holds a JRE, your jar and a start command. A typical `Dockerfile` uses a **multi-stage build**: compile with a full JDK, then copy only the jar into a small runtime image.

Dockerfile (not run here: no Docker daemon in the build sandbox) DOCKERFILE

```
# Stage 1: build
FROM eclipse-temurin:21-jdk AS build
WORKDIR /src
COPY . .
RUN ./mvnw -q -DskipTests package

# Stage 2: run
FROM eclipse-temurin:21-jre
WORKDIR /app
COPY --from=build /src/target/app.jar app.jar
RUN useradd --system appuser
USER appuser
EXPOSE 8080
ENTRYPOINT ["java", "-XX:MaxRAMPercentage=75", "-jar", "app.jar"]
```

Notes on this file: the second stage contains only a JRE and the jar, so the image is far smaller and has fewer tools for an attacker; running as a non-root user is a basic security habit; `ENTRYPOINT` in exec form (a JSON array) makes `java` process number 1, so it receives `SIGTERM` when the container stops and shutdown hooks run. Image tags such as `eclipse-temurin:21-jre` select a vendor distribution; pin a specific tag in production.

Memory inside a container

The JVM is **container aware** (since Java 10, backported to 8u191): it reads the cgroup memory limit and CPU quota rather than the host's totals. By default the maximum heap is 25% of the container memory (`MaxRAMPercentage=25`). That default is conservative because the process also needs memory outside the heap: Metaspace, thread stacks, the code cache, direct buffers and the GC's own structures. You can see the flag logic on any machine:

Heap sizing flags (this build machine's container limit is about 6 GB; typical values) TERMINAL

```
$ java -XX:+PrintFlagsFinal -version | grep -E "MaxRAMPercentage|UseContainerSupport"
    double MaxRAMPercentage       = 25.000000     {product} {default}
      bool UseContainerSupport    = true          {product} {default}
$ java -XX:+PrintFlagsFinal -version | grep " MaxHeapSize"
    size_t MaxHeapSize            = 1568669696    {product} {ergonomic}
$ java -XX:MaxRAMPercentage=50 -XX:+PrintFlagsFinal -version | grep " MaxHeapSize"
    size_t MaxHeapSize            = 3137339392    {product} {ergonomic}
$ java -Xmx256m -XX:+PrintFlagsFinal -version | grep " MaxHeapSize"
    size_t MaxHeapSize            = 268435456     {product} {command line}
```

Practical rules: prefer percentages (`-XX:MaxRAMPercentage=70` or 75) over a fixed `-Xmx`, so the same image works with different container sizes; leave 25-30% headroom for non-heap memory, otherwise the kernel's OOM killer ends the container (exit code 137) while the Java heap looks fine; and set a CPU limit knowing that it changes how many processors the JVM sees, which sizes the GC and `ForkJoinPool` thread counts. A container with less than two CPUs or about 1.8 GB of memory is treated as a "small" machine and, before Java 27, selects the Serial collector instead of G1; the last chapter of this Part explains the change in Java 27.

WATCH OUT

A container with a memory limit of 512 MB and `-Xmx512m` will be killed. The limit covers the whole process, not just the heap. Start from `MaxRAMPercentage` of 50-75 and watch real usage with `jcmd <pid> VM.native_memory` (needs `-XX:NativeMemoryTracking=summary`).

REAL WORLD

Spring Boot can build an image without a Dockerfile (`./mvnw spring-boot:build-image`) using Cloud Native Buildpacks, which also calculate sensible memory settings. Startup time matters for autoscaling; techniques to cut it (class-data sharing, AOT caches, GraalVM native image) are the subject of the next chapters.

INTERVIEW ANGLE

Your Java service keeps being killed in Kubernetes with exit code 137, but heap dumps show the heap is fine. What is going on? Exit 137 means the process received `SIGKILL`, usually from the OOM killer, because total process memory exceeded the container limit. The heap is only part of the memory: Metaspace, thread stacks, direct buffers and the code cache count too. Reduce `MaxRAMPercentage`, cap thread counts and direct memory, and measure with Native Memory Tracking.

Check yourself

1. What does the `Main-Class` manifest entry do?
2. What is the difference between `jlink` and `jpackage`?
3. Why is a multi-stage Dockerfile preferred?
4. A container has 2 GB of memory and you run `java -jar app.jar` with default flags. About how large can the heap grow?
5. What is wrong with `ENTRYPOINT java -jar app.jar` (shell form) for graceful shutdown?

Answers

1. It names the class whose `main` method `java -jar` runs.
2. `jlink` builds a trimmed Java runtime image from selected modules; `jpackage` wraps an application plus runtime into a native installer for one operating system.
3. The final image contains only a JRE and the jar, so it is smaller and exposes fewer tools; build tools stay in the first stage.
4. About 25% by default, roughly 512 MB.
5. The shell becomes process 1 and may not forward `SIGTERM` to `java`, so shutdown hooks may not run; use the exec form.

KEY POINTS

- A jar is a zip of classes plus a manifest; `java -jar` needs `Main-Class`, and dependencies need a classpath or a fat jar.
- `jdeps` plus `jlink` produce a small custom runtime; `jpackage` builds native installers for desktop apps.
- Containers: multi-stage build, JRE-only final image, non-root user, exec-form `ENTRYPOINT`.
- The JVM reads container limits; heap defaults to 25% of memory, so tune `MaxRAMPercentage` and leave room for non-heap memory.
- Exit code 137 usually means the OOM killer, not a Java `OutOfMemoryError`.

11.5 Java 25 LTS highlights in depth

Java 25 is the LTS release that most teams will move to from Java 17 or 21 over the next two years, so it is worth understanding what it actually changes. Think of it in three groups: features that make **small programs and learning easier** (compact source files, instance `main`, module imports), features that make **code safer and cleaner** (flexible constructor bodies, scoped values), and features that make the **JVM leaner and faster to start** (compact object headers, AOT ergonomics).

WATCH OUT

The sandbox that built this book has only JDK 21. Listings marked "Java 25; not run on 21" are checked against the JEP text (openjdk.org/jeps) but were not executed. Where a Java 21 preview gives a runnable cousin of the feature, that is shown and labelled. Verify new listings on a JDK 25 before relying on them.

11.5.1 Compact source files and instance main (JEP 512)

Every Java tutorial starts with a ceremony that students cannot yet explain: `public class`, `public static void main(String[] args)`, `System.out.println`. JEP 512 (final in Java 25, the result of previews in 21 to 24) removes the ceremony for small programs without creating a second language.

The rules:

- A **compact source file** may contain top-level methods and fields. The compiler wraps them in an implicitly declared class (final, in the unnamed package, named after the file, and not referable from other code).
- The `main` method no longer has to be `public static void main(String[] args)`. The launcher looks, in order, for a `static main(String[])`, a `static main()`, an instance `main(String[])`, then an instance `main()`. For an instance method it creates an object using a non-private no-argument constructor.
- Compact source files automatically `import module java.base`, so `List`, `Map`, `Stream` and `Path` need no imports.
- `java.lang.IO` has `IO.println`, `IO.print`, `IO.readln` for simple console input and output, without `System.out` or `Scanner`.

Hi.java (Java 25; not run on 21) JAVA

```java
String greeting = "Hello";

void main() {
    IO.println(greeting + ", " + readName());
    var langs = List.of("Java", "Kotlin", "Scala");   // List needs no import
    IO.println(langs);
}

String readName() {
    return IO.readln("Your name: ");
}
```

On Java 25 you run it with `java Hi.java`. On Java 21 the closest thing is the preview of the same idea (JEP 445), which does need `--enable-preview`, and where `System.out.println` is still required. This is real output from the build machine:

Hi.java (Java 21 preview version; this was run) JAVA

```java
String greeting = "Hello";

void main() {
    System.out.println(greeting + ", compact source file");
    System.out.println(helper(20));
}

int helper(int n) { return n * 2 + 2; }
```

Terminal TERMINAL

```
$ java --enable-preview --source 21 Hi.java
Note: Hi.java uses preview features of Java SE 21.
Note: Recompile with -Xlint:preview for details.
Hello, compact source file
42
```

NOTE
A compact source file is a convenience for scripts, exercises and learning. A growing program is meant to graduate: add a `class` declaration and the file becomes an ordinary one. Real services keep using normal classes, and frameworks such as Spring Boot do not care about any of this.

11.5.2 Module import declarations (JEP 511)

Imports are usually a source of noise: a dozen `import java.util.*` style lines. `import module java.base;` imports, on demand, every public top-level class and interface of every package that the module `java.base` exports. So `List`, `Map`, `Stream`, `Path`, `BigDecimal` all become available at once.

Report.java (Java 25; not run on 21)

```
import module java.base;

public class Report {
    public static void main(String[] args) {
        Map<String, List<Integer>> grades = new TreeMap<>();
        grades.computeIfAbsent("Asha", k -> new ArrayList<>()).add(90);
        System.out.println(grades + " " + Path.of("data").toAbsolutePath().getFileName());
    }
}
```

Two things to know. First, if two imported modules export a class with the same simple name (the classic is `java.util.Date` against `java.sql.Date` after `import module java.sql;`), a use of the simple name is ambiguous and does not compile; a single-type import such as `import java.util.Date;` resolves it. Second, the feature changes the source only, not the program's runtime dependencies: the code still needs the module to be present. Many style guides will probably still prefer explicit imports in long-lived code, so treat module imports as a tool for scripts and teaching, which is what the JEP targets.

11.5.3 Flexible constructor bodies (JEP 513)

Java has always required `super(...)` or `this(...)` to be the first statement of a constructor. That forced awkward code: to validate an argument before calling the superclass constructor you had to hide the check inside a static helper called from the argument expression. JEP 513 (final in 25) allows statements **before** the explicit constructor call, as long as they do not touch the object that is still being built.

The constructor body is now conceptually two parts: the **prologue** (before `super(...)`) and the **epilogue** (after). In the prologue you may validate arguments, compute values to pass up, and assign fields declared in the same class (only fields without an initializer); you may not read `this`, call instance methods, or use `super.something`.

Employee.java (Java 25; not run on 21) JAVA

```java
class Person {
    protected final int age;
    Person(int age) {
        this.age = age;
        describe();                       // may call an overridden method!
    }
    void describe() { System.out.println("Person " + age); }
}
```

Employee.java (Java 25; not run on 21) (continued) JAVA

```java
class Employee extends Person {
    private final String officeId;

    Employee(int age, String officeId) {
        if (age < 18 || age > 67)                       // prologue: fail fast
            throw new IllegalArgumentException("Age out of range: " + age);
        this.officeId = officeId;                       // prologue: assign own field
        super(age);                                     // now call the superclass constructor
    }

    @Override void describe() { System.out.println("Employee " + age + " @" + officeId); }
}
```

What to notice: in old Java, `Person`'s constructor calling the overridden `describe()` saw `officeId == null`, because the subclass field was assigned only after `super(...)` returned. The new rule lets `Employee` initialise `officeId` first, so the overridden method sees the real value. It is a fix for a classic trap (calling overridable methods from constructors, covered in Part 3), but the better advice remains: do not call overridable methods from constructors at all. The fail-fast validation before `super(...)` is the more common use. Record canonical constructors still cannot call `this(...)` or `super(...)` explicitly.

11.5.4 Scoped values (JEP 506)

Frameworks need to carry context through many layers of calls: the current user, a request id, a transaction. Passing it as a parameter through every method is noisy. The traditional answer is `ThreadLocal`, but it has three problems: it is mutable (any code can `set`), it lives until you `remove` it (or the thread ends, which in a thread pool may never happen, a classic leak), and child threads inherit a *copy*, which is costly when you have millions of virtual threads.

A **scoped value** is the better tool for sharing *immutable* data downward through a call chain. A value is **bound** for exactly the duration of a call (`run` or `call`); inside it, every callee in the same thread, however deep, can `get()` it; when the call returns, the binding disappears automatically. There is no `set` and nothing to remove. A nested `where(...).run(...)` can *rebind* the same value for its own callees; the outer value reappears afterwards.

Scoped values were previews in Java 21 to 24 and are final in 25. The version below uses the same API shape and ran on Java 21 with `--enable-preview`. On 25 you omit the flag.

Sv.java (preview on 21; final in 25) JAVA

```java
public class Sv {
    private static final ScopedValue<String> USER = ScopedValue.newInstance();

    static void handle() {
        System.out.println("user=" + (USER.isBound() ? USER.get() : "unbound"));
    }

    public static void main(String[] args) {
        handle();
        ScopedValue.where(USER, "alice").run(() -> {
            handle();
            ScopedValue.where(USER, "bob").run(Sv::handle);   // rebinding
            handle();
        });
        handle();
    }
}
```

Terminal TERMINAL

```
$ java --enable-preview --source 21 Sv.java
Note: Sv.java uses preview features of Java SE 21.
Note: Recompile with -Xlint:preview for details.
user=unbound
user=alice
user=bob
user=alice
user=unbound
```

The output tells the story: unbound outside, `alice` inside, `bob` during the nested call, `alice` again when it returns, and unbound at the end. Without the explicit `isBound()` check, `get()` on an unbound value throws an exception.

Scoped values also work with structured concurrency: threads forked inside a `StructuredTaskScope` (still a preview API in 25, the fifth preview, JEP 505) inherit the binding without copying. Ordinary thread pools do not inherit bindings. `ThreadLocal` is not obsolete: it remains right for per-thread caches (a non-thread-safe formatter object per thread) and for mutable per-thread state.

> **REAL WORLD**
> Spring and other frameworks store request context (security principal, MDC logging fields, tracing ids) in `ThreadLocal`. With virtual threads this still works, but migrating such context to scoped values is a topic for framework authors. As an application developer, remember the pooled-thread leak: always clean up a `ThreadLocal` in a `finally` block.

11.5.5 Compact object headers (JEP 519)

Every object in the heap begins with a header holding GC and locking information and a pointer to its class (Part 9 shows the layout). Shrinking that header saves memory on every one of millions of objects, which also means fewer cache misses and less GC work. Compact object headers, an experimental feature in Java 24, are a **product feature** in Java 25 but **opt-in**:

Terminal — TERMINAL

```
$ java -XX:+UseCompactObjectHeaders -jar app.jar
```

Oracle's JEP reports, on the benchmarks they ran, about 22% less heap and 8% less CPU time on one SPECjbb2015 configuration, 15% fewer collections on another, and about 10% less time for a highly parallel JSON parser. Treat these as upper-end examples, not promises: your gains depend on how many small objects you allocate. The JEP also notes that the layout was used in hundreds of Amazon production services. Because it is not the default in 25, enable it deliberately and measure with your real workload. This flag does not exist on Java 21 (the JVM prints `Unrecognized VM option`). Java 27 makes it the default; see the next chapter.

11.5.6 Faster startup: the AOT cache story (JEP 514, JEP 515)

A JVM starts cold: it loads and links classes, interprets bytecode, and only after profiling does the JIT produce fast code. For short-lived services, serverless functions and CLI tools this warm-up is the cost. Project Leyden's answer is the **AOT cache**: record what a typical run of your application loads and links, store it in a file, and reuse it on the next start.

Java 24 (JEP 483) introduced the cache with a two-step workflow (record, then create). Java 25 made it ergonomic (JEP 514): one command creates the cache, and production starts use it.

Creating and using an AOT cache (Java 25; not run on 21) — TERMINAL

```
$ java -XX:AOTCacheOutput=app.aot -cp app.jar com.example.App
$ java -XX:AOTCache=app.aot -cp app.jar com.example.App
```

The first command runs a *training run* of your application, and when it exits the JVM writes `app.aot`. The second uses the cache. Use a realistic training run (exercise the code paths that matter) and run production on the same JDK and classpath that created the cache; the cache is tied to them. JEP 515 additionally lets the cache carry method profiles collected during training, so the JIT can reach optimised code sooner after startup. Per the JEP, you can keep the older two-step approach (`-XX:AOTMode=record`, then `-XX:AOTMode=create`) when you train on a small machine and build the cache elsewhere.

The idea is not new on 21. Application class-data sharing (AppCDS) has been available since Java 13 and gives a part of the benefit. This is a real run on 21:

AppCDS dynamic archive on Java 21 (the application is a 12-line toy; no timing claimed) — TERMINAL

```
$ java -XX:ArchiveClassesAtExit=app.jsa -cp . App
504500
$ java -XX:SharedArchiveFile=app.jsa -cp . App
504500
```

The AOT cache extends this by caching loaded and linked state and, with JEP 515, profiling data. The sizes of the wins depend on how many classes your application loads; Spring Boot applications with thousands of classes see the most benefit.

GOTCHA
An AOT or CDS archive is not portable across JDK versions or vendors, and a changed classpath makes the JVM ignore it (sometimes with only a warning). Build the cache in the same pipeline stage and image as the one that runs the service, not on your laptop.

INTERVIEW ANGLE
What is new in Java 25 that a backend developer should know? Scoped values (final) as a safer alternative to `ThreadLocal` for immutable context; flexible constructor bodies for validating before `super(...)`; compact source files and instance `main` (and module imports) that simplify scripts and teaching; compact object headers (opt-in, less heap) and easier AOT cache creation for faster startup. Also that structured concurrency, stable values and primitive patterns are still in preview in 25.

Check yourself

1. Which `main` signature does Java 25's launcher try first, and which last?
2. What does `import module java.base;` import, and what ambiguity can appear with `java.sql`?
3. What is wrong with this code in Java 21? `Child(int x) { if (x < 0) throw new IllegalArgumentException(); super(x); }` What changes in Java 25?
4. List two weaknesses of `ThreadLocal` that scoped values fix.
5. How do you turn on compact object headers in Java 25, and what is the default?

Answers

1. First `static void main(String[] args)`; last an instance `void main()` (static before instance, with arguments before without).
2. All public top-level types in the packages exported by the module. `java.util.Date` and `java.sql.Date` clash by simple name; add an explicit single-type import.
3. It does not compile: `super(...)` must be first. Java 25 (JEP 513) allows validation statements in the prologue as long as they do not use `this`.
4. Mutable at any time, must be removed manually (leaks in pools), and inherited copies are expensive for child threads. Scoped values are immutable, bound for a bounded scope and not copied.
5. `-XX:+UseCompactObjectHeaders`; it is off by default in 25.

KEY POINTS
- Java 25 (Sept 2025) is the current LTS: JEPs 506, 511, 512, 513 and 519 are final or product features.
- Compact source files, instance `main`, `IO.println` and module imports cut ceremony for scripts and teaching.
- Flexible constructor bodies allow validation and field assignment before `super(...)`, without touching `this`.
- Scoped values give immutable, automatically scoped context; `ThreadLocal` remains for per-thread caches.
- Compact headers are opt-in in 25; AOT caches are created with `-XX:AOTCacheOutput` and used with `-XX:AOTCache`.

11.6 Java 26, Java 27 and the road ahead

Two releases have shipped since Java 25 LTS: Java 26 in March 2026 and Java 27 on 15 September 2026. Neither is an LTS, so most production systems will not run them, but they show where the platform is heading and several changes will reach everyone through the next LTS (expected September 2027). Only items confirmed against the OpenJDK release lists are described here.

11.6.1 Java 26 (March 2026)

JEP	Title	What it means for you
500	Prepare to Make Final Mean Final	Mutating a `final` field via deep reflection (`setAccessible`) will be warned about now and refused in the future
504	Remove the Applet API	The long-deprecated `java.applet` package is gone
516	AOT Object Caching with Any GC	The AOT cache can be used with any garbage collector, not only one
517	HTTP/3 for the HTTP Client	`java.net.http.HttpClient` can speak HTTP/3 (QUIC)
522	G1 throughput	Less synchronisation between application threads and G1, improving throughput
524	PEM Encodings (2nd preview)	Read and write PEM text for keys and certificates
525	Structured Concurrency (6th preview)	Still preview; refinement continues
526	Lazy Constants (2nd preview)	Values computed on first use, then treated as constants
529	Vector API (11th incubator)	SIMD API still incubating; waits for Valhalla
530	Primitive Types in Patterns (4th preview)	`switch` and `instanceof` over primitives

Java 26 JEPs worth knowing.

Two of these deserve a closer look.

Final means final (JEP 500). A `final` field is supposed to be immutable, and the JIT and the memory model assume so. Yet reflection with `setAccessible(true)` can overwrite a final instance field. Serialization libraries and some mocking tools rely on this. JEP 500 makes the JDK warn on such mutation now, as the first step to disallowing it by default, and the JVM can be told how strict to be (check the JEP for the exact option name before you use it). If your tests or frameworks mutate finals, expect warnings after upgrading. The rule of thumb is unchanged: write immutable classes properly (records, final fields) and do not depend on reflection to change them.

HTTP/3 (JEP 517). HTTP/3 runs over QUIC (UDP) instead of TCP, which avoids head-of-line blocking between streams and speeds up connection setup. The client from Part 5 gets support for it; you opt in when you build the client by asking for the HTTP/3 version, and the client can fall back when the server does not support it. As with HTTP/2, your application code (`client.send(request, handler)`) does not change.

11.6.2 Java 27 (15 September 2026)

JEP	Title	What it means for you
523	G1 as the default GC everywhere	No more silent fall back to Serial on small machines
527	Post-Quantum Hybrid Key Exchange for TLS 1.3	TLS can combine classical and post-quantum key exchange
534	Compact Object Headers by Default	The smaller header (from Java 25) is on without a flag
536	JFR In-Process Data Redaction	Sensitive values can be removed from recordings
531	Lazy Constants (3rd preview)	Still preview
532	Primitive Types in Patterns (5th preview)	Still preview
533	Structured Concurrency (7th preview)	Still preview
538	PEM Encodings (3rd preview)	Still preview
537	Vector API (12th incubator)	Still incubating

Java 27 JEPs worth knowing.

G1 everywhere (JEP 523). Since Java 9 the JVM picks G1 on "server-class" machines and Serial otherwise (fewer than 2 CPUs or under about 1.8 GB). Containers with small limits therefore ran Serial without anyone asking. From 27, G1 is chosen regardless of processor count and heap size, because G1 has improved enough to match Serial's throughput and footprint at all sizes. If you run tiny containers and want the old behaviour, name the collector yourself with `-XX:+UseSerialGC`; an explicit choice always wins. Check GC logs and memory (`-Xlog:gc*`) after upgrading small services.

Compact headers by default (JEP 534). What was opt-in in 25 becomes the default. The usual reason to care is a library or tool that assumes the old header layout, such as agents or off-heap code that computes object sizes. Check

that your tooling supports it, and use the flag in the negative form (`-XX:-UseCompactObjectHeaders`) as an escape hatch if it does not.

Lazy constants (JEP 526/531). Formerly "stable values" (JEP 502, preview in 25). The use case: a field you want to initialise lazily (a logger, a heavyweight object, a lookup table) but that the JIT should still treat as a constant once set, which `final` plus lazy initialisation could not give you. It is a preview; the API changed between previews and I am deliberately not showing code for it. Read the current JEP before using it.

> **NOTE**
> Features that have been preview for several releases (structured concurrency in 25, 26 and 27; primitive patterns; the Vector API) are not abandoned. They are waiting either for feedback or for other projects. Use previews to learn, not in production.

11.6.3 The road ahead: the big projects

The next years of Java are driven by a handful of long-running OpenJDK projects. They are described here at the level of intent; consult openjdk.org/projects for each one's current state, because the details move.

Project	Goal and status in short
Amber	Smaller, more expressive language: `var`, records, sealed types, pattern matching, text blocks, simple `main`. Delivered through the releases in this Part and still adding pattern features.
Loom	Cheap concurrency: virtual threads (final in 21), `synchronized` without pinning (24), scoped values (25). Structured concurrency is still in preview.
Panama	Safe, fast access to native code and memory: the Foreign Function and Memory API (final in 22), the Vector API (incubating), replacing JNI.
Valhalla	Value classes: objects without identity that the JVM can lay out flat in memory, plus eventually generics over primitives. It is a long-running effort; the Vector API and some compact-header decisions are linked to it.
Leyden	Faster startup and warm-up through ahead-of-time work: class-data sharing, AOT cache (24, 25, 26), method profiles.
Lilliput	Smaller object headers; delivered as compact object headers (25, default in 27).

Major OpenJDK projects.

The pattern behind these projects is consistent: add the capability as a preview or incubator, gather feedback for a few releases, finalise, and keep compatibility. Code written for Java 8 still runs on Java 27 in nearly all cases. The costs of upgrading are mostly in the surrounding ecosystem: removed APIs, stronger encapsulation, agents and bytecode libraries that need new versions.

11.6.4 An upgrade checklist, with real commands

Most upgrade pain comes from three sources: new APIs used by mistake, internals that are now locked, and libraries that are too old. Each has a cheap early warning.

1. Compile against the target release, not just the target syntax. `javac -source 17` still lets you call APIs from the JDK you are compiling with. `--release` also restricts the API to what the target version has. On the build machine (JDK 21) a call to `List.reversed()` (new in 21) is rejected when targeting 17:

```
Terminal                                                    TERMINAL
$ javac --release 17 Rel.java
Rel.java:2: error: cannot find symbol
... System.out.println(l.reversed()); } }
                       ^
  symbol:   method reversed()
  location: variable l of type ArrayList<Integer>
1 error
$ javac --release 21 Rel.java && java Rel
[2, 1]
```

2. Strong encapsulation. Since Java 16 (JEP 396) and finally in 17 (JEP 403), reflection cannot open non-public members of JDK classes unless the module opens them. Libraries from the Java 8 era that poke at `String.value` or `sun.misc` internals fail on first use:

Terminal TERMINAL

```
$ java Priv.java
Exception in thread "main" java.lang.reflect.InaccessibleObjectException: Unable to make field
private final byte[] java.lang.String.value accessible: module java.base does not "opens
java.lang" to unnamed module @3e77a1ed
$ java --add-opens java.base/java.lang=ALL-UNNAMED Priv.java
[B@12405818
```

(The hash after `@` and after `[B@` varies between runs.) `--add-opens` is a stop-gap flag: it lets the old library run while you wait for a fixed version. Find the offenders before the upgrade with `jdeps --jdk-internals app.jar`, which lists the uses of internal JDK APIs and often suggests the supported replacement.

3. Removed and changed things. Between 8 and 25 some pieces were removed from the JDK: the Java EE and CORBA modules in 11 (`javax.xml.bind` now needs a dependency), Nashorn in 15, the applet API in 26, and the Security Manager is permanently disabled since 24. The default charset became UTF-8 in 18, which can change how a program reads files that were written with the platform charset. A GC flag you tuned on Java 8 may no longer exist; the JVM prints a warning, or in some cases refuses to start, for unknown options.

4. Run with the new JVM in a staging environment and compare behaviour under load, not just whether the unit tests pass. Watch for changes in GC choice (see JEP 523 above), memory and startup.

11.6.5 Which Java should you choose?

- **New production project:** Java 25 LTS (or 21 if your platform, framework or vendor does not support 25 yet). Check the framework's compatibility matrix first; Spring Boot, for instance, supports particular Java baselines per release.
- **Existing Java 8 or 11 system:** plan a stepped upgrade to 17 then 21 or 25. Expect work around removed modules (`javax.xml.bind`, Nashorn), strong encapsulation (`--add-opens`), and library upgrades. Run tests on each step.
- **Experimenting:** the newest release (27) to try upcoming features, never as the production base.
- **Always:** use a vendor build that receives security updates (Temurin, Corretto, Oracle JDK, Red Hat, Azul and others) and update to the latest patch of your LTS every quarter.

TIP
Interviewers like candidates who can say why a team stays on an LTS and what they would check before moving: dependency support, GC and memory behaviour under load, startup, removed APIs, and a rollback plan.

INTERVIEW ANGLE
How would you decide whether to upgrade a service from Java 17 to 25? Check that the framework and all libraries and agents support 25; compile with `--release 25` and run the full test suite; look for warnings about reflection on final fields and illegal access; compare GC logs, memory (try compact headers) and latency under a realistic load test; then roll out gradually with a quick rollback. Benefits to name: virtual threads and sequenced collections from 21, scoped values and flexible constructors in 25, and lower memory and faster startup options.

Check yourself

1. What changes about the default garbage collector in Java 27, and how do you keep the old behaviour for a tiny container?
2. A library you use changes a `final` field with reflection. What do you expect after upgrading to Java 26?
3. Which of these are still preview in Java 27: structured concurrency, compact object headers, primitive patterns?
4. Which OpenJDK project delivered virtual threads, and which delivered the Foreign Function and Memory API?
5. Is Java 26 an LTS release? Which is the next LTS after 25?

Answers

1. G1 becomes the default in all environments (JEP 523); Serial was previously chosen on small machines. Keep Serial with `-XX:+UseSerialGC`.
2. A warning (the first step toward refusing such mutation, JEP 500); check the JEP for the strictness option.
3. Structured concurrency (7th preview) and primitive patterns (5th preview). Compact object headers are on by default in 27.
4. Loom and Panama.
5. No; the LTS releases are 8, 11, 17, 21, 25. The next after 25 is expected two years later, in September 2027.

KEY POINTS

- Java 26 (March 2026) and 27 (September 2026) are non-LTS; important items: final-means-final warnings, HTTP/3, G1 everywhere, compact headers by default.
- Previews and incubators (structured concurrency, lazy constants, primitive patterns, Vector API) are being refined across releases; do not ship them.
- Amber, Loom, Panama, Valhalla, Leyden and Lilliput are the projects shaping the platform.
- Choose an LTS with a supported vendor build, check framework support, and upgrade step by step with tests and a rollback plan.

PART 12

Design Patterns and Best Practices

From reusable designs to Effective Java habits, clean code, secure coding and performance rules of thumb.

In this part

12.1 Creational patterns in modern Java
12.2 Structural and behavioural patterns
12.3 Effective Java in practice
12.4 Clean code and code smells in Java
12.5 Secure coding in Java
12.6 Performance rules of thumb

12.1 Creational patterns in modern Java

A design pattern is a named, proven solution to a problem that keeps coming back. The value is less in the code than in the vocabulary: when a colleague says "wrap it in a decorator" or "make that a builder", both of you instantly picture the same structure. Interviewers use the names for the same reason.

Patterns also age. The classic 1994 book was written for C++ and early Java, and several patterns shrank or vanished when the language gained lambdas, records, enums, sealed types and pattern matching. This Part therefore shows each pattern in the form you would actually write today, and says plainly when the language already does the job.

Creational patterns answer one question: **who creates the object, and how much of the construction does the caller need to know?** We cover four: Singleton, Factory, Builder and Prototype.

12.1.1 Singleton: exactly one instance

Problem. Some things should exist once per JVM: a configuration holder, an ID generator, a registry. A global variable would work but gives you no control over creation. A singleton is a class that (1) hides its constructor, (2) creates its single instance itself and (3) hands it out through one access point.

There are three implementations worth knowing. Ranked from best to worst for most code: the enum, the holder idiom, and the eager static field. A fourth, double-checked locking, is mostly of interview interest.

SingletonDemo.java — JAVA

```
import java.lang.reflect.Constructor;
import java.util.concurrent.atomic.AtomicInteger;

public class SingletonDemo {

    // 1. Enum singleton: the JVM guarantees one instance, even against reflection/serialization.
    enum AppConfig {
        INSTANCE;
        private final String env = System.getProperty("app.env", "dev");
        String env() { return env; }
    }

    // 2. Initialization-on-demand holder: lazy, thread-safe, no locks in our code.
    static final class IdGenerator {
        private static final AtomicInteger created = new AtomicInteger();
        private final AtomicInteger next = new AtomicInteger(1000);

        private IdGenerator() { created.incrementAndGet(); }

        private static class Holder {
            static final IdGenerator INSTANCE = new IdGenerator();
        }

        static IdGenerator getInstance() { return Holder.INSTANCE; }
        int nextId() { return next.getAndIncrement(); }
    }

    // 3. Eager singleton with a private constructor: reflection can still break it.
    static final class Registry {
```

SingletonDemo.java (continued) JAVA

```
        static final Registry INSTANCE = new Registry();
        private Registry() { }
    }

    public static void main(String[] args) throws Exception {
        System.out.println("env = " + AppConfig.INSTANCE.env());
        boolean same = AppConfig.INSTANCE == AppConfig.valueOf("INSTANCE");
        System.out.println("same enum instance: " + same);

        System.out.println("instances before first use: " + IdGenerator.created.get());
        System.out.println("id " + IdGenerator.getInstance().nextId());
        System.out.println("id " + IdGenerator.getInstance().nextId());
        System.out.println("instances after: " + IdGenerator.created.get());

        Constructor<Registry> c = Registry.class.getDeclaredConstructor();
        c.setAccessible(true);
        Registry second = c.newInstance();
        System.out.println("registry broken by reflection: " + (second != Registry.INSTANCE));

        try {
            Constructor<?> ec = AppConfig.class.getDeclaredConstructors()[0];
            ec.setAccessible(true);
            ec.newInstance("X", 1);
        } catch (Exception e) {
            System.out.println("enum reflection: " + e.getClass().getSimpleName()
                + ": " + e.getMessage());
        }
    }
}
```

Terminal TERMINAL

```
$ java SingletonDemo.java
env = dev
same enum instance: true
instances before first use: 0
id 1000
id 1001
instances after: 1
registry broken by reflection: true
enum reflection: IllegalArgumentException: Cannot reflectively create enum objects
```

What to notice

- `enum AppConfig { INSTANCE; }` is the simplest correct singleton. The language guarantees one instance per class loader, and the JVM refuses reflective construction (`Cannot reflectively create enum objects`). Serialization also preserves identity for enums. Effective Java recommends it as the default.
- The holder idiom (`IdGenerator.Holder`) is lazy for free. The nested class `Holder` is not initialized until `getInstance()` first touches it, and class initialization is thread-safe by the JVM specification. The output shows `instances before first use: 0`, proving the laziness.
- A plain `private` constructor is only a convention against reflection: `setAccessible(true)` created a second `Registry`. (On a modular application the module system can block that, but do not rely on it.)

Double-checked locking

Before the holder idiom was popular, people wrote lazy singletons with a synchronized check. The correct version needs `volatile`, and it is easy to get wrong:

DoubleChecked.java JAVA

```
public final class LazyService {
    private static volatile LazyService instance;   // volatile is essential

    private LazyService() { }

    public static LazyService getInstance() {
        LazyService local = instance;               // one volatile read on the fast path
        if (local == null) {
            synchronized (LazyService.class) {
                local = instance;
                if (local == null) {
```

DoubleChecked.java (continued) JAVA

```
                    instance = local = new LazyService();
                }
            }
        }
        return local;
    }
}
```

Without `volatile`, another thread may see a non-null `instance` whose fields are not yet visible, because the compiler and CPU may reorder the stores inside the constructor with the store to `instance`. Part 8 explains this with the Java memory model (happens-before). The holder idiom gives the same result with no locking code at all, so prefer it.

GOTCHA

Singletons and testing A singleton is global state. Code that calls `AppConfig.INSTANCE` directly is hard to unit test, because you cannot substitute a fake. The usual cure is to create the object once in `main` (or let Spring do it) and pass it to whoever needs it through a constructor. That keeps "one instance" without making every class depend on a global. Spring beans are singletons in exactly this sense: one per application context, but not enforced by the class itself.

INTERVIEW ANGLE

How do you make a thread-safe singleton in Java, and which way do you prefer? Enum singleton (`enum X { INSTANCE }`) when no lazy loading is needed, or the initialization-on-demand holder when it is. Both rely on JVM class initialization, which is thread-safe. Double-checked locking works only with a `volatile` field. Mention that reflection can break a plain private constructor and that the enum cannot be broken that way.

12.1.2 Factory: hide which class gets created

Problem. `new Circle(1.5)` ties the caller to a concrete class. If the caller really needs "a shape from this description" or "a notifier for this channel", the choice of class should live in one place that can change.

Three flavours cover almost every real use:

- **Static factory method**: a named `static` method on the class or interface (`List.of`, `Integer.valueOf`, `LocalDate.of`). It can have a descriptive name, return a cached instance, return a subtype and validate. Effective Java makes this the first item.
- **Simple factory**: one method with a `switch` that picks the class from a key, as `of(String kind, ...)` below. With a sealed interface the compiler can also check that you covered every subtype elsewhere.
- **Registry of suppliers**: a `Map<String, Supplier<T>>` so that new kinds can be registered without editing a `switch`. This replaces the heavyweight "Abstract Factory class hierarchy" of the original book.

FactoryDemo.java JAVA

```
import java.util.List;
import java.util.Map;
import java.util.function.Supplier;

public class FactoryDemo {

    sealed interface Shape permits Circle, Rect, Square { double area(); }
    record Circle(double r) implements Shape {
        public double area() { return Math.PI * r * r; }
    }
    record Rect(double w, double h) implements Shape {
        public double area() { return w * h; }
    }
    record Square(double side) implements Shape {
        public double area() { return side * side; }
    }

    // Static factory method with a descriptive name and validation.
    static Shape of(String kind, double... dims) {
        return switch (kind) {
            case "circle" -> new Circle(dims[0]);
            case "rect"   -> new Rect(dims[0], dims[1]);
            case "square" -> new Square(dims[0]);
            default -> throw new IllegalArgumentException("unknown shape: " + kind);
        };
    }
```

FactoryDemo.java (continued) JAVA

```
    // A registry of suppliers: new kinds can be added without editing a switch.
    interface Notifier { String send(String msg); }
    static final Map<String, Supplier<Notifier>> NOTIFIERS = Map.of(
        "email", () -> msg -> "EMAIL: " + msg,
        "sms",   () -> msg -> "SMS: " + msg
    );

    static Notifier notifier(String channel) {
        Supplier<Notifier> s = NOTIFIERS.get(channel);
        if (s == null) throw new IllegalArgumentException("no channel " + channel);
        return s.get();
    }

    public static void main(String[] args) {
        List<Shape> shapes = List.of(of("circle", 1.5), of("rect", 2, 3), of("square", 4));
        for (Shape s : shapes) {
            System.out.printf("%-28s area=%.2f%n", s, s.area());
        }
        System.out.println(notifier("sms").send("order shipped"));
        try {
            of("hexagon");
        } catch (IllegalArgumentException e) {
            System.out.println("caught: " + e.getMessage());
        }
        // JDK examples of static factories
        System.out.println(List.of(1, 2, 3) + " " + Integer.valueOf(127)
            + " " + java.time.LocalDate.of(2026, 10, 11));
    }
}
```

Terminal TERMINAL

```
$ java FactoryDemo.java
Circle[r=1.5]                area=7.07
Rect[w=2.0, h=3.0]           area=6.00
Square[side=4.0]             area=16.00
SMS: order shipped
caught: unknown shape: hexagon
[1, 2, 3] 127 2026-10-11
```

The caller sees `Shape` and `Notifier`, never `Circle` or the lambda. Adding a `triangle` means changing the factory only. Adding `push` as a notification channel means adding one map entry.

TIP

Pick the lightest form that works If there is one product type and the choice is made once, a static method is enough. Reach for the registry when the set of kinds is open (plugins, configuration-driven choices). Reach for a full factory class hierarchy almost never; if you find yourself writing `AbstractWidgetFactory`, check whether a lambda would do.

12.1.3 Builder: construct complex objects step by step

Problem. A class with many fields, several of them optional, produces either a telescope of constructors (`Order(a)`, `Order(a, b)`, `Order(a, b, c, d)`) or a constructor call full of `null, false, 0`. Nobody can tell what the arguments mean.

A **builder** is a separate mutable object with a method per field and a final `build()`. Calls chain because each setter returns `this`. The finished product can then be immutable, and `build()` is the right place to enforce cross-field rules.

BuilderDemo.java JAVA

```
import java.util.ArrayList;
import java.util.List;

public class BuilderDemo {

    static final class Order {
        private final String customer;
        private final List<String> items;
        private final boolean express;
        private final String note;
```

BuilderDemo.java (continued) JAVA

```
        private Order(Builder b) {
            this.customer = b.customer;
            this.items = List.copyOf(b.items);
            this.express = b.express;
            this.note = b.note;
        }

        static Builder builder(String customer) { return new Builder(customer); }

        @Override public String toString() {
            return "Order[" + customer + ", items=" + items + ", express=" + express
                + ", note=" + note + "]";
        }

        static final class Builder {
            private final String customer;
            private final List<String> items = new ArrayList<>();
            private boolean express;
            private String note = "";

            private Builder(String customer) { this.customer = customer; }

            Builder item(String name) { items.add(name); return this; }
            Builder express(boolean v) { this.express = v; return this; }
            Builder note(String n) { this.note = n; return this; }

            Order build() {
                if (customer == null || customer.isBlank())
                    throw new IllegalStateException("customer required");
                if (items.isEmpty())
                    throw new IllegalStateException("at least one item required");
                if (express && items.size() > 3)
                    throw new IllegalStateException("express orders hold at most 3 items");
                return new Order(this);
            }
        }
    }

    // Records need no builder for small cases: use a compact constructor for validation.
    record Point(int x, int y) {
        Point {
            if (x < 0 || y < 0) throw new IllegalArgumentException("negative coordinate");
        }
    }

    public static void main(String[] args) {
        Order o = Order.builder("Asha").item("book").item("pen").express(true)
                       .note("gift wrap").build();
        System.out.println(o);

        try {
            Order.builder("Ravi").build();
        } catch (IllegalStateException e) {
            System.out.println("rejected: " + e.getMessage());
        }
        try {
            Order.builder("Ravi").item("a").item("b").item("c").item("d").express(true).build();
        } catch (IllegalStateException e) {
            System.out.println("rejected: " + e.getMessage());
        }
        System.out.println(new Point(1, 2));
        // StringBuilder and Stream.Builder are builders in the JDK
        System.out.println(new StringBuilder().append("a").append(1).append(true));
    }
}
```

Terminal TERMINAL

```
$ java BuilderDemo.java
Order[Asha, items=[book, pen], express=true, note=gift wrap]
rejected: at least one item required
rejected: express orders hold at most 3 items
Point[x=1, y=2]
a1true
```

What to notice

- The `Order` constructor is private and takes the builder. The only way to obtain an `Order` is through `build()`, so an invalid order cannot exist.
- `List.copyOf(b.items)` snapshots the list. If the builder were reused afterwards, the already-built order would not change. That is the defensive copy discussed again in the Effective Java chapter.
- The rule "express orders hold at most 3 items" involves two fields, which a setter could not check on its own. `build()` sees them all.
- A `record` with a compact constructor is often all you need for a handful of required fields. Use a builder when there are many optional parameters or when construction has several steps.

REAL WORLD
In real code You meet builders constantly: `StringBuilder`, `Stream.builder()`, `HttpRequest.newBuilder()`, `HttpClient.newBuilder()`, Spring's `ResponseEntity.ok().header(...).body(...)`, and Lombok's `@Builder`, which generates the boilerplate shown above. Lombok is an annotation processor and not part of the JDK; many teams use it, others prefer writing builders or using records.

12.1.4 Prototype: create by copying

Problem. Sometimes building an object from scratch is expensive or fiddly, but copying a ready-made one and tweaking it is cheap: a document template, a configured test fixture, a game unit. The **prototype** pattern says: give the object a way to produce a copy of itself.

Java has `Object.clone()` and the `Cloneable` marker interface for this, and both are widely regarded as a design mistake: `clone()` is protected, bypasses constructors, is shallow by default and has a confusing contract. Effective Java advises a **copy constructor** or **copy factory** instead. The program contrasts the options and shows the classic trap of a shallow copy.

PrototypeDemo.java JAVA

```
import java.util.ArrayList;
import java.util.List;

public class PrototypeDemo {

    static class Template {
        String title;
        List<String> tags;

        Template(String title, List<String> tags) {
            this.title = title;
            this.tags = tags;
        }

        // Copy constructor: the recommended replacement for clone()
        Template(Template other) {
            this.title = other.title;
            this.tags = new ArrayList<>(other.tags);   // deep copy of the mutable part
        }

        Template shallowCopy() { return new Template(title, tags); } // shares the list!

        @Override public String toString() { return title + tags; }
    }

    static class LegacyDoc implements Cloneable {
        int[] data = {1, 2, 3};
        @Override public LegacyDoc clone() {
            try {
                return (LegacyDoc) super.clone();        // shallow: data array is shared
            } catch (CloneNotSupportedException e) {
                throw new AssertionError(e);
            }
        }
    }

    public static void main(String[] args) {
        Template base = new Template("Report", new ArrayList<>(List.of("draft")));

        Template shallow = base.shallowCopy();
```

PrototypeDemo.java (continued) JAVA

```
        shallow.tags.add("shared-bug");
        System.out.println("after shallow edit, base = " + base);

        Template base2 = new Template("Report", new ArrayList<>(List.of("draft")));
        Template deep = new Template(base2);
        deep.title = "Report (copy)";
        deep.tags.add("v2");
        System.out.println("base2 = " + base2 + "  deep = " + deep);

        LegacyDoc a = new LegacyDoc();
        LegacyDoc b = a.clone();
        b.data[0] = 99;
        System.out.println("clone() shares the array: a.data[0] = " + a.data[0]);
    }
}
```

Terminal TERMINAL

```
$ java PrototypeDemo.java
after shallow edit, base = Report[draft, shared-bug]
base2 = Report[draft]  deep = Report (copy)[draft, v2]
clone() shares the array: a.data[0] = 99
```

The shallow copy shares the `tags` list with the original, so editing the copy changed `base` too. The copy constructor builds a new `ArrayList`, so the two objects are independent. The same trap exists with `clone()`: it copies the reference to the `int[]`, not the array.

Immutable objects need no prototype at all, because sharing them is safe. Records give you a cheap version of "copy with a change": write a `withTitle(...)` method that calls the canonical constructor.

12.1.5 Choosing among the four

Pattern	Use it when	Modern Java form
Singleton	exactly one shared instance	`enum`, holder class, or a bean created once in `main`/Spring
Factory	the class to create depends on input or config	static factory method, `switch` on sealed type, `Map<String, Supplier<T>>`
Builder	many optional fields, validation across fields	nested `Builder` class; `record` with compact constructor for small cases
Prototype	copying beats constructing	copy constructor or copy factory; avoid `clone()`

Creational patterns at a glance.

Check yourself

1. Why is `enum Config { INSTANCE }` safe against both reflection and serialization attacks while a normal singleton is not?
2. What does this print, and why? `static class A { static { System.out.println("A loaded"); } static A get() { return H.I; } private static class H { static final A I = new A(); } }`, then `main` only declares `A a = null;` and prints `"start"`.
3. What is wrong with this double-checked locking field: `private static LazyService instance;`?
4. A builder is reused: `b.item("x"); Order o1 = b.build(); b.item("y"); Order o2 = b.build();`. In the builder shown in this chapter, does `o1` change when `item("y")` is called? Why?
5. Name two reasons Effective Java prefers a copy constructor over `Cloneable`.
6. When would you choose a registry of `Supplier`s over a `switch` factory?

Answers

1. The language specification forbids reflective instantiation of enum constants, and the serialization mechanism writes only the constant's name and resolves it back to the existing instance. A normal singleton can be duplicated with `setAccessible(true)` or by deserializing a copy.
2. It prints only `start`. `A` is never initialized because no static member is touched and `A a = null;` does not trigger initialization (and the holder is never touched either).

3. It is not `volatile`. Without it, a second thread can observe a non-null reference to an object whose construction is not yet visible, because of instruction reordering.
4. No. The constructor of `Order` calls `List.copyOf`, so each order owns an immutable snapshot of the items at build time.
5. `clone()` bypasses constructors (invariants and `final` fields are awkward) and is shallow by default with a fragile contract (`Cloneable` has no methods; the copy logic lives in `Object`). A copy constructor is type-safe, can take any related type and is explicit about depth.
6. When the set of kinds is open or configuration-driven (plugins, strategies registered at startup), so adding a kind does not mean editing the factory.

KEY POINTS

- A pattern is a vocabulary plus a trade-off, not a template to copy; language features have absorbed parts of several of them.
- Singleton: prefer `enum` or the holder idiom; plain private constructors can be bypassed with reflection; DCL needs `volatile`.
- Factory: static factory methods, a switch over a sealed type, or a map of suppliers cover nearly every need.
- Builder: use for many optional parameters and cross-field validation; snapshot mutable inputs in `build()`.
- Prototype: write a copy constructor and copy mutable parts deeply; avoid `clone()`.

12.2 Structural and behavioural patterns

Creational patterns decide how objects are born. **Structural patterns** decide how objects and classes are composed into larger structures. **Behavioural patterns** decide how objects divide responsibility and talk to each other. This chapter walks through eleven patterns, each in the form a Java 21 programmer would write, with a complete program you can run.

A recurring theme: since Java 8, many behavioural patterns collapse into "pass a lambda". Since Java 17 to 21, the Visitor pattern collapses into a `switch` over a sealed hierarchy. Knowing the classic shape still matters, because libraries, interviews and legacy code use it.

12.2.1 Adapter and Decorator

Adapter: make an incompatible class fit

Problem. You have code that expects interface `Thermometer`, and a third-party `LegacyFahrenheitSensor` you cannot change. An **adapter** implements the interface you want and delegates to the object you have, translating as it goes.

Decorator: add behaviour by wrapping

Problem. You want to add features (buffering, upper-casing, compression, logging) to an object, in any combination, at run time. Subclassing explodes into `BufferedGzipUpperCaseStream` classes. A **decorator** has the same interface as the thing it wraps, holds a reference to it and adds behaviour before or after delegating. Because the type is unchanged, decorators stack.

The JDK's `java.io` package is the textbook example. `InputStreamReader` is an adapter (bytes to characters). `BufferedReader` and `FilterInputStream` subclasses are decorators. The program builds one of each of our own and then plugs a custom decorator into a real `java.io` chain.

AdapterDecoratorDemo.java JAVA

```
import java.io.BufferedReader;
import java.io.ByteArrayInputStream;
import java.io.FilterInputStream;
import java.io.IOException;
import java.io.InputStream;
import java.io.InputStreamReader;
import java.nio.charset.StandardCharsets;

public class AdapterDecoratorDemo {

    // ---- Adapter: make an old API fit the interface our code expects ----
    interface Thermometer { double celsius(); }
```

AdapterDecoratorDemo.java (continued) JAVA

```
    static class LegacyFahrenheitSensor {          // we cannot change this class
        double readFahrenheit() { return 98.6; }
    }

    static class SensorAdapter implements Thermometer {
        private final LegacyFahrenheitSensor sensor;
        SensorAdapter(LegacyFahrenheitSensor sensor) { this.sensor = sensor; }
        @Override public double celsius() { return (sensor.readFahrenheit() - 32) * 5 / 9; }
    }

    // ---- Decorator on our own interface: wrap, add behaviour, same type ----
    interface Text { String render(); }
    record Plain(String s) implements Text { public String render() { return s; } }
    record Bold(Text inner) implements Text {
        public String render() { return "<b>" + inner.render() + "</b>"; }
    }
    record Shout(Text inner) implements Text {
        public String render() { return inner.render().toUpperCase() + "!"; }
    }

    // ---- Decorator in the style of java.io: a FilterInputStream subclass ----
    static class UpperCaseInputStream extends FilterInputStream {
        UpperCaseInputStream(InputStream in) { super(in); }
        @Override public int read() throws IOException {
            int c = super.read();
            return c == -1 ? -1 : Character.toUpperCase(c);
        }
        @Override public int read(byte[] b, int off, int len) throws IOException {
            int n = super.read(b, off, len);
            for (int i = off; i < off + Math.max(n, 0); i++) {
                b[i] = (byte) Character.toUpperCase(b[i]);
            }
            return n;
        }
    }

    public static void main(String[] args) throws IOException {
        Thermometer t = new SensorAdapter(new LegacyFahrenheitSensor());
        System.out.printf("body temperature: %.1f C%n", t.celsius());

        Text msg = new Shout(new Bold(new Plain("sale")));
        System.out.println(msg.render());
        System.out.println(new Bold(new Shout(new Plain("sale"))).render());

        byte[] bytes = "hello\ndecorators\n".getBytes(StandardCharsets.UTF_8);
        InputStream raw = new ByteArrayInputStream(bytes);
        // InputStreamReader ADAPTS bytes to chars; BufferedReader DECORATES a Reader with buffering
        try (BufferedReader r = new BufferedReader(
                new InputStreamReader(new UpperCaseInputStream(raw), StandardCharsets.UTF_8))) {
            String line;
            while ((line = r.readLine()) != null) System.out.println(line);
        }
    }
}
```

Terminal TERMINAL

```
$ java AdapterDecoratorDemo.java
body temperature: 37.0 C
<B>SALE</B>!
<b>SALE!</b>
HELLO
DECORATORS
```

What to notice

- `Shout(Bold(Plain(..)))` and `Bold(Shout(Plain(..)))` give different results (`<B>SALE</B>!` versus `<b>SALE!</b>`). Order of wrapping is part of the design; document it.
- `UpperCaseInputStream` overrides both `read()` forms of `FilterInputStream`. Forgetting the bulk `read(byte[], int, int)` is a classic bug: `BufferedReader` calls the bulk method, so single-byte conversion alone would not have shown.

- The chain `BufferedReader(InputStreamReader(UpperCaseInputStream(raw)))` reads inside out: raw bytes, then upper-casing, then charset decoding, then line buffering. The same shape appears with `new BufferedOutputStream(new GZIPOutputStream(new FileOutputStream(f)))`.

> **NOTE**
> Adapter versus decorator Both wrap an object. An adapter changes the interface (Fahrenheit sensor becomes `Thermometer`). A decorator keeps the interface and adds behaviour. If a code review asks "which one is this", answer with that sentence.

12.2.2 Facade and Proxy

Facade: one simple door to a complicated subsystem

Problem. Placing an order touches inventory, payment and shipping, and every caller must remember the order of calls and the failure handling. A **facade** is a class with a small, task-oriented API (`placeOrder`) that hides the subsystem. It adds no new behaviour; it reduces what callers need to know. Service classes in layered Spring applications often play exactly this role.

Proxy: a stand-in that controls access

Problem. You want to add access control, lazy loading, caching, logging, retries or transactions around an object without the caller noticing. A **proxy** implements the same interface as the real object and sits in front of it. Unlike a decorator, which is about adding features, a proxy is about controlling access to the target, but in code the two look nearly the same.

Java supports two ways: write the proxy by hand (`CachingRepository`), or generate one at run time with `java.lang.reflect.Proxy`, which creates a class implementing your interfaces and routes every call to one `InvocationHandler`. The second is the foundation of declarative features such as transaction management in Spring (which uses JDK proxies or CGLIB subclassing, depending on the bean).

FacadeProxyDemo.java — JAVA

```
import java.lang.reflect.InvocationHandler;
import java.lang.reflect.InvocationTargetException;
import java.lang.reflect.Proxy;
import java.util.ArrayList;
import java.util.HashMap;
import java.util.List;
import java.util.Map;

public class FacadeProxyDemo {

    // ---- Facade: one simple entry point over several subsystems ----
    static class Inventory {
        boolean reserve(String sku) { System.out.println("  reserve " + sku); return true; }
    }
    static class Payment {
        boolean charge(double amt) { System.out.println("  charge " + amt); return true; }
    }
    static class Shipping {
        void ship(String sku) { System.out.println("  ship " + sku); }
    }

    static class OrderFacade {
        private final Inventory inventory = new Inventory();
        private final Payment payment = new Payment();
        private final Shipping shipping = new Shipping();

        boolean placeOrder(String sku, double price) {
            if (!inventory.reserve(sku)) return false;
            if (!payment.charge(price)) return false;
            shipping.ship(sku);
            return true;
        }
    }

    // ---- Proxy: same interface, controls access to the real object ----
    interface Repository {
        String find(int id);
        void save(String value);
```

FacadeProxyDemo.java (continued) JAVA

```
    }

    static class RealRepository implements Repository {
        public String find(int id) { return "row-" + id; }
        public void save(String value) {
            if (value.isEmpty()) throw new IllegalArgumentException("empty");
        }
    }

    // Static proxy: a caching proxy written by hand
    static class CachingRepository implements Repository {
        private final Repository target;
        private final Map<Integer, String> cache = new HashMap<>();
        CachingRepository(Repository target) { this.target = target; }
        public String find(int id) {
            return cache.computeIfAbsent(id, k -> {
                System.out.println("  (cache miss " + k + ")");
                return target.find(k);
            });
        }
        public void save(String v) { cache.clear(); target.save(v); }
    }

    // Dynamic proxy: one handler can wrap ANY interface (the idea behind Spring AOP)
    @SuppressWarnings("unchecked")
    static <T> T logging(T target, Class<T> iface, List<String> log) {
        InvocationHandler h = (proxy, method, args) -> {
            try {
                Object result = method.invoke(target, args);
                log.add(method.getName() + " ok");
                return result;
            } catch (InvocationTargetException e) {
                log.add(method.getName() + " failed: " + e.getCause().getMessage());
                throw e.getCause();
            }
        };
        return (T) Proxy.newProxyInstance(iface.getClassLoader(), new Class<?>[] {iface}, h);
    }

    public static void main(String[] args) {
        System.out.println("facade:");
        System.out.println("  result " + new OrderFacade().placeOrder("BOOK-1", 12.5));

        Repository cached = new CachingRepository(new RealRepository());
        System.out.println("caching proxy:");
        System.out.println("  " + cached.find(7));
        System.out.println("  " + cached.find(7));

        List<String> log = new ArrayList<>();
        Repository repo = logging(new RealRepository(), Repository.class, log);
        repo.find(1);
        try {
            repo.save("");
        } catch (IllegalArgumentException e) {
            log.add("caller saw " + e.getMessage());
        }
        System.out.println("dynamic proxy log: " + log);
        System.out.println("proxy class is generated: " + Proxy.isProxyClass(repo.getClass()));
    }
}
```

Terminal TERMINAL

```
$ java FacadeProxyDemo.java
facade:
  reserve BOOK-1
  charge 12.5
  ship BOOK-1
  result true
caching proxy:
  (cache miss 7)
  row-7
  row-7
dynamic proxy log: [find ok, save failed: empty, caller saw empty]
proxy class is generated: true
```

A few details deserve attention. `method.invoke` wraps any exception thrown by the target in `InvocationTargetException`; the handler unwraps it with `getCause()` and rethrows, so the caller sees the original `IllegalArgumentException` and not a reflection wrapper. A JDK dynamic proxy needs an interface (it cannot proxy a plain class). The caching proxy is not thread safe because it uses a plain `HashMap`; a real one would use `ConcurrentHashMap` or a cache library.

REAL WORLD
Where proxies hide in production A Spring `@Transactional` or `@Cacheable` method works because the bean you inject is really a proxy: the proxy opens the transaction, calls your method, then commits or rolls back. This explains the famous gotcha that calling a `@Transactional` method from another method of the same class does nothing special: the call goes `this.method()`, bypassing the proxy.

12.2.3 Strategy, Observer and Template Method

Strategy: pass the algorithm in

Problem. A checkout needs a discount rule, and the rules change often (none, 10 percent, flat 15 off). Hard-coding `if (type == ...)` chains makes every new rule an edit of a stable class. A **strategy** is an interchangeable algorithm behind a small interface. Since Java 8, any interface with a single abstract method can be implemented with a lambda, so a strategy is usually just a lambda. `Comparator` is the best-known strategy in the JDK: `list.sort(comparator)` receives the ordering algorithm as an argument.

Observer: publish/subscribe inside one JVM

Problem. One object changes and several others must react, without the subject knowing who they are. The subject keeps a list of listeners and calls them on every change. Swing's `ActionListener`, JavaBeans `PropertyChangeListener` and Spring's `ApplicationEvent` all follow this shape. The old `java.util.Observable` and `Observer` classes were deprecated in Java 9 and should not be used (they are not type safe and `Observable` is a class, not an interface).

Template Method: fixed skeleton, variable steps

Problem. Several jobs follow the same sequence (load, print header, format rows, print count) and differ in some steps. Put the sequence in a `final` method of an abstract class and let subclasses implement the abstract steps. Servlets (`doGet`), `AbstractList` and JUnit's lifecycle are examples. Composition via Strategy is usually more flexible; use Template Method when the skeleton is truly fixed.

StrategyObserverDemo.java JAVA

```java
import java.util.ArrayList;
import java.util.List;
import java.util.function.Consumer;
import java.util.function.DoubleUnaryOperator;

public class StrategyObserverDemo {

    // ---- Strategy: the algorithm is a parameter. A functional interface + lambdas ----
    @FunctionalInterface
    interface DiscountPolicy { double apply(double total); }

    static double checkout(double total, DiscountPolicy policy) { return policy.apply(total); }

    // ---- Observer: subject keeps a list of listeners and notifies them ----
    static class PriceFeed {
        private final List<Consumer<Double>> listeners = new ArrayList<>();
```

StrategyObserverDemo.java (continued) JAVA

```
        Runnable subscribe(Consumer<Double> l) {
            listeners.add(l);
            return () -> listeners.remove(l);          // unsubscribe handle
        }
        void publish(double price) {
            // iterate over a copy so a listener may unsubscribe while being notified
            for (Consumer<Double> l : List.copyOf(listeners)) l.accept(price);
        }
    }

    // ---- Template Method: fixed skeleton, subclasses fill the steps ----
    abstract static class ReportJob {
        final void run() {                  // final: subclasses cannot reorder the steps
            List<String> rows = load();
            System.out.println(header());
            rows.stream().map(this::format).forEach(System.out::println);
            System.out.println("rows: " + rows.size());
        }
        abstract List<String> load();
        abstract String format(String row);
        String header() { return "== report =="; }       // hook with a default
    }

    static class CsvReport extends ReportJob {
        List<String> load() { return List.of("1,Asha", "2,Ravi"); }
        String format(String row) { return row.replace(',', ';'); }
    }

    public static void main(String[] args) {
        DiscountPolicy none = t -> t;
        DiscountPolicy tenPercent = t -> t * 0.9;
        DiscountPolicy flat = t -> Math.max(0, t - 15);
        for (DiscountPolicy p : List.of(none, tenPercent, flat)) {
            System.out.println(checkout(100, p));
        }
        DoubleUnaryOperator composed = ((DoubleUnaryOperator) (t -> t * 0.9)).andThen(t -> t - 5);
        System.out.println("composed: " + composed.applyAsDouble(100));

        PriceFeed feed = new PriceFeed();
        List<String> audit = new ArrayList<>();
        Runnable stopPrinting = feed.subscribe(p -> System.out.println("display: " + p));
        feed.subscribe(p -> audit.add("seen " + p));
        feed.publish(101.5);
        stopPrinting.run();
        feed.publish(102.0);
        System.out.println(audit);

        new CsvReport().run();
    }
}
```

Terminal TERMINAL

```
$ java StrategyObserverDemo.java
100.0
90.0
85.0
composed: 85.0
display: 101.5
[seen 101.5, seen 102.0]
== report ==
1;Asha
2;Ravi
rows: 2
```

What to notice

- The three discount policies are one-liners. The composed policy uses `andThen`, a hint that strategies built from `Function`-like types combine naturally.
- `subscribe` returns a `Runnable` that unsubscribes, which avoids a separate `unsubscribe(listener)` method. The notify loop iterates over `List.copyOf(listeners)`: without the copy, a listener that unsubscribes itself during a callback would trigger a `ConcurrentModificationException`.

- Observer callbacks run on the publisher's thread, so a slow or throwing listener affects everyone. In production, decide whether to catch exceptions per listener or dispatch asynchronously.
- `run()` is `final` in `ReportJob`: that is what makes it a template and not just an inheritance hook.

GOTCHA
Observer memory leaks A subject holds strong references to its listeners. If a short-lived object subscribes to a long-lived subject and never unsubscribes, it can never be garbage collected. This is one of the classic Java leak patterns (see the memory leak chapter in Part 9). Always provide, and use, an unsubscribe path.

12.2.4 Command and State

Command: a request as an object

Problem. You want to queue operations, log them, retry them or undo them. If each action is just a method call, it is gone the moment it returns. A **command** wraps one action and its data in an object with `execute()` (and optionally `undo()`). `Runnable` and `Callable` are commands, and a task submitted to an `ExecutorService` is a command in a queue.

State: behaviour depends on the current state

Problem. An order can be new, paid or shipped, and the allowed actions differ. Scattering `if (status == PAID)` checks across the code is how invalid transitions sneak in. In the **state** pattern, each state encapsulates its own behaviour and decides the next state. A Java `enum` with abstract methods, as below, is a compact and safe way to write it. The alternative for larger machines is a `Map<State, Set<State>>` of legal transitions.

CommandStateDemo.java JAVA

```
import java.util.ArrayDeque;
import java.util.Deque;

public class CommandStateDemo {

    // ---- Command: a request as an object, so it can be queued, logged, undone ----
    static class Editor { final StringBuilder text = new StringBuilder(); }

    interface Command { void execute(); void undo(); }

    record Append(Editor ed, String s) implements Command {
        public void execute() { ed.text.append(s); }
        public void undo() { ed.text.setLength(ed.text.length() - s.length()); }
    }

    static class History {
        private final Deque<Command> done = new ArrayDeque<>();
        void run(Command c) { c.execute(); done.push(c); }
        void undo() { if (!done.isEmpty()) done.pop().undo(); }
    }

    // ---- State: behaviour depends on the current state; each state decides the next ----
    enum OrderState {
        NEW {
            OrderState pay() { return PAID; }
            OrderState ship() { throw new IllegalStateException("pay first"); }
        },
        PAID {
            OrderState pay() { throw new IllegalStateException("already paid"); }
            OrderState ship() { return SHIPPED; }
        },
        SHIPPED {
            OrderState pay() { throw new IllegalStateException("already paid"); }
            OrderState ship() { throw new IllegalStateException("already shipped"); }
        };
        abstract OrderState pay();
        abstract OrderState ship();
    }

    public static void main(String[] args) {
        Editor ed = new Editor();
        History h = new History();
        h.run(new Append(ed, "Hello"));
        h.run(new Append(ed, ", world"));
```

CommandStateDemo.java (continued) JAVA

```
        System.out.println(ed.text);
        h.undo();
        System.out.println(ed.text);
        h.undo();
        System.out.println("[" + ed.text + "]");

        OrderState s = OrderState.NEW;
        try {
            s.ship();
        } catch (IllegalStateException e) {
            System.out.println("NEW.ship(): " + e.getMessage());
        }
        s = s.pay();
        System.out.println(s);
        s = s.ship();
        System.out.println(s);
        try {
            s.pay();
        } catch (IllegalStateException e) {
            System.out.println("SHIPPED.pay(): " + e.getMessage());
        }
    }
}
```

Terminal TERMINAL

```
$ java CommandStateDemo.java
Hello, world
Hello
[]
NEW.ship(): pay first
PAID
SHIPPED
SHIPPED.pay(): already paid
```

`History` keeps executed commands on a stack, so `undo()` reverses them in last-in first-out order. `Append.undo` works because the command remembers exactly what it appended. For a command whose effect cannot be computed backwards (say "delete line 3"), store a snapshot of the old state in the command.

12.2.5 Visitor and Iterator

Visitor: add operations without touching the classes

Problem. You have a family of types (expression nodes) and want many operations over them (evaluate, print, optimise, type-check) without adding a method to every class for each operation. The classic **visitor** uses double dispatch: each node has `accept(Visitor v)`, which calls `v.visitAdd(this)`. It is verbose, and adding a new node type breaks every visitor.

With a `sealed` interface and records (Part 3), the compiler knows the full list of subtypes. A `switch` with record patterns then does the same job with one method per operation, and the compiler reports an error if you forget a case. The cost of the trade is the usual one: it is easy to add operations and hard to add a node type. Visitor with `accept` has the opposite emphasis only slightly; both make "add a node" costly.

VisitorIteratorDemo.java JAVA

```
import java.util.Iterator;
import java.util.NoSuchElementException;

public class VisitorIteratorDemo {

    // ---- Visitor, modern style: a sealed hierarchy + pattern-matching switch ----
    sealed interface Expr permits Num, Add, Mul, Neg { }
    record Num(double value) implements Expr { }
    record Add(Expr left, Expr right) implements Expr { }
    record Mul(Expr left, Expr right) implements Expr { }
    record Neg(Expr operand) implements Expr { }

    // Each "visitor" is just a method. No accept(), no Visitor interface, exhaustive checking.
    static double eval(Expr e) {
        return switch (e) {
            case Num n -> n.value();
```

VisitorIteratorDemo.java (continued) JAVA

```
            case Add(Expr l, Expr r) -> eval(l) + eval(r);
            case Mul(Expr l, Expr r) -> eval(l) * eval(r);
            case Neg(Expr o) -> -eval(o);
        };
    }

    static String show(Expr e) {
        return switch (e) {
            case Num n -> String.valueOf(n.value());
            case Add(Expr l, Expr r) -> "(" + show(l) + " + " + show(r) + ")";
            case Mul(Expr l, Expr r) -> show(l) + " * " + show(r);
            case Neg(Expr o) -> "-" + show(o);
        };
    }

    // ---- Iterator: walk a structure without exposing it ----
    static class Countdown implements Iterable<Integer> {
        private final int from;
        Countdown(int from) { this.from = from; }

        @Override public Iterator<Integer> iterator() {
            return new Iterator<>() {
                private int next = from;
                @Override public boolean hasNext() { return next > 0; }
                @Override public Integer next() {
                    if (next <= 0) throw new NoSuchElementException();
                    return next--;
                }
            };
        }
    }

    public static void main(String[] args) {
        Expr e = new Add(new Num(2), new Mul(new Num(3), new Neg(new Num(4))));
        System.out.println(show(e) + " = " + eval(e));

        for (int i : new Countdown(3)) System.out.print(i + " ");
        System.out.println("liftoff");
    }
}
```

Terminal TERMINAL

```
$ java VisitorIteratorDemo.java
(2.0 + 3.0 * -4.0) = -10.0
3 2 1 liftoff
```

Iterator: walk without exposing the structure

Problem. Callers should traverse a collection (list, tree, database cursor) without knowing how it stores its data. Java built this into the language: any class implementing `Iterable<T>` works in an enhanced `for` loop, as `Countdown` does above. `hasNext()` and `next()` are the whole contract. `Iterator.remove()` is optional. Most collection iterators are **fail-fast**: modifying the collection during iteration, other than through the iterator, throws `ConcurrentModificationException` (the details are in Part 6).

12.2.6 Patterns in the JDK: a reading list

Pattern	In the JDK or common libraries
Adapter	`InputStreamReader`, `Arrays.asList`, `Collections.enumeration`
Decorator	`BufferedReader`, `GZIPOutputStream`, `Collections.unmodifiableList`
Facade	`java.net.http.HttpClient`, `javax.sql.DataSource`-based helpers
Proxy	`java.lang.reflect.Proxy`, Spring AOP, Hibernate lazy proxies
Strategy	`Comparator`, `ThreadPoolExecutor.RejectedExecutionHandler`
Observer	`PropertyChangeListener`, Swing listeners, `CompletableFuture` callbacks
Template Method	`AbstractList.get`, `HttpServlet.doGet`, `InputStream.read(byte[])`
Command	`Runnable`, `Callable`, tasks in an `ExecutorService`
State	`Thread.State`, enum state machines
Visitor	`java.nio.file.FileVisitor` with `Files.walkFileTree`
Iterator	`Iterator`, `Iterable`, `Scanner`, `ResultSet`

Where to see each pattern in code you already use.

INTERVIEW ANGLE

Which design patterns have you used, and where? Pick three you can defend with a concrete case: for example Strategy for pluggable discount or sorting rules (a `Comparator` or lambda), Builder for a request object with many optional parts, and Proxy via Spring's `@Transactional`. For each, state the problem, the structure in one sentence, and the trade-off (indirection, extra classes, harder debugging). Interviewers care that you chose a pattern because of a problem, not the other way round.

Check yourself

1. In one sentence each: how does a decorator differ from an adapter, and from a proxy?
2. What does this print? `Text t = new Bold(new Shout(new Plain("a"))); System.out.println(t.render());` using the `Bold`/`Shout` records of this chapter.
3. Why does the observer loop iterate over `List.copyOf(listeners)` and not `listeners`?
4. A `java.lang.reflect.Proxy` handler calls `method.invoke(target, args)` and the target throws `IllegalArgumentException`. What does `invoke` throw, and what should the handler do?
5. Why is a `final` method the heart of the Template Method pattern?
6. What does the compiler do for you when a `switch` over a sealed interface misses a subtype, and why does that matter for the Visitor replacement?

Answers

1. An adapter changes the interface, a decorator keeps the interface and adds behaviour, a proxy keeps the interface and controls access to the target (cache, security, lazy load, remote call).
2. `<b>A!</b>`. `Shout` wraps `Plain("a")` giving `A!`, then `Bold` wraps that.
3. A listener may unsubscribe (or subscribe another) during notification. Modifying the list while iterating it would throw `ConcurrentModificationException`; iterating over a snapshot is safe.
4. It throws `InvocationTargetException` whose cause is the original exception. The handler should unwrap it and rethrow `e.getCause()` so callers see the real exception.
5. `final` stops subclasses from changing the order of the steps; they can only fill the abstract steps and optional hooks.
6. A compile-time error ("the switch expression does not cover all possible input values"). So adding a new node type forces you to update every operation, which is the safety the classic Visitor tried to give.

KEY POINTS

- Adapter changes an interface, Decorator adds behaviour behind the same interface, Proxy controls access, Facade simplifies a subsystem.
- Strategy and Command are usually lambdas in modern Java; Observer needs an unsubscribe path to avoid leaks.
- Template Method fixes the skeleton in a `final` method; prefer composition when the skeleton is not truly fixed.
- State fits naturally in an `enum` with abstract methods; Visitor becomes a `switch` over a sealed hierarchy with exhaustiveness checking.
- `java.io` and `java.lang.reflect.Proxy` are the JDK's best examples of Decorator, Adapter and Proxy.

12.3 Effective Java in practice

Design patterns tell you how to arrange classes. This chapter is about something smaller and more frequent: the habits that separate code that merely works from code that stays correct as it grows. The best-known collection of these habits is *Effective Java* by Joshua Bloch, who designed much of `java.util` and the Java collections. Its 90 or so "items" are rules of thumb backed by real failures. We do not reproduce the book; we pick the dozen items that matter most to a working engineer and demonstrate each with code you can run. Where a rule has a reason, the reason is given, because a rule you understand can be applied to cases the book never mentioned.

The theme of nearly every item is the same: **make misuse impossible, or at least obvious**. Do it with types (enums, records, `Optional`), with immutability, and with small public surfaces.

12.3.1 Creating objects

Prefer static factories and builders

Covered in the creational patterns chapter, so only the summary. Use a static factory (`Money.ofCents(250)`) when a name or caching helps, and a builder when there are many optional parameters. Both let you keep the real constructor `private`, which leaves the class free to change how objects are made later. A public constructor is a promise you can never take back.

Prefer dependency injection to hard-wired resources

A class that does `new DatabaseConnection("prod-host")` inside its methods cannot be tested without a database. Accept the collaborator through the constructor instead. The clean code chapter that follows shows this step by step.

Avoid creating needless objects

`new String("hello")` creates a second object where the literal `"hello"` is already shared; `Boolean.valueOf(true)` reuses an instance while `new Boolean(true)` (deprecated for removal) does not. A subtler version: `Long sum = 0L; sum += i;` inside a loop allocates a new `Long` on every iteration (the performance chapter at the end of this Part measures it). The balance matters: do not contort code to save objects that the JIT will remove anyway (see escape analysis in Part 9). Reuse expensive objects (a compiled `Pattern`, a `DateTimeFormatter`, an `ObjectMapper`) by putting them in a `static final` field.

12.3.2 Immutability and defensive copies

An **immutable** class is one whose instances cannot change after construction (`String`, `Integer`, `LocalDate`, records of immutables). Immutable objects are thread-safe without locks, safe to share and cache, safe as `HashMap` keys, and trivial to reason about. Bloch's advice: *classes should be immutable unless there is a very good reason to make them mutable.* The recipe:

1. No setters or other methods that change state.
2. Make the class `final` (or make all constructors private), so a subclass cannot add mutable behaviour.
3. Make all fields `private final`.
4. Copy any mutable object that comes in through the constructor, and never hand out the internal mutable object.

Step 4 is where most mistakes hide, because `final` only protects the *reference*, not the object it points to. The next listing shows the attack in `BrokenPeriod`, and the cure.

Effective.java JAVA

```java
import java.util.*;

public class Effective {
    // Item: defensive copies. Broken version.
    static final class BrokenPeriod {
        private final Date start, end;
        BrokenPeriod(Date start, Date end) { this.start = start; this.end = end; }
        Date end() { return end; }
    }
    // Good version with java.time
    record Period(java.time.LocalDate start, java.time.LocalDate end) {
        Period {
            if (start.isAfter(end)) throw new IllegalArgumentException(start + " after " + end);
        }
    }

    // Immutable class with a mutable component
    static final class Roster {
        private final List<String> names;
        Roster(List<String> names) { this.names = List.copyOf(names); }
        List<String> names() { return names; }
    }
    static final class LeakyRoster {
        private final List<String> names;
        LeakyRoster(List<String> names) { this.names = names; }
        List<String> names() { return names; }
    }

    // Static factory with caching and a named constructor
    static final class Money {
        private static final Money ZERO = new Money(0);
        private final long cents;
        private Money(long cents) { this.cents = cents; }
        static Money ofCents(long c) { return c == 0 ? ZERO : new Money(c); }
        static Money ofDollars(double d) { return ofCents(Math.round(d * 100)); }
        @Override public String toString() {
            return "$" + cents / 100 + "." + String.format("%02d", cents % 100);
        }
    }

    // int constants vs enum
    static final int APPLE = 0, ORANGE = 1;
    enum Fruit { APPLE, ORANGE }

    // Optional as return type
    static Optional<String> findEmail(Map<String, String> db, String user) {
        return Optional.ofNullable(db.get(user));
    }

    // Composition vs inheritance: classic InstrumentedHashSet
    static class InstrumentedSet<E> extends HashSet<E> {
        int addCount;
        @Override public boolean add(E e) { addCount++; return super.add(e); }
        @Override public boolean addAll(Collection<? extends E> c) {
            addCount += c.size();
            return super.addAll(c);
        }
    }

    public static void main(String[] args) {
        Date start = new Date(0), end = new Date(86_400_000L);
        BrokenPeriod p = new BrokenPeriod(start, end);
        long before = p.end().getTime();
        end.setTime(0);              // attack from the outside
        System.out.println("end changed: " + (p.end().getTime() != before));
        p.end().setTime(-5);         // attack through the accessor
        System.out.println("end now: " + p.end().getTime());

        try {
            new Period(java.time.LocalDate.of(2026, 5, 2), java.time.LocalDate.of(2026, 5, 1));
        } catch (IllegalArgumentException e) { System.out.println(e.getMessage()); }

        List<String> src = new ArrayList<>(List.of("Asha", "Ravi"));
        Roster safe = new Roster(src);
        LeakyRoster leaky = new LeakyRoster(src);
```

Effective.java (continued) JAVA

```
        src.add("Mallory");
        System.out.println(safe.names() + " vs " + leaky.names());
        try { safe.names().add("x"); }
        catch (UnsupportedOperationException e) { System.out.println("immutable view"); }

        System.out.println(Money.ofDollars(12.5) + " " + (Money.ofCents(0) == Money.ofDollars(0)));

        System.out.println((APPLE == ORANGE - 1) + " " + Fruit.APPLE
            + " " + Fruit.valueOf("ORANGE").ordinal());

        Map<String, String> db = Map.of("asha", "asha@example.com");
        System.out.println(findEmail(db, "asha").map(String::toUpperCase).orElse("none"));
        System.out.println(findEmail(db, "bob").orElse("none"));

        InstrumentedSet<String> s = new InstrumentedSet<>();
        s.addAll(List.of("a", "b", "c"));
        System.out.println("addCount = " + s.addCount);
    }
}
```

Run it (Java 21) TERMINAL

```
$ java Effective.java
end changed: true
end now: -5
2026-05-02 after 2026-05-01
[Asha, Ravi] vs [Asha, Ravi, Mallory]
immutable view
$12.50 true
true APPLE 1
ASHA@EXAMPLE.COM
none
addCount = 6
```

`Date` is used here only to demonstrate the attack; the chapter on date and time in Part 4 explains why `java.time` replaced it.

What to notice. `BrokenPeriod` looks immutable: both fields are `private final`. Yet `end.setTime(0)`, called on the original `Date` the caller still holds, changed the period, and `p.end().setTime(-5)` changed it through the accessor. Output lines 1 and 2 prove it. There are two fixes: copy on the way in and out (`new Date(start.getTime())`), or, better, use a type that is itself immutable. `java.time.LocalDate` is immutable, which is why the `Period` record needs only a validation check in its compact constructor and no copying at all. Choose immutable field types and the problem disappears.

The `Roster`/`LeakyRoster` pair shows the same bug with collections. `LeakyRoster` kept the caller's list, so `Mallory` appeared later in a roster that "never changes". `Roster` takes a snapshot with `List.copyOf` and exposes an immutable list, so any attempt to modify it throws `UnsupportedOperationException`. Note the order of the checks: when you copy a mutable parameter, **copy first, then validate the copy**; otherwise another thread can change the original between the check and the copy (a time-of-check to time-of-use race).

WATCH OUT

Records are shallow. A record's fields are `final`, but `record Team(List<String> members)` still shares whatever list it is given. Use the compact constructor to copy: `members = List.copyOf(members);`. The records chapter in Part 3 covers this.

12.3.3 Methods every class gets: equals, hashCode, toString

The `equals` contract (reflexive, symmetric, transitive, consistent, never equal to `null`) and the rule *equal objects must have equal hash codes* were demonstrated in the chapter on the `Object` class in Part 3. The practical advice here is how to avoid hand-writing them:

- For a value type, **use a** `record`: `equals`, `hashCode` and `toString` are generated from all components and are correct by construction.
- If you must write them, use `Objects.equals(a, b)` and `Objects.hash(a, b, c)`, and compare exactly the fields that define identity.
- If you override `equals`, always override `hashCode`; breaking this makes `HashMap` and `HashSet` lose your objects.

- Always override `toString` on any class whose objects appear in logs. Include the fields useful for debugging, but never secrets (see the secure coding chapter).
- Never put mutable fields in `hashCode` of an object used as a map key. Mutating the object after insertion strands it in the wrong bucket.

Implement `Comparable` when a type has a natural order, and keep it consistent with `equals` (`compareTo` returning 0 should mean `equals` is true), or sorted collections and hash collections will disagree about duplicates. Compare primitives with `Integer.compare`, not by subtracting, which overflows.

12.3.4 Classes and inheritance

Favour composition over inheritance

Inheritance couples a subclass to the *implementation* of its superclass, not just its interface. The last block in the listing is Bloch's famous example. `InstrumentedSet` extends `HashSet` to count how many elements were added. We add three elements with `addAll` and expect `addCount` to be 3. It is **6**. The reason is hidden in `HashSet`'s implementation: `addAll` internally calls `add` for each element, so each element is counted twice, once in our `addAll` and once in our overridden `add`. A later JDK version could change `addAll` to not call `add`, and our class would silently change behaviour again.

The fix is **composition with forwarding**: hold a `Set` in a private field, implement the `Set` interface by forwarding each method to it, and add the counting in your own methods. Then you depend on the *contract* of `Set`, not on how `HashSet` happens to be written. Inheritance is appropriate when the subclass truly *is a* kind of the superclass, both are under your control, and the superclass was designed and documented for extension. Otherwise either make the class `final`, or forbid overriding by making methods `final`.

Minimise accessibility

Make every class and member as inaccessible as it can be: `private` by default, package-private next, `public` only for the real API. In a module (Part 11) export only the packages that form the API. Public fields are almost always wrong (except `public static final` constants of immutable type), because you can never validate or change them. Keep a class's public surface small: each public method is a commitment.

Use interfaces to define types; prefer them as parameter and return types

Declare `List<String> names = new ArrayList<>();` and write methods that accept `Collection<String>` or `List<String>` rather than `ArrayList<String>`. Callers keep their freedom, and you can change the implementation. Use `Set`, `Map` and `Queue` the same way.

12.3.5 Enums, not integer constants

Before Java 5 programmers wrote `static final int APPLE = 0, ORANGE = 1;`. Nothing stops `eat(42)`, printing shows `0`, and a name cannot be recovered. In the listing the `Fruit` enum fixes all of this: it is type-safe (a method taking a `Fruit` cannot be passed `42`), prints as `APPLE`, can carry fields and methods (Part 3, chapter on enums), works in `switch` with exhaustiveness checking, and has fast specialised collections `EnumMap` and `EnumSet`. Use enums for any fixed set known at compile time: days, states, roles, operations. Two warnings: never rely on `ordinal()` for anything persisted (reordering the constants silently changes it), and never use bit-field integer flags where an `EnumSet` expresses the same thing readably.

12.3.6 Generics and collections

- **Do not use raw types.** `List list = new ArrayList();` throws away compile-time checking and moves `ClassCastException` to run time. Use `List<String>`, or `List<?>` when you truly do not care.
- **Prefer lists to arrays** when combining with generics. Arrays are covariant and reified (`String[]` is a subtype of `Object[]`, and the store check happens at run time); generics are invariant and erased. Mixing them gives warnings for good reason.
- **Use bounded wildcards for flexibility: PECS.** *Producer extends, Consumer super.* A method that only reads items out of a collection takes `Collection<? extends E>`; one that only puts items in takes `Collection<? super E>`. This is why `Collections.copy(List<? super T> dest, List<? extends T> src)` has that signature (see the wildcard chapter in Part 6).
- **Eliminate unchecked warnings** or, when you can prove safety, suppress the narrowest scope possible with `@SuppressWarnings("unchecked")` plus a comment explaining why it is safe.

- **Return empty collections, never `null`.** `Collections.emptyList()` or `List.of()` costs nothing and saves every caller a `null` check.

12.3.7 Methods

Check parameters for validity, early, and document it

Validate arguments at the top of public methods and fail with a clear message: `Objects.requireNonNull(name, "name")`, `if (qty <= 0) throw new IllegalArgumentException("qty must be positive: " + qty)`. Failing fast puts the stack trace at the real cause. For private methods use `assert`.

Design signatures carefully

Choose names that follow convention, avoid long parameter lists (more than about four: introduce a parameter object, a record, or a builder), and prefer two-element enums to `boolean` parameters when the call site would be unreadable (`render(true, false)` versus `render(Mode.DRAFT, Colour.MONO)`). Avoid overloads with the same number of parameters in the same position, and especially ones that differ only by a boxed/primitive pair (`remove(int)` versus `remove(Object)` on `List` is the famous trap).

Return `Optional` for "might be absent", not `null`

`Optional<T>` as a *return type* tells the caller in the signature that there may be no result, and its methods (`map`, `orElse`, `ifPresent`) make handling it fluent, as `findEmail` does in the listing: `ASHA@EXAMPLE.COM` for a known user, `none` for an unknown one. Use it only for return values. Do not use `Optional` for fields, method parameters or collection elements (it adds an object and noise without benefit), never call `get()` without checking `isPresent()` (prefer `orElseThrow()`), and never return `null` from a method whose type is `Optional`. For collections and arrays return an empty one.

> **GOTCHA**
> `Optional.of(null)` throws. `Optional.of(x)` requires a non-null `x` and throws `NullPointerException` otherwise. Use `Optional.ofNullable(x)` when `x` may be null. Passing a possibly-null value to `of` is a common beginner bug.

12.3.8 Exceptions

Part 5 covers the mechanics. The Effective Java advice, in one list:

- Use exceptions only for exceptional conditions, never for ordinary control flow (such as ending a loop by catching `ArrayIndexOutOfBoundsException`).
- Use **checked** exceptions for conditions a caller can reasonably recover from, and **unchecked** (`RuntimeException`) for programming errors such as a violated precondition. Many modern codebases lean to unchecked exceptions everywhere, since checked ones do not compose well with lambdas.
- Prefer the standard exceptions: `IllegalArgumentException`, `IllegalStateException`, `NullPointerException`, `IndexOutOfBoundsException`, `UnsupportedOperationException`, `ConcurrentModificationException`. Readers already know what they mean.
- Throw exceptions that fit the abstraction level and keep the cause: `throw new OrderException("cannot load order 7", sqlException);`. Never swallow an exception with an empty `catch` block; if you really mean to ignore it, name the variable `ignored` and say why in a comment.
- Include the values that caused the failure in the message (`"qty must be positive: -3"`), but not secrets.
- Use try-with-resources for anything `AutoCloseable`; it closes in reverse order and records secondary failures as suppressed exceptions instead of losing them.

12.3.9 Other items worth knowing

Use `@Override` always

It turns a typo (`equals(Foo f)` instead of `equals(Object o)`, which overloads instead of overriding) into a compile error.

Prefer the standard library

Do not write your own random number code, sort, or `Pair`; use `ThreadLocalRandom`, `List.sort`, a record. The JDK is better tested and your readers know it.

Beware of `float` and `double` for money

They cannot represent 0.1 exactly. Use `BigDecimal`, or store integer cents as the `Money` class in the listing does (Part 4 shows why).

Prefer `for-each` loops and `StringBuilder`

The enhanced `for` loop avoids index and iterator bugs. Use `StringBuilder` for repeated appends in loops.

Do not use finalizers or `Cleaner` for resources

Finalization is deprecated for removal. Release resources with `AutoCloseable` and try-with-resources (Part 9 has the details).

12.3.10 Evolving an API without breaking users

A public API ages. Code written against version 1 must keep compiling and running against version 2, and the longer a library lives the more careful it must be. These rules apply to any shared library or internal module with many callers.

- **Never remove or change a public method in a way that breaks callers.** Instead add a new method and mark the old one `@Deprecated(since = "2.0", forRemoval = false)` with a Javadoc `@deprecated` tag pointing to the replacement. Use `forRemoval = true` only when you really plan to delete it; the compiler then emits a stronger warning, as it does for `Thread.stop`.
- **Adding a method to an interface** breaks every implementor unless you give it a `default` implementation. That is the reason `default` methods exist (they let `List.sort` and `Collection.stream` be added in Java 8 without breaking decade-old implementations).
- **Adding a component to a record** changes its canonical constructor, which breaks callers. Keep an old constructor delegating to the new one.
- **Distinguish source from binary compatibility.** Changing a method's return type or making a parameter type more general can still compile but breaks already compiled callers, which look methods up by the exact descriptor (see the class file chapter in Part 1). Tools such as `japicmp` check this in a build.
- **Seal or hide what you may want to change.** Final classes, package-private helpers and unexported module packages cannot be depended on, so you can change them freely. A sealed interface lets you add subtypes without outside implementors existing.
- **Document the contract**, including thread safety, null handling and exceptions, with Javadoc, and follow *semantic versioning* (major.minor.patch) so users know what an upgrade may break.

INTERVIEW ANGLE

Why is composition preferred over inheritance? Give an example. Inheritance exposes a subclass to the superclass's implementation details, so a change in the parent, or a self-call inside it, can break the child, and it ties the child to one parent for life. Composition holds the other object in a field and delegates, so you depend only on its public contract, can swap it, and can combine several. Example: a counting `HashSet` subclass reports 6 after `addAll` of 3 elements because `HashSet.addAll` calls `add`; a wrapper that holds a `Set` and forwards counts correctly.

INTERVIEW ANGLE

How do you make a class immutable? Make it final (or have private constructors), make all fields private and final, provide no mutators, copy mutable inputs in the constructor and mutable outputs in accessors (or hold immutable types such as `List.copyOf`, `LocalDate`), and do not let `this` escape during construction. A record is a shortcut, but remember it is shallow for mutable components.

REAL WORLD

In Spring Boot code, DTOs are increasingly records, entities keep a no-argument constructor because the persistence framework needs it, and services are immutable objects with `final` fields set by constructor injection. Following the items above, `final` fields plus constructor injection, is what makes a Spring bean easy to test without starting any framework.

Check yourself

1. A class has `private final Date created;` assigned from its constructor argument and returned from `getCreated()`. Name two ways a caller can change `created`, and a one-line fix for each.
2. What does this print? `Long s = 0L; System.out.println(s.equals(0) + " " + s.equals(0L));`
3. Why does a `HashSet` subclass that counts `add` calls report 6 after `addAll` of three elements?
4. What is wrong with `Optional<String> name = Optional.of(map.get(key));` when `key` may be missing?
5. You change a public method `int size()` to `long size()` in a library. Callers' source code still compiles. Why might already compiled callers still break?
6. When should you use an enum instead of `int` constants, and what should you never persist from an enum?

Answers

1. Through the argument (the caller keeps the `Date` and mutates it) and through the getter (the caller mutates the returned `Date`). Fix: `this.created = new Date(created.getTime())` and `return new Date(created.getTime())`, or better use `Instant`, which is immutable.
2. `false true`. `0` is an `Integer`, so `Long.equals(Integer)` is false (different classes); `0L` is a `Long` with equal value.
3. `HashSet.addAll` calls `add` for each element, so each element is counted once by the overridden `addAll` and again by the overridden `add`: 3 + 3 = 6.
4. `Optional.of(null)` throws `NullPointerException`; use `Optional.ofNullable(map.get(key))`.
5. Compiled classes refer to the method by name plus exact descriptor (`size()I`); the new method is `size()J`, so the old one no longer exists, and linking fails with `NoSuchMethodError` until callers are recompiled.
6. Use an enum for any fixed set of related constants (type-safe, printable, with behaviour, `EnumMap`/`EnumSet`). Do not persist `ordinal()`; store the `name()` or an explicit code, because reordering constants changes ordinals.

KEY POINTS

Effective Java boils down to making wrong code hard to write.

- Prefer immutability: `private final` fields, no setters, copy mutable inputs and outputs, or choose immutable field types such as `LocalDate` and `List.copyOf`.
- Prefer composition and interfaces to inheritance; a subclass depends on how its parent is implemented, and the counting-set example shows how that goes wrong.
- Use enums instead of int constants, `Optional` as a return type for absence, empty collections instead of `null`, and standard exceptions with clear messages and causes.
- Validate arguments early, avoid raw types, apply PECS to wildcards, and override `equals` and `hashCode` together, or use a record.
- Evolve APIs by adding and deprecating, not by changing or removing; use `default` methods to extend interfaces.

12.4 Clean code and code smells in Java

Code is read far more often than it is written: by teammates, by reviewers, and by you six months from now. "Clean code" is simply code that a competent stranger can understand and change safely. This chapter turns that vague goal into specific habits: how to name things, how big a method should be, the design principles known as SOLID applied to small Java examples, a catalogue of **code smells** (surface signs that the design underneath is in trouble), a worked before/after refactoring that you can run, and **dependency injection by hand**, the one technique that makes the rest testable.

None of this is law. They are heuristics that experienced developers converge on, and each has exceptions. Learn them as questions to ask of your code, not as rules to follow blindly.

12.4.1 Naming

A good name removes the need for a comment. Names are the cheapest and most powerful tool you have.

- **Say what, not how.** `calculateTax(order)` over `doStuff(o)`. A reader should be able to guess what a method does from its call site.
- **Be consistent with Java conventions.** Classes and records `UpperCamelCase` nouns (`OrderService`); methods and variables `lowerCamelCase` (verbs for methods: `findByEmail`, `isEmpty`, `hasNext`); constants `UPPER_SNAKE_CASE`; packages all lower case. Booleans read as questions: `active`, `isExpired()`, `canRetry()`.

- **Length proportional to scope.** `i` in a three-line loop is fine; a field used across a 500-line class deserves a full name. Avoid abbreviations nobody else would guess (`cstmrLst`).
- **No noise words.** `OrderData`, `OrderInfo`, `OrderObject` add nothing over `Order`. Do not encode the type: `nameString`, `ListOfUsers`.
- **One word per concept.** If you call fetching `get` in one class, `fetch` in another and `retrieve` in a third, the reader wonders what the difference is. Pick one.
- **Use domain words.** If the business says "invoice", the class is `Invoice`, not `BillingDocument`.

12.4.2 Methods and classes: small, focused, at one level

A method should do one thing, and do it at one level of abstraction. A useful test: can you describe what the method does in one sentence without using "and"? A method that parses input, validates it, computes a price and writes a file does four things, and any change to one risks the other three. Typical guidelines: a method that does not fit on one screen (say 20 to 30 lines) deserves a look; more than three or four parameters suggest a missing object; a flag parameter (`process(order, true)`) means the method does two things and should be split in two.

Nesting is a smell. Deeply nested `if`s hide the main path. Use **guard clauses** to handle the odd cases first and return early:

(fragment) JAVA

```
// Nested
double discount(Customer c) {
    if (c != null) {
        if (c.isActive()) {
            if (c.orders() > 10) {
                return 0.1;
            }
        }
    }
    return 0;
}

// Guard clauses: the same logic, flat
double discountFlat(Customer c) {
    if (c == null || !c.isActive()) return 0;
    if (c.orders() <= 10) return 0;
    return 0.1;
}
```

Comments explain why, not what. `// increment i` is noise. `// the API returns 1-based pages, we use 0-based` explains a decision the code cannot show. If you feel the need to write a comment explaining a block, first try extracting the block into a method whose name says the same thing. Javadoc on public API is different: document the contract.

Classes should be cohesive. All the fields should be used by most of the methods. A class with two groups of fields each used by separate groups of methods is two classes. A good class can be described in one sentence.

12.4.3 SOLID, with small examples

SOLID is a set of five object-oriented design principles. Each one is short; the skill is spotting when your code violates it.

S: Single Responsibility Principle

A class should have one reason to change. An `OrderService` that calculates prices, saves to the database, formats e-mails and prints PDFs changes whenever pricing, storage, wording or layout changes. Split by responsibility: `PriceCalculator`, `OrderRepository`, `ReceiptFormatter`. Each can then be tested alone, and a change in e-mail wording cannot break pricing.

O: Open/Closed Principle

Open for extension, closed for modification. Adding a new kind of discount should not mean editing a growing `if/else` chain in a pricing method. Put the varying part behind an interface (strategy pattern) and add a new implementation. Enums with per-constant behaviour, and sealed interfaces with a pattern `switch`, are lighter-weight ways to reach the same goal. The refactoring below does exactly this.

L: Liskov Substitution Principle

A subtype must be usable wherever its supertype is expected, without surprises. The classic violation: `Square extends Rectangle` with `setWidth` that also changes height. Code that sets width and height independently and then computes the area gets the wrong answer when handed a `Square`. In Java terms, an overriding method must accept at least what the parent accepts, promise at least what it promises, and not throw new unexpected exceptions. If you find yourself overriding a method to throw `UnsupportedOperationException`, the inheritance is probably wrong (the JDK's own `Collections.unmodifiableList` and `List.of` are a known compromise, which is why `List.add` is documented as optional).

I: Interface Segregation Principle

Do not force clients to depend on methods they do not use. One fat `Worker` interface with `work()`, `eat()` and `sleep()` forces a robot class to implement `eat()`. Split it into small role interfaces. The JDK follows this: `Iterable`, `Comparable`, `Closeable` and `Runnable` are one-method roles that classes combine freely.

D: Dependency Inversion Principle

High-level code should depend on abstractions, not on concrete low-level classes. A `WelcomeService` should know that it can "notify someone", not that it sends mail through a specific SMTP library. Define a `Notifier` interface owned by the high-level code, and let the concrete class implement it. The mechanism that wires the concrete choice in is dependency injection, below.

12.4.4 A catalogue of code smells

A **code smell** does not mean the code is wrong; it means look closer. The most common ones in Java, each with the usual cure:

Smell	What it looks like	Usual cure
Long method	one method of 80 lines with comments marking "sections"	extract method (each section becomes a named method)
Long parameter list	`createUser(a, b, c, d, e, f, g)`	parameter object (a record) or builder
Primitive obsession	`String email`, `double price`, `int status` passed everywhere	small types: `record Email(String value)`, enum, `Money`
Flag arguments	`render(true)`	two methods, or an enum
Duplicated code	the same five lines pasted in three classes	extract method or class
Feature envy	method uses another object's getters more than its own fields	move the method to that object
God class	3000 lines, 60 fields, "Manager" or "Util" in its name	split by responsibility
Switch on type	`if (x instanceof A) ... else if (x instanceof B)` repeated	polymorphism, or sealed type with pattern `switch`
Magic numbers	`price * 1.05`	named constant or enum field
Mutable shared state	static mutable fields, public setters	immutability, pass state explicitly
Catch and ignore	`catch (Exception e) { }`	handle, rethrow with cause, or log with context
Train wreck	`a.getB().getC().getD().doIt()`	tell, don't ask; add a method on `a`

Common smells and the usual refactoring.

12.4.5 Worked refactoring: before and after

The listing shows one realistic small method, written the way it often arrives in a pull request, and a cleaned version with identical behaviour. Both are in one file, so the output proves they agree. The second half of the file is dependency injection, explained after it.

Refactor.java JAVA

```java
import java.util.*;

public class Refactor {
    // BEFORE: one method, mixed concerns, flag argument, magic numbers, deep nesting
    static double process(List<String[]> o, boolean vip, String type) {
        double t = 0;
        if (o != null) {
            for (String[] i : o) {
                if (i.length == 2) {
                    double p = Double.parseDouble(i[1]);
                    if (type.equals("food")) { t += p * 1.05; }
                    else { t += p * 1.2; }
                }
            }
            if (vip) { t = t * 0.9; }
        }
        return t;
    }

    // AFTER
    record Item(String name, double price) {}
    enum Category {
        FOOD(0.05), GENERAL(0.20);
        private final double taxRate;
        Category(double taxRate) { this.taxRate = taxRate; }
        double withTax(double price) { return price * (1 + taxRate); }
    }
    interface Discount { double apply(double total); }
    static final Discount NONE = total -> total;
    static final Discount VIP = total -> total * 0.9;

    static double total(List<Item> items, Category category, Discount discount) {
        double subtotal = items.stream().mapToDouble(i -> category.withTax(i.price())).sum();
        return discount.apply(subtotal);
    }

    // Dependency injection by hand
    interface Notifier { void send(String to, String msg); }
    interface UserRepo { Optional<String> emailOf(String user); }
    static class WelcomeService {
        private final UserRepo repo;
        private final Notifier notifier;
        WelcomeService(UserRepo repo, Notifier notifier) {
            this.repo = repo;
            this.notifier = notifier;
        }
        boolean welcome(String user) {
            return repo.emailOf(user)
                .map(email -> { notifier.send(email, "Welcome, " + user); return true; })
                .orElse(false);
        }
    }

    public static void main(String[] args) {
        List<String[]> old = List.of(new String[] {"rice", "100"}, new String[] {"dal", "50"});
        System.out.printf("before: %.2f%n", process(old, true, "food"));
        List<Item> items = List.of(new Item("rice", 100), new Item("dal", 50));
        System.out.printf("after:  %.2f%n", total(items, Category.FOOD, VIP));

        // production wiring
        UserRepo repo = u -> u.equals("asha") ? Optional.of("asha@example.com") : Optional.empty();
        Notifier console = (to, msg) -> System.out.println("MAIL to " + to + ": " + msg);
        System.out.println(new WelcomeService(repo, console).welcome("asha"));

        // test wiring: a fake notifier records what was sent
        List<String> sent = new ArrayList<>();
        WelcomeService svc = new WelcomeService(repo, (to, msg) -> sent.add(to + "|" + msg));
        System.out.println(svc.welcome("bob") + " " + sent);
        System.out.println(svc.welcome("asha") + " " + sent);
    }
}
```

Run it (Java 21) TERMINAL

```
$ java Refactor.java
before: 141.75
after:  141.75
MAIL to asha@example.com: Welcome, asha
true
false []
true [asha@example.com|Welcome, asha]
```

Both versions print `141.75` (two items of 100 and 50 with 5% food tax is 157.5, then 10% off for VIP). Now compare how each reads.

What was wrong with the before version

Every one-letter name (`o`, `t`, `i`, `p`) forces you to decode. The input is a `List<String[]>` of "name, price" pairs, a stringly-typed structure whose meaning lives only in the author's head (primitive obsession). The boolean `vip` and the string `type` are flag arguments: a typo such as `"Food"` silently takes the 20% branch. The numbers `1.05`, `1.2` and `0.9` are magic. The method parses, validates, taxes and discounts, four jobs in one, with four levels of nesting. Adding a third category or discount means editing the middle of it, an Open/Closed violation, and testing the discount logic means constructing string arrays.

What the after version does

- `Item` is a record: the data has a name and type. Parsing strings becomes the job of the layer that reads input, not of the pricing code.
- `Category` is an enum that owns its tax rate. A new category is one new constant, a typo is a compile error, and the arithmetic lives next to the data it uses. The `switch` on a string is gone.
- `Discount` is a one-method interface (a strategy), so `NONE` and `VIP` are lambdas, and a new discount is a new lambda. The `total` method has no flag.
- `total` itself is two lines at one level of abstraction: add up taxed prices, then apply the discount.

Behaviour is identical, but the new code is split into pieces that can each be understood and tested alone. Note also what we did **not** do: no `AbstractPricingStrategyFactory`. The cure for a smell should be smaller than the smell.

> **WATCH OUT**
> Never refactor without a safety net. A refactoring changes structure but not behaviour, and you can only claim that if something checks it. Write or run tests first (the testing chapter in Part 11 shows how), change in small steps, and run the tests after each step. In the listing, `main` itself acts as the net: both versions must print the same number.

12.4.6 Dependency injection by hand

A class that creates its own collaborators is welded to them. If `WelcomeService` did `new SmtpNotifier("mail.corp")` internally, you could not run it without a mail server, and every test would send e-mail. **Dependency injection** (DI) inverts this: the class declares what it needs, as constructor parameters of interface type, and whoever builds the object decides which concrete implementations to supply.

In the second half of `Refactor.java` there is no framework. `WelcomeService` takes a `UserRepo` and a `Notifier`, both interfaces. In `main`, "production" wiring passes a console notifier, and "test" wiring passes a fake notifier that records messages into a list. The test asserts on that list: `true [asha@example.com|Welcome, asha]` shows the right message went to the right address, and `false []` shows an unknown user sends nothing. No mail server, no database, no framework, just constructors.

Three reasons to prefer **constructor injection** over field or setter injection:

- Dependencies are `final`, so the object is never half-built.
- The constructor makes the dependencies visible. A constructor with eight parameters is the class telling you it violates single responsibility.
- The class can be created in a plain unit test with `new`.

The composition root

All the wiring should happen in one place near the program's entry point, often called the **composition root**: `main` creates the repository, the notifier and the services and connects them. Nothing else in the code base calls `new` on a service. A DI container such as Spring does the same job with annotations: `@Component` and `@Service` mark the classes, constructor parameters declare the needs, and the container acts as an automated composition root. Understanding the manual version first means Spring's `@Autowired` is no longer magic, just a machine that calls your constructors in the right order.

REAL WORLD

In a Spring Boot service the layering is almost always `Controller` (HTTP) calls `Service` (rules) calls `Repository` (database), each depending on the next through an interface and constructor injection. A unit test for the service builds it with `new`, passing a mock or fake repository (Mockito generates such fakes). Tests that need the whole container are slower and used sparingly.

INTERVIEW ANGLE

What are the SOLID principles? Give one example violation. Single responsibility (one reason to change), open/closed (extend without modifying), Liskov substitution (subtypes honour the parent's contract), interface segregation (small client-specific interfaces), dependency inversion (depend on abstractions). Example: `Square extends Rectangle` where `setWidth` also sets the height violates Liskov, because code written for rectangles breaks when given a square. Another: a `ReportService` with an `if (type.equals("pdf")) ... else if (type.equals("csv"))` chain violates open/closed; each new format edits the method, while with a `Formatter` interface it is a new class.

INTERVIEW ANGLE

What is the difference between dependency injection and dependency inversion? Dependency inversion is the design principle: high-level modules and low-level modules both depend on abstractions, and the abstraction is owned by the high-level side. Dependency injection is a technique to achieve it: objects receive their dependencies from outside (usually through the constructor) instead of creating them. Inversion of control is the broader idea, that a framework or caller controls object creation and flow rather than your code.

12.4.7 A habit to take away: the review checklist

Before you push code, read your own diff as a reviewer would, with these questions: Could a new teammate name what each method does from its name alone? Is anything duplicated? Is there a method I would have to scroll to read? Does any class have two reasons to change? Is every value that could be wrong (null, negative, empty) checked where it enters? Can I test this with plain `new`? Is there a comment explaining *what* the code does that a better name would make unnecessary? One or two "no" answers per change is normal; the habit is to notice them.

Check yourself

1. Rewrite as guard clauses: `if (user != null) { if (user.isActive()) { return user.name(); } } return "anonymous";`
2. Which smell is `parse(text, true, false)` and what is a better design?
3. A `Bird` class has `fly()`. `Penguin extends Bird` overrides `fly()` to throw `UnsupportedOperationException`. Which SOLID principle is violated, and what would you change?
4. What is wrong with this class, for testing? `class Mailer { private final Smtp smtp = new Smtp("mail.corp"); void send(String s) { smtp.send(s); } }`
5. What does the following print? Use the `Category` enum from the listing: `System.out.println(Category.FOOD.withTax(100) + " " + Category.GENERAL.withTax(100));`
6. Why is constructor injection preferred over setter injection?

Answers

1. `if (user == null || !user.isActive()) return "anonymous"; return user.name();` The special cases leave first and the main path stays unindented.
2. Flag arguments (and primitive obsession): the call site does not say what `true, false` mean. Use an options record or enums, or separate methods such as `parseStrict(text)`.
3. Liskov substitution: code that calls `fly()` on any `Bird` breaks for a `Penguin`. Redesign: do not put `fly()` on `Bird`; add a `Flyer` interface (interface segregation) implemented only by birds that fly.

4. The collaborator is created inside the class, so a test cannot replace it and would talk to a real mail server. Inject an interface through the constructor.
5. `105.0 120.0`. The value is `100 * (1 + 0.05)` and `100 * (1 + 0.20)`. (Doubles happen to print cleanly here; for real money use `BigDecimal` or integer cents.)
6. Dependencies become `final`, the object is fully built once constructed, a class with too many dependencies is visible at a glance, and no framework is needed to instantiate it in a test.

KEY POINTS

Clean code is code that is cheap to change safely.

- Choose names from the domain, keep methods small and at one level of abstraction, flatten nesting with guard clauses, and comment the why.
- SOLID in one line each: one reason to change; extend without editing; subtypes honour contracts; small interfaces; depend on abstractions.
- Learn the smell catalogue (long method, primitive obsession, flag arguments, god class, feature envy, switch on type), and cure each with the smallest refactoring that works.
- Refactor only under test, in small steps.
- Inject dependencies through constructors, wire everything in one composition root, and your classes become testable with plain `new`.

12.5 Secure coding in Java

Security is not a feature you add at the end; it is a property of how every line handles data it did not create. Most real-world breaches of Java services are not exotic cryptographic attacks. They are old, boring bugs: a string pasted into a SQL query, a file path taken from a request, an object deserialized from the network, a password in a log file, a library with a known hole that nobody upgraded. This chapter covers those, because they account for most of the damage and because each has a small, specific habit that prevents it.

Java gives you a head start: no buffer overflows or dangling pointers (the JVM checks array bounds and manages memory), and a strong standard library. It does not protect you from trusting the wrong input. The central mindset is therefore one sentence: **treat all data that crosses a trust boundary as hostile until validated**. Trust boundaries are the HTTP request, the file you read, the message from a queue, the environment variable, the response from another service, and the bytes of a serialized object.

12.5.1 Input validation: allow-lists, at the boundary

Problem. Code downstream assumes its inputs are sane: that a username has no quotes, a quantity is positive, a name is not 10 MB long. If the assumption is false, something breaks, sometimes something exploitable.

Idea. Validate at the boundary, where untrusted data enters, and validate against what is *allowed*, not against what is known to be bad. A **deny-list** ("reject input containing `DROP`") is endlessly bypassable: `dRoP`, comment tricks, encodings. An **allow-list** ("a username is 3 to 16 letters, digits or underscores") is short and complete. In the listing below, `validateUsername` is exactly that, and the result shows `a; DROP` rejected without any attempt to recognise attacks.

Practical rules:

- Check **type, length, range and format**, in that order. Reject, do not silently "fix" input.
- Convert strings to real types as early as possible (`int`, `UUID`, enum, `LocalDate`, a record such as `Email`). A parsed `UUID` cannot contain a quote.
- Cap sizes: maximum request body, maximum string length, maximum collection size, maximum nesting. Unbounded input is a denial-of-service vector.
- **Canonicalise before you check**: decode and normalise (Unicode `Normalizer`, `Path.normalize()`), then validate the canonical form. Checking first and decoding later is a classic bypass.
- Validation is not escaping. Validating that input "looks fine" does not make it safe to paste into SQL, HTML or a shell command. Each destination needs its own safe mechanism (below).
- Frameworks help: Bean Validation (`@NotNull`, `@Size`, `@Pattern`) in Spring Boot declares these rules on request objects. They still need to be applied on every endpoint.

Beware your own regular expressions: a pattern with nested quantifiers such as `(a+)+$` can take exponential time on crafted input (see the regex chapter in Part 4). Keep patterns simple and bounded, which also helps here since `{3,16}` caps the length.

12.5.2 Injection: never build commands out of strings

Injection is the family of attacks in which data is interpreted as code. SQL injection is the famous one, but the same bug exists for operating-system commands, LDAP queries, JNDI lookups and HTML (cross-site scripting).

SQL injection and the PreparedStatement

Look at what string concatenation produces. In the listing the user name is `x' OR '1'='1`, and the query built by `+` comes out as:

```
SELECT * FROM users WHERE name = 'x' OR '1'='1'
```

The quote in the data ended the string literal, and the rest became part of the SQL: the condition is now always true and every row matches. With a semicolon the attacker could also append a second statement. The fix is **parameterised queries**: the SQL text and the values travel separately, and the database driver treats the value strictly as data, whatever characters it contains.

(fragment, needs a database to run) JAVA

```java
// Safe: the ? placeholder is bound as a value, never parsed as SQL
String findName(Connection con, String user) throws SQLException {
    String sql = "SELECT display_name FROM users WHERE name = ?";
    try (PreparedStatement ps = con.prepareStatement(sql)) {
        ps.setString(1, user);
        try (ResultSet rs = ps.executeQuery()) {
            return rs.next() ? rs.getString(1) : null;
        }
    }
}
```

Rules that follow: always use `PreparedStatement` (or the equivalent in your framework: Spring Data's derived queries, JPA's named parameters `:name`, `JdbcTemplate` with `?`); never concatenate or `String.format` values into SQL; and remember that **placeholders only stand for values**, not for table names, column names or `ORDER BY` directions. If those must vary, choose them from a fixed allow-list (an enum or a `Map`) in your own code, never from raw input. JDBC itself is outside the scope of this book, but this habit is not.

> **WATCH OUT**
> ORMs do not make you immune. JPA/Hibernate protects you only when you use parameters. A JPQL string built by concatenation (`"from User where name = '" + n + "'"`) is just as injectable as raw SQL. The same holds for `@Query` strings and native queries.

Command injection

`Runtime.exec("convert " + filename)` is the same bug for the shell. Use `ProcessBuilder` with a list of arguments, which never goes through a shell: `new ProcessBuilder("convert", filename, "out.png")`. Even so, a value starting with `-` can be read as an option by the program, so validate it first or place it after `--` where the tool supports that. If you can avoid starting a process at all (use a library), do.

Output encoding

When data goes into HTML, escape it for HTML; into JSON, use a JSON library rather than concatenation; into a log line, strip newlines (log injection lets an attacker forge entries). Template engines such as Thymeleaf escape by default; turning that off is a decision to review.

12.5.3 Path traversal

Problem. A download endpoint takes `?file=report.pdf` and opens `uploads/` plus that name. An attacker sends `../../etc/passwd` and reads files outside the folder. This is **path traversal**.

Fix. Resolve the user-supplied part against the base directory, normalise, and check the result still starts with the base. The helper `resolveSafe` in the listing does that, and the output shows each case.

The interesting part is the last pair of lines of the first block: `base.resolve("../../etc/passwd").normalize()` gives `/srv/etc/passwd`, a path outside `/srv/app/uploads`, which is what the `startsWith` check catches. Three details matter:

- `Path.normalize()` removes `.` and `..` segments **lexically**, without touching the file system. It does not resolve symbolic links. If users can create symlinks inside the base directory, additionally compare `toRealPath()` results.
- `Path.resolve(other)` returns `other` unchanged when it is absolute. That is why `"/etc/passwd"` is also rejected: resolving it against the base yields `/etc/passwd`, which fails the check.
- `Path.startsWith(Path)` compares **path elements**, not characters. `Path.of("/srv/up2/f").startsWith("/srv/up")` is `false`, whereas a plain string `startsWith` would wrongly say true. Always use the `Path` method, never `toString().startsWith`.

Also never trust a client-supplied file *name* for the saved name: generate your own (a `UUID`), and keep the original only as metadata. Check the real content type, not just the extension, and cap the size.

12.5.4 Deserialization

Java's built-in serialization (Part 5) rebuilds objects from a byte stream by choosing classes named *inside the stream*, and runs their `readObject` code while doing so. If an attacker controls the bytes, and a class on your class path has a "gadget" (code that does something dangerous when deserialized, such as running a command through a chain of method calls), they can execute code on your server before your `readObject` call even returns. Libraries such as Apache Commons Collections have been the source of many such chains, and this has been behind remote-code-execution bugs in many well-known Java products.

The rules, in order of strength:

1. **Never deserialize untrusted data with `ObjectInputStream`.** Use a data format instead: JSON (Jackson, Gson), Protocol Buffers, or plain text. These do not name arbitrary classes in the stream.
2. If you must, install a **deserialization filter** (JEP 290, Java 9, backported; context-specific filters from Java 17 on). A filter is an allow-list of classes, plus limits on depth and size, checked *before* each class is instantiated.
3. Remember that a JSON library can also be configured dangerously: enabling polymorphic "default typing" on Jackson recreates the same problem in JSON form. Bind to concrete, known types.
4. Records and sealed hierarchies deserialized through a JSON binding are safer than `Serializable` classes, but still validate the values.

The listing shows rule 2 in about ten lines. The filter `java.lang.Integer;java.lang.Number;java.lang.Object;java.util.ArrayList;!*` means: allow these four, reject everything else (`!*`). An `ArrayList` of integers is accepted and prints `[1, 2]`; a `HashSet` is rejected with `InvalidClassException: filter status: REJECTED` before any of its code runs. Why do `Number` and `Object` appear? Because `Integer` extends `Number`, the stream describes the superclass as well, and an `ArrayList` is stored internally with an `Object[]` array, and the filter sees each of those. Finding the minimal allow-list is trial and error, which is a good indication of how much code runs during deserialization.

12.5.5 The complete example

Secure.java JAVA

```
import java.io.*;
import java.nio.file.*;
import java.security.*;
import java.util.*;

public class Secure {
    // Path traversal: resolve, normalize, then check the result is still inside the base
    static Path resolveSafe(Path base, String userPath) {
        Path p = base.resolve(userPath).normalize();
        if (!p.startsWith(base)) throw new SecurityException("path escapes base: " + userPath);
        return p;
    }

    // Allow-list validation
    static void validateUsername(String s) {
        if (!s.matches("[A-Za-z0-9_]{3,16}")) throw new IllegalArgumentException("bad username");
    }

    // Secrets must not leak through toString
    record Token(String value) {
        @Override public String toString() { return "Token[****]"; }
    }

    static byte[] serialize(Object o) throws IOException {
        ByteArrayOutputStream bos = new ByteArrayOutputStream();
        try (ObjectOutputStream oos = new ObjectOutputStream(bos)) { oos.writeObject(o); }
        return bos.toByteArray();
    }

    static Object deserialize(byte[] data, ObjectInputFilter filter) throws Exception {
        try (ObjectInputStream ois = new ObjectInputStream(new ByteArrayInputStream(data))) {
            ois.setObjectInputFilter(filter);
            return ois.readObject();
        }
    }

    public static void main(String[] args) throws Exception {
        Path base = Path.of("/srv/app/uploads").toAbsolutePath().normalize();
        System.out.println(resolveSafe(base, "report.pdf"));
        for (String evil : new String[] {"../../etc/passwd", "/etc/passwd"}) {
            try { System.out.println(resolveSafe(base, evil)); }
            catch (SecurityException e) { System.out.println(e.getMessage()); }
        }
        System.out.println("unchecked: " + base.resolve("../../etc/passwd").normalize());

        String user = "x' OR '1'='1";
        System.out.println("SELECT * FROM users WHERE name = '" + user + "'");

        for (String name : new String[] {"asha_01", "a; DROP"}) {
            try { validateUsername(name); System.out.println("ok " + name); }
            catch (IllegalArgumentException e) { System.out.println(e.getMessage() + ": " + name); }
        }

        byte[] raw = new byte[16];
        new SecureRandom().nextBytes(raw);
        System.out.println("token chars: " + HexFormat.of().formatHex(raw).length());

        byte[] a = "secret".getBytes(), b = "secret".getBytes();
        System.out.println("constant-time equal: " + MessageDigest.isEqual(a, b));
        System.out.println(new Token("abc123"));

        ObjectInputFilter allow = ObjectInputFilter.Config.createFilter(
            "java.lang.Integer;java.lang.Number;java.lang.Object;java.util.ArrayList;!*");
        System.out.println(deserialize(serialize(new ArrayList<>(List.of(1, 2))), allow));
        try {
            deserialize(serialize(new HashSet<>(List.of("x"))), allow);
        } catch (InvalidClassException e) {
            System.out.println("rejected: " + e.getMessage());
        }
    }
}
```

Run it (Java 21) TERMINAL

```
$ java Secure.java
/srv/app/uploads/report.pdf
path escapes base: ../../etc/passwd
path escapes base: /etc/passwd
unchecked: /srv/etc/passwd
SELECT * FROM users WHERE name = 'x' OR '1'='1'
ok asha_01
bad username: a; DROP
token chars: 32
constant-time equal: true
Token[****]
[1, 2]
rejected: filter status: REJECTED
```

12.5.6 Secrets: passwords, keys, tokens

Never put a secret in source code or in version control. Once a password or API key has been committed, it lives in the repository history forever, in every clone; the only safe response is to revoke it and issue a new one. Bots scan public repositories for keys within minutes.

Where secrets should live instead:

- **Environment variables or mounted files** injected at deploy time: `System.getenv("DB_PASSWORD")`. Better than code, but visible to anyone who can inspect the process or its container, and often leaked by crash dumps and debug endpoints.
- **A secrets manager** (HashiCorp Vault, AWS Secrets Manager, Azure Key Vault, Kubernetes Secrets with proper access control) that issues short-lived credentials and logs every access. For production this is the standard answer.
- Separate secrets per environment, with the least privilege each service needs (a read-only database user for a read-only service), and a rotation plan.

In code: keep secrets out of `toString` (the `Token` record in the listing prints `Token[****]`; a record's default would print the value), out of exception messages, and out of logs. A console password prompt (`System.console().readPassword()`) returns a `char[]` that you can overwrite with `Arrays.fill(chars, '\0')` once done, which shortens the time it lives in memory, a defence in depth rather than a guarantee, since the garbage collector may copy objects.

Do not store passwords; store password hashes, and never with a plain fast hash such as `SHA-256` or `MD5`, which can be attacked at billions of guesses per second on a GPU. Use a deliberately slow, salted algorithm designed for passwords: **bcrypt**, **scrypt** or **Argon2** (Spring Security supplies `BCryptPasswordEncoder` and `Argon2PasswordEncoder`). The JDK itself does not include Argon2 or bcrypt, so use a maintained library. The same advice covers cryptography generally: do not invent your own scheme. Use `AES/GCM/NoPadding` for symmetric encryption (not ECB), the platform's TLS for transport, and well-reviewed libraries for anything else.

12.5.7 Randomness and comparisons

`java.util.Random` and `ThreadLocalRandom` are fast and **predictable**: from a few outputs one can compute all later ones. They are for simulations and tests. Anything that must not be guessed (session identifiers, password-reset tokens, API keys, salts, one-time codes) must come from `java.security.SecureRandom`, which draws from the operating system's entropy source. In the listing, `new SecureRandom().nextBytes(raw)` fills 16 bytes (32 hex characters). Use at least 128 bits for tokens. `UUID.randomUUID()` uses `SecureRandom` internally and is fine for unguessable identifiers, though a dedicated 256-bit token is better for credentials.

When comparing secrets (a token from a request against the stored one), `String.equals` stops at the first differing byte, so response time leaks how many leading bytes were correct, a **timing attack**. Use `MessageDigest.isEqual(a, b)`, which takes the same time wherever the bytes differ (it prints `true` in the listing for equal arrays).

12.5.8 Logging without leaking

Logs are widely readable, copied to third-party services and retained for months. Do not log: passwords, tokens, API keys, session ids, full card numbers, national ids, or whole request bodies by default. Mask what must be shown (`****-****-****-1111`). Log identifiers (user id, order id), not personal data, and write structured fields instead of concatenated strings. Strip or encode newline characters in user-controlled values before logging, so an attacker cannot inject fake log lines. A `toString` that includes every field is convenient for debugging and dangerous for logging; override it for sensitive types, as `Token` does.

12.5.9 Dependency hygiene

A modern service is perhaps 10% your code and 90% libraries. The December 2021 Log4Shell vulnerability in the popular Log4j logging library, which allowed remote code execution by logging a crafted string, made the point: you are responsible for the security of everything on your class path, including libraries pulled in indirectly (*transitive* dependencies).

- **Know what you ship.** `mvn dependency:tree` or `gradle dependencies` lists everything. Generate an SBOM (software bill of materials, e.g. CycloneDX) in the build.
- **Scan automatically.** OWASP Dependency-Check, GitHub Dependabot or Renovate, and commercial scanners compare your dependencies against vulnerability databases (CVEs) and open upgrade pull requests.
- **Upgrade routinely**, in small steps, with tests, rather than every three years in one terrifying jump. Run a supported JDK: patch releases of the LTS versions carry security fixes.
- **Minimise dependencies.** Each one is attack surface. Do not add a library for one function the JDK already has.
- **Pin and verify.** Use exact versions (or a lock file), a trusted artifact repository, and checksum or signature verification, because supply-chain attacks publish malicious packages with names similar to real ones.
- **Do not disable certificate validation** "just to make it work". A trust-all `TrustManager` copied from a forum answer turns TLS into theatre.

NOTE
The Security Manager is gone. Older books show `System.setSecurityManager` as a sandbox for untrusted code. It was deprecated in Java 17 and permanently disabled in Java 24, and it was never a good isolation boundary. Today isolation means separate processes, containers, and least-privilege operating-system accounts.

INTERVIEW ANGLE
How do you prevent SQL injection in Java? Never concatenate user input into SQL. Use `PreparedStatement` with `?` placeholders (or a framework that does, like JPA named parameters or Spring `JdbcTemplate`) so values are sent separately from the query text and cannot change its structure. Placeholders cannot stand for identifiers, so table or column names that vary must be chosen from an allow-list. Add input validation and a least-privilege database account as extra layers.

INTERVIEW ANGLE
Why is Java deserialization dangerous and what do you do about it? `ObjectInputStream` instantiates classes named in the byte stream and runs their `readObject` logic. With a suitable gadget class on the class path, crafted bytes lead to code execution before the application sees the object. Avoid it for untrusted input: use JSON or Protobuf into known types. If serialization is unavoidable, set an `ObjectInputFilter` allow-list with depth and size limits, and keep dependencies patched.

REAL WORLD
In Spring Boot services most of this is covered by defaults you must not turn off: Spring Data and `JdbcTemplate` use parameters, Thymeleaf escapes output, Spring Security stores bcrypt hashes and protects against CSRF, and secrets arrive from environment variables or a vault through `application.yml` placeholders. Typical audit findings are not framework bugs, but: a secret in `application.yml` committed to Git, an actuator endpoint exposed without authentication, verbose stack traces returned to clients, and outdated transitive dependencies.

Check yourself

1. Why is `"..."` plus a user-supplied string unsafe in `statement.executeQuery("SELECT * FROM t WHERE n = '" + n + "'")`, and what is the fix in one sentence?

2. What does this print? `System.out.println(Path.of("/srv/up").resolve("/etc/passwd"));` and what does it imply for a path-traversal check?
3. Does `Path.of("/srv/up2/f").normalize().startsWith("/srv/up")` return true or false, and why does that matter?
4. Name two properties that make `SecureRandom` the right choice for a session token and `Random` the wrong one.
5. A token comparison is written `stored.equals(provided)`. What attack does this enable and what do you use instead?
6. List three places a password should never appear.

Answers

1. A quote in `n` ends the literal and the rest of the input is parsed as SQL (injection). Use a `PreparedStatement` with `?` and `setString`, so the value is never parsed as SQL.
2. It prints `/etc/passwd`. Resolving an absolute path against a base returns the absolute path itself, so you must check the resolved, normalised result against the base (`startsWith`), not just reject `..`.
3. `false`. `Path.startsWith` compares whole path elements, so `up2` is not `up`. A string `startsWith` would say true and allow a sibling directory.
4. `SecureRandom` uses an unpredictable OS entropy source with a cryptographic algorithm; `Random` is a linear generator whose future output can be computed from a few observed values (and has only 48 bits of state).
5. A timing attack: `equals` returns at the first difference, so response time reveals correct prefixes. Use `MessageDigest.isEqual` for a constant-time comparison.
6. Any three of: source code or Git history, log files, exception messages, `toString` output, URLs or query strings, plain text in a database, client-side code.

KEY POINTS

Secure coding is mostly a few disciplined habits.

- Treat everything crossing a trust boundary as hostile: validate with allow-lists, parse to real types early, cap sizes, canonicalise before checking.
- Keep data and commands apart: `PreparedStatement` for SQL, `ProcessBuilder` with an argument list for processes, escaping for HTML and logs.
- Defeat path traversal with `resolve`, `normalize` and `startsWith` on `Path`; avoid `ObjectInputStream` on untrusted data and use filters or JSON.
- Use `SecureRandom` for anything secret, `MessageDigest.isEqual` for comparing secrets, slow salted hashes (bcrypt, Argon2) for passwords, and keep secrets out of code and logs.
- Patch and scan dependencies continuously; you ship your libraries' vulnerabilities too.

12.6 Performance rules of thumb

Most Java code is fast enough, and the code that is not is rarely slow where you expect. This chapter gives you a short list of rules that pay off in practice, the order in which to apply them, and a way to check your beliefs with numbers. It deliberately avoids folklore such as "use `final` to speed up methods" or "avoid creating objects": the JIT compiler (Part 9) removes many such costs, and chasing them makes code worse.

12.6.1 Rule zero: measure first

Donald Knuth's line is worth knowing in full: "*We should forget about small efficiencies, say about 97% of the time: premature optimization is the root of all evil. Yet we should not pass up our opportunities in that critical 3%.*" The method that follows from it:

1. Define the goal in numbers (latency at the 99th percentile under 200 ms, 5,000 requests per second, startup under 3 s).
2. Measure the real system under realistic load, with a **profiler**, not a hunch. Java Flight Recorder (`-XX:StartFlightRecording`, Part 10) is built in and cheap enough for production; async-profiler and your IDE's profiler are common alternatives.
3. Fix the biggest cost you found, re-measure, stop when the goal is met.

The usual finding: the time goes into one database query, one remote call, one accidental O(n squared) loop, or the garbage collector, not into the places a developer would guess.

12.6.2 Rule one: algorithms and data structures beat tuning

Choosing a better algorithm changes the growth curve; tuning only changes the constant. The program below measures a few such choices, plus others from this chapter, on the build machine.

PerfDemo.java JAVA

```
import java.util.ArrayList;
import java.util.HashSet;
import java.util.List;
import java.util.Set;
import java.util.function.Supplier;
import java.util.stream.IntStream;

public class PerfDemo {
    static long sink;                       // consume results so the JIT cannot discard the work

    // Crude harness: warm up, then take the best of 5. NOT a substitute for JMH.
    static void time(String label, Supplier<Object> work) {
        for (int i = 0; i < 3; i++) sink += work.get().hashCode();
        long best = Long.MAX_VALUE;
        for (int i = 0; i < 5; i++) {
            long t0 = System.nanoTime();
            sink += work.get().hashCode();
            best = Math.min(best, System.nanoTime() - t0);
        }
        System.out.printf("%-34s %9.2f ms%n", label, best / 1e6);
    }

    public static void main(String[] args) {
        // 1. String building
        time("string += in a loop (20k)", () -> {
            String s = "";
            for (int i = 0; i < 20_000; i++) s += "x";
            return s;
        });
        time("StringBuilder (20k)", () -> {
            StringBuilder sb = new StringBuilder();
            for (int i = 0; i < 20_000; i++) sb.append("x");
            return sb.toString();
        });

        // 2. Boxing
        time("sum with Long (20M)", () -> {
            Long sum = 0L;
            for (int i = 0; i < 20_000_000; i++) sum += i;
            return sum;
        });
        time("sum with long (20M)", () -> {
            long sum = 0L;
            for (int i = 0; i < 20_000_000; i++) sum += i;
            return sum;
        });

        // 3. Collection sizing
        time("ArrayList default, 3M adds", () -> {
            List<Integer> l = new ArrayList<>();
            for (int i = 0; i < 3_000_000; i++) l.add(i);
            return l.size();
        });
        time("ArrayList presized, 3M adds", () -> {
            List<Integer> l = new ArrayList<>(3_000_000);
            for (int i = 0; i < 3_000_000; i++) l.add(i);
            return l.size();
        });

        // 4. Algorithm choice: 20k lookups in 100k elements
        List<Integer> list = new ArrayList<>();
        for (int i = 0; i < 100_000; i++) list.add(i * 2);
        Set<Integer> set = new HashSet<>(list);
        time("List.contains x 20k", () -> {
            int hits = 0;
```

PerfDemo.java (continued) JAVA

```
            for (int i = 0; i < 20_000; i++) if (list.contains(i * 7)) hits++;
            return hits;
        });
        time("HashSet.contains x 20k", () -> {
            int hits = 0;
            for (int i = 0; i < 20_000; i++) if (set.contains(i * 7)) hits++;
            return hits;
        });

        // 5. Streams vs loops over primitives
        int[] data = IntStream.range(0, 10_000_000).toArray();
        time("loop: sum of squares (10M)", () -> {
            long s = 0;
            for (int v : data) s += (long) v * v;
            return s;
        });
        time("IntStream: sum of squares", () ->
            IntStream.of(data).mapToLong(v -> (long) v * v).sum());
        time("boxed Stream<Integer> version", () ->
            java.util.Arrays.stream(data).boxed().mapToLong(v -> (long) v * v).sum());
    }
}
```

Terminal TERMINAL

```
$ java PerfDemo.java
string += in a loop (20k)              37.43 ms
StringBuilder (20k)                     0.13 ms
sum with Long (20M)                    95.60 ms
sum with long (20M)                     7.16 ms
ArrayList default, 3M adds            107.71 ms
ArrayList presized, 3M adds            31.61 ms
List.contains x 20k                  1106.91 ms
HashSet.contains x 20k                  0.38 ms
loop: sum of squares (10M)              1.59 ms
IntStream: sum of squares               1.70 ms
boxed Stream<Integer> version          48.27 ms
```

NOTE

About these numbers Measured on the build machine with JDK 21; typical figures only. Your numbers will differ, and individual lines vary a lot between runs (the `ArrayList` lines moved by a factor of five between two runs because garbage collection kicked in at different moments). Read them as orders of magnitude. The harness is a crude warm-up-and-take-the-best; it is **not** a trustworthy benchmark, which is exactly why the JMH section below exists.

What the numbers show

- `List.contains` **versus** `HashSet.contains`: roughly a thousand milliseconds against a fraction of one. `List.contains` scans (O(n)); the hash lookup is O(1) on average. Twenty thousand lookups in a 100,000-element list is 2 billion comparisons. This is the single most common real-world performance bug: a `contains` or `remove` on a list inside a loop. Put the lookup data into a `HashSet` or `HashMap` first.
- `String +=` **in a loop** versus `StringBuilder`: hundreds of times slower at 20,000 iterations and getting worse quadratically, because each `+=` copies the whole string built so far. A single expression such as `"a" + b + "c"` is fine (the compiler turns it into one efficient `invokedynamic` call); the problem is repeated concatenation across loop iterations. Details are in the strings chapter in Part 4.
- `Long` **versus** `long`: the boxed loop is about ten times slower, because every `sum += i` unboxes, adds and allocates a new `Long` (for values outside the small cache of -128 to 127). Boxing also costs memory: an `Integer` object is 16 bytes plus a reference; an `int` in an array is 4 bytes. `List<Integer>` with a million elements is several times larger than an `int[]`.
- **Pre-sizing an** `ArrayList`: when the final size is known, `new ArrayList<>(n)` avoids the repeated grow-and-copy cycles (each growth allocates about 1.5 times the old capacity and copies the elements). For a `HashMap`, Java 19 added `HashMap.newHashMap(expectedEntries)` which computes the right table size for the 0.75 load factor.
- **Streams versus loops**: on primitives, `IntStream` matched the hand-written loop in this run. The version that boxes every element with `.boxed()` was far slower. Streams are not slow; boxing is. For ordinary business code (filter, map, collect over a few thousand objects) readability should decide. Do not use `parallelStream()` as a speed-up without measuring; it helps only for large, CPU-bound, independent work (Part 7).

GOTCHA

Suspiciously fast results The loop doing 10 million multiplications in under 2 ms is plausible only because the JIT compiler can vectorise and unroll simple loops over arrays. A loop whose result is never used may be removed entirely, which is why the harness accumulates every result into `sink`. If a benchmark is "impossibly fast", the JIT probably deleted your work.

12.6.3 Rule two: do less I/O, and buffer it

Each call to a system service (`read`, `write`, a network send, a database round trip) costs thousands of times more than a method call. The remedy is the same everywhere: **batch the work**.

PerfIoDemo.java JAVA

```
import java.io.BufferedOutputStream;
import java.io.FileOutputStream;
import java.io.IOException;
import java.io.OutputStream;
import java.nio.file.Files;
import java.nio.file.Path;
import java.util.HashMap;
import java.util.Map;
import java.util.regex.Pattern;

public class PerfIoDemo {
    static final int BYTES = 300_000;
    static final Pattern ORDER_ID = Pattern.compile("\\w+-\\d+");     // compile once, reuse
    static final Map<Integer, Long> memo = new HashMap<>();

    static long fibSlow(int n) { return n < 2 ? n : fibSlow(n - 1) + fibSlow(n - 2); }
    static long fibMemo(int n) {
        if (n < 2) return n;
        Long cached = memo.get(n);
        if (cached != null) return cached;
        long v = fibMemo(n - 1) + fibMemo(n - 2);
        memo.put(n, v);
        return v;
    }

    static long timeMs(Runnable r) {
        long t0 = System.nanoTime();
        r.run();
        return (System.nanoTime() - t0) / 1_000_000;
    }

    static void write(Path p, boolean buffered) {
        try (OutputStream raw = new FileOutputStream(p.toFile());
             OutputStream out = buffered ? new BufferedOutputStream(raw) : raw) {
            for (int i = 0; i < BYTES; i++) out.write('x');
        } catch (IOException e) {
            throw new RuntimeException(e);
        }
    }

    public static void main(String[] args) throws IOException {
        Path file = Files.createTempFile("perf", ".bin");
        write(file, true);                                    // warm up
        System.out.println("unbuffered, " + BYTES + " write() calls: "
            + timeMs(() -> write(file, false)) + " ms");
        System.out.println("buffered,   " + BYTES + " write() calls: "
            + timeMs(() -> write(file, true)) + " ms");
        Files.delete(file);

        System.out.println("fib(35) recursive: " + timeMs(() -> fibSlow(35)) + " ms");
        System.out.println("fib(35) memoized:  " + timeMs(() -> fibMemo(35)) + " ms");

        long t = timeMs(() -> {
            int c = 0;
            for (int i = 0; i < 200_000; i++) if ("order-12345".matches("\\w+-\\d+")) c++;
        });
        long u = timeMs(() -> {
            int c = 0;
            for (int i = 0; i < 200_000; i++) if (ORDER_ID.matcher("order-12345").matches()) c++;
        });
```

PerfIoDemo.java (continued) JAVA

```
        System.out.println("String.matches, 200k calls: " + t + " ms");
        System.out.println("precompiled Pattern, 200k:  " + u + " ms");
    }
}
```

Terminal TERMINAL

```
$ java PerfIoDemo.java
unbuffered, 300000 write() calls: 331 ms
buffered,   300000 write() calls: 58 ms
fib(35) recursive: 101 ms
fib(35) memoized:  0 ms
String.matches, 200k calls: 392 ms
precompiled Pattern, 200k:  53 ms
```

What to notice

- Writing 300,000 single bytes straight to a `FileOutputStream` makes 300,000 system calls. Wrapping it in a `BufferedOutputStream` (an 8 KiB buffer by default) turns that into a few dozen. That is a ten-fold or better gain from one line, and the decorator pattern from the earlier chapter is what makes it a one-line change. Wrap `FileReader` in `BufferedReader`, and use `Files.readAllLines`/`Files.lines` or `Files.newBufferedReader` for text.
- The same principle drives database code (JDBC batch inserts, `IN` lists instead of N queries, avoiding the "N+1 selects" pattern of ORMs), network code (fewer, larger requests) and logging (asynchronous appenders).
- **Caching** turned `fib(35)` from tens of milliseconds into near zero: the memo table makes the recursion linear instead of exponential. A cache trades memory and staleness for speed, so decide the invalidation rule (time to live, size bound, explicit evict) before you write it. In production use a library such as Caffeine rather than a bare `HashMap`, which is unbounded and not thread safe. Spring's `@Cacheable` puts a cache in front of a method.
- Compiling a regular expression is expensive. `String.matches(regex)` compiles the pattern on every call; a `static final Pattern` compiles it once. The precompiled version is roughly ten times faster here. The same holds for `DateTimeFormatter`, `ObjectMapper`, `MessageDigest` factories and other costly-to-build, safe-to-share objects: build once, reuse (check that the class is thread safe first; `SimpleDateFormat` is not).

12.6.4 Rule three: understand allocation, but do not fear it

The JVM allocates small objects quickly: a bump of a pointer in the thread-local allocation buffer, plus work later in the young generation (Part 9). Short-lived objects are cheap to collect, because the collector only pays for what survives. The practical consequences:

- **Do not pool ordinary objects.** An object pool is almost always slower and buggier than letting the allocator work. Pool only objects that are expensive to create and manage outside the heap: database connections (HikariCP), threads (`ExecutorService`), direct buffers.
- **Do avoid allocation in the hottest loops** when a profiler shows allocation pressure (a high allocation rate in the JFR "Allocation" view, or frequent young collections in `-Xlog:gc`): use primitives instead of boxes, reuse a `StringBuilder`, return a count instead of a new list.
- **Watch for hidden allocation**: lambdas that capture variables allocate on each evaluation, `String.format` allocates a formatter and several strings, `stream()` creates a pipeline of objects, varargs creates an array, autoboxing creates `Integer`s.
- **Prefer primitive collections** only after measuring: `List<Integer>` and `Map<Integer, Integer>` box everything, and libraries such as Eclipse Collections, fastutil or Trove provide `int`-keyed versions. For most programs the standard collections are fine.
- **Memory is also speed.** Data that fits in the CPU cache is much faster to process. Iterating an `int[]` is faster than iterating a `LinkedList<Integer>`, partly because the array is contiguous and partly because there are no objects to dereference. This is why `ArrayList` beats `LinkedList` in most benchmarks, including many where the textbook says `LinkedList` should win.

WATCH OUT

Never tune the GC first Garbage collector flags are the last knob, not the first. The default G1 collector is a good choice for most services. Reduce the amount of garbage and the size of the live set, and choose heap size from measurements (`-Xmx`), before you consider switching collectors. The GC tuning chapter in Part 9 shows how to read the logs.

12.6.5 Rule four: know your concurrency costs

Contention is performance's silent killer. A `synchronized` method or a single lock that every request needs serialises the program no matter how many cores it has. Prefer immutable data, thread-confined state, `ConcurrentHashMap`, `LongAdder` for hot counters, and sensible thread pool sizing (CPU-bound work: about the number of cores; blocking I/O: more threads or virtual threads). Part 8 covers each of these. For database-heavy services the connection pool size and the slowest query usually determine throughput long before Java code does.

12.6.6 Benchmarking properly: JMH

A naive `System.nanoTime()` around a loop, like the harness above, is fragile for reasons you now know: the JVM interprets code first and compiles it later (warm-up), the JIT may remove work whose result is unused (dead code elimination), compute constants at compile time (constant folding), inline differently depending on what ran before (profile pollution), and garbage collection pauses land at random.

JMH (the Java Microbenchmark Harness, an OpenJDK project, used by the JDK team itself) handles forks, warm-up, measurement iterations and the "black hole" that stops dead-code elimination. A benchmark looks like this (JMH is a separate dependency, so this listing is shown as text and was not run here):

StringBench.java (needs the JMH dependency; not run here) JAVA

```
import org.openjdk.jmh.annotations.*;
import java.util.concurrent.TimeUnit;

@State(Scope.Thread)
@BenchmarkMode(Mode.AverageTime)
@OutputTimeUnit(TimeUnit.NANOSECONDS)
@Warmup(iterations = 3, time = 1)
@Measurement(iterations = 5, time = 1)
@Fork(2)
public class StringBench {
    @Param({"10", "1000"})
    int n;

    @Benchmark
    public String builder() {
        StringBuilder sb = new StringBuilder();
        for (int i = 0; i < n; i++) sb.append('x');
        return sb.toString();            // returning the value keeps it "used"
    }
}
```

You build it with the JMH archetype in Maven or the JMH Gradle plugin and run the resulting jar. Even with JMH, treat microbenchmarks as hints: the real question is always whether the change helps the whole application under load.

12.6.7 A practical checklist

Habit	Typical gain
Use the right data structure (`HashSet` or `HashMap` for lookups)	orders of magnitude when n is large
Fix N+1 queries, batch database and network calls	often the largest win in a service
Buffer I/O, reuse connections and threads through pools	10x or more for chatty I/O
Cache results of expensive pure computations, with a bound	large; adds staleness risk
Build with `StringBuilder` in loops, reuse `Pattern` and formatters	10x to 100x in the hot spot
Avoid boxing in numeric hot paths (`long`, `IntStream`)	several times
Pre-size collections when the size is known	modest (up to 2x on building)
Measure with a profiler, then with JMH	prevents wasted effort

Performance habits in order of payoff.

INTERVIEW ANGLE

How would you approach a Java service that is slow? Reproduce the problem and measure: look at the metrics (CPU, memory, GC, thread states, database time), take a profile or a Flight Recorder recording, and find where the time goes before touching code. Typical culprits are N+1 queries and slow SQL, lock contention, excessive allocation or garbage collection, blocking calls on too few threads, and algorithmic inefficiency. Change one thing, re-measure, and keep the result only if it helps.

REAL WORLD

In Spring Boot services Expect the first wins from: connection pool sizing, query tuning and `JOIN FETCH` or batching instead of lazy-loading in a loop, response compression and pagination, `@Cacheable` for hot reference data, and keeping startup lean (fewer auto-configurations, lazy beans). Micrometer metrics plus Actuator expose what to measure.

Check yourself

1. You find `ids.contains(x)` inside a loop over 50,000 orders, where `ids` is an `ArrayList<Long>` of 100,000 entries. What is the problem and the fix?
2. Why is `String s = ""; for (...) s += part;` slow, while `String t = a + b + c;` is fine?
3. What does `Long total = 0L; for (int i = 0; i < n; i++) total += i;` do wrong compared with `long total`?
4. A colleague reports "method A is 3x faster than B" from timing one run with `System.nanoTime()`. Give three reasons the result may be wrong.
5. Why does wrapping a `FileOutputStream` in a `BufferedOutputStream` speed up writing single bytes?
6. When is it right to pre-size an `ArrayList` or `HashMap`, and why is it not a big win in most programs?

Answers

1. Each `contains` scans the list (O(n)), so the loop is O(n times m), billions of comparisons. Copy the ids into a `HashSet<Long>` once and use its `contains` (O(1) average).
2. `+=` creates a new string and copies all previous characters every iteration, which is quadratic overall. A single `+` expression is compiled to one concatenation call that sizes the result once; for loops, use a `StringBuilder`.
3. It boxes: every iteration unboxes `total`, adds, and allocates a new `Long` (outside the -128 to 127 cache). It is several times slower and creates garbage. Use `long`.
4. No warm-up (the code ran interpreted), JIT optimisations such as dead-code elimination or constant folding removed the work, GC pauses or other processes on the machine disturbed one run, and the result may differ for other inputs. Use JMH and several forks.
5. Each `write(int)` on the raw stream is a system call; the buffered stream collects bytes in memory (8 KiB by default) and issues one system call per full buffer.
6. When you know the approximate final size, which avoids repeated resize-and-copy. The gain is modest (growth is amortised O(1)); it matters mainly for very large collections or hot paths found by a profiler.

KEY POINTS

- Measure first with a profiler (JFR, async-profiler); set a numeric goal and stop when you reach it.
- Algorithms and data structures dominate: replace list scans with hash lookups, avoid N+1 queries, avoid quadratic string building.
- Batch and buffer I/O, cache expensive pure results with a bound, and precompile or reuse costly shared objects such as `Pattern`.
- Avoid boxing in numeric hot paths, and pre-size collections when the size is known, but do not pool ordinary objects or tune GC flags before reducing garbage.
- Naive timing loops lie; use JMH for microbenchmarks and treat their results as hints.

PART 13

Projects

Four complete programs you can type in, run and extend. Everything here compiles with plain javac; no build tool is needed.

In this part

13.1 How to work through the projects
13.2 Project A: A bank account manager
13.3 Project B: A log file analyzer
13.4 Project C: A concurrent order processor
13.5 Project D: An in-memory key-value store

13.1 How to work through the projects

Reading about Java is not the same as building with it. Interviewers and teammates trust people who have finished programs, not people who know the vocabulary. This Part gives you four projects, each small enough to finish in an evening or two and each chosen to exercise a different slice of the book: object-oriented design and file persistence, streams and java.time, concurrency, and data structures with locking. Every listing was compiled and run on a JDK 21 build machine and the output shown is the real output.

13.1.1 Ground rules shared by all four projects

Each project uses the same conventions, so you only learn them once.

- **Plain tools only.** You need `javac` and `java`, nothing else. Maven, Gradle and JUnit are what you would use at work (see the chapters on testing and packaging in Part 11), but a project you cannot build without downloading half the internet is hard to study.
- **Package per project.** Code lives under `com.example.<project>`, split into sub-packages by role (`model`, `repo`, `service`, `app`). The directory structure must mirror the package names.
- **Sources and tests are separate trees.** `src/` holds production code, `test/` holds tests. Compiled classes go to `out/` and `testout/`, which you can delete at any time.
- **Time and randomness are injected.** Code that calls `LocalDate.now()` or `new Random()` directly cannot be tested. Our classes take a `Clock` (see the java.time chapter) or a seed, so the output in this book is repeatable.
- **A hand-rolled test runner.** JUnit is not assumed to be installed, so every project ships a tiny runner (about 40 lines) that registers named checks, runs them, prints PASS/FAIL and sets the process exit code. You will read it once in the first project and reuse the idea afterwards.
- **Scripted input.** Interactive programs read from `System.in`. We feed them a text file with shell redirection (`< script.txt`), which makes demos reproducible and also makes the program testable.

13.1.2 The standard workflow

For each project, work in this order. It mirrors how professional work goes, just faster.

1. Read the goal and the design section. Draw the classes on paper before reading code.
2. Create the directory tree, then type the files in. Typing, not pasting, is what makes you notice details.
3. Compile. Fix errors one at a time, reading the first error message only; later ones are often consequences.
4. Run the program with the scripted input and compare with the output in the book.
5. Run the tests. Then break something on purpose (change a `>` to `>=`) and watch a test fail. A test you have never seen fail proves little.
6. Do at least one extension idea from the end of the chapter without looking for a solution.

TIP
The compile command used throughout is `javac -d out $(find src -name '*.java')`. The `-d out` flag tells javac where to put `.class` files and it creates the package directories for you. On Windows PowerShell use `javac -d out (Get-ChildItem -Recurse src -Filter *.java).FullName`.

NOTE
The projects deliberately avoid frameworks, but they follow the layering you will meet in Spring Boot: a controller-like entry point (`app`), a service layer with the business rules, a repository interface hiding storage, and plain model classes. When you reach Spring later, you will recognise every box.

Check yourself

1. Why does the project pass a `Clock` into the service instead of calling `LocalDate.now()` inside it?
2. Source file `src/com/example/bank/model/Account.java` declares `package com.example.bank.models;`. What goes wrong and where?
3. What does `javac -d out` do that plain `javac` does not?
4. Why does a test runner need to call `System.exit` with a non-zero code on failure?

Answers

1. A fixed clock makes dates deterministic, so tests and book output never change from day to day.
2. javac compiles it, but the class file lands in `out/com/example/bank/models/` and other classes importing `...model.Account` will not find it; the error appears when compiling or running the importing code. Directory and package should match.
3. It writes class files into the given directory, creating directories that match the packages; without it they land next to the sources.
4. Build servers and shell scripts decide success from the exit code; printing FAIL alone would still look like success to `&&` chains and CI.

KEY POINTS
Every project: plain javac, `src` and `test` trees, package per project with role sub-packages.

KEY POINTS
Inject `Clock` and seeds so behaviour is repeatable and testable.

KEY POINTS
Scripted input plus a tiny test runner make console programs verifiable without any framework.

KEY POINTS
Break something on purpose to prove your tests can fail.

13.2 Project A: A bank account manager

13.2.1 Goal

Build a console program that opens accounts, deposits, withdraws, transfers between accounts, prints statements and remembers everything between runs. It looks like a toy, but it exercises most of Part 3 (classes, interfaces, enums, records, immutability, exceptions) plus collections and NIO file handling. The same design works for an inventory app (items instead of accounts, stock instead of balance); see the extension ideas.

Requirements:

- Accounts have an id, an owner and a type. A savings account may overdraw by up to 5.00; a checking account may not.
- Money is stored as whole cents in a `long`. Never use `double` for money (see the numbers chapter in Part 4).
- Business failures (not enough money) and input mistakes (negative amount, unknown account) are reported differently.

- Data is stored in a CSV file and survives restarts. A crash during saving must not corrupt the file.
- A text menu reads commands such as `deposit AC001 25.25`.

13.2.2 Design

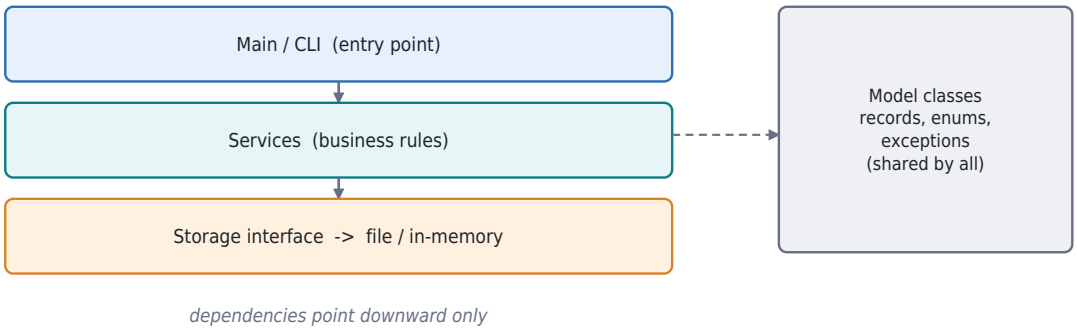


Figure 13.1. The layers used in every project: the entry point talks to services, services talk to a storage interface, and model classes are shared by all layers. Dependencies point downward only.

Responsibilities, from bottom to top:

- **model**: `Account` (mutable state with rules), `AccountType` (enum carrying the overdraft limit), `Transaction` (immutable record), `Money` (parse and format cents), and two exceptions.
- **repo**: the `Repository<T, ID>` interface and two implementations. `InMemoryAccountRepository` keeps a `TreeMap` so listings are sorted by id. `FileAccountRepository` loads a CSV at start and rewrites it after every save.
- **service**: `BankService` holds the use cases (open, deposit, withdraw, transfer). It depends on the `Repository` interface, not on a concrete class, so tests can use the in-memory version.
- **app**: `Main` parses command lines and prints results. No business rule lives here.

Two design decisions deserve a sentence each. First, `InsufficientFundsException` is **checked** because running out of money is a normal outcome that every caller must handle; `InvalidAmountException` extends `IllegalArgumentException` (unchecked) because a negative amount is a bug or bad input, not an outcome to plan for (see the exceptions chapter in Part 5). Second, `Transaction` is a **record** because it is pure immutable data.

13.2.3 The code

Create the tree below. All paths are relative to the project directory.

Project layout TERMINAL

```
$ find src test -name '*.java' | sort
src/com/example/bank/app/Main.java
src/com/example/bank/model/Account.java
src/com/example/bank/model/AccountType.java
src/com/example/bank/model/InsufficientFundsException.java
src/com/example/bank/model/InvalidAmountException.java
src/com/example/bank/model/Money.java
src/com/example/bank/model/Transaction.java
src/com/example/bank/repo/FileAccountRepository.java
src/com/example/bank/repo/InMemoryAccountRepository.java
src/com/example/bank/repo/Repository.java
src/com/example/bank/service/BankService.java
test/com/example/bank/BankTests.java
test/com/example/bank/MiniTest.java
```

We start with the small model types.

AccountType.java JAVA

```java
package com.example.bank.model;

public enum AccountType {
    CHECKING(0), SAVINGS(500);   // overdraft limit in cents

    private final long overdraftLimit;

    AccountType(long overdraftLimit) {
        this.overdraftLimit = overdraftLimit;
```

AccountType.java (continued) JAVA

```
    }

    public long overdraftLimit() {
        return overdraftLimit;
    }
}
```

Transaction.java JAVA

```
package com.example.bank.model;

import java.time.LocalDate;

public record Transaction(LocalDate date, String kind, long amountCents, long balanceAfter) {
    public Transaction {
        if (kind == null || kind.isBlank()) {
            throw new IllegalArgumentException("kind required");
        }
    }

    @Override
    public String toString() {
        return "%s %-12s %10s  balance %10s".formatted(
                date, kind, Money.format(amountCents), Money.format(balanceAfter));
    }
}
```

Money.java JAVA

```
package com.example.bank.model;

public final class Money {
    private Money() { }

    /** Parses "12.34" or "12" into cents. */
    public static long parse(String text) {
        String t = text.trim();
        int dot = t.indexOf('.');
        try {
            if (dot < 0) {
                return Math.multiplyExact(Long.parseLong(t), 100L);
            }
            String frac = t.substring(dot + 1);
            if (frac.length() != 2) {
                throw new NumberFormatException("need exactly 2 decimals");
            }
            long whole = Long.parseLong(t.substring(0, dot));
            return Math.addExact(Math.multiplyExact(whole, 100L), Long.parseLong(frac));
        } catch (NumberFormatException | ArithmeticException e) {
            throw new InvalidAmountException("bad amount: '" + text + "'", e);
        }
    }

    public static String format(long cents) {
        String sign = cents < 0 ? "-" : "";
        long abs = Math.abs(cents);
        return "%s%d.%02d".formatted(sign, abs / 100, abs % 100);
    }
}
```

GOTCHA

`Money.parse("1.5")` is rejected on purpose. Accepting one decimal digit invites the bug where "1.5" becomes 105 cents. Being strict at the boundary is cheaper than debugging wrong balances later. Note also `Math.multiplyExact` and `Math.addExact`: they throw `ArithmeticException` on overflow instead of silently wrapping around (see the numbers chapters), and we translate that into our own exception.

The two exceptions show the checked versus unchecked choice in code.

InsufficientFundsException.java JAVA

```java
package com.example.bank.model;

/** Checked: callers must decide what to do when the money is not there. */
public class InsufficientFundsException extends Exception {
    private final long shortfallCents;

    public InsufficientFundsException(String accountId, long shortfallCents) {
        super("account " + accountId + " is short by " + Money.format(shortfallCents));
        this.shortfallCents = shortfallCents;
    }

    public long shortfallCents() {
        return shortfallCents;
    }
}
```

InvalidAmountException.java JAVA

```java
package com.example.bank.model;

/** Unchecked: a programming or input error, not a business outcome. */
public class InvalidAmountException extends IllegalArgumentException {
    public InvalidAmountException(String message) {
        super(message);
    }

    public InvalidAmountException(String message, Throwable cause) {
        super(message, cause);
    }
}
```

Now the central class, `Account`.

Account.java JAVA

```java
package com.example.bank.model;

import java.time.Clock;
import java.time.LocalDate;
import java.util.ArrayList;
import java.util.Collections;
import java.util.List;
import java.util.Objects;

public class Account {
    private final String id;
    private final String owner;
    private final AccountType type;
    private long balanceCents;
    private final List<Transaction> history = new ArrayList<>();

    public Account(String id, String owner, AccountType type, long openingCents) {
        this.id = Objects.requireNonNull(id);
        this.owner = Objects.requireNonNull(owner);
        this.type = Objects.requireNonNull(type);
        if (openingCents < 0) {
            throw new InvalidAmountException("opening balance must not be negative");
        }
        this.balanceCents = openingCents;
    }

    public String id() { return id; }
    public String owner() { return owner; }
    public AccountType type() { return type; }
    public long balanceCents() { return balanceCents; }

    /** Read-only view: callers cannot modify our history. */
    public List<Transaction> history() {
        return Collections.unmodifiableList(history);
    }

    public void deposit(long cents, Clock clock) {
        requirePositive(cents);
        balanceCents = Math.addExact(balanceCents, cents);
        history.add(new Transaction(LocalDate.now(clock), "DEPOSIT", cents, balanceCents));
```

Account.java (continued) JAVA

```
    }

    public void withdraw(long cents, Clock clock) throws InsufficientFundsException {
        requirePositive(cents);
        long available = balanceCents + type.overdraftLimit();
        if (cents > available) {
            throw new InsufficientFundsException(id, cents - available);
        }
        balanceCents -= cents;
        history.add(new Transaction(LocalDate.now(clock), "WITHDRAW", cents, balanceCents));
    }

    private static void requirePositive(long cents) {
        if (cents <= 0) {
            throw new InvalidAmountException("amount must be positive, got " + Money.format(cents));
        }
    }

    /** Used by the repository when loading from disk (history is not persisted). */
    public static Account restore(String id, String owner, AccountType type, long cents) {
        return new Account(id, owner, type, Math.max(cents, 0)).withBalance(cents);
    }

    private Account withBalance(long cents) {
        this.balanceCents = cents;
        return this;
    }

    @Override
    public String toString() {
        return "%s [%s] %s %s".formatted(id, type, owner, Money.format(balanceCents));
    }
}
```

What to notice

- Fields are `private`; the only way to change the balance is `deposit` or `withdraw`, so the rules cannot be bypassed (encapsulation).
- `history()` returns `Collections.unmodifiableList(history)`, a read-only view. Without it a caller could call `account.history().clear()` and erase the audit trail. A test checks this.
- `withdraw` checks the rule before changing any state, then updates balance and history together. The order matters: if validation threw after the balance changed, the object would be left inconsistent.
- `restore` is a named factory used only when loading from disk. A comment says history is not persisted; that is a known limitation we list under extensions.

Next the storage layer: one interface, two implementations.

Repository.java JAVA

```
package com.example.bank.repo;

import java.util.List;
import java.util.Optional;

public interface Repository<T, ID> {
    Optional<T> findById(ID id);

    List<T> findAll();

    void save(T entity);
}
```

InMemoryAccountRepository.java JAVA

```
package com.example.bank.repo;

import com.example.bank.model.Account;
import java.util.ArrayList;
import java.util.List;
import java.util.Map;
import java.util.Optional;
import java.util.TreeMap;
```

InMemoryAccountRepository.java (continued) JAVA

```java
public class InMemoryAccountRepository implements Repository<Account, String> {
    private final Map<String, Account> accounts = new TreeMap<>();   // sorted by id

    @Override
    public Optional<Account> findById(String id) {
        return Optional.ofNullable(accounts.get(id));
    }

    @Override
    public List<Account> findAll() {
        return new ArrayList<>(accounts.values());
    }

    @Override
    public void save(Account account) {
        accounts.put(account.id(), account);
    }
}
```

FileAccountRepository.java JAVA

```java
package com.example.bank.repo;

import com.example.bank.model.Account;
import com.example.bank.model.AccountType;
import java.io.IOException;
import java.io.UncheckedIOException;
import java.nio.charset.StandardCharsets;
import java.nio.file.Files;
import java.nio.file.Path;
import java.nio.file.StandardCopyOption;
import java.util.ArrayList;
import java.util.List;
import java.util.Optional;

/** Wraps another repository and persists all accounts to a CSV file after every save. */
public class FileAccountRepository implements Repository<Account, String> {
    private final Path file;
    private final InMemoryAccountRepository cache = new InMemoryAccountRepository();

    public FileAccountRepository(Path file) {
        this.file = file;
        load();
    }

    private void load() {
        if (!Files.exists(file)) {
            return;
        }
        try {
            for (String line : Files.readAllLines(file, StandardCharsets.UTF_8)) {
                if (line.isBlank() || line.startsWith("#")) {
                    continue;
                }
                String[] p = line.split(",", -1);
                cache.save(Account.restore(p[0], p[1], AccountType.valueOf(p[2]),
                        Long.parseLong(p[3])));
            }
        } catch (IOException e) {
            throw new UncheckedIOException("cannot read " + file, e);
        }
    }

    @Override
    public Optional<Account> findById(String id) {
        return cache.findById(id);
    }

    @Override
    public List<Account> findAll() {
        return cache.findAll();
    }
```

FileAccountRepository.java (continued) JAVA

```
    @Override
    public void save(Account account) {
        cache.save(account);
        flush();
    }

    /** Write to a temp file, then move it over the real one: a crash cannot leave half a file. */
    private void flush() {
        List<String> lines = new ArrayList<>();
        lines.add("# id,owner,type,balanceCents");
        for (Account a : cache.findAll()) {
            lines.add(String.join(",", a.id(), a.owner(), a.type().name(),
                    Long.toString(a.balanceCents())));
        }
        try {
            Path tmp = file.resolveSibling(file.getFileName() + ".tmp");
            Files.write(tmp, lines, StandardCharsets.UTF_8);
            Files.move(tmp, file, StandardCopyOption.REPLACE_EXISTING,
                    StandardCopyOption.ATOMIC_MOVE);
        } catch (IOException e) {
            throw new UncheckedIOException("cannot write " + file, e);
        }
    }
}
```

What to notice

- `findById` returns `Optional`, so "not found" cannot be confused with a `null` bug.
- `FileAccountRepository` reuses the in-memory version as a cache (composition over inheritance). Only `save` touches the disk.
- `flush` writes `accounts.csv.tmp` and then moves it over the real file with `ATOMIC_MOVE`. If the program dies mid-write, the old file is still intact. This is the standard "write temp, then rename" pattern (see the NIO.2 chapter in Part 5).
- Both IO methods convert `IOException` into `UncheckedIOException` with a clear message and the original cause. The repository interface stays free of checked IO exceptions, so services need not know the storage is a file.

WATCH OUT

`ATOMIC_MOVE` is not supported on every file system and combining it with `REPLACE_EXISTING` may be ignored on some platforms. It works on normal local disks on Linux, macOS and Windows. If you target network drives, catch `AtomicMoveNotSupportedException` and fall back to a plain replace.

The service holds the use cases.

BankService.java JAVA

```
package com.example.bank.service;

import com.example.bank.model.Account;
import com.example.bank.model.AccountType;
import com.example.bank.model.InsufficientFundsException;
import com.example.bank.model.InvalidAmountException;
import com.example.bank.repo.Repository;
import java.time.Clock;
import java.util.List;
import java.util.NoSuchElementException;

public class BankService {
    private final Repository<Account, String> repo;
    private final Clock clock;
    private int nextNumber;

    public BankService(Repository<Account, String> repo, Clock clock) {
        this.repo = repo;
        this.clock = clock;
        this.nextNumber = repo.findAll().size() + 1;
    }

    public Account open(String owner, AccountType type, long openingCents) {
        Account a = new Account("AC%03d".formatted(nextNumber++), owner, type, openingCents);
        repo.save(a);
```

BankService.java (continued) JAVA

```
        return a;
    }

    public Account get(String id) {
        return repo.findById(id).orElseThrow(() -> new NoSuchElementException("no account " + id));
    }

    public List<Account> all() {
        return repo.findAll();
    }

    public void deposit(String id, long cents) {
        Account a = get(id);
        a.deposit(cents, clock);
        repo.save(a);
    }

    public void withdraw(String id, long cents) throws InsufficientFundsException {
        Account a = get(id);
        a.withdraw(cents, clock);
        repo.save(a);
    }

    /** Both legs succeed or neither does: withdraw first (it can fail), then deposit. */
    public void transfer(String from, String to, long cents) throws InsufficientFundsException {
        if (from.equals(to)) {
            throw new InvalidAmountException("cannot transfer to the same account");
        }
        Account src = get(from);
        Account dst = get(to);          // look up both BEFORE changing anything
        src.withdraw(cents, clock);
        dst.deposit(cents, clock);
        repo.save(src);
        repo.save(dst);
    }
}
```

The important method is `transfer`. It looks up **both** accounts before changing either, so an unknown destination cannot leave money withdrawn. It withdraws first because that is the step that can fail with the checked exception; the deposit cannot fail after that. In a real database you would use a transaction instead of relying on step order (see the extension ideas for a rollback exercise).

Finally the text menu.

Main.java JAVA

```
package com.example.bank.app;

import com.example.bank.model.AccountType;
import com.example.bank.model.InsufficientFundsException;
import com.example.bank.model.Money;
import com.example.bank.repo.FileAccountRepository;
import com.example.bank.service.BankService;
import java.io.InputStream;
import java.nio.file.Path;
import java.time.Clock;
import java.time.Instant;
import java.time.ZoneOffset;
import java.util.Scanner;

public class Main {
    private final BankService bank;
    private final Scanner in;

    public Main(BankService bank, InputStream input) {
        this.bank = bank;
        this.in = new Scanner(input);
    }

    public void run() {
        while (in.hasNextLine()) {
            String line = in.nextLine().trim();
            if (line.isEmpty()) {
                continue;
```

Main.java (continued) JAVA

```
            }
            System.out.println("> " + line);          // echo, since input is scripted
            String[] w = line.split("\\s+");
            try {
                if (!dispatch(w)) {
                    return;
                }
            } catch (InsufficientFundsException e) {
                System.out.println("DECLINED: " + e.getMessage());
            } catch (IllegalArgumentException | java.util.NoSuchElementException e) {
                System.out.println("ERROR: " + e.getMessage());
            }
        }
    }

    /** Returns false when the user asked to quit. */
    private boolean dispatch(String[] w) throws InsufficientFundsException {
        switch (w[0]) {
            case "open" -> {
                var a = bank.open(w[1], AccountType.valueOf(w[2].toUpperCase()), Money.parse(w[3]));
                System.out.println("opened " + a);
            }
            case "deposit" -> bank.deposit(w[1], Money.parse(w[2]));
            case "withdraw" -> bank.withdraw(w[1], Money.parse(w[2]));
            case "transfer" -> bank.transfer(w[1], w[2], Money.parse(w[3]));
            case "list" -> bank.all().forEach(a -> System.out.println("  " + a));
            case "history" -> bank.get(w[1]).history().forEach(t -> System.out.println("  " + t));
            case "quit" -> {
                return false;
            }
            default -> System.out.println("unknown command: " + w[0]);
        }
        return true;
    }

    public static void main(String[] args) {
        Path file = Path.of(args.length > 0 ? args[0] : "accounts.csv");
        Clock clock = Clock.fixed(Instant.parse("2026-03-01T10:00:00Z"), ZoneOffset.UTC);
        new Main(new BankService(new FileAccountRepository(file), clock), System.in).run();
    }
}
```

What to notice

- The `switch` is the arrow form with no fall-through. The method returns `false` for `quit`, which ends the loop.
- Two catch blocks give two behaviours: a checked business failure prints `DECLINED`, input and lookup mistakes print `ERROR`. One bad command never kills the program.
- The constructor takes an `InputStream`, so a test could pass a `ByteArrayInputStream`.

13.2.4 Running it

Save the commands below as `script1.txt`. Each line is typed into the menu; some are deliberately wrong.

script1.txt

```
open Asha checking 100.00
open Ravi savings 20.50
deposit AC001 25.25
withdraw AC001 500
transfer AC001 AC002 50
withdraw AC002 99.99
deposit AC002 -5
deposit AC009 5
deposit AC001 abc
list
history AC001
quit
```

Compile and run the scripted session TERMINAL

```
$ rm -rf out accounts.csv
$ javac -d out $(find src -name '*.java')
$ java -cp out com.example.bank.app.Main accounts.csv < script1.txt
> open Asha checking 100.00
opened AC001 [CHECKING] Asha 100.00
> open Ravi savings 20.50
opened AC002 [SAVINGS] Ravi 20.50
> deposit AC001 25.25
> withdraw AC001 500
DECLINED: account AC001 is short by 374.75
> transfer AC001 AC002 50
> withdraw AC002 99.99
DECLINED: account AC002 is short by 24.49
> deposit AC002 -5
ERROR: amount must be positive, got -5.00
> deposit AC009 5
ERROR: no account AC009
> deposit AC001 abc
ERROR: bad amount: 'abc'
> list
  AC001 [CHECKING] Asha 75.25
  AC002 [SAVINGS] Ravi 70.50
> history AC001
  2026-03-01 DEPOSIT           25.25  balance     125.25
  2026-03-01 WITHDRAW          50.00  balance      75.25
> quit
```

Look at each declined or failed line. The 500.00 withdrawal is declined with the exact shortfall (checking accounts have no overdraft). The savings account may dip 5.00 below zero, so withdrawing 99.99 from 70.50 is short by 24.49, not 29.49. Negative amounts, unknown accounts and the text "abc" are all reported without crashing.

The data file is plain text and the second run proves persistence.

The CSV file and a second run TERMINAL

```
$ cat accounts.csv
# id,owner,type,balanceCents
AC001,Asha,CHECKING,7525
AC002,Ravi,SAVINGS,7050
$ printf 'list\nwithdraw AC001 1000\nquit\n' | java -cp out com.example.bank.app.Main accounts.csv
> list
  AC001 [CHECKING] Asha 75.25
  AC002 [SAVINGS] Ravi 70.50
> withdraw AC001 1000
DECLINED: account AC001 is short by 924.75
> quit
```

13.2.5 Tests with a hand-rolled runner

Here is the whole test framework. It is intentionally tiny: tests are named lambdas; `assertThrows` takes the expected exception type.

MiniTest.java JAVA

```java
package com.example.bank;

import java.util.ArrayList;
import java.util.List;

/** A tiny stand-in for JUnit: register named checks, run them, report, exit non-zero. */
public final class MiniTest {
    public interface Body {
        void run() throws Exception;
    }

    private record Case(String name, Body body) { }

    private final List<Case> cases = new ArrayList<>();

    public void test(String name, Body body) {
        cases.add(new Case(name, body));
    }
```

MiniTest.java (continued) JAVA

```
    public static void assertEquals(Object expected, Object actual) {
        if (!java.util.Objects.equals(expected, actual)) {
            throw new AssertionError("expected <" + expected + "> but was <" + actual + ">");
        }
    }

    public static void assertTrue(boolean condition, String message) {
        if (!condition) {
            throw new AssertionError(message);
        }
    }

    public static <T extends Throwable> void assertThrows(Class<T> type, Body body) {
        try {
            body.run();
        } catch (Throwable t) {
            if (type.isInstance(t)) {
                return;
            }
            throw new AssertionError("expected " + type.getSimpleName() + " but got " + t);
        }
        throw new AssertionError("expected " + type.getSimpleName() + " but nothing was thrown");
    }

    public int runAll() {
        int failed = 0;
        for (Case c : cases) {
            try {
                c.body().run();
                System.out.println("PASS " + c.name());
            } catch (Throwable t) {
                failed++;
                System.out.println("FAIL " + c.name() + " -> " + t.getMessage());
            }
        }
        System.out.printf("%d tests, %d failed%n", cases.size(), failed);
        return failed;
    }
}
```

Tests use it like JUnit, except they are registered in `main`.

BankTests.java JAVA

```
package com.example.bank;

import static com.example.bank.MiniTest.assertEquals;
import static com.example.bank.MiniTest.assertThrows;

import com.example.bank.model.AccountType;
import com.example.bank.model.InsufficientFundsException;
import com.example.bank.model.InvalidAmountException;
import com.example.bank.model.Money;
import com.example.bank.repo.FileAccountRepository;
import com.example.bank.repo.InMemoryAccountRepository;
import com.example.bank.service.BankService;
import java.nio.file.Files;
import java.nio.file.Path;
import java.time.Clock;
import java.time.Instant;
import java.time.ZoneOffset;

public class BankTests {
    static final Clock CLOCK = Clock.fixed(Instant.parse("2026-03-01T00:00:00Z"), ZoneOffset.UTC);

    static BankService newBank() {
        return new BankService(new InMemoryAccountRepository(), CLOCK);
    }

    public static void main(String[] args) {
        MiniTest t = new MiniTest();

        t.test("Money.parse handles whole and fractional amounts", () -> {
            assertEquals(1250L, Money.parse("12.50"));
            assertEquals(700L, Money.parse("7"));
```

BankTests.java (continued) JAVA

```
        });
        t.test("Money.parse rejects one-decimal input", () ->
                assertThrows(InvalidAmountException.class, () -> Money.parse("1.5")));
        t.test("Money.format round-trips negatives", () ->
                assertEquals("-0.05", Money.format(-5)));
        t.test("deposit increases balance and records history", () -> {
            var bank = newBank();
            var a = bank.open("Asha", AccountType.CHECKING, 1000);
            bank.deposit(a.id(), 250);
            assertEquals(1250L, bank.get(a.id()).balanceCents());
            assertEquals(1, bank.get(a.id()).history().size());
        });
        t.test("withdraw beyond balance throws checked exception", () -> {
            var bank = newBank();
            var a = bank.open("Asha", AccountType.CHECKING, 1000);
            assertThrows(InsufficientFundsException.class, () -> bank.withdraw(a.id(), 1001));
            assertEquals(1000L, bank.get(a.id()).balanceCents());
        });
        t.test("savings may overdraw up to the limit", () -> {
            var bank = newBank();
            var a = bank.open("Ravi", AccountType.SAVINGS, 1000);
            bank.withdraw(a.id(), 1500);
            assertEquals(-500L, bank.get(a.id()).balanceCents());
        });
        t.test("failed transfer changes neither account", () -> {
            var bank = newBank();
            var a = bank.open("A", AccountType.CHECKING, 1000);
            var b = bank.open("B", AccountType.CHECKING, 0);
            assertThrows(InsufficientFundsException.class,
                    () -> bank.transfer(a.id(), b.id(), 5000));
            assertEquals(1000L, bank.get(a.id()).balanceCents());
            assertEquals(0L, bank.get(b.id()).balanceCents());
        });
        t.test("history view cannot be modified", () -> {
            var a = newBank().open("A", AccountType.CHECKING, 0);
            assertThrows(UnsupportedOperationException.class, () -> a.history().clear());
        });
        t.test("file repository survives a restart", () -> {
            Path f = Files.createTempFile("accounts", ".csv");
            var bank = new BankService(new FileAccountRepository(f), CLOCK);
            var a = bank.open("Asha", AccountType.CHECKING, 4200);
            var reloaded = new BankService(new FileAccountRepository(f), CLOCK);
            assertEquals(4200L, reloaded.get(a.id()).balanceCents());
            Files.delete(f);
        });

        System.exit(t.runAll() == 0 ? 0 : 1);
    }
}
```

Compile and run the tests TERMINAL

```
$ javac -d testout -cp out $(find test -name '*.java')
$ java -cp out:testout com.example.bank.BankTests
PASS Money.parse handles whole and fractional amounts
PASS Money.parse rejects one-decimal input
PASS Money.format round-trips negatives
PASS deposit increases balance and records history
PASS withdraw beyond balance throws checked exception
PASS savings may overdraw up to the limit
PASS failed transfer changes neither account
PASS history view cannot be modified
PASS file repository survives a restart
9 tests, 0 failed
$ echo "exit code: $?"
exit code: 0
```

NOTE

On Windows the classpath separator is `;` instead of `:`.

What the tests cover

Pure helpers (parsing and formatting), the happy path, the checked exception, the overdraft rule, the all-or-nothing property of `transfer`, the read-only history and persistence across a "restart" (a second repository reading the same file). Each test builds its own fresh bank so tests cannot affect each other; that independence is the single most important property of a test suite.

INTERVIEW ANGLE

Why is money stored as a `long` of cents rather than `double` or `BigDecimal`? `double` cannot represent most decimal fractions exactly (0.1 + 0.2 is not 0.3), so sums drift. `BigDecimal` is exact and is the right tool when you need arbitrary scale or rounding rules. Whole minor units in a `long` are exact, fast and simple when the currency has a fixed number of decimals, and `Math.addExact` guards overflow. Many payment systems store minor units for exactly this reason.

REAL WORLD

In a Spring Boot version, `BankService` becomes a `@Service`, `Repository` becomes a Spring Data repository backed by a database table, and `transfer` gets `@Transactional` so that both legs commit or roll back together. The shape of the code stays the same.

13.2.6 Extension ideas

1. **Inventory variant.** Replace `Account` with `Item` (sku, name, quantity) and `withdraw` with `remove`; throwing a checked `OutOfStockException`. Add a `low` command that lists items under a threshold using a stream.
2. Persist the transaction history too (one line per transaction in a second file) and load it in `restore`.
3. Make `transfer` genuinely atomic: if `repo.save(dst)` fails after `repo.save(src)`, undo the first save.
4. Add a monthly interest method on savings accounts using `BigDecimal` with `RoundingMode.HALF_EVEN`, and test the rounding.
5. Add a `Statement` formatter that prints a table using a text block template and `String.formatted`.
6. Replace the CSV with a JSON-lines file written by hand, then think about what a library such as Jackson would save you.

Check yourself

1. In `Account`, what happens if `history()` returned the field directly, and how would you notice in the tests?
2. What does this print? `System.out.println(Money.format(-5) + " " + Money.format(1005));`
3. Why does `transfer` call `get(to)` before `src.withdraw(...)`?
4. Why is `InsufficientFundsException` checked while `InvalidAmountException` is not?
5. What is the purpose of writing a `.tmp` file and then moving it?
6. What is wrong with this alternative, and why? `catch (Exception e) { System.out.println("ERROR"); }` around the whole dispatch.

Answers

1. Callers could mutate or clear the history; the "history view cannot be modified" test would fail because no `UnsupportedOperationException` is thrown.
2. `-0.05 10.05` (verified by the unit test for the first value; the second follows the same formula).
3. So an unknown destination fails before any money moves; otherwise the source would already be debited.
4. Running out of money is an expected business outcome callers must handle; a negative amount is a caller bug or bad input.
5. A crash while writing leaves only a half-written `.tmp`; the rename is atomic, so the real file is always complete.
6. It hides the cause (message lost), swallows programming bugs such as `NullPointerException`, and treats business declines like errors. Catch the specific exceptions you can handle.

KEY POINTS

Layers (app, service, repo, model) keep rules in one place and storage replaceable behind an interface.

KEY POINTS

Store money in minor units; use `Math.*Exact`; validate at the boundary.

KEY POINTS
Checked exceptions for expected business outcomes, unchecked for bad input and bugs.

KEY POINTS
Write-then-rename makes file persistence crash-safe; read-only views protect internal state.

13.3 Project B: A log file analyzer

13.3.1 Goal

Operations people read logs for a living, and "count the errors per hour and show me the slow endpoints" is a question you will answer with code many times. This project builds a command-line analyzer that reads a web-server style log, tolerates bad lines, and prints a report. It exercises regular expressions, `java.time`, the Stream API with collectors, `Optional`, records, text blocks and `Files.lines`.

Because we have no real log, the project also contains a **generator** that writes a reproducible fake one. Seeding `java.util.Random` makes the same seed produce the same file on every machine, which is what makes the report below repeatable.

The log format, one entry per line:

Log line format

```
2026-03-01T08:00:01.131Z INFO  [billing] PUT /orders 200 17ms user=u13
timestamp(UTC)           level service   method path   status latency user
```

13.3.2 Design

- **model**: `Level` enum (ordered DEBUG to ERROR) and `LogEntry` record. The record has a derived `endpoint()` that turns `GET /orders/42` and `GET /orders/77` into the same key `GET /orders/{id}`. Without this grouping, every id would be its own "endpoint".
- **io**: `LogGenerator` writes the file; `LogParser` turns one line into an `Optional<LogEntry>`.
- **analysis**: `Analyzer` runs all the statistics and returns an immutable `Report` record.
- **app**: `Main` formats the report with a text block.

Two design choices to notice. The parser returns `Optional` instead of throwing, because malformed lines are expected in real logs and should be counted, not fatal. And the report is **data** (a record), not printed text; formatting lives only in `Main`. That separation is what lets the tests assert on numbers rather than on strings.

13.3.3 The code

Model

Level.java JAVA

```java
package com.example.logs.model;

public enum Level {
    DEBUG, INFO, WARN, ERROR;

    public boolean atLeast(Level other) {
        return compareTo(other) >= 0;
    }
}
```

LogEntry.java JAVA

```java
package com.example.logs.model;

import java.time.Instant;

public record LogEntry(Instant time, Level level, String service, String method,
                       String path, int status, int millis, String user) {
```

LogEntry.java (continued) JAVA

```
    /** "/orders/42" and "/orders/77" are the same endpoint: collapse numeric ids. */
    public String endpoint() {
        return method + " " + path.replaceAll("/\\d+", "/{id}");
    }

    public boolean isServerError() {
        return status >= 500;
    }
}
```

The generator

LogGenerator.java JAVA

```java
package com.example.logs.io;

import com.example.logs.model.Level;
import java.io.IOException;
import java.io.UncheckedIOException;
import java.nio.file.Files;
import java.nio.file.Path;
import java.time.Instant;
import java.util.ArrayList;
import java.util.List;
import java.util.Random;

/** Writes a reproducible fake log: the same seed always gives the same file. */
public final class LogGenerator {
    private static final String[] SERVICES = {"api", "auth", "billing"};
    private static final String[] PATHS = {"/orders", "/orders/%d", "/login", "/users/%d",
        "/invoices/%d", "/health"};
    private static final String[] METHODS = {"GET", "GET", "GET", "POST", "PUT"};

    private LogGenerator() { }

    public static void write(Path file, int lines, long seed, Instant start) {
        Random rnd = new Random(seed);
        List<String> out = new ArrayList<>(lines);
        Instant t = start;
        for (int i = 0; i < lines; i++) {
            t = t.plusMillis(1 + rnd.nextInt(15_000));          // up to 15 s between lines
            if (i % 97 == 96) {
                out.add("### corrupted line " + i);              // malformed on purpose
                continue;
            }
            int status = pickStatus(rnd);
            Level level = status >= 500 ? Level.ERROR : status >= 400 ? Level.WARN
                    : rnd.nextInt(10) == 0 ? Level.DEBUG : Level.INFO;
            String service = SERVICES[rnd.nextInt(SERVICES.length)];
            String path = PATHS[rnd.nextInt(PATHS.length)].formatted(1 + rnd.nextInt(500));
            int millis = 5 + (int) (-Math.log(1 - rnd.nextDouble()) * 40)
                    + (status >= 500 ? 300 : 0);
            out.add("%s %-5s [%s] %s %s %d %dms user=u%02d".formatted(t, level, service,
                    METHODS[rnd.nextInt(METHODS.length)], path, status, millis, rnd.nextInt(40)));
        }
        try {
            Files.write(file, out);
        } catch (IOException e) {
            throw new UncheckedIOException(e);
        }
    }

    private static int pickStatus(Random rnd) {
        int r = rnd.nextInt(100);
        if (r < 85) {
            return 200;
        } else if (r < 90) {
            return 201;
        } else if (r < 95) {
            return 404;
        } else if (r < 97) {
            return 401;
        }
        return 500;
    }
}
```

What to notice: `new Random(seed)` gives a fixed sequence; every 97th line is replaced with garbage so we can test error handling; latencies follow an exponential distribution (`-Math.log(1 - u)` times a mean), which resembles real latency data with a long tail; 5xx responses get an extra 300 ms. The format string uses `%-5s` to pad the level and `Instant.toString()` for the timestamp, which is always ISO-8601 in UTC.

The parser

LogParser.java JAVA

```java
package com.example.logs.io;

import com.example.logs.model.Level;
import com.example.logs.model.LogEntry;
import java.time.Instant;
import java.time.format.DateTimeParseException;
import java.util.Optional;
import java.util.regex.Matcher;
import java.util.regex.Pattern;

public final class LogParser {
    // named groups make the pattern self-documenting
    private static final Pattern LINE = Pattern.compile(
            "^(?<ts>\\S+)\\s+(?<level>[A-Z]+)\\s+\\[(?<svc>[a-z]+)]\\s+"
            + "(?<method>[A-Z]+)\\s+(?<path>\\S+)\\s+(?<status>\\d{3})\\s+"
            + "(?<ms>\\d+)ms\\s+user=(?<user>\\w+)$");

    private LogParser() { }

    /** Empty means "this line is not a valid entry"; the caller counts it. */
    public static Optional<LogEntry> parse(String line) {
        Matcher m = LINE.matcher(line);
        if (!m.matches()) {
            return Optional.empty();
        }
        try {
            return Optional.of(new LogEntry(
                    Instant.parse(m.group("ts")),
                    Level.valueOf(m.group("level")),
                    m.group("svc"),
                    m.group("method"),
                    m.group("path"),
                    Integer.parseInt(m.group("status")),
                    Integer.parseInt(m.group("ms")),
                    m.group("user")));
        } catch (DateTimeParseException | IllegalArgumentException e) {
            return Optional.empty();
        }
    }
}
```

The regular expression uses **named groups** (`(?<status>\d{3})`) read with `m.group("status")`. Named groups survive edits to the pattern; numbered groups break silently when you add one. The pattern is a `static final Pattern`, compiled once: compiling inside the per-line method would repeat the work two thousand times. `matches()` requires the whole line to fit the pattern, and `^...$` makes the intent explicit.

> **GOTCHA**
> The `catch` covers `IllegalArgumentException` because `Level.valueOf("LOUD")` throws it, and `NumberFormatException` (a subclass) from a number that does not fit an `int`. `DateTimeParseException` is not a subclass of it, so both appear in the multi-catch (which forbids listing a class and its own subclass together).

The analyzer

Report.java JAVA

```java
package com.example.logs.analysis;

import com.example.logs.model.Level;
import com.example.logs.model.LogEntry;
import java.time.Instant;
import java.util.List;
import java.util.Map;

/** Everything the analyzer computed; plain data so tests can inspect each piece. */
public record Report(int total, int malformed, Instant first, Instant last,
                     Map<Level, Long> byLevel,
                     List<Map.Entry<String, Long>> topEndpoints,
                     Map<String, Double> avgMillisByService,
                     long p95Millis,
                     Map<Instant, Long> errorsPerHour,
                     Map<Boolean, Long> okVsFailed,
                     List<LogEntry> slowest) { }
```

Analyzer.java JAVA

```java
package com.example.logs.analysis;

import com.example.logs.io.LogParser;
import com.example.logs.model.Level;
import com.example.logs.model.LogEntry;
import java.io.IOException;
import java.io.UncheckedIOException;
import java.nio.file.Files;
import java.nio.file.Path;
import java.time.temporal.ChronoUnit;
import java.util.Comparator;
import java.util.List;
import java.util.Map;
import java.util.Optional;
import java.util.TreeMap;
import java.util.stream.Collectors;
import java.util.stream.Stream;

public final class Analyzer {
    private Analyzer() { }

    public static Report analyze(Path file) {
        try (Stream<String> lines = Files.lines(file)) {
            List<String> all = lines.filter(l -> !l.isBlank()).toList();
            List<LogEntry> entries = all.stream()
                    .map(LogParser::parse)
                    .flatMap(Optional::stream)          // drops the empty Optionals
                    .toList();
            return analyze(entries, all.size() - entries.size());
        } catch (IOException e) {
            throw new UncheckedIOException("cannot read " + file, e);
        }
    }

    public static Report analyze(List<LogEntry> entries, int malformed) {
        Map<Level, Long> byLevel = entries.stream().collect(Collectors.groupingBy(
                LogEntry::level, () -> new TreeMap<>(), Collectors.counting()));

        List<Map.Entry<String, Long>> top = entries.stream()
                .collect(Collectors.groupingBy(LogEntry::endpoint, Collectors.counting()))
                .entrySet().stream()
                .sorted(Map.Entry.<String, Long>comparingByValue().reversed()
                        .thenComparing(Map.Entry.comparingByKey()))
                .limit(5)
                .toList();

        Map<String, Double> avg = entries.stream().collect(Collectors.groupingBy(
                LogEntry::service, TreeMap::new, Collectors.averagingInt(LogEntry::millis)));

        long[] sorted = entries.stream().mapToLong(LogEntry::millis).sorted().toArray();
        long p95 = sorted.length == 0 ? 0 : sorted[(int) Math.ceil(sorted.length * 0.95) - 1];

        Map<java.time.Instant, Long> errorsPerHour = entries.stream()
```

Analyzer.java (continued) JAVA

```
                .filter(e -> e.level() == Level.ERROR)
                .collect(Collectors.groupingBy(e -> e.time().truncatedTo(ChronoUnit.HOURS),
                        TreeMap::new, Collectors.counting()));

        Map<Boolean, Long> okVsFailed = entries.stream()
                .collect(Collectors.partitioningBy(e -> e.status() < 400, Collectors.counting()));

        List<LogEntry> slowest = entries.stream()
                .sorted(Comparator.comparingInt(LogEntry::millis).reversed()
                        .thenComparing(LogEntry::time))
                .limit(3)
                .toList();

        return new Report(entries.size(), malformed,
                entries.stream().map(LogEntry::time).min(Comparator.naturalOrder()).orElse(null),
                entries.stream().map(LogEntry::time).max(Comparator.naturalOrder()).orElse(null),
                byLevel, top, avg, p95, errorsPerHour, okVsFailed, slowest);
    }
}
```

Walk through the collectors, because they are the point of this project (see the collectors chapter in Part 7).

- `groupingBy(LogEntry::level, () -> new TreeMap<>(), counting())` counts entries per level. The map factory argument picks a `TreeMap`, so keys come out in enum declaration order instead of hash order.
- Top endpoints: group by endpoint, then stream the map's `entrySet()`, sort by count descending with ties broken by key (so output is deterministic), and `limit(5)`. `comparingByValue().reversed()` needs the explicit type witness `Map.Entry.<String, Long>` because Java cannot infer the type through the chained call.
- `averagingInt` computes the mean per service in one pass.
- p95 sorts latencies into a `long[]` and picks the element at rank `ceil(0.95 * n)`. This "nearest rank" definition is simple and common; other definitions interpolate and give slightly different numbers.
- `truncatedTo(ChronoUnit.HOURS)` rounds an `Instant` down to the hour, which makes a natural grouping key for errors per hour.
- `partitioningBy` always produces both keys (`true` and `false`), even if one group is empty, so `get(true)` never returns `null`.
- `.flatMap(Optional::stream)` is the idiom for "keep the present values, drop the empties" (Java 9+).

WATCH OUT

`Files.lines` holds an open file handle until the stream is closed. Always wrap it in try-with-resources as shown. Forgetting it leaks file descriptors, and on Windows it can stop you deleting the file.

The report printer

Main.java JAVA

```java
package com.example.logs.app;

import com.example.logs.analysis.Analyzer;
import com.example.logs.analysis.Report;
import com.example.logs.io.LogGenerator;
import java.nio.file.Files;
import java.nio.file.Path;
import java.time.Instant;
import java.time.ZoneOffset;
import java.time.format.DateTimeFormatter;
import java.util.stream.Collectors;

public class Main {
    private static final DateTimeFormatter HOUR =
            DateTimeFormatter.ofPattern("yyyy-MM-dd HH':00'").withZone(ZoneOffset.UTC);

    public static void main(String[] args) throws Exception {
        Path file = Path.of(args.length > 0 ? args[0] : "sample.log");
        if (!Files.exists(file)) {
            LogGenerator.write(file, 2000, 42L, Instant.parse("2026-03-01T08:00:00Z"));
            System.out.println("generated " + file);
        }
        System.out.print(render(Analyzer.analyze(file)));
    }

    static String render(Report r) {
        String levels = r.byLevel().entrySet().stream()
                .map(e -> "  %-6s %5d".formatted(e.getKey(), e.getValue()))
                .collect(Collectors.joining("\n"));
        String top = r.topEndpoints().stream()
                .map(e -> "  %-22s %5d".formatted(e.getKey(), e.getValue()))
                .collect(Collectors.joining("\n"));
        String avg = r.avgMillisByService().entrySet().stream()
                .map(e -> "  %-8s %6.1f ms".formatted(e.getKey(), e.getValue()))
                .collect(Collectors.joining("\n"));
        String errors = r.errorsPerHour().entrySet().stream()
                .map(e -> "  %s  %3d".formatted(HOUR.format(e.getKey()), e.getValue()))
                .collect(Collectors.joining("\n"));
        String slow = r.slowest().stream()
                .map(e -> "  %5d ms  %s %s".formatted(e.millis(), e.service(), e.endpoint()))
                .collect(Collectors.joining("\n"));
        return """
                LOG REPORT
                entries: %d (malformed lines skipped: %d)
                window : %s to %s

                By level
                %s

                Top endpoints
                %s

                Average latency by service
                %s
                p95 latency: %d ms

                Errors per hour (UTC)
                %s

                Succeeded (<400): %d   Failed (>=400): %d

                Slowest requests
                %s
                """.formatted(r.total(), r.malformed(), r.first(), r.last(), levels, top, avg,
                r.p95Millis(), errors, r.okVsFailed().get(true), r.okVsFailed().get(false), slow);
    }
}
```

The text block uses `%s` and `%d` placeholders and `.formatted(...)` (see the strings chapter). The closing `"""` sits at the left margin of the content, which defines how much indentation is stripped. Each section is first built into a string with `Collectors.joining("\n")`, a common way to turn a collection into multi-line text.

13.3.4 Running it

Generate the sample log and analyze it TERMINAL

```
$ rm -rf out sample.log
$ javac -d out $(find src -name '*.java')
$ java -cp out com.example.logs.app.Main
generated sample.log
LOG REPORT
entries: 1980 (malformed lines skipped: 20)
window : 2026-03-01T08:00:01.131Z to 2026-03-01T12:11:24.770Z

By level
  DEBUG    177
  INFO    1599
  WARN     141
  ERROR     63

Top endpoints
  GET /health              216
  GET /login               215
  GET /orders              204
  GET /invoices/{id}       198
  GET /users/{id}          191

Average latency by service
  api        55.0 ms
  auth       53.1 ms
  billing    56.7 ms
p95 latency: 166 ms

Errors per hour (UTC)
  2026-03-01 08:00   19
  2026-03-01 09:00   15
  2026-03-01 10:00   14
  2026-03-01 11:00   14
  2026-03-01 12:00    1

Succeeded (<400): 1776   Failed (>=400): 204

Slowest requests
    523 ms  billing GET /login
    446 ms  billing GET /users/{id}
    436 ms  auth POST /health
```

A glance at the generated file shows the format, including the deliberately corrupt lines (the analyzer reports exactly 20 skipped out of 2000, one in 97 as designed).

Peek at the raw log TERMINAL

```
$ head -3 sample.log
2026-03-01T08:00:01.131Z INFO  [billing] PUT /orders 200 17ms user=u13
2026-03-01T08:00:13.314Z INFO  [api] POST /login 200 29ms user=u09
2026-03-01T08:00:14.715Z INFO  [api] GET /health 200 24ms user=u06
$ grep -n '###' sample.log | head -2
97:### corrupted line 96
194:### corrupted line 193
$ wc -l sample.log
2000 sample.log
```

Since the generator is seeded, you should see the same numbers. If you change the seed, every figure changes but the invariants hold: the level counts sum to the number of entries, and the successes plus failures equal the same total.

13.3.5 Tests

The tests reuse the `MiniTest` runner from the previous project (copy it into the `com.example.logs` test package). Two things are worth studying: the first test builds data by hand so every expected value can be computed on paper (100 latencies of 1 to 100 ms, so the 95th-percentile by nearest rank is exactly 95), and the generator test proves determinism by writing the same seed twice and comparing the files.

LogTests.java JAVA

```java
package com.example.logs;

import static com.example.logs.MiniTest.assertEquals;
import static com.example.logs.MiniTest.assertTrue;

import com.example.logs.analysis.Analyzer;
import com.example.logs.analysis.Report;
import com.example.logs.io.LogGenerator;
import com.example.logs.io.LogParser;
import com.example.logs.model.Level;
import com.example.logs.model.LogEntry;
import java.nio.file.Files;
import java.nio.file.Path;
import java.time.Instant;
import java.util.List;

public class LogTests {
    static final String GOOD =
            "2026-03-01T08:00:01.131Z INFO  [api] GET /orders/42 200 17ms user=u13";

    static LogEntry entry(String time, Level level, int status, int millis) {
        return new LogEntry(Instant.parse(time), level, "api", "GET", "/orders/1",
                status, millis, "u1");
    }

    public static void main(String[] args) {
        MiniTest t = new MiniTest();

        t.test("parse reads every field", () -> {
            LogEntry e = LogParser.parse(GOOD).orElseThrow();
            assertEquals(Level.INFO, e.level());
            assertEquals("api", e.service());
            assertEquals(200, e.status());
            assertEquals(17, e.millis());
            assertEquals("GET /orders/{id}", e.endpoint());
        });
        t.test("parse rejects garbage and bad timestamps", () -> {
            assertTrue(LogParser.parse("### corrupted").isEmpty(), "garbage accepted");
            assertTrue(LogParser.parse(GOOD.replace("2026-03-01", "2026-13-45")).isEmpty(),
                    "bad date accepted");
            assertTrue(LogParser.parse(GOOD.replace("INFO ", "LOUD ")).isEmpty(),
                    "unknown level accepted");
        });
        t.test("p95 and failure partition on a hand-made list", () -> {
            List<LogEntry> list = new java.util.ArrayList<>();
            for (int i = 1; i <= 100; i++) {
                list.add(entry("2026-03-01T08:00:00Z", Level.INFO, i <= 90 ? 200 : 500, i));
            }
            Report r = Analyzer.analyze(list, 3);
            assertEquals(95L, r.p95Millis());
            assertEquals(90L, r.okVsFailed().get(true));
            assertEquals(3, r.malformed());
        });
        t.test("errors are grouped by hour", () -> {
            Report r = Analyzer.analyze(List.of(
                    entry("2026-03-01T08:10:00Z", Level.ERROR, 500, 10),
                    entry("2026-03-01T08:50:00Z", Level.ERROR, 500, 10),
                    entry("2026-03-01T09:05:00Z", Level.ERROR, 500, 10)), 0);
            assertEquals(2L, r.errorsPerHour().get(Instant.parse("2026-03-01T08:00:00Z")));
            assertEquals(1L, r.errorsPerHour().get(Instant.parse("2026-03-01T09:00:00Z")));
        });
        t.test("generator is deterministic and analyzer counts bad lines", () -> {
            Path a = Files.createTempFile("a", ".log");
            Path b = Files.createTempFile("b", ".log");
            Instant start = Instant.parse("2026-03-01T00:00:00Z");
            LogGenerator.write(a, 500, 7L, start);
            LogGenerator.write(b, 500, 7L, start);
            assertEquals(Files.readString(a), Files.readString(b));
            Report r = Analyzer.analyze(a);
            assertEquals(500, r.total() + r.malformed());
            assertEquals(5, r.malformed());          // lines 97,194,291,388,485
            Files.delete(a);
            Files.delete(b);
        });
        t.test("Level.atLeast follows declaration order", () -> {
```

LogTests.java (continued) JAVA

```
            assertTrue(Level.ERROR.atLeast(Level.WARN), "ERROR >= WARN");
            assertTrue(!Level.DEBUG.atLeast(Level.INFO), "DEBUG < INFO");
        });

        System.exit(t.runAll() == 0 ? 0 : 1);
    }
}
```

Run the tests TERMINAL

```
$ javac -d testout -cp out $(find test -name '*.java')
$ java -cp out:testout com.example.logs.LogTests
PASS parse reads every field
PASS parse rejects garbage and bad timestamps
PASS p95 and failure partition on a hand-made list
PASS errors are grouped by hour
PASS generator is deterministic and analyzer counts bad lines
PASS Level.atLeast follows declaration order
6 tests, 0 failed
```

INTERVIEW ANGLE

How would you compute a top-N of a very large log file that does not fit in memory? Stream the file line by line (`Files.lines`), never collect all lines, and keep only counters. If the number of distinct keys also does not fit, use an approximate structure (count-min sketch) or process in partitions: hash each key to one of K temporary files, count each partition separately, then merge the top-N. For the in-memory case, a size-N min-heap (`PriorityQueue`) beats sorting everything.

REAL WORLD

Real systems rarely analyze log files with hand-written code; they ship structured JSON logs to Elasticsearch, Loki or similar. But the same operations (parse, group by a time bucket, percentile) are exactly what dashboards perform, and log parsing code still appears in migration scripts and incident investigations.

13.3.6 Extension ideas

1. Add command-line options: `--min-level WARN` using `Level.atLeast`, and `--service billing`, applied as stream filters before the statistics.
2. Parse the file in parallel with `lines.parallel()` and compare timings with the sequential version. Is it faster for 2000 lines? For 5 million? Explain why (see the parallel streams discussion in Part 7).
3. Add a "busiest minute" statistic (group by `truncatedTo(MINUTES)`, take the `max` by value).
4. Support a second log format by introducing a `LineFormat` interface with two implementations and choosing by a command-line flag.
5. Print an ASCII bar chart of errors per hour using `"#".repeat(n)`.
6. Detect bursts: report any 5-minute window containing more than 10 errors (sliding window over sorted timestamps).

Check yourself

1. What does this print? `Optional.of(1).stream().count() + Optional.empty().stream().count()`
2. Why is `Pattern` a `static final` field in `LogParser` and not created inside `parse`?
3. What is the difference between `Matcher.matches()` and `Matcher.find()`?
4. What is wrong with `Stream<String> s = Files.lines(p); return s.count();` in a long-running program?
5. In the analyzer, why is `partitioningBy` a better fit than `groupingBy` for the success-versus-failure split?
6. Two entries have the same latency. Why does the "slowest" sort add `.thenComparing(LogEntry::time)`?

Answers

1. `1` (a present Optional streams one element, an empty one streams none; 1 + 0 as longs).
2. Compiling a regex is expensive and `Pattern` is immutable and thread-safe, so one shared instance is correct and faster; only `Matcher` is per-use and not thread-safe.
3. `matches()` requires the entire input to match; `find()` looks for the next match anywhere inside.
4. The stream (and its file handle) is never closed; use try-with-resources.

5. It is for a boolean split and always returns both keys, so there are no missing-key surprises and the intent (two buckets) is clear.
6. `sorted` on ties follows input order in the sequential case, but an explicit tie-breaker makes the result deterministic regardless of how the data was produced.

KEY POINTS
Return `Optional` from parsers of untrusted input and count the empties instead of crashing.

KEY POINTS
Collectors (`groupingBy`, `counting`, `averagingInt`, `partitioningBy`) express reports declaratively; pass a map factory when key order matters.

KEY POINTS
Keep results as data (records) and formatting separate so tests check numbers.

KEY POINTS
Always close `Files.lines` streams, and compile regex `Pattern` objects once.

13.4 Project C: A concurrent order processor

13.4.1 Goal

A shop receives orders and must validate and charge each one. Each charge involves a slow call to a payment service, so one order takes about 10 ms of waiting, not computing. Processing 200 orders one by one takes over two seconds; processing them concurrently should take a fraction of that. This project implements the same job **five ways** and checks that all five agree:

1. sequentially (the baseline),
2. with a hand-built producer-consumer pipeline on a `BlockingQueue`,
3. with an `ExecutorService` thread pool and `Future`s,
4. with `CompletableFuture` stages,
5. with Java 21 virtual threads.

It also shows a graceful shutdown, a deliberate data race, and a test strategy for concurrent code. Read the concurrency Part first (threads, the memory model, executors, CompletableFuture, virtual threads); this chapter applies it.

13.4.2 Design

The work itself is separated from the way it is scheduled. That is the main design idea: `OrderProcessor.process(Order)` knows nothing about threads, and each `Runner` implementation decides how to call it for many orders.

- **model**: `Order` and `Result` records, and `Summary`, an order-independent digest (counts plus a checksum of ids). Concurrent runs finish in unpredictable order, so we cannot compare result lists directly. A summary that ignores order lets us ask "did every runner produce the same outcome?".
- **work**: `OrderGenerator` (seeded) and `OrderProcessor`, which sleeps for the simulated latency and then applies two business rules: reject amounts over 900.00, reject customers whose name ends in 13.
- **run**: the `Runner` interface and five implementations.
- **app**: `Main` times all runners; `ShutdownDemo` and `RaceDemo` illustrate two ideas separately.

NOTE
Using `Thread.sleep` to imitate I/O is a deliberate simplification. A sleeping thread, like a thread blocked on a socket, uses no CPU. That is the situation where more threads than cores help. For CPU-bound work (hashing, image resizing) a pool larger than the core count gains nothing.

13.4.3 The code

Model and work

Order.java JAVA

```java
package com.example.orders.model;

public record Order(int id, String customer, long amountCents) { }
```

Result.java JAVA

```java
package com.example.orders.model;

public record Result(int orderId, Status status, String detail) {
    public enum Status { ACCEPTED, REJECTED }

    public static Result accepted(Order o) {
        return new Result(o.id(), Status.ACCEPTED, "ok");
    }

    public static Result rejected(Order o, String why) {
        return new Result(o.id(), Status.REJECTED, why);
    }
}
```

Summary.java JAVA

```java
package com.example.orders.model;

import java.util.Collection;

/** Order-independent digest of a run: two correct runs must produce equal summaries. */
public record Summary(int accepted, int rejected, long checksum) {
    public static Summary of(Collection<Result> results) {
        int ok = 0;
        int bad = 0;
        long sum = 0;
        for (Result r : results) {
            if (r.status() == Result.Status.ACCEPTED) {
                ok++;
            } else {
                bad++;
            }
            sum += r.orderId() * (r.status() == Result.Status.ACCEPTED ? 1L : 2L);
        }
        return new Summary(ok, bad, sum);
    }
}
```

OrderGenerator.java JAVA

```java
package com.example.orders.work;

import com.example.orders.model.Order;
import java.util.ArrayList;
import java.util.List;
import java.util.Random;

public final class OrderGenerator {
    private OrderGenerator() { }

    public static List<Order> generate(int count, long seed) {
        Random rnd = new Random(seed);
        List<Order> orders = new ArrayList<>(count);
        for (int i = 1; i <= count; i++) {
            orders.add(new Order(i, "cust" + rnd.nextInt(30), 500 + rnd.nextInt(99_000)));
        }
        return orders;
    }
}
```

OrderProcessor.java JAVA

```java
package com.example.orders.work;

import com.example.orders.model.Order;
import com.example.orders.model.Result;

/** The "business logic": pretend to call a payment service (a blocking wait), then decide. */
public final class OrderProcessor {
    private final long latencyMillis;

    public OrderProcessor(long latencyMillis) {
        this.latencyMillis = latencyMillis;
    }

    public Result process(Order o) {
        try {
            Thread.sleep(latencyMillis);                  // stands in for network I/O
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();           // restore the flag, then bail out
            throw new IllegalStateException("interrupted while processing " + o.id(), e);
        }
        if (o.amountCents() > 90_000) {
            return Result.rejected(o, "over limit");
        }
        if (o.customer().endsWith("13")) {
            return Result.rejected(o, "blocked customer");
        }
        return Result.accepted(o);
    }
}
```

The catch block in `process` is the right way to handle `InterruptedException`: restore the flag with `Thread.currentThread().interrupt()` and stop what you were doing. Swallowing it silently makes threads impossible to cancel. Here we convert it into an unchecked exception because the caller (a task in some pool) cannot do anything else with a checked one.

The runner interface and the baseline

Runner.java JAVA

```java
package com.example.orders.run;

import com.example.orders.model.Order;
import com.example.orders.model.Result;
import java.util.List;

public interface Runner {
    String name();

    List<Result> run(List<Order> orders) throws Exception;
}
```

SequentialRunner.java JAVA

```java
package com.example.orders.run;

import com.example.orders.model.Order;
import com.example.orders.model.Result;
import com.example.orders.work.OrderProcessor;
import java.util.ArrayList;
import java.util.List;

public class SequentialRunner implements Runner {
    private final OrderProcessor processor;

    public SequentialRunner(OrderProcessor processor) {
        this.processor = processor;
    }

    @Override
    public String name() {
        return "sequential";
    }
```

SequentialRunner.java (continued) JAVA

```
    @Override
    public List<Result> run(List<Order> orders) {
        List<Result> results = new ArrayList<>();
        for (Order o : orders) {
            results.add(processor.process(o));
        }
        return results;
    }
}
```

Variant 1: producer-consumer with a BlockingQueue

QueueRunner.java JAVA

```java
package com.example.orders.run;

import com.example.orders.model.Order;
import com.example.orders.model.Result;
import com.example.orders.work.OrderProcessor;
import java.util.ArrayList;
import java.util.List;
import java.util.concurrent.ArrayBlockingQueue;
import java.util.concurrent.BlockingQueue;
import java.util.concurrent.ConcurrentLinkedQueue;

/** Classic producer-consumer: one producer thread, N consumer threads, a bounded queue. */
public class QueueRunner implements Runner {
    private static final Order POISON = new Order(-1, "poison", 0);

    private final OrderProcessor processor;
    private final int consumers;
    private final int capacity;

    public QueueRunner(OrderProcessor processor, int consumers, int capacity) {
        this.processor = processor;
        this.consumers = consumers;
        this.capacity = capacity;
    }

    @Override
    public String name() {
        return "queue(" + consumers + " consumers)";
    }

    @Override
    public List<Result> run(List<Order> orders) throws InterruptedException {
        BlockingQueue<Order> queue = new ArrayBlockingQueue<>(capacity);
        ConcurrentLinkedQueue<Result> results = new ConcurrentLinkedQueue<>();

        Thread producer = new Thread(() -> {
            try {
                for (Order o : orders) {
                    queue.put(o);                    // blocks when the queue is full: back-pressure
                }
                for (int i = 0; i < consumers; i++) {
                    queue.put(POISON);               // one pill per consumer
                }
            } catch (InterruptedException e) {
                Thread.currentThread().interrupt();
            }
        }, "producer");

        List<Thread> workers = new ArrayList<>();
        for (int i = 0; i < consumers; i++) {
            Thread w = new Thread(() -> {
                try {
                    while (true) {
                        Order o = queue.take();      // blocks when the queue is empty
                        if (o == POISON) {
                            return;
                        }
                        results.add(processor.process(o));
                    }
                } catch (InterruptedException e) {
                    Thread.currentThread().interrupt();
                }
            }, "consumer-" + i);
            workers.add(w);
        }

        producer.start();
        workers.forEach(Thread::start);
        producer.join();
        for (Thread w : workers) {
            w.join();
```

QueueRunner.java (continued) JAVA

```
        }
        return new ArrayList<>(results);
    }
}
```

What to notice

- The queue is **bounded** (`ArrayBlockingQueue`). When consumers are slower than the producer, `put` blocks the producer. That is back-pressure: memory use stays bounded instead of growing without limit, which is how an unbounded queue turns a slow downstream into an OutOfMemoryError.
- Shutdown uses a **poison pill**: a special sentinel object, one per consumer. Each consumer exits when it takes one. Comparing with `==` is intentional; it is the identity of that exact object that matters.
- Results go into a `ConcurrentLinkedQueue`, a thread-safe collection, so consumers do not need explicit locks.
- `join()` on every thread establishes a happens-before edge: after the joins, the main thread is guaranteed to see everything the workers wrote (see the memory model chapter).
- Threads have names (`consumer-0`). Always name threads you create; stack dumps become readable.

Variant 2: a thread pool

PoolRunner.java JAVA

```java
package com.example.orders.run;

import com.example.orders.model.Order;
import com.example.orders.model.Result;
import com.example.orders.work.OrderProcessor;
import java.util.ArrayList;
import java.util.List;
import java.util.concurrent.Callable;
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;
import java.util.concurrent.Future;
import java.util.concurrent.TimeUnit;

/** A fixed thread pool; invokeAll submits everything and waits for all the futures. */
public class PoolRunner implements Runner {
    private final OrderProcessor processor;
    private final int threads;

    public PoolRunner(OrderProcessor processor, int threads) {
        this.processor = processor;
        this.threads = threads;
    }

    @Override
    public String name() {
        return "pool(" + threads + " threads)";
    }

    @Override
    public List<Result> run(List<Order> orders) throws Exception {
        ExecutorService pool = Executors.newFixedThreadPool(threads);
        try {
            List<Callable<Result>> tasks = new ArrayList<>();
            for (Order o : orders) {
                tasks.add(() -> processor.process(o));
            }
            List<Result> results = new ArrayList<>();
            for (Future<Result> f : pool.invokeAll(tasks)) {
                results.add(f.get());                // rethrows task failures as ExecutionException
            }
            return results;
        } finally {
            shutdownGracefully(pool);
        }
    }

    /** Stop accepting work, wait a while, then interrupt whatever is still running. */
    static void shutdownGracefully(ExecutorService pool) throws InterruptedException {
        pool.shutdown();
        if (!pool.awaitTermination(5, TimeUnit.SECONDS)) {
            pool.shutdownNow();
            pool.awaitTermination(5, TimeUnit.SECONDS);
        }
    }
}
```

`invokeAll` submits every task and blocks until all are done, returning futures in the same order as the tasks. `f.get()` returns the value or throws `ExecutionException` wrapping whatever the task threw. The `finally` block always shuts the pool down, using the standard three steps in `shutdownGracefully`: `shutdown()` stops intake, `awaitTermination` waits, `shutdownNow()` interrupts the stragglers. Forgetting to shut down a fixed pool leaves non-daemon worker threads alive, and the JVM never exits.

Variant 3: CompletableFuture

FutureRunner.java JAVA

```java
package com.example.orders.run;

import com.example.orders.model.Order;
import com.example.orders.model.Result;
import com.example.orders.work.OrderProcessor;
import java.util.List;
import java.util.concurrent.CompletableFuture;
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;

/** One CompletableFuture per order; failures become REJECTED results instead of lost tasks. */
public class FutureRunner implements Runner {
    private final OrderProcessor processor;
    private final int threads;

    public FutureRunner(OrderProcessor processor, int threads) {
        this.processor = processor;
        this.threads = threads;
    }

    @Override
    public String name() {
        return "completable(" + threads + " threads)";
    }

    @Override
    public List<Result> run(List<Order> orders) throws Exception {
        ExecutorService pool = Executors.newFixedThreadPool(threads);
        try {
            List<CompletableFuture<Result>> futures = orders.stream()
                    .map(o -> CompletableFuture
                            .supplyAsync(() -> validate(o), pool)
                            .thenApplyAsync(processor::process, pool)
                            .exceptionally(ex -> Result.rejected(o, "failed: " + ex.getMessage())))
                    .toList();
            CompletableFuture.allOf(futures.toArray(CompletableFuture[]::new)).join();
            return futures.stream().map(CompletableFuture::join).toList();
        } finally {
            PoolRunner.shutdownGracefully(pool);
        }
    }

    private static Order validate(Order o) {
        if (o.amountCents() <= 0) {
            throw new IllegalArgumentException("non-positive amount");
        }
        return o;
    }
}
```

Each order becomes a small pipeline: validate, then process, with `exceptionally` turning any failure into a rejected result. Both stages run on our own pool; passing the executor explicitly avoids silently using the shared `ForkJoinPool.commonPool()`, which is sized for CPU work, not blocking calls. `allOf(...).join()` waits for every future. A realistic extension would add a stage per external call (inventory, payment, notification).

Variant 4: virtual threads

VirtualRunner.java JAVA

```java
package com.example.orders.run;

import com.example.orders.model.Order;
import com.example.orders.model.Result;
import com.example.orders.work.OrderProcessor;
import java.util.ArrayList;
import java.util.List;
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;
import java.util.concurrent.Future;

/** Java 21: one cheap virtual thread per order, no pool size to tune. */
public class VirtualRunner implements Runner {
    private final OrderProcessor processor;

    public VirtualRunner(OrderProcessor processor) {
        this.processor = processor;
    }

    @Override
    public String name() {
        return "virtual threads";
    }

    @Override
    public List<Result> run(List<Order> orders) throws Exception {
        try (ExecutorService exec = Executors.newVirtualThreadPerTaskExecutor()) {
            List<Future<Result>> futures = new ArrayList<>();
            for (Order o : orders) {
                futures.add(exec.submit(() -> processor.process(o)));
            }
            List<Result> results = new ArrayList<>();
            for (Future<Result> f : futures) {
                results.add(f.get());
            }
            return results;
        }     // close() waits for the tasks: shutdown() + awaitTermination()
    }
}
```

Virtual threads (Java 21, see the virtual threads chapter) are so cheap that you create one **per task** and never pool them. When a virtual thread blocks in `sleep` or on a socket, the JVM unmounts it from its carrier thread and runs another one. The try-with-resources block works because `ExecutorService` implements `AutoCloseable` since Java 19; `close()` waits for submitted tasks to finish.

The driver

Main.java JAVA

```java
package com.example.orders.app;

import com.example.orders.model.Order;
import com.example.orders.model.Result;
import com.example.orders.model.Summary;
import com.example.orders.run.FutureRunner;
import com.example.orders.run.PoolRunner;
import com.example.orders.run.QueueRunner;
import com.example.orders.run.Runner;
import com.example.orders.run.SequentialRunner;
import com.example.orders.run.VirtualRunner;
import com.example.orders.work.OrderGenerator;
import com.example.orders.work.OrderProcessor;
import java.util.List;

public class Main {
    public static void main(String[] args) throws Exception {
        int count = args.length > 0 ? Integer.parseInt(args[0]) : 200;
        long latency = args.length > 1 ? Long.parseLong(args[1]) : 10;
        List<Order> orders = OrderGenerator.generate(count, 42L);
        OrderProcessor processor = new OrderProcessor(latency);

        List<Runner> runners = List.of(
                new SequentialRunner(processor),
                new QueueRunner(processor, 8, 16),
                new PoolRunner(processor, 8),
                new FutureRunner(processor, 8),
                new VirtualRunner(processor));

        System.out.printf("%d orders, %d ms simulated latency each%n", count, latency);
        Summary expected = null;
        for (Runner r : runners) {
            long t0 = System.nanoTime();
            List<Result> results = r.run(orders);
            long ms = (System.nanoTime() - t0) / 1_000_000;
            Summary s = Summary.of(results);
            if (expected == null) {
                expected = s;
            }
            System.out.printf("%-24s %5d ms  ok=%d rejected=%d sum=%d %s%n", r.name(), ms,
                    s.accepted(), s.rejected(), s.checksum(),
                    s.equals(expected) ? "(matches baseline)" : "(MISMATCH!)");
        }
    }
}
```

13.4.4 Running it

Compile and run all five variants TERMINAL

```
$ rm -rf out testout
$ javac -d out $(find src -name '*.java')
$ java -cp out com.example.orders.app.Main
200 orders, 10 ms simulated latency each
sequential               2122 ms  ok=172 rejected=28 sum=22627 (matches baseline)
queue(8 consumers)        271 ms  ok=172 rejected=28 sum=22627 (matches baseline)
pool(8 threads)           264 ms  ok=172 rejected=28 sum=22627 (matches baseline)
completable(8 threads)    263 ms  ok=172 rejected=28 sum=22627 (matches baseline)
virtual threads            38 ms  ok=172 rejected=28 sum=22627 (matches baseline)
```

The timings above are typical for the build machine (2 CPU cores) and will differ on yours, but the pattern should be the same.

Variant	About	Why
sequential	N x latency	each order waits for the previous one: 200 x 10 ms is just over 2 s
queue, pool, completable (8 threads)	N / 8 x latency	eight orders wait at once; about 8 times faster, the wait dominates
virtual threads	about one latency	all 200 wait at once; no limit of 8 to hit

Reading the result (timings are typical, not guaranteed).

The last column of every line says "matches baseline": the accepted and rejected counts and the checksum are identical. Without that check, a faster-but-wrong implementation would look like a win.

> **GOTCHA**
> Do not write micro-benchmarks like this and then quote them as facts about Java. A single cold run includes JIT warm-up, thread start cost and noise. We use it to see an order-of-magnitude effect (about 8x and then about 80x), which is large enough to survive the noise. For real measurements use JMH, the benchmark harness from the OpenJDK project.

13.4.5 Graceful shutdown

Real services must stop cleanly: finish what is running, refuse new work, and report what was dropped. `ShutdownDemo` queues 40 tasks of 50 ms on 4 threads, waits a short time, then stops the pool.

ShutdownDemo.java JAVA

```java
package com.example.orders.app;

import com.example.orders.model.Order;
import com.example.orders.work.OrderProcessor;
import java.util.List;
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;
import java.util.concurrent.TimeUnit;
import java.util.concurrent.atomic.AtomicInteger;

/** Shows the three-step shutdown: stop intake, wait politely, then interrupt. */
public class ShutdownDemo {
    public static void main(String[] args) throws InterruptedException {
        OrderProcessor processor = new OrderProcessor(50);
        AtomicInteger done = new AtomicInteger();
        AtomicInteger interrupted = new AtomicInteger();

        ExecutorService pool = Executors.newFixedThreadPool(4);
        for (int i = 1; i <= 40; i++) {
            Order o = new Order(i, "cust" + i, 1000);
            pool.submit(() -> {
                try {
                    processor.process(o);
                    done.incrementAndGet();
                } catch (IllegalStateException e) {
                    interrupted.incrementAndGet();
                }
            });
        }

        Thread.sleep(130);                      // let some work happen, then decide to stop
        pool.shutdown();                        // 1. no new tasks; queued ones still run
        boolean clean = pool.awaitTermination(60, TimeUnit.MILLISECONDS);   // 2. wait politely
        List<Runnable> neverStarted = List.of();
        if (!clean) {
            neverStarted = pool.shutdownNow();   // 3. drain the queue and interrupt workers
            pool.awaitTermination(1, TimeUnit.SECONDS);
        }
        System.out.println("clean shutdown : " + clean);
        System.out.println("completed      : " + done.get());
        System.out.println("interrupted    : " + interrupted.get());
        System.out.println("never started  : " + neverStarted.size());
        int total = done.get() + interrupted.get() + neverStarted.size();
        System.out.println("accounted for  : " + total);
    }
}
```

Run the shutdown demo (typical output) TERMINAL

```
$ java -cp out com.example.orders.app.ShutdownDemo
clean shutdown : false
completed      : 12
interrupted    : 4
never started  : 24
accounted for  : 40
```

The exact split between completed, interrupted and never-started varies from run to run because it depends on timing, but the **accounted for** line must always equal 40: no task disappears silently. `shutdownNow()` returns the queued tasks that never began, so you can log or persist them. Tasks already running get an interrupt; they stop quickly only because `OrderProcessor` reacts to interruption. A task that ignores interrupts (a tight CPU loop that never checks the flag) will keep running regardless.

In a real application you would hook this into JVM shutdown with `Runtime.getRuntime().addShutdownHook(...)`, so that Ctrl+C or a container stop signal triggers the same three steps.

13.4.6 Why shared counters are dangerous

The processors above never share mutable counters. Here is what happens when you do. Four threads each increment four counters 250,000 times.

RaceDemo.java JAVA

```java
package com.example.orders.app;

import java.util.concurrent.atomic.AtomicInteger;
import java.util.concurrent.atomic.LongAdder;

/** Why shared counters need care: 4 threads x 250,000 increments should give 1,000,000. */
public class RaceDemo {
    static int plain;
    static volatile int volatileCounter;
    static final AtomicInteger atomic = new AtomicInteger();
    static final LongAdder adder = new LongAdder();

    public static void main(String[] args) throws InterruptedException {
        Thread[] threads = new Thread[4];
        for (int t = 0; t < threads.length; t++) {
            threads[t] = new Thread(() -> {
                for (int i = 0; i < 250_000; i++) {
                    plain++;                  // read, add, write: three steps, not atomic
                    volatileCounter++;        // visible, but still three steps
                    atomic.incrementAndGet(); // one atomic hardware operation
                    adder.increment();        // striped cells, merged on sum()
                }
            });
            threads[t].start();
        }
        for (Thread t : threads) {
            t.join();
        }
        System.out.println("plain int       : " + plain);
        System.out.println("volatile int    : " + volatileCounter);
        System.out.println("AtomicInteger   : " + atomic.get());
        System.out.println("LongAdder       : " + adder.sum());
    }
}
```

Run the race demo (typical output; plain and volatile vary) TERMINAL

```
$ java -cp out com.example.orders.app.RaceDemo
plain int       : 967756
volatile int    : 968756
AtomicInteger   : 1000000
LongAdder       : 1000000
```

`plain++` is read, add one, write: three steps. Two threads can read the same value and both write back the same result, so updates are **lost**. `volatile` guarantees visibility of the latest write, but does not make the three steps atomic, so it loses updates too. `AtomicInteger` uses a compare-and-set instruction and is exact. `LongAdder` keeps several cells to reduce contention and sums them on demand; it is the best choice for a hot statistics counter that is written often and read rarely. If your run happened to print exactly 1,000,000 for the first two, that is luck, not correctness; the bug is still there.

13.4.7 Tests

Concurrent code is tested by checking **properties that must hold on every schedule**, not exact interleavings: all runners agree with the baseline, no id is lost or duplicated, and the system terminates. Tests use a 1 ms latency to stay fast. The queue test with capacity 1 and 16 consumers is a stress shape: a tiny queue and many consumers force constant blocking and wake-ups, which is where shutdown bugs show up.

OrderTests.java JAVA

```java
package com.example.orders;

import static com.example.orders.MiniTest.assertEquals;
import static com.example.orders.MiniTest.assertThrows;
import static com.example.orders.MiniTest.assertTrue;

import com.example.orders.model.Order;
import com.example.orders.model.Result;
import com.example.orders.model.Summary;
import com.example.orders.run.FutureRunner;
import com.example.orders.run.PoolRunner;
import com.example.orders.run.QueueRunner;
import com.example.orders.run.Runner;
import com.example.orders.run.SequentialRunner;
import com.example.orders.run.VirtualRunner;
import com.example.orders.work.OrderGenerator;
import com.example.orders.work.OrderProcessor;
import java.util.List;
import java.util.Set;
import java.util.stream.Collectors;

public class OrderTests {
    static final OrderProcessor FAST = new OrderProcessor(1);

    public static void main(String[] args) {
        MiniTest t = new MiniTest();
        List<Order> orders = OrderGenerator.generate(120, 7L);
        Summary baseline = Summary.of(new SequentialRunner(FAST).run(orders));

        t.test("business rules: limit and blocked customer", () -> {
            assertEquals(Result.Status.REJECTED,
                    FAST.process(new Order(1, "cust1", 95_000)).status());
            assertEquals(Result.Status.REJECTED,
                    FAST.process(new Order(2, "cust13", 100)).status());
            assertEquals(Result.Status.ACCEPTED,
                    FAST.process(new Order(3, "cust2", 100)).status());
        });
        t.test("generator is deterministic", () ->
                assertEquals(OrderGenerator.generate(50, 1L), OrderGenerator.generate(50, 1L)));

        List<Runner> runners = List.of(new QueueRunner(FAST, 4, 8), new PoolRunner(FAST, 4),
                new FutureRunner(FAST, 4), new VirtualRunner(FAST));
        for (Runner r : runners) {
            t.test(r.name() + " matches the sequential baseline", () -> {
                List<Result> results = r.run(orders);
                assertEquals(baseline, Summary.of(results));
                Set<Integer> ids = results.stream().map(Result::orderId)
                        .collect(Collectors.toSet());
                assertEquals(orders.size(), ids.size());     // no order lost or duplicated
            });
        }
        t.test("queue runner copes with a tiny queue and many consumers", () ->
                assertEquals(baseline, Summary.of(new QueueRunner(FAST, 16, 1).run(orders))));
        t.test("interrupt makes the processor fail fast and keeps the flag", () -> {
            Thread.currentThread().interrupt();
            assertThrows(IllegalStateException.class, () -> new OrderProcessor(50)
                    .process(new Order(1, "c", 1)));
            assertTrue(Thread.interrupted(), "interrupt flag was lost");   // also clears it
        });

        System.exit(t.runAll() == 0 ? 0 : 1);
    }
}
```

Run the tests TERMINAL

```
$ javac -d testout -cp out $(find test -name '*.java')
$ java -cp out:testout com.example.orders.OrderTests
PASS business rules: limit and blocked customer
PASS generator is deterministic
PASS queue(4 consumers) matches the sequential baseline
PASS pool(4 threads) matches the sequential baseline
PASS completable(4 threads) matches the sequential baseline
PASS virtual threads matches the sequential baseline
PASS queue runner copes with a tiny queue and many consumers
PASS interrupt makes the processor fail fast and keeps the flag
8 tests, 0 failed
```

WATCH OUT

A passing concurrency test is weak evidence. Races depend on timing and may fail one run in a thousand. Run such tests many times (a loop in the shell is enough), keep thread counts above the CPU count to provoke interleavings, and prefer designs that are correct by construction (no shared mutable state, immutable messages, thread-safe collections) over designs you have to test into confidence.

INTERVIEW ANGLE

When would you pick a platform-thread pool, and when virtual threads? Virtual threads suit many concurrent tasks that mostly block on I/O (HTTP calls, database queries): create one per task and do not pool them. A bounded pool of platform threads suits CPU-bound work (size about the number of cores) or when you must cap concurrency against a limited resource such as a database. Virtual threads do not make computation faster, and a semaphore is the tool for limiting them, not a small pool.

REAL WORLD

Spring Boot 3.2+ can run request handling on virtual threads with one property (`spring.threads.virtual.enabled=true`). The service code stays blocking and simple, which is the appeal: you get thread-per-request readability with close to reactive scalability for I/O-bound services.

13.4.8 Extension ideas

1. Add a `StructuredTaskScope` variant (preview in Java 21 and later releases; run with `--enable-preview`) and compare error handling.
2. Add retries: an order whose charge "times out" (random failure) is retried up to three times with a delay; count retries in a `LongAdder`.
3. Limit in-flight payments to 20 with a `Semaphore` in the virtual-thread variant.
4. Add a priority: premium customers' orders jump the queue using `PriorityBlockingQueue`.
5. Make the consumer count a command-line argument and plot (by hand) time against 1, 2, 4, 8, 16, 32 threads. Where does it stop improving, and why?
6. Replace `ConcurrentLinkedQueue` for results with a per-thread list merged at the end, and think about which is faster and why.

Check yourself

1. What does `ExecutorService.shutdown()` do to tasks already queued? And `shutdownNow()`?
2. Why does the producer put one poison pill per consumer instead of just one?
3. What is wrong with this handler, and what should it do? `catch (InterruptedException e) { }`
4. Four threads each run `count++` on a shared `volatile int` 1000 times. Can the final value be less than 4000? Why?
5. Why pass an explicit executor to `supplyAsync` for blocking work?
6. Why is the queue in `QueueRunner` bounded?

Answers

1. `shutdown()` lets queued and running tasks finish but accepts no new ones. `shutdownNow()` drops the queue (returning those tasks), and interrupts running tasks.
2. A consumer exits after taking one pill and never takes another. With N consumers, one pill would stop only one of them and the rest would block on `take` forever.

3. It swallows the interrupt, so the thread cannot be cancelled. Restore the flag with `Thread.currentThread().interrupt()` or propagate the exception, and stop the work.
4. Yes. `count++` is read-modify-write; `volatile` ensures visibility but not atomicity, so concurrent increments can be lost (use `AtomicInteger` or `LongAdder`).
5. The default common pool has about as many threads as cores; blocking tasks would occupy them and starve other CompletableFuture work in the JVM.
6. To apply back-pressure: a fast producer blocks instead of filling memory with queued orders.

KEY POINTS
Separate the work from the scheduling; compare every concurrent version against a sequential baseline using an order-independent summary.

KEY POINTS
Bounded queues give back-pressure; poison pills (one per consumer) give a clean stop; always join or close to get happens-before.

KEY POINTS
Shut down in three steps: `shutdown`, `awaitTermination`, `shutdownNow`, and make tasks respond to interruption.

KEY POINTS
Blocking I/O workloads gain from many threads; virtual threads make "one thread per task" affordable. Shared counters need atomics.

13.5 Project D: An in-memory key-value store

13.5.1 Goal

Redis, Memcached and the caches inside every web framework are, at heart, a map with rules: values can expire, memory is bounded, and many threads use it at once. This project builds a small version and exposes it through a text shell:

Shell commands

```
SET token abc123 100     store a value that expires after 100 ms
GET token                read it, or (nil)
INCR visits              atomic counter
DEL / SIZE / KEYS / PURGE
```

You will build **three** store implementations behind one interface, so you can compare designs side by side:

1. `LockedStore`: a `HashMap` protected by a `ReentrantReadWriteLock`.
2. `ConcurrentStore`: a `ConcurrentHashMap` with no explicit locks.
3. `LruStore`: a size-bounded cache using `LinkedHashMap` in access order.

A background `ExpiryCleaner` built on `ScheduledExecutorService` removes dead entries, and the same contract tests run against every implementation.

13.5.2 Design

- **Interface first.** `KeyValueStore` lists the operations: `put` (with optional TTL, time to live), `get`, `delete`, `incr`, `size`, `keys`, `purgeExpired`. Every implementation must obey the same contract; the tests enforce it.
- **Entries are immutable records.** `Entry(value, expiresAtMillis)`. To change a value you replace the entry. Immutable entries are safe to share between threads without locking.
- **Time is injected** as a `LongSupplier` of milliseconds. Production passes `System::currentTimeMillis`; tests pass a clock they control and call `advance(100)` instead of sleeping. Tests that sleep are slow and flaky.
- **Two kinds of expiry.** *Lazy*: `get` treats an expired entry as missing. *Active*: the cleaner periodically calls `purgeExpired`. Lazy expiry alone would leave dead entries in memory forever if nobody reads them; active expiry alone would return stale data between sweeps. Real caches (Redis included) use both.

- **Strings only.** Values are strings without spaces so the shell parser stays tiny. A real store would use bytes and a proper protocol.

13.5.3 The code

Entry and the interface

Entry.java JAVA

```java
package com.example.kv.store;

/** An immutable stored value. expiresAtMillis == 0 means "never expires". */
public record Entry(String value, long expiresAtMillis) {
    public boolean expired(long nowMillis) {
        return expiresAtMillis != 0 && nowMillis >= expiresAtMillis;
    }

    public static Entry of(String value, long ttlMillis, long nowMillis) {
        return new Entry(value, ttlMillis <= 0 ? 0 : nowMillis + ttlMillis);
    }
}
```

KeyValueStore.java JAVA

```java
package com.example.kv.store;

import java.util.List;
import java.util.Optional;

public interface KeyValueStore {
    /** Store a value that never expires. */
    default void put(String key, String value) {
        put(key, value, 0);
    }

    /** Store a value that expires ttlMillis from now (ttlMillis <= 0 means never). */
    void put(String key, String value, long ttlMillis);

    Optional<String> get(String key);

    /** Returns true if a live entry existed. */
    boolean delete(String key);

    /** Atomically add one to a numeric value (a missing key counts as 0); returns the result. */
    long incr(String key);

    /** Number of live (not expired) entries. */
    int size();

    List<String> keys();

    /** Physically remove expired entries; returns how many were removed. */
    int purgeExpired();
}
```

An expiry time of `0` means "never". The check `nowMillis >= expiresAtMillis` means an entry is dead **at** its expiry instant, not after it; a test pins that boundary down.

Version 1: HashMap plus ReadWriteLock

LockedStore.java JAVA

```java
package com.example.kv.store;

import java.util.ArrayList;
import java.util.HashMap;
import java.util.List;
import java.util.Map;
import java.util.Optional;
import java.util.concurrent.locks.ReadWriteLock;
import java.util.concurrent.locks.ReentrantReadWriteLock;
import java.util.function.LongSupplier;

/** Version 1: a plain HashMap guarded by a read-write lock. Many readers OR one writer. */
public class LockedStore implements KeyValueStore {
    private final Map<String, Entry> map = new HashMap<>();
    private final ReadWriteLock lock = new ReentrantReadWriteLock();
    private final LongSupplier clock;

    public LockedStore(LongSupplier clock) {
        this.clock = clock;
    }

    @Override
    public void put(String key, String value, long ttlMillis) {
        lock.writeLock().lock();
        try {
            map.put(key, Entry.of(value, ttlMillis, clock.getAsLong()));
        } finally {
            lock.writeLock().unlock();
        }
    }

    @Override
    public Optional<String> get(String key) {
        lock.readLock().lock();
        try {
            Entry e = map.get(key);
            // Expired entries are only hidden here; removing needs the write lock (see purge).
            return e == null || e.expired(clock.getAsLong())
                    ? Optional.empty() : Optional.of(e.value());
        } finally {
            lock.readLock().unlock();
        }
    }

    @Override
    public boolean delete(String key) {
        lock.writeLock().lock();
        try {
            Entry e = map.remove(key);
            return e != null && !e.expired(clock.getAsLong());
        } finally {
            lock.writeLock().unlock();
        }
    }

    @Override
    public long incr(String key) {
        lock.writeLock().lock();
        try {
            long now = clock.getAsLong();
            Entry old = map.get(key);
            boolean live = old != null && !old.expired(now);
            long next = (live ? Long.parseLong(old.value()) : 0) + 1;
            map.put(key, new Entry(Long.toString(next), live ? old.expiresAtMillis() : 0));
            return next;
        } finally {
            lock.writeLock().unlock();
        }
    }

    @Override
    public int size() {
        lock.readLock().lock();
```

LockedStore.java (continued) JAVA

```
        try {
            long now = clock.getAsLong();
            return (int) map.values().stream().filter(e -> !e.expired(now)).count();
        } finally {
            lock.readLock().unlock();
        }
    }

    @Override
    public List<String> keys() {
        lock.readLock().lock();
        try {
            long now = clock.getAsLong();
            List<String> keys = new ArrayList<>();
            map.forEach((k, e) -> {
                if (!e.expired(now)) {
                    keys.add(k);
                }
            });
            keys.sort(null);
            return keys;
        } finally {
            lock.readLock().unlock();
        }
    }

    @Override
    public int purgeExpired() {
        lock.writeLock().lock();
        try {
            int before = map.size();
            long now = clock.getAsLong();
            map.values().removeIf(e -> e.expired(now));
            return before - map.size();
        } finally {
            lock.writeLock().unlock();
        }
    }
}
```

What to notice

- A `ReadWriteLock` allows **many readers at once or one writer alone**. For read-heavy workloads this beats a single `synchronized` because readers do not block each other. `get`, `size` and `keys` take the read lock; everything that changes the map takes the write lock.
- Every `lock()` is paired with `unlock()` in a `finally` block. If an exception escapes between them, the lock must still be released, or the next caller blocks forever.
- `get` does **not** delete an expired entry. Removing needs the write lock, and a read lock cannot be upgraded to a write lock: trying to take the write lock while holding the read lock **deadlocks** the thread against itself. So the read path only hides dead entries, and `purgeExpired` (write lock) removes them.
- `incr` is read-modify-write. It must happen under one write lock; two separate calls "get, then put" would lose updates.

GOTCHA

`ReentrantReadWriteLock` is not always faster than a plain lock. Taking a read lock still updates shared state, so with very short critical sections (like a `HashMap.get`) the lock bookkeeping costs more than the work it protects. It pays off when the protected work is longer, or reads vastly outnumber writes.

Version 2: ConcurrentHashMap

ConcurrentStore.java JAVA

```java
package com.example.kv.store;

import java.util.List;
import java.util.Map;
import java.util.Optional;
import java.util.concurrent.ConcurrentHashMap;
import java.util.function.LongSupplier;

/** Version 2: no explicit locks; ConcurrentHashMap makes each single-key operation atomic. */
public class ConcurrentStore implements KeyValueStore {
    private final ConcurrentHashMap<String, Entry> map = new ConcurrentHashMap<>();
    private final LongSupplier clock;

    public ConcurrentStore(LongSupplier clock) {
        this.clock = clock;
    }

    @Override
    public void put(String key, String value, long ttlMillis) {
        map.put(key, Entry.of(value, ttlMillis, clock.getAsLong()));
    }

    @Override
    public Optional<String> get(String key) {
        Entry e = map.get(key);
        if (e == null) {
            return Optional.empty();
        }
        if (e.expired(clock.getAsLong())) {
            map.remove(key, e);          // only if still the same entry: a fresh put survives
            return Optional.empty();
        }
        return Optional.of(e.value());
    }

    @Override
    public boolean delete(String key) {
        Entry e = map.remove(key);
        return e != null && !e.expired(clock.getAsLong());
    }

    @Override
    public long incr(String key) {
        long now = clock.getAsLong();
        long[] result = new long[1];
        map.compute(key, (k, old) -> {           // runs atomically for this key
            boolean live = old != null && !old.expired(now);
            result[0] = (live ? Long.parseLong(old.value()) : 0) + 1;
            return new Entry(Long.toString(result[0]), live ? old.expiresAtMillis() : 0);
        });
        return result[0];
    }

    @Override
    public int size() {
        long now = clock.getAsLong();
        return (int) map.values().stream().filter(e -> !e.expired(now)).count();
    }

    @Override
    public List<String> keys() {
        long now = clock.getAsLong();
        return map.entrySet().stream().filter(e -> !e.getValue().expired(now))
                .map(Map.Entry::getKey).sorted().toList();
    }

    @Override
    public int purgeExpired() {
        long now = clock.getAsLong();
        int removed = 0;
        for (Map.Entry<String, Entry> e : map.entrySet()) {
            if (e.getValue().expired(now) && map.remove(e.getKey(), e.getValue())) {
                removed++;
```

ConcurrentStore.java (continued) JAVA

```
            }
        }
        return removed;
    }
}
```

What to notice

- There are no locks in our code. `ConcurrentHashMap` makes each single-key operation (`get`, `put`, `remove`, `compute`) atomic internally, using compare-and-swap for empty buckets and short per-bucket locking otherwise (see the concurrent collections chapter).
- `get` can safely remove an expired entry using the **two-argument** `map.remove(key, e)`, which removes only if the key is still mapped to that exact entry. Between our `get` and our `remove`, another thread may have stored a fresh value; the conditional remove leaves it alone. This is the check-then-act race solved with one atomic call.
- `incr` uses `compute`, which runs our lambda atomically for that key. The lambda must be short and must not touch the same map. The one-element array `result` is the usual trick to get a value out of a lambda.
- `purgeExpired` iterates while other threads modify the map. That is allowed: `ConcurrentHashMap` iterators are *weakly consistent*, never throw `ConcurrentModificationException`, and may or may not see concurrent changes.
- `size()` here counts live entries, so it is O(n). The built-in `map.size()` would be cheap but would also count expired entries. Production stores keep counters; we keep the code simple.

WATCH OUT

The classic bug is `if (!map.containsKey(k)) map.put(k, v);` on a concurrent map. Each call is atomic but the pair is not: two threads can both pass the check. Use `putIfAbsent`, `computeIfAbsent` or `compute`, which do the check and the write as one operation.

Version 3: a bounded LRU cache

An **LRU** (least recently used) cache holds at most N entries and, when full, evicts the one that has gone unused the longest. `LinkedHashMap` has this built in: constructed with `accessOrder = true`, it moves each accessed entry to the end of its internal linked list, and overriding `removeEldestEntry` lets you evict the head after each insert.

LruStore.java JAVA

```
package com.example.kv.store;

import java.util.ArrayList;
import java.util.LinkedHashMap;
import java.util.List;
import java.util.Map;
import java.util.Optional;
import java.util.function.LongSupplier;

/**
 * Version 3: a bounded cache. LinkedHashMap in access order moves every entry we read to the
 * "newest" end, so the eldest entry is the least recently used. Because even get() changes the
 * order, reads are writes: a read-write lock would be wrong here, so we use plain synchronized.
 */
public class LruStore implements KeyValueStore {
    private final LongSupplier clock;
    private final int capacity;
    private long evictions;
    private final Map<String, Entry> map;

    public LruStore(int capacity, LongSupplier clock) {
        this.capacity = capacity;
        this.clock = clock;
        this.map = new LinkedHashMap<>(16, 0.75f, true) {
            @Override
            protected boolean removeEldestEntry(Map.Entry<String, Entry> eldest) {
                boolean evict = size() > LruStore.this.capacity;
                if (evict) {
                    evictions++;
                }
                return evict;
            }
        };
```

LruStore.java (continued) JAVA

```
    }

    public synchronized long evictions() {
        return evictions;
    }

    @Override
    public synchronized void put(String key, String value, long ttlMillis) {
        map.put(key, Entry.of(value, ttlMillis, clock.getAsLong()));
    }

    @Override
    public synchronized Optional<String> get(String key) {
        Entry e = map.get(key);                // this call also marks the key as recently used
        if (e == null) {
            return Optional.empty();
        }
        if (e.expired(clock.getAsLong())) {
            map.remove(key);
            return Optional.empty();
        }
        return Optional.of(e.value());
    }

    @Override
    public synchronized boolean delete(String key) {
        Entry e = map.remove(key);
        return e != null && !e.expired(clock.getAsLong());
    }

    @Override
    public synchronized long incr(String key) {
        long now = clock.getAsLong();
        Entry old = map.get(key);
        boolean live = old != null && !old.expired(now);
        long next = (live ? Long.parseLong(old.value()) : 0) + 1;
        map.put(key, new Entry(Long.toString(next), live ? old.expiresAtMillis() : 0));
        return next;
    }

    @Override
    public synchronized int size() {
        long now = clock.getAsLong();
        return (int) map.values().stream().filter(e -> !e.expired(now)).count();
    }

    @Override
    public synchronized List<String> keys() {
        long now = clock.getAsLong();
        List<String> keys = new ArrayList<>();
        map.forEach((k, e) -> {
            if (!e.expired(now)) {
                keys.add(k);
            }
        });
        keys.sort(null);
        return keys;
    }

    @Override
    public synchronized int purgeExpired() {
        int before = map.size();
        long now = clock.getAsLong();
        map.values().removeIf(e -> e.expired(now));
        return before - map.size();
    }
}
```

Here the lesson is the opposite of version 1: with access ordering, **even `get` modifies the map** (it relinks a node). A read lock would let several threads reorder the list at once and corrupt it. So every method is `synchronized`. If you need an LRU under heavy contention, real libraries (Caffeine) use sophisticated buffered designs; for most programs a synchronized `LinkedHashMap` is the right size of solution.

The anonymous subclass of `LinkedHashMap` also demonstrates an inner class reading the outer instance's field (`LruStore.this.capacity`) and its own state, which you met in the nested classes chapter.

The background cleaner

ExpiryCleaner.java JAVA

```java
package com.example.kv.store;

import java.util.concurrent.Executors;
import java.util.concurrent.ScheduledExecutorService;
import java.util.concurrent.TimeUnit;
import java.util.concurrent.atomic.AtomicLong;

/** Background sweeper: calls purgeExpired() on a fixed delay until closed. */
public class ExpiryCleaner implements AutoCloseable {
    private final ScheduledExecutorService scheduler;
    private final AtomicLong totalRemoved = new AtomicLong();

    public ExpiryCleaner(KeyValueStore store, long periodMillis) {
        this.scheduler = Executors.newSingleThreadScheduledExecutor(r -> {
            Thread t = new Thread(r, "kv-expiry-cleaner");
            t.setDaemon(true);                    // never keeps the JVM alive by itself
            return t;
        });
        scheduler.scheduleWithFixedDelay(() -> {
            try {
                totalRemoved.addAndGet(store.purgeExpired());
            } catch (RuntimeException e) {        // an uncaught exception would cancel the schedule
                System.err.println("cleaner error: " + e);
            }
        }, periodMillis, periodMillis, TimeUnit.MILLISECONDS);
    }

    public long totalRemoved() {
        return totalRemoved.get();
    }

    @Override
    public void close() throws InterruptedException {
        scheduler.shutdown();
        scheduler.awaitTermination(1, TimeUnit.SECONDS);
    }
}
```

Points to study: the thread factory names the thread and marks it **daemon**, so a forgotten cleaner cannot keep the program alive. `scheduleWithFixedDelay` waits the period *after each run finishes*, so runs never overlap or pile up (`scheduleAtFixedRate` would try to catch up). The `try/catch` inside the task is essential: if a scheduled task throws, the executor silently cancels all future runs of it. The class implements `AutoCloseable` so try-with-resources shuts it down.

The command shell

Shell.java JAVA

```java
package com.example.kv.shell;

import com.example.kv.store.KeyValueStore;
import java.util.Locale;
import java.util.Optional;

/** Parses one command line and returns the reply text. Redis-flavoured on purpose. */
public class Shell {
    private final KeyValueStore store;

    public Shell(KeyValueStore store) {
        this.store = store;
    }

    public String execute(String line) {
        String[] w = line.trim().split("\\s+");
        if (w.length == 0 || w[0].isEmpty()) {
            return "";
        }
        try {
            return switch (w[0].toUpperCase(Locale.ROOT)) {
                case "SET" -> set(w);
                case "GET" -> {
                    need(w, 2, "GET key");
                    Optional<String> v = store.get(w[1]);
                    yield v.map(s -> "\"" + s + "\"").orElse("(nil)");
                }
                case "DEL" -> {
                    need(w, 2, "DEL key");
                    yield "(integer) " + (store.delete(w[1]) ? 1 : 0);
                }
                case "INCR" -> {
                    need(w, 2, "INCR key");
                    yield "(integer) " + store.incr(w[1]);
                }
                case "SIZE" -> "(integer) " + store.size();
                case "KEYS" -> store.keys().isEmpty() ? "(empty)" : String.join(" ", store.keys());
                case "PURGE" -> "(integer) " + store.purgeExpired();
                case "SLEEP" -> {                    // demo helper: let real time pass
                    need(w, 2, "SLEEP millis");
                    Thread.sleep(Long.parseLong(w[1]));
                    yield "OK";
                }
                default -> "ERR unknown command '" + w[0] + "'";
            };
        } catch (IllegalArgumentException e) {
            return "ERR " + e.getMessage();
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
            return "ERR interrupted";
        }
    }

    private String set(String[] w) {
        if (w.length != 3 && w.length != 4) {
            throw new IllegalArgumentException("usage: SET key value [ttlMillis]");
        }
        long ttl = w.length == 4 ? Long.parseLong(w[3]) : 0;
        store.put(w[1], w[2], ttl);
        return "OK";
    }

    private static void need(String[] w, int count, String usage) {
        if (w.length != count) {
            throw new IllegalArgumentException("usage: " + usage);
        }
    }
}
```

Main.java JAVA

```java
package com.example.kv.shell;

import com.example.kv.store.ConcurrentStore;
import com.example.kv.store.ExpiryCleaner;
import com.example.kv.store.KeyValueStore;
import com.example.kv.store.LockedStore;
import com.example.kv.store.LruStore;
import java.util.Scanner;

public class Main {
    public static void main(String[] args) throws Exception {
        String kind = args.length > 0 ? args[0] : "concurrent";
        KeyValueStore store = switch (kind) {
            case "locked" -> new LockedStore(System::currentTimeMillis);
            case "lru" -> new LruStore(3, System::currentTimeMillis);
            default -> new ConcurrentStore(System::currentTimeMillis);
        };
        System.out.println("store: " + store.getClass().getSimpleName());
        Shell shell = new Shell(store);
        try (ExpiryCleaner cleaner = new ExpiryCleaner(store, 50);
             Scanner in = new Scanner(System.in)) {
            while (in.hasNextLine()) {
                String line = in.nextLine();
                if (line.isBlank()) {
                    continue;
                }
                if (line.trim().equalsIgnoreCase("QUIT")) {
                    break;
                }
                System.out.println("> " + line);
                System.out.println(shell.execute(line));
            }
            System.out.println("cleaner removed " + cleaner.totalRemoved() + " expired entries");
        }
    }
}
```

The shell returns a string for each command instead of printing, so it can be tested. The `switch` expression with `yield` gives each branch a value. `Locale.ROOT` in `toUpperCase` avoids the Turkish-i problem described in the Unicode chapter (a command `get` must map to `GET` on every machine). Errors from bad input come back as `ERR ...` text, never as exceptions that kill the loop. `Main` picks the implementation from a command-line argument, which is the strategy pattern in its simplest form.

13.5.4 Running it

Save the session below as `session.txt`. The `SLEEP` command makes the shell pause so a 100 ms entry really expires and the cleaner runs.

session.txt

```
SET name asha
GET name
SET token abc123 100
GET token
INCR visits
INCR visits
GET visits
SIZE
SLEEP 250
GET token
SIZE
DEL name
DEL name
GET name
SET a
FLY away
KEYS
QUIT
```

Compile and run the default (ConcurrentStore) TERMINAL

```
$ rm -rf out testout
$ javac -d out $(find src -name '*.java')
$ java -cp out com.example.kv.shell.Main < session.txt
store: ConcurrentStore
> SET name asha
OK
> GET name
"asha"
> SET token abc123 100
OK
> GET token
"abc123"
> INCR visits
(integer) 1
> INCR visits
(integer) 2
> GET visits
"2"
> SIZE
(integer) 3
> SLEEP 250
OK
> GET token
(nil)
> SIZE
(integer) 2
> DEL name
(integer) 1
> DEL name
(integer) 0
> GET name
(nil)
> SET a
ERR usage: SET key value [ttlMillis]
> FLY away
ERR unknown command 'FLY'
> KEYS
visits
cleaner removed 1 expired entries
```

The SIZE before the sleep is 3 (name, token, visits); after, 2, because `token` expired. The final line shows the cleaner removed one entry. Whether the cleaner or the lazy `GET` removes an expired entry first depends on timing, so on your machine that count can be 0 or 1; the replies to the commands do not change either way.

Now the LRU variant with capacity 3. We touch `a` so that `b` becomes the eldest, then insert a fourth key:

lru.txt

```
SET a 1
SET b 2
SET c 3
GET a
SET d 4
KEYS
QUIT
```

LRU eviction in action TERMINAL

```
$ java -cp out com.example.kv.shell.Main lru < lru.txt
store: LruStore
> SET a 1
OK
> SET b 2
OK
> SET c 3
OK
> GET a
"1"
> SET d 4
OK
> KEYS
a c d
cleaner removed 0 expired entries
```

`b` is gone, although it was inserted before `c`; `a` survived because reading it counted as use.

13.5.5 Tests

The tests have three layers. First, **contract tests** run in a loop over a list of factories, so each behaviour is asserted for all three stores with no copy-paste. Second, tests of behaviour unique to one class (LRU order, the cleaner thread). Third, the shell's parsing and error replies. A `ManualTime` clock makes expiry tests instant and exact.

KvTests.java JAVA

```
package com.example.kv;

import static com.example.kv.MiniTest.assertEquals;
import static com.example.kv.MiniTest.assertThrows;
import static com.example.kv.MiniTest.assertTrue;

import com.example.kv.shell.Shell;
import com.example.kv.store.ConcurrentStore;
import com.example.kv.store.ExpiryCleaner;
import com.example.kv.store.KeyValueStore;
import com.example.kv.store.LockedStore;
import com.example.kv.store.LruStore;
import java.util.List;
import java.util.Optional;
import java.util.concurrent.atomic.AtomicLong;
import java.util.function.Function;
import java.util.function.LongSupplier;

public class KvTests {
    /** A clock we control: tests move time forward by hand instead of sleeping. */
    static final class ManualTime implements LongSupplier {
        final AtomicLong now = new AtomicLong(1_000);

        @Override
        public long getAsLong() {
            return now.get();
        }

        void advance(long millis) {
            now.addAndGet(millis);
        }
    }

    public static void main(String[] args) {
        MiniTest t = new MiniTest();

        // The same contract tests run against every implementation.
        List<Function<LongSupplier, KeyValueStore>> factories = List.of(
                LockedStore::new, ConcurrentStore::new, clock -> new LruStore(100, clock));
        for (var factory : factories) {
            String n = factory.apply(() -> 0).getClass().getSimpleName();

            t.test(n + ": put, get, overwrite, delete", () -> {
                KeyValueStore s = factory.apply(new ManualTime());
                s.put("k", "v1");
                s.put("k", "v2");
                assertEquals(Optional.of("v2"), s.get("k"));
                assertTrue(s.delete("k"), "delete should report true");
                assertTrue(!s.delete("k"), "second delete should report false");
                assertEquals(Optional.empty(), s.get("k"));
            });
            t.test(n + ": entries expire after their ttl", () -> {
                ManualTime time = new ManualTime();
                KeyValueStore s = factory.apply(time);
                s.put("short", "x", 100);
                s.put("forever", "y");
                time.advance(99);
                assertEquals(Optional.of("x"), s.get("short"));
                time.advance(1);                                  // exactly at expiry: gone
                assertEquals(Optional.empty(), s.get("short"));
                assertEquals(1, s.size());
                assertEquals(List.of("forever"), s.keys());
            });
            t.test(n + ": purgeExpired removes only expired entries", () -> {
                ManualTime time = new ManualTime();
```

KvTests.java (continued) JAVA

```
                KeyValueStore s = factory.apply(time);
                s.put("a", "1", 10);
                s.put("b", "2", 10);
                s.put("c", "3", 1000);
                time.advance(50);
                assertEquals(2, s.purgeExpired());
                assertEquals(0, s.purgeExpired());
                assertEquals(List.of("c"), s.keys());
            });
            t.test(n + ": incr is atomic across 4 threads", () -> {
                KeyValueStore s = factory.apply(new ManualTime());
                Thread[] ts = new Thread[4];
                for (int i = 0; i < ts.length; i++) {
                    ts[i] = new Thread(() -> {
                        for (int j = 0; j < 10_000; j++) {
                            s.incr("counter");
                        }
                    });
                    ts[i].start();
                }
                for (Thread th : ts) {
                    th.join();
                }
                assertEquals(Optional.of("40000"), s.get("counter"));
            });
        }

        t.test("LRU evicts the least recently used entry", () -> {
            LruStore s = new LruStore(3, new ManualTime());
            s.put("a", "1");
            s.put("b", "2");
            s.put("c", "3");
            s.get("a");                       // a is now the newest; b is the eldest
            s.put("d", "4");
            assertEquals(List.of("a", "c", "d"), s.keys());
            assertEquals(1L, s.evictions());
        });
        t.test("cleaner thread removes expired entries on its own", () -> {
            KeyValueStore s = new LockedStore(System::currentTimeMillis);
            try (ExpiryCleaner cleaner = new ExpiryCleaner(s, 10)) {
                s.put("x", "1", 20);
                Thread.sleep(150);
                assertEquals(1L, cleaner.totalRemoved());
            }
        });
        t.test("shell parses commands and reports errors", () -> {
            Shell sh = new Shell(new ConcurrentStore(new ManualTime()));
            assertEquals("OK", sh.execute("set a 1"));
            assertEquals("\"1\"", sh.execute("GET a"));
            assertEquals("(nil)", sh.execute("GET zzz"));
            assertEquals("ERR usage: SET key value [ttlMillis]", sh.execute("SET a"));
            assertEquals("ERR unknown command 'NOPE'", sh.execute("NOPE"));
            assertTrue(sh.execute("SET a 1 abc").startsWith("ERR"), "bad ttl must be an error");
        });
        t.test("incr on a non-number value fails clearly", () -> {
            KeyValueStore s = new ConcurrentStore(new ManualTime());
            s.put("name", "asha");
            assertThrows(NumberFormatException.class, () -> s.incr("name"));
        });

        System.exit(t.runAll() == 0 ? 0 : 1);
    }
}
```

Compile and run the tests TERMINAL

```
$ javac -d testout -cp out $(find test -name '*.java')
$ java -cp out:testout com.example.kv.KvTests
PASS LockedStore: put, get, overwrite, delete
PASS LockedStore: entries expire after their ttl
PASS LockedStore: purgeExpired removes only expired entries
PASS LockedStore: incr is atomic across 4 threads
PASS ConcurrentStore: put, get, overwrite, delete
PASS ConcurrentStore: entries expire after their ttl
```

Compile and run the tests (continued) TERMINAL

```
PASS ConcurrentStore: purgeExpired removes only expired entries
PASS ConcurrentStore: incr is atomic across 4 threads
PASS LruStore: put, get, overwrite, delete
PASS LruStore: entries expire after their ttl
PASS LruStore: purgeExpired removes only expired entries
PASS LruStore: incr is atomic across 4 threads
PASS LRU evicts the least recently used entry
PASS cleaner thread removes expired entries on its own
PASS shell parses commands and reports errors
PASS incr on a non-number value fails clearly
16 tests, 0 failed
```

The `incr` test starts four threads that each increment 10,000 times and expects exactly 40,000. If you replace the lock in `LockedStore.incr` by nothing, this test fails with a lower number; try it.

13.5.6 Which one is faster? A measurement, and its limits

`Bench` runs 4 threads doing 4 million operations in total (95% reads) against the two concurrent stores, three rounds each.

Throughput comparison (typical, noisy) TERMINAL

```
$ java -cp out com.example.kv.shell.Bench
round 1
  LockedStore        762 ms  (4000000 ops)
  ConcurrentStore    383 ms  (4000000 ops)
round 2
  LockedStore        489 ms  (4000000 ops)
  ConcurrentStore    379 ms  (4000000 ops)
round 3
  LockedStore        443 ms  (4000000 ops)
  ConcurrentStore    345 ms  (4000000 ops)
```

Read the numbers honestly. On the 2-core build machine neither store wins consistently; the differences between rounds are as large as the differences between stores. Much of the time is spent building the `"key" + n` strings and generating random numbers, not in the store. With 2 cores there is little real parallelism to exploit, and the first round includes JIT warm-up. The lesson is methodological: a hand-made benchmark can show *whether something is badly wrong*, not which of two similar designs is better. Use JMH (the OpenJDK Microbenchmark Harness) for real decisions, and decide on correctness and simplicity first: `ConcurrentStore` has less code, no locks to forget to release, and scales when cores are many.

INTERVIEW ANGLE

Implement an LRU cache. What data structures and what complexity? A hash map for O(1) lookup plus a doubly linked list ordering keys by recency, with the map pointing into the list; get moves the node to the front, put inserts at the front and evicts the tail when over capacity: O(1) for both. In Java, `LinkedHashMap(capacity, 0.75f, true)` with `removeEldestEntry` does exactly this. Thread safety must be added separately, and note that `get` mutates the structure, so a read lock is not enough.

INTERVIEW ANGLE

`HashMap`, `Hashtable`, `Collections.synchronizedMap`, `ConcurrentHashMap`: the differences? `HashMap` is not thread-safe. `Hashtable` and `synchronizedMap` lock the whole map on every call and still need external locking for compound actions like iteration or check-then-put. `ConcurrentHashMap` locks at a much finer grain (CAS for empty bins, per-bin locks), allows lock-free reads, forbids null keys and values, and offers atomic compound operations (`compute`, `merge`, `putIfAbsent`).

REAL WORLD

In production you would use Redis or Caffeine instead of writing your own. Spring Boot's cache abstraction (`@Cacheable`) plugs those in. Knowing how TTL, eviction and atomic counters work inside tells you what to configure and what failure modes to expect: a cache without a size bound or a TTL is a memory leak with a nice name.

13.5.7 Extension ideas

1. Add an `EXPIRE key ms` command and a `TTL key` command that reports remaining milliseconds.
2. Persist the store: write a snapshot to a file on `SAVE` and load it at startup; skip entries that already expired. Reuse the write-then-rename trick from the bank project.
3. Make `LruStore` evict expired entries before live ones, using `purgeExpired` when the map is full.
4. Replace `size()`'s O(n) scan with an `AtomicInteger` or `LongAdder` maintained on put and delete. What race makes it hard with TTL?
5. Serve the shell over TCP: a `ServerSocket` that creates a virtual thread per connection and uses one shared store (see the networking mentions in Part 5).
6. Add a `MULTI`/`EXEC`-like batch that applies several commands atomically in `LockedStore` by holding the write lock, and explain why `ConcurrentStore` cannot do that cheaply.

Check yourself

1. Why can `LockedStore.get` not simply call `map.remove(key)` when it finds an expired entry?
2. What is the bug? `if (!map.containsKey(k)) { map.put(k, 1); }` where `map` is a `ConcurrentHashMap`.
3. Why would a `ReadWriteLock` be wrong for the LRU cache?
4. What does this print? `var m = new LinkedHashMap<String,Integer>(16, 0.75f, true); m.put("a",1); m.put("b",2); m.get("a"); System.out.println(m.keySet());`
5. What happens to a `scheduleWithFixedDelay` task whose body throws an uncaught exception?
6. Why do the tests use a manual clock instead of `Thread.sleep`?

Answers

1. It holds only the read lock, and a read lock cannot be upgraded to a write lock; requesting it deadlocks. Also, removing under a read lock would be a data race with other readers.
2. Check-then-act race: two threads can both see the key absent and both put. Use `putIfAbsent` or `compute`.
3. In access order `get` relinks nodes, i.e. it writes to the map, so concurrent "readers" would corrupt the list. Use `synchronized` or a write lock.
4. `[b, a]`, because the `get("a")` moved `a` to the most-recent end (verify in jshell).
5. The executor suppresses the exception and never runs the task again, silently. Catch exceptions inside the task.
6. A manual clock makes expiry exact, instant and deterministic; sleeping makes tests slow and flaky on busy machines.

KEY POINTS
Design to an interface and run one contract test suite against every implementation.

KEY POINTS
`ReadWriteLock` helps read-heavy code but cannot upgrade a read lock; `ConcurrentHashMap` needs atomic compound operations (`compute`, `putIfAbsent`, conditional `remove`) instead of check-then-act.

KEY POINTS
`LinkedHashMap` with access order and `removeEldestEntry` is a three-line LRU, but because `get` mutates it, guard it with a single lock.

KEY POINTS
Combine lazy and active expiry; run the cleaner on a daemon scheduled thread and catch exceptions inside it.

KEY POINTS
Inject time so tests are fast and exact; hand-made benchmarks show gross problems only.

PART 14

Interview Preparation

Four chapters of model answers (165 questions), then puzzles and coding problems. Use the answers as a spine, not a script.

In this part

14.1 How Java Interviews Work
14.2 Interview Q&A: Language, OOP, Strings, Exceptions and I/O
14.3 Interview Q&A: Collections, Generics, Streams and Functional Java
14.4 Interview Q&A: Concurrency, the Memory Model and the JVM
14.5 Interview Q&A: Modern Java
14.6 Interview Q&A: Design and Best Practices
14.7 What Does This Print? Fifty Puzzles
14.8 Coding Problems: Strings and Arrays
14.9 Coding Problems: Collections, Recursion and Dynamic Programming
14.10 Concurrency Coding Tasks
14.11 Output Prediction and Debugging Drills

14.1 How Java Interviews Work

You can know a great deal of Java and still fumble an interview, because an interview tests something slightly different from knowledge: whether you can explain, reason aloud, and stay calm when you hit the edge of what you know. This chapter explains how Java hiring processes are usually organised, what each round is really checking, and how to answer so that the interviewer can see your thinking. The chapters after it contain 160 questions with model answers that you can use to practise.

14.1.1 The shape of the process

Processes differ by company, but a Java backend or SDE hiring loop usually has the same building blocks.

Round	What it checks	How to prepare
Online assessment	Data structures and algorithms under a clock, sometimes with MCQs on Java, SQL and OS	Practise timed problems; revise Big-O; know the standard library
Technical round 1	Core Java: OOP, collections, strings, exceptions, plus one or two coding problems	The Q&A chapters in this Part, and the coding chapters that follow
Technical round 2	Depth: concurrency, JVM and memory, your project, design of a small system	Your own project, the concurrency and JVM Parts, a few design sketches
Managerial / HR	Communication, ownership, motivation, how you handle conflict and failure	Two or three honest stories using situation, action, result

Typical rounds for a fresher or early-career Java developer.

Many companies merge the first two technical rounds. Product companies lean toward algorithms; service companies lean toward breadth of Java and framework vocabulary; startups lean toward "build this small thing now". Read the job description and weigh your preparation accordingly.

14.1.2 What interviewers are really testing

Behind every question there is usually one of four intentions.

- **Do you have a correct mental model?** "Is Java pass-by-value?" is not a trivia question. A wrong model produces bugs, and a right one lets you predict what code does without running it.
- **Can you connect a concept to a consequence?** Knowing that `HashMap` uses `hashCode` is trivia. Knowing that a mutable key with a changing hash makes the entry unreachable is understanding.
- **Do you know the trade-off?** "ArrayList or LinkedList?" has no single answer. A good answer names the access pattern, the cost model, and the practical reality (cache locality usually wins).

- **Have you actually written code?** Follow-ups such as "what did you do in your project when that failed?" separate people who read from people who built. This is why the project chapters of this book matter: they give you real experience to quote.

14.1.3 How to structure an answer

A reliable shape for a conceptual question is **definition, mechanism, example, trade-off**, delivered in 30 to 90 seconds.

1. One-sentence definition, in your own words.
2. How it works underneath (one or two sentences: this is where you stand out).
3. A tiny example or a consequence ("so if you do X, Y happens").
4. The catch or the alternative, so you show judgement.

Then stop. Interviewers will probe the part they care about. Monologues of five minutes feel safe but they hide where your understanding ends, and a clumsy tangent can open a trap you did not need to enter.

INTERVIEW ANGLE
"What is the difference between an abstract class and an interface?" A weak answer lists syntax differences. A strong answer says: an interface defines a capability contract that unrelated classes can implement, and a class can implement many; an abstract class is a partially built base for a family of closely related classes, with state and constructors, and a class can extend only one. Then add the modern nuance: since Java 8 interfaces can have `default` methods, so the line is about state and identity, not about "interfaces cannot have code".

14.1.4 Whiteboard and live-coding habits

- **Restate the problem** and ask about inputs: size, duplicates, nulls, sorted or not, memory limits. Interviewers often leave gaps on purpose.
- **Start with a brute force and say its complexity.** Then improve. A correct slow solution beats a half-finished clever one.
- **Talk while you think.** Silence is ambiguous; narration lets the interviewer help you.
- **Write real Java**, not pseudo-code: correct types, generics, and imports named. Use meaningful names; `i` and `j` are fine for loops, nothing else is.
- **Test with a small example by hand** before saying "done": an empty input, one element, and a typical case.
- **State complexity** in time and space, and name what dominates.
- **If you are stuck**, say what you know, what you tried, and what you would try next. "I would first check the edge case when the list has one element" is a good sentence.

TIP
Never bluff. "I am not certain, but I believe X because Y, and I would confirm in the Javadoc" is a strong answer. Confident wrong answers are remembered; honest partial answers are forgiven.

14.1.5 Talking about your project

Every technical round includes "tell me about a project". Prepare a two-minute version with this skeleton: the problem, your role, the design in three sentences, the hardest bug or trade-off, and what you would change. Be ready to open the code in your head: why that collection, how errors are handled, what happens under concurrency, how you tested it. If you built one of the projects from this book, extend it (a new feature, a failure mode) so you have a genuine story.

14.1.6 How to use the Q&A chapters

Each question carries a difficulty tag: **basic** (expected from everyone), **intermediate** (expected from a good candidate), **advanced** (separates strong candidates). Cover the answer, say yours aloud, then compare. The model answers are deliberately short; the chapters earlier in the book hold the full explanation, and the answers name the topic so you can go back. Code snippets in the answers were run on JDK 21 unless marked otherwise.

Check yourself

1. Name the four parts of the answer shape recommended above.
2. You are given a coding problem. What is the first thing you should do before writing code?
3. An interviewer asks something you do not know. Give a better reply than "I don't know".
4. Why is "I know the syntax" a weak answer to "difference between abstract class and interface"?

Answers

1. Definition, mechanism, example, trade-off (or catch).
2. Restate the problem and clarify inputs, constraints and edge cases; then propose a brute force and its complexity.
3. State what you do know, reason from it to a best guess, say how you would verify it (Javadoc, a quick test), and stay calm.
4. Because interviews test the design reasoning (capability vs shared base, state, single inheritance), not just the keywords.

KEY POINTS

Interviews test models, consequences, trade-offs and real experience, not recitation.

- Answer with definition, mechanism, example, trade-off, then stop.
- Clarify, brute-force, optimise, test by hand, state complexity.
- Prepare a two-minute project story and know its weak points.
- Honest partial answers beat confident wrong ones.

14.2 Interview Q&A: Language, OOP, Strings, Exceptions and I/O

Forty questions, in two groups. Answers are written to be spoken in under a minute. Difficulty is marked on each heading.

14.2.1 Language and object-oriented Java

Q1. [basic] What are the JDK, JRE and JVM?

The **JVM** executes bytecode: it loads classes, manages memory with a garbage collector, and runs code through an interpreter plus a JIT compiler. The **JRE** is the JVM plus the standard class library. The **JDK** adds the development tools (`javac`, `jar`, `jshell`, `javap`, `jcmd`). Since Java 11 Oracle no longer ships a standalone JRE; you install a JDK, or use `jlink` to build a trimmed runtime.

Q2. [basic] Why is Java called platform independent?

The compiler produces bytecode for an abstract machine, not for a CPU. Each operating system has its own JVM that turns that bytecode into native instructions. The same `.class` file therefore runs anywhere a JVM exists. The JVM itself is platform specific; the bytecode and the language semantics (fixed sizes of `int`, `long`) are not.

Q3. [basic] What is the difference between == and equals()?

`==` compares primitive values, or for references whether both refer to the same object. `equals()` compares logical content, if the class overrides it; the default `Object.equals` is just `==`. Never compare strings or boxed values with `==`.

Boxing and equality JAVA

```
Integer a = 127, b = 127, c = 128, d = 128;
System.out.println((a == b) + " " + (c == d) + " " + c.equals(d));
```

Output TERMINAL

```
true false true
```

Boxing uses `Integer.valueOf`, which caches -128 to 127, so `a == b` is true by accident and `c == d` is false.

Q4. [basic] Is Java pass-by-value or pass-by-reference?

Always pass-by-value. For objects, the value passed is a copy of the reference, so the callee can mutate the object the caller sees, but reassigning the parameter does not affect the caller's variable.

Pass by value JAVA

```java
static void change(int[] a, int n, StringBuilder sb, String s) {
    a[0] = 99;        // mutates the shared array
    n = 99;           // changes the local copy only
    sb.append("!");   // mutates the shared object
    s = s + "!";      // makes a new String; local reassignment
}
// caller: int[] arr={1}; int n=1; sb=new StringBuilder("x"); s="x"; change(arr,n,sb,s)
```

Values after the call (arr[0], n, sb, s) TERMINAL

```
99 1 x! x
```

Q5. [basic] Primitive types versus wrapper classes. When do you use each?

There are eight primitives (`byte short int long float double char boolean`). They hold values directly, cannot be null, and are fast. Wrappers (`Integer`, `Long`, ...) are objects, needed for generics (`List<Integer>`), nullability and library APIs. Prefer primitives for arithmetic and fields that always have a value. Danger: unboxing a null throws `NullPointerException`, and boxing in tight loops allocates objects.

Q6. [basic] Explain final, finally and finalize.

`final` is a modifier: a final variable is assigned once, a final method cannot be overridden, a final class cannot be extended. `finally` is a block that runs after `try`/`catch` whether or not an exception occurred. `finalize()` was a pre-GC hook; it is deprecated for removal since Java 18 and must not be used. Use try-with-resources or `Cleaner` instead.

Q7. [basic] What is the difference between overloading and overriding?

Overloading: same method name, different parameter lists in the same class (or hierarchy); chosen at compile time by the static types of the arguments. Overriding: a subclass supplies a new body for an inherited instance method with the same signature; chosen at run time by the object's actual class (dynamic dispatch). Use `@Override` so the compiler checks you really are overriding.

Q8. [intermediate] Can you override a static or private method?

No. Static methods belong to the class and are *hidden*, not overridden: the method chosen depends on the declared type of the reference. Private methods are not inherited, so a same-named method in a subclass is a new, unrelated method. Fields are hidden too, never overridden.

Hiding and dynamic dispatch (Parent/Child with the same field and static method) JAVA

```java
Parent p = new Child();
System.out.println(p.name + " " + p.who());   // field: declared type; method: actual type
p.hello();                                    // static: declared type
```

Output TERMINAL

```
parent Child
Parent.hello
```

Q9. [basic] Abstract class or interface?

Use an interface for a capability that unrelated types can have (`Comparable`, `Runnable`); a class can implement many. Use an abstract class for a base with shared state, constructors and partial implementation; a class extends only one. Interfaces cannot hold instance state, but since Java 8 can have `default` and `static` methods, and since 9 `private` ones.

Q10. [intermediate] Two interfaces provide the same default method. What happens?

The implementing class does not compile until it resolves the conflict by overriding the method. Inside it you can pick one version with `InterfaceName.super.method()`. A concrete method inherited from a class always wins over an interface default ("class wins" rule).

Q11. [basic] What are the four pillars of OOP?

Encapsulation (hide state behind methods, enforce invariants), **abstraction** (expose what, hide how), **inheritance** (reuse and specialise via a base type) and **polymorphism** (one type, many behaviours via overriding and interfaces). Be ready to give one concrete Java example of each, and to say that composition is often better than inheritance.

Q12. [intermediate] Composition or inheritance?

Inheritance models "is-a" and couples the subclass to the superclass's implementation: a change in the parent can break children (the fragile base class problem). Composition ("has-a") holds another object and delegates, which is easier to change and test. Rule of thumb: inherit only when the subclass truly is a specialised parent and the parent was designed for extension; otherwise compose and depend on an interface.

Q13. [intermediate] Why does Java not support multiple inheritance of classes?

To avoid the diamond problem (ambiguity about which inherited state and method to use) and the complexity of combining state from several parents. Java allows multiple inheritance of *type* via interfaces, and of behaviour via default methods, which have an explicit resolution rule (Q10). Multiple inheritance of *state* is deliberately absent.

Q14. [intermediate] In what order are fields, blocks and constructors initialised? What is the trap?

Class loading first: static fields and static blocks, once, in textual order. Per `new`: the superclass constructor runs first (itself initialising its fields, then its body), then the subclass's field initialisers and instance blocks, then the subclass constructor body. The trap: if a superclass constructor calls an overridable method, the subclass override runs before the subclass fields are initialised, so it sees defaults.

Overridable call in a constructor JAVA

```java
class Parent {
    Parent() { init(); }
    void init() { System.out.println("Parent.init"); }
}
class Child extends Parent {
    int value = 42;
    Child() { super(); System.out.println("Child ctor value=" + value); }
    @Override void init() { System.out.println("Child.init value=" + value); }
}
// new Child();
```

Output TERMINAL

```
Child.init value=0
Child ctor value=42
```

Never call overridable methods from constructors; make them `final` or private if you must.

Q15. [intermediate] What is the equals/hashCode contract, and what breaks if you violate it?

Equal objects must have equal hash codes; `equals` must be reflexive, symmetric, transitive, consistent, and `x.equals(null)` is false. Hash-based collections find a bucket with `hashCode` and then confirm with `equals`. If you override only `equals`, two equal objects usually land in different buckets and `contains` misses them.

Overriding equals without hashCode JAVA

```java
Set<Point> set = new HashSet<>();
set.add(new Point(1, 2));                    // Point overrides only equals
System.out.println(set.contains(new Point(1, 2)));
```

Output (the default identity hash differs, so the bucket differs; typical) TERMINAL

```
false
```

The result is *typically* false, not guaranteed; that is what makes the bug nasty. Override both, based on the same fields, or use a `record`.

Q16. [intermediate] How do you build an immutable class?

Declare the class `final` (or make constructors private), make all fields `private final`, set them in the constructor, provide no setters, and defensively copy any mutable inputs and outputs (arrays, `Date`, collections; use `List.copyOf`). Immutable objects are thread-safe by construction, safe as map keys and easy to reason about. A `record` gives most of this for free, but is only *shallowly* immutable: a `List` component is still mutable unless you copy it.

Q17. [basic] What are the access modifiers?

`private` (same class), package-private (no keyword; same package), `protected` (same package plus subclasses), `public` (everywhere). With modules (Java 9) a package must also be `exports`-ed for outside access. Default to the narrowest access that works.

Q18. [intermediate] What kinds of nested classes exist?

A **static nested** class has no reference to an outer instance. An **inner** (non-static) class holds a hidden reference to its enclosing instance, which can keep the outer object alive (a classic leak) and complicates serialization. A **local** class is declared in a method. An **anonymous** class is a one-off subclass or implementation; lambdas replace most uses for functional interfaces. Prefer static nested unless you need the outer instance.

Q19. [basic] What is special about enums?

An enum is a class with a fixed set of instances, created once by the JVM. It can have fields, constructors, methods and per-constant bodies, and implement interfaces. They are type-safe, work in `switch`, are singletons per constant, and serialize safely by name. Use `EnumMap` and `EnumSet` for compact, fast collections keyed by enum.

Q20. [intermediate] What is a record and when do you not use one?

A record (Java 16) is a transparent, shallowly immutable data carrier: the compiler generates the canonical constructor, accessors, `equals`, `hashCode` and `toString`. Use it for DTOs, map keys, return tuples. Do not use it for entities with identity and lifecycle, classes that need mutable state, or inheritance (records are final and cannot extend a class).

Q21. [intermediate] Shallow copy versus deep copy; is clone() a good idea?

A shallow copy copies references, so the copy and original share inner objects; a deep copy duplicates the whole object graph. `Object.clone()` does a shallow field copy, requires `Cloneable`, bypasses constructors and is widely considered broken (Effective Java). Prefer a copy constructor or static factory, and copy mutable parts explicitly.

Q22. [intermediate] Which numeric traps appear most often?

Integer overflow wraps silently; compound assignment hides a cast; `char` arithmetic gives `int`; ternary operands undergo numeric promotion; floating point is binary. Run this to see them:

Numeric traps JAVA

```java
short sh = 10; sh += 5;                          // allowed: implicit cast back to short
System.out.println(Integer.MAX_VALUE + 1);       // wraps
System.out.println('a' + 1);                     // int arithmetic
System.out.println((char) ('a' + 1));            // cast back to char
System.out.println(true ? 1 : 2.0);              // 1 is promoted to double
System.out.println(0.1 + 0.2);
System.out.println(new BigDecimal("0.1").add(new BigDecimal("0.2")));
```

Output TERMINAL

```
-2147483648
98
b
1.0
0.30000000000000004
0.3
```

Use `Math.addExact` to fail on overflow, `BigDecimal` (built from strings) for money, and never compare floating-point values with `==`.

14.2.2 Strings, exceptions and I/O

Q23. [basic] Why is String immutable?

Immutability makes strings safe to share (the string pool), safe as `HashMap` keys (the hash can be cached), thread-safe without locks, and secure (a path or class name cannot change after validation). The cost is that every modification creates a new object; use `StringBuilder` for repeated edits.

Q24. [basic] What is the string pool? What does new String("x") do?

Literals and compile-time constants are interned in a pool (on the heap since Java 7), so identical literals share one object. `new String("x")` always creates a new object, distinct from the pooled one; `intern()` returns the pooled instance.

Pool behaviour JAVA

```java
String x = "hi", y = "hi", z = new String("hi");
System.out.println((x == y) + " " + (x == z) + " " + (x == z.intern()));
```

Output TERMINAL

```
true false true
```

Q25. [basic] String, StringBuilder or StringBuffer?

`String` is immutable. `StringBuilder` is a mutable, unsynchronized buffer: the default choice for building text in a loop. `StringBuffer` is the synchronized legacy version; you almost never need it because a builder is rarely shared across threads. A single `a + b + c` expression is fine (the compiler uses `invokedynamic`); concatenating inside a loop creates a new string per iteration, so use a builder there.

Q26. [advanced] Which of these is true: "a"+"b" == "ab", and the same with a variable?

Compile-time constants are folded and pooled, so `"a" + "b" == "ab"` is true, and so is a concatenation of `final` variables initialised with constants. A concatenation involving a non-final variable is computed at run time and creates a new object.

Constant folding JAVA

```java
String t = "a" + "b";
String u = "a"; String v = u + "b";
final String w = "a"; String x = w + "b";
System.out.println((t == "ab") + " " + (v == "ab") + " " + (x == "ab"));
```

Output TERMINAL

```
true false true
```

Q27. [intermediate] A common split surprise?

`String.split` takes a regex and drops *trailing* empty strings. `"a,b,,".split(",")` has length 2, not 4. Pass a negative limit (`split(",", -1)`) to keep them. Regex metacharacters must be escaped: `split("\\.")`, not `split(".")`.

Q28. [basic] Checked versus unchecked exceptions?

Checked exceptions (subclasses of `Exception` other than `RuntimeException`) must be caught or declared; the compiler enforces it. They model recoverable conditions outside the caller's control, such as `IOException`. Unchecked exceptions (`RuntimeException` and `Error` subclasses) need no declaration and usually signal programming errors (`NullPointerException`, `IllegalArgumentException`). Modern libraries and frameworks (Spring) favour unchecked exceptions because checked ones leak through lambdas and layers.

Q29. [basic] Exception versus Error?

Both extend `Throwable`. `Exception` is for conditions an application may handle. `Error` signals JVM-level failures (`OutOfMemoryError`, `StackOverflowError`, `NoClassDefFoundError`); catching them is almost always wrong, because the program state may be unsound. At most, log and shut down.

Q30. [intermediate] How does try-with-resources work, and what are suppressed exceptions?

Any `AutoCloseable` declared in the header is closed automatically, in *reverse* order, whether the body succeeds or throws. If the body throws and `close()` throws too, the body's exception is the one propagated and the close exceptions are attached with `addSuppressed` (read them via `getSuppressed()`).

Resources close in reverse order JAVA

```java
try (Res a = new Res("A"); Res b = new Res("B")) {   // close() throws IllegalStateException
    throw new RuntimeException("body");
} catch (Exception e) {
    System.out.println(e.getMessage() + " suppressed=" + Arrays.toString(e.getSuppressed()));
}
```

Output TERMINAL

```
open A
open B
close B
close A
body suppressed=[java.lang.IllegalStateException: close-B, java.lang.IllegalStateException: close-A]
```

Q31. [intermediate] What does return inside finally do?

A `finally` block always runs (barring `System.exit` or a JVM crash). If it contains `return`, that value replaces any pending return or exception, and the exception is silently lost. `try { return 1; } finally { return 2; }` returns 2. Never return, break or throw from `finally`.

Q32. [intermediate] How should you design and throw your own exceptions?

Pick checked only if callers can realistically recover; otherwise extend `RuntimeException`. Name it after the problem (`InsufficientFundsException`), include useful context fields, and always pass the original as the `cause` so the stack trace chain is kept: `new ServiceException("load failed", e)`. Do not use exceptions for normal control flow and do not catch and swallow silently. Log or rethrow, not both.

Q33. [basic] throw versus throws; catch ordering and multi-catch?

`throw` raises an exception object; `throws` declares what a method may throw. Catch blocks are tried in order, so a subclass catch must come before its superclass (otherwise it does not compile). Multi-catch `catch (IOException | SQLException e)` handles unrelated types in one block; the alternatives cannot be related by inheritance.

Q34. [intermediate] How does Java serialization work? What do transient and serialVersionUID do?

A class implementing `Serializable` can be written by `ObjectOutputStream`, which walks the object graph and writes non-static, non-transient fields. `transient` fields are skipped and get default values on read (null, 0). Static fields belong to the class, not the instance, so they are not written. `serialVersionUID` identifies the class version; declare it explicitly so harmless changes do not break old data. Native serialization is fragile and a known security risk (deserialization attacks); prefer JSON or Protobuf and never deserialize untrusted data.

transient in action JAVA

```java
// class User implements Serializable { String name = "ann"; transient String pw = "secret"; }
// after write + read:
System.out.println(u.name + " " + u.pw);
```

Output TERMINAL

```
ann null
```

Q35. [intermediate] Byte streams versus character streams; why specify a charset?

`InputStream`/`OutputStream` move raw bytes; `Reader`/`Writer` decode bytes to characters using a charset. Always pass the charset explicitly (`StandardCharsets.UTF_8`), because the platform default differed between machines before Java 18, which made files unreadable elsewhere. Since JDK 18 (JEP 400) the default is UTF-8, but explicit is still clearer. Wrap sources in `BufferedReader`/`BufferedWriter` to avoid one system call per byte.

Q36. [intermediate] java.io.File or java.nio.file?

Prefer NIO.2 (`Path`, `Files`): it has real error reporting through exceptions (`File.delete` just returns false), symbolic link and attribute support, directory walking, and one-liners such as `Files.readString` and `Files.lines`. `Files.lines` and `Files.list` return streams that hold an open file, so use them in try-with-resources.

Q37. [basic] Which runtime exceptions do you meet most?

`NullPointerException` (use of null; helpful messages since Java 14 name the null variable), `IllegalArgumentException`, `IllegalStateException`, `IndexOutOfBoundsException`, `ClassCastException` (bad downcast), `ArithmeticException` (integer divide by zero), `NumberFormatException` (bad `parseInt` input), `ConcurrentModificationException` and `ArrayStoreException` (an `Object[]` that is really a `String[]`). Knowing each cause is a quick win.

Q38. [advanced] ClassNotFoundException versus NoClassDefFoundError?

`ClassNotFoundException` is a checked exception thrown when code asks for a class by name (`Class.forName`, a classloader) and it is not found. `NoClassDefFoundError` is an `Error`: the class was present at compile time but is missing, or failed static initialisation, at run time. A failed static initialiser first throws `ExceptionInInitializerError`, and later uses of the class give `NoClassDefFoundError`.

Q39. [advanced] Can you catch StackOverflowError or OutOfMemoryError?

Syntactically yes. `StackOverflowError` can sometimes be survived (the stack unwinds and you lose only that call), but deep recursion should be turned into iteration. `OutOfMemoryError` is rarely recoverable, since other threads may also fail to allocate; you can catch it to log or release a big cache, but the sane response is to fix memory use or let the process die and restart. Do not use these for control flow.

Q40. [advanced] How does Java choose among overloaded methods?

In three phases: first by subtyping and widening only (no boxing, no varargs); then with boxing/unboxing; then varargs. Within a phase, the most specific method wins. So `g(5)` with `g(long)`, `g(Integer)` and `g(int...)` calls `g(long)` (widening beats boxing). `f(null)` with `f(Object)` and `f(String)` picks `String` as the more specific type. The choice uses the *static* type: `Object o = "s"; f(o)` calls `f(Object)`.

Overload resolution JAVA

```java
static void f(Object o)  { System.out.println("Object"); }
static void f(String s)  { System.out.println("String"); }
static void g(long x)    { System.out.println("long"); }
static void g(Integer x) { System.out.println("Integer"); }
static void g(int... x)  { System.out.println("varargs"); }
// f(null); Object o = "s"; f(o); g(5);
```

Output TERMINAL

```
String
Object
long
```

INTERVIEW ANGLE

"Why should you override `hashCode` whenever you override `equals`?" Because `HashMap`, `HashSet` and friends use `hashCode` to choose a bucket and only then call `equals`. Equal objects with different hash codes are searched for in the wrong bucket and are not found. State the contract (equal objects have equal hashes; unequal ones may collide), mention that the fields used in both must match, and add that mutating a key field after insertion loses the entry.

REAL WORLD

In Spring applications the same ideas appear as: `@Entity` classes with careful `equals`/`hashCode` (often on a business key, not the generated id), DTOs as records, custom runtime exceptions mapped to HTTP statuses by `@ControllerAdvice`, and try-with-resources for JDBC connections and streams.

Check yourself

1. What does this print? `Integer a = 1000, b = 1000; System.out.println(a == b);`
2. A subclass overrides a method that the superclass constructor calls. The subclass has `int n = 5;`. What does the override see for `n` during construction?
3. What is wrong with this class used as a `HashMap` key? `class K { int id; public boolean equals(Object o) { return o instanceof K k && k.id == id; } }`
4. What does `try { return 1; } finally { return 2; }` return, and what happens to an exception thrown in the `try`?
5. What is the length of `"x,,y,,".split(",")`?
6. Why is `catch (Exception e) { }` with an empty body bad practice?

Answers

1. `false` in practice: 1000 is outside the -128..127 cache, so two distinct objects (use `equals`).
2. 0, the default value, because subclass field initialisers have not run yet.
3. `hashCode` is not overridden, so equal keys usually hash to different buckets and lookups fail.
4. It returns 2; an exception thrown in the `try` is discarded.

5. 3 (`x`, empty, `y`): trailing empty strings are removed.
6. It hides failures, makes bugs undiscoverable and may swallow `InterruptedException` or `Error`-like conditions; log with context or rethrow wrapped.

KEY POINTS

The Java core language is a set of consistent rules; interviews probe whether you can apply them.

- Java is pass-by-value; reference values are copies of pointers.
- Overloading is compile-time, overriding is run-time; fields and statics are hidden, not overridden.
- Override `equals` and `hashCode` together, prefer immutability and records for value types.
- Strings are immutable and pooled; compile-time constants fold, run-time concatenation does not.
- Prefer try-with-resources, wrap with cause, never swallow or return from `finally`.

14.3 Interview Q&A: Collections, Generics, Streams and Functional Java

Forty more questions. The collections questions are the most reliably asked in Java interviews of any level, usually in the form "how does HashMap work" followed by a chain of follow-ups. Read the answers, then explain them aloud with a drawing.

14.3.1 Collections and generics

Q41. [basic] Describe the Collections Framework hierarchy.

`Iterable` is the root of `Collection`, which splits into `List` (ordered, indexed, duplicates allowed), `Set` (no duplicates) and `Queue`/`Deque` (head and tail operations). `Map` is *not* a `Collection`; it maps keys to values and exposes `keySet()`, `values()` and `entrySet()` views. Main implementations: `ArrayList`, `LinkedList`, `HashSet`, `LinkedHashSet`, `TreeSet`, `ArrayDeque`, `PriorityQueue`, `HashMap`, `LinkedHashMap`, `TreeMap`. Always declare variables by the interface (`List<String> xs = new ArrayList<>()`).

Q42. [basic] ArrayList or LinkedList?

`ArrayList` is a resizable array: `get(i)` is O(1), append is amortised O(1), insert or remove in the middle is O(n) because of shifting (a very fast `System.arraycopy`). `LinkedList` is a doubly linked list: insert/remove at a known node is O(1), but `get(i)` is O(n) and every element is a separate node object, which is bad for memory and CPU caches. In practice `ArrayList` wins almost always; for a queue or stack use `ArrayDeque`.

Q43. [intermediate] How does ArrayList grow?

It keeps an `Object[]`. The no-arg constructor starts with an empty shared array and allocates capacity 10 on the first `add`. When full it allocates a new array of about 1.5 times the size (`oldCapacity + (oldCapacity >> 1)`) and copies. If you know the final size, use `new ArrayList<>(n)` to avoid repeated copying. `trimToSize()` releases unused slots.

Q44. [basic] How does HashMap work internally?

It holds an array of buckets (default capacity 16, load factor 0.75). `put(k, v)` computes `hash = h ^ (h >>> 16)` from `k.hashCode()` to mix high bits into low bits, picks the bucket `(n - 1) & hash` (n is always a power of two), and walks the chain in that bucket comparing hashes and then `equals`. If the key exists the value is replaced; otherwise a node is appended. When `size` exceeds `capacity * 0.75`, the table doubles and entries are redistributed (each entry stays in the same bucket or moves by exactly the old capacity). Average `get`/`put` is O(1).

Q45. [advanced] What changed in Java 8 for HashMap collisions?

A bucket chain with at least 8 nodes, in a table of at least 64 buckets, is converted into a red-black tree (lookup O(log n) instead of O(n)); smaller tables resize instead. A tree shrinks back to a list at 6 nodes or fewer. This protects against poor hash functions and hash-flooding attacks. Tree nodes order by hash, then by `Comparable` if the key class implements it, so comparable keys behave better under collision.

Q46. [intermediate] HashMap, Hashtable, Collections.synchronizedMap and ConcurrentHashMap?

`HashMap` is unsynchronized, allows one null key and null values. `Hashtable` is legacy: every method is synchronized on one lock and no nulls are allowed. `synchronizedMap` wraps a map with one lock; iteration still needs manual locking. `ConcurrentHashMap` allows concurrent reads and fine-grained concurrent writes (CAS and per-bin locking), offers atomic operations such as `computeIfAbsent`, `merge`, and does not allow null keys or values. Use it for shared mutable maps. Part 8 covers the details.

Q47. [basic] How does HashSet ensure uniqueness?

A `HashSet` is a `HashMap` in disguise: elements are keys, and every key maps to one shared dummy value. `add` returns false when the key already exists, decided by `hashCode` then `equals`. `LinkedHashSet` keeps insertion order; `TreeSet` keeps sorted order and uses `compareTo`/`Comparator` instead of `equals`.

Q48. [intermediate] What happens if a key is mutated after insertion?

The entry sits in the bucket for the *old* hash. Looking up the mutated key hashes to a different bucket and misses; looking up an equal copy of the old value finds the bucket but fails `equals` against the changed key. The entry is effectively lost and leaks. Use immutable keys.

A mutated key is lost JAVA

```java
class K { int id; K(int i) { id = i; }
    public boolean equals(Object o) { return o instanceof K k && k.id == id; }
    public int hashCode() { return id; } }
Map<K,String> mk = new HashMap<>();
K k = new K(1); mk.put(k, "v"); k.id = 2;
System.out.println(mk.get(k) + " " + mk.get(new K(1)) + " " + mk.size());
```

Output TERMINAL

```
null null 1
```

Q49. [intermediate] Comparable versus Comparator?

`Comparable<T>` gives a class one natural ordering via `compareTo` (strings, numbers, dates). `Comparator<T>` is an external ordering, so you can sort the same class several ways without changing it. Build comparators with `Comparator.comparing(...)`, `thenComparing`, `reversed`, `nullsFirst`. The ordering must be consistent with `equals` when used in `TreeSet`/`TreeMap`, because those use compare results as equality (`compare == 0` means duplicate).

Q50. [intermediate] How do you build an LRU cache in Java?

Extend `LinkedHashMap` with access order enabled (third constructor argument `true`) and override `removeEldestEntry`. Every `get` moves the entry to the tail; the head is the least recently used and is evicted when the size limit is exceeded. It is not thread-safe; wrap it with a lock for concurrent use.

LRU with LinkedHashMap JAVA

```java
Map<Integer,String> lru = new LinkedHashMap<>(16, 0.75f, true) {
    protected boolean removeEldestEntry(Map.Entry<Integer,String> e) { return size() > 2; }
};
lru.put(1, "a"); lru.put(2, "b"); lru.get(1); lru.put(3, "c");
System.out.println(lru.keySet());
```

Output TERMINAL

```
[1, 3]
```

Key 2 was the least recently used, so it was evicted.

Q51. [intermediate] What is ConcurrentModificationException? Is it only about threads?

Fail-fast iterators of `ArrayList`, `HashMap` and others compare a `modCount` snapshot with the collection's count on each `next()`. Structural modification during iteration (other than through the iterator's own `remove`) makes the next call throw. It happens in single-threaded code too. It is best-effort, not guaranteed: removing the second-to-last element ends the loop silently because `hasNext()` returns false first.

Remove inside a for-each loop JAVA

```java
List<Integer> l = new ArrayList<>(List.of(1, 2, 3, 4));
for (Integer x : l) if (x == 2) l.remove(x);   // CME on the next iteration
List<Integer> l2 = new ArrayList<>(List.of(1, 2, 3, 4));
for (Integer x : l2) if (x == 3) l2.remove(x); // no exception: loop just ends
System.out.println(l2);
```

Output (the first loop throws ConcurrentModificationException, shown here caught) TERMINAL

```
[1, 2, 4]
```

Fixes: `Iterator.remove()`, `removeIf(predicate)`, or collect into a new list. Concurrent collections (`CopyOnWriteArrayList`, `ConcurrentHashMap`) have weakly consistent iterators that never throw it.

Q52. [basic] What is the trap in List.remove?

`remove(int index)` and `remove(Object o)` are overloads. On a `List<Integer>`, `list.remove(1)` removes the element at index 1, while `list.remove(Integer.valueOf(1))` removes the value 1.

Q53. [intermediate] List.of, Arrays.asList and Collections.unmodifiableList?

`List.of` (Java 9) is truly immutable, rejects nulls, and any mutation throws `UnsupportedOperationException`. `Arrays.asList` is a fixed-size view over the array: `set` works and writes through to the array, but `add`/`remove` throw. `Collections.unmodifiableList` is a read-only *view*: it blocks changes through the wrapper but reflects changes made to the underlying list. `List.copyOf` makes an independent immutable copy.

Three flavours of "unmodifiable" JAVA

```java
List<Integer> fixed = Arrays.asList(1, 2, 3);
fixed.set(0, 9);                      // ok
System.out.println(fixed);
// fixed.add(1)  -> UnsupportedOperationException
// List.of(1).add(2) -> UnsupportedOperationException
```

Output TERMINAL

```
[9, 2, 3]
```

Q54. [intermediate] Which maps accept null keys?

`HashMap` and `LinkedHashMap` allow one null key and null values. `TreeMap` throws `NullPointerException` for a null key (it must compare) when using natural ordering. `ConcurrentHashMap`, `Hashtable` and the `Map.of` family reject nulls. The concurrent ban exists so that "get returned null" unambiguously means "absent".

Q55. [intermediate] How does PriorityQueue work, and what does iterating it show?

It is a binary min-heap stored in an array; `offer` and `poll` are O(log n), `peek` is O(1). Only the head is guaranteed to be the smallest. The array order, and therefore `toString` and iteration, is *not* sorted. To get sorted output, `poll` repeatedly. For a max-heap supply `Comparator.reverseOrder()`.

Heap order is not sorted order JAVA

```java
PriorityQueue<Integer> pq = new PriorityQueue<>(List.of(5, 1, 4, 2, 3));
System.out.println(pq + " " + pq.poll() + " " + pq.peek());
```

Output TERMINAL

```
[1, 2, 4, 5, 3] 1 2
```

Q56. [basic] Which class do you use as a stack or a queue?

`ArrayDeque`: `push`/`pop`/`peek` for a stack, `offer`/`poll` for a queue. It is faster than `LinkedList` and `Stack`. `Stack` and `Vector` are legacy synchronized classes. `ArrayDeque` does not allow nulls. For thread-handoff use `BlockingQueue` implementations.

Q57. [intermediate] Which sorting algorithm does Java use, and is it stable?

`Collections.sort`/`List.sort`/`Arrays.sort(Object[])` use TimSort (a merge sort and insertion sort hybrid): stable, O(n log n) worst case, close to O(n) on nearly sorted data. `Arrays.sort` on primitive arrays uses dual-pivot quicksort: not stable (stability is meaningless for primitives), average O(n log n). Stability matters when sorting by one key after another.

Q58. [advanced] When would you use EnumMap, IdentityHashMap or WeakHashMap?

`EnumMap` is an array indexed by enum ordinal: fastest and smallest map for enum keys. `IdentityHashMap` compares keys with `==`, useful for graph traversal bookkeeping and serializers. `WeakHashMap` holds keys weakly, so an entry disappears once the key is otherwise unreachable; handy for canonicalising caches, but values must not strongly reference their key or the entry never goes.

Q59. [intermediate] What are sequenced collections (Java 21)?

Java 21 added `SequencedCollection`, `SequencedSet` and `SequencedMap` for types with a defined encounter order. They give uniform `getFirst()`, `getLast()`, `addFirst`, `removeLast` and `reversed()` on `List`, `Deque`, `LinkedHashSet`, `LinkedHashMap`, `TreeSet`. Before 21, getting the last element of a `List` was `list.get(list.size() - 1)`, and of a `LinkedHashSet` required iterating. `HashSet`/`HashMap` are not sequenced.

Q60. [intermediate] What is type erasure?

Generics are a compile-time feature. The compiler checks types, inserts casts, and erases type parameters to their bounds (`Object` if unbounded) in the bytecode. At run time `List<String>` and `List<Integer>` are the same class. Consequences: no `new T()`, no `instanceof List<String>`, no generic arrays, no overloads that differ only in type arguments. Erasure kept Java 5 code compatible with older libraries.

Same class at run time JAVA

```java
List<String> ls = new ArrayList<>(); List<Integer> li = new ArrayList<>();
System.out.println(ls.getClass() == li.getClass());
```

Output TERMINAL

```
true
```

Q61. [advanced] Explain ? extends, ? super and PECS.

Generics are invariant: `List<Integer>` is not a `List<Number>`. Wildcards add flexibility. **Producer Extends, Consumer Super**: if a parameter only *produces* values you read, use `? extends T`; if it only *consumes* values you write, use `? super T`.

PECS JAVA

```java
static double sum(Collection<? extends Number> src) {       // read Numbers out
    double s = 0; for (Number n : src) s += n.doubleValue(); return s;
}
static void fill(List<? super Integer> dst) { dst.add(1); dst.add(2); }  // write Integers in
// List<Number> nums = new ArrayList<>(); fill(nums); sum(nums)  ->  3.0
```

You cannot add to a `List<? extends Number>` (the real element type is unknown), nor safely read specific types from `? super`. `Collections.copy(dest, src)` has exactly this signature.

Q62. [advanced] What is heap pollution, and why are raw types discouraged?

A raw type (`List` without arguments) turns off generic checking. You can add an `Integer` to what is really a `List<String>`; the compiler only warns, and the failure appears later as a `ClassCastException` at an innocent-looking `String x = list.get(0)`. Avoid raw types, and treat unchecked warnings as bugs. Generic varargs (`T...`) can pollute the heap as well; `@SafeVarargs` is only for methods that do not write into the array.

Q63. [intermediate] How do you write a generic method with a bound?

Put the type parameter before the return type, and bound it for the operations you need.

Bounded generic method JAVA

```java
static <T extends Comparable<T>> T max(List<T> list) {
    T best = list.get(0);
    for (T t : list) if (t.compareTo(best) > 0) best = t;
    return best;
}
// max(List.of("b", "z", "a")) -> "z"
```

`Comparable<T>` as a bound lets you call `compareTo` on the elements. Many libraries use the more permissive `<T extends Comparable<? super T>>`.

Q64. [advanced] Why can't you create new T[] or use primitives as type arguments?

Because of erasure, `T` is unknown at run time, so arrays (which are reified and check element type at run time) cannot be created for it; use `(T[]) new Object[n]` with care, or `List<T>`. Type arguments must be reference types because erased code uses `Object`; `List<int>` is not allowed (boxing to `Integer` is used, with its cost). Project Valhalla aims at changing this, but it is not part of the current LTS.

14.3.2 Streams and functional Java

Q65. [basic] What is a stream, and how is it different from a collection?

A stream is a one-shot, lazy pipeline over a data source: source, zero or more *intermediate* operations (`filter`, `map`, `sorted`), and one *terminal* operation (`collect`, `forEach`, `count`) that triggers execution. A collection stores data; a stream does not store, it describes computation. Streams do not modify their source and can be consumed only once.

Q66. [intermediate] What does "lazy" mean here? Show it.

Intermediate operations only record steps. Nothing runs until the terminal operation, and elements flow one at a time through the whole pipeline (vertical processing), so short-circuiting operations such as `limit` or `findFirst` stop early.

Laziness and short-circuit JAVA

```java
Stream<Integer> s = Stream.of(1, 2, 3).peek(x -> System.out.println("peek " + x));
System.out.println("built");
s.limit(1).forEach(x -> System.out.println("got " + x));
```

Output TERMINAL

```
built
peek 1
got 1
```

Nothing printed while building; only element 1 was ever pulled.

Q67. [basic] map versus flatMap?

`map` transforms each element into exactly one output element. `flatMap` takes a function returning a stream per element and flattens all those streams into one: use it for "one-to-many" such as lines to words, or orders to line items. Think `Stream<List<X>>` to `Stream<X>`. On `Optional`, `flatMap` avoids `Optional<Optional<T>>`.

Q68. [intermediate] Show groupingBy and partitioningBy.

`groupingBy(classifier)` builds a `Map<K, List<T>>`; a downstream collector (`counting`, `averagingInt`, `mapping`, `summingInt`) reduces each group; a third argument selects the map type. `partitioningBy(predicate)` always yields exactly two keys, `false` and `true`.

Collectors JAVA

```java
record P(String n, int age, String city) {}
List<P> ps = List.of(new P("a", 20, "X"), new P("b", 30, "Y"), new P("c", 40, "X"));
Map<String, Double> avg = ps.stream()
    .collect(Collectors.groupingBy(P::city, TreeMap::new, Collectors.averagingInt(P::age)));
System.out.println(avg);
System.out.println(ps.stream().collect(
    Collectors.partitioningBy(p -> p.age() > 25, Collectors.counting())));
```

Output TERMINAL

```
{X=30.0, Y=30.0}
{false=1, true=2}
```

Q69. [intermediate] What goes wrong with Collectors.toMap?

Two elements with the same key make `toMap` throw `IllegalStateException: Duplicate key a (attempted merging values 1 and 1)`. Supply a merge function as the third argument (`(x, y) -> x`) and, if you need ordering, a map supplier as the fourth. A null value also throws `NullPointerException`.

Q70. [basic] Can you reuse a stream?

No. After a terminal operation it is consumed, and a second operation throws `IllegalStateException: stream has already been operated upon or closed`. Create a new stream from the source, or keep a `Supplier<Stream<T>>`.

Q71. [intermediate] Stream.toList() or Collectors.toList()?

`Stream.toList()` (Java 16) returns an unmodifiable list and permits nulls. `Collectors.toList()` returns a `List` whose mutability and type are unspecified (today an `ArrayList`), so do not depend on it. Use `toList()` by default; use `Collectors.toCollection(ArrayList::new)` when you need a mutable one.

Q72. [intermediate] When is a parallel stream a bad idea?

Parallel streams split the source and run on the common `ForkJoinPool`. They pay off for large data, CPU-bound, stateless, associative operations on easily splittable sources (arrays, `ArrayList`, ranges). They hurt with small data, I/O or blocking calls (starving the shared pool), `LinkedList` or `Stream.iterate` sources, and *shared mutable state*: `list.add` from a parallel `forEach` loses elements or throws. Measure before using. Ordered terminals such as `forEach` lose ordering; `forEachOrdered` keeps it at a cost.

Q73. [basic] How should you use Optional?

It is a return type that says "there may be no value". Use `map`, `flatMap`, `filter`, `orElse`, `orElseGet`, `orElseThrow`, `ifPresent` rather than `isPresent()` plus `get()`. Do not use it for fields, method parameters or collections (return an empty collection). `orElse(expensive())` always evaluates its argument; `orElseGet(() -> expensive())` is lazy.

Optional chaining JAVA

```java
Optional<String> o = Optional.empty();
System.out.println(o.map(String::length).orElse(-1));
```

Output TERMINAL

```
-1
```

Q74. [basic] Name the core functional interfaces.

`Function<T,R>` (T to R), `Predicate<T>` (T to boolean), `Consumer<T>` (T to nothing), `Supplier<T>` (nothing to T), `UnaryOperator<T>`, `BinaryOperator<T>`, `BiFunction<T,U,R>`, plus primitive forms (`IntPredicate`, `ToIntFunction`) that avoid boxing. A functional interface has exactly one abstract method; `@FunctionalInterface` makes the compiler check it. `Function` composes with `andThen` and `compose`, `Predicate` with `and`, `or`, `negate`.

Composition order JAVA

```java
Function<Integer, Integer> f = x -> x + 1, g = x -> x * 2;
System.out.println(f.andThen(g).apply(3) + " " + f.compose(g).apply(3));
```

Output TERMINAL

```
8 7
```

`f.andThen(g)` runs f first: `(3+1)*2 = 8`. `f.compose(g)` runs g first: `3*2+1 = 7`.

Q75. [intermediate] Lambda versus anonymous inner class?

A lambda is compiled to an `invokedynamic` call site bootstrapped by `LambdaMetafactory`; no extra `.class` file is generated at compile time and the JVM spins the implementation at first use. An anonymous class is a real class file with its own `this`. Inside a lambda `this` means the enclosing instance, inside an anonymous class it means the anonymous object. Lambdas work only for functional interfaces; anonymous classes can implement interfaces with many methods or extend a class and hold state.

Q76. [intermediate] Why must captured local variables be effectively final?

A lambda can outlive the method and run on another thread, and locals live on a stack frame that will be gone. Java therefore copies the value and requires that it never changes, so copy and original cannot disagree. The usual escape hatch is a one-element array or an `AtomicInteger`, but that is a signal to restructure (for example with `reduce` or a collector). Fields are not captured by value; they are read through `this` and can change.

The array trick, and why not to rely on it JAVA

```java
int[] cnt = {0};
Runnable run = () -> cnt[0]++;
run.run(); run.run();
System.out.println(cnt[0]);
```

Output TERMINAL

```
2
```

Q77. [intermediate] What are the four kinds of method reference?

Static (`Integer::parseInt`), bound instance (`System.out::println`, the receiver is fixed), unbound instance (`String::length`, the first argument is the receiver), and constructor (`ArrayList::new`). A reference is just a shorter lambda; choose it when it reads better.

Q78. [intermediate] Build a multi-key sort.

Chain comparators and name the key extractor so the intent is obvious.

Comparator chain JAVA

```java
Comparator<P> c = Comparator.comparing(P::city)
                            .thenComparing(P::age, Comparator.reverseOrder());
System.out.println(ps.stream().sorted(c).map(P::n).toList());
```

Output TERMINAL

```
[c, a, b]
```

City X first (c is older than a, so it comes first), then Y.

Q79. [advanced] Which stream operations are stateful, and what is their cost?

`sorted` must see every element before emitting the first; `distinct` and `limit`/`skip` (on ordered streams) keep state. A pipeline with `sorted` cannot stream a huge source in constant memory. Prefer filtering before sorting, and `IntStream`/`LongStream`/`DoubleStream` for numeric work to avoid boxing: `IntStream.rangeClosed(1, 5).sum()` allocates no `Integer` objects.

Q80. [advanced] Why avoid side effects in stream operations?

Lambdas in `map`, `filter` and friends should be pure: no mutation of outside state. Side effects break under parallelism, depend on evaluation order that the spec does not promise (for example, `peek` may be skipped if the terminal operation can compute the result without it, such as `count()` on a sized source), and make code hard to read. Use `collect` or `reduce` to accumulate instead of `forEach` plus an outside list. Reserve `peek` for debugging.

INTERVIEW ANGLE

"Explain how HashMap handles collisions and what happens when it gets full." Hash spreading chooses a bucket; nodes with the same bucket form a linked list; once a bucket has 8 or more nodes and the table has at least 64 buckets it becomes a red-black tree, and it reverts at 6. "Full" means exceeding capacity times load factor (0.75): the table doubles, and entries are redistributed with one bit test each. Mention that a good `hashCode` and immutable keys keep operations O(1) on average, and that a pre-sized map (`new HashMap<>(expected / 0.75 + 1)`, or `HashMap.newHashMap(n)` since Java 19) avoids rehashing.

REAL WORLD

Spring Data repositories return `Optional<T>` and `Stream<T>` (which must be closed). Controllers map entities to records with streams, group report rows with `groupingBy`, and cache with `ConcurrentHashMap.computeIfAbsent` or Caffeine, whose eviction policy is an improved LRU.

Check yourself

1. What does this print? `List<Integer> l = new ArrayList<>(List.of(10, 20, 30)); l.remove(1); System.out.println(l);`
2. Why does `new HashSet<>(List.of("a","a"))` throw nothing, while `List.of("a","a").stream().collect(Collectors.toMap(s -> s, s -> 1))` throws?
3. You need a thread-safe map with atomic "insert if absent". Which class and method?
4. Why can `List<Object> xs = new ArrayList<String>();` not compile?
5. What is printed? `Stream.of(1, 2, 3).peek(System.out::print);` (with nothing after it)
6. Give one reason a parallel stream may be slower than a sequential one.

Answers

1. `[10, 30]`: `remove(1)` is `remove(int index)`, so it removes the element at index 1, which is 20.
2. A set silently ignores duplicates (`add` returns false). `toMap` has no merge function, so a duplicate key raises `IllegalStateException`.
3. `ConcurrentHashMap.computeIfAbsent` (or `putIfAbsent`).
4. Generics are invariant: `ArrayList<String>` is not an `ArrayList<Object>`, otherwise you could add an `Integer` through the `Object` view. Use `List<?>` or `List<? extends Object>` for read-only access.
5. Nothing: there is no terminal operation, so the pipeline never runs.
6. Small data, splitting and merging overhead, a blocking task that starves the common pool, or a poorly splittable source.

KEY POINTS

Collections questions are about mechanics; streams questions are about laziness and purity.

- Pick `ArrayList`, `HashMap`, `ArrayDeque` by default; know why `LinkedList` rarely wins.
- HashMap: spread hash, power-of-two table, load factor 0.75, treeify at 8 (table 64+), untreeify at 6.
- Keys must be immutable with consistent `equals`/`hashCode`; fail-fast iterators throw `ConcurrentModificationException`.
- Generics are compile-time only (erasure); use PECS for wildcards.
- Streams are lazy, single-use and should be side-effect free; collect rather than mutate.

14.4 Interview Q&A: Concurrency, the Memory Model and the JVM

Concurrency and JVM internals are where interviews separate candidates, because the answers cannot be memorised from syntax: they require a mental model. The forty questions below use experiments run on JDK 21. Where a result depends on timing it is marked typical.

14.4.1 Concurrency and the Java Memory Model

Q81. [basic] Process versus thread?

A process is an isolated program instance with its own address space. Threads are execution paths inside a process that share its heap and open resources but have their own stack and program counter. Sharing makes communication cheap and bugs easy: shared mutable state needs synchronisation.

Q82. [basic] How can you create a thread? Runnable versus Callable?

Implement `Runnable` (or pass a lambda) to a `Thread`, subclass `Thread` (rarely good), or, best, submit tasks to an `ExecutorService`. `Runnable.run()` returns nothing and cannot throw checked exceptions; `Callable<V>.call()` returns a value and may throw, and is submitted to an executor, which returns a `Future<V>`. Separate the task from how it is run.

Q83. [basic] start() versus run()?

`start()` asks the JVM to create a new thread, which then calls `run()`. Calling `run()` directly just executes the method on the current thread; no concurrency. Calling `start()` twice throws `IllegalThreadStateException`.

Q84. [intermediate] List the thread states.

`NEW` (created, not started), `RUNNABLE` (running or ready; includes blocked in native I/O), `BLOCKED` (waiting to enter a `synchronized` block), `WAITING` (`wait()`, `join()`, `park()` with no timeout), `TIMED_WAITING` (`sleep`, timed `wait`/`join`), `TERMINATED`.

Observing states JAVA

```java
Thread t = new Thread(() -> { try { Thread.sleep(200); } catch (InterruptedException e) { } });
System.out.println(t.getState());
t.start(); Thread.sleep(50);
System.out.println(t.getState());
t.join();
System.out.println(t.getState());
```

Output TERMINAL

```
NEW
TIMED_WAITING
TERMINATED
```

Q85. [basic] What does synchronized do? What is locked?

It acquires the *monitor* of an object, giving mutual exclusion and also memory visibility (release flushes, acquire sees earlier writes). On an instance method the lock is `this`; on a static method it is the `Class` object; on a block it is the object you name. Two threads using *different* lock objects do not exclude each other. The bytecode uses `monitorenter` and `monitorexit` (the latter on both normal and exceptional exits).

Q86. [intermediate] What does volatile guarantee and not guarantee?

It guarantees visibility (a write is seen by subsequent reads in other threads) and ordering (no reordering across the volatile access: a write happens-before a later read of the same variable). It does **not** make compound actions atomic: `count++` is read, add, write.

Q87. [intermediate] Demonstrate a race condition and the fixes.

Two threads each increment four counters 100,000 times.

Four counters, two threads JAVA

```java
static int plain; static volatile int vol; static int sync;
static final AtomicInteger at = new AtomicInteger();
static synchronized void incSync() { sync++; }
// each thread: for 100_000 iterations { plain++; vol++; incSync(); at.incrementAndGet(); }
```

Result after both threads join (typical; plain and vol are racy, so their totals vary) TERMINAL

```
vol  : below 200000
sync : 200000
atomic: 200000
```

In the run on the build machine `vol` lost updates, while `plain` happened to reach 200000: a race may stay hidden, which is exactly why you reason from the memory model rather than from tests. Fixes: `synchronized`, a lock, `AtomicInteger`/`LongAdder` (lock-free CAS), or avoid sharing.

Q88. [advanced] What is the Java Memory Model and "happens-before"?

The JMM defines when a write by one thread is guaranteed visible to another. Without ordering, compilers and CPUs may reorder and cache. *Happens-before* edges create guarantees: program order within a thread; unlock of a monitor before every later lock of it; volatile write before every later volatile read; `Thread.start()` before the started thread's actions; a thread's actions before `join()` returns; writes to a `final` field in a constructor are visible once the reference is safely published. If there is no happens-before between a write and a read, it is a data race.

Q89. [intermediate] wait/notify versus sleep versus join?

`sleep(ms)` pauses the current thread and keeps any locks. `wait()` must be called while holding the object's monitor, releases it, and waits until `notify`/`notifyAll` (always inside a `while` loop re-checking the condition, because of spurious wakeups). `join()` waits for another thread to finish. Prefer higher-level tools: `BlockingQueue`, `CountDownLatch`, `Condition`.

Q90. [intermediate] What is a deadlock? How do you prevent it?

Threads wait on each other's locks forever. Four conditions hold together: mutual exclusion, hold-and-wait, no preemption, circular wait. Break one: acquire locks in a global order (breaks circular wait), use `tryLock` with timeout, hold fewer locks, or use higher-level structures. Diagnose with `jstack <pid>`, `jcmd <pid> Thread.print` (both print "Found one Java-level deadlock") or `ThreadMXBean.findDeadlockedThreads()`.

Provoking a deadlock (lock order l1,l2 versus l2,l1) JAVA

```java
Thread d1 = new Thread(() -> { synchronized (l1) { sleep(100); synchronized (l2) { } } });
Thread d2 = new Thread(() -> { synchronized (l2) { sleep(100); synchronized (l1) { } } });
```

Output after 400 ms: both threads BLOCKED; findDeadlockedThreads() reports 2 TERMINAL

```
BLOCKED BLOCKED
deadlocked=2
```

Q91. [intermediate] synchronized versus ReentrantLock?

Both are reentrant mutexes. `ReentrantLock` adds `tryLock()` (with timeout), interruptible acquisition, optional fairness, multiple `Condition`s, and `ReadWriteLock` variants; you must `unlock()` in `finally`. `synchronized` is simpler, cannot leak a lock, and has been heavily optimised. Use `synchronized` unless you need those features.

Q92. [intermediate] Explain ThreadPoolExecutor parameters. Why is Executors.newFixedThreadPool risky?

Parameters: core size, max size, keep-alive, work queue, thread factory, rejection policy. Behaviour: tasks go to new threads up to core size; then into the queue; only when the queue is full are threads added up to max; when both are full the handler rejects. `newFixedThreadPool` uses an *unbounded* queue, so a producer outrunning consumers eats memory until `OutOfMemoryError`; `newCachedThreadPool` can create unbounded threads. In production, construct a pool with a bounded queue and a deliberate rejection policy, and name your threads. Always `shutdown()`.

Q93. [intermediate] How do exceptions in tasks reach the caller? What is CompletableFuture for?

`Future.get()` rethrows a task's exception wrapped in `ExecutionException`; if you never call `get`, the failure is silent. `CompletableFuture` composes asynchronous stages without blocking: `supplyAsync`, `thenApply`, `thenCompose` (flatMap), `thenCombine`, `allOf`, `exceptionally`, `handle`. By default async stages run on the common `ForkJoinPool`; pass your own executor for blocking work.

Exception travel and a pipeline JAVA

```java
ExecutorService pool = Executors.newFixedThreadPool(2);
Future<Integer> fu = pool.submit(() -> { throw new IllegalStateException("boom"); });
try { fu.get(); } catch (ExecutionException e) { System.out.println(e.getCause()); }
pool.shutdown();
System.out.println(CompletableFuture.supplyAsync(() -> 6).thenApply(x -> x * 7).get());
```

Output TERMINAL

```
java.lang.IllegalStateException: boom
42
```

Q94. [intermediate] How do ConcurrentHashMap and CopyOnWriteArrayList work?

`ConcurrentHashMap` (since Java 8) uses lock-free reads, CAS to insert into an empty bin and a lock on the first node of a bin for other writes, so writers on different bins do not contend. Size is computed from striped counters. `CopyOnWriteArrayList` copies the whole array on each write and gives iterators a snapshot: excellent for read-mostly listener lists, terrible for write-heavy use. Neither supports a compound check-then-act across calls; use `computeIfAbsent` or `merge`.

Q95. [intermediate] When do you use CountDownLatch, CyclicBarrier, Semaphore?

`CountDownLatch`: one or more threads wait until N events complete; single use. `CyclicBarrier`: N threads wait for each other at a point; reusable. `Semaphore`: limit concurrent access to a resource with N permits (connection throttling). `Phaser` generalises barriers.

Q96. [basic] How do you implement producer-consumer?

Use a `BlockingQueue` (`ArrayBlockingQueue` bounded, `LinkedBlockingQueue`): producers `put` (blocks when full), consumers `take` (blocks when empty). A bounded queue provides back-pressure. Shutdown uses a poison pill or interruption. The hand-rolled `wait`/`notifyAll` version works but is easy to get wrong.

Q97. [intermediate] How do you stop a thread properly?

There is no safe forced stop (`Thread.stop` was removed from use and now throws `UnsupportedOperationException`). Use cooperative cancellation: `interrupt()` sets the interrupt flag, blocking methods such as `sleep` and `wait` throw `InterruptedException` and **clear** the flag. Handle it by either propagating the exception or restoring the flag with `Thread.currentThread().interrupt()`. Never swallow it silently.

Interrupt clears the flag JAVA

```java
Thread it = new Thread(() -> {
    try { Thread.sleep(5000); }
    catch (InterruptedException e) {
        System.out.println("interrupted, flag=" + Thread.currentThread().isInterrupted());
    }
});
it.start(); it.interrupt(); it.join();
```

Output TERMINAL

```
interrupted, flag=false
```

Q98. [advanced] What is ThreadLocal, and what is the leak risk?

It gives each thread its own copy of a value (per-request context, non-thread-safe helpers). In a pool, threads outlive requests, so a value set and never `remove()`d stays reachable through the thread (and can pin a classloader in app servers). Always `remove()` in `finally`. Virtual threads support `ThreadLocal` too, but with millions of threads prefer explicit parameters, or `ScopedValue` (final in Java 25, not run here).

Q99. [intermediate] What are virtual threads (Java 21), and when do they help?

Virtual threads are lightweight threads scheduled by the JVM onto a small pool of carrier (platform) threads. When one blocks on I/O or a lock, it unmounts and the carrier runs another. They let you write simple blocking code, one thread per request, at high concurrency. They do not speed up CPU-bound work. Do not pool them; create one per task with `Executors.newVirtualThreadPerTaskExecutor()` or `Thread.ofVirtual().start(...)`. Before Java 24, blocking inside `synchronized` pinned the carrier; JEP 491 (Java 24) removed that for monitors.

One virtual thread per task JAVA

```java
try (var ex = Executors.newVirtualThreadPerTaskExecutor()) {
    Future<String> f = ex.submit(() -> "virtual=" + Thread.currentThread().isVirtual());
    System.out.println(f.get());
}
```

Output TERMINAL

```
virtual=true
```

Q100. [advanced] Write a thread-safe lazy singleton.

The simplest correct options are an eager `static final` instance, an enum with one constant, or the holder idiom, which relies on class initialisation being thread-safe and lazy. Double-checked locking is correct only with `volatile`, because without it another thread may see a non-null reference to a partially constructed object.

Holder idiom and double-checked locking JAVA

```java
final class Registry {
    private Registry() { }
    private static class Holder { static final Registry INSTANCE = new Registry(); }
    static Registry get() { return Holder.INSTANCE; }          // lazy, thread-safe, no locks
}
```

Holder idiom and double-checked locking (continued) JAVA

```
final class Lazy {
    private static volatile Lazy instance;                    // volatile is essential
    static Lazy get() {
        Lazy r = instance;
        if (r == null) {
            synchronized (Lazy.class) {
                r = instance;
                if (r == null) instance = r = new Lazy();
            }
        }
        return r;
    }
}
```

Q101. [advanced] How do atomics work? What is the ABA problem?

`AtomicInteger` and friends use compare-and-set (CAS), one hardware instruction: set the value only if it still equals what I read; otherwise retry. No lock, no blocking, but heavy contention causes many retries; `LongAdder` spreads updates across cells and sums on read, better for counters. ABA: a value changes A to B and back to A, so CAS succeeds although things changed. `AtomicStampedReference` adds a version stamp.

Q102. [intermediate] Concurrency versus parallelism? What makes a class thread-safe?

Concurrency is dealing with many tasks at once (interleaving, possible on one core); parallelism is executing simultaneously on several cores. A class is thread-safe if it behaves correctly under any interleaving without callers adding synchronisation. The strongest strategies, in order: no shared state, immutable state, confinement to one thread, then proper locking or atomics. Document the guarantee (`@ThreadSafe` or Javadoc).

14.4.2 JVM, memory and garbage collection

Q103. [basic] What are the JVM runtime memory areas?

Per JVM: the **heap** (all objects and arrays, split into young and old generations in generational collectors), **Metaspace** (class metadata, in native memory since Java 8, replacing PermGen), the **code cache** (JIT-compiled code), plus direct buffers and other native memory. Per thread: a **stack** (frames with locals and operand stack), a **program counter**, and a native method stack. Heap and Metaspace are shared; stacks are private.

Q104. [basic] What lives on the stack and what on the heap?

Local primitives and object *references* live in stack frames; every object lives on the heap, wherever the reference is. Each method call pushes a frame, popped on return. Stack memory is reclaimed automatically when the method returns, so it needs no GC. The JIT can sometimes avoid allocating a non-escaping object altogether (escape analysis, scalar replacement), but you cannot rely on it.

Q105. [intermediate] Explain class loading and the delegation model.

Loading finds the bytes of a class; linking verifies them, prepares static fields with default values and resolves symbols; initialisation runs static initialisers, once, when the class is first actively used. Loaders form a hierarchy: bootstrap (core `java.base`), platform, application (your classpath), and custom ones. A loader first *delegates to its parent* and only loads itself if the parent fails. This prevents user code from replacing `java.lang.String` and keeps core types unique. Two classes with the same name from different loaders are different types.

Q106. [intermediate] When does a class get initialised?

On first `new`, first static method call, first non-constant static field access, `Class.forName` (default), or initialisation of a subclass. Compile-time constants (`static final int X = 5`) are inlined and do not trigger it. A static initialiser that throws leaves the class unusable: `ExceptionInInitializerError` the first time, `NoClassDefFoundError` afterwards.

Q107. [basic] How does the garbage collector decide an object is garbage?

By *reachability*, not by reference counting. The collector starts from GC roots (live thread stacks, static fields, JNI references) and marks everything reachable; the rest is garbage. Cycles of unreachable objects are collected. Objects do not need to be set to `null` in normal code; the common exception is clearing slots in your own array-based structures (a stack that does not null popped slots leaks).

Q108. [intermediate] What is the generational hypothesis? How does a young collection work?

Most objects die young. The heap is divided into a young generation (eden plus two survivor spaces) and an old generation. New objects are allocated in eden. A *minor* collection copies live objects from eden and one survivor to the other survivor (cheap, since most is dead); after surviving a number of collections (the tenuring threshold, at most 15) they are promoted to old. The old generation is collected less often, by marking and compaction or concurrent cycles. Big pauses come from old-generation or full collections.

Q109. [intermediate] Which collectors exist, and what is the default?

Serial (single thread, tiny heaps), Parallel (throughput, multi-threaded stop-the-world), **G1** (region-based, pause-time goal, the default on server-class machines since Java 9), ZGC (concurrent, pauses in the sub-millisecond range, large heaps; generational and the default ZGC mode since Java 23) and Shenandoah (concurrent, in OpenJDK builds). The JVM picks Serial instead of G1 when it sees fewer than 2 CPUs or less than about 1.8 GB of memory; Java 27 makes G1 the default everywhere (JEP 523). Select with `-XX:+UseG1GC`, `-XX:+UseZGC`, `-XX:+UseParallelGC`.

Which collector did my JVM choose? (2 CPUs, 8 GB build machine, JDK 21) JAVA

```
// run:  java -Xlog:gc -Xmx64m Program     and the first line says:
// [0.012s][info][gc] Using G1
//
// a young collection later in the same log (typical; times vary):
// [1.642s][info][gc] GC(0) Pause Young (Normal) (G1 Evacuation Pause) 7M->3M(64M) 9.114ms
```

Read a log line as: collection number, type of pause, cause, heap used before to after (and committed size), pause duration.

Q110. [intermediate] What reference types exist?

Strong (ordinary), **soft** (cleared only when memory is needed; suited to memory-sensitive caches), **weak** (cleared at the next GC once the object is not strongly reachable; used by `WeakHashMap`), **phantom** (for post-mortem cleanup via `Cleaner`). A weak reference is cleared promptly; a soft one is kept while memory allows.

Weak versus soft (after System.gc(), typical on HotSpot) JAVA

```
WeakReference<Object> w = new WeakReference<>(new Object());
Object o = new Object(); SoftReference<Object> s = new SoftReference<>(o); o = null;
System.gc();
System.out.println("weak cleared=" + (w.get() == null) + ", soft kept=" + (s.get() != null));
```

Output TERMINAL

```
weak cleared=true, soft kept=true
```

Q111. [intermediate] Can Java leak memory? Give examples.

Yes: not through lost pointers, but through objects that stay reachable and are never used again. Typical causes: static collections or caches that only grow; listeners or callbacks never deregistered; `ThreadLocal` values in pooled threads; mutable keys left in maps; unclosed resources (streams, connections); inner classes holding their outer instance; classloader leaks on redeploy. Diagnose with a heap dump (`jcmd <pid> GC.heap_dump file.hprof`) in a tool such as Eclipse MAT or VisualVM, and look at the dominator tree and GC roots.

Q112. [basic] Which OutOfMemoryError messages exist, and what does each mean?

`Java heap space`: the heap is full of live objects (leak or heap too small). `GC overhead limit exceeded`: the JVM spends almost all time in GC with little reclaimed. `Metaspace`: too many classes (leaks of dynamic classes, classloader leak). `unable to create native thread`: the OS refused a thread (too many threads or stack memory). `Direct buffer memory`: off-heap NIO buffers exhausted. `StackOverflowError` is a different error: recursion too deep for the thread's stack.

Provoking two errors (run with -Xmx32m) JAVA

```
try { rec(); } catch (StackOverflowError e) { /* depth in the thousands */ }
try { List<byte[]> l = new ArrayList<>(); while (true) l.add(new byte[1_000_000]); }
catch (OutOfMemoryError e) { System.out.println(e.getMessage()); }
```

Output of the second catch TERMINAL

```
Java heap space
```

Add `-XX:+HeapDumpOnOutOfMemoryError -XX:HeapDumpPath=/tmp` in production so you get evidence.

Q113. [intermediate] How does the JIT compiler work?

The JVM starts by interpreting bytecode and counts invocations and loop back-edges. Hot methods are compiled to machine code by tiered compilation: C1 (quick, light optimisation, with profiling) and C2 (slow to compile, aggressive optimisations based on the profile: inlining, escape analysis, loop unrolling, devirtualisation). Speculation can be undone by *deoptimisation* when an assumption fails (a new subclass is loaded). This is why Java needs warm-up and why micro-benchmarks need JMH. Code is stored in the code cache (default 240 MB on this build).

Q114. [basic] Which JVM flags matter most for memory?

`-Xms` and `-Xmx` (initial and maximum heap; setting them equal avoids resizing pauses), `-Xss` (thread stack size, default 1 MB on 64-bit Linux), `-XX:MaxMetaspaceSize`, `-XX:MaxRAMPercentage` (container-friendly sizing), `-XX:+UseG1GC`/`-XX:+UseZGC`, `-XX:MaxGCPauseMillis` (G1 target, default 200), `-Xlog:gc*`, `-XX:+HeapDumpOnOutOfMemoryError`. On the build machine (8 GB RAM) the default maximum heap was about 1.5 GB, one quarter of physical memory; `-XX:+PrintFlagsFinal` shows every effective value.

Default sizing on the build machine (JDK 21, 8 GB RAM, 2 CPUs) TERMINAL

```
$ java -XX:+PrintFlagsFinal -version | grep -E "MaxHeapSize|ThreadStackSize |MaxTenuringThreshold"
   size_t MaxHeapSize                              = 1568669696      {product} {ergonomic}
    uintx MaxTenuringThreshold                     = 15              {product} {default}
     intx ThreadStackSize                          = 1024            {pd product} {default}
```

Q115. [intermediate] A production service is slow or hung. What do you do?

Gather evidence before changing anything. `jcmd <pid> Thread.print` or `jstack` shows what threads are doing (deadlocks, a pool exhausted, everything waiting on I/O). `jstat -gc <pid> 1000` and GC logs show GC pressure. `jcmd <pid> GC.heap_info` and a heap dump show memory. Java Flight Recorder (`-XX:StartFlightRecording` or `jcmd <pid> JFR.start`) records CPU, allocation, locks and I/O with low overhead. Take several thread dumps a few seconds apart; one dump is a photograph, several show movement.

Q116. [intermediate] What are compact strings and string deduplication?

Since Java 9, `String` stores its characters in a `byte[]` plus a coder flag: Latin-1 text uses one byte per character, other text two (UTF-16). Roughly halves memory for typical text. G1 can additionally deduplicate identical backing arrays of strings (`-XX:+UseStringDeduplication`, off by default). `intern()` shares strings through the pool, but a pool grown unboundedly is a leak risk.

Q117. [advanced] How big is an object?

An object has a header (mark word and class pointer; about 12 bytes with compressed class pointers on 64-bit HotSpot), then fields, padded to a multiple of 8 bytes. So `new Object()` is 16 bytes and an `Integer` is 16. Compressed oops (references as 32-bit offsets) are on by default for heaps under about 32 GB. Compact object headers (`-XX:+UseCompactObjectHeaders`, product feature in Java 25, on by default in Java 27 per JEP 534) shrink the header to 8 bytes. Use the JOL library to inspect layouts. Arrays add a 4-byte length.

Q118. [basic] What is Metaspace, and how does it differ from PermGen?

PermGen was a fixed-size region of the heap holding class metadata and interned strings (before Java 7), prone to `OutOfMemoryError: PermGen space`. Java 8 replaced it with Metaspace in native memory, which grows on demand and is limited only by `-XX:MaxMetaspaceSize` (unlimited by default) or the OS. Interned strings moved to the heap in Java 7. Static fields' values live in the heap, with the class's mirror object.

Q119. [intermediate] "Java Memory Model" versus "JVM memory structure": are they the same?

No, and interviewers like to check. The runtime memory structure (heap, stacks, Metaspace) describes where data lives. The Java Memory Model (JLS chapter 17) defines the rules for visibility and ordering of reads and writes between threads. Mixing them up is a common red flag.

Q120. [intermediate] Should you call System.gc() or use finalizers?

No. `System.gc()` is only a hint (it triggered a full pause in the log above: `Pause Full (System.gc())`) and can hurt latency; `-XX:+DisableExplicitGC` ignores it. Finalizers are deprecated for removal: unpredictable timing, resurrection and performance cost. For cleanup use try-with-resources and `AutoCloseable`; for a safety net use `java.lang.ref.Cleaner`.

INTERVIEW ANGLE

"Walk me through what happens when you run `new Order()` in terms of memory." The class is loaded and initialised if this is the first use (Metaspace holds its metadata). The JVM allocates space for the object in eden, normally by bumping a thread-local allocation buffer pointer (TLAB), which is why allocation is fast. Header and fields are zeroed, then the constructor chain runs. The reference is stored in a stack slot or a field. When no GC root can reach the object it becomes garbage and is reclaimed in a young collection, or promoted to old if it survives enough collections.

REAL WORLD

Containerised Spring Boot services should set heap relative to the container (`-XX:MaxRAMPercentage=75`), enable GC logging and heap-dump-on-OOM, expose thread and heap metrics through Micrometer, and keep a JFR recording available. A pod killed by the container runtime without a Java OOM usually means total native memory (Metaspace, thread stacks, direct buffers) exceeded the limit, not the heap alone.

Check yourself

1. What is printed? `Object lock = new Object(); synchronized (lock) { lock.wait(); }` with no other thread, and what is the thread state while it waits?
2. A `volatile int count` is incremented by ten threads, 1000 times each. Is the final value guaranteed to be 10000?
3. A thread catches `InterruptedException` and does nothing. What is wrong?
4. A cache `static Map<String, Report>` grows forever. Which `OutOfMemoryError` message do you expect, and which tool finds the cause?
5. Why does `-Xmx` not cap the total memory the Java process uses?
6. What does the JMM guarantee when thread A calls `t.start()` and the new thread reads a variable A wrote earlier?

Answers

1. It prints nothing and blocks forever; the state is `WAITING`. (`wait()` requires holding the monitor, so it does not throw `IllegalMonitorStateException` here.)
2. No. `volatile` gives visibility, not atomicity of `count++`; use `AtomicInteger` or a lock.
3. The interrupt is lost, because catching clears the flag; the thread cannot be asked to stop. Re-set the flag or propagate.
4. `java.lang.OutOfMemoryError: Java heap space`; take a heap dump and read the dominator tree in MAT or VisualVM.
5. `-Xmx` limits only the Java heap; Metaspace, thread stacks, code cache, direct buffers and JVM internals use native memory on top.
6. `start()` happens-before everything the new thread does, so it sees A's earlier writes.

KEY POINTS

Concurrency and JVM answers must come from models, not memorised slogans.

- Shared mutable state needs happens-before: locks, volatile, atomics, safe publication; volatile is not atomic.
- Prefer executors, bounded queues and concurrent collections to raw threads; handle interrupts and exceptions deliberately.
- Virtual threads make blocking cheap; they do not make CPU work faster.
- Reachability, not reference counts, defines garbage; leaks are reachable-but-unused objects.
- Know the areas (heap, Metaspace, stacks), G1/ZGC basics, and the commands: `jcmd`, `jstack`, `jstat`, heap dump, JFR.

14.5 Interview Q&A: Modern Java

Interviewers use this group to find out whether you have kept up with the language and whether you can design code that other people can live with. For the modern-Java questions, always say *which release* finalised a feature; that single habit separates people who read the release notes from people who memorised a list. Questions continue the numbering of the previous chapters and are not repeated from them.

14.5.1 Language and library features, Java 8 to 25

Q121. [basic] Which Java versions are LTS, and which one should a new project use?

Long-term-support releases are 8, 11, 17, 21 and 25 (September 2025); since Java 17 a new LTS arrives every two years, and a non-LTS feature release appears every March and September. A new project should target the newest LTS that its frameworks support, today Java 25 (or 21 if a dependency lags). Non-LTS releases are supported for six months only, so they suit experiments and early feedback, not long-lived production services. Java 26 (March 2026) and 27 (September 2026) are such releases.

Q122. [basic] What are records, and what do you get for free?

A record is a transparent, shallowly immutable data carrier (final in Java 16). From the component list the compiler generates a private final field per component, a canonical constructor, accessors named like the components (`r.lo()`, not `getLo()`), and `equals`, `hashCode` and `toString` based on all components. A record cannot extend a class and cannot declare instance fields beyond its components, but it can implement interfaces, have static members and extra constructors.

Compact constructor validates; equals is by value JAVA

```
record Range(int lo, int hi) {
    Range {
        if (lo > hi) throw new IllegalArgumentException("lo > hi: " + lo + " > " + hi);
    }
}
// new Range(5, 1)                -> IllegalArgumentException: lo > hi: 5 > 1
// new Range(1, 2).equals(new Range(1, 2))   -> true, with equal hashCodes
// new Range(1, 2).toString()     -> Range[lo=1, hi=2]
```

Q123. [intermediate] Are records really immutable?

Only shallowly. The fields are final, but a component that refers to a mutable object can still be changed through that object. A record holding a `List` shows later additions to the caller's list. A record holding an array also compares arrays by reference in the generated `equals`. Defend in the compact constructor: `this.items = List.copyOf(items)` (a compact constructor assigns the parameters, so write `items = List.copyOf(items);`).

Shallow immutability and array components (run on JDK 21) JAVA

```
record Names(List<String> list) {}
List<String> src = new ArrayList<>(List.of("a"));
Names n = new Names(src);
src.add("b");
System.out.println(n);                                  // Names[list=[a, b]]
record P(int[] a) {}
System.out.println(new P(new int[]{1}).equals(new P(new int[]{1})));   // false
```

Q124. [basic] What is a sealed type, and why does it matter?

`sealed` (final in Java 17) restricts which classes may extend or implement a type, listed in `permits` (or inferred when all are in the same file). Each permitted subtype must be `final`, `sealed` or `non-sealed`. The payoff is that the compiler knows the complete set of subtypes, so a `switch` over the type can be checked for exhaustiveness without a `default`, and adding a new subtype turns every incomplete switch into a compile error. Sealed types plus records give you algebraic data types, the Java way to model "one of these shapes".

Q125. [intermediate] Explain pattern matching for switch and record patterns.

Pattern matching for `instanceof` (Java 16) tests and binds in one step: `if (o instanceof String s && s.length() > 3)`. Pattern matching for `switch` (final in Java 21) lets `case` labels be type patterns with optional `when` guards, and `case null` is allowed. Record patterns (also Java 21) deconstruct a record in the label and may nest: `case Rect(double w, double h) -> w * h`. A switch expression must be exhaustive; for a sealed hierarchy the compiler checks it. Order matters: a more general label before a more specific one is a compile error (dominance).

Sealed hierarchy, record patterns, guard (output from JDK 21) JAVA

```
sealed interface Shape permits Circle, Rect, Tri {}
record Circle(double r) implements Shape {}
record Rect(double w, double h) implements Shape {}
record Tri(double b, double h) implements Shape {}

static double area(Shape s) {
    return switch (s) {
        case Circle c -> Math.PI * c.r() * c.r();
        case Rect(double w, double h) -> w * h;
        case Tri t when t.b() <= 0 -> 0;
        case Tri(double b, double h) -> 0.5 * b * h;
    };
}
// area(new Circle(1)) = 3.14, area(new Rect(2,3)) = 6.0, area(new Tri(4,5)) = 10.0
```

No `default` is needed: remove the `Tri` cases and the code no longer compiles.

Q126. [intermediate] What does case null change in a switch?

Before Java 21 a `switch` on a reference threw `NullPointerException` for `null`. With pattern switches you may write `case null -> ...` (alone or as `case null, default`). Without it, `null` still throws, so old code keeps its behaviour. This makes "handle everything including null" a single expression.

Switch on Object with null and a guard (JDK 21 output) JAVA

```
static String describe(Object o) {
    return switch (o) {
        case null -> "null!";
        case Integer i when i > 10 -> "big int " + i;
        case Integer i -> "int " + i;
        case String s -> "string of " + s.length();
        case int[] a -> "int array " + a.length;
        default -> "other " + o.getClass().getSimpleName();
    };
}
// null, 5, 50, "hey", new int[3], 2.5  ->
// null!   int 5   big int 50   string of 3   int array 3   other Double
```

Q127. [basic] Switch expression versus switch statement?

A switch *expression* (Java 14) produces a value, uses `->` arms that do not fall through, and must be exhaustive; use `yield` to return a value from a block arm. The classic *statement* with `case X:` falls through unless you `break`, which is the source of many bugs. Prefer arrows even in statements.

yield inside a block arm JAVA

```
enum Level { LOW, HIGH }
Level l = Level.HIGH;
int v = switch (l) {
    case LOW -> 1;
    case HIGH -> { int x = 10; yield x * 2; }
};
System.out.println(v);   // 20
```

Q128. [basic] What are text blocks?

Multi-line string literals delimited by `"""` (Java 15). The compiler removes the common leading indentation (determined by the closing delimiter and the least-indented line), normalises line endings to `\n`, and allows quotes without escaping. A trailing backslash joins lines, and `\s` keeps a trailing space. They are ideal for JSON, SQL and HTML in code and tests.

A text block with a joined line JAVA

```
String json = """
    {
      "name": "Ada",\
     "x": 1
    }
    """;
// prints:
// {
//   "name": "Ada", "x": 1
// }
```

Q129. [basic] What does var do? When should you avoid it?

`var` (Java 10) infers a *local variable*'s static type from its initialiser; it is not dynamic typing and the type is still fixed at compile time. It works for locals, `for` variables and lambda parameters, but not for fields, parameters, return types or `var x = null`. Use it when the type is obvious from the right-hand side (`var users = new ArrayList<User>()`), avoid it when it hides information (`var x = service.process()`), and keep the diamond meaningful: `var list = new ArrayList<>()` infers `ArrayList<Object>`.

Q130. [intermediate] What are sequenced collections (Java 21)?

Three new interfaces, `SequencedCollection`, `SequencedSet` and `SequencedMap`, give every collection with a defined encounter order the same methods: `getFirst`, `getLast`, `addFirst`, `addLast`, `removeFirst`, `removeLast` and `reversed()` (a live view). `List`, `Deque`, `LinkedHashSet`, `LinkedHashMap` and `SortedSet`/`SortedMap` types implement them. Before this, getting the last element of a `List` was `list.get(list.size() - 1)` and of a `LinkedHashSet` required iteration.

Sequenced collections (JDK 21 output) JAVA

```
var sl = new ArrayList<>(List.of(1, 2, 3));
System.out.println(sl.getFirst() + " " + sl.getLast() + " " + sl.reversed());   // 1 3 [3, 2, 1]
var m = new LinkedHashMap<String, Integer>();
m.put("a", 1); m.put("b", 2);
System.out.println(m.firstEntry() + " " + m.lastEntry() + " " + m.reversed());
// a=1 b=2 {b=2, a=1}
```

Q131. [intermediate] Difference between List.of, Collections.unmodifiableList and Stream.toList()?

`List.of(...)` creates a truly immutable list: `add` throws `UnsupportedOperationException`, and `null` elements are rejected with `NullPointerException`. `Collections.unmodifiableList(src)` is only an unmodifiable *view*: you cannot change it through the view, but changes to `src` show through. `List.copyOf(src)` is a snapshot. `Stream.toList()` (Java 16) returns an unmodifiable list but, unlike `List.of`, allows `null` elements; `Collectors.toList()` gives an unspecified, in practice modifiable, `ArrayList`.

View versus copy (JDK 21 output) JAVA

```
List<String> src = new ArrayList<>(List.of("a", "b"));
var view = Collections.unmodifiableList(src);
var copy = List.copyOf(src);
src.add("c");
System.out.println(view + " vs " + copy);    // [a, b, c] vs [a, b]
// List.of(1, null) -> NullPointerException
```

Q132. [intermediate] What are virtual threads, and what do they not fix?

Finalised in Java 21 (JEP 444). A virtual thread is a `Thread` scheduled by the JVM instead of the OS: it is mounted on a small pool of carrier (platform) threads and unmounted when it blocks on I/O, so you can run hundreds of thousands of them. Use them for blocking, request-per-thread code: `Executors.newVirtualThreadPerTaskExecutor()` or `Thread.ofVirtual().start(task)`. They do not speed up CPU-bound work, should not be pooled (create one per task), and until Java 24 a virtual thread blocked inside `synchronized` pinned its carrier (JEP 491 removed that pinning in 24). Limit concurrency to scarce resources (a database) with a `Semaphore`, not with a pool size.

Q133. [advanced] What are scoped values and structured concurrency, and what is their status?

`ScopedValue` (final in Java 25, JEP 506) shares immutable data with a method call tree and its child threads in a bounded scope; it is a cheaper, safer replacement for most `ThreadLocal` use with virtual threads. Structured concurrency (`StructuredTaskScope`) treats a group of subtasks as one unit of work whose lifetime is bounded by a block, so failure and cancellation propagate; it is still a *preview* API in Java 25, and in the later releases (JEPs 525, 533), so it needs `--enable-preview` and its shape may still change. State the status exactly; do not call it final.

Q134. [intermediate] What do Optional and its misuse look like?

`Optional<T>` is a return type that says "there may be no result" and makes callers handle it with `map`, `flatMap`, `orElse`, `orElseGet`, `orElseThrow`, `ifPresentOrElse` and `or`. Misuse: as a field, as a method parameter, in collections, calling `get()` without checking (use `orElseThrow()`), or `Optional.of(x)` with a possible null (use `ofNullable`). `orElse(expr)` always evaluates `expr`; use `orElseGet(supplier)` when it is expensive.

Optional chaining (JDK 21 output) JAVA

```
Optional<String> op = Optional.of("x");
System.out.println(op.map(String::toUpperCase).orElse("none"));                // X
System.out.println(Optional.ofNullable((String) null)
        .or(() -> Optional.of("fallback")).get());                             // fallback
```

Q135. [intermediate] What useful String and Stream methods arrived after Java 8?

`String`: `isBlank`, `strip`, `lines`, `repeat` (11), `formatted`, `stripIndent` (15), `indent`, `transform`. `Stream`: `takeWhile`, `dropWhile`, the three-argument `iterate(seed, hasNext, next)` (9), `Stream.toList()` (16), `mapMulti` (16) and `Gatherers` with `gather` (final in Java 24, JEP 485) for custom intermediate operations such as windowing. `Optional`: `ifPresentOrElse`, `or`, `stream`, `isEmpty`.

Bounded iterate and the split pitfall (JDK 21 output) JAVA

```
System.out.println(Stream.iterate(1, i -> i < 50, i -> i * 3).toList());  // [1, 3, 9, 27]
System.out.println("a,b,,".split(",").length);        // 2 (trailing empties dropped)
System.out.println("  hi ".strip() + "|" + " ".isBlank() + "ab".repeat(2));  // hi|trueabab
```

Q136. [intermediate] How do you make an HTTP call without a library?

`java.net.http.HttpClient` (Java 11): build one client, reuse it, create an `HttpRequest`, and call `send` (blocking) or `sendAsync` (returns a `CompletableFuture`). It supports HTTP/2 by default and WebSocket; Java 26 adds HTTP/3 support (JEP 517). In services you normally use the framework's client (Spring `RestClient`/`WebClient`), but the JDK client is fine for small tools and tests.

Skeleton of a GET (not run: needs network) JAVA

```
HttpClient client = HttpClient.newHttpClient();
HttpRequest req = HttpRequest.newBuilder(URI.create("https://example.org/")).GET().build();
HttpResponse<String> res = client.send(req, HttpResponse.BodyHandlers.ofString());
System.out.println(res.statusCode());
```

Q137. [intermediate] What changed for running small programs?

Since Java 11 `java Hello.java` compiles and runs one source file without a separate `javac` step. Java 25 finalises compact source files and instance `main` methods (JEP 512): a file can contain a top-level `void main()` with no class, no `static` and no `String[] args`, plus the `IO.println` helper. Java 25 also finalises module import declarations (`import module java.base;`, JEP 511) and flexible constructor bodies (JEP 513), which allow statements before `super(...)` as long as they do not use `this`. These are for teaching, scripting and validation code; they do not change how real applications are structured.

Q138. [advanced] What are unnamed variables and patterns?

Final in Java 22 (JEP 456): the underscore `_` declares a variable or pattern you intend not to use, for example `catch (NumberFormatException _)`, `for (var _ : list)` or `case Rect(double w, _)`. It documents intent and the compiler will not warn about it. It is not allowed to refer to it later. This is a syntax feature, not a runtime one.

Q139. [advanced] What is new at the JVM level in Java 24 and 25 that a backend engineer should know?

Items that are confirmed: compact object headers (JEP 519, product feature in Java 25, enabled with `-XX:+UseCompactObjectHeaders`, shrinking the header from 12 to 8 bytes in typical configurations and reducing heap use; JEP 534 makes it the default in Java 27); virtual threads no longer pin on `synchronized` (24); AOT class loading and linking and AOT method profiling to speed start-up (24, 25); generational ZGC as the only ZGC mode (non-generational was removed in 24); the Security Manager removed for good (24). Say that you would measure on your own workload before adopting any of them.

Q140. [intermediate] Explain the Stream Collectors you use most.

`toList`, `toSet`, `toMap(key, value, merge)` (the two-argument form throws `IllegalStateException` on duplicate keys), `groupingBy(classifier, downstream)` for a `Map<K, List<V>>` or counts and sums, `partitioningBy(predicate)` for a two-key map, `joining`, `counting`, `summingInt`, `averagingDouble`, `mapping`, `teeing` (two collectors combined, Java 12). `Collectors.toUnmodifiableList` exists when you need immutability.

partitioningBy always has both keys (JDK 21 output) JAVA

```
System.out.println(IntStream.rangeClosed(1, 5).boxed()
        .collect(Collectors.partitioningBy(i -> i % 2 == 0)));
// {false=[1, 3, 5], true=[2, 4]}
```

Q141. [intermediate] What was removed or deprecated that candidates still mention?

`finalize()` is deprecated for removal (Java 18); use try-with-resources or `Cleaner`. The Security Manager was removed in Java 24. The Applet API was removed in Java 26. `Thread.stop`, `suspend` and `resume` are dead; the `Date`/`Calendar` classes remain but are replaced in new code by `java.time`. PermGen vanished in Java 8, Nashorn in 15, and CMS in 14. Knowing what is gone prevents recommending it in a design review.

Q142. [advanced] How would you adopt a new LTS in a large codebase?

Treat it as a controlled migration. Build and test on the new JDK in CI next to the old one; fix removed APIs and illegal reflective access (`--add-opens` flags as a short-term bridge); upgrade the build plugins, bytecode libraries (ASM, Byte Buddy) and frameworks first; read the release's compatibility notes; run the same load test and compare GC logs and latency; roll out gradually (canary) and keep an easy rollback. Adopt new language features only after the runtime change is stable, in separate commits.

Check yourself

1. What does `System.out.println("a,b,,".split(",").length);` print, and why?
2. `List<Integer> l = List.of(1, null);` What happens?
3. A record `P(int[] a)`: does `new P(new int[]{1}).equals(new P(new int[]{1}))` return true?
4. A `switch (shape)` over a sealed interface has all permitted cases and no `default`. A fifth subtype is added. What happens when you recompile?
5. What is the difference between `Collections.unmodifiableList(src)` and `List.copyOf(src)` after `src.add("x")`?
6. Which of these is final in Java 25: scoped values, structured concurrency, module imports?

Answers

1. `2`. `split` removes trailing empty strings; the result is `["a", "b"]`.
2. `NullPointerException`; `List.of` rejects null elements.
3. `false`. The generated `equals` compares components with `equals`, and arrays use reference equality.
4. The switch no longer compiles (not exhaustive), which is the point of sealed types.
5. The first reflects the new element (it is a view); the second does not (it is a snapshot).
6. Scoped values (JEP 506) and module imports (JEP 511) are final; structured concurrency is still a preview.

KEY POINTS

Modern Java questions reward precision about which release made what final.

- Records, sealed types and pattern matching together model data safely; the compiler enforces exhaustiveness.
- Records are only shallowly immutable; copy mutable components in the compact constructor.
- Virtual threads make blocking cheap but do not speed up CPU work; scoped values are final in 25, structured concurrency is still preview.
- Know the immutable-collection family (`List.of`, `copyOf`, `unmodifiableList`, `toList`) and what each allows.
- Plan LTS upgrades as a tested migration, not a version bump.

14.6 Interview Q&A: Design and Best Practices

The last group of questions has no single correct answer, so interviewers listen for reasoning: what you optimise for, what it costs, and how you would test it. Give a rule, a reason, and an exception. Use the small domain below (orders and notifications) for your own examples when you answer aloud.

14.6.1 Object-oriented design and SOLID

Q143. [basic] State the SOLID principles in one sentence each.

Single responsibility: a class has one reason to change. **O**pen/closed: extend behaviour by adding code (new implementations), not by editing tested code. **L**iskov substitution: a subtype must be usable wherever its supertype is expected without surprising the caller (same contracts, no stronger preconditions, no weaker postconditions). **I**nterface segregation: many small, client-specific interfaces beat one fat interface. **D**ependency inversion: high-level policy depends on abstractions, and details (databases, email) implement them. These are heuristics for managing change, not laws.

Q144. [intermediate] Give a Liskov violation in Java.

The classic one: `Square extends Rectangle` and overrides `setWidth` to also change the height; code that sets width and height independently and then asserts the area breaks. A real example from the JDK: `List.of(...)` is a `List` that throws `UnsupportedOperationException` from `add`, which is why the `List` contract allows optional operations and why you should document or encode mutability in types. Fix by modelling the true relationship (a shared `Shape` interface), or prefer composition.

Q145. [intermediate] What is dependency injection, and do you need a framework?

Dependency injection means an object receives its collaborators from outside (usually through the constructor) instead of creating them with `new`. That makes the class depend on an interface and lets tests pass a fake. You do not need a framework to do it; Spring, Guice or Jakarta CDI just automate wiring, scoping and lifecycle. Prefer constructor injection: dependencies are `final`, mandatory and visible, and the object is never half-built.

Constructor injection with a test double (JDK 21 output) JAVA

```
interface Notifier { void send(String to, String msg); }
record EmailNotifier() implements Notifier {
    public void send(String to, String m) { System.out.println("email " + to + ": " + m); }
}
static class OrderService {
    private final Notifier notifier;
    OrderService(Notifier notifier) { this.notifier = Objects.requireNonNull(notifier); }
    void place(String user) { notifier.send(user, "order placed"); }
}
// new OrderService(new EmailNotifier()).place("ada");           -> email ada: order placed
// new OrderService((to, m) -> System.out.println("test double got " + m)).place("bob");
//                                                               -> test double got order placed
// new OrderService(null)  -> NullPointerException at construction (fail fast)
```

Q146. [intermediate] Composition or inheritance, and what is the fragile base class problem?

Prefer composition. A subclass depends on how its parent calls its own overridable methods, so a harmless-looking change in the parent can break children (for example `HashSet.addAll` calling `add`, which a counting subclass also overrides, double counts). If you design a class for inheritance, document the self-use of overridable methods; otherwise make it `final` or give it a non-public constructor. Inheritance is right for a true is-a relationship with a stable, designed-for-extension parent.

Q147. [intermediate] How do you make a class immutable?

Declare the class `final` (or use a record), make all fields `private final`, set them only in the constructor, provide no setters, make defensive copies of mutable inputs *and* outputs (`List.copyOf`, `clone` of arrays, `Date` avoided), and do not leak `this` from the constructor. Benefits: thread safety without locks, safe as map keys, simple reasoning, safe caching. Cost: allocate a new object per change, which is usually cheap.

An immutable value class (JDK 21 output) JAVA

```java
static final class Money {
    private final long cents;
    private final String cur;
    Money(long c, String cur) { this.cents = c; this.cur = cur; }
    Money plus(Money o) { return new Money(cents + o.cents, cur); }
    @Override public String toString() {
        return cents / 100 + "." + String.format("%02d", cents % 100) + " " + cur;
    }
}
// new Money(1050, "EUR").plus(new Money(295, "EUR")) -> 13.45 EUR
```

Money never stores amounts as `double`; `long` cents or `BigDecimal` avoids rounding errors.

Q148. [intermediate] When do you use the Builder pattern?

When a type has many optional parameters or must be validated as a whole before construction. A builder gives named, readable calls, lets the target be immutable, and builds only when `build()` is called. The telescoping-constructor alternative is unreadable at five arguments. For a record with few components the canonical constructor is enough, and a static factory method with a good name (`Money.of`, `List.of`) often beats a builder.

Builder producing an immutable object (JDK 21 output) JAVA

```java
static class Pizza {
    private final String size;
    private final List<String> toppings;
    private Pizza(Builder b) { size = b.size; toppings = List.copyOf(b.toppings); }
    static Builder builder() { return new Builder(); }
    static class Builder {
        private String size = "M";
        private final List<String> toppings = new ArrayList<>();
        Builder size(String s) { size = s; return this; }
        Builder top(String t) { toppings.add(t); return this; }
        Pizza build() { return new Pizza(this); }
    }
    public String toString() { return size + toppings; }
}
// Pizza.builder().size("L").top("olives").top("basil").build() -> L[olives, basil]
```

Q149. [intermediate] How do you implement a singleton, and should you?

The safest simple way is a single-element `enum` (serialization and reflection safe), or a holder class for lazy initialisation. Most "singletons" in real applications are managed by the container (a Spring bean with singleton scope) and injected, which keeps them testable. Hand-written global singletons hide dependencies and make tests share state; use them only for genuinely process-wide, stateless or thread-safe things.

Enum singleton (JDK 21 output) JAVA

```java
enum Singleton {
    INSTANCE;
    private int hits;
    int hit() { return ++hits; }   // not thread-safe: guard or use AtomicInteger if shared
}
// Singleton.INSTANCE.hit(); System.out.println(Singleton.INSTANCE.hit());   // 2
```

Q150. [intermediate] Strategy and Factory in modern Java: do you still need classes?

Often a lambda or a method reference replaces a one-method strategy class. A map from a name to a function is a tiny factory/registry. Keep real classes when the strategy has state, several methods, or needs a name in logs and tests.

Strategy as a map of functions (JDK 21 output) JAVA

```
Map<String, BinaryOperator<Integer>> ops = Map.of("add", Integer::sum, "mul", (x, y) -> x * y);
System.out.println(ops.get("mul").apply(6, 7));    // 42
```

Q151. [intermediate] What other patterns do you meet daily in Java code?

Observer (listeners, event buses), Decorator (`BufferedInputStream` wraps an `InputStream`), Template method (`AbstractList`, Spring's `JdbcTemplate`), Iterator (`Iterable`), Adapter (`Arrays.asList`), Proxy (dynamic proxies behind Spring AOP and `@Transactional`), Builder, Factory method (`List.of`, `Optional.of`), and Singleton as a scope. Name the pattern, the problem it solves and a JDK example; interviewers care more about the example than the name.

Q152. [advanced] What is the difference between an abstract class and an interface now that interfaces have default methods?

An interface still has no instance state and no constructors, and a class can implement many interfaces but extend only one class. Abstract classes hold shared state and enforce construction, and can have non-public members. Use an interface for a capability or a contract (often with a few `default` helpers, evolving an API without breaking implementers); use an abstract class when subclasses share fields and a skeleton algorithm. A common pairing: interface plus a skeletal abstract class (as `List` and `AbstractList`).

14.6.2 Writing code others can maintain

Q153. [basic] What makes a good method or class name and size?

A name states the intent in the domain's words (`calculateShippingCost`, not `process`), a method does one thing at one level of abstraction, and a class has a clear responsibility that you can state in one sentence. Long parameter lists (more than about four) usually signal a missing type. Size is a symptom, not a rule: split when you find yourself writing comments to separate sections.

Q154. [intermediate] How should you handle null?

Do not return `null` for collections (return empty ones); use `Optional` for "no result" in return values; validate arguments at public boundaries with `Objects.requireNonNull(x, "x")` so failure is early and clear; use `Objects.equals` when either side can be null; annotate intent (`@Nullable`) where tooling supports it. Make fields non-null by construction. The "billion-dollar mistake" is not that null exists but that types do not say where it is allowed.

Q155. [intermediate] When do you use checked versus unchecked exceptions in your own API?

Use a checked exception when the caller can reasonably recover and the failure is part of the contract (a file may be missing); use an unchecked one for programming errors (`IllegalArgumentException`, `IllegalStateException`) and for failures the caller cannot handle meaningfully. Many modern libraries, including Spring, prefer unchecked exceptions to avoid `throws` clauses leaking through layers. Whatever you choose, include context in the message, wrap the cause, and translate low-level exceptions at layer boundaries (`SQLException` should not reach the controller).

Q156. [intermediate] How do you log properly?

Log through a facade (SLF4J) with levels: `ERROR` for failures needing attention, `WARN` for suspicious but handled cases, `INFO` for business milestones, `DEBUG` and `TRACE` for diagnosis. Use parameterised messages (`log.info("order {} placed", id)`) rather than string concatenation, log an exception once with its stack trace and cause (`log.error("payment failed for {}", id, e)`), never log secrets or personal data, and include a correlation or trace ID so one request can be followed across services. Do not both log and rethrow at every layer, or one failure appears ten times.

Q157. [intermediate] What makes a good unit test?

It is fast, isolated, deterministic and checks one behaviour, with a name that states it (`rejectsNegativeAmounts`). Arrange, act, assert. Inject the clock, random source and I/O so that results are repeatable. Prefer real small objects over mocks, and mock only the boundary (HTTP clients, repositories). Keep a pyramid: many unit tests, fewer integration tests (database through Testcontainers), few end-to-end tests. Coverage numbers show what is not tested, not that the tested parts are right.

Q158. [intermediate] How do you handle equals/hashCode, toString and Comparable in domain classes?

Records give you value `equals`/`hashCode`/`toString` for free. For classes, implement `equals` and `hashCode` from the same business fields, and never from fields that change while the object sits in a hash set. For JPA entities base equality on the identifier or a natural key, with a constant `hashCode` or a careful implementation, because generated IDs are `null` before saving. Keep `compareTo` consistent with `equals`, or document why not (`BigDecimal` is the famous exception). Never put secrets in `toString`.

Q159. [advanced] What do you watch for in API design?

Make misuse hard: immutable inputs, no flag parameters (two methods instead of `process(true)`), strong types over `String` and `int` (a `CustomerId` record), return the narrowest useful type, accept the broadest (`Collection` or `Iterable`, or a functional interface), keep the surface small because everything public is a promise, document the contract (null handling, thread-safety, exceptions), and deprecate before removing. Prefer returning empty collections to `null`, and avoid exposing internal mutable state.

Q160. [advanced] How do you approach performance work?

Measure first. Define the target (p99 latency, throughput), reproduce with a realistic load, find the bottleneck with a profiler or JFR (CPU, allocation, locks, I/O), change one thing, and measure again. Typical wins are algorithms and data structures (a `HashMap` instead of a list scan), fewer allocations in hot loops, batching I/O and database round trips, caching with an eviction policy, and fixing N+1 queries, long before micro-tuning. Micro-benchmarks need JMH because the JIT, warm-up and dead-code elimination distort naive timing loops.

Q161. [advanced] How do you design a thread-safe, in-memory cache in an interview?

State the requirements first: size bound, eviction (LRU or TTL), concurrency level, loading behaviour. A start: `ConcurrentHashMap<K, V>` with `computeIfAbsent` so concurrent misses for one key load only once (keep the loader short and never touch the same map from inside it), plus a bound and eviction. For LRU, `LinkedHashMap` with `accessOrder=true` and `removeEldestEntry` guarded by a lock is simplest. In production say you would use Caffeine (size and time-based eviction, statistics, async loading) rather than write your own, and discuss stale data and stampedes.

Q162. [advanced] A service gets slow in production. What is your first hour?

Establish facts before changing anything. Check recent deploys and config; look at the dashboard (latency, error rate, saturation: CPU, memory, GC pauses, thread-pool queues, connection-pool waits); read GC logs and take two or three thread dumps with `jcmd <pid> Thread.print` to see where threads wait; check downstream dependencies and the database slow-query log; capture a short JFR recording. Roll back or shed load if users are hurting, then find the cause. Afterwards write the timeline and a regression test or alert so it cannot return silently.

Q163. [intermediate] How do you review code, and what do you look for?

Correctness and edge cases first (null, empty, overflow, concurrency, resource closing), then design (fits the structure, no duplicated logic, right abstraction), tests (do they fail without the change?), naming and readability, security (input validation, injection, secrets), and operability (logs, metrics, config). Comment on the code, not the person; explain *why*; separate must-fix from preference; approve quickly when the risk is low. Small pull requests are the best review tool.

Q164. [intermediate] How do you keep dependencies and builds healthy?

Pin versions through a BOM or dependency management, update regularly in small steps (automated update pull requests), scan for known vulnerabilities in CI, prefer few well-maintained libraries over many, avoid depending on a library's internals, and keep builds reproducible (`mvn -B verify` or `gradle build` from a clean checkout must pass). Separate API modules from implementations so that you can change one without recompiling everything.

Q165. [basic] Why should we hire you? Tell us about a project in Java.

Prepare a two-minute story with this shape: the problem and the users, your role, one design decision with its alternatives and why you chose what you did, one thing that went wrong and what you learned, and a measurable result (latency reduced, tests added, bug rate lowered). Choose a project you can discuss several layers deep, because the interviewer will follow your answer down to the code. Honest scope beats inflated claims: "I built this part, I read that part" is credible.

Check yourself

1. What does `new OrderService(null)` do if the constructor uses `Objects.requireNonNull`, and why is that preferable to failing later?
2. Is `Collections.unmodifiableList(list)` an immutable class? Explain.
3. A `Square extends Rectangle` whose `setWidth` also sets the height: which SOLID principle does it break?
4. What is wrong with logging `log.error("failed " + id + ": " + e.getMessage())`?
5. What does this print? `enum S { I; int n; int hit() { return ++n; } }` then `S.I.hit(); System.out.println(S.I.hit());`
6. Why is a `hashCode` based on a mutable field dangerous for objects stored in a `HashSet`?

Answers

1. It throws `NullPointerException` immediately at construction, so the bug is reported where the bad value was passed rather than in some later method call.
2. No. It is an unmodifiable view; whoever holds the original list can still change what the view shows.
3. Liskov substitution: code written against `Rectangle` no longer behaves correctly with the subtype.
4. It concatenates eagerly, loses the stack trace and the cause (only the message is kept), and makes searching harder; pass the exception as the last argument.
5. `2`, because the enum constant is a single instance and the counter is shared.
6. Changing the field changes the hash, so the element sits in the wrong bucket and `contains` and `remove` fail to find it.

KEY POINTS

Design answers are arguments: a rule, the reason, the cost, the exception.

- Prefer constructor injection, composition and immutability; they make code testable and thread-safe.
- Patterns are named solutions; know a JDK example for each and whether a lambda is enough.
- Validate at boundaries, return empty collections, wrap and contextualise exceptions, log once with the cause.
- Measure before optimising; diagnose production with dashboards, thread dumps, GC logs and JFR.
- Prepare one project story you can defend several layers deep.

14.7 What Does This Print? Fifty Puzzles

Interviewers love small programs whose output surprises you, and not because they are cruel. A snippet that tests boxing, initialisation order or overload resolution shows in thirty seconds whether you have a mental model of the language or only a habit of typing code until it works. This chapter gives you fifty such snippets. Every output below was produced by running the listing on OpenJDK 21; where a result depends on the machine it says so.

How to use the puzzles

Cover the output and the explanation. Read the code once, write your prediction on paper, and only then look. A wrong prediction is the valuable case: it points at a rule you had wrong. After the explanation, ask "which rule decided this?" and name it (for example "constant folding", "most specific overload", "left operand evaluated first"). Naming the rule is what you will do in an interview.

INTERVIEW ANGLE

How should I answer a "what does this print" question? Do not blurt out an answer. Walk through the evaluation order aloud, name the rule that applies at each step, and then state the output. If you are unsure between two outcomes, say which rule would decide it. Interviewers reward visible reasoning, and a wrong answer with a correct method often scores higher than a lucky right one.

Each puzzle is labelled basic, intermediate or advanced. Treat any puzzle you got wrong as a flashcard to revisit in a few days. The listings are complete programs; save one as `P01.java` and run it with `java P01.java`.

14.7.1 Boxing, identity and the string pool

Puzzle 1. Integer cache and equals across wrapper types (basic)

P01.java JAVA

```java
public class P01 {
    public static void main(String[] args) {
        Integer a = 127, b = 127;
        Integer c = 128, d = 128;
        System.out.println(a == b);
        System.out.println(c == d);
        System.out.println(c.equals(d));
        Long big = 128L;
        System.out.println(big.equals(128));
        System.out.println(big == 128);
    }
}
```

Output TERMINAL

```
$ java P01.java
true
false
true
false
true
```

`Integer.valueOf` (which autoboxing calls) returns cached objects for -128..127, so `a == b` compares the same object. 128 is outside the cache, so `c` and `d` are two different objects and `==` compares references. `equals` compares values and is the right tool. The last two lines mix wrapper types: `Long.equals(Object)` first checks that the argument is a `Long`, and the autoboxed `128` is an `Integer`, so it is `false`. In `big == 128` one operand is a primitive, so `big` is unboxed and the comparison is numeric.

Puzzle 2. More wrapper identity (intermediate)

P02.java JAVA

```java
public class P02 {
    public static void main(String[] args) {
        Integer i1 = 1000, i2 = 1000;
        int i3 = 1000;
        System.out.println(i1 == i2);
        System.out.println(i1 == i3);
        Integer a = Integer.valueOf(100), b = Integer.valueOf(100);
        System.out.println(a == b);
        System.out.println(new Integer[] {1}[0] == new Integer[] {1}[0]);
        Character c1 = 'x', c2 = 'x';
        System.out.println(c1 == c2);
        Boolean t1 = true, t2 = Boolean.valueOf("TRUE");
        System.out.println(t1 == t2);
        Double d1 = 1.0, d2 = 1.0;
        System.out.println(d1 == d2);
    }
}
```

Output TERMINAL

```
$ java P02.java
false
true
true
true
true
true
false
```

Only `Integer` (and `Long`, `Short`, `Byte`, `Character` up to 127) have caches; `Boolean` has two instances; `Double` and `Float` have no cache, so two boxed `1.0` values are different objects. `i1 == i3` is `true` because one side is a primitive `int`, which forces unboxing. Never rely on identity of wrappers: use `equals` or unbox first.

Puzzle 3. Literal, new String and intern (basic)

P03.java JAVA

```java
public class P03 {
    public static void main(String[] args) {
        String s1 = "java";
        String s2 = "java";
        String s3 = new String("java");
        String s4 = new String("java").intern();
        System.out.println(s1 == s2);
        System.out.println(s1 == s3);
        System.out.println(s1 == s4);
        System.out.println(s1.equals(s3));
    }
}
```

Output TERMINAL

```
$ java P03.java
true
false
true
true
```

Literals with equal text share one pooled object, so `s1 == s2`. `new String("java")` always creates a fresh heap object, so `s1 == s3` is `false`, though `equals` is `true`. `intern()` returns the pooled instance, which is the same object as the literal. In real code compare strings with `equals`; the pool is an optimisation, not a contract.

Puzzle 4. Compile-time constants and concatenation (intermediate)

P04.java JAVA

```java
public class P04 {
    public static void main(String[] args) {
        String hel = "hel";
        final String fhel = "hel";
        String a = "hello";
        String b = "hel" + "lo";
        String c = hel + "lo";
        String d = fhel + "lo";
        System.out.println(a == b);
        System.out.println(a == c);
        System.out.println(a == d);
        System.out.println(a == c.intern());
    }
}
```

Output TERMINAL

```
$ java P04.java
true
false
true
true
```

`"hel" + "lo"` is a constant expression, so the compiler folds it to `"hello"` and it is pooled: `a == b`. `hel + "lo"` uses a non-final variable, so the concatenation happens at run time and creates a new object. Declaring the variable `final` makes it a constant variable, which is folded again, so `a == d` is `true`. Interning the run-time result gives the pooled instance.

Puzzle 5. Strings are immutable, StringBuilder has no equals (basic)

P05.java JAVA

```java
public class P05 {
    public static void main(String[] args) {
        StringBuilder a = new StringBuilder("x");
        StringBuilder b = new StringBuilder("x");
        System.out.println(a.equals(b));
        System.out.println(a.toString().equals(b.toString()));
        System.out.println(a.compareTo(b));
        String s = "abc";
        s.concat("def");
        s.toUpperCase();
        System.out.println(s);
        s = s.concat("def").toUpperCase();
        System.out.println(s);
    }
}
```

Output TERMINAL

```
$ java P05.java
false
true
0
abc
ABCDEF
```

`StringBuilder` does not override `equals`, so it uses identity. `compareTo` (since Java 11) compares contents, giving `0`. `String` methods never change the receiver: `concat` and `toUpperCase` return new strings, and ignoring the result does nothing. Reassign the result if you want it.

14.7.2 Initialization order and dispatch

Puzzle 6. Static and instance initialisation (basic)

P06.java JAVA

```java
public class P06 {
    static int x = init("x");
    static { System.out.println("static block"); }
    int y = init("y");
    { System.out.println("instance block"); }
    P06() { System.out.println("constructor"); }

    static int init(String name) {
        System.out.println("init " + name);
        return 1;
    }

    public static void main(String[] args) {
        System.out.println("main starts");
        new P06();
        new P06();
    }
}
```

Output TERMINAL

```
$ java P06.java
init x
static block
main starts
init y
instance block
constructor
init y
instance block
constructor
```

Static initialisers run once, when the class is first initialised (here before `main`'s first line, because `main` is in the same class), in textual order. Each `new` then runs field initialisers and instance blocks in textual order, followed by the

constructor body. Mixing the order of the field and the block would change the output, so keep initialisers at the top of the class.

Puzzle 7. Overridable method called from a constructor (intermediate)

P07.java JAVA

```java
public class P07 {
    static class Base {
        Base() { show(); }
        void show() { System.out.println("Base.show"); }
    }
    static class Child extends Base {
        private String name = "child";
        private int count = 7;
        Child() { super(); System.out.println("Child ctor " + name); }
        @Override void show() { System.out.println("Child.show " + name + " " + count); }
    }
    public static void main(String[] args) {
        new Child();
    }
}
```

Output TERMINAL

```
$ java P07.java
Child.show null 0
Child ctor child
```

The `Base` constructor runs before `Child`'s field initialisers, but dynamic dispatch already selects `Child.show`. The fields still hold their default values (`null` and `0`) because `Child`'s initialisers have not run. Calling an overridable method from a constructor exposes a half-built object: make the method `private` or `final`, or do the work in a factory method.

Puzzle 8. Class and constructor chain order (intermediate)

P08.java JAVA

```java
public class P08 {
    static class Parent {
        static { System.out.println("Parent static"); }
        { System.out.println("Parent instance"); }
        Parent() { System.out.println("Parent ctor"); }
    }
    static class Child extends Parent {
        static { System.out.println("Child static"); }
        { System.out.println("Child instance"); }
        Child() { this(1); System.out.println("Child ctor()"); }
        Child(int x) { super(); System.out.println("Child ctor(int)"); }
    }
    public static void main(String[] args) {
        new Child();
    }
}
```

Output TERMINAL

```
$ java P08.java
Parent static
Child static
Parent instance
Parent ctor
Child instance
Child ctor(int)
Child ctor()
```

The sequence is: superclass static, subclass static (once per class load), then for each instance the superclass initialisers and constructor, then the subclass initialisers and the constructor chain. `Child()` calls `this(1)`, which calls `super()`, so the instance block of `Child` runs once, inside the constructor that calls `super()`, never twice.

Puzzle 9. When is a class initialised? (advanced)

P09.java JAVA

```java
public class P09 {
    static class Config {
        static final int LIMIT = 100;
        static final Integer BOXED = 200;
        static { System.out.println("Config initialised"); }
    }
    public static void main(String[] args) {
        System.out.println(Config.LIMIT);
        System.out.println("between");
        System.out.println(Config.BOXED);
    }
}
```

Output TERMINAL

```
$ java P09.java
100
between
Config initialised
200
```

`LIMIT` is a compile-time constant (`static final` primitive with a constant initialiser). The compiler copies the value into `main`, so reading it never touches `Config` and its static block does not run. `BOXED` is not a constant (`Integer` is an object), so the first access initialises the class. This is why changing a constant in a library needs recompiling its users.

Puzzle 10. Fields and static methods are not virtual (intermediate)

P10.java JAVA

```java
public class P10 {
    static class Animal {
        String name = "animal";
        static String kind() { return "Animal.kind"; }
        String who() { return "Animal.who"; }
    }
    static class Dog extends Animal {
        String name = "dog";
        static String kind() { return "Dog.kind"; }
        @Override String who() { return "Dog.who"; }
    }
    public static void main(String[] args) {
        Animal a = new Dog();
        System.out.println(a.name);
        System.out.println(((Dog) a).name);
        System.out.println(a.who());
        System.out.println(a.kind());
    }
}
```

Output TERMINAL

```
$ java P10.java
animal
dog
Dog.who
Animal.kind
```

Only instance methods are dispatched on the run-time type. Fields and static methods are resolved with the declared type: `a` is an `Animal`, so `a.name` and `a.kind()` use `Animal`'s versions, while `a.who()` runs `Dog.who`. Cast to `Dog` to see the hiding field. Avoid hiding fields; call static methods through the class name.

Puzzle 11. Private methods are not overridden (intermediate)

P11.java JAVA

```java
public class P11 {
    static class A {
        private String hello() { return "A.hello"; }
        String call() { return hello(); }
    }
    static class B extends A {
        String hello() { return "B.hello"; }
    }
    public static void main(String[] args) {
        System.out.println(new B().call());
        System.out.println(new B().hello());
    }
}
```

Output TERMINAL

```
$ java P11.java
A.hello
B.hello
```

`A.call()` calls `hello()`; because `A.hello` is private it is bound at compile time to `A`'s version and `B.hello` is an unrelated method that merely has the same name. Without `private`, the call would dispatch to `B.hello`. Adding `@Override` to `B.hello` would be a compile error, which is exactly why you should use the annotation.

Puzzle 12. Default method resolution (advanced)

P12.java JAVA

```java
public class P12 {
    interface A { default String hi() { return "A"; } }
    interface B extends A { default String hi() { return "B"; } }
    static class Base { public String hi() { return "Base"; } }
    static class C extends Base implements B { }
    static class D implements A, B { }
    public static void main(String[] args) {
        System.out.println(new C().hi());
        System.out.println(new D().hi());
    }
}
```

Output TERMINAL

```
$ java P12.java
Base
B
```

The rules in order: a class method (even inherited) wins over any interface default, so `C` prints `Base`. Otherwise the most specific interface wins: `B` extends `A`, so for `D` the default from `B` is chosen even though `A` is listed too. Only two unrelated interfaces with the same default cause a compile error that you must resolve with `A.super.hi()`.

14.7.3 Overloads, varargs and the ternary operator

Puzzle 13. Overload resolution phases (intermediate)

P13.java JAVA

```java
public class P13 {
    static void m(Object o) { System.out.println("Object"); }
    static void m(String s) { System.out.println("String"); }
    static void m(Integer i) { System.out.println("Integer"); }
    static void n(long x) { System.out.println("long"); }
    static void n(Integer x) { System.out.println("Integer"); }
    static void n(int... x) { System.out.println("varargs"); }
    public static void main(String[] args) {
        m("text");
        m(42);
        m((Object) "text");
        n(5);
        char c = 'a';
        n(c);
    }
}
```

Output TERMINAL

```
$ java P13.java
String
Integer
Object
long
long
```

The compiler tries three phases in order: no boxing and no varargs; then boxing; then varargs. `m(42)` needs boxing, and `Integer` is more specific than `Object`. `n(5)` finds `n(long)` in phase 1 by widening, so it wins over boxing to `Integer`, and varargs is never considered. `m((Object) "text")` picks `m(Object)` because overloads are chosen from the static type.

Puzzle 14. Most specific method (advanced)

P14.java JAVA

```java
public class P14 {
    static void p(Object o) { System.out.println("Object"); }
    static void p(char[] cs) { System.out.println("char[]"); }
    static void q(double d) { System.out.println("double"); }
    static void q(float f) { System.out.println("float"); }
    public static void main(String[] args) {
        p(null);
        q(1);
        q(1L);
        q(1.0);
    }
}
```

Output TERMINAL

```
$ java P14.java
char[]
float
float
double
```

`p(null)`: both `Object` and `char[]` accept `null`; `char[]` is a subtype of `Object` and therefore more specific. (With `String` and `Integer` overloads it would be an ambiguity error.) For `q(1)` and `q(1L)`, both `float` and `double` are reachable by widening, and `float` is more specific because it widens to `double`. Only the literal `1.0` is a `double`.

Puzzle 15. Varargs and null (advanced)

P15.java JAVA

```java
import java.util.Arrays;
import java.util.List;

public class P15 {
    static void f(String... a) { System.out.println(a == null ? "null array" : "len " + a.length); }
    static void g(Object... a) { System.out.println(a == null ? "null array" : "len " + a.length); }
    public static void main(String[] args) {
        f();
        f((String) null);
        f((String[]) null);
        g((Object[]) new String[] {"a", "b"});
        g((Object) new String[] {"a", "b"});
        int[] prim = {1, 2, 3};
        Integer[] boxed = {1, 2, 3};
        List<int[]> l1 = Arrays.asList(prim);
        List<Integer> l2 = Arrays.asList(boxed);
        System.out.println(l1.size() + " " + l2.size());
    }
}
```

Output TERMINAL

```
$ java P15.java
len 0
len 1
null array
len 2
len 1
1 3
```

`f()` creates an empty array. `f((String) null)` passes one element that is `null`, giving an array of length 1. `f((String[]) null)` passes the array itself as `null`. The cast on `g` decides whether the array is the argument list or a single element. `Arrays.asList(int[])` cannot spread a primitive array, so you get a list with one element of type `int[]`; with `Integer[]` you get three elements.

Puzzle 16. Ternary typing rules (advanced)

P16.java JAVA

```java
public class P16 {
    public static void main(String[] args) {
        boolean flag = true;
        Object o1 = flag ? 1 : "s";
        Object o2 = flag ? 1 : 2.0;
        System.out.println(o1 + " " + o2);
        int i = 0;
        System.out.println(true ? 'a' : 0);
        System.out.println(true ? 'a' : i);
        Integer boxed = null;
        try {
            int r = flag ? boxed : 0;
            System.out.println(r);
        } catch (NullPointerException e) {
            System.out.println("NPE from unboxing");
        }
        Integer ok = flag ? null : 0;
        System.out.println(ok);
    }
}
```

Output TERMINAL

```
$ java P16.java
1 1.0
a
97
NPE from unboxing
null
```

If one branch is a numeric type and the other is a different numeric type, the result is promoted: `flag ? 1 : 2.0` is `1.0`. A `char` and an `int` constant that fits in a `char` stay a `char` (prints `a`); a non-constant `int` variable promotes to `int` (prints

97). If one branch is a primitive and the other is an `Integer`, the result is the primitive, so a `null Integer` is unboxed and throws a `NullPointerException`. With `null` and `int` literal, the result type is `Integer`, so no unboxing occurs.

Puzzle 17. Static call on null, instanceof, println(char[]) (intermediate)

P17.java JAVA

```java
public class P17 {
    static class Util {
        static String hello() { return "hello"; }
    }
    public static void main(String[] args) {
        Util u = null;
        System.out.println(u.hello());
        System.out.println(u instanceof Util);
        System.out.println(null instanceof Object);
        char[] cs = {'h', 'i'};
        System.out.println(cs);
        System.out.println(("" + cs).startsWith("[C@"));
        System.out.println(String.valueOf(cs));
    }
}
```

Output TERMINAL

```
$ java P17.java
hello
false
false
hi
true
hi
```

A static method is resolved from the declared type, so `u.hello()` does not dereference `u` and works even if `u` is `null`. `instanceof` is `false` for `null`, so it never throws. `println(char[])` is a special overload that prints the characters, but string concatenation calls `String.valueOf(Object)`, which gives the type-and-hash form such as `[C@1b6d3586`. That is why the code checks only the prefix.

14.7.4 Control flow, finally and exceptions

Puzzle 18. Return and finally (intermediate)

P18.java JAVA

```java
public class P18 {
    @SuppressWarnings("finally")
    static int a() {
        try { return 1; } finally { return 2; }
    }
    static int b() {
        int x = 10;
        try { return x; } finally { x = 20; }
    }
    static StringBuilder c() {
        StringBuilder sb = new StringBuilder("a");
        try { return sb; } finally { sb.append("b"); }
    }
    @SuppressWarnings("finally")
    static int d() {
        try { throw new RuntimeException("lost"); } finally { return 3; }
    }
    public static void main(String[] args) {
        System.out.println(a());
        System.out.println(b());
        System.out.println(c());
        System.out.println(d());
    }
}
```

Output TERMINAL

```
$ java P18.java
2
10
ab
3
```

The return value is computed first, then `finally` runs. For the primitive in `b()` the saved copy is `10`, so changing `x` later has no effect. In `c()` the saved value is a reference, so the mutation inside `finally` is visible: `ab`. A `return` inside `finally` replaces the result of the `try` and even discards a thrown exception, as in `d()`. Never return from `finally`; the compiler warns about it.

Puzzle 19. try-with-resources, close order and suppression (advanced)

P19.java JAVA

```java
public class P19 {
    static class Res implements AutoCloseable {
        final String n;
        Res(String n) { this.n = n; System.out.println("open " + n); }
        @Override public void close() {
            System.out.println("close " + n);
            throw new IllegalStateException("close " + n);
        }
    }
    public static void main(String[] args) {
        try (Res a = new Res("A"); Res b = new Res("B")) {
            System.out.println("body");
            throw new RuntimeException("body failed");
        } catch (Exception e) {
            System.out.println("caught " + e.getMessage());
            for (Throwable t : e.getSuppressed()) {
                System.out.println("suppressed " + t.getMessage());
            }
        }
    }
}
```

Output TERMINAL

```
$ java P19.java
open A
open B
body
close B
close A
caught body failed
suppressed close B
suppressed close A
```

Resources close in reverse order of opening, before the `catch` block runs. The body's exception is the primary one; the exceptions thrown by both `close` calls are attached as suppressed exceptions and read with `getSuppressed()`. In a hand-written `try/finally`, an exception from `close` would hide the real failure.

Puzzle 20. Short-circuit and non-short-circuit operators (intermediate)

P20.java JAVA

```java
public class P20 {
    static int calls = 0;
    static boolean t(String s) { calls++; System.out.print(s + " "); return true; }
    static boolean f(String s) { calls++; System.out.print(s + " "); return false; }
    public static void main(String[] args) {
        boolean r1 = f("a") && t("b") || t("c");
        System.out.println("-> " + r1);
        boolean r2 = t("d") | f("e") & t("f");
        System.out.println("-> " + r2);
        boolean r3 = f("g") & t("h");
        System.out.println("-> " + r3 + " calls=" + calls);
        int x = 5;
        boolean r4 = x > 3 || x++ > 0;
        System.out.println(x);
        System.out.println(true ^ true);
    }
}
```

Output TERMINAL

```
$ java P20.java
a c -> true
d e f -> true
g h -> false calls=7
5
false
```

`&&` binds tighter than `||`. In `f(a) && t(b) || t(c)`, `b` is skipped, then `c` runs. `|` and `&` always evaluate both operands, and `&` binds tighter than `|`, so `d | (e & f)` evaluates all three. In `r4`, `x > 3` is true so `x++` never runs and `x` stays `5`. `^` on booleans is exclusive or.

Puzzle 21. Switch fall-through (intermediate)

P21.java JAVA

```java
public class P21 {
    public static void main(String[] args) {
        for (int n = 1; n <= 4; n++) {
            System.out.print("n=" + n + ":");
            switch (n) {
                case 1: System.out.print(" one");
                case 2: System.out.print(" two"); break;
                case 3: System.out.print(" three");
                default: System.out.print(" default");
                case 4: System.out.print(" four");
            }
            System.out.println();
        }
    }
}
```

Output TERMINAL

```
$ java P21.java
n=1: one two
n=2: two
n=3: three default four
n=4: four
```

Old-style `case` labels fall through until a `break`. For `n=1`: `one`, then falls into `two`. For `n=3`: `three`, falls into `default`, falls into `case 4`: `default` position does not matter for matching (it is used only if nothing matches) but it matters for fall-through. Arrow cases (`case 1 ->`) never fall through, which is one reason to prefer them.

Puzzle 22. Pattern switch, guards and null (advanced)

P22.java JAVA

```java
public class P22 {
    static String describe(Object o) {
        return switch (o) {
            case Integer i when i > 10 -> "big int " + i;
            case Integer i -> "int " + i;
            case String s when s.isEmpty() -> "empty string";
            case String s -> "string " + s.length();
            case null -> "null!";
            default -> "other " + o.getClass().getSimpleName();
        };
    }
    public static void main(String[] args) {
        System.out.println(describe(5));
        System.out.println(describe(50));
        System.out.println(describe(""));
        System.out.println(describe("abc"));
        System.out.println(describe(null));
        System.out.println(describe(2.5));
        String sw = null;
        try {
            switch (sw) { case "a": break; default: break; }
        } catch (NullPointerException e) {
            System.out.println("old switch on null string: NPE");
        }
    }
}
```

Output TERMINAL

```
$ java P22.java
int 5
big int 50
empty string
string 3
null!
other Double
old switch on null string: NPE
```

Cases are tested top to bottom, so the guarded `Integer i when i > 10` must come before the plain `Integer i` (the other order is a compile error: dominance). `case null` is allowed in pattern switches since Java 21; without it a `null` selector throws a `NullPointerException`, which is still what the old string switch does.

14.7.5 Numbers, operators and characters

Puzzle 23. Integer overflow (basic)

P23.java JAVA

```java
public class P23 {
    public static void main(String[] args) {
        int max = Integer.MAX_VALUE;
        System.out.println(max + 1);
        System.out.println(Math.abs(Integer.MIN_VALUE));
        System.out.println((byte) 200);
        System.out.println((int) 3_000_000_000L);
        System.out.println(1_000_000 * 1_000_000);
        System.out.println(1_000_000L * 1_000_000);
        long micros = 24 * 60 * 60 * 1000 * 1000;
        System.out.println(micros);
    }
}
```

Output TERMINAL

```
$ java P23.java
-2147483648
-2147483648
-56
-1294967296
-727379968
1000000000000
500654080
```

Integer arithmetic wraps around silently in two's complement. `Math.abs(Integer.MIN_VALUE)` is still negative because `+2147483648` does not fit. `(byte) 200` keeps the low 8 bits, giving `-56`. `1_000_000 * 1_000_000` is computed in `int` before any widening, so the result is wrong even though the target would hold it; make one operand `long`. The microsecond constant has the same bug: `24 * 60 * 60 * 1000 * 1000` overflows `int`. Use `Math.multiplyExact` when overflow is an error.

Puzzle 24. Floating point surprises (intermediate)

P24.java JAVA

```java
public class P24 {
    public static void main(String[] args) {
        System.out.println(0.1 + 0.2);
        System.out.println(0.1f + 0.2f == 0.3f);
        System.out.println(0.1 + 0.2 == 0.3);
        System.out.println(0.1f == 0.1);
        double nan = 0.0 / 0.0;
        System.out.println(nan == nan);
        System.out.println(Double.valueOf(nan).equals(nan));
        System.out.println(0.0 == -0.0);
        System.out.println(Double.valueOf(0.0).equals(-0.0));
        System.out.println(1.0 / 0);
        System.out.println((int) Double.NaN);
    }
}
```

Output TERMINAL

```
$ java P24.java
0.30000000000000004
true
false
false
false
true
true
false
Infinity
0
```

`0.1 + 0.2` is not exactly `0.3` in binary floating point. The `float` version accidentally rounds to the same `float` as `0.3f`, which shows that you cannot reason about equality by feel. `0.1f == 0.1` is `false` because the float is widened to a different double. `NaN` is not equal to itself with `==`, but `Double.equals` treats it as equal (so it works as a map key), and `0.0 == -0.0` is `true` while `Double.equals` says `false`. Casting `NaN` to `int` gives `0`.

Puzzle 25. Compound assignment and increments (intermediate)

P25.java JAVA

```java
public class P25 {
    public static void main(String[] args) {
        byte b = 10;
        b += 300;
        System.out.println(b);
        short s = 1;
        s += 1.9;
        System.out.println(s);
        int i = 5;
        i = i++ + ++i;
        System.out.println(i);
        int j = 3;
        j = j++;
        System.out.println(j);
        int k = 1;
        k += k++ + ++k;
        System.out.println(k);
    }
}
```

Output TERMINAL

```
$ java P25.java
54
2
12
3
5
```

Compound assignment has a hidden cast: `b += 300` is `b = (byte)(b + 300)`, which wraps to `54`, and `s += 1.9` is `s = (short)(s + 1.9)`, truncating to `2`. For `i = i++ + ++i`: left operand evaluates to 5 (and `i` becomes 6), `++i` makes 7, sum 12. `j = j++` assigns the old value back, so `j` stays `3`. In `k += expr` the old value of `k` is saved before the right side runs: `1 + (1 + 3) = 5`. Do not write such expressions.

Puzzle 26. Rounding, division and remainder with negatives (intermediate)

P26.java JAVA

```java
public class P26 {
    public static void main(String[] args) {
        System.out.println(Math.round(2.5));
        System.out.println(Math.round(-2.5));
        System.out.println(Math.round(-2.6));
        System.out.println((int) -2.7);
        System.out.println(Math.floor(-2.7));
        System.out.println(-7 % 3);
        System.out.println(7 % -3);
        System.out.println(Math.floorMod(-7, 3));
        System.out.println(-7 / 2);
        System.out.println(Math.floorDiv(-7, 2));
        System.out.println(5.5 % 2);
    }
}
```

Output TERMINAL

```
$ java P26.java
3
-2
-3
-2
-3.0
-1
1
2
-3
-4
1.5
```

`Math.round` rounds half up (towards positive infinity): `2.5` becomes `3`, `-2.5` becomes `-2`. A cast truncates towards zero (`-2.7` becomes `-2`), while `Math.floor` goes down. Integer division truncates towards zero, and `%` takes the sign of the

dividend, so `-7 % 3 == -1` and `7 % -3 == 1`. `Math.floorMod` and `Math.floorDiv` give the mathematical (floor) versions, which is what you want for array wrap-around indices.

Puzzle 27. Division by zero and edge values (intermediate)

P27.java JAVA

```java
public class P27 {
    public static void main(String[] args) {
        System.out.println(1.0 / 0);
        System.out.println(-1.0 / 0);
        System.out.println(0.0 / 0);
        System.out.println(5 % 0.0);
        try {
            System.out.println(1 / 0);
        } catch (ArithmeticException e) {
            System.out.println(e.getMessage());
        }
        try {
            System.out.println(5 % 0);
        } catch (ArithmeticException e) {
            System.out.println(e.getMessage());
        }
        System.out.println(Double.MIN_VALUE > 0);
        System.out.println(Integer.MIN_VALUE == -Integer.MIN_VALUE);
    }
}
```

Output TERMINAL

```
$ java P27.java
Infinity
-Infinity
NaN
NaN
/ by zero
/ by zero
true
true
```

Integer division or remainder by zero throws `ArithmeticException: / by zero`. Floating point never throws: it yields `Infinity`, `-Infinity` or `NaN`. `Double.MIN_VALUE` is the smallest positive double, not the most negative one. `Integer.MIN_VALUE` equals its own negation because of overflow.

Puzzle 28. Shifts and literals (advanced)

P28.java JAVA

```java
public class P28 {
    public static void main(String[] args) {
        System.out.println(1 << 32);
        System.out.println(1L << 32);
        System.out.println(1 << 33);
        System.out.println(-1 >> 28);
        System.out.println(-1 >>> 28);
        byte b = (byte) 0xF0;
        System.out.println(b >>> 4);
        System.out.println((byte) (b >>> 4));
        b >>>= 4;
        System.out.println(b);
        System.out.println(010 + 0x10 + 0b10);
        long l = 0xFFFFFFFF;
        System.out.println(l);
    }
}
```

Output TERMINAL

```
$ java P28.java
1
4294967296
2
-1
15
268435455
-1
-1
26
-1
```

Shift distances are masked to 5 bits for `int` (6 for `long`), so `1 << 32` is `1 << 0` and `1 << 33` is `2`. `>>` keeps the sign, `>>>` fills with zeros. A `byte` is promoted to `int` before shifting, so `b >>> 4` on `-16` shifts a 32-bit value and gives `268435455`; casting back to `byte` yields `-1`. A leading zero means octal (`010` is 8). `0xFFFFFFFF` is an `int` literal with value `-1`, widened to `long` as `-1`; write `0xFFFFFFFFL` for the large value.

Puzzle 29. Floating point in loops (intermediate)

P29.java JAVA

```java
public class P29 {
    public static void main(String[] args) {
        int count = 0;
        for (double d = 0.0; d != 1.0 && count < 15; d += 0.1) {
            count++;
        }
        System.out.println(count);
        double sum = 0;
        for (int i = 0; i < 10; i++) sum += 0.1;
        System.out.println(sum);
        System.out.println(sum == 1.0);
        float f = 16_777_216f;
        System.out.println(f + 1 == f);
        System.out.println((long) 1e19);
    }
}
```

Output TERMINAL

```
$ java P29.java
15
0.9999999999999999
false
true
9223372036854775807
```

Adding `0.1` ten times gives `0.9999999999999999`, so a loop with `d != 1.0` never ends (the code stops it with a counter). Always loop with an integer counter or use `<`. A `float` has 24 bits of mantissa: above 16,777,216 adding 1 changes nothing. Casting a too-large `double` to `long` saturates at `Long.MAX_VALUE` rather than wrapping.

Puzzle 30. char arithmetic and string concatenation (intermediate)

P30.java JAVA

```java
public class P30 {
    public static void main(String[] args) {
        System.out.println('a' + 1);
        System.out.println((char) ('a' + 1));
        System.out.println('a' + 'b');
        System.out.println("" + 'a' + 'b');
        System.out.println('a' + 'b' + "c");
        System.out.println(1 + 2 + "x" + 1 + 2);
        char c = 'a';
        c += 1;
        c++;
        System.out.println(c);
        System.out.println(new StringBuilder('a').append("b").length());
    }
}
```

Output TERMINAL

```
$ java P30.java
98
b
195
ab
195c
3x12
c
1
```

`'a' + 1` is an `int` (98); cast back for the character. Two chars added are an `int` (195), unless a `String` is already on the left. Evaluation is left to right: `1 + 2 + "x" + 1 + 2` is `3x12`. Compound assignment and `++` on a `char` keep the type, so `c` ends as `c`. `new StringBuilder('a')` picks the `StringBuilder(int capacity)` constructor with capacity 97, not the text `a`, so the builder holds only `b`, length 1.

14.7.6 Strings, arrays and collections

Puzzle 31. split surprises (intermediate)

P31.java JAVA

```java
import java.util.Arrays;

public class P31 {
    public static void main(String[] args) {
        System.out.println("a,b,,".split(",").length);
        System.out.println(",a".split(",").length);
        System.out.println("".split(",").length);
        System.out.println(Arrays.toString("a|b".split("|")));
        System.out.println(Arrays.toString("a|b".split("\\|")));
        System.out.println("a.b".split(".").length);
        System.out.println("a,b,,".split(",", -1).length);
    }
}
```

Output TERMINAL

```
$ java P31.java
2
2
1
[a, |, b]
[a, b]
0
4
```

`split` removes trailing empty strings, but keeps a leading one: `"a,b,,"` gives 2, `",a"` gives 2. `"".split(",")` returns an array with one empty string. The argument is a regular expression, so `"|"` means an empty alternation that splits between every character, and `"."` matches everything, leaving only empty strings that are trimmed away (length 0). Escape with `\\|` or use `Pattern.quote`. A negative limit keeps trailing empties.

Puzzle 32. Mutable keys in a HashMap (intermediate)

P32.java JAVA

```java
import java.util.HashMap;
import java.util.Map;

public class P32 {
    static class Key {
        int id;
        Key(int id) { this.id = id; }
        @Override public boolean equals(Object o) { return o instanceof Key k && k.id == id; }
        @Override public int hashCode() { return id; }
    }
    public static void main(String[] args) {
        Map<Key, String> map = new HashMap<>();
        Key k = new Key(1);
        map.put(k, "one");
        k.id = 2;
        System.out.println(map.get(k));
        System.out.println(map.get(new Key(1)));
        System.out.println(map.containsKey(new Key(2)));
        System.out.println(map.size());
        System.out.println("Aa".hashCode() == "BB".hashCode());
    }
}
```

Output TERMINAL

```
$ java P32.java
null
null
false
1
true
```

The entry was stored in the bucket for hash `1`. After `k.id = 2`, lookup with `k` goes to the bucket for hash 2 and finds nothing. A new `Key(1)` goes to the right bucket but the stored key is no longer equal to it. The entry is still in the map (size 1) but unreachable. Use immutable keys: records, strings, wrappers. Also note `"Aa"` and `"BB"` have equal hash codes: equal hashes do not imply equal objects.

Puzzle 33. equals without hashCode (intermediate)

P33.java JAVA

```java
import java.util.HashMap;
import java.util.HashSet;
import java.util.Map;
import java.util.Set;

public class P33 {
    static class Point {
        final int x, y;
        Point(int x, int y) { this.x = x; this.y = y; }
        @Override public boolean equals(Object o) {
            return o instanceof Point p && p.x == x && p.y == y;
        }
    }
    public static void main(String[] args) {
        Set<Point> set = new HashSet<>();
        set.add(new Point(1, 2));
        set.add(new Point(1, 2));
        System.out.println(set.size());
        System.out.println(new Point(1, 2).equals(new Point(1, 2)));
        System.out.println(set.contains(new Point(1, 2)));
    }
}
```

Output TERMINAL

```
$ java P33.java
2
true
false
```

Two points that are `equals` land in different buckets because the inherited identity `hashCode` differs, so the set holds both and `contains` fails (almost always; a collision would be luck). The contract is: equal objects must have equal hash codes. Override both together, or use a record.

Puzzle 34. List.remove(int) versus remove(Object) (basic)

P34.java JAVA

```java
import java.util.ArrayList;
import java.util.List;

public class P34 {
    public static void main(String[] args) {
        List<Integer> list = new ArrayList<>(List.of(10, 20, 30, 1));
        list.remove(1);
        System.out.println(list);
        list.remove(Integer.valueOf(1));
        System.out.println(list);
        list.remove((Object) 30);
        System.out.println(list);
    }
}
```

Output TERMINAL

```
$ java P34.java
[10, 30, 1]
[10, 30]
[10]
```

With `List<Integer>`, `remove(1)` picks the `remove(int index)` overload and removes the element at index 1 (`20`). To remove the value, box it explicitly: `Integer.valueOf(1)` or a cast to `Object`. Overload resolution prefers the primitive overload in phase 1.

Puzzle 35. Arrays.asList and List.of (intermediate)

P35.java JAVA

```java
import java.util.Arrays;
import java.util.List;

public class P35 {
    public static void main(String[] args) {
        String[] arr = {"a", "b", "c"};
        List<String> view = Arrays.asList(arr);
        view.set(0, "Z");
        System.out.println(arr[0]);
        arr[1] = "Y";
        System.out.println(view);
        try {
            view.add("d");
        } catch (UnsupportedOperationException e) {
            System.out.println("add: " + e.getClass().getSimpleName());
        }
        try {
            List.of("x").set(0, "y");
        } catch (UnsupportedOperationException e) {
            System.out.println("List.of is immutable");
        }
    }
}
```

Output TERMINAL

```
$ java P35.java
Z
[Z, Y, c]
add: UnsupportedOperationException
List.of is immutable
```

`Arrays.asList` returns a fixed-size list that is a view of the array: `set` writes through to the array, and changes to the array are visible in the list, but `add` and `remove` throw `UnsupportedOperationException`. `List.of` returns a truly immutable list, where even `set` throws. Wrap in `new ArrayList<>(...)` if you need an independent mutable copy.

Puzzle 36. Removing inside a for-each loop (advanced)

P36.java JAVA

```java
import java.util.ArrayList;
import java.util.List;

public class P36 {
    static void removeSecond(String target) {
        List<String> l = new ArrayList<>(List.of("a", "b", "c"));
        try {
            for (String s : l) if (s.equals(target)) l.remove(s);
            System.out.println("removing " + target + " -> " + l);
        } catch (java.util.ConcurrentModificationException e) {
            System.out.println("removing " + target + " -> CME");
        }
    }
    public static void main(String[] args) {
        removeSecond("a");
        removeSecond("b");
        removeSecond("c");
    }
}
```

Output TERMINAL

```
$ java P36.java
removing a -> CME
removing b -> [a, c]
removing c -> CME
```

Removing from an `ArrayList` during iteration usually throws `ConcurrentModificationException`, but not always. `hasNext()` just compares the cursor with the size. After removing the second-to-last element, the cursor equals the new size, so the loop ends before `next()` can check the modification counter. Removing the first or last element does throw. Do not depend on this; use `Iterator.remove()` or `removeIf`.

Puzzle 37. Autoboxing in collection calls (advanced)

P37.java JAVA

```java
import java.util.HashSet;
import java.util.Set;

public class P37 {
    public static void main(String[] args) {
        Set<Short> set = new HashSet<>();
        for (short i = 0; i < 10; i++) {
            set.add(i);
            set.remove(i - 1);
        }
        System.out.println(set.size());
    }
}
```

Output TERMINAL

```
$ java P37.java
10
```

`i - 1` is an `int`, so `set.remove(i - 1)` boxes to `Integer`, but the set contains `Short` objects. `Integer.equals(Short)` is always `false`, so nothing is ever removed and the size is `10`. The fix is `set.remove((short) (i - 1))`. `remove(Object)` takes any object, so the compiler cannot warn you.

Puzzle 38. Null keys, null values and Map methods (intermediate)

P38.java JAVA

```java
import java.util.HashMap;
import java.util.Map;

public class P38 {
    public static void main(String[] args) {
        Map<String, Integer> m = new HashMap<>();
        m.put(null, 1);
        m.put("a", null);
        System.out.println(m.get(null) + " " + m.get("a") + " " + m.get("zzz"));
        System.out.println(m.containsKey("a") + " " + m.containsKey("zzz"));
        m.putIfAbsent("a", 5);
        System.out.println(m.get("a"));
        m.merge("a", 7, Integer::sum);
        System.out.println(m.get("a"));
        System.out.println(m.getOrDefault("a", 0) + " " + m.getOrDefault("q", 0));
        try {
            java.util.Map.of("k", 1, "k", 2);
        } catch (IllegalArgumentException e) {
            System.out.println("Map.of duplicate key");
        }
    }
}
```

Output TERMINAL

```
$ java P38.java
1 null null
true false
5
12
12 0
Map.of duplicate key
```

`HashMap` permits one `null` key and `null` values, so `get` returning `null` cannot tell absence from a stored `null`: use `containsKey`. `putIfAbsent` treats a mapping to `null` as absent, so it stores `5`. `merge` combines with the existing value (`5 + 7`). `getOrDefault` only supplies a default when the key is missing. `Map.of` rejects duplicate keys and nulls.

Puzzle 39. BigDecimal equality (advanced)

P39.java JAVA

```java
import java.math.BigDecimal;
import java.util.HashSet;
import java.util.Set;
import java.util.TreeSet;

public class P39 {
    public static void main(String[] args) {
        BigDecimal x = new BigDecimal("2.0");
        BigDecimal y = new BigDecimal("2.00");
        System.out.println(x.equals(y) + " " + x.compareTo(y));
        Set<BigDecimal> hs = new HashSet<>(Set.of(x));
        hs.add(y);
        Set<BigDecimal> ts = new TreeSet<>(Set.of(x));
        ts.add(y);
        System.out.println(hs.size() + " " + ts.size());
        System.out.println(new BigDecimal(0.1));
        System.out.println(BigDecimal.valueOf(0.1));
    }
}
```

Output TERMINAL

```
$ java P39.java
false 0
2 1
0.1000000000000000055511151231257827021181583404541015625
0.1
```

`BigDecimal.equals` compares value and scale, so `2.0` and `2.00` are not equal, while `compareTo` ignores scale. `HashSet` uses `equals`/`hashCode` and keeps both; `TreeSet` uses `compareTo` and keeps one. Neither collection is wrong: they define

"same" differently. `new BigDecimal(0.1)` shows the exact binary value of the double; `BigDecimal.valueOf(0.1)` uses the canonical string, which is what you nearly always want.

Puzzle 40. Subtraction comparators (advanced)

P40.java JAVA

```java
import java.util.Arrays;
import java.util.Comparator;

public class P40 {
    public static void main(String[] args) {
        Integer[] a = {Integer.MIN_VALUE, 5, Integer.MAX_VALUE, -3};
        Integer[] bad = a.clone();
        Arrays.sort(bad, (x, y) -> x - y);
        System.out.println(Arrays.toString(bad));
        Integer[] good = a.clone();
        Arrays.sort(good, Comparator.naturalOrder());
        System.out.println(Arrays.toString(good));
    }
}
```

Output TERMINAL

```
$ java P40.java
[5, 2147483647, -2147483648, -3]
[-2147483648, -3, 5, 2147483647]
```

`(x, y) -> x - y` overflows when the operands are far apart: `5 - MIN_VALUE` is negative although 5 is larger, so the comparator is inconsistent and the sort result is garbage (and may even throw `Comparison method violates its general contract!` for larger inputs). Use `Integer.compare(x, y)` or `Comparator.naturalOrder()`. The wrong order shown is what this JDK printed; other inputs fail differently.

Puzzle 41. Arrays: covariance, aliasing and clone (intermediate)

P41.java JAVA

```java
import java.util.ArrayList;
import java.util.List;

public class P41 {
    public static void main(String[] args) {
        Object[] objs = new String[1];
        try {
            objs[0] = 1;
        } catch (ArrayStoreException e) {
            System.out.println("ArrayStoreException: " + e.getMessage());
        }
        int[] a = {1, 2, 3};
        int[] b = a;
        b[0] = 99;
        System.out.println(a[0]);
        int[][] grid = new int[2][];
        System.out.println(grid[0]);
        int[][] rows = new int[2][2];
        int[][] copy = rows.clone();
        copy[0][0] = 5;
        System.out.println(rows[0][0]);
    }
}
```

Output TERMINAL

```
$ java P41.java
ArrayStoreException: java.lang.Integer
99
null
5
```

Arrays are covariant, so `Object[] objs = new String[1]` compiles, but the array remembers its real type and stores check it at run time: `ArrayStoreException`. Assigning an array copies the reference, so `a[0]` sees the change. A new `int[2][]` has `null` rows. `clone()` on a 2D array is shallow: the rows are shared, so `rows[0][0]` changes too.

14.7.7 Generics, records, enums, lambdas, streams and threads

Puzzle 42. Type erasure and heap pollution (advanced)

P42.java JAVA

```java
import java.util.ArrayList;
import java.util.List;

public class P42 {
    public static void main(String[] args) {
        List<String> strings = new ArrayList<>();
        List<Integer> ints = new ArrayList<>();
        System.out.println(strings.getClass() == ints.getClass());
        @SuppressWarnings({"unchecked", "rawtypes"})
        List<Object> raw = (List) strings;
        raw.add(42);
        System.out.println(strings.size());
        try {
            String s = strings.get(0);
            System.out.println(s);
        } catch (ClassCastException e) {
            System.out.println("ClassCastException at the use site");
        }
    }
}
```

Output TERMINAL

```
$ java P42.java
true
1
ClassCastException at the use site
```

Generic type arguments are erased, so at run time both lists are plain `ArrayList` and `getClass()` returns the same class. A raw-type cast lets an `Integer` into a `List<String>` without any error at that point. The failure appears later, at the compiler-inserted cast where the element is used as a `String`. This is the reason to avoid raw types and unchecked casts.

Puzzle 43. Record equality with arrays (intermediate)

P43.java JAVA

```java
import java.util.Arrays;

public class P43 {
    record Pair(String a, int[] nums) { }
    record Money(long cents) { }
    public static void main(String[] args) {
        int[] arr = {1, 2};
        Pair p1 = new Pair("x", arr);
        Pair p2 = new Pair("x", arr);
        Pair p3 = new Pair("x", new int[] {1, 2});
        System.out.println(p1.equals(p2));
        System.out.println(p1.equals(p3));
        System.out.println(new Money(5).equals(new Money(5)));
        System.out.println(new Money(5));
        System.out.println(Arrays.equals(arr, new int[] {1, 2}));
        System.out.println(arr.equals(new int[] {1, 2}));
    }
}
```

Output TERMINAL

```
$ java P43.java
true
false
true
Money[cents=5]
true
false
```

A record's `equals` compares components with `equals` (or `==` for primitives). An `int[]` component uses array identity, so two records holding equal-looking but different arrays are not equal, while records sharing the same array instance are.

`Money` has only a primitive, so value equality works and `toString` is generated. Prefer `List` over arrays in records, or override `equals` and `hashCode`.

Puzzle 44. Enum basics (basic)

P44.java JAVA

```java
public class P44 {
    enum Level { LOW, MID, HIGH }
    public static void main(String[] args) {
        Level a = Level.valueOf("MID");
        System.out.println(a == Level.MID);
        System.out.println(a.ordinal() + " " + a.compareTo(Level.HIGH));
        try {
            Level.valueOf("mid");
        } catch (IllegalArgumentException e) {
            System.out.println(e.getMessage());
        }
        System.out.println(Level.HIGH.getClass() + " " + Level.HIGH.getDeclaringClass());
    }
}
```

Output TERMINAL

```
$ java P44.java
true
1 -1
No enum constant P44.Level.mid
class P44$Level class P44$Level
```

Enum constants are singletons, so `==` is correct and null-safe. `valueOf` is case-sensitive and throws `IllegalArgumentException` for an unknown name. `compareTo` compares ordinals (`MID` is 1, `HIGH` is 2, difference -1). A constant without a body has the enum type as its class; a constant with a body would be an anonymous subclass, which is why `getDeclaringClass()` exists.

Puzzle 45. What a lambda captures (intermediate)

P45.java JAVA

```java
import java.util.ArrayList;
import java.util.List;
import java.util.function.Supplier;

public class P45 {
    public static void main(String[] args) {
        List<Supplier<Integer>> list = new ArrayList<>();
        for (int i = 0; i < 3; i++) {
            int copy = i * 10;
            list.add(() -> copy);
        }
        for (Supplier<Integer> s : list) System.out.print(s.get() + " ");
        System.out.println();
        int[] counter = {0};
        Runnable r = () -> counter[0]++;
        r.run(); r.run(); r.run();
        System.out.println(counter[0]);
        String text = "before";
        Supplier<String> sup = () -> "lambda sees a copy of the value";
        text = "after";
        System.out.println(text + " / " + sup.get());
    }
}
```

Output TERMINAL

```
$ java P45.java
0 10 20
3
after / lambda sees a copy of the value
```

A lambda captures the value of local variables, which must be effectively final. The loop variable `i` is not effectively final, so the code copies it to `copy` first, and each lambda has its own value. To accumulate state, the usual trick is a one-element array (or `AtomicInteger`), because the variable is final but its content is not. Reassigning `text` afterwards is fine here only because the lambda did not capture it.

Puzzle 46. Java is always pass by value (basic)

P46.java JAVA

```java
public class P46 {
    static void change(int n, int[] arr, StringBuilder sb, String s) {
        n = 99;
        arr[0] = 99;
        sb.append("!");
        s = s + "!";
        arr = new int[] {-1};
        sb = new StringBuilder("replaced");
    }
    public static void main(String[] args) {
        int n = 1;
        int[] arr = {1};
        StringBuilder sb = new StringBuilder("sb");
        String s = "s";
        change(n, arr, sb, s);
        System.out.println(n + " " + arr[0] + " " + sb + " " + s);
    }
}
```

Output TERMINAL

```
$ java P46.java
1 99 sb! s
```

The method receives copies of the values: copies of the primitive and of the references. Mutating through the copy of a reference (`arr[0] = 99`, `sb.append`) changes the shared object, while reassigning the parameter (`arr = new int[]...`, `sb = new...`, `s = s + "!"`) only changes the local copy. Strings are immutable, so the caller's `s` can never change.

Puzzle 47. orElse versus orElseGet (intermediate)

P47.java JAVA

```java
import java.util.Optional;

public class P47 {
    static String expensive(String why) {
        System.out.println("computing for " + why);
        return "default";
    }
    public static void main(String[] args) {
        Optional<String> present = Optional.of("value");
        System.out.println(present.orElse(expensive("orElse")));
        System.out.println(present.orElseGet(() -> expensive("orElseGet")));
        Optional<String> empty = Optional.empty();
        System.out.println(empty.orElseGet(() -> expensive("empty")));
        System.out.println(Optional.ofNullable(null).isPresent());
    }
}
```

Output TERMINAL

```
$ java P47.java
computing for orElse
value
value
computing for empty
default
false
```

The argument of `orElse` is an ordinary expression and is always evaluated, even when the `Optional` has a value. `orElseGet` takes a supplier that runs only when needed. Use `orElseGet` when the default is expensive or has side effects.

Puzzle 48. Lazy streams and element-wise processing (intermediate)

P48.java JAVA

```java
import java.util.stream.Stream;

public class P48 {
    public static void main(String[] args) {
        Stream<Integer> s = Stream.of(1, 2, 3, 4);
        s.peek(x -> System.out.println("peek " + x)).map(x -> x * 2);
        System.out.println("no terminal operation yet");
        Stream.of("a", "b", "c")
            .filter(x -> { System.out.println("filter " + x); return true; })
            .map(x -> { System.out.println("map " + x); return x.toUpperCase(); })
            .limit(2)
            .forEach(x -> System.out.println("got " + x));
    }
}
```

Output TERMINAL

```
$ java P48.java
no terminal operation yet
filter a
map a
got A
filter b
map b
got B
```

Intermediate operations only describe the pipeline; with no terminal operation nothing runs, so `peek` printed nothing. Elements flow through the whole pipeline one at a time (filter, map, forEach for `a`, then for `b`), not stage by stage. `limit(2)` stops the pipeline after two elements, so `c` is never touched. This laziness is also why side effects in stream stages are a bad idea.

Puzzle 49. Streams are single use (basic)

P49.java JAVA

```java
import java.util.List;
import java.util.stream.Stream;

public class P49 {
    public static void main(String[] args) {
        Stream<String> s = Stream.of("a", "b");
        System.out.println(s.count());
        try {
            System.out.println(s.count());
        } catch (IllegalStateException e) {
            System.out.println(e.getMessage());
        }
        List<Integer> src = List.of(3, 1, 2);
        src.stream().sorted().forEach(x -> { });
        System.out.println(src);
    }
}
```

Output TERMINAL

```
$ java P49.java
2
stream has already been operated upon or closed
[3, 1, 2]
```

A stream can be consumed once; the second terminal operation throws `IllegalStateException`. Stream operations never modify the source collection: `sorted()` sorts a temporary copy and the list stays `[3, 1, 2]`.

Puzzle 50. run versus start (intermediate)

P50.java JAVA

```java
public class P50 {
    public static void main(String[] args) throws Exception {
        Runnable r = () -> System.out.println("in " + Thread.currentThread().getName());
        Thread t = new Thread(r, "worker");
        t.run();
        t.start();
        t.join();
        try {
            t.start();
        } catch (IllegalThreadStateException e) {
            System.out.println("cannot restart: " + t.getState());
        }
    }
}
```

Output TERMINAL

```
$ java P50.java
in main
in worker
cannot restart: TERMINATED
```

Calling `run()` just calls the method on the current thread, which is why it prints `main`. `start()` creates a new thread that executes `run()`. A thread can be started only once; a second `start` throws `IllegalThreadStateException`, and the thread state is `TERMINATED` after it finishes.

14.7.8 Patterns behind the puzzles

Looking back, almost every puzzle falls into one of a handful of families. Learning the families is faster than memorising fifty answers.

Family	Rule to remember	Puzzles
Identity versus equality	`==` compares references for objects; caches and constant folding make some of them true	boxing, string pool
Evaluation order	operands left to right, `&&` and `||` short-circuit, `&` and `|` do not	increments, ternary, finally
Static versus dynamic	only instance methods are virtual; overloads are chosen at compile time	dispatch, overloads
Numeric promotion	operands widen to `int` at least; compound assignment casts back silently	overflow, shifts, chars
Contracts	equals and hashCode together; comparators must be consistent	HashMap, sorting, BigDecimal
Views and copies	`asList`, `subList`, shallow `clone` share data	arrays, collections
Laziness	streams and `orElseGet` defer work until needed	streams, Optional

Families of puzzles and the rule behind each.

Check yourself

1. What does this print: `Integer a = 127, b = 127; Long c = 127L; System.out.println((a == b) + " " + c.equals(127));`
2. What does this print, and why: `String s = "ja"; String t = s + "va"; System.out.println(t == "java");`
3. What is `x` after `int x = 7; x = x++ * 2 + x;`?
4. What does `System.out.println(1 + 2 + "3" + 4 + 5);` print?
5. What does this print: `char ch = 'a'; System.out.println(ch + 1 + "" + (char) (ch + 1));`
6. What is `sh` after `short sh = 10; sh *= 1000; sh *= 10;` and why does the code compile?

Answers

1. `true false`. 127 is cached, so `a == b`; `Long.equals` rejects an `Integer` argument.
2. `false`. `s` is not a constant, so the concatenation happens at run time and creates a new object; `t.intern() == "java"` would be `true`.
3. `22`. The left `x++` yields 7 and sets `x` to 8, so the expression is `7 * 2 + 8`.
4. `3345`. `1 + 2` is `3`, then string concatenation takes over: `"3" + "3" + 4 + 5`.
5. `98b`. `ch + 1` is an `int` (98), then `""` and the character `b` are appended.
6. `-31072`. `sh *= 10` computes `100000` in `int` and the implicit cast to `short` wraps it. Compound assignment contains a hidden cast, so it compiles.

KEY POINTS

Use these ideas in every output question.

- Name the rule: constant folding, cache range, most specific overload, short-circuit, numeric promotion.
- Reference types compare with `equals`; `==` on boxes and strings is only identity.
- Static initialisers run once per class, instance initialisers once per object, super before sub.
- Only instance methods dispatch dynamically; fields, statics and private methods do not.
- Compound assignment and narrowing casts hide overflow; use `Math.xxxExact` where it matters.
- Equal objects must have equal hash codes, and comparators must never overflow.

14.8 Coding Problems: Strings and Arrays

Coding rounds reward a repeatable method more than memorised answers. For every problem in this chapter we follow the same loop: restate the problem with one example, state the brute-force solution and its cost, ask what work is repeated or wasted, then write the improved version and say its time and space complexity. Say these steps out loud in an interview; the interviewer is grading your thinking as much as your code.

Every solution below was compiled and run on OpenJDK 21. The listings are static methods of one class, `Ch7`, and the driver at the end of the chapter calls all of them on small sample inputs. Complexities use n for the input length. Strings and arrays are the bread and butter of first-round interviews because they test loops, indices and edge cases without needing a library.

14.8.1 The method, in five steps

1. **Clarify.** Ask about empty input, null, duplicates, negative numbers, case sensitivity, and size limits. "Can the array be empty?" is a good question; a wrong assumption costs more than the ten seconds it takes to ask.
2. **Brute force.** State the obvious O(n^2) or O(n^3) approach first. It proves you understand the problem and gives you a correctness oracle.
3. **Find the waste.** Which work repeats? A nested loop that recomputes a sum suggests a running sum; a nested loop that looks for a match suggests a hash set or sorting; a window that restarts suggests two pointers.
4. **Code it cleanly.** Small methods, meaningful names, no clever one-liners.
5. **Test by hand.** Walk one normal case and two edge cases (empty, single element) through your code before saying "done".

Pattern	Typical trigger	Cost after
Two pointers	pair in sorted data, palindromes, in-place filtering	O(n)
Sliding window	longest or shortest contiguous range with a property	O(n)
Frequency array or map	anagrams, first unique, counting	O(n)
Running value (Kadane, min so far)	best subarray, best buy and sell	O(n)
Prefix and suffix products	"everything except me" without division	O(n)
Reversal tricks	rotate, reverse words	O(n), O(1) extra space

Patterns you will use in this chapter.

14.8.2 String problems

Problem 1. Reverse a string and the words of a sentence (basic)

Reversing characters is a two-pointer swap on a `char[]`. Reversing the order of words splits on runs of whitespace and walks the pieces backwards; `trim` plus `split("\\s+")` handles leading, trailing and repeated spaces.

Reversal JAVA

```
static String reverseWords(String s) {
    String[] parts = s.trim().split("\\s+");
    StringBuilder sb = new StringBuilder();
    for (int i = parts.length - 1; i >= 0; i--) {
        sb.append(parts[i]);
        if (i > 0) sb.append(' ');
    }
    return sb.toString();
}

static String reverse(String s) {            // reverse characters, in place on a char[]
    char[] c = s.toCharArray();
    for (int i = 0, j = c.length - 1; i < j; i++, j--) {
        char t = c[i]; c[i] = c[j]; c[j] = t;
    }
    return new String(c);
}
```

Both run in O(n) time. The character reversal needs O(n) extra space for the array copy because `String` is immutable; `new StringBuilder(s).reverse().toString()` does the same in one line, and you may say so, but be ready to write the loop.

Problem 2. Valid palindrome ignoring punctuation (basic)

Two pointers start at the ends and move inward, skipping characters that are not letters or digits. No cleaned copy of the string is built, so the space is O(1).

Palindrome JAVA

```
static boolean isPalindrome(String s) {
    int i = 0, j = s.length() - 1;
    while (i < j) {
        while (i < j && !Character.isLetterOrDigit(s.charAt(i))) i++;
        while (i < j && !Character.isLetterOrDigit(s.charAt(j))) j--;
        if (Character.toLowerCase(s.charAt(i++)) != Character.toLowerCase(s.charAt(j--)))
            return false;
    }
    return true;
}
```

Notice the inner `i < j` guards: without them a string made only of punctuation would run the pointers off the ends.

Problem 3. Valid anagram (basic)

Two strings are anagrams when every letter occurs equally often in both. Sorting both gives O(n log n); a counting array gives O(n) time and O(1) space (26 counters). One array is enough: add for the first string, subtract for the second, and check that all counters return to zero.

Anagram JAVA

```
static boolean isAnagram(String a, String b) {
    if (a.length() != b.length()) return false;
    int[] count = new int[26];
    for (int i = 0; i < a.length(); i++) {
        count[a.charAt(i) - 'a']++;
        count[b.charAt(i) - 'a']--;
    }
    for (int n : count) if (n != 0) return false;
    return true;
}
```

This version assumes lowercase `a` to `z`. For arbitrary Unicode, replace the array with a `HashMap<Character, Integer>` (or `int[65536]` for the Basic Multilingual Plane).

Problem 4. First non-repeating character (basic)

Two passes: count every character, then scan the string in order and return the first index whose count is one. Two passes beat the brute-force "for each character scan the rest" (O(n^2)).

FirstUnique JAVA

```
static int firstUniqueChar(String s) {
    int[] freq = new int[128];
    for (char c : s.toCharArray()) freq[c]++;
    for (int i = 0; i < s.length(); i++)
        if (freq[s.charAt(i)] == 1) return i;
    return -1;
}
```

Problem 5. Longest substring without repeating characters (intermediate)

The brute force tries every start and extends until a repeat appears: O(n^2). The waste is that after a repeat at position j we throw the window away and rebuild it. A **sliding window** keeps two indices, `start` and `end`, and a table of where each character was last seen. When the new character was seen inside the current window, `start` jumps to just past that earlier occurrence.

NoRepeat JAVA

```
static int longestUniqueBrute(String s) {            // O(n^2) time, O(min(n, 128)) space
    int best = 0;
    for (int i = 0; i < s.length(); i++) {
        Set<Character> seen = new HashSet<>();
        for (int j = i; j < s.length() && seen.add(s.charAt(j)); j++)
            best = Math.max(best, j - i + 1);
    }
    return best;
}

static int longestUnique(String s) {                 // O(n) time: sliding window
    int[] lastSeen = new int[128];
    Arrays.fill(lastSeen, -1);
    int best = 0, start = 0;
    for (int end = 0; end < s.length(); end++) {
        char c = s.charAt(end);
        if (lastSeen[c] >= start) start = lastSeen[c] + 1;   // jump past the duplicate
        lastSeen[c] = end;
        best = Math.max(best, end - start + 1);
    }
    return best;
}
```

The condition `lastSeen[c] >= start` matters. A character seen before the window began is not a conflict; without the comparison, input such as `abba` would drag `start` backwards. Each index moves forward only, so the time is O(n); space is O(alphabet).

Problem 6. Run-length compression (basic)

Walk the string, count runs of equal characters, and emit character-plus-count. The loop runs to `i <= length` so the last run is flushed by the same code path as the others. Return the original if the "compressed" form is not shorter.

Compress JAVA

```
static String compress(String s) {
    if (s.isEmpty()) return s;
    StringBuilder sb = new StringBuilder();
    int run = 1;
    for (int i = 1; i <= s.length(); i++) {
        if (i < s.length() && s.charAt(i) == s.charAt(i - 1)) {
            run++;
        } else {
            sb.append(s.charAt(i - 1)).append(run);
            run = 1;
        }
    }
    return sb.length() < s.length() ? sb.toString() : s;
}
```

Building with `StringBuilder` keeps this O(n); concatenating with `+=` in the loop would be O(n^2) because each concatenation copies.

Problem 7. Longest common prefix (basic)

Start with the first word as the candidate prefix and shorten it until every word starts with it. Worst case O(total characters).

Prefix JAVA

```
static String longestCommonPrefix(String[] words) {
    if (words.length == 0) return "";
    String prefix = words[0];
    for (String w : words) {
        while (!w.startsWith(prefix)) prefix = prefix.substring(0, prefix.length() - 1);
    }
    return prefix;
}
```

Problem 8. Longest palindromic substring (intermediate)

A palindrome mirrors around its centre. There are 2n-1 centres (n characters plus n-1 gaps), and expanding outward from each costs O(n), so the whole search is O(n^2) time and O(1) space. A brute force that tests every substring is O(n^3). Dynamic programming also gives O(n^2) but needs O(n^2) memory, so expansion is the better interview answer. (Manacher's algorithm reaches O(n) but is rarely expected.)

LongestPalindrome JAVA

```
static String longestPalindrome(String s) {          // expand around centre: O(n^2), O(1) space
    int lo = 0, hi = 0;
    for (int centre = 0; centre < s.length(); centre++) {
        for (int odd = 0; odd < 2; odd++) {          // odd: centre is a char; even: between two
            int l = centre, r = centre + odd;
            while (l >= 0 && r < s.length() && s.charAt(l) == s.charAt(r)) { l--; r++; }
            if (r - l - 1 > hi - lo) { lo = l + 1; hi = r; }
        }
    }
    return s.substring(lo, hi);
}
```

For `babad` both `bab` and `aba` are valid; the strict `>` keeps the first one found. State that tie-breaking rule out loud.

14.8.3 Array problems

Problem 9. Move zeroes to the end, keeping order (basic)

Keep a `write` index marking where the next non-zero belongs, and swap each non-zero element into it. One pass, in place, O(n) time and O(1) space. Swapping (rather than copying and then zero-filling) keeps the loop to a single pass.

MoveZeroes JAVA

```
static void moveZeroes(int[] a) {
    int write = 0;
    for (int read = 0; read < a.length; read++)
        if (a[read] != 0) {
            int t = a[write]; a[write] = a[read]; a[read] = t;
            write++;
        }
}
```

Problem 10. Rotate an array right by k (intermediate)

The easy answer copies into a new array at `(i + k) % n`: O(n) time, O(n) space. The in-place trick is three reversals: reverse everything, then reverse the first k elements, then the remaining n-k. For `[1..7]` and k = 3: reversing all gives `7 6 5 4 3 2 1`; reversing the first three gives `5 6 7 4 3 2 1`; reversing the last four gives `5 6 7 1 2 3 4`.

Rotate JAVA

```java
static void rotate(int[] a, int k) {                // rotate right by k, O(n) time, O(1) space
    k %= a.length;
    reverse(a, 0, a.length - 1);
    reverse(a, 0, k - 1);
    reverse(a, k, a.length - 1);
}
static void reverse(int[] a, int i, int j) {
    while (i < j) { int t = a[i]; a[i++] = a[j]; a[j--] = t; }
}
```

The `k %= a.length` line handles k larger than n. Rotating an empty array would divide by zero, so guard it if empty input is allowed.

Problem 11. Maximum subarray sum (intermediate)

Brute force sums every subarray: O(n^2) with a running sum (O(n^3) if you recompute sums). **Kadane's algorithm** asks, at each element, "is it better to extend the best subarray ending at the previous element, or to start fresh here?" That is one comparison per element.

MaxSubarray JAVA

```java
static int maxSubarrayBrute(int[] a) {              // O(n^2)
    int best = Integer.MIN_VALUE;
    for (int i = 0; i < a.length; i++) {
        int sum = 0;
        for (int j = i; j < a.length; j++) {
            sum += a[j];
            best = Math.max(best, sum);
        }
    }
    return best;
}

static int maxSubarray(int[] a) {                   // Kadane: O(n) time, O(1) space
    int best = a[0], endingHere = a[0];
    for (int i = 1; i < a.length; i++) {
        endingHere = Math.max(a[i], endingHere + a[i]);   // extend, or start fresh here
        best = Math.max(best, endingHere);
    }
    return best;
}
```

Initialise from `a[0]`, not from 0, or an all-negative array wrongly returns 0 (for `{-3, -1, -2}` the right answer is `-1`).

Problem 12. Best time to buy and sell a stock once (basic)

Brute force tries every buy/sell pair, O(n^2). The waste is looking backwards repeatedly: the best buy day for any sell day is simply the minimum price so far. Track it while scanning.

Stock JAVA

```java
static int maxProfit(int[] prices) {                // one buy, one later sell
    int minSoFar = Integer.MAX_VALUE, best = 0;
    for (int p : prices) {
        minSoFar = Math.min(minSoFar, p);
        best = Math.max(best, p - minSoFar);
    }
    return best;
}
```

Starting `best` at 0 means "do not trade" is a valid answer, which is why the descending prices case returns 0.

Problem 13. Missing number in 0..n (basic)

An array of n distinct numbers drawn from 0..n has exactly one value missing. Sum formula (`n(n+1)/2` minus the actual sum) works but the sum can overflow for large n. XOR is safe: `x ^ x == 0`, so XOR-ing every index and every value cancels all pairs and leaves the missing value.

Missing JAVA

```java
static int missingNumber(int[] a) {                 // a holds 0..n except one value
    int xor = a.length;
    for (int i = 0; i < a.length; i++) xor ^= i ^ a[i];
    return xor;
}
```

Problem 14. Product of array except self (intermediate)

The brute force multiplies the other n-1 elements for each position: O(n^2). Dividing the total product by `a[i]` fails when the array contains zeros (and the question usually forbids division). The fix is two sweeps: the left sweep stores the product of everything to the left in `out[i]`, and the right sweep multiplies in the product of everything to the right using a single running variable.

Product JAVA

```java
static int[] productExceptSelf(int[] a) {           // no division, O(n) time
    int n = a.length;
    int[] out = new int[n];
    out[0] = 1;
    for (int i = 1; i < n; i++) out[i] = out[i - 1] * a[i - 1];   // product of everything left
    int right = 1;
    for (int i = n - 1; i >= 0; i--) {
        out[i] *= right;                                          // times everything right
        right *= a[i];
    }
    return out;
}
```

Time O(n), extra space O(1) beyond the output array. Watch for `int` overflow if the values are large; use `long` or `BigInteger` if the constraints demand it.

Problem 15. Binary search, and the overflow bug (basic)

On sorted data, halve the range each step: O(log n). The classic bug is `mid = (lo + hi) / 2`, which overflows `int` when the sum exceeds 2,147,483,647. That bug sat in the JDK's own `Arrays.binarySearch` and in many textbooks for years; `lo + (hi - lo) / 2` is safe. Returning `-(lo + 1)` for a miss matches `Arrays.binarySearch`: the negative value encodes the insertion point.

BinarySearch JAVA

```java
static int binarySearch(int[] sorted, int target) {
    int lo = 0, hi = sorted.length - 1;
    while (lo <= hi) {
        int mid = lo + (hi - lo) / 2;               // (lo + hi) / 2 can overflow int
        if (sorted[mid] == target) return mid;
        if (sorted[mid] < target) lo = mid + 1; else hi = mid - 1;
    }
    return -(lo + 1);                               // same convention as Arrays.binarySearch
}
```

Problem 16. Majority element (intermediate)

A majority element appears more than n/2 times. A `HashMap` of counts is O(n) time and O(n) space. **Boyer-Moore voting** does it in O(1) space: keep a candidate and a vote count; matching elements add a vote, others cancel one, and when the votes hit zero the next element becomes the candidate. Because the majority element outnumbers everything else combined, it survives all the cancellations.

Majority JAVA

```java
static int majority(int[] a) {                      // Boyer-Moore voting, O(n) time, O(1) space
    int candidate = 0, votes = 0;
    for (int x : a) {
        if (votes == 0) candidate = x;
        votes += (x == candidate) ? 1 : -1;
    }
    return candidate;                               // valid only if a majority is guaranteed
}
```

If a majority is not guaranteed you need a second pass to verify the candidate.

14.8.4 Running all the solutions

Ch7.java (driver, inside main) JAVA

```
System.out.println(reverseWords("  the sky   is blue "));
System.out.println(reverse("stressed"));
String p = "A man, a plan, a canal: Panama";
System.out.println(isPalindrome(p) + " " + isPalindrome("race a car"));
System.out.println(isAnagram("listen", "silent") + " " + isAnagram("rat", "car"));
System.out.println(firstUniqueChar("leetcode") + " " + firstUniqueChar("aabb"));
System.out.println(longestUniqueBrute("abcabcbb") + " " + longestUnique("abcabcbb")
    + " " + longestUnique("pwwkew"));
System.out.println(compress("aabcccccaaa") + " " + compress("abc"));
System.out.println("[" + longestCommonPrefix(new String[]{"flower", "flow", "flight"}) + "] ["
    + longestCommonPrefix(new String[]{"dog", "car"}) + "]");
System.out.println(longestPalindrome("babad") + " " + longestPalindrome("cbbd"));
int[] z = {0, 1, 0, 3, 12};
moveZeroes(z);
System.out.println(Arrays.toString(z));
int[] r = {1, 2, 3, 4, 5, 6, 7};
rotate(r, 3);
System.out.println(Arrays.toString(r));
int[] m = {-2, 1, -3, 4, -1, 2, 1, -5, 4};
System.out.println(maxSubarrayBrute(m) + " " + maxSubarray(m));
int[] up = {7, 1, 5, 3, 6, 4}, down = {7, 6, 4, 3, 1};
System.out.println(maxProfit(up) + " " + maxProfit(down));
System.out.println(missingNumber(new int[]{3, 0, 1}));
System.out.println(Arrays.toString(productExceptSelf(new int[]{1, 2, 3, 4})));
int[] sorted = {1, 3, 5, 7, 9, 11};
System.out.println(binarySearch(sorted, 7) + " " + binarySearch(sorted, 4));
System.out.println(majority(new int[]{2, 2, 1, 1, 1, 2, 2}));
```

Output of the driver TERMINAL

```
$ java Ch7.java
blue is sky the
desserts
true false
true false
0 -1
3 3 3
a2b1c5a3 abc
[fl] []
bab bb
[1, 3, 12, 0, 0]
[5, 6, 7, 1, 2, 3, 4]
6 6
5 0
2
[24, 12, 8, 6]
3 -3
2
```

Every line matches the expected answer for the sample input: for instance 6 is the best subarray `4, -1, 2, 1`, and 5 is the best profit (buy at 1, sell at 6).

INTERVIEW ANGLE

How do you pick the right technique when you see a new array or string problem? Look at what is being asked. "Contiguous range with a property" points to a sliding window or a running value (Kadane). "Pairs or ends" in sorted or palindromic data points to two pointers. "Count or compare letters" points to a frequency array. "Everything except me" points to prefix and suffix passes. If none applies, state the brute force, then ask which work it repeats; the answer usually names the technique.

TIP

Test edge cases before the interviewer asks: empty input, one element, all equal, all negative, already sorted. Most "it passed my example" bugs live in those five.

Check yourself

1. What does `(lo + hi) / 2` evaluate to when `lo = 2_000_000_000` and `hi = 2_100_000_000` (both `int`)? What is the safe formula and its result?
2. What does `longestUnique("abba")` return, and what would go wrong if the test `lastSeen[c] >= start` were replaced by `lastSeen[c] != -1`?
3. What does `compress("aab")` return, and why?
4. What is the time complexity of `rotate` and how many extra array elements does it allocate?
5. What does `maxSubarray(new int[]{-3, -1, -2})` return, and why must the loop initialise from `a[0]`?
6. Why is the sum formula `n*(n+1)/2` a risky way to find the missing number?

Answers

1. `-97483648`, because the `int` sum wraps around to a negative number. The safe form `lo + (hi - lo) / 2` gives `2050000000`.
2. It returns `2` (the window `ba`). With `!= -1`, the second `a` would see the old `a` at index 0, outside the window, and move `start` backwards from 2 to 1. The window then contains the repeated `b`, and the method wrongly returns `3`.
3. `aab`. The compressed form `a2b1` is four characters, longer than the three-character input, so the original is returned.
4. O(n) time (three passes of reversal), zero extra elements: it works in place with O(1) space.
5. It returns `-1`, the largest single element. Initialising `best` at 0 would return 0, which corresponds to an empty subarray that the problem does not allow.
6. For large n the intermediate product or sum can overflow `int` before the subtraction. XOR cannot overflow.

KEY POINTS

Strings and arrays problems are solved by a few reusable patterns:

- Always state the brute force and its cost first, then name the repeated work you are removing.
- Two pointers, sliding windows and running values (Kadane, minimum so far) turn nested loops into one pass.
- Frequency arrays and hash maps trade O(alphabet) or O(n) space for linear time.
- Prefix/suffix passes and reversal tricks give O(1) extra space for "everything except me" and rotation.
- Use `lo + (hi - lo) / 2`, initialise from the first element, and test empty, single and all-negative inputs.

14.9 Coding Problems: Collections, Recursion and Dynamic Programming

The second family of coding problems is about choosing the right data structure and the right way to break a problem into smaller copies of itself. A `HashMap` turns a nested loop into a lookup; a `Deque` turns nested brackets into a stack; recursion with backtracking enumerates choices; dynamic programming (DP) remembers sub-answers so that they are computed once. All solutions here compile and run on OpenJDK 21; the driver and its real output close the chapter. The listings are members of one class, `Ch8`.

14.9.1 Hash maps, stacks, heaps and sorting

Problem 1. Two sum (basic)

Given an array and a target, return the indices of two numbers adding up to it. Brute force checks every pair: O(n^2) time, O(1) space. The waste is searching for the partner by scanning. Store each value's index in a `HashMap` as you go, and for each element look up `target - value` in O(1): O(n) time, O(n) space. One pass is enough because the partner is either already seen (found now) or will find this element later.

TwoSum JAVA

```
static int[] twoSum(int[] nums, int target) {         // O(n) time, O(n) space
    Map<Integer, Integer> indexOf = new HashMap<>();   // value -> index where seen
    for (int i = 0; i < nums.length; i++) {
        Integer j = indexOf.get(target - nums[i]);
        if (j != null) return new int[]{j, i};
        indexOf.put(nums[i], i);
    }
    return new int[0];
}
```

Problem 2. Contains duplicate (basic)

`HashSet.add` returns `false` when the element is already present, so insertion doubles as the test. O(n) time, O(n) space; sorting first and comparing neighbours is O(n log n) time with no extra set.

Duplicates JAVA

```
static boolean hasDuplicate(int[] nums) {
    Set<Integer> seen = new HashSet<>();
    for (int n : nums) if (!seen.add(n)) return true;   // add returns false if already present
    return false;
}
```

Problem 3. Group anagrams (intermediate)

All anagrams share one **canonical key**: the word with its letters sorted. Use that key in a map from key to list of words. Sorting each word costs O(k log k) for word length k, so the total is O(n k log k). `LinkedHashMap` keeps groups in first-seen order, which makes the output deterministic.

GroupAnagrams JAVA

```
static Collection<List<String>> groupAnagrams(String[] words) {
    Map<String, List<String>> groups = new LinkedHashMap<>();
    for (String w : words) {
        char[] c = w.toCharArray();
        Arrays.sort(c);
        groups.computeIfAbsent(new String(c), k -> new ArrayList<>()).add(w);
    }
    return groups.values();
}
```

`computeIfAbsent` creates the list only for the first word of each group. A faster key for lowercase words is a 26-element count array turned into a string, giving O(n k).

Problem 4. Top K frequent words (intermediate)

Count with `merge`, then pick the k largest counts. Sorting all distinct words costs O(m log m) for m distinct words. A **min-heap of size k** costs O(m log k): offer every entry, and whenever the heap grows past k, `poll` removes the weakest. The comparator puts lower counts first and, for equal counts, the alphabetically later word first, so that ties resolve toward the alphabetically smaller word.

TopK JAVA

```
static List<String> topK(String[] words, int k) {      // O(n log k)
    Map<String, Integer> freq = new HashMap<>();
    for (String w : words) freq.merge(w, 1, Integer::sum);
    // min-heap of size k: the weakest of the current top k sits at the head
    PriorityQueue<Map.Entry<String, Integer>> heap = new PriorityQueue<>(
        Map.Entry.<String, Integer>comparingByValue()
            .thenComparing(Map.Entry.<String, Integer>comparingByKey().reversed()));
    for (Map.Entry<String, Integer> e : freq.entrySet()) {
        heap.offer(e);
        if (heap.size() > k) heap.poll();
    }
    List<String> out = new ArrayList<>();
    while (!heap.isEmpty()) out.add(heap.poll().getKey());
    Collections.reverse(out);                           // strongest first
    return out;
}
```

This is the standard use of `PriorityQueue`: the heap is a min-heap, so to keep the k largest you evict the smallest.

Problem 5. Balanced brackets (basic)

A stack matches the most recent opener with each closer. The trick that keeps the code short is to push the closer you expect, so the check on a closing bracket is a single comparison. Use `ArrayDeque` as the stack; `java.util.Stack` is a legacy synchronised class. O(n) time, O(n) space.

Brackets JAVA

```
static boolean balanced(String s) {
    Deque<Character> stack = new ArrayDeque<>();
    for (char c : s.toCharArray()) {
        switch (c) {
            case '(' -> stack.push(')');
            case '[' -> stack.push(']');
            case '{' -> stack.push('}');
            case ')', ']', '}' -> {
                if (stack.isEmpty() || stack.pop() != c) return false;
            }
            default -> { }                              // ignore other characters
        }
    }
    return stack.isEmpty();
}
```

Two failure cases must be handled: a closer arriving on an empty stack (`)(`), and leftover openers at the end (`((`).

Problem 6. Merge intervals (intermediate)

Sort by start. Then one sweep: if the next interval starts after the last output interval ends, there is a gap and it begins a new interval; otherwise they overlap and the last interval's end is extended to the larger of the two ends. Sorting dominates: O(n log n).

MergeIntervals JAVA

```
static int[][] mergeIntervals(int[][] in) {
    int[][] sorted = in.clone();
    Arrays.sort(sorted, Comparator.comparingInt(iv -> iv[0]));
    List<int[]> out = new ArrayList<>();
    for (int[] iv : sorted) {
        if (out.isEmpty() || out.get(out.size() - 1)[1] < iv[0]) {
            out.add(new int[]{iv[0], iv[1]});           // gap: start a new interval
        } else {
            int[] last = out.get(out.size() - 1);
            last[1] = Math.max(last[1], iv[1]);         // overlap: extend the last one
        }
    }
    return out.toArray(new int[0][]);
}
```

Remember `Math.max` when extending: `[1, 10]` followed by `[2, 3]` must keep the end at 10.

Problem 7. LRU cache (intermediate)

A least-recently-used cache evicts the entry that has gone unused longest. In Java the shortest correct answer is `LinkedHashMap` in **access-order** mode, with `removeEldestEntry` overridden to evict when the size passes the capacity. `get` and `put` are O(1).

LruCache JAVA

```
static class LruCache<K, V> extends LinkedHashMap<K, V> {
    private final int capacity;
    LruCache(int capacity) {
        super(16, 0.75f, true);                         // true = access order
        this.capacity = capacity;
    }
    @Override
    protected boolean removeEldestEntry(Map.Entry<K, V> eldest) {
        return size() > capacity;                       // called after every put
    }
}
```

Interviewers often ask for it without `LinkedHashMap`. The classic structure is a hash map from key to node plus a doubly linked list ordered by recency, with sentinel head and tail nodes so that inserting and unlinking never need null checks.

ManualLru JAVA

```java
static class ManualLru {
    private static class Node {
        int key, value; Node prev, next;
        Node(int k, int v) { key = k; value = v; }
    }
    private final int capacity;
    private final Map<Integer, Node> map = new HashMap<>();
    private final Node head = new Node(0, 0), tail = new Node(0, 0);   // sentinels
    ManualLru(int capacity) { this.capacity = capacity; head.next = tail; tail.prev = head; }

    int get(int key) {
        Node n = map.get(key);
        if (n == null) return -1;
        unlink(n); addFirst(n);                       // most recently used goes to the front
        return n.value;
    }
    void put(int key, int value) {
        Node n = map.get(key);
        if (n != null) { n.value = value; unlink(n); addFirst(n); return; }
        if (map.size() == capacity) {                 // evict the node before the tail
            Node lru = tail.prev;
            unlink(lru); map.remove(lru.key);
        }
        n = new Node(key, value);
        map.put(key, n); addFirst(n);
    }
    private void unlink(Node n) { n.prev.next = n.next; n.next.prev = n.prev; }
    private void addFirst(Node n) {
        n.next = head.next; n.prev = head;
        head.next.prev = n; head.next = n;
    }
}
```

`LinkedHashMap` is the same design built into the JDK. Neither version is thread-safe; wrap with `Collections.synchronizedMap` or use a library such as Caffeine in production.

14.9.2 Recursion and backtracking

Problem 8. All permutations (intermediate)

Backtracking builds a candidate step by step: choose, explore, un-choose. The `used` array records which elements are in the current permutation; when the permutation is complete, add a *copy* of it to the result (adding `cur` itself would store the same list n! times, which then ends up empty). There are n! results, so the time is O(n * n!).

Permutations JAVA

```java
static List<List<Integer>> permutations(int[] nums) {
    List<List<Integer>> out = new ArrayList<>();
    backtrack(nums, new boolean[nums.length], new ArrayList<>(), out);
    return out;
}
private static void backtrack(int[] nums, boolean[] used, List<Integer> cur,
                              List<List<Integer>> out) {
    if (cur.size() == nums.length) { out.add(new ArrayList<>(cur)); return; }   // copy!
    for (int i = 0; i < nums.length; i++) {
        if (used[i]) continue;
        used[i] = true; cur.add(nums[i]);             // choose
        backtrack(nums, used, cur, out);              // explore
        cur.remove(cur.size() - 1); used[i] = false;  // un-choose
    }
}
```

Problem 9. All subsets (basic)

The iterative form is neat: start with the empty subset; for each element, copy every existing subset and add the element to the copy. The count doubles each time, so the result has 2^n subsets and the work is O(n * 2^n).

Subsets JAVA

```java
static List<List<Integer>> subsets(int[] nums) {
    List<List<Integer>> out = new ArrayList<>();
    out.add(new ArrayList<>());
    for (int x : nums) {
        int size = out.size();
        for (int i = 0; i < size; i++) {              // every existing subset, plus x
            List<Integer> copy = new ArrayList<>(out.get(i));
            copy.add(x);
            out.add(copy);
        }
    }
    return out;
}
```

14.9.3 Graph search on grids and trees

Problem 10. Number of islands (intermediate)

In a grid of `1` (land) and `0` (water), count connected groups of land. Scan the grid; on meeting unvisited land, count one island and run a **depth-first search** that flips every connected land cell to water so it is never counted again. O(rows * cols) time. The recursion depth can reach the number of cells, so for very large grids use an explicit stack or BFS to avoid `StackOverflowError`.

Islands JAVA

```java
static int countIslands(char[][] grid) {
    int count = 0;
    for (int r = 0; r < grid.length; r++)
        for (int c = 0; c < grid[0].length; c++)
            if (grid[r][c] == '1') { count++; sink(grid, r, c); }
    return count;
}
private static void sink(char[][] g, int r, int c) {   // DFS: flood the whole island
    if (r < 0 || c < 0 || r >= g.length || c >= g[0].length || g[r][c] != '1') return;
    g[r][c] = '0';
    sink(g, r + 1, c); sink(g, r - 1, c); sink(g, r, c + 1); sink(g, r, c - 1);
}
```

Problem 11. Shortest path in a maze (intermediate)

When every step costs the same, **breadth-first search** (BFS) finds the shortest path, because it visits cells in order of distance. Use a queue and a distance table that doubles as the visited set. O(rows * cols).

ShortestPath JAVA

```java
static int shortestPath(char[][] maze) {              // 'S' start, 'E' end, '#' wall
    int rows = maze.length, cols = maze[0].length;
    int[][] dist = new int[rows][cols];
    for (int[] row : dist) Arrays.fill(row, -1);
    Deque<int[]> queue = new ArrayDeque<>();
    for (int r = 0; r < rows; r++)
        for (int c = 0; c < cols; c++)
            if (maze[r][c] == 'S') { queue.add(new int[]{r, c}); dist[r][c] = 0; }
    int[][] dirs = {{1, 0}, {-1, 0}, {0, 1}, {0, -1}};
    while (!queue.isEmpty()) {
        int[] cur = queue.poll();
        if (maze[cur[0]][cur[1]] == 'E') return dist[cur[0]][cur[1]];
        for (int[] d : dirs) {
            int nr = cur[0] + d[0], nc = cur[1] + d[1];
            if (nr >= 0 && nc >= 0 && nr < rows && nc < cols
                    && maze[nr][nc] != '#' && dist[nr][nc] == -1) {
                dist[nr][nc] = dist[cur[0]][cur[1]] + 1;
                queue.add(new int[]{nr, nc});
            }
        }
    }
    return -1;
}
```

DFS goes deep first and uses a stack (or recursion); BFS goes level by level and uses a queue. For shortest paths in unweighted graphs, choose BFS; for weighted graphs use Dijkstra.

Problem 12. Level-order traversal of a binary tree (basic)

BFS on a tree. The inner loop runs exactly `queue.size()` times, which is the number of nodes on the current level, so each level lands in its own list. A recursive `height` shows the other natural style: the answer for a node is built from the answers of its children.

LevelOrder JAVA

```
record TreeNode(int val, TreeNode left, TreeNode right) {}

static List<List<Integer>> levelOrder(TreeNode root) {
    List<List<Integer>> out = new ArrayList<>();
    if (root == null) return out;
    Deque<TreeNode> queue = new ArrayDeque<>();
    queue.add(root);
    while (!queue.isEmpty()) {
        int width = queue.size();                          // nodes on this level
        List<Integer> level = new ArrayList<>();
        for (int i = 0; i < width; i++) {
            TreeNode n = queue.poll();
            level.add(n.val());
            if (n.left() != null) queue.add(n.left());
            if (n.right() != null) queue.add(n.right());
        }
        out.add(level);
    }
    return out;
}

static int height(TreeNode n) {
    return n == null ? 0 : 1 + Math.max(height(n.left()), height(n.right()));
}
```

14.9.4 Dynamic programming basics

DP applies when a problem has **overlapping subproblems** (the same sub-question recurs) and **optimal substructure** (the best answer is built from best answers of the parts). The recipe: define what `dp[i]` means in words, write the recurrence, fix the base case, and choose the evaluation order. Top-down (recursion plus a memo table) and bottom-up (a loop filling a table) give the same answer.

Problem 13. Fibonacci in three costs (basic)

The naive recursion recomputes the same values and takes O(2^n) time. Memoising each result brings it to O(n). The bottom-up loop needs only the last two values: O(n) time, O(1) space. `long` overflows at `fib(93)`, which is why the driver stops at 90.

Fibonacci JAVA

```
static long fibNaive(int n) { return n < 2 ? n : fibNaive(n - 1) + fibNaive(n - 2); }  // O(2^n)

static long fibMemo(int n, long[] memo) {                  // top-down: O(n) time, O(n) space
    if (n < 2) return n;
    if (memo[n] != 0) return memo[n];
    return memo[n] = fibMemo(n - 1, memo) + fibMemo(n - 2, memo);
}

static long fibBottomUp(int n) {                           // bottom-up: O(n) time, O(1) space
    long a = 0, b = 1;
    for (int i = 0; i < n; i++) { long t = a + b; a = b; b = t; }
    return a;
}
```

The climbing-stairs problem (one or two steps at a time) is the same recurrence with different base cases.

Problem 14. Coin change, fewest coins (intermediate)

`dp[a]` is the fewest coins that make amount `a`. For each coin that fits, `dp[a] = min(dp[a], dp[a - coin] + 1)`. Starting every cell at `amount + 1` acts as infinity without risking overflow. Time O(amount * coins), space O(amount). A greedy "take the largest coin" fails: for coins 1, 5, 11 and amount 15 greedy picks 11 + 1 + 1 + 1 + 1 (five coins) while the best is 5 + 5 + 5.

CoinChange JAVA

```
static int coinChange(int[] coins, int amount) {          // fewest coins, or -1
    int[] dp = new int[amount + 1];                       // dp[a] = fewest coins to make a
    Arrays.fill(dp, amount + 1);                          // "infinity"
    dp[0] = 0;
    for (int a = 1; a <= amount; a++)
        for (int coin : coins)
            if (coin <= a) dp[a] = Math.min(dp[a], dp[a - coin] + 1);
    return dp[amount] > amount ? -1 : dp[amount];
}
```

Problem 15. Longest common subsequence (intermediate)

`dp[i][j]` is the LCS length of the first i characters of `a` and the first j of `b`. If the last characters match, extend the diagonal; otherwise take the better of dropping one character from either string. O(n * m) time and space; keeping only two rows reduces space to O(min(n, m)) when you need just the length.

Lcs JAVA

```
static int lcs(String a, String b) {                      // O(n*m) time and space
    int[][] dp = new int[a.length() + 1][b.length() + 1];
    for (int i = 1; i <= a.length(); i++)
        for (int j = 1; j <= b.length(); j++)
            dp[i][j] = a.charAt(i - 1) == b.charAt(j - 1)
                ? dp[i - 1][j - 1] + 1
                : Math.max(dp[i - 1][j], dp[i][j - 1]);
    return dp[a.length()][b.length()];
}
```

Problem 16. 0/1 knapsack (intermediate)

Each item can be taken once. `dp[w]` is the best value achievable with capacity `w`. The detail interviewers probe is the loop direction: iterating `w` **downwards** guarantees each item is considered at most once, because `dp[w - weight]` still holds the value from before this item. Iterating upwards would let an item be reused, which solves the *unbounded* knapsack instead.

Knapsack JAVA

```
static int knapsack(int[] weight, int[] value, int capacity) {   // 0/1: each item once
    int[] dp = new int[capacity + 1];                     // dp[w] = best value within weight w
    for (int i = 0; i < weight.length; i++)
        for (int w = capacity; w >= weight[i]; w--)       // downwards: reuse is impossible
            dp[w] = Math.max(dp[w], dp[w - weight[i]] + value[i]);
    return dp[capacity];
}
```

14.9.5 Running all the solutions

Ch8.java (driver, inside main) JAVA

```
System.out.println(Arrays.toString(twoSum(new int[]{2, 7, 11, 15}, 9)));
System.out.println(hasDuplicate(new int[]{1, 2, 3, 1}) + " " + hasDuplicate(new int[]{1, 2, 3}));
System.out.println(groupAnagrams(new String[]{"eat", "tea", "tan", "ate", "nat", "bat"}));
String[] words = {"b", "a", "c", "a", "b", "a", "c", "d"};
System.out.println(topK(words, 2));
System.out.println(balanced("{[()]}") + " " + balanced("([)]") + " " + balanced("(("));
int[][] merged = mergeIntervals(new int[][]{{8, 10}, {1, 3}, {2, 6}, {15, 18}, {17, 20}});
System.out.println(Arrays.deepToString(merged));

LruCache<String, Integer> cache = new LruCache<>(2);
cache.put("a", 1); cache.put("b", 2);
cache.get("a");                                  // a is now most recent
cache.put("c", 3);                               // evicts b
System.out.println(cache.keySet());
ManualLru lru = new ManualLru(2);
lru.put(1, 10); lru.put(2, 20);
lru.get(1); lru.put(3, 30);                      // evicts key 2
System.out.println(lru.get(2) + " " + lru.get(1) + " " + lru.get(3));

System.out.println(permutations(new int[]{1, 2, 3}));
System.out.println(subsets(new int[]{1, 2, 3}));
char[][] grid = {"11000".toCharArray(), "11000".toCharArray(),
                 "00100".toCharArray(), "00011".toCharArray()};
System.out.println(countIslands(grid));
char[][] maze = {"S.#.".toCharArray(), "..#E".toCharArray(), "....".toCharArray()};
System.out.println(shortestPath(maze));
System.out.println(fibNaive(20) + " " + fibMemo(50, new long[51]) + " " + fibBottomUp(90));
System.out.println(coinChange(new int[]{1, 5, 11}, 15) + " " + coinChange(new int[]{2}, 3));
System.out.println(lcs("ABCBDAB", "BDCABA"));
System.out.println(knapsack(new int[]{1, 3, 4, 5}, new int[]{1, 4, 5, 7}, 7));
TreeNode tree = new TreeNode(3, new TreeNode(9, null, null),
    new TreeNode(20, new TreeNode(15, null, null), new TreeNode(7, null, null)));
System.out.println(levelOrder(tree) + " height=" + height(tree));
```

Output of the driver TERMINAL

```
$ java Ch8.java
[0, 1]
true false
[[eat, tea, ate], [tan, nat], [bat]]
[a, b]
true false false
[[1, 6], [8, 10], [15, 20]]
[a, c]
-1 10 30
[[1, 2, 3], [1, 3, 2], [2, 1, 3], [2, 3, 1], [3, 1, 2], [3, 2, 1]]
[[], [1], [2], [1, 2], [3], [1, 3], [2, 3], [1, 2, 3]]
3
6
6765 12586269025 2880067194370816120
3 -1
4
9
[[3], [9, 20], [15, 7]] height=3
```

Check a few by hand: the best coin change for 15 is 3 coins; the LCS of `ABCBDAB` and `BDCABA` is 4; the knapsack picks the items of weight 3 and 4 for value 9; and the maze path of length 6 goes down, right along the bottom row, then up to `E`.

INTERVIEW ANGLE

When do you use DP instead of plain recursion? When the recursion tree contains the same subproblem many times (overlapping subproblems) and the best answer for a problem can be assembled from best answers to its parts. Naive Fibonacci is the textbook example: 2^n calls but only n distinct inputs. Add a memo table (top-down) or fill a table in dependency order (bottom-up) and the time drops to the number of distinct states times the work per state.

GOTCHA

In backtracking, `out.add(cur)` stores a reference to a list that you keep mutating. Always add `new ArrayList<>(cur)`. The symptom is a result full of identical (often empty) lists.

Check yourself

1. What is the complexity of `twoSum` and why is one pass enough?
2. Why does a min-heap of size k find the k largest elements?
3. What is wrong with `if (cur.size() == n) out.add(cur);` in a permutation generator?
4. In 0/1 knapsack, what changes if the inner loop is written `for (int w = weight[i]; w <= capacity; w++)`? Test with weights `{2}`, values `{3}`, capacity 6.
5. What does `balanced("(]")` return and what is on the stack when the `]` arrives?
6. Why does the greedy strategy fail for coin change with coins 1, 5, 11 and amount 15?

Answers

1. O(n) time and O(n) space. Every pair (i, j) with i < j is found when j is processed, because the earlier element i is already in the map.
2. The heap head is the smallest of the k kept so far. Any new element larger than the head replaces it, so what remains after all elements are seen are the k largest. Each offer or poll costs O(log k).
3. It stores the same mutable list repeatedly. As backtracking removes elements, every stored reference ends up empty. Add a copy: `new ArrayList<>(cur)`.
4. The item can then be used repeatedly (unbounded knapsack). With weight 2, value 3 and capacity 6 the code returns 9 instead of the correct 3.
5. `false`. The `(` pushed `)`, so the stack holds `)` and `]` does not equal it, so `pop() != c` is true.
6. Greedy takes 11 first and needs four 1s (5 coins); the optimum is 5 + 5 + 5 (3 coins). Locally best choices do not compose into a global optimum, which is why DP is needed.

KEY POINTS

Collections and DP problems come down to choosing state and structure:

- A `HashMap` or `HashSet` replaces an inner search loop; a `Deque` is the stack or queue; a `PriorityQueue` of size k solves top-k in O(n log k).
- Backtracking is choose, explore, un-choose, and results must be copies.
- BFS finds shortest paths in unweighted graphs; DFS counts or explores components.
- DP needs a clear meaning for each state, a recurrence and a base case; loop direction matters (knapsack).
- `LinkedHashMap` with access order and `removeEldestEntry` is a complete LRU cache; know the hand-written list-plus-map design too.

14.10 Concurrency Coding Tasks

Concurrency questions in interviews rarely ask you to recite the Java Memory Model. They ask you to write a small, correct, thread-safe program on a whiteboard: "print odd and even numbers from two threads in order", "implement a producer-consumer", "fix this deadlock". This chapter gives eight such tasks. Each has the problem, the idea, a complete solution and the real output of running it on OpenJDK 21. The listings are members of one class, `Ch9`, and a single driver at the end runs them all. For the theory behind the tools used here (monitors, `volatile`, the executors) see the concurrency Part earlier in the book.

14.10.1 Tools you will reach for

Before writing code, decide which of these a task needs. Choosing the tool is most of the answer.

Need	Tool	Why
Hand-off between threads	`BlockingQueue` (`ArrayBlockingQueue`)	built-in waiting and back-pressure
Alternate turns	`wait`/`notifyAll` or `Semaphore`	explicit signalling
Counter shared by many threads	`AtomicInteger`, `LongAdder`	lock-free; `synchronized` also works
Shared map with compute-if-absent	`ConcurrentHashMap.computeIfAbsent`	atomic per key
Several locks held together	one global lock order, or `tryLock` with timeout	prevents circular wait
Split work and combine results	`ExecutorService` + `Future`, `ForkJoinPool`	pooled threads, results by `get`/`join`

Concurrency tools and when to choose them.

WATCH OUT
Never create threads with `new Thread` in production code for task-style work; use an `ExecutorService`. The solutions below use raw threads in two tasks only because the task is *about* thread coordination.

14.10.2 Task 1. Print odd and even numbers alternately with two threads

One thread prints odd numbers, the other even numbers, and the output must interleave in order. The shared state is the next number to print. A thread whose turn it is not waits on the monitor; after printing it calls `notifyAll` so the partner re-checks its condition. The `wait` call is always inside a `while` loop that re-tests the condition, because of spurious wakeups and because `notifyAll` wakes every waiter.

OddEven JAVA

```
static class OddEven {
    private final Object lock = new Object();
    private int next = 1;                                  // guarded by lock
    private final int max;
    OddEven(int max) { this.max = max; }

    void print(boolean odd, StringBuilder out) throws InterruptedException {
        synchronized (lock) {
            while (next <= max) {
                while (next <= max && (next % 2 == 1) != odd) lock.wait();   // not my turn
                if (next > max) break;
                out.append(odd ? "odd:" : "even:").append(next++).append(' ');
                lock.notifyAll();                          // wake the partner
            }
            lock.notifyAll();                              // let a waiting partner exit
        }
    }
}
```

What to notice: all reads and writes of `next` happen while holding `lock`; the final `notifyAll` after the loop frees a partner that is still waiting when the last number has been printed. Without it one thread can wait forever, a hang you only see about once in a hundred runs. A `Semaphore` pair (each thread releases the other's permit) is an equally good answer.

14.10.3 Task 2. Producer-consumer with a bounded queue

A producer creates items faster than a consumer processes them. A bounded `BlockingQueue` solves two problems at once: `put` blocks when the queue is full (back-pressure, so memory stays bounded) and `take` blocks when it is empty (no busy-waiting). A **poison pill**, a special value, tells the consumer to stop.

ProducerConsumer JAVA

```
static long producerConsumer() throws Exception {
    BlockingQueue<Integer> queue = new ArrayBlockingQueue<>(4);   // bounded: back-pressure
    final int POISON = -1;                                 // tells the consumer to stop
    AtomicLong sum = new AtomicLong();
    Thread producer = new Thread(() -> {
        try {
            for (int i = 1; i <= 100; i++) queue.put(i);          // blocks when full
            queue.put(POISON);
        } catch (InterruptedException e) { Thread.currentThread().interrupt(); }
    });
    Thread consumer = new Thread(() -> {
        try {
            for (int v; (v = queue.take()) != POISON; ) sum.addAndGet(v);   // blocks when empty
        } catch (InterruptedException e) { Thread.currentThread().interrupt(); }
    });
    producer.start(); consumer.start();
    producer.join(); consumer.join();
    return sum.get();
}
```

Handling `InterruptedException` by restoring the interrupt flag (`Thread.currentThread().interrupt()`) is the correct habit; swallowing it hides shutdown requests. With several consumers you need one poison pill per consumer.

14.10.4 Task 3. Create a deadlock, then fix it

Two threads each need two locks. Thread 1 takes A then B; thread 2 takes B then A. If each grabs its first lock before the other grabs its second, both wait forever. That is the four-part deadlock condition (mutual exclusion, hold and wait, no pre-emption, **circular wait**). Break any one and it disappears; the cheapest to break is circular wait by giving all locks a global order.

Deadlock JAVA

```
static class Account {
    final int id; long balance;
    final Lock lock = new ReentrantLock();
    Account(int id, long balance) { this.id = id; this.balance = balance; }
}

static void transferBad(Account from, Account to, long amount) {
    from.lock.lock();                                   // thread 1: A then B
    try {                                               // thread 2: B then A  -> deadlock
        sleepQuietly(50);
        to.lock.lock();
        try { from.balance -= amount; to.balance += amount; } finally { to.lock.unlock(); }
    } finally { from.lock.unlock(); }
}

static void transferGood(Account from, Account to, long amount) {
    Account first = from.id < to.id ? from : to;        // always lock the lower id first
    Account second = first == from ? to : from;
    first.lock.lock();
    try {
        sleepQuietly(50);
        second.lock.lock();
        try { from.balance -= amount; to.balance += amount; } finally { second.lock.unlock(); }
    } finally { first.lock.unlock(); }
}
```

The `sleepQuietly(50)` calls only make the bad interleaving certain for the demo. The good version orders locks by account id, so both threads lock account 1 before account 2 whichever direction the money flows. An alternative that avoids ordering is `tryLock(timeout)` on the second lock, releasing the first and retrying on failure. The driver starts each pair as daemon threads, waits half a second, and reports how many are still stuck; for the bad version the JVM itself can name them through `ThreadMXBean.findDeadlockedThreads()` (the same information `jstack` prints as "Found one Java-level deadlock").

INTERVIEW ANGLE

How do you prevent and detect deadlocks? Prevent: acquire locks in a single global order, hold locks for as short a time as possible, never call unknown code (callbacks, overridable methods) while holding a lock, or use `tryLock` with a timeout. Detect: `jstack <pid>` or `jcmd <pid> Thread.print` lists Java-level deadlocks, and `ThreadMXBean.findDeadlockedThreads()` does it programmatically.

14.10.5 Task 4. A thread-safe counter

`count++` is three steps (read, add, write), so two threads can both read 5 and both write 6. The experiment below runs four threads, each incrementing four counters 100,000 times: a plain `int`, a `synchronized` method, an `AtomicInteger` and a `LongAdder`.

Counters JAVA

```
static class Counters {
    int plain;                                          // NOT thread-safe
    int synced;
    final AtomicInteger atomic = new AtomicInteger();
    final LongAdder adder = new LongAdder();
    synchronized void incSynced() { synced++; }
}

static Counters countWithThreads(int threads, int perThread) throws Exception {
    Counters c = new Counters();
    ExecutorService pool = Executors.newFixedThreadPool(threads);
    for (int t = 0; t < threads; t++) {
        pool.submit(() -> {
            for (int i = 0; i < perThread; i++) {
                c.plain++;                              // read-modify-write: updates get lost
```

Counters (continued) JAVA

```
                c.incSynced();
                c.atomic.incrementAndGet();
                c.adder.increment();
            }
        });
    }
    pool.shutdown();
    pool.awaitTermination(30, TimeUnit.SECONDS);
    return c;
}
```

The plain counter loses updates; the other three always reach exactly 400,000. `AtomicInteger` uses a hardware compare-and-set loop. `LongAdder` spreads contention over several cells and adds them up in `sum()`, which wins under heavy contention but is only a snapshot while writers run. Use `synchronized` when the counter must change together with other fields (an invariant spanning several variables), atomics for a single independent counter.

14.10.6 Task 5. A thread-safe cache

`computeIfAbsent` on a `ConcurrentHashMap` is atomic for the key: if many threads ask for the same missing key, the loader runs once and the others wait for the value. The check-then-act version (`if (!map.containsKey(k)) map.put(k, load(k))`) is not atomic and may load the same key repeatedly.

Cache JAVA

```
static class Cache<K, V> {
    private final ConcurrentHashMap<K, V> map = new ConcurrentHashMap<>();
    private final java.util.function.Function<K, V> loader;
    Cache(java.util.function.Function<K, V> loader) { this.loader = loader; }

    V get(K key) {
        return map.computeIfAbsent(key, loader);        // atomic per key: loader runs once
    }
}
```

Keep the loader short and never let it touch the same map: `computeIfAbsent` holds an internal lock on the bin while the loader runs, and a loader that updates the map can deadlock or throw. If loading is slow, cache a `CompletableFuture<V>` instead.

14.10.7 Task 6. A thread-safe singleton

Three correct answers. The **holder idiom** relies on the JVM's class initialisation guarantee: `Holder` is initialised once, lazily, under the class-init lock, so no explicit synchronisation is needed. The **enum singleton** is the simplest and is also safe against serialization and reflection attacks. **Double-checked locking** works only if the field is `volatile`; mention it, but prefer the first two.

Singleton JAVA

```
static class Config {
    static final AtomicInteger created = new AtomicInteger();
    private Config() { created.incrementAndGet(); }
    private static class Holder {                       // loaded lazily, once, thread-safely
        static final Config INSTANCE = new Config();
    }
    static Config getInstance() { return Holder.INSTANCE; }
}

enum Registry { INSTANCE; int hits; }                   // the enum singleton
```

14.10.8 Task 7. A rate limiter (token bucket)

Allow bursts up to `capacity` requests and a sustained `refillPerSecond`. State: the number of tokens and the time of the last refill. On each request, add the tokens earned since last time (capped at capacity), then spend one if at least one is available. The class takes the clock as a `LongSupplier` so a test can control time instead of sleeping.

TokenBucket JAVA

```
static class TokenBucket {
    private final long capacity, refillPerSecond;
    private final LongSupplier nanoClock;              // injectable, so tests are deterministic
    private double tokens;                             // guarded by this
    private long lastRefill;

    TokenBucket(long capacity, long refillPerSecond, LongSupplier nanoClock) {
        this.capacity = capacity; this.refillPerSecond = refillPerSecond;
        this.nanoClock = nanoClock; this.tokens = capacity; this.lastRefill = nanoClock.getAsLong();
    }

    synchronized boolean tryAcquire() {
        long now = nanoClock.getAsLong();
        tokens = Math.min(capacity, tokens + (now - lastRefill) / 1e9 * refillPerSecond);
        lastRefill = now;
        if (tokens >= 1) { tokens -= 1; return true; }
        return false;
    }
}
```

All state changes happen in one `synchronized` method, so refilling and spending are a single atomic step. For a distributed system the same logic lives in a shared store such as Redis; libraries like Bucket4j or Resilience4j provide production versions.

14.10.9 Task 8. Parallel sum

Sum a large array using several threads. Three styles are shown: manual chunks submitted to a fixed pool (the pool lives only for the call and is shut down in `finally`), a `RecursiveTask` that splits until the pieces are small, and a parallel stream.

ParallelSum JAVA

```
static long sumWithPool(long[] data, int parts) throws Exception {
    ExecutorService pool = Executors.newFixedThreadPool(parts);
    try {
        List<Future<Long>> futures = new ArrayList<>();
        int chunk = (data.length + parts - 1) / parts;
        for (int p = 0; p < parts; p++) {
            int from = p * chunk, to = Math.min(data.length, from + chunk);
            futures.add(pool.submit(() -> {
                long s = 0;
                for (int i = from; i < to; i++) s += data[i];
                return s;
            }));
        }
        long total = 0;
        for (Future<Long> f : futures) total += f.get();
        return total;
    } finally {
        pool.shutdown();
    }
}

static class SumTask extends RecursiveTask<Long> {
    private final long[] data; private final int from, to;
    SumTask(long[] data, int from, int to) { this.data = data; this.from = from; this.to = to; }
    @Override protected Long compute() {
        if (to - from <= 10_000) {                     // small enough: do it directly
            long s = 0;
            for (int i = from; i < to; i++) s += data[i];
            return s;
        }
        int mid = (from + to) >>> 1;
        SumTask left = new SumTask(data, from, mid);
        left.fork();                                   // run the left half asynchronously
        long right = new SumTask(data, mid, to).compute();
        return left.join() + right;
    }
}
```

Parallelism pays only when the work per element is large enough to outweigh splitting and thread hand-off. Summing a million numbers is so cheap that the sequential loop is typically as fast or faster; this task is about the structure, and you should say so in an interview.

14.10.10 Running all eight tasks

Ch9.java (helper) JAVA

```
static void printTurns(OddEven oe, boolean odd, StringBuilder out) {
    try {
        oe.print(odd, out);
    } catch (InterruptedException e) {
        Thread.currentThread().interrupt();
    }
}
```

Ch9.java (driver, inside main) JAVA

```
StringBuilder out = new StringBuilder();
OddEven oe = new OddEven(8);
Thread odd = new Thread(() -> printTurns(oe, true, out));
Thread even = new Thread(() -> printTurns(oe, false, out));
odd.start(); even.start(); odd.join(); even.join();
System.out.println(out.toString().trim());

System.out.println("queue sum = " + producerConsumer());

Account a = new Account(1, 100), b = new Account(2, 100);
int bad = stuckThreads(() -> transferBad(a, b, 10), () -> transferBad(b, a, 20));
System.out.println("bad transfer: stuck threads = " + bad
    + ", JVM reports deadlocked = " + jvmReportedDeadlocks());
Account c = new Account(1, 100), d = new Account(2, 100);
int good = stuckThreads(() -> transferGood(c, d, 10), () -> transferGood(d, c, 20));
System.out.println("good transfer: stuck threads = " + good
    + ", balances = " + c.balance + "/" + d.balance);

Counters counters = countWithThreads(4, 100_000);
System.out.println("expected 400000: synced=" + counters.synced + " atomic=" + counters.atomic
    + " adder=" + counters.adder.sum() + " plain lost updates=" + (counters.plain != 400_000));

AtomicInteger loads = new AtomicInteger();
Cache<Integer, String> cache = new Cache<>(k -> { loads.incrementAndGet(); return "v" + k; });
ExecutorService pool = Executors.newFixedThreadPool(8);
for (int i = 0; i < 1000; i++) { int key = i % 5; pool.submit(() -> cache.get(key)); }
pool.shutdown(); pool.awaitTermination(10, TimeUnit.SECONDS);
System.out.println("cache loads for 5 keys, 1000 gets = " + loads.get());

ExecutorService pool2 = Executors.newFixedThreadPool(8);
Set<Config> seen = ConcurrentHashMap.newKeySet();
for (int i = 0; i < 100; i++) pool2.submit(() -> seen.add(Config.getInstance()));
pool2.shutdown(); pool2.awaitTermination(10, TimeUnit.SECONDS);
boolean enumSame = Registry.INSTANCE == Registry.valueOf("INSTANCE");
System.out.println("singleton instances = " + seen.size() + ", constructor calls = "
    + Config.created.get() + ", enum same = " + enumSame);

long[] fakeNanos = {0};
TokenBucket bucket = new TokenBucket(3, 2, () -> fakeNanos[0]);   // 3 burst, 2 per second
StringBuilder rl = new StringBuilder();
for (int i = 0; i < 5; i++) rl.append(bucket.tryAcquire() ? 'Y' : 'n');
fakeNanos[0] += 1_000_000_000L;                       // one second passes: 2 tokens
for (int i = 0; i < 3; i++) rl.append(bucket.tryAcquire() ? 'Y' : 'n');
System.out.println("rate limiter: " + rl);

long[] data = LongStream.rangeClosed(1, 1_000_000).toArray();
System.out.println(sumWithPool(data, 4) + " " + ForkJoinPool.commonPool().invoke(
    new SumTask(data, 0, data.length)) + " " + LongStream.of(data).parallel().sum());
```

Output of the driver TERMINAL

```
$ java Ch9.java
odd:1 even:2 odd:3 even:4 odd:5 even:6 odd:7 even:8
queue sum = 5050
bad transfer: stuck threads = 2, JVM reports deadlocked = 2
good transfer: stuck threads = 0, balances = 110/90
expected 400000: synced=400000 atomic=400000 adder=400000 plain lost updates=true
cache loads for 5 keys, 1000 gets = 5
singleton instances = 1, constructor calls = 1, enum same = true
rate limiter: YYYnnYYn
500000500000 500000500000 500000500000
```

Every line is deterministic except the content of the lost-update count itself: the driver prints only whether the plain counter lost updates, which in practice it does with four threads, but that outcome is a race, so treat `plain lost updates=true` as typical rather than guaranteed. The numbers `500000500000` are n(n+1)/2 for n = 1,000,000.

REAL WORLD

In Spring applications you rarely write these primitives yourself: `@Async` and `TaskExecutor` give you pools, Caffeine or Spring's cache abstraction give you caches, and the container manages singletons. You still need this chapter to read thread dumps, to reason about shared mutable state in a singleton bean, and to answer the questions.

Check yourself

1. Why must `wait` be called inside a `while` loop and not an `if`?
2. In `OddEven`, what could go wrong if the final `lock.notifyAll()` after the outer loop were removed?
3. What is wrong with this cache code, and how do you fix it? `if (!map.containsKey(k)) { map.put(k, load(k)); } return map.get(k);` on a `ConcurrentHashMap`.
4. What will `transferGood(a, b, 5)` and `transferGood(b, a, 5)` running concurrently never do, and which of the four deadlock conditions does the id ordering break?
5. A `TokenBucket(3, 2, clock)` is asked five times at time zero, then three times after one second has passed. What pattern of results do you get?
6. Why is `LongAdder.sum()` not a precise "current value" while other threads are still incrementing?

Answers

1. A woken thread must re-test its condition: another thread may have changed the state first, `notifyAll` wakes all waiters, and spurious wakeups are allowed. With `if`, the thread would proceed on a false condition.
2. If one thread prints the last number and exits the loop while its partner is still waiting, nobody wakes the partner and it waits forever, so the program never finishes.
3. The check and the put are separate steps, so two threads can both see the key missing and both load. Use `map.computeIfAbsent(k, this::load)`, which is atomic per key.
4. They never deadlock, because both threads lock account 1 before account 2. The ordering breaks **circular wait**.
5. `YYYnn` then `YYn`: the burst of three succeeds and the next two fail; after one second at 2 tokens per second two tokens have been added, so the next two succeed and the third fails. The driver output shows `YYYnnYYn`.
6. `sum()` adds the cells one by one without stopping writers, so increments made during the summation may or may not be included. It is exact once the writers have finished.

KEY POINTS

Concurrency tasks are won by choosing the right tool and protecting every shared access:

- Alternating threads need a shared state variable, `wait` in a `while`, and `notifyAll` (or semaphores).
- A bounded `BlockingQueue` gives hand-off and back-pressure; stop consumers with a poison pill per consumer.
- Break deadlocks by acquiring locks in one global order; detect them with `jstack` or `ThreadMXBean`.
- Use atomics or `synchronized` for counters, `computeIfAbsent` for caches, the holder idiom or an enum for singletons.
- A token bucket is two numbers (tokens, last refill) updated atomically; inject the clock to test it.

14.11 Output Prediction and Debugging Drills

The last chapter of this Part trains the skill that separates a person who has written Java from a person who can maintain it: reading a short, plausible program, predicting what it really does, and fixing it. Each drill below has the same shape. First read the buggy code and decide what you expect. Then look at the real output from OpenJDK 21, the diagnosis, and the fix. Cover the diagnosis with your hand before reading it; the learning is in the attempt. All ten drills are members of one class, `Ch10`, whose `main` prints a line prefixed `D1:` to `D10:` per observation; each drill shows only its own lines.

14.11.1 A habit for debugging by reading

Ask these four questions of any suspicious snippet, in this order. They find most bugs without a debugger.

1. **What are the types, exactly?** `int` or `long`, `Integer` or `int`, `Object` or a subclass. Many bugs are an implicit conversion.
2. **Who owns this object?** Is it shared, mutated after being stored, or backed by something else?
3. **When does each piece run?** Constructor order, lazy streams, which thread.
4. **What does the API promise?** Does the method return a new value, or mutate? Does it throw on this input?

14.11.2 Drill 1. The `equals` that is never called

A developer writes `Point` with value equality, then finds that `contains` never matches.

Point.java — JAVA

```java
static class Point {
    final int x, y;
    Point(int x, int y) { this.x = x; this.y = y; }
    public boolean equals(Point other) {               // BUG: overloads, does not override
        return other != null && x == other.x && y == other.y;
    }
    @Override public int hashCode() { return Objects.hash(x, y); }
}
```

Driver (inside main) — JAVA

```java
List<Point> list = List.of(new Point(1, 2));
System.out.println("D1: contains = " + list.contains(new Point(1, 2)));
Object o = new Point(1, 2);
System.out.println("D1: equals(Object) = " + o.equals(new Point(1, 2)));
System.out.println("D1f: fixed = " + List.of(new GoodPoint(1, 2)).contains(new GoodPoint(1, 2)));
```

Output — TERMINAL

```
$ java Ch10.java | grep -E "^D1f?:"
D1: contains = false
D1: equals(Object) = false
D1f: fixed = true
```

Diagnosis. `equals(Point)` has a different parameter type from `Object.equals(Object)`, so it is an *overload*, not an override. `List.contains` and every collection call `equals(Object)`, which is still the identity version inherited from `Object`. **Fix:** write `equals(Object)`, and put `@Override` on it so that the compiler refuses a wrong signature.

GoodPoint.java — JAVA

```java
static class GoodPoint {
    final int x, y;
    GoodPoint(int x, int y) { this.x = x; this.y = y; }
    @Override public boolean equals(Object o) {        // @Override makes the compiler check it
        return o instanceof GoodPoint p && x == p.x && y == p.y;
    }
    @Override public int hashCode() { return Objects.hash(x, y); }
}
```

14.11.3 Drill 2. Removing inside a for-each loop

The goal is to remove the even numbers from a list.

Loop (inside main) JAVA

```java
List<Integer> nums = new ArrayList<>(List.of(1, 2, 3, 4));
try {
    for (Integer n : nums) {
        if (n % 2 == 0) nums.remove(n);            // BUG: structural change during iteration
    }
} catch (ConcurrentModificationException e) {
    System.out.println("D2: " + e.getClass().getSimpleName());
}
```

Output TERMINAL

```
$ java Ch10.java | grep -E "^D2f?:"
D2: ConcurrentModificationException
D2f: fixed = [1, 3]
```

Diagnosis. The enhanced `for` loop uses an `Iterator`. `ArrayList.remove` changes the list's modification count behind the iterator's back, and the next call to `next()` notices and throws `ConcurrentModificationException` (CME). The exception is a fail-fast diagnostic, and it has nothing to do with threads here. There is a famous trap: if you remove the *second-to-last* element, `hasNext()` returns false and no exception is thrown, so the bug hides until the data changes. **Fix:** `removeIf`, or use the iterator's own `remove()`, or collect and `removeAll`.

Fix (inside main) JAVA

```java
List<Integer> fixed = new ArrayList<>(List.of(1, 2, 3, 4));
fixed.removeIf(n -> n % 2 == 0);
System.out.println("D2f: fixed = " + fixed);
```

Note also that `nums.remove(n)` with an `Integer` argument calls `remove(Object)`; with an `int` it would remove by *index*, a separate classic mistake.

14.11.4 Drill 3. Comparing wrapper objects with `==`

Two lists hold equal numbers, yet the comparison disagrees for one of them.

Boxes (inside main) JAVA

```java
List<Integer> a = List.of(1000, 5);
List<Integer> b = List.of(1000, 5);
System.out.println("D3: " + (a.get(0) == b.get(0)) + " " + (a.get(1) == b.get(1)));
System.out.println("D3f: fixed = " + a.get(0).equals(b.get(0)));
```

Output TERMINAL

```
$ java Ch10.java | grep -E "^D3f?:"
D3: false true
D3f: fixed = true
```

Diagnosis. `get` returns `Integer` objects, and `==` on references compares identity. `Integer.valueOf` caches the values -128 to 127, so the two `5`s are the same object while the two `1000`s are different objects. The code "works" in tests with small numbers and fails in production. **Fix:** `equals`, or unbox to `int` before comparing (`a.get(0).intValue() == b.get(0)`).

14.11.5 Drill 4. Strings are immutable

A helper should trim and upper-case its input.

Shout.java JAVA

```java
static String shout(String s) {
    s.trim();
    s.toUpperCase();                                  // BUG: result thrown away
    return s;
}
```

Output of shout(" hello ") TERMINAL

```
$ java Ch10.java | grep -E "^D4f?:"
D4: [  hello ]
```

Diagnosis. `trim` and `toUpperCase` return *new* strings; the original is untouched, and the new strings were discarded. Static analysis tools flag this as "return value ignored". **Fix:** use the result: `return s.trim().toUpperCase();`. The same applies to `BigDecimal`, `LocalDate`, `Optional` and every other immutable type.

14.11.6 Drill 5. Overflow before the widening

Compute the number of microseconds in a day.

Micros (inside main) JAVA

```
long micros = 24 * 60 * 60 * 1000 * 1000;          // BUG: int arithmetic overflows first
long micros2 = 24L * 60 * 60 * 1000 * 1000;        // fix: one long operand makes it long
System.out.println("D5: " + micros + " vs " + micros2);
```

Output TERMINAL

```
$ java Ch10.java | grep -E "^D5f?:"
D5: 500654080 vs 86400000000
```

Diagnosis. All operands are `int` literals, so the multiplication is done in 32-bit `int` arithmetic and wraps around; only afterwards is the wrong result widened to `long`. **Fix:** make at least one operand `long` *before* the first multiplication (`24L * 60 * ...`), or use `TimeUnit.DAYS.toMicros(1)`. Widening cannot repair a value that has already overflowed.

14.11.7 Drill 6. A list you cannot grow

A method receives names from `Arrays.asList` and adds one.

Names (inside main) JAVA

```
List<String> names = Arrays.asList("ann", "bob");
names.set(0, "amy");                                // allowed: set is not a size change
try {
    names.add("cy");                                // BUG: fixed-size list backed by an array
} catch (UnsupportedOperationException e) {
    System.out.println("D6: " + e.getClass().getSimpleName() + " " + names);
}
List<String> growable = new ArrayList<>(names);    // fix: copy into a real ArrayList
growable.add("cy");
System.out.println("D6f: fixed = " + growable);
```

Output TERMINAL

```
$ java Ch10.java | grep -E "^D6f?:"
D6: UnsupportedOperationException [amy, bob]
D6f: fixed = [amy, bob, cy]
```

Diagnosis. `Arrays.asList` returns a fixed-size list that is a *view* of the array: `set` works (and writes through to the array), but `add` and `remove` throw `UnsupportedOperationException`. `List.of` and `Stream.toList()` are fully immutable; `Collections.unmodifiableList` is a read-only view. **Fix:** copy into `new ArrayList<>(...)` when you need to grow it.

14.11.8 Drill 7. A thread-safe map used in a non-thread-safe way

A developer picks `ConcurrentHashMap` "so it is safe", then counts hits from four threads.

Hits (inside main) JAVA

```java
Map<String, Integer> hits = new ConcurrentHashMap<>();
ExecutorService pool = Executors.newFixedThreadPool(4);
for (int t = 0; t < 4; t++) {
    pool.submit(() -> {
        for (int i = 0; i < 50_000; i++) {
            hits.put("k", hits.getOrDefault("k", 0) + 1);   // BUG: read, then write
        }
    });
}
pool.shutdown(); pool.awaitTermination(30, TimeUnit.SECONDS);
System.out.println("D7: lost updates = " + (hits.get("k") != 200_000));
```

Output (typical: the loss is a race, but it shows with this load) TERMINAL

```
$ java Ch10.java | grep -E "^D7f?:"
D7: lost updates = true
D7f: fixed = 200000
```

Diagnosis. Each call (`getOrDefault`, `put`) is individually thread-safe, but the *sequence* "read, add one, write" is not atomic. Two threads read 7 and both write 8. Thread-safe components do not make compound operations safe. **Fix:** an atomic method of the map itself.

Fix (inside main) JAVA

```java
Map<String, Integer> safe = new ConcurrentHashMap<>();
ExecutorService pool2 = Executors.newFixedThreadPool(4);
for (int t = 0; t < 4; t++) {
    pool2.submit(() -> {
        for (int i = 0; i < 50_000; i++) safe.merge("k", 1, Integer::sum);   // atomic
    });
}
pool2.shutdown(); pool2.awaitTermination(30, TimeUnit.SECONDS);
System.out.println("D7f: fixed = " + safe.get("k"));
```

The second output line (`D7f`) is the fixed run: exactly 200000.

14.11.9 Drill 8. The key that was changed after insertion

A mutable object is used as a `HashMap` key, then edited.

MutableKey.java JAVA

```java
static class Key {
    String name;
    Key(String name) { this.name = name; }
    @Override public boolean equals(Object o) { return o instanceof Key k && name.equals(k.name); }
    @Override public int hashCode() { return name.hashCode(); }
}
```

Driver (inside main) JAVA

```java
Key key = new Key("alpha");
Map<Key, String> map = new HashMap<>();
map.put(key, "value");
key.name = "beta";                                  // BUG: key changed after insertion
System.out.println("D8: get(key) = " + map.get(key) + ", size = " + map.size());
System.out.println("D8: get(old) = " + map.get(new Key("alpha")));
```

Output TERMINAL

```
$ java Ch10.java | grep -E "^D8f?:"
D8: get(key) = null, size = 1
D8: get(old) = null
```

Diagnosis. `HashMap` stores each entry in the bucket chosen by the key's hash *at insertion time*. After `name` changes, `hashCode()` returns a different value, so a lookup with the modified key searches the wrong bucket (`null`), and a lookup with the original value finds the right bucket but `equals` now fails. The entry is still counted in `size()` but is unreachable: a memory leak. **Fix:** use immutable keys (`String`, `Integer`, records with immutable components), or remove the entry before editing and re-insert it afterwards.

14.11.10 Drill 9. Money in `double`

A loop adds ten dimes and compares with one euro.

Money (inside main) JAVA

```java
double total = 0;
for (int i = 0; i < 10; i++) total += 0.10;        // BUG: 0.10 has no exact binary form
System.out.println("D9: " + total + " " + (total == 1.0));
BigDecimal exact = BigDecimal.ZERO;
for (int i = 0; i < 10; i++) exact = exact.add(new BigDecimal("0.10"));
System.out.println("D9f: fixed = " + exact + " " + (exact.compareTo(BigDecimal.ONE) == 0));
```

Output TERMINAL

```
$ java Ch10.java | grep -E "^D9f?:"
D9: 0.9999999999999999 false
D9f: fixed = 1.00 true
```

Diagnosis. `0.10` is not exactly representable in binary floating point, so the errors accumulate and ten additions fall one unit in the last place short of 1.0. **Fix:** for money use `BigDecimal` built from *strings* (`new BigDecimal("0.10")`; `new BigDecimal(0.10)` imports the binary error) or integer cents in a `long`. Compare `BigDecimal` with `compareTo`, not `equals`, because `equals` also compares scale (`1.0` is not equal to `1.00`).

14.11.11 Drill 10. A constructor that calls an overridable method

The base class announces itself in its constructor by calling `describe()`.

Inheritance.java JAVA

```java
static class Base {
    Base() { describe(); }                          // BUG: calls an overridable method
    void describe() { System.out.println("D10: base"); }
}
static class Derived extends Base {
    private final String label = "derived";
    @Override void describe() { System.out.println("D10: final label = " + label.length()); }
}

static class Derived2 extends Base {
    private String label = "derived";
    @Override void describe() { System.out.println("D10: label = " + label); }
}
```

Driver (inside main) JAVA

```java
new Derived();
new Derived2();
```

Output TERMINAL

```
$ java Ch10.java | grep -E "^D10f?:"
D10: final label = 7
D10: label = null
```

Diagnosis. Construction runs the superclass constructor *first*. It calls `describe()`, which dispatches dynamically to the subclass override while the subclass's own fields have not been initialised yet: the non-final `label` is still `null`. (The first output line, `final label = 7`, is a second surprise: a `final` field initialised with a compile-time constant is inlined by the compiler, so it appears initialised even this early. Remove `final`, or make the initialiser a non-constant expression, and it prints `null` as well.) **Fix:** never call overridable methods from a constructor; make the method `private` or `final`, or move the call into an explicit `init()` that the caller invokes after construction.

INTERVIEW ANGLE

How do you debug a bug you cannot immediately see? Reproduce it with the smallest input; read the exact exception type and the first line of your own code in the stack trace; form a hypothesis and test it with a print, a debugger breakpoint or a unit test; then check the usual suspects: identity versus equality, integer overflow, mutated shared state, ignored return values, iteration order assumptions. Fix the cause, add a test that fails before the fix, and explain the root cause in the commit message.

Check yourself

1. What does this print? `Long a = 127L, b = 127L, c = 128L, d = 128L; System.out.println((a == b) + " " + (c == d));`
2. A teammate adds `@Override` to `public boolean equals(Point p)` and the code stops compiling. Is that good or bad news? Why?
3. What does `new ArrayList<>(List.of(1, 2, 3)).remove(1)` remove: the value 1 or the element at index 1? What is the result list?
4. After `map.put(key, v)` you call `key.setId(99)` where `hashCode` uses `id`. Why can `map.get(key)` now return `null`?
5. What is wrong with `int seconds = 3_000_000; long ms = seconds * 1000;`, and what is the value of `ms`?
6. Why should `BigDecimal` values be compared with `compareTo` rather than `equals`?

Answers

1. `true false`. `Long.valueOf` also caches -128 to 127, so `127L` boxes to the same object and `128L` to two different objects.
2. Good news: the compiler has found that the method does not override `Object.equals(Object)`, an overload bug that would silently use identity equality.
3. It removes the element at index 1, because `remove(int)` is chosen for an `int` argument. The result is `[1, 3]`.
4. The entry sits in the bucket for the old hash code; the changed key hashes to a different bucket, so the lookup never reaches it.
5. `seconds * 1000` is evaluated as `int` and overflows before being assigned to `long`; 3,000,000,000 wraps to `-1294967296`. Use `seconds * 1000L`.
6. `equals` also compares scale, so `new BigDecimal("1.0").equals(new BigDecimal("1.00"))` is false, while `compareTo` returns 0 for numerically equal values.

KEY POINTS

Debugging drills reduce to a short list of recurring root causes:

- Use `@Override`; an overload instead of an override is silent and breaks collections.
- Wrapper `==` compares identity; strings and other immutables return new values that you must use.
- Compound operations on thread-safe parts are not atomic; use the atomic API (`merge`, `compute`, atomics).
- Keys must not change while in a hash map; money is not `double`; widening cannot undo overflow.
- Constructors must not call overridable methods, and fail-fast iterators complain when you modify during iteration.

PART 15

Study Plan, Cheat Sheets and Quizzes

Turn the book into a schedule, a reference you can scan in minutes, and a way to find out what you really know.

In this part

15.1 The 25-Day, 4-Week Study Plan
15.2 Cheat Sheets
15.3 Part-by-Part Self-Test Quizzes
15.4 Final Mock Interview and What to Do Next

15.1 The 25-Day, 4-Week Study Plan

This book has fourteen Parts and several hundred pages. Read front to back without a plan, most people stall around the collections chapters and never reach concurrency or the JVM, which are exactly the topics interviewers like. This chapter turns the book into a schedule: **25 study days spread over four weeks**, with a fixed daily loop, a checkpoint at the end of each week, and a list of what to skip if you are short of time.

The plan is deliberately **JVM first**. Java feels like a language, but it is really a language plus a runtime. Once you can picture the heap, the stack, class loading and the JIT, later topics (string pool, `HashMap` memory, thread visibility, garbage collection) stop being facts to memorise and become consequences you can derive.

15.1.1 The daily loop

Every study day has the same three blocks. Keep the order; the third block is the one people skip and the one that makes the knowledge stick.

Block	Time	What you do
Read	40 min	Read the day's chapters once at normal speed. Do not type yet. Mark every sentence you could not explain to a friend.
Code	40 min	Re-type the listings (do not paste). Change one thing in each and predict the result before you run it. Use `jshell` for one-liners.
Test	20 to 30 min	Close the book. Answer the chapter's Check yourself questions from memory, then the matching quiz from the last Part of this book. Write wrong answers in your error log.

The daily loop (about 100 to 110 minutes).

The error log

Keep a plain text file called `errors.txt`. For every question you get wrong, write one line: the topic, what you answered, the correct idea in your own words. At the end of each week re-read only this file. It is a few pages long and it is a map of your personal weak spots, which is far more valuable than another pass through material you already know.

Rules that keep the plan honest

- Typing beats reading. A listing you have re-typed and changed is remembered; a listing you have scrolled past is not.
- Predict, then run. Before you execute any program, write down what you think it prints. A wrong prediction is the most useful event in the whole session.
- Do not move on with an unexplained term. If "happens-before" or "type erasure" is still fuzzy, spend ten minutes with the chapter's examples before the next day starts.
- One missed day is fine. Two in a row means you drop the lowest-priority item of the next day (see the skip list below), not that you cram.

15.1.2 Week 1: the JVM and the language (days 1 to 7)

Goal: you can explain what happens between `javac Hello.java` and the output appearing, and you can write correct code using primitives, control flow, arrays and methods.

Day	Read	Code focus
1	Part 1: What Java is; From source to running program	Install a JDK, run `java -version`, compile and run Hello with `javac` and with `java Hello.java`
2	Part 1: JVM architecture	Use `javap -c -p` on a small class; draw heap, stack and metaspace by hand
3	Part 1: Your first programs and the toolchain; How to study this book	Write a small program that reads arguments and `Scanner` input; read a real stack trace
4	Part 2: Variables, primitive types and literals; Operators and expressions	Overflow, integer division, `==` on doubles, compound assignment
5	Part 2: Type conversion, casting and promotion; Control flow	`switch` expression with `yield`, labelled `break`
6	Part 2: Arrays; Methods	Overload resolution experiments, recursion until `StackOverflowError`
7	Part 2: Text blocks and basic I/O; then the Week 1 checkpoint	Quizzes for Parts 1 and 2

Week 1 schedule. Chapter names refer to titles in this book.

15.1.3 Week 2: object-oriented Java, strings and exceptions (days 8 to 13)

Goal: you can design a small class hierarchy, explain dynamic dispatch and the `equals`/`hashCode` contract, and you know why strings are immutable and how exceptions flow.

Day	Read	Code focus
8	Part 3: Classes and objects; Static, access control and packages	Print initialisation order; write an immutable class with a defensive copy
9	Part 3: Inheritance; Polymorphism and dynamic dispatch; Abstract classes and interfaces	`javap` an `invokevirtual`; overloading versus overriding experiments
10	Part 3: The Object class; Nested, inner, local and anonymous classes; Enums	Break `hashCode` on purpose and watch a `HashMap` lose an entry
11	Part 3: Records; Sealed classes and interfaces; Pattern matching; Wrapper classes; Designing classes well	A sealed hierarchy with an exhaustive `switch`
12	Part 4: Strings in depth; Formatting, regex and text; Unicode, charsets and Locale	Pool experiments with `==` and `intern`; one regex with groups
13	Part 4: Numbers beyond primitives; Date and time with java.time. Part 5: Exceptions. Week 2 checkpoint	`BigDecimal` money maths; `return` in `finally`; quizzes for Parts 3 to 5

Week 2 schedule.

Day 13 is heavy. If it overruns, move the java.time chapter to the start of day 14.

15.1.4 Week 3: collections, generics and streams (days 14 to 19)

Goal: you can pick the right collection and justify it with Big-O, explain how `HashMap` works inside, and write readable stream pipelines.

Day	Read	Code focus
14	Part 5: Logging and debugging basics; Classic I/O; NIO.2; Serialization and its dangers; Properties, system and runtime	Read a file with `Files.lines`; list a directory tree
15	Part 6: Generics; The Collections Framework overview	Trigger `ConcurrentModificationException`, then fix it
16	Part 6: Lists in depth; Sorting and comparing; Immutable, unmodifiable and sequenced collections	`Arrays.asList` and `List.of` traps; a multi-key `Comparator`
17	Part 6: Sets; HashMap in depth; Other maps	LRU cache with `LinkedHashMap`; bad `hashCode` experiment

Day	Read	Code focus
18	Part 6: Queues and deques; Choosing a data structure and Big-O; Implement your own	Write your own dynamic array and hash map
19	Part 7: Lambdas and functional interfaces; Optional; Streams fundamentals; Collectors. Week 3 checkpoint	`groupingBy` with a downstream collector; quizzes for Parts 6 and 7

Week 3 schedule.

15.1.5 Week 4: concurrency, memory, projects and interviews (days 20 to 25)

Goal: you can write and reason about thread-safe code, explain garbage collection and the common `OutOfMemoryError` messages, and talk through a project you built.

Day	Read	Code focus
20	Part 8: Concurrency concepts; Threads; Races and the Java Memory Model; synchronized and monitors	Reproduce a lost update with `count++`; fix it three ways
21	Part 8: Locks and atomics; Executors and thread pools; CompletableFuture; Synchronizers and concurrent collections; Virtual threads	A `ThreadPoolExecutor` with a bounded queue; virtual-thread executor
22	Part 9: Objects in memory; Heap structure; Garbage collection fundamentals; The collectors; Reading GC logs and tuning; Memory leaks and OutOfMemoryError	Run a program with `-Xlog:gc*`; provoke `OutOfMemoryError` with a small `-Xmx`
23	Part 10: Diagnostics toolbox; Troubleshooting playbook. Part 11: From Java 8 to 25; Testing. Part 12: Effective Java in practice	Take a thread dump with `jstack`; write a hand-rolled test runner
24	Part 13: one project of your choice, built and extended. Part 14: skim the Q&A for your weak topics	Build the project from an empty directory with only `javac`
25	Part 15: Final mock interview, timed (90 minutes). Review the error log.	Fix every wrong answer by writing a tiny program that proves the right one

Week 4 schedule.

15.1.6 Weekly checkpoints

At the end of each week, spend 30 minutes on these tests before you continue. If you fail more than a quarter of a checkpoint, repeat that week's weakest day instead of starting the next week.

Week	You pass the checkpoint when you can
1	Draw the runtime data areas and say where a local variable, an object and a `static` field live; predict the output of integer overflow and `char` arithmetic snippets; score at least 9 of 12 on the quizzes for Parts 1 and 2.
2	Explain overriding versus overloading with an example; state the `equals`/`hashCode` contract and show a bug caused by breaking it; explain why `"a" + "b" == "ab"` can be true; score at least 9 of 12 on Parts 3 to 5.
3	Describe `HashMap` from key to bucket to resize; choose between `ArrayList`, `LinkedList`, `ArrayDeque`, `TreeMap`; write a `groupingBy` pipeline without looking; score at least 9 of 12 on Parts 6 and 7.
4	Explain happens-before and give one example each for `volatile`, `synchronized` and an atomic; name the heap regions and three collectors; read a thread dump; score at least 40 of 60 on the final mock interview.

Checkpoints.

15.1.7 If you are short of time

Not everyone has 25 free days. Here is what to cut, in the order to cut it. Never cut the Check yourself blocks; cut reading breadth first.

Time you have	Keep	Skip or skim
15 days	Everything in the plan, one chapter per block	Skip Part 5 serialization and properties, the unicode chapter, Part 10 reflection and modules, Part 11 build tools

Time you have	Keep	Skip or skim
10 days	Parts 1 to 3, collections and `HashMap`, streams basics, threads and `synchronized`, JVM memory and GC	All of Parts 4 (except strings), 5 (except exceptions), 10, 11, 12; read only the Q&A of Part 14
5 days	Part 1 JVM architecture, Part 3 OOP summary boxes, `HashMap`, `synchronized`/`volatile`, GC basics, then Part 14 and the mock	Everything else; use the cheat sheets in the next chapter as your notes

Skip list by available time.

Whatever you skip, do not skip Part 13. A project that you built, ran and can explain is worth more in an interview than any number of memorised answers.

15.1.8 Tracking progress

Pick one method and use it every day; the method matters less than the habit.

- **A grid.** Draw 25 boxes on paper and tick one each day you finish all three blocks. A streak you can see is motivating.
- **A score sheet.** After each quiz, write the date and score. Re-take any quiz scoring below 9 of 12 three days later without looking at the answers.
- **A GitHub repository.** Create one repository with a folder per Part and commit every listing you re-typed, with your own changes. In an interview it shows real practice, and it gives you something concrete to talk about.

INTERVIEW ANGLE

"How did you learn Java?" A strong answer is specific: "I studied the JVM first, then rebuilt each concept in code. For example, I broke `hashCode` on purpose to see a `HashMap` degrade, reproduced a lost update with `count++`, and read GC logs from a program I made allocate heavily." Concrete experiments are far more convincing than a list of books or courses.

TIP

Study in the same place at the same time each day. A fixed slot removes the daily decision "should I study today", which is the real enemy of a 25-day plan.

Check yourself

1. Name the three blocks of the daily loop and say which one is most often skipped.
2. Why does the plan start with the JVM instead of syntax?
3. What goes into the error log, and when do you re-read it?
4. You have 10 days. Name two Parts you keep in full and two you cut.
5. Week 3 checkpoint: you score 6 of 12 on the Part 7 quiz. What do you do?

Answers

1. Read, code, test. The test block (closed book, Check yourself plus quiz) is the one usually skipped.
2. Many later topics (string pool, `HashMap`, memory visibility, GC) are consequences of how the JVM works; knowing the runtime lets you derive behaviour instead of memorising it.
3. One line per wrong answer: topic, your answer, the correct idea in your own words. Re-read it at the end of each week and before the mock interview.
4. Keep Part 1 and Part 3 (also collections, concurrency basics, GC); cut Part 10 and Part 11 (also most of Parts 4, 5 and 12).
5. Below the pass mark, so repeat the weakest day (lambdas, Optional, streams or collectors, whichever the errors point to), then re-take the quiz three days later.

KEY POINTS

Plan: 25 days, four weeks, JVM first.

- Daily loop: read 40 minutes, code 40 minutes, test 20 to 30 minutes. The test block is not optional.
- Keep an error log and re-read it weekly; it is a personal map of weak spots.
- Use the four weekly checkpoints; repeat a weak day rather than rush on.
- If short of time, cut breadth (serialization, modules, build tools) but never the projects or the Check yourself blocks.

15.2 Cheat Sheets

These tables are meant to be scanned, not read. They compress the book into reference pages you can review the night before an interview or keep open while coding. Each sheet names the Part where the details live. A cheat sheet is a memory aid, not a teacher: if a row surprises you, go back to the chapter.

15.2.1 Keywords and modifiers

Java has about 50 reserved keywords (including `const` and `goto`, which are reserved but unused), plus the literals `true`, `false`, `null`. Words such as `var`, `record`, `sealed`, `permits`, `yield` and `when` are *contextual* keywords: special only in some positions, still legal as variable names in most places (`var` and `yield` cannot be type names).

Group	Keywords
`Primitive types`	byte short int long float double char boolean void
`Control flow`	if else switch case default while do for break continue return
`Exceptions`	try catch finally throw throws assert
`Classes and types`	class interface enum extends implements new this super instanceof package import
`Access`	public protected private
`Other modifiers`	static final abstract native synchronized transient volatile strictfp default
`Reserved, unused`	const goto
`Contextual`	var record sealed non-sealed permits yield when module requires exports opens

Keywords grouped by purpose.

Modifier	Applies to	Meaning
`public`	class, member	Visible everywhere
`protected`	member	Same package plus subclasses
`(none)`	class, member	Package-private: same package only
`private`	member	Same top-level class only
`static`	member, nested class	Belongs to the class, not an instance
`final`	class, method, variable	Cannot extend / override / reassign
`abstract`	class, method	No implementation here; cannot instantiate an abstract class
`sealed`	class, interface	Only the listed `permits` subtypes may extend it
`synchronized`	method, block	Acquire the object's (or class's) monitor
`volatile`	field	Visibility and ordering across threads; not atomic `x++`
`transient`	field	Skipped by default Java serialization
`native`	method	Implemented in native code (JNI)
`default`	interface method	Method with a body in an interface

Modifiers: where they apply and what they mean.

15.2.2 Primitive types and ranges

The ranges are exact; the "about" values are for quick mental use. All integral types are signed except `char`.

Type	Bits	Default	Range	Wrapper
`byte`	8	`0`	-128 to 127	Byte
`short`	16	`0`	-32,768 to 32,767	Short
`char`	16	`'\u0000'`	0 to 65,535 (UTF-16 code unit)	Character
`int`	32	`0`	-2,147,483,648 to 2,147,483,647 (about 2.1e9)	Integer
`long`	64	`0L`	-9.22e18 to 9.22e18 (`Long.MAX_VALUE` = 9223372036854775807)	Long
`float`	32	`0.0f`	about 3.4e38, 6 to 7 decimal digits	Float
`double`	64	`0.0`	about 1.8e308, 15 to 16 decimal digits	Double
`boolean`	JVM-defined	`false`	true or false	Boolean

Primitives (Part 2).

Remember: `Integer.MAX_VALUE + 1` wraps to `Integer.MIN_VALUE`; `Math.abs(Integer.MIN_VALUE)` is still negative; `Math.addExact` throws instead of wrapping; `Integer` objects between -128 and 127 are cached, so `==` works there and nowhere else dependably; `0.1 + 0.2 != 0.3`, use `BigDecimal` for money.

15.2.3 Operator precedence

From highest (binds first) to lowest. Operators on one row have equal precedence. When in doubt, add parentheses; readers will thank you.

Level	Operators	Associativity
1	`a[i] a.b a(args) a++ a--`	left to right
2	`++a --a +a -a ! ~ (cast) new`	right to left
3	`* / %`	left to right
4	`+ - (also string concatenation)`	left to right
5	`<< >> >>>`	left to right
6	`< <= > >= instanceof`	left to right
7	`== !=`	left to right
8	`&`	left to right
9	`^`	left to right
10	`bitwise OR (one vertical bar)`	left to right
11	`&&`	left to right
12	`logical OR (two vertical bars)`	left to right
13	`? :`	right to left
14	`= += -= *= /= %= &= ^= <<= >>= >>>= (and the OR-assign form) and ->`	right to left

Operator precedence (Part 2).

Traps: `&&` binds tighter than `||`; `1 + 2 + "x"` is `"3x"` but `"x" + 1 + 2` is `"x12"`; compound assignment hides a cast (`byte b = 10; b += 5;` compiles); `%` keeps the sign of the dividend (`-7 % 3` is `-1`).

15.2.4 Collections: Big-O and when to use

n is the number of elements. "Amortised" means averaged over many operations. Hash costs assume a decent `hashCode`; a bad one degrades toward O(n) (O(log n) after treeification).

Structure	get / contains	add	remove	iteration order	Use it when
`ArrayList`	O(1) by index; O(n) contains	O(1) amortised at end	O(n)	insertion	Default list; random access, mostly appending
`LinkedList`	O(n)	O(1) at ends	O(1) at known node	insertion	Rarely; `ArrayDeque` is usually better
`ArrayDeque`	O(n)	O(1) at both ends	O(1) at ends	insertion	Stack or queue; no nulls
`PriorityQueue`	O(n) contains; peek O(1)	O(log n)	O(log n) poll	heap order, not sorted	Top-k, schedulers, Dijkstra
`HashSet / HashMap`	O(1) average	O(1) average	O(1) average	none guaranteed	Default set / map
`LinkedHashSet / Map`	O(1)	O(1)	O(1)	insertion (or access)	Predictable order, LRU cache
`TreeSet / TreeMap`	O(log n)	O(log n)	O(log n)	sorted	Sorted keys, range queries, floor/ceiling
`EnumSet / EnumMap`	O(1)	O(1)	O(1)	enum order	Keys are enum constants
`ConcurrentHashMap`	O(1) average	O(1) average	O(1) average	none guaranteed	Shared map across threads
`CopyOnWriteArrayList`	O(1) by index	O(n) copy	O(n) copy	insertion	Many reads, very rare writes (listeners)

Time complexity of common operations (Part 6).

Decision shortcuts: need key to value lookup, use `HashMap`; need it sorted, `TreeMap`; need insertion order, `LinkedHashMap`; need a queue or stack, `ArrayDeque`; need unique elements, `HashSet`; need immutable, `List.of`, `Set.of`, `Map.of` (no nulls, unmodifiable). `HashMap` defaults: capacity 16, load factor 0.75, a bucket becomes a tree at 8 entries when the table has at least 64 slots, and goes back to a list at 6.

15.2.5 Stream operations

A stream pipeline is source, zero or more intermediate operations (lazy), one terminal operation (triggers the work). A stream can be consumed once (Part 7).

Operation	What it does	Notes
`filter(p)`	Keep elements matching the predicate	Stateless
`map(f)`	Transform each element	`mapToInt` etc. avoid boxing
`flatMap(f)`	Map each element to a stream and flatten	One-to-many
`distinct()`	Remove duplicates using `equals`	Stateful
`sorted() / sorted(c)`	Sort	Stateful, buffers everything
`limit(n) / skip(n)`	First n / drop first n	Short-circuiting (`limit`)
`peek(c)`	Run a side action	For debugging only
`takeWhile(p) / dropWhile(p)`	Prefix by predicate (Java 9)	Order matters
`mapMulti(f)`	Emit zero or more per element (Java 16)	Alternative to `flatMap`

Intermediate operations (return a Stream).

Operation	Result	Notes
`forEach(c)`	void	Order not guaranteed on parallel streams; `forEachOrdered`
`collect(c)`	Any result via a `Collector`	`Collectors.toList()`, `groupingBy`, `joining`
`toList()`	Unmodifiable `List` (Java 16)	Differs from `Collectors.toList()`
`reduce(id, op)`	Single value	`op` must be associative
`count()`	long	Skips work if size is known
`min(c) / max(c)`	Optional	Needs a comparator
`findFirst() / findAny()`	Optional	Short-circuiting
`anyMatch / allMatch / noneMatch`	boolean	Short-circuiting
`toArray()`	Object[] or typed array	`toArray(String[]::new)`
`sum() / average() / summaryStatistics()`	Primitive streams only	`IntStream`, `LongStream`, `DoubleStream`

Terminal operations.

Collector	Produces
`toList() / toSet()`	`List` / `Set` (no guarantee about the exact class)
`toMap(k, v, merge)`	`Map`; without a merge function a duplicate key throws `IllegalStateException`
`groupingBy(f) / groupingBy(f, downstream)`	`Map<K, List<T>>` or a map of downstream results (`counting()`, `summingInt`, `mapping`)
`partitioningBy(p)`	`Map<Boolean, List<T>>` with both keys always present
`joining(sep, prefix, suffix)`	One `String` from a `CharSequence` stream
`counting(), summingInt(f), averagingDouble(f)`	Aggregates, mostly used as downstream collectors
`teeing(c1, c2, merger)`	Run two collectors in one pass and combine (Java 12)

Frequently used Collectors.

15.2.6 The java.util.function interfaces

Learn the pattern, not 43 names: a function consumes (`Consumer`), produces (`Supplier`), tests (`Predicate`), transforms (`Function`), or combines (`BiFunction`). Prefixes `Bi` (two arguments), `Unary` and `Binary` (same type in and out), and `Int`, `Long`, `Double` (primitive specialisations that avoid boxing) modify them.

Interface	Method	Signature	Typical use
`Function<T,R>`	`apply`	`T → R`	`map`
`Predicate<T>`	`test`	`T → boolean`	`filter`, `removeIf`
`Consumer<T>`	`accept`	`T → void`	`forEach`
`Supplier<T>`	`get`	`() → T`	lazy values, `orElseGet`
`UnaryOperator<T>`	`apply`	`T → T`	`replaceAll`
`BinaryOperator<T>`	`apply`	`(T, T) → T`	`reduce`, `merge`
`BiFunction<T,U,R>`	`apply`	`(T, U) → R`	map `compute`
`BiConsumer<T,U>`	`accept`	`(T, U) → void`	`Map.forEach`
`IntFunction, ToIntFunction`	`apply, applyAsInt`	`int → R, T → int`	primitive streams
`Runnable / Callable<V>`	`run / call`	`() → void / () → V (may throw)`	threads, executors

Interface	Method	Signature	Typical use
`Comparator<T>`	`compare`	`(T, T) → int`	sorting

Core functional interfaces (Part 7).

Method references come in four kinds: static (`Integer::parseInt`), bound instance (`System.out::println`), unbound instance (`String::length`), constructor (`ArrayList::new`). Captured local variables must be effectively final.

15.2.7 Concurrency toolbox chooser

Start from the problem, not the class. Prefer the highest-level tool that solves it (Part 8).

Problem	Reach for	Avoid
One shared counter	`AtomicInteger` / `LongAdder` (heavy contention)	`volatile int` with `x++`
A flag seen by other threads	`volatile boolean`	plain field
Guard a compound check-then-act	`synchronized` or `ReentrantLock`	separate atomics for related state
Need timeout / `tryLock` / fairness / many conditions	`ReentrantLock` and `Condition`	`synchronized` when you need those
Many readers, rare writers	`ReadWriteLock`, `StampedLock`, `CopyOnWriteArrayList`	one big lock for everything
Shared map	`ConcurrentHashMap` (`merge`, `computeIfAbsent`)	`Collections.synchronizedMap` with check-then-put
Hand work between threads	`BlockingQueue` (`ArrayBlockingQueue`, `LinkedBlockingQueue`)	`wait`/`notify` by hand
Run tasks	`ExecutorService` with a bounded queue	`new Thread` per task, unbounded `newCachedThreadPool` under load
Compose async results	`CompletableFuture` with your own executor	blocking `get()` inside the common pool
Wait for N events / start together	`CountDownLatch` / `CyclicBarrier`	busy-wait loops with `sleep`
Limit concurrent access	`Semaphore`	ad-hoc counters
Divide and conquer CPU work	Fork/Join, parallel streams (pure functions only)	parallel streams doing I/O
Many blocking I/O tasks	virtual threads (`newVirtualThreadPerTaskExecutor`)	pooling virtual threads; long `synchronized` blocks on Java 21
Per-thread state	`ThreadLocal` (careful with pools), `ScopedValue` (Java 25)	static mutable state

Which tool for which problem.

Golden rules: prefer immutability; confine mutable state to one thread when possible; always acquire locks in a fixed global order; never call unknown code while holding a lock; always shut executors down; treat `InterruptedException` as a request to stop, and restore the flag with `Thread.currentThread().interrupt()` if you cannot rethrow.

15.2.8 JVM flags

Flags starting with `-X` are standard-ish and stable; `-XX:` are HotSpot options (`+` turns a boolean on, `-` off, `=` sets a value). Defaults below were checked with `-XX:+PrintFlagsFinal` on JDK 21 on the build machine and depend on the machine.

Flag	Effect	Note
`-Xms / -Xmx`	Initial / maximum heap size	`-Xmx512m`; setting both equal avoids resizing
`-Xss`	Thread stack size	default 1 MB on 64-bit Linux (`ThreadStackSize=1024`)
`-XX:MaxRAMPercentage`	Max heap as a percent of container or machine memory	default 25.0
`-XX:MaxMetaspaceSize`	Cap on class metadata	unlimited by default

Flag	Effect	Note
`-XX:+UseG1GC / UseParallelGC / UseSerialGC / UseZGC`	Choose a collector	G1 is the default on server-class machines since 9
`-XX:MaxGCPauseMillis=n`	G1 pause goal	default 200, a goal not a guarantee
`-Xlog:gc*`	Unified GC logging	`-Xlog:gc*:file=gc.log`
`-XX:+HeapDumpOnOutOfMemoryError`	Write a heap dump on OOM	add `-XX:HeapDumpPath=...`
`-XX:+ExitOnOutOfMemoryError`	Exit on OOM so a supervisor restarts the process	default off
`-XX:+PrintFlagsFinal`	Print every flag with its final value	`java -XX:+PrintFlagsFinal -version`
`-XX:+PrintCompilation`	Log JIT compilation events	diagnostics
`-XX:StartFlightRecording`	Start JFR at launch	`-XX:StartFlightRecording=duration=60s,filename=app.jfr`
`-XX:TieredStopAtLevel=1`	Use only C1	faster startup, lower peak speed
`-XX:+UseCompactObjectHeaders`	Smaller object headers	Java 25 opt-in; default in Java 27
`-Dname=value`	Set a system property	`System.getProperty("name")`
`-ea`	Enable `assert`	off by default
`--enable-preview`	Allow preview features	needs matching `--release` / `--source`

Flags worth knowing (Parts 9 and 10).

15.2.9 jcmd commands

`jcmd` talks to a running JVM by process id. Run `jps -l` or `jcmd` with no arguments to find the pid, and `jcmd <pid> help` to list what that JVM supports. All commands below exist on JDK 21.

Command	What it gives you
`jcmd <pid> Thread.print`	Thread dump with locks and deadlock report (same as `jstack`)
`jcmd <pid> GC.heap_info`	Heap regions and usage right now
`jcmd <pid> GC.class_histogram`	Instance count and bytes per class (leak hunting)
`jcmd <pid> GC.heap_dump file.hprof`	Heap dump for Eclipse MAT or VisualVM
`jcmd <pid> GC.run`	Request a full GC (a hint, for experiments)
`jcmd <pid> VM.flags`	Flags the JVM is running with
`jcmd <pid> VM.system_properties`	All system properties
`jcmd <pid> VM.command_line`	The launch command line
`jcmd <pid> VM.uptime`	How long the JVM has run
`jcmd <pid> VM.native_memory`	Native memory report (needs `-XX:NativeMemoryTracking=summary`)
`jcmd <pid> JFR.start / JFR.dump / JFR.stop`	Control Java Flight Recorder
`jcmd <pid> Compiler.codecache`	Code cache usage

Common jcmd commands (Part 10).

15.2.10 Common exceptions

Unchecked exceptions extend `RuntimeException` or `Error`; everything else under `Throwable` is checked and must be caught or declared (Part 5).

Exception	Kind	Typical cause
`NullPointerException`	unchecked	Dereferencing `null`; unboxing a null `Integer`
`ArrayIndexOutOfBoundsException`	unchecked	Index outside `0 .. length-1`
`StringIndexOutOfBoundsException`	unchecked	Bad `substring` or `charAt` index
`ClassCastException`	unchecked	Wrong downcast; mixed raw types
`NumberFormatException`	unchecked	`Integer.parseInt("abc")`
`IllegalArgumentException`	unchecked	Bad argument value passed in
`IllegalStateException`	unchecked	Method called at the wrong time; `toMap` duplicate key
`ArithmeticException`	unchecked	Integer division by zero
`ConcurrentModificationException`	unchecked	Modifying a collection while iterating it
`UnsupportedOperationException`	unchecked	Writing to `List.of(...)` or an `Arrays.asList` size change
`ArrayStoreException`	unchecked	Wrong type stored in a covariant array
`IOException`	checked	Any I/O failure; `FileNotFoundException`, `NoSuchFileException` are subtypes
`InterruptedException`	checked	A blocking call was interrupted
`ClassNotFoundException`	checked	Class named by string not on the classpath
`StackOverflowError`	Error	Runaway recursion
`OutOfMemoryError`	Error	Heap, Metaspace, native thread or direct buffer exhausted
`NoClassDefFoundError`	Error	Class was present at compile time, missing at run time
`ExceptionInInitializerError`	Error	A static initialiser threw

Exceptions you will meet daily.

INTERVIEW ANGLE

"Which exception does each of these throw?" Practise saying, without hesitating: `1 / 0` throws `ArithmeticException`, but `1.0 / 0` is `Infinity` and no exception; `List.of(1).add(2)` throws `UnsupportedOperationException`; `Integer x = null; int y = x;` throws `NullPointerException`; `Object o = "s"; Integer i = (Integer) o;` throws `ClassCastException`.

15.2.11 Command-line toolbox

Command	Purpose
`java -version`	Show the installed runtime version
`java Hello.java`	Compile in memory and run a single source file (Java 11+)
`javac -d out src/com/example/*.java`	Compile into the directory `out`, preserving packages
`java -cp out com.example.Main`	Run a class from the classpath
`javap -c -p Foo`	Disassemble bytecode, including private members
`jshell`	Interactive REPL for trying expressions
`jar cfe app.jar com.example.Main -C out .`	Package classes into an executable jar
`java -jar app.jar`	Run an executable jar
`jps -l`	List running Java processes with main class
`jstack <pid>`	Thread dump (deadlock detection included)
`jmap -histo <pid>`	Object histogram (similar to `GC.class_histogram`)
`jstat -gcutil <pid> 1000`	GC utilisation sampled every second
`jlink --add-modules java.base --output rt`	Build a minimal runtime image
`jfr print app.jfr`	Print events from a recording

Everyday commands (Parts 1, 10 and 11).

15.2.12 Garbage collectors

Collector	Strength	Typical use
`Serial`	Single-threaded, tiny footprint	Small containers, tools, tiny heaps
`Parallel`	Maximum throughput, longer pauses	Batch jobs where pause time does not matter
`G1`	Balanced; region-based; pause goal	Default for most servers
`ZGC (generational by default since 23)`	Very short pauses even on large heaps	Latency-sensitive services
`Shenandoah`	Low pauses, concurrent compaction	Latency-sensitive, where the build includes it
`Epsilon`	Allocates but never collects	Benchmarks and very short-lived programs

Collector choices (Part 9).

15.2.13 Thread states and version milestones

State	Meaning
`NEW`	Created, `start()` not yet called
`RUNNABLE`	Running or ready to run (also while blocked in native I/O)
`BLOCKED`	Waiting to enter a `synchronized` block
`WAITING`	`wait()`, `join()` or `park()` without a timeout
`TIMED_WAITING`	`sleep(ms)` or waits with a timeout
`TERMINATED`	`run()` has finished

Thread.State values (Part 8).

Java	Feature
8	Lambdas, streams, `Optional`, `java.time`, default methods
9	Modules, `jshell`, `List.of` factory methods
10 / 11	`var` / `HttpClient`, running a single source file, new `String` methods

Java	Feature
14 / 15	Switch expressions / text blocks
16 / 17	Records and `instanceof` patterns / sealed classes
21	Virtual threads, record patterns, pattern `switch`, sequenced collections
22	Unnamed variables `_`, Foreign Function and Memory API
24	Stream Gatherers, virtual threads without pinning on `synchronized`
25	Scoped values, module imports, compact source files, flexible constructor bodies (all final)

Language features and the version that finalised them (Part 11).

Check yourself

1. What is `byte b = 120; b += 10;` followed by `System.out.println(b)`? Does it compile?
2. A bucket in a `HashMap` holds 9 entries and the table has 16 slots. Does it become a tree?
3. Which of `map`, `filter`, `sorted`, `count` are terminal operations?
4. You need a thread-safe counter incremented by 50 threads. Name the best tool and one wrong choice.
5. Which `jcmd` command lists how many instances of each class are alive?
6. Is `InterruptedException` checked or unchecked, and what do you do when you catch it and cannot rethrow?

Answers

1. It compiles, because compound assignment casts back to `byte`. 130 does not fit, so it wraps to -126.
2. No. Treeification needs at least 8 entries in a bucket and a table of at least 64 slots; with fewer, `HashMap` resizes the table instead.
3. Only `count`. The other three are lazy intermediate operations.
4. `AtomicInteger` or `LongAdder`. A plain `int` or a `volatile int` with `count++` is wrong, because the increment is read, add, write and loses updates.
5. `jcmd <pid> GC.class_histogram`.
6. Checked. Restore the flag with `Thread.currentThread().interrupt()` so callers can still see the interruption, then stop the work.

KEY POINTS

Reference sheets for the whole book.

- Know the ranges: `int` is about 2.1e9, `long` about 9.2e18, `char` is unsigned 16 bits; overflow wraps silently.
- Pick collections by access pattern: `HashMap` for lookup, `TreeMap` for sorted, `ArrayDeque` for queues, `ArrayList` as the default list.
- Streams are lazy until a terminal operation; match the functional interface to the job (consume, produce, test, transform).
- Choose the highest-level concurrency tool that fits; use atomics, locks, queues and executors before raw `wait`/`notify`.
- For a live JVM, `jcmd <pid> Thread.print`, `GC.heap_info` and `GC.class_histogram` answer most first questions.

15.3 Part-by-Part Self-Test Quizzes

Reading feels like learning; recalling is learning. Each quiz below covers one Part of the book with twelve questions: a mix of multiple choice (A to D) and short answer, with tiny code snippets. Do them closed-book, write your answers down, and only then read the answers section that follows each quiz. Every snippet in these quizzes was compiled and run with JDK 21, and the printed results come from those runs.

How to score: one point per question. Eleven or twelve means move on. Nine or ten means re-read the explanations you missed and add them to your error log. Eight or fewer means go back to that Part's chapter and redo its code block. Questions marked "short" want one or two sentences, not an essay.

15.3.1 Quiz 1: The Java Platform and the JVM

1. (MC) Which of these contains the `javac` compiler? A) JRE B) JDK C) JVM D) JIT
2. (MC) What actually executes the contents of a `.class` file? A) the operating system directly B) the JVM (interpreter plus JIT compiler) C) `javac` D) the class loader
3. (short) Name the three phases of class loading and what the middle phase does.
4. (MC) Which runtime area is private to each thread? A) heap B) Metaspace C) JVM stack D) code cache
5. (short) State the class loader delegation rule in one sentence.
6. (MC) The file `Main.java` contains `class Main { public static void main(String[] args) { System.out.println(args.length); } }`. What does `java Main.java a "b c"` print? A) 0 B) 2 C) 3 D) a compile error
7. (MC) What does `java Hello.java` (Java 11 and later) do? A) Fails: you must run `javac` first B) Compiles in memory and runs the program C) Interprets the source text line by line D) Downloads the dependencies
8. (MC) Which release is not a long-term-support (LTS) release? A) 11 B) 17 C) 22 D) 25
9. (short) What replaced PermGen in Java 8, and where does it live?
10. (MC) Which statement about the JIT compiler is true? A) It compiles every method before the program starts B) It compiles frequently executed methods to machine code while the program runs C) It converts Java source to bytecode D) It is only used by the garbage collector
11. (short) What does `javap -c -p Foo` show that `javap Foo` does not?
12. (short) Where do local variables of primitive type, objects, and `static` fields live?

Answers: Quiz 1

1. B. The JDK ships `javac`, `jar`, `jshell` and the tools; the JRE (runtime only) is no longer shipped separately by most vendors.
2. B. The JVM interprets bytecode and the JIT compiles hot code to machine code.
3. Loading, linking, initialisation. Linking has three steps: verify the bytecode, prepare (allocate and default static fields), resolve symbolic references.
4. C. Each thread has its own JVM stack (and PC register); heap, Metaspace and code cache are shared.
5. A loader first asks its parent to load the class and only tries itself if the parent cannot.
6. B. `args.length` is 2 (`a` and `b c`; the quotes make one argument). Verified by running.
7. B. The source launcher compiles in memory and runs it; no `.class` file is written.
8. C. The LTS releases are 8, 11, 17, 21 and 25.
9. Metaspace, in native (off-heap) memory, holding class metadata.
10. B. It profiles at run time and compiles hot methods (C1 first, then C2).
11. `-c` prints the bytecode instructions, `-p` includes private members.
12. Primitive locals live in the stack frame of the running method, objects on the heap, and `static` fields belong to the class (class-level data, not to any object).

15.3.2 Quiz 2: Language Fundamentals

1. (what prints) `int x = 5; x = x++ + ++x; System.out.println(x);`
2. (what prints) `System.out.println(1 + 2 + "3" + 4 + 5);`
3. (what prints) `byte b = 120; b += 10; System.out.println(b);`
4. (what prints) `char c = 'a'; c += 1; System.out.println(c); System.out.println(c + 1);`
5. (what prints) `System.out.println((int) 3.99 + " " + (byte) 200 + " " + 7 / 2 + " " + -7 % 3);`
6. (what prints) `System.out.println(Integer.MAX_VALUE + 1);`
7. (MC) What does `System.out.println(0.1 + 0.2 == 0.3)` print? A) true B) false C) compile error D) 0.3
8. (short) What do `1.0 / 0` and `1 / 0` do?
9. (what prints) `System.out.println((5 > 3) ? 1 : 2.0);`
10. (short) Which keyword gives a value back from a block in a `switch` expression?
11. (short) What is the default value of each element of `new boolean[2]`, and what does `a[3] = 1` throw when `int[] a = new int[3]`?
12. Given overloads `g(int)`, `g(long)`, `g(Integer)` and `g(int...)`, what do `g((short) 5)`, `Integer boxed = 5; g(boxed)` and `g()` print, in that order?

Answers: Quiz 2

1. 12. `x++` yields 5 (x becomes 6), `++x` makes it 7 and yields 7; 5 + 7 = 12.
2. `3345`. Evaluation is left to right: `1 + 2` is 3, then `"3"` makes the string `"33"`, then `4` and `5` are appended.
3. -126. `b += 10` is `b = (byte)(b + 10)`; 130 wraps to -126.
4. `b` and `99`. `c += 1` keeps type `char`; `c + 1` is an `int` (98 + 1).
5. `3 -56 3 -1`. Casting double to int truncates, 200 wraps to -56 as a byte, integer division truncates, and `%` takes the sign of the left operand.
6. -2147483648. Overflow wraps silently.
7. B. `0.1 + 0.2` is 0.30000000000000004 in binary floating point.
8. `1.0 / 0` is `Infinity` (no exception); `1 / 0` throws `ArithmeticException: / by zero`.
9. `1.0`. The two branches are promoted to a common type, `double`.
10. `yield`.
11. A `boolean` array element defaults to `false`; `a[3] = 1` on a 3-element array throws `ArrayIndexOutOfBoundsException`.
12. `int`, `Integer`, `varargs`. For `(short) 5` widening to `int` wins over boxing and varargs; the phases are widening, then boxing, then varargs. Verified by running.

15.3.3 Quiz 3: Object-Oriented Java

1. (what prints) A class has `static { System.out.print("S "); }`, an instance block `{ System.out.print("I "); }` and a constructor printing `"C "`. What does `new Init(); new Init();` print?
2. (what prints) With overloads `f(Object o)` and `f(String s)`, what does `f(null)` call?
3. (what prints) `class A { String n = "A"; String who() { return "A"; } } class B extends A { String n = "B"; String who() { return "B"; } }` Then `A a = new B(); System.out.println(a.n + " " + a.who());`
4. (what prints) A class `P` overrides `equals` (same `x` means equal) but not `hashCode`. After adding two `new P(1)` to a `HashSet<P>`, what is `size()`?
5. (what prints) `Integer a = 127, b = 127, c = 128, d = 128;` What do `a == b`, `c == d` and `c.equals(d)` print?
6. (what prints) With `record Pt(int x, int y) {}`, what does `System.out.println(new Pt(1, 2).equals(new Pt(1, 2)) + " " + new Pt(1, 2));` print?
7. (what prints) `Object o = "text"; String d = switch (o) { case Integer n -> "int"; case String s when s.length() > 3 -> "long str"; case String s -> "str"; default -> "other"; };` What is `d`?
8. (MC) Which members can an interface contain since Java 9? A) only abstract methods B) default, static and private methods C) instance fields D) constructors
9. (MC) Which statement about enums is true? A) An enum can extend another class B) An enum can implement interfaces C) An enum can have a public constructor D) Enums cannot have methods
10. (short) State the difference between overloading and overriding, including when each is resolved.
11. (short) Why must you override `hashCode` whenever you override `equals`?
12. (short) What does `sealed ... permits` buy you with `switch`?

Answers: Quiz 3

1. `S I C I C` . The static block runs once at class initialisation; the instance block and constructor run for each object.
2. The `String` overload: the most specific applicable method is chosen at compile time.
3. `A B`. Fields are hidden, not overridden, and are resolved by the static type (`A`); methods are resolved dynamically by the runtime type (`B`).
4. 2 (almost certainly). Equal objects have different identity hash codes, so they land in different buckets; this breaks the contract.
5. `true false true`. Values from -128 to 127 are cached by `Integer.valueOf`; 128 creates distinct objects.
6. `true Pt[x=1, y=2]`. Records generate value-based `equals` and `toString`.
7. `long str`. Guards (`when`) refine a pattern; order matters, and the more specific case comes first.
8. B. Interfaces can have default, static and private methods, and constants, but no instance fields or constructors.
9. B. Enums implicitly extend `java.lang.Enum`, so they cannot extend a class; constructors are implicitly private.

10. Overloading: same name, different parameter lists, chosen at compile time from static types. Overriding: a subclass replaces an inherited instance method with the same signature, chosen at run time from the object's actual class.
11. The contract says equal objects must have equal hash codes. Otherwise hash-based collections look in the wrong bucket and treat equal objects as different.
12. The compiler knows every permitted subtype, so a `switch` over the type can be exhaustive without `default`, and adding a new subtype produces compile errors where you forgot to handle it.

15.3.4 Quiz 4: Strings, Text and Time

1. (what prints) `String s1 = "hi", s2 = "hi", s3 = new String("hi");` What do `s1 == s2`, `s1 == s3`, `s1 == s3.intern()` and `s1.equals(s3)` print?
2. (what prints) `System.out.println("a" + "b" == "ab");` and then `String ab = "a"; System.out.println(ab + "b" == "ab");`
3. (what prints) `new BigDecimal("1.10").equals(new BigDecimal("1.1"))` and `new BigDecimal("1.10").compareTo(new BigDecimal("1.1"))`
4. (what prints) `System.out.println(LocalDate.of(2024, 1, 31).plusMonths(1));`
5. (what prints) `"a,b,,".split(",").length`
6. (what prints) `String s = "x"; s.concat("y"); System.out.println(s);`
7. (what prints) `String e = "\uD83D\uDE00"; System.out.println(e.length() + " " + e.codePointCount(0, 2));` (this pair of escapes is one smiley character, outside the Basic Multilingual Plane)
8. (MC) `StringBuffer` differs from `StringBuilder` in that: A) it is synchronized B) it is immutable C) it is faster D) it supports Unicode only
9. (short) Why should you write `new BigDecimal("0.1")` and not `new BigDecimal(0.1)`?
10. (MC) Which java.time type represents a point on the machine timeline, independent of time zone? A) `LocalDateTime` B) `ZonedDateTime` C) `Instant` D) `Period`
11. (what prints) `Duration.ofMinutes(90)` and `Period.ofDays(10)` converted with `toString`.
12. (MC) Which formatter is safe to share between threads? A) `SimpleDateFormat` B) `DateTimeFormatter` C) both D) neither

Answers: Quiz 4

1. `true false true true`. Literals share one pooled object; `new String` makes a separate object; `intern()` returns the pooled one.
2. `true`, then `false`. The first is a compile-time constant folded to the literal `"ab"`; the second concatenates at run time and creates a new object.
3. `false` and `0`. `equals` compares scale too (1.10 versus 1.1); `compareTo` compares numeric value.
4. `2024-02-29`. The day is clamped to the last valid day of the month (2024 is a leap year).
5. 2. Trailing empty strings are removed by `split`, leaving `a` and `b`.
6. `x`. Strings are immutable; the result of `concat` was discarded.
7. `2 1`. `length()` counts UTF-16 code units (a surrogate pair); one code point.
8. A. Both have the same API; `StringBuffer` synchronizes every method and is rarely needed.
9. `new BigDecimal(0.1)` captures the exact binary approximation of the double (0.1000000000000000055...); the string constructor gives exactly 0.1.
10. C. `Instant` is a count from the epoch; the others carry local fields or a zone.
11. `PT1H30M` and `P10D`. `Duration` is time-based, `Period` is date-based.
12. B. `DateTimeFormatter` is immutable and thread-safe; `SimpleDateFormat` is not.

15.3.5 Quiz 5: Exceptions, I/O and Utilities

1. (what prints) `static int fin() { try { return 1; } finally { return 2; } }` and `static int fin2() { int x = 1; try { return x; } finally { x = 5; } }`. What do `fin()` and `fin2()` return?
2. (what prints) A class `R implements AutoCloseable` prints `open1` or `open2` in its constructor and `close1` or `close2` in `close()`. What is printed by `try (R r1 = new R("1"); R r2 = new R("2")) { System.out.print("body "); }`?
3. (what prints) `System.out.println(Path.of("/a/b/../c").normalize());`

4. (what prints) What does `e.getMessage()` return when you catch the `NumberFormatException` from `Integer.parseInt("12x")`?
5. (MC) Which of these is a checked exception? A) `NullPointerException` B) `IOException` C) `ArithmeticException` D) `StackOverflowError`
6. (short) Why is `catch (Exception e) { }` with an empty body considered harmful?
7. (MC) What happens with `catch (IOException | FileNotFoundException e)`? A) It works B) Compile error: the alternatives are related by subclassing C) Only the first is caught D) Runtime error
8. (short) What is a suppressed exception and how do you read it?
9. (MC) Why is Java's built-in serialization considered a security risk? A) It is slow B) Deserializing untrusted bytes can run attacker-chosen code paths C) It needs a database D) It only works on Windows
10. (short) Which method reads a file line by line lazily, and what must you do with the result?
11. (MC) You start a program with `java -Dmode=fast App`. How does the code read the value? A) `System.getenv("mode")` B) `System.getProperty("mode")` C) `args[0]` D) `Runtime.mode()`
12. (what prints) `int x = -1; assert x > 0 : "neg"; System.out.println("ran");` is run as `java N.java`. What happens, and what changes with `-ea`?

Answers: Quiz 5

1. `2` and `1`. A `return` in `finally` overrides the earlier return (and swallows exceptions; never do it). In `fin2` the return value `1` was already evaluated before `finally` changed `x`.
2. `open1 open2 body close2 close1` . Resources are closed in reverse order of creation.
3. `/a/c`. `normalize()` removes `.` and `..` segments lexically.
4. `For input string: "12x"`.
5. B. `IOException` is checked. `StackOverflowError` is an `Error`; the others are unchecked.
6. It hides failures: the program continues in an unknown state and nobody learns why. Catch specific exceptions, handle them or rethrow, and at minimum log with the stack trace.
7. B. Verified by compiling: "Alternatives in a multi-catch statement cannot be related by subclassing".
8. When a resource's `close()` throws while another exception is already propagating, the close exception is attached to the primary one. Read it with `getSuppressed()`; stack traces show it as "Suppressed:".
9. B. The deserializer constructs objects from the byte stream, so crafted data can chain existing classes into harmful behaviour (gadget chains). Prefer JSON or records and use an `ObjectInputFilter` if you must deserialize.
10. `Files.lines(path)`. It returns a `Stream<String>` that holds the file open, so close it with try-with-resources.
11. B. `-D` defines a system property. `getenv` reads environment variables, and `args` holds program arguments after the class name.
12. It prints `ran`, because assertions are disabled by default. With `java -ea N.java` it throws `AssertionError: neg`. Verified by running.

15.3.6 Quiz 6: Generics and Collections

1. (what prints) `Arrays.asList(1, 2, 3).add(4);` What happens?
2. (what prints) `List<Integer> l = new ArrayList<>(List.of(1, 2, 3)); l.remove(1);` prints `[1, 3]`. What does `l.remove(Integer.valueOf(1))` leave on a fresh `[1, 2, 3]`, and why is the first call different?
3. (what prints) `for (Integer v : list) if (v == 1) list.remove(v);` on `[1, 2, 3]` throws `ConcurrentModificationException`. What happens if the test is `v == 2` instead, and what is the final list?
4. (what prints) `List<String> ls = new ArrayList<>(); List raw = ls; raw.add(1); String s = ls.get(0);` Which line fails and with what?
5. (what prints) `TreeSet<String> ts = new TreeSet<>(String.CASE_INSENSITIVE_ORDER); ts.add("a"); ts.add("A"); System.out.println(ts);`
6. (what prints) `Bad` has a `hashCode` based on mutable `v` and an `equals` on `v`. After `m.put(k, "v"); k.v = 2;` with `k.v` originally 1, what do `m.get(k)`, `m.get(new Bad(1))` and `m.size()` return?
7. (MC) Which line throws? A) `List.of(1, null)` B) `new ArrayList<>().add(null)` C) `new HashMap<>().put(null, 1)` D) `new ArrayDeque<>().add(null)`. Pick every one that throws `NullPointerException`.
8. (what prints) `Deque<Integer> dq = new ArrayDeque<>(); dq.push(1); dq.push(2); dq.addLast(3);` What does `System.out.println(dq + " " + dq.pop());` print?

9. (what prints) `PriorityQueue<Integer> pq = new PriorityQueue<>(List.of(5, 1, 3));` What do two successive `pq.poll()` calls return?
10. (what prints) `List<Integer> src = new ArrayList<>(List.of(1, 2, 3, 4)); src.subList(1, 3).set(0, 99);` What is `src` now?
11. (MC) `HashMap` defaults are: A) capacity 10, load factor 0.5 B) capacity 16, load factor 0.75 C) capacity 16, load factor 1.0 D) capacity 8, load factor 0.75
12. (short) State PECS and name one method signature that uses it.

Answers: Quiz 6

1. It throws `UnsupportedOperationException`. `Arrays.asList` is a fixed-size view of the array: `set` works, `add` and `remove` do not.
2. `[2, 3]`. `remove(int index)` is chosen for the literal `1` (it removes the element at index 1, leaving `[1, 3]`); `remove(Object)` is chosen for the boxed `Integer`.
3. Nothing is thrown, and the list ends as `[1, 3]`. After removing the second-to-last element, `hasNext()` sees `cursor == size` and the loop ends before `next()` checks for modification. This is a famous quirk, not a safe pattern. Use `removeIf` or an explicit `Iterator`.
4. The last line: `ls.get(0)` throws `ClassCastException` because the compiler inserts a cast to `String`. The raw `add` only produced a compiler warning (heap pollution).
5. `[a]`. The comparator treats `A` and `a` as equal, so the set rejects the second add.
6. `null`, `null`, `1`. The entry sits in the bucket for hash 1; looking up `k` now hashes to 2, and looking up `new Bad(1)` finds the entry but `equals` fails because the stored key now has `v == 2`. The entry is unreachable. Never mutate keys.
7. A and D throw. `List.of` rejects nulls and `ArrayDeque` forbids null elements; `ArrayList` and `HashMap` allow them.
8. `[2, 1, 3] 2`. `push` adds at the head; `pop` removes from the head. String concatenation evaluates `dq` first, then `pop()`.
9. `1` then `3`. A priority queue returns the smallest element first (natural order).
10. `[1, 99, 3, 4]`. `subList` is a view backed by the original list.
11. B. Capacity 16 and load factor 0.75 (resize at 12 entries).
12. "Producer extends, consumer super": use `? extends T` to read from a structure, `? super T` to write into it. Example: `static <T> void copy(List<? super T> dest, List<? extends T> src)`.

15.3.7 Quiz 7: Functional Java: Lambdas and Streams

1. (what prints) `Stream.of(1, 2, 3).peek(v -> System.out.print("peek" + v + " ")).filter(v -> v > 1);` and nothing else. What is printed?
2. (what prints) `Stream.of(1, 2, 3).peek(v -> System.out.print("p" + v + " ")).filter(v -> v > 1).findFirst().ifPresent(v -> System.out.println("found " + v));`
3. (what prints) `Stream.of("a", "bb", "cc").collect(Collectors.groupingBy(String::length))`
4. (what prints) `Stream.of(1, 2, 3, 4, 5).collect(Collectors.partitioningBy(v -> v % 2 == 0))`
5. (what prints) `Stream<Integer> st = Stream.of(1); st.count(); st.count();` What happens on the second call?
6. (what prints) `Optional.of(null)` versus `Optional.ofNullable((String) null).orElse("x")`.
7. (what prints) `Stream.of(1, 2, 3, 4).reduce(10, Integer::sum)` and `IntStream.rangeClosed(1, 5).sum()`
8. (what prints) `Stream.of("a", "a").collect(Collectors.toMap(v -> v, v -> 1))`: what happens?
9. (what prints) `Function<Integer,Integer> inc = q -> q + 1, dbl = q -> q * 2;` What do `inc.andThen(dbl).apply(3)` and `inc.compose(dbl).apply(3)` return?
10. (MC) `int count = 0; list.forEach(x -> count++);` fails because: A) `forEach` is not allowed B) captured local variables must be effectively final C) `count` is a primitive D) lambdas cannot return void
11. (short) When is `orElseGet(supplier)` better than `orElse(value)`?
12. (short) How does the JVM implement a lambda: anonymous inner class or something else?

Answers: Quiz 7

1. Nothing. Intermediate operations are lazy and there is no terminal operation, so `peek` never runs.
2. `p1 p2 found 2`. Elements flow one at a time, and `findFirst` short-circuits as soon as 2 passes the filter, so 3 is never touched.
3. `{1=[a], 2=[bb, cc]}`.
4. `{false=[1, 3, 5], true=[2, 4]}`. `partitioningBy` always has both keys.
5. `IllegalStateException: stream has already been operated upon or closed`. A stream is one-shot.
6. `Optional.of(null)` throws `NullPointerException`; the second expression gives `x`.
7. `20` (10 + 1 + 2 + 3 + 4) and `15`.
8. `IllegalStateException: Duplicate key a (attempted merging values 1 and 1)`. Pass a merge function as the third argument to handle duplicates.
9. `8` and `7`. `andThen` applies `inc` first ((3 + 1) * 2); `compose` applies `dbl` first (3 * 2 + 1).
10. B. Verified by compiling: "local variables referenced from a lambda expression must be final or effectively final". Use an `AtomicInteger`, or better, a stream `count()`.
11. When computing the default is expensive or has side effects: `orElse` evaluates its argument always, `orElseGet` only when the `Optional` is empty.
12. Neither as an inner class file: the compiler emits an `invokedynamic` instruction and the `LambdaMetafactory` creates the implementing class at run time. `this` inside a lambda refers to the enclosing instance.

15.3.8 Quiz 8: Concurrency

1. (short) Two threads each run `count++` 100,000 times on a shared `int`. Can the final value be less than 200,000? Why?
2. (what prints) `Thread t = new Thread(() -> System.out.println(Thread.currentThread().getName())); t.run();` What name is printed?
3. (what prints) `Thread tt = new Thread(() -> {}); tt.start(); tt.join(); tt.start();` What happens, and what is `tt.getState()` before `start`?
4. (what prints) `Thread v = Thread.ofVirtual().unstarted(() -> {}); System.out.println(v.isDaemon());`
5. (what prints) `ConcurrentHashMap<String,Integer> m = new ConcurrentHashMap<>(); m.merge("a", 1, Integer::sum); m.merge("a", 1, Integer::sum); System.out.println(m);`
6. (what prints) `AtomicInteger ai = new AtomicInteger(5); System.out.println(ai.getAndIncrement() + " " + ai.incrementAndGet() + " " + ai.compareAndSet(7, 10) + " " + ai);`
7. (what prints) A thread runs `try { Thread.sleep(5000); } catch (InterruptedException e) { System.out.println(Thread.currentThread().isInterrupted()); }` and is interrupted from outside while sleeping. What is printed?
8. (what prints) `CompletableFuture.supplyAsync(() -> 6).thenApply(q -> q * 7).get()`
9. (MC) `volatile` on an `int` field guarantees: A) atomic `x++` B) visibility of writes to other threads C) mutual exclusion D) a lock
10. (short) Name one way to avoid deadlock between two locks.
11. (MC) `Executors.newFixedThreadPool(4)` uses which queue? A) a bounded one of size 4 B) an unbounded `LinkedBlockingQueue` C) a `SynchronousQueue` D) none
12. (short) What does the Java Memory Model guarantee about `Thread.start()` and `Thread.join()`?

Answers: Quiz 8

1. Yes. `count++` is read, add, write; two threads can read the same value and one update is lost. Fix with `AtomicInteger`, `synchronized` or a lock.
2. `main`. Calling `run()` executes the body on the calling thread. Only `start()` creates a new thread.
3. The second `start()` throws `IllegalThreadStateException`; before `start()` the state is `NEW` (after `join` it is `TERMINATED`).
4. `true`. Virtual threads are always daemon threads.
5. `{a=2}`. `merge` is atomic per key.
6. `5 7 true 10`.
7. `false`. When `InterruptedException` is thrown, the interrupt flag is cleared; call `Thread.currentThread().interrupt()` if you need to preserve it.

8. `42`.
9. B. `volatile` gives visibility and ordering, not atomicity of compound operations.
10. Always acquire locks in the same global order, or use `tryLock` with a timeout, or avoid holding two locks.
11. B. The unbounded queue means tasks pile up under load (a memory risk); for production, build a `ThreadPoolExecutor` with a bounded queue and a rejection policy.
12. Everything the starting thread did before `start()` happens-before the new thread's actions, and everything a thread did happens-before another thread's successful return from `join()` on it.

15.3.9 Quiz 9: The JVM in Depth

1. (what prints) `try { long[] big = new long[Integer.MAX_VALUE]; } catch (OutOfMemoryError e) { System.out.println(e.getMessage()); }` What message appears on JDK 21?
2. (what prints) `static int depth = 0; static void rec() { depth++; rec(); }` Calling `rec()` inside `try { } catch (StackOverflowError e) { }` and then printing `depth > 1000` shows what?
3. (what prints) `Object ref = new Object(); WeakReference<Object> wr = new WeakReference<>(ref); System.out.println(wr.get() != null);` while `ref` is still in scope.
4. (MC) On a machine with 4 CPUs and 8 GB of RAM running Java 21 with no flags, the default collector is: A) Serial B) Parallel C) G1 D) ZGC
5. (MC) The error message `java.lang.OutOfMemoryError: Metaspace` most likely means: A) too many objects on the heap B) too many classes loaded (often class loader leaks) C) too many threads D) a huge array
6. (short) Name the spaces of the young generation.
7. (MC) An object reachable only through a `WeakReference` is: A) never collected B) collectable at the next GC C) collected only on `OutOfMemoryError` D) collected after 60 seconds
8. (short) What does `-Xlog:gc*` do?
9. (short) Which flag makes the JVM write a heap dump when it runs out of heap, and what do you open it with?
10. (short) What is a safepoint, and why does it matter for GC pauses?
11. (short) What are escape analysis and scalar replacement?
12. (short) What are compressed oops, and what heap size limit is usually associated with them?

Answers: Quiz 9

1. `Requested array size exceeds VM limit`. The array length is beyond what the VM allows, so this fails before any allocation is attempted. (A merely too-large array gives `Java heap space`.)
2. `true`. `StackOverflowError` is an `Error`, but it can be caught; the depth reached is typically thousands to tens of thousands of frames.
3. `true`. A strong reference (`ref`) still exists, so the object is not eligible for collection.
4. C. G1 is the default on server-class machines (2 or more CPUs and at least about 1.8 GB). Java 27 makes G1 the default everywhere.
5. B. Metaspace holds class metadata; running out usually means classes are being generated or loaded repeatedly, or class loaders cannot be collected.
6. Eden and the two survivor spaces (S0 and S1, also called from and to).
7. B. Weakly reachable objects are cleared when the collector runs; soft references survive until memory is tight.
8. It enables the unified logging framework for all GC-related tags (`gc`, `gc+heap`, `gc+age` and so on), printing pause times, heap before and after, and more.
9. `-XX:+HeapDumpOnOutOfMemoryError` (with `-XX:HeapDumpPath=...`). Open the `.hprof` with Eclipse MAT or VisualVM.
10. A point in the code where a thread can be safely stopped with a consistent view of its stack. Stop-the-world phases wait for all threads to reach a safepoint, so a thread that takes long to get there lengthens the pause.
11. The JIT proves an object never escapes a method; it can then skip heap allocation and keep the fields in registers (scalar replacement).
12. Object references stored in 32 bits instead of 64 bits, saving memory and cache. They work for heaps up to about 32 GB; just above that limit the JVM switches to full 64-bit references.

15.3.10 Quiz 10: Runtime Features and Diagnostics

1. (what prints) `System.out.println(String.class.getDeclaredMethod("length").invoke("abc"));`
2. (what prints) An annotation `@interface Dflt {}` has no `@Retention`; `@Rt` is declared with `@Retention(RetentionPolicy.RUNTIME)`. Both are put on class `Ann`. What do `Ann.class.isAnnotationPresent(Rt.class)` and `Ann.class.isAnnotationPresent(Dflt.class)` return?
3. (what prints) A dynamic proxy for `interface Greeter { String hi(String n); }` has a handler that increments a counter and returns `method.getName() + args[0]`. After `px.hi("X")`, what are the returned string and the counter?
4. (MC) A JDK dynamic proxy (`java.lang.reflect.Proxy`) can proxy: A) any class B) interfaces only C) final classes D) enums
5. (short) What does `ServiceLoader` do?
6. (short) In `module-info.java`, what is the difference between `exports` and `opens`?
7. (MC) Which tool prints a thread dump of a running JVM? A) `jmap` B) `jstack` C) `javap` D) `jlink`
8. (short) Which `jcmd` command prints the number of instances per class, and when do you use it?
9. (short) How do you record a Flight Recorder file at startup and print its events?
10. (short) A Java process uses 100 percent CPU. Describe how you find the responsible thread.
11. (short) What problem does the Foreign Function and Memory API solve, and in which release did it become final?
12. (MC) `field.get(obj)` on a private field throws `IllegalAccessException` unless you call: A) `field.setFinal()` B) `field.setAccessible(true)` C) `obj.unlock()` D) nothing is possible

Answers: Quiz 10

1. `3`. Reflection finds and invokes the method.
2. `true false`. Without `@Retention`, the default is CLASS: kept in the class file but not available at run time.
3. `hiX` and `1`. Every call on the proxy goes through the `InvocationHandler`.
4. B. For classes, frameworks use subclass-generating libraries instead (Spring uses CGLIB, Mockito uses ByteBuddy).
5. It discovers and instantiates implementations of an interface listed in `META-INF/services` or declared with `provides ... with` in a module. It is the basis of plug-in architectures (JDBC drivers, for example).
6. `exports` makes public types of a package accessible at compile time and run time. `opens` additionally allows deep reflection (private members) at run time, which frameworks need.
7. B. `jcmd <pid> Thread.print` is the equivalent.
8. `jcmd <pid> GC.class_histogram`. Use it to find which class is growing when you suspect a memory leak; take two histograms a few minutes apart and compare.
9. Start with `-XX:StartFlightRecording=filename=app.jfr,duration=60s`, then run `jfr print app.jfr` (or `jfr summary`) or open it in JDK Mission Control.
10. Find the busy thread (`top -H -p <pid>` gives the thread id), convert the id to hex, then search for `nid=0x...` in the `jstack` output to see its stack.
11. It replaces JNI with a safer, pure-Java way to call native functions and manage off-heap memory. It became final in Java 22.
12. B. (And in a named module, the package must also be opened to your module.)

15.3.11 Quiz 11: Modern Java and Tooling

1. (what prints) `int r = switch ("b") { case "a" -> 1; case "b" -> 2; default -> 0; }; System.out.println(r);`
2. (short) In a text block, what does a backslash at the end of a line do?
3. (MC) In which release did records become a final (non-preview) feature? A) 14 B) 16 C) 17 D) 21
4. (MC) Which Maven phase runs the unit tests? A) compile B) test C) package D) install
5. (short) What does the `test` dependency scope mean in Maven?
6. (short) What does `jlink` produce?
7. (MC) Which of these is a final feature in Java 25? A) Scoped Values B) Structured Concurrency C) Stable Values D) Vector API
8. (short) What changes about the default garbage collector in Java 27?
9. (short) Which Java release made virtual threads final, and which JEP?
10. (short) What combination of features gives you algebraic data types in modern Java?

11. (short) Which flag helps size the heap relative to the container memory limit?
12. (short) Name two JUnit 5 lifecycle annotations and when each runs.

Answers: Quiz 11

1. `2`.
2. It suppresses the line break, joining the next line to the current one (verified by running).
3. B. Records were preview in 14 and 15, final in 16.
4. B. The lifecycle runs validate, compile, test, package, verify, install, deploy; running a later phase runs the earlier ones.
5. The dependency is on the classpath for compiling and running tests only, and is not packaged into the application.
6. A custom runtime image containing only the modules your application needs.
7. A. Scoped Values are final in Java 25 (JEP 506); Structured Concurrency is still a preview feature, as are Stable Values; the Vector API is still incubating.
8. G1 becomes the default collector in all environments (JEP 523); before, small machines (fewer than 2 CPUs or under about 1.8 GB) got Serial.
9. Java 21, JEP 444.
10. Sealed interfaces (a closed set of subtypes), records (data carriers) and pattern matching `switch` (exhaustive deconstruction).
11. `-XX:MaxRAMPercentage` (default 25.0), together with the container's memory limit; `-Xmx` sets an absolute size instead.
12. `@BeforeEach` runs before every test method and `@AfterEach` after it; `@BeforeAll` and `@AfterAll` run once per class (static methods by default).

15.3.12 Quiz 12: Design Patterns and Best Practices

1. (what prints) `class Holder { private final List<String> items; Holder(List<String> i) { items = i; } int size() { return items.size(); } }`. With `List<String> shared = new ArrayList<>(List.of("a")); Holder h = new Holder(shared); shared.add("b");`, what does `h.size()` return, and how do you fix the class?
2. (what prints) `Objects.requireNonNullElse(null, "dflt")`
3. (MC) The simplest thread-safe, serialization-safe singleton is: A) a synchronized `getInstance` B) double-checked locking C) a single-element `enum` D) a static `volatile` field
4. (short) How do lambdas simplify the Strategy pattern?
5. (MC) `new BufferedReader(new FileReader(f))` is an example of: A) Singleton B) Decorator C) Visitor D) Prototype
6. (short) Why use `PreparedStatement` instead of building SQL by concatenation?
7. (short) Why use `SecureRandom` for session tokens?
8. (MC) When is the Builder pattern most useful? A) classes with many optional parameters B) classes with no fields C) static utility classes D) enums
9. (short) Explain "composition over inheritance" in two sentences.
10. (what prints) `new StringBuilder("abc").reverse().append(1).insert(0, 'z')`
11. (short) How do you protect against path traversal when serving files from a base directory?
12. (short) Why measure before optimising, and what tool do you use for micro-benchmarks?

Answers: Quiz 12

1. `2`. The class stores the caller's mutable list, so outside changes leak in. Copy defensively: `items = List.copyOf(i);` (immutable copy; rejects nulls).
2. `dflt`.
3. C. An `enum` with one constant gives lazy, thread-safe initialisation and is safe against reflection and serialization attacks.
4. A strategy is just a function: pass a lambda or method reference (`Comparator`, `Predicate`) instead of defining an interface plus one class per algorithm.
5. B. `BufferedReader` wraps another `Reader` and adds buffering behind the same interface.
6. Parameters are sent separately from the SQL text, so input is never interpreted as SQL, which prevents SQL injection; it also allows statement reuse.

7. `Random` is predictable from its seed. `SecureRandom` draws from a cryptographically strong source, so tokens cannot be guessed.
8. A. It gives named, readable construction and a place to validate before building an immutable object.
9. Instead of inheriting behaviour from a parent class, hold another object as a field and delegate to it. It avoids the tight coupling of a subclass to a parent's internals and lets you swap parts at run time.
10. `zcba1`. Reverse gives `cba`, then `1` is appended, then `z` inserted at index 0.
11. Resolve the user input against the base path, call `normalize()`, and check that the result `startsWith(base)`; reject it otherwise.
12. Intuition about performance is often wrong; the JIT, GC and caches change results. Profile first (JFR), and use JMH for micro-benchmarks, since hand-written timing loops are distorted by warm-up and dead-code elimination.

KEY POINTS

How to use the Part quizzes.

- Answer closed-book and in writing, then check; the effort of recalling is what builds memory.
- Score 11 or 12 and move on; 9 or 10, log the misses; 8 or fewer, redo that Part's code.
- For every wrong "what prints" answer, write the tiny program and run it. A run that surprises you is worth ten re-reads.
- Re-take any quiz you scored below 9 on after three days, without looking at the answers.

15.4 Final Mock Interview and What to Do Next

This is a single 60-question test that mixes everything. Treat it like a real interview: set a timer for **90 minutes**, close the book, and write your answers on paper. Snippets were compiled and run on JDK 21. Short-answer questions are marked (short); answer in one to three sentences, as you would speak in an interview. The key follows the questions.

Scoring: 52 to 60 means you are interview-ready on the fundamentals; 40 to 51 is solid, so review the topics of your misses; 30 to 39 means repeat weeks 2 to 4 of the study plan for your weak areas; below 30, go back to Part 1 and re-do the daily loop. Give half a point for a short answer that is right in spirit but vague.

15.4.1 Round 1: Language and object-oriented Java

1. (what prints) `String s = null; s += "a"; System.out.println(s);`
2. (what prints) `int i = 0; i = i++; System.out.println(i);`
3. (what prints) `long l = Integer.MAX_VALUE * 2; System.out.println(l);`
4. (what prints) `System.out.println(Math.round(-2.5) + " " + Math.round(2.5));`
5. (what happens) `Integer n = null; int r = true ? n : 0;`
6. (what prints) `int x = 2; switch (x) { case 1: System.out.print("1"); case 2: System.out.print("2"); case 3: System.out.print("3"); break; default: System.out.print("d"); }`
7. (what prints) `class Base { Base() { init(); } void init() { System.out.print("Base.init "); } }` and `class Derived extends Base { int v = 7; void init() { System.out.print("v=" + v + " "); } }`. What does `new Derived();` print?
8. (what happens) `interface X { default String n() { return "X"; } }` and `interface Y { default String n() { return "Y"; } }`. A class implements both. What must it do, and what does `X.super.n() + Y.super.n()` return?
9. (what prints) `float f = 0.1f; System.out.println(f == 0.1);`
10. (short) When do you choose an abstract class over an interface?
11. (MC) `final List<String> names = new ArrayList<>();` means: A) the list cannot change B) `names` cannot be reassigned, but the list can change C) the list is thread-safe D) the variable is a constant String
12. (short) What is a covariant return type? Give a one-line example.

15.4.2 Round 2: Strings, exceptions and collections

1. (what prints) `Set<Short> ss = new HashSet<>(); for (short k = 0; k < 5; k++) { ss.add(k); ss.remove(k - 1); } System.out.println(ss.size());`
2. (what prints) `System.out.println(Arrays.asList(new int[]{1, 2}).size());`
3. (what prints) `System.out.println(new StringBuilder("a").equals(new StringBuilder("a")));`
4. (what happens) `Collections.sort(List.of(3, 1, 2));`
5. (what prints) `Map<String,Integer> m = new TreeMap<>(Comparator.reverseOrder()); m.put("a", 1); m.put("c", 3); m.put("b", 2); System.out.println(m);`
6. (what prints) `System.out.println(new int[]{1}.equals(new int[]{1}) + " " + Arrays.equals(new int[]{1}, new int[]{1}));`
7. (what prints) `System.out.println(Objects.hash(1, 2, 3) + " " + List.of(1, 2, 3).hashCode());` Are the two numbers related?
8. (what prints) `System.out.println(String.valueOf((Object) null));`
9. (short) Why did Java's designers introduce checked exceptions, and what is the common criticism?
10. (short) What must a class implement to be used in try-with-resources?
11. (MC) Which allows a `null` key? A) `Hashtable` B) `ConcurrentHashMap` C) `HashMap` D) `TreeMap` with natural ordering
12. (short) When would you pick `LinkedList` over `ArrayList`?

15.4.3 Round 3: Streams and concurrency

1. (what prints) `System.out.println(Stream.of(1, 2, 3, 4).skip(1).limit(2).toList());`
2. (what prints) `System.out.println(Stream.of("a", "b", "a").collect(Collectors.groupingBy(Function.identity(), Collectors.counting())));`
3. (what prints) `List<Integer> li = new ArrayList<>(List.of(1, 2, 3)); li.removeIf(x -> x % 2 == 1); System.out.println(li);`
4. (what prints) `System.out.println(Optional.of(5).map(q -> q * 2).filter(q -> q > 20).isPresent());`
5. (what prints) `System.out.println(Long.valueOf(127) == Long.valueOf(127));`
6. (what prints) `System.out.println(Stream.of(1, 2, 3, 4, 5, 6).parallel().reduce(0, Integer::sum));` Is the result deterministic and why?
7. (short) Give two situations where a parallel stream is a bad idea.
8. (MC) `wait()` must be called: A) anywhere B) inside a `synchronized` block on the same object, ideally in a loop checking the condition C) only from the main thread D) after `notify()`
9. (short) Name two things `ReentrantLock` offers that `synchronized` does not.
10. (short) Why is `Executors.newCachedThreadPool()` risky under heavy load?
11. (short) When do virtual threads help, and when do they not?
12. (MC) Compared with `Collections.synchronizedMap`, `ConcurrentHashMap`: A) locks the whole map on each call B) allows concurrent updates and has atomic methods like `merge` C) keeps keys sorted D) is slower for all uses

15.4.4 Round 4: JVM, memory and modern Java

1. (short) What is stored on the heap and what on a thread's stack?
2. (MC) `-Xmx2g` sets: A) the initial heap B) the maximum heap C) the stack size D) Metaspace limit
3. (short) What distinguishes a minor (young) collection from a full collection?
4. (short) What is a memory leak in a garbage-collected language? Give one example.
5. (MC) `OutOfMemoryError: unable to create native thread` points to: A) a full heap B) too many threads or too little native memory/OS limit C) a huge array D) a bad `finalize`
6. (short) What do the C1 and C2 JIT compilers do differently?
7. (MC) `var x = 10;` is: A) dynamically typed B) statically typed, with the type inferred as `int` C) an `Object` D) allowed for fields
8. (short) What can a record not do that an ordinary class can?
9. (what prints) `Object o = 42; if (o instanceof Integer n && n > 40) System.out.println(n + 1);`
10. (short) How does `sealed` help a `switch` expression?
11. (short) Name three features finalised in Java 21.

12. (MC) Java 25 is: A) a short-term release B) an LTS release C) preview only D) the first version with records

15.4.5 Round 5: Mixed, design and tooling

1. (what prints) `System.out.println(Character.getNumericValue('7') + ('7' - '0') + " " + (char) ('a' + 2));`
2. (what prints) `int[] arr = {3, 1, 2}; int[] cp = arr; Arrays.sort(cp); System.out.println(arr[0]);`
3. (what prints) `String t = "a"; t = t + 1 + 2; System.out.println(t);`
4. (what prints) `System.out.println(Integer.parseInt("-0") + Integer.valueOf("08"));`
5. (what prints) `double dd = 10; dd /= 4; int ii = 10; ii /= 4; System.out.println(dd + " " + ii);`
6. (short) What does the "D" in SOLID mean, and how do you apply it in plain Java?
7. (short) Name two problems with the classic singleton and one remedy.
8. (short) List the properties of the `equals` contract.
9. (short) Your service hangs and CPU is idle. How do you check for a deadlock?
10. (short) Why are immutable classes easier to use in concurrent programs?
11. (short) What is the role of a build tool like Maven or Gradle?
12. (short) Describe what happens inside `HashMap.get(key)`.

15.4.6 Answer key

Questions 1 to 12:

1. `nulla`. `null + "a"` is string concatenation.
2. `0`. The old value is stored back after the increment, overwriting it.
3. `-2`. The multiplication happens in `int` and overflows before the widening to `long`.
4. `-2 3`. `Math.round` rounds half up (towards positive infinity).
5. `NullPointerException`: the conditional has type `int`, so `n` is unboxed.
6. `23`. Without `break`, execution falls through to the next case.
7. `v=0`. The superclass constructor runs before the subclass's field initialisers, so the overridden method sees the default 0. Never call overridable methods from a constructor.
8. The class must override `n()`, otherwise it does not compile. The expression gives `XY`.
9. `false`. `0.1f` widened to double is not 0.1.
10. When subclasses share state or constructor logic, you need non-public members, or you want a template method. Use an interface for a capability that unrelated classes may have; a class can implement many interfaces but extend one class.
11. B.
12. An overriding method may return a subtype of the overridden method's return type, for example `Object clone()` overridden as `Point clone()`.

Questions 13 to 24:

1. `5`. `k - 1` is an `int`, boxed to `Integer`, which never equals a `Short`, so `remove` removes nothing.
2. `1`. `asList` of an `int[]` produces a `List<int[]>` with a single element.
3. `false`. `StringBuilder` does not override `equals`; use `contentEquals` or `toString`.
4. `UnsupportedOperationException`; `List.of` is immutable.
5. `{c=3, b=2, a=1}`.
6. `false true`. Arrays use identity `equals`; `Arrays.equals` compares contents.
7. `30817 30817`. Both compute 31 * h + element starting from 1, the `List` hash formula.
8. `null` (the text). `valueOf(Object)` returns the string "null".
9. To force callers to deal with recoverable failures. Criticism: they clutter code and leak implementation details across layers (lambdas and streams do not work well with them); many modern APIs and frameworks use unchecked exceptions.
10. `AutoCloseable` (or `Closeable`), with `close()`.
11. C. `Hashtable` and `ConcurrentHashMap` reject null keys, and a `TreeMap` with natural ordering throws on a null key.
12. Rarely. Mostly when you need constant-time insertion or removal through an iterator in the middle of a long list; for queues and stacks prefer `ArrayDeque`.

Questions 25 to 36:

1. `[2, 3]`.
2. `{a=2, b=1}`.
3. `[2]`.
4. `false`. 10 is not above 20.
5. `true`. `Long` caches -128 to 127 like `Integer`.
6. `21`. Deterministic, because addition is associative and the identity is 0; non-associative operations (such as subtraction) would give unpredictable results.
7. When tasks are tiny or the source is small (overhead dominates); when operations have side effects or touch shared mutable state; when tasks block on I/O (the common pool is small); when order matters and you rely on `forEach`.
8. B. A loop guards against spurious wakeups.
9. Any two of: `tryLock` with timeout, interruptible lock acquisition, fairness policy, multiple `Condition`s, and lock/unlock in different scopes.
10. It creates a new thread for every task when none is idle, with no upper bound, so a burst can exhaust memory or OS thread limits.
11. They help when many tasks mostly wait on blocking I/O (thread-per-request servers). They do not speed up CPU-bound work, and on Java 21 long `synchronized` blocks around blocking calls can pin the carrier thread.
12. B.

Questions 37 to 48:

1. Objects and arrays live on the heap (shared by all threads). Each thread's stack holds frames with local variables (primitives and references) and the operand stack.
2. B.
3. A minor collection cleans only the young generation (cheap and frequent, because most objects die young); a full collection processes the whole heap including the old generation and is more expensive.
4. Objects that stay reachable (so cannot be collected) although the program no longer uses them. Example: adding entries to a static `Map` cache and never removing them.
5. B.
6. C1 compiles quickly with light optimisation and profiling; C2 compiles hot code slowly with heavy optimisation (inlining, escape analysis) for peak speed. Tiered compilation uses both.
7. B.
8. It cannot extend another class (it implicitly extends `Record`), cannot declare extra instance fields beyond its components, and its fields are final.
9. `43`.
10. If the supertype is sealed, the compiler knows all subtypes, so the `switch` can be exhaustive without `default` and fails to compile when a subtype is missing.
11. Any three of: virtual threads, record patterns, pattern matching for `switch`, sequenced collections, generational ZGC.
12. B.

Questions 49 to 60:

1. `14 c`. 7 + 7 = 14, then the string concatenation; `'a' + 2` cast to `char` is `c`.
2. `1`. `cp` and `arr` are the same array; sorting through one is visible through the other.
3. `a12`. Left to right: `"a1"` then `"a12"`.
4. `8`. `-0` parses to 0 and `08` parses to 8 in `Integer.valueOf` (decimal, leading zero allowed).
5. `2.5 2`.
6. Dependency Inversion: depend on abstractions rather than concrete classes. Apply it by declaring a field as an interface and passing the implementation in (constructor injection), as Spring does.
7. Global mutable state that is hard to test, and thread-safety or serialization/reflection issues. Remedy: use an `enum` singleton, or avoid the singleton and inject a single instance through a framework.
8. Reflexive, symmetric, transitive, consistent (same result while nothing changes), and `x.equals(null)` is false.
9. Take a thread dump with `jstack <pid>` or `jcmd <pid> Thread.print`; the JVM reports "Found one Java-level deadlock" with the threads and the locks they hold and wait for.

10. They cannot change after construction, so they can be shared freely between threads without locks and with safe publication through `final` fields.
11. Dependency management, a standard project layout and lifecycle (compile, test, package), and reproducible builds from a declarative file (`pom.xml` or `build.gradle`).
12. Compute `hash(key)` (spread with `h ^ (h >>> 16)`), pick the bucket `(n - 1) & hash`, then walk the bucket's list or tree comparing hash and `equals`; return the value, or `null` if absent.

15.4.7 What to do next

Finishing a book is the start of the work, not the end. Here is a sensible order for a Java backend career, based on how most interview loops and junior jobs look.

1. Build and ship something (weeks 1 to 4)

Take one project from Part 13 and extend it: add a command-line interface, persistence with files, then a real database through JDBC. Put it on GitHub with a README that shows how to build and run it, a short design note and tests. Employers read repositories; one clear, working project beats several half-finished ones.

2. Spring Boot (weeks 3 to 8)

Most Java backend jobs use Spring Boot. You are ready for it because you know the core ideas it relies on: reflection and annotations, dynamic proxies (for transactions and AOP), dependency injection (the Dependency Inversion principle), class loading, and threads. Learn in this order: a REST controller; dependency injection and beans; configuration properties; Spring Data JPA with a relational database; validation and error handling; testing with JUnit and `MockMvc`; then security basics. Use the official Spring guides and reference documentation, and rebuild your Part 13 project as a Spring Boot service.

3. Data structures and algorithms (ongoing, 30 to 60 minutes a day)

Interviews for software engineers test problem solving in Java. Work through arrays and strings, hash maps and sets, two pointers and sliding window, stacks and queues, linked lists, recursion and backtracking, trees and graphs (BFS, DFS), heaps, binary search, and dynamic programming basics. For each problem, say the brute-force idea first, state its complexity, then improve it. Part 14 includes a set of coding problems with Java solutions as a starting point; the cheat sheet on collections tells you which structure gives the complexity you need.

4. System design basics (after the above)

Learn to reason about a service rather than a class: client, load balancer, stateless application servers, cache, database, queue. Study relational versus NoSQL storage, indexing, replication and sharding, caching strategies and invalidation, message queues and eventual consistency, idempotency and retries, and observability (logs, metrics, traces). Practise by designing small things end to end, such as a URL shortener or a rate limiter, and always state requirements and trade-offs before drawing boxes.

5. Keep the JVM skills alive

Read GC logs from a service you run, profile with JFR when something is slow, and read the release notes of each new Java version. The habit of testing a claim with a small program, which this book tried to teach, is the skill that keeps paying off.

INTERVIEW ANGLE

"Tell me about a project you built." Structure the answer: the goal, the design (two or three classes or components and why), one hard problem you hit and how you diagnosed it (a bug found with `jstack`, a performance problem found with a profiler, a race condition reproduced with a stress loop), and what you would improve. Concrete detail proves you did the work.

TIP

After your mock interview, do not just read the key. For every miss, write a three-line program that proves the correct answer, run it, and add it to your repository as a note.

KEY POINTS

Final mock interview and next steps.

- The mock tests recall under time pressure; 52 or more of 60 means the fundamentals are in place.
- Convert every wrong answer into a runnable experiment and an entry in the error log.
- Next: one finished project on GitHub, Spring Boot, daily data-structures practice, then system design basics.
- Keep using the JVM tools (`jcmd`, `jstack`, GC logs, JFR); production debugging skill sets strong engineers apart.